About This Book

The *OS/2 Presentation Device Driver Reference* defines what a presentation driver is, how it operates, and when to use one. In addition, a description of the types of presentation drivers, their interfaces, and the available system services is provided. System programmers can use the information found in this book to write their own presentation drivers.

Detailed descriptions of control structures, data structures, and I/O formats have been included to help in understanding and using the interfaces. For further information about structures and defined values, see the *OS/2 Presentation Manager Programming Reference* and the header files supplied with the *Developer's Toolkit for OS/2* .

Knowledge of a programming language (such as C or assembler) that is used for writing OS/2 applications is necessary, and the programmer must be familiar with the workings of the OS/2 operating system.

Note: Programmers who require information on 16-bit device drivers should refer to the *OS/2 Physical Device Driver Reference* .

How This Book is Organized

This book consists of 15 chapters, with supporting appendixes.

Chapters 1 through 11 cover general information about presentation drivers. Chapters 12, 13, 14, and 15 are references detailing each of three classes of presentation driver function:

- Exported Driver Functions, Chapter 12
- Mandatory and Simulated Graphic Engine Functions, Chapter 13
- Device Support Functions, Chapter 14
- Bidirectional (BIDI) Command Structures, Chapter 15

Introduction to OS/2 Presentation Drivers
: This chapter provides general information about presentation drivers, such where they reside in the system, and what they do.

Design Considerations for All Drivers
: This chapter contains a description of general design considerations for all drivers. and specific design considerations for display and hardcopy presentation drivers.

Graphics Engine/Presentation Driver Design Changes
: This chapter describes the changes to the graphics engine since the OS/2 2.1 version.

Design Considerations for Display Drivers
: This chapter contains a description of specific design considerations for display presentation drivers. It also includes an overview about the IBM Distributed Console Access Facility (DCAF) product and the Enhanced Direct Interface Video Extensions (EnDIVE).

Design Considerations for Hardcopy Drivers
: This chapter contains a description of specific design considerations for hardcopy presentation drivers.

Display Drivers
: This chapter contains a description of the entry points a display driver must export to the system, so that an application program can query the driver and the system can enable it.

Distributed Console Access Facility (DCAF) Architecture
: This chapter explains the Distributed Console Access Facility (DCAF), its architecture, and impact on code.

Graphics Engine Hardcopy Drivers
: This chapter contains a description of the hardcopy drivers that use the graphics engine dispatch table, and the related support functions that are provided by the spooler.

Queue Drivers
: This chapter contains a description of the functions queue drivers must export so that they can be used by the spooler.

Port Drivers
: This chapter contains a description of the functions port drivers must export so that they can be used by the Workplace

Shell and the spooler.

Presentation Manager Function Categories
This chapter discusses the types of functions within each category.

- Mandatory Functions for All Drivers

  This section contains descriptions of the internal functions that must be supported by handling routines in the presentation drivers. These functions must be supported by all presentation drivers.

- Mandatory Functions for Display Drivers

  This section contains a description of additional external functions that must be supported by the display drivers.

- Simulated Functions

  This section contains descriptions of the internal functions that are supported in the graphics engine and that can be hooked by the presentation drivers in order to exploit special features in the target device.

- Palette Manager Functions

  This section contains a description of the Palette Manager and its functions.

- Software Motion Video

  This section describes the function calls provided to support movie-like displays.

- Graphics Engine Internal Functions

  This section contains descriptions of the internal functions that are supported in the graphics engine and that can be called by the presentation drivers.

- System Functions

  This section contains descriptions of the system functions that are available to the presentation drivers.

Exported Driver Function Reference
This chapter contains detailed descriptions of the exported entry point functions. These functions are presented alphabetically except for the subfunctions to the "OS2_PM_DRV_ENABLE" entry point; these are given in ascending hexadecimal order.

Mandatory and Simulated Graphics Engine Function Reference
Detailed descriptions of all "mandatory" (must be supported by handling routines in the presentation drivers) and "simulated" (supported by handling routines in the graphics engine) presentation driver functions are given in this chapter. These functions are presented alphabetically.

Device Support Function Reference
Detailed descriptions of all device support or "helper" functions are given in this chapter. These functions are presented alphabetically.

BIDI Command Structures and Command Flow
Detailed descriptions of all the BIDI command structures are provided in this chapter.

Appendixes

Appendix A. OS/2 Version Compatibility Considerations
This table describes information added to or changed since the availability of OS/2 Warp, Version 3 in terms of version compatibility.

Appendix B. Syntax Conventions
This appendix shows the conventions that have been used for the parameter names found in the Presentation Manager library.

Appendix C. Format of the Journal File
This appendix shows the file format that the OS/2 graphics engine journal functions create in memory or on disk.

Appendix D. Bit Map Simulation for 16-Bit Hardcopy Drivers
This appendix contains a description of the bit-map simulation for hardcopy drivers.

Appendix E. Data Types
This appendix contains a description of the parameters for all data types.

Appendix F. Notices
This appendix contains legal notices and trademark acknowledgements.

Miscellaneous

A glossary and an index are included.

Assistance

Technical support for device driver development is provided by the IBM Driver Development Support Center (DDSC) through a bulletin board system (BBS). You are encouraged to use the DDSC to obtain support by sending in your questions and reviewing the question and answer database which can be downloaded for off-line review.

To access the DDSC BBS, dial 512-838-9717 (using a modem) to register and access the support system. For voice support in the United States, call 512-838-9493.

Additional assistance is available through the IBM Solution Developer Program. For membership information:

Internet:	ibmsdp@vnet.ibm.com
US/Canada:	800-627-8363
International:	770-835-9902
International Fax:	770-835-9444

Ordering Information

In addition to the actual tools and source code available on The IBM Developer Connection Device Driver Kit for OS/2, it also includes the following DDK reference books in online format:

- The Physical Device Driver Reference
- The Storage Device Driver Reference
- The Input/Output Device Driver Reference
- The Pen for OS/2 Device Driver Reference
- The Virtual Device Driver Reference
- The Presentation Device Driver Reference
- The Display Device Driver Reference
- The Printer Device Driver Reference
- The Graphics Adapter Device Driver Reference
- The MMPM/2 Device Driver Reference (Multimedia)

To order the DDK call:

U.S.A.:	1-800-633-8266	
Canada:	1-800-561-5293	
When calling from Europe, the Middle East, or Africa, the number depends on the language you use to place the order:	• English	(+45) 48101500
	• French	(+45) 48101200
	• Italian	(+45) 48101600
	• German	(+45) 48101000
	• Spanish	(+45) 48101100
	• Dutch	(+45) 48101400
	• Danish	(+45) 48101300
	• Finish	(+45) 48101650
	• Swedish	(+45) 48101150
	• Norwegian	(+45) 48101250
	• FAX	(+45) 48142207
When ordering from Latin America or South America, the number depends on the country from which you are calling:	• Bolivia	02-35 1840
	• Columbia	01-257-0111
	• Dominican Republic	566-5161
	• El Salvador	02-98 5011
	• Honduras	32-2319
	• Paraguay	021-444 094
	• Urugruay	02-923 617
	• Chile	02-633-4400
	• Costa Rica	223-6222
	• Ecuador	02-56 5100

	• Guatemala	02-31 5859
	• Panama	02-639 977
	• Peru	014-36 6345
	• Venezuela	02-908-8901
	• Argentina	01-313-0014
To order from Asia/ Pacific:	• All except Japan	(61) 2-354-7684
	• Japan	(81) 3-3495-2045(Fax) Fax request to: DAP-J, IBM Japan
To order from SE Brazil:	(021) 800-6120(Voice) (021) 800-6936(Fax)	
To order from Mexico:	• Mexico City	627-2444
	• Country	91-800-00639

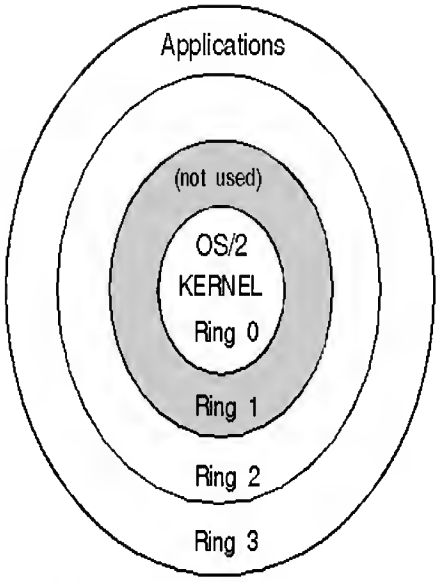
What's New

The following updates have been made for Version 4.02 of the DDK:

- New parameters and expanded parameter descriptions to the following functions:
  - GreFillPath
  - GreBeginArea
  - GreSelectClipPath
  - GreGetRegionRects
  - GreGetClipRects
  - QueryDeviceSurface
  - GreEndArea
- EnDIVE-related APIs GreEscape DEVESC_SETUPBLITTERNOTIFY and GreEscape DEVESC_VIDEOHIDDEN and their associated data types were included.

Introduction to OS/2 Presentation Drivers

OS/2* operates within the confines of a four-ring structure, Ring 0 through Ring 3. As shown in the following figure, Ring 0 is the most-protected ring and contains the OS/2 kernel, which is the main engine that drives the operating system. Ring 3 is the least-protected ring and contains applications and system services. Ring 1 is not used by OS/2. Ring 2 is similar to Ring 0 in that it provides access to physical devices such as printers and displays. Ring 2 is similar to Ring 3 in that it is prohibited from modifying kernel data structures. The following figure shows the ring structure.

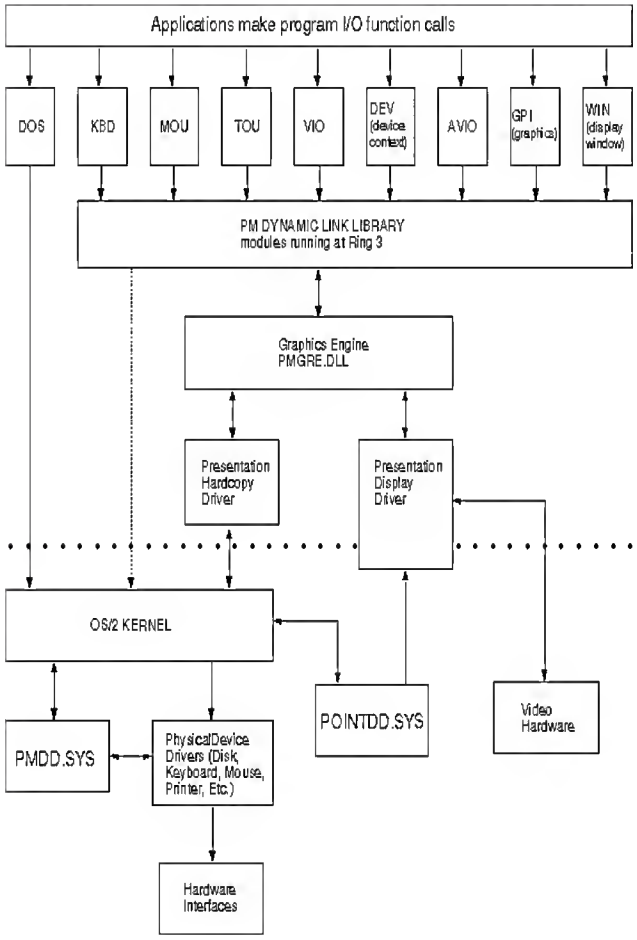


(System Services run at Ring 3 in OS/2 2.0)

Throughout the ring structure, functions can be called from the same ring level or any numerically lower ring level. Conversely, data can be accessed from the same ring level or any numerically higher ring level.

For the sake of compatibility between 16-bit and 32-bit code, all 32-bit code is treated as "Ring-2-Conforming". This means that 32-bit code runs at the ring level of its caller. For example, if the code is called from Ring 3, it runs at Ring 3. If the code is called from Ring 2, it runs at Ring 2. This allows easier access to functions and data, and eliminates many of the costly ring transitions encountered in the previous 16-bit system. To be Ring-2-conforming (rather than strictly Ring 2 or Ring 3), a function cannot call any strictly Ring 3 code or directly access the hardware. In other words, it must abide by the restrictions of both ring levels.

Many of the system services that exist at Ring 3 in dynamic link libraries (DLLs) access another DLL called PMGRE.DLL, which is the graphics engine or kernel of the OS/2 graphics subsystem. The graphics engine is composed entirely of Ring-2-conforming code that interfaces with the presentation drivers, which interface directly with the physical device drivers (at Ring 0) and the hardware. These DLLs are identified as presentation drivers for hardcopy devices (hardcopy drivers) by the file-name extension .DRV, and as presentation drivers for display devices (display drivers) by the file-name extension .DLL. The following figure shows a conceptual view of presentation drivers.



Conceptual view of presentation drivers in the flow of control from an application program to the hardcopy device and the display screen.

The graphics engine loads and enables the presentation drivers, and then dispatches calls to them through dispatch tables as Ring 2, Ring 3, or Ring-2-conforming code. For optimal system performance, write all of the functions in 32-bit presentation drivers be written as Ring-2-conforming. This eliminates the need for ring transitions from system services at Ring 3 and from 16-bit presentation drivers at Ring 2.

By exporting a table named OS2_PM_DRV_RING_LEVELS in 32-bit presentation drivers, the ring level of each function in the dispatch table can be selected.

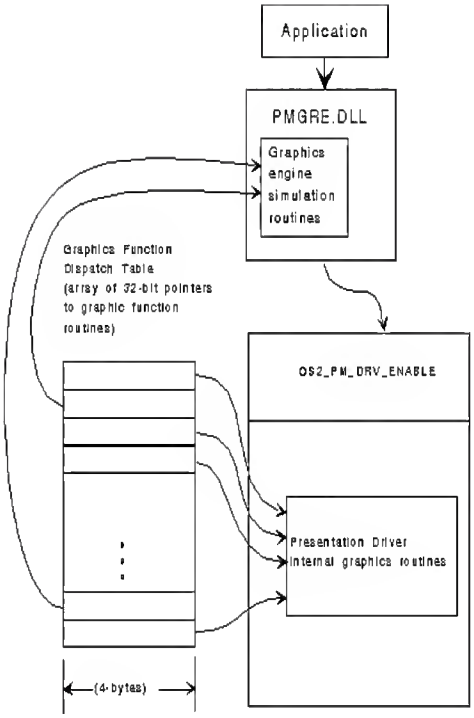
Note: If this table is not exported, all 32-bit functions will be dispatched as Ring-2-Conforming.

The presentation driver has certain responsibilities to the graphics engine. Specifically, a number of entry points exist within the graphics engine that the presentation driver is required to hook (a mechanism by which procedures are called) and support. Many of these functions (as they currently exist in the graphics engine) are not truly functional, and if calls were made to these entry points, nothing would happen. In many cases, they simply return when called.

There are other entry points in the graphics engine that can be optionally hooked by the presentation driver (for example, where only light processing is required, it might be preferable to use the presentation driver). These entry points can be called by the presentation driver for special processing.

The graphics engine calls the entry points within the presentation driver by means of a dispatch table. The dispatch table is a block of memory allocated by the graphics engine for the containment of entry points and is assigned for use by a presentation driver. Each presentation driver loaded by the system is given its own separate dispatch table by the graphics engine. When a presentation driver device context is enabled, the graphics engine allocates a dispatch table for that presentation driver and fills the dispatch table with pointers. Each entry in the table is a 32-bit pointer that points back to a specific routine existing in the graphics engine.

Because many of these routines must be hooked by the presentation driver, the graphics engine refers to the dispatch table to find the appropriate pointer any time that it calls a function in the presentation driver. At this point, however, all of the pointers in the dispatch table point back to routines in the graphics engine. The presentation driver must go into the dispatch table itself and replace some of the pointers with new pointers that point to corresponding routines within the presentation driver. This is mandatory for some routines, optional for others. The hooking of pointer entries in the dispatch table occurs the first time the presentation driver is called at its OS2_PM_DRV_ENABLE entry point for the first subfunction (see FillLogicalDeviceBlock). The following figure shows how the Graphics Function Dispatch Table works.



The exported entry point OS2_PM_DRV_ENABLE (see OS2_PM_DRV_ENABLE) has ten subfunctions. Four of these subfunctions are used in the enable process of a device context, three are used in the disable process, one is used to save the device context, one is used to restore the device context, and one is used to reset the device context.

When Enable Subfunction 01H - *FillLogicalDeviceBlock* is called, the graphics engine passes to it a pointer to the dispatch table that it allocated for the presentation driver. *FillLogicalDeviceBlock* then must substitute all of the mandatory pointers (and perhaps some optional routines) in the dispatch table with pointers to corresponding routines within the presentation driver itself.

The *Developer's Toolkit for OS/2* provides support for writing source code in either C or assembler language. This support consists of a set of header files that define the functions, structures, and constants used in the internal interface to the presentation driver. Presentation drivers might also need to include OS2.H or OS2.INC, which define system functions, structures, and constants.

As with other components of OS/2, the presentation driver architecture ensures that:

- Once loaded, the presentation driver can be enabled for use by multiple applications or processes.
- Once enabled, the presentation driver can support multiple instances of a device context for the owning application or process.

Note that although many of the functions must be hooked by the presentation driver, some might also be passed back to the graphics engine for processing when called. Save the original pointer values stored in the dispatch table by the graphics engine before hooking them. Having access to the original pointers to the graphics engine simulation routines allows the presentation driver to optionally make calls back to the graphics engine (that is, to have the graphics engine do the processing instead). This technique has often proven to be a tremendous time saver in developing OS/2 presentation drivers, especially when calling back to the graphics engine to perform clipping routines.

When the Presentation Manager* interface is initialized, the presentation driver for the attached display is loaded and enabled. This driver has direct access to the video hardware. The function calls passed to the presentation driver are processed and then passed to the adapter interface.

Hardcopy Devices (Printers and Plotters)

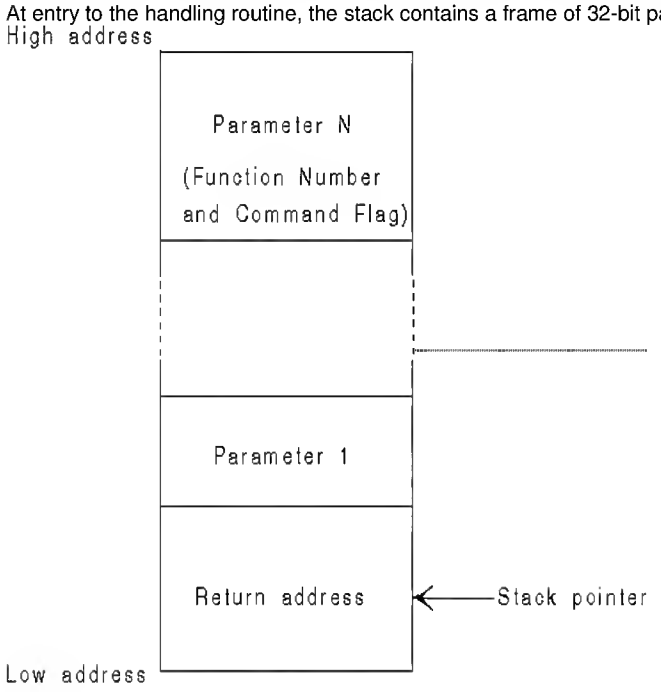
For hardcopy drivers, the presentation driver is loaded in response to an application or process calling *DevOpenDC* . Upon receipt of this routine, the Presentation Manager interface looks to see if the required presentation driver is loaded, and if it is able to handle the new device context (DC). If the presentation driver is loaded and can handle the DC, the driver is enabled for the new DC. If either of these conditions is not met, the required driver is loaded and then enabled.

Note: The amount of time required by the system to load a driver can be reduced by supplying Extended Attribute information in the .DRV file. See Extended Attributes for further information.

Calling Conventions

Presentation drivers interact with both the external API and the internal interface to the graphics engine and dispatch table. Parameters are passed as 32-bit values on the stack by using the C convention. These parameters are pushed to the stack in reverse order of how they appear in the statement. The hardware architecture locates the conceptual top of the stack at the high-order address of the stack space. When data is pushed to the stack, the stack pointer is decremented so that upon completion, the pointer addresses the first item of data.

At entry to the handling routine, the stack contains a frame of 32-bit parameters and a return address as shown in the following figure.



Function Number and Command Flags

The first doubleword (DWORD) pulled from the stack contains two fields:

Field	WORD Type	Description
Function Number	Low-order	Identifies a specific function. Header files define symbolic names for the Gre*xxx* function numbers.
Command Flags	High-order	See the following description.

Command Flags

The flags in this 16-bit field tell the presentation driver which operations need to be done while it processes the function:

COM_DRAW

Bit 0. If this flag is set, the presentation driver must draw the output of the function at the device. When not set, the driver does not draw the output. The function must still be processed to update internal data, such as current position and, if specified, boundary calculations.

COM_BOUND

Bit 1. If set, the presentation driver must calculate the bounding rectangle for the output of the specified function. Upon completion, the

	driver calls its own *GreAccumulateBounds* routine to accumulate the bounding rectangle (GPI_BOUNDS in model space coordinates). **Note:** All presentation drivers must be able to calculate bounds on any figure they can draw.
COM_CORR	Bit 2. This flag is significant only for display drivers. If set, the display driver must determine whether the output of the specified function intersects the pick window. The result, TRUE or FALSE, is passed back to the caller in the return code for the called function. For details, see the function descriptions in Mandatory Functions for All Drivers and Simulated Functions.
COM_ALT_BOUND	Bit 3. This flag is significant only for display drivers. It indicates that the display driver should accumulate USER_BOUNDS in screen coordinates. Notice that user bounds are those used by the Window Manager. Hardcopy drivers do not accumulate user bounds.
COM_AREA	Bit 4. If set, the function is part of an area. Calls to functions that define an area (for example, *GreSetCurrentPosition* and *GrePolyLine*), or that are invalid in an area definition, are passed back to the graphics engine for processing by the default handler. If all functions in the area component have been hooked by the presentation driver, it is not necessary to pass back functions received with COM_AREA set.
COM_PATH	Bit 5. This flag is similar to the COM_AREA flag. Calls to functions that define a path (for example, *GrePolyLine* and *GreSetCurrentPosition*), or that are invalid in a path definition, are passed back to the graphics engine for processing by the default handler. If all functions in the path component have been hooked by the presentation driver, it is not necessary to pass back functions received with COM_PATH set.
COM_TRANSFORM	Bit 6. This flag is set by the graphics engine. If set, the engine is working in world coordinates and the presentation driver should return world coordinates for queries, and convert to device space (using *GreConvert*) for drawing functions. If this bit is not set, the graphics engine is working in device space and the presentation driver should return device coordinates for queries and should not convert to device space for drawing functions.
COM_RECORDING	Bit 7. This flag is ignored.
COM_DEVICE	Bit 8. If set, the presentation driver must process the function. The driver should not pass the function to the graphics engine.
COM_PRECLIP	Bit 10. If set, the graphics engine has passed the function's data already clipped in screen coordinates. The COM_PRECLIP flag will only be used and set by the Graphics Engine when a presentation driver reports CAPS_CLIP_FILLS in the CAPS_ADDITIONAL_GRAPHICS capability field of the mandatory entry point *GreQueryDeviceCaps()* .

Note: This flag is supported if the Graphics Engine is Version 202 or greater.

Note: COM_DRAW, COM_BOUND, COM_CORR, COM_ALT_BOUND, COM_AREA, and COM_PATH apply only to drawing functions. They are ignored by all other functions. The remaining bits of the Command Flags field are not defined and their values are ignored.

Device Context

Device contexts provide the mechanism that the application program uses to write output data to devices. The application, or one of its processes, opens a device context with *DevOpenDC* , associates a presentation space to the DC, and writes or draws in that space. Each *DevOpenDC* creates an instance of a DC. That instance is eliminated when the application closes the device context. The created DC is seen internally as a dispatch table. Calls from the application program to the DC are passed, as one or more internal Gre*xxx* routines, through the dispatch table to the handling routines in the presentation driver for the DC, or are passed back to the Gre*xxx* simulation routines.

Each instance of a device context has:

- Device context types
- Data types (for only OD_QUEUED device context type)
- Instance data
- Program stack

Device Context Types

The type of device to be opened is passed to the presentation driver when the device context is enabled:

OD_INFO	The presentation driver does not generate any output. Information device contexts are used to retrieve information. All Gre*xxx* function routines passed to the presentation driver are processed as if the type were OD_DIRECT. In this way, the operating system can query statistics such as font metrics and boundaries.
OD_MEMORY	The presentation driver processes the output data as it does for an OD_DIRECT device type except the output is written to a bit map that is compatible with the physical device. (The application program creates the bit map and associates it to the device context.)
OD_DIRECT	The presentation driver processes the Gre*xxx* routines to generate device-specific output data. Hardcopy drivers use the file system Prt*xxx* interface to pass the output to the physical device driver. Display drivers use the adapter interface, such as the IBM* Personal System/2* Display Adapter 8514/A interface.
OD_QUEUED	(Hardcopy drivers only.) The hardcopy driver opens a spool file and uses the spooler Spl*xxx* interface to send output to that file. For spooled output, the hardcopy driver must consider the DC data type. See Data Types (Hardcopy Drivers Only).

Note: OD_METAFILE and OD_METAFILE_NOQUERY are handled by PMGPI.DLL and are never passed to the presentation driver.

Data Types (Hardcopy Drivers Only)

For presentation drivers that support the device context type, OD_QUEUED, the driver must support the PM_Q_STD and PM_Q_RAW data types as defined by the Presentation Manager interface. Support for other data types is optional.

The concept of *data types* only applies when the device context type is OD_QUEUED. For all other device context types (OD_DIRECT, OD_MEMORY, OD_METAFILE, OD_METAFILE_NOQUERY, and OD_INFO), the *pszDataType* field in the DEVOPENSTRUC structure has no meaning. See FillPhysicalDeviceBlock for details on when the DEVOPENSTRUC structure is passed to the presentation driver. However, a hardcopy driver is never requested to open an OD_METAFILE or OD_METAFILE_NOQUERY device context type because these types are handled by GPI. See Graphics Engine Hardcopy Drivers for details on how the hardcopy driver creates the spool file.

The basic differences between data types, PM_Q_STD and PM_Q_RAW, are described below:

PM_Q_STD	The hardcopy driver uses the spooler to create a device-independent spool file using the SplStd*xxx* and *SplQmxxx* interfaces.
PM_Q_RAW	The hardcopy driver processes the Gre*xxx* functions to generate device-specific output data. This data is written using the spooler SplQm*xxx* interface to a spool file.

Instance Data

For every instance of a device context, the system has a doubleword that is reserved for use by the presentation driver. Typically, this doubleword is used by the presentation driver to hold a pointer to information about the current state of the device context. This pointer is returned to the system by the driver when the device context is enabled. On subsequent calls through the dispatch table, the pointer is passed back to the presentation driver as a parameter (*pInstanc*e) on the program stack. For more information, see EnableDeviceContext.

Program Stack

A stack of 4KB is available for use when a function is passed to the driver at Ring 2. If it needs more than 4KB, the presentation driver should allocate its own stack space, switch to that stack on entry, and switch back to the original stack on exit. At Ring 3, the presentation driver will use the application's stack when a function is passed to the driver.

Calls to the presentation driver use C language calling conventions. Parameters are pushed to the stack in the opposite order as they are in the call statement.

Dynamic Link Library Functions

Functions exported and imported by a dynamic link library are identified in the library module definition file. These provide links between libraries and subsystems. For example, the components of the Presentation Manager interface must call an enable entry point in the presentation driver. The presentation driver needs access to the simulated functions in the graphics engine. For information on how to develop a dynamic link library (DLL), refer to the *OS/2 Programming Guid*e and *OS/2 Application Design Guid*e .

Note: The initialization routine for a dynamic link library, including presentation drivers, must be compiled to run at Ring 3 (privilege level 3).

Exported Functions

There are two types of exported functions used by OS/2:

- Presentation drivers
- Graphic engine functions

Presentation Drivers

Dynamic link libraries for presentation drivers must export the appropriate set of the following entry points:

```
EXPORTS
   MoveCursorBlock     @103           /* Display drivers only  */
   OS2_PM_DRV_QUERYSCREENRESOLUTIONS /* Optional              */
   OS2_PM_DRV_DEVMODE                /* Hardcopy drivers only */
   OS2_PM_DRV_DEVICENAMES            /* Hardcopy drivers only */
   DrvInstall                        /* Optional              */
   DrvRemove                         /* Optional              */
   OS2_PM_DRV_RING_LEVELS            /* All drivers           */
   OS2_PM_DRV_ENABLE_LEVELS          /* All drivers           */
   OS2_PM_DRV_ENABLE                 /* All drivers           */
```

In addition to the entry points listed above, hardcopy drivers should export entry points for the routines that handle dialogs with the user. See Exported Entry Points (Cursor); also see Exported Entry Points and Exported Driver Function Reference.

Graphics Engine Functions

The graphics engine exports its own set of entry points. Those that are significant to the presentation driver are:

```
EXPORTS
   GETDRIVERINFO     @30
   SETDRIVERINFO     @31
```

GETDRIVERINFO

Used by the presentation driver to get the instance pointer (*pInstance*) for a specified device context, or to get a pointer to the bit-map header for a specified bit map. See System Functions for more information.

SETDRIVERINFO

Used by the presentation driver to set a specified value in the instance pointer of a specified device context.

Note: Instance pointers (*pInstance*) and data are discussed under EnableDeviceContext.

Imported Functions

To call a Gre*xxx* function that is supported as a simulation or internal function in the graphics engine, call the imported entry point with the Gre*xxx* function parameters. The *pInstance* parameter should be NULL. For example, to call *GreCreateJournalFile* with the name assigned in the module definition file, use:

```
result = GreCreateJournal (pszFileName, flOption, cSize, 0L, NGreCreateJournalFile);
```

Note: *NGreCreateJournalFile* is defined in the header file. See the description of GreCreateJournalFile.

Simulations (functions for the presentation driver interface that are supported by handling routines in the graphics engine) can also be called at the addresses given in the default dispatch table. Use the addresses contained in the dispatch table that is passed to the presentation driver at Enable time.

Presentation Driver Interface

The internal Presentation Driver interface is comprised of a set of graphics engine (Gre*xxx*) functions that are called through a dispatch table. A dispatch table is an array of pointers to function-handling routines. The low-order byte of the function number identifies the member of the array that contains the pointer for the function. The functions called through the dispatch table fall into three main groups:

- Functions that all presentation drivers must support. See Mandatory Functions for All Drivers.
- Functions that must be supported by display drivers. See Mandatory Functions for Display Drivers.
- Functions that are supported by simulations in the graphics engine. See Simulated Functions.

The first instance of a loaded presentation driver is given a copy of the default dispatch table. The Enable routine in the presentation driver modifies this copy so that, for those functions supported in the driver, the pointers address the function-handling routines of the presentation driver.

When the function is called a second time (or any time thereafter) for the same presentation driver, a NULL dispatch table pointer can be given because the graphics engine already has the table correctly initialized. It is therefore not necessary to reinitialize the table.

Design Considerations for All Drivers

This chapter describes various design considerations involved in creating a presentation driver. The following list includes the topics covered in this chapter:

- Angles
- Bounds computations
- Clipping
- Closing figures in areas and paths
- Coordinate values
- Positions within text functions
- Return codes
- Transform matrix values
- Allocating Memory
- Error Strategy
- Presentation Manager Error Codes
- Exit List Processing
- Per-Thread Exception Management
- Using Macro Assembler Instructions
- Protecting Objects or Device Contexts

Angles

Angles are passed as signed 32-bit numbers. Zero refers to the direction of the positive X-axis; 360 represents 360˚. Positive values represent counterclockwise angles from the positive X-axis.

Bounds Computations

All presentation drivers must accumulate bounds for unclipped primitives. Application bounds (COM_BOUND) are accumulated in model space. User bounds (COM_ALT_BOUND) are accumulated in device-coordinate space.

Clipping

The presentation driver must perform clipping for drawing and text functions, except for *GreDrawLinesInPath* and *GrePolyShortLine* . Clipping for these two functions is done by the graphics engine. The minimum requirement is to render each primitive clipped to a single rectangle and to clip each rectangle in turn. The rectangles can be enumerated as described in GreGetClipRects.

Note: Rectangles might not always be valid. See Drawing to Display Devices.

Closing Figures in Areas and Paths

The graphics engine generates closure lines for figures within areas and paths unless the presentation driver has opted to hook all the path and area functions. In this case, the presentation driver is responsible for closing any figures. For details, see Area and Path Functions.

Coordinate Values

All coordinates are passed to the presentation driver as 32-bit values. Unless stated otherwise, these values represent world coordinates. The graphics engine function, *GreConvert* , can be called to convert coordinates from one type to another. Coordinates must be converted back to world coordinates before returning to the presentation driver. Notice that screen coordinates are device coordinates to which the DC origin has been added.

Positions within Text Functions

When positions are used, a text function takes the position from the base line of the text box. Descenders, such as the tail of a lowercase character "**y,**" are expressed as a negative value relative to the base line.

Return Codes

The presentation driver must always return a full 32-bit (LONG) value. For example, BOOLEAN TRUE and FALSE are defined as:

```
#define TRUE   (1L);
#define FALSE  (0L);
```

Transform Matrix Values

Transform matrix elements are represented in fixed point notation, that is, as a 16-bit signed integer and a 16-bit fractional part. These precision limits apply during graphics engine matrix multiplication for all initial, intermediate, and final matrix element values.

Allocating Memory

Presentation drivers can allocate and manage memory by using:

- A Dos*xxx* function such as *DosAllocMem* .
- The SS*xxx* functions described in System Functions.

Display drivers, or presentation drivers that share objects such as bit maps and regions, must use the SS*xxx* functions to allocate memory for these objects. Memory allocated through calls to these functions is "shared memory", controlled by the memory allocator component of the graphics engine. Ownership of the memory can be transferred or (when the owning device context ceases to exist) marked as having no owner.

Error Strategy

Presentation drivers support the error strategy implemented by the Presentation Manager interface. When an error occurs, the driver calls *WinSetErrorInfo* (see WinSetErrorInfo) to log the appropriate error code and set the return code to show that an error was detected.

The component that implements a function must provide error checking for the environment, objects, and resources associated with it. The presentation driver requires the following error checking:

- Fail-safe on routines that set attributes and transformation values. Any routine that changes attributes or transformation values must be able to restore the initial values if an error occurs during the change.
- Full error checking on symbol sets, fonts, bit maps, and regions.
- With segment drawing, drawing primitives, and primitive attributes in draw mode, unchecked parameters are passed directly to the graphics engine or the presentation driver. When one of these functions is hooked by the presentation driver, the handling routine must perform the necessary error checking and log any errors, or reset any invalid values to their defaults, as appropriate.
- For any function with coordinates as parameters, the presentation driver must check that the values passed are valid. When an invalid coordinate is detected, the handling routine must log an error or use a default coordinate value, as appropriate.

For any defined error, the application sees the same error code regardless of whether the error was logged by the Graphics Programming Interface (GPI), graphics engine, or presentation driver.

Severity

Four ascending severity levels are defined for error messages:

- Warning
- Error
- Severe error
- Unrecoverable error

Warning

The function detected a problem, took corrective action, and was able to complete processing successfully.

Error

The function detected a problem for which no sensible corrective action is possible. The function is not executed and the system remains at the same state as when the function was requested.

Severe Error

The function detected a problem from which the system cannot reestablish its state. The function has partially executed and the application must now make some corrective action to restore the system to a known state.

Unrecoverable Error

The function detected an error from which it is impossible for the system to reestablish the state that it held at the time that the function was called. It is also impossible for the application to restore the system to a known state.

Presentation Manager Error Codes

Error codes are defined in the header file. These codes fall into general and specific groups. General error codes that are appropriate to many Gre*xxx* functions include:

Error Code	Must be logged by:
PMERR_COORDINATE_OVERFLOW	Functions requiring matrix computation
PMERR_INSUFFICIENT_MEMORY	Functions resulting in memory allocation
PMERR_INV_HBITMAP	Functions with *hbm* as an explicit or implicit parameter
PMERR_INV_HRGN	Functions with *hrgn* as an explicit or implicit parameter
PMERR_INV_COORDINATE	Functions with coordinate parameters
PMERR_INV_IN_AREA	Functions that are invalid inside an open area bracket
PMERR_BASE_ERROR	Functions that directly or indirectly issue DOS routines
PMERR_DEV_FUNC_NOT_INSTALLED	Functions not supported by the presentation driver

Specific error codes listed in the descriptions of each Gre*xxx* function are found in Mandatory and Simulated Graphics Engine Function Reference.

To set an error code and the error's severity, the presentation driver must call *WinSetErrorInfo* (see WinSetErrorInfo). All error codes are listed and explained in the *Presentation Manager Programming Reference* .

Exit List Processing

An exit list is a list of routines that are given control when the current process ends, either normally or abnormally. The following is an example of exit list processing:

1. When the presentation driver's Enable Subfunction 01H - *FillLogicalDeviceBlock* is called, the driver can call function *DosExitList:*

```
#define ROUTINE_ORDER 0x1000
usResult = DosExitList (EXLST_ADD | ROUTINE_ORDER, (PFNEXITLIST)MyExitProc);
```

   This adds the function *MyExitProc* to the list of functions that are called when this process terminates (either normally or because of some error such as a General Protection fault).

2. When *MyExitProc* is called, the presentation driver can perform any necessary cleanup such as releasing any semaphores. The last call in *MyExitProc* is another call to *DosExitList* :

```
usResult = DosExitList (EXLST_EXIT, (PFNEXITLIST)MyExitProc);
```

   This allows the operating system to transfer control to the next function in the list of exit list processing functions for the process that has terminated. For more information, refer to *DosExitList* in the *OS/2 Control Program Programming Reference* and *OS/2 Programming Guide* .

At Enable time, the presentation driver must place an entry in the exit list for the application or process that opens the DC. This entry is a pointer identifying the routine in the presentation driver that releases all resources owned by the DC.

Note:

When writing a presentation driver, consider what would happen if another thread of the process were to terminate.

Per-Thread Exception Management

Per-thread exception management is available to 32-bit EXEs, DLLs, and presentation drivers. The per-thread exception management model is a superset of *DosExitList* processing found in 16-bit OS/2 programming.

Under 16-bit OS/2 architecture, a process cannot handle access violations. In the event of an access violation, the system terminates the process. Given that the trapping process had registered an exit-list function, it had one final chance to do clean up just prior to termination. Sixteen-bit drivers and DLLs usually add an e*xitlist* . At process termination, 16-bit drivers perform per-process clean up in exit-list functions.

Programmers of 32-bit drivers have a choice concerning per-process clean up. They may use the 32-bit version of *DosExitList()* , or they may add code for the DLL termination case in the driver function *_DLL_InitTerm()* . The *_DLL_InitTerm()* function, if present and exported from the driver, will be called by OS/2 with parameter 0 on DLL instantiation and parameter 1 on DLL termination or unload. The *_DLL_InitTerm()* function provides process-granular clean up.

Furthermore, 32-bit OS/2 has exception handlers that are thread-granular. OS/2 lets each thread in a process handle exceptions that arise on that thread. Some of these exceptions are:

- Accessing invalid or uncommitted memory
- Stack overruns
- Thread terminations
- Floating point errors
- Asynchronous (external) signals
- User-raised exceptions (synchronous signals)
- Invalid opcodes
- Bounds
- Divide by zero errors
- Overflow errors

The goals of a 32-bit driver that uses exception management are:

- When passed bad parameters that cause exceptions, drivers can call *WinSetErrorInfo* and return suitable result codes for the application to inspect.
- When threads are terminated while executing driver code, the driver can clean up before the thread exits.

The Exception Handler

A misconception about exception handlers is that they must be able to fix every bad situation and resume execution. This is not a realistic expectation.

Assume that the handler function will not resume every exception. Instead, it will perform clean up, report errors, and return an appropriate result code. In this case, one exception handling function can handle all exceptions.

At exception time, the handler's main activity is to jump out of the handler function back into the function that caused the trap. The handler does an inter-function jump using ANSI C *setjmp()* and *longjmp()*.

After the inter-function jump, the thread is now executing at a known point in the function that had a problem, and doing clean up there is far easier than in the handler.

Sample Code

The code fragment below shows how to set a handler while the calling thread is executing driver code. In the event of an exception, OS/2 calls the handler function, which jumps back to the point of setjmp().

```
#define INCL_DOS
#include <os2.h>
#include <setjmp.h>

// this exception registration record extends the OS/2-defined one in BSEXCPT.H
// by adding a C jmp_buf at the end
struct _regrec {
  PULONG    pNext;
  PFN       pfnHandler;
  jmp_buf   jmp;
}
typedef struct _regrec REGREC, *PREGREC;

// ------------------------------------------------------------------------
// an exception handler function that jumps back to setjmp()

ULONG _System DriverHandler( PEXCEPTIONREPORTRECORD p1,
                             PREGREC p2,
                             PCONTEXTRECORD p3,
                             PVOID pv )
{
  // handle interesting exceptions
  switch( p1->ExceptionNum ) {
  case XCPT_ACCESS_VIOLATION:
  case XCPT_INTEGER_DIVIDE_BY_ZERO:
  case XCPT_INTEGER_OVERFLOW:
  case XCPT_PROCESS_TERMINATE:
  case XCPT_ASYNC_PROCESS_TERMINATE:
    // setjmp() will return p1->ExceptionNum
    longjmp(p2)->jmp, p1->ExceptionNum );
  }
  // not interested; system should continue search
  return XCPT_CONTINUE_SEARCH;
}
```

```
// ------------------------------------------------------------------------
// example of an exported driver function; return TRUE upon success
LONG _System ExportedDriverFunction( PVOID p )
{
  REGREC        regrec;
  ULONG         ulException;
  LONG          lResult;

  regrec.pfnHandler = (PFN)DriverHandler;
  DosSetExceptionHandler( (PEXCEPTIONREGISTRATIONRECORD)&regrec );

  // get thread state; normally rets 0, rets non-zero after longjmp()
  ulException = setjmp( regrec.jmp );

  if( ulException ) {
    // an exception occurred
    // clean up here: release memory, semaphores
    // must check for exiting thread case
    switch( ulException ) {
    case XCPT_PROCESS_TERMINATE:
    case XCPT_ASYNC_PROCESS_TERMINATE:
      // thread is ending or being terminated
      DosUnsetExceptionHandler( (PEXCEPTIONREGISTRATIONRECORD)&regrec );
      DosExit( EXIT_THREAD, 0 );
    }
    WinSetErrorInfo( ... );
    lResult = FALSE;
    goto depart;
  }

  // do the work of the exported function here

  lResult = TRUE;

depart:
  DosUnsetExceptionHandler( (PEXCEPTIONREGISTRATIONRECORD)&regrec );
  return lResult;
}
```

Using Macro Assembler Instructions

Presentation drivers never use the CLI and STI macro assembler instructions because these instructions can interfere with some of the base OS/2 system operations.

Protecting Objects or Device Contexts

A process that attempts to use a locked object will return an error such as PMERR_HDC_BUSY. Although a device context is owned by a single application or process, the owner can access the device context (DC) through multiple threads. The presentation driver must provide a mechanism whereby a DC can register that it is busy and block access from other threads. In its simplest form, this is performed by the *EnterDriver* and *LeaveDriver* routines, which are called at the start and end of each function-handling routine in the presentation driver.

An example of a typical *EnterDriver* routine for a display driver is as follows:

```
 /***************** Typical EnterDriver Routine ***********************/

enter_driver()
{
 do {                /* Lock DC for exclusive use of the current thread */
   SemEnter(Device); /* Some functions do not pass a Device Context (DC)*/
                     /* handle  */
   if (hdc == NULL)
       return(SUCCESS);  /* Check validity of the passed DC handle */
```

```
    if (hdc == ERROR) {
        WinSetErrorInfo (SEVERITY_ERROR, PMERR_INV_HDC);
        SemLeave(Device);
        return(ERROR);
                        }  /* DC region must be validated before */
                           /* driver draws into it.              */

    if (hdc_is_not_dirty)  /* Test the HDC_IS_DIRTY flag. */
                           /* If the flag is set, the DC         */
        return(SUCCESS);   /* must be recalculated by the system */

    SemLeave(Device);      /* Unlock DC.  Call back to engine    */
                           /* to force DC calculation.           */

    VisRegionNotify(hdc);  /* Loop back to reset lock and recheck*/
   }  while (TRUE);
  }
```

Changes to the Graphics Engine

In the Presentation Manager for OS/2 2.1 and previous versions, a presentation driver had to support more than 70 functions. In addition, it was also mandatory that the presentation driver keep device-contextual state information and, in the case of display presentation drivers, run some functions at interrupt time.

The design of the graphics engine in OS/2 Warp, Version 3 focuses on taking the complexity out of developing a presentation driver while keeping the expandability and flexibility it was originally designed to have.

This graphics engine has two major changes from OS/2 2.1 and previous versions:

- Device-independent functions merged into the graphics engine
- System rasterization support (referred to as SOFTDRAW)

This design is intended to do the following:

- Make presentation drivers simple and easy to develop
- Allow developers to add functionality in increments
- Move contextual and state management out of the presentation drivers
- Move common rasterization management out of the presentation drivers

The following table identifies the support for the Presentation Driver Model that is available in various versions of OS/2.

OS/2 Release	16BPD	32BPD	NMPD
Runs on OS/2 1.X	X		
Runs on OS/2 2.0, 2.1	X	X	
Runs on OS/2 Warp, Version 3	X	X	X

Note:

16BPD	Presentation Driver compiled as 16 bit
32BPD	Presentation Driver compiled as 32 bit
NMPD	New model of Presentation Driver

Graphics Engine/Presentation Driver Design Changes

In the graphics engine for OS/2 Warp, Version 3, the presentation driver has to support only the current OS2_PM_DRV_ENABLE and DEVESC_ functions. A new enable subfunction call, OS2_PM_DRV_ENABLE: Subfunction 0Eh - QueryDeviceSurface, has been added so that the graphics engine can manage the formerly mandatory functions for presentation drivers.

This graphics engine is designed so that a developer can create a presentation driver with minimal effort, and then incrementally add function that supports specific hardware. In order to achieve this, the new graphics engine does the following:

- Manages Presentation Manager contextual and state information
- Performs rasterization into a linear address specified by the presentation driver
- Provides a set of device support routines

Mandatory Functions now handled by the OS/2 Graphics Engine

The following existing functions will be simulated by the graphics engine:

- PolyScanLine

  If the current presentation driver's dispatch table has not hooked out this function, the graphics engine will handle the PolyScanLine call. The graphics engine will convert the shortline structure to a set of rectangles and pass them as ROP_PATCOPY BITBLTS to the presentation driver.

- DrawLinesInPath

  The graphics engine will convert the PLINE structure (which is passed into the DrawLinesInPath function) into the LINEINFO structure required by the SDLine function.

- QueryDeviceCaps

  The device will now give the capabilities to the graphics engine during the OS2_PM_DRV_ENABLE: QueryDeviceSurface call. The graphics engine will then do the default handling on the information provided earlier. The presentation driver can dynamically change its device capabilities through the GreSetDeviceSurface call.

- Bit maps

  The graphics engine will create and manage bit maps if the presentation driver has not hooked the following bit map functions:

  - DeviceCreateBitmap
  - DeviceDeleteBitmap
  - DeviceSelectBitmap
  - GetBitmapBits
  - SetBitmapBits

  The engine will support all standard bitcounts (1, 4, 8, 16 and 24) in packed pixel format. All the bit maps are converted to the device's bitcount except monochrome bit maps. Features supported include the following:

  - All standard bitcount support
  - Only ONE plane (that is, packed pixel)
  - Bits are either BOTTOM to TOP or TOP to BOTTOM
  - BYTE, 2 BYTE or 4 BYTE alignment
  - RLE compression, can add other compression algorithms

- BitBlt

  Bitblt is fully supported. Presentation drivers need not hook this function or call back for simulation. Bit maps could be stretched, flipped, rotated and sheared. The engine will do the simulation and call Bitblt with COM_DEVICE set (if the Bitblt is hooked) or call SDBitBlt. If SDBitBlt is called, the bits are preclipped. Stretching (not rotation or shear) will be done in batches.

- SetPel

The graphics engine converts this call to a BitBlt call with a 1x1x[dst bpp] source bit map. It will behave as described in the *OS/2 Presentation Device Driver Reference* , Fourth Edition.

- GetPel

  The graphics engine converts this call to a BitBlt call with a 1x1x[dst bpp] destination bit map. It will behave as described in the *OS/2 Presentation Device Driver Reference* , Fourth Edition.

- CharString

  CharString will be handled by retrieving the current position and calling the Device Driver Interface (DDI) CharStringPos.

- CharStringPos

  CharStringPos simulation will start by handling the current default requirements of the Device Driver Interface. That is, the string is being output in the left/right direction and supports a raster image of the current font. All simulation currently being handled by the engine will still take place; then on the CharStringPos call with the COM_DEVICE flag set, the default handling will take place.

- PolyMarker

  PolyMarker simulation will be handled by using the default ATM scalable marker that is currently provided for hardcopy drivers. This simulation will give the display marker the ability to support attributes and different sizes with the default marker.

- GetCodePage/SetCodePage

  The engine now stores the code page in the engine's device context. The code page will be provided by the engine if requested by the driver.

- GetDCOrigin

  The engine currently stores the device origin in device coordinates on a device context basis. For simulation, the engine will return these values.

- DevEscape

  This is still a mandatory function. The driver is now required to pass through the DevEscape function to the graphic engine's DevEscape entry point for certain DevEscape simulation (QueryVIOCellSizes).

- Attributes

  Attributes are already stored by the engine. Now the engine will return only the correct values in the case of DeviceGetAttributes, and set either the appropriate dirty flags or physical attributes in the case of DeviceSetAttributes and DeviceSetGlobalAttribute.

- NOP Simulations

  The following functions do not require graphics engine simulation, and will simply provide the appropriate return code:

  - RealizeFont
  - DeviceQueryFontAttributes
  - DeviceQueryFonts
  - SaveScreenBits
  - DeviceSetDCOrigin

OpenDC Modifications

Because of the added driver support, presentation drivers have a smaller subset of mandatory functions. These mandatory functions differ between display and printer drivers. All drivers are still required to handle the Enable subfunctions and a subset of the DEVESC_functions. One new mandatory function is required - OS2_PM_DRV_ENABLE: Subfunction 0Eh - QueryDeviceSurface. This function provides to the graphics engine the information needed for handling the new Device Driver Interface architecture. This call is made from the graphics engine during an OpenDC call.

Design Considerations for Display Drivers

When an application requires a user to choose an object from a presentation space, the user typically selects the object by positioning the mouse cursor over the object and clicking the mouse buttons. This action sends a message to the application, informing the program of the current (X,Y) position of the mouse cursor. However, it is still up to the application to determine the object selected. This is accomplished by the application defining a rectangular area called the "pick aperture," which is centered on the reported mouse position, and determining which, if any, of the currently defined segments intersect or lie completely within the pick aperture. The process of determining intersection or inclusion within the pick aperture is called "correlation."

Naming Conventions

Because some 16-bit hardcopy drivers load a 16-bit DISPLAY.DLL and call it directly, OS/2 includes a 16-bit DISPLAY.DLL, which provides the necessary functions for these hardcopy drivers. Therefore, all 32-bit display drivers must have new and unique names.

Correlation

Correlation must be performed by all display drivers in page coordinates on fully clipped primitives. (Correlation is not required for hardcopy devices.) Correlating on areas is particularly complex because *GreSetCurrentPosition* and *GreEndArea* generate a closure line when the current position is not at the start of the current closed figure. This closure line can cause a correlation hit. Also, the area interior, itself, can cause a correlation hit that must be reported on the *GreEndArea* order.

The lines (arcs, full arcs, boxes, and fillets) defining the area boundary can cause a correlation hit if the area is specified with boundary. This hit must be reported when the function is issued.

Correlation and Retained Segments

To be a candidate for correlation, a retained segment must:

- Have a unique identifier
- Be a nondynamic object
- Be defined as detectable (see following explanation)

Each primitive or group of primitives within a given segment must be capable of maintaining tag information. *Tag information* is added to an object in response to an application calling *GpiSetTag*. The pick tag is a positive integer. If 0 is specified as the pick tag, detectability should be turned OFF for subsequent primitives. The pick tag is 0 by default. The tag specified in the most recent call to *GpiSetTag()* is the current tag, and remains in effect for all subsequent drawing operations until another call to *GpiSetTag()* is made. The tag is considered to be a part of the current attributes for the segment and, therefore, is affected by the current attribute mode of the segment.

An application can request correlation data for:

- Segments that have been defined as both detectable and visible
- All nonzero segments that intersect the pick aperture regardless of the object detectability and visibility attributes

The presentation driver returns the names of segments within the pick aperture in reverse order of their occurrence on the segment chain. This data is returned in the form of segment and tag pairs. Each unique segment and tag pair within the pick aperture is termed a *correlation hit* . If two or more primitives within the current pick aperture have the same tag, they are considered as a single correlation hit. When a called segment is picked, correlation data is also returned for all segments above it in the hierarchy (up to and including the root segment). This also constitutes a single correlation hit.

Correlation and Nonretained Segments

*Nonretained graphic*s are those objects that are correlated on *during* the drawing process. If nonretained objects are to be correlated on, they must have unique identifiers and be defined as detectable. The application must set the correlate bit of the Draw Control flag passed to the engine from the function *GpiSetDrawControl* .

Drawing to Display Devices

Because changes on the screen can affect more than one device context, the graphics engine notifies the display driver when a change occurs. Notification is performed through a call to the function (see GreDeviceInvalidateVisRegion) in the display driver. This routine identifies the affected device contexts and supplies a pointer (*pInstance*) to the instance data of each device context. The handling routine for *GreInvalidateVisRegion* sets a flag, such as HDC_IS_DIRTY, in the instance data for all identified device contexts.

All routines that draw to the screen test the HDC_IS_DIRTY flag. If this flag is set, the routines call VisRegionNotify before drawing the visible region.

Distributed Console Access Facility (DCAF)

The IBM Distributed Console Access Facility (DCAF) provides remote console capability via communication links. DCAF allows one programmable workstation to act as a controlling workstation. From this workstation the user can control and monitor the display output of another programmable workstation, called a target workstation.

Operating States

DCAF has two operating states: monitoring and active.

Monitoring State: When a DCAF session is in this state, the controlling workstation displays an image of the target workstation's display. The target workstation user, however, retains complete control of operations on the target workstation.

Active State: When a DCAF session is in this state, the user of the controlling workstation, referred to as the c*ontroller,* operates and controls the target workstation. Any keystrokes typed by the controller, and the resulting screen image, are displayed on the target workstation, just as if the target user had typed the keystrokes directly. As in the monitoring state, the controller sees an image of the target workstation's display. In the active state, keystroke input from the target workstation is not accepted. However, if the hot key combination is enabled for the target workstation, the target user can use the key combination to regain control of the workstation and change the session state.

Concurrent sessions can be established on multiple target workstations by a single controlling workstation. On the other hand, a target workstation can be connected to only one controlling workstation session at a time, even though multiple controlling workstations can be defined within the network.

For more information about DCAF, see Distributed Console Access Facility (DCAF) Architecture in Display Drivers, as well as the individual functions in Mandatory and Simulated Graphics Engine Function Reference.

Direct Interface Video Extensions (DIVE)

The first implementation of Graphics Driver interfaces for high speed access to graphics hardware were shipped in the OS/2 2.1 DDK as extensions or DEVESC_ functions. The DEVESC_GETAPERTURE, DEVESC_ACQUIREFB, DEVESC_SWITCHBANK and DEVESC_DEAQUIREFB functions provide a device independent interface for application and system software to access display memory directly. With these DEVESC_ functions, time-sensitive software requiring simple, fast graphics throughput achieves a sizable performance boost.

Using the DIVE architecture, the OS/2 multimedia subsystem writes directly to video memory. As a user of the DIVE driver functions, the subsystem performs all window clipping, color translation and scaling required to write the data correctly in a window. Additionally, on bank-switched configurations, software using the DIVE driver functions must detect when video memory bank boundaries are crossed and then switch the bank correctly

---

Enhanced Direct Interface Video Extensions (EnDIVE)

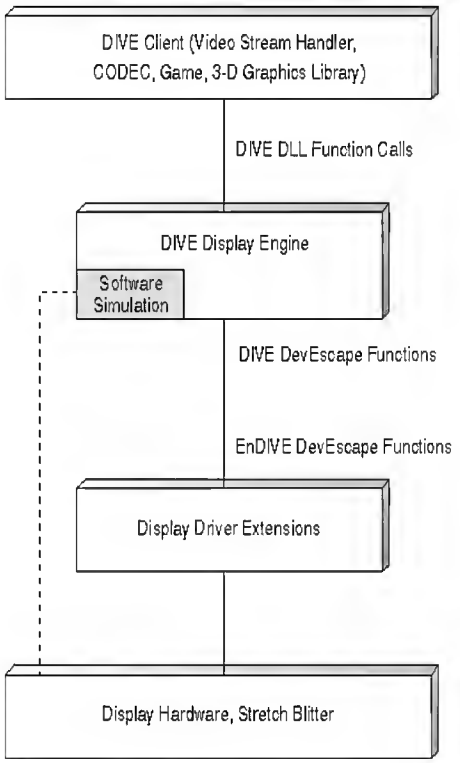
In OS/2 Version 3, the DIVE driver architecture has been enhanced to support the use of motion video accelerator hardware. This enhanced DIVE architecture is called EnDIVE. It provides additional DEVESC_ functions for querying information about the color formats supported by the hardware, video memory managment and image processing.

When the hardware is present in the graphics system, OS/2 display drivers look for and load a "buddy" DLL that supports the EnDIVE functions. In turn, software using DIVE architecture drivers will detect and use the additional functions to accelerate image processing operations.

OS/2 Version 3 DIVE Display Engine

The DIVE Display Engine simplifies the complexities of dealing with the DIVE and EnDIVE driver functions. As a single, system-level API, the Display Engine enables efficient transfer of data to video memory with clipping, scaling and color translation. The Display Engine also abstracts the DIVE and EnDIVE driver interfaces to a higher level.

Using the DIVE Display Engine, applications can either write directly to video memory or use the DIVE blitter to get a high level of screen update performance, image scaling, color translation and bank-switch display support. The DIVE Display Engine also detects and uses acceleration hardware (via EnDIVE) and substitutes software algorithms whenever necessary. In this way, the DIVE Display Engine abstracts the low-level DIVE and EnDIVE driver interfaces to a higher level, adding software emulation for operations that each DIVE driver user has to do.



Design Considerations for Hardcopy Drivers

For information on hardcopy drivers written to the graphics engine dispatch table, see the following chapters:

- Graphics Engine Hardcopy Drivers
- Presentation Manager Function Categories
- Exported Driver Function Reference
- Mandatory and Simulated Graphics Engine Function Reference

Design Considerations

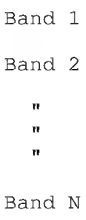
The following design considerations only apply to hardcopy drivers:

- Banding
- Document processing
- Dynamic Job Properties
- Extended attributes
- Hardcopy device names
- Hardcopy driver migration
- Hardcopy driver output to file
- Help
- Job error dialog
- Output threads

Banding

Banding is another technique that is available to presentation drivers for raster technology hardcopy devices. Banding is the process of dividing a whole page into two or more bands (or strips) of raster data, which is recorded in memory as a bit map, and is then sent to the physical device or spooler. It is used to reach a balance between memory requirements and performance. This technique uses the graphics engine's journaling functions to save and replay a journal file of the calls to the Gre*xxx* functions for a whole page.

The hardcopy driver handles the output page as a number of bands and creates a bit map large enough for one band at a time (see illustration below).



The device context origin of the bit map is manipulated so that it relates to each band in turn. The hardcopy driver replays the journal file as many times as necessary to write into each band. Note that Band 1 cannot be written into while the calls to the Gre*xxx* functions are being journaled. This is because the Command flag, COM_DRAW, is turned OFF between calls to *GreStartJournalFile* and *GreStopJournalFile* unless the JNL_DRAW_OPTIMIZATION flag is passed in on the call to function *GreCreateJournalFile()* . The hardcopy driver is told not to perform any output while the Gre*xxx* calls are being journaled unless the JNL_DRAW_OPTIMIZATION bit flag is set.

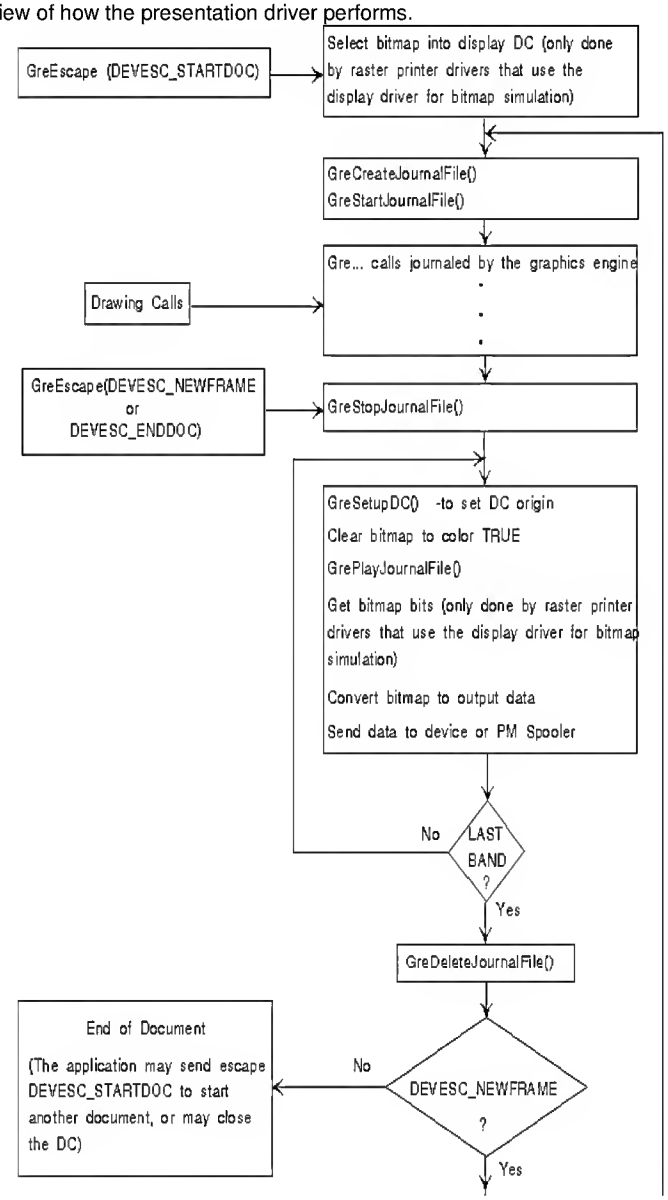
The size (width and height) of each band is determined by each hardcopy driver, dependent upon the type of physical device to which the output is to be sent and the amount of memory the hardcopy driver can use to build its bit map. As an example, a color laser printer might need the full 24 bits per pel, in which case, several bands might be needed to make a page. A simple dot-matrix printer that uses one bit per pel could treat the whole page as a single band.

The data for each band is sent as a single band to the physical device for an OD_DIRECT device context, or to the spooler for an OD_QUEUED device context with a data type of PM_Q_RAW. Note that the number of bytes of data that are sent might not be the same as the number of bytes required to create the bit map for a given band. This can be due to compression algorithms, which might be implemented in a given hardcopy driver and supported by a given physical device.

The technique of banding is performed by recording all of the graphics for a whole page in a journal file. When the journal file is complete (that is, the hardcopy driver has received a DEVESC_NEXTBAND escape, a DEVESC_NEWFRAME escape, or a DEVESC_ENDDOC escape), the hardcopy driver plays the journal file, reprocesses the calls recorded to produce each band in turn, and clips the graphics recorded in the journal file to each band output. After all bands are output, the journal file is deleted. This banding technique uses the graphics engine journaling functions to save and replay a journal file of the Gre*xxx* calls. These graphics engine journaling functions are documented in Journal Functions (Hardcopy Drivers Only).

Each page of output is handled as a separate entity. The *GreEscape* routine for DEVESC_STARTDOC opens a journal file for the first page and registers it in the device context instance data. When *GreEscape* DEVESC_NEWFRAME or DEVESC_ENDDOC is received, the hardcopy driver writes the bands and closes the journal. If the escape code was DEVESC_NEWFRAME, the *GreEscape* routine opens and registers the journal file for the next page.

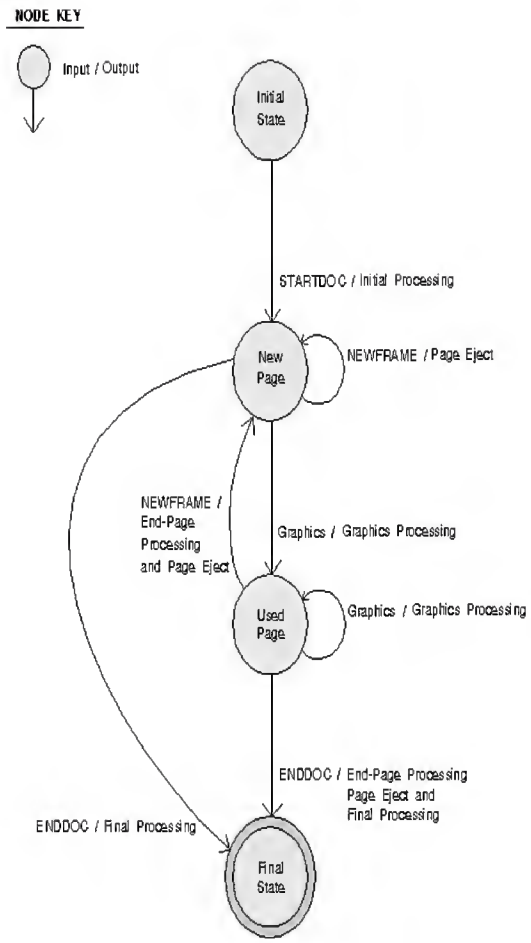
The following figure gives an overview of how the presentation driver performs.



Overview of Presentation Driver Performance

Document Processing

The following figure diagrams the state transitions for the *GreEscape* function calls used during document processing. The *GreEscape* function names are abbreviated for ease of reading. For example, *GreEscape* (DEVESC_STARTDOC) is shown as STARTDOC. See Spool File Creation for details on each of the output actions.



State Transitions for Document Processing by Hardcopy Drivers

---

Dynamic Job Properties

Dynamic Job Properties (DJP) provides two new printing enhancements. The first enhancement makes it possible to set up a print job without relying on user intervention. This is an improvement over the current "black box" configuration, which requires that the printer driver's job properties be displayed in a dialog box for a user to edit.

The second enhancement makes it possible to change the print format within the same job. Currently, a four page legal document that includes a high resolution color bitmap and a letter envelope would have to be printed out as three separate jobs, requiring the user to set the job properties of each job to match the specified format (monochrome legal text; high-resolution, 24-bit per pel bitmap; and monochrome envelope). With the DJP enhancement, a program can pass in different job properties with each page, rather than bringing every page up to the highest format in the job.

Refer to Mandatory and Simulated Graphics Engine Function Reference for a detailed description of each of the following functions:

- GreEscape DEVESC_DEFAULTJOBPROPERTIES
- GreEscape DEVESC_NEWFRAME_WPROP
- GreEscape DEVESC_QUERYESCSUPPORT
- GreEscape DEVESC_QUERYJOBPROPERTIES
- GreEscape DEVESC_QUERYSIZE
- GreEscape DEVESC_SETJOBPROPERTIES
- GreEscape DEVESC_STARTDOC_WPROP

Design Overview of DJP

DJP can be applied to two different scenarios. The first scenario applies to applications that query or modify DJP_ITEMs; these events are usually performed asynchronously. In this scenario, the application opens up a OD_INFO DC, sets up its job property information, and then turns around and opens up a OD_QUEUED DC to print. Setting or querying data uses only the printer property information in the device context (DC) and does not affect the print job in any other way.

In the other scenario for using DJPs, the DC characteristics provided in the DevOpenDC information are changed. This is either accomplished at the start of a document with GreEscape DEVESC_STARTDOC_WPROP or at the beginning of a new page with GreEscape DEVESC_NEWFRAME_WPROP. In this scenario, the application must query the printer driver again and again for DevQueryCaps, DevQueryHardcopyCaps, list of devices fonts, and any other DC information. The driver must use the same code path that DevOpenDC uses (FillLogicalDeviceBlock, FillPhysicalDeviceBlock, EnableDeviceContext, CompleteOpenDC). After the printer driver resets all of its information, it calls the new Gre call, GreResetDC2 (hdc, 0). This call instructs the graphics engine to query the printer driver again and recalculate its internal state data. This has the side effect of performing a ResetDC.

The DJP_ITEM is the new structure that contains DJP information. The structure DJP_ITEM is used by both applications and device drivers. The format of the structure follows.

```
/* Dynamic Job Property Item
*/
typedef struct _djpItem {
   ULONG       cb;              // size of this structure
   ULONG       ulProperty;      // Type of Property
   LONG        iType;           // INPUT : DJP_ALL - Returns all possible
                                //         values (placed at ulValue).
                                //         DJP_ALL is not valid if
                                //         setting properties.
                                //
                                //         DJP_CURRENT - returns current
                                //         value
                                //
                                // OUTPUT: DJP_ERROR_XXX if there was an
                                //         error
   ULONG       ulNumReturned;   // How many elements are returned in
                                // ulValue on a query. This should be
                                // 1 for DEVESC_SETJOBPROPERTIES.
   ULONG       ulValue;         // The start of the types supported by
                                // this property.
} DJP_ITEM, *PDJP_ITEM;
```

Because the individual properties have variable lengths, the properties are broken up into two different categories: simple and complex. The names of these properties are identified by either "_Sx_" for simple or "_Cx_" for complex. In addition, they are sorted into different subcategories such as "_xJ_" for job properties or "_xP_" for printer properties. Simple types are ULONG sized. This is the minimum size for the data. The complex types can be larger and have structures associated with them. All property types, simple and complex, are associated with a type; for example, DJP_SJ_ORIENTATION is a simple type of type PDJPT_ORIENTATION. The values for DJP_SJ_ORIENTATION can be DJP_ORI_PORTRAIT or DJP_ORI_LANDSCAPE. DJP_CJ_RESOLUTION is a complex type of type PDJPT_RESOLUTION; this is a structure composed of two members: x and y resolutions.

The following helper macros have been created to work with DJP_ITEMs: DJP_NEXT_STRUCTP(), DJP_ELEMENTP(), and DJP_SET_ELEMENT().

DJP_NEXT_STRUCTP():

This macro is used for walking the list of item structures. It takes a pointer to the current item and returns a pointer to the next item and depends on the size of the structure set in the variable cb.

```
// Bump to the next DJP_ITEM structure
pDJP = DJP_NEXT_STRUCTP (pDJP);
```

DJP_ELEMENTP():

This macro takes a pointer to the current item and returns a pointer to the element type passed in. Each job property type has a corresponding element type.

```
// Get the pointer to the resolution value
PDJPT_RESOLUTION    pElm;

pElm = DJP_ELEMENTP (pDJP, DJPT_RESOLUTION);
```

DJP_SET_ELEMENT():

This macro will take three parameters: The first is a DJP_ITEM structure, the second is the element type, and the third is the DJP element. This function helps to deal with complex structures, but is not useful for dealing with simple structures. The following example shows how to set DJP_CJ_RESOLUTION (complex property) to use a 720x720 resolution:

```
// Structure (from header file DJP.H)
typedef struct _djpResolution {
   USHORT     usXResolution; // X resolution (in dots per inch)
```

```
   USHORT     usYResolution; // Y resolution (in dots per inch)
} DJPT_RESOLUTION, *PDJPT_RESOLUTION;

// Set the # of bytes in the structure
pDJP->cb = DJP_HEADER_SIZE + sizeof (DJPT_RESOLUTION);

// Set the current property and type
pDJP->ulProperty = DJP_CJ_RESOLUTION;
pDJP->iType      = DJP_CURRENT;

// Set up the resolution structure
djpResolution.usXResolution = 720;
djpResolution.usYResolution = 720;

// Put the data into the DJP_ITEM structure.
DJP_SET_ELEMENT (*pDJP, DJPT_RESOLUTION, djpResolution);

To set a simple type such as the color mode use:

pDJP->cb         = sizeof (DJP_ITEM);
pDJP->ulProperty = DJP_SJ_COLOR;
pDJP->lType      = DJP_CURRENT;
pDJP->ulValue    = DJP_CLR_COLOR;

/* Move to next element. Either way, you use the same macro.
** Dependent on cb having the correct value.
*/
pDJP = DJP_NEXT_STRUCTP (pDJP);
```

The parts of the DJP property defines identified by _Sx_ and _Cx_ indicate whether the specified item is a simple item or a complex item. This information helps you to determine the correct strategy for setting the structure size and the data value.

DJP Design Restrictions

DevEscape is a preexisting API. This means that is a metafile. Metafile programs can work with old drivers (returning error codes), or both the new programs and new printer drivers can work on old operating system levels.

Unfortunately, DevEscape has only two pointers to data blocks, each with two corresponding sizes. This limits what can be passed through the API. If a DJP is changed, you must re-query all of the HDC (the device context handle) characteristics. This includes font information, DevQueryCaps, DevQueryHardcopyCaps. In addition, previously opened memory HDCs are no longer valid or compatible with this DC.

Engine Interfaces to the Printer Driver

DJP uses two paths to communicate with the driver. The first path is the preexisting DevEscape. This is hooked out in the dispatch table at location NGreEscape & 0x01FF. The second is a new API DevPostEscape. This is an exported entry point in the .DEF file (OS2_PM_DRV_POSTESCAPE @206).

The prototype for DevEscape follows:

```
LONG ENGENTRY Escape (HDC       hdc,
                      LONG      lEscape,
                      LONG      cInCount,
                      PBYTE     pInData,
                      PLONG     pcOutCount,
                      PBYTE     pOutData,
                      PDDC      pddc,
                      ULONG     ulFunction)
```

The prototype for DevPostEscape is:

```
ULONG APIENTRY OS2_PM_DRV_POSTESCAPE (PSZ       pszDriverName,
                                      PSZ       pszDeviceName,
                                      PSZ       pszQueueName,
                                      PSZ       pszSplPrinterName,
                                      ULONG     ulFuncNum,
```

```
                                        ULONG     cbParm1,
                                        PBYTE     pbParm1,
                                        ULONG     cbParm2,
                                        PBYTE     pbParm2)
```

The following information shows how these two functions are similar (note that most of the parameters are the same).

```
LONG      lEscape      <===> ULONG     ulFuncNum     *
LONG      cInCount     <===> ULONG     cbParm1
PBYTE     pInData      <===> PBYTE     pbParm1
PLONG     pcOutCount   <===> ULONG     cbParm2
PBYTE     pOutData     <===> PBYTE     pbParm2
```

The defines and values are different, but the concept is the same.

```
DevEscapes                      DevPostEscapes

GreEscape                       DEVPE_QUERYSUPPORT
DEVESC_QUERYESCSUPPORT

GreEscape DEVESC_QUERYSIZE      DEVPE_QUERYSIZE

GreEscape                       DEVPE_QUERYJOBPROPERTIES
DEVESC_QUERYJOBPROPERTIES
```

The major difference between DevEscapes and DevPostEscapes is that DevEscape uses an already opened device context handle (HDC), and printer property information is already "hardcoded" at DevOpenDC time. If another device's job properties are passed in, or if another queue's job properties is used with different printer property information, misleading information will be returned.

To overcome this limitation with DevEscape, DevPostEscape passes in four strings with all the information required to find both the printer and job property data. However, DevPostEscape also has a limitation: DevPostEscape behaves like DevPostDeviceModes() in that it loads and unloads the printer driver for each call to DevPostEscape. When DevPostEscape is used before a DevOpenDC() is issued against a specified printer driver, the number of opened instances against the printer driver is 0 (after DevOpenDC, the number will be at least 2 or more). When the number of references is 0, the printer driver will behave as if it is being called for the first time and will load all of its global resources, such as string tables and device data structures. This process can be time consuming, and, if DevPostEscape is called a number of times before DevOpenDC, performance will suffer.

Minimum Support for DJP

To provide minimum support for DJP, the printer driver must accomplish the following:

1. Export OS2_PM_DRV_POSTESCAPE @206 for the new DevPostEscape API.

2. Support the following DevEscapes and their equivalent DevPostEscapes:

```
DevEscapes                      DevPostEscapes

GreEscape                       DEVPE_QUERYSUPPORT
DEVESC_QUERYESCSUPPORT

GreEscape DEVESC_QUERYSIZE      DEVPE_QUERYSIZE

GreEscape                       DEVPE_QUERYJOBPROPERTIES
DEVESC_QUERYJOBPROPERTIES

GreEscape                       DEVPE_SETJOBPROPERTIES
DEVESC_SETJOBPROPERTIES

GreEscape                       DEVPE_DEFAULTJOBPROPERTIES
DEVESC_DEFAULTJOBPROPERTIES

GreEscape                       GreEscape
DEVESC_STARTDOC_WPROP           DEVESC_NEWFRAME_WPROP
```

3. Support the following DJP_ITEMs:

DJP_NONE
DJP_SJ_ORIENTATION
DJP_CJ_RESOLUTION
DJP_SJ_BITSPERPEL
DJP_SJ_COLOR
DJP_SJ_PRINTQUALITY
DJP_SJ_PAPERSIZE
DJP_CJ_FORM
DJP_SJ_COPIES

Note: You must support the GreEscape DEVESC_QUERYSIZE/DEVPE_QUERYSIZE for all DJP_ITEMs (returning a count of 1).

Unsupported DJP Items

If your device does not have a DJP_ITEM in its job properties, then the following strategy should be implemented:

```
for DEVESC_QUERYSIZE return the sizeof (DJP_ITEM)
   pQuerySize->ulSizeNeeded += sizeof (DJP_ITEM);

for DEVESC_QUERYJOBPROPERTIES
    pIQuery->ulNumReturned = 0;
    pIQuery->ulValue       = DJP_NONE;
    pIQuery->lType         = DJP_ERROR_NOT_SUPPORTED;

    and return DEV_WARNING

for DEVESC_SETJOBPROPERTIES
    pISet->lType = DJP_ERROR_NOT_SUPPORTED;

    and return DEV_WARNING
```

Using DEVESC_STARTDOC_WPROP and DEVESC_NEWFRAME

GreEscape DEVESC_STARTDOC_WPROP and GreEscape DEVESC_NEWFRAME_WPROP are used to change your device characteristics. When an application calls DevOpenDC, it passes your job property information. DJP allows an application to give new job property information on a page-by-page basis. The printer driver must reinitialize its internal structures based on the new job property information. If the device is placed into one mode and a new job property forces the device into a different mode, then it is acceptable for the printer driver to terminate the job and reinitialize the device. To keep the spooler from terminating and reinitializing, avoid using PrtClose() to close the job. (Because the device context is essentially different than it was at DevOpenDC time, all memory compatible DCs, any information returned from DevQueryCaps or DevQueryHardcopyCaps, and any font related information will have to be requeried or recreated by the application.)

Extended Attributes

Extended attributes (EA) are used for correct installation and operation of multifile drivers. They are also used for improved performance during installation.

Extended attributes serve three purposes:

- Installation of multifile drivers
- Version numbering
- Improved performance of drivers in the Workplace Shell*

  Performance is improved because the Workplace Shell no longer needs to open a Device Context to the driver, which causes the driver to be loaded each time it needs Icon and Device Name information.

Names are separated by commas when more than one name is defined in an extended attribute. The following examples of extended attributes are for the HP LaserJet** hardcopy driver.

Installation of Multifile Drivers

- **VENDORNAME (Optional)**

  Vendor name of the supported printers, such as VENDORNAME=HP.

- **REQUIREDVENDORFILES (Optional unless the driver requires other files)**

  Names of extra files required for correct operation by the driver files in the VENDORNAME directory, such as REQUIREDVENDORFILES=HP_ADDF.DLL.

- **OPTIONALVENDORFILES**

  Names of extra files that are optional, that is, files not required for correct operation, such as OPTIONALVENDORFILES=PCLHELP.HLP, HP_ADDF.SYM, HP_ADDF.MAP.

- **CLASSNAME (Optional)**

  Name of the output stream created by the hardcopy driver, such as CLASSNAME=PCL.

- **REQUIREDCLASSFILES (Optional unless the driver requires other files)**

  Names of extra files required for correct operation by the driver files in the CLASSNAME directory, such as REQUIREDCLASSFILES=GENERIC.DLL.

- **OPTIONALCLASSFILES**

  Names of extra files that are optional, that is, files not required for correct operation, such as OPTIONALCLASSFILES=*.FNT, GENERIC.SYM, GENERIC.MAP.

- **REQUIREDDRIVERFILES**

  Names of extra files required by the driver for correct operation, such as REQUIREDDRIVERFILES=LASERJET.DRV.

- **OPTIONALDRIVERFILES**

  Names of extra files that are optional, that is, files not required for correct operation, such as OPTIONALDRIVERFILES=LASERJET.SYM, LASERJET.MAP.

Subdirectory Structure

The subdirectory structure on the startup driver created by the hardcopy driver installation is as follows:

```
OS2\DLL\VENDORNAME\CLASSNAME\DRVNAME
```

An example of this subdirectory structure might be OS2\DLL\HP\PCL\LASERJET. Most hardcopy drivers need to be installed only in one subdirectory. In this case, omit CLASSNAME and ensure that VENDORNAME is equal to the name of the DRV file.

For example, to install a Postscript** driver in a subdirectory, you might use the following extended attributes:

```
VENDORNAME = PSCRIPT
REQUIREDDRIVERFILES = PSCRIPT.DRV
OPTIONALDRIVERFILES = PSCRIPT.HLP
```

This will install the PostScript driver into a subdirectory named PSCRIPT.

Version Numbering

- **.VERSION (Optional)**

  Indicates the version number of a hardcopy driver. For example to indicate a hardcopy version number of 13.328, the extended attribute might be `.VERSION=13.328`. Hardcopy drivers can load this attribute and display it in OS2_PM_DRV_DEVMODE dialogs. See Hardcopy Driver Migration for more information.

Improved Performance of Drivers in the Workplace Shell

- **.EXPAND (Optional)**

  Lists the device names supported by a hardcopy driver. Using this extended attribute improves performance over use of OS2_PM_DRV_DEVICENAMES (especially from a diskette). Names (values) for this extended attribute are NULL-terminated strings. An example of this extended attribute is `.EXPAND=HP LaserJet II\0HP LaserJet III\0\0`.

- **.ICON (Optional)**

  Gives the icon definition used by the Workplace Shell to display this hardcopy driver. This extended attribute is automatically created during the build process if the ICON keyword is used with a resource identifier of 1, or if the DEFAULTICON keyword is used in the RC file for a hardcopy driver. An example of this extended attribute would be `.ICON 1 LASER.ICO`.

- **.HIDDEN (Optional)**

  Indicates which hardcopy driver is used by files associated with a printer driver. This attribute is used by the Workplace Shell to prevent display of these files in a directory folder. An example of this extended attribute is `.HIDDEN=LASERJET.DRV`.

Hardcopy Device Names

If a hardcopy driver supports multiple devices, this is indicated at the API and user level. There are three parts:

- The hardcopy driver implements OS2_PM_DRV_DEVICENAMES.
- The hardcopy driver accepts a device name in the *szDeviceName* field of *pdriv* . See FillPhysicalDeviceBlock.
- The device name is accepted in OS2_PM_DRV_DEVMODE.

Hardcopy Driver Migration

Hardcopy drivers must be able to work with earlier levels as well as future levels of drivers across a network. Notice that there are several possible driver version numbers:

- EA Version Number. Derived from a build number (such as 13.160)
- Dialog Version Number. Derived from the build number. Service representatives can use this number to uniquely identify the build of the hardcopy driver and the fix level.
- INI File Data Version Number. Identifies to the driver the version number of the data stored in the OS2SYS.INI file. The numbering system and format is defined by the hardcopy driver.
- lVersion Number. Given in the *DrivData* structure of job or printer properties. The numbering scheme of this ULONG is defined by the hardcopy driver, but the following format of *lVersion* is suggested. It is a FIXED type of number.
    - High WORD. System level in BCD (such as 0020)
    - Low WORD. Build level in BCD (such as 0160).

The numbering scheme for drivers is as follows:

New Driver	20.0000
Minor Update	20.0001
Major Release	21.0000

For drivers reading data from an earlier version of the driver, the data must be interpreted (by using the *lVersion* number as a guide to its format) and any missing data must set to the device defaults. For drivers reading data from a later version of the driver, as much data as possible should be read and the rest ignored. This implies that later versions of the driver only add new driver data fields at the end of the data.

Hardcopy Driver Output to File

Hardcopy drivers can print data to a disk file so that the print file can be sent to the printer later without any need for application intervention. The print file can be used as a file interchange format. *Printing to file* means the hardcopy driver converts the *prespool* input into a device-dependent format and stores the output directly to a file. File system errors must be reported.

There are two methods the hardcopy driver can implement to allow printing to disk. The system-provided method for printing to disk requires that the hardcopy driver handle a file name as a *szLogAddr*ess on a OD_DIRECT Enable Subfunction 02H - *FillPhysicalDeviceBlock* . The other method for printing to disk enables the hardcopy driver to change the job properties dialog to allow the user to input a fully-qualified file name. There are circumstances in which the hardcopy driver must know the format of the required output file. For example, a hardcopy driver can output raw PostScript or Encapsulated PostScript (EPS).

Help

All hardcopy drivers have a Help function that invokes contextual assistance.

Job Error Dialog

Use the SplMessageBox function to display error messages from hardcopy drivers. The following push buttons in a message box are presented to an end user:

RETRY	Retry sending print data
ABORT	Delete job
IGNORE	Cancel dialog

The hardcopy driver will respond to each of the returns in the following manner:

MBID_RETRY	Continue sending data to the output buffer (*PrtWrite*)
MBID_ABORT	Issue a *PrtAbort* to tell the spooler to delete the current job and set a flag that the job has been ended; then return from the write thread

MBID_IGNORE Continue sending data to the output buffer (*PrtWrite*)

The Job Error dialog box contains a **Help** push button and associated help.

Output Threads

Printer drivers should buffer output into large blocks and call *PrtWrite* on a separate thread. Use *DosCreateThread* to start this separate thread when the first printer-specific commands are to be written.

Display Drivers

This chapter describes video cursor support and the installation utility program used by OS/2 display drivers.

Exported Entry Points (Cursor)

The following entry point is exported by the dynamic link library of a display driver:

```
EXPORTS
   MoveCursorBlock     @103
```

MoveCursorBlock

Display drivers must export an entry point for the *MoveCursorBlock* table. This table contains information about the display driver's *MoveCursor* routine (code) and data areas. The table values are checked after the display driver is initialized. This allows the driver to determine the correct values.

```
typedef struct _MCDESCRIPTION {
    PVOID    pMoveCursor;
    ULONG    ulCodeLength;
    PVOID    pCursorData;
    ULONG    ulDataLength;
} MCDESCRIPTION;
typedef MCDESCRIPTION * PMCDESCRIPTION;
```

The fields in the MCDESCRIPTION data structure are described below:

pMoveCursor	Flat address of the *MoveCursor* routine in the display driver.
ulCodeLength	Length in bytes of the *MoveCursor* routine
pCursorData	Flat address of the data area used by the *MoveCursor* routine
ulDataLength	Length in bytes of the data area used by the *MoveCursor* routine

This routine supports calls from the system timer (at interrupt time). The strategy for the *MoveCursor* routine is that the pointer is checked and, if necessary, redrawn or excluded at timed intervals.

The PMDD.SYS physical device driver creates a privilege level 0 alias for the data address and passes the alias to the routine in the EAX register when *MoveCursor* is called at interrupt time. Therefore, all data addressing within the routine must be performed relative to this address.

At entry to the *MoveCursor* routine, the stack contains the following:

```
VOID MoveCursor(LONG abs_x, LONG abs_y, PVOID pCursorData)
```

Using the C calling convention, the stack contains two LONGs, which hold the X- and Y-coordinates of the cursor hot spot, and a PVOID that is a pointer to the cursor data area that is valid in the current context. All references to this data area must be done relative to the address passed in. If the X- and Y-coordinates are set to 0x80000000, this signifies a *CheckCursor* call.

Regular timer interrupts give the presentation driver an opportunity to check whether the pointer is valid. For example, the driver can check the following:

- Have new X- and Y- coordinates been set?
- Is the pointer excluded because of a drawing operation? If so, has that operation been completed?
- Is the pointer currently visible (although it should be excluded) because it is in an area that is being redrawn?

At the end of the *MoveCursor* routine, a check is performed to see if a new location was given for the pointer while it was being drawn. If the pointer has moved again, it must be drawn at the new location or be excluded because it has moved into the protection rectangle. This implies that the routine needs to track both real (X,Y) and pointer (X,Y) coordinates.

Programming Considerations

Typical cursors are an arrow or a cross with an action point called the *hot spot* at the point of the arrow or the center of the cross. When the presentation driver draws a cursor, the origin of the image must be offset to place the action point at the required (X,Y) position. The required offset is specified in the call to GreSetCursor. Because the cursor entry point can be called at various times from many different places, the cursor routine uses semaphores to protect itself (protection is the responsibility of the presentation driver). Similarly, because cursor drawing can be a time-consuming operation, the display driver must also protect itself against re-entrance.

The display driver must resolve all interactions between cursor drawing at interrupt time and access to video hardware. While in the background, the display driver does not draw any cursor image.

Caution should be used when the display is a buffered device and the cursor is drawn into a bit map in the buffer. In this case, the display driver deletes the cursor and excludes it when a draw operation occurs at the cursor location. To do this, the driver does a "hit test" for each output operation to see if the cursor location is in the drawing area and to set a protection rectangle that is used to exclude the cursor.

Introduction to Display Driver Installation

The OS/2 2.0 Display Driver Installation utility program (DSPINSTL.EXE) provides all the facilities necessary to install and configure OS/2 display drivers. The OS/2 display driver installation installs and configures PM display drivers. It might also include the installation and configuration of WIN-OS/2 display drivers, and the installation of base video service (VIO) drivers and DOS virtual device drivers (VDDs).

Display driver installation and configuration is more complex than standard device driver installation. It typically includes the following activities:

1. Allowing the user to select OS/2 display characteristics - Specification of the type and resolution of the display driver to be installed. (See Allowing the User to Select OS/2 Display Driver Characteristics)

2. Copying display driver-related files - Copying display driver-related files from source directories (usually on diskettes) to target directories on the user's hard disk (see Copying Display Driver-Related Files).

3. Updating system configuration files - Appropriately updating various OS/2 and WIN-OS/2 configuration files to ready the new display driver for subsequent execution when the user's system is restarted (see Updating System Configuration Files).

Allowing the User to Select OS/2 Display Driver Characteristics

The user needs to specify whether the display driver to be installed will be used as the primary display adapter or as the secondary display adapter.

The user must then specify the type of display driver to install. This is important because many types of display adapter hardware can support a number of different display drivers. For instance, extended graphics array

XGA display adapter hardware can be operated using either the high-resolution XGA display driver or other lower resolution standard display drivers such as video graphics array (VGA) or enhanced graphics adapter (EGA) display drivers.

Finally, because some display drivers can support operation at more than one graphics resolution, the user might have to specify the display resolution at which the driver is to be operated. Display resolutions are typically specified by supplying the width and height of the display (in pels) and the maximum number of colors that can be simultaneously displayed. The user's choices for display resolution are often affected by the type of video hardware attached to the user's system. For example, both the size and type of the user's display and the amount of video RAM on the display adapter can affect which display resolutions can be run.

Copying Display Driver-Related Files

There are several factors that can complicate the copying of display driver-related files from source directories to target directories. First, display driver files might be sufficiently large and numerous to span more than one source diskette. This can easily occur when several display drivers that support different display resolutions are bundled together on the same diskettes (along with their associated font and resource files). Second, to save space within the source directory, it might be desirable to place packed files within the source directory. If so, the packed files should be unpacked when they are copied to the target directory. Third, the display driver packager might wish to specify file version checks that are to take place before file copying is performed. Such checks are valuable because they can prevent the inadvertent replacement of updated display driver files by older versions of the files.

Updating System Configuration Files

After display driver-related files have been copied to the user's system it is necessary to update different system configuration files to complete the installation process. Frequently, the CONFIG.SYS and OS2.INI files need to be updated to complete PM display driver installation and the SYSTEM.INI and WIN.INI files need to be updated to complete the WIN-OS/2 display driver installation. Depending on the display driver to be installed, other configuration files might also have to be updated.

Display Driver Installation Utility Program

DSPINSTL, the display driver installation utility program, is similar in some ways to the OS/2 Device Driver Installation utility program, DDINSTAL.EXE. DSPINSTL contains extensions that are specifically geared to simplify the steps that the user must perform to install and configure OS/2 display drivers. Because different types of OS/2 display drivers can have radically different installation and configuration requirements, DSPINSTL contains a set of features that allows a display driver developer to customize the installation process to meet the developer's special needs.

DSPINSTL consists of the following six major components:

- DSPINSTL Configuration File (DSC file) Interpreter
- Initial User Interface PM Panels - A standard set of PM panels that serve as a basic front-end user interface (see User Interface PM Panels).
- Action Routine Interface - An interface that allows the display driver developer to (optionally) add custom control logic (see Action Routine Interface).
- Service Functions - A set of standard service functions that can be called from within an action routine (see reference to service functions under Action Routine Interface and DSPINSTL Service Functions).
- Command Language PM Panels - A standard set of PM panels that act as the front end to the DSPINSTL Command Language Interpreter (see Command Language Presentation Manager Panels).
- Command Language Interpreter - The final stage of DSPINSTL that executes the installation and configuration commands that are packed in Display Driver Profile files (DSP files) (see DSPINSTL Command Language Interpreter).

DSPINSTL Configuration File Interpreter

The DSPINSTL Configuration file (DSC file) interpreter allows the display driver developer to specify, within the DSC files, the high level knowledge that characterizes one or more display driver packages. The information contained in the DSC files determines the contents of some of the fields in the standard set of PM panels. It also initializes a chain of data elements that will subsequently be passed to the DSPINSTL Command Language Interpreter that executes as the final stage of DSPINSTL operation.

Display Configuration Files

The DSC files are the key to DSPINSTL's ability to act as a generalized installation tool that can install many different types of display drivers. The display driver developer packages a set of drivers by placing the drivers on a diskette, creating one or more DSP files containing DSPINSTL installation and configuration commands. The developer then creates a configuration file entry within a DSC file that summarizes the key points concerning that display driver package. DSC files should be placed either in the directory that contains the DSPINSTL utility program or in a directory that is contained in the user's DPATH. DSPINSTL checks its directory first, and then if it cannot find a DSC file it searches the DPATH, stopping when it finds a directory that contains one or more DSC files. DSPINSTL reads all the DSC files it finds in that directory. DSC files are distinguished by a file-name extension of .DSC. A configuration file entry includes the following types of information:

- A title string that identifies the driver within the initial DSPINSTL PM panels (Primary Video Adapter and Secondary Video Adapter). These two panels are provided so that users can explicitly select which display drivers to install.
- The name of the dynamic link library (DLL) module (if any) that contains action routines specific to the installation of that display driver package.
- A keyword string that identifies whether the display drivers handle a standard type of display adapter. The following is a list of supported standard-type keyword strings:

Type of Adapter	Keyword Type String
Color Graphics Adapter	CGA
Enhanced Graphics Adapter	EGA
Video Graphics Adapter	VGA
Display Adapter 8514/A	8514
Extended Graphics Adapter	XGA
Super Video Graphics Adapter	SVGA

  These standard-type strings are provided so that DSPINSTL can create a logical association with a display driver as a default type. This association is used if the user accepts DSPINSTL's default selection in the Video Display Configuration PM panel (the initial DSPINSTL PM panel). For example, suppose the panel shows that the user's system has an 8514 adapter as its primary display. If a DSC file entry contains a standard-type string of "8514" and the user accepts the default choice by selecting the **OK** push button, then that file entry will be chosen automatically. If more than one DSC file entry contains standard-type strings of "8514", then DSPINSTL would display a PM panel to allow the user to resolve the ambiguity. If the user checks either the primary or secondary check box, to indicate the desire to override the default logic and explicitly choose a driver, then the PM panel that follows will include list box entries for all the entries in the DSC files that DSPINSTL has read.

- A default chain of data elements that characterize a sequence of installation and configuration steps which, in total, constitute the entire installation and configuration process. Each chain element can include the following data:
  - A required default keyword string that will trigger the interpretation of one or more DSP files in a given source directory.
  - A required default prompt string that will be displayed in a PM panel informing the user of the purpose of the next installation step. This prompt string is included to help the user decide which device or directory to point the utility program toward to accomplish that portion of installation.
  - An optional default prompt string that is used to tell the user which diskette to insert (if the next portion of installation is being performed from a diskette). This feature allows the driver package developer to inform the user of the exact title that is written on a diskette.
  - An optional diskette volume label that allows DSPINSTL to verify that the user has inserted the correct diskette.

- An optional display resolution string that can be used by the developer to display a default display resolution for installation. This string can be passed by an action routine to the Select_Video_Resolution service function.

Typically, there will be one data element within the chain for each diskette in the package. However, the display driver developer can use the chain as desired. For example, one data element in the chain might handle file copy commands, while the next might be used for configuration file updates. The chain of data elements is first passed to the action routines (if any) and is then interpreted by the DSPINSTL Command Language Interpreter in the last part of the process. In this manner a developer can create a default series of installation steps, dynamically modify those steps, and then have the DSPINSTL command interpreter execute all the substeps that constitute each step.

DSC files are composed of lines containing character string entries. The following syntax rules govern the lines within a DSC file:

- All character strings in the file are enclosed in double quotation marks.
- The \" pair can be used to embed a double-quotation character within a character string.
- Comment lines start with an asterisk (*).

DSC files contain one or more display driver package entries. Each entry contains a header section containing general information about the display driver package. The header section is followed by one or more subsections that define the default values for a data chain element. The format of the header section is shown in the following diagram:

```
<Adapter Type Title string>   /* Title strings are up to 256 characters   */

<DLL Module Name string>      /* DLL module, containing action routines   */
                              /*   for that type of adapter               */

<Standard Type Key string>    /* Key string that DSPINSTL uses to identify*/
                              /*   a type of adapter that it recognizes.  */
                              /*   Can be NULL, see below for the complete*/
                              /*   set of standard keys                   */
         "1"                  /* Required parameter, must be set to 1     */
         "0"                  /* Required parameter, must be set to 0     */

<# of Parameter Sets in Entry>/* 'n'                                      */
```

The format of the data chain element subsection is as follows:

```
<Default Key String>            /* Initial key string assigned to this     */
                                /*   DSPINSTL data chain element           */

<Default General Prompt String> /* User prompt string displayed in the     */
                                /*   Specify Source Directory panel when   */
                                /*   the DSPINSTL data chain element is
                                /*   interpreted                           */

<Default Diskette Prompt String>/* User prompt displayed in the Insert     */
                                /*   Diskette panel displayed by the       */
                                /*   Specify Source Directory panel logic  */

<Default Volume Label>          /* Optional volume label string used by    */
                                /*   the Specify Source Directory panel    */
                                /*   logic to verify that an inserted      */
                                /*   diskette is the correct diskette      */

<Resolution String>             /* Optional string, which indicates a      */
                                /*   display driver resolution.            */
                                /*   The string can be passed to the       */
                                /*   Select_Display_Resolutions            */
                                /*   service function or can be left NULL. */
                                /*   SVGA display driver sets have one     */
                                /*   resolution to a DSP file.             */
                                /*   Resolution strings should always      */
                                /*   be of the form, ####x####x####, where */
                                /*   each #### field is a separate integer */
```

There is no data element parameter provided to indicate a choice of default source directory. It is always assumed to be the logical A: drive. DSPINSTL provides a Specify Source Directory PM panel that allows the user to override the default source directory. The <Default General Prompt String> is displayed in the PM panel so that the user will know the purpose of the source directory about to be searched.

If the user changes the source directory to a nonremovable device, the source files can exist in the specified directory or in a subdirectory under the specified directory. The subdirectory name must be the same as the volume label for the entry with the underscore character (_) replacing spaces. To do this, volume labels must be limited to eight characters.

The following example shows a typical SVGA display adapter parameter entry:

```
"XYZ SVGA video adapter"
"XYZ.DLL"
"SVGA"
"1"
"0"
"3"
"XYZ1"
"XYZ SVGA display drivers (part 1)"
"XYZ SVGA drivers diskette #1"
"XYZ 1"
"640x480x256"
"XYZ2"
"XYZ SVGA display drivers (part 2)"
"XYZ SVGA drivers diskette #2"
"XYZ 2"
"1024x786x256"
"XYZ3"
"XYZ SVGA display drivers (part 3)"
"XYZ SVGA drivers diskette #3"
"XYZ 3"
"1024x768x256"
```

The above example includes three resolution strings, which can be displayed in the scrolling list box in the Select Display Resolutions panel.

If DSPINSTL cannot identify the type of graphics adapter, it displays a default type of **Other** and also sets the default internal key to "Other". The user can then go to the Primary Video Adapter PM panel or the Secondary Video Adapter PM panel and select a display type that is included in one of the DSC file entries.

If DSPINSTL can identify the type of graphics adapter but finds no matching configuration file entry with a matching standard-type keyword string, it displays the default type-name for the adapter in the Video Display Configuration panel (such as VGA) but does not allow the user to accept the default selection. The user must use either the primary display check box or the secondary display check box in order to override default DSPINSTL processing. This will cause a secondary PM panel to be displayed that will allow the user to select a display driver package from a list that includes all the packages described in the DSC files.

User Interface PM Panels

The front-end PM panels allow the user to specify whether primary or secondary display driver installation is desired. The panels also allow the user to override the default display driver type and specify which display driver package to install (based on the set of packages defined in the relevant DSC files on the user's system).

Action Routine Interface

The action routine interface allows the display driver developer to (optionally) add custom control logic that is executed after the user makes an initial selection. Action routines are supported so that the installation process can ask the user specialized questions (through the use of PM panels) or perform internal inquiries concerning the state of the hardware or software system. For example, an action routine can be used to ask the user what display resolution should be configured. Action routines are packaged in OS/2 Dynamic Link Library files (DLL files).

The action routines tailor a display driver installation by modifying the chain of data elements that are initialized by the DSPINSTL Configuration File Interpreter. The display driver developer packages DSPINSTL action routines in a DLL file and then includes the name of the DLL file in the DSC file entry for that display driver package. The DLL itself should be placed in a directory within the user's LIBPATH or in the directory containing the DSPINSTL.EXE file.

DSPINSTL will call specifically named action routines (if they are provided in the DLL that can be specified within the controlling configuration file entry) during the installation when the user:

- Accepts the selection of the primary display (in the initial Video Display Configuration PM panel)
- Accepts the selection of the secondary display (in the initial Video Display Configuration PM panel)
- Overrides the selection of the primary display and explicitly selects a primary display (through the use of a Primary Video Adapter

PM panel)

- Overrides the selection of the secondary display and explicitly selects a secondary display (through the use of a Secondary Video Adapter PM panel)

The four action routines must have the following names:

- Default_Primary()
- Default_Secondary()
- Selected_Primary()
- Selected_Secondary()

The display driver developer can provide any subset of the four action routines that is deemed appropriate. When DSPINSTL reaches one of the points at which it tries to call an action routine, it uses the *DosLoadModule* API to attempt to load the DLL module. If successful, it then uses *DosGetProcAddr* API to try to obtain the address of the action routine. If this is successful, DSPINSTL passes control to the action routine. If this is not successful, DSPINSTL continues.

DSPINSTL aids the function of the action routines by providing them with a set of service functions that enable them to perform various standard actions such as data chain maintenance. In this manner, the individual action routine does not have to be aware of the specific structure of the data element chain. Again, DSPINSTL is designed to insulate the action routine from the details of the utility program's internal structure. This greatly simplifies the creation of an action routine.

Each action routine is passed a pointer to a global DSPINSTL data structure, which contains values that can be used by the action routine. This data structure includes a revision field, which is always set to 1 by the utility program. The revision field is provided so that future revisions of DSPINSTL will be able to grow and evolve the data structure. Action routines should always check the revision field to determine how to access the data structure.

Procedure Call Interface for Action Routines

All four action routines follow the same procedure call interface, which is illustrated under Default_Primary.

DSPINSTL Service Functions

A set of standard service functions is provided to keep the action-routine developer from having to program commonly used functions. Service functions enable the user to examine and modify the DSPINSTL chain of data elements.

The global DSPINSTL data structure contains pointers to a set of DSPINSTL service functions that are available to all action routines. These service functions, listed below, are described on the pages that follow.

- Link_Chain_Element
- Unlink_Chain_Element
- Next_Chain_Element
- Select_Display_Resolutions

Link_Chain_Element

Link_Chain_Element - Syntax

Description:

Link_Chain_Element provides a standard mechanism for the linking of data chain elements within the DSPINSTL data chain.

```
#include <os2.h>

PDSPINSTL_CHAIN     new_element_ptr;
USHORT              create_flag;
USHORT              position_flag;
PDSPINSTL_CHAIN     existing_element_ptr;
ULONG               rc;                     /*  Return codes. */

rc = Link_Chain_Element(new_element_ptr,
       create_flag, position_flag, existing_element_ptr);
```

Link_Chain_Element Parameter - new_element_ptr

new_element_ptr (PDSPINSTL_CHAIN) - input
If create_flag is set to *1* , this argument will return a pointer to the new element. If create_flag is set to *0* , on input this argument points to an existing DSPINSTL data chain element.

Link_Chain_Element Parameter - create_flag

create_flag (USHORT) - input
The flag that indicates whether the caller wants the function to create a new element or merely link an element into the chain. A value of *1* indicates that a new element should be created and then linked into the chain. A value of *0* indicates that an existing element (pointed to by the new_element_ptr argument) should be linked into the chain.

Link_Chain_Element Parameter - position_flag

position_flag (USHORT) - input
The flag that indicates where the caller wishes the data chain element to be linked into the data chain. A value of *0* places the element at the head of the chain. A value of *1* places it at the tail of the chain. A value of *2* indicates that the element should be linked in front of the chain element pointed to by the existing_element_ptr argument. A value of *3* indicates that the element should be linked after the chain element pointed to by the existing_element_ptr argument.

Link_Chain_Element Parameter - existing_element_ptr

existing_element_ptr (PDSPINSTL_CHAIN) - input
If the position_flag is set to 2 or 3 on input, this argument points to another data element within the chain.

Link_Chain_Element Return Value - rc

rc (ULONG) - returns
Return codes.

This service function returns the following values:

0	Success
1	Element could not be linked into chain
2	New chain element could not be allocated

Link_Chain_Element - Parameters

new_element_ptr (PDSPINSTL_CHAIN) - input
If create_flag is set to *1* , this argument will return a pointer to the new element. If create_flag is set to *0* , on input this argument points to an existing DSPINSTL data chain element.

create_flag (USHORT) - input
The flag that indicates whether the caller wants the function to create a new element or merely link an element into the chain. A value of *1* indicates that a new element should be created and then linked into the chain. A value of *0* indicates that an existing element (pointed to by the new_element_ptr argument) should be linked into the chain.

position_flag (USHORT) - input
The flag that indicates where the caller wishes the data chain element to be linked into the data chain. A value of *0* places the element at the head of the chain. A value of *1* places it at the tail of the chain. A value of *2* indicates that the element should be linked in front of the chain element pointed to by the existing_element_ptr argument. A value of *3* indicates that the element should be linked after the chain element pointed to by the existing_element_ptr argument.

existing_element_ptr (PDSPINSTL_CHAIN) - input
If the position_flag is set to 2 or 3 on input, this argument points to another data element within the chain.

rc (ULONG) - returns
Return codes.

This service function returns the following values:

0	Success
1	Element could not be linked into chain
2	New chain element could not be allocated

Link_Chain_Element - Remarks

This function optionally can be used to allocate a new element that is then linked into the chain. The caller indicates the location in the chain where the data chain element should be linked. Elements can be linked at the head of the chain, the end of the chain, or after a specified chain element. After the function returns a pointer to a new data chain element, the action routine can then fill in the element's fields as appropriate.

Link_Chain_Element - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

Unlink_Chain_Element

Unlink_Chain_Element - Syntax

Description:

Unlink_Chain_Element provides a standard mechanism for unlinking data chain elements from the chain. It does not deallocate the memory associated with the element; it only unlinks it from the chain.

```
#include <os2.h>

PDSPINSTL_CHAIN     element_ptr;
ULONG               rc;           /*  Return codes. */

rc = Unlink_Chain_Element(element_ptr);
```

Unlink_Chain_Element Parameter - element_ptr

element_ptr (PDSPINSTL_CHAIN) - input
A pointer to a DSPINSTL data chain element.

Unlink_Chain_Element Return Value - rc

rc (ULONG) - returns
Return codes.

This service function returns the following values:

0	Success
1	Element could not be unlinked from chain

Unlink_Chain_Element - Parameters

element_ptr (PDSPINSTL_CHAIN) - input
A pointer to a DSPINSTL data chain element.

rc (ULONG) - returns
Return codes.

This service function returns the following values:

0	Success
1	Element could not be unlinked from chain

Unlink_Chain_Element - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

Next_Chain_Element

Next_Chain_Element - Syntax

Description:

Next_Chain_Element provides a standard mechanism for returning the address of specified data elements within the chain.

```
#include <os2.h>

PDSPINSTL_CHAIN    next_element_ptr;
USHORT             position_flag;
PDSPINSTL_CHAIN    existing_element_ptr;
ULONG              rc;                    /*  Return codes. */

rc = Next_Chain_Element(next_element_ptr,
       position_flag, existing_element_ptr);
```

Next_Chain_Element Parameter - next_element_ptr

next_element_ptr (PDSPINSTL_CHAIN) - input
Returns a pointer to the specified chain element.

Next_Chain_Element Parameter - position_flag

position_flag (USHORT) - input
A flag that indicates which data chain element is specified by the caller:

0	Indicates the head element of the chain

1	Indicates the tail element of the chain
2	Indicates the data element that follows the data element that is pointed to by the existing_element_ptr input argument
3	Indicates the data element that precedes the data element that is pointed to by the existing_element_ptr input argument

---

Next_Chain_Element Parameter - existing_element_ptr

existing_element_ptr (PDSPINSTL_CHAIN) - input
Pointer to a data chain element. Used only if position_flag is set to 2 or 3.

Next_Chain_Element Return Value - rc

rc (ULONG) - returns
Return codes.

This service function returns the following values:

0	Success
1	Next element found within the chain

Next_Chain_Element - Parameters

next_element_ptr (PDSPINSTL_CHAIN) - input
Returns a pointer to the specified chain element.

position_flag (USHORT) - input
A flag that indicates which data chain element is specified by the caller:

0	Indicates the head element of the chain
1	Indicates the tail element of the chain
2	Indicates the data element that follows the data element that is pointed to by the existing_element_ptr input argument
3	Indicates the data element that precedes the data element that is pointed to by the existing_element_ptr input argument

existing_element_ptr (PDSPINSTL_CHAIN) - input
Pointer to a data chain element. Used only if position_flag is set to 2 or 3.

rc (ULONG) - returns
Return codes.

This service function returns the following values:

0	Success
1	Next element found within the chain

Next_Chain_Element - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

Select_Display_Resolutions

Select_Display_Resolutions - Syntax

Description:

Select_Display_Resolutions can be used by an action routine to ask a user which display resolutions are desired as a result of this installation.

```
#include <os2.h>

HWND                  hWnd;
PSCREENRESOLUTION     resolutions;
ULONG                 num_resolutions;
BOOL                  fmultiple;
ULONG                 rc;               /*  Return codes. */

rc = Select_Display_Resolutions(hWnd,
       resolutions, num_resolutions, fmultiple);
```

Select_Display_Resolutions Parameter - hWnd

hWnd (HWND) - input
The handle of the main DSPINSTL window.

Select_Display_Resolutions Parameter - resolutions

resolutions (PSCREENRESOLUTION) - input
A pointer to an array of display resolution structures, where:

```
struct resolutions_struct
{
 UCHAR resolution_string[40]; /* String to go in multiple selection list box. */
 USHORT selected;             /* Flag set by the service function.            */
                              /* If 0, this resolution was not selected,      */
                              /* if 1, this resolution was selected.          */
};
```

Select_Display_Resolutions Parameter - num_resolutions

num_resolutions (ULONG) - input
The number of display resolutions in the array.

Select_Display_Resolutions Parameter - fmultiple

fmultiple (BOOL) - input
A flag that specifies whether the user will be allowed to select multiple resolutions (or only a single resolution). True indicates multiple selections are allowed.

Select_Display_Resolutions Return Value - rc

rc (ULONG) - returns
Return codes.

This service function returns the following values:

0	User selected an entry and exited the panel by selecting the **OK** push button
0xFFFFFFFF	User exited the panel by selecting the **Cancel** push button

---

Select_Display_Resolutions - Parameters

hWnd (HWND) - input
The handle of the main DSPINSTL window.

resolutions (PSCREENRESOLUTION) - input
A pointer to an array of display resolution structures, where:

```
struct resolutions_struct
{
 UCHAR resolution_string[40]; /* String to go in multiple selection list box. */
 USHORT selected;             /* Flag set by the service function.            */
                              /* If 0, this resolution was not selected,      */
                              /* if 1, this resolution was selected.          */
};
```

num_resolutions (ULONG) - input
The number of display resolutions in the array.

fmultiple (BOOL) - input
A flag that specifies whether the user will be allowed to select multiple resolutions (or only a single resolution). True indicates multiple selections are allowed.

rc (ULONG) - returns

Return codes.

This service function returns the following values:

0	User selected an entry and exited the panel by selecting the **OK** push button
0xFFFFFFFF	User exited the panel by selecting the **Cancel** push button

---

Select_Display_Resolutions - Remarks

This function displays a Select Display Resolutions PM panel, which lets the user choose from a set of display resolution strings that are displayed in a scrolling list box within the panel. The various strings are passed into the function within an array of structures. A flag that is passed to the function controls whether the panel will allow the user to select multiple display resolutions or only a single resolution. In this manner, action routines can look for one choice or multiple choices. On return from the function, each structure in the array will contain a flag value that indicates whether that array element was selected by the user.

Select_Display_Resolutions - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

Command Language Presentation Manager Panels

During the final stage of DSPINSTL, each data chain element will trigger a separate DSPINSTL Command Language interpretation step. One of these PM panels will allow the user to optionally override the source directory that will be used for that step. Another panel will prompt the user to insert an appropriately labeled diskette. The remainder of the PM panels will prompt the user when execution errors are encountered during this final portion of DSPINSTL execution.

DSPINSTL Command Language Interpreter

The interpretation of the DSPINSTL commands in the DSPINSTL Display Driver Profile files is the final stage of the DSPINSTL process. The interpretation is performed by the DSPINSTL Command Language Interpreter. The interpreter operates by interpreting the contents of DSPINSTL Display Driver Profile files (DSP files) that contain DSPINSTL commands. DSPINSTL commands specify the basic installation and configuration operations that will be performed by the utility program. Each DSP file is tagged with a keyword string that identifies the file.

The DSPINSTL Command Language Interpreter selects DSP files for interpretation by traversing the data element chain. The chain was first initialized by the Configuration File Interpreter, and then might have been modified through the operation of action routines. Each data element corresponds to an installation/configuration "step." Among other uses, this multistep architecture allows DSPINSTL installation and configuration to be partitioned over multiple source diskettes (with each diskette handled in separate steps).

DSPINSTL provides a PM panel that allows the user to override the default value for the source directory that is to be used for a given step. The data chain element also includes a DSP file keyword string. The DSPINSTL Command Language Interpreter will interpret all DSP files in the source directory that are tagged with that keyword string.

It is the DSPINSTL commands that do the actual work of display driver installation and configuration. All the preceding DSPINSTL activities act to properly set up this final stage on a given user's system. In addition to commands that copy files and modify configuration files, there is a special RUN command that can be used to run specified programs during DSP file interpretation. Among other things, such programs can generate DSPINSTL commands that are immediately interpreted by the DSPINSTL Command Language Interpreter. This is the final manner in which the display driver developer can customize DSPINSTL execution.

DSPINSTL Command Language

The DSPINSTL command language is highly flexible and accommodates the many different kinds of operations that must be accomplished in the installation and configuration of OS/2 display drivers.

A set of DSPINSTL commands is packaged as a DSP file. Each DSP file must contain a KEY command that tags that DSP file. When a data chain element is interpreted by DSPINSTL, any DSP files in the current source directory that are tagged with that keyword are then interpreted. This is the mechanism that allows automatic execution.

The FILES command tells DSPINSTL which files to copy to the user's hard disk. Date/Time checking or file version checking can be performed as part of FILES command processing to help prevent inappropriate file copies from being done. This capability is an important DSPINSTL safety feature. The FILES command supports the installation of both packed and unpacked files. The use of packed files can greatly reduce the amount of space consumed on the display driver developer's diskettes.

The CONFIG, OS2INI, and WININI commands tell DSPINSTL to update the configuration in different types of system configuration files. There are three separate commands because the different types of OS/2 system configuration files require different styles of editing. The CONFIG command is used to update the CONFIG.SYS file. The OS2INI command is used to edit the various *.INI OS/2 system configuration files (such as OS2.INI or OS2SYS.INI). The WININI command is used to update the different WIN-OS/2 configuration files (such as WIN.INI and SYSTEM.INI). The configuration of OS/2 display drivers is particularly tricky because of the coordinated updates that are needed in different OS/2 configuration files.

The SET_DRIVER and SET_RESOLUTION commands are also configuration editing commands. In these two cases, rather than edit a specific configuration file, an OS/2 Graphics Engine function updates the system configuration. The additional level of data abstraction is necessary to support the architecture of the Graphics Engine, which is yet another complexity that is supported by the DSPINSTL command language.

DSPINSTL Commands

The DSPINSTL command language bears similarities to the DDINSTAL command language. The DSPINSTL command language contains important keyword extensions. Like DDINSTAL, the DSPINSTL command language interpreter accepts comments in the form of lines beginning with an asterisk (*).

The DSPINSTL Command Language files support the following commands:

:KEY
:FILES
:CONFIG
:OS2INI
:WININI
:RUN
:SET_DRIVER
:SET_RESOLUTION

The use of a MODE qualifier is valid on all DSPINSTL commands except :KEY. Valid MODE qualifier values are:

MODE=

- PRIMARY - execute this command only if user has chosen to perform a primary display driver installation.
- SECONDARY - execute this command only if user has chosen to perform secondary display driver installation.
- DOS - execute this command only if DOS support is installed on the OS/2 system.
- WINDOWS - execute this command only if WIN-OS/2 support is installed.

KEY

This command identifies a DSP file with a unique keyword string so that it can be automatically selected for interpretation by DSPINSTL. The final stage of DSPINSTL processing consists of one or more steps of DSP file interpretation. In each step, all the DSP files in the current source directory whose keyword strings match the keyword string for that step are interpreted.

The DSP file line that immediately follows the :KEY command line contains a key string that begins with the first nonblank character in the line and ends at the first blank character or the end of the line. There can be no embedded blanks within a key string. Key strings can be up to 64 bytes in length. Here is an example of a :KEY command line:

```
:KEY
XYZ_800x600x16
```

There must be a single :KEY command line in each DSP file.

FILES

This command specifies the files that are to be copied from the source directory to different destination directories. It performs in a manner similar to the DDINSTAL :FILES command. Like its DDINSTAL counterpart, the DSPINSTL :FILES command line is followed by a set of file copy directives. However, unlike its DDINSTAL counterpart, the DSPINSTL :FILES command uses the UNPACK utility program to copy files. Therefore, it can handle both packed and unpacked file formats.

The :FILES command can be used to replace any executable modules (programs and DLLs,) that are currently in use. The replacement occurs the next time the system is started. DSPINSTL cannot replace nonexecutable files that are locked at the time DSPINSTL is run.

To protect against inadvertently copying an older version file over a newer version file, all file copying is mediated by either one of the following forms of file version checks:

- File Version Field checking
- Date/Time checking

File Version Field Checking

Executable files can be tagged with a file version string in the description field of their executable file header. This field can be set during program linking. The format of a file version string is as follows:

```
<header> <version string> <trailer>
```

where:

<header> is a two-character string: @#.

<version string> is twelve characters defined as xxx:YYYYYYYY, where *xxx* is used to identify the vendor that created the file (for example, IBM) and *YYYYYYYY* is a file version number (for example, 0006.307). This file version number can be interpreted as a floating point number or as an integer.

<trailer> is the two-character string: #@.

An example of a file version string would be:

@#IBM:0006.307#@.

If both the source and target versions of a file contain file version strings, the file version checking is performed. For a file version check to be considered successful, the vendor identifiers of the source and target files must match and the file version number of the source file must be greater than or equal to the file version number of the target file. If the file version check succeeds, then the source file is copied over the target file. If not, DSPINSTL prompts the user to decide whether to replace the target file. If the source file contains a file version field, but the target file does not, then the source file is assumed to be more recent than the target file and is copied over the target file.

Date/Time Checking

DSPINSTL will not automatically copy an older version of a file over a newer version when the Date/Time of the file to be copied is less recent than the Date/Time of the file to be replaced. Instead, it prompts the user to decide whether to replace the newer version of a file with the older one.

The :FILES command line is followed by one or more lines that name the files to be copied from the source directory and also the destination directory to which they are to be copied. The format of these lines is as follows:

```
<source file name> <destination directory pathname>
```

where:

<source file name> is the name of a file within the source directory that contains the DSP file. A source file can be packed or unpacked.

<destination directory pathname> can include a logical drive letter specification. The optional logical drive letter specification is always overridden by the system's startup drive. The destination directory path name is not a complete file name. This means that the source file is never renamed when it is copied.

Note: The special term %BOOTDRIVE% can be used to replace the startup drive letter in all DSPINSTL commands that allow path names to be specified. %BOOTDRIVE% is a special drive designator that indicates the drive letter on which the OS/2 system was started.

If a destination directory is provided, the file is copied to that directory. If no destination directory is provided and the source file is a packed file, the file is copied to the directory that is specified in the source file's pack header. If there is no corresponding directory on the drive, the directory is created. If no destination directory is provided and the source file is an unpacked file, then the file is copied to the current working directory.

The following examples illustrate DSP file command lines that can follow the :FILES keyword command line:

```
:FILES    :MODE=PRIMARY
IBMVGA32.DL@ %BOOTDRIVE%:\OS2\DLL
BVHVGA.DLL %BOOTDRIVE%:\OS2|DLL

:FILES  :MODE=PRIMARY  :MODE=DOS
VVGA.SY@ %BOOTDRIVE%:\OS2\MDOS
```

Note: The @ sign (at sign) at the end of some of the source file names indicates that these are packed files (packed by the OS/2 PACK utility program).

CONFIG

This command specifies updates to the CONFIG.SYS configuration file. It performs in a manner similar to the DDINSTAL :CONFIG command. Like its DDINSTAL counterpart, the DSPINSTL :CONFIG command line is followed by a set of CONFIG.SYS statements, which are to be added to the configuration file. However, the DSPINSTL version of the :CONFIG command attempts to replace the corresponding CONFIG.SYS statements instead of merely appending to the file.

The following examples illustrate DSP file command lines that can follow the :CONFIG keyword command line:

```
:CONFIG  :MODE=PRIMARY
SET VIDEO_DEVICES=BIO_VGA
SET VIO_VGA=DEVICE(BVHVGA)

:CONFIG :MODE=PRIMARY :MODE=DOS
DEVICE=%BOOTDRIVE%:\OS2\MDOS\VVGA.SYS
```

OS2INI

This command specifies updates to an OS/2 INI system configuration file and performs in a similar manner to the DDINSTAL :OS2_INI keyword. However, it is more general in nature than DDINSTAL because it can be used to specify updates to any OS/2 INI file (such as OS2.INI or OS2SYS.INI).

Like its DDINSTAL counterpart, the DSPINSTL :INI command line is followed by a set of INI line entries, which are to be placed in the configuration file. However, the first line following the :OS2INI command line specifies the complete path name of the OS/2 INI file to be modified. Like the :FILES command, the optional drive specification within the complete path name is always overridden by the system's startup drive. If only a simple OS/2 INI file name is specified, the \OS2 directory on the system's startup drive is assumed.

The following example illustrates the DSP file command lines that can follow the :OS2INI keyword command line:

```
:OS2INI  :MODE=PRIMARY
OS2.INI
PM_DISPLAYDRIVERS IBMVGA32 IBMVGA32
PM_DISPLAYDRIVERS CURRENTDRIVER IBMVGA32
```

WININI

This command specifies updates to the WIN-OS/2 configuration file. It has no counterpart in DDINSTAL. Like the :OS2INI keyword, the :WININI key is followed by the name or complete path name of a WIN-OS/2 configuration file. If a simple file name is provided, it is assumed that the file resides in the \OS2\MDOS\WINOS2 directory on the system's startup drive.

The file name line is followed by a series of lines that will be placed in the specified WIN-OS/2 configuration file. Each WIN-OS/2 configuration line consists of three fields: <section>, <keyword>, and <value>. The section field identifies a section within the WIN-OS/2 configuration file.

In a similar manner to the :CONFIG command, an attempt is made to replace corresponding lines in the file rather than simply appending the lines to the specified section of the file. WIN-OS/2 configuration lines are typically of the form:

```
<keyword string> = <value string>
```

where the <value string> can actually be several substrings that are divided by spaces. Any field (section, keyword, value) can contain spaces if the field is enclosed in double quotation marks. The :WININI command operates by replacing lines in the specified section that match <command string> fields, or (if no match is found) by appending the new line at the end of the specified section.

The following example illustrates the DSP file command lines that can follow the :WININI command line:

```
:WININI :MODE=PRIMARY :MODE=WINDOWS
SYSTEM.INI
BOOT DISPLAY.DRV VGA.DRV
BOOT SDISPLAY.DRV SWINVGA.DR
```

RUN

This command is used to specify one or more programs to execute. The programs and their associated parameters are listed in the lines that follow the :RUN command.

Each program that is run can dynamically create DSP commands by writing them to its standard output device. When the program terminates, any DSP commands that were written to the standard output device are immediately interpreted by DSPINSTL. The dynamically created DSP commands are interpreted before any DSP file lines, which follow the :RUN program line, are interpreted. The program should use only the standard output device for the writing of DSP commands.

If a :RUN program generates output other than DSP lines, it must be redirected to another file or to NULL (see the following example).

```
:RUN
 MYPROG.EXE >NUL 2>&1
```

The :RUN command ignores the return code from each of the programs that it executes. Each program specification line that follows the :RUN keyword command must adhere to the following format:

```
<program pathname> [<program parameters>]
```

The :RUN command can also be used to execute OS/2 command files (.CMD files).

SET_DRIVER

The :SET_DRIVER command is used to configure a 32-bit display driver for use by the 32-bit graphics engine. The parameters that are associated with this keyword are listed in the lines that follow the :SET_DRIVER command. These parameters are passed to the *DspSetDriverInfo* API that was discussed earlier in this chapter.

Each parameter is specified as a <keyword> = <value> pair.

The following list specifies the parameters that must be specified in the lines following the :SET_DRIVER command line. All the parameter values in this list are strings and must be enclosed in double quotation marks.

NAME = "<Name of display driver>"
DESC = "<String that describes driver driver (can be NULL)>"
PATHNAME = "<Name of display driver>"
PARMS = "<String containing parameters for driver (can be NULL)>"

Example:

```
:SET_DRIVER
NAME="IBMVGA32"
DESC="IBM VGA Display Driver"
PATHNAME="IBMVGA32"
PARMS=""
```

SET_RESOLUTION

The :SET_RESOLUTION command is used to inform the 32-bit graphics engine of the initial display resolution for the display driver that is being installed. This command is only necessary for display drivers that can support multiple display resolutions and use the *DspDefaultResolution* API to determine (during their initialization) which resolution to display. That API was described earlier in this chapter.

The parameters associated with this command are listed in the lines that follow the :SET_RESOLUTION command line. Each parameter is specified as a <keyword> = <value> pair.

The following list includes the parameters that must be specified after the :SET_RESOLUTION command line. All the parameter values in this list are numeric values and are specified as unsigned decimal numbers.

WIDTH = <Width of the display resolution in pels>
HEIGHT = <Height of the display resolution in pels>
COLORS = <Number of colors supported in this display resolution>
PLANES = <Number of planes in this display resolution>

Example:

```
:SET_RESOLUTION
WIDTH=1024
HEIGHT=768
COLORS=256
PLANES=1
```

Creating a Display Driver Installation Package

A complete display driver package will usually include the following types of files:

- DSPINSTL utility program
- A DLL, which includes the action routines (if action routines are used)
- One or more .DSC configuration files (containing DSC entries for each separate driver package)
- Display driver files (PM or WIN-OS/2 driver files)
- Any other files to be copied (if necessary)
- One or more DSP files (containing DSPINSTL installation and configuration commands)
- Programs to be triggered by the :RUN commands in DSP files (as needed)

A display driver package can span multiple diskettes. If so, each diskette must include the DSP files that explain what to do with the files on the diskette. The display driver files (and related files) can be packed (by the PACK.EXE utility program). If the files are packed, their last file-name character must be set to the "&" character. DSC files, DSP files, the action routine DLL, and the DSPINSTL utility program must not be packed.

Distributed Console Access Facility (DCAF) Architecture

The IBM Distributed Console Access Facility (DCAF) product provides a remote console function that allows one programmable workstation, called a *controlling* workstation, to control the keyboard and mouse input and monitor the display output of another programmable workstation, called a *target* workstation.

Operating States

The DCAF product has two main operating states: monitoring and active.

Monitoring State

When the DCAF session is in the monitoring state, the controlling-workstation user (hereafter referred to as the controller) sees a screen

image of the target workstation's display. The target workstation user (hereafter referred to as the target user) has complete control of the target workstation operations.

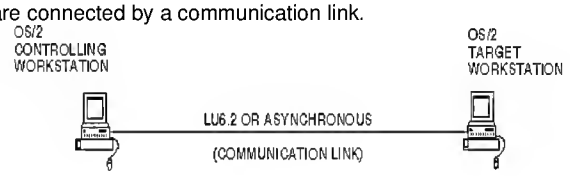
Active State

When the DCAF session is in the active state, the controller operates and controls the target workstation. The keystrokes typed by the controller are relayed to the target workstation and acted upon as if they were typed by the target user. The keystroke input and the resulting screen image are seen on both the controlling and target workstations.

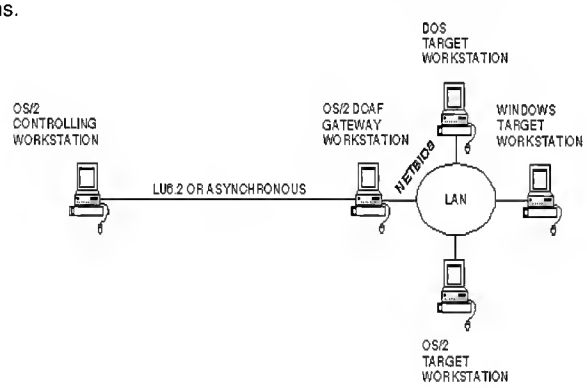
During the active state, keystroke input from the target workstation is not accepted. However, if the hot key combination is enabled for the target workstation, the target user can invoke this key combination to regain control of the target workstation. The target user also can choose to change the session state (for example, from active to monitoring).

The controller can establish concurrent sessions with multiple target workstations and can access each session by switching to a different window. Multiple controlling workstations can be defined within the network, however, only one of them at a time can establish a session with any single target workstation.

The controlling and target workstations are connected by a communication link.



The *Sample DCAF Direct Connection* figure above shows a direct connection between the controlling workstation and the OS/2 target workstation. Direct modem connections can be established between OS/2 workstations or between OS/2 controlling workstations and Windows Version 3.1 target workstations.



The *Sample DCAF Connection* figure above shows a connection via a DCAF gateway workstation that acts as an interface between the controlling workstation and the target workstations on a local area network (LAN). Connections through the DCAF gateway can be established from a DCAF controlling workstation to any kind of DCAF target (OS/2, DOS or Windows).

Benefits of using DCAF

Within a customer's network, DCAF's ability to monitor or control a remote workstation from a central site makes it a powerful tool for network management, network administration, network security, problem determination and diagnosis, and application assistance in the form of Help Desk functions.

This remote capability can save time and money, as well as increase the productivity of the administration and support personnel. Technical expertise can be centralized at the controlling site, reducing the need for highly skilled personnel at remote locations. Reduced operational requirements will decrease costs and allow for faster growth for distributed applications within the distributed intelligent workstation environment.

System management of a secure-session environment is streamlined and made easier with the location of the DCAF security administrator component at the controlling site. The DCAF secure-session security helps protect the customer's software assets because DCAF's logged connections create a security audit trail.

Typical Uses of DCAF

- View and control another PC
- Control unattended PCs remotely
- Support customers and provide help remotely
- Diagnose and solve problems on another PC by modem or over a LAN
- Transfer text and data between PCs
- Run DOS, Windows and OS/2 on an unattended PC by modem or over a LAN
- Remote Help Desk assistance for applications, online education, and maintenance of application programs.
- Remote problem determination for trace and dump analysis, including file transfer of data.
- Remote control of unattended workstations (for example, LAN servers).
- Remote management of Personal System/2 (PS/2) workstations and accessibility to data and programs stored on a PS/2 (for example, in the home or in the office) i.Remote access to system consoles when they are implemented on PS/2s.

Here is a typical scenario of the DCAF Help Desk function:

- A target user is having difficulty understanding the company's new accounting program.
- The target user contacts the Help Desk person who, in turn, opens a session with that particular target workstation.
- With the target workstation accounting program on the screen, the controller (Help Desk person) switches to active state and types the correct keystrokes to run the accounting program.
- The target user observes this process and learns how to use the new accounting program.
- After demonstrating how to use the product, the controller switches to the monitoring state and returns control to the target user.

Understanding the DCAF Environment

DCAF offers two main levels of security, which limit users' ability to take over your target workstations. These levels, basic DCAF (or nonsecure) and secure DCAF, offer protection for your workstations to varying degrees. For typical nonsecure environments that deal with nonconfidential information, basic DCAF provides adequate protection. Secure DCAF is more complex and caters to environments, such as banks and insurance companies, that work with confidential material and require greater protection.

In order to be part of the DCAF environment, the workstations must have DCAF components installed.

The DCAF Components

Before you begin the installation, you should be familiar with the DCAF components. The role of each workstation in your network determines the component(s) you install on that workstation. All components, except targets, run on OS/2 workstations only.

You can install one or more components on the same workstation, but it is recommended that you install only the necessary components on each workstation. For example, on one workstation you might install a controller, on another, the gateway and LAN directory, and on a third, a target component.

The Controller

The controller enables you to control the keyboard and mouse input and monitor the display output of a target workstation. It enables you to transfer files between the controller and target.

The Target

The target runs on three kinds of operating systems:

- The **OS/2** target enables an OS/2 workstation to be monitored and controlled by the controller.
- The **Windows** target enables a Windows workstation to be monitored and controlled by the controller. The controller connects to the workstation whether it is operating in DOS full-screen mode or in a Windows graphic environment.
- The **DOS** target enables a DOS workstation to be monitored and controlled by the controller. To install the DOS target, either create an installation package or use the CID method. Use the packager on an OS/2 workstation to create a DOS target installation package and then install the target on the DOS workstation.

In each case, the controller connects to the target directly if it is a basic DCAF target, or through a gateway if the workstation is secure.

The Gateway

The gateway links a controller with secure targets or with targets connected through an SNA backbone. The gateway always requires that you install a LAN directory on a workstation-any workstation-on the same LAN.

The gateway connects to all targets using the NetBIOS communication protocol. Therefore, install the gateway only in those parts of your network that use NetBIOS.

The gateway supports more than one session at a time so you probably need one gateway per logical LAN. To connect to secure targets, the gateway must be secure as well.

The LAN Directory

The LAN directory provides a list of the targets on a LAN that are available to the controller. When the controller connects directly with the targets via IPX/SPX, NetBIOS, or TCP/IP, the LAN directory is optional. When the controller connects with the targets through a gateway, the LAN directory is required.

Install one LAN directory per logical LAN. If you install more than one LAN directory on a logical LAN, give each a unique name.

The LAN directory also keeps a list that associates the workstation's physical address on the LAN with a nickname that makes the target name easy to remember. This list also enables the controller to find targets quickly. You can install the LAN directory on the target, gateway (if present), or on any workstation on the same LAN as the target.

The Security Administrator

The security administrator is necessary for secure DCAF only. It provides centralized maintenance of the databases for controllers, target access tables, secure target workstations, and security authenticators. Install one security administrator, typically on a controller, for your entire network if you are using secure DCAF. Safeguard this workstation from unauthorized access.

The security administrator keeps an encrypted database of authorized users, as well as an audit log file, EQNADMN.LOG. To be able to transfer security database files to the security authenticator, you must install either the security authenticator or a controller on the same workstation as the security administrator.

The Security Authenticator

The security authenticator is necessary for secure DCAF only. It authenticates sessions with secure targets. The security authenticator also gives access to secure targets based on security profiles for authorized controllers that it receives from the security administrator.

Install the security authenticator on an OS/2 target on the same LAN. Install it either on the security administrator workstation or on the secure OS/2 target workstation, so that it can receive security information from the security administrator.

If you install the security administrator on the LAN, you can install the security authenticator on the same workstation. Install at least one security authenticator for each logical LAN in your network.

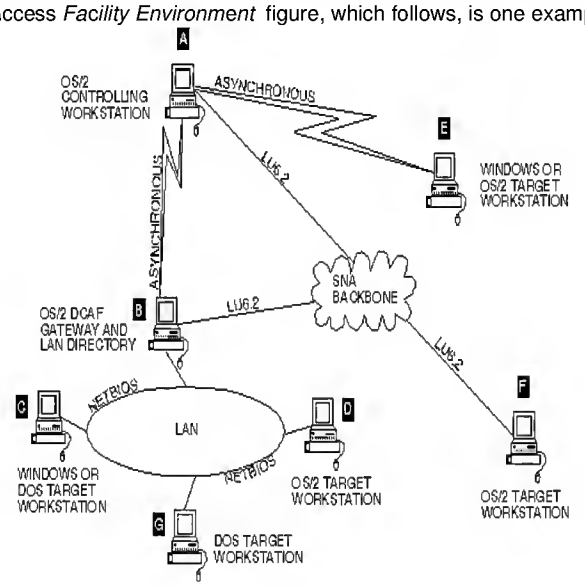
The Packager

The packager is an installation aid only. It creates installation packages of DCAF components that you can install on other workstations. In addition, the packager enables you to prepare a component with the same personalization features to be installed in large numbers all over your network. Always install DOS targets using the packager.

The DCAF Network

The DCAF network includes the workstations that have DCAF installed and the communications that link them.

The *Example of a Distributed Console Access Facility Environment* figure, which follows, is one example of a basic DCAF environment.



The DCAF workstations communicate with the following communication protocols:

- Advanced Peer-to-Peer Networking (APPN)
- Advanced Program-to-Program Communication or APPC); more specifically, Logical Unit Type 6.2
- Asynchronous (also called Asynchronous Communications Device Interface or ACDI)
- IPX/SPX (except for DOS targets)
- NetBIOS
- TCP/IP (except for DOS targets)

Controller **A** communicates directly with target **E** and with gateway **B** using switched asynchronous communication.

Controller **A** also communicates, via LU 6.2 over a Systems Network Architecture (SNA) network or backbone, through gateway **B** to the targets on the LAN. The SNA backbone represents the part of the network that connects (via switched lines, leased lines, or satellite communications) host computers, communication controllers, and other computer hardware. Any connections over an SNA backbone must go through a gateway.

Controller **A** communicates directly with target **F** via LU 6.2. For LU 6.2 communications, the physical unit type for the controller, target workstation, and DCAF gateway workstation must be configured as a type 2.1 (PU2.1).

Controller **A** communicates via TCP/IP through a router with LAN directory **I** and with target **H.**

Controller **A** communicates via IPX/SPX through a router with LAN directory **K** and with target **J.**

Controller **A** communicates via NetBIOS directly with LAN directory **M** and with targets **L, N,** and **O** on the LAN.

OS/2 target workstations on the LAN require that OS/2 Communications Manager, LAN Adapter and Protocol Support (LAPS) from NTS/2, or the LAN Server 2.0 be installed to provide the NetBIOS device drivers. In order to run DCAF with other applications that require NetBIOS services, you may need to increase the NetBIOS resources via the OS/2 Communications Manager or LAN Services.

DOS target workstations on the LAN require that a LAN support program be installed, such as the IBM LAN Support product. DCAF supports DOS target workstations on the LAN only.

Levels of DCAF Security

DCAF provides the following levels of security:

- Basic (Nonsecure)
  - No password
  - Password-only
- Secure
  - Secure session

Basic (Nonsecure) Level

A *no password* session exists between a controlling workstation and a gateway or target workstation for which a password is not defined and, therefore, is not necessary.

A *password-only* session exists between a controlling workstation and a gateway or target workstation for which a password is defined. The controller must type the password before the session can be established.

In both the *no password* and *password-only* cases, the target component was installed as basic, or nonsecure. Basic DCAF provides adequate protection for nonsecure environments that deal with nonconfidential information.

Secure Level

Secure DCAF offers a higher and more advanced level of security for environments such as banks and insurance companies that work with confidential material. You can make a target workstation almost impregnable to intruders by designating it as a secure target. The controller must access the secure target through a secure gateway.

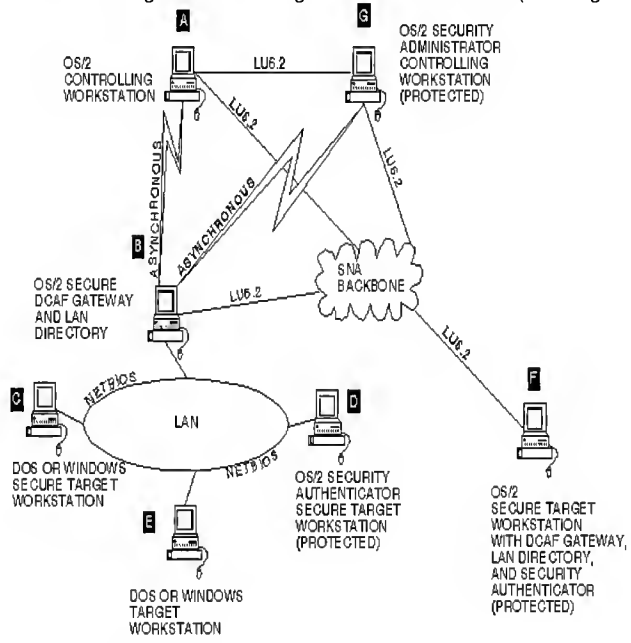
A *secure* session exists between a controlling workstation and a gateway or target workstation that has been installed as a secure gateway or target. These secure workstations do not have passwords. Instead, the controller, authorized to access a secure gateway or target, has a personal pass phrase (a compound password) and an access-level profile for a specific secure workstation.

DCAF security works with the following communication connections only:

- A controller can connect to a secure gateway via APPC/APPN or asynchronous.

- The secure Gateway can connect to a secure target via NetBIOS only.

The *Example of a DCAF Environment with Session Security* figure shows some workstations in a secure DCAF environment. The gateway can use only NetBIOS to communicate with the secure targets. Secure targets must be on a LAN (see targets **C** and **D** in the following figure.)



Controller **A** communicates via secure gateway **B** with the secure targets on the LAN. When the controller wants to take over secure target **C,** security authenticator **D** verifies that the controller is authorized to access that workstation.

DOS or Windows target **E** on the LAN is not secure and does not require authentication. It has password-only security. Controller **A** can also establish a session with secure target workstation **F.** Typically, the secure DCAF gateway, LAN directory, and security authenticator reside on target workstation **F.** In this case, controller **A** connects to gateway **F** via APPC, and gateway **F** connects internally to target **F** via NetBIOS.

The security administrator is typically installed on a controller workstation. It provides security file maintenance, a message log file, and control access information used by the DCAF security authenticators, all in a central location. Security files are transferred from the security administrator via a secure session to all of the remote security authenticators. Security administrator workstation **G** transfers new or updated security files to security authenticators **D** and **F.**

When the controller wants to change a pass phrase, the security authenticator verifies the authorization before a session is allowed, and keeps a log file of the authentication activity. The verification compares the controller's input with the ciphered data stored on the security authenticator. The advantage of this mechanism is that there is no way to intercept or discover the pass phrase during transmission, because only the ciphered data is sent.

For auditing purposes, the gateway logs all session connections and the communication errors that it filters.

Code Modifications Required in Base Driver for DCAF

The majority of the new code is contained within new modules that are simply linked with the base driver. The areas of the base driver code that must be modified include the following:

- The Dispatch Table, which contains the new Entry Points
- All drawing functions, which contain extra tests to determine whether they should accumulate clipped screen bounds and, if necessary, code to calculate the clipped bounding rectangle
- Seamless Windows cursor exclusion code, which is modified to call the new bounds accumulation routine.

Dispatch Table

In the EDDEFLDB.C, where the dispatch table is filled in with the display driver entry points, the five new entry points are included in the

Driver Dispatch Table with the following function numbers:

GreOpenScreenChangeArea	0x4012
GreGetScreenChangeArea	0x4013
GreCloseScreenChangeArea	0x4014
GreGetScreenBits	0x401D
GreSetScreenBits	0x401E

Accumulating and Querying Screen Bounds

Traditional Presentation Manager bounds are unclipped, and use a single rectangle to define their limits. In order to better define the bounding area, the DCAF-enabled display driver now has the ability to maintain one or more clipped, multirectangle regions (Screen Change Areas or SCAs) that are updated to indicate areas on the screen that have been drawn to.

This ability is provided by the following changes to the base driver:

- Three new entry points or functions to create, query and delete Screen Change Areas, as follows:
  - GreOpenScreenChangeArea
  - GreGetScreenChangeArea
  - GreCloseScreenChangeArea
- Two bounds accumulation internal routines, which add a single rectangle to all of the currently active SCAs.
- An extra flag test in the path of every display driver drawing function (for COM_SCR_BOUND in the high-order word in the last parameter of the GreXX calls, FunN).

  If this flag is set and the drawing operation is going to the screen, the drawing function passes a clipped bounding rectangle of the drawing primitive to the bounds accumulation functions described above. The code required to do this is driver-specific.
- Interception of Windows cursor exclusion calls and passing the supplied exclusion rectangle to the new bounds accumulation function.

Compressing and Decompressing Data

The bounding routines described above and under Performing the Bounding Accumulation efficiently track the regions on the screen that are updated by Presentation Manager drawing functions. The DCAF-enabled display driver also provides the ability to compress, decompress and, if necessary, convert the format of screen data.

This ability is provided by the following two new entry points or functions:

- GetScreenBits
- SetScreenBits

The task is performed by the internal routine CompressRect in COMPRESS.ASM. The compressed data that is passed between display drivers uses a private format (that is, no external application or program has the right to examine, alter or make any assumptions about the content of the data). This allows the compression method to be improved in later versions of the driver. Definitions of the data structures are as follows:

1. PACKET HEADER

   dd total_data_packet_length (including header)

   dw data_format

2. RECTANGLE HEADER

   dw xLeft

   dw yBottom

   dw xRight

dw yTop

3. RECTANGLE DATA

The rectangle data is split into individual rows. Each row is split into run-length encoded blocks (cells), each of which comprises a length field followed by one or more data fields. The sizes of both the length and data fields vary according to the format of the data being transmitted (as specified by the data_format field in the packet header), as follows:

4bpp	field size is one byte (8 bits)
8bpp, 16bpp	field size is two bytes (16 bits)

The following encoding rules are used:

- If the length field contains a ***positive*** value (most significant bit ***not*** set), then the following single data field is repeated (length) times. If the data field size is 8 bits, this value will be limited to a maximum of 127.
- If the length field contains a ***negative*** value (most significant bit set), then (length - most significant bit) fields of nonrepeating data follow. If the data field size is 8 bits, this value will be limited to a maximum of 127.
- If the length field is zero and the following field is nonzero, the nonzero field is a count of the number of times that the single previous row is repeated. If the data field size is 8 bits, this value will be limited to a maximum of 127. This will only appear as the first cell in a row, and only after there has been at least *one* row of data.
- If the length field is zero and the following field is zero, the next (third) field is a count of the number of times that the previous pair of rows are repeated. If the data field size is 8 bits, this value will be limited to a maximum of 127. This will only appear as the first cell in a row, and only after there have been at least *two* rows of data.

The following example shows the hexadecimal values of an 8bpp compressed bitmap:

```
0003 0004  00FA 0405 0706 0802 0104 0903 0001  ...........  0000 0003  0000 0000 0004  ...
 lf   df    lf   df   df   df   df   df   df                lf   df    lf   df   df
  cell1              cell2                                  celln     celln+1
```

This bitmap would expand as follows (two-digit values represent a color index for a single pixel):

```
00 04 00 04 00 04 04 05 07 06 08 02 01 04 09 03 00 01 ............ row1

                                                    do three more identical rows (celln):

00 04 00 04 00 04 04 05 07 06 08 02 01 04 09 03 00 01 ............ row2
00 04 00 04 00 04 04 05 07 06 08 02 01 04 09 03 00 01 ............ row3
00 04 00 04 00 04 04 05 07 06 08 02 01 04 09 03 00 01 ............ row4

                                                    do four pairs of identical couples (celln+1):

00 04 00 04 00 04 04 05 07 06 08 02 01 04 09 03 00 01 ............ row5
00 04 00 04 00 04 04 05 07 06 08 02 01 04 09 03 00 01 ............ row6

00 04 00 04 00 04 04 05 07 06 08 02 01 04 09 03 00 01 ............ row7
00 04 00 04 00 04 04 05 07 06 08 02 01 04 09 03 00 01 ............ row8

00 04 00 04 00 04 04 05 07 06 08 02 01 04 09 03 00 01 ............ row9
00 04 00 04 00 04 04 05 07 06 08 02 01 04 09 03 00 01 ............ row10

00 04 00 04 00 04 04 05 07 06 08 02 01 04 09 03 00 01 ............ row11
00 04 00 04 00 04 04 05 07 06 08 02 01 04 09 03 00 01 ............ row12
```

The following example shows the hexadecimal values of a 4bpp compressed bitmap:

```
03 04  FA 04 05 07 06 08 02 ...........  00 03  00 00 04  ...
lf df  lf df df df df df df              lf df  lf df df
cell1        cell2                       celln  celln+1
```

This bitmap would expand as follows (one-digit value represents a color index for a single pixel):

```
0 4 0 4 0 4 0 4 0 5 0 7 0 6 0 8 0 2 ............ row1

                                        do three more identical rows (celln):
```

```
0 4 0 4 0 4 0 4 0 5 0 7 0 6 0 8 0 2 ..... row2
0 4 0 4 0 4 0 4 0 5 0 7 0 6 0 8 0 2 ..... row3
0 4 0 4 0 4 0 4 0 5 0 7 0 6 0 8 0 2 ..... row4

                                    do four pairs of identical couples (celln+1):

0 4 0 4 0 4 0 4 0 5 0 7 0 6 0 8 0 2 ..... row5
0 4 0 4 0 4 0 4 0 5 0 7 0 6 0 8 0 2 ..... row6

0 4 0 4 0 4 0 4 0 5 0 7 0 6 0 8 0 2 ..... row7
0 4 0 4 0 4 0 4 0 5 0 7 0 6 0 8 0 2 ..... row8

0 4 0 4 0 4 0 4 0 5 0 7 0 6 0 8 0 2 ..... row9
0 4 0 4 0 4 0 4 0 5 0 7 0 6 0 8 0 2 ..... row10

0 4 0 4 0 4 0 4 0 5 0 7 0 6 0 8 0 2 ..... row11
0 4 0 4 0 4 0 4 0 5 0 7 0 6 0 8 0 2 ..... row12
```

Standard Golomb Run Length Encoding compression is inefficient at compressing 2x2 dither patterns, which are commonly used by Presentation Manager Display Drivers. The modified compression algorithm handles these patterns efficiently for the following reasons:

- For 4bpp and 8bpp, the data field size is such that, when a row is compressed, pairs of adjacent pels are put in each data field. Note that there is no dithering at 16bpp.
- When searching for duplicate scanlines, the algorithm also searches for duplicate scanline pairs that will match and compress patterns that repeat on alternate scanlines.

The actual pixel data is stored in Motorola format; that is, the most significant bits of a byte contain the leftmost pels. An example is shown below for a pair of pixels (PEL1,PEL2):

for the 4bpp format:
- PEL1 goes in bits 7..4
- PEL2 goes in bits 3..0

for the 8bpp format:
- PEL1 goes in bits 15..8
- PEL2 goes in bits 7..0

All 4bpp data is defined as indices into the standard VGA palette. All 8bpp data is defined as indices into the standard XGA palette. All 16bpp data is defined as XGA-format 16-bit color values. (See the appendix regarding default color palettes in the *IBM OS/2 Display Device Driver Reference* for details of these formats).

All changed drivers must convert their own internal format into one of these standard formats before transmission. Refer to the section on Converting Data later in this chapter.

Maintaining and Tracking Screen Change Areas (SCAs)

A key part of the modifications concerns the maintenance and tracking of SCAs.

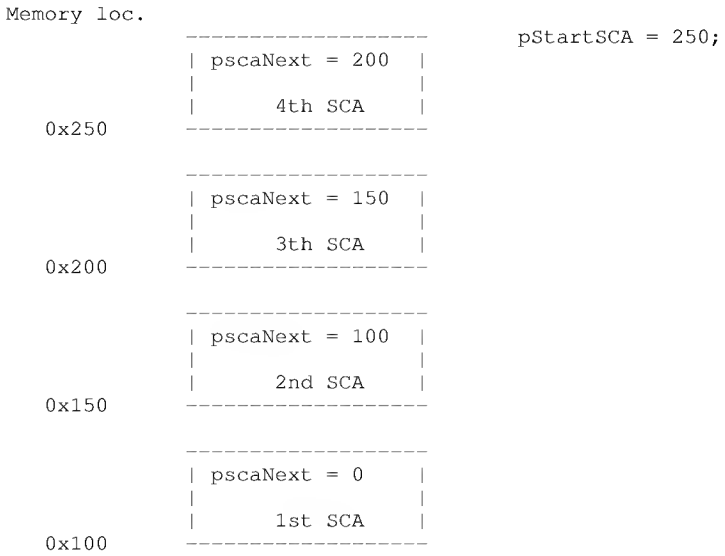
SCA Definition

These areas track, as efficiently as possible, regions of the screen that are altered by display driver drawing routines. This tracking is done by using multiple rectangles to define the area, rather than the usual single bounding rectangle provided by *traditional* Presentation Manager bounding functions.

SCAs are maintained within the driver in an SCA data structure, defined in the new DCAF.H file. For details, refer to the SCA data structure in the Appendixes of the *OS/2 Presentation Device Driver Reference* . Instances of this structure are dynamically created and destroyed upon calls to GreOpenScreenChangeArea and GreCloseScreenChangeArea, respectively. For details, refer to these functions in the chapter of the *OS/2 Presentation Device Driver Reference* that discusses mandatory and simulated graphics engine functions.

A global variable, *pStartSCA,* points to the latest SCA instance created. If *pStartSCA* is null, there are no active SCAs. All SCA instances are

linked together in a list using the *pscaNext* field of the SCA data structure. A null value in this field indicates the end of the list (the earliest created SCA). For example:



Each SCA instance can store multiple rectangles, up to MAX_SCA_RECTS (14), which define the area on the screen that has changed since the SCA was created or last queried. These rectangles are stored in the array arcl[]. The number of rectangles stored within the array is kept in the *cRects* field, which will never exceed MAX_SCA_RECTS. If *cRects* is zero, the SCA is a null area - the initial state.

The remaining field in the SCA data structure, *aulRectSize[],* is an array containing the sizes of the rectangles in arcl[]. This is not strictly necessary, because the sizes can be calculated on the fly using the dimensions in arcl[]. However, when accumulating a rectangle into a SCA, the size of each of the rectangles is frequently needed. Caching the rectangle sizes in this array saves having to recalculate the sizes every time, resulting in better performance.

The SCA data structure defines space for (MAX_SCA_RECTS+1) rectangles, but only MAX_SCA_RECTS are ever used to define the SCA. The extra rectangle is used to simplify the routine that accumulates rectangles into the SCA.

Creating a new SCA

Creating a new SCA, accomplished by GreOpenScreenChangeArea (SCRAREA.C), requires the following steps:

- Allocate memory for the new SCA instance
- Set the *cRects* field to be zero
- Set the *pStartSca* global variable to point to the new SCA address
- Link the instance to the linked list of SCAs

The newly created SCA will be identified by a 32-bit handle, which is the address of the SCA location in the display driver.

Accumulating a Rectangle into an SCA

All display driver functions that draw to the screen are modified to accumulate clipped bounding rectangles into all active SCAs when necessary. The drawing functions determine whether they should do this by examining the FunN (Function Number/COM_ flags) parameter. If the COM_SCR_BOUND flag is set, and the function is drawing to the screen, then bounding rectangles are accumulated into the active SCAs; otherwise, no accumulation takes place. The setting of COM_SCR_BOUND is controlled by the GreOpenScreenChangeArea and GreCloseScreenChangeArea functions. For details, refer to these functions in the chapter of the *OS/2 Presentation Device Driver Reference* that discusses mandatory and simulated graphics engine functions.

In every drawing call, the following test is performed:

```
    if (DCAFBoundsRequired(FunN))
    {
      accumulate the bound

    }
```

where the DCAFBoundsRequired macro is defined in DCAFEXTF.H as follows:

```
#define DCAFBoundsRequired(FunN)       \
    ( (FunN & COM_SCR_BOUND) &&        \     /* COM_SCR_BOUND is set    */
      (FunN & COM_DRAW) &&             \     /*  COM_DRAW is set     */
      (pdc->DCIDCType == OD_DIRECT) )        /*  the destination is the screen */
```

When change-area tracking is not needed, COM_SCR_BOUND will never be set. The difference in operation and performance of the DCAF-enabled base driver will be negligible-one additional check of the COM_SCR_BOUND flag per drawing function.

Performing the Bounding Accumulation

Two routines (in SCBOUNDS.C) perform the bounding accumulation:

- VOID AccumulateScreenBounds(PRECTL prclArgBound);
- VOID AccumulateScreenBoundsThroughClips(pDevRect pBoundCoords, ULONG ulCoordType);

AccumulateScreenBounds

This routine is called by GreSetPel and GrePolyShortLine, the drawing functions that are able to pass preclipped bounding rectangles. Its task is to take the passed rectangle and accumulate it into all the current SCAs. The passed rectangle is in exclusive SCREEN coordinates.

AccumulateScreenBoundsThroughClips

This routine takes the supplied (unclipped) bounding rectangle, intersects it with each of the clip rectangles in the DC and accumulates each of the clipped bounds into the active SCAs. The supplied bounding rectangle can be in one of the following types of coordinates:

16-bit AI coordinates (COORD_AI)	Origin is top-left of screen
16-bit Device coordinates (COORD_DEVICE_WORD)	Origin is current DC origin
32-bit Screen Coordinates	Origin is bottom-left of screen

The *ulCoordtype* field specifies which of these coordinates is being supplied.

AccumulateScreenBoundsThroughClips can be called from the same point in drawing functions as the ordinary (unclipped) bounds are accumulated. This minimizes the complexity and number of changes required in the main drawing code. Before accumulating the bounding rectangle, according to the following accumulation algorithm, the coordinate conversion (if required) to exclusive SCREEN coordinates and the clipping of the passed rectangle must be performed. The clipping can be done against the DC cache clip rectangles, or it may require a call back to the Graphics Engine to get the clip set. When a rectangle is added into an SCA, it is done in such a way as to minimize the increase in area of the SCA.

The following accumulation algorithm accomplishes this:

```
for (pscaCurrent = each SCA in the linked list)
:
: // First check whether the new rect is already contained within this SCA
: for (rclCurrent = each rectangle in current SCA)
```

```
: : if rclNew lies completely within rclCurrent
: : : no more work - skip straight to next SCA
: : endif
: endfor

: // We have to add the rectangle to the SCA.
: // First see if there is free space for the rectangle within the SCA.
: if pscaCurrent->cRects < MAX_SCA_RECTS
: : copy rect into SCA
: : calculate size and store in SCA
: : increment pscaCurrent->cRects
: else
: : // All of the SCA rects are used.
: : // Copy the new rect into the SCA at position (MAX_SCA_RECTS+1) and the
: : // problem then becomes:
: : // We have MAX_SCA_RECTS+1 rectangles, which we have to reduce
: : // to MAX_SCA_RECTS by merging two of the rectangles into a single rectangle.
: : // The pair of rects that we merge are the two that will cause the smallest
: : // increase in area.
: : initialize ulSmallestAreaIncrease to be maximum possible value
: : for (iRect1 = each rectangle in the SCA)
: : : for (iRect2 = iRect1+1 to MAX_SCA_RECTS+1)
: : : // This inner loop is the performance bottleneck.
: : : // Make it as fast as possible, if you can!!
: : : : if area increase of (iRect1,iRect2) merged < ulSmallestAreaIncrease
: : : : : set ulSmallestAreaIncrease to be area increase of (iRect1,iRect2) merged
: : : : : set best pair of rects to be (iRect1,iRect2)
: : : : endif
: : : endfor
: : endfor
: : 
: : merge best pair of rects found into the slot originally occupied by Rect1
: : if rclNew was not one of those merged
: : : copy rclNew into vacant slot made by merging pair
: : endif

: endif
endfor
```

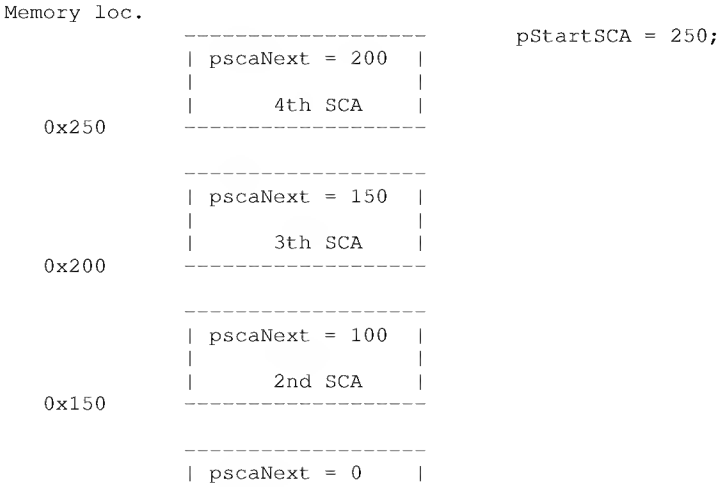
When change-area tracking is active, this routine is called by every function that draws to the screen. Therefore, the routine must be as efficient as possible (particularly in the inner loop) to minimize the hit on performance.

Deleting an SCA

This task is performed by GreCloseScreenChangeArea (SCRAREA.C), as follows:

- Unlink the SCA instance from the linked list of SCAs
- Free the memory for the SCA instance

In a typical example, if we close the second SCA:



```
              |                  |
              |      1st SCA     |
   0x100      --------------------
```

we will get:

```
Memory loc.
              --------------------       pStartSCA = 250;
              | pscaNext = 200   |
              |                  |
              |      4th SCA     |
   0x250      --------------------

              --------------------
              | pscaNext = 100   |
              |                  |
              |      3th SCA     |
   0x200      --------------------

              --------------------
              | pscaNext = 0     |
              |                  |
              |      1st SCA     |
   0x100      --------------------
```

If the last remaining SCA is being freed, pStartSCA is set to NULL. If the latest SCA created is being freed, pStartSCA is set to the address of the SCA created immediately prior to it (the previous SCA).

Converting Data

The changed display drivers use different internal data formats:

- VGA 4bpp planar
- XGA 4bpp packed, 8bpp packed, 16bpp packed

When data is transmitted between display drivers, it is done at the lower bpp of the two drivers (or at the lowest bpp; for example, a pair of 16bpp DCAF-enabled drivers could communicate at 4bpp to reduce the amount of data transmitted). Therefore, the following conversion routines are required by the display driver:

XGA (4bpp, 8bpp, 16bpp packed)

internal format required format

16bpp packed -> 8bpp packed (compression)
16bpp packed -> 4bpp packed (compression)
16bpp packed -> 4bpp planar (compression)
8bpp packed -> 8bpp packed (compression)
8bpp packed -> 4bpp packed (compression)
8bpp packed -> 4bpp planar (compression)
4bpp packed -> 4bpp planar (compression)

external data format internal format

(decompression) 8bpp packed -> 16bpp packed
(decompression) 8bpp packed -> 8bpp packed
(decompression) 4bpp packed -> 16bpp packed
(decompression) 4bpp packed -> 8bpp packed
(decompression) 4bpp planar -> 16bpp packed
(decompression) 4bpp planar -> 8bpp packed
(decompression) 4bpp planar -> 4bpp packed

The conversions from packed to planar and vice versa are assisted by the use of a lookup table to split packed bytes into bits that can be conveniently reassembled into planar format (and vice versa).

All conversions to planar format are done by first converting the bits per pel to 4 (still in packed format) and then performing an optimized 4bpp

packed-to-planar conversion. In conversions from 4bpp planar a similar, but reverse, process is performed- converting from 4bpp planar to 4bpp packed and then to the required packed destination format.

Conversions from 4bpp and 8bpp use a lookup table to efficiently translate the colors. Conversions from 16bpp cannot use a direct lookup table because the size is prohibitive. Therefore, colors have to be searched for on a *nearest color* basis in the destination color table. This is much slower than a simple table lookup. To improve performance, a cache of the most recently calculated colors is kept, which saves having to repeatedly recalculate commonly used colors.

Seamless Windows Support

In OS/2 2.1, Seamless Windows is supported by allowing the Windows display driver to draw directly on the Presentation Manager (PM) screen. This means that Seamless Windows updates do not go through the PM drawing functions and, therefore, will not update the active screen change area (SCA) in the usual way. As a result, Seamless Windows requires special treatment.

Prior to drawing on the PM screen, the Seamless Windows driver calls the PM driver through an exported entry point, SeamlessExcludeCursor. This call excludes the PM cursor from the area in which the Seamless Driver is about to draw. The modified DCAF-enabled display driver intercepts this call and passes the rectangle coordinates to AccumulateScreenBounds.

Under DCAF, during PM display driver initialization, Seamless Windows must be granted addressability to all data and code that it will access during the call to SeamlessExcludeCursor. In order to add the supplied rectangle to all active SCAs, which can reside anywhere in the display driver heap, there is a single, static SSA (Seamless Screen Area), called scaSeamless, used for this purpose. All Seamless bounding rectangles will be accumulated in scaSeamless; then, this SCA is merged with the other active SCAs when a query is issued via GreGetScreenChangeArea.

As a result, at initialization (in InitializeSeamless), the Seamless Windows driver is given access to AccumulateScreenBounds, to scaSeamless and to the DDT display driver control block, in order to retrieve the screen dimension at seamless time. The addresses of this data are stored in the SeamlessAddresses control block, owned by the Windows driver.

Before writing to the PM screen, the Seamless Windows driver will call SeamlessExcludeCursor. The exclusion rectangle will be passed in the following registers in so-called AI coordinates (i.e., 16-bit inclusive; 0,0 is top left of screen):

Left	cx
Top	dx
Right	si
Bottom	di

The display driver will determine whether the new bounds accumulation is needed by checking the value of the pStartSCA pointer and, in SeamlessExcludeCursor32, will call AccumulateSeamlessBounds to initiate the bounds accumulation. AccumulateSeamlessBounds converts the passed rectangle to EXCLUSIVE SCREEN coordinates, clips the rectangle to the screen dimensions and calls AccumulateScreenBounds. This causes the rectangle supplied to SeamlessExcludeCursor to be added to only scaSeamless. When an SCA is queried using GreGetScreenChangeArea, the Seamless SCA is merged with the active SCAs and then reset to NULL.

Graphics Engine Hardcopy Drivers

For the Presentation Manager interface, hardcopy devices, such as printers and plotters, are queued devices. When an application writes to one of these devices, the presentation driver creates a spool file and writes the data to that file. The data is printed when it is complete and the required device is free.

Two instances of a device context are required to support queued data. The first instance is opened as an OD_QUEUED device by the application. This DC buffers the data, does any processing that is required, and then writes the data to a spool file. The second instance is opened as an OD_DIRECT device by the queue processor. This DC receives data from the spool file, does any processing that is required, and by using the Prt*xxx* interface, sends the data to the final output destination, such as port, device, or UNC name.

When a device context is opened, the data type given (PM_Q_STD or PM_Q_RAW) is applicable only for the OD_QUEUED device context type.

Exported Entry Points

The following entry points must be exported by a hardcopy driver dynamic link library:

```
EXPORTS
   OS2_PM_DRV_DEVMODE
   OS2_PM_DRV_DEVICENAMES
   OS2_PM_DRV_ENABLE
   DrvInstall                /* Optional */
   DrvRemove                 /* Optional */

   DialogProc    @2
```

Note: DialogProc is exported by ordinal. The entry point is used by the Presentation Manager interface to manage the dialog initiated by the OS2_PM_DRV_DEVMODE.

Additional information on Exported Entry Points, including detailed descriptions of the functions and subfunctions can be found in Exported Driver Function Reference and OS2_PM_DRV_DEVMODE.

InnerGre Entries

The InnerGre entries are important to some 32-bit hardcopy presentation drivers that are coded in the "shadow DC" style. A shadow DC is actually a display OD_MEMORY DC. It is created by a printer driver to "steal" rasterization services from the currently installed display driver, and therefore save writing some code.

GRE Macro Expansion

For the following GRE macro:

```
GreCreateBitmap( hdc, pbmiHeader, flUsage, pBits, pbmi );
```

the compiler expands the call as follows:

```
Gre32Entry7( hdc, pbmiHeader, flUsage, pBits, pbmi, NULL, NGreCreateBitmap );
```

Note the two additional parameters. The NULL is a pointer to instance data for the device drawing context (DC). The graphics engine supplies the correct pointer when it dispatches the function to the device driver. NGreCreateBitmap is a combination of command drawing flags in the high word and requested function number in the low word. The graphics engine sets the command flags when it dispatches the function to the device driver.

InnerGre Macros

For each Gre32Entry there is also an InnerGre32Entry. Some 32-bit hardcopy drivers call both the standard GRE entries as well as the InnerGre entry. These hardcopy drivers use a shadow device context, which is an OD_MEMORY device context in the currently installed display driver, to assist with rasterization. When called to perform a drawing function, these hardcopy drivers relay the work to the shadow DC (thus the display driver) by substituting the hardcopy DC handle with the shadow DC handle and calling the appropriate engine entry. This style of hardcopy driver coding made necessary the InnerGre entries.

The InnerGre entries differ from normal GRE entries in these ways:

- For the InnerGre entries, the graphic engine does not reload the command flags parameter (the last parameter). Instead, the engine

uses the command flags parameter exactly as supplied by the caller.

- The graphics engine will not transform coordinates when called at the InnerGre level. The InnerGre entries always expect device coordinates.
- The graphics engine does not perform validation on the supplied device context handle when called at the InnerGre level.

The *Developer's Toolkit for OS/2* does not provide an include file for InnerGre macros. Refer to to INNERGRE.H in the MiniDriver code sample in the *IBM Device Driver Source Kit for OS/2* , (Version 1.1 or higher). The MiniDriver is an example of a hardcopy driver that uses a shadow device context.

File System Emulation

Presentation drivers for hardcopy devices use an internal interface to communicate with the device. Hardcopy drivers do not differentiate between different types of ports- LPT1 (parallel) and COM1 (serial). The Prt*xxx* API routes the data to the appropriate physical device driver. This API also handles semaphoring the port so that two threads do not intermix their output.

The internal interface is based on the DOS file system calls DosOpen, DosClose, DosWrite, and so forth. Presentation drivers open a device and receive a handle that identifies the device as a file. Subsequent operations such as writing to the device are implemented by writing to the returned handle.

The handle returned by PrtOpen is not an OS/2 file handle, and can only be used in other Prtxxxx APIs.

The following functions are used by the presentation driver:

- PrtAbort
- PrtClose
- PrtDevIOCtl
- PrtOpen
- PrtWrite

The following functions are new for OS/2 Warp, Version 3:

- PrtAbortDoc
- PrtResetAbort
- PrtNewPage

Spooler Support for PM_Q_STD Data Type

The following functions are available to help the hardcopy driver create a spool file containing PM_Q_STD data:

SplStdClose
SplStdDelete
SplStdGetBits
SplStdOpen
SplStdQueryLength
SplStdStart
SplStdStop

Spooler Components

The spooler interface (Spl...) is implemented in two libraries, PMSPL.DLL and PMPRINT.QPR, which support four activities:

- Management of spool buffers for PM_Q_STD data (SplStd*xxx* interface)
- Management of queued files (SplQm*xxx* interface)
- Processing of queued files (SplQp*xxx* interface)
- Displaying messages on the screen (SplMessageBox)

Output Strategy

The following illustration summarizes the spooler and direct output strategies.

DC_TYPE	OD_DIRECT	OD_QUEUED	
DATATYPE	PM_Q_RAW PM_Q_STD	PM_Q_RAW	PM_Q_STD
Spool File Creation SplQmXXX		X	X
Metafile Creation SplStdXXX			X
Output PrtXXX	X		
Output Printer Language	X	X	

Output Strategy

Spool File Creation

The steps taken by the presentation driver to create a spool file are determined by the data type for which the DC was enabled. All presentation drivers must support PM_Q_STD and PM_Q_RAW. Overviews of creating a spool file for these data types are shown below. See the following illustration.

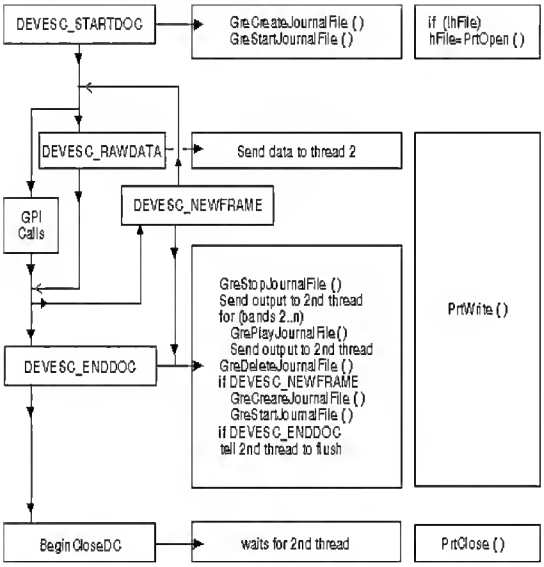
Data types can be defined by the user. A name should be chosen that is not likely to conflict with other user-defined data types. The name must be a string of up to 16 characters in the ranges 'A' through 'Z', '0' through '9', or _. Note that the data type is only useful to applications that know about it, and with presentation drivers that implement it.

Print jobs must be spooled by using the data type given on the call to OS2_PM_DRV_ENABLE Subfunction 02H - FillPhysicalDeviceBlock. Therefore, print jobs queued PM_Q_STD can be printed with queue processor options applied to the job.

Output Directly to Device

The handling of an OD_DIRECT DC is similar to an OD_QUEUED PM_Q_RAW DC, except that PRT*xxx* calls are used to write the data. You should use a second thread for writing to the device so that the application's thread is not blocked for long periods of time during a device timeout situation.

The following illustration shows this output method.



Output to Device - OD_DIRECT

PM_Q_STD

PM_Q_STD data is a file of Gpi*xxx* calls that describe the output document. This data type is independent of the device and the presentation driver. The presentation driver invokes the spooler's standard interface (SplStd*xxx*). All subsequent Gpi*xxx* calls and some escape codes sent to the DC are recorded in a spool buffer. When DEVESC_ENDDOC is detected, the presentation driver ends the recording and invokes the spooler's interface (SplQm*xxx*) to write the buffered data into a spool file.

Normal Sequence of Events;

1. Application calls DevOpenDC to open an OD_QUEUED device for printing PM_Q_STD data.
2. Hardcopy driver calls SplStdOpen to open a recording.
3. Application calls DevEscape with DEVESC_STARTDOC.
    a. Hardcopy driver calls SplStdStart to start recording.
    b. Hardcopy driver calls SplQmOpen and SplQmStartDoc to open and start a spool file.
    c. Spooler records all Gpi*xxx* calls and some escape codes in the spool buffer.

    Note: Recording does not stop the flow of Gpi*xxx* calls through the system. These calls are processed and the resulting Gre*xxx* calls are passed on to handling routines in the graphics engine and presentation driver. Special considerations apply for escape codes. For details, see the individual escape codes under GreEscape.
    d. Hardcopy driver DC is OD_INFO. The presentation driver tracks the current position, does any bounds calculation required, and responds to queries from the application. In particular, the presentation driver must be able to understand and reply to:
        - DevQueryCaps
        - DevQueryHardCopyCaps
        - DevQueryDeviceNames
        - DevPostDeviceModes
4. Application calls DevEscape with DEVESC_NEWFRAME.
    a. Hardcopy driver resets current position and bounds.
5. Application calls DevEscape with DEVESC_ENDDOC.
    a. Hardcopy driver calls SplStdStop to stop the recording.

b. Hardcopy driver calls SplStdQueryLength to get the length, in bytes, of the spooled data.

c. Hardcopy driver calls SplStdGetBits to get data from the spool buffer into memory that is owned by the presentation driver. (The driver might need to loop on this step and the next if the spooled data is larger than the available memory.)

d. Hardcopy driver calls SplQmWrite to write the data in the spool file.

e. Hardcopy driver calls SplStdDelete to delete the data in the spool buffer.

f. Hardcopy driver calls SplQmEndDoc to stop the spool file, and returns the Job ID to the application's DevEscape with DEVESC_ENDDOC.

6. Application calls DevEscape with DEVESC_STARTDOC (repeat Step 2).

7. Application calls DevEscape with DEVESC_NEWFRAME (repeat Step 3).

8. Application calls DevEscape with DEVESC_ENDDOC to spool the second job (repeat Step 4).

9. Application calls DevCloseDC.

a. Hardcopy driver calls SplStdClose and SplQmClose to close the spool buffer and the spool file.

End Sequence of Events:

1. Application calls DevOpenDC to open an OD_QUEUED device for printing PM_Q_STD data.

a. Hardcopy driver calls SplStdOpen to open a recording.

2. Application calls DevEscape with DEVESC_STARTDOC.

a. Hardcopy driver calls SplStdStart to start recording, and SplQmOpen to open a spool file if one is not yet open.
b. Spooler records all Gpi*xxx* calls and some escape codes in the spool buffer.

Note: Recording does not stop the flow of Gpi*xxx* calls through the system. These calls are processed and the resulting Gre*xxx* calls are passed on to handling routines in the graphics engine and presentation driver. Special considerations apply for escape codes. For details, see the individual escape codes under GreEscape.

c. Hardcopy driver performs as if the DC was opened as OD_INFO. The presentation driver tracks the current position, does any bounds calculation required, and responds to queries from the application. In particular, the presentation driver must be able to understand and reply to:
   - DevQueryCaps
   - DevQueryHardCopyCaps
   - DevQueryDeviceNames
   - DevPostDeviceModes

The calls can be journaled, but the journal file is not saved.

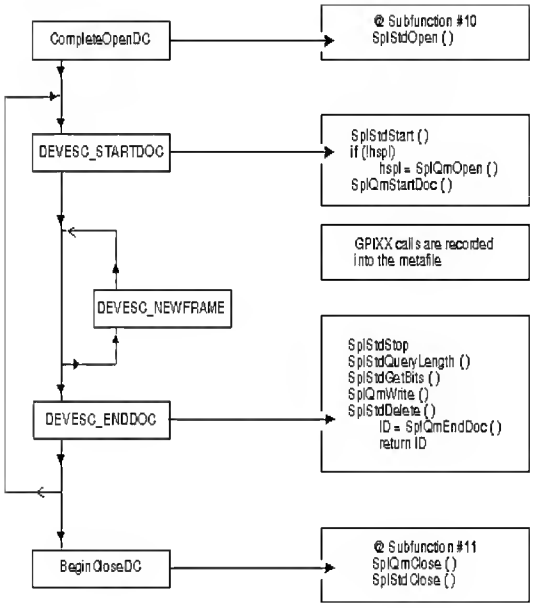
3. Application calls DevEscape with DEVESC_ABORTDOC to end the document.

a. Hardcopy driver calls GreStopJournalFile.
b. Hardcopy driver calls GreDeleteJournalFile.
c. Hardcopy driver calls SplStdStop to stop the recording.
d. Hardcopy driver calls SplStdDelete to delete the data in the spool buffer.
e. Hardcopy driver calls SplQmAbortDoc to stop the spool file.

4. The application calls DevCloseDC.

a. Hardcopy driver calls SplStdClose and SplQmClose to close the recording and the spool file.

This method of spool file creation is shown in the following illustration.



Spool File Creation - OD_QUEUED PM_Q_STD

PM_Q_RAW

PM_Q_RAW data is a device-dependent bit stream.

Normal Sequence of Events:

1. Application calls DevOpenDC to open an OD_QUEUED device for printing PM_Q_RAW data.
2. Application calls DevEscape with DEVESC_STARTDOC.
    a. Hardcopy driver calls SplQmOpen to open a spool file if one is not yet opened.
    b. Hardcopy driver calls GreCreateJournalFile to create a journal file.
    c. Hardcopy driver calls SplQmStartDoc to start the spool file.
    d. Hardcopy driver calls GreStartJournalFile to start recording Gre*xxx* calls.

    Note: Recording does not block the flow of Gre*xxx* calls to handling routines in the graphics engine and presentation driver.
    e. Hardcopy driver processes the incoming Gre*xxx* calls to create the first band of data for the spool file.
3. Application calls DevEscape with DEVESC_NEWFRAME to start a new page.
    a. Hardcopy driver calls GreStopJournalFile to stop the journal file.
    b. Hardcopy driver calls SplQmWrite to write the first band in the spool file.
    c. Hardcopy driver moves clip rectangle to next band in presentation space.
    d. Hardcopy driver calls GrePlayJournalFile to play the journal file.
    e. Hardcopy driver processes each Gre*xxx* call to create the second band of data for the spool file.
    f. Hardcopy driver calls SplQmWrite to write the second band in the spool file.
    g. Hardcopy driver plays the journal file repeatedly until all the bands have been processed and passed to the spooler.

h. Hardcopy driver issues a page eject, if necessary.

i. Hardcopy driver calls GreDeleteJournalFile.

j. Hardcopy driver calls SplQmEndDoc to stop the spool file, and returns the Job ID to the application's DevEscape with DEVESC_ENDDOC.

Note: If no Gre*xxx* calls have been made (that is, no output is required), only Steps 1 and 8 are executed.

4. Application calls DevEscape with DEVESC_ENDDOC to end the document.

   a. Hardcopy driver calls GreStopJournalFile to stop the journal file.

   b. Hardcopy driver calls SplQmWrite to write the first band in the spool file.

   c. Hardcopy driver calls GrePlayJournalFile to play the journal file.

   d. Hardcopy driver processes each Gre*xxx* call to create the second band of data for the spool file.

   e. Hardcopy driver calls SplQmWrite to write the second band in the spool file.

   f. Hardcopy driver plays the journal file repeatedly until all the bands have been processed and passed to the spooler.

   g. Hardcopy driver calls GreDeleteJournalFile.

   h. Hardcopy driver calls SplQmEndDoc to stop the spool file, and returns the Job ID to the application's DevEscape with DEVESC_ENDDOC.

   Note: If no Gre*xxx* calls have been made (that is, no output is required), only Steps 3.a, 3.i, and 3.j are executed. No page is ejected.

5. Application calls DevEscape with DEVESC_STARTDOC (repeat Step 2).

6. Application calls DevEscape with DEVESC_NEWFRAME to start new page (repeat Step 3 under Normal Sequence of Events).

7. Application calls DevEscape with DEVESC_ENDDOC to spool the second job (repeat Step 4).

8. Application calls DevCloseDC.

   a. Hardcopy driver calls SplQmClose.

End Sequence of Events:

1. Application calls DevOpenDC to open an OD_QUEUED device for printing PM_Q_RAW data.

2. Application calls DevEscape with DEVESC_STARTDOC to start the document.

   a. Hardcopy driver calls SplStdOpen to open a spool file.
   b. Hardcopy driver calls GreCreateJournalFile to create a journal file.
   c. Hardcopy driver calls SplQmStartDoc to start the spool file.
   d. Hardcopy driver calls GreStartJournalFile to start recording Gre*xxx* calls.

      Note: Recording does not block the flow of Gre*xxx* calls to handling routines in the graphics engine and presentation driver.

   e. Hardcopy driver processes the incoming Gre*xxx* calls to create the first band of data for the spool file.

3. Application calls DevEscape with DEVESC_ABORTDOC to end the document.

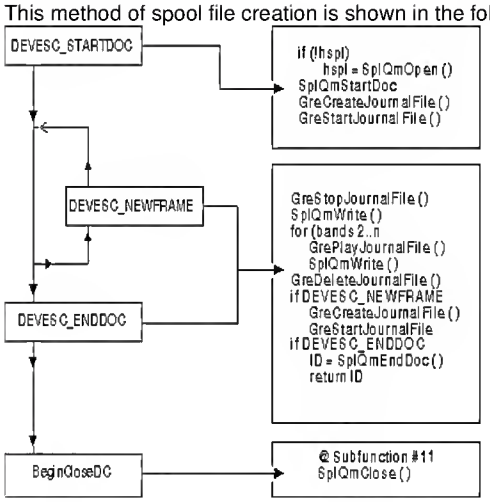
   a. Hardcopy driver calls GreStopJournalFile
   b. Hardcopy driver calls GreDeleteJournalFile
   c. Hardcopy driver calls SplQmAbortDoc to stop the spool file.

      Note: The hardcopy driver can be processing the DEVESC_ENDDOC (banding, for example) and a DEVESC_ABORT comes into the hardcopy driver on another thread.

4. Application calls DevCloseDC.

   a. Hardcopy driver calls SplQmClose to close the spool file.

This method of spool file creation is shown in the following illustration.



Spool File Creation - OD_QUEUED PM_Q_RAW

Querying and Setting Configuration Data

The configuration of printers and queues is stored in OS2SYS.INI. For optimum performance, use the *SplxxxDevice* and *SplxxxQueue* functions together as a high-level interface into OS2SYS.INI. Refer to the *OS/2 Presentation Manager Programming Reference* for further information.

Spooler Support Functions

The purpose of the spooler is to control the queues, create new spool files, and invoke the queue processor when a job is ready for printing. The spooler also provides a function, SplMessageBox, that can be called to display a message to the user.

The following functions are available in the spooler:

- SplMessageBox
- SplQmAbort
- SplQmAbortDoc
- SplQmClose
- SplQmEndDoc
- SplQmOpen
- SplQmStartDoc
- SplQmWrite

The following functions are new for OS/2 Warp, Version 3:

- SplQmNewPage
- SplQmGetJobID

For a complete description of each of the functions listed in this chapter, see Device Support Function Reference. They are listed alphabetically in the index table for that chapter.

Queue Drivers

The Workplace Shell uses the term "queue driver" to identify the queue processor. Each queue has its own queue processor, which prints a spool file. The Presentation Manager interface delivers two different queue drivers. Presentation Manager system queue drivers are supplied in the files PMPRINT.QPR and PMPLOT.QPR.

The spooler calls these functions by using DosLoadModule and DosGetProcAddr. Queue drivers can be 16-bit or 32-bit DLLs.

The user can supply queue drivers to support user data types, however, any queue driver created by the user must support PM_Q_STD and PM_Q_RAW standard data types.

Queue Driver Printing Methods

The method used by the queue driver to write data to the hardcopy device depends on whether the data type is PM_Q_STD or PM_Q_RAW.

PM_Q_STD

PM_Q_STD performs in the following manner:

1. Opens a DC for the hardcopy device by using DevOpenDC and enables it as an OD_DIRECT device
2. Calls DevEscape with DEVESC_STARTDOC
3. Writes data, which is in metafile format, by using GpiPlayMetafile
4. Calls DevEscape with DEVESC_ENDDOC
5. Closes hardcopy device DC by using DevCloseDC

PM_Q_RAW

PM_Q_RAW performs in the following manner:

1. Opens a DC for the hardcopy device by using DevOpenDC and enables it as an OD_DIRECT device
2. Calls DevEscape with DEVESC_STARTDOC
3. Writes the data using DevEscape with DEVESC_RAWDATA
4. Calls DevEscape with DEVESC_ENDDOC
5. Closes hardcopy device DC by using DevCloseDC

User Data Types

The processing required for user data types depends on the format of the data type. In some instances, it might be necessary to create a special queue processor to support the data type.

Interface Functions

Seven functions provide an interface to the queue driver.

- SplQpClose
- SplQpControl
- SplQpInstall
- SplQpOpen
- SplQpPrint
- SplQpQueryDt
- SplQpQueryFlags

Note: OS/2 2.1 supports both 16-bit and 32-bit queue drivers (DLLs).

For a complete description of each of the functions listed in this chapter, see Device Support Function Reference. The functions are listed alphabetically in the index table for that chapter.

Port Drivers

Port drivers are dynamic link libraries (DLLs) that contain a set of 32-bit functions, which provide helper functions for the spooler and Workplace Shell. The file type of a port driver is ".PDR".

There are two types of port drivers-the "old-style" port drivers written to run under OS/2 Version 2.x, and the "new-style" port drivers that will run only on versions of OS/2 that contain the bidirectional print spooler (OS/2 Warp, Version 3 and higher). See the "Bidirectional Communications" chapter in the *Printer Device Driver Reference for Os/2* for a description of the bidirectional spooler and its new interfaces.

The OS/2 2.x port driver DLLs required a physical port device driver (.SYS file) installed in the CONFIG.SYS file. These port drivers were only called at the beginning (SplPdInitPort) and at the end (SplPdTermPort) of each print job being sent to the port.

The OS/2 Warp, Version 3 port driver interface passes all print job data to the port driver and allows the port driver to return information returned by the printer. SplPdInitPort and SplPdTermPort are no longer necessary. Applications and print drivers can use the new PrtQuery API to get information about the printer. This API is useful for the printer driver when displaying its printer properties. See PrtQuery in Device Support Function Reference for a description of the API, its format and syntax. The file type of a port driver is .PDR.

The operating system, by default, provides two port drivers; SERIAL.PDR supports COM1 - 4, and PARALLEL.PDR supports LPT1 - 3. For any other port, the supplier of the physical port software is responsible for providing a Presentation Manager port driver if it is necessary.

The functions exported from a port driver are:

- SplPdEnumPort
- SplPdGetPortIcon
- SplPdInitPort
- SplPdInstallPort
- SplPdQueryPort
- SplPdRemovePort
- SplPdSetPort
- SplPdTermPort

OS/2 Warp, Version 3 port drivers must export the following APIs to replace SplPdInitPort and SplPdTermPort:

- SplPdOpen
- SplPdWrite
- SplPdClose
- SplPdAbortDoc
- SplPdNewPage
- SplPdQuery
- SplPdSet

OS/2 Warp, Version 3 port drivers can export the following optional APIs:

- SplPdResetAbort
- SplPdSendCmd
- SplPdRemoteSetPort

OS2SYS.INI File Structure

The contents of the INI file section for ports is as follows:

- Port driver PDR installation:

```
appname: PM_PORT_DRIVER  keyname: <name>  value: <full path to port driver>

For example:
```

```
PM_PORT_DRIVER, SERIAL,     C:\OS2\DLL\SERIAL.PDR
                PARALLEL,   C:\OS2\DLL\PARALLEL.PDR
                EXTLPT,     C:\OS2\DLL\EXTLPT.PDR
```

This is written by the Workplace Shell during installation.

- Ports known to the system:

```
appname: PM_<portname> keyname: DESCRIPTION    value: <port description>
                       keyname: INITIALIZATION value: <initialization values>
                       keyname: TERMINATION    value: <termination values>
                       keyname: PORTDRIVER     value: <name of port driver>
                       keyname: TIMEOUT        value: <timeout in seconds>

For example:

PM_COM1, DESCRIPTION,         Serial Port COM1;
PM_COM1, INITIALIZATION,      9600;0;W;8;1;
PM_COM1, TERMINATION,         ;
PM_COM1, PORTDRIVER,          SERIAL;
PM_COM1, TIMEOUT,             45;
```

For compatibility, the existing port structure is also supported by all port drivers.

```
appname: PM_SPOOLER_PORT  keyname: <port> value: <port init/term sting>

For example:

PM_SPOOLER_PORT, LPT1,     ;
                 COM1,     9600;0;W;8;1;
```

The port driver is expected to maintain its own sections in the OS2SYS.INI file. The format of the values stored under the keynames INITIALIZATION and TERMINATION is port-driver specific. Also, the port driver might need to hold extra data under the PM_<port> appname.

For a complete description of each of the functions listed in this chapter, see Device Support Function Reference. The functions are listed alphabetically in the index table for that chapter.

Presentation Manager Function Categories

This chapter categorizes the Presentation Manager functions as mandatory for all drivers, mandatory for display drivers or hardcopy drivers, or functions simulated by a handling routine in the graphics engine. Each type of function is described by category as well.

Mandatory Functions for All Drivers

This section describes those functions that must be supported for all devices. These functions are called through the dispatch table by handling routines in the presentation driver. The dispatch table contains the address of each function that the presentation driver can hook. Initially, when the presentation driver is first loaded, a copy of the default dispatch table is passed to the driver. The presentation driver's Enable routine modifies this copy so that the entries for functions supported in the driver point to the handling routines in the driver. Entries to the table that are to be modified can first be saved by the presentation driver in case they are subsequently needed. The original saved table entries, and those that are not modified, point to the engine-simulation routines for the functions concerned.

Functions listed in the dispatch table fall into two categories:

- Functions that the presentation driver must support (mandatory functions)
- Functions that are supported by the graphics engine but can be optionally hooked by the presentation driver

Descriptions of the mandatory functions are provided. Each description shows what the handling routine is expected to do, the parameters passed to the routine, and the values that the routine returns. The functions are grouped according to the conditional include sections of the header file:

- Attribute functions (INCL_GRE_DEVMISC1)
- Bit-map functions (INCL_GRE_BITMAPS)
- Color table functions (INCL_GRE_COLORTABLE)
- Device functions 2 (INCL_GRE_DEVMISC2)
- Device functions 3 (INCL_GRE_DEVMISC3)
- GreEscape functions (INCL_GRE_DEVICE)
- Line functions (INCL_GRE_LINES, INCL_GRE_SCANS)
- Marker functions (INCL_GRE_MARKERS)
- Query functions (INCL_GRE_DEVICE)
- Text (String) functions (INCL_GRE_STRINGS)

Additional functions that must be supported by presentation drivers for display devices are described in Mandatory Functions for Display Drivers. Hardcopy drivers must also provide some support for these display device functions. This support can be a common routine that returns Successful and posts the warning, PMERR_DEV_FUNC_NOT_INSTALLED.

The level of support provided in the handling routine for a particular function depends on the type of device and the level of support provided for that device. At the minimum, the handling routine must indicate a successful completion.

Attribute and Bundle Definitions

A general description of colors, mixes, patterns, and the attribute definitions for each attribute bundle type are provided. For area definitions, the area must be filled by using the pattern that is current when GreBeginArea is called.

Colors

All colors are passed as 32-bit signed values. These are either indexes into the Logical Color table, logical palette indexes, or representations of 24-bit Red-Green-Blue (RGB) values. Some special attribute values can be passed to the graphics engine and returned by GreGetAttributes:

CLR_FALSE	All color planes or bits, or both, are 0.
CLR_TRUE	All color planes or bits, or both, are 1.
CLR_WHITE	This index is never loaded explicitly. It always produces White when the default color table is in force or when the index is set to RGB. With a realized color table and an index that is not RGB, the value CLR_WHITE produces the background color CLR_BACKGROUND.
CLR_BLACK	This index is never loaded explicitly. It always produces Black when the default color table is in force or when the index is set to RGB. With a realized color table and an index that is not RGB, CLR_BLACK produces the neutral color CLR_NEUTRAL. See Color Functions.

CLR_FALSE and CLR_TRUE provide useful operands for BitBlt logical operations. CLR_DEFAULT is the default value at the API. It is a reserved value and is not passed to the presentation driver.

Mix Modes

All values are passed to the graphics engine which passes them unchanged to the presentation driver.

Foreground Mix Mode	Valid values are:	
	FM_OR	OR
	FM_OVERPAINT	Overpaint
	FM_XOR	Exclusive-OR
	FM_LEAVEALONE	Leave alone (invisible)
	FM_AND	AND

FM_SUBTRACT	(Inverse source) AND destination
FM_MASKSRCNOT	Source AND (inverse destination)
FM_ZERO	All zeros
FM_NOTMERGESRC	Inverse (source OR destination)
FM_NOTXORSRC	Inverse (source Exclusive-OR destination)
FM_INVERT	Inverse of destination
FM_MERGESRCNOT	Source OR (inverse destination)
FM_NOTCOPYSRC	Inverse of source
FM_MERGENOTSRC	(Inverse source) OR destination
FM_NOTMASKSRC	Inverse of (source AND destination)
FM_ONE	All ones

Note: FM_DEFAULT is the default value at the API. It is a reserved value and is not passed to the presentation driver.

The presentation driver must support FM_OR, FM_OVERPAINT, FM_XOR, and FM_LEAVEALONE. Other foreground mixes can be handled as FM_OVERPAINT. Mixing, other than for FM_LEAVEALONE or FM_OVERPAINT, is performed on the physical color index or logical palette index. When an indexed color table or palette has been realized, this corresponds to the logical color index. In other cases, the color resulting from a mix cannot be predicted.

Note: An exception to this rule occurs when the same object is drawn twice with FM_XOR and BM_LEAVEALONE, and no intermediate drawing is performed in other mixes. The implementation must guarantee that this always results in the object being erased cleanly.

Other valid mixes can also be supported for some primitive types. When a valid mix is not supported, the default is FM_OVERPAINT. An error is raised only when the specified mix value is not one of those listed above.

Background Mix Mode

Valid values are:

BM_ERROR	Error
BM_DEFAULT	Default
BM_OR	OR
BM_OVERPAINT	Overpaint
BM_XOR	Exclusive-OR
BM_LEAVEALONE	Leave alone (invisible)
BM_SRCTRANSPARENT	Provides a transparent overlay function by not copying (to the output bit map) the pels from the source bit map that match the presentation space background color.
BM_DESTTRANSPARENT	Provides a transparent underlay function by copying the pels from the source bit map only to the destination pels that match the presentation space

background color.

The presentation driver must support BM_OVERPAINT and BM_LEAVEALONE. Other background mixes can be handled as BM_LEAVEALONE.

GreQueryDeviceCaps allows the application to determine the mixes supported by the device.

Line Attributes

The device line attributes are bundled in a DLINEBUNDLE structure:

Parameter	Description
cAttr	Size of the logical attribute bundle.
cDefs	Size of the LINEDEFS structure.
lbnd	LINEBUNDLE structure. The logical line bundle as seen by the application. See ***LINEBUNDLE Mask:*** following this table.
ldef	LINEDEFS structure: defType Line definition. This is ignored by the presentation driver.

LINEBUNDLE Mask

This mask is used in calls to GreDeviceSetAttributes to identify fields in the LINEBUNDLE structure. Valid flags and the fields that they identify are:

Flag	Field
LBB_COLOR	lColor
LBB_BACK_COLOR	lBackColor
LBB_MIX_MODE	usMixMode
LBB_BACK_MIX_MODE	usBackMixMode
LBB_WIDTH	fxWidth
LBB_GEOM_WIDTH	lGeomWidth
LBB_TYPE	usType
LBB_END	usEnd
LBB_JOIN	usJoin

lbnd The fields of a LINEBUNDLE structure are:

lColor	Line foreground color.
lBackColor	Line background color.
usMixMode	Line foreground mix mode.
usBackMixMode	Line background mix mode.

fxWidth

Line-width multiplier. This value is expressed in fixed-point notation with a notational binary point between the second and third bytes. Therefore, 1.0 is represented by 10000h (or 65536). This multiplier is applied to the normal line width. Valid values are:

LINEWIDTH_DEFAULT	Default line width. This is the same as LINEWIDTH_NORMAL.
LINEWIDTH_NORMAL	Normal line width.
LINEWIDTH_THICK	Thick line width. This is double the normal width.

Typically, values equal or less than *1.0* are treated as LINEWIDTH_NORMAL and values greater than 1.0 as LINEWIDTH_THICK.

lGeomWidth

Geometric line thickness in world-coordinate space specified as an integer value. This is used only by GreStrokePath or when MPATH_STROKE is specified for GreModifyPath. A value of 0 (zero) results in the thinnest line possible regardless of the transform in force. Thick geometric lines are treated as polygons and are transformed accordingly.

usType

Specifies the cosmetic line type. Valid values are:

LINETYPE_DOT	Dotted
LINETYPE_SHORTDASH	Short-dashed
LINETYPE_DASHDOT	Dash, dot
LINETYPE_DOUBLEDOT	Double-dotte

d

LINETYPE_LONGDASH

Long-dashed

LINETYPE_DASHDOUBLEDOT

Dash, double-dot

LINETYPE_SOLID

Solid

LINETYPE_INVISIBLE

Invisible

LINETYPE_ALTERNATE

Everyalternatepelon

Note: LINETYPE_DEFAULT is the default value at the API. It is a reserved value and is not passed to the presentation driver.

usEnd

Valid values are:

	LINEEND_FLAT	Flat
	LINEEND_SQUARE	Square
	LINEEND_ROUND	Round

Note: LINEEND_DEFAULT is the default value at the API. It is a reserved value and is not passed to the presentation driver.

usJoin

Valid values are:

	LINEJOIN_BEVEL	Bevel
	LINEJOIN_ROUND	Round
	LINEJOIN_MITRE	Miter

Note: LINEJOIN_DEFAULT is the default value at the API. It is a reserved value and is not passed to the presentation driver.

When lines join at a very acute angle, and a mitered joint has been specified, the length of the miter line could extend to infinity. To prevent this, when the ratio of miter length to geometric line width exceeds 10:1, a bevel joint is drawn. The miter length is the distance between the inner and outer intersection points.

When a wide line is explicitly closed by a call to GreCloseFigure from within a path, the style at the closure point is JOIN style not END style. If enough points are given to implicitly close the figure, the END style is used at the closure points. Notice that the LINEJOIN attribute is only to be used at wide line ends when the figure has been closed by a call to GreCloseFigure.

Area (Pattern) Attributes

The device area (pattern) attributes are bundled in a DAREABUNDLE structure:

Parameter	Description
cAttr	Size of the logical area bundle.
cDefs	Size of the AREADEFS structure. This is 0 when no device area bundle exists.
abnd	AREABUNDLE structure. See text following *AREABUNDLE Mask:*.
adef	AREADEFS structure. See text following *AREABUNDLE Mask:*.

AREABUNDLE Mask

This mask is used in calls to GreDeviceSetAttributes to identify fields in the AREABUNDLE structure. Valid flags and the fields that they identify are:

Flag	Field
ABB_COLOR	lColor
ABB_BACK_COLOR	lBackColor
ABB_MIX_MODE	usMixMode
ABB_BACK_MIX_MODE	usBackMixMode
ABB_SET	usSet
ABB_SYMBOL	usSymbol

abnd The fields of an AREABUNDLE structure are:

lColor	Area foreground color.
lBackColor	Area background color.
usMixMode	Area foreground mix mode.
usBackMixMode	Area background mix mode.
usSet	Local identifier (lcid) for a logical font or a bit map. Valid values are:

0	Base pattern set.
Nonzero	Local identifier for the logical font or bit map defined by the cdef.defSet field in the area attributes bundle.

usSymbol Identity of the required pattern in the current pattern set or logical font. This attribute is ignored when the pattern set is a bit map. If the value is outside the range of the logical font, the standard default pattern is used.

Values in the range 1-255 are valid. The defined values are:

PATSYM_ERROR	Error.
PATSYM_DEFAULT	Default.
PATSYM_DENSE1	Solid shading with decreasing intensity.
PATSYM_DENSE2	Solid shading with decreasing intensity.
PATSYM_DENSE3	Solid shading with decreasing intensity.
PATSYM_DENSE4	Solid shading with decreasing intensity.
PATSYM_DENSE5	Solid shading with decreasing intensity.
PATSYM_DENSE6	Solid shading with decreasing intensity.
PATSYM_DENSE7	Solid shading with decreasing intensity.
PATSYM_DENSE8	Solid shading with decreasing intensity.
PATSYM_VERT	Vertical lines.
PATSYM_HORIZ	Horizontal lines.
PATSYM_DIAG1	Diagonal lines 1, bottom-left to top-right.
PATSYM_DIAG2	Diagonal lines 2, bottom-left to top-right.
PATSYM_DIAG3	Diagonal lines 1, top-left to bottom-right.
PATSYM_DIAG4	Diagonal lines 2, top-left to bottom-right.
PATSYM_NOSHADE	No shading.
PATSYM_SOLID	Solid shading.
PATSYM_BLANK	Blank (same as PATSYM_NOSHADE).
PATSYM_HALFTONE	Every alternate pel on. PATSYM_HALFTONE can be similar to PATSYM_DENSE4 and PATSYM_DENSE5 (solid patterns) but has a more stringent definition. It is useful for generating gray text.

See the *Presentation Manager Programming Reference* for definitions of these shading patterns.

ptlRefPoint

Specifies the pattern origin for areas and thick lines. The pattern is mapped into the area to be filled by conceptually replicating the pattern definition in both the horizontal and vertical directions.

The pattern reference point is subject to all of the transforms. When an area is moved by changing a transform and redrawing, the fill pattern also appears to move so as to retain its position relative to the area boundaries. This positioning capability allows part of a picture to be moved with a BitBlt operation and the remainder to be drawn by changing the appropriate transform with no discontinuity at the join.

The pattern reference point, which is specified in world coordinates, need not be inside the actual area to be filled and is not subject to clipping. However, the area to be filled is subject to clipping.

adef — The fields of an AREADEFS structure are:

defSet	Area definition. This can be a text pattern, predefined pattern, bit map, pointer to an engine font, device font handle, or bit map handle.
fFlags	The only valid flag is CDEF_GENERIC.
CodePage	Code Page ID.

Character Attributes

The device character attributes are bundled in a DCHARBUNDLE structure:

Parameter	Description
cAttr	Size of the attribute bundle.
cDefs	Size of the CHARDEFS structure.
cbnd	CHARBUNDLE structure. See text following ***CHARBUNDLE Mask:***.
cdef	CHARDEFS structure.

CHARBUNDLE Mask

This mask is used in calls to GreDeviceSetAttributes to identify fields in the CHARBUNDLE structure. Valid flags and the fields that they identify are:

Flag	Field
CBB_COLOR	lColor
CBB_BACK_COLOR	lBackColor
CBB_MIX_MODE	usMixMode
CBB_BACK_MIX_MODE	usBackMixMode
CBB_SET	usSet
CBB_MODE	usPrecision

CBB_BOX	sizfxCell
CBB_ANGLE	ptlAngle
CBB_SHEAR	ptlShear
CBB_DIRECTION	usDirection
CBB_TEXT_ALIGN	usTextAlign
CBB_EXTRA	fxExtra
CBB_BREAK_EXTRA	fxBreakExtra

cbnd The fields of a CHARBUNDLE structure are:

lColor — Character foreground color.

lBackColor — Character background color.

usMixMode — Character foreground mix mode.

usBackMixMode — Character background mix mode.

usSet — Specifies a local identifier (lcid) for a logical font. If usSet is 0, the current code page and character precision are used to resolve the selection of the base font. The code page (set by the function GreSetCodePage) identifies two base fonts: an outline font and an image font. The value of usPrecision determines which of these is selected. Valid values for usSet are:

0	Base font.
Nonzero	Local identifier for the logical font defined by the cdef.defSet field in the character attributes bundle.

usPrecision — Specifies the character mode. The value of usPrecision is used to select output quality from Precision 1 (the lowest) to Precision 3. Presentation drivers normally use Precision 3 except when performance is improved by using the specified precision. For example, the EGA and VGA drivers always use Precision 3 when the current font is an outline font, but switch to the specified precision for an image font.

Valid values for usPrecision are:

CM_MODE1	Precision 1. The selected font can be either an image or an outline font. When an image font is used, the first character is positioned with its reference point at the current position. Subsequent characters are positioned by using FONTMETRICS.
CM_MODE2	Precision 2. The selected font can be either an image or outline font. When an image font is used, the first character is positioned with its reference point at the current position. Subsequent characters are positioned by using the FONTMETRICS, CBB_BOX, CBB_ANGLE, and CBB_SHEAR attributes. This is done by constructing a transformation matrix that transforms the FONTMETRICS values sXDeviceRes and sYDeviceRes to the rotated, scaled, and sheared character box attributes, and translates the reference point of the first character to the current position. The actual character positions are then determined by applying this transform to the character positions found by using the FONTMETRICS.

Notice that the character box attribute is subjected to the rotation, scaling, and shear defined by the current transformations and the other character attributes.

Note: Shear attributes affect only the horizontal position of CM_MODE2 characters when the character direction is CHRDIRN_TOPBOTTOM or CHRDIRN_BOTTOMTOP.

CM_MODE3 — Precision 3. The selected font must be an outline font.

Note: CM_DEFAULT is the default value at the API. It is a reserved value and is not passed to the presentation driver.

For outline fonts, regardless of mode, all character attributes together with the FONTMETRICS are used for positioning, scaling, rotating, and shearing the characters. This is done by constructing a transformation matrix as described previously for CM_MODE2. The actual character vector stroke coordinates are then determined by applying this transform to the character-definition coordinates suitably modified (for character positioning) by the FONTMETRICS.

As with CM_MODE2, the character box attribute is subjected to the rotation, scaling, and shear defined by the current transformations and the other character attributes.

The character reference point is defined as the intersection of the base line and the left edge of the character. The baseline is defined as an offset, pCellOffset, from the top of the character cell.

sizfxCell — Specifies fixed-point numbers for the width and height of a character cell in world-coordinate space. This defines the background area for a character. Each dimension is represented as a signed four-byte integer with a notational binary point between bit 16 and bit 15. Therefore, +2.5 is represented by 00028000h and -2.5 is represented by FFFD8000h.

For CM_MODE1, the cell has no effect when characters are drawn from an image font. For CM_MODE2, the width determines the spacing of consecutive characters along the baseline. Both width and height can be positive, negative, or 0. When either parameter is negative, the spacing occurs in the direction opposite to normal and each character is drawn reflected in CM_MODE3. For example, a negative height in the standard direction in Mode 3 indicates that the characters are drawn upside down, and that the string is drawn below the baseline (assuming no other transformations cause inversion). A zero character width or height is also valid. The string of characters collapses into a line. If both are 0, the string is drawn as a single point in CM_MODE3.

ptlAngle — Specifies integer values X and Y for the coordinates of the end of a line starting at the origin (0, 0). The baseline for subsequent character strings is parallel to this line.

For CM_MODE1, the angle has no effect when characters are drawn.

For CM_MODE2, the angle is used to determine the position of each image character. However, the orientation of characters within the character box is inherent in their definitions. The characters are positioned so that the lower-left corners of the character definitions are placed at the lower-left corners of the character boxes.

For CM_MODE3, the angle is observed accurately and the character boxes are rotated to be normal to the character baseline. If the coordinate system is such that one X-axis unit is not physically equal to one Y-axis unit, a rotated character appears to be sheared.

ptlShear — Specifies integer values, which identify the end coordinates of a line originating at 0, 0. The vertical strokes in subsequent outline character strings are drawn parallel to the defined line. For CM_MODE1, the shear has no effect when image characters are drawn. For CM_MODE2, the shear affects the height of the character cell. Therefore, the position of characters drawn

with CDIRN_TOPBOTTOM or CDIRN_BOTTOMTOP. The top of the character box remains parallel to the character baseline.

If hx=0 and hy=1 (the standard default), upright characters result. If hx and hy are both positive or negative nonzero, the characters slope from bottom left to top right. If hx and hy are of opposite signs, the characters slope from top left to bottom right.

Note: The value for hy cannot be 0 (zero) because this implies an infinite shear.

No character inversion takes place as a result of shear alone. (Inversion can be done with the charCell attribute.)

usDirection

Valid values are:

CHDIRN_LEFTRIGHT	Left-to-right
CHDIRN_TOPBOTTOM	Top-to-bottom
CHDIRN_RIGHTLEFT	Right-to-left
CHDIRN_BOTTOMTOP	Bottom-to-top

Note: CHDIRN_DEFAULT is the default value at the API. It is a reserved value and is not passed to the presentation driver.

If the specified direction is not valid, the presentation driver uses CHDIRN_LEFTRIGHT as the default.

usTextAlign

Specifies the horizontal and vertical alignment of character strings. This alignment defines a reference point within the string, which is positioned on the starting point specified for the string.

The horizontal alignment values are as follows:

TA_NORMAL_HORIZ	Normal alignment. This is the initial default. The alignment assumed depends on the current character direction: CHDIRN_LEFTRIGHT Same as TA_LEFT CHDIRN_TOPBOTTOM Same

		Same as TA_CENTER
CHDIRN_RIGHTLEFT		Same as TA_RIGHT
		Same as TA_CENTER
CHDIRN_BOTTOMTOP		Same as TA_CENTER
	TA_LEFT	Left alignment. The string is aligned on the left edge of its leftmost character.
	TA_CENTER	Center alignment. The string is aligned on the arithmetic mean of Left and Right.
	TA_RIGHT	Right alignment. The string is aligned on the right edge of its rightmost

	character.
TA_STANDARD_HORIZ	Standard alignment. The alignment assumed depends on the current character direction: CHDIRN_LEFTRIGHT Same as TA_LEFT CHDIRN_TOPBOTTOM Same as TA_LEFT CHDIRN_RIGHTLEFT Same as TA_RIGHT CHDIRN_BOTTOMTOP Same as TA

The vertical alignment values are as follows:

TA_NORMAL_VERT	Normal alignment. This is the initial default. The alignment assumed depends on the current character direction: CHDIRN_LEFTRIGHT Same as TA_BASE CHDIRN_TOPBOTTOM Same as TA_TOP CHDIRN_RIGHTLEFT Same as TA_BASE CHDIRN_BOTTOMTOP Same as TA_BOTTOM
TA_TOP	Top alignment. The string is aligned on the top edge of its topmost character.
TA_HALF	Half alignment. The string is aligned on the arithmetic mean of Bottom and Top.
TA_BASE	Base alignment. The string is aligned on the base of its bottom character.
TA_BOTTOM	Bottom alignment. The string is aligned on the bottom edge of its bottom character.
TA_STANDARD	Standard alignment. The alignment assumed depends on the current character direction: CHDIRN_LEFTRIGHT Same as TA_BOTTOM CHDIRN_TOPBOTTOM Same as TA_TOP CHDIRN_RIGHTLEFT Same as TA_B

OTT
OM
CHDIRN_BOTTOMTOP
Same
as
TA_B
OTT
OM

The current position will also be modified relative to the current character direction as shown:

Horizontal

For horizontal character directions:

Horizontal Alignment

TA_LEFT

TA_CENTER

TA_RIGHT

Vertical Alignment

TA_TOP

TA_HALF

TA_BASE

TA_BOTTOM

Vertical

For vertical character directions:

Horizontal Alignment

TA_LEFT

TA_CENTER

TA_RIGHT

Vertical Alignment

TA_TOP	
TA_HALF	
TA_BASE	
TA_BOTTOM	

	fxExtra	A fixed point world-coordinate distance, which is added between every character as it is being placed.
	fxBreakExtra	A fixed point world-coordinate distance, which is added to the width of the break character as it is placed.

cdef The fields of a CHARDEFS structure are:

	defSet	Character set definition. If defSet is passed as 0, the presentation driver must use the default device font (zero is passed only when the driver provides and manages its own default font). Otherwise, the significance of defSet depends upon the state of the CDEF_GENERIC flag, as follows: • If the flag is set, defSet is a pointer to an engine font. • If not set, defSet is a device font identifier defined by the driver. When defSet is a pointer to an engine font, cdef is a pointer to an instance of the FOCAFONT data structure. The definition of the FOCAFONT data structure is included in the header file.
	fFlags	Valid flags are: CDEF_GENERIC — Engine font (not device font) CDEF_BOLD — Font must be made bold CDEF_ITALIC — Font must be made italic CDEF_UNDERLINE — Font must be underlined CDEF_STRIKEOUT — Font must be made overstrike
	CodePage	Current code page. The presentation driver ignores this field when the font is not a multicode page font that needs translating.

charSpacing	Character spacing.

See the *OS/2 Programming Guide* for examples of these attributes.

Image Attributes

The device image attributes are bundled in a DIMAGEBUNDLE structure:

Parameter	Description
cAttr	Size of the attributes structure.
cDefs	Set to 0. There is no IMAGEDEFS structure.
ibnd	IMAGEBUNDLE structure. See text following *IMAGEBUNDLE Mask:*

IMAGEBUNDLE Mask

This mask is used in calls to GreDeviceSetAttributes to identify fields in the IMAGEBUNDLE structure. Valid flags and the fields that they identify are:

Flag	Field
IBB_COLOR	lColor
IBB_BACK_COLOR	lBackColor
IBB_MIX_MODE	usMixMode
IBB_BACK_MIX_MODE	usBackMixMode

ibnd The fields of an IMAGEBUNDLE structure are:

lColor	Image foreground color
lBackColor	Image background color
usMixMode	Image foreground mix mode
usBackMixMode	Image background mix mode

Marker Attributes

The device marker attributes are bundled in a DMARKERBUNDLE structure:

Parameter	Description
cAttr	Size of MARKERBUNDLE structure.
cDefs	Size of MARKERDEFS structure.
mbnd	MARKERBUNDLE structure. See text following *MARKERBUNDLE Mask:*.
mdef	MARKERDEFS structure. See text following

MARKERBUNDLE Mask

This mask is used in calls to GreDeviceSetAttributes to identify fields in the MARKERBUNDLE structure. Valid flags and the fields that they identify are:

Flag	Field
MBB_COLOR	lColor
MBB_BACK_COLOR	lBackColor
MBB_MIX_MODE	usMixMode
MBB_BACK_MIX_MODE	usBackMixMode
MBB_SET	usSet
MBB_SYMBOL	usSymbol
MBB_BOX	sizfxCell

mbnd — The fields of a MARKERBUNDLE structure are:

lColor — Marker foreground color.

lBackColor — Marker background color.

usMixMode — Marker foreground mix mode.

usBackMixMode — Marker background mix mode.

usSet — Specifies a local identifier (lcid) for the logical font:

0	Base marker set
Nonzero	Local identifier for the font identified in the mdef.defSet field of the marker attributes bundle.

usSymbol — Specifies the identity of the required marker symbol in the current marker set. If the value of usSymbol does not identify a marker in the current font, the standard default for the font is used. The default for the default font is a cross. For a loaded font, it is the character identified by the usDefaultChar field of the FOCAMETRICS structure.

All values in the range 0-255 are valid. The defined values for the default marker set are:

MARKSYM_CROSS

MARKSYM_PLUS

MARKSYM_DIAMOND

MARKSYM_SQUARE

MARKSYM_SIXPOINTSTAR

MARKSYM_EIGHTPOINTSTAR

MARKSYM_SOLIDDIAMOND

MARKSYM_SOLIDSQUARE

		MARKSYM_DOT	
		MARKSYM_SMALLCIRCLE	
		MARKSYM_BLANK	
		Note: MARKSYM_DEFAULT is the default value at the API. It is a reserved value and is not passed to the presentation driver.	
	sizfxCell	Specifies fixed-point numbers for the width and height of a marker cell in world-coordinate space. This defines the background area for a marker. Each dimension is represented as a signed four-byte integer with a notational binary point between bit 16 and bit 15. Therefore, +2.5 is represented by 00028000h, and -2.5 is represented by FFFD8000h. The value of this attribute only affects the size of markers drawn with an outline (vector) font or marker set. Markers drawn from image (raster) sets are not affected.	
mdef	The fields of a MARKERDEFS structure are:		
	defSet	Marker set definition. If this value is passed as 0, the presentation driver must use the default marker set. If the CDEF_GENERIC flag is set, this is a pointer to an engine font. Otherwise, it is a device-font identifier defined by the presentation driver.	
	fFlags	Valid flags are:	
		CDEF_GENERIC	Engine font (not device font)
		CDEF_BOLD	Marker must be emboldened
		CDEF_ITALIC	Marker must be italicized
		CDEF_UNDERLINE	Marker must be underlined
		CDEF_STRIKEOUT	Marker must be struck out
	CodePage	Code page number.	

Bit-Map Functions

Presentation drivers for hardcopy vector devices can return Failure on all bit-map operations. The same bit-map file format is used for bit maps, icons, and pointers. For details, refer to the *Presentation Manager Programming Reference* .

Color Functions

By default, the color mode for a DC is set to index mode, and the DC has a Logical Color Table set to the values given in the following list. When in index mode, these defaults are always considered to be part of the color table unless they are explicitly overwritten by GreCreateLogColorTable.

Note: Presentation drivers that support less than 16 colors must map Value 0 (CLR_BACKGROUND) through Value 15 (CLR_PALEGRAY) to device colors. If GreQueryColorData is called while the default color table is the current color table, the presentation driver returns the device colors.

Default values for the Logical Color Table are:

CLR_FALSE	(-5). All color planes or bits, or both, are FALSE.
CLR_TRUE	(-4). All color planes or bits, or both, are TRUE.
CLR_DEFAULT	(-3). This is the API default. It is a reserved value and is not passed to the presentation driver.
CLR_WHITE	(-2). This index is never loaded explicitly. It always produces white when the default table is in force or when the index is set to RGB. When, with a realized color table and an index that is not RGB, this option is unavailable, it produces CLR_BACKGROUND.
CLR_BLACK	(-1). This index is never loaded explicitly. It always produces black when the default table is in force or when the index is set to RGB. When, with a realized color table and an index that is not RGB, this option is unavailable, it produces CLR_NEUTRAL.
CLR_BACKGROUND	(0). This is the natural background color of the device. For a hardcopy device, it is the paper color and for a display device it is the default window color, SYSCLR_WINDOW.
CLR_BLUE	(1)
CLR_RED	(2)
CLR_PINK	(3)
CLR_GREEN	(4)
CLR_CYAN	(5)
CLR_YELLOW	(6)
CLR_NEUTRAL	(7) This is a device-dependent contrasting color. For a display device, it is the default window text color, SYSCLR_WINDOWTEXT.
CLR_DARKGRAY	(8)
CLR_DARKBLUE	(9)
CLR_DARKRED	(10)
CLR_DARKPINK	(11)
CLR_DARKGREEN	(12)
CLR_DARKCYAN	(13)
CLR_BROWN	(14)
CLR_PALEGRAY	(15)

Colors with indexes greater than 15 are device-dependent defaults, which must be defined by the presentation driver. The effective range of the color table, which includes the default color table, is -5 through MaxIndex. Color indexes outside this range that have not been loaded are not used by applications because these colors cannot be guaranteed.

Where physically possible, the default colors are always available on a device. For devices that support more than 16 colors, requested colors can be mapped to colors other than the defaults (when they exist). Such colors cannot be guaranteed to be similar for different devices. They can be different for other releases of applications and presentation drivers. Applications that depend on precise colors beyond the defaults must query the available colors (see GreQueryRealColors) and, when necessary, realize their own color tables (see GreRealizeColorTable).

Support for Monochrome Devices

Presentation drivers for monochrome devices must be able to draw pictures intended for color devices. A simple solution for hardcopy drivers is to map as follows:

- Background color to paper color.
- Foreground color to printer foreground except when:
  - In RGB mode. If the foreground RGB matches the default background RGB, use paper color.
  - In index mode. If the RGB foreground index matches the RGB color for Index 0, use paper color.

To map the RGB colors to the device, the presentation driver must first establish the reset color, which is the base color for the device. The reset color can be one of the following:

- Paper color for a hardcopy device with no loaded color table
- SYSCLR_WINDOW for a monochrome display with no loaded color table
- CLR_BACKGROUND for any monochrome device that has a loaded color table

Any color that is not the reset color is considered to be the contrast color. The values for the reset color and contrast color are either 000000h or FFFFFFh. When the reset color is 000000h, the contrast color is FFFFFFh. CLR_TRUE, CLR_FALSE, and CLR_DEFAULT are always honored independently of the reset color. The interpretation of CLR_BLACK and CLR_WHITE depends on the reset color.

When GreQueryNearestColor is called for a monochrome device, the value returned is either the reset color or the contrast color. GreErasePS causes the color to be set to the value of the reset color. GreBitblt can also be used to transfer a color bit map to a monochrome device or bit map. In this case, the source image background color becomes the reset color and all other pels are represented by the contrast color. More sophisticated presentation drivers for monochrome devices should use half-toning for colors to provide more usable output. Half-toning can be applied to all graphic primitives.

GreEscape

The GreEscape handling routine in the presentation driver supports the DevEscape function and its escape codes at the API. While the primary function of GreEscape is to implement the required support for the defined escape codes, it can be used to implement additional escape codes. There is a set of defined ranges for additional escape codes. The range chosen determines how the operating system processes the escape code when it is received as a parameter to DevEscape. On entry to GreEscape, the value of lEscape on the stack identifies the escape code. The action taken is determined by the escape code and the physical device that the presentation driver supports.

Support

GreEscape is called by DevEscape. GreEscape with the escape code, DEVESC_QUERYESCSUPPORT, must be supported by all presentation drivers. Hardcopy drivers also support the DEVESC_STARTDOC, DEVESC_ABORTDOC, DEVESC_NEXTFRAME, and DEVESC_ENDDOC escape codes. The other escape codes are optional. See Defined Escape Codes and the individual escape codes that follow.

Stack Frame

On entry to the GreEscape routine, the stack frame contains:

Parameter	Data Type	Description
hdc	HDC	Device context handle.
lEscape	LONG	Escape code.
cInCount	LONG	Number of bytes pointed to by pInData.
pInData	PBYTE	Pointer to input data structure.
pcOutCount	PLONG	Pointer to the number of bytes in output data structure. If the escape code is one that returns data in the output

		data structure, the handling routine changes the value addressed by pcOutCount to show the number of bytes returned.
pOutData	PLONG	Pointer to output data structure.
pInstance	PVOID	Pointer to instance data.
lFunction	ULONG	High-order WORD=flags; Low-order WORD=NGreEscape.

Return Codes

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

Possible Errors Detected

When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

- PMERR_DEV_FUNC_NOT_INSTALLED
- PMERR_INV_LENGTH_OR_COUNT

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further information.

Defined Escape Codes

The following list shows the escape codes that have been defined for Presentation Manager and the devices to which they apply. GreEscape returns DEVESC_NOTIMPLEMENTED for escape codes that it does not support.

Ranges from DEV_DBE_FIRST (24450) to DEV_DBE_LAST (24455) are reserved for DBCS support and used by DBCS display driver.

DEVESC_ABORTDOC	(Hardcopy drivers only)
DEVESC_ACQUIREFB	(software motion video)
DEVESC_BREAK_EXTRA	(Hardcopy drivers only)
DEVESC_CHAR_EXTRA	(Hardcopy drivers only)
DEVESC_DBE_FONTMANAGEMENT	(DBCS support)
DEVESC_DEACQUIREFB	(software motion video)
DEVESC_DRAFTMODE	(Hardcopy drivers only)
DEVESC_ENDDOC	(Hardcopy drivers only)
DEVESC_EXTGET	(EnDIVE support)
DEVESC_EXTPUT	(EnDIVE support)

DEVESC_EXTQUERY	(EnDIVE support)
DEVESC_FLUSHOUTPUT	(Hardcopy drivers only)
DEVESC_GETAPERTURE	(EnDIVE support)
DEVESC_GETCP	(Hardcopy drivers only)
DEVESC_GETJOBID	(Hardcopy drivers only)
DEVESC_GETSCALINGFACTOR	(Hardcopy drivers only)
DEVESC_HWREQUEST	(EnDIVE support)
DEVESC_NEWFRAME	(Hardcopy drivers only)
DEVESC_NEXTBAND	(Hardcopy drivers only)
DEVESC_QUERYESCSUPPORT	(All drivers)
DEVESC_QUERYFB	(EnDIVE support)
DEVESC_QUERYVIOCELLSIZES	(Display drivers only)
DEVESC_RAWDATA	(Hardcopy drivers only)
DEVESC_REGISTER	(EnDIVE support)
DEVESC_SETMODE	(Hardcopy drivers only)
DEVESC_SETUPBLITTERNOTIFY	(EnDIVE support)
DEVESC_STARTDOC	(Hardcopy drivers only)
DEVESC_STD_JOURNAL	(Hardcopy drivers only)
DEVESC_SWITCHBANK	(software motion video)
DEVESC_VIDEOHIDDEN	(EnDIVE support)
DEVESC_VRAMALLOC	(EnDIVE support)

Ranges for Additional Escape Codes

The following table indicates the defined ranges for additional escape codes and shows how the operating system processes the escape codes when they are received as parameters to DevEscape:

32768-40959	Escape is not metafiled or recorded by the spooler. The escape is passed to the presentation driver in all cases.
40960-49151	Escape is metafiled but is not recorded by the spooler. For an OD_QUEUED device with PM_Q_STD data and for all device types other than OD_METAFILE, the escape is passed to the presentation driver.
49152-57343	Escape is metafiled and recorded by the spooler. For an OD_METAFILE device or for OD_QUEUED with PM_Q_STD data, the escape is not passed to the presentation driver.
57344-65535	Escape is recorded by the spooler. The escape is passed to the presentation driver except when the DC is an OD_QUEUED device with PM_Q_STD data.

Line Functions

The style of a line determines whether it is drawn solid, alternating, invisible, or as any one of a combination of dots and dashes. This style is determined by the usType parameter in the LINEBUND structure (see Line Attributes), which can have any one of the ten values shown in the following table. For each usType value, there is an associated 8-bit style mask whose bits form a template that corresponds to whether pels are set ON or OFF when the line is drawn on the device.

The 8-bit style masks used by OS/2 are as follows:

8-Bit Style Masks

usType	Value	StyleMask	Bits	Comment
LINETYPE_DEFAULT	0	0xFF	11111111	Solid line.
LINETYPE_DOT	1	0xAA	10101010	
LINETYPE_SHORTDASH	2	0xCC	11001100	
LINETYPE_DASHDOT	3	0xE4	11100100	
LINETYPE_DOUBLEDOT	4	0xA0	10100000	
LINETYPE_LONGDASH	5	0xE7	11100111	
LINETYPE_DASHDOUBLEDOT	6	0xEA	11101010	
LINETYPE_SOLID	7	0xFF	11111111	
LINETYPE_INVISIBLE	8	0x00	00000000	
LINETYPE_ALTERNATE	9	0xAA	10101010	

The style masks can be put into an array of bytes as illustrated in the following code example:

```
const BYTE bStyleMask[ ] =
{
 0xFF,0xAA,0xCC,0xE4,0xA0,0xE7,0xEA,0xFF,0x00,0xAA
};
```

Consider the following code example and comments:

```
DrawHorizontalLine(
  SHORT             sY,             /* Y-value               */
  SHORT             sX0,            /* Starting x-value      */
  SHORT             sX1,            /* Ending x-value        */
  USHORT            usStyleRatio,   /* Style ratio           */
  USHORT            usState         /* Current mask state    */
  PLINEBUNDLE       pLbnd)          /* pLbnd contains usType */
                                    /*   parameter           */
{
  USHORT            usStateOld;     /* Used for test         */
  USHORT            usOrigMask;     /* Original mask         */
  USHORT            usMask;         /* Current mask          */
```

The usState WORD structure is set up by the graphics engine. This structure is shown in the following table:

5 bits (not used)	3 bits (mask position, value 0-7)	8 bits (error-term byte, value 0-0xFF)

Note: This format is different from the way the graphics engine stores this value. The graphics engine keeps the high and low bytes swapped,

therefore, the calling routine must swap them before calling this function.

The mask position has three bits and eight possible values, 0-7, that correspond to the number of bits in the style mask. Notice that when the error-term byte overflows (exceeds 0xFF), it will increment the mask position value. To get the unshifted mask value:

```
usMask = usOrigMask = bStyleMasks[pLbnd -> usType];
```

The mask must be shifted to the current position as shown above:

```
usMask <<= ((usState & 0x700) >> 8);
```

For each pel:

```
for(sX = sX0; sX <= sX1; sX++){
```

Because many devices have small pel sizes, it is often necessary to set more than one pel ON per corresponding bit in the style mask. This setting is accomplished by using a style ratio, which determines how many pels on the line correspond to a bit in the style mask. As an example, for each pel on the line, the most significant bit of the style mask is consulted. If this bit is 1, the pel is drawn. If this bit is 0, the pel is not drawn, as shown in the following example:

```
if(usMask & 0x80)
   DrawPel(sX,sY);

usStateOld = usState;
```

The style ratio is then added to the error-term byte of the usState parameter. When the error-term byte overflows, the mask position value (high byte of usState) is incremented to the next position. If the mask position exceeds 7, it is reset to 1.0:

```
usState = (usState + usStyleRatio) & 0x7ff;
```

The mask itself must now be shifted appropriately:

```
if(HIBYTE(usState) | = HIBYTE(usStateOld)){     /* Has it changed?   */
   if(HIBYTE(usState) == 0)                    /* Back to zero?     */
      usMask - usOrigMask;                     /* Yes, reset it     */
   else
      usMask <<= 1;                            /* Else shift it     */
}
.
.
.
return(usState);                               /* Return the state
                                                  for the next line */
```

Styled line information is usually maintained by the presentation driver in the Device Context (DC) instance data structure. See Device Context. Remember that the DC is a structure that the presentation driver creates. The definition of the DC data structure is specific and can be unique for each unique presentation driver. This information can include:

```
typedef struct _DC {
.
.
.
USHORT       usStyleRatioX;    /* X-style ratio                  */
```

```
USHORT       usStyleRatioY;     /* Y-style ratio                    */
ULONG        lLineStyle;        /* Current state information        */
                                /*   (GRE Backward Format)          */
LINEBUNDLE   lbnd;              /* Linebundle information contains  */
                                /*   usType parameter               */
BYTE         bMyCurrentMask;    /* Current style mask               */
.
.
.
 DC;
typedef DC *PDC;
```

The lLineStyle value has much of the same information as the usState parameter in the example function. Its value is concurrently maintained by the graphics engine but has a different format from the example above. The graphics engine can set or query this value by calling the functions GreSetLineOrigin or GreGetLineOrigin, respectively. It is therefore reasonable to keep a copy in the graphics engine format.

The graphics engine's format for this value consists of three values:

- A flag indicating whether or not to draw the first pel of the current line
- The current mask position (explained earlier)
- The error-term byte (explained earlier)

They are stored in the following manner:

High WORD	16-bits. Draw first pel flag. Can be 0x0000 (do not draw first pel) or 0x0001 (draw first pel).	
Low WORD	16 bits, consisting of:	
	8 bits	Error-term byte
	5 bits	GRE internal flags
	3 bits	Mask position

If two bytes of this low WORD are compared to the WORD in the previous example, they are swapped.

Style Ratio

The most obvious way to generate styled lines is to draw a circle at the origin, and then draw a dashed line from the origin to all points on the circle. The number of dashes in each line will be the same and the lengths of all the dashes drawn will be equal. This is executed on microcode on some devices. However, it is too costly in terms of processor time to implement it in software. Therefore, an alternative method called the "maximum metric" is used in the graphics engine. This method can be visualized by drawing a square centered at the origin, and then drawing a dashed line to any point on the square. The number of dashes in each line will be the same but the lengths of the dashes will vary.

The maximum metric states that:

- If the line is Y-major as viewed on the device:

```
{y-major = ABS(y1-y0) > ABS(x1-x0)
where (x0, y0) and (x1, y1) are the endpoints of the line in
device-coordinate space},
add pDC -> usStyleRatioY to error-term value upon each
increment of Y as the line is drawn
```

- If the line is X-major as viewed on the device:

```
{x-major = ABS(x1-x0) > ABS(y1-y0) in device-coordinate space},
add pDC -> usStyleRatioX to error-term value upon each increment
of X as line is drawn
```

The line style as viewed on the device is determined, as follows:

```
if (ABS(pDC -> usStyleRatioX * sDeltaX) > ABS(pDC -> usStyleRatioY * sDeltaY))
    LineStyleAsItLooksOnTheDevice = X-MAJOR;
else
    LineStyleAsItLooksOnTheDevice = Y-MAJOR;
```

This might be coded similar to the following:

```
if (ABS(pDC -> usStyleRatioX * sDeltaX) > ABS(pDC -> usStyleRatioY * sDeltaY)) {
    usChangeInStateForOnePixel = pDC -> usStyleRatioX; fAddThisWhen = X_INCREMENTS;
} else {
        usChangeInStateForOnePixel = pDC -> usStyleRatioY; fAddThisWhen = Y_INCREMENTS;
}
```

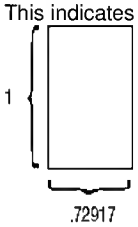
It follows that for a line (AB), the total change in style state can be expressed as:

```
usChangeInStateForWholeLine = MAX(ABS(pDC -> usStyleRatioX * sDeltaX),
                                  ABS(pDC -> usStyleRatioY * sDeltaY))
where sDeltaX = Bx-Ax and sDeltaY = By-Ay
```

For example, an EGA device that has a 640 x 350 (X-to-Y) resolution is displayed on a display that has an X-to-Y ratio of 1-to-.75. To calculate the aspect ratio:

```
X/Y Ratio = 350/(640*.75) = .72917   Therefore: X = Y*.72917 or Y = X/.72917
```

This indicates that a pel is taller than it is wide. The following figure visually describes the ratio.



Pel Aspect Ratio

Assume that four pels in the X-direction is the desirable size of a styled-line dot. Because this display is 9.5 inches across and there are 640 pels across, the length of the four pels is:

```
(9.5inches/640pels) * 4pels = .059375inches
This results in a pDC -> usStyleRatioX = 64&:
                                        64 = 256/4
```

Note that 256 = 0x100, which corresponds to an overflow of the error-term byte into the mask position. To get the equivalent pDC usStyleRatioY value, use the desired distance and multiply it by Y pels per inch, as follows:

```
(.059375inches * 350 pels)/7.125inches = 2.917pels
Therefore:pDC   usStyleRatioY = 256/2.917 = 87.76 (rounded to 88)
```

An easier method is to calculate pDC usStyleRatioY from pDC usStyleRatioX, using the following aspect ratio:

```
pDC   usStyleRatioY = pDC   usStyleRatioX / .72917 = 87.76 (rounded to 88)
```

Note that the values pDC   usStyleRatioX and pDC   usStyleRatioY are the same as those returned by GreGetStyleRatio.

An example of what is meant by "as viewed on the device" follows:

If a line is drawn from (0, 0) to (100, 100) pels on the device, it is drawn as a diagonal line but does not look diagonal to an observer. Instead, it looks like a line drawn to (100, 73).

This change of perspective is because of the aspect ratio. Each unit (pel) in the X-direction covers only .72917 as much distance as a pel in the Y-direction. The images on the device do not look skewed because this ratio is factored in when the application draws a diagonal line (for example, by drawing from (0, 0) to (73, 100)).

The styled lines are affected when a line that is drawn in pels as X-major appears on the device as Y-major. In this case, a line drawn from (0, 0) to (85, 64) is X-major as drawn in pels (because ABS(x1-x0) > ABS(y1-y0)), but appears on the device to be Y-major. Notice that this line must be styled Y-major to look correct on the device, as follows:

```
usChangeInStateForOnePixel = ABS(pDC -> usStyleRatioX * sDeltaX) > ABS(pDC -> usStyleRatioY
                             * sDeltaY)?
                             pDC -> usStyleRatioX :pDC -> usStyleRatioY;
where:
     ABS(pDC -> usStyleRatioX * sDeltaX) = 64 * 85 = 5440
     ABS(pDC -> usStyleRatioY * sDeltaY) = 88 * 64 = 5632
```

Because 5440 < 5632, the line is styled Y-major by using pDC   usStyleRatioY. The X-major diagonal line drawing routine usually adds pDC   usStyleRatioX to the error-term byte for every increment of X. This is correct if the aspect ratio is 1:1.

However, because the aspect ratio is not 1:1, pDC usStyleRatioY must be used in this case for every increment of Y, although the line is drawn as an X-major line with the X-major routine. This means that regardless of which way the line is drawn, it must be styled according to how it looks on the device, which is determined by the maximum metric method described previously.

LINETYPE_ALTERNATE is a special case of styled line. When drawing a line of this type (pLbnd   usType = LINETYPE_ALTERNATE), the X and Y style ratios are temporarily set to 256 to set every other pel on the line ON. Notice that changing the values returned by GreGetStyleRatio is not necessary because the graphics engine does not call this function if the line type is LINETYPE_ALTERNATE.

PolyShortLines and Styling

The graphics engine determines how to style the PolyShortLine, and either sets the PSL_YMAJOR bit of the style field or clears it to 0. Therefore:

```
if(psl   usStyle & PSL_YMAJOR) {

Style it Y-major by adding pDC   usStyleRatioY to the error-term value upon each increment
of y as it is drawn.
}
else {

Style it X-major by adding pDC   usStyleRatioX to the error-term value upon each increment
of x as it is drawn.
}
```

First and Last Pel Considerations

The presentation driver ensures that a series of line, arc, and fillet orders all join up correctly including the ON/OFF counts defined by the current line attributes. For example, when drawing connected lines (PolyLines), the handling routine must not draw the first pel of the second and subsequent lines. Typically, the presentation driver maintains a flag in the DC instance data structure to indicate whether the first pel of a line is to be drawn. This flag is set by GreSetCurrentPosition and cleared by any subsequent drawing primitive. To ensure that a figure is closed correctly, GreCloseFigure does not draw the last pel in the closure line.

Some orders are defined as move type operations. A move causes three things to happen:

- The line style sequence is reset.
- The next line, arc, fillet, or partial arc primitive is drawn with first and last pel (subject to the line style sequence).
- In an area, if the current figure is not closed (that is, the current device coordinate position is not the same as the start device coordinate position), an implicit closure line is drawn to close it.

Subsequent start line, arc, fillet, and partial arc primitives are drawn to include the last but not the first pel (subject to the line style sequence). Any closed figure (full arc, box, or pie slice drawn with a boundary), is drawn with its boundary complete (no missing pels) and with the line-pattern sequence honored around all the parts of its boundary. Such closed figures are not considered to be move type operations and, therefore, allow construction of complex area boundaries.

Move type operations are:

- GreSetCurrentPosition.
- Any GreSet*xxx* function, such as GreSetModelTransform or GreSetWindow/ViewportTransform, that changes or might change the transform from world-coordinate space to device coordinates.
- Any GreSet*xxx* function, such as GreSetViewingLimits, which changes or might change the current clipping.

A different set of rules is necessary to construct a boundary for scan-line area filling. For example a rule might be "ignore line style and draw all lines solid with first pel OFF, last pel ON" (see GreGetLineOrigin). This boundary is different from the boundary that is drawn on the screen after the interior is filled. Functions such as GrePolyLine, GreArc, and GrePolyFillet that are preceded by a move operation are drawn with the first pel ON and the last pel set OFF.

Mandatory Functions for All Drivers by Category

Related mandatory functions for all presentation drivers can be grouped together into the following categories:

Attribute Functions

- GreDeviceGetAttributes
- GreDeviceSetAttributes
- GreDeviceSetGlobalAttribute
- GreGetPairKerningTable

Bit-Map Functions

- GreBitblt
- GreDeviceCreateBitmap
- GreDeviceDeleteBitmap
- GreDeviceSelectBitmap
- GreDrawBits
- GreDrawBorder
- GreGetBitmapBits
- GreGetPel
- GreImageData
- GreSetBitmapBits
- GreSetPel

Color Table Functions

- GreCreateLogColorTable
- GreQueryColorData
- GreQueryColorIndex
- GreQueryLogColorTable
- GreQueryNearestColor
- GreQueryRealColors
- GreQueryRGBColor
- GreRealizeColorTable
- GreUnrealizeColorTable

Device Functions 2

- GreDeviceQueryFontAttributes
- GreDeviceQueryFonts
- GreErasePS
- GreNotifyClipChange
- GreNotifyTransformChange
- GreRealizeFont

Device Functions 3

- GreAccumulateBounds
- GreDeviceSetDCOrigin
- GreGetBoundsData
- GreGetCodePage
- GreGetDCOrigin
- GreGetLineOrigin
- GreLockDevice
- GreResetBounds
- GreSetCodePage
- GreSetLineOrigin
- GreUnlockDevice

GreEscape Functions

- GreEscape DEVESC_ABORTDOC
- GreEscape DEVESC_ACQUIREFB
- GreEscape DEVESC_BREAK_EXTRA
- GreEscape DEVESC_CHAR_EXTRA
- GreEscape DEVESC_DBE_FONTMANAGEMENT
- GreEscape DEVESC_DEACQUIREFB
- GreEscape DEVESC_DRAFTMODE
- GreEscape DEVESC_ENDDOC
- GreEscape DEVESC_EXTGET
- GreEscape DEVESC_EXTPUT
- GreEscape DEVESC_EXTQUERY
- GreEscape DEVESC_FLUSHOUTPUT
- GreEscape DEVESC_GETAPERTURE
- GreEscape DEVESC_GETCP
- GreEscape DEVESC_GETJOBID
- GreEscape DEVESC_GETSCALINGFACTOR
- GreEscape DEVESC_HWREQUEST
- GreEscape DEVESC_NEWFRAME
- GreEscape DEVESC_NEXTBAND
- GreEscape DEVESC_QUERYESCSUPPORT
- GreEscape DEVESC_QUERYFB
- GreEscape DEVESC_QUERYVIOCELLSIZES
- GreEscape DEVESC_RAWDATA
- GreEscape DEVESC_REGISTER
- GreEscape DEVESC_SETMODE
- GreEscape DEVESC_SETUPBLITTERNOTIFY
- GreEscape DEVESC_STARTDOC
- GreEscape DEVESC_STD_JOURNAL
- GreEscape DEVESC_SWITCHBANK
- GreEscape DEVESC_VIDEOHIDDEN
- GreEscape DEVESC_VRAMALLOC

Line Functions

- GreDisjointLines
- GreDrawLinesInPath
- GreGetCurrentPosition
- GrePolyLine
- GrePolyScanline
- GrePolyShortLine
- GreSetCurrentPosition

Marker Function

- GrePolyMarker

Query Functions

- GreQueryDeviceBitmaps
- GreQueryDeviceCaps
- GreQueryDevResource
- GreQueryDeviceCaps

Text Functions

- GreCharString
- GreCharStringPos
- GreQueryCharPositions
- GreQueryTextBox
- GreQueryWidthTable

For a complete description of each Mandatory function, see Mandatory and Simulated Graphics Engine Function Reference. The functions are listed alphabetically in the index table for that chapter.

Mandatory Functions for Display Drivers

This section describes the functions that must be supported by the display driver for the primary screen. Although these functions are not required for hardcopy drivers, hardcopy drivers should provide a default routine to handle the functions described in this section, and at FillLogicalDeviceBlock time, put a pointer to the default routine in the relevant entries in the dispatch table. The default routine will post a warning PMERR_DEV_FUNC_NOT_INSTALLED and return BOOLEAN TRUE.

Descriptions of these mandatory functions for display drivers are provided. The functions are grouped according to the *conditional include* sections of the header file, as follows:

- AVIO functions (INCL_AVIOP, INCL_GRE_DEVMISC1)
- Bit-map functions (INCL_GRE_BITMAPS)
- Device functions 2 (INCL_GRE_DEVMISC2)
- Device functions 3 (INCL_GRE_DEVMISC3)
- Miscellaneous screen functions (INCL_GRE_PICK, INCL_WINPOINTERS)

Each description shows what the handling routine is expected to do, the parameters passed to the routine, and the values that the routine returns.

AVIO Functions

Advanced VIO (AVIO) functions are used to display characters. These functions must be supported for Display DCs. Hardcopy drivers do not support AVIO functions. When writing to an AVIO presentation space, an application must ensure that windows containing alphanumeric data are device-cell-aligned, where appropriate. The presentation driver can determine whether any characters that are not cell-aligned are visible. Most column, row, length, width, and height values correspond to cells within the presentation space Logical Video Buffer (LVB), whose origin is assumed to be at the bottom-left corner of the buffer (0,0).

The presentation driver is expected to clip alphanumeric data to the DC region. This is performed in the same way as it is for normal graphics, by enumerating the rectangles using GreGetClipRects and clipping each line to a single rectangle. Although the presentation driver is neither expected to test for correlation hits nor to accumulate GPI_BOUNDS, it should accumulate USER_BOUNDS for AVIO functions if the COM_ALT_BOUNDS command flag is set.

The VIO presentation space is passed to the display driver as a pointer to a VioPresentationSpace structure. The display driver uses this structure to extract the current state data to allow it to update the display. The VioPresentationSpace structure is defined as:

PresentationSpaceLock	Lock (not used by the presentation driver).
pLVB	Pointer to the logical video buffer (LVB). The LVB is a two-dimensional array of character cells and is assumed to begin at offset 0 within the segment. The presentation driver must not alter the contents of this field.
pBVSCB	Not used by the presentation driver.
rgfAVio	Not used by the presentation driver.
CellByteSize	Size in bytes of a cell in the LVB. This value is either 2 or 4. The presentation

	driver must not alter the contents of this field.
BufferRowCount	Number of cell rows in the logical video buffer. The presentation driver must not alter the contents of this field.
BufferColumnCount	Number of cell columns in the logical video buffer. The presentation driver must not alter the contents of this field.
WindowOriginRow	Row index for the logical video buffer. This field, together with the parameter WindowOriginColumn, indicates the logical video buffer cell that is drawn in the bottom left of the window's client area. The presentation driver must not alter the contents of this field.
WindowOriginColumn	Column index for logical video buffer (see WindowOriginRow). The presentation driver must not alter the contents of this field.
TextCursorRow	Row coordinate of flashing text cursor relative to the logical video buffer. The presentation driver must not alter the contents of this field.
TextCursorColumn	Column coordinate of flashing text cursor relative to the logical video buffer. The presentation driver must not alter the contents of this field.
TextCursorStartLine	First scan line of a character-cell image overlaid by the text cursor. Lines in the cell image are numbered from top to bottom. The first line is 0. The presentation driver must not alter the contents of this field.
TextCursorEndLine	Last scan line of a character-cell image, overlaid by the text cursor. The presentation driver must not alter the contents of this field.
TextCursorWidth	Width of text cursor in pels. The presentation driver must not alter the contents of this field.
TextCursorVisible	Indicates whether the cursor is visible (nonzero) or invisible (zero). The presentation driver must not alter the contents of this field.
CellImageHeight	Height of character cell in pels. When the value passed is 0 or invalid, the presentation driver should reset it to the device default value.
CellImageWidth	Width of character cell in pels. When the value passed is 0 or invalid, the presentation driver should reset it to the device default value.
CodePageID	ID of current code page for this presentation space. When the value passed is 0 or invalid, the presentation driver should reset it to the device default value.
WindowHeight	Not used by the presentation driver.
WindowWidth	Not used by the presentation driver.
hConsoleDisplayContext	Device context handle associated with the presentation space. The presentation driver must not alter the contents of this field.
hVioWindow	Not used by the display driver.
RowOrgLatch	See "ColOrgLatch".
ColOrgLatch	Window-origin coordinates (WindowOriginRow and WindowOriginColumn) saved on completion of the last call to GreUpdateCursor. RowOrgLatch and ColOrgLatch are used by the display driver to record the state of the currently displayed cursor. They are interrogated by GreUpdateCursor to determine whether the cursor has moved.
CursorRow	See " CursorCol".
CursorCol	Cursor coordinates (TextCursorRow and TextCursorColumn) saved on completion of the last call to GreUpdateCursor. These fields are used by the display driver to record the state of the currently displayed cursor so that it can be successfully erased.
CursorStartLine	See "CursorEndLine".
CursorEndLine	Cursor start and end lines (TextCursorStartLine and TextCursorEndLine) saved on completion of the last call to GreUpdateCursor. These fields are used by the display driver to record the state of the currently displayed cursor so that it can be successfully erased.
CursorWidth	Cursor width (TextCursorWidth) saved on completion of the last call to

GreUpdateCursor. This field is used by the display driver to record the state of the currently displayed cursor so that it can be successfully erased.

PartialCellAdjust

This is a negative value reflecting the partial cell height that must be below the bottom of the window to ensure that a complete cell is positioned at the top of the window (actual window height = WindowHeight rounded up to next complete character cell - PartialCellAdjust). The presentation driver must not alter the contents of this field.

XLatch

See "YLatch".

YLatch

Pel coordinates of the bottom-left corner of the cursor rectangle relative to the bottom-left corner of the window.

WidthLatch

See "HeightLatch".

HeightLatch

The height and width in pels of the cursor saved on completion of the last call to GreUpdateCursor. These are taken to be CellImageHeight and the difference between TextCursorStartLine and TextCursorEndLine, taking account of any wrapping. The width and height latches are used by GreUpdateCursor to record the screen region corresponding to an exclusive-OR cursor.

CellHeightLatch

The height in pels of the character cell for which the cursor was last drawn. This parameter is used to detect cell height changes. These values were saved on completion of the last call to GreUpdateCursor.

rgfShieldStates

Flags:

CursorShowing (0x0001)	Cursor is visible on the screen. This flag is maintained by the display driver.
fHasTheFocus (0x0002)	This window has the input focus. This flag is not modified by the display driver.
fCursorIsOn (0x0004)	Set to indicate that this is the ON phase of the blink cycle. The cursor should be invisible during the OFF phase of the blink cycle. This flag is not modified by the display driver.

pFontsLoaded[3]

Array of pointers for AVIO fonts 1, 2, and 3.

pMapFontsLoaded[3]

Array of pointers to code page maps for AVIO Fonts 1, 2, and 3.

FormatID

Presentation-space format. This identifies the format of the attribute bytes in the LVB, as follows:

0H	CGA
0H	Extended
70H	World-wide format

The presentation driver must not alter the value of this field.

lpNLSExt	Pointer to a National Language Support (NLS) extension structure where Double-Byte Character Set (DBCS) environment vectors are set for AVIO Fonts 1, 2, and 3. If the display driver is required to display DBCS, it must maintain the array of DBCSEvInfox4x, instead of DBCSEvInfox0x. When the display driver provides NLS, it must set the DBCS vectors for lcid 1-3.

Each character cell is contained in a two-byte or four-byte array in the LVB. The format of the character-cell array is:

Code Point	Position of the character in the code table.	
CGA Attribute Byte	Character attributes. The four low-order bits represent the foreground color and the high-order bits represent the character background color. Each 4-bit color value corresponds to an explicit 24-bit RGB value. The RGB values are defined within the graphics engine and match the colors available on a CGA device.	
Extended Attribute Byte	Applies only to four-byte cells. It is defined as:	
	Bit 7	Underscore.
	Bit 6	Reverse video.
	Bit 4	Background transparency. When set, the background is transparent. When not set, the background is opaque.
	Bits 1-0	Character Set 0, 1, 2, or 3.
Extended Attribute Byte	(World-wide format). Applies only to four-byte cells. It is defined as:	
	Bit 7	Underscore.
	Bit 6	Reverse video.
	Bit 4	Background transparency. When set, the background is transparent. When not set, the background is opaque.
	Bit 3	Left vertical grid.
	Bit 4	Top horizontal grid.
	Bits 1-0	Character Set 0, 1, 2, or 3.
Spare Attribute Byte	Applies only to four-byte character cells.	
	It is reserved for the system.	
Spare Attribute Byte	(World-wide format). Applies only to four-byte character cells.	
	Bit 7	DBCS trailing byte maintained by the operating system.
	Bits 6-1	Reserved for the system.
	Bit 0	DBCS byte maintained by the operating system.
	Note that applications are not allowed to set bit 7 and bit 0.	

Mandatory Functions (for Display Drivers) by Category

Related mandatory functions for display drivers can be grouped together into the following categories:

AVIO Functions

- GreCharRect
- GreCharStr

- GreDeviceSetAVIOFont
- GreScrollRect
- GreUpdateCursor

Bit-Map Functions

- GreDeviceSetCursor
- GreRestoreScreenBits
- GreSaveScreenBits

Device Functions 2

- GreDeviceInvalidateVisRegion
- GreGetStyleRatio
- GreSetStyleRatio

Device Functions 3

- GreDeath
- GreResurrection

Miscellaneous Screen Functions

- GreGetPickWindow
- GreSetColorCursor
- GreSetPickWindow

For a complete description of each Mandatory function, see Mandatory and Simulated Graphics Engine Function Reference. The functions are listed alphabetically in the index table for that chapter.

Direct Interface Video Extensions (DIVE) Functions

The DIVE architecture provides a device-independent interface for direct access to display memory through DEVESC_ functions. Using the DIVE architecture, the OS/2 software motion video decompression subsystem writes directly to video memory. The subsystem performs window clipping, color translation and scaling required to write the data in a window. On bank-switched configurations, the software must detect when video memory bank boundaries are crossed, then switch the bank accordingly.

DIVE Display Engine Functional Characteristics

The OS/2 Version 3 DIVE Display Engine is a single, system-level API that enables efficient transfer of data to video memory with clipping, scaling and color translation. The Display Engine exports the following functions:

- Query for display capabilities
- Open instance
- Visible region specification
- Allocation of image data buffers
- Buffer to screen/buffer-to-buffer transfer (blitter) setup
- 8-bit CLUT color palette simulation/specification
- Transfer buffer-to-image display/transfer buffer-to-buffer
- Utillity functions useful for direct access, including window starting address calculation
- Frame buffer acquire/deacquire
- VRAM bank selection (for bank-switched displays)
- Close instance

Enhanced Direct Interface Video Extensions (EnDIVE)

The DIVE architecture has been enhanced in OS/2 Version 3 to support motion video accelerator hardware. This enhanced architecture is called EnDIVE. It provides additional DEVESC_ functions for querying information about the color formats supported by the hardware, video memory management and image processing.

The Display Engine detects and uses acceleration hardware (via EnDIVE) and substitutes software algorithms whenever necessary.

The DIVE and EnDIVE DEVESC_ functions are listed below:

- GreEscape DEVESC_ACQUIREFB
- GreEscape DEVESC_DEACQUIREFB
- GreEscape DEVESC_EXTGET
- GreEscape DEVESC_EXTPUT
- GreEscape DEVESC_EXTQUERY
- GreEscape DEVESC_GETAPERTURE
- GreEscape DEVESC_HWREQUEST
- GreEscape DEVESC_QUERYFB
- GreEscape DEVESC_REGISTER
- GreEscape DEVESC_SETUPBLITTERNOTIFY
- GreEscape DEVESC_SWITCHBANK
- GreEscape DEVESC_VIDEOHIDDEN
- GreEscape DEVESC_VRAMALLOC

For detailed information about the DIVE architecture, refer to the *OS/2 Multimedia Application Programming Guide* and to the individual functions included in Mandatory and Simulated Graphics Engine Function Reference. The functions are listed alphabetically in the index table for that chapter.

Simulated Functions

These functions are simulated by handling routines in the graphics engine and are called through pointers in the default dispatch table. Any simulated functions can be "hooked" to improve performance or to exploit special features of the device. Hooking is performed by overwriting the pointer in the presentation driver's copy of the dispatch table with a pointer to the presentation driver's handling routine. If this is done, the original pointer must be saved in order to pass calls to the engine's handling routine.

When the presentation driver has hooked a function, all calls to that function are passed through the dispatch table directly to the driver's handling routine. If the presentation driver cannot completely handle a hooked function, it can pass the call to the engine's routine for completion.

The functions in this section are grouped according to the conditional include sections of the header file:

- Arc functions (INCL_GRE_ARCS)
- Area and Path functions (INCL_GRE_PATHS)
- Clip functions (INCL_GRE_CLIP)
- Line functions (INCL_GRE_LINE)
- Palette Manager functions (INCL_GRE_PALETTE)
- Region functions (INCL_GRE_REGIONS)
- Transform functions (INCL_GRE_XFORMS)

Each description defines what the handling routine is expected to do, the parameters passed to the routine, and the values that the routine returns.

Arc Functions

Drawing functions pass individual drawing orders to the graphics engine. The graphics engine then draws, correlates, and takes, or takes bounds on, the drawing primitives as directed by the flags. The graphics engine is assumed to clip to the appropriate part of the window, which is the region excluding any window border or frills.

Coordinates are passed as signed 32-bit numbers in a logical space called "world-coordinate space". Angles are also passed as signed 32-bit numbers. Zero refers to the direction of the positive X-axis; 2(31st power) represents 360˚. Positive values are counterclockwise from the positive X-axis.

Area and Path Functions

A "path" is an area or a shape that can be used to define:

- Wide lines and curves to which changes of scale can be applied
- Shapes and areas for filling
- Irregular shapes to which subsequent primitives are clipped. This is known as a "clip path".

Clip Functions

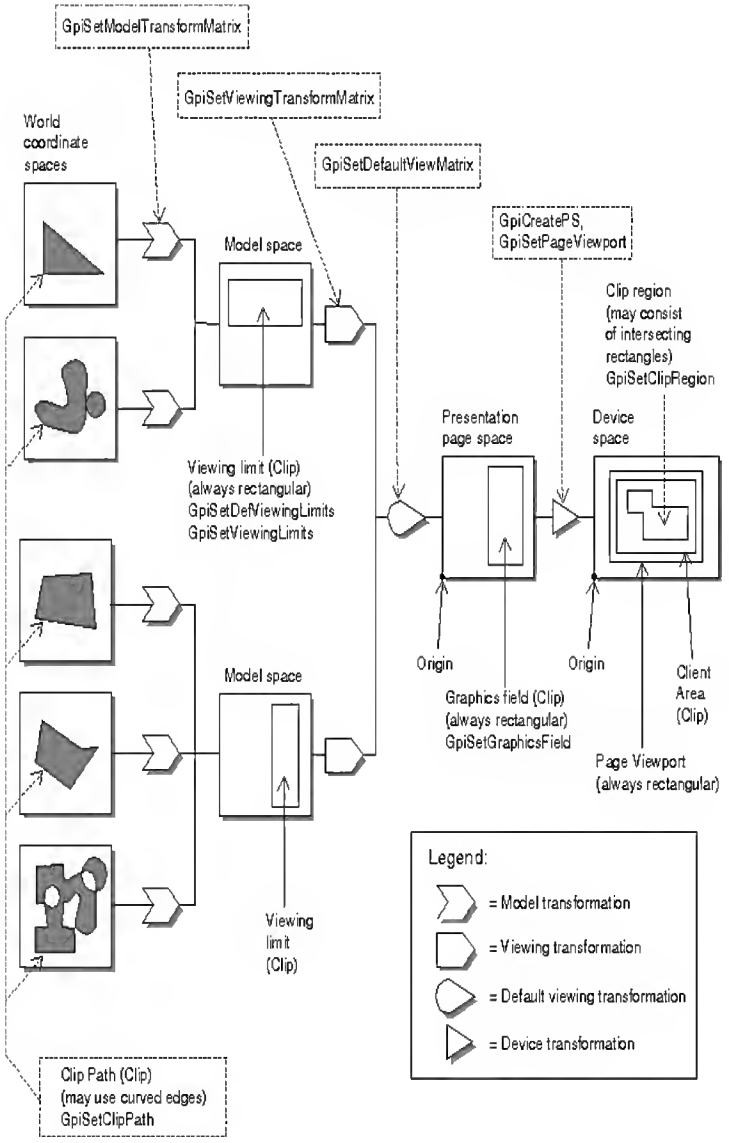
"Clip regions" are defined as rectangles in world coordinates. The boundaries of a clip region rectangle are inclusive of the rectangle they define.

Region Functions

"Regions" are defined as rectangles in device coordinates. They are inclusive at the bottom-left boundary and exclusive at the top-right boundary. That is, the top-right coordinates are outside the rectangle they define and the bottom-left coordinates are inside the rectangle. When both coordinate pairs are equal, the rectangle dimension is 0.

Transform Functions

"Transform functions" provide a complete viewing pipeline whereby coordinates are transformed from world-coordinates to model space to presentation page to device space. The following figure diagrams the viewing pipeline.



Matrix Element Format

The matrix elements for the model and viewing transforms are held in XFORM structures:

fxM11
fxM12
fxM21
fxM22
lM41
lM42

M11, M12, M21, and M22 are fixed-point numbers represented as signed four-byte integers with a notational binary point between bits 16 and 15:

```
+2.5 is represented by 00028000H
-2.5 is represented by FFFD8000H
-0.5 is represented by FFFF8000H
```

M41 and M42 are signed four-byte numbers.

Device Transform Definition by Presentation Page Viewport

For a transform, defined by viewport and window rectangles whose bottom-left and top-right coordinates are represented by (X1, Y1), (X2, Y2), (X3, Y3), and (X4, Y4), respectively, the matrix elements are determined as shown in the following example. The point (X3-1/2, Y3-1/2) transforms to (X1-1/2, Y1-1/2), and the point (X4+1/2, Y4+1/2) transforms to (X2+1/2, Y2+1/2). Therefore:

```
M12 = 0
M21 = 0

If X4 >= X3 then
M11 = (X2-X1+1) / (X4-X3+1)
M41 = (X1*X4-X3*X2+1/2 * (X2-X4 + X1-X3)) / (X4-X3+1)

If X4 < X3 then
M11 = (X2-X1+1) / (X4-X3-1)
M41 = (X1*X4-X3*X2-1/2 * (X2+X4+X1+X3)) / (X4-X3-1)

If Y4 >= Y3 then
M22 = (Y2-Y1+1) / (Y4-Y3+1)
M42 = (Y1*Y4-Y3*Y2+1/2 * (Y2-Y4 + Y1-Y3)) / (Y4-Y3+1)

If Y4 < Y3 then
M22 = (Y2-Y1+1) / (Y4-Y3-1)
M42 = (Y1*Y4-Y3*Y2-1/2 * (Y2+Y4+Y1+Y3)) / (Y4-Y3-1)
```

Note: X4 is always greater than X3 and Y4 is always greater than Y3.

In the case of device transforms, (X3, Y3) is always (0, 0), Y4 is always greater than Y3, and the device space coordinates (X2, Y2) are exclusive. This simplifies the formula to:

```
M12 = 0
M21 = 0

M11 = (X2-X1) / (X4+1)
M41 = (X1*X4+1/2 * (X2-1-X4+X1)) / (X4+1) = X1+1/2 (M11-1)

M22 = (Y2-Y1) / (Y4+1)
M42 = (Y1*Y4+1/2 * (Y2-1-Y4+Y1)) / (Y4+1) = Y1+1/2 (M22-1)
```

Bounds, Correlation, and Clipping

All presentation drivers must support bounds computation for both GPI bounds (COM_BOUND) and user bounds (COM_ALT_BOUND). Bounds are calculated on unclipped primitives for all operations that draw to the device, including AVIO functions. GPI bounds are passed in model space coordinates. User bounds are calculated in device-coordinate space. To prevent inaccuracies from occurring when the transform changes, the typical presentation driver maintains bounds in both coordinate sets in its instance data structure. It then accumulates the transforms as they occur.

Correlation is performed in page-coordinate space on the output of primitives that have been clipped only to the viewing limits and graphics field. Correlation is also performed on all operations that draw to the device, except the AVIO function. Note that hardcopy drivers are not required to perform correlation.

Simulated Functions by Category

Related simulated functions can be grouped together into the following categories:

Arc Functions

- GreArc
- GreBoxBoth
- GreBoxBoundary
- GreBoxInterior
- GreFullArcBoth
- GreFullArcBoundary
- GreFullArcInterior
- GreGetArcParameters
- GrePartialArc
- GrePolyFillet
- GrePolyFilletSharp
- GrePolySpline
- GreSetArcParameters

Area and Path Functions

- GreAreaSetAttributes
- GreBeginArea
- GreBeginPath
- GreCloseFigure
- GreEndArea
- GreEndPath
- GreFillPath
- GreModifyPath
- GreOutlinePath
- GreRestorePath
- GreSavePath
- GreSelectClipPath
- GreStrokePath

Clip Functions

- GreCopyClipRegion
- GreExcludeClipRectangle
- GreGetClipBox
- GreGetClipRects
- GreIntersectClipRectangle
- GreOffsetClipRegion
- GrePtVisible
- GreQueryClipRegion
- GreRectVisible
- GreRegionSelectBitmap
- GreRestoreRegion
- GreSaveRegion
- GreSelectClipRegion
- GreSelectPathRegion
- GreSetupDC
- GreSetXformRect

Line Functions

- GreDrawRLE
- GrePolygonSet

Palette Manager Functions

- GreDeviceAnimatePalette
- GreDeviceCreatePalette
- GreDeviceDeletePalette
- GreDeviceResizePalette
- GreDeviceSetPaletteEntries
- GreQueryHWPaletteInfo
- GreQueryPaletteRealization
- GreRealizePalette
- GreUpdateColors

Region Functions

- GreCombineRectRegion
- GreCombineRegion
- GreCombineShortLineRegion
- GreCreateRectRegion
- GreDestroyRegion
- GreEqualRegion
- GreGetRegionBox
- GreGetRegionRects
- GreOffsetRegion
- GrePaintRegion
- GrePtInRegion
- GreRectInRegion
- GreSetRectRegion

Transform Functions

- GreConvert
- GreGetGlobalViewingXform
- GreGetGraphicsField
- GreGetModelXform
- GreGetPageUnits
- GreGetPageViewport
- GreGetViewingLimits
- GreGetWindowViewportXform
- GreMultiplyXforms
- GreRestoreXform
- GreRestoreXformData

- GreSaveXform
- GreSaveXformData
- GreSetGlobalViewingXform
- GreSetGraphicsField
- GreSetModelXform
- GreSetPageUnits
- GreSetPageViewport
- GreSetViewingLimits
- GreSetWindowViewportXform

For a complete description of each Simulated function, see Mandatory and Simulated Graphics Engine Function Reference. The functions are listed alphabetically in the index table for that chapter.

Palette Manager Functions

The hardware (HW) palette, or device physical palette, is divided into two parts:

- Device default colors, which are the entries used by nonpalette manager DCs
- Palette Manager palette colors

At startup, the HW palette is assigned the device default colors because there are no Palette Manager DCs at this time. When Palette Manager palettes are created and subsequently selected and realized, their palette colors must be allocated slots in the HW palette. Therefore, the proportion of the HW palette allocated to the default colors is reduced to produce free slots, which then can be allocated to the Palette Manager palettes. If the number of Palette Manager slots is reduced, the number of slots allocated to the default colors is increased to improve the appearance of the non-Palette Manager DCs.

Non-Palette Manager DCs can only use the default colors found in the device HW palette. There are five possible sizes of device default colors: 256-, 128-, 64-, 32-, and 16-entries. The 256-entry default palette is the same as the standard 8514 8bpp palette. The 16-entry default palette is the same as VGA palette. For OD_MEMORY DCs, the 256-entry device default palette is always used. For OD_DIRECT DCs, the device HW palette can be changed to permit allocation of HW palette slots to Palette Manager DCs.

Within the HW palette, the device default colors are always positioned symmetrically in two halves at the top and bottom of the HW palette. The slots in the center are used for Palette Manager slots. Therefore, when mixing the device default colors, the result is always another device default color. The colors within the device default palette are arranged so that the inverse mixes always give the expected results.

Accessing the Device Default Palette

The device default palette is accessed through two pointers, DirectDeviceDefaultPalette and MemoryDeviceDefaultPalette, depending on whether an OD_DIRECT or OD_MEMORY DC is in use. The size of the DirectDeviceDefaultPalette is always kept in ulDirectDeviceDefaultPaletteSize.

The direct device default palette is changed by the ReduceDefaultPaletteSize and IncreaseDefaultPaletteSize functions. These functions change the ulDirectDeviceDefaultPaletteSize, DirectDeviceDefaultPalette, and DefaultDirectLogicalColorTable variables, and update the entries in the DirectSpecialColorTable.

Accessing the Default Logical Color Tables

There are five possible default logical color tables, which correspond to the five device default palette sets of colors. Each of these color tables contains the same RGB entries. However, they have different indexes into the HW palette. The indexes provide the nearest colors available when each of the device default color sets are loaded into the HW palette.

The default logical color tables are accessed through two pointers, DefaultDirectLogicalColorTable and DefaultMemoryLogicalColorTable, depending on whether an OD_DIRECT or OD_MEMORY DC is in use. There are also two copies of the special color table, which hold system colors and other special colors. These table copies, DirectSpecialColorTable and MemorySpecialColorTable, correspond to OD_DIRECT and OD_MEMORY DCs. Unlike the default logical color tables and the device default palette, these tables are changed by altering their entries rather than by pointing to different versions of the tables.

The special color tables should be updated by the display driver when the system colors change. All of the default logical color tables are updated immediately when the system colors are changed using CLR_BACKGROUND and CLR_NEUTRAL.

Updating the Logical Color Tables

To keep the logical color tables updated in all of the DCs, a field in each DC's instance data points to the device default palette that was current for that DC when the physical indexes into the palette were last calculated. When the EnterDriver function is called, it must check to see if this palette is the current device default palette for that DC. If it is not, EnterDriver updates the logical color table and the current device default palette field.

If the logical color table is one of the five defaults, it can be updated by changing it to point to the current default logical color table. If the logical color table is not one of the defaults, each index has to be recalculated using the current device default palette, which must calculate the nearest default physical index.

Palette Manager Function List

These functions are grouped according to the "conditional include" sections of the header file, **Palette Manager functions (INCL_GRE_PALETTE)**. Each description shows what the handling routine is expected to do, the parameters passed to the routine, and the values that the routine returns.

The Palette Manager functions and their page locations are listed below:

- GreDeviceAnimatePalette
- GreDeviceCreatePalette
- GreDeviceDeletePalette
- GreDeviceResizePalette
- GreDeviceSetPaletteEntries
- GreQueryHWPaletteInfo
- GreQueryPaletteRealization
- GreRealizePalette
- GreUpdateColors

For a complete description of each Palette Manager function, see Mandatory and Simulated Graphics Engine Function Reference. The functions are listed alphabetically in the index table for that chapter.

Software Motion Video Support

The software motion video component of the Multimedia Presentation Manager/2 must update bit maps very quickly. The system must be able to play movie files at 15 frames per second or faster, with each frame having a bit map size of 320 x 240 pels or larger.

GPI calls such as GpiBitBlt cannot meet these objectives. A set of DevEscape calls are provided so that the software motion video component can directly update the Video Random Access Memory (VRAM) of packed-pel display adapters.

Most of the movie frames are *delta frame*s , containing only those pels that have changed from the previously displayed frame. By directly updating VRAM, the software motion video component only needs to selectively update a small number of pels in a typical 320 x 240 pel image. This selective updating allows better performance than a GPI call, which requires redrawing of the entire bit map and makes a heavy call/return overhead when updating small runs of pels.

Software motion video DevEscape functions are provided to:

- Query the attributes of the aperture (DEVESC_GETAPERTURE)
- Switch the bank address (DEVESC_SWITCHBANK)
- Reserve a video adapter (DEVESC_ACQUIREFB)
- Release a video adapter (DEVESC_DEACQUIREFB)

Architecture Objectives

These DevEscape calls allow the Multimedia Presentation Manager/2 software motion video component to efficiently display rapidly changing pels in real time.

By allowing this efficient, device-independent access technique, there is sufficient processor power available for improved software video decompression, which results in the following benefits:

- Software motion video that scales video quality according to available system resources
- Improved video quality

Architecture Definition

These software motion video calls let MMPM/2 determine the exact location and size of memory-mapped VRAM. For video adapters with bank-switched VRAM, efficient, device-independent means are provided to safely gain access to the VRAM resources, manipulate the bank switch mechanism, and release the adapter. This architecture follows the same rationale as WIN-OS/2 se*aml*ess *window*s , using the same mechanisms to ensure reliable sharing of the device.

Description of Functions

Four DevEscape functions in the XGA and SVGA Presentation Manager device drivers are implemented. They provide the ability to allow calling routines to share access to memory-mapped VRAM.

For a complete description of each software motion video function, see Mandatory and Simulated Graphics Engine Function Reference. The functions are listed alphabetically in the index table for that chapter.

Enhanced Direct Interface Video Extensions (EnDIVE)

The Direct Interface Video Extensions (DIVE) architecture has been enhanced in OS/2 Version 3. The enhanced architecture is called EnDIVE and it supports the use of motion video accelerator hardware. Additional DEVESC_ functions are available for querying information about the color formats supported by the hardware, video memory management and image processing.

The additional DEVESC_ functions are provided to:

- Copy an image from the screen to system memory or to off-screen video memory in a compressed format (DEVESC_EXTGET)
- Copy an image to the screen (DEVESC_EXTPUT)
- Acquire information about the motion video accelerator (DEVESC_EXTQUERY)
- Take advantage of enhanced acquire functions and check on video memory buffer location (DEVESC_HWREQUEST)
- Acquire information about the frame buffer (DEVESC_QUERYFB)
- Register and unregister EnDIVE use (DEVESC_REGISTER)
- Allocate off-screen video memory (DEVESC_VRAMALLOC)
- Indicate if the hardware can support the current image blitting parameters (DEVESC_SETUPBLITTERNOTIFY)
- Inform the driver that video is hidden (DEVESC_VIDEOHIDDEN)

For a complete description of each EnDIVE function, see Mandatory and Simulated Graphics Engine Function Reference. The functions are listed alphabetically in the index table for that chapter.

Graphics Engine Internal Functions

The handling routines for these functions are in the graphics engine. Because they are not called through the dispatch table, they cannot be hooked by the presentation driver. Descriptions of these internal functions are provided. The functions are grouped according to the conditional include sections of the header file.

Each description defines what the function does, the parameters passed to the engine, and the values that the function returns.

Graphics Engine Functions by Category

Related graphics engine functions can be grouped together into the following categories:

Device Context Functions (INCL_GRE_DCS)

- GreCloseDC
- GreGetHandle
- GreGetProcessControl
- GreOpenDC
- GreQueryEngineVersion
- GreResetDC
- GreRestoreDC
- GreSaveDC
- GreSetHandle
- GreSetProcessControl

Device Support Functions (INCL_GRE_DEVSUPPORT)

- GreCreateBitmap
- GreDeleteBitmap
- GreGetAttributes
- GreGetBitmapDimension
- GreGetBitmapParameters
- GreGetDefaultArcParameters
- GreGetDefaultAttributes
- GreGetDefaultViewingLimits
- GreInitializeAttributes
- GreSelectBitmap
- GreSetAttributes
- GreSetBitmapDimension
- GreSetDefaultArcParameters
- GreSetDefaultAttributes
- GreSetDefaultViewingLimits
- GreSetGlobalAttribute

Font Functions (INCL_GRE_FONTS)

- GreCreateLogicalFont
- GreLoadFont
- GreQueryCodePageVector
- GreQueryFontAttributes
- GreQueryFontFileDescriptions
- GreQueryFonts
- GreQueryLogicalFont
- GreUnLoadFont

Journal Functions (Hardcopy drivers only) (INCL_GRE_JOURNALI

- GreCreateJournalFile
- GreDeleteJournalFile
- GreOpenJournalFile
- GrePlayJournalFile
- GreStartJournalFile
- GreStopJournalFile

LCID Functions (INCL_GRE_LCID)

- GreCopyDCLoadData
- GreQueryBitmapHandle
- GreSetBitmapID

Set ID Functions (INCL_GRE_SETID)

- GreDeleteSetId
- GreQueryNumberSetIds
- GreQuerySetIds

For a complete description of each Graphics Engine function, see Mandatory and Simulated Graphics Engine Function Reference. The functions are listed alphabetically in the index table for that chapter.

Font Functions

When Presentation Manager is initialized, the engine is responsible for establishing the default image and outline fonts, and the default pattern and marker sets.

The presentation driver can supply device fonts as default fonts. If this is done, the driver can request the graphics engine to manage the device fonts by setting the CAPS_FONT_OUTLINE_MANAGE or CAPS_FONT_IMAGE_MANAGE flag in the array of device capabilities (see GreQueryDeviceCaps). When these flags are set, the engine performs any mapping and transforms that might be necessary. To use a default font belonging to the presentation driver, the font handle is passed to the engine in response to GreQueryDevResource.

Font Matching

When requested by an application, the mapping of a font to a loaded font that is physically available is done by the graphics engine. A physical font can be one that is built into a particular device or a public font loaded from a .FON file when Presentation Manager is initialized. The engine matches fonts by checking the values for szFacename and IMatch (in the font metrics structures) against those passed to it in GreCreateLogicalFont. If no match is found, the engine proceeds as if the match number was passed with a value of 0. When a value of 0 is passed for either szFaceName or IMatch, the engine ignores that parameter and considers it to match anything. If no face name is specified, the engine supplies a default font from the presentation driver (see GreQueryDevResource), or if none is available, maps the requested font to one of its own default fonts.

When a face name is specified and the match number is 0, the engine checks to see if an image font was requested and, if so, searches for a

font of the specified dimensions. If this search fails, the engine then attempts to match the font to an outline font with the face name and selection flags requested. When the match number is 0 and the required font is an outline font, the engine searches for an outline font. Should all these searches fail, a default font is used. See GreRealizeFont.

Journal Functions (Hardcopy Drivers Only)

Journal functions allow presentation drivers to optimize the processing of data for raster devices such as dot-matrix or laser printers, which print bands of data. The size of a band is determined by the type of device and the amount of memory the driver can use to build its bit map. As an example, a color laser printer might need the full 24 bits per pel. In this case, several bands might be needed to make a page. A simple dot-matrix printer that uses one bit per pel could treat the whole page as a single band. Refer to Banding for a description of how a presentation driver would use journaling functions to perform the technique of banding.

The hardcopy driver creates a journal file when the DC is enabled and starts recording in response to GreEscape (DEVESC_STARTDOC). The graphics engine stores the Gre*xxx* functions in the journal file as they are passed to the DC until told to stop recording by the hardcopy driver.

If the presentation driver passes the bit flag JNL_DRAW_OPTIMIZATION when it calls GreCreateJournalFile, the hardcopy driver processes the calls to produce the first band while the journal file is being accumulated. Otherwise, the COM_DRAW command bit flag is turned OFF and the hardcopy driver is, in effect, told not to draw while the journal file is being accumulated.

When the journal file is complete, if the JNL_DRAW_OPTIMIZATION bit flag was set ON when the function GreCreateJournalFile was called, the hardcopy driver passes the completed band to the spooler or hardcopy device. It then plays the journal file and reprocesses the calls to produce the next band. Otherwise, the hardcopy driver plays the journal file and reprocesses the calls to produce all bands, including the first.

System Functions

This section describes system functions that are available to presentation drivers:

- GetDriverInfo
- SetDriverInfo
- SSAllocMem
- SSFreeMem
- VisRegionNotify
- WinSetErrorInfo

For a complete description of each System function, see Mandatory and Simulated Graphics Engine Function Reference. The functions are listed alphabetically in the index table for that chapter.

Exported Driver Function Reference

This chapter describes the entry points that are exported by the dynamic link library of a presentation driver:

```
EXPORTS
   OS2_PM_DRV_RING_LEVELS      /* All drivers - Optional  */
   OS2_PM_DRV_ENABLE_LEVELS    /* All drivers - Optional  */
   OS2_PM_DRV_ENABLE           /* All drivers - Mandatory */
```

The functions in this chapter are listed in the following index table. A detailed description of each function follows the table.

Function	Type
DrvInstall	GRE hardcopy
DrvRemove	GRE hardcopy
OS2_PM_DRV_DEVICENAMES	GRE hardcopy

OS2_PM_DRV_DEVMODE	GRE hardcopy
FillLogicalDeviceBlock	exported entry point
FillPhysicalDeviceBlock	exported entry point
DisablePhysicalDeviceBlock	exported entry point
EnableDeviceContext	exported entry point
DisableDeviceContext	exported entry point
SaveDCState	exported entry point
RestoreDCState	exported entry point
ResetDCState	exported entry point
CompleteOpenDC	exported entry point
BeginCloseDC	exported entry point
InstallPMEngineDisplayHooks	exported entry point
QueryDeviceSurface	exported entry point
OS2_PM_DRV_QUERYSCREENRESOLUTIONS	display driver

DrvInstall

DrvInstall - Syntax

NOTE: This entry point is optional.

Description:

DrvInstall informs the hardcopy driver that it is about to be installed or reinstalled. The driver is given the opportunity to update data in the INI file.

It is the responsibility of the caller to install the complete multifile driver by using extended attributes.

```
#include <os2.h>

DrvInstall();
```

DrvInstall - Return Value

None.

DrvInstall - Parameters

None.

DrvInstall - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

DrvRemove

DrvRemove - Syntax

Description:

DrvRemove informs the hardcopy driver that it is about to be removed from the system (deleted from the hard disk).

```
#include <os2.h>

DrvRemove();
```

DrvRemove - Return Value

None.

DrvRemove - Parameters

None.

DrvRemove - Remarks

The driver is given the opportunity to remove data it owns from the INI file. The hardcopy driver does not use this entry point to delete any of its own data files unless they are created after installation.

It is the responsibility of the caller to remove the complete multifile driver by using extended attributes.

DrvRemove - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

OS2_PM_DRV_DEVICENAMES

OS2_PM_DRV_DEVICENAMES - Syntax

Description:

OS2_PM_DRV_DEVICENAMES must be compiled to run at Ring 3 (privilege level 3).

The device names entry point is exported as OS2_PM_DRV_DEVICENAMES by the presentation driver to support the DevQueryDeviceNames function at the API level. This call returns the device names and device descriptions supported by this driver.

```
#include <os2.h>

PSZ       pszDriverName;
PLONG     pcNames;
PSTR32    paDeviceName;
PSTR64    paDeviceDesc;
PLONG     pcDataTypes;
PSZ       paDataType;
ULONG     lReserved1;
ULONG     lReserved2;
LONG      rc;             /*  Return codes. */

rc = OS2_PM_DRV_DEVICENAMES(pszDriverName,
       pcNames, paDeviceName, paDeviceDesc,
       pcDataTypes, paDataType, lReserved1,
       lReserved2);
```

OS2_PM_DRV_DEVICENAMES Parameter - pszDriverName

pszDriverName (PSZ) - input
A pointer to a string containing the name of the device driver, such as LaserJet.

OS2_PM_DRV_DEVICENAMES Parameter - pcNames

pcNames (PLONG) - input
A pointer to count of fields, cNames, in DeviceName and DeviceDesc arrays.

OS2_PM_DRV_DEVICENAMES Parameter - paDeviceName

paDeviceName (PSTR32) - input
A pointer to DeviceName array, CHAR cNames[32]. Device names are NULL-terminated strings such as 'HP LaserJet II'.

OS2_PM_DRV_DEVICENAMES Parameter - paDeviceDesc

paDeviceDesc (PSTR64) - input
A pointer to DeviceDesc array, CHAR cNames[64]. Device descriptions are NULL-terminated strings such as 'HP LaserJet II'.

OS2_PM_DRV_DEVICENAMES Parameter - pcDataTypes

pcDataTypes (PLONG) - input
A pointer to count of fields, cDataTypes in DataType array.

OS2_PM_DRV_DEVICENAMES Parameter - paDataType

paDataType (PSZ) - input
A pointer to DataType array, CHAR cDataTypes[16].

OS2_PM_DRV_DEVICENAMES Parameter - lReserved1

lReserved1 (ULONG) - input

Reserved.

OS2_PM_DRV_DEVICENAMES Parameter - lReserved2

lReserved2 (ULONG) - input
Reserved.

OS2_PM_DRV_DEVICENAMES Return Value - rc

rc (LONG) - returns
Return codes.

The handling routine in the presentation driver returns a LONG integer. Valid values are:

-1	Successful
0	Error

Note: The system expects the successful and error return codes from OS2_PM_DRV_DEVICENAMES to be the opposite of those from OS2_PM_DRV_DEVMODE and the Enable subfunctions.

OS2_PM_DRV_DEVICENAMES - Parameters

pszDriverName (PSZ) - input
A pointer to a string containing the name of the device driver, such as LaserJet.

pcNames (PLONG) - input
A pointer to count of fields, cNames, in DeviceName and DeviceDesc arrays.

paDeviceName (PSTR32) - input
A pointer to DeviceName array, CHAR cNames[32]. Device names are NULL-terminated strings such as 'HP LaserJet II'.

paDeviceDesc (PSTR64) - input
A pointer to DeviceDesc array, CHAR cNames[64]. Device descriptions are NULL-terminated strings such as 'HP LaserJet II'.

pcDataTypes (PLONG) - input
A pointer to count of fields, cDataTypes in DataType array.

paDataType (PSZ) - input
A pointer to DataType array, CHAR cDataTypes[16].

lReserved1 (ULONG) - input
Reserved.

lReserved2 (ULONG) - input
Reserved.

rc (LONG) - returns
Return codes.

The handling routine in the presentation driver returns a LONG integer. Valid values are:

-1	Successful
0	Error

Note: The system expects the successful and error return codes from OS2_PM_DRV_DEVICENAMES to be the opposite of those from

OS2_PM_DRV_DEVMODE and the Enable subfunctions.

OS2_PM_DRV_DEVICENAMES - Remarks

Applications such as the Presentation Manager Print Object call DevQueryDeviceNames to determine the device names and descriptions and the data types that the presentation driver supports. Hardcopy drivers must contain a handling routine for OS2_PM_DRV_DEVICENAMES.

Applications usually call this function twice. First, it is called with a NULL value for cNames and cDataTypes to query the number of names and data types. Second, after allocating the arrays, the application calls this function with valid values to get the data. If the value of cNames is NULL at the location addressed by pcNames, the handling routine must update cNames to the actual count of names. If cNames has a valid value, the routine must write device names and device descriptions into the arrays addressed by paDeviceName and paDeviceDesc. Similarly, for cDataTypes, the handling routine either writes a valid value into cDataTypes or writes data-type names into the array addressed by paDataType. Notice that when writing to an array, the routine does not write past the end of the array as defined by the associated count.

OS2_PM_DRV_DEVICENAMES - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

OS2_PM_DRV_DEVMODE

OS2_PM_DRV_DEVMODE - Syntax

Description:

OS2_PM_DRV_DEVMODE must be compiled to run at Ring 2 Conforming (privilege level 2) or Ring 3 (privilege level 3).

The Device Modes entry point is exported by hardcopy drivers as OS2_PM_DRV_DEVMODE to support the DevPostDeviceModes function at the application programming interface (API). This call is used to display the job and printer properties dialogs.

```
#include <os2.h>

PDRIVDATA    pDriverData;
PSZ          pszDriverName;
PSZ          pszDeviceName;
PSZ          pszPrinterName;
ULONG        lOptions;
ULONG        rc;             /*  Return codes. */

rc = OS2_PM_DRV_DEVMODE(pDriverData,
       pszDriverName, pszDeviceName, pszPrinterName,
       lOptions);
```

OS2_PM_DRV_DEVMODE Parameter - pDriverData

pDriverData (PDRIVDATA) - input
NULL or pointer to memory location for DRIVDATA structure:

cb	Number of bytes in the structure.
lVersion	Version number of the presentation driver. Subsequently used by the presentation driver to verify its entry in the INI file.
szDeviceName[32]	Device name.
abGeneralData	Driver-specific data for job or printer properties.

If pDriverData is NULL, the handling routine in the presentation driver must return the length of the driver's DRIVDATA structure for either job or printer properties.

OS2_PM_DRV_DEVMODE Parameter - pszDriverName

pszDriverName (PSZ) - input
A pointer to a string containing the name of the hardcopy driver.

OS2_PM_DRV_DEVMODE Parameter - pszDeviceName

pszDeviceName (PSZ) - input
A pointer to a string containing the device name, such as HP LASERJET II P, as defined by the presentation driver.

OS2_PM_DRV_DEVMODE Parameter - pszPrinterName

pszPrinterName (PSZ) - input
A NULL or pointer to a string containing the printer name, such as PRINTER1. A NULL pointer or NULL string are both valid conditions for this parameter.

OS2_PM_DRV_DEVMODE Parameter - lOptions

lOptions (ULONG) - input
Identifies the action that should be taken by the presentation driver:

DPDM_POSTJOBPROP	Display a dialog for job properties and return the DRIVDATA structure. Do not update OS2SYS.INI. The abGeneralData field contains the job properties.
DPDM_CHANGEPROP	Display a dialog for printer properties and device defaults, update the

	OS2SYS.INI file, and return the DRIVDATA structure. The abGeneralData field contains the printer properties.
DPDM_QUERYJOBPROP	Return the DRIVDATA structure with device default job properties. Do not display a dialog. The abGeneralData field contains the device default job properties.

All other values are reserved.

OS2_PM_DRV_DEVMODE Return Value - rc

rc (ULONG) - returns
Return codes.

The handling routine in the presentation driver returns a LONG integer. Valid values are:

DPDM_ERROR	Error.
DPDM_NONE	There are no settable options.
>0	The number of bytes for the required DRIVDATA structure when the input pointer pDriverData is equal to NULL.
DEV_OK	The data block pointed to by the input parameter pDriverData was initialized when the pDriverData input pointer is not equal to NULL.

OS2_PM_DRV_DEVMODE - Parameters

pDriverData (PDRIVDATA) - input
NULL or pointer to memory location for DRIVDATA structure:

cb	Number of bytes in the structure.
lVersion	Version number of the presentation driver. Subsequently used by the presentation driver to verify its entry in the INI file.
szDeviceName[32]	Device name.
abGeneralData	Driver-specific data for job or printer properties.

If pDriverData is NULL, the handling routine in the presentation driver must return the length of the driver's DRIVDATA structure for either job or printer properties.

pszDriverName (PSZ) - input
A pointer to a string containing the name of the hardcopy driver.

pszDeviceName (PSZ) - input
A pointer to a string containing the device name, such as HP LASERJET II P, as defined by the presentation driver.

pszPrinterName (PSZ) - input
A NULL or pointer to a string containing the printer name, such as PRINTER1. A NULL pointer or NULL string are both valid conditions for this parameter.

lOptions (ULONG) - input
Identifies the action that should be taken by the presentation driver:

DPDM_POSTJOBPROP	Display a dialog for job properties and return the DRIVDATA structure. Do not update OS2SYS.INI. The abGeneralData field contains the job properties.
DPDM_CHANGEPROP	Display a dialog for printer properties and device defaults, update the

	OS2SYS.INI file, and return the DRIVDATA structure. The abGeneralData field contains the printer properties.
DPDM_QUERYJOBPROP	Return the DRIVDATA structure with device default job properties. Do not display a dialog. The abGeneralData field contains the device default job properties.

All other values are reserved.

rc (ULONG) - returns
Return codes.

The handling routine in the presentation driver returns a LONG integer. Valid values are:

DPDM_ERROR	Error.
DPDM_NONE	There are no settable options.
>0	The number of bytes for the required DRIVDATA structure when the input pointer pDriverData is equal to NULL.
DEV_OK	The data block pointed to by the input parameter pDriverData was initialized when the pDriverData input pointer is not equal to NULL.

OS2_PM_DRV_DEVMODE - Remarks

In the hardcopy driver, the handling routine generates a DRIVDATA structure that defines the current setting of printer properties or job properties. These properties identify the options that are set when the job is printed. All hardcopy drivers must contain a handling routine for OS2_PM_DRV_DEVMODE.

Applications such as the Presentation Manager Print Object call DevPostDeviceModes to configure the device. Notice that such applications usually call this function twice, first with a NULL value for pDriverData to query the length of the driver's DRIVDATA structure, and then with a valid pointer to get the data.

Note: LONG, APIENTRY, PDRIVDATA (DRIVDATA *), and PSZ (char *) are defined in OS2DEF.H, which is included through the header file OS2.H.

The details about printer properties and job properties are stored as a set of flags or values in the array abGeneralData. Do not store pointers in this array because they might not be valid when they return. This array is driver-specific. The flags needed and the location of those flags in the array must be determined to fully exploit the capabilities of the device.

A list of related terms and their definitions follows.

- **Properties**

  This is a descriptive term for software and hardware characteristics of hardcopy devices, such as printers, plotters, and cameras. For a particular property, there is a list of possible values from which the user can select one or more values. This list can be extended by adding user-defined values, but can never be reduced by removing predefined values. For example, predefined forms, Letter and A4, can never be deleted. However, the user can add a form named "Blue Letter" to describe colored separator paper.

- **Job Properties**

  These are properties that can be changed from job to job. Typically, job property values are set in the printer by sending software commands. Some job properties can be derived from printer properties. Examples of typical job properties are:

  - Paper orientation
  - Forms required
  - Device resolution
  - Simplex (one-sided) or duplex (double-sided) printing

- **Printer Properties**

  These are properties that describe the physical characteristics of a printer. Printer properties are mutually exclusive of job properties (see the previous description). Examples of typical printer properties are:

  - Number of paper bins
  - Form size in each paper bin

- Hardware fonts installed
- Font cartridges installed (cartridge slots download fonts during system startup)
- Availability of optional add-on hardware, such as an envelope feeder or SCSI hard disk

- **User-Definable Values**

  For each individual property, there is a list of possible values that can be extended by making use of user-definable values (such as user-defined forms). User-definable values can only be defined or changed in the printer property dialogs, that is, when OS2_PM_DRV_DEVMODE is called with the DPDM_CHANGEPROP flag.

- **Selected Values**

  Given a particular property and its set of values, the user can choose one or more to be "selected" values. For example, Paper Bin 1 contains Letter paper, and the envelope bin accepts Com10, DL, or B5 envelopes.

- **Device Default Properties**

  Device defaults can be set in the printer properties dialog. These defaults do not usually contain user-defined values. The selectable values are chosen according to country code and the most common delivery configuration of the device. In a hardcopy driver that supports multiple devices, it is possible that the device default properties are different for different devices.

- **Option Flags**

  - DPDM_POSTJOBPROP
  - DPDM_CHANGEPROP
  - DPDM_QUERYJOBPROP

- **DPDM_POSTJOBPROP**

  Any application can call the OS2_PM_DRV_DEVMODE handling routine by using the flag option, DPDM_POSTJOBPROP. The calling program requires the user to select properties for a specific job such as draft or letter quality and size, style, and color of the default font. The source for the default value of the properties and device defaults is determined by the pszPrinterName parameter:

  - If pszPrinterName points to a valid string, the handling routine searches the PM_PrinterName,Driver.DeviceName section of OS2SYS.INI for the printer properties associated with the name.

  - If pszPrinterName is NULL or points to a NULL string, the handling routine uses the values from abGeneralData in the DRIVDATA structure addressed by pDriverData.

  In both cases, the initial default values are used if the handling routine cannot find a valid abGeneralData array.

  When called with the DPDM_POSTJOBPROP flag option, hardcopy drivers:

  - Examine the pDriverData parameter. If the pDriverData is valid, use these values as a basis of the values to be displayed in the user dialog. If the pDriverData is NULL , the driver version number is 0, or the pDriverData is not meaningful, use a set of device default job properties. There are two cases:

    -- If the printer name is given, use the job properties.
    -- If the printer name is NULL or not a valid printer name, use the job properties derived from the device defaults.

  - Update the job property values before displaying the user dialog. If the printer name is given, the printer properties stored by that printer are examined and compared with the job properties. There are two cases:

    -- Add extra values. If extra printer property values are defined, those that are applicable are added to the user dialog. For example, if a new user-defined form was added to the printer properties, it appears on the list of selectable forms.

    -- Remove values. If printer property values are deleted, the deleted values do not appear in the user dialog. If all the selected values are deleted from the printer properties, the device default becomes the selected value. For example, if a user-defined form is removed from the printer properties, it is replaced by a device default such as Letter (for the United States) or A4 (for France).

  - Display the user dialog (the user makes selections). Note that if the user selects the **Cancel** push button on the dialog, the hardcopy driver returns pDriverData unmodified. If pDriverData contains zeros or is not understood by the hardcopy driver, the driver should return the default job properties for the device.

  - Return the data to the caller. Any updates in the user dialog must be reflected in an updated pDriverData parameter, which is passed back to the caller of OS2_PM_DRV_DEVMODE. This update must occur regardless of whether an actual printer name or NULL printer name was passed in.

- **DPDM_CHANGEPROP**

  Only the Workplace Shell calls DevPostDeviceModes using this flag option. Applications that must change options for a particular job use DPDM_POSTJOBPROP.

The calling program requires the user to identify the current settings of the device defaults and select the device default properties. This usually requires two dialogs, one to identify options (such as the paper size currently in the device and details of any memory or font cartridges that are installed), and the other to establish a set of device default properties.

An error is returned if a NULL printer name is used. The pDriverData parameter is ignored. Given that a printer name is passed in, the hardcopy driver needs to retrieve the printer properties for that printer and the individual device for that driver. This means the hardcopy driver must store these properties separately from properties for other printers, in order to prevent properties and selected values of one printer from interacting with another. An example of printer property interaction might be if one printer has a user-defined form in Bin 1, and the other has no extra forms. It would be possible to send a job to the second printer that uses a form that is only available in the first printer.

The hardcopy driver builds a complex keyname containing the printer name, hardcopy driver and device name. For ease of use, store the printer properties under individual property keynames.

The format is easily extended for new printer properties. The format of the appname is:

```
PM_DD_<printer name>,<hardcopy driver>.<device name>
```

Two examples might be:

appname	= PM_DD_PRINTER1, LASERJET.HP LaserJet II
keyname	= BINFORMS
value	= Assignment of forms to paper bins
keyname	= FORMSDATA
value	= Definition of forms
appname	= PM_DD_PRINTER2, LASERJET.HP LaserJet II
keyname	= BINFORMS
value	= Assignment of forms to paper bins.

The advantage of this method is that the appname is standardized. Therefore:

- When a printer is renamed, the Workplace Shell can automatically move the data to a new appname without the hardcopy driver involved.
- The initialization file can be cleaned up by the Workplace Shell when a printer is deleted, or when the default hardcopy driver for a printer is changed.

- **DPDM_QUERYJOBPROP**

  Any application can call OS2_PM_DRV_DEVMODE by using the DPDM_QUERYJOBPROP flag option to find out the device default job properties. These defaults are derived from the printer properties. There are two cases:

  - The printer name is given so the properties can be retrieved from OS2SYS.INI under the PM_DD_Printer*xxx* application name.
  - The printer name is not given and the hardcopy driver uses the device default printer properties for that device.

Note: Information concerning the design of dialogs and menus is given in the *Common User Access Interface Design Guide* .

OS2_PM_DRV_DEVMODE - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

OS2_PM_DRV_ENABLE

The enable entry point is exported as OS2_PM_DRV_ENABLE by the presentation driver. The Enable routine in the driver supports a set of subfunctions that enable or disable the environment for a device context owned by a specific application or process. When a device context is opened or closed, the Presentation Manager interface issues a series of calls to subfunctions at the enable entry point. These calls initialize the presentation driver, the physical device, and the device context.

The syntax used by the Presentation Manager interface to call the Enable routine is:

```
LONG APIENTRY OS2_PM_DRV_ENABLE (Subfunc, Param1, Param2)

ULONG   Subfunc;
ULONG   Param1;
ULONG   Param2;
```

Note: LONG, ULONG (unsigned LONG), and APIENTRY are defined in OS2DEF.H, which is included through the header file OS2.H.

Stack Frame

At entry to the OS2_PM_DRV_ENABLE routine, the stack frame contains:

Parameter	Description
Subfunction	32-bit value identifying the subfunction
Param1	First parameter
Param2	Second parameter

Subfunctions

Presentation drivers must support the following OS2_PM_DRV_ENABLE subfunctions:

01h	FillLogicalDeviceBlock
02h	FillPhysicalDeviceBlock
04h	DisablePhysicalDeviceBlock
05h	EnableDeviceContext
06h	DisableDeviceContext
07h	SaveDCState
08h	RestoreDCState
09h	ResetDCState
0Ah	CompleteOpenDC
0Bh	BeginCloseDC
0Ch	InstallPMEngineDisplayHooks

Note: The OS2_PM_DRV_ENABLE function should return -1 (ERROR_MINUS) for Subfunction 03h, and numbers greater than 0Ch.

Device Context Management

Device contexts are opened in response to an application or process calling the DevOpenDC API function. On receiving this call, Presentation

Manager loads the presentation driver (if it is not already present and able to support the device context) and issues a series of calls to the enable entry point.

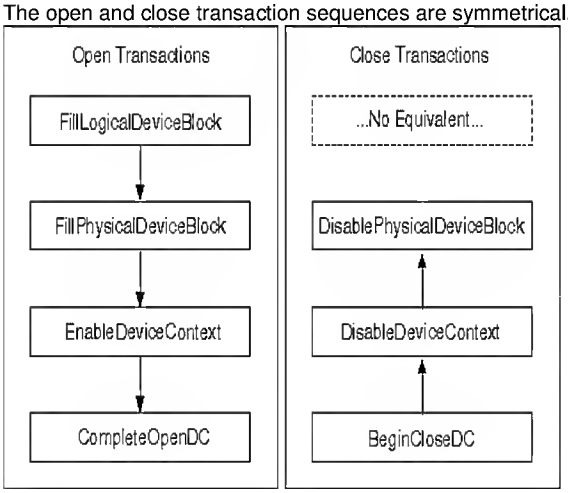
Initially, when the presentation driver has not been enabled for use by the calling application or process, the driver receives the series of calls shown in the following figure. When the driver has been enabled for an application or process, the sequence of calls to enable additional device contexts does not include subfunction FillLogicalDeviceBlock and, depending upon a requirement that the driver posts in the initial enable sequence, includes or excludes FillPhysicalDeviceBlock.

The OS2_PM_DRV_ENABLE entry point handles the DC management functions. Those functions can be placed in three categories:

- Transactions involved in opening a DC
- Transactions involved in closing a DC
- Other DC functions including SaveDC, RestoreDC and ResetDC

Opening and Closing DCs

The open and close transaction sequences are symmetrical.



Device contexts are opened in response to an application or process calling the DevOpenDC API function. On receiving this call, the graphics engine (PMGRE.DLL) loads the presentation driver, if it is not already present and able to support the device context, and issues a series of calls to the OS2_PM_DRV_ENABLE entry point.

Initially, when the presentation driver has not been enabled for use by the calling application or process, the driver receives the Open Transaction series of calls. When the driver has been enabled for an application or process, the sequence of calls to enable additional device contexts can include FillLogicalDeviceBlock and, depending upon a requirement that the driver posts in the initial enable sequence, includes or excludes FillPhysicalDeviceBlock.

Each of the Close Transaction functions undoes actions taken by a corresponding Open Transaction function. For example, theEnableDeviceContext subfunction allocates the DDC data area within the presentation driver and returns the hDDC handle. The DisableDeviceContext subfunction deallocates the DDC data area.

The FillLogicalDeviceBlock transaction occurs when a thread of a process opens the first instance of a DC for this presentation driver. During this transaction, a dispatch table will be created for later use by the graphics engine.

The FillPhysicalDeviceBlock transaction will occur only if the result flags from the FillLogicalDeviceBlock transaction indicate that it is necessary. See the description of Bit 2 under FillPhysicalDeviceBlock.

This subfunction is used by presentation drivers that support multiple types of physical devices. It will occur during the DevOpenDC call for every DC associated with a particular type of physical device. (This is where any memory allocation and initializations relating to a physical device should be done.) During the EnableDeviceContext transaction, the presentation driver will allocate and format the driver-specific data (DDC) associated with the DC being constructed. At CompleteOpenDC time, the DC is completely constructed and initialized. This transaction gives the presentation driver anopportunity to make final adjustments to the DC before the application gets access to it.

If either the FillLogicalDeviceBlock or FillPhysicalDeviceBlock transaction returns a failure return code, no other transactions are requested by the graphics engine.

If the EnableDeviceContext fails, the DisablePhysicalDeviceBlock transaction is sent to the presentation driver. If the CompleteOpenDC transaction fails, the BeginCloseDC, DisableDeviceContext, and DisablePhysicalDeviceBlock transactions are sent to the presentation driver.

If an application has opened a device context, but the application terminates before it closes the device context, the graphics engine still calls

the disable routines BeginCloseDC, DisableDeviceContext, and DisablePhysicalDeviceBlock, as expected by the presentation driver when a device context is closed explicitly by an application.

Saving and Restoring Device Contexts

The SaveDCState and RestoreDCState subfunctions manage a stack of entries that correspond to most of the state of a DC. SaveDCState pushes an entry on the stack. RestoreDCState retrieves one of the pushed entries and uses it to revert the DC back to its earlier saved state. Note that a particular RestoreDCState call can pop more than one entry from the stack of SaveDC entries.

Resetting a Device Context

This call resets a DC to its original initialized state, as it was just after the CompleteOpenDC transaction.

OS2_PM_DRV_ENABLE: Subfunction 01h - FillLogicalDeviceBlock

OS2_PM_DRV_ENABLE: Subfunction 01h - FillLogicalDeviceBlock

Description:

OS2_PM_DRV_ENABLE: FillLogicalDeviceBlock is called on the first occasion that a specific application or process opens a device context that uses the called presentation driver.

```
#include <os2.h>

ULONG     ulSubfunction;    /*  01h. */
PULONG    pParam1;
PLONG     pParam2;
PLONG     pDispatchTable;
LONG      rc;               /*  Return codes. */

rc = FillLogicalDeviceBlock(ulSubfunction,
       pParam1, pParam2, pDispatchTable);
```

OS2_PM_DRV_ENABLE: Subfunction 01h - FillLogicalDeviceBlock ulSubfunction

ulSubfunction (ULONG) - input
    01h.

OS2_PM_DRV_ENABLE: Subfunction 01h - FillLogicalDeviceBlock

pParam1

pParam1 (PULONG) - input
A pointer to the following structure:

ulVersion	Version number in binary coded decimal (BCD) of the graphic engine.
cTableSize	The number of entries in the dispatch table. The presentation driver should not replace pointers past the end of the table.

OS2_PM_DRV_ENABLE: Subfunction 01h - FillLogicalDeviceBlock pParam2

pParam2 (PLONG) - input
A pointer to the following structure:

pfsFlags — Pointer to a WORD of logical device flags. Flag bits 0 and 2 apply to the presentation driver and are set ON or OFF, as appropriate, by the presentation driver. All other flags are reserved for system use and must not be modified.

Bit 0	Set ON if each device context for this presentation driver requires its own physical device block. Even if the presentation driver sets this bit off, the FillPhysicalDeviceBlock subfunction can still be called more than once (although no work may need to be done after FillPhysicalDeviceBlock is called the first time).
Bit 2	Set ON if the szDeviceName and pszDriverName fields of a DevOpenDC call for this device are ignored. Setting Bit 2 ON indicates that the presentation driver supports only one physical device in one configuration, for example, the display driver. Hardcopy drivers do not set this bit unless the hardcopy driver supports only one physical device in one configuration.
Bit 7	This is a new bit in OS/2 2.1, Graphics Engine Version 0x202 or later. Setting Bit 7 ON indicates that the presentation driver may update the dispatch table in FillPhysicalDeviceBlock. Do this if your driver supports more than one type of device and you want to hook different functions based on the device type.

OS2_PM_DRV_ENABLE: Subfunction 01h - FillLogicalDeviceBlock pDispatchTable

pDispatchTable (PLONG) - input
A pointer to the presentation driver's copy of the dispatch table. This table is an array of 32-bit pointers to system-default function-handling routines. The low-order byte of a function number identifies the offset to the relevant pointer.

OS2_PM_DRV_ENABLE: Subfunction 01h - FillLogicalDeviceBlock

rc (LONG) - returns

Return codes.

The handling routine should return a LONG integer. Valid values are:

0	Successful
-1	Error

OS2_PM_DRV_ENABLE: Subfunction 01h - FillLogicalDeviceBlock

ulSubfunction (ULONG) - input
01h.

pParam1 (PULONG) - input
A pointer to the following structure:

ulVersion	Version number in binary coded decimal (BCD) of the graphic engine.
cTableSize	The number of entries in the dispatch table. The presentation driver should not replace pointers past the end of the table.

pParam2 (PLONG) - input
A pointer to the following structure:

pfsFlags — Pointer to a WORD of logical device flags. Flag bits 0 and 2 apply to the presentation driver and are set ON or OFF, as appropriate, by the presentation driver. All other flags are reserved for system use and must not be modified.

Bit 0	Set ON if each device context for this presentation driver requires its own physical device block. Even if the presentation driver sets this bit off, the FillPhysicalDeviceBlock subfunction can still be called more than once (although no work may need to be done after FillPhysicalDeviceBlock is called the first time).
Bit 2	Set ON if the szDeviceName and pszDriverName fields of a DevOpenDC call for this device are ignored. Setting Bit 2 ON indicates that the presentation driver supports only one physical device in one configuration, for example, the display driver. Hardcopy drivers do not set this bit unless the hardcopy driver supports only one physical device in one configuration.
Bit 7	This is a new bit in OS/2 2.1, Graphics Engine Version 0x202 or later. Setting Bit 7 ON indicates that the presentation driver may update the dispatch table in FillPhysicalDeviceBlock. Do this if your driver supports more than one type of device and you want to hook different functions based on the device type.

pDispatchTable (PLONG) - input
A pointer to the presentation driver's copy of the dispatch table. This table is an array of 32-bit pointers to system-default function-handling routines. The low-order byte of a function number identifies the offset to the relevant pointer.

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

0	Successful
-1	Error

OS2_PM_DRV_ENABLE: Subfunction 01h - FillLogicalDeviceBlock

This subfunction is not called when the same application or process opens additional device contexts with the same presentation driver. However, the graphics engine can call this subfunction at other times and the call should always be honored. If the value of pDispatchTable is

nonzero, the handling routine must initialize the dispatch table.

The major tasks that the handling routine must implement are:

1. Add an entry to the DosExitList to ensure that any allocated resources are freed when the owning application or process terminates.

2. Save those pointers in the dispatch table that are needed to pass hooked functions back to the graphics engine. A typical example is GreCharString, which must be hooked by the presentation driver, but can be passed back to the default handling routine.

3. Initialize the dispatch table. That is, modify the table so that the entries for functions hooked by the presentation driver contain pointers to the driver's handling routines.

4. Set flags to indicate how future DevOpenDC calls to this device should be handled.

In some typical presentation drivers, the handling routine for FillLogicalDeviceBlock allocates global heap space for use by the device contexts. The memory for this heap space is obtained by calling the SSAllocMem function. This global heap space is available to all instances of a device context that are opened by the application or process for which the presentation driver was enabled.

OS2_PM_DRV_ENABLE: Subfunction 01h - FillLogicalDeviceBlock

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

OS2_PM_DRV_ENABLE: Subfunction 02h - FillPhysicalDeviceBloc

OS2_PM_DRV_ENABLE: Subfunction 02h - FillPhysicalDeviceBloc

Description:

OS2_PM_DRV_ENABLE: FillPhysicalDeviceBlock is always called in the initial set of calls to the enable function for a specific application or process.

```
#include <os2.h>

ULONG     ulSubfunction;  /*  02h. */
PULONG    pParam1;
PLONG     pParam2;        /*  Pointer to the dispatch table */
LONG      rc;             /*  Return codes. */

rc = FillPhysicalDeviceBlock(ulSubfunction,
       pParam1, pParam2);
```

OS2_PM_DRV_ENABLE: Subfunction 02h - FillPhysicalDeviceBloc ulSubfunction

ulSubfunction (ULONG) - input
02h.

OS2_PM_DRV_ENABLE: Subfunction 02h - FillPhysicalDeviceBloc pParam1

pParam1 (PULONG) - input
A pointer to an extended DEVOPENSTRUC structure (the structure is extended by adding a DENPARAMS structure to give three additional fields: ulStateInfo, ulType, and ulHDC).

pszLogAddress — Pointer to the logical address of the device, such as "LPT1Q".

pszDriverName — Pointer to name of the presentation driver, such as 'LaserJet'.

pdriv — Pointer to DRIVDATA structure. This structure contains data generated by the presentation driver during the dialog that sets device modes. See OS2_PM_DRV_DEVMODE.

cb	Number of bytes in the structure
lVersion	Version number
szDeviceName[32]	Device name
abGeneralData	Driver-specific data

pszDataType — Pointer to the name of the queue file data type:

PM_Q_STD
PM_Q_RAW

pszComment — Pointer to a file description that the spooler could use in messages displayed to the user; usually the name of the application by convention.

pszQueueProcName — Pointer to name of queue driver (queue processor).

pszQueueProcParams — Pointer to a string of queue processor parameters.

pszSpoolerParams — Pointer to a string of spooler parameters separated by one or more blanks.

FORM=aaa	Identifies the form name for the print job. Multiple names are separated by commas (aaa,bbb,ccc). If this parameter is not present the job is printed on the current form. Form names are defined by the presentation driver. Valid names are those that are returned from a call to the driver's GreQueryHardcopyCaps handling routine.

pszNetworkParams — Pointer to a string of networking parameters.

ulStateInfo — Reserved. This field does not contain meaningful information at this time.

ulType — DC type (for example, OD_QUEUED).

ulHDC	DC handle.

OS2_PM_DRV_ENABLE: Subfunction 02h - FillPhysicalDeviceBloc pParam2

pParam2 (PLONG) - input
Pointer to the dispatch table

Pointer to the dispatch table if Bit 7 was set in FillLogicalDeviceBlock and the Graphics Engine version is 0x202 or later.

OS2_PM_DRV_ENABLE: Subfunction 02h - FillPhysicalDeviceBloc

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

-1	Error.
Other	Handle or pointer, ulStateInfo, to the physical device block. This pointer is passed back to the presentation driver in subsequent calls to the OS2_PM_DRV_ENABLE subfunctions 04h and 05h. **Note:** The name, pDCI, is used in place of ulStateInfo in some source code.

OS2_PM_DRV_ENABLE: Subfunction 02h - FillPhysicalDeviceBloc

ulSubfunction (ULONG) - input
02h.

pParam1 (PULONG) - input
A pointer to an extended DEVOPENSTRUC structure (the structure is extended by adding a DENPARAMS structure to give three additional fields: ulStateInfo, ulType, and ulHDC).

pszLogAddress	Pointer to the logical address of the device, such as "LPT1Q".	
pszDriverName	Pointer to name of the presentation driver, such as 'LaserJet'.	
pdriv	Pointer to DRIVDATA structure. This structure contains data generated by the presentation driver during the dialog that sets device modes. See OS2_PM_DRV_DEVMODE.	
	cb	Number of bytes in the structure
	lVersion	Version number
	szDeviceName[32]	Device name
	abGeneralData	

Driver-spec
ific data

pszDataType — Pointer to the name of the queue file data type:

PM_Q_STD
PM_Q_RAW

pszComment — Pointer to a file description that the spooler could use in messages displayed to the user; usually the name of the application by convention.

pszQueueProcName — Pointer to name of queue driver (queue processor).

pszQueueProcParams — Pointer to a string of queue processor parameters.

pszSpoolerParams — Pointer to a string of spooler parameters separated by one or more blanks.

FORM=aaa — Identifies the form name for the print job. Multiple names are separated by commas (aaa,bbb,ccc). If this parameter is not present the job is printed on the current form.

Form names are defined by the presentation driver. Valid names are those that are returned from a call to the driver's GreQueryHardcopyCaps handling routine.

pszNetworkParams — Pointer to a string of networking parameters.

ulStateInfo — Reserved. This field does not contain meaningful information at this time.

ulType — DC type (for example, OD_QUEUED).

ulHDC — DC handle.

pParam2 (PLONG) - input
Pointer to the dispatch table

Pointer to the dispatch table if Bit 7 was set in FillLogicalDeviceBlock and the Graphics Engine version is 0x202 or later.

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

-1 — Error.
Other — Handle or pointer, ulStateInfo, to the physical device block. This pointer is passed back to the presentation driver in subsequent calls to the OS2_PM_DRV_ENABLE subfunctions 04h and 05h.

Note: The name, pDCI, is used in place of ulStateInfo in some source code.

OS2_PM_DRV_ENABLE: Subfunction 02h - FillPhysicalDeviceBloc

This subfunction is called multiple times only when additional device contexts are enabled if the value of flag bit 0, as set in the initial call to FillLogicalDeviceBlock, shows that each device context requires its own physical device block. However, the graphics engine can call this subfunction at other times. If it is called more than once when the individual DCs do not require separate physical device blocks, the handling routine does nothing and returns the existing ulStateInfo handle or pointer from the DC instance data.

If the param2 (the pointer to the dispatch table) is not null, then the engine version you are running supports rehooking the dispatch table. You would want to change the hooked functions in the dispatch table if your driver supports multiple devices and the device you will be using supports different functions than were hooked in FillLogicalDeviceBlock.

Print jobs must be spooled using the data type given on the call to enable FillLogicalDeviceBlock. Therefore, print jobs queued PM_Q_STD can be printed with queue processor options applied to the job.

The physical device block is located in the driver's global heap. To initialize the block, the presentation driver uses:

- Default values set by the presentation driver
- Values read from the initialization file
- Values from the DEVOPENSTRUC structure

Physical device blocks hold information about the presentation driver and the device that is the same for every instance of a device context. The design of the presentation driver determines what information is held in the physical device block. The following example shows a typical printer physical device block.

```
typedef struct
{
  Dptr                 PDBDriverName;              /* String for driver name       */
  Dptr                 PDBOutputName;              /* String for output name       */
  unsigned             PDBHandle;                  /* Handle for DOS device        */
  WORD                 PDBOutputType;              /* Type of output; STD/ESC/RAW  */
  DeviceSemaphoreTable PDBDevice;                  /* Pointer to device table      */
  Byte                 PDBScratch[DCT_MAX_SIMPLE]; /* Scratch pad for printer      */
  DDTType              DDT;                        /* Copy of the DDT to be used   */
  WORD                 RasterMode;                 /* Raster type used             */
  WORD                 PrintQuality;               /* Draft or LQ printing         */
  WORD                 Orientation;                /* Portrait or landscape        */
  DWORD                Version;                    /* Driver version number        */
  DevRect              FormClipRegion;             /* Clip region for current form */
  Byte                 DeviceName[32];             /* Model number, and so forth   */
} PDBIType;
```

Hardcopy drivers must be able to accept a UNC queue name as a logical address on FillPhysicalDeviceBlock. The UNC queue name is passed to the SplQmOpen API that handles rerouting the spool job across the LAN.

Some applications use the SpoolerParams to submit form names. However, the forms can also be supplied in the driver data. SpoolerParams take precedence over the driver data. At FillPhysicalDeviceBlock time, any SpoolerParams are integrated with the *pdriv* field of the DEVOPENSTRUC.

If *pdriv* is NULL, it is created by obtaining the data from the system. The hardcopy driver attempts to find a printer in the .INI file that uses this port name, and derives job properties from the device defaults and printer properties.

To find the printer properties in the OS/2 SYS.INI file, the presentation driver should use the APPNAME format:

```
PM_DD_<printername>,<hardcopydriver>.<devicename>
```

To get the printername, call SplQueryQueue with the DEVOPENSTRUC pszLogAddress for OD_Queued jobs.

Call SplEnumDevice for OD_Direct jobs.

Search through the structures returned by SplEnumDevice or SplEnumQueue to find the one with the matching pszLogAddress. Use the *printername* field from the matching structure.

If you do not find your device, look for the pszLogAddress with the name "file".

Note: Hardcopy drivers process all fields except *pszQeueProcName, pszQueueProcParams,* and *pszNetworkParams*.

OS2_PM_DRV_ENABLE: Subfunction 02h - FillPhysicalDeviceBloc

Select an item:
Syntax

Parameters
Returns
Remarks
Glossary

OS2_PM_DRV_ENABLE: Subfunction 04h - DisablePhysicalDevice

OS2_PM_DRV_ENABLE: Subfunction 04h - DisablePhysicalDevice

Description:

OS2_PM_DRV_ENABLE: DisablePhysicalDeviceBlock is called by the system to disable the specified device and to free any associated memory.

```
#include <os2.h>

ULONG    ulSubfunction;  /*  04H. */
ULONG    ulParam1;
PLONG    pParam2;        /*  Not used. */
LONG     rc;             /*  Return codes. */

rc = DisablePhysicalDeviceBlock(ulSubfunction,
       ulParam1, pParam2);
```

OS2_PM_DRV_ENABLE: Subfunction 04h - DisablePhysicalDevice ulSubfunction

ulSubfunction (ULONG) - input
04H.

OS2_PM_DRV_ENABLE: Subfunction 04h - DisablePhysicalDevice ulParam1

ulParam1 (ULONG) - input
A handle or pointer, ulStateInfo, to physical device block.

OS2_PM_DRV_ENABLE: Subfunction 04h - DisablePhysicalDevice pParam2

pParam2 (PLONG) - input
Not used.

OS2_PM_DRV_ENABLE: Subfunction 04h - DisablePhysicalDevice
rc

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

0	Successful
-1	Error

OS2_PM_DRV_ENABLE: Subfunction 04h - DisablePhysicalDevice

ulSubfunction (ULONG) - input
04H.

ulParam1 (ULONG) - input
A handle or pointer, ulStateInfo, to physical device block.

pParam2 (PLONG) - input
Not used.

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

0	Successful
-1	Error

OS2_PM_DRV_ENABLE: Subfunction 04h - DisablePhysicalDevice

Presentation drivers for the primary display device return a value of 0 (zero) without taking any action.

Note: The operating system never calls this subfunction in the hardcopy driver if the driver uses one physical device block to support multiple device contexts. Presentation drivers notify the operating system of this capability by not setting bit 0 of the Logical Device flags returned to the system from the FillLogicalDeviceBlock subroutine.

OS2_PM_DRV_ENABLE: Subfunction 04h - DisablePhysicalDevice

Select an item:

Syntax
Parameters
Returns
Remarks
Glossary

OS2_PM_DRV_ENABLE: Subfunction 05h - EnableDeviceContext

OS2_PM_DRV_ENABLE: Subfunction 05h - EnableDeviceContext

Description:

In response to *OS2_PM_DRV_ENABLE: EnableDeviceContext* , the driver reserves the memory it needs to support this instance of a DC and initializes the instance data.

```
#include <os2.h>

ULONG     ulSubfunction;  /*  05h. */
PULONG    pParam1;
PLONG     pParam2;        /*  Not used. */
LONG      rc;             /*  Return codes. */

rc = EnableDeviceContext(ulSubfunction, pParam1,
       pParam2);
```

OS2_PM_DRV_ENABLE: Subfunction 05h - EnableDeviceContext ulSubfunction

ulSubfunction (ULONG) - input
05h.

OS2_PM_DRV_ENABLE: Subfunction 05h - EnableDeviceContext pParam1

pParam1 (PULONG) - input
A pointer to a DENPARAMS structure:

ulStateInfo	Value returned by FillPhysicalDeviceBlock.
ulType	Type of device context. Defined values are: OD_QUEUED OD_DIRECT OD_INFO

OD_MEMORY

For details, see DevOpenDC in the *Presentation Manager Programming Reference* .

ulHDC — Device context handle.

OS2_PM_DRV_ENABLE: Subfunction 05h - EnableDeviceContext pParam2

pParam2 (PLONG) - input
Not used.

OS2_PM_DRV_ENABLE: Subfunction 05h - EnableDeviceContext

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

-1	Error.
other	Pointer (pInstance) to the DC instance data.

OS2_PM_DRV_ENABLE: Subfunction 05h - EnableDeviceContext

ulSubfunction (ULONG) - input
05h.

pParam1 (PULONG) - input
A pointer to a DENPARAMS structure:

ulStateInfo — Value returned by FillPhysicalDeviceBlock.

ulType — Type of device context. Defined values are:

OD_QUEUED
OD_DIRECT
OD_INFO
OD_MEMORY

For details, see DevOpenDC in the *Presentation Manager Programming Reference* .

ulHDC — Device context handle.

pParam2 (PLONG) - input
Not used.

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

-1	Error.
other	Pointer (pInstance) to the DC instance data.

OS2_PM_DRV_ENABLE: Subfunction 05h - EnableDeviceContext

The value of the return code from this subfunction, if it is not -1, is saved by the system and passed back to the presentation driver on all future calls to this DC instance. This value is expected to be a pointer or handle to the instance data.

The handling routine initializes relevant fields in the instance data to their default values. For example, it calls WinQueryProcessCP to get the initial Code Page ID.

STARTDOC/ABORTDOC/ENDDOC (Hardcopy drivers only)

The hardcopy driver allows the following sequences to GreEscapes within the subfunctions EnableDeviceContext and BeginCloseDC brackets:

```
GreEscape DEVESC_STARTDOC
GreEscape DEVESC_ENDDOC                    - spools first job
GreEscape DEVESC_STARTDOC
GreEscape DEVESC_ENDDOC                    - spools second job
GreEscape DEVESC_STARTDOC
GreEscape DEVESC_ENDDOC

GreEscape DEVESC_STARTDOC
GreEscape DEVESC_ABORTDOC                  - terminates job
GreEscape DEVESC_STARTDOC
GreEscape DEVESC_ENDDOC                    - spools first job
```

Instance data refers to information (such as the name of a spool file and whether a bit map has been created) that applies to a specific instance of a device context. Global data, which applies to all instances of a device context that is using the same presentation driver, is held in the Physical Device Block (PDB). A pointer to this block is passed in the DENPARAMS structure to the EnableDeviceContext routine to be included in the instance data. See Instance Data. A typical example of instance data follows:

```
typedef struct _DC {
  USHORT            DCIIdentifier;     /* Contains DC_IDENTIFIER              */
  ULONG             DCIDCType;         /* DC type 0-8                         *
  HDC               DCIhdc;            /* DC handle                           */

  struct _DC *      DCINextEntry;      /* Next instance                       */
  LONG              DCISaveCount;      /* Number of saved DC states           */

  pBitmapHeader     DCISelListEntry;   /* Selected bit-map list entry         */
  USHORT            DCIBitmapType;     /* Current bit-map type                */

  POINTL            DCICurrPos;        /* Current position                    */

  DCHARBUNDLE       DCICurTxtAts;      /* Current Text attributes bundle      */
  DLINEBUNDLE       DCICurLinAts;      /* Current Line attributes bundle      */
  DAREABUNDLE       DCICurPtnAts;      /* Current Pattern attributes bundle   */
  DIMAGEBUNDLE      DCICurImgAts;      /* Current Image attributes bundle     */
  DMARKERBUNDLE     DCICurMrkAts;      /* Current Marker attributes bundle    */

  USHORT            DCILinePatCnt;     /* Current line pattern count          */
  USHORT            DCILineTypMask;    /* Mask used for line types            */

  POINTL            DCIPatternOrigin;  /* Pattern origin                      */

  pBitmapHeader     DCIMarker;         /* Pointer to marker definition        */
  BOOL              DCIMarkerSimReq;   /* On, if simulation required          */

  USHORT            DCIFontType;       /* Type of current font                */




  pRealizedFontType DCIRealizedFonts;  /* Font table array                    */
  USHORT            DCIFontTabNum;     /* Number of Font table entries        */
  BOOL              DCIPairKerning;    /* Enabled\disabled state              */

  ULONG             DCICodePage;       /* Currently selected code page        */

  BOOL              DCITextSim;        /* Set if text simulation required     */
```

```
  POINTL            DCIOrigin;         /* Current DC origin                    */

  PCOLORTABLE       DCIColorTable;     /* Pointer to color table               */
  BOOL              DCIBackgndDefined; /* Status of CLR_BACKGROUND              */
  BOOL              DCINeutralDefined; /* Status of CLR_NEUTRAL                */
  USHORT            DCIColTabSize;     /* Color table size                     */
  USHORT            DCIColFormat;      /* Format of color table                */
  USHORT            DCIColStatus;      /* Status of color table                */
  USHORT            DCILowIndex;       /* Lowest index in table                */
  USHORT            DCIHighIndex;      /* Highest index in table               */
  USHORT            DCISysColState;    /* Latest state of system colors        */

  COLORPAIR         DCILineColatts;    /* Line color indexes                   */
  COLORPAIR         DCIPattColatts;    /* Pattern colors indexes               */
  COLORPAIR         DCICharColatts;    /* Character colors indexes             */
  COLORPAIR         DCIImagColatts;    /* Image colors indexes                 */
  COLORPAIR         DCIMarkColatts;    /* Marker colors indexes                */

  ClipRectangle     DCIClipRects[CACHED_CLIPS];
                                       /* Clip rectangles                      */
  BOOL              ClipChanged;       /*                                      */
  USHORT            DCIClipOrder;      /* Order of clip rectangles             */
  USHORT            DCIClipNum;        /* Number of clip rectangles            */
  RECTL             DCIBoundingClip;   /* Current screen/bit-map area          */
  USHORT            DCIEngineClips;    /* Clip regions in engine               */
  BOOL              DCIIsDirty;        /* Indicates when clip regions          */
                                       /* are invalid

  RECTL             DCIGPIBounds;      /* Current GPI bounds                   */
  BOOL              DCIDefGPIBounds;   /* On, if GPI bounds are default        */
  RECTL             DCIUserBounds;     /* Current user bounds                  */
  BOOL              DCIDefUserBounds;  /* On, if user bounds are default       */
  SHORT             DCIConvFactor;     /* For device, conversion               */

  BOOL              DCICorrInvalid;    /* Correlation rectangles invalid       */
  RECTL             DCIPickWindowPage; /* Pick window in page coordinates      */
  DevRect           DCIPickWindowDevice;
                                       /* Pick window in device coordinates    */
  PRECTL            DCICorrRects;      /* Current correlation rectangles       */
  USHORT            DCICorrNum;        /* Number of correlation rectangles     */
  USHORT            DCICorrSize;       /* Number of correlation rectangles for */
                                       /* which storage has been allocated     */




  XFORM             DCITransform;      /* Transformation data                  */

  FONTDETAILS       DCIAvioFonts[CNT_LOADABLE_LCIDS + 1];
                                       /* AVIO loadable font definitions       */

  USHORT            DCIChanged;        /* Shows non-default bundles            */

  USHORT            DCISpacingType;    /* Spacing type of current font         */
  BOOL              DCIXFrmSimple;     /* Transform type                       */
  ULONG             StyleNumber;       /* Line style state and mask            */

  USHORT            DCICommandMask;    /* Used to mask command bits            */

  POINTL            DCICurrPosWorld;   /* Current world position               */
  pBitmapHeader     Pattern;           /* Pattern for direct DC                */

  AVIOINFO          DCIAvioInfo;       /* Avioparms material                   */
  FONTDETAILS       CurrentFont;       /* Presentation Mgr. format current font */
  PDEVPAL           Palette;           /* Handle/pointer of palette            */
  PRGB2             DCIDeviceDefaultPalette;
                                       /* Device default palette that was      */
                                       /* current the last time color table    */
                                       /* indexes were calculated              */
} DC;
```

OS2_PM_DRV_ENABLE: Subfunction 05h - EnableDeviceContext

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

OS2_PM_DRV_ENABLE: Subfunction 06h - DisableDeviceContext

OS2_PM_DRV_ENABLE: Subfunction 06h - DisableDeviceContext

Description:

OS2_PM_DRV_ENABLE: DisableDeviceContext is called when a device context is about to be deleted. In response, the presentation driver must release any memory and other resources that it has allocated for the DC.

```
#include <os2.h>

ULONG     ulSubfunction;  /*  06H. */
PULONG    pParam1;
PLONG     pParam2;        /*  Not used. */
LONG      rc;             /*  Return codes. */

rc = DisableDeviceContext(ulSubfunction, pParam1,
       pParam2);
```

OS2_PM_DRV_ENABLE: Subfunction 06h - DisableDeviceContext ulSubfunction

ulSubfunction (ULONG) - input
06H.

OS2_PM_DRV_ENABLE: Subfunction 06h - DisableDeviceContext pParam1

pParam1 (PULONG) - input
A pointer (pInstance) to the DC instance data.

OS2_PM_DRV_ENABLE: Subfunction 06h - DisableDeviceContext

pParam2

pParam2 (PLONG) - input
Not used.

OS2_PM_DRV_ENABLE: Subfunction 06h - DisableDeviceContext

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

0	Successful
-1	Error

OS2_PM_DRV_ENABLE: Subfunction 06h - DisableDeviceContext

ulSubfunction (ULONG) - input
06H.

pParam1 (PULONG) - input
A pointer (pInstance) to the DC instance data.

pParam2 (PLONG) - input
Not used.

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

0	Successful
-1	Error

OS2_PM_DRV_ENABLE: Subfunction 06h - DisableDeviceContext

The presentation driver uses the DC instance data to identify this memory.

OS2_PM_DRV_ENABLE: Subfunction 06h - DisableDeviceContext

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

OS2_PM_DRV_ENABLE: Subfunction 07h - SaveDCState

OS2_PM_DRV_ENABLE: Subfunction 07h - SaveDCState - Syntax

OS2_PM_DRV_ENABLE: SaveDCState requests the presentation driver to save all the information that it has about the device context.

```
#include <os2.h>

ULONG     ulSubfunction;  /*  07h. */
PULONG    pParam1;
PLONG     pParam2;        /*  Not used. */
LONG      rc;             /*  Return codes. */

rc = SaveDCState(ulSubfunction, pParam1, pParam2);
```

OS2_PM_DRV_ENABLE: Subfunction 07h - SaveDCState Parame

ulSubfunction (ULONG) - input
    07h.

OS2_PM_DRV_ENABLE: Subfunction 07h - SaveDCState Parame

pParam1 (PULONG) - input
    A pointer (pInstance) to the DC instance data.

OS2_PM_DRV_ENABLE: Subfunction 07h - SaveDCState Parame

pParam2 (PLONG) - input
    Not used.

OS2_PM_DRV_ENABLE: Subfunction 07h - SaveDCState Return

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

0	Successful
-1	Error

OS2_PM_DRV_ENABLE: Subfunction 07h - SaveDCState - Param

ulSubfunction (ULONG) - input
07h.

pParam1 (PULONG) - input
A pointer (pInstance) to the DC instance data.

pParam2 (PLONG) - input
Not used.

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

0	Successful
-1	Error

OS2_PM_DRV_ENABLE: Subfunction 07h - SaveDCState - Remar

The state of a DC might be saved multiple times in last-in-first-out (LIFO) order. The routine returns an error code if there is not enough memory available to save the state.

The handling routine must keep a count of the number of saved states.

RestoreDCState is related to SaveDCState.

OS2_PM_DRV_ENABLE: Subfunction 07h - SaveDCState - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

OS2_PM_DRV_ENABLE: Subfunction 08h - RestoreDCState

OS2_PM_DRV_ENABLE: Subfunction 08h - RestoreDCState - Syn

Description:

OS2_PM_DRV_ENABLE: RestoreDCState restores a specific DC state from the saved DC states.

```
#include <os2.h>

ULONG      ulSubfunction;  /*  08h. */
PULONG     pParam1;
PLONG      pParam2;
LONG       rc;             /*  Return codes. */

rc = RestoreDCState(ulSubfunction, pParam1,
       pParam2);
```

OS2_PM_DRV_ENABLE: Subfunction 08h - RestoreDCState Para

ulSubfunction (ULONG) - input
08h.

OS2_PM_DRV_ENABLE: Subfunction 08h - RestoreDCState Para

pParam1 (PULONG) - input
A pointer (pInstance) to the DC instance data.

OS2_PM_DRV_ENABLE: Subfunction 08h - RestoreDCState Para

pParam2 (PLONG) - input
Identifies which of the saved states are to be restored. Positive numbers indicate the specific state counting from the first saved state, that is, 1 equals the first, 2 the second, and so forth. All saved states following the one being restored are discarded. Preceding states remain saved.

Negative numbers indicate the specific state counting from the last saved state, a value of -2 indicates that the last state is discarded and the state before that is restored.

OS2_PM_DRV_ENABLE: Subfunction 08h - RestoreDCState Retu

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

0	Successful
-1	Error

OS2_PM_DRV_ENABLE: Subfunction 08h - RestoreDCState - Par

ulSubfunction (ULONG) - input
08h.

pParam1 (PULONG) - input
A pointer (pInstance) to the DC instance data.

pParam2 (PLONG) - input
Identifies which of the saved states are to be restored. Positive numbers indicate the specific state counting from the first saved state, that is, 1 equals the first, 2 the second, and so forth. All saved states following the one being restored are discarded. Preceding states remain saved.

Negative numbers indicate the specific state counting from the last saved state, a value of -2 indicates that the last state is discarded and the state before that is restored.

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

0	Successful
-1	Error

OS2_PM_DRV_ENABLE: Subfunction 08h - RestoreDCState - Rer

An index to the required state is supplied as Parameter 2. The presentation driver returns an error if the index is 0 or if it specifies a value that does not identify a saved state.

Display drivers must be careful when handling this call. Some of the data stored in the DC instance data must match the data held by the Window Manager. The handling routine for RestoreDCState does not restore:

- DC origin
- User bounds
- Cached clipping rectangles
- HDC_IS_DIRTY flag

OS2_PM_DRV_ENABLE: Subfunction 08h - RestoreDCState - Top

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

OS2_PM_DRV_ENABLE: Subfunction 09h - ResetDCState

OS2_PM_DRV_ENABLE: Subfunction 09h - ResetDCState - Synta

Description:

OS2_PM_DRV_ENABLE: ResetDCState resets the device context to its original initialized state.

```
#include <os2.h>

ULONG     ulSubfunction;  /*  09h */
PULONG    pParam1;
PLONG     pParam2;        /*  Reserved. */
LONG      rc;             /*  Return codes. */

rc = ResetDCState(ulSubfunction, pParam1,
       pParam2);
```

OS2_PM_DRV_ENABLE: Subfunction 09h - ResetDCState Param

ulSubfunction (ULONG) - input
09h

OS2_PM_DRV_ENABLE: Subfunction 09h - ResetDCState Param

pParam1 (PULONG) - input
A pointer (pInstance) to the DC instance data.

OS2_PM_DRV_ENABLE: Subfunction 09h - ResetDCState Param

pParam2 (PLONG) - input
Reserved.

OS2_PM_DRV_ENABLE: Subfunction 09h - ResetDCState Return

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

0	Successful
-1	Error

OS2_PM_DRV_ENABLE: Subfunction 09h - ResetDCState - Parar

ulSubfunction (ULONG) - input
09h

pParam1 (PULONG) - input
A pointer (pInstance) to the DC instance data.

pParam2 (PLONG) - input
Reserved.

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

0	Successful
-1	Error

OS2_PM_DRV_ENABLE: Subfunction 09h - ResetDCState - Rema

The presentation driver deletes all fonts, patterns, and paths, and resets all attributes to their default values. Note that when resources are not owned by the presentation driver, the driver saves the relevant values so that they are available to reset the device context. A typical example is when the default font is a graphics engine font. In this case, the presentation driver saves the font flags and address passed in the first call to GreDeviceSetAttributes.

The visible region and the DC origin are not affected by this function.

OS2_PM_DRV_ENABLE: Subfunction 09h - ResetDCState - Topic

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

OS2_PM_DRV_ENABLE: Subfunction 0Ah - CompleteOpenDC

OS2_PM_DRV_ENABLE: Subfunction 0Ah - CompleteOpenDC - S

Description:

OS2_PM_DRV_ENABLE: CompleteOpenDC is called upon completion of the DevOpenDC process to tell the presentation driver that the device context now has access to the graphics engine.

```
#include <os2.h>

ULONG     ulSubfunction;  /*  0AH. */
HDC       hdcParam1;
PLONG     pParam2;        /*  A pointer (pInstance) to the DC instance data. */
LONG      rc;             /*  Return codes. */

rc = CompleteOpenDC(ulSubfunction, hdcParam1,
       pParam2);
```

OS2_PM_DRV_ENABLE: Subfunction 0Ah - CompleteOpenDC Pa ulSubfunction

ulSubfunction (ULONG) - input
0AH.

OS2_PM_DRV_ENABLE: Subfunction 0Ah - CompleteOpenDC Pa

hdcParam1 (HDC) - input
A device context handle.

OS2_PM_DRV_ENABLE: Subfunction 0Ah - CompleteOpenDC Pa

pParam2 (PLONG) - input
A pointer (pInstance) to the DC instance data.

OS2_PM_DRV_ENABLE: Subfunction 0Ah - CompleteOpenDC Re

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

0	Successful
-1	Error

OS2_PM_DRV_ENABLE: Subfunction 0Ah - CompleteOpenDC - P

ulSubfunction (ULONG) - input
0AH.

hdcParam1 (HDC) - input
A device context handle.

pParam2 (PLONG) - input
A pointer (pInstance) to the DC instance data.

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

0	Successful
-1	Error

OS2_PM_DRV_ENABLE: Subfunction 0Ah - CompleteOpenDC - R

Presentation drivers for the primary display device return Successful without taking any action. For other devices, the handling routine in the presentation driver completes the initialization process for any resources, such as bit maps, that are obtained through calls to the graphics engine.

Hardcopy drivers do not use the CompleteOpenDC routine to open resources such as spool files or journal files. If these resources are required, they are opened in response to a call to GreEscape DEVESC_STARTDOC. Such drivers set a flag in the instance data to show that DEVESC_STARTDOC has been received, and do not process any output until that flag has been set.

Note: The presentation driver locks and unlocks resources such as bit maps and device contexts to prevent simultaneous use of the resource by two threads belonging to the same process. Typically, this is done by setting a semaphore or some other form of BUSY flag for the resource.

OS2_PM_DRV_ENABLE: Subfunction 0Ah - CompleteOpenDC - T

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

OS2_PM_DRV_ENABLE: Subfunction 0Bh - BeginCloseDC

OS2_PM_DRV_ENABLE: Subfunction 0Bh - BeginCloseDC - Synta

Description:

OS2_PM_DRV_ENABLE: BeginCloseDC is called to inform the presentation driver that the device context is being closed.

```
#include <os2.h>

ULONG     ulSubfunction;  /*  0Bh. */
HDC       hdcParam1;
PLONG     pParam2;        /*  A pointer (pInstance) to the DC instance data. */
LONG      rc;             /*  Return codes. */

rc = BeginCloseDC(ulSubfunction, hdcParam1,
       pParam2);
```

OS2_PM_DRV_ENABLE: Subfunction 0Bh - BeginCloseDC Param

ulSubfunction (ULONG) - input
0Bh.

OS2_PM_DRV_ENABLE: Subfunction 0Bh - BeginCloseDC Param

hdcParam1 (HDC) - input
A device context handle.

OS2_PM_DRV_ENABLE: Subfunction 0Bh - BeginCloseDC Param

pParam2 (PLONG) - input
A pointer (pInstance) to the DC instance data.

OS2_PM_DRV_ENABLE: Subfunction 0Bh - BeginCloseDC Returr

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

0	Successful
-1	Error

OS2_PM_DRV_ENABLE: Subfunction 0Bh - BeginCloseDC - Para

ulSubfunction (ULONG) - input
0Bh.

hdcParam1 (HDC) - input
A device context handle.

pParam2 (PLONG) - input
A pointer (pInstance) to the DC instance data.

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

0	Successful
-1	Error

OS2_PM_DRV_ENABLE: Subfunction 0Bh - BeginCloseDC - Rem

This is the last call made to the driver before it loses access to the graphics engine. Display drivers for the primary display device return Successful without taking any action. For other devices, the handling routine in the presentation driver has to close any resources (such as journal files and bit maps) that it owns.

Hardcopy drivers do not use the BeginCloseDC routine to complete tasks such as writing spool files. The tasks are completed in response to a call to GreEscape DEVESC_ENDDOC. The DEVESC_ENDDOC routine resets the DEVESC_STARTDOC flag in the instance data. The BeginCloseDC routine checks that the flag is reset before taking any action.

OS2_PM_DRV_ENABLE: Subfunction 0Bh - BeginCloseDC - Topic

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

OS2_PM_DRV_ENABLE: Subfunction 0Ch - InstallPMEngineDispl

OS2_PM_DRV_ENABLE: Subfunction 0Ch - InstallPMEngineDispl

Description:

OS2_PM_DRV_ENABLE: InstallPMEngineDisplayHooks provides the addresses of the graphics engine and display driver dispatch tables so that the dynamic link libraries for display hooks can intercept the dispatching of these functions for monitoring or other purposes.

```
#include <os2.h>
```

```
ULONG     ulSubfunction;  /*  0Ch. */
PLONG     pParam1;
PLONG     pParam2;
LONG      rc;             /*  Return codes. */

rc = InstallPMEngineDisplayHooks(ulSubfunction,
       pParam1, pParam2);
```

OS2_PM_DRV_ENABLE: Subfunction 0Ch - InstallPMEngineDispl ulSubfunction

ulSubfunction (ULONG) - input
0Ch.

OS2_PM_DRV_ENABLE: Subfunction 0Ch - InstallPMEngineDispl pParam1

pParam1 (PLONG) - input
A pointer to the following data structure:

ulVersion	Version number in binary coded decimal (BCD) of the graphics engine.
comptype	A structure consisting of two short flag variables:

fEngine32Bit	This flag will always be true.
fDriver32Bit	This flag is set if the current display driver is a 32-bit driver.

OS2_PM_DRV_ENABLE: Subfunction 0Ch - InstallPMEngineDispl pParam2

pParam2 (PLONG) - input
A pointer to the dispatch tables of the display driver and graphics engine:

pEngHooks	Pointer to the 32-bit entry points of the graphics engine.
pDrvHooks	Pointer to the entry points of the current display driver, which can consist of a table of 16-bit or 32-bit entry points:

```
typedef struct _CompType {
    USHORT fEngine32Bit;
    USHORT fDriver32Bit;
} COMPTYPE;
```

```
typedef struct _P1 {
```

```
    ULONG    version;
    COMPTYPE comptype;
} P1;
```

```
typedef struct _P2 {
    PLONG pEngTable;
    PLONG pDrvTable;
} P2;
```

OS2_PM_DRV_ENABLE: Subfunction 0Ch - InstallPMEngineDispl Value - rc

rc (LONG) - returns
Return codes.

The return value is 0 if the function is successful; otherwise, it is an error value.

If the error returned = PMERR_DRVR_NOT_SUPPORTED, the graphics engine will unload the hooking DLL and work as though no routine were installed. This would be the case if the hooking DLL only handled 32-bit entry points and received a display dispatch table of 16-bit entry points.

OS2_PM_DRV_ENABLE: Subfunction 0Ch - InstallPMEngineDispl Parameters

ulSubfunction (ULONG) - input
0Ch.

pParam1 (PLONG) - input
A pointer to the following data structure:

ulVersion	Version number in binary coded decimal (BCD) of the graphics engine.	
comptype	A structure consisting of two short flag variables:	
	fEngine32Bit	This flag will always be true.
	fDriver32Bit	This flag is set if the current display driver is a 32-bit driver.

pParam2 (PLONG) - input
A pointer to the dispatch tables of the display driver and graphics engine:

pEngHooks	Pointer to the 32-bit entry points of the graphics engine.
pDrvHooks	Pointer to the entry points of the current display driver, which can consist of a table of 16-bit or 32-bit entry points:

```
typedef struct _CompType {
    USHORT fEngine32Bit;
    USHORT fDriver32Bit;
} COMPTYPE;
```

```
typedef struct _P1 {
    ULONG    version;
    COMPTYPE comptype;
} P1;
```

```
typedef struct _P2 {
    PLONG pEngTable;
    PLONG pDrvTable;
} P2;
```

rc (LONG) - returns
Return codes.

The return value is 0 if the function is successful; otherwise, it is an error value.

If the error returned = PMERR_DRVR_NOT_SUPPORTED, the graphics engine will unload the hooking DLL and work as though no routine were installed. This would be the case if the hooking DLL only handled 32-bit entry points and received a display dispatch table of 16-bit entry points.

OS2_PM_DRV_ENABLE: Subfunction 0Ch - InstallPMEngineDispl

For a display hook's DLL to be installed, the name of the DLL must be added to OS2.INI with keyname = MODULENAME and application name = PM_ED_HOOKS.

The graphics engine will load the DLLs so identified in OS2.INI in turn, from left to right, so that the rightmost name will be the last display hook's DLL offered access to the dispatch tables.

Display hook DLLs need to be Ring-2-conforming. Support for 16-bit drivers runs at Ring 2. OS/2 2.0 32-bit drivers have a choice to run predetermined calls at Ring 2.

OS2_PM_DRV_ENABLE: Subfunction 0Ch - InstallPMEngineDispl

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

OS2_PM_DRV_ENABLE: Subfunction 0Eh - QueryDeviceSurface

OS2_PM_DRV_ENABLE: Subfunction 0Eh - QueryDeviceSurface

Description:

OS2_PM_DRV_ENABLE: QueryDeviceSurface provides the needed information to the graphics engine for handling the new Device Driver Interface (DDI) architecture.

```
#include <os2.h>

ULONG    ulSubfunction;  /*  0Eh. */
PLONG    pParam1;        /*  Pointer to the DEVICESURFACE data structure. */
LONG     pParam2;        /*  Long integer returned from FillPhysicalDeviceBlock. */
LONG     rc;             /*  Return codes. */

rc = QueryDeviceSurface(ulSubfunction, pParam1,
       pParam2);
```

OS2_PM_DRV_ENABLE: Subfunction 0Eh - QueryDeviceSurface | ulSubfunction

ulSubfunction (ULONG) - input
0Eh.

OS2_PM_DRV_ENABLE: Subfunction 0Eh - QueryDeviceSurface |

pParam1 (PLONG) - input
Pointer to the DEVICESURFACE data structure.

Points to information needed by the graphics engine.

ulLength	Length of the DEVICESURFACE data structure, in bytes.
SurfaceBmapInfo	Information used by the graphics engine when rastering to a linear address. (See BMAPINFO in Data Types.).
ulDSFlgs	Miscellaneous flags used by the graphics engine for rendering of rasterized data.

Color format flag

DS_MONO_INVERT	Mono ROPs are inverted.

Destination color format flags

DS_COLOR_RGB	Color is in RGB.
DS_COLOR_RGB2	Color is in RGB2 (default).
DS_COLOR_24BIT_BGR	First byte=blue (default).
DS_COLOR_24BIT_RGB	First byte=red, second byte=green, third byte=blue.
DS_COLOR_16BIT_565_BGR	

Blue

5 bits (default)

Green

6 bits

Red

5 bits

|b|b|b|b|b|g|g|g|g|g|g|r|r|r|

DS_COLOR_16BIT_565_RGB

Red

5 bits

Green

6 bits

Blue

5 bits

|r|r|r|r|r|g|g|g|g|g|g|b|b|b|

DS_COLOR_16BIT_1555_BGR

Overlay

1

Blue

5

b
i
t
s
Green

5
b
i
t
s
Red

5
b
i
t
s

|o|b|b|b|b|b|g|g|g|g|g|r|r|r|

DS_COLOR_16BIT_1555_RGB

Overlay

1
Red

5
b
i
t
s
Green

5
b
i
t
s
Blue

5
b
i
t
s

|o|r|r|r|r|r|g|g|g|g|g|b|b|b|

Direction flags

DS_BOTTOMTOP
Bottom-to-top is Y position.

DS_TOPBOTTOM
Top-to-bottom is Y position.

Hardcopy flag

DS_QUEUED_RAW_DEFAULT
Set QUEUED to RAW as default.

Notification flag

DS_NOTIFY_LAST_SDBITBLT
Last banded blit per API bit map transfer.

Scanline alignment flags

DS_BYTE_ALIGNMENT
: DS scan line alignment.

DS_2BYTE_ALIGNMENT
: DS scan line alignment.

DS_4BYTE_ALIGNMENT
: DS scan line alignment.

DS_KEEP_EXTFORMAT
: Variable alignment; let application decide.

Device Font Flags

Normal output to the device from the graphics subsystem is generated from the device origin with scan lines horizontal and incrementing both pels and scan lines. This output can be altered by the following flags:

DS_BITBLT_XROW_YSCAN
: Generates row data as sequential bit stream instead of scan line data.

DS_BITBLT_ROW_NEGATIVE
: Generates rows in opposite direction, for example, Start at Device Origin + Device Height.

DS_BITBLT_SCAN_NEGATIVE
: Generates scan lines in opposite direction, for example, Start at Device Origin + Device Length.

Output Surface Direction Flags

As of OS/2 Warp Version 4, the graphics subsystem added device font support for the new device driver model. Below are some of the device surface flags to support it:

DS_DOWNLOAD_FONTS
: (GRE_VERSION 234) - The device request downloading of system fonts via pfnTextBlt in new cached format.

DS_DEVICE_FONTS
: (GRE_VERSION 234) - The device has its own hardware fonts. It will provide font definition information, but wants the system to handle all query and character string operations. It will render fonts via the pfnTextBlt call.

DS_FONT_CLIP_SUPPORT
: (GRE_VERSION 238) - In Gre Version 234, only the precise rule is used.

DS_CLIP_100_RULE
: Graphics subsystem will not send the glyph to the driver unless it is fully visible.

DS_CLIP_66_RULE
: Graphics subsystem will only send the glyph to the driver if it is at least 66 % visible.

DS_CLIP_PRECISE_RULE
: Graphics subsystem will send the all glyphs to the driver with all the current clip rectangles.

Surface description flag

DS_MEMORY
: Memory surface.

ulStyleRatio — Default style ratio used by the current device.

abmapinfoDefPattern[ ] — Set of default patterns supplied by the graphics engine that can be changed by the presentation driver. [DEFAULT_PATTERNS_NUMBER].

pHWPalette

Default hardware palette for the current device.

DitherMatrix

Default is the generic dither support for hardcopy devices.

For additional details on dithering refer to the section, Dithering / Color Mapping Interface.

ulLength
: Length of dither matrix structure.

fExt2IntSup
: Gre dither flags for external to internal.

pExt2IntDI
: Used for GDM_USERDEF. Pointer to users dither information.

fStretchSup
: Gre dither flags for stretch support.

pStretchDI
: Used for GDM_USERDEF. Pointer to users dither information.

fLog2PhysSup
: Gre dither flags for logical to physical color mapping.

pLog2PhysDI
: Used for GDM_USERDEF. Pointer to users dither information.

pfnDither
: Used for GDM_USERDEF. Pointer to users dither function.

DitherMatrix Flags:

GDM_NO_DITHER
: No dithering in system.

GDM_USERDEF_DITHER
: Users supplies own dithering routines.

GDM_MATRIX_DITHER
: Use system-ordered dithering.

GDM_ERRORDIF_DITHER
: Use system error diffusion dithering.

GDM_DITHER_BEGIN
: Used with dithering that needs to save data across output bands of an image.

GDM_DITHER_END
: Used with dithering that needs to save data across output bands of an image.

GDM_COLOR_CONVERT
: Use device's color mapping functions.

pfnDevLockDC

Pointer to allow the device driver to retrieve its device-specific information from the graphics engine. Pointer is stored by the device driver.

pfnDevUnLockDC

Pointer to the function to release the lock on the current DC. Pointer is stored by the device driver.

pfnBitBlt

Pointer defaults to SOFTDRAW's handling of the new SDBitBlt function. If a device does not have linear address support or would rather handle preclipped bit map transfers itself, then it should hook out this function with its own routine.

pfnLine

Pointer defaults to SOFTDRAW's handling of the new SDLine function. If a device does not have linear address support or would rather handle preclipped line operations itself, then it should hook out this function with its own routine.

pfnTextBlt

Used in conjunction with DS_DOWNLOAD_FONTS and DS_DEVICE_FONTS. If flags are not specified, text strings will be simulated through bitblts.

Refer to Device Font Support for additional details.

pfnLockPhysDev

Pointer prevents other processes from accessing the physical hardware between the lock and unlock.

pfnUnLockPhysDev

Pointer releases the physical hardware for access by other processes.

ulhddc

Handle to device's device context.

ulReserved[4]	Reserved by system.
ulCapsCnt	Size in ULONGs of the *DevCaps* field.
DevCaps[ ]	The device capabilities array in OS/2 2.0 and previous versions. Array must be filled out in order for the graphics engine to handle a call to the DevQueryCaps function. [CAPS_MAX_CAPS + 1]

OS2_PM_DRV_ENABLE: Subfunction 0Eh - QueryDeviceSurface

pParam2 (LONG) - input
Long integer returned from FillPhysicalDeviceBlock.

OS2_PM_DRV_ENABLE: Subfunction 0Eh - QueryDeviceSurface

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

Zero (0)	Success
- 1	Error

If *pParam1* is a null pointer, return a zero (0) if you are requesting new graphics engine support; otherwise, return a - 1.

OS2_PM_DRV_ENABLE: Subfunction 0Eh - QueryDeviceSurface

ulSubfunction (ULONG) - input
0Eh.

pParam1 (PLONG) - input
Pointer to the DEVICESURFACE data structure.

Points to information needed by the graphics engine.

ulLength	Length of the DEVICESURFACE data structure, in bytes.
SurfaceBmapInfo	Information used by the graphics engine when rastering to a linear address. (See BMAPINFO in Data Types.).
ulDSFlgs	Miscellaneous flags used by the graphics engine for rendering of rasterized data.

Color format flag	
DS_MONO_INVERT	Mono ROPs are inverted.
Destination color format flags	
DS_COLOR_RGB	Color is in RGB.

DS_COLOR_RGB2

Color is in RGB2 (default).

DS_COLOR_24BIT_BGR

First byte=blue (default).

DS_COLOR_24BIT_RGB

First byte=red, second byte=green, third byte=blue.

DS_COLOR_16BIT_565_BGR

Blue

5 bits (default)

Green

6 bits

Red

5 bits

|b|b|b|b|b|g|g|g|g|g|g|r|r|r|

DS_COLOR_16BIT_565_RGB

Red

5 bits

Green

6 bits

Blue

5 bits

|r|r|r|r|r|g|g|g|g|g|g|b|b|b|

DS_COLOR_16BIT_1555_BGR

DS_COLOR_16BIT_1555_RGB

Direction flags

DS_BOTTOMTOP

Bottom-to-top is Y position.

DS_TOPBOTTOM

Top-to-bottom is Y position.

Hardcopy flag

DS_QUEUED_RAW_DEFAULT
: Set QUEUED to RAW as default.

Notification flag

DS_NOTIFY_LAST_SDBITBLT
: Last banded blit per API bit map transfer.

Scanline alignment flags

DS_BYTE_ALIGNMENT
: DS scan line alignment.

DS_2BYTE_ALIGNMENT
: DS scan line alignment.

DS_4BYTE_ALIGNMENT
: DS scan line alignment.

DS_KEEP_EXTFORMAT
: Variable alignment; let application decide.

Device Font Flags

Normal output to the device from the graphics subsystem is generated from the device origin with scan lines horizontal and incrementing both pels and scan lines. This output can be altered by the following flags:

DS_BITBLT_XROW_YSCAN
: Generates row data as sequential bit stream instead of scan line data.

DS_BITBLT_ROW_NEGATIVE
: Generates rows in opposite direction, for example, Start at Device Origin + Device Height.

DS_BITBLT_SCAN_NEGATIVE
: Generates scan lines in opposite direction, for example, Start at Device Origin + Device Length.

Output Surface Direction Flags

As of OS/2 Warp Version 4, the graphics subsystem added device font support for the new device driver model. Below are some of the device surface flags to support it:

DS_DOWNLOAD_FONTS
: (GRE_VERSION 234) - The device request downloading of system fonts via pfnTextBlt in new cached format.

DS_DEVICE_FONTS
: (GRE_VERSION 234) - The device has its own hardware fonts. It will provide font definition information, but wants the system to handle all query and character string operations. It will render fonts via the pfnTextBlt call.

DS_FONT_CLIP_SUPPORT
: (GRE_VERSION 238) - In Gre Version 234, only the precise rule is used.

DS_CLIP_100_RULE
: Graphics subsystem will not send the glyph to the driver unless it is fully visible.

DS_CLIP_66_RULE
: Graphics subsystem will only send the glyph to the driver if it is at least 66% visible.

DS_CLIP_PRECISE_RULE
: Graphics subsystem will send the all glyphs to the driver with all the current clip rectangles.

Surface description flag

DS_MEMORY
Memory surface.

ulStyleRatio
Default style ratio used by the current device.

abmapinfoDefPattern[ ]
Set of default patterns supplied by the graphics engine that can be changed by the presentation driver. [DEFAULT_PATTERNS_NUMBER].

pHWPalette
Default hardware palette for the current device.

DitherMatrix
Default is the generic dither support for hardcopy devices.

For additional details on dithering refer to the section, Dithering / Color Mapping Interface.

ulLength
Length of dither matrix structure.

fExt2IntSup
Gre dither flags for external to internal.

pExt2IntDI
Used for GDM_USERDEF. Pointer to users dither information.

fStretchSup
Gre dither flags for stretch support.

pStretchDI
Used for GDM_USERDEF. Pointer to users dither information.

fLog2PhysSup
Gre dither flags for logical to physical color mapping.

pLog2PhysDI
Used for GDM_USERDEF. Pointer to users dither information.

pfnDither
Used for GDM_USERDEF. Pointer to users dither function.

DitherMatrix Flags:

GDM_NO_DITHER
No dithering in system.

GDM_USERDEF_DITHER
Users supplies own dithering routines.

GDM_MATRIX_DITHER
Use system-ordered dithering.

GDM_ERRORDIF_DITHER
Use system error diffusion dithering.

GDM_DITHER_BEGIN
Used with dithering that needs to save data across output bands of an image.

GDM_DITHER_END
Used with dithering that needs to save data across output bands of an image.

GDM_COLOR_CONVERT
Use device's color mapping functions.

pfnDevLockDC
Pointer to allow the device driver to retrieve its device-specific information from the graphics engine. Pointer is stored by the device driver.

pfnDevUnLockDC
Pointer to the function to release the lock on the current DC. Pointer is stored by the device driver.

pfnBitBlt
Pointer defaults to SOFTDRAW's handling of the new SDBitBlt function. If a device does not have linear address support or would rather handle preclipped bit map transfers itself, then it should hook out this function with its own routine.

pfnLine
Pointer defaults to SOFTDRAW's handling of the new SDLine function. If a device does not have linear address support or would rather handle preclipped line operations itself, then it should hook out this function with its own routine.

pfnTextBlt
Used in conjunction with DS_DOWNLOAD_FONTS and

	DS_DEVICE_FONTS. If flags are not specified, text strings will be simulated through bitblts. Refer to Device Font Support for additional details.
pfnLockPhysDev	Pointer prevents other processes from accessing the physical hardware between the lock and unlock.
pfnUnLockPhysDev	Pointer releases the physical hardware for access by other processes.
ulhddc	Handle to device's device context.
ulReserved[4]	Reserved by system.
ulCapsCnt	Size in ULONGs of the *DevCaps* field.
DevCaps[ ]	The device capabilities array in OS/2 2.0 and previous versions. Array must be filled out in order for the graphics engine to handle a call to the DevQueryCaps function. [CAPS_MAX_CAPS + 1]

pParam2 (LONG) - input
Long integer returned from FillPhysicalDeviceBlock.

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

Zero (0)	Success
- 1	Error

If *pParam1* is a null pointer, return a zero (0) if you are requesting new graphics engine support; otherwise, return a - 1.

OS2_PM_DRV_ENABLE: Subfunction 0Eh - QueryDeviceSurface

For display drivers, the system layer is responsible for handling the QueryDeviceSurface subfunction call. This call is made from the graphics engine during an OpenDC request.

Dithering / Color Mapping Interface

This is the interface to let devices provide their own color mapping and dithering routines, that is, override the graphic system's defaults or provide routines for formats not supported by the graphics system (for example, a 32-bit 10R 10G 10B format for instance).

Notes

1. A device can specify a logical color format (via Device Capabilities) which is different from their physical color format (Device Surface ulBpp).
2. External format is the format in which an OS/2 application provides images. 1/4/8/24 Bpp, BGR, doubleword aligned. Internal format is the format that the application image is converted into relative to the device's logical format.

A device has the option to dither at external to internal conversion time (this is the only option for color conversion) or at logical to physical.

Note: A device can specify KEEP_EXTERNAL_FORMAT, in which case an external to physical conversion could take place. This should not matter to the conversion-handling routine because all the information in the image information structure should suffice in performing the conversion.

You can:

- Select dithering.
- Modify system dithering routines.
- Provide your own dithering routines.

- Provide support for unsupported system formats.

Selecting Dithering

Dithering is selected via:

- GDM_EXT2INT_DITHER
- GDM_LOG2PHY_DITHER

Add the appropriate GDM_DITHER flag into the fExt2IntSup and/or fLog2PhysSup fields (dither could be done at both levels).

Modifying System Dithering Routines

You can change just the dithering defaults for that requested type of dithering without providing funtion. To accomplish that, provide either a pExt2IntDI or a pLog2PhysDI structure.

Note: The whole structure must be filled out.

Below are two currently used system dither structures:

```
MATRIXDITHERINFO                  System Default Values

  ULONG       ulLength            sizeof(MATRIXDITHERINFO)
  ULONG       ulType              GDM_MATRIX_DITHER
  ULONG       fOptions            00
  ULONG       ulIntensity         00
  BYTE        bRedWt              25
  BYTE        bGreenWt            60
  BYTE        bBlueWt             15
  BYTE        bPad                00
  SIZEL       szMatrix            N/A
  BYTE        bHalftone[1]        N/A

DIFFUSIONDITHERINFO

  ULONG       ulLength            sizeof(DIFFUSIONDITHERINFO)
  ULONG       ulType              GDM_ERRORDIF_DITHER
  ULONG       fOptions            JJN_DIF
  ULONG       ulIntensity         00
  BYTE        bRedWt              25
  BYTE        bGreenWt            60
  BYTE        bBlueWt             15
  BYTE        bPad                00
  PBYTE       pBuffer             N/A
  SIZEL       szFilter            N/A
  BYTE        bFilterArry[1]      N/A

JJN_DIF        Currently only one supported
```

Providing Your Own Dithering Routines

To provide your own dither routines, place the GDM_USERDEF_DITHER in either the fExt2IntSup and/or fLog2PhysSup fields. Then provide a function for pfnDither:

```
pfnDither(PDITHERINFO pdi, PIMAGEINFO pii, ULONG ulFlags);
```

Where pdi = either pExt2IntDI or pLog2PhysDI. This should be your own allocated structure. For pii, see the detailed information about the IMAGEINFOS structure which follows.

Providing Support for Unsupported System Formats

To provide your own color conversion routines, place the GDM_COLOR_CONVERT in the fExt2IntSup field, and provide the three required functions in the COLORCONVERTINFO structure:

```
pfnRGB2ToPhyIndex   (ULONG ulClr)

pfnPhyIndexToRGB2   (ULONG ulClr)

pfnConvert          (PIMAGEINFO pimginfo)
```

Data Structures

```
COLORCONVERTINFO
```

```
  ULONG       ulLength                          sizeof(COLORCONVERTINFO)
  ULONG       ulType                            GDM_COLOR_CONVERT
  ULONG       ulOptions                         00
  ULONG       (* APIENTRY pfnRGB2ToPhyIndex)  () N/A - Mapping from RGB to Phy index
  ULONG       (* APIENTRY pfnPhyIndexToRGB2)  () N/A - Mapping from Phy index to RGB
  ULONG       (* APIENTRY pfnConvert)         () N/A - Actual Convert Bits Function

IMAGEINFOS

  PPOINTL     pptlSrc
  PPOINTL     pptlDst
  ULONG       ulSrcBpp               Source image bits per pixel
  ULONG       ulSrcBytesPerLine      Count of bytes per scan line (includes padding)
  PBYTE       pbSrcBits              Pointer to source data
  ULONG       ulcSrcClrs             Count of source colors
  ULONG       ulSrcClrType           Source color format see below
  PRGB2       pargb2Src              Pointer to source color table
  ULONG       ulTrgBpp               Target image bits per pixel
  ULONG       ulTrgBytesPerLine      Count of bytes per scan line (includes padding)
  PBYTE       pbTrgBits              Pointer to target data
  ULONG       ulcTrgClrs             Count of target colors
  ULONG       ulTrgClrType           Source target format see below
  PRGB2       pargb2Trg              Pointer to target color table
  ULONG       ulOptions              see below
```

IMAGEINFOS Color Types

When ulBpp <= 8:

```
  COLOR_RGB2            Color is in RGB2  (default)
  COLOR_RGB             Color is in RGB
```

When ulBpp = 24:

```
  COLOR_24BIT_BGR       1st byte=blue, 2nd byte=green 3rd byte=red  (default)
  COLOR_24BIT_RGB       1st byte=red,  2nd byte=green 3rd byte=blue
```

When ulBpp = 16:

```
  COLOR_16BIT_565_BGR   blue - 5 bits, green - 6 bits, red - 5 bits   (default)
                        ---------------------------------
                        |b|b|b|b|b|g|g|g|g|g|g|r|r|r|r|r|
                        ---------------------------------
  COLOR_16BIT_565_RGB   red - 5 bits, green - 6 bits, blue - 5 bits
                        ---------------------------------
                        |r|r|r|r|r|g|g|g|g|g|g|b|b|b|b|b|
                        ---------------------------------
  COLOR_16BIT_1555_BGR  overlay - 1, red - 5 bits, green - 5 bits, blue - 5 bits
                        ---------------------------------
                        |o|b|b|b|b|b|g|g|g|g|g|r|r|r|r|r|
                        ---------------------------------
  COLOR_16BIT_1555_RGB  overlay - 1, red - 5 bits, green - 5 bits, blue - 5 bits
                        ---------------------------------
                        |o|r|r|r|r|r|g|g|g|g|g|b|b|b|b|b|
                        ---------------------------------
```

When ulBpp = 32:

```
  COLOR_32BIT_BGR       1st byte=blue, 2nd byte=green 3rd byte=red,  4th byte=not used
  COLOR_32BIT_RGB       1st byte=red,  2nd byte=green 3rd byte=blue, 4th byte=not used
```

IMAGEINFO Options

```
  IMAGEINFO_CLR_TO_MONO_BG_EXACT     Src pels that are trg BG clr -> 1
                                     all other pels               -> 0
  IMAGEINFO_CLR_TO_MONO_BG_EXACT_INV Src pels that are trg BG clr -> 0
                                     all other pels               -> 1
                                     Nearest color match.
                                       Source pels that are close to target BG
                                       color become target BG color.
                                       All other pels become target FG color.
  IMAGEINFO_CLR_TO_MONO_BG_MAP       Src pels that are close to trg BG clr -> 1
                                     all other pels                        -> 0
  IMAGEINFO_CLR_TO_MONO_BG_MAP_INV   Src pels that are close to trg BG clr -> 0
                                     all other pels                        -> 1
  IMAGEINFO_SRC_Y_FLIP               Source is going Y negative direction
```

```
                                        i.e. second line = first line - ulBytesPerLine.
IMAGEINFO_TRG_Y_FLIP                Target is going Y negative direction
```

Device Font Support

Devices now have a couple of new font options. One option is to have the system download its own fonts to the device driver. The other option is to let the system manage text queries and strings while letting the device actually render the images.

Download Fonts

Two things must be done to tell the graphics subsystem that your device driver wants to be font-download enabled:

1. The DS_DOWNLOAD_FONTS flag must be set in the device surface flags.

2. The device driver must handle the pfnTextBlt function and set the respective pointer in the device surface.

Once this has been done, the graphics subsystem sends all text output to the device driver via pfnTextBlt.

```
APIRET APIENTRY pfnTextBlt(PTEXTBLTINFO ptbi, ULONG hddc);

TEXTBLTINFO

   ULONG          ulLength        sizeof(TEXTBLTINFO)
   ULONG          flOptions       00
   ULONG          lGlyphCnt       Count of Glyphs to render
   PLONG          pGlyphIndices   pointer to glyph indicies
   ULONG          ulFGMix         Foreground mix mode
   ULONG          ulBGMix         Background mix mode
   ULONG          ulFGColor       Foreground text color
   ULONG          ulBGColor       Background text color
   PBMAPINFO      pDstBmapInfo    Destination bitmap information
   PDEVFONTINFO   pDevFntInfo     Device font information
   ULONG          ulClpCnt        Non-Clipping Drivers Ignore
   PBLTRECT       abrClipRects    Non-Clipping Drivers Ignore
   PPOINTL        aptlSrcOrg      Clipping Drivers Ignore
   PBLTRECT       abrDst          Clipping Drivers Ignore .ulXExt & ulYExt

DEVFONTINFO

  ULONG          ulFntCnt        Maximum glyphs contained in this font
  ULONG          fFontInfo       Flags
  ULONG          ulEngTag        Renderer Tag - i.e. 'GRE'
  ULONG          ulUniqueFntID   Unique font ID
  ULONG          ulMaxHeight     Font maximum height
  ULONG          ulMaxWidth      Font maximum width
  PGHBTBL        pghbTbl         Pointer to glyph high-byte table see below
  ULONG          ulHalfWidth     N/A
  CHAR           szGlyphlist[16] Glyph list name (ie. PM383)
  ULONG          ulReserved1     N/A
  ULONG          ulReserved2     N/A

DFI_FIXED_FONT         Font is a fixed-width font
DFI_DBCS_FONT          Font is a DBCS font

GRETAG "GRE_"          Font is a PM system style font
WINTAG "WIN_"          Font is a WIN system style font
```

The Glyph Table is designed to be a dynamic 2-byte font table. As an example, lets take a PM383 font with three cached entries, index 34, 36 and 301:

```
      GHBTBL

0   pglbtbl1                                    >  A Space

1   pglbtbl2                                       B Space

2                                                  C Space

n                                                  Pad

                                                   Bitmapinfo
```

```
                        GLBTBL

                    >34   pGlyphInfo
                                                    >  A Space
                     35
                                                       B Space
                    >36   pGlyphInfo
                                                       C Space
                     37
                                                       Pad

                          GLBTBL                       Bitmapinfo

                     44

                    >45   pGlyphInfo                     >  A Space
                     46                                     B Space
                     47                                     C Space
                                                            Pad
                                                            Bitmapinfo
```

```
GHBTBL

  PGLBTBL         pglbTbl[1]      Up to 256 entries per table


GLBTBL
  PGLYPHINFO      pGlyphInfo[1]   Up to 256 entries per table


GLYPHINFO

  char      bAspace
  char      bBspace
  char      bCspace
  char      bPad
  BMAPINFO  bmapinfo
```

Device Fonts Handled By The System

Here again, the DS_DEVICE_FONTS flag needs to be set and the pfnTextBlt function must be hooked out and handled by the device driver. In order for the system to handle font queries and text positioning of the device font, the system will query the fonts and font attributes for all the supported device fonts.

The system will need to know what fonts the driver wants supported. The system will query the driver the first time a device font request is initiated via the following new dispatch entry:

```
APIRET APIENTRY QueryDeviceFontMetrics(HDC             hdc,
                                       PDEVFONTMETRICS abDFM,
                                       ULONG           lDFMFlags,
                                       ULONG           hddc,
                                       ULONG           ulFunN)

return: Count of device fonts


DEVFONTMETRICS

   ULONG         flOptions               Device font metrics options
   PFOCAFONT     pFoca                   pointer to focafont information
   PVOID         pUniPanhose             pointer to panhose info for uni fonts
   char          achFileName[260]        if pFoca is null then font file name to use
   PSZ           pszFullFamilyName       pointer family name null if pFoca null
   PSZ           pszFullFaceName         pointer to face name null if pFoca null
   char          achGlyphListName[16]    glyph list name
   PFN           pfnDevQueryCodePage     pointer to code page functionality (defaulted to PM383 )
   LONG          lDevMatch               Devices lmatch number (negative)

Options for flOptions of DEVFONTMETRICS

DEVFONT_DEFAULT           selects which font in the array of DEVFONTMETRICS
                          is the device's default font.
```

OS2_PM_DRV_ENABLE: Subfunction 0Eh - QueryDeviceSurface ·

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

OS2_PM_DRV_ENABLE_LEVELS

This entry point is the address of a table of ring levels required when calling each of the Enable subfunctions. The table is an array of bytes corresponding to the subfunction numbers, terminating with a byte of 0 (that is, an ASCIIZ string). The most significant six bits of each byte are reserved and must be 0. The remaining two bits of each byte represent the ring level to be used when dispatching the corresponding function in the dispatch table.

```
00 = End of Table          10 = Ring 2
01 = Ring-2-Conforming     11 = Ring 3
```

Any subfunction that does not have a corresponding byte in the table will be dispatched as Ring-2-conforming. This is the most desirable case from the standpoint of system performance.

Subfunction	Ring Level
Unused	0x01
FillLogicalDeviceBlock	0x01
FillPhysicalDeviceBlock	0x03
Unused	0x01
DisablePhysicalDeviceBlock	0x02
All others	0x00

The following example of six bytes would declare FillPhysicalDeviceBlock as Ring 3, DisablePhysicalDeviceBlock as Ring 2, and all other subfunctions as Ring-2-conforming:

```
0x01    0x01    0x03    0x01    0x02    0x00
```

If the address of this table is not exported, all subfunctions will be called as Ring-2-conforming as if the table had contained the single byte 0x00.

OS2_PM_DRV_RING_LEVELS

This entry point is the address of a table of ring levels required when dispatching each of the functions hooked in the dispatch table by FillLogicalDeviceBlock. The table is an array of bytes corresponding to the functions in the dispatch table, terminating with a byte of 0 (that is,

an ASCIIZ string). The most significant six bits of each byte are reserved and must be 0. The remaining two bits of each byte represent the ring level to be used when dispatching the corresponding function in the dispatch table.

```
00 = End of Table          10 = Ring 2
01 = Ring-2-Conforming     11 = Ring 3
```

Any function that does not have a corresponding byte in the table will be dispatched as Ring-2-conforming. This is the most desirable case from the standpoint of system performance.

Function	Ring Level
GreGetArcParameters	0x01
GreSetArcParameters	0x01
GreArc	0x03
GrePartialArc	0x02
All others	0x00

The following example of five bytes would declare GreArc as Ring 3, GrePartialArc as Ring 2, and all other functions as Ring-2-conforming:

```
0x01    0x01    0x03    0x02    0x00
```

If the address of this table is not exported, all functions will be dispatched as Ring-2-conforming as if the table had contained the single byte 0x00.

OS2_PM_DRV_QUERYSCREENRESOLUTIONS

OS2_PM_DRV_QUERYSCREENRESOLUTIONS - Syntax

OS2_PM_DRV_QUERYSCREENRESOLUTIONS allows the operating system to display to the user the display modes that are supported by the display driver.

```
#include <os2.h>

PVOID     pBuf;     /*  Pointer to the output buffer. */
PULONG    pcbBuf;   /*  Pointer to the number of bytes in the buffer. */
LONG      rc;       /*  Return codes */

rc = OS2_PM_DRV_QUERYSCREENRESOLUTIONS(
      pBuf, pcbBuf);
```

OS2_PM_DRV_QUERYSCREENRESOLUTIONS Parameter - pBuf

pBuf (PVOID) - input
Pointer to the output buffer.

OS2_PM_DRV_QUERYSCREENRESOLUTIONS Parameter - pcbE

pcbBuf (PULONG) - input
Pointer to the number of bytes in the buffer.

pBuf — Pointer to the buffer that receives the output from this function. The output is returned in a SCREENRESCOUNT structure followed by an array of SCREENRESOLUTION structures. *pBuf* should contain the following SCREENRESCOUNT information:

maxcount	Total number of screen resolutions supported.
count	Number of SCREENRESOLUTION structures returned. This count will be less than maxcount if the defined size of the *pBuf* buffer is too small as identified by *pcbBuf*.
res_struct_length	Length of a SCREENRESOLUTION structure. This value should be used to increment between structures so that any future increase in the size of the structure (to add information) will not cause a failure.

The SCREENRESCOUNT information should be followed by the SCREENRESOLUTION information:

width	Width of the device in pels.	
height	Height of the device in pels.	
colors	Number of colors supported in this mode.	
planes	Number of display planes in this mode.	
floptions	Optional information for each resolution. Valid values include:	
	DSP_RESOLUTION_OBTAINABLE	Obtainable with the hardware configuration
	DSP_RESOLUTION_DEFAULT	Default resolution for the hardware configuration

pcbBuf — Pointer to the number of bytes in the *pScreenResolution* buffer. If *pScreenResolution* is 0 on input, this function stores the size (in bytes) needed to retrieve all of the screen resolution data in *pcbScreenResolution*. If not 0, this field contains the number of bytes actually returned.

OS2_PM_DRV_QUERYSCREENRESOLUTIONS Return Value - rc

rc (LONG) - returns
Return codes

The returned value depends upon the input value of *pcbBuf*.

0	Return the size in bytes needed to retrieve all of the screen resolution data.
Nonzero	Return the number of bytes actually returned in pBuf.

OS2_PM_DRV_QUERYSCREENRESOLUTIONS - Parameters

pBuf (PVOID) - input
Pointer to the output buffer.

pcbBuf (PULONG) - input
Pointer to the number of bytes in the buffer.

pBuf — Pointer to the buffer that receives the output from this function. The output is returned in a SCREENRESCOUNT structure followed by an array of SCREENRESOLUTION structures. *pBuf* should contain the following SCREENRESCOUNT information:

maxcount	Total number of screen resolutions supported.
count	Number of SCREENRESOLUTION structures returned. This count will be less than maxcount if the defined size of the *pBuf* buffer is too small as identified by *pcbBuf*.
res_struct_length	Length of a SCREENRESOLUTION structure. This value should be used to increment between structures so that any future increase in the size of the structure (to add information) will not cause a failure.

The SCREENRESCOUNT information should be followed by the SCREENRESOLUTION information:

width	Width of the device in pels.	
height	Height of the device in pels.	
colors	Number of colors supported in this mode.	
planes	Number of display planes in this mode.	
floptions	Optional information for each resolution. Valid values include:	
	DSP_RESOLUTION_OBTAINABLE	Obtainable with the hardware configuration
	DSP_RESOLUTION_DEFAULT	Default resolution for the hardware configuration

pcbBuf — Pointer to the number of bytes in the *pScreenResolution* buffer. If *pScreenResolution* is 0 on input, this function stores the size (in bytes) needed to retrieve all of the screen resolution data in *pcbScreenResolution*. If not 0, this field contains the number of bytes actually returned.

rc (LONG) - returns
Return codes

The returned value depends upon the input value of *pcbBuf*.

0	Return the size in bytes needed to retrieve all of the screen resolution data.
Nonzero	Return the number of bytes actually returned in pBuf.

OS2_PM_DRV_QUERYSCREENRESOLUTIONS - Remarks

Support: This entry point should be exported by display drivers that support multiple display resolutions.

OS2_PM_DRV_QUERYSCREENRESOLUTIONS - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

Mandatory and Simulated Graphics Engine Function Reference

All mandatory and simulated Presentation Manager functions are listed alphabetically in the following table. The two rightmost columns indicate whether a function is required for Hardcopy or Display drivers, or for both. A detailed description of each function follows the table. For ease of recognition, mandatory functions also include at the top of the page an icon of a printer, a display, or both, as appropriate.

Function	Type	Hard Copy	Display
GreAccumulateBounds	mandatory	x	x
GreArc	simulated (arc function)		
GreAreaSetAttributes	simulated (area function)		
GreBeginArea	simulated (area function)		
GreBeginPath	simulated (arc function)		
GreBitblt	mandatory	x	x
GreBoxBoth	simulated (arc function)		
GreBoxBoundary	simulated (arc function)		
GreBoxInterior	simulated (area/path function)		
GreCharRect	mandatory (AVIO function)		x
GreCharStr	mandatory (AVIO function)		x
GreCharStr	mandatory	x	x
GreCharStringPos	mandatory	x	x
GreCloseFigure	simulated (arc function)		
GreCombineRectRegion	simulated (region function)		
GreCombineRegion	simulated (region function)		
GreCombineShortLineRegion	simulated (region function)		
GreConvert	simulated (transform function)		
GreConvertWithMatrix	simulated (transform function)		
GreCopyClipRegion	simulated (clip function)		
GreCreateLogColorTable	mandatory	x	x

GreCreateRectRegion	simulated (region function)		
GreDeath	mandatory (display devices) device function 3		x
GreDestroyRegion	simulated (region function)		
GreDeviceAnimatePalette	simulated (transform function)		
GreDeviceCreateBitmap	mandatory	x	x
GreDeviceCreatePalette	simulated (transform function)		
GreDeviceDeleteBitmap	mandatory	x	x
GreDeviceDeletePalette	simulated (palette manager)		
GreDeviceGetAttributes	mandatory	x	x
GreDeviceInvalidateVisRegion	mandatory (display devices) device function 2		x
GreDeviceQueryFontAttributes	mandatory	x	x
GreDeviceQueryFonts	mandatory	x	x
GreDeviceResizePalette	simulated (palette manager function)		
GreDeviceSelectBitmap	mandatory	x	x
GreDeviceSetAttributes	mandatory	x	x
GreDeviceSetAVIOFont	mandatory (display devices) AVIO function		x
GreDeviceSetCursor	mandatory (display devices) bit map function		x
GreDeviceSetDCOrigin	mandatory	x	x
GreDeviceSetGlobalAttribute	mandatory	x	x
GreDeviceSetPaletteEntries	simulated (palette manager function)		
GreDisjointLines	mandatory	x	x
GreDrawBits	mandatory	x	x
GreDrawBorder	mandatory	x	x
GreDrawLinesInPath	mandatory	x	x
GreDrawRLE	simulated (line function)		
GreEndArea	simulated (area function)		
GreEndPath	simulated (path		

	function)		
GreEqualRegion	simulated (region function)		
GreErasePS	mandatory	x	x
GreEscape DEVESC_ABORTDOC	mandatory (hardcopy devices)	x	
GreEscape DEVESC_ACQUIREFB	mandatory (software motion video support)		x
GreEscape DEVESC_BREAK_EXTRA	mandatory (hardcopy devices)	x	
GreEscape DEVESC_CHAR_EXTRA	mandatory (hardcopy devices)	x	
GreEscape DEVESC_DBE_FONTMANAGEMENT	mandatory (DBCS support)		
GreEscape DEVESC_DBE_LAST	mandatory (DBCS support)		
GreEscape DEVESC_DEACQUIREFB	mandatory (software motion video support)		x
GreEscape DEVESC_DRAFTMODE	mandatory (hardcopy devices)	x	
GreEscape DEVESC_ENDDOC	mandatory (hardcopy devices)	x	
GreEscape DEVESC_EXTGET	mandatory (software motion video support)		x
GreEscape DEVESC_EXTPUT	mandatory (software motion video support)		x
GreEscape DEVESC_EXTQUERY	mandatory (software motion video support)		x
GreEscape DEVESC_FLUSHOUTPUT	mandatory (hardcopy devices)	x	
GreEscape DEVESC_GETAPERTURE	mandatory (software motion video support)		x
GreEscape DEVESC_GETCP	mandatory (hardcopy devices)	x	
GreEscape DEVESC_GETJOBID	mandatory (hardcopy devices) ???	x	
GreEscape DEVESC_GETSCALINGFACTOR	mandatory (hardcopy devices)	x	
GreEscape DEVESC_HWREQUEST	mandatory (software motion video		x

	support)		
GreEscape DEVESC_NEWFRAME	mandatory (hardcopy devices)	x	
GreEscape DEVESC_NEXTBAND	mandatory (hardcopy devices)	x	
GreEscape DEVESC_QUERYESCSUPPORT	mandatory	x	x
GreEscape DEVESC_QUERYFB	mandatory (software motion video support)		x
GreEscape DEVESC_QUERYVIOCELLSIZES	mandatory (display devices)		x
GreEscape DEVESC_RAWDATA	mandatory (hardcopy devices)	x	
GreEscape DEVESC_REGISTER	mandatory (software motion video support)		x
GreEscape DEVESC_SETMODE	mandatory (hardcopy devices)	x	
GreEscape DEVESC_SETUPBLITTERNOTIFY	mandatory (software motion video support)		x
GreEscape DEVESC_STARTDOC	mandatory (hardcopy devices)	x	
GreEscape DEVESC_STD_JOURNAL	mandatory (hardcopy devices)	x	
GreEscape DEVESC_SWITCHBANK	mandatory (software motion video support)		x
GreEscape DEVESC_VIDEOHIDDEN	mandatory (software motion video support)		x
GreEscape DEVESC_VRAMALLOC	mandatory (software motion video support)		x
GreExcludeClipRectangle	simulated (clip function)		
GreFillPath	simulated (path function)		
GreFullArcBoth	simulated (arc function)		
GreFullArcBoundary	simulated (arc function)		
GreFullArcInterior	simulated (arc function)		
GreGetArcParameters	simulated (arc function)		
GreGetBitmapBits	mandatory	x	x

GreGetBoundsData	mandatory	x	x
GreGetClipBox	simulated (clip function)		
GreGetClipRects	simulated (clip function)		
GreGetCodePage	mandatory	x	x
GreGetCurrentPosition	mandatory	x	x
GreGetDCOrigin	mandatory	x	x
GreGetGlobalViewingXform	simulated (transform function)		
GreGetGraphicsField	simulated (transform function)		
GreGetLineOrigin	mandatory	x	x
GreGetModelXform	simulated (transform function)		
GreGetPageUnits	simulated (transform function)		
GreGetPageViewport	simulated (transform function)		
GreGetPairKerningTable	mandatory	x	x
GreGetPel	mandatory	x	x
GreGetPickWindow	mandatory (display devices) misc function		x
GreGetRegionBox	simulated (region function)		
GreGetRegionRects	simulated (transform function)		
GreGetStyleRatio	mandatory (display devices) device function 2		x
GreGetViewingLimits	simulated (transform function)		
GreGetWindowViewportXform	simulated (transform function)		
GreImageData	mandatory	x	x
GreIntersectClipRectangle	simulated (clip function)		
GreLockDevice	mandatory	x	x
GreModifyPath	simulated (path function)		
GreMultiplyXforms	simulated (transform function)		
GreNotifyClipChange	mandatory	x	x
GreNotifyTransformChange	mandatory	x	x

GreOffsetClipRegion	simulated (clip function)			
GreOffsetRegion	simulated (region function)			
GreOutlinePath	simulated (path function)			
GrePaintRegion	simulated (region function)			
GrePartialArc	simulated (arc function)			
GrePolyFillet	simulated (arc function)			
GrePolyFilletSharp	simulated (arc function)			
GrePolygonSet	simulated (line function)			
GrePolyLine	mandatory	x	x	
GrePolyMarker	mandatory	x	x	
GrePolyScanline	mandatory	x	x	
GrePolyShortLine	mandatory	x	x	
GrePolySpline	simulated (line function)			
GrePtInRegion	simulated (region function)			
GrePtVisible	simulated (clip function)			
GreQueryCharPositions	mandatory	x	x	
GreQueryClipRegion	simulated (clip function)			
GreQueryColorData	mandatory	x	x	
GreQueryColorIndex	mandatory	x	x	
GreQueryDeviceBitmaps	mandatory	x	x	
GreQueryDeviceCaps	mandatory	x	x	
GreQueryDevResource	mandatory	x	x	
GreQueryHardcopyCaps	mandatory (hardcopy devices)	x		
GreQueryHWPaletteInfo	simulated (palette manager)			
GreQueryLogColorTable	mandatory	x	x	
GreQueryNearestColor	mandatory	x	x	
GreQueryPaletteRealization	simulated (palette manager function)			
GreQueryRealColors	mandatory	x	x	
GreQueryRGBColor	mandatory	x	x	
GreQueryTextBox	mandatory	x	x	

GreQueryWidthTable	mandatory	x	x
GreRealizeColorTable	mandatory	x	x
GreRealizeFont	mandatory	x	x
GreRealizePalette	simulated		
GreRectInRegion	simulated (region function)		
GreRectVisible	simulated (clip function)		
GreRegionSelectBitmap	simulated (clip function)		
GreResetBounds	mandatory	x	x
GreRestorePath	simulated (path function)		
GreRestoreRegion	simulated (clip function)		
GreRestoreScreenBits	mandatory (display devices) bit map function		x
GreRestoreXform	simulated (transform function)		
GreRestoreXformData	simulated (transform function)		
GreResurrection	mandatory (display devices) device function 3		x
GreSavePath	simulated (path function)		
GreSaveRegion	simulated (clip function)		
GreSaveScreenBits	mandatory (display devices) bit map function		x
GreSaveXform	simulated (transform function)		
GreSaveXformData	simulated (transform function)		
GreScrollRect	mandatory (display devices) AVIO function		x
GreSelectClipPath	simulated (path function)		
GreSelectClipRegion	simulated (clip function)		
GreSelectPathRegion	simulated (clip function)		
GreSetArcParameters	simulated (arc function)		
GreSetBitmapBits	mandatory	x	x
GreSetCodePage	mandatory	x	x

GreSetColorCursor	mandatory (display devices) misc function		x
GreSetCurrentPosition	mandatory	x	x
GreSetGlobalViewingXform	simulated (transform function)		
GreSetGraphicsField	simulated (transform function)		
GreSetLineOrigin	mandatory	x	x
GreSetModelXform	simulated (transform function)		
GreSetPageUnits	simulated (transform function)		
GreSetPageViewport	simulated (transform function)		
GreSetPel	mandatory	x	x
GreSetPickWindow	mandatory (display devices) misc function		x
GreSetRectRegion	simulated (region function)		
GreSetStyleRatio	mandatory (display devices) device function 2		x
GreSetupDC	simulated (clip function)		
GreSetViewingLimits	simulated (transform function)		
GreSetWindowViewportXform	simulated (transform function)		
GreSetXformRect	simulated (clip function)		
GreStrokePath	simulated (path function)		
GreUnlockDevice	mandatory	x	x
GreUnrealizeColorTable	mandatory	x	x
GreUpdateColors	simulated (palette manager function)		
GreUpdateCursor	mandatory (display devices) AVIO function		x
SDBitBlt	simulated (device surface function)		
SDLine	mandatory (device surface		

function)

GreAccumulateBounds

GreAccumulateBounds - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreAccumulateBounds is called to merge bounds into the total bounds held by the presentation driver.

This function must be supported by the presentation driver. *GreAccumulateBounds* is used when a drawing is created to maintain a rectangle that forms the bounding box for the entire drawing. This rectangle is used in transforms and other functions that manipulate the entire drawing at once. *GreAccumulateBounds* can be handled by bit-map emulation.

```
#define INCL_GRE_DEVMISC3
#include <os2.h>

HDC       hdc;         /*  Device context handle. */
PRECTL    prclRect;    /*  Pointer to rectangle. */
PVOID     pInstance;   /*  Pointer to instance data. */
ULONG     lFunction;
BOOL      rc;          /*  Return codes. */

rc = GreAccumulateBounds(hdc, prclRect, pInstance,
       lFunction);
```

GreAccumulateBounds Parameter - hdc

hdc (HDC) - input
Device context handle.

GreAccumulateBounds Parameter - prclRect

prclRect (PRECTL) - input
Pointer to rectangle.

Pointer to rectangle, defined as a RECTL structure in device coordinates.

GreAccumulateBounds Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreAccumulateBounds Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=*flags* ; low-order WORD=*NGreAccumulateBounds* .

GreAccumulateBounds Return Value - rc

rc (BOOL) - returns
Return codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful.
FALSE	An error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call *WinSetErrorInfo* to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreAccumulateBounds - Parameters

hdc (HDC) - input
Device context handle.

prclRect (PRECTL) - input
Pointer to rectangle.

Pointer to rectangle, defined as a RECTL structure in device coordinates.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=*flags* ; low-order WORD=*NGreAccumulateBounds* .

rc (BOOL) - returns
Return codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful.
FALSE	An error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call *WinSetErrorInfo* to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreAccumulateBounds - Remarks

The presentation driver does bound calculations for all drawing primitives. It must convert the bounds to model space as they are accumulated before merging with the GPI bounds. This can be done with *GreConvert* . *GreAccumulateBounds* is related to GreResetBounds and GreGetBoundsData.

GreAccumulateBounds - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreArc

GreArc - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreArc draws an arc through three points. These are the current position and the two points specified in the data structure. Upon completion, the current position is the third point of the arc.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_ARCS
#include <os2.h>

HDC         hdc;          /*  Device context handle. */
PPOINTL     paptlPoint;   /*  Pointer to ArcData array. */
PVOID       pInstance;    /*  Pointer to instance data. */
ULONG       lFunction;    /*  High-order WORD=flags; low order WORD=NGreArc. */
LONG        rc;

rc = GreArc(hdc, paptlPoint, pInstance, lFunction);
```

GreArc Parameter - hdc

hdc (HDC) - input
Device context handle.

GreArc Parameter - paptlPoint

paptlPoint (PPOINTL) - input
Pointer to ArcData array.

Pointer to an array of POINTL structures giving the mid- and end-points of the arc. If the mid-point is coincident with the start or end point, a straight line is drawn from the start point to the end point. If COM_TRANSFORM is not set, the function expects the array of points to be in screen coordinates.

x	X-coordinate of point
y	Y-coordinate of point

GreArc Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreArc Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low order WORD=NGreArc.

GreArc Return Value - rc

rc (LONG) - returns
On completion, this function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected.

GPI_OK	Successful
GPI_HITS	Successful with correlation hit (returned by display drivers when the correlation flag is ON and a hit is detected)
GPI_ERROR	Error.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MATRIX_ELEMENT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreArc - Parameters

hdc (HDC) - input
Device context handle.

paptlPoint (PPOINTL) - input
Pointer to ArcData array.

Pointer to an array of POINTL structures giving the mid- and end-points of the arc. If the mid-point is coincident with the start or end point, a straight line is drawn from the start point to the end point. If COM_TRANSFORM is not set, the function expects the array of points to be in screen coordinates.

x	X-coordinate of point
y	Y-coordinate of point

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low order WORD=NGreArc.

rc (LONG) - returns
On completion, this function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected.

GPI_OK	Successful
GPI_HITS	Successful with correlation hit (returned by display drivers when the correlation flag is ON and a hit is detected)
GPI_ERROR	Error.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED

PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MATRIX_ELEMENT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreArc - Remarks

If GreArc is used within a path definition or an area definition to continue a figure following a GreBoxxxx or GreFullArcxxx function, the error PMERR_INV_NESTED_FIGURES is posted. This is because the GreBoxxxx and GreFullArcxxx functions generate a closed figure within an area or path definition.

GreArc - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreAreaSetAttributes

GreAreaSetAttributes - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreAreaSetAttributes is called by the graphics engine after processing a call to GreSetAttrs received inside an area or path bracket. The handling routine in the graphics engine does nothing. Its purpose is to provide an entry in the dispatch table that can be hooked by presentation drivers.

This function must be hooked by presentation drivers that perform their own area or path simulations.

```
#define INCL_GRE_PATHS
#include <os2.h>

HDC         hdc;           /*  Device context handle. */
LONG        lPrimType;     /*  Bundle primitive type. */
ULONG       flDefsMask;    /*  Flags attributes for default. */
ULONG       flAttrsMask;   /*  Flags attributes for modification. */
PBUNDLE     pAttrs;        /*  Pointer to fixed-format record bundle. */
PVOID       pInstance;     /*  Pointer to instance data. */
ULONG       lFunction;     /*  High-order WORD=flags; low-order WORD=NGreAreaSetAttributes. */
BOOL        fSuccess;      /*  This function returns BOOLEAN (fSuccess). */

fSuccess = GreAreaSetAttributes(hdc, lPrimType,
             flDefsMask, flAttrsMask, pAttrs,
             pInstance, lFunction);
```

GreAreaSetAttributes Parameter - hdc

hdc (HDC) - input
Device context handle.

GreAreaSetAttributes Parameter - lPrimType

lPrimType (LONG) - input
Bundle primitive type.

PRIM_LINE
: Line attribute bundle

PRIM_CHAR
: Character attribute bundle

PRIM_MARKER
: Marker attribute bundle

PRIM_AREA
: Pattern attribute bundle

PRIM_IMAGE
: Image attribute bundle

GreAreaSetAttributes Parameter - flDefsMask

flDefsMask (ULONG) - input
Flags attributes for default.

Flags indicating which attributes are to be set to default.

GreAreaSetAttributes Parameter - flAttrsMask

flAttrsMask (ULONG) - input

Flags attributes for modification.

Flags indicating which attributes are to be modified.

GreAreaSetAttributes Parameter - pAttrs

pAttrs (PBUNDLE) - input
Pointer to fixed-format record bundle.

This is a pointer to the fixed-format bundle record containing the attribute values to be set as specified by flAttrsMask. Only the attribute fields corresponding to attribute flags set in flAttrsMask, and not set in flDefsMask, contain valid values. This buffer must only be large enough to contain data for the highest offset attribute referenced.

GreAreaSetAttributes Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreAreaSetAttributes Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreAreaSetAttributes.

GreAreaSetAttributes Return Value - fSuccess

fSuccess (BOOL) - returns
This function returns BOOLEAN (fSuccess).

TRUE
Successful
FALSE
Error

GreAreaSetAttributes - Parameters

hdc (HDC) - input
Device context handle.

lPrimType (LONG) - input
Bundle primitive type.

PRIM_LINE
Line attribute bundle
PRIM_CHAR
Character attribute bundle
PRIM_MARKER
Marker attribute bundle
PRIM_AREA
Pattern attribute bundle
PRIM_IMAGE
Image attribute bundle

flDefsMask (ULONG) - input
Flags attributes for default.

Flags indicating which attributes are to be set to default.

flAttrsMask (ULONG) - input
Flags attributes for modification.

Flags indicating which attributes are to be modified.

pAttrs (PBUNDLE) - input
Pointer to fixed-format record bundle.

This is a pointer to the fixed-format bundle record containing the attribute values to be set as specified by flAttrsMask. Only the attribute fields corresponding to attribute flags set in flAttrsMask, and not set in flDefsMask, contain valid values. This buffer must only be large enough to contain data for the highest offset attribute referenced.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreAreaSetAttributes.

fSuccess (BOOL) - returns
This function returns BOOLEAN (fSuccess).

TRUE
Successful
FALSE
Error

GreAreaSetAttributes - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreBeginArea

GreBeginArea - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreBeginArea indicates the beginning of a set of drawing functions that define the boundary of an area. All of the boundaries of the area are considered to be part of the interior and are filled.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_PATHS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    flOptions;
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;
BOOL     fSuccess;

fSuccess = GreBeginArea(hdc, flOptions, pInstance,
             lFunction);
```

GreBeginArea Parameter - hdc

hdc (HDC) - input
Device context handle.

GreBeginArea Parameter - flOptions

flOptions (ULONG) - input
These flags designate whether the boundary is drawn and what the drawing mode is.

The defaults are BA_NOBOUNDARY and BA_ALTERNATE.

BA_NOBOUNDARY
Do not draw boundary lines.
BA_BOUNDARY
Draw boundary lines.
BA_ALTERNATE
Alternate mode.
BA_WINDING
Winding mode.
BA_EXCL
Fill exclusive of bottom and right border in device coordinate. New on graphical engine versions 236 and later.

GreBeginArea Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreBeginArea Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low order WORD=NGreBeginArea.

GreBeginArea Return Value - fSuccess

fSuccess (BOOL) - returns
This function returns BOOLEAN (fSuccess).

Area correlation hits are returned at End Area time. No hits are returned for primitives, such as lines and arcs, that form part of the area definition.

TRUE
Successful
FALSE
Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_ALREADY_IN_AREA
PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_AREA_CONTROL
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreBeginArea - Parameters

hdc (HDC) - input
Device context handle.

flOptions (ULONG) - input
These flags designate whether the boundary is drawn and what the drawing mode is.

The defaults are BA_NOBOUNDARY and BA_ALTERNATE.

BA_NOBOUNDARY
Do not draw boundary lines.
BA_BOUNDARY
Draw boundary lines.
BA_ALTERNATE
Alternate mode.
BA_WINDING
Winding mode.
BA_EXCL
Fill exclusive of bottom and right border in device coordinate. New on graphical engine versions 236 and later.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low order WORD=NGreBeginArea.

fSuccess (BOOL) - returns
This function returns BOOLEAN (fSuccess).

Area correlation hits are returned at End Area time. No hits are returned for primitives, such as lines and arcs, that form part of the area definition.

TRUE
Successful

FALSE
Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_ALREADY_IN_AREA
PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_AREA_CONTROL
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreBeginArea - Remarks

GreBeginArea has no direct effect on current position, although it can be affected by drawing orders within the boundary definition. When GreBox*xxx* or GreFullArc*xxx* functions are used within an area definition, they generate closed figures and must not be used within another figure definition. For more information, see "GpiBeginArea" in the *Presentation Manager Programming Reference* .

The following functions are valid when received after GreBeginArea and before GreEndArea:

- GreArc
- GreAreaSetAttributes (valid only for color, mix, and valid line attributes)
- GreBoxBoundary
- GreDeviceSetAttributes (valid only for color, mix, and valid line attributes)
- GreDeviceSetGlobalAttribute (valid only for foreground color and mix)
- GreFullArcBoundary
- GrePartialArc
- GrePolyFillet
- GrePolyFilletSharp
- GrePolyLine
- GrePolySpline
- GreQueryCharStringPos
- GreQueryTextBox
- GreSetArcParameters
- GreSetAttributes (valid only for color, mix, and valid line attributes)
- GreSetCurrentPosition
- GreSetGlobalAttribute (valid only for foreground color and mix)
- GreSetModelXform

GreBeginArea - Topics

Select an item:
Syntax
Parameters
Returns

Remarks
Glossary

GreBeginPath

GreBeginPath - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreBeginPath identifies the start of a sequence of figures that define a path. Note that functions that set character attributes are not allowed within a path definition. When GreBox*xxx* or GreFullArc*xxx* functions are used within a path definition, they generate closed figures and must not be used within another figure definition. Refer to function "GpiBeginPath" in the *Presentation Manager Programming Reference* for further information.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_PATHS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
LONG     idPath;      /*  Path identifier. This value must be 1. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low order WORD=NGreBeginPath. */
BOOL     fSuccess;    /*  This function returns BOOLEAN (fSuccess). */

fSuccess = GreBeginPath(hdc, idPath, pInstance,
             lFunction);
```

GreBeginPath Parameter - hdc

hdc (HDC) - input
Device context handle.

GreBeginPath Parameter - idPath

idPath (LONG) - input
Path identifier. This value must be 1.

GreBeginPath Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreBeginPath Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low order WORD=NGreBeginPath.

GreBeginPath Return Value - fSuccess

fSuccess (BOOL) - returns
This function returns BOOLEAN (fSuccess).

TRUE
Successful
FALSE
Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_ALREADY_IN_PATH
PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PATH_ID
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreBeginPath - Parameters

hdc (HDC) - input
Device context handle.

idPath (LONG) - input
Path identifier. This value must be 1.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low order WORD=NGreBeginPath.

fSuccess (BOOL) - returns
This function returns BOOLEAN (fSuccess).

TRUE
Successful

FALSE
Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_ALREADY_IN_PATH
PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PATH_ID
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreBeginPath - Remarks

The following functions are valid when received after GreBeginArea and before GreEndArea:

- GreArc
- GreAreaSetAttributes (valid only for color, mix, and valid line attributes)
- GreBoxBoundary
- GreDeviceSetAttributes (valid only for color, mix, and valid line attributes)
- GreDeviceSetGlobalAttribute (valid only for foreground color and mix)
- GreFullArcBoundary
- GrePartialArc
- GrePolyFillet
- GrePolyFilletSharp
- GrePolyLine
- GrePolySpline
- GreQueryCharStringPos
- GreQueryTextBox
- GreSetArcParameters
- GreSetAttributes (valid only for color, mix, and valid line attributes)
- GreSetCurrentPosition
- GreSetGlobalAttribute (valid only for foreground color and mix)
- GreSetModelXform

GreBeginPath - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreBitblt

GreBitblt - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreBitBlt modifies bit-map data at a target rectangle in the current device context (DC). The modification can copy a rectangle of data from a specified source DC to the target or perform a raster operation on the target.

This function must be supported by the presentation driver. GreBitblt is called by the function GpiBitBlt and is used to modify bit-map data within a target rectangle of the current device context. However, if the destination is larger or smaller than the source, the presentation driver can pass this function to the graphics engine by using the original pointer copied from the dispatch table.

```
#define INCL_GRE_BITMAPS
#include <os2.h>

HDC             hdc;          /*  Device context handle. */
HDC             hdcSrc;       /*  Handle to the source DC or bit map. */
LONG            cPoints;      /*  Number of (X,Y) pairs in paptlPoint. */
PPOINTL         paptlPoint;   /*  Pointer to an array of (X, Y) coordinate pairs. */
LONG            lRop;         /*  Raster operation code. */
ULONG           flOptions;
PBITBLTATTRS    pBattrs;      /*  Pointer to attributes. */
PVOID           pInstance;    /*  Pointer to instance data. */
ULONG           lFunction;
LONG            rc;           /*  Return Codes. */

rc = GreBitblt(hdc, hdcSrc, cPoints, paptlPoint,
       lRop, flOptions, pBattrs, pInstance,
       lFunction);
```

GreBitblt Parameter - hdc

hdc (HDC) - input
Device context handle.

GreBitblt Parameter - hdcSrc

hdcSrc (HDC) - input
Handle to the source DC or bit map.

When lRop does not require a source, hdcSrc is passed as NULL. The handling routine then copies the current pattern to the currently selected bit map or device.

GreBitblt Parameter - cPoints

cPoints (LONG) - input
Number of (X,Y) pairs in paptlPoint.

A count of the number of (X,Y) pairs in the paptlPoint array. The count can be 2, 3, or 4.

cPoints=2	A raster operation (as determined by lRop) on the destination rectangle.	
cPoints=3	A copy between two rectangles of the same size. Only the bottom-left corner is given for the source rectangle.	
cPoints=4	An operation is determined by comparing the sizes of the two rectangles:	
	Target<Source	Compress the source rectangle into the target rectangle. The flOptions flags determine how to handle eliminated rows and columns. In this case, the function can be passed to the GreBitblt routine in the graphics engine.
	Target=Source	Copy between equal rectangles.
	Target>Source	Stretch the source rectangle into the target rectangle. In this case, the function can be passed to the GreBitblt routine in the graphics engine.

GreBitblt Parameter - paptlPoint

paptlPoint (PPOINTL) - input
Pointer to an array of (X, Y) coordinate pairs.

Pointer to an array of (X,Y) coordinate pairs that define the target and source rectangles. The coordinates, which can be passed as a pair of RECTL structures, define the bottom-left and top-right corners of the target and source rectangles (see previous parameter, cPoints, for exceptions).

```
(xTgtBL, yTgtBL), (xTgtTR, yTgtTR), (xSrcBL, ySrcBL), (xSrcTR, ySrcTR)
```

When the source rectangle is totally or partially outside the source bit map (or device), the operation is implementation-dependent for that area (that is, the programmer of the called presentation driver must decide what to do).

Note: When BBO_TARGWORLD is not set, the rectangles are noninclusive. That is, they include the left and lower boundaries in device units but not the top and right boundaries. When the bottom-left corner of a rectangle maps to the same device pel as the top-right corner, that rectangle is considered to be empty.

When BBO_TARGWORLD is set, the target rectangle is inclusive at all boundaries. The source is noninclusive.

GreBitblt Parameter - lRop

lRop (LONG) - input
Raster operation code.

The low-order byte represents a mix value in the range hex 00-hex FF. Raster operation code values and the mix-bit table are defined in the *Presentation Manager Programming Reference* . The handling routine uses lRop to determine the operations to perform on the pattern, source, and target to get the required mix.

In addition to the ROP values defined at the API, the presentation driver must support ROP_GRAY (000080CAH). This value is used to shade the text for menu items that are not currently selectable. When ROP_GRAY is set, the handling routine overpaints the foreground pattern by using the current pattern and the background pattern color (background pels for the pattern are not changed). For the PATSYM_HALFTONE pattern, this setting overpaints the background pattern color onto alternate pels leaving those in between unchanged.

GreBitblt Parameter - flOptions

flOptions (ULONG) - input
Specifies treatment of eliminated lines and columns when compression is done. Option flag values are:

BBO_OR	Stretch and compress, as necessary, ORing any eliminated rows and columns. Used for White on Black.
BBO_AND	Stretch and compress, as necessary, ANDing any eliminated rows and columns. Used for Black on White.
BBO_IGNORE	Stretch and compress, as necessary, IGNORing any eliminated rows and columns. Used for color.
BBO_TARGWORLD	The target rectangle is defined in world coordinates in the target PS. When this option is specified, the target rectangle is transformed to device coordinates. Where any shear or rotation has occurred, this rectangle must be converted to an upright rectangle that bounds the transformed figure. This rectangle is then used as the target for the operation. No inversion of the image takes place.
BBO_NO_COLOR_INFO	No color information is associated with the source. The source bit map should be copied directly to the destination without translating the pels. This option must be supported if the device supports palette management.
BLTMODE_SRC_BITMAP	hdcSrc is a bit-map handle. The bit map must not be currently selected into a device context. If this flag is not set, hdcSrc is a device context handle.
BLTMODE_ATTRS_PRES	If set, the pBattrs parameter is present. This option can be ORed with any of the above options.

Note: Flags 15 - 31 are not used by the system. They are reserved for use by the presentation driver.

GreBitblt Parameter - pBattrs

pBattrs (PBITBLTATTRS) - input
Pointer to attributes.

Pointer to a BITBLTATTRS structure containing the attributes. The BITBLTATTRS structure consists of the following fields:

cSize	Size of this structure
lColor	Foreground color of source
lBackColor	Background color of source

The color values are used in conversions between monochrome and color data, and are the only format conversions required. The conversions are required for:

- Output of a monochrome pattern to a color device. In this case, the source pattern is converted to a color pattern. This is performed by using the colors provided in the BITBLTATTRS structure. If these colors are not provided, the handling routine uses the current area colors for the target device context. See Area (Pattern) Attributes. The bits are then transferred so that:
  - Source 1s become (target area) foreground color
  - Source 0s become (target area) background color
- Transfer from a monochrome bit map to a color bit map or device. In this case, the source bits are converted by using the current image colors. These are the colors provided in the BITBLTATTRS structure. If these colors are not provided, the handling routine uses the current image colors for the target device context. See Image Attributes. The bits are then transferred so that:

- Source 1s become (target image) foreground color
- Source 0s become (target image) background color

- Transfer from a color bit map to a monochrome bit map or device. In this case, the source bit map is converted by using the source and target image colors. The target colors are provided in the BITBLTATTRS structure. If these colors are not provided, the handling routine uses those in the image attributes bundle for the target device context. See Image Attributes. When the source is a device context, the source-image background color is that from the source DC. When the source is a bit-map handle, the background color is taken from the BITBLTATTRS structure, if provided, or from the background-image color of the target DC. The bits are then transferred so that:

  - Source pels that are the source-image background color become target-image background color.

  - All other pels become target-image foreground color.

When IRop does not call for a pattern, the pattern set and pattern symbol are not used. Neither the source nor the pattern is required when a bit map or part of a bit map is being cleared to a particular color. When a pattern is required, dithering can be done for solid patterns in a color that is not available on the device. Color dithering is described in GreQueryNearestColor

---

GreBitblt Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreBitblt Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreBitblt.

GreBitblt Return Value - rc

rc (LONG) - returns
Return Codes.

On completion, the handling routine must return a LONG integer (cHits), indicating, where appropriate, whether correlation hits have been detected:

GPI_OK	Successful
GPI_HITS	Successful with correlation hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_IS_SELECTED
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INCOMPATIBLE_BITMAP
PMERR_INCORRECT_DC_TYPE

PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_BITBLT_MIX
PMERR_INV_BITBLT_STYLE
PMERR_INV_COLOR_ATTR
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_FORMAT
PMERR_INV_COLOR_INDEX
PMERR_INV_COLOR_OPTIONS
PMERR_INV_COLOR_START_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_DC_DATA
PMERR_INV_DC_TYPE
PMERR_INV_DRIVER_NAME
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_ID
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_SCAN_START
PMERR_INV_USAGE_PARM
PMERR_UNSUPPORTED_ATTR_VALUE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreBitblt - Parameters

hdc (HDC) - input
Device context handle.

hdcSrc (HDC) - input
Handle to the source DC or bit map.

When lRop does not require a source, hdcSrc is passed as NULL. The handling routine then copies the current pattern to the currently selected bit map or device.

cPoints (LONG) - input
Number of (X,Y) pairs in paptlPoint.

A count of the number of (X,Y) pairs in the paptlPoint array. The count can be 2, 3, or 4.

cPoints=2	A raster operation (as determined by lRop) on the destination rectangle.
cPoints=3	A copy between two rectangles of the same size. Only the bottom-left corner is given for the source rectangle.
cPoints=4	An operation is determined by comparing the sizes of the two rectangles:

Target<Source	Compress the source rectangle into the target rectangle. The flOptions flags determine how to handle eliminated rows and columns. In this case, the function can be passed to the GreBitblt routine in the graphics engine.
Target=Source	Copy between equal rectangles.
Target>Source	Stretch the source rectangle into the target rectangle. In this case, the function can be passed to the GreBitblt routine in the graphics engine.

paptlPoint (PPOINTL) - input

Pointer to an array of (X, Y) coordinate pairs.

Pointer to an array of (X,Y) coordinate pairs that define the target and source rectangles. The coordinates, which can be passed as a pair of RECTL structures, define the bottom-left and top-right corners of the target and source rectangles (see previous parameter, cPoints, for exceptions).

```
(xTgtBL, yTgtBL), (xTgtTR, yTgtTR), (xSrcBL, ySrcBL), (xSrcTR, ySrcTR)
```

When the source rectangle is totally or partially outside the source bit map (or device), the operation is implementation-dependent for that area (that is, the programmer of the called presentation driver must decide what to do).

Note: When BBO_TARGWORLD is not set, the rectangles are noninclusive. That is, they include the left and lower boundaries in device units but not the top and right boundaries. When the bottom-left corner of a rectangle maps to the same device pel as the top-right corner, that rectangle is considered to be empty.

When BBO_TARGWORLD is set, the target rectangle is inclusive at all boundaries. The source is noninclusive.

lRop (LONG) - input
Raster operation code.

The low-order byte represents a mix value in the range hex 00-hex FF. Raster operation code values and the mix-bit table are defined in the *Presentation Manager Programming Reference* . The handling routine uses lRop to determine the operations to perform on the pattern, source, and target to get the required mix.

In addition to the ROP values defined at the API, the presentation driver must support ROP_GRAY (000080CAH). This value is used to shade the text for menu items that are not currently selectable. When ROP_GRAY is set, the handling routine overpaints the foreground pattern by using the current pattern and the background pattern color (background pels for the pattern are not changed). For the PATSYM_HALFTONE pattern, this setting overpaints the background pattern color onto alternate pels leaving those in between unchanged.

flOptions (ULONG) - input
Specifies treatment of eliminated lines and columns when compression is done. Option flag values are:

BBO_OR	Stretch and compress, as necessary, ORing any eliminated rows and columns. Used for White on Black.
BBO_AND	Stretch and compress, as necessary, ANDing any eliminated rows and columns. Used for Black on White.
BBO_IGNORE	Stretch and compress, as necessary, IGNORing any eliminated rows and columns. Used for color.
BBO_TARGWORLD	The target rectangle is defined in world coordinates in the target PS. When this option is specified, the target rectangle is transformed to device coordinates. Where any shear or rotation has occurred, this rectangle must be converted to an upright rectangle that bounds the transformed figure. This rectangle is then used as the target for the operation. No inversion of the image takes place.
BBO_NO_COLOR_INFO	No color information is associated with the source. The source bit map should be copied directly to the destination without translating the pels. This option must be supported if the device supports palette management.
BLTMODE_SRC_BITMAP	hdcSrc is a bit-map handle. The bit map must not be currently selected into a device context. If this flag is not set, hdcSrc is a device context handle.
BLTMODE_ATTRS_PRES	If set, the pBattrs parameter is present. This option can be ORed with any of the above options.

Note: Flags 15 - 31 are not used by the system. They are reserved for use by the presentation driver.

pBattrs (PBITBLTATTRS) - input
Pointer to attributes.

Pointer to a BITBLTATTRS structure containing the attributes. The BITBLTATTRS structure consists of the following fields:

cSize	Size of this structure
lColor	Foreground color of source
lBackColor	Background color of source

The color values are used in conversions between monochrome and color data, and are the only format conversions required. The

conversions are required for:

- Output of a monochrome pattern to a color device. In this case, the source pattern is converted to a color pattern. This is performed by using the colors provided in the BITBLTATTRS structure. If these colors are not provided, the handling routine uses the current area colors for the target device context. See Area (Pattern) Attributes. The bits are then transferred so that:
  - Source 1s become (target area) foreground color
  - Source 0s become (target area) background color
- Transfer from a monochrome bit map to a color bit map or device. In this case, the source bits are converted by using the current image colors. These are the colors provided in the BITBLTATTRS structure. If these colors are not provided, the handling routine uses the current image colors for the target device context. See Image Attributes. The bits are then transferred so that:
  - Source 1s become (target image) foreground color
  - Source 0s become (target image) background color
- Transfer from a color bit map to a monochrome bit map or device. In this case, the source bit map is converted by using the source and target image colors. The target colors are provided in the BITBLTATTRS structure. If these colors are not provided, the handling routine uses those in the image attributes bundle for the target device context. See Image Attributes. When the source is a device context, the source-image background color is that from the source DC. When the source is a bit-map handle, the background color is taken from the BITBLTATTRS structure, if provided, or from the background-image color of the target DC. The bits are then transferred so that:
  - Source pels that are the source-image background color become target-image background color.
  - All other pels become target-image foreground color.

When IRop does not call for a pattern, the pattern set and pattern symbol are not used. Neither the source nor the pattern is required when a bit map or part of a bit map is being cleared to a particular color. When a pattern is required, dithering can be done for solid patterns in a color that is not available on the device. Color dithering is described in GreQueryNearestColor

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreBitblt.

rc (LONG) - returns
Return Codes.

On completion, the handling routine must return a LONG integer (cHits), indicating, where appropriate, whether correlation hits have been detected:

GPI_OK	Successful
GPI_HITS	Successful with correlation hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_IS_SELECTED
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INCOMPATIBLE_BITMAP
PMERR_INCORRECT_DC_TYPE
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_BITBLT_MIX
PMERR_INV_BITBLT_STYLE
PMERR_INV_COLOR_ATTR
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_FORMAT
PMERR_INV_COLOR_INDEX
PMERR_INV_COLOR_OPTIONS
PMERR_INV_COLOR_START_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE

PMERR_INV_DC_DATA
PMERR_INV_DC_TYPE
PMERR_INV_DRIVER_NAME
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_ID
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_SCAN_START
PMERR_INV_USAGE_PARM
PMERR_UNSUPPORTED_ATTR_VALUE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreBitblt - Remarks

The device contexts can be memory DCs with bit maps selected or DCs belonging to devices that support raster operations.

When copying bits from a color bit map to a monochrome bit map or device, only those pels that are in the source background color are copied to the target as background color. All other pels are copied to the target as foreground color. Copying is nondestructive. When the target and source rectangles are in the same DC, no information is lost from the source if the rectangles overlap. When the target is expressed in world coordinates (that is, the BBO_TARGWORLD flag is set in flOptions), the coordinates must be transformed device coordinates. The bits are transferred to an upright rectangle in device space, regardless of any rotational elements that might have been present in the transforms.

The attribute structure identified by the pBattrs parameter defines the bit-map foreground and background colors. If pBattrs is NULL, the handling routine uses the current foreground and background colors.

When the mix specified by lRop requires both source and pattern, a 3-way operation is performed by using the current pattern in the target device context. If pattern mixing is not required, a 2-way operation is done. If any of the source data is unavailable, the handling routine transfers those bits that are present and returns without error. This might occur when the source device context is a window on the screen that has been overlaid by another. In this example, the handling routine must proceed by reading what is there.

Note: The device driver may pass back Bitblt calls that call for stretching. However during the stretching, GRE will perform dithering only if the device supports more than two colors and a bit map format of more than one bit per pel in its response to a GreQueryDeviceCaps call.

A driver may alter these capabilities before returning a Bitblt call to the GRE so that dithering will (or will not) be done.

GreBitblt - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreBoxBoth

GreBoxBoth - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreBoxBoth draws and fills a rectangular box with one corner at the current (X,Y) position and the opposite corner at the specified (X,Y) position. The current (X,Y) position does not change. When this function occurs within an area or path definition, it generates a closed figure. GreBoxBoth must not occur within any other figure definition.

This function is supported by the graphics engine and can be hooked by the presentation driver.

```
#define INCL_GRE_ARCS
#include <os2.h>

HDC             hdc;         /*  Device context handle. */
PBOXPARAMS      pBox;        /*  Pointer to a BOXPARAMS structure: */
PVOID           pInstance;   /*  Pointer to instance data. */
ULONG           lFunction;   /*  High-order WORD=flags; low-order WORD=NGreBoxBoth. */
LONG            rc;

rc = GreBoxBoth(hdc, pBox, pInstance, lFunction);
```

GreBoxBoth Parameter - hdc

hdc (HDC) - input
Device context handle.

GreBoxBoth Parameter - pBox

pBox (PBOXPARAMS) - input
Pointer to a BOXPARAMS structure:

ptl
POINTL structure defining the opposite corner of the box. If COM_TRANSFORM is not set, the function expects the point to be in screen coordinates.
x X-coordinate of opposite corner
y Y-coordinate of opposite corner

sizl.cx
Horizontal length of the full axis of an ellipse. This field is used for rounding each corner.

sizl.cy
Vertical length of the full axis of an ellipse. This field is used for rounding each corner.

GreBoxBoth Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreBoxBoth Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreBoxBoth.

GreBoxBoth Return Value - rc

rc (LONG) - returns
On completion, this function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected.

GPI_OK
Successful

GPI_HITS
Successful with correlation hit (returned by display drivers when the correlation flag is ON and a hit is detected)

GPI_ERROR
Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_ALREADY_IN_AREA
PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_AREA_CONTROL
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_BOX_ROUNDING_PARM
PMERR_INV_CHAR_DIRECTION_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_MIX_ATTR
PMERR_INV_NESTED_FIGURES
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_NOT_IN_AREA
PMERR_NOT_IN_PATH
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN

PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreBoxBoth - Parameters

hdc (HDC) - input
Device context handle.

pBox (PBOXPARAMS) - input
Pointer to a BOXPARAMS structure:

ptl
POINTL structure defining the opposite corner of the box. If COM_TRANSFORM is not set, the function expects the point to be in screen coordinates.
x X-coordinate of opposite corner
y Y-coordinate of opposite corner

sizl.cx
Horizontal length of the full axis of an ellipse. This field is used for rounding each corner.

sizl.cy
Vertical length of the full axis of an ellipse. This field is used for rounding each corner.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreBoxBoth.

rc (LONG) - returns
On completion, this function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected.

GPI_OK
Successful

GPI_HITS
Successful with correlation hit (returned by display drivers when the correlation flag is ON and a hit is detected)

GPI_ERROR
Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_ALREADY_IN_AREA
PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_AREA_CONTROL
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_BOX_ROUNDING_PARM
PMERR_INV_CHAR_DIRECTION_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_MIX_ATTR

PMERR_INV_NESTED_FIGURES
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_NOT_IN_AREA
PMERR_NOT_IN_PATH
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreBoxBoth - Remarks

The sides of the box (before transformation) are parallel to the X-axis and the Y-axis. The corners of the box can be rounded by means of quarter ellipses of the specified diameters. When the value of either diameter is 0, no rounding occurs. When the value of either diameter exceeds the length of the corresponding side, that length is used as the diameter instead. When the value of the diameters are equal to the value of the sides, the corners are rounded with a quarter circle. If the current position is (x0, y0), the box is drawn from the current position in a counterclockwise direction.

When correlating, the handling routine records a hit when the pick aperture intersects the boundary or interior, or is completely within the interior (even if the mix used for the fill operation is LEAVEALONE).

GreBoxBoth - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreBoxBoundary

GreBoxBoundary - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreBoxBoundary draws a rectangular box with one corner at the current (X,Y) position and the opposite corner at the specified (X,Y) position. The current (X,Y) position does not change.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_ARCS
#include <os2.h>

HDC         hdc;         /*  Device context handle. */
PPOINTL     pBox;        /*  Pointer to a BOXDATA structure: */
PVOID       pInstance;   /*  Pointer to instance data. */
ULONG       lFunction;   /*  High-order WORD=flags; low-order WORD=NGreBoxBoundary. */
LONG        rc;

rc = GreBoxBoundary(hdc, pBox, pInstance,
       lFunction);
```

GreBoxBoundary Parameter - hdc

hdc (HDC) - input
Device context handle.

GreBoxBoundary Parameter - pBox

pBox (PPOINTL) - input
Pointer to a BOXDATA structure:

ptlOpposite
POINTL structure defining the opposite corner of the box. If COM_TRANSFORM is not set, the function expects the point to be in screen coordinates.
x X-coordinate of opposite corner
y Y-coordinate of opposite corner

lHRound
Horizontal length of the full axis of an ellipse. This field is used for rounding each corner.

lVRound
Vertical length of the full axis of an ellipse. This field is used for rounding each corner.

GreBoxBoundary Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreBoxBoundary Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreBoxBoundary.

GreBoxBoundary Return Value - rc

rc (LONG) - returns
On completion, this function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected.

GPI_OK
Successful
GPI_HITS
Successful with correlate hit (returned by display drivers when the correlate flag is ON and a hit is detected)
GPI_ERROR
Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_BOX_ROUNDING_PARM
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_NESTED_FIGURES
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_NOT_IN_PATH
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreBoxBoundary - Parameters

hdc (HDC) - input
Device context handle.

pBox (PPOINTL) - input
Pointer to a BOXDATA structure:

ptlOpposite
POINTL structure defining the opposite corner of the box. If COM_TRANSFORM is not set, the function expects the point to be in screen coordinates.
x X-coordinate of opposite corner
y Y-coordinate of opposite corner
lHRound
Horizontal length of the full axis of an ellipse. This field is used for rounding each corner.
lVRound
Vertical length of the full axis of an ellipse. This field is used for rounding each corner.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreBoxBoundary.

rc (LONG) - returns
On completion, this function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected.

GPI_OK
Successful

GPI_HITS

Successful with correlate hit (returned by display drivers when the correlate flag is ON and a hit is detected)

GPI_ERROR

Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_BOX_ROUNDING_PARM
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_NESTED_FIGURES
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_NOT_IN_PATH
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreBoxBoundary - Remarks

The sides of the box (before transformation) are parallel to the X-axis and the Y-axis. The corners of the box can be rounded by means of quarter ellipses of the specified diameters. When the value of either diameter is 0, no rounding occurs. When the value of either diameter exceeds the length of the corresponding side, that length is used as the diameter instead. When the value of the diameters are equal to the value of the sides, the corners are rounded with a quarter circle. If the current position is (x0, y0), the box is drawn from the current position in a counterclockwise direction.

When correlating, the handling routine records a hit when the pick aperture intersects the boundary.

GreBoxBoundary - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreBoxInterior

GreBoxInterior - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description *GreBoxInterior* draws a rectangular box with one corner at the current (X,Y) position and the opposite corner at the specified (X,Y) position. The current (X,Y) position does not change. When this function occurs within an area or path definition, it generates a closed figure. GreBoxInterior must not occur within any other figure definition.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_ARCS
#include <os2.h>

HDC         hdc;         /*  Device context handle. */
PPOINTL     pBox;        /*  Pointer to a BOXDATA structure. */
PVOID       pInstance;   /*  Pointer to instance data. */
ULONG       lFunction;   /*  High-order WORD=flags; low-order WORD=NGreBoxInterior. */
LONG        rc;

rc = GreBoxInterior(hdc, pBox, pInstance,
       lFunction);
```

GreBoxInterior Parameter - hdc

hdc (HDC) - input
Device context handle.

GreBoxInterior Parameter - pBox

pBox (PPOINTL) - input
Pointer to a BOXDATA structure.

ptlOpposite	POINTL structure defining the opposite corner of the box. If COM_TRANSFORM is not set, the function expects the point to be in screen coordinates. x — X-coordinate of opposite corner y — Y-coordinate of opposite corner
lHRound	Horizontal length of the full axis of an ellipse. This field is used for rounding each corner.
lVRound	Vertical length of the full axis of an ellipse. This field is used for rounding each corner.

GreBoxInterior Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreBoxInterior Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreBoxInterior.

GreBoxInterior Return Value - rc

rc (LONG) - returns
On completion, this function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected.

GreBoxInterior - Parameters

hdc (HDC) - input
Device context handle.

pBox (PPOINTL) - input
Pointer to a BOXDATA structure.

ptlOpposite	POINTL structure defining the opposite corner of the box. If COM_TRANSFORM is not set, the function expects the point to be in screen coordinates.
	x X-coordinate of opposite corner y Y-coordinate of opposite corner
lHRound	Horizontal length of the full axis of an ellipse. This field is used for rounding each corner.
lVRound	Vertical length of the full axis of an ellipse. This field is used for rounding each corner.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreBoxInterior.

rc (LONG) - returns
On completion, this function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected.

GreBoxInterior - Remarks

The sides of the box (before transformation) are parallel to the X-axis and Y-axis. The corners of the box can be rounded by means of quarter ellipses of the specified diameters. When the value of either diameter is 0, no rounding occurs. When the value of either diameter exceeds the length of the corresponding side, that length is used as the diameter instead. When the value of the diameters are equal to the value of the sides, the corners are rounded with a quarter circle. If the current position is (x0, y0), the box is drawn from the current position in a counterclockwise direction. This is significant when, for example, the area mode is BA_WINDING.

When correlating, the handling routine records a hit when the pick aperture intersects, or is completely within, the interior (even if the mix used for the fill operation is LEAVEALONE).

GreBoxInterior - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreCharRect

GreCharRect - Syntax

Simulation support:

This function is mandatory for display drivers. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreCharRect draws a rectangle of character cells from the LVB to the device context. The attributes for each character are applied by the handling routine as the character is drawn.

```
#define INCL_AVIOP
#include <os2.h>

HDC                     hdc;         /*  The device context handle. */
VioPresentationSpace   *pVioPS;      /*  A pointer to the Vio presentation space. */
LPGridRectRef           pCharRect;
PVOID                   pInstance;   /*  A pointer to instance data. */
ULONG                   lFunction;   /*  High-order WORD=flags; low-order WORD=NGreCharRect. */
LONG                    rc;          /*  Return codes. */

rc = GreCharRect(hdc, pVioPS, pCharRect, pInstance,
       lFunction);
```

GreCharRect Parameter - hdc

hdc (HDC) - input
The device context handle.

GreCharRect Parameter - pVioPS

pVioPS (VioPresentationSpace *) - input
A pointer to the Vio presentation space.

GreCharRect Parameter - pCharRect

pCharRect (LPGridRectRef) - input
A pointer to a parameter block. This is a GridRectRef structure:

StartRow	The starting row (relative to the bottom left of the LVB) of the character rectangle to be drawn
StartCol	The starting column (relative to the bottom left of the LVB) of the character rectangle to be drawn
RectWidth	The width in character cells of the rectangle to be updated
RectHeight	The height of the rectangle to be updated

GreCharRect Parameter - pInstance

pInstance (PVOID) - input
A pointer to instance data.

GreCharRect Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCharRect.

GreCharRect Return Value - rc

rc (LONG) - returns
Return codes.

This function returns a LONG value as an error indicator.

NO_ERROR	Successful.
CE_INVALID_PRESENTATION_SPACE	Error. For example, this message might be returned as a result of an invalid CellByteSize.

GreCharRect - Parameters

hdc (HDC) - input
The device context handle.

pVioPS (VioPresentationSpace *) - input
A pointer to the Vio presentation space.

pCharRect (LPGridRectRef) - input
A pointer to a parameter block. This is a GridRectRef structure:

StartRow	The starting row (relative to the bottom left of the LVB) of the character rectangle to be drawn
StartCol	The starting column (relative to the bottom left of the LVB) of the character rectangle to be drawn
RectWidth	The width in character cells of the rectangle to be updated
RectHeight	The height of the rectangle to be updated

pInstance (PVOID) - input
A pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCharRect.

rc (LONG) - returns
Return codes.

This function returns a LONG value as an error indicator.

NO_ERROR	Successful.
CE_INVALID_PRESENTATION_SPACE	Error. For example, this message might be returned as a result of an invalid CellByteSize.

GreCharRect - Remarks

This function is used to implement the advanced Vio function, VioSetOrg.

GreCharRect - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreCharStr

GreCharStr - Syntax

Simulation support:

This function is mandatory for display drivers. For other drivers, it is simulated by a handling routine in the graphics engine.

Support:

GreCharStr draws a string of character cells from the LVB to the device context.

```
#define INCL_AVIOP
#include <os2.h>

HDC                     hdc;         /*  The device context handle. */
VioPresentationSpace   *pVioPS;      /*  A pointer to the Vio presentation space. */
LPGridRectRef           pCharRect;
PVOID                   pInstance;   /*  A pointer to instance data. */
ULONG                   lFunction;   /*  High-order WORD=flags; low-order WORD=NGreCharStr. */
LONG                    rc;          /*  Return codes. */

rc = GreCharStr(hdc, pVioPS, pCharRect, pInstance,
       lFunction);
```

GreCharStr Parameter - hdc

hdc (HDC) - input
The device context handle.

GreCharStr Parameter - pVioPS

pVioPS (VioPresentationSpace *) - input
A pointer to the Vio presentation space.

GreCharStr Parameter - pCharRect

pCharRect (LPGridRectRef) - input
A pointer to parameter block. This is a GridStringRef structure:

StartRow	The starting row (relative to the bottom left of the LVB) of the character rectangle to be drawn
StartCol	The starting column (relative to the bottom left of the LVB) of the character rectangle to be drawn
StringLength	Number of characters to be written

GreCharStr Parameter - pInstance

pInstance (PVOID) - input
A pointer to instance data.

GreCharStr Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCharStr.

GreCharStr Return Value - rc

rc (LONG) - returns
Return codes.

This function returns a LONG value as an error indicator.

NO_ERROR	Successful.
GRE_INVALID_COLUMN_INDEX	Invalid column index.
CE_INVALID_PRESENTATION_SPACE	Error. For example, this message might be returned as a result of an invalid CellByteSize.
CE_INVALID_ROW_INDEX	Invalid row index.
GRE_NEGATIVE_LENGTH	Negative length.

GreCharStr - Parameters

hdc (HDC) - input
The device context handle.

pVioPS (VioPresentationSpace *) - input
A pointer to the Vio presentation space.

pCharRect (LPGridRectRef) - input
A pointer to parameter block. This is a GridStringRef structure:

StartRow	The starting row (relative to the bottom left of the LVB) of the character rectangle to be drawn
StartCol	The starting column (relative to the bottom left of the LVB) of the character rectangle to be drawn
StringLength	Number of characters to be written

pInstance (PVOID) - input
A pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCharStr.

rc (LONG) - returns
Return codes.

This function returns a LONG value as an error indicator.

NO_ERROR	Successful.
GRE_INVALID_COLUMN_INDEX	Invalid column index.
CE_INVALID_PRESENTATION_SPACE	Error. For example, this message might be returned as a result of an invalid CellByteSize.
CE_INVALID_ROW_INDEX	Invalid row index.
GRE_NEGATIVE_LENGTH	Negative length.

GreCharStr - Remarks

The attributes for each character are applied by the handling routine as the character is drawn. When the end of a row is reached, the next character is drawn in the first cell of the next row. Character drawing continues until either the string or the logical video buffer is exhausted.

GreCharStr - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreCharString

GreCharString - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreCharString draws a character string starting at the current (X,Y) position. Upon completion, the current (X,Y) position is the start point for the character cell immediately after the last character in the string.

GreCharString must be supported by the presentation driver. The handling routine must provide full support for drawing characters from an image font in CM_MODE1 when the character direction is CHDIRN_LEFTRIGHT (see Character Attributes). For outline characters or characters in any other mode or direction, the handling routine can dispatch the call to the graphics engine at the address given for this call in the default dispatch table.

GreCharString is called by the function *GpiCharString* . *GreCharString* is used to draw a character string from the current position within the presentation space. It updates the current presentation space position upon completion of output and produces a call to *GreSetCurrentPosition* .

```
#define INCL_GRE_STRINGS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
LONG     cChars;      /*  Number of characters in the string. */
PCH      pchString;   /*  Pointer to the character string. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;
LONG     rc;          /*  Return Codes. */

rc = GreCharString(hdc, cChars, pchString,
       pInstance, lFunction);
```

GreCharString Parameter - hdc

hdc (HDC) - input
Device context handle.

GreCharString Parameter - cChars

cChars (LONG) - input
Number of characters in the string.

GreCharString Parameter - pchString

pchString (PCH) - input
Pointer to the character string.

GreCharString Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreCharString Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=*flags* ; low-order WORD=*NGreCharString* .

GreCharString Return Value - rc

rc (LONG) - returns
Return Codes.

On completion, the handling routine must return a LONG value (*cHits*) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error.

Possible Errors Detected: When an error is detected, the handling routine must call *WinSetErrorInfo* to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_FONT_AND_MODE_MISMATCH
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_LENGTH_OR_COUNT
PMERR_PATH_LIMIT_EXCEEDED.

GreCharString - Parameters

hdc (HDC) - input
Device context handle.

cChars (LONG) - input
Number of characters in the string.

pchString (PCH) - input
Pointer to the character string.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=*flags* ; low-order WORD=*NGreCharString* .

rc (LONG) - returns
Return Codes.

On completion, the handling routine must return a LONG value (*cHits*) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error.

Possible Errors Detected: When an error is detected, the handling routine must call *WinSetErrorInfo* to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_FONT_AND_MODE_MISMATCH
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_LENGTH_OR_COUNT
PMERR_PATH_LIMIT_EXCEEDED.

GreCharString - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreCharStringPos

GreCharStringPos - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreCharStringPos draws a character string. The string can be drawn from the current (X,Y) position or from a position specified.

This function must be supported by the presentation driver. The handling routine must provide full support for drawing characters from an image font in CM_MODE1 when the character direction is CHDIRN_LEFTRIGHT (see Character Attributes). For outline characters or characters in any other mode or direction, the handling routine can dispatch the call to the graphics engine at the address given for this call in the default dispatch table.

GreCharStringPos is called by the function GpiCharStringAt. GreCharStringPos is used to draw a character string either at the current position or at a specified position. It will also update the current presentation space position upon completion of output.

```
#define INCL_GRE_STRINGS
#include <os2.h>

HDC          hdc;         /*  Device context handle. */
PPOINTL      pptlStart;   /*  Pointer to (X,Y) coordinates of start position. */
PRECTL       prclRect;    /*  Pointer to an opaque or clip rectangle. */
ULONG        flOptions;   /*  Options flag. */
LONG         cChars;      /*  Number of characters in the string. */
PCH          pchString;   /*  Pointer to the character string. */
PLONG        pAdx;        /*  Pointer to Increment array. */
PCSP_INFO    pAttrs;      /*  Pointer to attributes. */
PVOID        pInstance;   /*  Pointer to instance data. */
ULONG        lFunction;   /*  High-order WORD=flags; low-order WORD=NGreCharStringPos. */
LONG         rc;          /*  Return Codes. */
```

```
rc = GreCharStringPos(hdc, pptlStart, prclRect,
       flOptions, cChars, pchString, pAdx,
       pAttrs, pInstance, lFunction);
```

GreCharStringPos Parameter - hdc

hdc (HDC) - input
Device context handle.

GreCharStringPos Parameter - pptlStart

pptlStart (PPOINTL) - input
Pointer to (X,Y) coordinates of start position.

GreCharStringPos Parameter - prclRect

prclRect (PRECTL) - input
Pointer to an opaque or clip rectangle.

The clipping rectangle pointed to by this parameter is defined as a RECTL structure:

xLeft	Minimum X-coordinate of rectangle
yBottom	Minimum Y-coordinate
xRight	Maximum X-coordinate of rectangle
yTop	Maximum Y-coordinate

This rectangle, which is in world coordinates, is used as the clipping rectangle or as the background for the string, or both, depending on the value of flOptions. When the CHS_OPAQUE flag is set, normal background mix attributes are ignored and the rectangle is drawn using overpaint and the character background color attribute. When the CHS_OPAQUE is not set, the background is drawn using the normal method. When neither CHS_OPAQUE nor CHS_CLIP are specified, this parameter is ignored. Notice that points on the boundary of this rectangle are considered to be inside the rectangle.

GreCharStringPos Parameter - flOptions

flOptions (ULONG) - input
Options flag.

The following flags can be used in combination:

CHS_OPAQUE	Background of characters is defined by the rectangle prclRect. The rectangle is to be shaded (with background color and overpaint) before drawing.
CHS_VECTOR	Increment vector supplied (pAdx). If 0, pAdx is ignored.

CHS_LEAVEPOS	Leave current position at the start of the string.
CHS_CLIP	Clip string to rectangle.
CHS_START_XY	Start position of the string. When set, the handling routine must draw the string from the position indicated by pptlStart. If this flag is not set, the current position is used.
CHS_ATTR_INFO	Attributes to be used. When this flag is set, pAttrs indicates the foreground and background colors. Current attributes are unchanged. If the flag is not set, the string is drawn using the current character attributes. See Character Attributes.
CHS_UNDERSCORE	Underscore the characters. See the FATTRS structure in GreCreateLogicalFont.
CHS_STRIKEOUT	Overstrike the characters.

GreCharStringPos Parameter - cChars

cChars (LONG) - input
Number of characters in the string.

GreCharStringPos Parameter - pchString

pchString (PCH) - input
Pointer to the character string.

GreCharStringPos Parameter - pAdx

pAdx (PLONG) - input
Pointer to Increment array.

Pointer to an array of LONG integers, one element for each character in the string. When CHS_VECTOR is set, this array is used to set the spacing between characters. Each element is the distance in world coordinates from the bottom-left corner of the corresponding character in the string to the bottom-left corner of the next. The distance is measured along the baseline for left-to-right and right-to-left character directions, and along the shear line for top-to-bottom and bottom-to-top character directions. The final element is used to reposition the current position, when necessary.

GreCharStringPos Parameter - pAttrs

pAttrs (PCSP_INFO) - input

Pointer to attributes.

Pointer to a CSP_INFO structure. This structure contains the attributes to be used to draw the string when the CHS_ATTR_INFO flag is set. These do not alter the current character attributes (see Character Attributes). The CSP_INFO structure is defined as:

cSize	Number of bytes in structure
lColor	Use foreground color
lBackColor	Use background color

GreCharStringPos Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreCharStringPos Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCharStringPos.

GreCharStringPos Return Value - rc

rc (LONG) - returns
Return Codes.

On completion, the handling routine must return a LONG value (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlation hit (returned by display drivers when the correlation flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_FONT_AND_MODE_MISMATCH
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_LENGTH_OR_COUNT
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreCharStringPos - Parameters

hdc (HDC) - input
Device context handle.

pptlStart (PPOINTL) - input
Pointer to (X,Y) coordinates of start position.

prclRect (PRECTL) - input
Pointer to an opaque or clip rectangle.

The clipping rectangle pointed to by this parameter is defined as a RECTL structure:

xLeft	Minimum X-coordinate of rectangle
yBottom	Minimum Y-coordinate
xRight	Maximum X-coordinate of rectangle
yTop	Maximum Y-coordinate

This rectangle, which is in world coordinates, is used as the clipping rectangle or as the background for the string, or both, depending on the value of flOptions. When the CHS_OPAQUE flag is set, normal background mix attributes are ignored and the rectangle is drawn using overpaint and the character background color attribute. When the CHS_OPAQUE is not set, the background is drawn using the normal method. When neither CHS_OPAQUE nor CHS_CLIP are specified, this parameter is ignored. Notice that points on the boundary of this rectangle are considered to be inside the rectangle.

flOptions (ULONG) - input
Options flag.

The following flags can be used in combination:

CHS_OPAQUE	Background of characters is defined by the rectangle prclRect. The rectangle is to be shaded (with background color and overpaint) before drawing.
CHS_VECTOR	Increment vector supplied (pAdx). If 0, pAdx is ignored.
CHS_LEAVEPOS	Leave current position at the start of the string.
CHS_CLIP	Clip string to rectangle.
CHS_START_XY	Start position of the string. When set, the handling routine must draw the string from the position indicated by pptlStart. If this flag is not set, the current position is used.
CHS_ATTR_INFO	Attributes to be used. When this flag is set, pAttrs indicates the foreground and background colors. Current attributes are unchanged. If the flag is not set, the string is drawn using the current character attributes. See Character Attributes.
CHS_UNDERSCORE	Underscore the characters. See the FATTRS structure in GreCreateLogicalFont.
CHS_STRIKEOUT	Overstrike the characters.

cChars (LONG) - input
Number of characters in the string.

pchString (PCH) - input
Pointer to the character string.

pAdx (PLONG) - input
Pointer to Increment array.

Pointer to an array of LONG integers, one element for each character in the string. When CHS_VECTOR is set, this array is used to set the spacing between characters. Each element is the distance in world coordinates from the bottom-left corner of the corresponding character in the string to the bottom-left corner of the next. The distance is measured along the baseline for left-to-right and right-to-left

character directions, and along the shear line for top-to-bottom and bottom-to-top character directions. The final element is used to reposition the current position, when necessary.

pAttrs (PCSP_INFO) - input
Pointer to attributes.

Pointer to a CSP_INFO structure. This structure contains the attributes to be used to draw the string when the CHS_ATTR_INFO flag is set. These do not alter the current character attributes (see Character Attributes). The CSP_INFO structure is defined as:

cSize	Number of bytes in structure
lColor	Use foreground color
lBackColor	Use background color

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCharStringPos.

rc (LONG) - returns
Return Codes.

On completion, the handling routine must return a LONG value (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlation hit (returned by display drivers when the correlation flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_FONT_AND_MODE_MISMATCH
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_LENGTH_OR_COUNT
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreCharStringPos - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreCloseFigure

GreCloseFigure - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

*GreCloseFigur*e closes a figure within a path definition by drawing a line from the current (X,Y) position to the start point of the figure. Upon completion, the current position is the start point of the figure. "Open figures" can be generated by starting a new figure (with a Move function) or by ending the path without first closing the figure. GreCloseFigure is valid outside of a path definition. When this occurs, this function has no effect and the handling routine ignores it. For more information, see "GpiCloseFigure" in the *Presentation Manager Programming Reference* .

This function can be hooked by the presentation driver.

```
#define INCL_GRE_PATHS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreCloseFigure. */
BOOL     rc;          /*  This function returns BOOLEAN (fSuccess). */

rc = GreCloseFigure(hdc, pInstance, lFunction);
```

GreCloseFigure Parameter - hdc

hdc (HDC) - input
Device context handle.

GreCloseFigure Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreCloseFigure Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCloseFigure.

GreCloseFigure Return Value - rc

rc (BOOL) - returns
This function returns BOOLEAN (fSuccess).

TRUE
Successful

FALSE
Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_NOT_IN_PATH
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreCloseFigure - Parameters

hdc (HDC) - input
Device context handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCloseFigure.

rc (BOOL) - returns
This function returns BOOLEAN (fSuccess).

TRUE
Successful

FALSE
Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_NOT_IN_PATH
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreCloseFigure - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreCloseScreenChangeArea

GreCloseScreenChangeArea - Syntax

Simulation support:

This function is mandatory for display drivers that want to be DCAF-enabled.

Description *GreCloseScreenChangeArea* frees the data area internal to the display driver, identified by the SCA handle, that was accumulating screen changes.

```
#define INCL_GRE_SCREEN
#include <os2.h>
#include <pmddi.h>

HDC      hdc;     /*  Any valid device context. */
HSCA     hsca;    /*  Handle of the SCA to be closed. */
PDC      pdcArg;  /*  Pointer to instance data. */
ULONG    FunN;    /*  High-order word=Flags; Low-order word=NGreCloseScreenChangeArea. */
BOOL     rc;

rc = GreCloseScreenChangeArea(hdc, hsca, pdcArg,
       FunN);
```

GreCloseScreenChangeArea Parameter - hdc

hdc (HDC) - input
Any valid device context.

GreCloseScreenChangeArea Parameter - hsca

hsca (HSCA) - input
Handle of the SCA to be closed.

To be valid, the handle must be obtained from a previous call to GreOpenScreenChangeArea.

GreCloseScreenChangeArea Parameter - pdcArg

pdcArg (PDC) - input
Pointer to instance data.

GreCloseScreenChangeArea Parameter - FunN

FunN (ULONG) - input
High-order word=Flags; Low-order word=NGreCloseScreenChangeArea.

GreCloseScreenChangeArea Return Value - rc

rc (BOOL) - returns
This function returns one of the following:

TRUE	If the handle passed is recognized and no errors occurred.
FALSE	All other cases.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_HDC
PMERR_NO_HANDLE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreCloseScreenChangeArea - Parameters

hdc (HDC) - input
Any valid device context.

hsca (HSCA) - input
Handle of the SCA to be closed.

To be valid, the handle must be obtained from a previous call to GreOpenScreenChangeArea.

pdcArg (PDC) - input
Pointer to instance data.

FunN (ULONG) - input
High-order word=Flags; Low-order word=NGreCloseScreenChangeArea.

rc (BOOL) - returns
This function returns one of the following:

TRUE	If the handle passed is recognized and no errors occurred.
FALSE	All other cases.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_HDC
PMERR_NO_HANDLE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreCloseScreenChangeArea - Remarks

It is permissible to implement this function by returning 0 to indicate that the bits were not saved and, therefore, must be saved by the calling routine.

GreCloseScreenChangeArea calls EnterDriver and performs entry checks. The function checks that the *hsca* field matches one of the existing SCAs. If it does, the function removes the SCA from the linked list, frees the SCA memory via FreeMem and updates the linked list.

If the SCA to be closed is the last created (that is, there are no more in the linked list), then GreSetProcessControl is called to turn off the COM_SCR_BOUND bit on subsequent calls to the display driver. Resetting this bit prevents any further screen bounds accumulation until another call is made to GreOpenScreenChangeArea.

Before returning either TRUE or FALSE, GreCloseScreenChangeArea calls ExitDriver.

GreCloseScreenChangeArea - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreCombineRectRegion

GreCombineRectRegion - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreCombineRectRegion combines a region with a rectangle to make a new region. If COM_TRANSFORM is not set, the function expects the point to be in device coordinates. The destination region can be the same as the source region. An error is raised if either of the regions specified is currently selected as the clip region. The source and destination regions must be of the same device class.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_REGIONS
#include <os2.h>

HDC       hdc;         /*  Device context handle. */
HRGN      hrgnDst;     /*  Destination region handle. */
PRECTL    prclRect;    /*  Pointer to rectangle. */
HRGN      hrgnSrc;     /*  Region handle. */
LONG      cmdMode;     /*  Method of combination. */
PVOID     pInstance;   /*  Pointer to instance data. */
ULONG     lFunction;   /*  High-order WORD=flags; low-order WORD=NGreCombineRectRegion. */
LONG      rc;
```

```
rc = GreCombineRectRegion(hdc, hrgnDst, prclRect,
       hrgnSrc, cmdMode, pInstance, lFunction);
```

GreCombineRectRegion Parameter - hdc

hdc (HDC) - input
Device context handle.

GreCombineRectRegion Parameter - hrgnDst

hrgnDst (HRGN) - input
Destination region handle.

GreCombineRectRegion Parameter - prclRect

prclRect (PRECTL) - input
Pointer to rectangle.

GreCombineRectRegion Parameter - hrgnSrc

hrgnSrc (HRGN) - input
Region handle.

GreCombineRectRegion Parameter - cmdMode

cmdMode (LONG) - input
Method of combination.

CRGN_OR
Union of hrgnSrc and prclRect.

CRGN_COPY
Uses prclRect only. The hrgnSrc parameter is ignored.

CRGN_XOR
Symmetric difference of hrgnSrc and prclRect.

CRGN_AND
Intersection of hrgnSrc and prclRect.

CRGN_DIFF
Uses hrgnSrc and **not** (prclRect).

GreCombineRectRegion Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreCombineRectRegion Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCombineRectRegion.

GreCombineRectRegion Return Value - rc

rc (LONG) - returns
This function returns an integer (lComplexity) indicating the complexity of the new region:

RGN_ERROR
Error
RGN_NULL
Null region
RGN_RECT
Rectangular region
RGN_COMPLEX
Complex region (more than one rectangle)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORDINATE
PMERR_INV_HRGN
PMERR_INV_RECT
PMERR_INV_REGION_MIX_MODE
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreCombineRectRegion - Parameters

hdc (HDC) - input
Device context handle.

hrgnDst (HRGN) - input
Destination region handle.

prclRect (PRECTL) - input
Pointer to rectangle.

hrgnSrc (HRGN) - input

Region handle.

cmdMode (LONG) - input
Method of combination.

CRGN_OR
Union of hrgnSrc and prclRect.
CRGN_COPY
Uses prclRect only. The hrgnSrc parameter is ignored.
CRGN_XOR
Symmetric difference of hrgnSrc and prclRect.
CRGN_AND
Intersection of hrgnSrc and prclRect.
CRGN_DIFF
Uses hrgnSrc and **not** (prclRect).

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCombineRectRegion.

rc (LONG) - returns
This function returns an integer (lComplexity) indicating the complexity of the new region:

RGN_ERROR
Error
RGN_NULL
Null region
RGN_RECT
Rectangular region
RGN_COMPLEX
Complex region (more than one rectangle)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORDINATE
PMERR_INV_HRGN
PMERR_INV_RECT
PMERR_INV_REGION_MIX_MODE
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreCombineRectRegion - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreCombineRegion

GreCombineRegion - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreCombineRegion combines two regions to make a third. The destination region can be the same as one of the source regions. An error is raised when any one of the specified regions is currently selected as the clip region. All source and target regions must be of the same device class.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_REGIONS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
HRGN     hrgnDst;     /*  Destination region handle. */
HRGN     hrgnSrc1;    /*  First region handle. */
HRGN     hrgnSrc2;    /*  Second region handle. */
LONG     cmdMode;     /*  Method of combination. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreCombineRegion. */
LONG     rc;

rc = GreCombineRegion(hdc, hrgnDst, hrgnSrc1,
       hrgnSrc2, cmdMode, pInstance, lFunction);
```

GreCombineRegion Parameter - hdc

hdc (HDC) - input
Device context handle.

GreCombineRegion Parameter - hrgnDst

hrgnDst (HRGN) - input
Destination region handle.

GreCombineRegion Parameter - hrgnSrc1

hrgnSrc1 (HRGN) - input
First region handle.

GreCombineRegion Parameter - hrgnSrc2

hrgnSrc2 (HRGN) - input
Second region handle.

GreCombineRegion Parameter - cmdMode

cmdMode (LONG) - input
Method of combination.

CRGN_OR
Union of hrgnSrc1 and hrgnSrc2.
CRGN_COPY
Uses hrgnSrc1 only. hrgnSrc2 is ignored.
CRGN_XOR
Symmetric difference of hrgnSrc1 and hrgnSrc2.
CRGN_AND
Intersection of hrgnSrc1 and hrgnSrc2.
CRGN_DIFF
Uses hrgnSrc1 and **not** (hrgnSrc2).

GreCombineRegion Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreCombineRegion Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCombineRegion.

GreCombineRegion Return Value - rc

rc (LONG) - returns
This function returns an integer (lComplexity) indicating the complexity of the new region:

RGN_ERROR
Error
RGN_NULL
Null region
RGN_RECT
Rectangular region
RGN_COMPLEX
Complex region (more than 1 rectangle)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:
PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY

PMERR_INV_COORDINATE
PMERR_INV_HRGN
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreCombineRegion - Parameters

hdc (HDC) - input
Device context handle.

hrgnDst (HRGN) - input
Destination region handle.

hrgnSrc1 (HRGN) - input
First region handle.

hrgnSrc2 (HRGN) - input
Second region handle.

cmdMode (LONG) - input
Method of combination.

CRGN_OR
Union of hrgnSrc1 and hrgnSrc2.
CRGN_COPY
Uses hrgnSrc1 only. hrgnSrc2 is ignored.
CRGN_XOR
Symmetric difference of hrgnSrc1 and hrgnSrc2.
CRGN_AND
Intersection of hrgnSrc1 and hrgnSrc2.
CRGN_DIFF
Uses hrgnSrc1 and **not** (hrgnSrc2).

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCombineRegion.

rc (LONG) - returns
This function returns an integer (lComplexity) indicating the complexity of the new region:

RGN_ERROR
Error
RGN_NULL
Null region
RGN_RECT
Rectangular region
RGN_COMPLEX
Complex region (more than 1 rectangle)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:
PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORDINATE
PMERR_INV_HRGN
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreCombineRegion - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreCombineShortLineRegion

GreCombineShortLineRegion - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreCombineShortLineRegion combines an area lying between polyshortline pairs, which is represented by a SCANDATA structure, with a region. The pScanData parameter is ORed into the region. This function is used to build regions for path simulation. The function always expects the points in the shortlines to be in device coordinates. An error is raised when the region specified is currently selected as the clip region.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_REGIONS
#include <os2.h>

HDC          hdc;         /*  Device context handle. */
HRGN         hrgn;        /*  Region handle. */
PSCANDATA    pScanData;   /*  Pointer to a SCANDATA structure: */
PVOID        pInstance;   /*  Pointer to instance data. */
ULONG        lFunction;   /*  High-order WORD=flags; low-order WORD=NGreCombineShortLineRegion. */
BOOL         rc;          /*  This function returns BOOLEAN (fSuccess). */

rc = GreCombineShortLineRegion(hdc, hrgn,
       pScanData, pInstance, lFunction);
```

GreCombineShortLineRegion Parameter - hdc

hdc (HDC) - input
Device context handle.

GreCombineShortLineRegion Parameter - hrgn

hrgn (HRGN) - input
Region handle.

GreCombineShortLineRegion Parameter - pScanData

pScanData (PSCANDATA) - input
Pointer to a SCANDATA structure:

pslFirstLeft
Pointer to the left end of the first polyshortline

pslLastLeft
Pointer the left end of the last polyshortline

pslFirstRight
Pointer to right edge of first polyshortline

pslLastRight
Pointer to right edge of last polyshortline

c
Number of scan lines

rclBound
RECTL structure defining the bounding rectangle

slh
SHORTLINEHEADER structure:

ulStyle	Line style
ulFormat	Line format
ptlStart	(x, y) position of start
ptlStop	(x, y) position of end
lxLeft	Left edge of bounding rectangle
lxRight	Right edge of bounding rectangle
pslhNext	Pointer to next shortline
pslhPrev	Pointer to previous shortline

This structure is a discrete representation of a curve that starts at point (x0, y0) and ends at point (x1, y1). For each of the (y1-y0+1) rows, there is exactly one X value contained in the X array. The final point in the series is not drawn.

ax
Array of X values, as device coordinates.

GreCombineShortLineRegion Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreCombineShortLineRegion Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCombineShortLineRegion.

GreCombineShortLineRegion Return Value - rc

rc (BOOL) - returns
This function returns BOOLEAN (fSuccess).

TRUE
Successful
FALSE
Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:
PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_HRGN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreCombineShortLineRegion - Parameters

hdc (HDC) - input
Device context handle.

hrgn (HRGN) - input
Region handle.

pScanData (PSCANDATA) - input
Pointer to a SCANDATA structure:

pslFirstLeft
Pointer to the left end of the first polyshortline
pslLastLeft
Pointer the left end of the last polyshortline
pslFirstRight
Pointer to right edge of first polyshortline
pslLastRight
Pointer to right edge of last polyshortline
c
Number of scan lines
rclBound
RECTL structure defining the bounding rectangle
slh
SHORTLINEHEADER structure:

ulStyle	Line style
ulFormat	Line format
ptlStart	(x, y) position of start
ptlStop	(x, y) position of end
lxLeft	Left edge of bounding rectangle
lxRight	Right edge of bounding rectangle
pslhNext	Pointer to next shortline
pslhPrev	Pointer to previous shortline

This structure is a discrete representation of a curve that starts at point (x0, y0) and ends at point (x1, y1). For each of the (y1-y0+1) rows, there is exactly one X value contained in the X array. The final point in the series is not drawn.
ax
Array of X values, as device coordinates.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCombineShortLineRegion.

rc (BOOL) - returns
This function returns BOOLEAN (fSuccess).

TRUE

Successful

FALSE

Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_HRGN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreCombineShortLineRegion - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreConvert

GreConvert - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreConvert converts the specified coordinates from one coordinate space to another by using the current values of the transforms.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_XFORMS
#include <os2.h>

HDC        hdc;          /*  Device context handle. */
LONG       lSrc;         /*  Source-coordinate space. */
LONG       lDst;         /*  Target-coordinate space.  Defined by the same values as lSrc. */
PPOINTL    paptlPoint;
LONG       cPoints;      /*  Count of coordinate pairs in the array. */
PVOID      pInstance;    /*  Pointer to instance data. */
ULONG      lFunction;    /*  High-order WORD=flags; low-order WORD=NGreConvert. */
BOOL       fSuccess;     /*  This function returns BOOLEAN (fSuccess). */

fSuccess = GreConvert(hdc, lSrc, lDst, paptlPoint,
            cPoints, pInstance, lFunction);
```

GreConvert Parameter - hdc

hdc (HDC) - input
Device context handle.

GreConvert Parameter - lSrc

lSrc (LONG) - input
Source-coordinate space.

CVTC_WORLD
World-coordinate space
CVTC_MODEL
Model space
CVTC_DEFAULTPAGE
Default page-coordinate space
CVTC_PAGE
Page-coordinate space
CVTC_DEVICE
Device-coordinate space. Screen coordinates are 32-bit signed integers and are used by the presentation driver as screen pel addresses.

GreConvert Parameter - lDst

lDst (LONG) - input
Target-coordinate space. Defined by the same values as lSrc.

GreConvert Parameter - paptlPoint

paptlPoint (PPOINTL) - input
Pointer to array of (x, y) coordinates to transform. The result is also returned to this parameter.

GreConvert Parameter - cPoints

cPoints (LONG) - input
Count of coordinate pairs in the array.

GreConvert Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreConvert Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreConvert.

GreConvert Return Value - fSuccess

fSuccess (BOOL) - returns
This function returns BOOLEAN (fSuccess).

TRUE
Successful
FALSE
Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreConvert - Parameters

hdc (HDC) - input
Device context handle.

lSrc (LONG) - input
Source-coordinate space.

CVTC_WORLD
World-coordinate space
CVTC_MODEL
Model space
CVTC_DEFAULTPAGE
Default page-coordinate space
CVTC_PAGE
Page-coordinate space
CVTC_DEVICE
Device-coordinate space. Screen coordinates are 32-bit signed integers and are used by the presentation driver as screen pel addresses.

lDst (LONG) - input
Target-coordinate space. Defined by the same values as lSrc.

paptlPoint (PPOINTL) - input
Pointer to array of (x, y) coordinates to transform. The result is also returned to this parameter.

cPoints (LONG) - input
Count of coordinate pairs in the array.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreConvert.

fSuccess (BOOL) - returns
This function returns BOOLEAN (fSuccess).

TRUE
Successful
FALSE
Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:
PMERR_COORDINATE_OVERFLOW
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreConvert - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreConvertWithMatrix

GreConvertWithMatrix - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreConvertWithMatrix converts a series of points by using the indicated matrix. Other current transform matrixes are ignored.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_XFORMS
#include <os2.h>

HDC         hdc;          /*  Device context handle. */
PPOINTL     paptlPoint;
LONG        cPoints;      /*  Count of coordinate pairs in the array. */
```

```
PXFORM      paXform;
PVOID       pInstance;    /*  Pointer to instance data. */
ULONG       lFunction;    /*  High-order WORD=flags; low-order WORD=NGreConvertWithMatrix. */
BOOL        fSuccess;     /*  This function returns BOOLEAN (fSuccess). */

fSuccess = GreConvertWithMatrix(hdc, paptlPoint,
             cPoints, paXform, pInstance,
             lFunction);
```

GreConvertWithMatrix Parameter - hdc

hdc (HDC) - input
Device context handle.

GreConvertWithMatrix Parameter - paptlPoint

paptlPoint (PPOINTL) - input
Pointer to array of (X,Y) coordinates to transform. The result is also returned to this parameter.

GreConvertWithMatrix Parameter - cPoints

cPoints (LONG) - input
Count of coordinate pairs in the array.

GreConvertWithMatrix Parameter - paXform

paXform (PXFORM) - input
Pointer to an array of six matrix elements for two-dimensional formation. These are M11, M12, M21, M22, M41, and M42.

GreConvertWithMatrix Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreConvertWithMatrix Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreConvertWithMatrix.

GreConvertWithMatrix Return Value - fSuccess

fSuccess (BOOL) - returns
This function returns BOOLEAN (fSuccess).

TRUE
Successful
FALSE
Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:
PMERR_COORDINATE_OVERFLOW
PMERR_INV_LENGTH_OR_COUNT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreConvertWithMatrix - Parameters

hdc (HDC) - input
Device context handle.

paptlPoint (PPOINTL) - input
Pointer to array of (X,Y) coordinates to transform. The result is also returned to this parameter.

cPoints (LONG) - input
Count of coordinate pairs in the array.

paXform (PXFORM) - input
Pointer to an array of six matrix elements for two-dimensional formation. These are M11, M12, M21, M22, M41, and M42.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreConvertWithMatrix.

fSuccess (BOOL) - returns
This function returns BOOLEAN (fSuccess).

TRUE
Successful
FALSE
Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:
PMERR_COORDINATE_OVERFLOW
PMERR_INV_LENGTH_OR_COUNT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreConvertWithMatrix - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreCopyClipRegion

GreCopyClipRegion - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreCopyClipRegion copies the visible region, clip region, or DC region, and returns the complexity and bounds of the resulting region.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_CLIP
#include <os2.h>

HDC       hdc;          /*  Device context handle. */
HRGN      hrgn;         /*  Visible region handle. */
PRECTL    prclBounds;   /*  Pointer to the bounding rectangle of the returned region. */
ULONG     flOptions;    /*  These flags determine the type of region to be returned in hrgn: */
PVOID     pInstance;    /*  Pointer to instance data. */
ULONG     lFunction;    /*  High-order WORD=flags; low-order WORD=NGreCopyClipRegion. */
LONG      rc;           /*  Return codes. */

rc = GreCopyClipRegion(hdc, hrgn, prclBounds,
       flOptions, pInstance, lFunction);
```

GreCopyClipRegion Parameter - hdc

hdc (HDC) - input
Device context handle.

GreCopyClipRegion Parameter - hrgn

hrgn (HRGN) - input
Visible region handle.

GreCopyClipRegion Parameter - prclBounds

prclBounds (PRECTL) - input
Pointer to the bounding rectangle of the returned region.

The bounding rectangle is returned in the same coordinate system as the region defined by flOptions. This rectangle is inclusive at the bottom and left boundaries, exclusive at the top and right boundaries. When specified as NULL, the bounding rectangle is not returned.

GreCopyClipRegion Parameter - flOptions

flOptions (ULONG) - input
These flags determine the type of region to be returned in hrgn:

COPYCRGN_ALLINTERSECT
The function must return the intersection of all clipping. This value describes the DC region and is expressed in screen coordinates.
COPYCRGN_VISRGN
The function must return a copy of the visible region only. This value is returned in screen coordinates.
COPYCRGN_CLIPRGN
The function must return a copy of the clip region only. The clip region is expressed in device coordinates.

GreCopyClipRegion Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreCopyClipRegion Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCopyClipRegion.

GreCopyClipRegion Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (lComplexity) indicating the complexity of the region:

RGN_ERROR	Error
RGN_NULL	Null region
RGN_RECT	Rectangular region

RGN_COMPLEX Complex region (more than one rectangle)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_RECT
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreCopyClipRegion - Parameters

hdc (HDC) - input
Device context handle.

hrgn (HRGN) - input
Visible region handle.

prclBounds (PRECTL) - input
Pointer to the bounding rectangle of the returned region.

The bounding rectangle is returned in the same coordinate system as the region defined by flOptions. This rectangle is inclusive at the bottom and left boundaries, exclusive at the top and right boundaries. When specified as NULL, the bounding rectangle is not returned.

flOptions (ULONG) - input
These flags determine the type of region to be returned in hrgn:

COPYCRGN_ALLINTERSECT
The function must return the intersection of all clipping. This value describes the DC region and is expressed in screen coordinates.
COPYCRGN_VISRGN
The function must return a copy of the visible region only. This value is returned in screen coordinates.
COPYCRGN_CLIPRGN
The function must return a copy of the clip region only. The clip region is expressed in device coordinates.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCopyClipRegion.

rc (LONG) - returns
Return codes.

This function returns an integer (lComplexity) indicating the complexity of the region:

RGN_ERROR Error
RGN_NULL Null region
RGN_RECT Rectangular region
RGN_COMPLEX Complex region (more than one rectangle)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORDINATE
PMERR_INV_HDC

PMERR_INV_HRGN
PMERR_INV_RECT
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreCopyClipRegion - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreCreateLogColorTable

GreCreateLogColorTable - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreCreateLogColorTable defines the entries of the Logical Color Table.

This function must be supported by the presentation driver. GreCreateLogColorTable is called by GpiCreateLogColorTable to create a Logical Color Table, which is used in subsequent drawing operations.

```
#define INCL_GRE_COLORTABLE
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    flOptions;   /*  Options flag. */
LONG     lFormat;     /*  Format of entries in the table. */
LONG     lStart;      /*  Starting index. */
LONG     cCount;      /*  Number of elements supplied in the application data area. */
PLONG    pData;       /*  Pointer to the application data area. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;
BOOL     rc;          /*  Return Codes. */

rc = GreCreateLogColorTable(hdc, flOptions,
       lFormat, lStart, cCount, pData, pInstance,
       lFunction);
```

GreCreateLogColorTable Parameter - hdc

hdc (HDC) - input
Device context handle.

GreCreateLogColorTable Parameter - flOptions

flOptions (ULONG) - input
Options flag.

Valid options are:

LCOL_RESET	Indicates that the handling routine must reset the color table to default before processing the remainder of this function. **Note:** This option is assumed when the color table is changed from LCOLF_RGB to LCOLF_INDRGB or LCOLF_CONSECRGB.
LCOL_REALIZABLE	Indicates that the application can call GreRealizeColorTable at the appropriate time. This can affect the way the handling routine maps the indexes when the Logical Color Table is not realized. A realizable color table is only required to provide color mapping for its color indexes while it is realized. If this flag is not set, GreRealizeColorTable has no effect and posts a warning.
LCOL_PURECOLOR	For solid patterns, pattern colors that are not available can be approximated by dithering. When this flag is set, only pure colors are used; the handling routine must not dither colors. The default is to allow color dithering.

Other flags are reserved and must be 0.

GreCreateLogColorTable Parameter - lFormat

lFormat (LONG) - input
Format of entries in the table.

Valid formats are:

LCOLF_INDRGB	Array of (index, RGB) values. Each pair of values contains eight bytes, a four-byte index and a four-byte color. This flag sets the color table into index mode, and forces LCOL_RESET if it is in RGB mode.
LCOLF_CONSECRGB	Array of (RGB) values corresponding to color indexes starting from lStart upwards. Each entry is a four-byte value. This flag sets the color table into index mode, and forces LCOL_RESET if it is in RGB mode.
LCOLF_RGB	Color index = RGB. This flag sets the color table to RGB mode.

GreCreateLogColorTable Parameter - lStart

lStart (LONG) - input
Starting index.

Starting index. Only relevant when lFormat is set to LCOLF_CONSECRGB.

GreCreateLogColorTable Parameter - cCount

cCount (LONG) - input
Number of elements supplied in the application data area.

The number of elements supplied in pData. This flag can be set to 0 if the color table is to be reset to the default or LCOLF_RGB. When this is 0, LCOLF_INDRGB and LCOLF_CONSECRGB have the same effect.

For LCOLF_INDRGB, *cCount* must be an even number.

GreCreateLogColorTable Parameter - pData

pData (PLONG) - input
Pointer to the application data area.

Pointer to the data area containing the color table definition data. The format depends on the value of lFormat. Each color value is a four-byte integer with a value of:

```
(R*65536) + (G*256) + B

Where:
      R = red color intensity value
      G = green color intensity value
      B = blue color intensity value
```

The maximum intensity for each primary is 255.

GreCreateLogColorTable Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreCreateLogColorTable Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCreateLogColorTable.

GreCreateLogColorTable Return Value - rc

rc (BOOL) - returns
Return Codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: Error checking for this function is performed by the graphics engine. Error codes for conditions the handling routine can expect to be passed by the graphics engine include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_FORMAT
PMERR_INV_COLOR_INDEX
PMERR_INV_COLOR_START_INDEX
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_REALIZE_NOT_SUPPORTED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreCreateLogColorTable - Parameters

hdc (HDC) - input
Device context handle.

flOptions (ULONG) - input
Options flag.

Valid options are:

LCOL_RESET	Indicates that the handling routine must reset the color table to default before processing the remainder of this function. **Note:** This option is assumed when the color table is changed from LCOLF_RGB to LCOLF_INDRGB or LCOLF_CONSECRGB.
LCOL_REALIZABLE	Indicates that the application can call GreRealizeColorTable at the appropriate time. This can affect the way the handling routine maps the indexes when the Logical Color Table is not realized. A realizable color table is only required to provide color mapping for its color indexes while it is realized. If this flag is not set, GreRealizeColorTable has no effect and posts a warning.
LCOL_PURECOLOR	For solid patterns, pattern colors that are not available can be approximated by dithering. When this flag is set, only pure colors are used; the handling routine must not dither colors. The default is to allow color dithering.

Other flags are reserved and must be 0.

lFormat (LONG) - input
Format of entries in the table.

Valid formats are:

LCOLF_INDRGB	Array of (index, RGB) values. Each pair of values contains eight bytes, a four-byte index and a four-byte color. This flag sets the color table into index mode, and forces LCOL_RESET if it is in RGB mode.
LCOLF_CONSECRGB	Array of (RGB) values corresponding to color indexes starting from lStart

upwards. Each entry is a four-byte value. This flag sets the color table into index mode, and forces LCOL_RESET if it is in RGB mode.

LCOLF_RGB — Color index = RGB. This flag sets the color table to RGB mode.

lStart (LONG) - input
Starting index.

Starting index. Only relevant when lFormat is set to LCOLF_CONSECRGB.

cCount (LONG) - input
Number of elements supplied in the application data area.

The number of elements supplied in pData. This flag can be set to 0 if the color table is to be reset to the default or LCOLF_RGB. When this is 0, LCOLF_INDRGB and LCOLF_CONSECRGB have the same effect.

For LCOLF_INDRGB, *cCount* must be an even number.

pData (PLONG) - input
Pointer to the application data area.

Pointer to the data area containing the color table definition data. The format depends on the value of lFormat. Each color value is a four-byte integer with a value of:

```
(R*65536) + (G*256) + B

Where:
      R = red color intensity value
      G = green color intensity value
      B = blue color intensity value
```

The maximum intensity for each primary is 255.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCreateLogColorTable.

rc (BOOL) - returns
Return Codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: Error checking for this function is performed by the graphics engine. Error codes for conditions the handling routine can expect to be passed by the graphics engine include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_FORMAT
PMERR_INV_COLOR_INDEX
PMERR_INV_COLOR_START_INDEX
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_REALIZE_NOT_SUPPORTED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreCreateLogColorTable - Remarks

Refer to the function "GpiCreateLogColorTable" in the *Presentation Manager Programming Reference* for more information.

GreCreateLogColorTable - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreCreateRectRegion

GreCreateRectRegion - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreCreateRectRegion creates a region by taking the OR of a series of rectangles. When no rectangles are specified (that is, cRect is 0), an empty region is created. If COM_TRANSFORM is not set, the function expects the points to be in device coordinates.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_REGIONS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
PRECT    paRegion;    /*  Pointer to the region definition. */
LONG     cRect;       /*  Number of rectangles in the region definition. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreCreateRectRegion. */
HRGN     rc;          /*  Return codes. */

rc = GreCreateRectRegion(hdc, paRegion, cRect,
       pInstance, lFunction);
```

GreCreateRectRegion Parameter - hdc

hdc (HDC) - input
Device context handle.

GreCreateRectRegion Parameter - paRegion

paRegion (PRECT) - input
Pointer to the region definition.

This is a pointer to an array of rectangle structures, which defines the region. Each rectangle is described by a RECTL structure:

xLeft	Minimum x-coordinate of rectangle
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of rectangle
yTop	Maximum y-coordinate

For each rectangle, xRight must be equal to or greater than xLeft. yTop must be equal to or greater than yBottom. The bottom and left boundaries of each rectangle are part of the interior of the region; the top and right boundaries are not.

GreCreateRectRegion Parameter - cRect

cRect (LONG) - input
Number of rectangles in the region definition.

If this is *0* , an empty region is created.

GreCreateRectRegion Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreCreateRectRegion Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCreateRectRegion.

GreCreateRectRegion Return Value - rc

rc (HRGN) - returns
Return codes.

On completion, the handling routine returns the region handle (hrgn), or RGN_ERROR if an error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORDINATE
PMERR_INV_HRGN
PMERR_INV_LENGTH_OR_COUNT

PMERR_INV_RECT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreCreateRectRegion - Parameters

hdc (HDC) - input
Device context handle.

paRegion (PRECT) - input
Pointer to the region definition.

This is a pointer to an array of rectangle structures, which defines the region. Each rectangle is described by a RECTL structure:

xLeft	Minimum x-coordinate of rectangle
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of rectangle
yTop	Maximum y-coordinate

For each rectangle, xRight must be equal to or greater than xLeft. yTop must be equal to or greater than yBottom. The bottom and left boundaries of each rectangle are part of the interior of the region; the top and right boundaries are not.

cRect (LONG) - input
Number of rectangles in the region definition.

If this is *0* , an empty region is created.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCreateRectRegion.

rc (HRGN) - returns
Return codes.

On completion, the handling routine returns the region handle (hrgn), or RGN_ERROR if an error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORDINATE
PMERR_INV_HRGN
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreCreateRectRegion - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreDeath

GreDeath - Syntax

Simulation support:

This function is mandatory for display drivers. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreDeath informs the presentation driver that the entire screen is required by another screen group (an application that is not running under the Presentation Manager interface).

```
#define INCL_GRE_DEVMISC3
#include <os2.h>

HDC      hdc;         /*  The device context handle. */
PVOID    pInstance;   /*  A pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreDeath. */
BOOL     fSuccess;    /*  Return codes. */

fSuccess = GreDeath(hdc, pInstance, lFunction);
```

GreDeath Parameter - hdc

hdc (HDC) - input
The device context handle.

GreDeath Parameter - pInstance

pInstance (PVOID) - input
A pointer to instance data.

GreDeath Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeath.

GreDeath Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. An error code for conditions that the handling routine is expected to check:

PMERR_DEV_FUNC_NOT_INSTALLED.

GreDeath - Parameters

hdc (HDC) - input
The device context handle.

pInstance (PVOID) - input
A pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeath.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. An error code for conditions that the handling routine is expected to check:

PMERR_DEV_FUNC_NOT_INSTALLED.

GreDeath - Remarks

Any current Presentation Manager applications are set to the background. While this condition exists, the presentation driver must make all calls as usual. However, it might not affect the underlying hardware, that is, the presentation driver must continue to accumulate bounds and respond to queries but it might not actually draw to the display. When GreResurrection is called, the missing output will be re-created by the system sending a WM_PAINT message to the application.

This function goes directly to the Presentation Manager interface.

GreDeath - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreDestroyRegion

GreDestroyRegion - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreDestroyRegion deletes the specified region unless it has been selected as a clipping region. In this case, an error is raised. When a null region is specified, GreDestroyRegion does not delete any region and returns without logging an error.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_REGIONS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
HRGN     hrgn;        /*  Region handle. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreDestroyRegion. */
BOOL     fSuccess;    /*  Reurn codes. */

fSuccess = GreDestroyRegion(hdc, hrgn, pInstance,
            lFunction);
```

GreDestroyRegion Parameter - hdc

hdc (HDC) - input
Device context handle.

GreDestroyRegion Parameter - hrgn

hrgn (HRGN) - input
Region handle.

GreDestroyRegion Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreDestroyRegion Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDestroyRegion.

GreDestroyRegion Return Value - fSuccess

fSuccess (BOOL) - returns
Reurn codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HRGN_BUSY
PMERR_INV_HRGN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDestroyRegion - Parameters

hdc (HDC) - input
Device context handle.

hrgn (HRGN) - input
Region handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDestroyRegion.

fSuccess (BOOL) - returns
Reurn codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HRGN_BUSY
PMERR_INV_HRGN

PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDestroyRegion - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreDeviceAnimatePalette

GreDeviceAnimatePalette - Syntax

Simulation support:

This function is simulated by a handling routine in Revision 2.0 (or higher) of the graphics engine.

Description

GreDeviceAnimatePalette is the presentation driver version of GreAnimatePalette. It is called to process an application's GreAnimatePalette request. The purpose of this function is to modify the RGB values in the hardware palette rapidly and make the changes apparent immediately. Changes are reflected immediately for each logical palette entry that is marked as PC-RESERVED and is associated with a hardware slot. The RGB value on the logical application's palette, as well as the hardware palette itself, are updated.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_PALETTE
#include <os2.h>

HDC       hdc;         /*  Device context handle. */
ULONG     hdevpal;     /*  Device palette handle. */
ULONG     ulFormat;    /*  Specifies the entry format. Must be LCOLF_CONSECRGB. */
ULONG     ulStart;     /*  Starting index, that is, the first palette entry to change. */
ULONG     cclr;        /*  Count of the palette entries to change. */
PULONG    pclr;        /*  Pointer to table of new RGB2 palette entries. */
PVOID     pInstance;   /*  Pointer to instance data. */
ULONG     lFunction;   /*  High-order WORD=flags; low-order WORD=NGreDeviceAnimatePalette. */
ULONG     rc;          /*  Return codes. */

rc = GreDeviceAnimatePalette(hdc, hdevpal,
       ulFormat, ulStart, cclr, pclr, pInstance,
       lFunction);
```

GreDeviceAnimatePalette Parameter - hdc

hdc (HDC) - input
Device context handle.

GreDeviceAnimatePalette Parameter - hdevpal

hdevpal (ULONG) - input
Device palette handle.

GreDeviceAnimatePalette Parameter - ulFormat

ulFormat (ULONG) - input
Specifies the entry format. Must be LCOLF_CONSECRGB.

GreDeviceAnimatePalette Parameter - ulStart

ulStart (ULONG) - input
Starting index, that is, the first palette entry to change.

GreDeviceAnimatePalette Parameter - cclr

cclr (ULONG) - input
Count of the palette entries to change.

GreDeviceAnimatePalette Parameter - pclr

pclr (PULONG) - input
Pointer to table of new RGB2 palette entries.

GreDeviceAnimatePalette Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreDeviceAnimatePalette Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceAnimatePalette.

GreDeviceAnimatePalette Return Value - rc

rc (ULONG) - returns
Return codes.

PAL-ERROR Error	Count of the hardware palette slots changed.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_PALETTE_BUSY
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH.

GreDeviceAnimatePalette - Parameters

hdc (HDC) - input
Device context handle.

hdevpal (ULONG) - input
Device palette handle.

ulFormat (ULONG) - input
Specifies the entry format. Must be LCOLF_CONSECRGB.

ulStart (ULONG) - input
Starting index, that is, the first palette entry to change.

cclr (ULONG) - input
Count of the palette entries to change.

pclr (PULONG) - input
Pointer to table of new RGB2 palette entries.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceAnimatePalette.

rc (ULONG) - returns
Return codes.

PAL-ERROR Error	Count of the hardware palette slots changed.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_PALETTE_BUSY

PMERR_INV_IN_AREA
PMERR_INV_IN_PATH.

GreDeviceAnimatePalette - Remarks

This function modifies the colors in a palette. Changes are reflected immediately for each animated index that has a hardware slot.

GreDeviceAnimatePalette - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreDeviceCreateBitmap

GreDeviceCreateBitmap - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreDeviceCreateBitmap creates a bit map and obtains its handle.

This function must be supported by the presentation driver. GreDeviceCreateBitmap is called from GreCreateBitmap, which is one of the graphics engine internal device support functions.

```
#define INCL_GRE_BITMAPS
#include <os2.h>

HDC                 hdc;         /*  Device context handle. */
PBITMAPINFOHEADER   pInfoHd;     /*  Pointer to data structure. */
ULONG               flUsage;
PBYTE               pBitmap;     /*  Pointer to bit-map initialization data. */
PBITMAPINFO         pInfo;       /*  Pointer to BITMAPINFO or BITMAPINFO2 structure. */
PVOID               pInstance;   /*  Pointer to instance data. */
ULONG               lFunction;   /*  High-order WORD=flags; low-order WORD=NGreDeviceCreateBitmap. */
ULONG               rc;          /*  Return Code. */

rc = GreDeviceCreateBitmap(hdc, pInfoHd, flUsage,
       pBitmap, pInfo, pInstance, lFunction);
```

GreDeviceCreateBitmap Parameter - hdc

hdc (HDC) - input
Device context handle.

GreDeviceCreateBitmap Parameter - pInfoHd

pInfoHd (PBITMAPINFOHEADER) - input
Pointer to data structure.

Pointer to a BITMAPINFOHEADER or BITMAPINFOHEADER2 data structure that defines the new bit map.

The BITMAPINFOHEADER data structure contains the following fields:

cbFix	Length of this structure in bytes
cx	Bit-map width
cy	Bit-map height
cPlanes	Number of color planes; 1 if standard format
cBitCount	Number of adjacent color bits per pel Notice that each plane has ((cx*cBitCount+31)/32*4*cy) bytes.

The BITMAPINFOHEADER2 data structure contains the following fields:

cbFix	Length in bytes of this structure
cx	Bit-map width
cy	Bit-map height
cPlanes	Number of color planes; 1 if standard format
cBitCount	Number of adjacent color bits per pel
ulCompression	Compression scheme used to store the bit map: BCA_UNCOMP — Bit map is uncompressed (the only valid value).
cbImage	Length of bit-map storage data in bytes. If the bit map is uncompressed, 0 (default) can be specified for this.
cxResolution	Horizontal component of the resolution of the target device. That is, the resolution of the device the bit map is intended for in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cyResolution	Vertical component of the resolution of the target device. That is, the resolution of the device the bit map is intended for in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cclrUsed	The number of color indexes from the color table that are used by the bit map. If it is 0 (the default), all the indexes are used. If it is nonzero, only the first cclrUsed

entries in the table are accessed by the system. Further entries can be omitted.

For standard formats with a cBitCount of one, four, or eight (and cPlanes=1), any indexes beyond cclrUsed are not valid. For example, a bit map with 64 colors can use the 8-bitcount format without having to supply the other 192 entries in the color table. For the 24-bitcount standard format, cclrUsed is the number of colors used by the bit map.

cclrImportant

Minimum number of color indexes for satisfactory appearance of the bit map. More colors can be used in the bit map, however, it is not necessary to assign them to the device palette. These additional colors can be mapped to the nearest colors available. Zero (the default) means that all entries are important. For a 24-bitcount standard format, the cclrImportant colors are also listed in the color table relating to this bit map.

usUnits

Units of measure of the horizontal and vertical components of resolution:

BRU_METRIC	(Default) Pels per meter

usReserved

Reserved field. If present, it must be 0.

usRecording

Recording algorithm, the format in which bit-map data is recorded:

BRA_BOTTOMUP	(Default) Scanlines are recorded from bottom to top

usRendering

Halftoning algorithm used to record bit-map data that has been digitally halftoned:

BRH_NOTHALFTONED

BRH_ERRORDIFFUSION

BRH_PANDA

BRH_SUPERCIRCLE

cSize1	Size value 1. If BRH_ERRORDIFFUSION is specified in usRendering, cSize1 is the error damping as a percentage in the range 0-100. A value of 100% indicates no damping. A value of 0% indicates that any errors are not diffused. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize1 is the X-dimension of the pattern used in pels.	
cSize2	Size value 2. If BRH_ERRORDIFFUSION is specified in usRendering, this parameter is ignored. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize2 is the Y-dimension of the pattern used in pels.	
ulColorEncoding	Color encoding:	
	BCE_RGB	(Default) Each element in the color array is an RGB2 data type.
ulIdentifier	Reserved for application use	

GreDeviceCreateBitmap Parameter - flUsage

flUsage (ULONG) - input
Additional information used when creating a new bit map. The only valid flag is:

CBM_INIT	When set, the pBitmap and paInfo parameters are used to initialize the newly created bit map. It is assumed that enough data is passed to initialize the whole bit map.

Other flags are reserved and must be ignored by the handling routine.

GreDeviceCreateBitmap Parameter - pBitmap

pBitmap (PBYTE) - input
Pointer to bit-map initialization data.

Pointer to bit-map initialization data, starting on a DWORD-aligned address. This data is stored in the order that the coordinates appear on a display screen, that is, the pel in the lower-left corner is the first in the bit map. Pels are scanned to the right, and upward, from that position. The bits of the first pel are stored beginning with the most significant bits of the first byte. The data for pels in each scan line is packed together tightly. However, all scan lines are padded at the end so that each one begins on a ULONG boundary. That is, three bytes of pel data will hold one 24-bit pel, three 8-bit pels, six 4-bit pels, or twenty-four 1-bit pels. If those three bytes are the only pel data for that scan line, one more byte of zeros would be required to pad the line to a ULONG boundary.

GreDeviceCreateBitmap Parameter - pInfo

pInfo (PBITMAPINFO) - input
Pointer to BITMAPINFO or BITMAPINFO2 structure.

Pointer to either a BITMAPINFO structure:

cbFix	Length of structure
cx	Bit-map width
cy	Bit-map height
cPlanes	Number of color planes; 1 if standard format
cBitCount	Number of adjacent color bits per pel
argbColor[ ]	Color table array of RGB structures: bBlue bGreen bRed

or pointer to a BITMAPINFO2 structure:

cbFix	Length of structure
cx	Bit-map width
cy	Bit-map height
cPlanes	Number of color planes, 1 if standard format
cBitCount	Number of adjacent color bits per pel
ulCompression	Compression scheme used to store the bit map: BCA_UNCOMP — Bit map is uncompressed (the only valid value).
cbImage	Length of bit-map storage data in bytes. If the bit map is uncompressed, 0 (default) can be specified for this.
cxResolution	Horizontal component of the resolution of the target device. That is, the resolution of the device the bit map is intended for in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cyResolution	Vertical component of the resolution of the target device. That is, the resolution of the device the bit map is intended for in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cclrUsed	The number of color indexes from the color table that are used by the bit map. If it is 0 (the default), all the indexes are used. If it is nonzero, only the first cclrUsed entries in the table are accessed by the system. Further entries can be omitted. For standard formats with a cBitCount of 1, 4, or 8 (and cPlanes=1), any indexes beyond cclrUsed are not valid. For example, a bit map with 64 colors can use the 8-bitcount format without having to supply the other 192 entries in the color table. For the 24-bitcount standard format, cclrUsed is the number of colors used by the bit map.
cclrImportant	Minimum number of color indexes for satisfactory appearance of the bit map. More colors can be used in the bit map, however, it is not necessary to assign them to the device palette. These additional colors can be mapped to the nearest colors available. Zero (default) means that all entries are important. For a 24-bitcount standard format, the cclrImportant colors are also listed

in the color table relating to this bit map.

usUnits

Units of measure of the horizontal and vertical components of resolution:

BRU_METRIC (Default) Pels per meter.

usReserved

Reserved field. If present, it must be 0.

usRecording

Recording algorithm, the format in which bit-map data is recorded:

BRA_BOTTOMUP (Default) Scanlines are recorded from bottom to top.

usRendering

Halftoning algorithm used to record bit-map data that has been digitally halftoned:

BRH_NOTHALFTONED

BRH_ERRORDIFFUSION

BRH_PANDA

BRH_SUPERCIRCLE

cSize1

Size value 1. If BRH_ERRORDIFFUSION is specified in usRendering, cSize1 is the error damping as a percentage in the range 0-100. A value of 100% indicates no damping. A value of 0% indicates that any errors are not diffused.

If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize1 is the X-dimension of the pattern used in pels.

cSize2	Size value 2. If BRH_ERRORDIFFUSION is specified in usRendering, this parameter is ignored. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize2 is the X-dimension of the pattern used in pels.	
ulColorEncoding	Color encoding:	
	BCE_RGB	(Default) Each element in the color array is an RGB2 data type.
ulIdentifier	Reserved for application use	
argbColor[]	Color table array of RGB2 structures:	
	bBlue bGreen bRed fcOptions	Reserved

GreDeviceCreateBitmap Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreDeviceCreateBitmap Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceCreateBitmap.

GreDeviceCreateBitmap Return Value - rc

rc (ULONG) - returns
Return Code.

On completion, the handling routine must return the bit-map handle (hbm), or GPI_ERROR if an error is detected.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_HDC
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SCAN_START

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDeviceCreateBitmap - Parameters

hdc (HDC) - input
Device context handle.

pInfoHd (PBITMAPINFOHEADER) - input
Pointer to data structure.

Pointer to a BITMAPINFOHEADER or BITMAPINFOHEADER2 data structure that defines the new bit map.

The BITMAPINFOHEADER data structure contains the following fields:

cbFix	Length of this structure in bytes
cx	Bit-map width
cy	Bit-map height
cPlanes	Number of color planes; 1 if standard format
cBitCount	Number of adjacent color bits per pel Notice that each plane has ((cx*cBitCount+31)/32*4*cy) bytes.

The BITMAPINFOHEADER2 data structure contains the following fields:

cbFix	Length in bytes of this structure
cx	Bit-map width
cy	Bit-map height
cPlanes	Number of color planes; 1 if standard format
cBitCount	Number of adjacent color bits per pel
ulCompression	Compression scheme used to store the bit map: BCA_UNCOMP — Bit map is uncompressed (the only valid value).
cbImage	Length of bit-map storage data in bytes. If the bit map is uncompressed, 0 (default) can be specified for this.
cxResolution	Horizontal component of the resolution of the target device. That is, the resolution of the device the bit map is intended for in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cyResolution	Vertical component of the resolution of the target device. That is, the resolution of the device the bit map is intended for in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cclrUsed	The number of color indexes from the color table that are used by the bit map. If it is 0 (the default), all the indexes are used. If it is nonzero, only the first cclrUsed entries in the table are accessed by the system. Further entries can be omitted. For standard formats with a cBitCount of one, four, or eight (and cPlanes=1), any indexes beyond cclrUsed are not valid. For example, a bit map with 64 colors can use the 8-bitcount format without having to supply the other 192 entries in the color table. For the 24-bitcount standard format, cclrUsed is the number of colors used

by the bit map.

cclrImportant

Minimum number of color indexes for satisfactory appearance of the bit map. More colors can be used in the bit map, however, it is not necessary to assign them to the device palette. These additional colors can be mapped to the nearest colors available. Zero (the default) means that all entries are important. For a 24-bitcount standard format, the cclrImportant colors are also listed in the color table relating to this bit map.

usUnits

Units of measure of the horizontal and vertical components of resolution:

BRU_METRIC	(Default) Pels per meter

usReserved

Reserved field. If present, it must be 0.

usRecording

Recording algorithm, the format in which bit-map data is recorded:

BRA_BOTTOMUP	(Default) Scanlines are recorded from bottom to top

usRendering

Halftoning algorithm used to record bit-map data that has been digitally halftoned:

BRH_NOTHALFTONED

BRH_ERRORDIFFUSION

BRH_PANDA

BRH_SUPERCIRCLE

cSize1	Size value 1. If BRH_ERRORDIFFUSION is specified in usRendering, cSize1 is the error damping as a

percentage in the range 0-100. A value of 100% indicates no damping. A value of 0% indicates that any errors are not diffused.

If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize1 is the X-dimension of the pattern used in pels.

cSize2 — Size value 2. If BRH_ERRORDIFFUSION is specified in usRendering, this parameter is ignored. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize2 is the Y-dimension of the pattern used in pels.

ulColorEncoding — Color encoding:

BCE_RGB	(Default) Each element in the color array is an RGB2 data type.

ulIdentifier — Reserved for application use

flUsage (ULONG) - input
Additional information used when creating a new bit map. The only valid flag is:

CBM_INIT	When set, the pBitmap and paInfo parameters are used to initialize the newly created bit map. It is assumed that enough data is passed to initialize the whole bit map.

Other flags are reserved and must be ignored by the handling routine.

pBitmap (PBYTE) - input
Pointer to bit-map initialization data.

Pointer to bit-map initialization data, starting on a DWORD-aligned address. This data is stored in the order that the coordinates appear on a display screen, that is, the pel in the lower-left corner is the first in the bit map. Pels are scanned to the right, and upward, from that position. The bits of the first pel are stored beginning with the most significant bits of the first byte. The data for pels in each scan line is packed together tightly. However, all scan lines are padded at the end so that each one begins on a ULONG boundary. That is, three bytes of pel data will hold one 24-bit pel, three 8-bit pels, six 4-bit pels, or twenty-four 1-bit pels. If those three bytes are the only pel data for that scan line, one more byte of zeros would be required to pad the line to a ULONG boundary.

pInfo (PBITMAPINFO) - input
Pointer to BITMAPINFO or BITMAPINFO2 structure.

Pointer to either a BITMAPINFO structure:

cbFix	Length of structure
cx	Bit-map width
cy	Bit-map height
cPlanes	Number of color planes; 1 if standard format
cBitCount	Number of adjacent color bits per pel
argbColor[ ]	Color table array of RGB structures: bBlue bGreen bRed

or pointer to a BITMAPINFO2 structure:

cbFix — Length of structure

cx — Bit-map width

cy — Bit-map height

cPlanes — Number of color planes, 1 if standard format

cBitCount — Number of adjacent color bits per pel

ulCompression — Compression scheme used to store the bit map:

BCA_UNCOMP	Bit map is uncompressed (the only

valid value).

cbImage — Length of bit-map storage data in bytes. If the bit map is uncompressed, 0 (default) can be specified for this.

cxResolution — Horizontal component of the resolution of the target device. That is, the resolution of the device the bit map is intended for in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.

cyResolution — Vertical component of the resolution of the target device. That is, the resolution of the device the bit map is intended for in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.

cclrUsed — The number of color indexes from the color table that are used by the bit map. If it is 0 (the default), all the indexes are used. If it is nonzero, only the first cclrUsed entries in the table are accessed by the system. Further entries can be omitted.

For standard formats with a cBitCount of 1, 4, or 8 (and cPlanes=1), any indexes beyond cclrUsed are not valid. For example, a bit map with 64 colors can use the 8-bitcount format without having to supply the other 192 entries in the color table. For the 24-bitcount standard format, cclrUsed is the number of colors used by the bit map.

cclrImportant — Minimum number of color indexes for satisfactory appearance of the bit map. More colors can be used in the bit map, however, it is not necessary to assign them to the device palette. These additional colors can be mapped to the nearest colors available. Zero (default) means that all entries are important. For a 24-bitcount standard format, the cclrImportant colors are also listed in the color table relating to this bit map.

usUnits — Units of measure of the horizontal and vertical components of resolution:

BRU_METRIC	(Default) Pels per meter.

usReserved — Reserved field. If present, it must be 0.

usRecording — Recording algorithm, the format in which bit-map data is recorded:

BRA_BOTTOMUP	(Default) Scanlines are re

coredfrombottomtotop.

usRendering	Halftoning algorithm used to record bit-map data that has been digitally halftoned: BRH_NOTHALFTONED

BRH_PANDA

		BRH_SUPERCIRCLE
cSize1	Size value 1. If BRH_ERRORDIFFUSION is specified in usRendering, cSize1 is the error damping as a percentage in the range 0-100. A value of 100% indicates no damping. A value of 0% indicates that any errors are not diffused. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize1 is the X-dimension of the pattern used in pels.	
cSize2	Size value 2. If BRH_ERRORDIFFUSION is specified in usRendering, this parameter is ignored. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize2 is the X-dimension of the pattern used in pels.	
ulColorEncoding	Color encoding:	
	BCE_RGB	(Default) Each element in the color array is an RGB2 data type.
ulIdentifier	Reserved for application use	
argbColor[]	Color table array of RGB2 structures:	
	bBlue bGreen bRed fcOptions	Reserved

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceCreateBitmap.

rc (ULONG) - returns
Return Code.

On completion, the handling routine must return the bit-map handle (hbm), or GPI_ERROR if an error is detected.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_HDC
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SCAN_START

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDeviceCreateBitmap - Remarks

Bit-map size is limited by available memory. The maximum width and height are 64KB. Typically, the following standard bit-map formats are used:

Bitcount	Planes
1	1
4	1
8	1
24	1

All presentation drivers must be able to create and use all of the standard formats. However, presentation drivers for two-color devices will lose the color information.

The device context handle supplied to this function must never be NULL, because bit maps always belong to some device. The bit map is created on the device specified and can be selected to a different device later because the graphics engine can handle transfer of bits from one device to the other. When a presentation driver supports only a single color format, requests for other color bit-map formats are mapped to the supported function. No error is returned.

GreDeviceCreateBitmap - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreDeviceCreatePalette

GreDeviceCreatePalette - Syntax

Simulation support:

This function is simulated by a handling routine in Revision 2.0 (or later) of the graphics engine.

Description

GreDeviceCreatePalette is called by GreSelectPalette, to process an application's GpiSelectPalette request. The presentation driver is expected to perform any allocation and data structure initialization needed for a subsequent GreDeviceRealizePalette to succeed.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_PALETTE
#include <os2.h>

HDC             hdc;         /*  Device context handle. */
PPALETTEINFO    ppalinfo;    /*  Pointer to table of palette data. */
ULONG           hdevpal;     /*  Device palette handle (NULL, if new). */
PVOID           pInstance;   /*  Pointer to instance data. */
ULONG           lFunction;   /*  High-order WORD=flags; low-order WORD=NGreDeviceCreatePalette. */
ULONG           rc;          /*  Return codes. */

rc = GreDeviceCreatePalette(hdc, ppalinfo,
       hdevpal, pInstance, lFunction);
```

GreDeviceCreatePalette Parameter - hdc

hdc (HDC) - input
Device context handle.

GreDeviceCreatePalette Parameter - ppalinfo

ppalinfo (PPALETTEINFO) - input
Pointer to table of palette data.

Pointer to table of palette data from GreCreatePalette. The structure of palinfo is as follows:

```
struct _PALETTEINFO { /* palinfo */

ULONG        flCmd;          /* Options from create                   */
ULONG        ulFormat;       /* Specifies format of entries at create */
                             /* Must be LCOLF-CONSECRGB               */
ULONG        clColorData;    /* Count of Palette entries              */
RGB2   argb[0];              /* Palette entries                       */

} PALETTEINFO; Additional RGB@ entries follow so that CLColorData-1 entries are
defined.
```

GreDeviceCreatePalette Parameter - hdevpal

hdevpal (ULONG) - input
Device palette handle (NULL, if new).

If hdevpal=0, ppalinfo points to a palette information structure, which must be set into a new palette. The handle for this new palette is

returned in hdevpal. The new palette is then selected into the DC.

If hdevpal is nonzero, ppalinfo is ignored, the palette is selected into the DC, and hdevpal is a device palette handle previously returned by the presentation driver. In this case, hdevpal is used to bind the palette with the new DC. This technique is the driver-level mechanism for sharing palettes among contexts on the same device.

GreDeviceCreatePalette Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreDeviceCreatePalette Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceCreatePalette.

GreDeviceCreatePalette Return Value - rc

rc (ULONG) - returns
Return codes.

On completion, this function returns the handle to the device palette (hdevpal).

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_IN_AREA
PMERR_INV_IN_PATH

GreDeviceCreatePalette - Parameters

hdc (HDC) - input
Device context handle.

ppalinfo (PPALETTEINFO) - input
Pointer to table of palette data.

Pointer to table of palette data from GreCreatePalette. The structure of palinfo is as follows:

```
struct _PALETTEINFO { /* palinfo */

ULONG        flCmd;         /* Options from create                  */
ULONG        ulFormat;      /* Specifies format of entries at create */
                            /* Must be LCOLF-CONSECRGB               */
ULONG        clColorData;   /* Count of Palette entries             */
RGB2    argb[0];            /* Palette entries                      */
```

```
} PALETTEINFO; Additional RGB@ entries follow so that CLColorData-1 entries are
defined.
```

hdevpal (ULONG) - input
Device palette handle (NULL, if new).

If hdevpal=0, ppalinfo points to a palette information structure, which must be set into a new palette. The handle for this new palette is returned in hdevpal. The new palette is then selected into the DC.

If hdevpal is nonzero, ppalinfo is ignored, the palette is selected into the DC, and hdevpal is a device palette handle previously returned by the presentation driver. In this case, hdevpal is used to bind the palette with the new DC. This technique is the driver-level mechanism for sharing palettes among contexts on the same device.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceCreatePalette.

rc (ULONG) - returns
Return codes.

On completion, this function returns the handle to the device palette (hdevpal).

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_IN_AREA
PMERR_INV_IN_PATH

GreDeviceCreatePalette - Remarks

When an application sets a bit map, the bits are specified as indexes into the color table supplied with the bit map. The presentation driver allocates its own copy of the bit map.

The driver-level pels contain indexes to the nearest color in the default physical palette.

Note: This means that GreGetBitMapBits does not necessarily return the same bit map that the application set. Information may be lost if the bit map is taken to a less color-capable system and then brought back to the (more capable) system on which it was created.

However, an application can select a palette into a memory DC. This causes the color table and the bit-map bits to change to the nearest mapping to the new palette. Calls to GreRealizePalette are not performed on memory DCs. The palette takes effect as soon as it is selected.

The presentation driver is expected to perform any allocation and data structure needed for a subsequent GreDeviceResizePalette to succeed.

GreDeviceCreatePalette - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreDeviceDeleteBitmap

GreDeviceDeleteBitmap - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreDeviceDeleteBitmap destroys a bit map.

This function must be supported by the presentation driver.

```
#define INCL_GRE_BITMAPS
#include <os2.h>

HDC              hdc;         /*  Reserved. */
ULONG            hbm;         /*  Handle of the bit map to be destroyed. */
PDELETERETURN    pReturns;    /*  Pointer to returned bit-map parameters. */
ULONG            flOptions;
ULONG            pInstance;   /*  Reserved. */
ULONG            lFunction;
BOOL             rc;          /*  Return Codes. */

rc = GreDeviceDeleteBitmap(hdc, hbm, pReturns,
       flOptions, pInstance, lFunction);
```

GreDeviceDeleteBitmap Parameter - hdc

hdc (HDC) - input
Reserved.

Note: The handling routine must not use the value passed on the stack in the location reserved for hdc. This location contains undefined data.

GreDeviceDeleteBitmap Parameter - hbm

hbm (ULONG) - input
Handle of the bit map to be destroyed.

GreDeviceDeleteBitmap Parameter - pReturns

pReturns (PDELETERETURN) - input

Pointer to returned bit-map parameters.

Pointer to a DELETERETURN structure, which consists of the following fields:

pInfo	Pointer to a BITMAPINFO or BITMAPINFO2 structure
pBits	Pointer to bit map

GreDeviceDeleteBitmap Parameter - flOptions

flOptions (ULONG) - input
Additional information used by the engine when creating or deleting a bit map. The only valid flag is:

CBM_INIT	When set, bit-map parameters must be returned in pReturns. This means that before deleting the bit map, the handling routine must translate it into one of the standard formats. The presentation driver must then allocate two blocks of memory, one for the bit map and another for the bit-map parameters and color translation table. The presentation driver can use any of the standard formats. However, it must take into account the parameters originally specified in GreDeviceCreateBitmap. The handling routine should use the format that uses the least amount of memory without losing any bit-map information. When this flag is not set, bit-map data is not returned.

Other flags are reserved and should be ignored by the handling routine.

GreDeviceDeleteBitmap Parameter - pInstance

pInstance (ULONG) - input
Reserved.

Note: The handling routine must not use the value passed on the stack in the location reserved for pInstance. This location contains undefined data.

GreDeviceDeleteBitmap Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceDeleteBitmap.

GreDeviceDeleteBitmap Return Value - rc

rc (BOOL) - returns
Return Codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_HDC
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SCAN_START

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDeviceDeleteBitmap - Parameters

hdc (HDC) - input
Reserved.

Note: The handling routine must not use the value passed on the stack in the location reserved for hdc. This location contains undefined data.

hbm (ULONG) - input
Handle of the bit map to be destroyed.

pReturns (PDELETERETURN) - input
Pointer to returned bit-map parameters.

Pointer to a DELETERETURN structure, which consists of the following fields:

pInfo	Pointer to a BITMAPINFO or BITMAPINFO2 structure
pBits	Pointer to bit map

flOptions (ULONG) - input
Additional information used by the engine when creating or deleting a bit map. The only valid flag is:

CBM_INIT	When set, bit-map parameters must be returned in pReturns. This means that before deleting the bit map, the handling routine must translate it into one of the standard formats. The presentation driver must then allocate two blocks of memory, one for the bit map and another for the bit-map parameters and color translation table. The presentation driver can use any of the standard formats. However, it must take into account the parameters originally specified in GreDeviceCreateBitmap. The handling routine should use the format that uses the least amount of memory without losing any bit-map information. When this flag is not set, bit-map data is not returned.

Other flags are reserved and should be ignored by the handling routine.

pInstance (ULONG) - input
Reserved.

Note: The handling routine must not use the value passed on the stack in the location reserved for pInstance. This location contains undefined data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceDeleteBitmap.

rc (BOOL) - returns
Return Codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_HDC
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SCAN_START

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDeviceDeleteBitmap - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreDeviceDeletePalette

GreDeviceDeletePalette - Syntax

Simulation support:

This function is simulated by a handling routine in Revision 2.0 (or later) of the graphics engine.

Description

GreDeviceDeletePalette is called by the graphics engine when a palette is selected out of a device context. This can occur in many situations, including when an application selects a new palette using GpiSelectPalette or deletes a palette using GpiDeletePalette. It informs the device to delete its instance of the palette for the given device context. The DC should be reset to the default color table. It is the responsibility of the presentation driver to determine if any other device contexts are using the palette. In this case, the presentation driver does not free the internal data structures needed for palette realization.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_PALETTE
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    hdevpal;     /*  Handle to the device palette from GreDeviceCreatePalette. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreDeviceDeletePalette.

 */
LONG     rc;          /*  Return codes. */

rc = GreDeviceDeletePalette(hdc, hdevpal,
       pInstance, lFunction);
```

GreDeviceDeletePalette Parameter - hdc

hdc (HDC) - input
Device context handle.

GreDeviceDeletePalette Parameter - hdevpal

hdevpal (ULONG) - input
Handle to the device palette from GreDeviceCreatePalette.

GreDeviceDeletePalette Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreDeviceDeletePalette Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceDeletePalette.

GreDeviceDeletePalette Return Value - rc

rc (LONG) - returns
Return codes.

On completion, this function returns the following value:

Error	(0)
Successful	(1)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_IN_AREA
PMERR_INV_IN_PATH

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDeviceDeletePalette - Parameters

hdc (HDC) - input
Device context handle.

hdevpal (ULONG) - input
Handle to the device palette from GreDeviceCreatePalette.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceDeletePalette.

rc (LONG) - returns
Return codes.

On completion, this function returns the following value:

Error	(0)
Successful	(1)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_IN_AREA
PMERR_INV_IN_PATH

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDeviceDeletePalette - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreDeviceGetAttributes

GreDeviceGetAttributes - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreDeviceGetAttributes queries the attribute values currently set in the device.

This function must be supported by the presentation driver.

```
#define INCL_GRE_DEVMISC1
#include <os2.h>

HDC         hdc;           /*  Device context handle. */
LONG        lPrimType;     /*  Bundle primitive type. */
ULONG       flAttrsMask;   /*  Attribute mask. */
PBUNDLE     pAttrs;
PVOID       pInstance;     /*  Pointer to instance data. */
ULONG       lFunction;
BOOL        rc;            /*  Return Codes. */

rc = GreDeviceGetAttributes(hdc, lPrimType,
       flAttrsMask, pAttrs, pInstance, lFunction);
```

GreDeviceGetAttributes Parameter - hdc

hdc (HDC) - input
Device context handle.

GreDeviceGetAttributes Parameter - lPrimType

lPrimType (LONG) - input
Bundle primitive type.

Indicates the bundle type. Valid primitive values are:

PRIM_LINE	Line attribute bundle (see Line Attributes.
PRIM_CHAR	Character attribute bundle see Character Attributes.
PRIM_MARKER	Marker attribute bundle see Marker Attributes.
PRIM_AREA	Area (Pattern) attribute bundle see Area (Pattern) Attributes.
PRIM_IMAGE	Image attribute bundle see Image Attributes.

GreDeviceGetAttributes Parameter - flAttrsMask

flAttrsMask (ULONG) - input
Attribute mask.

Specifies the attributes to be returned. This mask contains a bit corresponding to each attribute in the bundle record that is required. For each set bit, the handling routine must return the corresponding attribute values and default mask bits. Only the foreground color and background color attributes can be requested for any primitive type.

GreDeviceGetAttributes Parameter - pAttrs

pAttrs (PBUNDLE) - input
Pointer to the fixed-format bundle record to which the attributes are returned. Fields other than those indicated by flAttrsMask must not be modified.

GreDeviceGetAttributes Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreDeviceGetAttributes Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceGetAttributes.

GreDeviceGetAttributes Return Value - rc

rc (BOOL) - returns
Return Codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDeviceGetAttributes - Parameters

hdc (HDC) - input
Device context handle.

lPrimType (LONG) - input
Bundle primitive type.

Indicates the bundle type. Valid primitive values are:

PRIM_LINE	Line attribute bundle (see Line Attributes.
PRIM_CHAR	Character attribute bundle see Character Attributes.
PRIM_MARKER	Marker attribute bundle see Marker Attributes.
PRIM_AREA	Area (Pattern) attribute bundle see Area (Pattern) Attributes.
PRIM_IMAGE	Image attribute bundle see Image Attributes.

flAttrsMask (ULONG) - input
Attribute mask.

Specifies the attributes to be returned. This mask contains a bit corresponding to each attribute in the bundle record that is required. For each set bit, the handling routine must return the corresponding attribute values and default mask bits. Only the foreground color and background color attributes can be requested for any primitive type.

pAttrs (PBUNDLE) - input
Pointer to the fixed-format bundle record to which the attributes are returned. Fields other than those indicated by flAttrsMask must not be modified.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceGetAttributes.

rc (BOOL) - returns
Return Codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDeviceGetAttributes - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreDeviceInvalidateVisRegion

GreDeviceInvalidateVisRegion - Syntax

Simulation support:

This function is mandatory for display drivers. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreDeviceInvalidateVisRegion notifies the presentation driver that the visible region and DC region of one or more DCs has changed, and that the affected DCs must revalidate their visible regions before drawing in them.

```
#define INCL_GRE_DEVMISC2
#include <os2.h>
```

```
HDC          hdc;         /*  The device context handle. */
LONG         cArray;      /*  The number of elements in the array. */
PDC_BLOCK    paBlock;
PVOID        pInstance;   /*  A pointer to instance data. */
ULONG        lFunction;   /*  High-order WORD=flags; low-order WORD=NGreDeath. */
BOOL         fSuccess;    /*  Return codes. */

fSuccess = GreDeviceInvalidateVisRegion(hdc,
             cArray, paBlock, pInstance, lFunction);
```

GreDeviceInvalidateVisRegion Parameter - hdc

hdc (HDC) - input
The device context handle.

GreDeviceInvalidateVisRegion Parameter - cArray

cArray (LONG) - input
The number of elements in the array.

GreDeviceInvalidateVisRegion Parameter - paBlock

paBlock (PDC_BLOCK) - input
A pointer to an array of DC_BLOCK structures:

hdc	A device context handle.
pDcl	A pointer to instance data.

GreDeviceInvalidateVisRegion Parameter - pInstance

pInstance (PVOID) - input
A pointer to instance data.

GreDeviceInvalidateVisRegion Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeath.

GreDeviceInvalidateVisRegion Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. An error code for conditions that the handling routine is expected to check:

PMERR_DEV_FUNC_NOT_INSTALLED

GreDeviceInvalidateVisRegion - Parameters

hdc (HDC) - input
The device context handle.

cArray (LONG) - input
The number of elements in the array.

paBlock (PDC_BLOCK) - input
A pointer to an array of DC_BLOCK structures:

hdc	A device context handle.
pDcl	A pointer to instance data.

pInstance (PVOID) - input
A pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeath.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. An error code for conditions that the handling routine is expected to check:

PMERR_DEV_FUNC_NOT_INSTALLED

GreDeviceInvalidateVisRegion - Remarks

The array identified by paBlock contains a series of structures, each of which identifies a DC and supplies the pointer (pInstance) to its instance data. The display driver responds by setting a flag (HDC_IS_DIRTY) in the instance data of each DC identified in the array. The handling routines for all drawing functions should check the HDC_IS_DIRTY flag before drawing. If the flag is set, VisRegionNotify must be called to revalidate the DC's visible region.

This function allows the system to defer the calculations caused by visible region changes. This enables menus and dialogs to perform more

efficiently.

GreDeviceInvalidateVisRegion - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreDeviceQueryFontAttributes

GreDeviceQueryFontAttributes - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreDeviceQueryFontAttributes stores the metrics of the currently selected font at the location addressed by pfmMetrics. Note that the handling routine must transform device coordinates to world coordinates before returning the results to the calling routine. This transformation can be done by using GreConvert.

This function must be supported by the presentation driver.

```
#define INCL_GRE_DEVMISC2
#include <os2.h>

HDC             hdc;          /*  Device context handle. */
ULONG           cMetrics;     /*  Size of FONTMETRICS structure. */
PFONTMETRICS    pfmMetrics;   /*  Pointer to FONTMETRICS structure. */
PVOID           pInstance;    /*  Pointer to instance data. */
ULONG           lFunction;    /*  High-order WORD=flags; low-order WORD=NGreDeviceQueryFontAttributes. */
BOOL            rc;           /*  Return Codes. */

rc = GreDeviceQueryFontAttributes(hdc, cMetrics,
       pfmMetrics, pInstance, lFunction);
```

GreDeviceQueryFontAttributes Parameter - hdc

hdc (HDC) - input
Device context handle.

GreDeviceQueryFontAttributes Parameter - cMetrics

cMetrics (ULONG) - input
Size of FONTMETRICS structure.

GreDeviceQueryFontAttributes Parameter - pfmMetrics

pfmMetrics (PFONTMETRICS) - input
Pointer to FONTMETRICS structure.

GreDeviceQueryFontAttributes Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreDeviceQueryFontAttributes Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceQueryFontAttributes.

GreDeviceQueryFontAttributes Return Value - rc

rc (BOOL) - returns
Return Codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDeviceQueryFontAttributes - Parameters

hdc (HDC) - input
Device context handle.

cMetrics (ULONG) - input
Size of FONTMETRICS structure.

pfmMetrics (PFONTMETRICS) - input
Pointer to FONTMETRICS structure.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceQueryFontAttributes.

rc (BOOL) - returns
Return Codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDeviceQueryFontAttributes - Remarks

GreDeviceQueryFontAttributes is called from the graphics engine internal function GreQueryFontAttributes in response to an application calling one of the GpiQueryFont*xxx* () APIs. This call can be handled by bit-map simulation.

GreDeviceQueryFontAttributes - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreDeviceQueryFonts

GreDeviceQueryFonts - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreDeviceQueryFonts returns the characteristics of device fonts in an array of FONTMETRICS structures, if the QF_PUBLIC option flag (see the flOptions parameter) is set. The returned fonts include those that correspond to device modes such as "expanded" and "expanded-bold".

This function must be supported by the presentation driver.

```
#define INCL_GRE_DEVMISC2
#include <os2.h>

HDC             hdc;          /*  Device context handle. */
ULONG           flOptions;    /*  Option flags. */
PSZ             pszFaceName;  /*  Pointer to FaceName to match. */
PFONTMETRICS    pfmMetrics;   /*  Pointer to array of FONTMETRICS structures. */
LONG            cMetrics;     /*  Number of bytes of each metrics structure in the metrics array. */
PLONG           pcFonts;      /*  Pointer to the number of fonts requested. */
PVOID           pInstance;    /*  Pointer to instance data. */
ULONG           lFunction;    /*  High-order WORD=flags; low-order WORD=NGreDeviceQueryFonts. */
LONG            rc;           /*  Return Codes. */

rc = GreDeviceQueryFonts(hdc, flOptions, pszFaceName,
       pfmMetrics, cMetrics, pcFonts, pInstance,
       lFunction);
```

GreDeviceQueryFonts Parameter - hdc

hdc (HDC) - input
Device context handle.

GreDeviceQueryFonts Parameter - flOptions

flOptions (ULONG) - input
Option flags.

The only valid flag for this function is:

QF_PUBLIC	When this flag is set, the handling routine must return all device fonts. "Device fonts" are public fonts and they are returned in the array addressed by pfmMetrics. If this flag is not set, the handling routine should not return any fonts.

GreDeviceQueryFonts Parameter - pszFaceName

pszFaceName (PSZ) - input
Pointer to FaceName to match.

If this is a NULL pointer, all faces are matched.

GreDeviceQueryFonts Parameter - pfmMetrics

pfmMetrics (PFONTMETRICS) - input
Pointer to array of FONTMETRICS structures.

GreDeviceQueryFonts Parameter - cMetrics

cMetrics (LONG) - input
Number of bytes of each metrics structure in the metrics array.

GreDeviceQueryFonts Parameter - pcFonts

pcFonts (PLONG) - input
Pointer to the number of fonts requested.

This is a pointer to the number of fonts requested. On completion, the handling routine modifies the value indicated to the number of fonts returned. An application can determine the number of public fonts available to it by passing a value of 0 at the address indicated by this pointer.

GreDeviceQueryFonts Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreDeviceQueryFonts Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceQueryFonts.

GreDeviceQueryFonts Return Value - rc

rc (LONG) - returns
Return Codes.

The handling routine should return the number of fonts not returned or else return GPI_ALTERROR if an error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreDeviceQueryFonts - Parameters

hdc (HDC) - input
Device context handle.

flOptions (ULONG) - input
Option flags.

The only valid flag for this function is:

QF_PUBLIC	When this flag is set, the handling routine must return all device fonts. "Device fonts" are public fonts and they are returned in the array addressed by pfmMetrics. If this flag is not set, the handling routine should not return any fonts.

pszFaceName (PSZ) - input
Pointer to FaceName to match.

If this is a NULL pointer, all faces are matched.

pfmMetrics (PFONTMETRICS) - input
Pointer to array of FONTMETRICS structures.

cMetrics (LONG) - input
Number of bytes of each metrics structure in the metrics array.

pcFonts (PLONG) - input
Pointer to the number of fonts requested.

This is a pointer to the number of fonts requested. On completion, the handling routine modifies the value indicated to the number of fonts returned. An application can determine the number of public fonts available to it by passing a value of 0 at the address indicated by this pointer.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceQueryFonts.

rc (LONG) - returns
Return Codes.

The handling routine should return the number of fonts not returned or else return GPI_ALTERROR if an error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreDeviceQueryFonts - Remarks

When the device context is not set to draft mode, the returned fonts are those that can be positioned to the nearest pel. Such precision is not

necessary if the device context is in draft mode. Draft mode is set by the system calling the GreEscape DEVESC_DRAFTMODE.

In the FONTMETRICS structures, the handling routine sets:

- *szFacename* field to a meaningful name, such as 'Courier Bold'.
- *usCodepage* field to 0. (This field has no significance in this context.)
- *lMatch* field to a negative value. This allows the presentation driver to map the font when the value is specified in a call to GreRealizeFont.

The presentation driver must transform device coordinates to world coordinates before it returns the results to the calling routine. This can be done by using GreConvert. For presentation drivers that support only outline fonts, the return values are for outline fonts even when image fonts have been loaded.

GreDeviceQueryFonts - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreDeviceResizePalette

GreDeviceResizePalette - Syntax

Simulation support:

This function is simulated by a handling routine in Revision 2.0 (or higher) of the graphics engine.

Description

GreDeviceResizePalette changes the size of a logical palette. If the size is reduced, the removed entries are deleted. If the size is increased, the new entries are set to Black.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_PALETTE
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    hdevpal;     /*  Device palette handle. */
ULONG    ulSize;      /*  New palette size. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreDeviceResizePalette. */
LONG     rc;          /*  Return codes. */

rc = GreDeviceResizePalette(hdc, hdevpal,
       ulSize, pInstance, lFunction);
```

GreDeviceResizePalette Parameter - hdc

hdc (HDC) - input
Device context handle.

GreDeviceResizePalette Parameter - hdevpal

hdevpal (ULONG) - input
Device palette handle.

GreDeviceResizePalette Parameter - ulSize

ulSize (ULONG) - input
New palette size.

GreDeviceResizePalette Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreDeviceResizePalette Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceResizePalette.

GreDeviceResizePalette Return Value - rc

rc (LONG) - returns
Return codes.

On completion, this function returns the following values:

Error	(0)
Successful	(1)

GreDeviceResizePalette - Parameters

hdc (HDC) - input
Device context handle.

hdevpal (ULONG) - input
Device palette handle.

ulSize (ULONG) - input
New palette size.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceResizePalette.

rc (LONG) - returns
Return codes.

On completion, this function returns the following values:

Error	(0)
Successful	(1)

GreDeviceResizePalette - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreDeviceSelectBitmap

GreDeviceSelectBitmap - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreDeviceSelectBitmap informs the presentation driver that a new bit map is selected into the device context. See also GreSelectBitmap.

This function must be supported by the presentation driver.

```
#define INCL_GRE_BITMAPS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    hbm;         /*  Device bit-map handle. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;
```

```
BOOL     rc;          /*  Return Codes. */

rc = GreDeviceSelectBitmap(hdc, hbm, pInstance,
       lFunction);
```

GreDeviceSelectBitmap Parameter - hdc

hdc (HDC) - input
Device context handle.

GreDeviceSelectBitmap Parameter - hbm

hbm (ULONG) - input
Device bit-map handle.

GreDeviceSelectBitmap Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreDeviceSelectBitmap Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceSelectBitmap.

GreDeviceSelectBitmap Return Value - rc

rc (BOOL) - returns
Return Codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED

PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDeviceSelectBitmap - Parameters

hdc (HDC) - input
Device context handle.

hbm (ULONG) - input
Device bit-map handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceSelectBitmap.

rc (BOOL) - returns
Return Codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDeviceSelectBitmap - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreDeviceSetAttributes

GreDeviceSetAttributes - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreDeviceSetAttributes sets attributes in the attribute bundle.

This function must be supported by the presentation driver.

```
#define INCL_GRE_DEVMISC1
#include <os2.h>

HDC         hdc;           /*  Device context handle. */
LONG        lBType;        /*  Bundle type. */
ULONG       flDefsMask;
ULONG       flAttrsMask;   /*  Mask indicating the attributes to be modified. */
PBUNDLE     pAttrs;        /*  Pointer to a bundle structure. */
PVOID       pInstance;     /*  Pointer to instance data. */
ULONG       lFunction;
BOOL        rc;            /*  Return Codes. */

rc = GreDeviceSetAttributes(hdc, lBType, flDefsMask,
       flAttrsMask, pAttrs, pInstance, lFunction);
```

GreDeviceSetAttributes Parameter - hdc

hdc (HDC) - input
Device context handle.

GreDeviceSetAttributes Parameter - lBType

lBType (LONG) - input
Bundle type.

Device attribute bundle. The following device bundles are defined:

PRIM_AREA	Area (Pattern) attribute bundle see Area (Pattern) Attributes.
PRIM_CHAR	Character attribute bundle see Character Attributes.
PRIM_IMAGE	Image attribute bundle see Image Attributes.
PRIM_LINE	Line attribute bundle (see Line Attributes.
PRIM_MARKER	Marker attribute bundle see Marker Attributes.

All device bundles share a similar format. They consist of two bundles, a bundle of logical attributes and a bundle of device information.

GreDeviceSetAttributes Parameter - flDefsMask

flDefsMask (ULONG) - input
Mask indicating the attributes to be set to their standard default values.

GreDeviceSetAttributes Parameter - flAttrsMask

flAttrsMask (ULONG) - input
Mask indicating the attributes to be modified.

GreDeviceSetAttributes Parameter - pAttrs

pAttrs (PBUNDLE) - input
Pointer to a bundle structure.

Pointer to the DLINEBUNDLE, DCHARBUNDLE, DMARKERBUNDLE, DAREABUNDLE, or DIMAGEBUNDLE structure containing the new attributes.

GreDeviceSetAttributes Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreDeviceSetAttributes Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceSetAttributes.

GreDeviceSetAttributes Return Value - rc

rc (BOOL) - returns
Return Codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_CHAR_DIRECTION_ATTR
PMERR_INV_CHAR_MODE_ATTR

PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_MIX_ATTR
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreDeviceSetAttributes - Parameters

hdc (HDC) - input
Device context handle.

lBType (LONG) - input
Bundle type.

Device attribute bundle. The following device bundles are defined:

PRIM_AREA	Area (Pattern) attribute bundle see Area (Pattern) Attributes.
PRIM_CHAR	Character attribute bundle see Character Attributes.
PRIM_IMAGE	Image attribute bundle see Image Attributes.
PRIM_LINE	Line attribute bundle (see Line Attributes.
PRIM_MARKER	Marker attribute bundle see Marker Attributes.

All device bundles share a similar format. They consist of two bundles, a bundle of logical attributes and a bundle of device information.

flDefsMask (ULONG) - input
Mask indicating the attributes to be set to their standard default values.

flAttrsMask (ULONG) - input
Mask indicating the attributes to be modified.

pAttrs (PBUNDLE) - input
Pointer to a bundle structure.

Pointer to the DLINEBUNDLE, DCHARBUNDLE, DMARKERBUNDLE, DAREABUNDLE, or DIMAGEBUNDLE structure containing the new attributes.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceSetAttributes.

rc (BOOL) - returns
Return Codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_CHAR_DIRECTION_ATTR

PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_MIX_ATTR
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreDeviceSetAttributes - Remarks

This function sets attributes in the attribute bundle specified by lBType. Pointers to the current attribute bundles are maintained by presentation drivers in the instance data structure. The handling routine for GreDeviceSetAttributes modifies the specified bundle as directed by flAttrsMask and flDefsMask (see "Bundle Masks").

The handling routine must allow any attribute to be set to any value in the defined range for that attribute even when the value cannot be implemented on the device. For example, the presentation driver for a vector hardcopy device must accept BM_XOR background mix. When the hardcopy driver is called to write to the device, it should map values that cannot be implemented to the default value. If this call would set any of the attributes to a value that is not in the defined range of values for that attribute, the handling routine must restore all attributes to the value they had on entry to this routine.

When this function is called for the first time to set the character attributes, the handling routine should set the default font in the usSet parameter of the character attribute bundle (see Character Attributes). If the default font is an engine font, the presentation driver must save the address and flags of the font. This ensures that the default font is restored if the device context is reset (see ResetDCState).

Bundle Masks

The parameters flDefsMask and flAttrsMask are instances of the mask for the specified attribute bundle. Valid flags are listed under the following bundle definitions:

- Line Attributes
- Area (Pattern) Attributes
- Character Attributes
- Image Attributes
- Marker Attributes

Flags set in flAttrsMask identify which fields in the attribute bundle are to be changed. For each flag that is set in flAttrsMask, the state of that flag in flDefsMask determines the source for the new value of the field. If the flag is set in both masks, the corresponding field should be set to its default value. If the flag is set in flAttrsMask and not set in flDefsMask, the corresponding field will be set from the relevant field in the bundle addressed by pAttrs.

In the attribute bundle addressed by pAttrs, the only fields that contain valid values are those that will be used to modify the device context's attribute bundle.

When setting pattern and area attributes, the pattern origin from world coordinates must be converted (in the attributes bundle) to device coordinates (in the DC instance data).

GreDeviceSetAttributes - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreDeviceSetAVIOFont

GreDeviceSetAVIOFont - Syntax

Simulation support:

This function is mandatory for display drivers. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreDeviceSetAVIOFont loads or deletes an image font used by the AVIO presentation space associated with the device context.

```
#define INCL_GRE_DEVMISC1
#include <os2.h>

HDC         hdc;         /*  The device context handle. */
PFATTRS     pLogFont;    /*  A pointer to a logical font. */
PBYTE       pFontDef;
LONG        lcid;        /*  Local identifier value of -2, -3, or -4. */
PVOID       pInstance;   /*  A pointer to instance data. */
ULONG       lFunction;   /*  High-order WORD=flags; low-order WORD=NGreDeviceSetAVIOFont. */
BOOL        fSuccess;    /*  Return codes. */

fSuccess = GreDeviceSetAVIOFont(hdc, pLogFont,
             pFontDef, lcid, pInstance, lFunction);
```

GreDeviceSetAVIOFont Parameter - hdc

hdc (HDC) - input
The device context handle.

GreDeviceSetAVIOFont Parameter - pLogFont

pLogFont (PFATTRS) - input
A pointer to a logical font.

GreDeviceSetAVIOFont Parameter - pFontDef

pFontDef (PBYTE) - input
A pointer to a physical font data structure. When the value passed is 0, the handling routine must delete the loaded font corresponding to lLcid. If the high-order bit of the high-order WORD in lLcid is set, the handle is for a device font.

GreDeviceSetAVIOFont Parameter - lcid

lcid (LONG) - input
Local identifier value of -2, -3, or -4.

GreDeviceSetAVIOFont Parameter - pInstance

pInstance (PVOID) - input
A pointer to instance data.

GreDeviceSetAVIOFont Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceSetAVIOFont.

GreDeviceSetAVIOFont Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful. The font is acceptable for use with an AVIO presentation space.
FALSE	Error.

GreDeviceSetAVIOFont - Parameters

hdc (HDC) - input
The device context handle.

pLogFont (PFATTRS) - input
A pointer to a logical font.

pFontDef (PBYTE) - input
A pointer to a physical font data structure. When the value passed is 0, the handling routine must delete the loaded font corresponding to lLcid. If the high-order bit of the high-order WORD in lLcid is set, the handle is for a device font.

lcid (LONG) - input
Local identifier value of -2, -3, or -4.

pInstance (PVOID) - input
A pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceSetAVIOFont.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful. The font is acceptable for use with an AVIO presentation space.
FALSE	Error.

GreDeviceSetAVIOFont - Remarks

When loaded, the font is used by subsequent GreCharRect, GreCharStr, and GreScrollRect calls to draw the character images for the appropriate AVIO set. lLcid identifiers LCID_AVIO_1, LCID_AVIO_2, and LCID_AVIO_3 correspond to AVIO sets 1, 2, and 3 respectively.

If the font is not acceptable for use with an AVIO presentation space, the handling routine returns FALSE to indicate an error. The font must be a fixed-pitch image (raster) font that matches one of the cell sizes for the default font. If the font does not match a supported cell size, the characters are displayed in the presentation space as black cells.

A possible approach is for the presentation driver to realize the AVIO fonts required and store the pointers to the fonts in the DC instance data structure.

GreDeviceSetAVIOFont - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreDeviceSetCursor

GreDeviceSetCursor - Syntax

Simulation support:

This function is mandatory for display drivers. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreDeviceSetCursor sets the cursor bit map that defines the cursor shape.

```
#define INCL_GRE_BITMAPS
```

```
#include <os2.h>

HDC         hdc;           /*  The device context handle. */
PPOINTL     pptlHotSpot;   /*  A pointer to hot spot coordinates. */
ULONG       hbm;           /*  Bit-map handle used for the cursor image. */
PVOID       pInstance;     /*  A pointer to instance data. */
ULONG       lFunction;     /*  High-order WORD=flags; low-order WORD=NGreDeviceSetCursor. */
BOOL        fSuccess;      /*  Return codes. */

fSuccess = GreDeviceSetCursor(hdc, pptlHotSpot,
             hbm, pInstance, lFunction);
```

GreDeviceSetCursor Parameter - hdc

hdc (HDC) - input
The device context handle.

GreDeviceSetCursor Parameter - pptlHotSpot

pptlHotSpot (PPOINTL) - input
A pointer to hot spot coordinates.

x	X-position of the hotspot within the cursor bit map
y	Y-position of the hotspot within the cursor bit map.

GreDeviceSetCursor Parameter - hbm

hbm (ULONG) - input
Bit-map handle used for the cursor image.

GreDeviceSetCursor Parameter - pInstance

pInstance (PVOID) - input
A pointer to instance data.

GreDeviceSetCursor Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceSetCursor.

GreDeviceSetCursor Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_COORDINATE
PMERR_INV_CURSOR_BITMAP
PMERR_INV_HDC

GreDeviceSetCursor - Parameters

hdc (HDC) - input
The device context handle.

pptlHotSpot (PPOINTL) - input
A pointer to hot spot coordinates.

x	X-position of the hotspot within the cursor bit map
y	Y-position of the hotspot within the cursor bit map.

hbm (ULONG) - input
Bit-map handle used for the cursor image.

pInstance (PVOID) - input
A pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceSetCursor.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_COORDINATE
PMERR_INV_CURSOR_BITMAP
PMERR_INV_HDC

GreDeviceSetCursor - Remarks

GreDeviceSetCursor is subject to all clipping.

The handling routine takes the previous cursor bit map and replaces it with the one indicated by hbm. If hbm is NULL, the cursor has no shape and its image is removed from the display screen.

GreDeviceSetCursor - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreDeviceSetDCOrigin

GreDeviceSetDCOrigin - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreDeviceSetDCOrigin sets the origin of the device context which, when created, has its origin set to 1.0,0.

This function must be supported by presentation drivers for display devices and for hardcopy devices that use banding. The minimum requirement for other hardcopy devices is for the handling routine to return TRUE if the origin addressed by pptlDC is set to 0, or to log an error and return FALSE.

```
#define INCL_GRE_DEVMISC3
#include <os2.h>

HDC         hdc;         /*  Device context handle. */
PPOINTL     pptlDC;      /*  Pointer to the DC origin. */
PVOID       pInstance;   /*  Pointer to instance data. */
ULONG       lFunction;
BOOL        rc;          /*  Return Codes. */

rc = GreDeviceSetDCOrigin(hdc, pptlDC, pInstance,
       lFunction);
```

GreDeviceSetDCOrigin Parameter - hdc

hdc (HDC) - input
Device context handle.

GreDeviceSetDCOrigin Parameter - pptlDC

pptlDC (PPOINTL) - input
Pointer to the DC origin.

This is the offset to the origin of the device context indicated by hdc. Convert does not add in this offset (see GreConvert). Therefore, the presentation driver must add it to all device coordinates to make them screen coordinates.

Note: WORLD_COORDINATE to SCREEN_COORDINATE is not a valid conversion.

GreDeviceSetDCOrigin Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreDeviceSetDCOrigin Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceSetDCOrigin.

GreDeviceSetDCOrigin Return Value - rc

rc (BOOL) - returns
Return Codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDeviceSetDCOrigin - Parameters

hdc (HDC) - input
Device context handle.

pptlDC (PPOINTL) - input
Pointer to the DC origin.

This is the offset to the origin of the device context indicated by hdc. Convert does not add in this offset (see GreConvert). Therefore, the presentation driver must add it to all device coordinates to make them screen coordinates.

Note: WORLD_COORDINATE to SCREEN_COORDINATE is not a valid conversion.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceSetDCOrigin.

rc (BOOL) - returns
Return Codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDeviceSetDCOrigin - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreDeviceSetGlobalAttribute

GreDeviceSetGlobalAttribute - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreDeviceSetGlobalAttribute sets the individual primitive attributes to the specified value in the line, area, character, image, and marker bundles.

This function must be supported by the presentation driver.

```
#define INCL_GRE_DEVMISC1
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lAttrType;    /*  Specifies the attribute. */
LONG     lAttribute;   /*  New attribute value. */
ULONG    flOptions;    /*  Options flag. */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;
BOOL     rc;           /*  Return Codes. */

rc = GreDeviceSetGlobalAttribute(hdc, lAttrType,
       lAttribute, flOptions, pInstance, lFunction);
```

GreDeviceSetGlobalAttribute Parameter - hdc

hdc (HDC) - input
Device context handle.

GreDeviceSetGlobalAttribute Parameter - lAttrType

lAttrType (LONG) - input
Specifies the attribute.

Possible attribute types are:

ATYPE_COLOR	Foreground color
ATYPE_BACK_COLOR	Background color
ATYPE_MIX_MODE	Foreground mix
ATYPE_BACK_MIX_MODE	Background mix ATYPE_BACK_COLOR and ATYPE_BACK_MIX_MODE do not apply to the line bundle.

GreDeviceSetGlobalAttribute Parameter - lAttribute

lAttribute (LONG) - input
New attribute value.

GreDeviceSetGlobalAttribute Parameter - flOptions

flOptions (ULONG) - input
Options flag.

The only allowable option is:

GATTR_DEFAULT	When set, the attribute indicated by lAttrType is set to its default value. When this

flag is not set, the attribute is set to the value of lAttribute.

GreDeviceSetGlobalAttribute Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreDeviceSetGlobalAttribute Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceSetGlobalAttribute.

GreDeviceSetGlobalAttribute Return Value - rc

rc (BOOL) - returns
Return Codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_COLOR_ATTR
PMERR_INV_HDC
PMERR_INV_MIX_ATTR

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDeviceSetGlobalAttribute - Parameters

hdc (HDC) - input
Device context handle.

lAttrType (LONG) - input
Specifies the attribute.

Possible attribute types are:

ATYPE_COLOR	Foreground color
ATYPE_BACK_COLOR	Background color
ATYPE_MIX_MODE	Foreground mix

ATYPE_BACK_MIX_MODE — Background mix ATYPE_BACK_COLOR and ATYPE_BACK_MIX_MODE do not apply to the line bundle.

lAttribute (LONG) - input
New attribute value.

flOptions (ULONG) - input
Options flag.

The only allowable option is:

GATTR_DEFAULT — When set, the attribute indicated by lAttrType is set to its default value. When this flag is not set, the attribute is set to the value of lAttribute.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceSetGlobalAttribute.

rc (BOOL) - returns
Return Codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_COLOR_ATTR
PMERR_INV_HDC
PMERR_INV_MIX_ATTR

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDeviceSetGlobalAttribute - Remarks

If this call sets any attributes to a value that is not in the defined range of values for that attribute, the handling routine must restore all attributes to the value they had on entry to this routine.

GreDeviceSetGlobalAttribute - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreDeviceSetPaletteEntries

GreDeviceSetPaletteEntries - Syntax

Simulation support:

This function is simulated by a handling routine in Revision 2.0 (or later) of the graphics engine.

Description

GreDeviceSetPaletteEntries changes the entries in a palette as a result of an application's GpiSetPalette entries request. These changes do not become apparent until an application calls WinRealizePalette. If an application wants to change its palette rapidly and makes those changes immediately apparent, it calls GpiAnimatePalette.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_PALETTE
#include <os2.h>

HDC       hdc;         /*  Device context handle. */
ULONG     hdevpal;     /*  Device palette handle. */
ULONG     ulFormat;    /*  Specifies the entry format. Must be LCOLF_CONSECRGB. */
ULONG     ulStart;     /*  Starting index, that is, first palette entry to change. */
ULONG     cclr;        /*  Count of palette entries to change. */
PULONG    pclr;        /*  Pointer to table of new RGB2 palette entries. */
PVOID     pInstance;   /*  Pointer to instance data. */
ULONG     lFunction;   /*  High-order WORD=flags; low-order WORD=NGreDeviceSetPaletteEntries. */
BOOL      rc;          /*  Return codes. */

rc = GreDeviceSetPaletteEntries(hdc, hdevpal,
       ulFormat, ulStart, cclr, pclr, pInstance,
       lFunction);
```

GreDeviceSetPaletteEntries Parameter - hdc

hdc (HDC) - input
Device context handle.

GreDeviceSetPaletteEntries Parameter - hdevpal

hdevpal (ULONG) - input
Device palette handle.

GreDeviceSetPaletteEntries Parameter - ulFormat

ulFormat (ULONG) - input
Specifies the entry format. Must be LCOLF_CONSECRGB.

GreDeviceSetPaletteEntries Parameter - ulStart

ulStart (ULONG) - input
Starting index, that is, first palette entry to change.

GreDeviceSetPaletteEntries Parameter - cclr

cclr (ULONG) - input
Count of palette entries to change.

GreDeviceSetPaletteEntries Parameter - pclr

pclr (PULONG) - input
Pointer to table of new RGB2 palette entries.

GreDeviceSetPaletteEntries Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreDeviceSetPaletteEntries Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceSetPaletteEntries.

GreDeviceSetPaletteEntries Return Value - rc

rc (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful, if entries set without error
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_COLOR_INDEX

GreDeviceSetPaletteEntries - Parameters

hdc (HDC) - input
Device context handle.

hdevpal (ULONG) - input
Device palette handle.

ulFormat (ULONG) - input
Specifies the entry format. Must be LCOLF_CONSECRGB.

ulStart (ULONG) - input
Starting index, that is, first palette entry to change.

cclr (ULONG) - input
Count of palette entries to change.

pclr (PULONG) - input
Pointer to table of new RGB2 palette entries.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceSetPaletteEntries.

rc (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful, if entries set without error
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_COLOR_INDEX

GreDeviceSetPaletteEntries - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreDisjointLines

GreDisjointLines - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreDisjointLines draws a sequence of disjoint straight lines using the end-point pairs specified. If COM_TRANSFORM is not set, the pairs are expected in screen coordinates.

This function is supported by the graphics engine and can be hooked by the presentation driver.

```
#define INCL_GRE_LINES
#include <os2.h>

HDC         hdc;          /*  Device context handle. */
PPOINTL     paptlPoint;   /*  Pointer to an array of cPoints (X,Y) pairs. */
LONG        cPoints;      /*  Number of (X,Y) pairs in the points array. */
PVOID       pInstance;    /*  Pointer to instance data. */
ULONG       lFunction;
LONG        rc;           /*  Return Codes. */

GreDisjointLines(hdc, paptlPoint, cPoints,
       pInstance, lFunction, rc);
```

GreDisjointLines Parameter - hdc

hdc (HDC) - input
Device context handle.

GreDisjointLines Parameter - paptlPoint

paptlPoint (PPOINTL) - input
Pointer to an array of cPoints (X,Y) pairs.

Pointer to an array of cPoints (X,Y) pairs containing the end-points for the lines.

GreDisjointLines Parameter - cPoints

cPoints (LONG) - input
Number of (X,Y) pairs in the points array.

When this is passed as 0, the handling routine takes no action except to return Success.

GreDisjointLines Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreDisjointLines Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDisjointLines.

GreDisjointLines Parameter - rc

rc (LONG)
Return Codes.

On completion, the handling routine must return an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlation hit (returned by display drivers when the correlation flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN.
PMERR_INV_RECT
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDisjointLines - Parameters

hdc (HDC) - input
Device context handle.

paptlPoint (PPOINTL) - input
Pointer to an array of cPoints (X,Y) pairs.

Pointer to an array of cPoints (X,Y) pairs containing the end-points for the lines.

cPoints (LONG) - input
Number of (X,Y) pairs in the points array.

When this is passed as 0, the handling routine takes no action except to return Success.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDisjointLines.

rc (LONG)
Return Codes.

On completion, the handling routine must return an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlation hit (returned by display drivers when the correlation flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN.
PMERR_INV_RECT
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDisjointLines - Topics

Select an item:
Syntax
Parameters
Glossary

GreDrawBits

GreDrawBits - Syntax

Simulation support:

This function is simulated by a handling routine in Revision 2.0 (or higher) of the graphics engine and can be hooked by the presentation driver.

Description:

GreDrawBits draws a rectangle of bits.

```
#define INCL_GRE_BITMAPS
#include <os2.h>

HDC             hdc;          /*  Device context handle. */
PBYTE           pBitmap;      /*  Pointer to bit map data. */
PBITMAPINFO     pInfo;
LONG            cPoints;      /*  Number of (X, Y) pairs in paptlPoint. */
PPOINTL         paptlPoint;   /*  Pointer to an array of (X, Y) coordinate pairs. */
LONG            lRop;         /*  Raster operation code. */
ULONG           flOptions;
PVOID           pInstance;    /*  Pointer to instance data. */
ULONG           lFunction;
ULONG           rc;           /*  Return Codes. */

rc = GreDrawBits(hdc, pBitmap, pInfo, cPoints,
       paptlPoint, lRop, flOptions, pInstance,
       lFunction);
```

GreDrawBits Parameter - hdc

hdc (HDC) - input
Device context handle.

GreDrawBits Parameter - pBitmap

pBitmap (PBYTE) - input
Pointer to bit map data.

Pointer to the pel data of the bit map. This data is stored in the order that the coordinates appear on a display screen, that is, the pel in the lower-left corner is the first in the bit map. Pels are scanned to the right, and upward, from that position. The bits of the first pel are stored beginning with the most significant bits of the first byte. The data for pels in each scan line is packed together tightly. However, all scan lines are padded at the end so that each one begins on a ULONG boundary. That is, three bytes of pel data will hold one 24-bit pel, three 8-bit pels, six 4-bit pels, or twenty-four 1-bit pels. If those three bytes are the only pel data for that scan line, one more byte of zeros would be required to pad the line to a ULONG boundary.

GreDrawBits Parameter - pInfo

pInfo (PBITMAPINFO) - input
Pointer to data structure defining the new bit map. Pointer to either a BITMAPINFO structure:

cbFix	Length of structure
cx	Bit-map width
cy	Bit-map height
cPlanes	Number of color planes, one if standard format
cBitCount	Number of adjacent color bits per pel
argbColor[]	Color table array of RGB structures: bBlue bGreen bRed

Or pointer to a BITMAPINFO2 structure:

cbFix	Length of structure
cx	Bit-map width
cy	Bit-map height
cPlanes	Number of color planes, one if standard format
cBitCount	Number of adjacent color bits per pel
ulCompression	Compression scheme used to store the bit map: BCA_UNCOMP — Bit map is uncompressed (the only valid value).
cbImage	Length of bit map storage data in bytes. If the bit map is uncompressed, 0 (the default) can be specified.
cxResolution	Horizontal component of the resolution of the target device. That is, the resolution of the device the bit map is intended for in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cyResolution	Vertical component of the resolution of the target device. That is, the resolution of the device the bit map is intended for in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cclrUsed	The number of color indexes from the color table that are used by the bit map. If it is 0 (default), all the indexes are used. If it is nonzero, only the first cclrUsed entries in the table are accessed by the system; further entries can be omitted. For standard formats with a cBitCount of 1, 4, or 8 (and cPlanes=1), any indexes beyond cclrUsed are not valid. For example, a bit map with 64 colors can use the 8-bitcount format without having to supply the other 192 entries in the color table. For the 24-bitcount standard format, cclrUsed is the number of colors used by the bit map.
cclrImportant	Minimum number of color indexes for satisfactory appearance of the bit map. More colors can be used in the bit map but it is not necessary to assign them to the device palette. These additional colors can be mapped to the nearest colors available. Zero (default) means that all entries are important. For a 24-bitcount standard format, the cclrImportant colors are also listed in the color table relating to this bit map.
usUnits	Units of measure of the horizontal and vertical components of resolution:

	BRU_METRIC	(Default) Pels per meter

usReserved

Reserved field. If present, it must be 0.

usRecording

Recording algorithm, the format in which bit map data is recorded:

BRA_BOTTOMUP (Default) Scanlines are recorded from bottom to top.

usRendering

Half-toning algorithm used to record bit map data that has been digitally halftoned:

BRH_NOTHALFTONED

BRH_ERRORDIFFUSION

BRH_PANDA

BRH_SUPERCIRCLE

cSize1	Size value 1. If BRH_ERRORDIFFUSION is specified in usRendering, cSize1 is the error damping as a percentage in the range 0-100. A value of 100% indicates no damping. A value of 0% indicates that any errors are not diffused. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize1 is the X-dimension of the pattern used in pels.
cSize2	Size value 2. If BRH_ERRORDIFFUSION is specified in usRendering, this parameter is ignored. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize2 is the Y-dimension of the pattern used in pels.

ulColorEncoding	Color encoding:	
	BCE_RGB	(Default) Each element in the color array is an RGB2 data type.
ulIdentifier	Reserved for application use	
argbColor[]	Color table array of RGB2 structures:	
	bBlue bGreen bRed	
	fcOptions	Reserved. Must be 0.

GreDrawBits Parameter - cPoints

cPoints (LONG) - input
Number of (X, Y) pairs in paptlPoint.

This count must be set to 4.

The operation is determined by comparing the sizes of the two rectangles:

Target<Source	Compress the source rectangle into the target rectangle. The flOptions flags determine how eliminated rows and columns should be handled.
Target=Source	Copy between equal rectangles.
Target>Source	Stretch the source rectangle into the target rectangle. In this case, the call can be passed to the GreBitblt routine in the graphics engine.

GreDrawBits Parameter - paptlPoint

paptlPoint (PPOINTL) - input
Pointer to an array of (X, Y) coordinate pairs.

A pointer to a block of (X, Y) coordinate pairs that define the target and source rectangles. The coordinates, which can be passed as a pair of RECTL structures, define the bottom-left and top-right corners of the target and source rectangles (see "cPoints" above for more information):

```
(xTgtBL, yTgtBL), (xTgtTR, yTgtTR), (xSrcBL, ySrcBL), (xSrcTR, ySrcTR)
```

When the source rectangle is totally or partially outside the source bit map (or device), the operation is implementation-dependent for that area, that is, the user must decide what to do.

Note: When BBO_TARGWORLD is set, the target rectangle is inclusive at all boundaries. The source is noninclusive.

When BBO_TARGWORLD is not set, the rectangles are noninclusive. That is, they include the left and lower boundaries in device units but not the top and right boundaries. When the bottom-left corner of a rectangle maps to the same device pel as the top-right corner, that rectangle is considered to be empty.

GreDrawBits Parameter - lRop

lRop (LONG) - input
Raster operation code.

The low-order byte represents a mix value in the range, hex 00 - hex FF. Raster operation code values and the mix-bit table are defined in the *Presentation Manager Programming Reference* . The handling routine uses lRop to determine the operations to perform on pattern, source, and target to get the required mix.

In addition to the raster operation values defined at the API, the presentation driver must support ROP_GRAY (hex 000080CA). This value is used to shade the text for menu items that are not currently selectable. When ROP_GRAY is set, the handling routine overpaints the foreground pattern using the current pattern and the background pattern color (background pels for the pattern are not changed). For the PATSYM_HALFTONE pattern, this overpaints the background pattern color onto alternate pels, leaving those in between unchanged.

GreDrawBits Parameter - flOptions

flOptions (ULONG) - input
Specified treatment of eliminated lines and columns. Option flags:

BBO_OR	Stretch and compress, as necessary, ORing any eliminated rows and columns. Used for White on Black.
BBO_AND	Stretch and compress, as necessary, ANDing any eliminated rows and columns. Used for Black on White.
BBO_IGNORE	Stretch and compress, as necessary, IGNORing any eliminated rows and columns. Used for color.
BBO_TARGWORLD	The target rectangle is defined in world coordinates in the target PS. When this option is specified, the target rectangle is transformed to device coordinates. Where any shear or rotation has occurred, the target rectangle must be converted to an upright rectangle that bounds the transformed figure. This rectangle is then used as the target for the operation. No inversion of the image takes place.
BBO_COLORS	The source color map is defined in terms of indexes into the destination color map instead of RGB values.
BBO_PAL_COLORS	The color values in BITMAPINFOHEADER2 are not colors; they are indexes into the currently realized palette.
BBO_NO_COLOR_INFO	No color information is associated with the source. The source bit map should be copied directly to the destination without translating the pels. This option must be supported if the device supports palette management.
BBO_DITHER	Use an enhanced stretching algorithm when stretching a bit map.
BLTMODE_SRC_BITMAP	hdcSrc is a bit map handle. The bit map must not be currently selected into a device context. If this flag is not set, hdcSrc is a DC handle.
BLTMODE_ATTRS_PRES	If set, the pBattrs parameter is present. This option can be ORed with any of the options above.

Note: Flags 15-31 are not used by the system; they are reserved for use by the presentation driver.

GreDrawBits Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreDrawBits Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDrawBits.

GreDrawBits Return Value - rc

rc (ULONG) - returns
Return Codes.

On completion, the handling routine must return a LONG integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlation hit (returned by the display driver when the correlation flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_HDC_BUSY
PMERR_INCORRECT_DC_TYPE
PMERR_INV_BITBLT_MIX
PMERR_INV_BITBLT_STYLE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDrawBits - Parameters

hdc (HDC) - input
Device context handle.

pBitmap (PBYTE) - input
Pointer to bit map data.

Pointer to the pel data of the bit map. This data is stored in the order that the coordinates appear on a display screen, that is, the pel in the lower-left corner is the first in the bit map. Pels are scanned to the right, and upward, from that position. The bits of the first pel are stored beginning with the most significant bits of the first byte. The data for pels in each scan line is packed together tightly. However, all scan lines are padded at the end so that each one begins on a ULONG boundary. That is, three bytes of pel data will hold one 24-bit pel, three 8-bit pels, six 4-bit pels, or twenty-four 1-bit pels. If those three bytes are the only pel data for that scan line, one more byte of zeros would be required to pad the line to a ULONG boundary.

pInfo (PBITMAPINFO) - input
Pointer to data structure defining the new bit map. Pointer to either a BITMAPINFO structure:

cbFix	Length of structure
cx	Bit-map width

cy	Bit-map height
cPlanes	Number of color planes, one if standard format
cBitCount	Number of adjacent color bits per pel
argbColor[]	Color table array of RGB structures: bBlue bGreen bRed

Or pointer to a BITMAPINFO2 structure:

cbFix	Length of structure
cx	Bit-map width
cy	Bit-map height
cPlanes	Number of color planes, one if standard format
cBitCount	Number of adjacent color bits per pel
ulCompression	Compression scheme used to store the bit map: BCA_UNCOMP — Bit map is uncompressed (the only valid value).
cbImage	Length of bit map storage data in bytes. If the bit map is uncompressed, 0 (the default) can be specified.
cxResolution	Horizontal component of the resolution of the target device. That is, the resolution of the device the bit map is intended for in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cyResolution	Vertical component of the resolution of the target device. That is, the resolution of the device the bit map is intended for in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cclrUsed	The number of color indexes from the color table that are used by the bit map. If it is 0 (default), all the indexes are used. If it is nonzero, only the first cclrUsed entries in the table are accessed by the system; further entries can be omitted. For standard formats with a cBitCount of 1, 4, or 8 (and cPlanes=1), any indexes beyond cclrUsed are not valid. For example, a bit map with 64 colors can use the 8-bitcount format without having to supply the other 192 entries in the color table. For the 24-bitcount standard format, cclrUsed is the number of colors used by the bit map.
cclrImportant	Minimum number of color indexes for satisfactory appearance of the bit map. More colors can be used in the bit map but it is not necessary to assign them to the device palette. These additional colors can be mapped to the nearest colors available. Zero (default) means that all entries are important. For a 24-bitcount standard format, the cclrImportant colors are also listed in the color table relating to this bit map.
usUnits	Units of measure of the horizontal and vertical components of resolution: BRU_METRIC — (Default) Pels per

meter

usReserved	Reserved field. If present, it must be 0.
usRecording	Recording algorithm, the format in which bit map data is recorded: BRA_BOTTOMUP (Default) Scanlines are recorded from bottom to top.
usRendering	Half-toning algorithm used to record bit map data that has been digitally halftoned: BRH_NOTHALFTONED

BRH_ERRORDIFFUSION

BRH_PANDA

BRH_SUPERCIRCLE

cSize1

Size value 1. If BRH_ERRORDIFFUSION is specified in usRendering, cSize1 is the error damping as a percentage in the range 0-100. A value of 100% indicates no damping. A value of 0% indicates that any errors are not diffused.

If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize1 is the X-dimension of the pattern used in pels.

cSize2

Size value 2. If BRH_ERRORDIFFUSION is specified in usRendering, this parameter is ignored. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize2 is the Y-dimension of the pattern used in pels.

ulColorEncoding

Color encoding:

BCE_RGB (Default) Each

element in the color array is an RGB2 data type.

ulIdentifier	Reserved for application use
argbColor[]	Color table array of RGB2 structures:
bBlue	
bGreen	
bRed	
fcOptions	Reserved. Must be 0.

cPoints (LONG) - input
Number of (X, Y) pairs in paptlPoint.

This count must be set to 4.

The operation is determined by comparing the sizes of the two rectangles:

Target<Source	Compress the source rectangle into the target rectangle. The flOptions flags determine how eliminated rows and columns should be handled.
Target=Source	Copy between equal rectangles.
Target>Source	Stretch the source rectangle into the target rectangle. In this case, the call can be passed to the GreBitblt routine in the graphics engine.

paptlPoint (PPOINTL) - input
Pointer to an array of (X, Y) coordinate pairs.

A pointer to a block of (X, Y) coordinate pairs that define the target and source rectangles. The coordinates, which can be passed as a pair of RECTL structures, define the bottom-left and top-right corners of the target and source rectangles (see "cPoints" above for more information):

```
(xTgtBL, yTgtBL), (xTgtTR, yTgtTR), (xSrcBL, ySrcBL), (xSrcTR, ySrcTR)
```

When the source rectangle is totally or partially outside the source bit map (or device), the operation is implementation-dependent for that area, that is, the user must decide what to do.

Note: When BBO_TARGWORLD is set, the target rectangle is inclusive at all boundaries. The source is noninclusive.

When BBO_TARGWORLD is not set, the rectangles are noninclusive. That is, they include the left and lower boundaries in device units but not the top and right boundaries. When the bottom-left corner of a rectangle maps to the same device pel as the top-right corner, that rectangle is considered to be empty.

lRop (LONG) - input
Raster operation code.

The low-order byte represents a mix value in the range, hex 00 - hex FF. Raster operation code values and the mix-bit table are defined in the *Presentation Manager Programming Reference* . The handling routine uses lRop to determine the operations to perform on pattern, source, and target to get the required mix.

In addition to the raster operation values defined at the API, the presentation driver must support ROP_GRAY (hex 000080CA). This value is used to shade the text for menu items that are not currently selectable. When ROP_GRAY is set, the handling routine overpaints the foreground pattern using the current pattern and the background pattern color (background pels for the pattern are not changed). For the PATSYM_HALFTONE pattern, this overpaints the background pattern color onto alternate pels, leaving those in between unchanged.

flOptions (ULONG) - input
Specified treatment of eliminated lines and columns. Option flags:

BBO_OR	Stretch and compress, as necessary, ORing any eliminated rows and columns. Used for White on Black.
BBO_AND	Stretch and compress, as necessary, ANDing any eliminated rows and columns. Used for Black on White.
BBO_IGNORE	Stretch and compress, as necessary, IGNORing any eliminated rows and columns. Used for color.
BBO_TARGWORLD	The target rectangle is defined in world coordinates in the target PS. When this option is specified, the target rectangle is transformed to

	device coordinates. Where any shear or rotation has occurred, the target rectangle must be converted to an upright rectangle that bounds the transformed figure. This rectangle is then used as the target for the operation. No inversion of the image takes place.
BBO_COLORS	The source color map is defined in terms of indexes into the destination color map instead of RGB values.
BBO_PAL_COLORS	The color values in BITMAPINFOHEADER2 are not colors; they are indexes into the currently realized palette.
BBO_NO_COLOR_INFO	No color information is associated with the source. The source bit map should be copied directly to the destination without translating the pels. This option must be supported if the device supports palette management.
BBO_DITHER	Use an enhanced stretching algorithm when stretching a bit map.
BLTMODE_SRC_BITMAP	hdcSrc is a bit map handle. The bit map must not be currently selected into a device context. If this flag is not set, hdcSrc is a DC handle.
BLTMODE_ATTRS_PRES	If set, the pBattrs parameter is present. This option can be ORed with any of the options above.

Note: Flags 15-31 are not used by the system; they are reserved for use by the presentation driver.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDrawBits.

rc (ULONG) - returns
Return Codes.

On completion, the handling routine must return a LONG integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlation hit (returned by the display driver when the correlation flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_HDC_BUSY
PMERR_INCORRECT_DC_TYPE
PMERR_INV_BITBLT_MIX
PMERR_INV_BITBLT_STYLE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDrawBits - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreDrawBorder

GreDrawBorder - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreDrawBorder draws an internal border in a rectangular frame. The interior can also be filled.

This function must be hooked by all presentation drivers. Drivers for hardcopy devices do nothing except return TRUE (Successful). Display drivers must return Failure if fast-frame drawing is not supported. GreDrawBorder is not called by any specific function; it is used to do fast frame drawing.

```
#define INCL_GRE_BITMAPS
#include <os2.h>

HDC       hdc;           /*  Device context handle. */
PRECTL    prclFrame;
ULONG     cxBorder;      /*  Thickness of vertical border in device coordinates. */
ULONG     cyBorder;      /*  Thickness of horizontal border in device coordinates. */
LONG      clrBorder;     /*  Color of border in any valid format. */
LONG      clrInterior;   /*  Color of interior in any valid format. */
ULONG     flCmd;         /*  Options flag. */
PVOID     pInstance;     /*  Pointer to instance data. */
ULONG     lFunction;
BOOL      rc;            /*  Return Codes. */

rc = GreDrawBorder(hdc, prclFrame, cxBorder,
       cyBorder, clrBorder, clrInterior, flCmd,
       pInstance, lFunction);
```

GreDrawBorder Parameter - hdc

hdc (HDC) - input
Device context handle.

GreDrawBorder Parameter - prclFrame

prclFrame (PRECTL) - input
Pointer to a RECTL structure.

Pointer that defines the frame in screen coordinates. Coordinates are passed in screen coordinates, which are inclusive at the bottom-left corner and exclusive at the top-right corner.

GreDrawBorder Parameter - cxBorder

cxBorder (ULONG) - input
Thickness of vertical border in device coordinates.

GreDrawBorder Parameter - cyBorder

cyBorder (ULONG) - input
Thickness of horizontal border in device coordinates.

GreDrawBorder Parameter - clrBorder

clrBorder (LONG) - input
Color of border in any valid format.

GreDrawBorder Parameter - clrInterior

clrInterior (LONG) - input
Color of interior in any valid format.

GreDrawBorder Parameter - flCmd

flCmd (ULONG) - input
Options flag.

Valid values are:

DB_ROP
: An option flag that defines the mix to be used for the border and the interior. The mixes are mutually exclusive with one another but can be combined independently with DB_INTERIOR or DB_AREAATTRS. This option can have any of the following values:

  DB_PATCOPY
  : (Default) Use ROP_PATCOPY (see GreBitblt). This is a copy of the pattern to the destination.

  DB_PATINVERT
  : Exclusive-OR the current pattern and the destination (ROP_PATINVERT). Current mix and color parameters are ignored.

DB_DESTINVERT
: Inverts the destination (ROP_DESTINVERT).

DB_AREAMIXMODE
: Maps the current area foreground-mix attribute to a BitBlt raster operation. The area background-mix mode is ignored.

DB_INTERIOR
: Fills the area defined by prclFrame, excluding the border defined by cxBorder and cyBorder.

DB_AREAATTRS
: When set, the pattern used for the border is the one currently defined in the area attribute. The pattern used for the interior is the one that would be obtained by calling GreSetAttributes with the area attribute background color passed for the foreground color, and the area attribute foreground color passed for the background. See GreSetAttributes.

  When this flag is not set (default), the border pattern is equivalent to using GreSetAttributes for the area attributes, which use clrBorder as foreground color and clrInterior as the background.

  The Interior pattern is equivalent to using GreSetAttributes for the area attributes, which use clrInterior color as foreground color and clrBorder as the background.

The handling routine should ignore the remaining flags, DB_STANDARD and DB_DLGBORDER, which are used by the Frame Manager.

GreDrawBorder Parameter - pInstance

pInstance (PVOID) - input
: Pointer to instance data.

GreDrawBorder Parameter - lFunction

lFunction (ULONG) - input
: High-order WORD=flags; low-order WORD=NGreDrawBorder.

GreDrawBorder Return Value - rc

rc (BOOL) - returns
: Return Codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes

for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_IS_SELECTED
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INCOMPATIBLE_BITMAP
PMERR_INCORRECT_DC_TYPE
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BITBLT_MIX
PMERR_INV_BITBLT_STYLE
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_COLOR_ATTR
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_FORMAT
PMERR_INV_COLOR_INDEX
PMERR_INV_COLOR_OPTIONS
PMERR_INV_COLOR_START_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_DC_DATA
PMERR_INV_DC_TYPE
PMERR_INV_DRAW_BORDER_OPTION
PMERR_INV_DRIVER_NAME
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_ID
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_SCAN_START
PMERR_INV_USAGE_PARM
PMERR_UNSUPPORTED_ATTR_VALUE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDrawBorder - Parameters

hdc (HDC) - input
Device context handle.

prclFrame (PRECTL) - input
Pointer to a RECTL structure.

Pointer that defines the frame in screen coordinates. Coordinates are passed in screen coordinates, which are inclusive at the bottom-left corner and exclusive at the top-right corner.

cxBorder (ULONG) - input
Thickness of vertical border in device coordinates.

cyBorder (ULONG) - input
Thickness of horizontal border in device coordinates.

clrBorder (LONG) - input
Color of border in any valid format.

clrInterior (LONG) - input
Color of interior in any valid format.

flCmd (ULONG) - input
Options flag.

Valid values are:

DB_ROP
An option flag that defines the mix to be used for the border and the interior. The mixes are mutually exclusive with one another but can be combined independently with DB_INTERIOR or DB_AREAATTRS. This option can have any of the following values:

DB_PATCOPY
(Default) Use ROP_PATCOPY (see GreBitblt). This is a copy of the pattern to the destination.

DB_PATINVERT
Exclusive-OR the current pattern and the destination (ROP_PATINVERT). Current mix and color parameters are ignored.

DB_DESTINVERT
Inverts the destination (ROP_DESTINVERT).

DB_AREAMIXMODE
Maps the current area foreground-mix attribute to a BitBlt raster operation. The area background-mix mode is ignored.

DB_INTERIOR
Fills the area defined by prclFrame, excluding the border defined by cxBorder and cyBorder.

DB_AREAATTRS
When set, the pattern used for the border is the one currently defined in the area attribute. The pattern used for the interior is the one that would be obtained by calling GreSetAttributes with the area attribute background color passed for the foreground color, and the area attribute foreground color passed for the background. See GreSetAttributes.

When this flag is not set (default), the border pattern is equivalent to using GreSetAttributes for the area attributes, which use clrBorder as foreground color and clrInterior as the background.

The Interior pattern is equivalent to using GreSetAttributes for the area attributes, which use clrInterior color as foreground color and clrBorder as the background.

The handling routine should ignore the remaining flags, DB_STANDARD and DB_DLGBORDER, which are used by the Frame Manager.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDrawBorder.

rc (BOOL) - returns
Return Codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_IS_SELECTED
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INCOMPATIBLE_BITMAP
PMERR_INCORRECT_DC_TYPE
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BITBLT_MIX
PMERR_INV_BITBLT_STYLE
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_COLOR_ATTR
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_FORMAT
PMERR_INV_COLOR_INDEX
PMERR_INV_COLOR_OPTIONS
PMERR_INV_COLOR_START_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_DC_DATA
PMERR_INV_DC_TYPE
PMERR_INV_DRAW_BORDER_OPTION
PMERR_INV_DRIVER_NAME
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_ID
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_SCAN_START
PMERR_INV_USAGE_PARM
PMERR_UNSUPPORTED_ATTR_VALUE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDrawBorder - Remarks

The parameters cxBorder and cyBorder can be passed as 0. When both are 0, the interior must still be drawn. When the X-borders and Y-borders overlap, the border is drawn as a single rectangle with no interior.

GreDrawBorder is a BitBlt accelerator and is similar in function and limitation to GreBitblt, GreDeviceSetAttributes, and GreSetAttributes.

See the *Presentation Manager Programming Reference* for a description of the WinDrawBorder function.

GreDrawBorder - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreDrawLinesInPath

GreDrawLinesInPath - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreDrawLinesInPath draws a sequence of one or more straight lines from the sequence of linked structures addressed by pLine.

This function must be supported by all presentation drivers. However, it is not called by any specific function. GreDrawLinesInPath is used by the filing routines and the path rendering routines within the graphics engine to produce output.

```
#define INCL_GRE_LINES
#include <os2.h>

HDC       hdc;         /*  Device context handle. */
PRECTL    prclBound;   /*  Pointer to a bounding box for the lines. */
PCURVE    pLine;       /*  Pointer to the first of a series of linked structures. */
ULONG     cLines;      /*  Count of LINE structures to draw. */
PVOID     pInstance;   /*  Pointer to instance data. */
ULONG     lFunction;
LONG      rc;          /*  Return Codes. */

rc = GreDrawLinesInPath(hdc, prclBound, pLine,
       cLines, pInstance, lFunction);
```

GreDrawLinesInPath Parameter - hdc

hdc (HDC) - input
Device context handle.

GreDrawLinesInPath Parameter - prclBound

prclBound (PRECTL) - input
Pointer to a bounding box for the lines.

Pointer to a RECTL structure:

xLeft	Minimum X-coordinate of rectangle
yBottom	Minimum Y-coordinate of rectangle
xRight	Maximum X-coordinate of rectangle
yTop	Maximum Y-coordinate of rectangle

GreDrawLinesInPath Parameter - pLine

pLine (PCURVE) - input
Pointer to the first of a series of linked structures.

A pointer to the first structure in the path definition containing the lines that have to be drawn. The LINE structure, shown below, is an example of the more general CURVE structure. These two structures and the FILLETSHARP structure are defined in the header file. They all take the same general form and are distinguished by the value of bType.

The LINE structure is defined as:

bIdent	Identifier. This value has no significance for the presentation driver.
bType	Structure type. The only significant value is LINE_IDENTIFIER, which indicates that this structure is a LINE structure. If any other value is detected, the handling routine should skip to the structure addressed by npcvNext.
ulStyle	Line style. See Line Attributes.
fl	Flags. The only flag is: CURVE_DO_FIRST_PEL When set, the handling routine must draw the first pel in the line. Se

e a l s o F i r s t a n d L a s t P e l C o n s i d e r a t i o n s .

pcvNext	Pointer to next structure in the sequence.
pcvPrev	Pointer to previous structure.
Reserved1[2]	Reserved parameter.
ptfxA	Start of the already clipped line (inclusive).
ptfxC	End of the already clipped line (inclusive).
ptlA	Start point of unclipped line (inclusive).
ptlC	End point of unclipped line (inclusive).
lRslope	Ignored by the presentation driver.
Reserved2[4]	Reserved parameter.

GreDrawLinesInPath Parameter - cLines

cLines (ULONG) - input
Count of LINE structures to draw.

GreDrawLinesInPath Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreDrawLinesInPath Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDrawLinesInPath.

GreDrawLinesInPath Return Value - rc

rc (LONG) - returns
Return Codes.

On completion, the handling routine must return an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlation hit (returned by display drivers when the correlation flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDrawLinesInPath - Parameters

hdc (HDC) - input
Device context handle.

prclBound (PRECTL) - input
Pointer to a bounding box for the lines.

Pointer to a RECTL structure:

xLeft	Minimum X-coordinate of rectangle
yBottom	Minimum Y-coordinate of rectangle
xRight	Maximum X-coordinate of rectangle
yTop	Maximum Y-coordinate of rectangle

pLine (PCURVE) - input
Pointer to the first of a series of linked structures.

A pointer to the first structure in the path definition containing the lines that have to be drawn. The LINE structure, shown below, is an example of the more general CURVE structure. These two structures and the FILLETSHARP structure are defined in the header file. They all take the same general form and are distinguished by the value of bType.

The LINE structure is defined as:

bIdent	Identifier. This value has no significance for the presentation driver.
bType	Structure type. The only significant value is LINE_IDENTIFIER, which indicates that this structure is a LINE structure. If any other value is detected, the handling routine should skip to the structure addressed by npcvNext.
ulStyle	Line style. See Line Attributes.
fl	Flags. The only flag is: CURVE_DO_FIRST_PEL When set, the handling routine must draw the first pel in the line. See also F

pcvNext	Pointer to next structure in the sequence.
pcvPrev	Pointer to previous structure.
Reserved1[2]	Reserved parameter.
ptfxA	Start of the already clipped line (inclusive).
ptfxC	End of the already clipped line (inclusive).
ptlA	Start point of unclipped line (inclusive).
ptlC	End point of unclipped line (inclusive).
lRslope	Ignored by the presentation driver.
Reserved2[4]	Reserved parameter.

cLines (ULONG) - input
Count of LINE structures to draw.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDrawLinesInPath.

rc (LONG) - returns
Return Codes.

On completion, the handling routine must return an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlation hit (returned by display drivers when the correlation flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX

PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDrawLinesInPath - Remarks

The structures can be a mixture of LINE, CURVE, and FILLETSHARP structures. These have similar forms and the second field, *bType,* identifies the type of the structure.

Starting at the structure addressed by *pLine,* the handling routine examines the *bType* field of each structure in turn. If *bType* is LINE_IDENTIFIER, the handling routine draws the line. Otherwise, it uses the value of the *npcvNext* field to skip to the next structure. This process continues until the handling routine has drawn the number of lines specified by cLines. Before drawing a line, the handling routine must check the CURVE_DO_FIRST_PEL flag to determine whether it should draw the first pel of the line. When the line passes between two pel positions, the presentation driver should round down to the nearest pel for values of 0.5 or less.

The call to GreDrawLinesInPath in the presentation driver is made by the graphics engine. Note that the coordinates are passed as screen coordinates (device coordinates + DC origin), and the lines are already completely clipped.

GreDrawLinesInPath - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreDrawRLE

GreDrawRLE - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreDrawRLE draws a run-length-encoded shape. If the call is passed back to the graphics engine, it calls the GrePolyScanline entry point.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_LINE
#include <os2.h>
```

```
HDC         hdc;         /*  Device context handle. */
PRLEHDR     pRLEHDR;     /*  Pointer to run-length-encoding data header. */
PVOID       pInstance;   /*  Pointer to instance data. */
ULONG       lFunction;   /*  High-order WORD=flags; low-order WORD=NGreDrawRLE. */
ULONG       rc;          /*  Return codes. */

rc = GreDrawRLE(hdc, pRLEHDR, pInstance, lFunction);
```

GreDrawRLE Parameter - hdc

hdc (HDC) - input
Device context handle.

GreDrawRLE Parameter - pRLEHDR

pRLEHDR (PRLEHDR) - input
Pointer to run-length-encoding data header.

Pointer an RLEHDR structure:

```
typedef struct {
                  LONG        lType;
                  BRECTL      brectlBounds;
                  PVOID       pRLE;
                } RLEHDR;
```

GreDrawRLE always supports lType=0. In this format, the pRLE field in the RLE header points to an array of POINTL structures (see below). The brectlBounds rectangle contains the tightest rectangle that fits around the shape.

The RLE array starts with an (X,Y) pair, where Y is the Y-coordinate, and X is the number of runs on the line. The number of (X,Y) pairs follow, where X is the left side of the run and Y is the right side, exclusive. The next (X,Y) pair holds the header of the next line. The Y-coordinate must increase. If the number of points is 0, the end of the data has been reached. For example, the following array of eight POINTL structures defines two scan lines of run-length-encoded data:

```
(3,45), (20,25), (42,56), (100,350),   (Three pairs of runs at Scan Line 45)
(2,46), (19,26), (43,56),              (Two pairs of runs at Scan Line 46)
(0,47)                                 (End of RLE data indicator)
```

The presentation driver draws lines between the following device-coordinate-space pairs, inclusive, given the data in the above example:

```
(20 ,45) to (24 ,45)
(42 ,45) to (55 ,45)
(100,45) to (349,45)
(19 ,46) to (25 ,46)
(43 ,46) to (55 ,46)
```

Notice that pels are drawn up to, but not including, the right coordinate of the run. That is, runs are filled inclusive or exclusive.

GreDrawRLE Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreDrawRLE Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDrawRLE.

GreDrawRLE Return Value - rc

rc (ULONG) - returns
Return codes.

On completion, the handling routine must return an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlation hit (returned by display drivers when the correlation flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE

PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDrawRLE - Parameters

hdc (HDC) - input
Device context handle.

pRLEHDR (PRLEHDR) - input
Pointer to run-length-encoding data header.

Pointer an RLEHDR structure:

```
typedef struct {
                  LONG        lType;
```

```
  BRECTL      brectlBounds;
  PVOID       pRLE;
} RLEHDR;
```

GreDrawRLE always supports lType=0. In this format, the pRLE field in the RLE header points to an array of POINTL structures (see below). The brectlBounds rectangle contains the tightest rectangle that fits around the shape.

The RLE array starts with an (X,Y) pair, where Y is the Y-coordinate, and X is the number of runs on the line. The number of (X,Y) pairs follow, where X is the left side of the run and Y is the right side, exclusive. The next (X,Y) pair holds the header of the next line. The Y-coordinate must increase. If the number of points is 0, the end of the data has been reached. For example, the following array of eight POINTL structures defines two scan lines of run-length-encoded data:

```
(3,45), (20,25), (42,56), (100,350),   (Three pairs of runs at Scan Line 45)
(2,46), (19,26), (43,56),              (Two pairs of runs at Scan Line 46)
(0,47)                                 (End of RLE data indicator)
```

The presentation driver draws lines between the following device-coordinate-space pairs, inclusive, given the data in the above example:

```
(20 ,45) to (24 ,45)
(42 ,45) to (55 ,45)
(100,45) to (349,45)
(19 ,46) to (25 ,46)
(43 ,46) to (55 ,46)
```

Notice that pels are drawn up to, but not including, the right coordinate of the run. That is, runs are filled inclusive or exclusive.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDrawRLE.

rc (ULONG) - returns
Return codes.

On completion, the handling routine must return an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlation hit (returned by display drivers when the correlation flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE

PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreDrawRLE - Remarks

GreDrawRLE is called by the area fill code in the graphics engine. The calling function uses the driver's

CAPS_ADDITIONAL_GRAPHICS/CAPS_CLIP_FILLS field, which is queried during the first instance of DevOpenDC. If it is set, the data delivered to GreDrawRLE has already been clipped and the COM_PRECLIP flag will be set in the flag's WORD of the function number. All coordinates are passed as screen coordinates.

GreDrawRLE - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEndArea

GreEndArea - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreEndArea indicates the end of a set of drawing functions that define the boundary of an area. If the final (X,Y) position is not the same as the starting position of the last figure in the area, the handling routine must close the figure by drawing a line from the current position to the start of the final figure. Upon completion, the current (X,Y) position is the last (X,Y) position specified in the area boundary unless a closure line was drawn. In this case, the start of the last figure in the area definition becomes the current (X,Y) position.

The area fill can be built up in memory or on devices that have hardware assistance for area fill in the device. For convex figures, performance gain can be increased by recording the start and end pel positions across each scan line. Whatever algorithm is used to fill the area, the interior fill must be identical in each occurrence. If it is not identical, bit-map operations can fail to join correctly when copied to the screen.

This function can be hooked by the presentation driver.

Note: If the flOptions bit BA_EXCL (specified on the GreBeginArea call) was set on and the driver is calling the engine with fill path for simulation, the fill path flOptions parameter must have the FPATH_EXCL bit set.

```
#define INCL_GRE_PATHS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    flCancel;
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreEndArea. */
LONG     rc;          /*  Return codes. */

rc = GreEndArea(hdc, flCancel, pInstance,
       lFunction);
```

GreEndArea Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEndArea Parameter - flCancel

flCancel (ULONG) - input
The value of this flag specifies whether this call cancels the area definition:

EA_DRAW	Draw the area. This flag is the default.
EA_CANCEL	Cancel the area definition. Otherwise, draw the area.

Note: When GreBeginArea is not called and EA_CANCEL is set, GreEndArea is valid but has no effect, thus allowing the handling routine to reset an area bracket to a known state when it has no knowledge of the actual current state.

GreEndArea Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreEndArea Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEndArea.

GreEndArea Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlation hit (returned by display drivers when the correlation flag is ON, and a hit is detected) on any part of the interior, regardless of mix
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH

PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_AREA_CONTROL
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_CHAR_DIRECTION_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_MIX_ATTR
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_NOT_IN_AREA
PMERR_PATH_LIMIT_EXCEEDED
PMERR_REGION_IS_CLIP_REGION.

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreEndArea - Parameters

hdc (HDC) - input
Device context handle.

flCancel (ULONG) - input
The value of this flag specifies whether this call cancels the area definition:

EA_DRAW	Draw the area. This flag is the default.
EA_CANCEL	Cancel the area definition. Otherwise, draw the area.

Note: When GreBeginArea is not called and EA_CANCEL is set, GreEndArea is valid but has no effect, thus allowing the handling routine to reset an area bracket to a known state when it has no knowledge of the actual current state.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEndArea.

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlation hit (returned by display drivers when the correlation flag is ON, and a hit is detected) on any part of the interior, regardless of mix
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_AREA_CONTROL
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_CHAR_DIRECTION_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_MIX_ATTR
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_NOT_IN_AREA
PMERR_PATH_LIMIT_EXCEEDED
PMERR_REGION_IS_CLIP_REGION.

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreEndArea - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreEndPath

GreEndPath - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreEndPath identifies the end of a sequence of figures that define a path. This function is valid outside a path definition but has no effect. When this function occurs outside a path, the handling routine should ignore it.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_PATHS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    flCancel;
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreEndPath. */
BOOL     fSuccess;    /*  Return codes. */

fSuccess = GreEndPath(hdc, flCancel, pInstance,
             lFunction);
```

GreEndPath Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEndPath Parameter - flCancel

flCancel (ULONG) - input
The value of flCancel specifies whether this call cancels the path definition:

EA_DRAW	Create the current path
EA_CANCEL	Cancel the path definition When GreBeginPath is not called and EA_CANCEL is set,

GreEndPath is valid but has no effect. This allows the handling routine to reset a path bracket to a known state when it has no knowledge of the actual current state.

GreEndPath Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreEndPath Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEndPath.

GreEndPath Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_END_PATH_OPTIONS
PMERR_INV_HDC
PMERR_NOT_IN_PATH
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreEndPath - Parameters

hdc (HDC) - input
Device context handle.

flCancel (ULONG) - input
The value of flCancel specifies whether this call cancels the path definition:

EA_DRAW	Create the current path
EA_CANCEL	Cancel the path definition When GreBeginPath is not called and EA_CANCEL is set,

GreEndPath is valid but has no effect. This allows the handling routine to reset a path bracket to a known state when it has no knowledge of the actual current state.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEndPath.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_END_PATH_OPTIONS
PMERR_INV_HDC
PMERR_NOT_IN_PATH
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreEndPath - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreEqualRegion

GreEqualRegion - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreEqualRegion checks whether two regions, owned by the device that is identified by hdc, are identical. An error is raised when either region is currently selected as the clip region.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_REGIONS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
HRGN     hrgnSrc1;    /*  First region handle. */
HRGN     hrgnSrc2;    /*  Second region handle. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreEqualRegion. */
LONG     rc;          /*  Return codes. */

rc = GreEqualRegion(hdc, hrgnSrc1, hrgnSrc2,
       pInstance, lFunction);
```

GreEqualRegion Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEqualRegion Parameter - hrgnSrc1

hrgnSrc1 (HRGN) - input
First region handle.

GreEqualRegion Parameter - hrgnSrc2

hrgnSrc2 (HRGN) - input
Second region handle.

GreEqualRegion Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreEqualRegion Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEqualRegion.

GreEqualRegion Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (lEquality) indicating whether the regions are equal:

EQRGN_EQUAL	Equal
EQRGN_ERROR	Error
EQRGN_NOTEQUAL	Not equal

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HRGN_BUSY
PMERR_INV_HRGN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreEqualRegion - Parameters

hdc (HDC) - input
Device context handle.

hrgnSrc1 (HRGN) - input
First region handle.

hrgnSrc2 (HRGN) - input
Second region handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEqualRegion.

rc (LONG) - returns
Return codes.

This function returns an integer (lEquality) indicating whether the regions are equal:

EQRGN_EQUAL	Equal
EQRGN_ERROR	Error
EQRGN_NOTEQUAL	Not equal

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HRGN_BUSY
PMERR_INV_HRGN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreEqualRegion - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreErasePS

GreErasePS - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreErasePS resets the presentation space of the device context to the background color CLR_BACKGROUND. Hardcopy drivers should return TRUE without taking any action.

This function must be supported by the presentation driver. GreErasePS is called from GpiErase, and is used to erase the contents of the

presentation space currently associated with the device context. Hardcopy drivers should not take any action except to return TRUE (Successful).

```
#define INCL_GRE_DEVMISC2
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;
BOOL     rc;          /*  Return Code. */

rc = GreErasePS(hdc, pInstance, lFunction);
```

GreErasePS Parameter - hdc

hdc (HDC) - input
Device context handle.

GreErasePS Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreErasePS Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreErasePS.

GreErasePS Return Value - rc

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreErasePS - Parameters

hdc (HDC) - input
Device context handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreErasePS.

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreErasePS - Remarks

The handling routine does not update GPI_BOUNDS or return correlation data, and it is not affected by the PCTL_DRAW control or COM_DRAW command flag. However, in display drivers, the handling routine should update USER_BOUNDS if the COM_ALT_BOUND command flag is set.

This function is subject to all clipping.

GreErasePS - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_ABORTDOC

GreEscape DEVESC_ABORTDOC - Syntax

Simulation support:

This function is mandatory for hardcopy drivers and must be implemented in order to have a valid OS/2 hardcopy driver. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreEscape DEVESC_ABORTDOC ends the current document.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_ABORTDOC escape code. */
LONG     cInCount;     /*  The handling routine ignores this parameter. */
PBYTE    pInData;      /*  The handling routine ignores this parameter. */
PLONG    pcOutCount;   /*  The handling routine ignores this parameter. */
PLONG    pOutData;     /*  The handling routine ignores this parameter. */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreEscape. */
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_ABORTDOC Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_ABORTDOC Parameter - lEscape

lEscape (LONG) - input
DEVESC_ABORTDOC escape code.

GreEscape DEVESC_ABORTDOC Parameter - cInCount

cInCount (LONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_ABORTDOC Parameter - pInData

pInData (PBYTE) - input

The handling routine ignores this parameter.

GreEscape DEVESC_ABORTDOC Parameter - pcOutCount

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_ABORTDOC Parameter - pOutData

pOutData (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_ABORTDOC Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_ABORTDOC Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_ABORTDOC Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	Error

GreEscape DEVESC_ABORTDOC - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_ABORTDOC escape code.

clnCount (LONG) - input
The handling routine ignores this parameter.

plnData (PBYTE) - input
The handling routine ignores this parameter.

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

pOutData (PLONG) - input
The handling routine ignores this parameter.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	Error

GreEscape DEVESC_ABORTDOC - Remarks

The handling routine in the presentation driver discards all data (such as data in a spooler buffer or journal file) received for the current document and closes any files associated with it. The current document is defined as any data back to, and including, the DEVESC_STARTDOC statement.

This function applies to hardcopy drivers only.

The following are three scenarios for ending a job and the steps to be taken by a hardcopy driver:

1. The user ends a job:
   a. The spooler calls SplQpControl(SPLC_ABORT).
   b. The queue driver calls DevEscape(DEVESC_ABORTDOC).
   c. The hardcopy driver sets a flag indicating that the job was aborted and returns the DevEscape(DEVESC_ABORTDOC) thread. The next time PrtWrite returns, the hardcopy driver completes the current page, for example, by sending a form feed and some null data to make sure the printer is not in graphics mode.
   d. The hardcopy driver calls PrtClose.

   Note: The hardcopy driver must be able to accept DEVESC_ABORT while processing data or DEVESC_ENDDOC.

2. The printer runs out of paper or is offline:
   a. The spooler function PrtWrite fails and returns an error to the hardcopy driver.
   b. If the job ended, the hardcopy driver calls PrtClose. Otherwise, it calls SplMessageBox.
   c. The spooler brings up a message box or sends a message to the user, holds the job, and waits on a semaphore until the job is released (possibly across the network), in which case it will return RETRY. If the user selects ABORT, the hardcopy driver calls PrtAbort and PrtClose. If the user selects RETRY, the hardcopy driver will try PrtWrite again (see above).

Note: The spooler will ignore all PrtWrite operations after PrtAbort is called. After the hardcopy driver has called PrtClose, it should return errors until the DC is closed.

3. The application ends the job while spooling:

   a. The application calls DevEscape(DEVESC_ABORTDOC).
   b. The hardcopy driver calls SplQmAbortDoc.
   c. The hardcopy driver flags the DC as being aborted.
   d. The application calls either DevCloseDC or DevEscape(DEVESC_STARTDOC) to start another job on the same DC.

Note: This escape code is metafiled but not recorded.

GreEscape DEVESC_ABORTDOC - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_ACQUIREFB

GreEscape DEVESC_ACQUIREFB - Syntax

Simulation support:

This function is mandatory for display drivers supporting software motion video. For other drivers, it is simulated by a handling routine in the graphics engine.

Description

GreEscape DEVESC_ACQUIREFB reserves a video adapter by acquiring the necessary internal Presentation Manager driver resources so that the calling routine has sole access to the video frame buffer and bank-switching registers. No use of the adapter by the Presentation Manager or any other process is allowed while the adapter is reserved.

The escape is intended to allow the calling routine to access memory directly (causing bank switches as needed), without concern for another process changing the bank registers. The device drivers are alerted that the software motion video component is using the bank address switch DevEscape.

This escape is used in conjunction with DEVESC_SWITCHBANK and DEVESC_DEACQUIREFB. When used, the handling routine saves the necessary state to allow the calling routine to access video memory directly and use DEV_SWITCHBANK. After the calling routine completes its access of video memory it must use DEVESC_DEACQUIREFB to restore the adapter state, and allow other processes to access the adapter.

If the adapter cannot be acquired, the handling routine immediately responds with DEVESC_ERROR.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     ICODE;        /*  DEVESC_ACQUIREFB escape code. */
LONG     lInCount;     /*  Number of bytes pointed to by pInData. */
```

```
PBYTE    pbInData;
PLONG    plOutCount;  /*  The handling routine ignores this parameter. */
PLONG    pbOutData;   /*  The handling routine ignores this parameter. */
LONG     rc;          /*  Return Code. */

rc = GreEscape(hdc, ICODE, lInCount, pbInData,
       plOutCount, pbOutData);
```

GreEscape DEVESC_ACQUIREFB Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_ACQUIREFB Parameter - ICODE

ICODE (LONG) - input
DEVESC_ACQUIREFB escape code.

GreEscape DEVESC_ACQUIREFB Parameter - lInCount

lInCount (LONG) - input
Number of bytes pointed to by pInData.

GreEscape DEVESC_ACQUIREFB Parameter - pbInData

pbInData (PBYTE) - input
Pointer to ACQUIREFB structure containing:

fAFBFlags	Only one flag is currently defined. AFB_SWITCH is set if the *ulBankNumber* field contains the value of a bank switch, which is to be active when the call returns. AFB_SWITCH should not be set for apertures that are not bank-switched.
ulBankNumber	This is an integer value specifying the initial video memory bank index; 0 (zero) for the first bank, 1 (one) for the second bank, and so on.
rctlRegion	This field gives the rectangle, in device coordinates, that will be used for "BLTing". The device driver will perform "Exclude" processing to hide the mouse cursor if it is in this area. If it is hidden when DEVESC_DEACQUIREFB is called, it will be restored.

GreEscape DEVESC_ACQUIREFB Parameter - plOutCount

plOutCount (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_ACQUIREFB Parameter - pbOutData

pbOutData (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_ACQUIREFB Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	Error

GreEscape DEVESC_ACQUIREFB - Parameters

hdc (HDC) - input
Device context handle.

lCODE (LONG) - input
DEVESC_ACQUIREFB escape code.

lInCount (LONG) - input
Number of bytes pointed to by pInData.

pbInData (PBYTE) - input
Pointer to ACQUIREFB structure containing:

fAFBFlags	Only one flag is currently defined. AFB_SWITCH is set if the *ulBankNumber* field contains the value of a bank switch, which is to be active when the call returns. AFB_SWITCH should not be set for apertures that are not bank-switched.
ulBankNumber	This is an integer value specifying the initial video memory bank index; 0 (zero) for the first bank, 1 (one) for the second bank, and so on.
rctlRegion	This field gives the rectangle, in device coordinates, that will be used for "BLTing". The device driver will perform "Exclude" processing to hide the mouse cursor if it is in this area. If it is hidden when DEVESC_DEACQUIREFB is called, it will be restored.

plOutCount (PLONG) - input
The handling routine ignores this parameter.

pbOutData (PLONG) - input
The handling routine ignores this parameter.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	Error

GreEscape DEVESC_ACQUIREFB - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreEscape DEVESC_BREAK_EXTRA

GreEscape DEVESC_BREAK_EXTRA - Syntax

Simulation support:

This function is mandatory for hardcopy drivers, but may simply return a DEVESC_NOTIMPLEMENTED code. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreEscape DEVESC_BREAK_EXTRA changes the width of the break character on a hardcopy device. The handling routine sets or resets, as determined by the value of cInCount, an extra width value for the break character. Upon completion, the width of the break character is the default width specified by the font plus any extra widths set by DEVESC_BREAK_EXTRA and DEVESC_CHAR_EXTRA. The extra widths can be positive, zero, or negative.

Note: This escape code is implemented by old hardcopy drivers. The function of this escape code is replaced by the character attribute fxBreakExtra. See Character Attributes.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_BREAK_EXTRA escape code. */
LONG     cInCount;     /*  Number of bytes pointed to by pInData. */
PBYTE    pInData;      /*  If cInCount is not equal to 0, pInData is a pointer to a fixed value. */
PLONG    pcOutCount;   /*  The handling routine ignores this parameter. */
PLONG    pOutData;     /*  The handling routine ignores this parameter. */
PVOID    pInstance;    /*  Pointer to instance data. */
```

```
ULONG     lFunction;    /*  High-order WORD=flags; low-order WORD=NGreEscape. */
LONG      rc;           /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_BREAK_EXTRA Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_BREAK_EXTRA Parameter - lEscape

lEscape (LONG) - input
DEVESC_BREAK_EXTRA escape code.

GreEscape DEVESC_BREAK_EXTRA Parameter - cInCount

cInCount (LONG) - input
Number of bytes pointed to by pInData.

When cInCount=0, the handling routine resets the extra width to 0.

GreEscape DEVESC_BREAK_EXTRA Parameter - pInData

pInData (PBYTE) - input
If cInCount is not equal to 0, pInData is a pointer to a fixed value.

This value is the required extra width defined in world coordinates.

GreEscape DEVESC_BREAK_EXTRA Parameter - pcOutCount

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_BREAK_EXTRA Parameter - pOutData

pOutData (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_BREAK_EXTRA Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_BREAK_EXTRA Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_BREAK_EXTRA Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_BREAK_EXTRA - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_BREAK_EXTRA escape code.

cInCount (LONG) - input
Number of bytes pointed to by pInData.

When cInCount=0, the handling routine resets the extra width to 0.

pInData (PBYTE) - input
If cInCount is not equal to 0, pInData is a pointer to a fixed value.

This value is the required extra width defined in world coordinates.

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

pOutData (PLONG) - input
The handling routine ignores this parameter.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_BREAK_EXTRA - Remarks

This escape code is metafiled and recorded.

GreEscape DEVESC_BREAK_EXTRA - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_CHAR_EXTRA

GreEscape DEVESC_CHAR_EXTRA - Syntax

Simulation support:

This function is mandatory for hardcopy drivers, but may simply return a DEVESC_NOTIMPLEMENTED code. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreEscape DEVESC_CHAR_EXTRA changes the width of all characters, including the break character on a hardcopy device. The handling routine sets or resets, as defined by the value of clnCount, an extra width value. Upon completion, the width of a character is the default width specified by the font plus the extra width set by DEVESC_CHAR_EXTRA. The extra width can be positive, zero, or negative.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_CHAR_EXTRA escape code. */
LONG     cInCount;     /*  Number of bytes pointed to by pInData. */
PBYTE    pInData;
PLONG    pcOutCount;   /*  The handling routine ignores this parameter. */
PLONG    pOutData;     /*  The handling routine ignores this parameter. */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_CHAR_EXTRA Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_CHAR_EXTRA Parameter - lEscape

lEscape (LONG) - input
DEVESC_CHAR_EXTRA escape code.

GreEscape DEVESC_CHAR_EXTRA Parameter - cInCount

cInCount (LONG) - input
Number of bytes pointed to by pInData.

When cInCount=0, the handling routine should reset the extra width to 0.

GreEscape DEVESC_CHAR_EXTRA Parameter - pInData

pInData (PBYTE) - input
If cInCount is not equal to 0, pInData is a pointer to a FIXED value. This value is the required extra width defined in world coordinates.

GreEscape DEVESC_CHAR_EXTRA Parameter - pcOutCount

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_CHAR_EXTRA Parameter - pOutData

pOutData (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_CHAR_EXTRA Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_CHAR_EXTRA Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_CHAR_EXTRA Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error.

GreEscape DEVESC_CHAR_EXTRA - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_CHAR_EXTRA escape code.

cInCount (LONG) - input
Number of bytes pointed to by pInData.

When cInCount=0, the handling routine should reset the extra width to 0.

pInData (PBYTE) - input
If cInCount is not equal to 0, pInData is a pointer to a FIXED value. This value is the required extra width defined in world coordinates.

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

pOutData (PLONG) - input
The handling routine ignores this parameter.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error.

GreEscape DEVESC_CHAR_EXTRA - Remarks

Note: This escape code is implemented by old hardcopy drivers. The function of this escape code is replaced by the character attribute, fxExtra. See Character Attributes.

This escape code is metafiled and recorded.

GreEscape DEVESC_CHAR_EXTRA - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_DBE_FONTMANAGEMENT

GreEscape DEVESC_DBE_FONTMANAGEMENT - Syntax

Simulation support: NONE

This function is mandatory for display drivers supporting the Double-Byte Character Set (DBCS). For other drivers, DEVESC_NOTIMPLEMENTED should be returned.

Description

GreEscape DEVESC_DBE_FONTMANAGEMENT is the entry point to sub-functions to support DBCS device font.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_DBE_FONTMANAGEMENT (24450) escape code. */
LONG     cInCount;     /*  The number of bytes pointed to by pInData. */
PBYTE    pInData;      /*  Pointer to the FONTMGRPARM structure. */
PLONG    pcOutCount;   /*  The handling routine ignores this parameter. */
PLONG    pOutData;
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_DBE_FONTMANAGEMENT Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_DBE_FONTMANAGEMENT Parameter - lEsc

lEscape (LONG) - input
DEVESC_DBE_FONTMANAGEMENT (24450) escape code.

GreEscape DEVESC_DBE_FONTMANAGEMENT Parameter - cInC

cInCount (LONG) - input
The number of bytes pointed to by pInData.

GreEscape DEVESC_DBE_FONTMANAGEMENT Parameter - pInI

pInData (PBYTE) - input
Pointer to the FONTMGRPARM structure.

GreEscape DEVESC_DBE_FONTMANAGEMENT Parameter - pcC

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_DBE_FONTMANAGEMENT Parameter - pOu

pOutData (PLONG) - input
When fdi.fdi_lFunctionID in pInData is QUERY_VIO_FONTATTRIBUTE, pOutData points to the FATTRS structure. FATTRS correspondS to the specified cell size, which is one of size returned by the GreEscape DEVESC_QUERYVIOCELLSIZES escape function.

The handling routine must fill lMatch number in the FATTRS structure. Be sure that the lMaxBaselineExt and lAveCharWidth field are returned in device coordinates. For other fdi.fdi_lFunctionID, the handling routine ignores this parameter.

GreEscape DEVESC_DBE_FONTMANAGEMENT Parameter - pIns

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_DBE_FONTMANAGEMENT Parameter - lFur

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_DBE_FONTMANAGEMENT Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_LENGTH_OR_COUNT

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreEscape DEVESC_DBE_FONTMANAGEMENT - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_DBE_FONTMANAGEMENT (24450) escape code.

cInCount (LONG) - input
The number of bytes pointed to by pInData.

pInData (PBYTE) - input
Pointer to the FONTMGRPARM structure.

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

pOutData (PLONG) - input
When fdi.fdi_lFunctionID in pInData is QUERY_VIO_FONTATTRIBUTE, pOutData points to the FATTRS structure. FATTRS correspondS to the specified cell size, which is one of size returned by the GreEscape DEVESC_QUERYVIOCELLSIZES escape function.

The handling routine must fill lMatch number in the FATTRS structure. Be sure that the lMaxBaselineExt and lAveCharWidth field are returned in device coordinates. For other fdi.fdi_lFunctionID, the handling routine ignores this parameter.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_LENGTH_OR_COUNT

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreEscape DEVESC_DBE_FONTMANAGEMENT - Remarks

For an OD_METAFILE device, DEVESC_DBE_FONTMANAGEMENT is passed to the presentation driver but is not metafiled. For an OD_QUEUED device with PM_Q_STD, the spooler records this escape, but it is not passed to the presentation driver.

GreEscape DEVESC_DBE_FONTMANAGEMENT - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_DEACQUIREFB

GreEscape DEVESC_DEACQUIREFB - Syntax

Simulation support:

This function is mandatory for display drivers supporting software motion video. For other drivers, it is simulated by a handling routine in the graphics engine.

Description

GreEscape DEVESC_DEACQUIREFB releases the internal Presentation Manager driver resources acquired by DEVESC_ACQUIREFB. The escape results in the release of the video adapter device for use by other threads.

The device drivers are informed that the process that reserved the adapter has completed its operation.

This function is used in conjunction with DEVESC_SWITCHBANK and DEVESC_ACQUIREFB. When used, the handling routine restores the state saved by DEVESC_ACQUIREFB and returns.

If DEVESC_ACQUIREFB had caused the mouse cursor to be hidden, this call will perform "unexclude" processing to restore the mouse cursor.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lCode;        /*  DEVESC_DEACQUIREFB escape code. */
LONG     lInCount;     /*  The handling routine ignores this parameter. */
PBYTE    pbInData;     /*  The handling routine ignores this parameter. */
PLONG    plOutCount;   /*  The handling routine ignores this parameter. */
PLONG    pbOutData;    /*  The handling routine ignores this parameter. */
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lCode, lInCount, pbInData,
       plOutCount, pbOutData);
```

GreEscape DEVESC_DEACQUIREFB Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_DEACQUIREFB Parameter - lCode

lCode (LONG) - input
DEVESC_DEACQUIREFB escape code.

GreEscape DEVESC_DEACQUIREFB Parameter - lInCount

lInCount (LONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_DEACQUIREFB Parameter - pbInData

pbInData (PBYTE) - input
The handling routine ignores this parameter.

GreEscape DEVESC_DEACQUIREFB Parameter - plOutCount

plOutCount (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_DEACQUIREFB Parameter - pbOutData

pbOutData (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_DEACQUIREFB Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	Error

GreEscape DEVESC_DEACQUIREFB - Parameters

hdc (HDC) - input
Device context handle.

lCode (LONG) - input
DEVESC_DEACQUIREFB escape code.

lInCount (LONG) - input
The handling routine ignores this parameter.

pbInData (PBYTE) - input
The handling routine ignores this parameter.

plOutCount (PLONG) - input
The handling routine ignores this parameter.

pbOutData (PLONG) - input
The handling routine ignores this parameter.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	Error

GreEscape DEVESC_DEACQUIREFB - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreEscape DEVESC_DEFAULTJOBPROPERTIES

GreEscape DEVESC_DEFAULTJOBPROPERTIES - Syntax

Simulation support:

This function is mandatory for all drivers and must be implemented in order to have a valid OS/2 hardcopy driver.

Description:

GreEscape DEVESC_DEFAULTJOBPROPERTIES resets (or defaults) the specified job property types inside the job properties structure that is passed in. Refer to Dynamic Job Properties for related information.

```
#define INCL_DEV
#define INCL_DEVDJP
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_DEFAULTJOBPROPERTIES escape code. */
LONG     cInCount;     /*  Number of bytes pointed to by pInData. */
PBYTE    pInData;      /*  Pointer to a list of DJP_ITEM data structures. */
PLONG    pcOutCount;   /*  Pointer to the number of bytes pointed to by pOutData. */
PLONG    pOutData;     /*  Pointer to the printer driver's job properties structure */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreEscape. */
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_DEFAULTJOBPROPERTIES Parameter - hd

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_DEFAULTJOBPROPERTIES Parameter - lEs

lEscape (LONG) - input
DEVESC_DEFAULTJOBPROPERTIES escape code.

GreEscape DEVESC_DEFAULTJOBPROPERTIES Parameter - cIn

cInCount (LONG) - input
Number of bytes pointed to by *pInData*.

GreEscape DEVESC_DEFAULTJOBPROPERTIES Parameter - pIr

pInData (PBYTE) - output
Pointer to a list of DJP_ITEM data structures.

The end of the list is indicated by a DJP_ITEM structure that has DJP_NONE as its *ulProperty* value.

Note: The values contained within the DJP_ITEMs are ignored. The corresponding job property information will be reset.

GreEscape DEVESC_DEFAULTJOBPROPERTIES Parameter - pc

pcOutCount (PLONG) - input
Pointer to the number of bytes pointed to by *pOutData*.

GreEscape DEVESC_DEFAULTJOBPROPERTIES Parameter - pC

pOutData (PLONG) - input
Pointer to the printer driver's job properties structure

as returned by SplQueryQueue. These may have been modified by GreEscape DEVESC_SETJOBPROPERTIES.

Note: On output, the job properties are changed.

GreEscape DEVESC_DEFAULTJOBPROPERTIES Parameter - pIr

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_DEFAULTJOBPROPERTIES Parameter - lFu

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_DEFAULTJOBPROPERTIES Return Value -

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK
Successful.

DEV_PROP_BUF_TOO_SMALL
The job properties buffer was too small to contain the proper job properties data or expanded DJP_ITEMs. The buffer will not be updated.

DEV_INV_INP_JOBPROPERTIES
Invalid job properties were passed in. The driver changed them to default values. This is an error and the item buffer will not be updated.

DEVESC_ERROR
Error

GreEscape DEVESC_DEFAULTJOBPROPERTIES - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_DEFAULTJOBPROPERTIES escape code.

cInCount (LONG) - input
Number of bytes pointed to by *pInData*.

pInData (PBYTE) - output
Pointer to a list of DJP_ITEM data structures.

The end of the list is indicated by a DJP_ITEM structure that has DJP_NONE as its *ulProperty* value.

Note: The values contained within the DJP_ITEMs are ignored. The corresponding job property information will be reset.

pcOutCount (PLONG) - input
Pointer to the number of bytes pointed to by *pOutData*.

pOutData (PLONG) - input
Pointer to the printer driver's job properties structure

as returned by SplQueryQueue. These may have been modified by GreEscape DEVESC_SETJOBPROPERTIES.

Note: On output, the job properties are changed.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK
Successful.

DEV_PROP_BUF_TOO_SMALL
The job properties buffer was too small to contain the proper job properties data or expanded DJP_ITEMs. The buffer will not be updated.

DEV_INV_INP_JOBPROPERTIES
Invalid job properties were passed in. The driver changed them to default values. This is an error and the item buffer will not be updated.

DEVESC_ERROR
Error

GreEscape DEVESC_DEFAULTJOBPROPERTIES - Remarks

GreEscape DEVESC_DEFAULTJOBPROPERTIES is used to default *individual* DJP_ITEMs rather than defaulting the entire job property structure. This structure is known only to the printer presentation driver, and this escape is the only way for an application to reset values inside the structure.

Since this function is *new* , applications will call GreEscape DEVESC_QUERYESCSUPPORT to determine if you support this DevEscape.

Note that the DevEscape path has a handle to a device context (HDC) that should already have printer property information associated with the HDC. Job property information may change or may be dependent upon the printer property information in the HDC. In that case, if an

application passes job properties for a different device or for a different spooler printer name, then misleading information will be returned.

Remember to test the input job properties to make sure that they are valid. This includes passing tests such as whether they contain your driver's signature string, whether the device name is yours, and whether values are within the proper ranges (only landscape or portrait). If these tests fail, then fail the call.

```
LONG ENGENTRY Escape (HDC       hdc,
                      LONG      lEscape,
                      LONG      cInCount,
                      PBYTE     pInData,
                      PLONG     pcOutCount,
                      PBYTE     pOutData,
                      PDDC      pddc,
                      ULONG     ulFunction)
{
   switch (lEscape)
   {
   case DEVESC_DEFAULTJOBPROPERTIES:
   {
      INT            cbQueries   = cInCount;
      PBYTE          pbQueries   = pInData;
      PDJP_ITEM      pISet       = (PDJP_ITEM)pbQueries;
      BOOL           fError      = FALSE;
      ULONG          ulMaxSize;

      pGoodDrivData = ReturnDriverData (pddc->pdb,
                                        pdi,
                                        pddc->pdb->hmcbHeap,
                                        pDrivData,
                                        pddc->pdb->pszPrinterName,
                                        TRUE,
                                        pDevice,
                                        pDriver);
      assertF (pGoodDrivData);

      /* Is the size of the input driver data is not the same as
      ** the size of our good data?
      */
      if (pDrivData->cb != pGoodDrivData->cb)
         lrc = DEV_PROP_BUF_TOO_SMALL;

      /* Or, if the two don't compare, then fail the call.
      */
      if (0 != memcmp (pDrivData, pGoodDrivData, pGoodDrivData->cb))
         lrc = DEV_INV_INP_JOBPROPERTIES;

      // Update the job properties with the input set list
      do
      {
         DBPRINTF (("Set default '%s'/%d.  Query type '%s'/%d. Simple value = %d\n",
                    pszProperty (pISet->ulProperty),
                    pISet->ulProperty,
                    pszType (pISet->lType),
                    pISet->lType,
                    pISet->ulValue));

         switch (pISet->ulProperty)
         {
         case DJP_SJ_ORIENTATION:
         {
            pJobProp->ulOrientation = pDevice->DeviceDefaults.ulOrientation;
            break;
         }

         case DJP_CJ_RESOLUTION:
         {
            pJobProp->ulDefResID = pDevice->DeviceDefaults.ulDefResID;
            break;
         }

         case DJP_SJ_BITSPERPEL:
         case DJP_SJ_COLOR:
         {
            pJobProp->ulDefPrintModeID = pDevice->DeviceDefaults.ulDefPrintModeID;
            break;
         }

         case DJP_SJ_PAPERSIZE:
         case DJP_CJ_FORM:
         {
            pJobProp->ulDefConnID = pDevice->DeviceDefaults.ulDefConnID;
```

```
            break;
         }

         case DJP_SJ_COPIES:
         {
            pJobProp->ulCopies = pDevice->DeviceDefaults.ulCopies;
            break;
         }

         case DJP_SJ_PRINTQUALITY:
         case DJP_SJ_TRAYTYPE:
         case DJP_SJ_MEDIA:
         case DJP_SJ_MEDIA_COLOR:
         case DJP_CJ_MIXEDFORMS:
         case DJP_SJ_FONTDOWNLOADING:
         case DJP_SJ_DUPLEX:
         case DJP_SJ_COLLATE:
         case DJP_SJ_FEED:
         case DJP_SJ_SCALING:
         case DJP_SJ_FORMFEEDCONTROL:
         case DJP_SJ_N_UP:
         default:
            fError = TRUE;
            pISet->lType = DJP_ERROR_NOT_SUPPORTED;
            DBPRINTF (("Unknow set default '%s' = %d!\n",
                       pszProperty (pISet->ulProperty),
                       pISet->ulProperty));
            break;
         }

         // Move to the set item
         pISet = DJP_NEXT_STRUCTP (pISet);

      } while (pISet->ulProperty != DJP_NONE);

      cbDrivData = cbParm2;
      pDrivData  = (PDRIVDATA)pbParm2;

      /* Copy the updated job properties onto the user's job properties
      */
      ulMaxSize = min (pGoodDrivData->cb, cbDrivData);
      memcpy (pDrivData, pGoodDrivData, ulMaxSize);
      pDrivData->cb = ulMaxSize;

      if (fError)
      {
         GplErrSetWarning (PMERR_DATATYPE_ENTRY_INVALID);
         ulrc = DEV_ERROR;
      }
      else
      {
         ulrc = DEV_OK;
      }
      break;
   }
   }

   return lrc;
}
```

GreEscape DEVESC_DEFAULTJOBPROPERTIES - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_DRAFTMODE

GreEscape DEVESC_DRAFTMODE - Syntax

Simulation support:

This function is mandatory for hardcopy drivers, but may simply return a DEVESC_NOTIMPLEMENTED code. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreEscape DEVESC_DRAFTMODE sets draft (text) mode ON or OFF. Setting draft mode ON tells the presentation driver that the coming page need not contain any graphics or all-points-addressable output.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_DRAFTMODE escape code. */
LONG     cInCount;     /*  Number of bytes pointed to by pInData. */
PBYTE    pInData;      /*  Short integer value specifying the mode. */
PLONG    pcOutCount;   /*  The handling routine ignores this parameter. */
PLONG    pOutData;     /*  The handling routine ignores this parameter. */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreEscape. */
LONG     rc;           /*  Return codes. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_DRAFTMODE Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_DRAFTMODE Parameter - lEscape

lEscape (LONG) - input
DEVESC_DRAFTMODE escape code.

GreEscape DEVESC_DRAFTMODE Parameter - cInCount

cInCount (LONG) - input
Number of bytes pointed to by pInData.

This parameter should be set to 2.

GreEscape DEVESC_DRAFTMODE Parameter - pInData

pInData (PBYTE) - input
Short integer value specifying the mode.

1 for draft mode ON, 0 for OFF.

GreEscape DEVESC_DRAFTMODE Parameter - pcOutCount

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_DRAFTMODE Parameter - pOutData

pOutData (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_DRAFTMODE Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_DRAFTMODE Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_DRAFTMODE Return Value - rc

rc (LONG) - returns
Return codes.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_DRAFTMODE - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_DRAFTMODE escape code.

cInCount (LONG) - input
Number of bytes pointed to by pInData.

This parameter should be set to 2.

pInData (PBYTE) - input
Short integer value specifying the mode.

1 for draft mode ON, 0 for OFF.

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

pOutData (PLONG) - input
The handling routine ignores this parameter.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return codes.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_DRAFTMODE - Remarks

This escape code is valid only at a page boundary, for example, after DEVESC_STARTDOC or DEVESC_NEWFRAME.

Note: This escape code is implemented by old hardcopy drivers and is being phased out.

This escape code is metafiled and recorded.

When draft mode is ON, the presentation driver can choose to optimize throughput by:

- Ignoring all graphics primitives such as lines, arcs, and areas
- Using the fonts provided by the output device
- Approximating positions received in calls to functions such as GreSetCurrentPosition and GreCharStringPos to the nearest character position that the output device supports for the current font.

The presentation driver must maintain current attributes such as color, mix, and transforms when draft mode is ON even though they might have

no effect on the draft output. Similarly, the driver needs to track font changes and respond by setting the appropriate device font such as enlarged, condensed, or italics.

Note: For an OD_QUEUED device with PM_Q_STD data, the spooler records this escape in the buffer and does not pass it to the presentation driver.

GreEscape DEVESC_DRAFTMODE - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_ENDDOC

GreEscape DEVESC_ENDDOC - Syntax

Simulation support:

This function is mandatory for hardcopy drivers and must be implemented in order to have a valid OS/2 hardcopy driver. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreEscape DEVESC_ENDDOC ends the current document.

Note: DEVESC_ENDDOC is mandatory at the API when writing PM_Q_STD data to an OD_QUEUED device.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_ENDDOC escape code. */
LONG     cInCount;     /*  The handling routine ignores this parameter. */
PBYTE    pInData;      /*  The handling routine ignores this parameter. */
PLONG    pcOutCount;   /*  Pointer to size parameter. */
PLONG    pOutData;     /*  Pointer to data area. */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreEscape. */
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_ENDDOC Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_ENDDOC Parameter - lEscape

lEscape (LONG) - input
DEVESC_ENDDOC escape code.

GreEscape DEVESC_ENDDOC Parameter - cInCount

cInCount (LONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_ENDDOC Parameter - pInData

pInData (PBYTE) - input
The handling routine ignores this parameter.

GreEscape DEVESC_ENDDOC Parameter - pcOutCount

pcOutCount (PLONG) - input
Pointer to size parameter.

On input, this parameter specifies the size of the buffer pointed to by pOutData. On output, it is set to the number of data bytes returned in this buffer. The input value of this parameter is usually 2. On completion, this is set to 0 if no ID is returned in pOutData.

GreEscape DEVESC_ENDDOC Parameter - pOutData

pOutData (PLONG) - input
Pointer to data area.

Pointer to a data area in which the Job ID of the spooled print job is returned. Set to NULL if there is no Job ID, for example, when the hardcopy DC is OD_DIRECT.

GreEscape DEVESC_ENDDOC Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_ENDDOC Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_ENDDOC Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_ENDDOC - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_ENDDOC escape code.

cInCount (LONG) - input
The handling routine ignores this parameter.

pInData (PBYTE) - input
The handling routine ignores this parameter.

pcOutCount (PLONG) - input
Pointer to size parameter.

On input, this parameter specifies the size of the buffer pointed to by pOutData. On output, it is set to the number of data bytes returned in this buffer. The input value of this parameter is usually 2. On completion, this is set to 0 if no ID is returned in pOutData.

pOutData (PLONG) - input
Pointer to data area.

Pointer to a data area in which the Job ID of the spooled print job is returned. Set to NULL if there is no Job ID, for example, when the hardcopy DC is OD_DIRECT.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_ENDDOC - Remarks

The handling routine does whatever work is required to complete the job. For example, a DC for an OD_QUEUED device with PM_Q_STD data would close the spooler buffer and transfer the buffered data to a spool file. As with DEVESC_STARTDOC, do not assume that this escape code is always issued at the end of a document. When it has not been issued, the DEVESC_ENDDOC work must be done in the BeginCloseDC or CloseDC routine.

This escape code is metafiled but not recorded.

GreEscape DEVESC_ENDDOC - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_EXTGET

GreEscape DEVESC_EXTGET - Syntax

Simulation support:

None.

Description

GreEscape DEVESC_EXTGET is called by EnDIVE users to copy an image from the screen to system memory or to off-screen video memory. This function should be called only when the image needs to be returned in a compressed format. If the hardware does not support compression, then this call should not be implemented.

The various compression formats supported by the hardware are returned in the GreEscape DEVESC_EXTQUERY function.

Users of this function are responsible for bracketing access to video memory and this function with GreEscape DEVESC_HWREQUEST.

```
#define INCL_GRE_DEVICE
```

```
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     ICODE;        /*  DEVESC_EXTGET escape code. */
LONG     lInCount;     /*  Number of bytes pointed to by pbInData. */
PBYTE    pbInData;     /*  Pointer to IMAGEPACK data structure. */
PLONG    plOutCount;   /*  NULL. */
PLONG    pbOutData;    /*  NULL. */
LONG     rc;           /*  Return Codes. */

rc = GreEscape(hdc, ICODE, lInCount, pbInData,
       plOutCount, pbOutData);
```

GreEscape DEVESC_EXTGET Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_EXTGET Parameter - ICODE

ICODE (LONG) - input
DEVESC_EXTGET escape code.

GreEscape DEVESC_EXTGET Parameter - lInCount

lInCount (LONG) - input
Number of bytes pointed to by pbInData.

GreEscape DEVESC_EXTGET Parameter - pbInData

pbInData (PBYTE) - input
Pointer to IMAGEPACK data structure.

GreEscape DEVESC_EXTGET Parameter - plOutCount

plOutCount (PLONG) - input
NULL.

GreEscape DEVESC_EXTGET Parameter - pbOutData

pbOutData (PLONG) - input
NULL.

GreEscape DEVESC_EXTGET Return Value - rc

rc (LONG) - returns
Return Codes.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	Error

GreEscape DEVESC_EXTGET - Parameters

hdc (HDC) - input
Device context handle.

ICODE (LONG) - input
DEVESC_EXTGET escape code.

lInCount (LONG) - input
Number of bytes pointed to by pbInData.

pbInData (PBYTE) - input
Pointer to IMAGEPACK data structure.

plOutCount (PLONG) - input
NULL.

pbOutData (PLONG) - input
NULL.

rc (LONG) - returns
Return Codes.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	Error

GreEscape DEVESC_EXTGET - Remarks

The GreEscape DEVESC_EXTPUT and GreEscape DEVESC_EXTQUERY functions are related to GreEscape DEVESC_EXTGET.

GreEscape DEVESC_EXTGET - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_EXTPUT

GreEscape DEVESC_EXTPUT - Syntax

Simulation support:

None.

Description

GreEscape DEVESC_EXTPUT is called by EnDIVE users to copy an image to the screen. The source image must be scaled to the dimensions specified in the IMAGEPACK input data structure.

The source data of this operation can be either linear system memory or single aperture off-screen VRAM that was previously allocated by the display driver's GreEscape DEVESC_VRAMALLOC function.

Stretch Blt Acceleration Processing

Since Stretch Blt accelerators support a wide variety of input formats that are typically different from the graphics adapter's VRAM, a COLORINFO data structure is included in the IMAGEBUF data structure to describe the source data. COLORINFO is one of the structures returned by the driver's call to GreEscape DEVESC_EXTQUERY. As long as the source color space is one of the driver's accelerated color spaces, the driver is expected to accelerate the Blt by using the Stretch Blt assist. Only COLORINFO data structures returned by GreEscape DEVESC_EXTQUERY can be specified for source and target color spaces.

Users of this function are responsible for bracketing access to video memory and this function with GreEscape DEVESC_HWREQUEST.

CODEC Processing

Fields in the IMAGEPACK data structure specify compression and decompression services from within the driver. When the corresponding capabilities are specified in the IMAGECAPS flags of the GreEscape DEVESC_EXTQUERY function, the IMAGEPACK CODEC fields must be supported. To support CODEC processing of video data, the driver must examine the flags in the CODECINFO structure to determine whether compression or decompression is required.

Roundtrip Processing

Through the use of bits in the command mask parameter, data can be passed to the driver for decompression and the output returned to the caller. Another bit controls whether the output is displayed. Conversely, uncompressed data can be passed in GreEscape DEVESC_EXTGET, be compressed, and returned to the caller.

```
#define INCL_GRE_DEVICE
#include <os2.h>
```

```
HDC         hdc;          /*  Device context handle. */
LONG        ICODE;        /*  DEVESC_EXTPUT escape code. */
LONG        lInCount;     /*  Number of bytes pointed to by pInData. */
PBYTE       pbInData;     /*  Pointer to IMAGEPACK data structure. */
PLONG       plOutCount;   /*  NULL. */
PULONG      pbOutData;    /*  NULL. */
LONG        rc;           /*  Return Codes. */

rc = GreEscape(hdc, ICODE, lInCount, pbInData,
       plOutCount, pbOutData);
```

GreEscape DEVESC_EXTPUT Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_EXTPUT Parameter - ICODE

ICODE (LONG) - input
DEVESC_EXTPUT escape code.

GreEscape DEVESC_EXTPUT Parameter - lInCount

lInCount (LONG) - input
Number of bytes pointed to by pInData.

GreEscape DEVESC_EXTPUT Parameter - pbInData

pbInData (PBYTE) - input
Pointer to IMAGEPACK data structure.

GreEscape DEVESC_EXTPUT Parameter - plOutCount

plOutCount (PLONG) - input
NULL.

GreEscape DEVESC_EXTPUT Parameter - pbOutData

pbOutData (PULONG) - input
NULL.

GreEscape DEVESC_EXTPUT Return Value - rc

rc (LONG) - returns
Return Codes.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	Error

GreEscape DEVESC_EXTPUT - Parameters

hdc (HDC) - input
Device context handle.

lCODE (LONG) - input
DEVESC_EXTPUT escape code.

lInCount (LONG) - input
Number of bytes pointed to by pInData.

pbInData (PBYTE) - input
Pointer to IMAGEPACK data structure.

plOutCount (PLONG) - input
NULL.

pbOutData (PULONG) - input
NULL.

rc (LONG) - returns
Return Codes.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	Error

GreEscape DEVESC_EXTPUT - Remarks

The GreEscape DEVESC_EXTQUERY and GreEscape DEVESC_EXTGET functions are related to GreEscape DEVESC_EXTPUT.

GreEscape DEVESC_EXTPUT - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_EXTQUERY

GreEscape DEVESC_EXTQUERY - Syntax

Simulation support:

None.

Description

GreEscape DEVESC_EXTQUERY is called by EnDIVE users to get information describing the characteristics of the motion video accelerator. The function returns the capabilities and formats supported by the device.

When called, GreEscape DEVESC_EXTQUERY returns a pointer to an array of filled-in IMAGECAPS structures. The number of structures in the array is returned in the *ulENDIVEDrivers* field of the FBINFO data structure, which is returned by the GreEscape DEVESC_QUERYFB function.

The IMAGECAPS data structure contains capabilities flags and arrays of supported CODECs and color space definitions for both source and target data. For source color spaces, the COLORINFO data structure is organized in order of preference from the driver's perspective, from the most preferable to the least.

To use GreEscape DEVESC_EXTQUERY, the caller allocates enough memory for 8 CODECs, 8 source color formats and 8 target color formats. If the allocated memory is insufficient for a particular configuration, the driver returns an error code with the suggested values in the appropriate *ulNumxxxx* field in the IMAGECAPS data structure. The caller must then reallocate enough memory and call the driver again.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     ICODE;        /*  DEVESC_EXTQUERY escape code. */
LONG     lInCount;     /*  Zero (0). */
PBYTE    pbInData;     /*  NULL. */
PLONG    plOutCount;   /*  Number of bytes pointed to by pbOutData. */
PLONG    pbOutData;    /*  Pointer to IMAGECAPS data structure. */
LONG     rc;           /*  Return Codes. */

rc = GreEscape(hdc, ICODE, lInCount, pbInData,
       plOutCount, pbOutData);
```

GreEscape DEVESC_EXTQUERY Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_EXTQUERY Parameter - lCODE

lCODE (LONG) - input
DEVESC_EXTQUERY escape code.

GreEscape DEVESC_EXTQUERY Parameter - lInCount

lInCount (LONG) - input
Zero (0).

GreEscape DEVESC_EXTQUERY Parameter - pbInData

pbInData (PBYTE) - input
NULL.

GreEscape DEVESC_EXTQUERY Parameter - plOutCount

plOutCount (PLONG) - input
Number of bytes pointed to by pbOutData.

GreEscape DEVESC_EXTQUERY Parameter - pbOutData

pbOutData (PLONG) - input
Pointer to IMAGECAPS data structure.

GreEscape DEVESC_EXTQUERY Return Value - rc

rc (LONG) - returns

Return Codes.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	Error

GreEscape DEVESC_EXTQUERY - Parameters

hdc (HDC) - input
Device context handle.

ICODE (LONG) - input
DEVESC_EXTQUERY escape code.

lInCount (LONG) - input
Zero (0).

pbInData (PBYTE) - input
NULL.

plOutCount (PLONG) - input
Number of bytes pointed to by pbOutData.

pbOutData (PLONG) - input
Pointer to IMAGECAPS data structure.

rc (LONG) - returns
Return Codes.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	Error

GreEscape DEVESC_EXTQUERY - Remarks

The GreEscape DEVESC_EXTPUT and GreEscape DEVESC_EXTGET functions are related to GreEscape DEVESC_EXTQUERY.

GreEscape DEVESC_EXTQUERY - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_FLUSHOUTPUT

GreEscape DEVESC_FLUSHOUTPUT - Syntax

Simulation support:

This function is mandatory for hardcopy drivers, but may simply return a DEVESC_NOTIMPLEMENTED code. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreEscape DEVESC_FLUSHOUTPUT flushes any output received for the current document. The handling routine discards all data (such as data stored in a spooler buffer or journal file) received for the current document.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_FLUSHOUTPUT escape code. */
LONG     cInCount;     /*  The handling routine ignores this parameter. */
PBYTE    pInData;      /*  The handling routine ignores this parameter. */
PLONG    pcOutCount;   /*  The handling routine ignores this parameter. */
PLONG    pOutData;     /*  The handling routine ignores this parameter. */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreEscape. */
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_FLUSHOUTPUT Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_FLUSHOUTPUT Parameter - lEscape

lEscape (LONG) - input
DEVESC_FLUSHOUTPUT escape code.

GreEscape DEVESC_FLUSHOUTPUT Parameter - cInCount

cInCount (LONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_FLUSHOUTPUT Parameter - pInData

pInData (PBYTE) - input
The handling routine ignores this parameter.

GreEscape DEVESC_FLUSHOUTPUT Parameter - pcOutCount

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_FLUSHOUTPUT Parameter - pOutData

pOutData (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_FLUSHOUTPUT Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_FLUSHOUTPUT Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_FLUSHOUTPUT Return Value - rc

rc (LONG) - returns

Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_FLUSHOUTPUT - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_FLUSHOUTPUT escape code.

cInCount (LONG) - input
The handling routine ignores this parameter.

pInData (PBYTE) - input
The handling routine ignores this parameter.

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

pOutData (PLONG) - input
The handling routine ignores this parameter.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_FLUSHOUTPUT - Remarks

Note: This escape code is implemented by old hardcopy drivers and is being phased out.

This escape code is metafiled and recorded.

GreEscape DEVESC_FLUSHOUTPUT - Topics

Select an item:
Syntax
Parameters
Returns
Remarks

GreEscape DEVESC_GETAPERTURE

GreEscape DEVESC_GETAPERTURE - Syntax

Simulation support:

This function is mandatory for display drivers supporting software motion video. For other drivers, it is simulated by a handling routine in the graphics engine.

Description

GreEscape DEVESC_GETAPERTURE returns the size and physical address of the video aperture. For example, it would return "0A000h" and "64KB" for most Super VGA adapters. Care must be taken to determine if VGA emulation is active. If it is, a nonconflicting aperture is returned for the aperture address.

When multiple video adapters are present, this call returns the information for the adapter in use by the Presentation Manager for displaying the desktop.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lCode;        /*  DEVESC_GETAPERTURE escape code. */
LONG     lInCount;     /*  The handling routine should ignore this parameter. */
PBYTE    pbInData;     /*  The handling routine should ignore this parameter. */
PLONG    plOutCount;   /*  Number of bytes pointed to by pOutData. */
PLONG    pbOutData;
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lCode, lInCount, pbInData,
       plOutCount, pbOutData);
```

GreEscape DEVESC_GETAPERTURE Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_GETAPERTURE Parameter - lCode

lCode (LONG) - input
DEVESC_GETAPERTURE escape code.

GreEscape DEVESC_GETAPERTURE Parameter - lInCount

lInCount (LONG) - input
The handling routine should ignore this parameter.

GreEscape DEVESC_GETAPERTURE Parameter - pbInData

pbInData (PBYTE) - input
The handling routine should ignore this parameter.

GreEscape DEVESC_GETAPERTURE Parameter - plOutCount

plOutCount (PLONG) - input
Number of bytes pointed to by pOutData.

GreEscape DEVESC_GETAPERTURE Parameter - pbOutData

pbOutData (PLONG) - input
Pointer to an APERTURE structure containing:

ulPhysAddr	The physical address of the video memory aperture that the calling routine should use.
ulApertureSize	The size in bytes of the video memory aperture.
ulScanLineSize	The offset in VRAM between scan lines. This value tells the caller how many bytes of VRAM (not visible memory) are used for each scan line. For example, an 800-byte visible scan line could be mapped onto a 1024-byte physical scan line.
rctlScreen	A rectangle describing the visible region of VRAM. For example, a 640 x 480 screen would normally return an upper-left corner of (0,0) and a lower-right corner of (639,479).

GreEscape DEVESC_GETAPERTURE Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	Error

GreEscape DEVESC_GETAPERTURE - Parameters

hdc (HDC) - input
Device context handle.

lCode (LONG) - input
DEVESC_GETAPERTURE escape code.

lInCount (LONG) - input
The handling routine should ignore this parameter.

pbInData (PBYTE) - input
The handling routine should ignore this parameter.

plOutCount (PLONG) - input
Number of bytes pointed to by pOutData.

pbOutData (PLONG) - input
Pointer to an APERTURE structure containing:

ulPhysAddr	The physical address of the video memory aperture that the calling routine should use.
ulApertureSize	The size in bytes of the video memory aperture.
ulScanLineSize	The offset in VRAM between scan lines. This value tells the caller how many bytes of VRAM (not visible memory) are used for each scan line. For example, an 800-byte visible scan line could be mapped onto a 1024-byte physical scan line.
rctlScreen	A rectangle describing the visible region of VRAM. For example, a 640 x 480 screen would normally return an upper-left corner of (0,0) and a lower-right corner of (639,479).

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	Error

GreEscape DEVESC_GETAPERTURE - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreEscape DEVESC_GETCP

GreEscape DEVESC_GETCP - Syntax

Simulation support:

This function is mandatory for hardcopy drivers, but may simply return a DEVESC_NOTIMPLEMENTED code. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreEscape DEVESC_GETCP gets a hardcopy data stream that would set the specified code page in the hardcopy device.

Note: This escape code is used only by hardcopy drivers with built-in fonts.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_GETCP escape code. */
LONG     cInCount;     /*  The handling routine ignores this parameter. */
PBYTE    pInData;      /*  The handling routine ignores this parameter. */
PLONG    pcOutCount;   /*  Pointer to the number of bytes addressed by pOutData. */
PLONG    pOutData;     /*  Pointer to the buffer in which the handling routine returns the required data. */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreEscape. */
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_GETCP Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_GETCP Parameter - lEscape

lEscape (LONG) - input
DEVESC_GETCP escape code.

GreEscape DEVESC_GETCP Parameter - cInCount

cInCount (LONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_GETCP Parameter - pInData

pInData (PBYTE) - input
The handling routine ignores this parameter.

GreEscape DEVESC_GETCP Parameter - pcOutCount

pcOutCount (PLONG) - input
Pointer to the number of bytes addressed by pOutData.

The handling routine should update this value to show the number of bytes returned in the buffer.

GreEscape DEVESC_GETCP Parameter - pOutData

pOutData (PLONG) - input
Pointer to the buffer in which the handling routine returns the required data.

GreEscape DEVESC_GETCP Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_GETCP Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_GETCP Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_GETCP - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_GETCP escape code.

cInCount (LONG) - input
The handling routine ignores this parameter.

pInData (PBYTE) - input
The handling routine ignores this parameter.

pcOutCount (PLONG) - input
Pointer to the number of bytes addressed by pOutData.

The handling routine should update this value to show the number of bytes returned in the buffer.

pOutData (PLONG) - input
Pointer to the buffer in which the handling routine returns the required data.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_GETCP - Remarks

The data stream should provide for options, such as "draft" and "NLQ," as defined in the OS2_PM_DRV_DEVMODE dialog. The handling routine in the presentation driver responds by writing the data stream in the buffer addressed by pOutData and the count of bytes into the LONG value addressed by pcOutCount.

This escape code is not metafiled or recorded.

GreEscape DEVESC_GETCP - Topics

Select an item:

Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_GETJOBID

GreEscape DEVESC_GETJOBID - Syntax

Simulation support:

This function is optional for hardcopy drivers.

Description:

GreEscape DEVESC_GETJOBID (8160L) is called by an application to get the spooler job ID, print queue name, and print server name for the job the application is spooling.

This escape is not metafiled and is allowed only when printing OD_QUEUED.

```
#define INCL_GRE_DEVICE
#include <os2.h>

LONG     lInCount;     /*  Not used; must be set to 0 (zero). */
PBYTE    pbInData;     /*  Not used; must be set to NULL. */
PLONG    plOutCount;   /*  The number of bytes pointed to or used by pbOutData. */
PLONG    pbOutData;    /*  Buffer to receive QMJOBINFO data structure. */
LONG     rc;           /*  Return Code. */

rc = GreEscape(lInCount, pbInData, plOutCount,
       pbOutData);
```

GreEscape DEVESC_GETJOBID Parameter - lInCount

lInCount (LONG) - input
Not used; must be set to 0 (zero).

GreEscape DEVESC_GETJOBID Parameter - pbInData

pbInData (PBYTE) - input
Not used; must be set to NULL.

GreEscape DEVESC_GETJOBID Parameter - plOutCount

plOutCount (PLONG) - output
The number of bytes pointed to or used by *pbOutData* .

There are two cases with this parameter.

Case 1:

INPUT: *plOutCount* points to a LONG (which holds the size needed) and *pbOutData* is NULL.

OUTPUT: *plOutCount* contains the size needed.

Case 2:

INPUT: *plOutCount* points to a LONG (which has the size allocated) and *pbOutData* points to the block of memory.

OUTPUT: *pbOutData* is filled in with the QMJOBINFO structure.

GreEscape DEVESC_GETJOBID Parameter - pbOutData

pbOutData (PLONG) - output
Buffer to receive QMJOBINFO data structure.

See *plOutCount* for additional details.

GreEscape DEVESC_GETJOBID Return Value - rc

rc (LONG) - returns
Return Code.

0	Success
6(ERROR_INVALID_HANDLE)	Invalid handle given.
87(ERROR_INVALID_PARAMETER)	Invalid parameter given.
234(ERROR_MORE_DATA)	Part of the information requested was returned, but there was not enough room for all the data. **plOutCount* contains the size of buffer needed to retrieve all the query response data.
2123(NERR_BufTooSmall)	No information was returned because the output buffer was too small. **plOutCount* contains the size of buffer needed to retrieve all the query response data.
2151(NERR_JobNotFound)	No spooler job found.

GreEscape DEVESC_GETJOBID - Parameters

llnCount (LONG) - input
Not used; must be set to 0 (zero).

pbInData (PBYTE) - input
Not used; must be set to NULL.

plOutCount (PLONG) - output
The number of bytes pointed to or used by *pbOutData* .

There are two cases with this parameter.

Case 1:

INPUT: *plOutCount* points to a LONG (which holds the size needed) and *pbOutData* is NULL.

OUTPUT: *plOutCount* contains the size needed.

Case 2:

INPUT: *plOutCount* points to a LONG (which has the size allocated) and *pbOutData* points to the block of memory.

OUTPUT: *pbOutData* is filled in with the QMJOBINFO structure.

pbOutData (PLONG) - output
Buffer to receive QMJOBINFO data structure.

See *plOutCount* for additional details.

rc (LONG) - returns
Return Code.

0	Success
6(ERROR_INVALID_HANDLE)	Invalid handle given.
87(ERROR_INVALID_PARAMETER)	Invalid parameter given.
234(ERROR_MORE_DATA)	Part of the information requested was returned, but there was not enough room for all the data. **plOutCount* contains the size of buffer needed to retrieve all the query response data.
2123(NERR_BufTooSmall)	No information was returned because the output buffer was too small. **plOutCount* contains the size of buffer needed to retrieve all the query response data.
2151(NERR_JobNotFound)	No spooler job found.

GreEscape DEVESC_GETJOBID - Remarks

The following function is related to GreEscape DEVESC_GETJOBID(8160L): GreEscape DEVESC_ENDDOC.

GreEscape DEVESC_GETJOBID - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_GETSCALINGFACTOR

GreEscape DEVESC_GETSCALINGFACTOR - Syntax

Simulation support:

This function is mandatory for hardcopy drivers, but may simply return a DEVESC_NOTIMPLEMENTED code. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreEscape DEVESC_GETSCALINGFACTOR gets the incremental scaling factors for the (X,Y) axes of a hardcopy device.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_GETSCALINGFACTOR escape code. */
LONG     cInCount;     /*  The handling routine ignores this parameter. */
PBYTE    pInData;      /*  The handling routine ignores this parameter. */
PLONG    pcOutCount;   /*  Number of bytes pointed to by pOutData. */
PLONG    pOutData;     /*  Pointer to SFACTORS data structure: */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreEscape. */
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_GETSCALINGFACTOR Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_GETSCALINGFACTOR Parameter - lEscape

lEscape (LONG) - input
DEVESC_GETSCALINGFACTOR escape code.

GreEscape DEVESC_GETSCALINGFACTOR Parameter - cInCoun

clnCount (LONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_GETSCALINGFACTOR Parameter - plnData

plnData (PBYTE) - input
The handling routine ignores this parameter.

GreEscape DEVESC_GETSCALINGFACTOR Parameter - pcOutC

pcOutCount (PLONG) - input
Number of bytes pointed to by pOutData.

On return, this is updated by the handling routine to the number of bytes actually returned.

GreEscape DEVESC_GETSCALINGFACTOR Parameter - pOutDat

pOutData (PLONG) - input
Pointer to SFACTORS data structure:

lXscale	X-scaling factor, as an exponent of 2
lYscale	Y-scaling factor, as an exponent of 2

GreEscape DEVESC_GETSCALINGFACTOR Parameter - pInstanc

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_GETSCALINGFACTOR Parameter - lFunctio

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_GETSCALINGFACTOR Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_GETSCALINGFACTOR - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_GETSCALINGFACTOR escape code.

cInCount (LONG) - input
The handling routine ignores this parameter.

pInData (PBYTE) - input
The handling routine ignores this parameter.

pcOutCount (PLONG) - input
Number of bytes pointed to by pOutData.

On return, this is updated by the handling routine to the number of bytes actually returned.

pOutData (PLONG) - input
Pointer to SFACTORS data structure:

lXscale	X-scaling factor, as an exponent of 2
lYscale	Y-scaling factor, as an exponent of 2

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_GETSCALINGFACTOR - Remarks

The handling routine returns the scaling factors as an SFACTORS structure at the location addressed by pOutData, and changes the value addressed by pcOutCount to show the number of bytes in the returned structure.

This escape code is not metafiled or recorded.

Scaling factors are used for physical devices where one unit on the X axis is not equal to one unit on the Y axis. The factors show an arbitrary unit length expressed in X units and Y units. The length is chosen so that the number of X units and Y units can be expressed as an exponent of 2 and the exponents are returned in the SFACTORS structure. For example, if there are eight units of X in the arbitrary unit length, lXscale

is set to 3.

Note: For an OD_QUEUED device with PM_Q_STD data, the spooler passes this escape to the presentation driver without recording it.

GreEscape DEVESC_GETSCALINGFACTOR - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_HWREQUEST

GreEscape DEVESC_HWREQUEST - Syntax

Simulation support:

None.

Description

GreEscape DEVESC_HWREQUEST is called by EnDIVE users to take advantage of enhanced aquire functions. In addition to performing the same serialization functions, GreEscape DEVESC_HWREQUEST lets each using process find out if its video memory buffer has been relocated or if system memory could not be allocated.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     ICODE;        /*  DEVESC_HWREQUEST escape code. */
LONG     lInCount;     /*  Number of bytes pointed to by pbInData. */
PBYTE    pbInData;     /*  Pointer to VRAMREQUESTIN data structure. */
PLONG    plOutCount;   /*  Number of bytes pointed to by pbOutData. */
PLONG    pbOutData;    /*  Pointer to VRAMREQUESTOUT data structure. */
LONG     rc;           /*  Return Codes. */

rc = GreEscape(hdc, ICODE, lInCount, pbInData,
       plOutCount, pbOutData);
```

GreEscape DEVESC_HWREQUEST Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_HWREQUEST Parameter - lCODE

lCODE (LONG) - input
DEVESC_HWREQUEST escape code.

GreEscape DEVESC_HWREQUEST Parameter - lInCount

lInCount (LONG) - input
Number of bytes pointed to by pbInData.

GreEscape DEVESC_HWREQUEST Parameter - pbInData

pbInData (PBYTE) - input
Pointer to VRAMREQUESTIN data structure.

GreEscape DEVESC_HWREQUEST Parameter - plOutCount

plOutCount (PLONG) - input
Number of bytes pointed to by pbOutData.

GreEscape DEVESC_HWREQUEST Parameter - pbOutData

pbOutData (PLONG) - input
Pointer to VRAMREQUESTOUT data structure.

GreEscape DEVESC_HWREQUEST Return Value - rc

rc (LONG) - returns
Return Codes.

The handling routine returns:

DEV_OK
Successful

DEVESC_NOTIMPLEMENTED
Escape is not implemented for the specified code

DEVESC_ERROR
Error

WINGETLASTERROR can be called for specific error information.

GreEscape DEVESC_HWREQUEST - Parameters

hdc (HDC) - input
Device context handle.

lCODE (LONG) - input
DEVESC_HWREQUEST escape code.

lInCount (LONG) - input
Number of bytes pointed to by pbInData.

pbInData (PBYTE) - input
Pointer to VRAMREQUESTIN data structure.

plOutCount (PLONG) - input
Number of bytes pointed to by pbOutData.

pbOutData (PLONG) - input
Pointer to VRAMREQUESTOUT data structure.

rc (LONG) - returns
Return Codes.

The handling routine returns:

DEV_OK
Successful

DEVESC_NOTIMPLEMENTED
Escape is not implemented for the specified code

DEVESC_ERROR
Error

WINGETLASTERROR can be called for specific error information.

GreEscape DEVESC_HWREQUEST - Remarks

None.

GreEscape DEVESC_HWREQUEST - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_NEWFRAME

GreEscape DEVESC_NEWFRAME - Syntax

Simulation support:

This function is mandatory for hardcopy drivers and must be implemented in order to have a valid OS/2 hardcopy driver. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreEscape DEVESC_NEWFRAME indicates that there is no more data for the current page.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_NEWFRAME escape code. */
LONG     cInCount;     /*  The handling routine ignores this parameter. */
PBYTE    pInData;      /*  The handling routine ignores this parameter. */
PLONG    pcOutCount;   /*  The handling routine ignores this parameter. */
PLONG    pOutData;     /*  The handling routine ignores this parameter. */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreEscape. */
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_NEWFRAME Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_NEWFRAME Parameter - lEscape

lEscape (LONG) - input
DEVESC_NEWFRAME escape code.

GreEscape DEVESC_NEWFRAME Parameter - cInCount

cInCount (LONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_NEWFRAME Parameter - pInData

pInData (PBYTE) - input
The handling routine ignores this parameter.

GreEscape DEVESC_NEWFRAME Parameter - pcOutCount

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_NEWFRAME Parameter - pOutData

pOutData (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_NEWFRAME Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_NEWFRAME Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_NEWFRAME Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code

DEVESC_ERROR	Error

GreEscape DEVESC_NEWFRAME - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_NEWFRAME escape code.

cInCount (LONG) - input
The handling routine ignores this parameter.

pInData (PBYTE) - input
The handling routine ignores this parameter.

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

pOutData (PLONG) - input
The handling routine ignores this parameter.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_NEWFRAME - Remarks

For a display device, DEVESC_NEWFRAME is similar to GreErasePS. However, the handling routine should reset the attributes (color and mix). For hardcopy devices, the handling routine would advance the paper to a new page.

Note: For an OD_QUEUED device with PM_Q_STD data, the spooler records this escape in the buffer, but does not pass it on to the presentation driver.

The following sequence implies that the previous page is completed and ejected, and then a blank page is ejected, even if no other functions are called between the two GreEscapes.

```
GreEscape(DEVESC_NEWFRAME)
GreEscape(DEVESC_NEWFRAME)
```

This escape code is metafiled and recorded.

GreEscape DEVESC_NEWFRAME - Topics

Select an item:

Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_NEWFRAME_WPROP

GreEscape DEVESC_NEWFRAME_WPROP - Syntax

Simulation support:

This function is mandatory for hardcopy drivers and must be implemented in order to have a valid OS/2 hardcopy driver.

Description:

Based on the new job properties that are passed in, *GreEscape DEVESC_NEWFRAME_WPROP* , will reset the device context information and presentation space information. This action should be equivalent to the result of a new DevOpenDC with the new job properties. Refer to Dynamic Job Properties for related information.

```
#define INCL_DEV
#define INCL_DEVDJP
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_NEWFRAME_WPROP escape code. */
LONG     cInCount;     /*  Not used-should be 0. */
PBYTE    pInData;      /*  Not used-should be NULL. */
PLONG    pcOutCount;   /*  A pointer to the number of bytes pointed to by pOutData. */
PLONG    pOutData;     /*  Pointer to the printer driver's job properties */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreEscape. */
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_NEWFRAME_WPROP Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_NEWFRAME_WPROP Parameter - lEscape

lEscape (LONG) - input
DEVESC_NEWFRAME_WPROP escape code.

GreEscape DEVESC_NEWFRAME_WPROP Parameter - cInCount

cInCount (LONG) - input
Not used-should be 0.

GreEscape DEVESC_NEWFRAME_WPROP Parameter - pInData

pInData (PBYTE) - input
Not used-should be NULL.

GreEscape DEVESC_NEWFRAME_WPROP Parameter - pcOutCo

pcOutCount (PLONG) - input
A pointer to the number of bytes pointed to by *pOutData*.

GreEscape DEVESC_NEWFRAME_WPROP Parameter - pOutData

pOutData (PLONG) - input
Pointer to the printer driver's job properties

as returned by SplQueryQueue. These job properties may possibly have been modified by GreEscape DEVESC_SETJOBPROPERTIES.

Note: *pOutData* is used as an input parameter for this call.

GreEscape DEVESC_NEWFRAME_WPROP Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_NEWFRAME_WPROP Parameter - lFunction

lFunction (ULONG) - input

High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_NEWFRAME_WPROP Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_ERROR	Error

GreEscape DEVESC_NEWFRAME_WPROP - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_NEWFRAME_WPROP escape code.

cInCount (LONG) - input
Not used-should be 0.

pInData (PBYTE) - input
Not used-should be NULL.

pcOutCount (PLONG) - input
A pointer to the number of bytes pointed to by *pOutData*.

pOutData (PLONG) - input
Pointer to the printer driver's job properties

as returned by SplQueryQueue. These job properties may possibly have been modified by GreEscape DEVESC_SETJOBPROPERTIES.

Note: *pOutData* is used as an input parameter for this call.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_ERROR	Error

GreEscape DEVESC_NEWFRAME_WPROP - Remarks

GreEscape DEVESC_NEWFRAME_WPROP is used to change the job properties at the beginning of a new page. Since this function is *new* , applications will call GreEscape DEVESC_QUERYESCSUPPORT to determine if you support this DevEscape.

The DevEscape path has a handle to a device context (HDC) that should already have printer property information associated with the HDC. Job property information may change or may be dependent upon the printer property information in the HDC. In that case, if an application passes job properties for a different device or for a different spooler printer name, then misleading information will be returned.

This function will change all characteristics of the HDC and PS. An application *must* query again information such as DevQueryCaps, DevQueryHardcopyCaps, the list of device fonts, and so on. In addition, all previously opened memory DCs are no longer considered compatible with the DC.

You also should check the version level of the graphics engine, since GreResetDC2 is a new API and only exists on engine levels 0x0234 and above. If you are using an older version level, then treat DEVESC_NEWFRAME_WPROP as a DEVESC_NEWFRAME.

This function also performs the same actions as calling DEVESC_NEWFRAME.

```
LONG ENGENTRY Escape (HDC       hdc,
                      LONG      lEscape,
                      LONG      cInCount,
                      PBYTE     pInData,
                      PLONG     pcOutCount,
                      PBYTE     pOutData,
                      PDDC      pddc,
                      ULONG     ulFunction)
{
   switch (lEscape)
   {
   case DEVESC_NEWFRAME:
   case DEVESC_NEWFRAME_WPROP:
   {
      if (GRE_234 > globals.ulGreVersion)
      {
         /* Switch DEVESC_NEWFRAME_WPROP to DEVESC_NEWFRAME
         ** on back level versions of the engine.
         */
         lEscape = DEVESC_NEWFRAME;
         break;
      }
      if (DEVESC_NEWFRAME_WPROP == lEscape)
      {
         /* This does a number of things such as:
         **   1) Sanity check input driver data
         **   2) Walk through new job properties and set up
         **      DC information (papersize, orientation, color depth, etc)
         **   3) Set up device surface structure
         */
         ReinitializeWithJobProps (pddc, (PDRIVDATA)pOutData);

         /* This does a number of things such as:
         **   1) Allocate the device surface memory
         **   2) Call engine's SetDeviceSurface function.
         */
         CreateAndSetDeviceSurface (pddc);
         /* Inform the engine that your characteristics have changed
         ** and it should requery the information.
         */
         GreResetDC2 (pddc->pdb->hdc, 0);
      }

      // Normal NEWFRAME processing...
      break;
   }
   }

   return lrc;
}
```

GreEscape DEVESC_NEWFRAME_WPROP - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_NEXTBAND

GreEscape DEVESC_NEXTBAND - Syntax

Simulation support:

This function is mandatory for hardcopy drivers, but may simply return a DEVESC_NOTIMPLEMENTED code. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreEscape DEVESC_NEXTBAND indicates that there is no more data for the current band, and gets the coordinates of the next band. If there is no current band, the handling routine returns the coordinates of the first band.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_NEXTBAND escape code. */
LONG     cInCount;     /*  The handling routine ignores this parameter. */
PBYTE    pInData;      /*  The handling routine ignores this parameter. */
PLONG    pcOutCount;   /*  Number of bytes of data pointed to by pOutData. */
PLONG    pOutData;     /*  Pointer to BANDRECT structure. */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreEscape. */
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_NEXTBAND Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_NEXTBAND Parameter - lEscape

lEscape (LONG) - input

DEVESC_NEXTBAND escape code.

GreEscape DEVESC_NEXTBAND Parameter - cInCount

cInCount (LONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_NEXTBAND Parameter - pInData

pInData (PBYTE) - input
The handling routine ignores this parameter.

GreEscape DEVESC_NEXTBAND Parameter - pcOutCount

pcOutCount (PLONG) - input
Number of bytes of data pointed to by pOutData.

On return, this is updated to the number of bytes actually returned.

GreEscape DEVESC_NEXTBAND Parameter - pOutData

pOutData (PLONG) - input
Pointer to BANDRECT structure.

Points to where the device coordinates of the next band are returned:

xLeft	X-coordinate of the lower-left corner of the rectangular band
yBottom	Y-coordinate of the lower-left corner of the rectangular band
xRight	X-coordinate of the top-right corner of the rectangular band
yTop	Y-coordinate of the top-right corner of the rectangular band

An empty rectangle (xLeft=xRight or yTop=yBottom) marks the end of the banding operation.

GreEscape DEVESC_NEXTBAND Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_NEXTBAND Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_NEXTBAND Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_NEXTBAND - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_NEXTBAND escape code.

cInCount (LONG) - input
The handling routine ignores this parameter.

pInData (PBYTE) - input
The handling routine ignores this parameter.

pcOutCount (PLONG) - input
Number of bytes of data pointed to by pOutData.

On return, this is updated to the number of bytes actually returned.

pOutData (PLONG) - input
Pointer to BANDRECT structure.

Points to where the device coordinates of the next band are returned:

xLeft	X-coordinate of the lower-left corner of the rectangular band
yBottom	Y-coordinate of the lower-left corner of the rectangular band
xRight	X-coordinate of the top-right corner of the rectangular band
yTop	Y-coordinate of the top-right corner of the rectangular band

An empty rectangle (xLeft=xRight or yTop=yBottom) marks the end of the banding operation.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_NEXTBAND - Remarks

DEVESC_NEXTBAND is used by programs that do their own banding. It is not necessary to support this escape code in hardcopy drivers for devices that cannot use banded output. See Banding. Notice that DEVESC_NEWFRAME is issued to start a new page.

Note: This escape code is implemented by old hardcopy drivers that support banding and is being phased out.

This escape code is metafiled but not recorded.

GreEscape DEVESC_NEXTBAND - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_QUERYESCSUPPORT

GreEscape DEVESC_QUERYESCSUPPORT - Syntax

Simulation support:

This function is mandatory for all drivers and must be implemented in order to have a valid OS/2 hardcopy driver.

Description:

GreEscape DEVESC_QUERYESCSUPPORT queries whether a particular escape code is implemented by the presentation driver. The presentation driver returns DEV_OK if the specified escape is supported.

Refer to Dynamic Job Properties for related information

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_QUERYESCSUPPORT escape code. */
LONG     cInCount;     /*  Number of bytes pointed to by pInData. */
PBYTE    pInData;      /*  Pointer to an escape code value specifying the escape function to be checked. */
PLONG    pcOutCount;   /*  The handling routine ignores this parameter. */
PLONG    pOutData;     /*  The handling routine ignores this parameter. */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreEscape. */
```

```
LONG     rc;          /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_QUERYESCSUPPORT Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_QUERYESCSUPPORT Parameter - lEscape

lEscape (LONG) - input
DEVESC_QUERYESCSUPPORT escape code.

GreEscape DEVESC_QUERYESCSUPPORT Parameter - cInCoun

cInCount (LONG) - input
Number of bytes pointed to by pInData.

GreEscape DEVESC_QUERYESCSUPPORT Parameter - pInData

pInData (PBYTE) - input
Pointer to an escape code value specifying the escape function to be checked.

GreEscape DEVESC_QUERYESCSUPPORT Parameter - pcOutCo

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_QUERYESCSUPPORT Parameter - pOutDat

pOutData (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_QUERYESCSUPPORT Parameter - pInstanc

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_QUERYESCSUPPORT Parameter - lFunctior

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_QUERYESCSUPPORT Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_QUERYESCSUPPORT - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_QUERYESCSUPPORT escape code.

cInCount (LONG) - input
Number of bytes pointed to by pInData.

pInData (PBYTE) - input
Pointer to an escape code value specifying the escape function to be checked.

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

pOutData (PLONG) - input
The handling routine ignores this parameter.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_QUERYESCSUPPORT - Remarks

Note: For an OD_QUEUED device with PM_Q_STD data, the spooler passes this escape on to the presentation driver without recording it.

This escape code is not metafiled or recorded.

GreEscape DEVESC_QUERYESCSUPPORT - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_QUERYFB

GreEscape DEVESC_QUERYFB - Syntax

Simulation support:

None.

Description

GreEscape DEVESC_QUERYFB is called by EnDIVE users to get information describing the characteristics of the frame buffer.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     ICODE;        /*  DEVESC_QUERYFB escape code. */
LONG     lInCount;     /*  Zero (0). */
PBYTE    pbInData;     /*  NULL. */
PLONG    plOutCount;   /*  Number of bytes pointed to by pbOutData. */
PLONG    pbOutData;    /*  Pointer to FBINFO data structure. */
```

```
LONG     rc;          /*  Return Codes. */

rc = GreEscape(hdc, ICODE, lInCount, pbInData,
       plOutCount, pbOutData);
```

GreEscape DEVESC_QUERYFB Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_QUERYFB Parameter - ICODE

ICODE (LONG) - input
DEVESC_QUERYFB escape code.

GreEscape DEVESC_QUERYFB Parameter - lInCount

lInCount (LONG) - input
Zero (0).

GreEscape DEVESC_QUERYFB Parameter - pbInData

pbInData (PBYTE) - input
NULL.

GreEscape DEVESC_QUERYFB Parameter - plOutCount

plOutCount (PLONG) - input
Number of bytes pointed to by pbOutData.

GreEscape DEVESC_QUERYFB Parameter - pbOutData

pbOutData (PLONG) - input
Pointer to FBINFO data structure.

GreEscape DEVESC_QUERYFB Return Value - rc

rc (LONG) - returns
Return Codes.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	Error

WINGETLASTERROR can be called for specific error information.

GreEscape DEVESC_QUERYFB - Parameters

hdc (HDC) - input
Device context handle.

lCODE (LONG) - input
DEVESC_QUERYFB escape code.

lInCount (LONG) - input
Zero (0).

pbInData (PBYTE) - input
NULL.

plOutCount (PLONG) - input
Number of bytes pointed to by pbOutData.

pbOutData (PLONG) - input
Pointer to FBINFO data structure.

rc (LONG) - returns
Return Codes.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	Error

WINGETLASTERROR can be called for specific error information.

GreEscape DEVESC_QUERYFB - Remarks

None.

GreEscape DEVESC_QUERYFB - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_QUERYJOBPROPERTIES

GreEscape DEVESC_QUERYJOBPROPERTIES - Syntax

Simulation support:

This function is mandatory for all drivers and must be implemented in order to have a valid OS/2 hardcopy driver. Refer to Dynamic Job Properties for related information.

Description:

GreEscape DEVESC_QUERYJOBPROPERTIES queries the specified job property types and values inside the job properties structure that is passed in. Refer to Dynamic Job Properties for related information.

```
#define INCL_DEV
#define INCL_DEVDJP
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_QUERYJOBPROPERTIES escape code. */
LONG     cInCount;     /*  Number of bytes pointed to by pInData. */
PBYTE    pInData;      /*  Pointer to a list of DJP_ITEM data structures. */
PLONG    pcOutCount;   /*  Pointer to the number of bytes pointed to by pOutData. */
PLONG    pOutData;     /*  Pointer to the printer driver's job properties structure. */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreEscape. */
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_QUERYJOBPROPERTIES Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_QUERYJOBPROPERTIES Parameter - lEsca

lEscape (LONG) - input
DEVESC_QUERYJOBPROPERTIES escape code.

GreEscape DEVESC_QUERYJOBPROPERTIES Parameter - cInC

cInCount (LONG) - input
Number of bytes pointed to by *pInData*.

GreEscape DEVESC_QUERYJOBPROPERTIES Parameter - pInD

pInData (PBYTE) - output
Pointer to a list of DJP_ITEM data structures.

The end of the list is indicated by a DJP_ITEM structure that has DJP_NONE as its *ulProperty* value. On input, the DJP_ITEM structures should be sized as *simple* (that is, == sizeof (DJP_ITEM)). On output, if lType was DJP_ALL, then the DJP_ITEM size will grow and its new size will be reflected in the *cb* field.

GreEscape DEVESC_QUERYJOBPROPERTIES Parameter - pcOu

pcOutCount (PLONG) - input
Pointer to the number of bytes pointed to by *pOutData*.

GreEscape DEVESC_QUERYJOBPROPERTIES Parameter - pOut

pOutData (PLONG) - input
Pointer to the printer driver's job properties structure.

Note: *pOutData* is used as an input parameter for this call.

GreEscape DEVESC_QUERYJOBPROPERTIES Parameter - pInst

pInstance (PVOID) - input

Pointer to instance data.

GreEscape DEVESC_QUERYJOBPROPERTIES Parameter - lFunc

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_QUERYJOBPROPERTIES Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK
Successful.

DEV_PROP_BUF_TOO_SMALL
The job properties buffer was too small to contain the proper job properties data or expanded DJP_ITEMs. The buffer will not be updated.

DEV_INV_INP_JOBPROPERTIES
Invalid job properties were passed in. The driver changed them to default values. This is an error and the item buffer will not be updated.

DEV_WARNING
One or more of the DJP_ITEMs has an error in the *lType* field.

DEVESC_ERROR
Error

GreEscape DEVESC_QUERYJOBPROPERTIES - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_QUERYJOBPROPERTIES escape code.

cInCount (LONG) - input
Number of bytes pointed to by *pInData*.

pInData (PBYTE) - output
Pointer to a list of DJP_ITEM data structures.

The end of the list is indicated by a DJP_ITEM structure that has DJP_NONE as its *ulProperty* value. On input, the DJP_ITEM structures should be sized as *simple* (that is, == sizeof (DJP_ITEM)). On output, if lType was DJP_ALL, then the DJP_ITEM size will grow and its new size will be reflected in the *cb* field.

pcOutCount (PLONG) - input
Pointer to the number of bytes pointed to by *pOutData*.

pOutData (PLONG) - input
Pointer to the printer driver's job properties structure.

Note: *pOutData* is used as an input parameter for this call.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK
Successful.

DEV_PROP_BUF_TOO_SMALL
The job properties buffer was too small to contain the proper job properties data or expanded DJP_ITEMs. The buffer will not be updated.

DEV_INV_INP_JOBPROPERTIES
Invalid job properties were passed in. The driver changed them to default values. This is an error and the item buffer will not be updated.

DEV_WARNING
One or more of the DJP_ITEMs has an error in the *IType* field.

DEVESC_ERROR
Error

GreEscape DEVESC_QUERYJOBPROPERTIES - Remarks

GreEscape DEVESC_QUERYJOBPROPERTIES queries the information that is within the printer's job properties structure. This structure is known only to the printer presentation driver and this escape is the only way for an application to look inside the structure.

Since DEVESC_QUERYJOBPROPERTIES is *new* , applications will call GreEscape DEVESC_QUERYESCSUPPORT to determine if you support this DevEscape.

Note that the DevEscape path has a handle to a device context (HDC) that should already have printer property information associated with the HDC. Job property information may change or may be dependent upon the printer property information in the HDC. In that case, if an application passes job properties for a different device or for a different spooler printer name, then misleading information will be returned.

Remember to test the input job properties to make sure that they are valid. This includes passing tests such as whether they contain your driver's signature string, whether the device name is yours, and whether values are within the proper ranges (only landscape or portrait). If these tests fail, then fail the call.

```
LONG ENGENTRY Escape (HDC       hdc,
                      LONG      lEscape,
                      LONG      cInCount,
                      PBYTE     pInData,
                      PLONG     pcOutCount,
                      PBYTE     pOutData,
                      PDDC      pddc,
                      ULONG     ulFunction)
{
   switch (lEscape)
   {
   case DEVESC_QUERYJOBPROPERTIES:
   {
      INT            cbQueries    = cInCount;
      PBYTE          pbQueries    = pInData;
      PDJP_ITEM      pIQuery      = (PDJP_ITEM)pbQueries;
      BOOL           fError       = FALSE;
      PBYTE          pbNext;
      INT            iNumQueries;
      PQUERYTUPLE    pTuples      = NULL,
                     pTupleCur;
```

```
if (GRE_234 > globals.ulGreVersion)
{
   assertstring ("Not supported on pre-DAX engine!\n");
   lrc = DEVESC_NOTIMPLEMENTED;
   break;
}

pGoodDrivData = ReturnDriverData (pddc->pdb,
                                  pdi,
                                  pddc->pdb->hmcbHeap,
                                  pDrivData,
                                  pddc->pdb->pszPrinterName,
                                  TRUE,
                                  pDevice,
                                  pDriver);
assertF (pGoodDrivData);

/* Is the size of the input driver data is not the same as
** the size of our good data?
*/
if (pDrivData->cb != pGoodDrivData->cb)
   lrc = DEV_PROP_BUF_TOO_SMALL;

/* Or, if the two don't compare, then fail the call.
*/
if (0 != memcmp (pDrivData, pGoodDrivData, pGoodDrivData->cb))
   lrc = DEV_INV_INP_JOBPROPERTIES;

// Output buffer must be at least one item in size
if (sizeof (DJP_ITEM) > cbQueries)
{
   ulrc = DEV_BAD_PARAMETERS;
   break;
}

// Step 1.  Count the number of queries
iNumQueries = 0;
pIQuery     = (PDJP_ITEM)pbQueries;
while (pIQuery->ulProperty != DJP_NONE)
{
   iNumQueries++;
   pIQuery = DJP_NEXT_STRUCTP (pIQuery);
}
iNumQueries++;   // Add an EOL marker

// Step 2.  Allocate and copy the query types
pTuples = (PQUERYTUPLE)GplMemoryAlloc (globals.pvSharedHeap,
                                       iNumQueries * sizeof (QUERYTUPLE));
assertF (pTuples);
if (!pTuples)
{
   ulrc = DEV_ERROR;
   GplErrSetError (PMERR_INSUFFICIENT_MEMORY);
   break;
}

/* Since we will be overwriting the input list to the output list, we
** need to copy the original input list into an array of tuples
** (the only thing that matters)
*/
pIQuery   = (PDJP_ITEM)pbQueries;
pTupleCur = pTuples;
while (pIQuery->ulProperty != DJP_NONE)
{
   pTupleCur->ulProperty = pIQuery->ulProperty;
   pTupleCur->lType      = pIQuery->lType;

   pIQuery = DJP_NEXT_STRUCTP (pIQuery);
   pTupleCur++;
}
// And copy the EOL
pTupleCur->ulProperty = DJP_NONE;
pTupleCur->lType      = DJP_NONE;

// Step 3.  Fill in the output list
pIQuery   = (PDJP_ITEM)pbQueries;
pTupleCur = pTuples;
while (0 < iNumQueries)
{
   BOOL  fCurError = FALSE;

   pIQuery->ulProperty    = pTupleCur->ulProperty;
   pIQuery->lType         = pTupleCur->lType;
```

```
pIQuery->ulNumReturned = 0;

if (sizeof (DJP_ITEM) > cbQueries)
{
   fError = DEV_PROP_BUF_TOO_SMALL;
   assertstring ("Not enough space left!\n");
   break;
}

DBPRINTF (("Query '%s'/%d.  Query type '%s'/%d\n",
           pszProperty (pTupleCur->ulProperty),
           pTupleCur->ulProperty,
           pszType (pTupleCur->lType),
           pTupleCur->lType));

if (pTupleCur->lType == DJP_CURRENT)
{
   pIQuery->ulNumReturned = 1;

   // Assume a simple type... complex types will override it
   pbNext = (PBYTE)(pIQuery + 1);

   switch (pTupleCur->ulProperty)
   {
   case DJP_SJ_ORIENTATION:
      if (ORIENTATION_PORTRAIT == pJobProp->ulOrientation)
         pIQuery->ulValue = DJP_ORI_PORTRAIT;
      else
         pIQuery->ulValue = DJP_ORI_LANDSCAPE;
      break;

   case DJP_CJ_RESOLUTION:
   {
      PDJPT_RESOLUTION   pRes   = DJP_ELEMENTP (*pIQuery, DJPT_RESOLUTION);
      PRESINFO           pResInfo;
      DJPT_RESOLUTION    Res;
      LONG               lRet;

      pResInfo = GetpResFromResID (pDriver, pJobProp->ulDefResID);

      if (pResInfo)
      {
         Res.usXResolution = pResInfo->ulXRes;
         Res.usYResolution = pResInfo->ulYRes;
         *pRes++ = Res;

         pbNext = (PBYTE)pRes;
      }
      else
      {
         fCurError = TRUE;
      }
      break;
   }

   case DJP_SJ_BITSPERPEL:
   case DJP_SJ_COLOR:
   {
      PPRINTMODE pPrintMode;

      pPrintMode = GetpPrintModeFromID (pDriver,
                                        pJobProp->ulDefPrintModeID);

      if (pPrintMode)
      {
         if (DJP_SJ_BITSPERPEL == pTupleCur->ulProperty)
         {
            pIQuery->ulValue = pPrintMode->usLogBitCount;
         }
         else
         {
            if (1 == pPrintMode->usBitsPerPel)
               pIQuery->ulValue = DJP_CLR_MONOCHROME;
            else
               pIQuery->ulValue = DJP_CLR_COLOR;
         }
      }
      else
      {
         fCurError = TRUE;
      }
      break;
   }
```

```
   case DJP_CJ_FORM:
   case DJP_SJ_PAPERSIZE:
   {
      PDJPT_FORM       pDJPForm       = DJP_ELEMENTP (*pIQuery, DJPT_FORM);
      FORMINFO2        FormInfo;
      ULONG            ulDefaultFormID,
                       ulDefaultTrayID,
                       ulDefaultMediaID;
      LONG             lRet;

      GetIDsFromConnID (pDriver,
                        pDevice,
                        pJobProp->ulDefConnID,
                        &ulDefaultTrayID,
                        &ulDefaultFormID,
                        &ulDefaultMediaID);

      if (DJP_CJ_FORM == pTupleCur->ulProperty)
      {
         FillFormInfo (pDJPForm,
                       ulDefaultTrayID,
                       ulDefaultFormID,
                       ulDefaultMediaID,
                       pdb,
                       pJobProp->ulDefResID);

         pDJPForm++;
         pbNext = (PBYTE)pDJPForm;
      }
      else
      {
         lRet = GetDefaultFormInfo (pdb,
                                    ulDefaultFormID,
                                    pJobProp->ulDefResID,
                                    &FormInfo);

         if (lRet)
            lRet = FormInfo.ulDJPid;
         else
            lRet = DJP_PSI_NONE;

         if (DJP_PSI_NONE == lRet)
            fCurError = TRUE;
         else
            pIQuery->ulValue = lRet;
      }
      break;
   }

   case DJP_SJ_COPIES:
      pIQuery->ulValue = pJobProp->ulCopies;
      break;

   case DJP_NONE:
      // Nothing to do!
      pIQuery->ulValue = DJP_NONE;
      break;

   case DJP_SJ_PRINTQUALITY:
   case DJP_SJ_TRAYTYPE:
   case DJP_SJ_MEDIA:
   case DJP_SJ_MEDIA_COLOR:
   case DJP_CJ_MIXEDFORMS:
   case DJP_SJ_FONTDOWNLOADING:
   case DJP_SJ_DUPLEX:
   case DJP_SJ_COLLATE:
   case DJP_SJ_FEED:
   case DJP_SJ_SCALING:
   case DJP_SJ_FORMFEEDCONTROL:
   case DJP_SJ_N_UP:
   default:
      fCurError = TRUE;
      DBPRINTF (("Unknow query '%s' = %d!\n",
                 pszProperty (pTupleCur->ulProperty),
                 pTupleCur->ulProperty));
      break;
   }
}
else if (pTupleCur->lType == DJP_ALL)
{
   /* No assumptions are made here.  pbNext must be set up by each
   ** case statement
```

```
*/

switch (pTupleCur->ulProperty)
{
case DJP_SJ_ORIENTATION:
{
   PDJPT_ORIENTATION pTmp = DJP_ELEMENTP (*pIQuery, DJPT_ORIENTATION);

   *pTmp++ = DJP_ORI_PORTRAIT;
   *pTmp++ = DJP_ORI_LANDSCAPE;

   pIQuery->ulNumReturned = 2;
   pbNext = (PBYTE)pTmp;
   break;
}

case DJP_CJ_RESOLUTION:
{
   PDJPT_RESOLUTION   pRes         = DJP_ELEMENTP (*pIQuery, DJPT_RESOLUTION);
   PRESINFO           pResInfo     = pDriver->pRES;
   ULONG              ulNumDefined = pDriver->ulNumRes;
   PULONG             pulResDevice = pDevice->pulRES;
   ULONG              ulNumDevice  = pDevice->usNumRes;
   DJPT_RESOLUTION    Res;
   INT                iNumReturned = 0;
   LONG               lRet;
   BOOL               fFound;
   register INT       i, j;

   for (i = 0; i < ulNumDevice; i++)
   {
      fFound = FALSE;

      for (j = 0; j < ulNumDefined; j++)
      {
         if (pResInfo[j].ulResID == *pulResDevice)
         {
            fFound = TRUE;

            Res.usXResolution = pResInfo[j].ulXRes;
            Res.usYResolution = pResInfo[j].ulYRes;
            *pRes++ = Res;

            pbNext = (PBYTE)pRes;
         }
      }

      if (!fFound)
      {
         assertstring ("ResInfo not found!\n");
      }
      else
      {
         iNumReturned++;
      }

      pulResDevice++;
   }

   pIQuery->ulNumReturned = iNumReturned;

   if (0 == iNumReturned)
   {
      pbNext = (PBYTE)(pIQuery + 1);
      fCurError = TRUE;
   }
   break;
}

case DJP_SJ_BITSPERPEL:
case DJP_SJ_COLOR:
{
   PPRINTMODE       pPrintMode    = pDriver->pPrintModes;
   ULONG            ulNumDefined  = pDriver->ulNumPrintModes;
   PULONG           pulPrintModes = pDevice->pulPrintModes;
   ULONG            ulNumDevice   = pDevice->usNumPrintModes;
   PDJPT_BITSPERPEL pTmp          = DJP_ELEMENTP (*pIQuery, DJPT_BITSPERPEL);
   INT              iNumReturned  = 0;
   BOOL             fFoundMono    = FALSE,
                    fFoundColor   = FALSE;
   register INT     i, j;

   for (i = 0; i < ulNumDevice; i++)
```

```
    {
       for (j = 0; j < ulNumDefined; j++)
       {
          if (pPrintMode[j].ulPrintModeID == pulPrintModes[i])
          {
             if (DJP_SJ_BITSPERPEL == pTupleCur->ulProperty)
             {
                *pTmp++ = pPrintMode[j].usLogBitCount;
                iNumReturned++;
             }
             else
             {
                /* Only return 1 instance for each no matter how
                ** many print modes there are.
                */
                if (1 == pPrintMode[j].usBitsPerPel &&
                    !fFoundMono                      )
                {
                   fFoundMono = TRUE;
                   *pTmp++ = DJP_CLR_MONOCHROME;
                   iNumReturned++;
                }
                else if (!fFoundColor)
                {
                   fFoundColor = TRUE;
                   *pTmp++ = DJP_CLR_COLOR;
                   iNumReturned++;
                }
             }
          }
       }

       pulPrintModes++;
    }

    pIQuery->ulNumReturned = iNumReturned;

    if (0 == iNumReturned)
    {
       pbNext = (PBYTE)(pIQuery + 1);
       fCurError = TRUE;
    }
    else
       pbNext = (PBYTE)pTmp;
    break;
}

case DJP_SJ_PAPERSIZE:
{
    ULONG           ulNumDefined = pDevice->ulNumForms;
    PFORMINFO2      pFormInfo    = pDevice->pFORMS;
    PDJPT_PAPERSIZE pSize        = DJP_ELEMENTP (*pIQuery, DJPT_PAPERSIZE);
    INT             iNumReturned = 0;
    register INT    i;

    for (i = 0; i < ulNumDefined; i++)
    {
       LONG  lRet;

       lRet = pFormInfo[i].ulDJPid;

       if (DJP_PSI_NONE == lRet)
       {
          fCurError = TRUE;
       }
       else
       {
          iNumReturned++;
          *pSize++ = lRet;

          pbNext = (PBYTE)pSize;
       }
    }

    pIQuery->ulNumReturned = iNumReturned;

    if (0 == iNumReturned)
    {
       pbNext = (PBYTE)(pIQuery + 1);
       fCurError = TRUE;
    }
    break;
}
```

```
case DJP_CJ_FORM:
{
   PDJPT_FORM   pDJPForm    = DJP_ELEMENTP (*pIQuery, DJPT_FORM);
   PUSERCONNECT pUserConn;
   ULONG        ulFormID,
                ulTrayID,
                ulMediaID;
   INT          iNumReturned = 0;
   register INT i;

   iNumReturned = 0;

   for (i = 0; i < pDevice->usNumConnects; i++)
   {
      GetIDsFromConnID (pDriver,
                        pDevice,
                        pDevice->pulCONNECTS[i],
                        &ulTrayID,
                        &ulFormID,
                        &ulMediaID);

      FillFormInfo (pDJPForm,
                    ulTrayID,
                    ulFormID,
                    ulMediaID,
                    pdb,
                    pJobProp->ulDefResID);

      pDJPForm++;
      pbNext = (PBYTE)pDJPForm;
      iNumReturned++;
   }

   // get pointer to first user-defined form connection
   pUserConn = pDevice->pUserDefData->pUserCONNECTS;

   // as long as we have user-defined form connections
   while (pUserConn)
   {
      FillFormInfo (pDJPForm,
                    pUserConn->fcUser.ulTrayID,
                    pUserConn->fcUser.ulFormID,
                    pUserConn->fcUser.ulMediaID,
                    pdb,
                    pJobProp->ulDefResID);

      pDJPForm++;
      pbNext = (PBYTE)pDJPForm;
      iNumReturned++;

      // Move to the next connection
      pUserConn = pUserConn->pNextConnect;
   }

   pIQuery->ulNumReturned = iNumReturned;

   if (0 == iNumReturned)
   {
      pbNext = (PBYTE)(pIQuery + 1);
      fCurError = TRUE;
   }
   break;
}

case DJP_SJ_COPIES:
{
   pIQuery->lType         = DJP_ERROR_NOT_ENUM;
   pIQuery->ulNumReturned = 0;
   pIQuery->ulValue       = DJP_NONE;

   pbNext = (PBYTE)(pIQuery + 1);

   fError = TRUE;
   break;
}

case DJP_NONE:
{
   // Nothing to do!
   pIQuery->ulNumReturned = 0;
   pIQuery->ulValue       = DJP_NONE;
```

```
            pbNext = (PBYTE)(pIQuery + 1);
            break;
         }

         case DJP_SJ_PRINTQUALITY:
         case DJP_SJ_TRAYTYPE:
         case DJP_SJ_MEDIA:
         case DJP_SJ_MEDIA_COLOR:
         case DJP_CJ_MIXEDFORMS:
         case DJP_SJ_FONTDOWNLOADING:
         case DJP_SJ_DUPLEX:
         case DJP_SJ_COLLATE:
         case DJP_SJ_FEED:
         case DJP_SJ_SCALING:
         case DJP_SJ_FORMFEEDCONTROL:
         case DJP_SJ_N_UP:
         default:
            fCurError = TRUE;

            pbNext = (PBYTE)(pIQuery + 1);

            DBPRINTF (("Unknow query '%s' = %d!\n",
                       pszProperty (pTupleCur->ulProperty),
                       pTupleCur->ulProperty));
            break;
         }
      }
      else if (DJP_NONE != pTupleCur->lType)
      {
         fCurError = TRUE;

         assertstring ("Unknown query type!\n");
      }

      // We now know the final size
      pIQuery->cb = pbNext - (PBYTE)pIQuery;

      /* @TBD We need some way to stop overwriting memory
      */
      assertT (cbQueries < pIQuery->cb);

      cbQueries -= pIQuery->cb;

      if (fCurError)
      {
         pIQuery->ulNumReturned = 0;
         pIQuery->ulValue       = DJP_NONE;
         pIQuery->lType         = DJP_ERROR_NOT_SUPPORTED;
         fError                 = TRUE;
      }

      // Move to the next query item
      pIQuery = DJP_NEXT_STRUCTP (pIQuery);

      iNumQueries--;
      pTupleCur++;
   }

   // Clean up
   if (pTuples)
      GplMemoryFree (pTuples);

   if (fError)
   {
      GplErrSetWarning (PMERR_DATATYPE_ENTRY_INVALID);
      ulrc = DEV_WARNING;
   }
   else
   {
      // good result
      ulrc = DEV_OK;
   }
   break;
}
}

return lrc;
}
```

GreEscape DEVESC_QUERYJOBPROPERTIES - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_QUERYSIZE

GreEscape DEVESC_QUERYSIZE - Syntax

Simulation support:

This function is mandatory for all drivers and must be implemented in order to have a valid OS/2 hardcopy driver.

Description:

GreEscape DEVESC_QUERYSIZE queries the size of memory (in bytes) required for the corresponding list of specified job property types.

Refer to Dynamic Job Properties for related information.

```
#define INCL_DEV
#define INCL_DEVDJP
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_QUERYSIZE escape code. */
LONG     cInCount;     /*  Number of bytes pointed to by pInData. */
PBYTE    pInData;      /*  Pointer to the QUERYSIZE data structure. */
PLONG    pcOutCount;   /*  A pointer to the number of bytes pointed to by pOutData. */
PLONG    pOutData;     /*  Pointer to the printer driver's job properties. */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreEscape. */
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_QUERYSIZE Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_QUERYSIZE Parameter - lEscape

lEscape (LONG) - input
DEVESC_QUERYSIZE escape code.

GreEscape DEVESC_QUERYSIZE Parameter - cInCount

cInCount (LONG) - input
Number of bytes pointed to by *pInData*.

GreEscape DEVESC_QUERYSIZE Parameter - pInData

pInData (PBYTE) - output
Pointer to the QUERYSIZE data structure.

Note: *pQuerySize ->ulSizeNeeded* is used as output.

GreEscape DEVESC_QUERYSIZE Parameter - pcOutCount

pcOutCount (PLONG) - input
A pointer to the number of bytes pointed to by *pOutData*.

GreEscape DEVESC_QUERYSIZE Parameter - pOutData

pOutData (PLONG) - input
Pointer to the printer driver's job properties.

This should be complete and not missing data or partially sized.

GreEscape DEVESC_QUERYSIZE Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_QUERYSIZE Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_QUERYSIZE Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK
Successful.

DEV_PROP_BUF_TOO_SMALL
Job properties buffer did not have adequate space to contain the proper job properties data. The buffer will not be updated.

DEV_INV_INP_JOBPROPERTIES
Invalid job properties were passed in. The driver changed them to default values. This is an error and the item buffer will not be updated.

DEVESC_ERROR
Error

GreEscape DEVESC_QUERYSIZE - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_QUERYSIZE escape code.

cInCount (LONG) - input
Number of bytes pointed to by *pInData*.

pInData (PBYTE) - output
Pointer to the QUERYSIZE data structure.

Note: *pQuerySize ->ulSizeNeeded* is used as output.

pcOutCount (PLONG) - input
A pointer to the number of bytes pointed to by *pOutData*.

pOutData (PLONG) - input
Pointer to the printer driver's job properties.

This should be complete and not missing data or partially sized.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input

High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK
Successful.

DEV_PROP_BUF_TOO_SMALL
Job properties buffer did not have adequate space to contain the proper job properties data. The buffer will not be updated.

DEV_INV_INP_JOBPROPERTIES
Invalid job properties were passed in. The driver changed them to default values. This is an error and the item buffer will not be updated.

DEVESC_ERROR
Error

GreEscape DEVESC_QUERYSIZE - Remarks

DEVESC_QUERYSIZE is used to determine the amount of buffer space necessary to query or set single or multiple job properties. Since DEVESC_QUERYSIZE is *new* , applications will call GreEscape DEVESC_QUERYESCSUPPORT to determine if you support this DevEscape.

This function has a handle to a device context (HDC) that should already have printer property information associated with the HDC. Job property information may change or may be dependent upon the printer property information in the HDC. In that case, if an application passes job properties for a different device or for a different spooler printer name, then misleading information will be returned.

If a property is not supported, DEVESC_QUERYSIZE will allow space to hold the size of a simple DJP_ITEM structure. This space allows the next part of the process to continue without error. The application will be notified by GreEscape DEVESC_QUERYJOBPROPERTIES that the propery is not supported.

Make sure to test the input job properties to make sure that they are valid. This includes passing tests such as whether they contain your driver's signature string, whether the device name is yours, and whether the values are within proper ranges (only landscape or portrait). If these tests fail, then fail the call.

```
LONG ENGENTRY Escape (HDC       hdc,
                      LONG      lEscape,
                      LONG      cInCount,
                      PBYTE     pInData,
                      PLONG     pcOutCount,
                      PBYTE     pOutData,
                      PDDC      pddc,
                      ULONG     ulFunction)
{
   switch (lEscape)
   {
   case DEVESC_QUERYSIZE:
   {
      INT         cbQueries  = cInCount;
      PQUERYSIZE  pQuerySize = (PQUERYSIZE)pInData;
      PQUERYTUPLE pTuple     = pQuerySize->aTuples;
      INT         cbElmSize  = sizeof (QUERYTUPLE);
      INT         iCount;
      INT         iMaxCount;
      INT         iSize;

      if (GRE_234 > globals.ulGreVersion)
      {
         assertstring ("Not supported on pre-DAX engine!\n");
         lrc = DEVESC_NOTIMPLEMENTED;
         break;
      }

      pGoodDrivData = ReturnDriverData (pddc->pdb,
                                        pdi,
                                        pddc->pdb->hmcbHeap,
                                        pDrivData,
```

```
                                    pddc->pdb->pszPrinterName,
                                    TRUE,
                                    pDevice,
                                    pDriver);
assertF (pGoodDrivData);

/* Is the size of the input driver data is not the same as
** the size of our good data?
*/
if (pDrivData->cb != pGoodDrivData->cb)
   lrc = DEV_PROP_BUF_TOO_SMALL;

/* Or, if the two don't compare, then fail the call.
*/
if (0 != memcmp (pDrivData, pGoodDrivData, pGoodDrivData->cb))
   lrc = DEV_INV_INP_JOBPROPERTIES;

// Take off the header
cbQueries -= QUERYSIZE_HEADER_SIZE;

// Must be an even multiple
assertT (cbQueries % cbElmSize);
if (cbQueries % cbElmSize)
   break;

// Initialize the returned size
pQuerySize->ulSizeNeeded = 0;

while (cbQueries)
{
   switch (pTuple->ulProperty)
   {
   case DJP_NONE:
      iSize     = sizeof (((PDJP_ITEM)NULL)->ulValue);
      iMaxCount = 1;
      break;

   case DJP_SJ_ORIENTATION:
      iSize     = sizeof (DJPT_ORIENTATION);
      iMaxCount = 2;
      break;

   case DJP_CJ_RESOLUTION:
      iSize     = sizeof (DJPT_RESOLUTION);
      iMaxCount = pDevice->usNumRes;
      break;

   case DJP_SJ_BITSPERPEL:
      iSize     = sizeof (DJPT_BITSPERPEL);
      iMaxCount = pDevice->usNumPrintModes;
      break;

   case DJP_SJ_COLOR:
   {
      PPRINTMODE       pPrintMode    = pDriver->pPrintModes;
      ULONG            ulNumDefined  = pDriver->ulNumPrintModes;
      PULONG           pulPrintModes = pDevice->pulPrintModes;
      ULONG            ulNumDevice   = pDevice->usNumPrintModes;
      BOOL             fFoundMono    = FALSE,
                       fFoundColor   = FALSE;
      register INT     i, j;

      for (i = 0; i < ulNumDevice; i++)
      {
         for (j = 0; j < ulNumDefined; j++)
         {
            if (pPrintMode[j].ulPrintModeID == pulPrintModes[i])
            {
               if (1 == pPrintMode[j].usBitsPerPel)
                  fFoundMono = TRUE;
               else
                  fFoundColor = TRUE;
            }
         }
      }

      iSize     = sizeof (DJPT_COLOR);
      iMaxCount = 0;
      if (fFoundMono)
         iMaxCount++;
      if (fFoundColor)
         iMaxCount++;
      if (!fFoundMono &&
```

```
        !fFoundColor )
        iMaxCount = 1;
    break;
}

case DJP_CJ_FORM:
case DJP_SJ_PAPERSIZE:
{
    PUSERCONNECT pUserConn;
    ULONG        ulFormID,
                 ulTrayID,
                 ulMediaID;
    FORMINFO2    FormInfo;
    PTRAYINFO    pTrayInfo      = NULL;
    LONG         lRet;
    register INT i;

    iMaxCount = 0;

    switch (pTuple->ulProperty)
    {
    case DJP_CJ_FORM:
        iSize = sizeof (DJPT_FORM);
        break;

    case DJP_SJ_PAPERSIZE:
        iSize = sizeof (DJPT_PAPERSIZE);
        break;
    }

    for (i = 0; i < pDevice->usNumConnects; i++)
    {
        GetIDsFromConnID (pDriver,
                          pDevice,
                          pDevice->pulCONNECTS[i],
                          &ulTrayID,
                          &ulFormID,
                          &ulMediaID);

        if (DJP_CJ_FORM == pTuple->ulProperty)
        {
            iMaxCount++;
        }
        else if (DJP_SJ_PAPERSIZE == pTuple->ulProperty)
        {
            lRet = GetDefaultFormInfo (pdb,
                                       ulFormID,
                                       pJobProp->ulDefResID,
                                       &FormInfo);

            if (lRet)
                lRet = FormInfo.ulDJPid;
            else
                lRet = DJP_PSI_NONE;

            if (DJP_PSI_NONE != lRet)
                iMaxCount++;
        }
    }

    // get pointer to first user-defined form connection
    pUserConn = pDevice->pUserDefData->pUserCONNECTS;

    // as long as we have user-defined form connections
    while (pUserConn)
    {
        if (DJP_CJ_FORM == pTuple->ulProperty)
        {
            iMaxCount++;
        }
        else if (DJP_SJ_PAPERSIZE == pTuple->ulProperty)
        {
            lRet = GetDefaultFormInfo (pdb,
                                       ulFormID,
                                       pJobProp->ulDefResID,
                                       &FormInfo);

            if (lRet)
                lRet = FormInfo.ulDJPid;
            else
                lRet = DJP_PSI_NONE;

            if (DJP_PSI_NONE != lRet)
```

```
                iMaxCount++;
            }

            // Move to the next connection
            pUserConn = pUserConn->pNextConnect;
         }

         if (0 == iMaxCount)
            // Make sure its at least 1
            iMaxCount = 1;
         break;
      }

      case DJP_SJ_COPIES:
         iSize     = sizeof (DJPT_COPIES);
         iMaxCount = 1;
         break;

      case DJP_SJ_PRINTQUALITY:
      case DJP_SJ_TRAYTYPE:
      case DJP_SJ_MEDIA:
      case DJP_SJ_MEDIA_COLOR:
      case DJP_CJ_MIXEDFORMS:
      case DJP_SJ_FONTDOWNLOADING:
      case DJP_SJ_DUPLEX:
      case DJP_SJ_COLLATE:
      case DJP_SJ_FEED:
      case DJP_SJ_SCALING:
      case DJP_SJ_FORMFEEDCONTROL:
      case DJP_SJ_N_UP:
      default:
         iSize         = sizeof (((PDJP_ITEM)NULL)->ulValue);
         iMaxCount     = 1;
         DBPRINTF (("Unknow query '%s' = %d!\n",
                    pszProperty (pTuple->ulProperty),
                    pTuple->ulProperty));
         break;
      }

      DBPRINTF (("Query size '%s'/%d = %d.  Max count = %d\n",
                 pszProperty (pTuple->ulProperty),
                 pTuple->ulProperty,
                 iSize, iMaxCount));

      if (DJP_NONE    == pTuple->ulProperty ||
          DJP_CURRENT == pTuple->lType       )
      {
         iCount = 1;
      }
      else if (DJP_ALL == pTuple->lType)
      {
         iCount = iMaxCount;
      }
      else
      {
         iCount = 1;
         assertstring ("Unknown type!\n");
      }

      // Bump up the returned size
      pQuerySize->ulSizeNeeded += DJP_HEADER_SIZE + iCount * iSize;

      cbQueries -= cbElmSize;
      pTuple++;
   }

   // good result
   ulrc = DEV_OK;
   break;
}
}

return lrc;
}
```

GreEscape DEVESC_QUERYSIZE - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_QUERYVIOCELLSIZES

GreEscape DEVESC_QUERYVIOCELLSIZES - Syntax

Simulation support:

This function is mandatory for display drivers. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreEscape DEVESC_QUERYVIOCELLSIZES obtains the details of the VIO cell sizes supported by the presentation driver.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_QUERYVIOCELLSIZES escape code. */
LONG     cInCount;     /*  The handling routine ignores this parameter. */
PBYTE    pInData;      /*  The handling routine ignores this parameter. */
PLONG    pcOutCount;   /*  Pointer to the number of bytes pointed to by pOutData. */
PLONG    pOutData;     /*  Pointer to the address of the data returned. */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreEscape. */
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_QUERYVIOCELLSIZES Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_QUERYVIOCELLSIZES Parameter - lEscape

lEscape (LONG) - input
DEVESC_QUERYVIOCELLSIZES escape code.

GreEscape DEVESC_QUERYVIOCELLSIZES Parameter - cInCour

cInCount (LONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_QUERYVIOCELLSIZES Parameter - pInData

pInData (PBYTE) - input
The handling routine ignores this parameter.

GreEscape DEVESC_QUERYVIOCELLSIZES Parameter - pcOutC

pcOutCount (PLONG) - input
Pointer to the number of bytes pointed to by pOutData.

On return, the handling routine updates the value indicated by this pointer to the number of bytes actually returned. This value must be an even multiple of sizeof(LONG). When the value passed is less than sizeof(LONG), the handling routine must change it to 0. Nothing is loaded at the address indicated by *pOutData* .

When the value passed equals sizeof(LONG), the handling routine must return the number of supported VIO character-cell sizes. The value indicated by *pcOutCount* is unchanged. The contents of the address indicated by *pOutData* are updated so that maxcount is the number of VIO cell sizes provided by the device. When the value passed is greater than sizeof(LONG), the handling routine must update the buffer addressed by *pOutData* so that:

- "maxcount" is the number of VIO cell sizes provided by the device.
- "count" is the number of VIOFONTCELLSIZE structures returned. This can be 0 when *OutCount* is 2*sizeof(LONG). For example:

```
count==((pcOutCount-2*sizeof(LONG))/2*sizeof(LONG))
```

GreEscape DEVESC_QUERYVIOCELLSIZES Parameter - pOutDa

pOutData (PLONG) - input
Pointer to the address of the data returned.

This is the location at which the handling routine stores the following structures:

maxcount	Total number of VIO cell sizes provided by the device
count	Number of VIOFONTCELLSIZE structures that follow

Followed by an array of VIOFONTCELLSIZE structures:

xWidth	Width of the VIO character cell
yHeight	Height of the VIO character cell

GreEscape DEVESC_QUERYVIOCELLSIZES Parameter - pInstanc

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_QUERYVIOCELLSIZES Parameter - lFunctio

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_QUERYVIOCELLSIZES Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_QUERYVIOCELLSIZES - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_QUERYVIOCELLSIZES escape code.

cInCount (LONG) - input
The handling routine ignores this parameter.

pInData (PBYTE) - input
The handling routine ignores this parameter.

pcOutCount (PLONG) - input
Pointer to the number of bytes pointed to by pOutData.

On return, the handling routine updates the value indicated by this pointer to the number of bytes actually returned. This value must be an even multiple of sizeof(LONG). When the value passed is less than sizeof(LONG), the handling routine must change it to 0. Nothing is loaded at the address indicated by *pOutData* .

When the value passed equals sizeof(LONG), the handling routine must return the number of supported VIO character-cell sizes. The value indicated by *pcOutCount* is unchanged. The contents of the address indicated by *pOutData* are updated so that maxcount is the

number of VIO cell sizes provided by the device. When the value passed is greater than sizeof(LONG), the handling routine must update the buffer addressed by *pOutData* so that:

- "maxcount" is the number of VIO cell sizes provided by the device.
- "count" is the number of VIOFONTCELLSIZE structures returned. This can be 0 when *OutCount* is 2*sizeof(LONG). For example:

```
count==((pcOutCount-2*sizeof(LONG))/2*sizeof(LONG))
```

pOutData (PLONG) - input
Pointer to the address of the data returned.

This is the location at which the handling routine stores the following structures:

maxcount	Total number of VIO cell sizes provided by the device
count	Number of VIOFONTCELLSIZE structures that follow

Followed by an array of VIOFONTCELLSIZE structures:

xWidth	Width of the VIO character cell
yHeight	Height of the VIO character cell

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_QUERYVIOCELLSIZES - Remarks

The DEVESC_QUERYVIOCELLSIZES must be implemented by presentation drivers for display devices that provide two or more VIO fonts with different cell sizes. Other presentation drivers should not implement this escape code.

This escape code is not metafiled or recorded.

GreEscape DEVESC_QUERYVIOCELLSIZES - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_RAWDATA

GreEscape DEVESC_RAWDATA - Syntax

Simulation support:

This function is mandatory for hardcopy drivers and must be implemented in order to have a valid OS/2 hardcopy driver. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreEscape DEVESC_RAWDATA sends device-specific data directly to the spooler or device. The action taken by the handling routine is determined by the DC type.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_RAWDATA escape code. */
LONG     cInCount;     /*  Number of bytes pointed to by pInData. */
PBYTE    pInData;      /*  Pointer to raw data. */
PLONG    pcOutCount;   /*  The handling routine ignores this parameter. */
PLONG    pOutData;     /*  The handling routine ignores this parameter. */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreEscape. */
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_RAWDATA Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_RAWDATA Parameter - lEscape

lEscape (LONG) - input
DEVESC_RAWDATA escape code.

GreEscape DEVESC_RAWDATA Parameter - cInCount

cInCount (LONG) - input
Number of bytes pointed to by pInData.

GreEscape DEVESC_RAWDATA Parameter - pInData

pInData (PBYTE) - input
Pointer to raw data.

GreEscape DEVESC_RAWDATA Parameter - pcOutCount

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_RAWDATA Parameter - pOutData

pOutData (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_RAWDATA Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_RAWDATA Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_RAWDATA Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_RAWDATA - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_RAWDATA escape code.

cInCount (LONG) - input
Number of bytes pointed to by pInData.

pInData (PBYTE) - input
Pointer to raw data.

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

pOutData (PLONG) - input
The handling routine ignores this parameter.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_RAWDATA - Remarks

The action taken by the handling routine is determined by the DC type. For example, OD_DIRECT DC would send the raw data directly to the device using the Prt*xxx* APIs. See File System Emulation.

This escape code is metafiled and recorded.

As a general rule, an application should use DEVESC_RAWDATA only for a complete document or frame within a document. DEVESC_RAWDATA must not be mixed with other drawing functions. If DEVESC_RAWDATA and other drawing functions are called in a single frame, the results are dependent on the implementation. For example, the presentation driver might choose to print the raw data and ignore the other drawing calls.

Note: For an OD_QUEUED device with PM_Q_STD data, the spooler records this call in the buffer, but does not pass it on to the presentation driver.

GreEscape DEVESC_RAWDATA - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_REGISTER

GreEscape DEVESC_REGISTER - Syntax

Simulation support:

None.

Description

GreEscape DEVESC_REGISTER is called by EnDIVE users to register and unregister use with the driver. This function allows each calling process to be uniquely identified by the handle returned on a register operation.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     ICODE;        /*  DEVESC_REGISTER escape code. */
LONG     lInCount;     /*  Number of bytes pointed to by pbInData. */
PBYTE    pbInData;     /*  Pointer to VRAMREGISTERIN data structure. */
PLONG    plOutCount;   /*  Number of bytes pointed to by pbOutData. */
PLONG    pbOutData;    /*  Pointer to VRAMREGISTEROUT data structure. */
LONG     rc;           /*  Return Codes. */

rc = GreEscape(hdc, ICODE, lInCount, pbInData,
       plOutCount, pbOutData);
```

GreEscape DEVESC_REGISTER Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_REGISTER Parameter - ICODE

ICODE (LONG) - input
DEVESC_REGISTER escape code.

GreEscape DEVESC_REGISTER Parameter - lInCount

llnCount (LONG) - input
Number of bytes pointed to by pbInData.

GreEscape DEVESC_REGISTER Parameter - pbInData

pbInData (PBYTE) - input
Pointer to VRAMREGISTERIN data structure.

GreEscape DEVESC_REGISTER Parameter - plOutCount

plOutCount (PLONG) - input
Number of bytes pointed to by pbOutData.

GreEscape DEVESC_REGISTER Parameter - pbOutData

pbOutData (PLONG) - input
Pointer to VRAMREGISTEROUT data structure.

GreEscape DEVESC_REGISTER Return Value - rc

rc (LONG) - returns
Return Codes.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	Error

WINGETLASTERROR can be called for specific error information.

GreEscape DEVESC_REGISTER - Parameters

hdc (HDC) - input
Device context handle.

lCODE (LONG) - input

DEVESC_REGISTER escape code.

llnCount (LONG) - input
Number of bytes pointed to by pbInData.

pbInData (PBYTE) - input
Pointer to VRAMREGISTERIN data structure.

plOutCount (PLONG) - input
Number of bytes pointed to by pbOutData.

pbOutData (PLONG) - input
Pointer to VRAMREGISTEROUT data structure.

rc (LONG) - returns
Return Codes.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	Error

WINGETLASTERROR can be called for specific error information.

GreEscape DEVESC_REGISTER - Remarks

None.

GreEscape DEVESC_REGISTER - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_SETJOBPROPERTIES

GreEscape DEVESC_SETJOBPROPERTIES - Syntax

Simulation support:

This function is mandatory for all drivers and must be implemented in order to have a valid OS/2 hardcopy driver.

Description:

GreEscape DEVESC_SETJOBPROPERTIES writes the the specified job property types and values into the job properties structure that is passed in.

Refer to Dynamic Job Properties for related information.

```
#define INCL_DEV
#define INCL_DEVDJP
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_SETJOBPROPERTIES escape code. */
LONG     cInCount;     /*  Number of bytes pointed to by pInData. */
PBYTE    pInData;      /*  Pointer to a list of DJP_ITEM data structures. */
PLONG    pcOutCount;   /*  Number of bytes pointed to by pOutData. */
PLONG    pOutData;     /*  Pointer to the printer driver's job properties structure */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreEscape. */
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_SETJOBPROPERTIES Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_SETJOBPROPERTIES Parameter - lEscape

lEscape (LONG) - input
DEVESC_SETJOBPROPERTIES escape code.

GreEscape DEVESC_SETJOBPROPERTIES Parameter - cInCoun

cInCount (LONG) - input
Number of bytes pointed to by *pInData*.

GreEscape DEVESC_SETJOBPROPERTIES Parameter - pInData

pInData (PBYTE) - output
Pointer to a list of DJP_ITEM data structures.

The end of the list is indicated by a DJP_ITEM data structure that has DJP_NONE as its *ulProperty* value.

GreEscape DEVESC_SETJOBPROPERTIES Parameter - pcOutCc

pcOutCount (PLONG) - input
Number of bytes pointed to by *pOutData*.

GreEscape DEVESC_SETJOBPROPERTIES Parameter - pOutDat

pOutData (PLONG) - input
Pointer to the printer driver's job properties structure

as returned by SplQueryQueue. On output, the job properties are changed.

GreEscape DEVESC_SETJOBPROPERTIES Parameter - pInstanc

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_SETJOBPROPERTIES Parameter - lFunctior

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_SETJOBPROPERTIES Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK
: Successful.

DEV_PROP_BUF_TOO_SMALL
: Job properties buffer was too small to contain the proper job properties data. The buffer will not be updated.

DEV_INV_INP_JOBPROPERTIES
: Invalid job properties were passed in. The driver changed them to default values. This is an error and the item buffer will not be updated.

DEV_WARNING

One or more of the DJP_ITEMs has an error in the *lType* field.

Valid input for *lType* is as follows:

DJP_CURRENT
: The application requests the current value to be changed in the input job properties.

The following are valid output for the *lType* field:

DJP_CURRENT
: Unchanged; successful query.

DJP_ERROR_NOT_SUPPORTED
: The driver does not support the value in the *ulProperty* field of the DJP_ITEMs data structure.

DEVESC_ERROR_INV_PARMS
: The value in the *ulValue* field of the DJP_ITEMs data structure is not valid.

DJP_ERROR_OUT_OF_RANGE
: The value in *ulValue* is not in the valid range for the specified property.

DJP_ERROR
: Error

GreEscape DEVESC_SETJOBPROPERTIES - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_SETJOBPROPERTIES escape code.

cInCount (LONG) - input
Number of bytes pointed to by *pInData*.

pInData (PBYTE) - output
Pointer to a list of DJP_ITEM data structures.

The end of the list is indicated by a DJP_ITEM data structure that has DJP_NONE as its *ulProperty* value.

pcOutCount (PLONG) - input
Number of bytes pointed to by *pOutData*.

pOutData (PLONG) - input
Pointer to the printer driver's job properties structure

as returned by SplQueryQueue. On output, the job properties are changed.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK
: Successful.

DEV_PROP_BUF_TOO_SMALL
: Job properties buffer was too small to contain the proper job properties data. The buffer will not be

updated.

DEV_INV_INP_JOBPROPERTIES

Invalid job properties were passed in. The driver changed them to default values. This is an error and the item buffer will not be updated.

DEV_WARNING

One or more of the DJP_ITEMs has an error in the *lType* field.

Valid input for *lType* is as follows:

DJP_CURRENT

The application requests the current value to be changed in the input job properties.

The following are valid output for the *lType* field:

DJP_CURRENT

Unchanged; successful query.

DJP_ERROR_NOT_SUPPORTED

The driver does not support the value in the *ulProperty* field of the DJP_ITEMs data structure.

DEVESC_ERROR_INV_PARMS

The value in the *ulValue* field of the DJP_ITEMs data structure is not valid.

DJP_ERROR_OUT_OF_RANGE

The value in *ulValue* is not in the valid range for the specified property.

DJP_ERROR

Error

GreEscape DEVESC_SETJOBPROPERTIES - Remarks

DEVESC_SETJOBPROPERTIES is used to set information in the printer's job properties structure. This structure is known only to the printer presentation driver and this escape is the only way for an application to set values inside the structure.

Since DEVESC_SETJOBPROPERTIES is *new* , applications will call GreEscape DEVESC_QUERYESCSUPPORT to determine if you support this DevEscape.

The DevEscape path has a handle to a device context (HDC) that should already have printer property information associated with the HDC. Job property information may change or may be dependent upon the printer property information in the HDC. In that case, if an application passes job properties for a different device or for a different spooler printer name, then misleading information will be returned.

You should test the input job properties to make sure that they are valid. This includes passing tests such as whether they contain your driver's signature string, whether the device name is yours, and whether the values are within proper ranges (only landscape or portrait). If these tests fail, then fail the call.

```
LONG ENGENTRY Escape (HDC       hdc,
                      LONG      lEscape,
                      LONG      cInCount,
                      PBYTE     pInData,
                      PLONG     pcOutCount,
                      PBYTE     pOutData,
                      PDDC      pddc,
                      ULONG     ulFunction)
{
   switch (lEscape)
   {
   case DEVESC_SETJOBPROPERTIES:
   {
      INT            cbQueries   = cInCount;
      PBYTE          pbQueries   = pInData;
      PDJP_ITEM      pISet       = (PDJP_ITEM)pbQueries;
      BOOL           fError      = FALSE;
      ULONG          ulMaxSize;
```

```
if (GRE_234 > globals.ulGreVersion)
{
   assertstring ("Not supported on pre-DAX engine!\n");
   lrc = DEVESC_NOTIMPLEMENTED;
   break;
}

pGoodDrivData = ReturnDriverData (pddc->pdb,
                                  pdi,
                                  pddc->pdb->hmcbHeap,
                                  pDrivData,
                                  pddc->pdb->pszPrinterName,
                                  TRUE,
                                  pDevice,
                                  pDriver);
assertF (pGoodDrivData);

/* Is the size of the input driver data is not the same as
** the size of our good data?
*/
if (pDrivData->cb != pGoodDrivData->cb)
   lrc = DEV_PROP_BUF_TOO_SMALL;

/* Or, if the two don't compare, then fail the call.
*/
if (0 != memcmp (pDrivData, pGoodDrivData, pGoodDrivData->cb))
   lrc = DEV_INV_INP_JOBPROPERTIES;

// Update the job properties with the input set list
do
{
   DBPRINTF (("Set '%s'/%d.  Query type '%s'/%d. Simple value = %d\n",
              pszProperty (pISet->ulProperty),
              pISet->ulProperty,
              pszType (pISet->lType),
              pISet->lType,
              pISet->ulValue));

   switch (pISet->ulProperty)
   {
   case DJP_SJ_ORIENTATION:
   {
      if (pISet->ulValue == DJP_ORI_PORTRAIT)
      {
         pJobProp->ulOrientation = ORIENTATION_PORTRAIT;
      }
      else if (pISet->ulValue == DJP_ORI_LANDSCAPE)
      {
         pJobProp->ulOrientation = ORIENTATION_LANDSCAPE;
      }
      else
      {
         pISet->lType = DJP_ERROR_OUT_OF_RANGE;
      }
      break;
   }

   case DJP_CJ_RESOLUTION:
   {
      PDJPT_RESOLUTION   pRes         = DJP_ELEMENTP (*pISet, DJPT_RESOLUTION);
      PRESINFO           pResInfo     = pDriver->pRES;
      ULONG              ulNumDefined = pDriver->ulNumRes;
      PULONG             pulResDevice = pDevice->pulRES;
      ULONG              ulNumDevice  = pDevice->usNumRes;
      BOOL               fFound       = FALSE;
      register INT       i, j;

      for (i = 0; !fFound && i < ulNumDevice; i++, pulResDevice++)
      {
         for (j = 0; j < ulNumDefined; j++)
         {
            if (pResInfo[j].ulResID == *pulResDevice)
            {
               if (pRes->usXResolution == pResInfo[j].ulXRes &&
                   pRes->usYResolution == pResInfo[j].ulYRes  )
               {
                  fFound = TRUE;
                  pJobProp->ulDefResID = *pulResDevice;
                  break;
               }
            }
         }
      }
```

```
   if (!fFound)
      pISet->lType = DJP_ERROR_OUT_OF_RANGE;
   break;
}

case DJP_SJ_BITSPERPEL:
case DJP_SJ_COLOR:
{
   PPRINTMODE       pPrintMode    = pDriver->pPrintModes;
   ULONG            ulNumDefined  = pDriver->ulNumPrintModes;
   PULONG           pulPrintModes = pDevice->pulPrintModes;
   ULONG            ulNumDevice   = pDevice->usNumPrintModes;
   PDJPT_BITSPERPEL pTmp          = DJP_ELEMENTP (*pISet, DJPT_BITSPERPEL);
   BOOL             fFound        = FALSE;
   register INT     i, j;

   for (i = 0; !fFound && i < ulNumDevice; i++, pulPrintModes++)
   {
      fFound = FALSE;

      for (j = 0; j < ulNumDefined; j++)
      {
         if (pPrintMode[j].ulPrintModeID == *pulPrintModes)
         {
            if (DJP_SJ_BITSPERPEL == pISet->ulProperty)
            {
               if (*pTmp == pPrintMode[j].usLogBitCount)
               {
                  pJobProp->ulDefPrintModeID = *pulPrintModes;
                  fFound = TRUE;
                  break;
               }
            }
            else
            {
               if (((1     == pPrintMode[j].usBitsPerPel) &&
                    (*pTmp == DJP_CLR_MONOCHROME)          ) ||
                   ((1     <  pPrintMode[j].usBitsPerPel) &&
                    (*pTmp == DJP_CLR_COLOR)               )  )
               {
                  pJobProp->ulDefPrintModeID = *pulPrintModes;
                  fFound = TRUE;
                  break;
               }
            }
         }
      }
   }

   if (!fFound)
      pISet->lType = DJP_ERROR_OUT_OF_RANGE;
   break;
}

case DJP_SJ_PAPERSIZE:
{
   ULONG           ulNumDefined = pDevice->ulNumForms;
   PFORMINFO2      pFormInfo    = pDevice->pFORMS;
   PDJPT_PAPERSIZE pSize        = DJP_ELEMENTP (*pISet, DJPT_PAPERSIZE);
   BOOL            fFound       = FALSE;
   LONG            lRet;
   register INT i;

   for (i = 0; i < ulNumDefined; i++)
   {
      lRet = pFormInfo[i].ulDJPid;

      if (DJP_PSI_NONE != lRet && *pSize == lRet)
      {
         fFound = TRUE;
         break;
      }
   }

   if (fFound)
   {
      ULONG  ulDefaultConnID  = pJobProp->ulDefConnID,
             ulDefaultFormID  = 0,
             ulDefaultTrayID  = 0,
             ulDefaultMediaID = 0;
      BOOL   bOK;
```

```
      // Ask for the current connection id
      bOK = GetIDsFromConnID (pDriver,
                              pDevice,
                              ulDefaultConnID,
                              &ulDefaultTrayID,
                              &ulDefaultFormID,
                              &ulDefaultMediaID);

      if (bOK)
         // Get a new connection id
         bOK = NewConnection (pdb,
                              pGoodDrivData->szDeviceName,
                              &ulDefaultConnID,
                              pFormInfo[i].ulFormID, // Override form
                              ulDefaultTrayID,
                              ulDefaultMediaID,
                              FALSE);                // Dont create a new one!

      // Set it!
      if (bOK)
         pJobProp->ulDefConnID = ulDefaultConnID;
      else
         pISet->lType = DJP_ERROR_OUT_OF_RANGE;
   }
   else
   {
      pISet->lType = DJP_ERROR_OUT_OF_RANGE;
   }
   break;
}

case DJP_CJ_FORM:
{
   ULONG           ulNumDefined;
   ULONG           ulNumDevice;
   PDJPT_FORM      pDJPForm     = DJP_ELEMENTP (*pISet, DJPT_FORM);
   PFORMINFO2      pFormInfo    = (PFORMINFO2)NULL;
   PTRAYINFO       pTrayInfo    = (PTRAYINFO)NULL;
   PMEDIAINFO      pMediaInfo   = (PMEDIAINFO)NULL;
   CHAR            szTempName[32];
   BOOL            bOK;
   register INT    i, j;

   ulNumDevice = pDevice->ulNumForms;
   for (i = 0; i < ulNumDevice; i++)
   {
      FormNameFromID (pDriver,
                      pDevice,
                      szTempName,
                      pDevice->pFORMS[i].ulFormID);

      if (0 == strcmp (pDJPForm->szFormname, szTempName))
      {
         pFormInfo = (PFORMINFO2) &pDevice->pFORMS[i];
         break;
      }
   }

   if (pDJPForm->szTrayname[0])
   {
      ulNumDevice  = pDevice->usNumTrays;
      ulNumDefined = pDriver->ulNumTrays;

      for (i = 0; i < ulNumDefined && !pTrayInfo; i++)
      {
         for (j = 0; j < ulNumDevice; j++)
         {
            if (pDriver->pTRAYS[i].ulTrayID == pDevice->pulTRAYS[j])
            {
               TrayNameFromID (pDriver,
                               pDevice,
                               szTempName,
                               pDriver->pTRAYS[i].ulTrayID);

               if (0 == strcmp (pDJPForm->szTrayname, szTempName))
               {
                  pTrayInfo = (PTRAYINFO) &pDriver->pTRAYS[i];
                  break;
               }
            }
         }
      }
   }
```

```
   if (pDJPForm->szMedianame[0])
   {
      ulNumDevice  = pDevice->ulNumMedias;
      ulNumDefined = pDriver->ulNumMedias;

      for (i = 0; i < ulNumDefined && !pMediaInfo; i++)
      {
         for (j = 0; j < ulNumDevice; j++)
         {
            if (pDriver->pMEDIAS[i].ulMediaID == pDevice->pulMEDIAS[j])
            {
               MediaNameFromID (pDriver,
                                pDevice,
                                szTempName,
                                pDriver->pMEDIAS[i].ulMediaID);

               if (0 == strcmp (pDJPForm->szMedianame, pszTempName))
               {
                  pMediaInfo = (PMEDIAINFO) &pDriver->pMEDIAS[i];
                  break;
               }
            }
         }
      }
   }

   if (pFormInfo)
   {
      ULONG  ulDefaultConnID  = pJobProp->ulDefConnID,
             ulDefaultFormID  = 0,
             ulDefaultTrayID  = 0,
             ulDefaultMediaID = 0;

      // Ask for the current connection id
      bOK = GetIDsFromConnID (pDriver,
                              pDevice,
                              ulDefaultConnID,
                              &ulDefaultTrayID,
                              &ulDefaultFormID,
                              &ulDefaultMediaID);

      ulDefaultFormID = pFormInfo->ulFormID;
      if (pTrayInfo)
         ulDefaultTrayID = pTrayInfo->ulTrayID;
      if (pMediaInfo)
         ulDefaultMediaID = pMediaInfo->ulMediaID;

      bOK = NewConnection (pdb,
                           pGoodDrivData->szDeviceName,
                           &ulDefaultConnID,
                           ulDefaultFormID,
                           ulDefaultTrayID,
                           ulDefaultMediaID,
                           FALSE);           // Dont create a new one!
   }
   else
      bOK = FALSE;

   if (bOK)
      pJobProp->ulDefConnID = ulDefaultConnID;
   else
      pISet->lType = DJP_ERROR_OUT_OF_RANGE;
   break;
}

case DJP_SJ_COPIES:
{
   if (0 < pISet->ulValue && pISet->ulValue < 100)
      pJobProp->ulCopies = pISet->ulValue;
   else
      pISet->lType = DJP_ERROR_OUT_OF_RANGE;
   break;
}

case DJP_SJ_PRINTQUALITY:
case DJP_SJ_TRAYTYPE:
case DJP_SJ_MEDIA:
case DJP_SJ_MEDIA_COLOR:
case DJP_CJ_MIXEDFORMS:
case DJP_SJ_FONTDOWNLOADING:
case DJP_SJ_DUPLEX:
case DJP_SJ_COLLATE:
```

```
         case DJP_SJ_FEED:
         case DJP_SJ_SCALING:
         case DJP_SJ_FORMFEEDCONTROL:
         case DJP_SJ_N_UP:
         default:
            pISet->lType = DJP_ERROR_NOT_SUPPORTED;
            DBPRINTF (("Unknow set '%s' = %d!\n",
                       pszProperty (pISet->ulProperty),
                       pISet->ulProperty));
            break;
         }

         // Move to the set item
         pISet = DJP_NEXT_STRUCTP (pISet);

      } while (pISet->ulProperty != DJP_NONE);

      cbDrivData = cbParm2;
      pDrivData  = (PDRIVDATA)pbParm2;

      /* Copy the updated job properties onto the user's job properties
      */
      ulMaxSize = min (pGoodDrivData->cb, cbDrivData);
      memcpy (pDrivData, pGoodDrivData, ulMaxSize);
      pDrivData->cb = ulMaxSize;

      if (fError)
      {
         GplErrSetWarning (PMERR_DATATYPE_ENTRY_INVALID);
         ulrc = DEV_ERROR;
      }
      else
      {
         ulrc = DEV_OK;
      }
      break;
   }
   }

   return lrc;
}
```

GreEscape DEVESC_SETJOBPROPERTIES - Topics

Select an item:

Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_SETMODE

GreEscape DEVESC_SETMODE - Syntax

Simulation support:

This function is mandatory for hardcopy drivers, but may simply return a DEVESC_NOTIMPLEMENTED code. For other drivers, it is

simulated by a handling routine in the graphics engine.

Description:

GreEscape DEVESC_SETMODE sets the printer into a particular mode.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_SETMODE escape code. */
LONG     cInCount;     /*  Number of bytes pointed to by pInData. */
PBYTE    pInData;      /*  Pointer to the buffer that contains an ESCMODE structure. */
PLONG    pcOutCount;   /*  The handling routine ignores this parameter. */
PLONG    pOutData;     /*  The handling routine ignores this parameter. */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreEscape. */
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_SETMODE Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_SETMODE Parameter - lEscape

lEscape (LONG) - input
DEVESC_SETMODE escape code.

GreEscape DEVESC_SETMODE Parameter - cInCount

cInCount (LONG) - input
Number of bytes pointed to by pInData.

GreEscape DEVESC_SETMODE Parameter - pInData

pInData (PBYTE) - input
Pointer to the buffer that contains an ESCMODE structure.

GreEscape DEVESC_SETMODE Parameter - pcOutCount

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_SETMODE Parameter - pOutData

pOutData (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_SETMODE Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_SETMODE Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_SETMODE Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_SETMODE - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_SETMODE escape code.

cInCount (LONG) - input
Number of bytes pointed to by pInData.

pInData (PBYTE) - input
Pointer to the buffer that contains an ESCMODE structure.

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

pOutData (PLONG) - input
The handling routine ignores this parameter.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_SETMODE - Remarks

Although it is optional for hardcopy drivers to support this function, those that do support it need to know the code page of any of the built-in fonts. For example, if only Code Page 437 is built-in, it is the code page used if 437 is requested by GreEscape DEVESC_SETMODE. If Code Page 865 is requested, a suitable code page or font could be downloaded.

This escape code is metafiled and recorded.

GreEscape DEVESC_SETMODE - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_SETUPBLITTERNOTIFY

GreEscape DEVESC_SETUPBLITTERNOTIFY - Syntax

Simulation support:

None.

Description

GreEscape DEVESC_SETUPBLITTERNOTIFY is called by EnDIVE users to indicate whether the hardware can support the current image blitting parameters. If the hardware cannot support the blitting parameters for some reason (such as stretching or shrinking), the *ulFlags* of the SETUPBLITTERNOTIFYOUT data structure should be set. In such cases, the hardware will revert to software for blitting operation. If the user does not support this function, it is assumed that the hardware can handle the blitter operation being set.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     ICODE;        /*  DEVESC_SETUPBLITTERNOTIFY escape code. */
LONG     lInCount;     /*  Number of bytes pointed to by pbInData. */
PBYTE    pbInData;     /*  Pointer to SETUPBLITTERNOTIFYIN data structure. */
PLONG    plOutCount;   /*  Number of bytes pointed to by pbOutData. */
PBYTE    pbOutData;    /*  Pointer to SETUPBLITTERNOTIFYOUT data structure. */
LONG     rc;           /*  Return Codes. */

rc = GreEscape(hdc, ICODE, lInCount, pbInData,
       plOutCount, pbOutData);
```

GreEscape DEVESC_SETUPBLITTERNOTIFY Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_SETUPBLITTERNOTIFY Parameter - ICODE

ICODE (LONG) - input
DEVESC_SETUPBLITTERNOTIFY escape code.

GreEscape DEVESC_SETUPBLITTERNOTIFY Parameter - lInCou

lInCount (LONG) - input
Number of bytes pointed to by pbInData.

GreEscape DEVESC_SETUPBLITTERNOTIFY Parameter - pbInDa

pbInData (PBYTE) - input
Pointer to SETUPBLITTERNOTIFYIN data structure.

GreEscape DEVESC_SETUPBLITTERNOTIFY Parameter - plOutC

plOutCount (PLONG) - input
Number of bytes pointed to by pbOutData.

GreEscape DEVESC_SETUPBLITTERNOTIFY Parameter - pbOutD

pbOutData (PBYTE) - output
Pointer to SETUPBLITTERNOTIFYOUT data structure.

GreEscape DEVESC_SETUPBLITTERNOTIFY Return Value - rc

rc (LONG) - returns
Return Codes.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code.
DEVESC_ERROR	Error

GreEscape DEVESC_SETUPBLITTERNOTIFY - Parameters

hdc (HDC) - input
Device context handle.

lCODE (LONG) - input
DEVESC_SETUPBLITTERNOTIFY escape code.

lInCount (LONG) - input
Number of bytes pointed to by pbInData.

pbInData (PBYTE) - input
Pointer to SETUPBLITTERNOTIFYIN data structure.

plOutCount (PLONG) - input
Number of bytes pointed to by pbOutData.

pbOutData (PBYTE) - output
Pointer to SETUPBLITTERNOTIFYOUT data structure.

rc (LONG) - returns
Return Codes.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	

Error

GreEscape DEVESC_SETUPBLITTERNOTIFY - Remarks

None.

GreEscape DEVESC_SETUPBLITTERNOTIFY - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_STARTDOC

GreEscape DEVESC_STARTDOC - Syntax

Simulation support:

This function is mandatory for hardcopy drivers and must be implemented in order to have a valid OS/2 hardcopy driver. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreEscape DEVESC_STARTDOC starts a new document. The handling routine in the presentation driver does whatever initialization is required to spool or print the document.

Note: DEVESC_STARTDOC is mandatory at the API for an OD_QUEUED device with PM_Q_STD data.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_STARTDOC escape code. */
LONG     cInCount;     /*  Number of bytes pointed to by pInData. */
PBYTE    pInData;      /*  Pointer to a string containing the name of the document. */
PLONG    pcOutCount;   /*  The handling routine ignores this parameter. */
PLONG    pOutData;     /*  The handling routine ignores this parameter. */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreEscape. */
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_STARTDOC Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_STARTDOC Parameter - lEscape

lEscape (LONG) - input
DEVESC_STARTDOC escape code.

GreEscape DEVESC_STARTDOC Parameter - cInCount

cInCount (LONG) - input
Number of bytes pointed to by pInData.

GreEscape DEVESC_STARTDOC Parameter - pInData

pInData (PBYTE) - input
Pointer to a string containing the name of the document.

GreEscape DEVESC_STARTDOC Parameter - pcOutCount

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_STARTDOC Parameter - pOutData

pOutData (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_STARTDOC Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_STARTDOC Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_STARTDOC Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_STARTDOC - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_STARTDOC escape code.

cInCount (LONG) - input
Number of bytes pointed to by pInData.

pInData (PBYTE) - input
Pointer to a string containing the name of the document.

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

pOutData (PLONG) - input
The handling routine ignores this parameter.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code

---

GreEscape DEVESC_STARTDOC - Remarks

The driver continues to spool or print data until it receives DEVESC_ENDDOC. This ensures that documents longer than one page are not interspersed with other jobs.

Because DEVESC_STARTDOC is not mandatory at the API, it cannot be assumed that an application (when opening a DC for printing) will pass DEVESC_STARTDOC to the presentation driver. In this case, the presentation driver must initialize a spool file with no name or a journal file in the *CompleteOpenDC* handling routine. If this is done, the presentation driver should set a flag so that initialization is not repeated if DEVESC_STARTDOC is received. Notice that a handling routine is required for DEVESC_STARTDOC to save the specified document name in the DC instance data.

When this function call is issued for an OD_QUEUED device, the presentation driver must start the recording of data into the spool file by calling SplStdStart. It should also call SplQmStartDoc to pass the name of the spool file to the visual spooler. See SplStdStart and SplQmStartDoc.

This escape code is metafiled but not recorded.

GreEscape DEVESC_STARTDOC - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_STARTDOC_WPROP

GreEscape DEVESC_STARTDOC_WPROP - Syntax

Simulation support:

This function is mandatory for hardcopy drivers and must be implemented in order to have a valid OS/2 hardcopy driver.

Refer to Dynamic Job Properties for related information.

Description:

GreEscape DEVESC_STARTDOC_WPROP will reset the device context information and presentation space information to be based on the new job properties that are passed in. This should be equivalent to the result of a new DevOpenDC with the new job properties.

```
#define INCL_DEV
#define INCL_DEVDJP
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_STARTDOC_WPROP escape code. */
```

```
LONG     cInCount;     /*  Number of bytes pointed to by pInData. */
PBYTE    pInData;      /*  The title of the document (same as GreEscape DEVESC_STARTDOC). */
PLONG    pcOutCount;   /*  Pointer to the number of bytes pointed to by pOutData. */
PLONG    pOutData;     /*  Pointer to the printer driver's job properties as returned by SplQueryQueue. */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreEscape. */
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_STARTDOC_WPROP Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_STARTDOC_WPROP Parameter - lEscape

lEscape (LONG) - input
DEVESC_STARTDOC_WPROP escape code.

GreEscape DEVESC_STARTDOC_WPROP Parameter - cInCount

cInCount (LONG) - input
Number of bytes pointed to by *pInData* .

GreEscape DEVESC_STARTDOC_WPROP Parameter - pInData

pInData (PBYTE) - input
The title of the document (same as GreEscape DEVESC_STARTDOC).

GreEscape DEVESC_STARTDOC_WPROP Parameter - pcOutCou

pcOutCount (PLONG) - input
Pointer to the number of bytes pointed to by *pOutData* .

GreEscape DEVESC_STARTDOC_WPROP Parameter - pOutData

pOutData (PLONG) - input
Pointer to the printer driver's job properties as returned by SplQueryQueue.

These job properties may also have been modified by GreEscape DEVESC_SETJOBPROPERTIES.

GreEscape DEVESC_STARTDOC_WPROP Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_STARTDOC_WPROP Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_STARTDOC_WPROP Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK
Successful

DEVESC_ERROR
Error

GreEscape DEVESC_STARTDOC_WPROP - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_STARTDOC_WPROP escape code.

cInCount (LONG) - input
Number of bytes pointed to by *pInData* .

pInData (PBYTE) - input
The title of the document (same as GreEscape DEVESC_STARTDOC).

pcOutCount (PLONG) - input
Pointer to the number of bytes pointed to by *pOutData* .

pOutData (PLONG) - input
Pointer to the printer driver's job properties as returned by SplQueryQueue.

These job properties may also have been modified by GreEscape DEVESC_SETJOBPROPERTIES.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK
Successful

DEVESC_ERROR
Error

GreEscape DEVESC_STARTDOC_WPROP - Remarks

DEVESC_STARTDOC_WPROP is used to change the job properties at the beginning of a document. Since this function is *new* , applications will call GreEscape DEVESC_QUERYESCSUPPORT to determine if you support this DevEscape.

The DevEscape path has a handle to a device context (HDC) that should already have printer property information associated with the HDC. Job property information may change or may be dependent upon the printer property information in the HDC. In that case, if an application passes job properties for a different device or for a different spooler printer name, then misleading information will be returned.

This function will change all characteristics of the HDC and PS. An application *must* query again information such as DevQueryCaps, DevQueryHardcopyCaps, the list of device fonts, and so on. In addition, all previously opened memory DCs are no longer considered compatible with the DC.

You also should check the version level of the graphics engine, since GreResetDC2 is a new API and only exists on engine levels 0x0234 and above. If you are using an older version level, then treat DEVESC_STARTDOC_WPROP as a DEVESC_STARTDOC.

This function also performs the same actions as calling DEVESC_STARTDOC.

```
LONG ENGENTRY Escape (HDC       hdc,
                      LONG      lEscape,
                      LONG      cInCount,
                      PBYTE     pInData,
                      PLONG     pcOutCount,
                      PBYTE     pOutData,
                      PDDC      pddc,
                      ULONG     ulFunction)
{
   switch (lEscape)
   {
   case DEVESC_STARTDOC:
   case DEVESC_STARTDOC_WPROP:
   {
      if (GRE_234 > globals.ulGreVersion)
      {
         /* Switch DEVESC_STARTDOC_WPROP to DEVESC_STARTDOC
         ** on back level versions of the engine.
         */
         lEscape = DEVESC_STARTDOC;
         break;
      }

      if (DEVESC_STARTDOC_WPROP == lEscape)
      {
         /* This does a number of things such as:
         **   1) Sanity check input driver data
         **   2) Walk through new job properties and set up
         **      DC information (papersize, orientation, color depth, etc)
         **   3) Set up device surface structure
```

```
         */
         ReinitializeWithJobProps (pddc, (PDRIVDATA)pOutData);
      }

      // For Raw Data only jobs do not need to reset device surface
      if (!pddc->pdb->fRawOnly)
         /* This does a number of things such as:
         **   1) Allocate the device surface memory
         **   2) Call engine's SetDeviceSurface function.
         */
         CreateAndSetDeviceSurface (pddc);

      if (DEVESC_STARTDOC_WPROP == lEscape)
      {
         /* Inform the engine that your characteristics have changed
         ** and it should requery the information.
         */
         GreResetDC2 (pddc->pdb->hdc, 0);
      }

      // Normal STARTDOC processing follows...
      break;
   }
   }

   return lrc;
}
```

GreEscape DEVESC_STARTDOC_WPROP - Topics

Select an item:

Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_STD_JOURNAL

GreEscape DEVESC_STD_JOURNAL - Syntax

Simulation support:

This function is mandatory for hardcopy drivers, but may simply return a DEVESC_NOTIMPLEMENTED code. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreEscape DEVESC_STD_JOURNAL sends a standard journal file to the presentation driver.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lEscape;      /*  DEVESC_STD_JOURNAL escape code. */
```

```
LONG     cInCount;     /*  Number of bytes pointed to by pInData. */
PBYTE    pInData;      /*  Pointer to journal data. */
PLONG    pcOutCount;   /*  The handling routine ignores this parameter. */
PLONG    pOutData;     /*  The handling routine ignores this parameter. */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreEscape. */
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lEscape, cInCount, pInData,
       pcOutCount, pOutData, pInstance, lFunction);
```

GreEscape DEVESC_STD_JOURNAL Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_STD_JOURNAL Parameter - lEscape

lEscape (LONG) - input
DEVESC_STD_JOURNAL escape code.

GreEscape DEVESC_STD_JOURNAL Parameter - cInCount

cInCount (LONG) - input
Number of bytes pointed to by pInData.

GreEscape DEVESC_STD_JOURNAL Parameter - pInData

pInData (PBYTE) - input
Pointer to journal data.

GreEscape DEVESC_STD_JOURNAL Parameter - pcOutCount

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_STD_JOURNAL Parameter - pOutData

pOutData (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_STD_JOURNAL Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreEscape DEVESC_STD_JOURNAL Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

GreEscape DEVESC_STD_JOURNAL Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_STD_JOURNAL - Parameters

hdc (HDC) - input
Device context handle.

lEscape (LONG) - input
DEVESC_STD_JOURNAL escape code.

cInCount (LONG) - input
Number of bytes pointed to by pInData.

pInData (PBYTE) - input
Pointer to journal data.

pcOutCount (PLONG) - input
The handling routine ignores this parameter.

pOutData (PLONG) - input
The handling routine ignores this parameter.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreEscape.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_STD_JOURNAL - Remarks

Note: This escape is implemented by old hardcopy drivers and is being phased out.

GreEscape DEVESC_STD_JOURNAL - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_SWITCHBANK

GreEscape DEVESC_SWITCHBANK - Syntax

Simulation support:

This function is mandatory for display drivers supporting software motion video. For other drivers, it is simulated by a handling routine in the graphics engine.

Description

GreEscape DEVESC_SWITCHBANK changes which bank of video memory is mapped to the aperture that is returned by DEVESC_GETAPERTURE. The escape is intended to allow the calling routine to switch banks in a device-independent fashion, and then access video memory directly.

This escape must be bracketed by calls to DEVESC_ACQUIREFB and DEVESC_DEACQUIREFB. These calls ensure serial access to the bank-switching resources of the device.

Currently, this call will only be used on adapters configured with apertures smaller than the visible video memory. Adapters with adequately

sized apertures do not require the calling routine to acquire and deacquire the device in order to share the bank-switching resource.

This call is required for adapters that have a 64KB aperture in the 256-color mode. An XGA adapter has a 64KB aperture mode and most Super VGA adapters also use a 64KB aperture.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lCode;        /*  DEVESC_SWITCHBANK escape code. */
LONG     lInCount;     /*  Number of bytes pointed to by pInData. */
PBYTE    pbInData;     /*  Pointer to a SWITCHBANK structure containing: */
PLONG    plOutCount;   /*  The handling routine ignores this parameter. */
PLONG    pbOutData;    /*  The handling routine ignores this parameter. */
LONG     rc;           /*  Return Code. */

rc = GreEscape(hdc, lCode, lInCount, pbInData,
       plOutCount, pbOutData);
```

GreEscape DEVESC_SWITCHBANK Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_SWITCHBANK Parameter - lCode

lCode (LONG) - input
DEVESC_SWITCHBANK escape code.

GreEscape DEVESC_SWITCHBANK Parameter - lInCount

lInCount (LONG) - input
Number of bytes pointed to by pInData.

GreEscape DEVESC_SWITCHBANK Parameter - pbInData

pbInData (PBYTE) - input
Pointer to a SWITCHBANK structure containing:

ulBankNum	Specifies the aperture bank to be made active: zero (0) for the first bank, one (1) for the second, and so on.

GreEscape DEVESC_SWITCHBANK Parameter - plOutCount

plOutCount (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_SWITCHBANK Parameter - pbOutData

pbOutData (PLONG) - input
The handling routine ignores this parameter.

GreEscape DEVESC_SWITCHBANK Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_SWITCHBANK - Parameters

hdc (HDC) - input
Device context handle.

lCode (LONG) - input
DEVESC_SWITCHBANK escape code.

lInCount (LONG) - input
Number of bytes pointed to by pInData.

pbInData (PBYTE) - input
Pointer to a SWITCHBANK structure containing:

ulBankNum	Specifies the aperture bank to be made active: zero (0) for the first bank, one (1) for the second, and so on.

plOutCount (PLONG) - input
The handling routine ignores this parameter.

pbOutData (PLONG) - input
The handling routine ignores this parameter.

rc (LONG) - returns
Return Code.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape not implemented for specified code
DEVESC_ERROR	Error

GreEscape DEVESC_SWITCHBANK - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreEscape DEVESC_VIDEOHIDDEN

GreEscape DEVESC_VIDEOHIDDEN - Syntax

Simulation support:

None.

Description

GreEscape DEVESC_VIDEOHIDDEN is called by EnDIVE users to specify that the video is hidden whenever the video is completely occluded (for example, clipped out, minimized, and so on). This will give the driver a chance to turn off video acceleration if necessary.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle */
LONG     ICODE;        /*  DEVESC_VIDEOHIDDEN escape code */
LONG     lInCount;     /*  sizeof(ULONG) */
PUCHAR   pbInData;     /*  fBlitterEnabled  (TRUE or FALSE) */
PLONG    plOutCount;   /*  Zero  */
PBYTE    pbOutData;    /*  Zero  */
LONG     rc;           /*  Return Codes */

rc = GreEscape(hdc, ICODE, lInCount, pbInData,
       plOutCount, pbOutData);
```

GreEscape DEVESC_VIDEOHIDDEN Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_VIDEOHIDDEN Parameter - ICODE

lCODE (LONG) - input
DEVESC_VIDEOHIDDEN escape code.

GreEscape DEVESC_VIDEOHIDDEN Parameter - lInCount

lInCount (LONG) - input
Number of bytes pointed to by pbInData.

GreEscape DEVESC_VIDEOHIDDEN Parameter - pbInData

pbInData (PUCHAR) - input
Pointer to fBlitterEnabled.

GreEscape DEVESC_VIDEOHIDDEN Parameter - plOutCount

plOutCount (PLONG) - input
Number of bytes pointed to by pbOutData.

GreEscape DEVESC_VIDEOHIDDEN Parameter - pbOutData

pbOutData (PBYTE) - output
Pointer to VRAMREGISTEROUT data structure.

GreEscape DEVESC_VIDEOHIDDEN Return Value - rc

rc (LONG) - returns
Return Codes.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code.
DEVESC_ERROR	Error

GreEscape DEVESC_VIDEOHIDDEN - Parameters

hdc (HDC) - input
Device context handle.

lCODE (LONG) - input
DEVESC_VIDEOHIDDEN escape code.

lInCount (LONG) - input
Number of bytes pointed to by pbInData.

pbInData (PUCHAR) - input
Pointer to fBlitterEnabled.

plOutCount (PLONG) - input
Number of bytes pointed to by pbOutData.

pbOutData (PBYTE) - output
Pointer to VRAMREGISTEROUT data structure.

rc (LONG) - returns
Return Codes.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	Error

GreEscape DEVESC_VIDEOHIDDEN - Remarks

EnDIVE calls a driver escape (PutImage) for every frame to be put on the screen. When the window is completely obscured or minimized, there are no calls into the driver, even if the video continues to play. For devices that are blitting to the screen, this sequence is acceptable. However, for overlay devices, the video is still on. If another window uses the driver color key color in the area, there will be bleed through. Furthermore, there exists certain situations where RGB and YUV data can be present simultaneously in the frame buffer. If the YUV video data is overpainted by RGB graphics data without adjustments to the data mapping, some of the RGB data will be misinterpreted and cause large patches of video noise on the screen. DEVESC_VIDEOHIDDEN informs the driver that the video is hidden, so that the driver has a chance to turn off video acceleration if necessary.

GreEscape DEVESC_VIDEOHIDDEN - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreEscape DEVESC_VRAMALLOC

GreEscape DEVESC_VRAMALLOC - Syntax

Simulation support:

None.

Description

GreEscape DEVESC_VRAMALLOC is called by EnDIVE users to allocate off-screen video memory, into which the function puts images. The driver is responsible for allocating and deallocating memory from its available video memory. The VRAMALLOCIN data structure contains a function code (*ulFunction*), which specifies allocation or deallocation. If video memory is not supported or available, the EnDIVE user is responsible for allocating system memory used to hold the image for output by the driver.

Two types of memory allocation can be requested: shared or private. **Shared** memory reuses memory that is shared among all EnDIVE users in the system. Users of shared memory must assume that it is valid only when bracketed betweenDEVESC_HWREQUEST calls. **Private** or unshared memory is assumed by the EnDIVE user to be retained solely for its use until the memory is deallocated.

For the user of GreEscape DEVESC_VRAMALLOC, the information in GreEscape DEVESC_EXTQUERY is assumed to correctly describe off-screen VRAM. However, a duplicate *ulScanLineBytes* field is provided here in GreEscape DEVESC_VRAMALLOC, in case the memory allocated by this function has a different scanline length.

The *ulSize* field in the VRAMALLOCIN data structure is modified by the display driver to reflect the exact number of bytes allocated (will be DWORD aligned). A nonzero return code indicates the allocation failed.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     ICODE;        /*  DEVESC_VRAMALLOC escape code. */
LONG     lInCount;     /*  Number of bytes pointed to by pbInData. */
PBYTE    pbInData;     /*  Pointer to VRAMALLOCIN data structure. */
PLONG    plOutCount;   /*  Number of bytes pointed to by pbOutData. */
PLONG    pbOutData;    /*  Pointer to VRAMALLOCOUT data structure. */
LONG     rc;           /*  Return Codes. */

rc = GreEscape(hdc, ICODE, lInCount, pbInData,
       plOutCount, pbOutData);
```

GreEscape DEVESC_VRAMALLOC Parameter - hdc

hdc (HDC) - input
Device context handle.

GreEscape DEVESC_VRAMALLOC Parameter - ICODE

ICODE (LONG) - input
DEVESC_VRAMALLOC escape code.

GreEscape DEVESC_VRAMALLOC Parameter - lInCount

lInCount (LONG) - input
Number of bytes pointed to by pbInData.

GreEscape DEVESC_VRAMALLOC Parameter - pbInData

pbInData (PBYTE) - input
Pointer to VRAMALLOCIN data structure.

GreEscape DEVESC_VRAMALLOC Parameter - plOutCount

plOutCount (PLONG) - input
Number of bytes pointed to by pbOutData.

GreEscape DEVESC_VRAMALLOC Parameter - pbOutData

pbOutData (PLONG) - input
Pointer to VRAMALLOCOUT data structure.

GreEscape DEVESC_VRAMALLOC Return Value - rc

rc (LONG) - returns
Return Codes.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	Error
	WINGETLASTERROR can be called for specific error information.

GreEscape DEVESC_VRAMALLOC - Parameters

hdc (HDC) - input
Device context handle.

lCODE (LONG) - input
DEVESC_VRAMALLOC escape code.

lInCount (LONG) - input
Number of bytes pointed to by pbInData.

pbInData (PBYTE) - input
Pointer to VRAMALLOCIN data structure.

plOutCount (PLONG) - input
Number of bytes pointed to by pbOutData.

pbOutData (PLONG) - input
Pointer to VRAMALLOCOUT data structure.

rc (LONG) - returns
Return Codes.

The handling routine returns:

DEV_OK	Successful
DEVESC_NOTIMPLEMENTED	Escape is not implemented for the specified code
DEVESC_ERROR	Error

WINGETLASTERROR can be called for specific error information.

GreEscape DEVESC_VRAMALLOC - Remarks

None.

GreEscape DEVESC_VRAMALLOC - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreExcludeClipRectangle

GreExcludeClipRectangle - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreExcludeClipRectangle excludes the specified rectangle from the clipping region, that is, the area covered by the rectangle will be outside the resulting clip region. If COM_TRANSFORM is not set, the function expects the rectangle points to be in device coordinates. GreExcludeClipRectangle creates a clip region when none exists. The application is responsible for deleting this clip region when it is finished. Otherwise, it is not deleted until the DC is closed.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_CLIP
#include <os2.h>

HDC       hdc;        /*  Device context handle. */
PRECTL    prclRect;   /*  Pointer to rectangle in world or device coordinates. */
PVOID     pInstance;  /*  Pointer to instance data. */
ULONG     lFunction;  /*  High-order WORD=flags; low-order  WORD=NGreExcludeClipRectangle. */
LONG      rc;         /*  Return codes. */

rc = GreExcludeClipRectangle(hdc, prclRect,
       pInstance, lFunction);
```

GreExcludeClipRectangle Parameter - hdc

hdc (HDC) - input
Device context handle.

GreExcludeClipRectangle Parameter - prclRect

prclRect (PRECTL) - input
Pointer to rectangle in world or device coordinates.

RECTL structure:

xLeft	Minimum x-coordinate of rectangle
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of rectangle
yTop	Maximum y-coordinate

All the boundaries of this rectangle are considered to be part of the interior and are clipped accordingly.

GreExcludeClipRectangle Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreExcludeClipRectangle Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreExcludeClipRectangle.

GreExcludeClipRectangle Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (lComplexity) indicating the complexity of the DC region:

RGN_ERROR	Error
RGN_NULL	Null region
RGN_RECT	Rectangular region
RGN_COMPLEX	Complex region (more than 1 rectangle)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreExcludeClipRectangle - Parameters

hdc (HDC) - input
Device context handle.

prclRect (PRECTL) - input
Pointer to rectangle in world or device coordinates.

RECTL structure:

xLeft	Minimum x-coordinate of rectangle
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of rectangle
yTop	Maximum y-coordinate

All the boundaries of this rectangle are considered to be part of the interior and are clipped accordingly.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreExcludeClipRectangle.

rc (LONG) - returns

Return codes.

This function returns an integer (lComplexity) indicating the complexity of the DC region:

RGN_ERROR	Error
RGN_NULL	Null region
RGN_RECT	Rectangular region
RGN_COMPLEX	Complex region (more than 1 rectangle)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreExcludeClipRectangle - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreFillPath

GreFillPath - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreFillPath fills the interior of the closed figure defined in the path by using the current pattern attributes. Before filling the path, the handling routine must close any open figures in the path definition. On completion, it must delete the path. All of the boundaries of the area are considered to be part of the interior and are filled.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_PATHS
#include <os2.h>
```

```
HDC      hdc;         /*  Device context handle. */
LONG     idPath;      /*  Path ID.  It must be 1. */
ULONG    flOptions;   /*  Option flags. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreFillPath. */
LONG     rc;          /*  Return codes. */

rc = GreFillPath(hdc, idPath, flOptions, pInstance,
       lFunction);
```

GreFillPath Parameter - hdc

hdc (HDC) - input
Device context handle.

GreFillPath Parameter - idPath

idPath (LONG) - input
Path ID. It must be 1.

GreFillPath Parameter - flOptions

flOptions (ULONG) - input
Option flags.

These flags determine how the path is to be filled:

FPATH_ALTERNATE	Fill is performed by using the odd/even (alternate) rule.
FPATH_WINDING	Fill is performed by using the winding rule.
FPATH_EXCL	Fill exclusive of bottom and right border in device coordinate.

GreFillPath Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreFillPath Parameter - lFunction

lFunction (ULONG) - input

High-order WORD=flags; low-order WORD=NGreFillPath.

GreFillPath Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_FILL_PATH_OPTIONS
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PATH_ID
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreFillPath - Parameters

hdc (HDC) - input
Device context handle.

idPath (LONG) - input
Path ID. It must be 1.

flOptions (ULONG) - input
Option flags.

These flags determine how the path is to be filled:

FPATH_ALTERNATE	Fill is performed by using the odd/even (alternate) rule.
FPATH_WINDING	Fill is performed by using the winding rule.
FPATH_EXCL	Fill exclusive of bottom and right border in device coordinate.

pInstance (PVOID) - input

Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreFillPath.

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_FILL_PATH_OPTIONS
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PATH_ID
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreFillPath - Remarks

There is an alternate method of filling the interior of closed figures for devices that have their own hardware filling; see GreOutlinePath for more information.

GreFillPath - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreFullArcBoth

GreFullArcBoth - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreFullArcBoth draws and fills a full arc centered on the current (X,Y) position. The current line attributes apply to the boundary line, and the current pattern attributes apply to the interior. The dimensions of the arc are defined as a multiplier that is applied to the current arc parameters. GreFullArcBoth does not affect the current position. When the COM_PATH or the COM_AREA flag is set, this function must raise an error.

When correlating, the handling routine must return a hit when the pick aperture intersects the boundary, or is completely within the interior, even when the mix is LEAVEALONE. See Mix Modes.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_ARCS
#include <os2.h>

HDC      hdc;           /*  Device context handle. */
FIXED    fxMultiplier;  /*  Multiplier. */
PVOID    pInstance;     /*  Pointer to instance data. */
ULONG    lFunction;     /*  High-order WORD=flags; low-order WORD=NGreFullArcBoth. */
LONG     rc;            /*  Return codes. */

rc = GreFullArcBoth(hdc, fxMultiplier, pInstance,
       lFunction);
```

GreFullArcBoth Parameter - hdc

hdc (HDC) - input
Device context handle.

GreFullArcBoth Parameter - fxMultiplier

fxMultiplier (FIXED) - input
Multiplier.

The value of this parameter defines the size of the required arc in relation to an arc drawn with the current arc parameters. The multiplier is a fixed-point value. The high-order WORD contains the integer portion; the low-order WORD contains the fractional portion. A value of 64KB gives a multiplier of 1. The implementation limit of the multiplier is 255. This value must not be negative.

GreFullArcBoth Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreFullArcBoth Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreFullArcBoth.

GreFullArcBoth Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_ALREADY_IN_AREA
PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_AREA_CONTROL
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_CHAR_DIRECTION_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_MIX_ATTR
PMERR_INV_MULTIPLIER
PMERR_INV_NESTED_FIGURES
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

PMERR_NOT_IN_AREA
PMERR_NOT_IN_PATH
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreFullArcBoth - Parameters

hdc (HDC) - input
Device context handle.

fxMultiplier (FIXED) - input
Multiplier.

The value of this parameter defines the size of the required arc in relation to an arc drawn with the current arc parameters. The multiplier is a fixed-point value. The high-order WORD contains the integer portion; the low-order WORD contains the fractional portion. A value of 64KB gives a multiplier of 1. The implementation limit of the multiplier is 255. This value must not be negative.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreFullArcBoth.

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_ALREADY_IN_AREA
PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_AREA_CONTROL
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_CHAR_DIRECTION_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_MIX_ATTR
PMERR_INV_MULTIPLIER
PMERR_INV_NESTED_FIGURES

PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_NOT_IN_AREA
PMERR_NOT_IN_PATH
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreFullArcBoth - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreFullArcBoundary

GreFullArcBoundary - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreFullArcBoundary draws a line by using the current line attributes around the edge of a full arc centered on the current (X,Y) position. The dimensions of the arc are defined as a multiplier that is applied to the current arc parameters. GreFullArc*xxx* functions do not affect the current position.

When correlating, the handling routine must return a hit when the pick aperture intersects the boundary. See Mix Modes.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_ARCS
#include <os2.h>

HDC      hdc;           /*  Device context handle. */
FIXED    fxMultiplier;  /*  Multiplier. */
PVOID    pInstance;     /*  Pointer to instance data. */
ULONG    lFunction;     /*  High-order WORD=flags; low-order WORD=NGreFullArcBoundary. */
LONG     rc;            /*  Return codes. */

rc = GreFullArcBoundary(hdc, fxMultiplier,
       pInstance, lFunction);
```

GreFullArcBoundary Parameter - hdc

hdc (HDC) - input
Device context handle.

GreFullArcBoundary Parameter - fxMultiplier

fxMultiplier (FIXED) - input
Multiplier.

The value of this parameter defines the size of the required arc in relation to an arc drawn with the current arc parameters. The multiplier is a fixed-point value. The high-order WORD contains the integer portion; the low-order WORD contains the fractional portion. A value of 64KB gives a multiplier of 1. The implementation limit of the multiplier is 255. This value must not be negative.

GreFullArcBoundary Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreFullArcBoundary Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreFullArcBoundary.

GreFullArcBoundary Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MULTIPLIER
PMERR_INV_NESTED_FIGURES
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_NOT_IN_PATH
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreFullArcBoundary - Parameters

hdc (HDC) - input
Device context handle.

fxMultiplier (FIXED) - input
Multiplier.

The value of this parameter defines the size of the required arc in relation to an arc drawn with the current arc parameters. The multiplier is a fixed-point value. The high-order WORD contains the integer portion; the low-order WORD contains the fractional portion. A value of 64KB gives a multiplier of 1. The implementation limit of the multiplier is 255. This value must not be negative.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreFullArcBoundary.

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MULTIPLIER
PMERR_INV_NESTED_FIGURES
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_NOT_IN_PATH
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreFullArcBoundary - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreFullArcInterior

GreFullArcInterior - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreFullArcInterior draws a filled, full arc by using the current pattern attributes with its center at the current (X,Y) position. The dimensions of the arc are defined by a multiplier that is applied to the current arc parameters. GreFullArc*xxx* functions do not affect the current position. The arc boundary is not drawn. When the COM_PATH or the COM_AREA flag is set, this function must raise an error.

When correlating, the handling routine must return a hit when the pick aperture intersects the interior, or is completely within the interior, even when the mix is LEAVEALONE. See Mix Modes.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_ARCS
#include <os2.h>

HDC      hdc;           /*  Device context handle. */
FIXED    fxMultiplier;  /*  Multiplier. */
PVOID    pInstance;     /*  Pointer to instance data. */
ULONG    lFunction;     /*  High-order WORD=flags; low-order WORD=NGreFullArcInterior. */
LONG     rc;            /*  Return codes. */

rc = GreFullArcInterior(hdc, fxMultiplier,
       pInstance, lFunction);
```

GreFullArcInterior Parameter - hdc

hdc (HDC) - input
Device context handle.

GreFullArcInterior Parameter - fxMultiplier

fxMultiplier (FIXED) - input
Multiplier.

This parameter defines the size of the required arc in relation to an arc drawn with the current arc parameters. The multiplier is a fixed-point value. The high-order WORD contains the integer portion; the low-order WORD contains the fractional portion. A value of 64KB gives a multiplier of 1. The implementation limit of the multiplier is 255. This value must be positive or 0.

GreFullArcInterior Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreFullArcInterior Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreFullArcInterior.

GreFullArcInterior Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_ALREADY_IN_AREA
PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_AREA_CONTROL
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_CHAR_DIRECTION_ATTR

PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_MIX_ATTR
PMERR_INV_MULTIPLIER
PMERR_INV_NESTED_FIGURES
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_NOT_IN_AREA
PMERR_NOT_IN_PATH
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreFullArcInterior - Parameters

hdc (HDC) - input
Device context handle.

fxMultiplier (FIXED) - input
Multiplier.

This parameter defines the size of the required arc in relation to an arc drawn with the current arc parameters. The multiplier is a fixed-point value. The high-order WORD contains the integer portion; the low-order WORD contains the fractional portion. A value of 64KB gives a multiplier of 1. The implementation limit of the multiplier is 255. This value must be positive or 0.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreFullArcInterior.

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_ALREADY_IN_AREA
PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_HRGN_BUSY

PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_AREA_CONTROL
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_CHAR_DIRECTION_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_MIX_ATTR
PMERR_INV_MULTIPLIER
PMERR_INV_NESTED_FIGURES
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_NOT_IN_AREA
PMERR_NOT_IN_PATH
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreFullArcInterior - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreGetArcParameters

GreGetArcParameters - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreGetArcParameters stores the current arc parameters in the buffer addressed by pArcParms.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_ARCS
#include <os2.h>

HDC           hdc;         /*  Device context handle. */
PARCPARAMS    pArcParms;   /*  Pointer to ARCPARAMS structure. */
PVOID         pInstance;   /*  Pointer to instance data. */
ULONG         lFunction;   /*  High-order WORD=flags; low-order WORD=NGreGetArcParameters. */
BOOL          fSuccess;    /*  Return codes. */

fSuccess = GreGetArcParameters(hdc, pArcParms,
             pInstance, lFunction);
```

GreGetArcParameters Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetArcParameters Parameter - pArcParms

pArcParms (PARCPARAMS) - input
Pointer to ARCPARAMS structure.

GreGetArcParameters Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetArcParameters Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetArcParameters.

GreGetArcParameters Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetArcParameters - Parameters

hdc (HDC) - input
Device context handle.

pArcParms (PARCPARAMS) - input
Pointer to ARCPARAMS structure.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetArcParameters.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetArcParameters - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreGetBitmapBits

GreGetBitmapBits - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreGetBitmapBits can be called for two reasons. If the pointer to application storage (pBitmap) is not NULL, the function transfers bit-map data from a bit map to application storage. If pBitmap is NULL, this function must only fill in the RGB values in the BITMAPINFO or BITMAPINFO2 data structure, which is pointed to by pInfo, and then return the value 0.

This function must be supported by the presentation driver. GreGetBitmapBits is called from GpiQueryBitmapBits, and is used to transfer bit-map data from the device context to application storage. It can be handled by bit-map simulation.

```
#define INCL_GRE_BITMAPS
#include <os2.h>

HDC             hdc;          /*  Device context handle. */
HBITMAP         hbm;          /*  Bit-map handle. */
LONG            lScanStart;   /*  Scan line number from where data transfer starts. */
LONG            cScanCount;   /*  Number of scan lines to be transferred. */
PBYTE           pBitmap;      /*  Pointer to bit-map data or NULL. */
PBITMAPINFO     pInfo;        /*  Pointer to BITMAPINFO or BITMAPINFO2 structure. */
PVOID           pInstance;    /*  Pointer to instance data. */
ULONG           lFunction;    /*  High-order WORD=flags; low-order WORD=NGreGetBitmapBits. */
LONG            rc;           /*  Return Code. */

rc = GreGetBitmapBits(hdc, hbm, lScanStart,
       cScanCount, pBitmap, pInfo, pInstance,
       lFunction);
```

GreGetBitmapBits Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetBitmapBits Parameter - hbm

hbm (HBITMAP) - input
Bit-map handle.

If 0, the DC bit map is used.

GreGetBitmapBits Parameter - lScanStart

lScanStart (LONG) - input

Scan line number from where data transfer starts.

0 is the first scan line number.

GreGetBitmapBits Parameter - cScanCount

cScanCount (LONG) - input
Number of scan lines to be transferred.

GreGetBitmapBits Parameter - pBitmap

pBitmap (PBYTE) - input
Pointer to bit-map data or NULL.

Pointer to the pel data of the bit map. This data is stored in the order that the coordinates appear on a display screen, that is, the pel in the lower-left corner is the first in the bit map. Pels are scanned to the right, and upward, from that position. The bits of the first pel are stored beginning with the most significant bits of the first byte. The data for pels in each scan line is packed together tightly. However, all scan lines are padded at the end so that each one begins on a ULONG boundary. That is, three bytes of pel data will hold one 24-bit pel, three 8-bit pels, six 4-bit pels, or twenty-four 1-bit pels. If those three bytes are the only pel data for that scan line, one more byte of 0s would be required to pad the line to a ULONG boundary.

GreGetBitmapBits Parameter - pInfo

pInfo (PBITMAPINFO) - input
Pointer to BITMAPINFO or BITMAPINFO2 structure.

Pointer to either a BITMAPINFO structure:

cbFix	Length of structure
cx	Bit-map width. This value is updated by the handling routine.
cy	Bit-map height. This value is updated by the handling routine.
cPlanes	Number of color planes, 1 if standard format
cBitCount	Number of adjacent color bits per pel
argbColor[]	Color table array of RGB structures: bBlue bGreen bRed

Or pointer to a BITMAPINFO2 structure:

cbFix	Length of structure	
cx	Bit-map width	
cy	Bit-map height	
cPlanes	Number of color planes, 1 if standard format	
cBitCount	Number of adjacent color bits per pel	
ulCompression	Compression scheme used to store the bit map:	
	BCA_UNCOMP	Bit map is uncomp

	ressed (the only valid value).
cbImage	Length of bit-map storage data in bytes. If the bit map is uncompressed, 0 (default) can be specified for this.
cxResolution	Horizontal component of the resolution of the target device; that is, the resolution of the device the bit map is intended for in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cyResolution	Vertical component of the resolution of the target device; that is, the resolution of the device the bit map is intended for in the units specified by usUnits. This information enables an application to select from a resource group, the bit map that best matches the characteristics of the current output device.
cclrUsed	The number of color indexes from the color table that are used by the bit map. If it is 0 (default), all the indexes are used. If it is not 0, only the first cclrUsed entries in the table are accessed by the system; further entries can be omitted. For standard formats with a cBitCount of 1, 4, or 8 (and cPlanes=1), any indexes beyond cclrUsed are not valid. For example, a bit map with 64 colors can use the 8-bitcount format without having to supply the other 192 entries in the color table. For the 24-bitcount standard format, cclrUsed is the number of colors used by the bit map.
cclrImportant	Minimum number of color indexes for satisfactory appearance of the bit map. More colors can be used in the bit map, but it is not necessary to assign them to the device palette. These additional colors can be mapped to the nearest colors available. Zero (default) means that all entries are important. For a 24-bitcount standard format, the cclrImportant colors are also listed in the color table relating to this bit map.
usUnits	Units of measure of the horizontal and vertical components of resolution: BRU_METRIC (Default) Pels per meter
usReserved	Reserved field. If present, it must be 0.
usRecording	Recording algorithm, the format in which bit-map data is recorded: BRA_BOTTOMUP (Default) Scanlinesar

recordedfrombottomtotop.

usRendering	Half-toning algorithm used to record bit-map data that has been digitally half-toned: BRH_NOTHALFTONED

BRH_PANDA

	BRH_SUPERCIRCLE	
cSize1	Size value 1. If BRH_ERRORDIFFUSION is specified in usRendering, cSize1 is the error damping as a percentage in the range 0 - 100. A value of 100% indicates no damping. A value of 0% indicates that no errors are diffused. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize1 is the X-dimension of the pattern used in pels.	
cSize2	Size value 2. If BRH_ERRORDIFFUSION is specified in usRendering, this parameter is ignored. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize2 is the Y-dimension of the pattern used in pels.	
ulColorEncoding	Color encoding:	
	BCE_RGB	(Default) Each element in the color array is an RGB2 data type.
ulIdentifier	Reserved for application use.	
argbColor[]	Color table array of RGB2 structures:	
	bBlue bGreen bRed fcOptions	 Reserved. Must be 0.

GreGetBitmapBits Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetBitmapBits Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetBitmapBits.

GreGetBitmapBits Return Value - rc

rc (LONG) - returns
Return Code.

On completion, the handling routine must return a LONG value (lLines), indicating the number of lines transferred, or GPI_ALTERROR if an error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BITMAP_NOT_SELECTED
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INCORRECT_DC_TYPE
PMERR_INV_HBITMAP
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SCAN_START

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetBitmapBits - Parameters

hdc (HDC) - input
Device context handle.

hbm (HBITMAP) - input
Bit-map handle.

If 0, the DC bit map is used.

lScanStart (LONG) - input
Scan line number from where data transfer starts.

0 is the first scan line number.

cScanCount (LONG) - input
Number of scan lines to be transferred.

pBitmap (PBYTE) - input
Pointer to bit-map data or NULL.

Pointer to the pel data of the bit map. This data is stored in the order that the coordinates appear on a display screen, that is, the pel in the lower-left corner is the first in the bit map. Pels are scanned to the right, and upward, from that position. The bits of the first pel are stored beginning with the most significant bits of the first byte. The data for pels in each scan line is packed together tightly. However, all scan lines are padded at the end so that each one begins on a ULONG boundary. That is, three bytes of pel data will hold one 24-bit pel,

three 8-bit pels, six 4-bit pels, or twenty-four 1-bit pels. If those three bytes are the only pel data for that scan line, one more byte of 0s would be required to pad the line to a ULONG boundary.

pInfo (PBITMAPINFO) - input
Pointer to BITMAPINFO or BITMAPINFO2 structure.

Pointer to either a BITMAPINFO structure:

cbFix	Length of structure
cx	Bit-map width. This value is updated by the handling routine.
cy	Bit-map height. This value is updated by the handling routine.
cPlanes	Number of color planes, 1 if standard format
cBitCount	Number of adjacent color bits per pel
argbColor[]	Color table array of RGB structures: bBlue bGreen bRed

Or pointer to a BITMAPINFO2 structure:

cbFix	Length of structure
cx	Bit-map width
cy	Bit-map height
cPlanes	Number of color planes, 1 if standard format
cBitCount	Number of adjacent color bits per pel
ulCompression	Compression scheme used to store the bit map: BCA_UNCOMP — Bit map is uncompressed (the only valid value).
cbImage	Length of bit-map storage data in bytes. If the bit map is uncompressed, 0 (default) can be specified for this.
cxResolution	Horizontal component of the resolution of the target device; that is, the resolution of the device the bit map is intended for in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cyResolution	Vertical component of the resolution of the target device; that is, the resolution of the device the bit map is intended for in the units specified by usUnits. This information enables an application to select from a resource group, the bit map that best matches the characteristics of the current output device.
cclrUsed	The number of color indexes from the color table that are used by the bit map. If it is 0 (default), all the indexes are used. If it is not 0, only the first cclrUsed entries in the table are accessed by the system; further entries can be omitted. For standard formats with a cBitCount of 1, 4, or 8 (and cPlanes=1), any indexes beyond cclrUsed are not valid. For example, a bit map with 64 colors can use the 8-bitcount format without having to supply the other 192 entries in the color table. For the 24-bitcount standard format, cclrUsed is the number of colors used by the bit map.
cclrImportant	Minimum number of color indexes for satisfactory appearance of the bit map. More colors can be used in the bit map, but it is not necessary to assign them to the device palette. These additional colors can be mapped

to the nearest colors available. Zero (default) means that all entries are important. For a 24-bitcount standard format, the cclrImportant colors are also listed in the color table relating to this bit map.

usUnits

Units of measure of the horizontal and vertical components of resolution:

BRU_METRIC (Default) Pels per meter

usReserved

Reserved field. If present, it must be 0.

usRecording

Recording algorithm, the format in which bit-map data is recorded:

BRA_BOTTOMUP (Default) Scanlines are recorded from bottom to top.

usRendering

Half-toning algorithm used to record bit-map data that has been digitally half-toned:

BRH_NOTHALFTONED

BRH_ERRORDIFFUSION

BRH_PANDA

BRH_SUPERCIRCLE

cSize1

Size value 1. If BRH_ERRORDIFFUSION is specified in usRendering, cSize1 is the error damping as a percentage in the range 0 - 100. A value of 100% indicates no damping. A value of 0% indicates that no errors are diffused.

If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize1 is the X-dimension of the pattern used

in pels.

cSize2

Size value 2. If BRH_ERRORDIFFUSION is specified in usRendering, this parameter is ignored. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize2 is the Y-dimension of the pattern used in pels.

ulColorEncoding

Color encoding:

BCE_RGB — (Default) Each element in the color array is an RGB2 data type.

ulIdentifier

Reserved for application use.

argbColor[]

Color table array of RGB2 structures:

bBlue
bGreen
bRed
fcOptions — Reserved. Must be 0.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetBitmapBits.

rc (LONG) - returns
Return Code.

On completion, the handling routine must return a LONG value (lLines), indicating the number of lines transferred, or GPI_ALTERROR if an error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BITMAP_NOT_SELECTED
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INCORRECT_DC_TYPE
PMERR_INV_HBITMAP
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SCAN_START

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetBitmapBits - Remarks

The bit map can be specified by a bit-map handle, or (if this is NULL) a DC handle, in which case the device context must be a memory DC with a bit map currently selected.

When the bit-map handle is NULL, the DC must be a memory DC with a bit map currently selected. Otherwise, the DC handle must be valid for that device. The BITMAPINFO or BITMAPINFO2 structure must be initialized with the values of cPlanes and cBitcount for the format of data required. This must be one of the standard formats or a device-specific format that matches the DC. On return, cx, cy, and argbColors are supplied by the system. Conversion of the bit-map data is carried out, if necessary.

pBitmap must point to a storage area large enough to contain data for the requested number of scan lines. The amount of storage required for one scan line can be determined by calling GetBitmapParameters:

```
((cBitcount * cx + 31)/32) * cPlanes * 4 bytes
```

GreGetBitmapBits - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreGetBoundsData

GreGetBoundsData - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreGetBoundsData stores the bounding rectangle of previous drawing primitives at the address indicated by pBoundsData.

This function must be supported by the presentation driver. GreGetBoundsData is called by GpiQueryBoundaryData in response to an application's request for the current boundary data for a presentation space and device context pair. This function can be handled by bit-map simulation.

```
#define INCL_GRE_DEVMISC3
#include <os2.h>

HDC       hdc;          /*  Device context handle. */
ULONG     flOptions;    /*  Option flags. */
PRECTL    pBoundsData;
PVOID     pInstance;    /*  Pointer to instance data. */
ULONG     lFunction;    /*  High-order WORD=flags; low-order WORD=NGreGetBoundsData. */
BOOL      rc;           /*  Return Code. */

rc = GreGetBoundsData(hdc, flOptions, pBoundsData,
       pInstance, lFunction);
```

GreGetBoundsData Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetBoundsData Parameter - flOptions

flOptions (ULONG) - input
Option flags.

The option flags define which bounding rectangle the handling routine should return. Valid values are:

GBD_GPI	Return GPI bounds in model space coordinates
GBD_USER	Return current user bounds in screen coordinates and reset user bounds to their initial value

---

GreGetBoundsData Parameter - pBoundsData

pBoundsData (PRECTL) - input
Pointer to the address for the returned RECTL structure that defines the specified bounding rectangle.

GreGetBoundsData Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetBoundsData Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetBoundsData.

GreGetBoundsData Return Value - rc

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetBoundsData - Parameters

hdc (HDC) - input
Device context handle.

flOptions (ULONG) - input
Option flags.

The option flags define which bounding rectangle the handling routine should return. Valid values are:

GBD_GPI	Return GPI bounds in model space coordinates
GBD_USER	Return current user bounds in screen coordinates and reset user bounds to their initial value

pBoundsData (PRECTL) - input
Pointer to the address for the returned RECTL structure that defines the specified bounding rectangle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetBoundsData.

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetBoundsData - Remarks

All presentation drivers must support GPI bounds. These bounds should be transformed to model space coordinates when they are accumulated. Display drivers must also support user bounds in screen coordinates. Bounds are inclusive. A NULL boundary is represented by the minimum coordinates of the rectangle, which are greater than its maximum coordinates. If the bounds have been reset, a NULL value is returned for pBoundsData.

GreGetBoundsData - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreGetClipBox

GreGetClipBox - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreGetClipBox determines the dimensions of the tightest rectangle around the DC region in world coordinates. The DC region is defined as the intersection of the visible region, clip region, viewing limits, graphics field, and clip path. The bounding rectangle is inclusive. Points on its boundary are deemed to be inside it. If COM_TRANSFORM is not set, the rectangle points are returned in screen coordinates.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_CLIP
#include <os2.h>

HDC       hdc;         /*  Device context handle. */
PRECTL    prclRect;    /*  Pointer to rectangle in world or screen coordinates. */
PVOID     pInstance;   /*  Pointer to instance data. */
ULONG     lFunction;   /*  High-order WORD=flags; low-order WORD=NGreGetClipBox. */
LONG      rc;          /*  Return codes. */

rc = GreGetClipBox(hdc, prclRect, pInstance,
       lFunction);
```

GreGetClipBox Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetClipBox Parameter - prclRect

prclRect (PRECTL) - input
Pointer to rectangle in world or screen coordinates.

RECTL structure:

xLeft	Minimum x-coordinate of rectangle
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of rectangle
yTop	Maximum y-coordinate

The prclRect parameter is a NULL rectangle when the value of xRight is less than xLeft, and yTop is less than yBottom.

GreGetClipBox Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetClipBox Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetClipBox.

GreGetClipBox Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (lComplexity) indicating the complexity of the resultant "clipping region", which is defined as the intersection of all clipping (that is, the clip path, viewing limits, graphics field, clip region, and visible region).

RGN_ERROR	Error
RGN_NULL	Null region
RGN_RECT	Rectangular region
RGN_COMPLEX	Complex region (more than 1 rectangle)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetClipBox - Parameters

hdc (HDC) - input
Device context handle.

prclRect (PRECTL) - input
Pointer to rectangle in world or screen coordinates.

RECTL structure:

xLeft	Minimum x-coordinate of rectangle
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of rectangle
yTop	Maximum y-coordinate

The prclRect parameter is a NULL rectangle when the value of xRight is less than xLeft, and yTop is less than yBottom.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetClipBox.

rc (LONG) - returns
Return codes.

This function returns an integer (lComplexity) indicating the complexity of the resultant "clipping region", which is defined as the intersection of all clipping (that is, the clip path, viewing limits, graphics field, clip region, and visible region).

RGN_ERROR	Error
RGN_NULL	Null region
RGN_RECT	Rectangular region
RGN_COMPLEX	Complex region (more than 1 rectangle)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetClipBox - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreGetClipRects

GreGetClipRects - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreGetClipRects fills the buffer indicated by parclRect with a list of (X,Y) coordinate pairs specifying the rectangles, which define the DC region and intersect an optional bounding rectangle (prclRect). The DC region is the intersection of the visible region, clip region, viewing limits, graphics field, and clip path.

Clipping, when performed by the graphics engine, carries a heavy processing overhead. For simple clipping, the presentation driver gains a significant performance advantage by calling this function to enumerate the clip region, and clipping the line to each rectangle in turn.

This function can be used to determine if the banding rectangle is inside, outside, or crossing the device's clip rectangle.

Note: If COM_TRANSFORM is set, the rectangle points are returned in device coordinates. If COM_TRANSFORM is not set, the rectangle points are returned in screen coordinates.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_CLIP
#include <os2.h>

HDC         hdc;        /*  Device context handle. */
PRECTL      prclBound;  /*  Pointer to bounding rectangle in device or screen coordinates. */
PRGNRECT    pControl;   /*  Pointer to a control structure containing additional parameters: */
PRECTL      parclRect;  /*  Pointer to array of rectangle (RECTL) structures. */
PVOID       pInstance;  /*  Pointer to instance data. */
ULONG       lFunction;  /*  High-order WORD=flags; low-order WORD=NGreGetClipRects. */
BOOL        fSuccess;   /*  Return codes. */

fSuccess = GreGetClipRects(hdc, prclBound,
             pControl, parclRect, pInstance,
             lFunction);
```

GreGetClipRects Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetClipRects Parameter - prclBound

prclBound (PRECTL) - input
Pointer to bounding rectangle in device or screen coordinates.

Pointer to a RECTL structure:

xLeft	Minimum X-coordinate of rectangle
yBottom	Minimum Y-coordinate
xRight	Maximum X-coordinate of rectangle
yTop	Maximum Y-coordinate

Only rectangles intersecting this bounding rectangle are returned. When this pointer is NULL, all rectangles in the region are enumerated. When this pointer is not NULL, each rectangle returned is the intersection of the bounding rectangle with a rectangle in the region.

If COM_TRANSFORM is set, the function expects the bounding rectangle to be in device coordinates. If COM_TRANSFORM is not set, the function expects the bounding rectangle to be in screen coordinates.

GreGetClipRects Parameter - pControl

pControl (PRGNRECT) - input
Pointer to a control structure containing additional parameters:

lStart	The rectangle number to start enumerating (1 indicates start at the beginning). By updating this value, the function can be called repeatedly to allow for more rectangles than can be

stored in the receiving buffer.

If parclRect is NULL, this field is always 1.

crc — The number of rectangles that can fit into the buffer (at least 1).

If parclRect is NULL and the clip region is completely within prclBoundRect, crc contains the value RRGN_INSIDE.

If parclRect is NULL and the clip region is completely outside prclBoundRect, crc contains the value RRGN_OUTSIDE.

If parclRect is NULL and the clip region spans prclBoundRect, crc contains the value RRGN_PARTIAL.

crcReturned — Number of rectangles that were written into the buffer. If less than lStart, there are no more rectangles to enumerate.

If parclRect is NULL and the clip region is completely within prclBoundRect, crcReturned contains the value 1.

If parclRect is NULL and the clip region is completely outside prclBoundRect, crcReturned contains the value 0.

If parclRect is NULL and the clip region spans prclBoundRect, crcReturned contains the number of clip rectangles.

usDirection — The direction in which the rectangles are listed:

RECTDIR_LFRT_TOPBOT	Left-to-right, top-to-bottom
RECTDIR_RTLF_TOPBOT	Right-to-left, top-to-bottom
RECTDIR_LFRT_BOTTOP	Left-to-right, bottom-to-top
RECTDIR_RTLF_BOTTOP	Right-to-left, bottom-to-top

If parclRect is NULL, usDirection is not applicable.

GreGetClipRects Parameter - parclRect

parclRect (PRECTL) - input
Pointer to array of rectangle (RECTL) structures.

All rectangles are inclusive on the left and bottom borders, and exclusive on the right and top borders.

GreGetClipRects Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetClipRects Parameter - lFunction

lFunction (ULONG) - input

High-order WORD=flags; low-order WORD=NGreGetClipRects.

GreGetClipRects Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetClipRects - Parameters

hdc (HDC) - input
Device context handle.

prclBound (PRECTL) - input
Pointer to bounding rectangle in device or screen coordinates.

Pointer to a RECTL structure:

xLeft	Minimum X-coordinate of rectangle
yBottom	Minimum Y-coordinate
xRight	Maximum X-coordinate of rectangle
yTop	Maximum Y-coordinate

Only rectangles intersecting this bounding rectangle are returned. When this pointer is NULL, all rectangles in the region are enumerated. When this pointer is not NULL, each rectangle returned is the intersection of the bounding rectangle with a rectangle in the region.

If COM_TRANSFORM is set, the function expects the bounding rectangle to be in device coordinates. If COM_TRANSFORM is not set, the function expects the bounding rectangle to be in screen coordinates.

pControl (PRGNRECT) - input
Pointer to a control structure containing additional parameters:

lrcStart	The rectangle number to start enumerating (1 indicates start at the beginning). If parclRect is NULL, this field is always 1.
crc	The number of rectangles that can fit into the buffer (at least 1). If parclRect is NULL and the clip region is completely within prclBoundRect, crc contains the value RRGN_INSIDE. If parclRect is NULL and the clip region is completely outside prclBoundRect, crc contains the value RRGN_OUTSIDE. If parclRect is NULL and the clip region spans prclBoundRect, crc contains the value RRGN_PARTIAL.

crcReturned	Number of rectangles that were written into the buffer. If less than lrcStart, there are no more rectangles to enumerate. If parclRect is NULL and the clip region is completely within prclBoundRect, crcReturned contains the value 1. If parclRect is NULL and the clip region is completely outside prclBoundRect, crcReturned contains the value 0. If parclRect is NULL and the clip region spans prclBoundRect, crcReturned contains the number of clip rectangles.	
usDirection	The direction in which the rectangles are listed:	
	RECTDIR_LFRT_TOPBOT	Left-to-right, top-to-bottom
	RECTDIR_RTLF_TOPBOT	Right-to-left, top-to-bottom
	RECTDIR_LFRT_BOTTOP	Left-to-right, bottom-to-top
	RECTDIR_RTLF_BOTTOP	Right-to-left, bottom-to-top
	If parclRect is NULL, usDirection is not applicable.	

parclRect (PRECTL) - input
Pointer to array of rectangle (RECTL) structures.

All rectangles are inclusive on the left and bottom borders, and exclusive on the right and top borders.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetClipRects.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetClipRects - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreGetCodePage

GreGetCodePage - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreGetCodePage returns the current code page. This is the default code page obtained by WinQueryProcessCp during the enabling of the DC (see EnableDeviceContext) or the code page set by GreSetCodePage.

This function must be supported by the presentation driver. GreGetCodePage is called by GpiQueryCP in response to an application requesting the currently selected code page for the device context.

```
#define INCL_GRE_DEVMISC3
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreGetCodePage. */
LONG     rc;          /*  Return Code. */

rc = GreGetCodePage(hdc, pInstance, lFunction);
```

GreGetCodePage Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetCodePage Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetCodePage Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetCodePage.

GreGetCodePage Return Value - rc

rc (LONG) - returns
Return Code.

On completion, the handling routine must return the current code page (lCodePage) or GPI_ERROR.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. An error code for conditions that the handling routine is expected to check is:

PMERR_DEV_FUNC_NOT_INSTALLED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetCodePage - Parameters

hdc (HDC) - input
Device context handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetCodePage.

rc (LONG) - returns
Return Code.

On completion, the handling routine must return the current code page (lCodePage) or GPI_ERROR.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. An error code for conditions that the handling routine is expected to check is:

PMERR_DEV_FUNC_NOT_INSTALLED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetCodePage - Remarks

This function applies to the default font, not the currently selected font, which can be determined with a call to GreQueryFontAttributes (see GreQueryFontAttributes).

GreGetCodePage - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreGetCurrentPosition

GreGetCurrentPosition - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreGetCurrentPosition takes the current (X,Y) position in world coordinates from the DC instance data structure and stores it at the location addressed by pptlPosition. If COM_TRANSFORM is not set, the current position is returned in screen coordinates.

This function must be supported by all presentation drivers. GreGetCurrentPosition is called by the GpiQueryCurrentPosition function. GreGetCurrentPosition might also be called in response to any of the Presentation Manager drawing functions that begin their operation from the current position within the presentation space.

```
#define INCL_GRE_LINES
#include <os2.h>

HDC         hdc;            /*  Device context handle. */
PPOINTL     pptlPosition;   /*  Pointer to current position. */
PVOID       pInstance;      /*  Pointer to instance data. */
ULONG       lFunction;      /*  High-order WORD=flags; low-order WORD=NGreGetCurrentPosition.

 */
BOOL        rc;             /*  Return Code. */

rc = GreGetCurrentPosition(hdc, pptlPosition,
       pInstance, lFunction);
```

GreGetCurrentPosition Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetCurrentPosition Parameter - pptlPosition

pptlPosition (PPOINTL) - input
Pointer to current position.

GreGetCurrentPosition Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetCurrentPosition Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetCurrentPosition.

GreGetCurrentPosition Return Value - rc

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetCurrentPosition - Parameters

hdc (HDC) - input
Device context handle.

pptlPosition (PPOINTL) - input
Pointer to current position.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetCurrentPosition.

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED

PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetCurrentPosition - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreGetDCOrigin

GreGetDCOrigin - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreGetDCOrigin queries the origin of the device context that defines the bottom-left drawing origin for a display device or a banded hardcopy device.

This function must be supported by presentation drivers for display devices and for hardcopy devices that use banding. For other devices, the minimum requirement is for the handling routine to return Successful with pptlOrigin set to (0,0).

```
#define INCL_GRE_DEVMISC3
#include <os2.h>

HDC         hdc;          /*  Device context handle. */
PPOINTL     pptlOrigin;   /*  Pointer to the (X,Y) coordinates of the returned DC origin in screen coordinates. */
PVOID       pInstance;    /*  Pointer to instance data. */
ULONG       lFunction;    /*  High-order WORD=flags; low-order WORD=NGreGetDCOrigin. */
BOOL        rc;           /*  Return Code. */

rc = GreGetDCOrigin(hdc, pptlOrigin, pInstance,
       lFunction);
```

GreGetDCOrigin Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetDCOrigin Parameter - pptlOrigin

pptlOrigin (PPOINTL) - input
Pointer to the (X,Y) coordinates of the returned DC origin in screen coordinates.

GreGetDCOrigin Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetDCOrigin Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetDCOrigin.

GreGetDCOrigin Return Value - rc

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

GreGetDCOrigin - Parameters

hdc (HDC) - input
Device context handle.

pptlOrigin (PPOINTL) - input
Pointer to the (X,Y) coordinates of the returned DC origin in screen coordinates.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetDCOrigin.

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

GreGetDCOrigin - Remarks

The DC origin is set by GreSetupDC (see GreSetupDC) at the graphics engine, and by GreDeviceSetDCOrigin (see GreDeviceSetDCOrigin) at the presentation driver. This device context origin is stored in the Device Context Instance (DCI) data structure addressed by *pInstance*. The DC origin is always returned in screen coordinates.

GreGetDCOrigin - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreGetGlobalViewingXform

GreGetGlobalViewingXform - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreGetGlobalViewingXform queries the global viewing transform matrix. On completion, paXformData points to an array of two-dimensional values that defines the global viewing transform matrix.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_XFORMS
#include <os2.h>

HDC       hdc;          /*  Device context handle. */
PXFORM    paXformData;  /*  Pointer to return data */
PVOID     pInstance;    /*  Pointer to instance data. */
ULONG     lFunction;    /*  High-order WORD=flags; low-order WORD=NGreGetGlobalViewingXform. */
BOOL      fSuccess;     /*  Return codes. */

fSuccess = GreGetGlobalViewingXform(hdc, paXformData,
             pInstance, lFunction);
```

GreGetGlobalViewingXform Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetGlobalViewingXform Parameter - paXformData

paXformData (PXFORM) - input
Pointer to return data

Pointer to return data in which the array of six matrix elements (M11, M12, M21, M22, M41, and M42) are to be stored.

GreGetGlobalViewingXform Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetGlobalViewingXform Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetGlobalViewingXform.

GreGetGlobalViewingXform Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetGlobalViewingXform - Parameters

hdc (HDC) - input
Device context handle.

paXformData (PXFORM) - input
Pointer to return data

Pointer to return data in which the array of six matrix elements (M11, M12, M21, M22, M41, and M42) are to be stored.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetGlobalViewingXform.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetGlobalViewingXform - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreGetGraphicsField

GreGetGraphicsField - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreGetGraphicsField loads the buffer indicated by prclGraphicsField with the integer values that identify the boundaries of the graphics field.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_XFORMS
#include <os2.h>

HDC       hdc;                /*  Device context handle. */
PRECTL    prclGraphicsField;  /*  Pointer to graphics field. */
PVOID     pInstance;          /*  Pointer to instance data. */
ULONG     lFunction;          /*  High-order WORD=flags; low-order WORD=NGreGetGraphicsField. */
BOOL      fSuccess;           /*  Return codes. */

fSuccess = GreGetGraphicsField(hdc, prclGraphicsField,
             pInstance, lFunction);
```

GreGetGraphicsField Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetGraphicsField Parameter - prclGraphicsField

prclGraphicsField (PRECTL) - input
Pointer to graphics field.

RECTL structure:

xLeft	Minimum X-coordinate of graphics field
yBottom	Minimum Y-coordinate
xRight	Maximum X-coordinate of graphics field
yTop	Maximum Y-coordinate

GreGetGraphicsField Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetGraphicsField Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetGraphicsField.

GreGetGraphicsField Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetGraphicsField - Parameters

hdc (HDC) - input
Device context handle.

prclGraphicsField (PRECTL) - input
Pointer to graphics field.

RECTL structure:

xLeft	Minimum X-coordinate of graphics field
yBottom	Minimum Y-coordinate
xRight	Maximum X-coordinate of graphics field
yTop	Maximum Y-coordinate

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetGraphicsField.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetGraphicsField - Topics

Select an item:

Syntax
Parameters
Returns
Glossary

GreGetLineOrigin

GreGetLineOrigin - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreGetLineOrigin returns the current line style from the DC instance data and stores the current position to the address indicated by pptlXY. The presentation driver maintains the line style information.

This function must be supported by the presentation driver. GreGetLineOrigin is used to get the line style and current position simultaneously. This function call can be handled by bit-map simulation.

```
#define INCL_GRE_DEVMISC3
#include <os2.h>

HDC         hdc;         /*  Device context handle. */
PPOINTL     pptlXY;      /*  Pointer to an (X,Y) coordinate pair to which the current position is returned. */
PVOID       pInstance;   /*  Pointer to instance data. */
ULONG       lFunction;   /*  High-order WORD=flags; low-order WORD=NGreGetLineOrigin. */
LONG        rc;          /*  Return Code. */

rc = GreGetLineOrigin(hdc, pptlXY, pInstance,
       lFunction);
```

GreGetLineOrigin Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetLineOrigin Parameter - pptlXY

pptlXY (PPOINTL) - input
Pointer to an (X,Y) coordinate pair to which the current position is returned.

GreGetLineOrigin Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetLineOrigin Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetLineOrigin.

GreGetLineOrigin Return Value - rc

rc (LONG) - returns
Return Code.

On completion, this function returns the style number (lStyle), or GPI_ALTERROR if an error occurs.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetLineOrigin - Parameters

hdc (HDC) - input
Device context handle.

pptlXY (PPOINTL) - input
Pointer to an (X,Y) coordinate pair to which the current position is returned.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetLineOrigin.

rc (LONG) - returns
Return Code.

On completion, this function returns the style number (lStyle), or GPI_ALTERROR if an error occurs.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetLineOrigin - Remarks

Some lines and curves can be drawn either by the presentation driver or by simulations that must be able to query and set the style as required. If COM_TRANSFORM is not set, the coordinate pair at pptlXY is returned in screen coordinates.

The high-order WORD of the style number contains the first/last pel information. If the value of this byte is 0, the first pel is not drawn. (See First and Last Pel Considerations.) The low-order byte indicates the position in the style mask. This is a count, held in the three least significant bits, of the number of positions that the style mask byte is rotated. The next byte is the state of the style error value.

GreGetLineOrigin - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreGetModelXform

GreGetModelXform - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreGetModelXform queries the current model transform matrix. On completion, paXformData is an array of two-dimensional values defining the current model transform matrix.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_XFORMS
#include <os2.h>

HDC       hdc;          /*  Device context handle. */
PXFORM    paXformData;  /*  Pointer to return data. */
PVOID     pInstance;    /*  Pointer to instance data. */
ULONG     lFunction;    /*  High-order WORD=flags; low-order WORD=NGreGetModelXform. */
BOOL      fSuccess;     /*  Return codes. */

fSuccess = GreGetModelXform(hdc, paXformData,
             pInstance, lFunction);
```

GreGetModelXform Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetModelXform Parameter - paXformData

paXformData (PXFORM) - input
Pointer to return data.

A pointer to return data in which the array of six matrix elements (M11, M12, M21, M22, M41, and M42) are to be stored.

GreGetModelXform Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetModelXform Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetModelXform.

GreGetModelXform Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetModelXform - Parameters

hdc (HDC) - input
Device context handle.

paXformData (PXFORM) - input
Pointer to return data.

A pointer to return data in which the array of six matrix elements (M11, M12, M21, M22, M41, and M42) are to be stored.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetModelXform.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetModelXform - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreGetPageUnits

GreGetPageUnits - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreGetPageUnits returns the page units for the specified display context. On completion, the structure addressed by pExtent contains the dimensions of the page expressed in the units indicated by the return value of this function.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_XFORMS
#include <os2.h>

HDC       hdc;         /*  Device context handle. */
PSIZEL    pExtent;     /*  Pointer to a SIZEL structure where the page dimensions are returned. */
PVOID     pInstance;   /*  Pointer to instance data. */
ULONG     lFunction;   /*  High-order WORD=flags; low-order WORD=NGreGetPageUnits. */
ULONG     rc;          /*  Return codes. */

rc = GreGetPageUnits(hdc, pExtent, pInstance,
       lFunction);
```

GreGetPageUnits Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetPageUnits Parameter - pExtent

pExtent (PSIZEL) - input
Pointer to a SIZEL structure where the page dimensions are returned.

On completion, the SIZEL structure contains the actual dimensions of the page:

lWidth	Page width.
lHeight	Page height.

GreGetPageUnits Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetPageUnits Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetPageUnits.

GreGetPageUnits Return Value - rc

rc (ULONG) - returns
Return codes.

This function returns the page units, or GPI_ERROR if an error occurred. The defined page units are described under GreSetPageUnits.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetPageUnits - Parameters

hdc (HDC) - input
Device context handle.

pExtent (PSIZEL) - input
Pointer to a SIZEL structure where the page dimensions are returned.

On completion, the SIZEL structure contains the actual dimensions of the page:

lWidth	Page width.
lHeight	Page height.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetPageUnits.

rc (ULONG) - returns
Return codes.

This function returns the page units, or GPI_ERROR if an error occurred. The defined page units are described under GreSetPageUnits.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetPageUnits - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreGetPageViewport

GreGetPageViewport - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreGetPageViewport loads the buffer indicated by prclViewport with the page viewport coordinates.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_XFORMS
#include <os2.h>

HDC       hdc;           /*  Device context handle. */
PRECTL    prclViewport;  /*  Pointer to page viewport boundaries. */
PVOID     pInstance;     /*  Pointer to instance data. */
ULONG     lFunction;     /*  High-order WORD=flags; low-order WORD=NGreGetPageViewport. */
BOOL      fSuccess;      /*  Return codes. */

fSuccess = GreGetPageViewport(hdc, prclViewport,
             pInstance, lFunction);
```

GreGetPageViewport Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetPageViewport Parameter - prclViewport

prclViewport (PRECTL) - input
Pointer to page viewport boundaries.

This is a RECTL structure in device coordinates:

xLeft	Minimum x-coordinate of viewport
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of viewport
yTop	Maximum y-coordinate

Boundaries are inclusive at the bottom-left corner, and exclusive at the top-right corner.

GreGetPageViewport Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetPageViewport Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetPageViewport.

GreGetPageViewport Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetPageViewport - Parameters

hdc (HDC) - input
Device context handle.

prclViewport (PRECTL) - input
Pointer to page viewport boundaries.

This is a RECTL structure in device coordinates:

xLeft	Minimum x-coordinate of viewport
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of viewport
yTop	Maximum y-coordinate

Boundaries are inclusive at the bottom-left corner, and exclusive at the top-right corner.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetPageViewport.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetPageViewport - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreGetPairKerningTable

GreGetPairKerningTable - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreGetPairKerningTable stores the kerning pairs of the current font to the buffer addressed by pKernPairs.

This function must be supported by the presentation driver. The call parameters are passed unchanged to the display driver's dispatch table. GreGetPairKerningTable is called from GpiQueryKerningPairs. GreGetPairKerningTable is used to return the kerning data for the currently selected font. This function can be handled by bit-map simulation.

```
#define INCL_GRE_DEVMISC1
#include <os2.h>

HDC           hdc;          /*  Device context handle. */
LONG          cKernPairs;   /*  Number of kerning pairs requested by the application. */
PKERNPAIRS    pKernPairs;   /*  Pointer to kerning pair records. */
PVOID         pInstance;    /*  Pointer to instance data. */
ULONG         lFunction;    /*  High-order WORD=flags;. low-order WORD=NGreGetPairKerningTable. */
LONG          rc;           /*  Return Code. */

rc = GreGetPairKerningTable(hdc, cKernPairs,
       pKernPairs, pInstance, lFunction);
```

GreGetPairKerningTable Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetPairKerningTable Parameter - cKernPairs

cKernPairs (LONG) - input
Number of kerning pairs requested by the application.

GreGetPairKerningTable Parameter - pKernPairs

pKernPairs (PKERNPAIRS) - input
Pointer to kerning pair records.

KERNINGPAIRS data structure:

sFirstChar	Code point for the first character
sSecondChar	Code point for the second character
sKerningAmount	Two-byte signed integer indicating the amount of kerning. Positive numbers specify increased inter-character spacing.

GreGetPairKerningTable Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetPairKerningTable Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags;. low-order WORD=NGreGetPairKerningTable.

GreGetPairKerningTable Return Value - rc

rc (LONG) - returns
Return Code.

On completion, the handling routine must return the number of kerning pairs returned in pKernPairs (cPairs), or GPI_ALTERROR if an error occurs.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CODEPAGE
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SETID

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetPairKerningTable - Parameters

hdc (HDC) - input
Device context handle.

cKernPairs (LONG) - input
Number of kerning pairs requested by the application.

pKernPairs (PKERNPAIRS) - input
Pointer to kerning pair records.

KERNINGPAIRS data structure:

sFirstChar	Code point for the first character
sSecondChar	Code point for the second character
sKerningAmount	Two-byte signed integer indicating the amount of kerning. Positive numbers specify increased inter-character spacing.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags;. low-order WORD=NGreGetPairKerningTable.

rc (LONG) - returns
Return Code.

On completion, the handling routine must return the number of kerning pairs returned in pKernPairs (cPairs), or GPI_ALTERROR if an error occurs.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CODEPAGE
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SETID

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetPairKerningTable - Remarks

The handling routine must transform all kerning-pair coordinates from device to world coordinates before sending the data to the calling routine. This can be done by using GreConvert. If it is unable to do this because the transform matrix is singular, it must log PMERR_COORDINATE_OVERFLOW.

The number of kerning pairs is a field in the FONTMETRICS structure.

GreGetPairKerningTable - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreGetPel

GreGetPel - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreGetPel returns the color of a pel at a specified position.

This function must be supported by the presentation driver, and is called by GpiQueryPel. GreGetPel is used to query the value of a pel at a specified (X,Y) coordinate within the DC. This function can be handled by bit-map simulation.

```
#define INCL_GRE_BITMAPS
#include <os2.h>

HDC         hdc;         /*  Device context handle. */
PPOINTL     pptlPel;     /*  Pointer to the coordinate (X, Y) pair structure. */
PVOID       pInstance;   /*  Pointer to instance data. */
ULONG       lFunction;   /*  High-order WORD=flags; low-order WORD=NGreGetPel. */
LONG        rc;          /*  Return Code. */

rc = GreGetPel(hdc, pptlPel, pInstance, lFunction);
```

GreGetPel Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetPel Parameter - pptlPel

pptlPel (PPOINTL) - input
Pointer to the coordinate (X, Y) pair structure.

GreGetPel Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetPel Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetPel.

GreGetPel Return Value - rc

rc (LONG) - returns
Return Code.

This function must return the color index value for the pel, or CLR_NOINDEX if there is no index corresponding to the color. In addition, the handling routine must raise an error when the point is subject to any clipping.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT
PMERR_PEL_IS_CLIPPED
PMERR_PEL_NOT_AVAILABLE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetPel - Parameters

hdc (HDC) - input
Device context handle.

pptlPel (PPOINTL) - input
Pointer to the coordinate (X, Y) pair structure.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetPel.

rc (LONG) - returns
Return Code.

This function must return the color index value for the pel, or CLR_NOINDEX if there is no index corresponding to the color. In addition, the handling routine must raise an error when the point is subject to any clipping.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT
PMERR_PEL_IS_CLIPPED
PMERR_PEL_NOT_AVAILABLE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetPel - Remarks

If COM_TRANSFORM is set, this position is in world coordinates. If not set, the position is in screen coordinates. The return value of this function is either the color index of the pel or its RGB value depending on the color mode of the device.

GreGetPel - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreGetPickWindow

GreGetPickWindow - Syntax

Simulation support:

This function is mandatory for display drivers. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreGetPickWindow stores (at the location addressed by pPick) a RECTL structure giving the position and size of the pick window in page-coordinate space.

```
#define INCL_GRE_PICK
#include <os2.h>

HDC       hdc;         /*  The device context handle. */
PRECTL    pPick;       /*  A pointer to a pick window. */
PVOID     pInstance;   /*  A pointer to instance data. */
ULONG     lFunction;   /*  High-order WORD=flags; low-order WORD=NGreGetPickWindow. */
BOOL      fSuccess;    /*  Return codes. */

fSuccess = GreGetPickWindow(hdc, pPick, pInstance,
             lFunction);
```

GreGetPickWindow Parameter - hdc

hdc (HDC) - input
The device context handle.

GreGetPickWindow Parameter - pPick

pPick (PRECTL) - input
A pointer to a pick window.

The pick window is defined as a RECTL structure in page coordinate space:

xLeft	Minimum x-coordinate of window
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of window
yTop	Maximum y-coordinate

GreGetPickWindow Parameter - pInstance

pInstance (PVOID) - input
A pointer to instance data.

GreGetPickWindow Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetPickWindow.

GreGetPickWindow Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

GreGetPickWindow - Parameters

hdc (HDC) - input
The device context handle.

pPick (PRECTL) - input
A pointer to a pick window.

The pick window is defined as a RECTL structure in page coordinate space:

xLeft	Minimum x-coordinate of window
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of window
yTop	Maximum y-coordinate

pInstance (PVOID) - input
A pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetPickWindow.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

GreGetPickWindow - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreGetRegionBox

GreGetRegionBox - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreGetRegionBox loads the buffer pointed to by prclRect with the dimensions of the tightest rectangle around the region indicated by hrgn. Note that the rectangle is always returned in device coordinates. GreGetRegionBox raises an error when hrgn is the handle of the currently selected clip region.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_REGIONS
#include <os2.h>

HDC       hdc;         /*  Device context handle. */
HRGN      hrgn;        /*  Region handle. */
PRECTL    prclRect;    /*  Pointer to rectangle in device coordinates. */
PVOID     pInstance;   /*  Pointer to instance data. */
ULONG     lFunction;   /*  High-order WORD=flags; low-order WORD=NGreGetRegionBox. */
LONG      rc;          /*  Return codes. */

rc = GreGetRegionBox(hdc, hrgn, prclRect,
       pInstance, lFunction);
```

GreGetRegionBox Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetRegionBox Parameter - hrgn

hrgn (HRGN) - input
Region handle.

GreGetRegionBox Parameter - prclRect

prclRect (PRECTL) - input
Pointer to rectangle in device coordinates.

RECTL structure:

xLeft	Minimum X-coordinate of rectangle
yBottom	Minimum Y-coordinate
xRight	Maximum X-coordinate of rectangle
yTop	Maximum Y-coordinate

GreGetRegionBox Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetRegionBox Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetRegionBox.

GreGetRegionBox Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (lComplexity) indicating the complexity of the region:

RGN_ERROR	Error
RGN_NULL	Null region
RGN_RECT	Rectangular region
RGN_COMPLEX	Complex region (more than 1 rectangle)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HRGN_BUSY
PMERR_INV_HRGN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetRegionBox - Parameters

hdc (HDC) - input
Device context handle.

hrgn (HRGN) - input
Region handle.

prclRect (PRECTL) - input
Pointer to rectangle in device coordinates.

RECTL structure:

xLeft	Minimum X-coordinate of rectangle
yBottom	Minimum Y-coordinate
xRight	Maximum X-coordinate of rectangle
yTop	Maximum Y-coordinate

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetRegionBox.

rc (LONG) - returns
Return codes.

This function returns an integer (lComplexity) indicating the complexity of the region:

RGN_ERROR	Error
RGN_NULL	Null region
RGN_RECT	Rectangular region
RGN_COMPLEX	Complex region (more than 1 rectangle)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HRGN_BUSY
PMERR_INV_HRGN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetRegionBox - Remarks

When the region is empty, the rectangle is returned with the right boundary equal to the left, and the top boundary equal to the bottom. GreGetRegionBox must return an error when the region indicated by hrgn is selected into the DC as a clip region.

GreGetRegionBox - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreGetRegionRects

GreGetRegionRects - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreGetRegionRects fills the array pointed to by parclRect with a list of (X,Y) coordinate pairs specifying the rectangles that together define the region associated with hrgn. All rectangle coordinates are inclusive at the bottom-left corner, and exclusive at the top-right corner. Notice that the coordinates of the bounding box, and the rectangles returned, will be in device coordinates. GreGetRegionRects raises an error when hrgn is the handle of a currently selected clip region.

This function can be hooked by the presentation driver.

This function can be used to determine if the banding rectangle is inside, outside, or crossing the device's clip rectangle.

```
#define INCL_GRE_REGIONS
#include <os2.h>

HDC         hdc;             /*  Device context handle. */
HRGN        hrgn;            /*  Region handle. */
PRECTL      prclBoundRect;   /*  Pointer to bounding rectangle. */
PRGNRECT    pControl;        /*  This is a pointer to a RGNRECT structure: */
PRECTL      parclRect;       /*  Pointer to array of rectangle structures that define the region. */
PVOID       pInstance;       /*  Pointer to instance data. */
ULONG       lFunction;       /*  High-order WORD=flags; low-order WORD=NGreGetRegionRects. */
BOOL        rc;              /*  Return codes. */

rc = GreGetRegionRects(hdc, hrgn, prclBoundRect,
       pControl, parclRect, pInstance, lFunction);
```

GreGetRegionRects Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetRegionRects Parameter - hrgn

hrgn (HRGN) - input
Region handle.

GreGetRegionRects Parameter - prclBoundRect

prclBoundRect (PRECTL) - input
Pointer to bounding rectangle.

RECTL structure:

xLeft	Minimum x-coordinate of window
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of window
yTop	Maximum y-coordinate

Only rectangles intersecting this bounding rectangle are returned. If this pointer is NULL, all rectangles in the region are enumerated. If this pointer is not NULL, only rectangles that intersect the bounding rectangle are returned. Each of these rectangles is the intersection of the bounding rectangle with a rectangle in the region.

GreGetRegionRects Parameter - pControl

pControl (PRGNRECT) - input
This is a pointer to a RGNRECT structure:

lrcStart	The rectangle number to start enumerating (1 indicates start at the beginning). If parclRect is NULL, this field is always 1.
crc	The number of rectangles that can fit into the buffer (at least 1). If parclRect is NULL and the clip region is completely within prclBoundRect, crc contains the value RRGN_INSIDE. If parclRect is NULL and the clip region is completely outside prclBoundRect, crc contains the value RRGN_OUTSIDE. If parclRect is NULL and the clip region spans prclBoundRect, crc contains the value RRGN_PARTIAL.
crcReturned	Number of rectangles that were written into the buffer. If less than lrcStart, there are no more rectangles to enumerate. If parclRect is NULL and the clip region is completely within prclBoundRect, crcReturned contains the value 1. If parclRect is NULL and the clip region is completely outside prclBoundRect, crcReturned contains the value 0. If parclRect is NULL and the clip region spans prclBoundRect, crcReturned contains the number of clip rectangles.
usDirection	The direction in which the rectangles are listed: RECTDIR_LFRT_TOPBOT — Left-to-right, top-to-bottom RECTDIR_RTLF_TOPBOT — Right-to-left, top-to-bottom RECTDIR_LFRT_BOTTOP — Left-to-right, bottom-to-top RECTDIR_RTLF_BOTTOP — Right-to-left, bottom-to-top If parclRect is NULL, usDirection is not applicable.

GreGetRegionRects Parameter - parclRect

parclRect (PRECTL) - input
Pointer to array of rectangle structures that define the region.

GreGetRegionRects Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetRegionRects Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetRegionRects.

GreGetRegionRects Return Value - rc

rc (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HRGN_BUSY
PMERR_INV_COORDINATE
PMERR_INV_HRGN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetRegionRects - Parameters

hdc (HDC) - input
Device context handle.

hrgn (HRGN) - input
Region handle.

prclBoundRect (PRECTL) - input

Pointer to bounding rectangle.

RECTL structure:

xLeft	Minimum x-coordinate of window
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of window
yTop	Maximum y-coordinate

Only rectangles intersecting this bounding rectangle are returned. If this pointer is NULL, all rectangles in the region are enumerated. If this pointer is not NULL, only rectangles that intersect the bounding rectangle are returned. Each of these rectangles is the intersection of the bounding rectangle with a rectangle in the region.

pControl (PRGNRECT) - input
This is a pointer to a RGNRECT structure:

lrcStart	The rectangle number to start enumerating (1 indicates start at the beginning). If parclRect is NULL, this field is always 1.
crc	The number of rectangles that can fit into the buffer (at least 1). If parclRect is NULL and the clip region is completely within prclBoundRect, crc contains the value RRGN_INSIDE. If parclRect is NULL and the clip region is completely outside prclBoundRect, crc contains the value RRGN_OUTSIDE. If parclRect is NULL and the clip region spans prclBoundRect, crc contains the value RRGN_PARTIAL.
crcReturned	Number of rectangles that were written into the buffer. If less than lrcStart, there are no more rectangles to enumerate. If parclRect is NULL and the clip region is completely within prclBoundRect, crcReturned contains the value 1. If parclRect is NULL and the clip region is completely outside prclBoundRect, crcReturned contains the value 0. If parclRect is NULL and the clip region spans prclBoundRect, crcReturned contains the number of clip rectangles.
usDirection	The direction in which the rectangles are listed: RECTDIR_LFRT_TOPBOT — Left-to-right, top-to-bottom RECTDIR_RTLF_TOPBOT — Right-to-left, top-to-bottom RECTDIR_LFRT_BOTTOP — Left-to-right, bottom-to-top RECTDIR_RTLF_BOTTOP — Right-to-left, bottom-to-top If parclRect is NULL, usDirection is not applicable.

parclRect (PRECTL) - input
Pointer to array of rectangle structures that define the region.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetRegionRects.

rc (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HRGN_BUSY
PMERR_INV_COORDINATE
PMERR_INV_HRGN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetRegionRects - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreGetScreenBits

GreGetScreenBits - Syntax

Simulation support:

This function is mandatory for display drivers that want to be DCAF-enabled.

Description *GreGetScreenBits* queries a region of screen pixel data and saves it in the memory provided by the caller. The data is compressed, and can be converted into a format suitable for another supported display device. The process will stop when either of the following situations occurs:

- The supplied memory area is full
- The requested region has been returned

The area to be queried can be identified in one of two ways:

- As a pointer to a single rectangle (RECTL, long values)
- As a region handle

The GSB_OPT_HRGN flag in the *flCmd* field identifies the region: if the flag is set, it indicates a region handle; if the flag is not set, it indicates a pointer to a rectangle. If a RECTL is specified, the RECTL is assumed to be inclusive/inclusive.

The GreGetScreenBits function modifies the supplied rectangle or region to indicate the area that was NOT returned in the call. If the entire requested region was returned, the rectangle or region will be a null area.

The supplied device context must be direct; it is the source of the pixel data. Since this is not a drawing primitive, no correlation, bounds accumulation or drawing will take place.

```
#define INCL_GRE_SCREEN
#include <os2.h>
#include <pmddi.h>

HDC       hdc;        /*  Any valid, direct (screen) device context handle. */
HRGN      hrgnApp;    /*  Area of the screen to be acquired. */
PBYTE     pDest;
```

```
PULONG     pulLength;  /*  Pointer to a ULONG. */
ULONG      flCmd;      /*  Option flags. */
PDC        pdcArg;     /*  Pointer to instance data. */
ULONG      FunN;       /*  High-order word=Flags; Low-order word=NGreGetScreenBits. */
LONG       rc;

rc = GreGetScreenBits(hdc, hrgnApp, pDest,
       pulLength, flCmd, pdcArg, FunN);
```

GreGetScreenBits Parameter - hdc

hdc (HDC) - input
Any valid, direct (screen) device context handle.

GreGetScreenBits Parameter - hrgnApp

hrgnApp (HRGN) - input
Area of the screen to be acquired.

Area can be either a valid region handle or a pointer to an inclusive RECTL, specified by a flag in the *flCmd* field.

GreGetScreenBits Parameter - pDest

pDest (PBYTE) - input
Pointer to the memory buffer where the compressed data will be written.

GreGetScreenBits Parameter - pulLength

pulLength (PULONG) - input
Pointer to a ULONG.

The ULONG must contain the length of the memory buffer pointed to by *pDest* . Valid values range from 2071 bytes to 64K bytes. Upon exit, the ULONG contains the number of bytes stored in the memory buffer.

GreGetScreenBits Parameter - flCmd

flCmd (ULONG) - input
Option flags.

These flags specify the format (bits per pel, linear or planar) of the data to be put in the memory buffer. *flCmd* also indicates whether *hrgn* contains a region handle or a pointer to a RECTL. Valid options include the following:

GSB_OPT_4BPP	0000H
GSB_OPT_8BPP	0001H
GSB_OPT_16BPP	0002H
GSB_OPT_LINEAR	0000H
GSB_OPT_PLANAR	0008H
GSB_OPT_HRGN	0010H

These option flags are ORed together to create the *flCmd* field.

GreGetScreenBits Parameter - pdcArg

pdcArg (PDC) - input
Pointer to instance data.

GreGetScreenBits Parameter - FunN

FunN (ULONG) - input
High-order word=Flags; Low-order word=NGreGetScreenBits.

GreGetScreenBits Return Value - rc

rc (LONG) - returns
This function returns one of the following:

1	The entire area was successfully compressed in the memory buffer. Upon exit, the supplied region or rectangle will be NULL.
2	A subset of the area was saved in the memory buffer because the buffer was not big enough for the whole area. Upon exit, the supplied region or rectangle will be updated to contain the area that was NOT compressed. This implies that GreGetScreenBits will have to be called again to complete the area compression.
0	An error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_DC_TYPE
PMERR_INV_FORMAT_CONTROL
PMERR_INV_IMAGE_DIMENSION
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_PEL_NOT_AVAILABLE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetScreenBits - Parameters

hdc (HDC) - input
Any valid, direct (screen) device context handle.

hrgnApp (HRGN) - input
Area of the screen to be acquired.

Area can be either a valid region handle or a pointer to an inclusive RECTL, specified by a flag in the *flCmd* field.

pDest (PBYTE) - input
Pointer to the memory buffer where the compressed data will be written.

pulLength (PULONG) - input
Pointer to a ULONG.

The ULONG must contain the length of the memory buffer pointed to by *pDest* . Valid values range from 2071 bytes to 64K bytes. Upon exit, the ULONG contains the number of bytes stored in the memory buffer.

flCmd (ULONG) - input
Option flags.

These flags specify the format (bits per pel, linear or planar) of the data to be put in the memory buffer. *flCmd* also indicates whether *hrgn* contains a region handle or a pointer to a RECTL. Valid options include the following:

GSB_OPT_4BPP	0000H
GSB_OPT_8BPP	0001H
GSB_OPT_16BPP	0002H
GSB_OPT_LINEAR	0000H
GSB_OPT_PLANAR	0008H
GSB_OPT_HRGN	0010H

These option flags are ORed together to create the *flCmd* field.

pdcArg (PDC) - input
Pointer to instance data.

FunN (ULONG) - input
High-order word=Flags; Low-order word=NGreGetScreenBits.

rc (LONG) - returns
This function returns one of the following:

1	The entire area was successfully compressed in the memory buffer. Upon exit, the supplied region or rectangle will be NULL.
2	A subset of the area was saved in the memory buffer because the buffer was not big enough for the whole area. Upon exit, the supplied region or rectangle will be updated to contain the area that was NOT compressed. This implies that GreGetScreenBits will have to be called again to complete the area compression.
0	An error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_DC_TYPE
PMERR_INV_FORMAT_CONTROL
PMERR_INV_IMAGE_DIMENSION
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_PEL_NOT_AVAILABLE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetScreenBits - Remarks

It is permissible to implement this function by returning 0 to indicate that the bits were not saved and, therefore, must be saved by the calling routine.

GreGetScreenBits calls EnterDriver and performs some initial error tests, checking the *FunN* field and the DC type. If the COM_PATH or the COM_AREA flags are set in *FunN,* the function logs PMERR_INV_IN_PATH or PMERR_INV_IN_AREA, respectively, and returns 0 (zero).

Next, the parameters are checked for validity.

- The buffer size must be in the following range:

MIN_BUFFER_SIZE	2071 bytes; the length of a buffer containing only a compressed row at the maximum resolution, and when the input row only contains nonrepeating data
65536 bytes	Limit due to PHUNK memory

  Otherwise, the PMERR_INV_LENGTH_OR_COUNT error is logged and the call returns 0 (zero).

- The *flCmd* field must be set with one or more valid flags.

  Otherwise, the PMERR_INV_FORMAT_CONTROL error is logged and the call returns 0 (zero).

- Presentation Manager must be in the foreground (*fXGAdead* is set to TRUE).

  Otherwise, PMERR_PEL_NOT_AVAILABLE error is logged and the call returns 0 (zero).

If all these checks are successful, the CompressScreenBits routine (in GETSCR.C) is called. CompressScreenBits checks whether the format is valid by creating a word containing the internal (screen) format and the external (requested) format. Then the aGSBValidDataFormats table is scanned for a match. This table is an array of valid source and destination format pairs, as defined in the VALID_DATA_FORMATS data structure. See VALID_DATA_FORMATS in Data Types for details on this data structure.

If a match is found, the format is valid. Otherwise, the format is invalid, the PMERR_INV_FORMAT_CONTROL error is logged and the call returns 0. Then, the pointer to the compression function for the required format combination is retrieved from the aGSBValidDataFormats structure and copied into the Spad global structure.

The following steps initialize the conversion cache by calling the InitializeCache routine in DCAFCNV.ASM (this is currently only required if source is 16bpp and destination is 8bpp or 4bpp). Then a check is done to determine whether a conversion table is needed for this combination of formats (actually, it is required for all the pairs, except for those with 16 bpp as a source. These latest pairs have a null pointer in the last two fields of their aGSBValidDataFormats structures.

- If the internal format is 16bpp, a conversion table is not needed, whatever the target format may be.
- If the requested format is 8bpp:
  - if the hardware palette is the default, then no conversion is required.

    This is checked by comparing two global variables, hForeGroundPal and DirectDeviceDefaultPalette. If there is an old conversion table, it will be freed by FreeConvertTable (in DCAFCNVT.C).
  - Otherwise, a palette has been realized and the internal indices need to be converted into standard external format; that is, indices are converted into the default 256-color XGA (fudged) palette.

    If a conversion table has been previously created, ensure that the mapping refers to the current palette (the internal palette may have changed since the conversion table was calculated). This is done via a global variable, Seamlessdata.ulLastPalUpdate, which is incremented when the hardware palette is changed. GreGetScreenBits takes a copy of this variable when the conversion table is created. Subsequent calls check whether the value has changed before using the table. If the value has changed, the old table is deleted and a new one is created.
- If the requested format is 4bpp:
  - A conversion table is always needed. It must correctly map the current hardware palette.
  - In this case, the same scheme as for 8bpp is used.

If a conversion table is needed (according to the scheme above), the proper creation function (stored in the *pfnCreateConvertTable* field of the specific pair) is called and the pointer to the just created table is stored in the *ppConvertTable* field. The creation calls for the conversion table of valid format pairs; all of them are defined in DCAFCNVT.C:

- CreateConvertTable_8int_4ext creates a 256-entry table of bytes to convert 8bpp to 4bpp.

  To do this, after the memory allocation, the NearestRestrictedColourIndex (in CONVINT.C) is called, in order to map the RGB values in the HWPalette table to the nearest entry in the VGA standard default palette (see the appendix regarding default color palettes in the *IBM OS/2 Display Device Driver Reference*).

- CreateConvertTable_8int_8ext creates a 256-entry table of bytes to convert 8bpp internal indices to 8bpp external indices (XGA default palette).

  To do this, after the memory allocation, the NearestRestrictedColourIndex (in CONVINT.C) is called, in order to map the RGB values in the HWPalette table to the nearest entry in the FullSizeDeviceDefaultPalette palette (the 256-entry XGA default palette defined in EDDDATA.C).

- CreateConvertTable_4int_4ext creates a 256-entry table of bytes to convert 4bpp internal (fudged) format to 4bpp external (standard VGA) format.

  Pels are converted a pair at a time, which is why the conversion table is 256 entries, rather than the expected 16. To do this, after the memory allocation, the NearestRestrictedColourIndex (in CONVINT.C) is called, in order to map the RGB values in the Reduced16DeviceDefaultPalette table (the 16-entry XGA default palette defined in EDDDATA.C) to the nearest entry in the VGA standard default palette (see the appendix regarding default color palettes in the *IBM OS/2 Display Device Driver Reference*).

  The remaining values in the external table are obtained by combining pairs of conversion values, in the following way:

```
for (i = 0; i < 256; i++)
{
     pConvertTable[i] = (abLocalConvertTable[i >> 4] << 4) |
               (abLocalConvertTable[i & 0x0F]) ; }
```

The next step is to calculate the bounding rectangle of the supplied area to be queried using GreGetRegionBox. If the query area is a region, the Graphics Engine is asked for the first 10 rectangles from the query area and puts their addresses in the Spad.prclCurrent pointer (our local buffer). If the query area is a rectangle, it is made EXCLUSIVE and its address is put in Spad.prclCurrent. A check is done to see if the bounding box (for the region) or the rectangle is a valid ordered rectangle. If it is not, a PMERR_INV_IMAGE_DIMENSION is logged and the call returns 0 (zero). The call also will return 0 if the rectangle is empty. If the bounding box or the rectangle exceeds the screen dimension, PMERR_INV_IMAGE_DIMENSION is logged and the call returns 0.

Next, the destination pointer is moved past the packet header. Then, the cursor is excluded from the bounding rectangle using eddm_excludeCursor in EDDMCCRS.C, after rounding the bounding rectangle up to 8 pel boundaries, because the rectangles may be rounded up to 2 or 8 pel boundaries below (depending upon bpp). The drawing mode is set to always use the real XGA hardware, calling the SetDrawModeHard macro (in EDDMACRO.H).

After setting some ShadowXGARegs fields, WaitForRealHWFunction in HWACCESS.ASM is called to ensure that the hardware is ready. Then, some of the registers that were just set up (calling TransferShadowRegisters(TSR_MAP_A | TSR_COLOUR_MIX) in HWACCESS.ASM) are transferred. Currently, only MAP_A and COLOR_MIX registers can be transferred. MAP_B, COORDINATES and PIXELOP registers will be transferred later (when all of the values are known), since they depend on the rectangle in processing.

At this point, the main loop starts:

- For each rectangle in the Spad.prclCurrent local buffer

```
: adjust the current rectangle coordinates if necessary. We alter the
         coordinates according to format and bits per pel so that we do not
         have to worry about the masking associated with compressing and
         decompressing partial bytes.

         4bpp formats are rounded to 8 pel boundaries because the
         destination could be planar VGA.

         8bpp formats are rounded to even pel boundaries because
         we transmit data in 16-bit fields i.e. two pels per data
         field.

: call CompressRect in COMPRESS.ASM, getting as a return whether or not
         the output buffer is full
: if the output buffer is full
: : break
: endif
: if no more rectangles in the local buffer and a region was supplied and
         there are more rects in engine
: : subtract the already compressed rectangles from the supplied region
```

```
            using GreCombineRectRegion (CRGN_DIFF)
        : : reload local buffer with more rects from engine
        : endif
        endfor
```

- We can get out from the cycle above for two reasons:
  - No more rectangles to process; reset the provided region or rectangle using GreSetRectRegion
  - No more room in the output buffer; subtract the already processed rectangles from the region or the processed area from the provided rectangle.

The number of bytes written is reported in the packet header and the cursor is renabled via reenable_cursor in EDDMCCRS.C. Then, ExitDriver is called and a return code indicating full (return code 1) or partial data returned (return code 2) is issued.

CompressRect (in COMPRESS.ASM)

```
if free bytes in output buffer < size of (RECTANGLE HEADER + WORST_CASE_ROW_LENGTH)
: set a variable saying that there is no room in the output buffer
: return
else
: write out rect header
: for iRow = each row of rect
: : // Check for duplicate scanlines
: : if iRow > first row
: : : if row[iRow] matches row[iRow-1]
: : : : count subsequent matching rows
: : : : write duplicate scanline code + count
: : : endif
: : endif
: : // Check for duplicate scanline pairs
: : if iRow > second row and iRow < last row
: : : if row[iRow] matches row[iRow-2] and row[iRow+1] matches row[iRow-1]
: : : : count subsequent matching row pairs
: : : : write duplicate scanline pair code + count
: : : endif
: : endif
: : // Compress the row
: : call appropriate row compression function
: : if free bytes in output buffer < WORST_CASE_ROW_LENGTH
: : : set a variable saying that there is no room in the output buffer
: : : update the passed rectangle to contain the area that was not compressed
: : : return
: : endif
: endfor
return
endif
```

There is a separate compression function for each of the valid src/dst format combinations (in COMPRESS.ASM), as follows:

```
compress_row_16_16          (src 16bpp packed, dst 16bpp packed)
                                    calling only compression routine

compress_row_16_8           (src 16bpp packed, dst 8bpp packed)
                                    calling: convert_row_16pk_8pk;
                                             compression routine

compress_row_16_4           (src 16bpp packed, dst 4bpp packed)
                                    calling: convert_row_16pk_4pk;
                                             compression routine

compress_row_16_4pl         (src 16bpp packed, dst 4bpp planar)
                                    calling: convert_row_16pk_4pk;
                                             convert_row_4pk_4pl;
                                             compression routine

compress_row_8_8            (src  8bpp packed, dst 8bpp packed)
                                    calling: convert_row_8pkint_8pkext;
                                             compression routine

compress_row_8_4            (src  8bpp packed, dst  4bpp packed)
                                    calling: convert_row_8pk_4pk;
                                             compression routine

compress_row_8_4pl          (src  8bpp packed, dst 4bpp planar)
```

```
                                        calling: convert_row_8pk_4pk;
                                                 convert_row_4pk_4pl;
                                                 compression routine

compress_row_4_4            (src  4bpp packed, dst 4bpp packed)
                                        calling: convert_row_4pkint_4pkext;
                                                 compression routine

compress_row_4_4pl          (src  4bpp packed, dst 4bpp planar)
                                        calling: convert_row_4pkint_4pkext;
                                                 convert_row_4pk_4pl;
                                                 compression routine
```

Conversion between different data formats (for example, 8bpp and 4bpp) is done independently of the data compression. That is, if the source and destination formats differ, the source data is first converted to the destination format (in an intermediate buffer defined on the PHUNK), and then is compressed into the destination buffer. If the source and destination formats match, the data is compressed directly from the source to the destination. The data conversion routines use tables wherever possible to improve performance (see Converting Data in Code Modifications Required in Base Driver for DCAF for more information). The routines defined in COMPRESS.ASM are as follows:

```
convert_row_16pk_8pk        (src 16bpp packed, dst 8bpp packed)
                                 using NearestRestrictedDeviceDefaultPalette
                                 to find, for every pixel, the nearest entry in the
                                 FullSizeDeviceDefaultPalette
                                   (the 256 entry default palette defined
                                     in EDDDATA.C).

convert_row_16pk_4pk        (src 16bpp packed, dst 4bpp packed)
                                 using NearestRestrictedDeviceDefaultPalette
                                 to find, for every pixel, the nearest entry in the
                                 StandardVGADefaultPalette

convert_row_8pkint_8pkext   (scr 8bpp packed, dst 8bpp packed)
                                 using ConvertTable_8int_8ext

convert_row_8pk_4pk         (src 8bpp packed, dst 4bpp packed)
                                 using ConvertTable_8int_4ext

convert_row_4pk_4pl         (src 4bpp packed, dst 4bpp planar)
                                 changes the packed to planar format

convert_row_4pkint_4pkext   (src 4bpp packed, dst 4bpp packed)
                                 using ConvertTable_4int_4ext
```

Additional work is done to make the required rows available for compression. Checks are performed to ensure that a row is available before accessing it. In the DCAF-enabled driver, VRAM is not directly accessible so the required rows have to be loaded into a locked buffer in system memory before they can be worked on. There already exists a 64K buffer in the Driver called the PHUNK (PHysical chUNK), which is ideal for this purpose. Rectangles can easily exceed 64K in size, so this buffer will often be reloaded multiple times during the processing of a single rectangle.

GreGetScreenBits - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreGetScreenChangeArea

GreGetScreenChangeArea - Syntax

Simulation support:

This function is mandatory for display drivers that want to be DCAF-enabled.

Description *GreGetScreenChangeArea* takes an SCA handle and, for the identified SCA, adds its rectangles to the region pointed to by the *phrgn* field. The SCA is reset to NULL as a result of this call.

```
#define INCL_GRE_SCREEN
#include <os2.h>
#include <pmddi.h>

HDC       hdc;      /*  Any valid device context. */
HSCA      hsca;     /*  Handle of the SCA to be queried. */
PHRGN     phrgn;    /*  Pointer to a region handle. */
PDC       pdcArg;   /*  Pointer to instance data. */
ULONG     FunN;     /*  High-order word=Flags; Low-order word=NGreGetScreenArea. */
LONG      rc;

rc = GreGetScreenChangeArea(hdc, hsca, phrgn,
       pdcArg, FunN);
```

GreGetScreenChangeArea Parameter - hdc

hdc (HDC) - input
Any valid device context.

GreGetScreenChangeArea Parameter - hsca

hsca (HSCA) - input
Handle of the SCA to be queried.

To be valid, the handle must be obtained from a previous call to GreOpenScreenChangeArea.

GreGetScreenChangeArea Parameter - phrgn

phrgn (PHRGN) - input
Pointer to a region handle.

GreGetScreenChangeArea Parameter - pdcArg

pdcArg (PDC) - input
Pointer to instance data.

GreGetScreenChangeArea Parameter - FunN

FunN (ULONG) - input
High-order word=Flags; Low-order word=NGreGetScreenArea.

GreGetScreenChangeArea Return Value - rc

rc (LONG) - returns
This function returns one of the following:

TRUE	If the handle passed is recognized and no errors occurred.
FALSE	All other cases.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_HDC
PMERR_NO_HANDLE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetScreenChangeArea - Parameters

hdc (HDC) - input
Any valid device context.

hsca (HSCA) - input
Handle of the SCA to be queried.

To be valid, the handle must be obtained from a previous call to GreOpenScreenChangeArea.

phrgn (PHRGN) - input
Pointer to a region handle.

pdcArg (PDC) - input
Pointer to instance data.

FunN (ULONG) - input
High-order word=Flags; Low-order word=NGreGetScreenArea.

rc (LONG) - returns
This function returns one of the following:

TRUE	If the handle passed is recognized and no errors occurred.
FALSE	All other cases.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_HDC
PMERR_NO_HANDLE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetScreenChangeArea - Remarks

It is permissible to implement this function by returning 0 to indicate that the bits were not saved and, therefore, must be saved by the calling routine.

GreGetScreenChangeArea calls EnterDriver and performs entry checks. Using the MergeSeamlessSCAWithGlobalSCAs routine, GreGetScreenChangeArea then checks to see if there are any rectangles in the Seamless SCA (called scaSeamless). If there are, they are merged with all of the currently active SCAs. MergeSeamlessSCAWithGlobalSCAs will call AccumulateScreenBound for every rectangle in scaSeamless. At the end, the c*Rects* field of scaSeamless is set to zero. See Seamless Windows Support in Distributed Console Access Facility (DCAF) Architecture for more information on Seamless Windows support.

Next, the linked list is tracked until the SCA to be queried is found or the end of the linked list is reached. **If the SCA to be queried is found,** GreCombineRectRegion is called for all the rectangles in the SCA and adds the rectangles to the region passed by the user. If the call is always successful, the c*Rects* field of the SCA is set to zero, resetting the SCA. **If the end of the list is reached without finding the SCA,** a PMERR_NO_ERROR is logged and the call returns FALSE.

If there were no errors, GreGetScreenChangeArea calls ExitDriver and returns TRUE.

GreGetScreenChangeArea - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreGetStyleRatio

GreGetStyleRatio - Syntax

Simulation support:

This function is mandatory for display drivers. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreGetStyleRatio stores the style ratio X-direction and Y-direction step values at the location addressed by pRatio.

```
#define INCL_GRE_DEVMISC2
#include <os2.h>

HDC      hdc;        /*  The device context handle. */
```

```
PBYTE    pRatio;     /*  A pointer to a style ratio value. */
PVOID    pInstance;  /*  A pointer to instance data. */
ULONG    lFunction;  /*  High-order WORD=flags; low-order WORD=NGreGetStyleRatio. */
BOOL     fSuccess;   /*  Return codes. */

fSuccess = GreGetStyleRatio(hdc, pRatio, pInstance,
             lFunction);
```

GreGetStyleRatio Parameter - hdc

hdc (HDC) - input
The device context handle.

GreGetStyleRatio Parameter - pRatio

pRatio (PBYTE) - input
A pointer to a style ratio value.

The style ratio is defined as a 2-byte value. The low-order byte indicates a step in the X-direction, the high-order byte a step in the Y-direction.

GreGetStyleRatio Parameter - pInstance

pInstance (PVOID) - input
A pointer to instance data.

GreGetStyleRatio Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetStyleRatio.

GreGetStyleRatio Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE Successful

FALSE Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

GreGetStyleRatio - Parameters

hdc (HDC) - input
The device context handle.

pRatio (PBYTE) - input
A pointer to a style ratio value.

The style ratio is defined as a 2-byte value. The low-order byte indicates a step in the X-direction, the high-order byte a step in the Y-direction.

pInstance (PVOID) - input
A pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetStyleRatio.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE Successful
FALSE Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

GreGetStyleRatio - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreGetViewingLimits

GreGetViewingLimits - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreGetViewingLimits loads the array indicated by prclViewingLimits with the boundaries of the viewing window in graphic model-space coordinates. All boundaries are inclusive.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_XFORMS
#include <os2.h>

HDC       hdc;                /*  Device context handle. */
PRECTL    prclViewingLimits;  /*  Pointer to limits of viewing area (RECTL structure): */
PVOID     pInstance;          /*  Pointer to instance data. */
ULONG     lFunction;          /*  High-order WORD=flags; low-order WORD=NGreGetViewingLimits. */
BOOL      fSuccess;           /*  Return codes. */

fSuccess = GreGetViewingLimits(hdc, prclViewingLimits,
             pInstance, lFunction);
```

GreGetViewingLimits Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetViewingLimits Parameter - prclViewingLimits

prclViewingLimits (PRECTL) - input
Pointer to limits of viewing area (RECTL structure):

xLeft	Minimum x-coordinate of viewing limits
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of viewing limits
yTop	Maximum y-coordinate

GreGetViewingLimits Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetViewingLimits Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetViewingLimits.

GreGetViewingLimits Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetViewingLimits - Parameters

hdc (HDC) - input
Device context handle.

prclViewingLimits (PRECTL) - input
Pointer to limits of viewing area (RECTL structure):

xLeft	Minimum x-coordinate of viewing limits
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of viewing limits
yTop	Maximum y-coordinate

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetViewingLimits.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetViewingLimits - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreGetWindowViewportXform

GreGetWindowViewportXform - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreGetWindowViewportXform queries the current window or viewport transform matrix. On completion, paXformData is an array of two-dimensional values defining the current window or viewport transform matrix.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_XFORMS
#include <os2.h>

HDC       hdc;          /*  Device context handle. */
PXFORM    paXformData;  /*  Pointer to return data. */
PVOID     pInstance;    /*  Pointer to instance data. */
ULONG     lFunction;    /*  High-order WORD=flags; low-order WORD=NGreGetWindowViewportXform. */
BOOL      fSuccess;     /*  Return codes. */

fSuccess = GreGetWindowViewportXform(hdc,
            paXformData, pInstance, lFunction);
```

GreGetWindowViewportXform Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetWindowViewportXform Parameter - paXformData

paXformData (PXFORM) - input
Pointer to return data.

Pointer to return data in which the array of six matrix elements (M11, M12, M21, M22, M41, and M42) are to be stored.

GreGetWindowViewportXform Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetWindowViewportXform Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetWindowViewportXform.

GreGetWindowViewportXform Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetWindowViewportXform - Parameters

hdc (HDC) - input
Device context handle.

paXformData (PXFORM) - input
Pointer to return data.

Pointer to return data in which the array of six matrix elements (M11, M12, M21, M22, M41, and M42) are to be stored.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input

High-order WORD=flags; low-order WORD=NGreGetWindowViewportXform.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreGetWindowViewportXform - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreImageData

GreImageData - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreImageData draws a single row of image data with one bit per pel by using the current image foreground and background color and mix attributes.

This function must be supported by the presentation driver. GreImageData is called by the function GpiImage. GreImageData is called multiple times for each call made to GpiImage, once for each scan line in the monochrome bit map.

```
#define INCL_GRE_BITMAPS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
PBYTE    pData;       /*  Pointer to data string. */
LONG     cBits;       /*  Number of bits in row (maximum is 2040). */
ULONG    offRow;      /*  Row number relative to the current y-position. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreImageData. */
LONG     rc;          /*  Return Code. */

rc = GreImageData(hdc, pData, cBits, offRow,
       pInstance, lFunction);
```

GreImageData Parameter - hdc

hdc (HDC) - input
Device context handle.

GreImageData Parameter - pData

pData (PBYTE) - input
Pointer to data string.

GreImageData Parameter - cBits

cBits (LONG) - input
Number of bits in row (maximum is 2040).

GreImageData Parameter - offRow

offRow (ULONG) - input
Row number relative to the current y-position.

Zero is the current position. A value of 1 indicates one row below the current position.

GreImageData Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreImageData Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreImageData.

GreImageData Return Value - rc

rc (LONG) - returns
Return Code.

On completion, the handling routine must return a LONG integer (cHits), indicating whether correlation hits have been detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_IMAGE_DATA_LENGTH
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_SCAN_START

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreImageData - Parameters

hdc (HDC) - input
Device context handle.

pData (PBYTE) - input
Pointer to data string.

cBits (LONG) - input
Number of bits in row (maximum is 2040).

offRow (ULONG) - input
Row number relative to the current y-position.

Zero is the current position. A value of 1 indicates one row below the current position.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreImageData.

rc (LONG) - returns
Return Code.

On completion, the handling routine must return a LONG integer (cHits), indicating whether correlation hits have been detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_IMAGE_DATA_LENGTH
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_SCAN_START

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreImageData - Remarks

Drawing starts at the current X-position and at a Y-position defined as a row offset below the current Y-position. Data is supplied as a series of bytes and a count of the bits to be drawn. The handling routine cannot assume that unused bits at the end of the stream are set to 0. Bits are drawn from left to right. The high-order bit of each byte is the leftmost-image bit. The 1 bits are foreground and the 0 bits are background. Note that this function does not affect the current position.

GreImageData - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreIntersectClipRectangle

GreIntersectClipRectangle - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreIntersectClipRectangle sets the new clipping region to the intersection of the current clip region and the specified rectangle.

This function is supported by the graphics engine and can be hooked by the presentation driver.

```
#define INCL_GRE_CLIP
#include <os2.h>

HDC       hdc;         /*  Device context handle. */
PRECTL    prclRect;    /*  Pointer to rectangle (RECTL structure) in world or device coordinates: */
PVOID     pInstance;   /*  Pointer to instance data. */
ULONG     lFunction;   /*  High-order WORD=flags; low-order WORD=NGreIntersectClipRectangle. */
LONG      rc;          /*  Return codes. */

rc = GreIntersectClipRectangle(hdc, prclRect,
       pInstance, lFunction);
```

GreIntersectClipRectangle Parameter - hdc

hdc (HDC) - input
Device context handle.

GreIntersectClipRectangle Parameter - prclRect

prclRect (PRECTL) - input
Pointer to rectangle (RECTL structure) in world or device coordinates:

xLeft	Minimum x-coordinate of rectangle
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of rectangle
yTop	Maximum y-coordinate All the boundaries of the rectangle are inclusive (part of the interior) and are not clipped. If COM_TRANSFORM is not set, the function expects the rectangle to be in device coordinates.

GreIntersectClipRectangle Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreIntersectClipRectangle Parameter - lFunction

lFunction (ULONG) - input

High-order WORD=flags; low-order WORD=NGreIntersectClipRectangle.

GreIntersectClipRectangle Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (lComplexity) indicating the complexity of the DC region:

RGN_ERROR	Error
RGN_NULL	Null region
RGN_RECT	Rectangular region
RGN_COMPLEX	Complex region (more than 1 rectangle)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreIntersectClipRectangle - Parameters

hdc (HDC) - input
Device context handle.

prclRect (PRECTL) - input
Pointer to rectangle (RECTL structure) in world or device coordinates:

xLeft	Minimum x-coordinate of rectangle
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of rectangle
yTop	Maximum y-coordinate All the boundaries of the rectangle are inclusive (part of the interior) and are not clipped. If COM_TRANSFORM is not set, the function expects the rectangle to be in device coordinates.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreIntersectClipRectangle.

rc (LONG) - returns
Return codes.

This function returns an integer (lComplexity) indicating the complexity of the DC region:

RGN_ERROR	Error
RGN_NULL	Null region
RGN_RECT	Rectangular region

RGN_COMPLEX Complex region (more than 1 rectangle)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreIntersectClipRectangle - Remarks

When no clip region exists, GreIntersectClipRectangle must create one. The application must then free the handle when it subsequently selects another clip region. The return value of this function is the complexity of the resultant DC region, which is defined as the intersection of all clipping.

GreIntersectClipRectangle - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreLockDevice

GreLockDevice - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreLockDevice locks a device for use by a single thread.

This function must be supported by all presentation drivers. GreLockDevice prevents two separate processes from accessing the resource (device context) at the same time. Hardcopy drivers need do nothing except return TRUE (Successful).

```
#define INCL_GRE_DEVMISC3
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreLockDevice. */
BOOL     rc;          /*  Return Code. */

rc = GreLockDevice(hdc, pInstance, lFunction);
```

GreLockDevice Parameter - hdc

hdc (HDC) - input
Device context handle.

GreLockDevice Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreLockDevice Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreLockDevice.

GreLockDevice Return Value - rc

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreLockDevice - Parameters

hdc (HDC) - input
Device context handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreLockDevice.

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreLockDevice - Remarks

This function synchronizes the use and update of the visible region by allowing all current and pending drawings to finish and then blocking any requests to draw from other threads until GreUnlockDevice is called. On exit, the only thread allowed to continue with screen operations is the one that acquires the lock. To prevent deadlock, GreDeath cannot be called while the visible region is locked.

GreLockDevice - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreModifyPath

GreModifyPath - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreModifyPath modifies a path. When the transform is singular, GreModifyPath is invalid and causes an error to be logged.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_PATHS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
LONG     idPath;      /*  Path ID. */
LONG     cmdMode;     /*  Modify mode. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreModifyPath. */
BOOL     fSuccess;    /*  Return codes. */

fSuccess = GreModifyPath(hdc, idPath, cmdMode,
             pInstance, lFunction);
```

GreModifyPath Parameter - hdc

hdc (HDC) - input
Device context handle.

GreModifyPath Parameter - idPath

idPath (LONG) - input
Path ID.

GreModifyPath Parameter - cmdMode

cmdMode (LONG) - input
Modify mode.

The only valid flag is:

MPATH_STROKE	This replaces the original path (with a path that encloses the shape produced by stroking the original path) using the current geometric wide line attribute. Any open figures within the path are not closed. The envelope includes the effects of "line joins" and "line ends" according to the current values of these attributes. For example, where a line joins an arc, the common point is handled according to the line join attribute in the current line attribute bundle. (See Line Attributes.) When a figure is closed by using GreCloseFigure, the line join attribute in the current line bundle is applied to the start and end points. The envelope takes account of any crossings. For example, a stroked X does not have a hole in the middle if it is subsequently drawn in

Exclusive-OR mode.

GreModifyPath Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreModifyPath Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreModifyPath.

GreModifyPath Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_FILL_PATH_OPTIONS
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MATRIX_ELEMENT
PMERR_INV_MODIFY_PATH_MODE
PMERR_INV_PATH_ID
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_TRANSFORM_TYPE
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreModifyPath - Parameters

hdc (HDC) - input
Device context handle.

idPath (LONG) - input
Path ID.

cmdMode (LONG) - input
Modify mode.

The only valid flag is:

MPATH_STROKE	This replaces the original path (with a path that encloses the shape produced by stroking the original path) using the current geometric wide line attribute. Any open figures within the path are not closed. The envelope includes the effects of "line joins" and "line ends" according to the current values of these attributes. For example, where a line joins an arc, the common point is handled according to the line join attribute in the current line attribute bundle. (See Line Attributes.) When a figure is closed by using GreCloseFigure, the line join attribute in the current line bundle is applied to the start and end points. The envelope takes account of any crossings. For example, a stroked X does not have a hole in the middle if it is subsequently drawn in Exclusive-OR mode.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreModifyPath.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_FILL_PATH_OPTIONS
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MATRIX_ELEMENT
PMERR_INV_MODIFY_PATH_MODE
PMERR_INV_PATH_ID
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

PMERR_INV_TRANSFORM_TYPE
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreModifyPath - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreMultiplyXforms

GreMultiplyXforms - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

*GreMultiplyXform*s multiplies the transform matrix.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_XFORMS
#include <os2.h>

HDC       hdc;              /*  Device context handle. */
PXFORM    paXform;
PXFORM    paNewXformData;
LONG      lMode;            /*  Specifies how supplied array is used to set matrix: */
PVOID     pInstance;        /*  Pointer to instance data. */
ULONG     lFunction;
BOOL      fSuccess;         /*  Return codes. */

fSuccess = GreMultiplyXforms(hdc, paXform,
             paNewXformData, lMode, pInstance,
             lFunction);
```

GreMultiplyXforms Parameter - hdc

hdc (HDC) - input

Device context handle.

GreMultiplyXforms Parameter - paXform

paXform (PXFORM) - output
On input: Transform used during lMode operation.

On output: Result of the lMode operation.

GreMultiplyXforms Parameter - paNewXformData

paNewXformData (PXFORM) - input
Transform used during lMode operation.

GreMultiplyXforms Parameter - lMode

lMode (LONG) - input
Specifies how supplied array is used to set matrix:

SX_UNITY	Set unity transform, ignore array values
SX_CAT_AFTER	Concatenate after
SX_CAT_BEFORE	Concatenate before
SX_OVERWRITE	Overwrite

GreMultiplyXforms Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreMultiplyXforms Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreMultiplyXforms.

GreMultiplyXforms Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_MATRIX_ELEMENT
PMERR_INV_TRANSFORM_TYPE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreMultiplyXforms - Parameters

hdc (HDC) - input
Device context handle.

paXform (PXFORM) - output
On input: Transform used during lMode operation.

On output: Result of the lMode operation.

paNewXformData (PXFORM) - input
Transform used during lMode operation.

lMode (LONG) - input
Specifies how supplied array is used to set matrix:

SX_UNITY	Set unity transform, ignore array values
SX_CAT_AFTER	Concatenate after
SX_CAT_BEFORE	Concatenate before
SX_OVERWRITE	Overwrite

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreMultiplyXforms.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_MATRIX_ELEMENT
PMERR_INV_TRANSFORM_TYPE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreMultiplyXforms - Remarks

This function multiplies the transform matrix (defined by paNewXformData) by the corresponding matrix in paXform. The result is stored in paXform. When this function is used to make a series of matrix multiplications on the same matrix, some loss of accuracy can occur due to rounding because no higher precision can be retained across calls.

GreMultiplyXforms - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreNotifyClipChange

GreNotifyClipChange - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreNotifyClipChange is called whenever there is any change to the DC region. The call gives the presentation driver an opportunity to optimize clipping by enumerating the clip rectangles and caching them whenever they change.

This function must be supported by the presentation driver. GreNotifyClipChange is called from the graphics engine whenever an application calls a GPI function that modifies the clipping rectangles within the device context. This function can be handled by bit-map simulation.

```
#define INCL_GRE_DEVMISC2
#include <os2.h>

HDC       hdc;          /*  Device context handle. */
PRECTL    prclBound;    /*  Pointer to a rectangle that bounds the new clip region in screen coordinates. */
ULONG     cRect;        /*  Number of rectangles in the new clip region as returned by GetClipRects. */
ULONG     idClipPath;   /*  Current ClipPath ID. */
PVOID     pInstance;    /*  Pointer to instance data. */
ULONG     lFunction;    /*  High-order WORD=flags; low-order WORD=NGreNotifyClipChange. */
BOOL      rc;           /*  Return Code. */

rc = GreNotifyClipChange(hdc, prclBound, cRect,
       idClipPath, pInstance, lFunction);
```

GreNotifyClipChange Parameter - hdc

hdc (HDC) - input

Device context handle.

GreNotifyClipChange Parameter - prclBound

prclBound (PRECTL) - input
Pointer to a rectangle that bounds the new clip region in screen coordinates.

This rectangle is inclusive on the left and bottom borders, and exclusive on the right and top borders.

GreNotifyClipChange Parameter - cRect

cRect (ULONG) - input
Number of rectangles in the new clip region as returned by GetClipRects.

GreNotifyClipChange Parameter - idClipPath

idClipPath (ULONG) - input
Current ClipPath ID.

For display devices, when idClipPath is passed as NCC_CLEANDC, the handling routine should clear the HDC_IS_DIRTY flag and return Successful. See VisRegionNotify and GreDeviceInvalidateVisRegion.

Presentation drivers for other devices ignore this parameter.

GreNotifyClipChange Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreNotifyClipChange Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreNotifyClipChange.

GreNotifyClipChange Return Value - rc

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Note: Presentation drivers that do not cache the clip rectangles should return TRUE, if there are no errors.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreNotifyClipChange - Parameters

hdc (HDC) - input
Device context handle.

prclBound (PRECTL) - input
Pointer to a rectangle that bounds the new clip region in screen coordinates.

This rectangle is inclusive on the left and bottom borders, and exclusive on the right and top borders.

cRect (ULONG) - input
Number of rectangles in the new clip region as returned by GetClipRects.

idClipPath (ULONG) - input
Current ClipPath ID.

For display devices, when idClipPath is passed as NCC_CLEANDC, the handling routine should clear the HDC_IS_DIRTY flag and return Successful. See VisRegionNotify and GreDeviceInvalidateVisRegion.

Presentation drivers for other devices ignore this parameter.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreNotifyClipChange.

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Note: Presentation drivers that do not cache the clip rectangles should return TRUE, if there are no errors.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORDINATE
PMERR_INV_HDC

PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreNotifyClipChange - Remarks

Typically, the handling routine would allocate more memory for the new DC region (when necessary) and call GreGetClipRects (see GreGetClipRects) to get the set of rectangles that define the new DC region. Also (for display devices only), the handling routine must clear the HDC_IS_DIRTY flag.

On completion, the handling routine must re-dispatch this call to the graphics engine using the pointer supplied in the default dispatch table.

GreNotifyClipChange - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreNotifyTransformChange

GreNotifyTransformChange - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

*GreNotifyTransformChang*e notifies the presentation driver of a change in the transform from world coordinates to device coordinates.

This function must be supported by the presentation driver.

```
#define INCL_GRE_DEVMISC2
#include <os2.h>

HDC                     hdc;         /*  Device context handle. */
ULONG                   flFlags;     /*  Flags. */
PNOTIFYTRANSFORMDATA    pXformdata;  /*  Pointer to NOTIFYTRANSFORMDATA structure: */
PVOID                   pInstance;   /*  Pointer to instance data. */
ULONG                   lFunction;   /*  High-order WORD=flags; low-order WORD=NGreNotifyTransformChange. */
BOOL                    rc;          /*  Return Code. */

rc = GreNotifyTransformChange(hdc, flFlags,
       pXformdata, pInstance, lFunction);
```

GreNotifyTransformChange Parameter - hdc

hdc (HDC) - input
Device context handle.

GreNotifyTransformChange Parameter - flFlags

flFlags (ULONG) - input
Flags.

A set of flags that pass information regarding the complexity of the 2x2 matrix of the M11, M12, M21, and M22 components (of the composite transform from world coordinates to device coordinates). These flags indicate if a translation is required. If the MATRIX_SIMPLE flag is not set, none of the other flags are valid.

MATRIX_SIMPLE	Two entries are *0* .
MATRIX_UNITS	All entries are *+1* or *-1* .
MATRIX_XY_EXCHANGE	Zeros are on the diagonal.
MATRIX_X_NEGATE	X is hit by negative (see Note).
MATRIX_Y_NEGATE	Y is hit by negative (see Note).
MATRIX_TRANSLATION	Non-zero translation.

Note: MATRIX_X_NEGATE and MATRIX_Y_NEGATE are only meaningful when both MATRIX_SIMPLE and MATRIX_UNITS are set.

Examples:

```
Matrix = { 1.0, 0.0, 0.0, 1.0, 0, 0 } =>
Flags = MATRIX_SIMPLE | MATRIX_UNITS

Matrix = { 1.0, 0.0, 0.0, 1.0, 5, 10 } =>
Flags = MATRIX_SIMPLE | MATRIX_UNITS | MATRIX_TRANSLATION

Matrix = { 0.0, -1.0, 1.0, 0.0, 17, 5 } =>
Flags = MATRIX_SIMPLE | MATRIX_UNITS | MATRIX_XY_EXCHANGE |
MATRIX_Y_NEGATE | MATRIX_TRANSLATION
```

GreNotifyTransformChange Parameter - pXformdata

pXformdata (PNOTIFYTRANSFORMDATA) - input
Pointer to NOTIFYTRANSFORMDATA structure:

usType	Indicates fixed-point notation	
xform	XFORM data structure contains fixed-point values	
	fxM11	Fixed-point matrix elements
	fxM12	Fixed-point matrix elements
	fxM21	Fixed-point matrix elements
	fxM22	Fixed-point matrix elements
	lM41	Long translations
	lM42	Long translations

GreNotifyTransformChange Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreNotifyTransformChange Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreNotifyTransformChange.

GreNotifyTransformChange Return Value - rc

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PICK_APERTURE_POSN
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreNotifyTransformChange - Parameters

hdc (HDC) - input
Device context handle.

flFlags (ULONG) - input
Flags.

A set of flags that pass information regarding the complexity of the 2x2 matrix of the M11, M12, M21, and M22 components (of the composite transform from world coordinates to device coordinates). These flags indicate if a translation is required. If the MATRIX_SIMPLE flag is not set, none of the other flags are valid.

MATRIX_SIMPLE	Two entries are *0* .
MATRIX_UNITS	All entries are *+1* or *-1* .
MATRIX_XY_EXCHANGE	Zeros are on the diagonal.
MATRIX_X_NEGATE	X is hit by negative (see Note).
MATRIX_Y_NEGATE	Y is hit by negative (see Note).
MATRIX_TRANSLATION	Non-zero translation.

Note: MATRIX_X_NEGATE and MATRIX_Y_NEGATE are only meaningful when both MATRIX_SIMPLE and MATRIX_UNITS are set.

Examples:

```
Matrix = { 1.0, 0.0, 0.0, 1.0, 0, 0 } =>
Flags = MATRIX_SIMPLE | MATRIX_UNITS

Matrix = { 1.0, 0.0, 0.0, 1.0, 5, 10 } =>
Flags = MATRIX_SIMPLE | MATRIX_UNITS | MATRIX_TRANSLATION

Matrix = { 0.0, -1.0, 1.0, 0.0, 17, 5 } =>
Flags = MATRIX_SIMPLE | MATRIX_UNITS | MATRIX_XY_EXCHANGE |
MATRIX_Y_NEGATE | MATRIX_TRANSLATION
```

pXformdata (PNOTIFYTRANSFORMDATA) - input
Pointer to NOTIFYTRANSFORMDATA structure:

usType	Indicates fixed-point notation	
xform	XFORM data structure contains fixed-point values	
	fxM11	Fixed-point matrix elements
	fxM12	Fixed-point matrix elements
	fxM21	Fixed-point matrix elements
	fxM22	Fixed-point matrix elements
	lM41	Long translations
	lM42	Long translations

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreNotifyTransformChange.

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PICK_APERTURE_POSN
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreNotifyTransformChange - Remarks

This function is called by the graphics engine to provide sufficient information to allow the device to optimize its calling of the GreConvert function or, where possible, to make all point transformations itself. The minimum requirement is for the handling routine to update the current position and pattern origin, and then call back to the GreNotifyTransformChange routine in the graphics engine. Note that the handling routine must:

- Check that the new current position is valid.
- Check that the change will not cause an overflow of the 16-bit coordinates for the device space.
- Fail safe. If an error is detected or the call back to the graphics engine fails, the routine must restore the initial values before returning FALSE.

GreNotifyTransformChange - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreOffsetClipRegion

GreOffsetClipRegion - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreOffsetClipRegion moves the clipping region by the specified amount.

This function is supported by the graphics engine and can be hooked by the presentation driver.

```
#define INCL_GRE_CLIP
#include <os2.h>

HDC         hdc;         /*  Device context handle. */
PPOINTL     pdpt;        /*  Pointer to offset. */
PVOID       pInstance;   /*  Pointer to instance data. */
ULONG       lFunction;   /*  High-order WORD=flags; low-order WORD=NGreOffsetClipRegion. */
LONG        rc;          /*  Return codes. */

rc = GreOffsetClipRegion(hdc, pdpt, pInstance,
       lFunction);
```

GreOffsetClipRegion Parameter - hdc

hdc (HDC) - input
Device context handle.

GreOffsetClipRegion Parameter - pdpt

pdpt (PPOINTL) - input
Pointer to offset.

Pointer to offset by which the clipping region is to be moved, in world coordinates:

dx	X offset
dy	Y offset

GreOffsetClipRegion Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreOffsetClipRegion Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreOffsetClipRegion.

GreOffsetClipRegion Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (lComplexity) indicating the complexity of the region:

RGN_ERROR	Error
RGN_NULL	Null region
RGN_RECT	Rectangular region
RGN_COMPLEX	Complex region (more than 1 rectangle)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORD_OFFSET
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE

PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreOffsetClipRegion - Parameters

hdc (HDC) - input
Device context handle.

pdpt (PPOINTL) - input
Pointer to offset.

Pointer to offset by which the clipping region is to be moved, in world coordinates:

dx	X offset
dy	Y offset

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreOffsetClipRegion.

rc (LONG) - returns
Return codes.

This function returns an integer (lComplexity) indicating the complexity of the region:

RGN_ERROR	Error
RGN_NULL	Null region
RGN_RECT	Rectangular region
RGN_COMPLEX	Complex region (more than 1 rectangle)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORD_OFFSET
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreOffsetClipRegion - Remarks

The value returned is the complexity of the resultant DC region, that is, the intersection of all clipping (such as clip path, viewing limits, graphics field, clip region, and visible region).

GreOffsetClipRegion - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreOffsetRegion

GreOffsetRegion - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreOffsetRegion moves the given region by the specified amounts (unless the region is in use as a clipping region when an error is raised).

This function can be hooked by the presentation driver.

```
#define INCL_GRE_REGIONS
#include <os2.h>

HDC         hdc;         /*  Device context handle. */
HRGN        hrgn;        /*  Region handle. */
PPOINTL     pdpt;        /*  Pointer to offset by which region is to be moved, in device coordinates: */
PVOID       pInstance;   /*  Pointer to instance data. */
ULONG       lFunction;   /*  High-order WORD=flags; low-order WORD=NGreOffsetRegion. */
BOOL        fSuccess;    /*  Return codes. */

fSuccess = GreOffsetRegion(hdc, hrgn, pdpt,
             pInstance, lFunction);
```

GreOffsetRegion Parameter - hdc

hdc (HDC) - input
Device context handle.

GreOffsetRegion Parameter - hrgn

hrgn (HRGN) - input
Region handle.

GreOffsetRegion Parameter - pdpt

pdpt (PPOINTL) - input
Pointer to offset by which region is to be moved, in device coordinates:

dx	X offset
dy	Y offset

GreOffsetRegion Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreOffsetRegion Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreOffsetRegion.

GreOffsetRegion Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_HRGN_BUSY
PMERR_INV_COORDINATE
PMERR_INV_HRGN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreOffsetRegion - Parameters

hdc (HDC) - input
Device context handle.

hrgn (HRGN) - input
Region handle.

pdpt (PPOINTL) - input
Pointer to offset by which region is to be moved, in device coordinates:

dx	X offset
dy	Y offset

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreOffsetRegion.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_HRGN_BUSY
PMERR_INV_COORDINATE
PMERR_INV_HRGN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreOffsetRegion - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreOpenScreenChangeArea

GreOpenScreenChangeArea - Syntax

Simulation support:

This function is mandatory for display drivers that want to be DCAF-enabled.

Description *GreOpenScreenChangeArea* allocates a data area internal to the display driver, in which the driver will accumulate screen changes. The function returns a 32-bit handle that is required to identify the area in GreGetScreenChangeArea and GreCloseScreenChangeArea calls.

```
#define INCL_GRE_SCREEN
#include <os2.h>
#include <pmddi.h>

HDC       hdc;      /*  Any device context handle. */
PDC       pdcArg;   /*  Pointer to instance data. */
ULONG     FunN;     /*  High-order word=Flags; Low-order word=NGreOpenScreenChangeArea. */
LONG      rc;

rc = GreOpenScreenChangeArea(hdc, pdcArg,
       FunN);
```

GreOpenScreenChangeArea Parameter - hdc

hdc (HDC) - input
Any device context handle.

GreOpenScreenChangeArea Parameter - pdcArg

pdcArg (PDC) - input
Pointer to instance data.

GreOpenScreenChangeArea Parameter - FunN

FunN (ULONG) - input
High-order word=Flags; Low-order word=NGreOpenScreenChangeArea.

GreOpenScreenChangeArea Return Value - rc

rc (LONG) - returns
This function returns one of the following:

hsca	The handle of the new screen change area (SCA).
GPI_ERROR	There was no memory available. The memory allocation failure will have been logged by the memory allocation routine.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_HDC
PMERR_MEMORY_ALLOCATION_ERR

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreOpenScreenChangeArea - Parameters

hdc (HDC) - input
Any device context handle.

pdcArg (PDC) - input
Pointer to instance data.

FunN (ULONG) - input
High-order word=Flags; Low-order word=NGreOpenScreenChangeArea.

rc (LONG) - returns
This function returns one of the following:

hsca	The handle of the new screen change area (SCA).
GPI_ERROR	There was no memory available. The memory allocation failure will have been logged by the memory allocation routine.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_HDC
PMERR_MEMORY_ALLOCATION_ERR

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreOpenScreenChangeArea - Remarks

It is permissible to implement this function by returning 0 (zero) to indicate that the bits were not saved and, therefore, must be saved by the calling routine.

If EnterDriver is successful, GreOpenScreenChangeArea attempts to create a new SCA instance by calling AllocateMemory, a common routine in MEMMAN.C. The new SCA gets initialized to NULL and is added to the SCA linked list.

If the creation is successful and this is the first SCA to be opened, GreSetProcessControl is used to tell the graphics engine to turn on the COM_SCR_BOUND flag and start accumulating screen bounds. Setting this bit causes subsequent screen drawing functions to accumulate the clipped bounds into the active SCAs.

If there is not enough memory to create the SCA, PMERR_MEMORY_ALLOCATION_ERROR is logged and GPI_ERROR is returned.

Before returning either the handle of the new SCA or GPI_ERROR, ExitDriver is called.

GreOpenScreenChangeArea - Topics

Select an item:
Syntax

Parameters
Returns
Remarks
Glossary

GreOutlinePath

GreOutlinePath - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreOutlinePath draws the boundary of the path indicated by idPath. GreOutlinePath is also used to draw area boundaries.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_PATHS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
LONG     idPath;      /*  Path ID. It must be 1. */
ULONG    flOptions;   /*  Outline path flags. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreOutlinePath. */
LONG     rc;          /*  Return codes. */

rc = GreOutlinePath(hdc, idPath, flOptions,
       pInstance, lFunction);
```

GreOutlinePath Parameter - hdc

hdc (HDC) - input
Device context handle.

GreOutlinePath Parameter - idPath

idPath (LONG) - input
Path ID. It must be 1.

GreOutlinePath Parameter - flOptions

flOptions (ULONG) - input
Outline path flags.

Possible values are:

OPTH_NO_CLIPPING_REQD	The graphics engine will outline the path without clipping and line attributes set. This value can be used for devices that have hardware support for area fills with at least simple clipping. This output will be sent to the presentation driver via GreDrawLinesInPath for each subpath. This flag should only be used with the graphics engine version number 0x202 or later (see GreQueryEngineVersion).
OPTH_QRY_PATH_POINTS	Returns the number of points in a path. The path is not deleted. This flag should only be used with the graphics engine version number 0x202 or later (see GreQueryEngineVersion).

GreOutlinePath Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreOutlinePath Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreOutlinePath.

GreOutlinePath Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_FILL_PATH_OPTIONS
PMERR_INV_HDC

PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PATH_ID
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_PATH_UNKNOWN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreOutlinePath - Parameters

hdc (HDC) - input
Device context handle.

idPath (LONG) - input
Path ID. It must be 1.

flOptions (ULONG) - input
Outline path flags.

Possible values are:

OPTH_NO_CLIPPING_REQD	The graphics engine will outline the path without clipping and line attributes set. This value can be used for devices that have hardware support for area fills with at least simple clipping. This output will be sent to the presentation driver via GreDrawLinesInPath for each subpath. This flag should only be used with the graphics engine version number 0x202 or later (see GreQueryEngineVersion).
OPTH_QRY_PATH_POINTS	Returns the number of points in a path. The path is not deleted. This flag should only be used with the graphics engine version number 0x202 or later (see GreQueryEngineVersion).

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreOutlinePath.

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_FILL_PATH_OPTIONS
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PATH_ID
PMERR_INV_PICK_APERTURE_POSN

PMERR_INV_RECT
PMERR_PATH_UNKNOWN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreOutlinePath - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GrePaintRegion

GrePaintRegion - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GrePaintRegion paints the specified region by using the current area foreground and background colors.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_REGIONS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
HRGN     hrgn;        /*  Region handle. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGrePaintRegion. */
LONG     rc;          /*  Return codes. */

rc = GrePaintRegion(hdc, hrgn, pInstance,
       lFunction);
```

GrePaintRegion Parameter - hdc

hdc (HDC) - input
Device context handle.

GrePaintRegion Parameter - hrgn

hrgn (HRGN) - input
Region handle.

GrePaintRegion Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GrePaintRegion Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePaintRegion.

GrePaintRegion Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePaintRegion - Parameters

hdc (HDC) - input

Device context handle.

hrgn (HRGN) - input
Region handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePaintRegion.

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePaintRegion - Remarks

Mixing is controlled only by the area foreground mix. GrePaintRegion returns an error when the region is currently selected as a clip region.

If the preprocessor in the graphics engine receives a GrePaintRegion call for a null region, the preprocessor returns GPI_OK and does not forward the call through the dispatch table to the handling routine. Presentation drivers for devices that do not support BitBlt operations (for example, vector devices such as plotters) do not hook this function. Such drivers hook only the function to return an error code.

GrePaintRegion - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GrePartialArc

GrePartialArc - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GrePartialArc draws a straight line from the current position to the starting point of a partial arc and draws the specified partial arc.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_ARCS
#include <os2.h>

HDC         hdc;            /*  Device context handle. */
PPOINTL     pptlCenter;     /*  Pointer to (X,Y) coordinates for center of arc. */
FIXED       fxMultiplier;   /*  Multiplier. */
FIXED       fxStart;        /*  Start angle. */
FIXED       fxSweep;        /*  Sweep angle. */
PVOID       pInstance;      /*  Pointer to instance data. */
ULONG       lFunction;      /*  High-order WORD=flags; low-order WORD=NGrePartialArc. */
LONG        rc;             /*  Return codes. */

rc = GrePartialArc(hdc, pptlCenter, fxMultiplier,
       fxStart, fxSweep, pInstance, lFunction);
```

GrePartialArc Parameter - hdc

hdc (HDC) - input
Device context handle.

GrePartialArc Parameter - pptlCenter

pptlCenter (PPOINTL) - input
Pointer to (X,Y) coordinates for center of arc.

If COM_TRANSFORM is not set, the function expects the center of the arc position to be in screen coordinates.

GrePartialArc Parameter - fxMultiplier

fxMultiplier (FIXED) - input
Multiplier.

This parameter defines the size of the full arc in relation to an arc drawn with the current arc parameters. The multiplier is a fixed-point value. The high-order WORD contains the integer portion; the low-order WORD contains the fractional portion. A value of 64KB gives a multiplier of 1. There is an implementation limit of 255 for this value, which must not be negative.

GrePartialArc Parameter - fxStart

fxStart (FIXED) - input
Start angle.

Start angle that defines the starting point on the curve. Start and sweep angles are measured counterclockwise from the X-axis at the center of the arc before the arc parameters are applied. If the current arc parameters do not specify a circle, the angles are skewed to the same degree that the full arc is a skewed circle.

The angles are specified as doubleword values in fixed-point format. The high-order WORD contains the integer portion; the low-order WORD contains the fractional portion. A value of 6553 gives an angle of 1°. Both angles must be positive. Whether the arc is drawn in a clockwise or counterclockwise direction is determined by the arc parameters. An angle greater than 360° is also valid. In this case, after the initial line, a full arc is drawn followed by a partial arc of (lSweep MOD 360)°. See also the function "GpiPartialArc" in the*Presentation Manager Programming Referenc*e .

GrePartialArc Parameter - fxSweep

fxSweep (FIXED) - input
Sweep angle.

Sweep angle that defines the extent of the curve to be drawn. Start and sweep angles are measured counterclockwise from the X-axis at the center of the arc before the arc parameters are applied. If the current arc parameters do not specify a circle, the angles are skewed to the same degree that the full arc is a skewed circle.

The angles are specified as doubleword values in fixed-point format. The high-order WORD contains the integer portion; the low-order WORD contains the fractional portion. A value of 6553 gives an angle of 1°. Both angles must be positive. Whether the arc is drawn in a clockwise or counterclockwise direction is determined by the arc parameters. An angle greater than 360° is also valid. In this case, after the initial line, a full arc is drawn followed by a partial arc of (lSweep MOD 360)°. See also the function "GpiPartialArc" in the*Presentation Manager Programming Referenc*e .

GrePartialArc Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GrePartialArc Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePartialArc.

GrePartialArc Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_ANGLE_PARM
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MULTIPLIER
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePartialArc - Parameters

hdc (HDC) - input
Device context handle.

pptlCenter (PPOINTL) - input
Pointer to (X,Y) coordinates for center of arc.

If COM_TRANSFORM is not set, the function expects the center of the arc position to be in screen coordinates.

fxMultiplier (FIXED) - input
Multiplier.

This parameter defines the size of the full arc in relation to an arc drawn with the current arc parameters. The multiplier is a fixed-point value. The high-order WORD contains the integer portion; the low-order WORD contains the fractional portion. A value of 64KB gives a multiplier of 1. There is an implementation limit of 255 for this value, which must not be negative.

fxStart (FIXED) - input
Start angle.

Start angle that defines the starting point on the curve. Start and sweep angles are measured counterclockwise from the X-axis at the center of the arc before the arc parameters are applied. If the current arc parameters do not specify a circle, the angles are skewed to the same degree that the full arc is a skewed circle.

The angles are specified as doubleword values in fixed-point format. The high-order WORD contains the integer portion; the low-order WORD contains the fractional portion. A value of 6553 gives an angle of 1°. Both angles must be positive. Whether the arc is drawn in a clockwise or counterclockwise direction is determined by the arc parameters. An angle greater than 360° is also valid. In this case, after the initial line, a full arc is drawn followed by a partial arc of (lSweep MOD 360)°. See also the function "GpiPartialArc" in the*Presentation Manager Programming Reference* .

fxSweep (FIXED) - input
Sweep angle.

Sweep angle that defines the extent of the curve to be drawn. Start and sweep angles are measured counterclockwise from the X-axis at the center of the arc before the arc parameters are applied. If the current arc parameters do not specify a circle, the angles are skewed to the same degree that the full arc is a skewed circle.

The angles are specified as doubleword values in fixed-point format. The high-order WORD contains the integer portion; the low-order WORD contains the fractional portion. A value of 6553 gives an angle of 1°. Both angles must be positive. Whether the arc is drawn in a

clockwise or counterclockwise direction is determined by the arc parameters. An angle greater than 360° is also valid. In this case, after the initial line, a full arc is drawn followed by a partial arc of (lSweep MOD 360)°. See also the function "GpiPartialArc" in the *Presentation Manager Programming Reference* .

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePartialArc.

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_ANGLE_PARM
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MULTIPLIER
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePartialArc - Remarks

Upon completion, the current (X,Y) position is the end of the partial arc. The dimensions of the full arc are defined as a multiplier, which is applied to the current arc parameters. The partial arc that is drawn is the section of the full arc that is enclosed by the specified start and sweep angles.

If GrePartialArc is used within a path or an area definition to continue a figure following a GreBox*xxx* or GreFullArc*xxx* function, the error PMERR_INV_NESTED_FIGURES is posted. This error occurs because GreBox*xxx* and GreFullArc*xxx* generate a closed figure within an area or path definition.

GrePartialArc - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GrePolyFillet

GrePolyFillet - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GrePolyFillet draws a fillet on a series of connected lines with the first line starting at the current (X,Y) position. Upon completion, the current (X,Y) position is set to the last point in the series.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_ARCS
#include <os2.h>

HDC         hdc;          /*  Device context handle. */
PPOINTL     paptlPoint;   /*  Pointer to coordinates array. */
LONG        cPoints;      /*  Number of coordinate pairs. */
PVOID       pInstance;    /*  Pointer to instance data. */
ULONG       lFunction;    /*  High-order WORD=flags; low-order WORD=NGrePolyFillet. */
LONG        rc;           /*  Return codes. */

rc = GrePolyFillet(hdc, paptlPoint, cPoints,
       pInstance, lFunction);
```

GrePolyFillet Parameter - hdc

hdc (HDC) - input
Device context handle.

GrePolyFillet Parameter - paptlPoint

paptlPoint (PPOINTL) - input
Pointer to coordinates array.

An array of cPoints (X,Y) pairs, which contain the (X,Y) coordinates of the end points for the lines. If COM_TRANSFORM is not set, the function expects the points to be in screen coordinates.

GrePolyFillet Parameter - cPoints

cPoints (LONG) - input
Number of coordinate pairs.

If this value is passed as 0, the handling routine does nothing and returns Successful.

GrePolyFillet Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GrePolyFillet Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePolyFillet.

GrePolyFillet Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePolyFillet - Parameters

hdc (HDC) - input
Device context handle.

paptlPoint (PPOINTL) - input
Pointer to coordinates array.

An array of cPoints (X,Y) pairs, which contain the (X,Y) coordinates of the end points for the lines. If COM_TRANSFORM is not set, the function expects the points to be in screen coordinates.

cPoints (LONG) - input
Number of coordinate pairs.

If this value is passed as 0, the handling routine does nothing and returns Successful.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePolyFillet.

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePolyFillet - Remarks

The shape of the fillet is controlled by a set of coordinates for a series of two or more connected lines. The fillet is tangential to the start point of the first line and to the end point of the last line. When more than two sets of coordinates are supplied, the fillet passes through the mid-points of the intermediate lines. An individual fillet always lies within the area bounded by the start, end, and control points.

Refer to function "GpiPolyFillet" in the *Presentation Manager Programming Reference* for more information.

GrePolyFillet - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GrePolyFilletSharp

GrePolyFilletSharp - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GrePolyFilletSharp draws a sequence of one or more sharp fillets starting at the current (X,Y) position.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_ARCS
#include <os2.h>

HDC         hdc;          /*  Device context handle. */
PPOINTL     paptlPoint;   /*  Pointer to Coordinates array. */
LONG        cPoints;      /*  Number of coordinate pairs. */
PFIXED      pfxSharp;     /*  Pointer to Sharpness array. */
PVOID       pInstance;    /*  Pointer to instance data. */
ULONG       lFunction;    /*  High-order WORD=flags; low-order WORD=NGrePolyFilletSharp. */
LONG        rc;           /*  Return codes. */

rc = GrePolyFilletSharp(hdc, paptlPoint, cPoints,
       pfxSharp, pInstance, lFunction);
```

GrePolyFilletSharp Parameter - hdc

hdc (HDC) - input
Device context handle.

GrePolyFilletSharp Parameter - paptlPoint

paptlPoint (PPOINTL) - input
Pointer to Coordinates array.

An array of cPoints (X,Y) pairs where each group contains two pairs of (X,Y) coordinates for the control and end points of the fillets. If COM_TRANSFORM is not set, the function expects the points to be in screen coordinates.

GrePolyFilletSharp Parameter - cPoints

cPoints (LONG) - input
Number of coordinate pairs.

When this is passed as 0, it returns Successful.

GrePolyFilletSharp Parameter - pfxSharp

pfxSharp (PFIXED) - input
Pointer to Sharpness array.

An array of sharpness values with one element for each fillet. "Sharpness" is defined as a fixed-point value. The high-order WORD contains the integer portion; the low-order WORD contains the fractional portion. A value of 64KB gives a sharpness of 1.

GrePolyFilletSharp Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GrePolyFilletSharp Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePolyFilletSharp.

GrePolyFilletSharp Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON and

	a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePolyFilletSharp - Parameters

hdc (HDC) - input
Device context handle.

paptlPoint (PPOINTL) - input
Pointer to Coordinates array.

An array of cPoints (X,Y) pairs where each group contains two pairs of (X,Y) coordinates for the control and end points of the fillets. If COM_TRANSFORM is not set, the function expects the points to be in screen coordinates.

cPoints (LONG) - input
Number of coordinate pairs.

When this is passed as 0, it returns Successful.

pfxSharp (PFIXED) - input
Pointer to Sharpness array.

An array of sharpness values with one element for each fillet. "Sharpness" is defined as a fixed-point value. The high-order WORD contains the integer portion; the low-order WORD contains the fractional portion. A value of 64KB gives a sharpness of 1.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePolyFilletSharp.

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePolyFilletSharp - Remarks

As each fillet is drawn, the end point for the fillet becomes the start point for the next fillet. Upon completion, the current (X,Y) position is the end point of the last fillet. Each fillet is controlled by a set of coordinates for two connected lines and by a sharpness value. The fillet is tangential to the start point of the first line and to the end point of the second line.

The sharpness value is determined from:

```
Sharpness = WD/DB
where:
       W is the mid-point of the line joining the end points
       B is the control point above the top of the fillet
       D is the point where the fillet intersects the line WC
```

Refer to the function "GpiPolyFillet" in the *Presentation Manager Programming Reference* for more information.

GrePolyFilletSharp - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GrePolygonSet

GrePolygonSet - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GrePolygonSet draws a set of closed polygons.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_LINE
#include <os2.h>

HDC         hdc;         /*  Device context handle. */
LONG        flModel;     /*  Model flags. */
ULONG       flOptions;   /*  Option flags. */
PPOLYGON    paPolygon;   /*  Pointer to an array of POLYGON structures: */
LONG        cPolygons;   /*  Number of POLYGON structures. */
PVOID       pInstance;   /*  Pointer to instance data. */
ULONG       lFunction;   /*  High-order WORD=flags; low-order WORD = NGrePolygonSet. */
LONG        rc;          /*  Return codes. */

rc = GrePolygonSet(hdc, flModel, flOptions,
       paPolygon, cPolygons, pInstance, lFunction);
```

GrePolygonSet Parameter - hdc

hdc (HDC) - input
Device context handle.

GrePolygonSet Parameter - flModel

flModel (LONG) - input
Model flags.

These flags determine how polygons are drawn:

POLYGON_INCL	Default setting. Fill is inclusive of bottom right.
POLYGON_EXCL	Fill is exclusive of bottom right. Aids migration from other graphics models.

GrePolygonSet Parameter - flOptions

flOptions (ULONG) - input
Option flags.

These flags determine how the polygons are filled:

POLYGON_ALTERNATE	Fill is performed by using the alternate mode.
POLYGON_WINDING	Fill is performed by using the winding mode.
POLYGON_BOUNDARY	Draw boundary lines.
POLYGON_NOBOUNDARY	Do not draw boundary lines.

GrePolygonSet Parameter - paPolygon

paPolygon (PPOLYGON) - input
Pointer to an array of POLYGON structures:

ulPoints	Number of points in this polygon
aPointl	Array of points defining polygon

If COM_TRANSFORM is not set, the function expects the points to be in screen coordinates.

GrePolygonSet Parameter - cPolygons

cPolygons (LONG) - input
Number of POLYGON structures.

When this is passed as *0* , no action is taken except to return Successful.

GrePolygonSet Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GrePolygonSet Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD = NGrePolygonSet.

GrePolygonSet Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePolygonSet - Parameters

hdc (HDC) - input
Device context handle.

flModel (LONG) - input
Model flags.

These flags determine how polygons are drawn:

POLYGON_INCL	Default setting. Fill is inclusive of bottom right.
POLYGON_EXCL	Fill is exclusive of bottom right. Aids migration from other graphics models.

flOptions (ULONG) - input
Option flags.

These flags determine how the polygons are filled:

POLYGON_ALTERNATE	Fill is performed by using the alternate mode.
POLYGON_WINDING	Fill is performed by using the winding mode.
POLYGON_BOUNDARY	Draw boundary lines.
POLYGON_NOBOUNDARY	Do not draw boundary lines.

paPolygon (PPOLYGON) - input
Pointer to an array of POLYGON structures:

ulPoints	Number of points in this polygon
aPointl	Array of points defining polygon

If COM_TRANSFORM is not set, the function expects the points to be in screen coordinates.

cPolygons (LONG) - input
Number of POLYGON structures.

When this is passed as *0* , no action is taken except to return Successful.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD = NGrePolygonSet.

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePolygonSet - Remarks

The polygons are filled using the current AREABUNDLE structure values. For the first polygon, the current position is the first point. For all subsequent polygons, all points that define the polygon are given explicitly. The polygons are automatically closed, if necessary, by drawing a line from the last vertex to the first. Notice that polygons can overlap, if needed.

Note: GrePolygonSet is not valid when the COM_AREA or COM_PATH flag is set.

GrePolygonSet - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GrePolyLine

GrePolyLine - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GrePolyLine draws a sequence of one or more lines starting at the current (X,Y) position. As each line is drawn, its end point becomes the start point for the next line. Upon completion, the current (X,Y) position is the end point of the last line.

For performance reasons, all presentation drivers should support this function when drawing a polyline to a single clipping rectangle. When the clip region is more complex, the handling routine can forward the call to the graphics engine using the pointer supplied in the dispatch table when the presentation driver was enabled. The engine will clip each line and return it to the presentation driver as a call to GreDrawLinesInPath.

GrePolyLine is called by the function GpiPolyLine. GrePolyLine is also used by many of the complex object-rendering routines within the graphics engine.

```
#define INCL_GRE_LINES
#include <os2.h>

HDC         hdc;          /*  Device context handle. */
PPOINTL     paptlPoint;   /*  Pointer to an array of (X,Y) points. */
LONG        cPoints;      /*  Number of (X,Y) pairs in points array. */
PVOID       pInstance;    /*  Pointer to instance data. */
ULONG       lFunction;    /*  High-order WORD=flags; low-order WORD=NGrePolyLine. */
LONG        rc;           /*  Return Code. */

rc = GrePolyLine(hdc, paptlPoint, cPoints,
       pInstance, lFunction);
```

GrePolyLine Parameter - hdc

hdc (HDC) - input
Device context handle.

GrePolyLine Parameter - paptlPoint

paptlPoint (PPOINTL) - input
Pointer to an array of (X,Y) points.

Pointer to an array of cPoints (X,Y) pairs containing the (X,Y) coordinates of the end points for the lines.

GrePolyLine Parameter - cPoints

cPoints (LONG) - input
Number of (X,Y) pairs in points array.

When this is passed as 0, the handling routine takes no action except to return Successful.

GrePolyLine Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GrePolyLine Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePolyLine.

GrePolyLine Return Value - rc

rc (LONG) - returns
Return Code.

On completion, the handling routine must return an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is *on* , and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePolyLine - Parameters

hdc (HDC) - input
Device context handle.

paptlPoint (PPOINTL) - input
Pointer to an array of (X,Y) points.

Pointer to an array of cPoints (X,Y) pairs containing the (X,Y) coordinates of the end points for the lines.

cPoints (LONG) - input
Number of (X,Y) pairs in points array.

When this is passed as 0, the handling routine takes no action except to return Successful.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePolyLine.

rc (LONG) - returns
Return Code.

On completion, the handling routine must return an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is *on* , and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX

PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePolyLine - Remarks

When COM_TRANSFORM is not set, the function expects the array of points to be in screen coordinates.

When the COM_AREA or COM_PATH flag is set, this function is part of an area or path definition. In either case, the handling routine would usually pass the call back to the graphics engine for processing by the default handling routine. The call would not be passed back to the graphics engine if the presentation driver had hooked all of the area and path functions.

When this function is used to draw a closed figure, the handling routine must not draw the last point of the last (closure) line. The handling routine can call back to the graphics engine to do any necessary clipping.

GrePolyLine - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GrePolyMarker

GrePolyMarker - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GrePolyMarker draws a sequence of one or more markers. The first marker is drawn at the current (X,Y) position. Subsequent markers are drawn at the specified (X,Y) positions that indicate the centers of the markers. Upon completion, the current (X,Y) position is the center of the last marker.

This function must be supported by the presentation driver. GrePolyMarker is called by GpiPolyMarker and GpiMarker. GrePolyMarker is used to render one or more markers. The first marker is drawn at the current position and the positions of any subsequent markers must be specified.

```
#define INCL_GRE_MARKERS
#include <os2.h>

HDC         hdc;          /*  Device context handle. */
PPOINTL     paptlPoint;   /*  Pointer to points array. */
LONG        cPoints;      /*  Number of markers to be drawn. */
PVOID       pInstance;    /*  Pointer to instance data. */
ULONG       lFunction;    /*  High-order WORD=flags; low-order WORD=NGrePolyMarker. */
LONG        rc;           /*  Return Code. */

rc = GrePolyMarker(hdc, paptlPoint, cPoints,
       pInstance, lFunction);
```

GrePolyMarker Parameter - hdc

hdc (HDC) - input
Device context handle.

GrePolyMarker Parameter - paptlPoint

paptlPoint (PPOINTL) - input
Pointer to points array.

An array of (cPoints) (X,Y) pairs that contain the (X,Y) coordinates of the markers.

GrePolyMarker Parameter - cPoints

cPoints (LONG) - input
Number of markers to be drawn.

When this is passed as NULL, the handling routine takes no action except to return Successful.

GrePolyMarker Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GrePolyMarker Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePolyMarker.

GrePolyMarker Return Value - rc

rc (LONG) - returns
Return Code.

On completion, the handling routine must return an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_IN_AREA
PMERR_INV_MARKER_SYMBOL_ATTR
PMERR_PATH_LIMIT_EXCEEDED
PMERR_UNSUPPORTED_ATTR
PMERR_UNSUPPORTED_ATTR_VALUE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePolyMarker - Parameters

hdc (HDC) - input
Device context handle.

paptlPoint (PPOINTL) - input
Pointer to points array.

An array of (cPoints) (X,Y) pairs that contain the (X,Y) coordinates of the markers.

cPoints (LONG) - input
Number of markers to be drawn.

When this is passed as NULL, the handling routine takes no action except to return Successful.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePolyMarker.

rc (LONG) - returns
Return Code.

On completion, the handling routine must return an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)

GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_IN_AREA
PMERR_INV_MARKER_SYMBOL_ATTR
PMERR_PATH_LIMIT_EXCEEDED
PMERR_UNSUPPORTED_ATTR
PMERR_UNSUPPORTED_ATTR_VALUE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePolyMarker - Remarks

When COM_TRANSFORM is not set, the function expects the array of points to be in screen coordinates.

Note: GrePolyMarker has no related functions.

GrePolyMarker - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GrePolyScanline

GrePolyScanline - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GrePolyScanline fills an area lying between polyshortline pairs by using the current pattern attribute.

This function must be supported by all presentation drivers except those that hook the GreDrawRLE, GreEndArea, and GreFillPath functions.

All function calls to GrePolyScanline come from the GreDrawRLE, GreEndArea, or GreFillPath handling routines in the graphics engine. GrePolyScanline is not called by any specific function. However, it is likely to be accessed by any area-filling functions.

```
#define INCL_GRE_SCANS
#include <os2.h>

HDC          hdc;         /*  Device context handle. */
PSCANDATA    pScanData;   /*  Pointer to a SCANDATA structure. */
PVOID        pInstance;   /*  Pointer to instance data. */
ULONG        lFunction;   /*  High-order WORD=flags; low-order WORD=NGrePolyScanline. */
LONG         rc;          /*  Return Code. */

rc = GrePolyScanline(hdc, pScanData, pInstance,
       lFunction);
```

GrePolyScanline Parameter - hdc

hdc (HDC) - input
Device context handle.

GrePolyScanline Parameter - pScanData

pScanData (PSCANDATA) - input
Pointer to a SCANDATA structure.

Pointer to a SCANDATA structure:

pslFirstLeft	Pointer to the left end of the first polyshortline
pslLastLeft	Pointer the left end of the last polyshortline
pslFirstRight	Pointer to right edge of first polyshortline
pslLastRight	Pointer to right edge of last polyshortline
c	Number of scan lines
rclBound	RECTL structure defining the bounding rectangle

Notice that a polyshortline consists of a list of linked SHORTLINE structures:

slh — SHORTLINEHEADER structure:

ulStyle	Line style
ulFormat	Line format
ptlStart	(X,Y) position of start
ptlStop	(X,Y) position of end
lxLeft	Left edge of bounding rectangle
lxRight	Right edge of bounding rectangle
pslhNext	Pointer to next shortline
pslhPrev	Pointer to previous shortline

This structure is a discrete representation of a curve that starts at point (x0,y0) and ends at point (x1,y1). For each of the (y1-y0+1) rows, there is exactly one X value contained in the X array. The final point in the series is not drawn.

ax — Array of X values as screen coordinates.

GrePolyScanline Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GrePolyScanline Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePolyScanline.

GrePolyScanline Return Value - rc

rc (LONG) - returns
Return Code.

On completion, the handling routine must return an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePolyScanline - Parameters

hdc (HDC) - input
Device context handle.

pScanData (PSCANDATA) - input
Pointer to a SCANDATA structure.

Pointer to a SCANDATA structure:

pslFirstLeft	Pointer to the left end of the first polyshortline

pslLastLeft	Pointer the left end of the last polyshortline
pslFirstRight	Pointer to right edge of first polyshortline
pslLastRight	Pointer to right edge of last polyshortline
c	Number of scan lines
rclBound	RECTL structure defining the bounding rectangle

Notice that a polyshortline consists of a list of linked SHORTLINE structures:

slh — SHORTLINEHEADER structure:

ulStyle	Line style
ulFormat	Line format
ptlStart	(X,Y) position of start
ptlStop	(X,Y) position of end
lxLeft	Left edge of bounding rectangle
lxRight	Right edge of bounding rectangle
pslhNext	Pointer to next shortline
pslhPrev	Pointer to previous shortline

This structure is a discrete representation of a curve that starts at point (x0,y0) and ends at point (x1,y1). For each of the (y1-y0+1) rows, there is exactly one X value contained in the X array. The final point in the series is not drawn.

ax — Array of X values as screen coordinates.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePolyScanline.

rc (LONG) - returns
Return Code.

On completion, the handling routine must return an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePolyScanline - Remarks

The calling function uses the driver's CAPS_ADDITIONAL_GRAPHICS / CAPS_CLIP_FILLS field, which is queried during the first instance of DevOpenDC. If it is set, the data delivered to GrePolyScanline has already been clipped and the COM_PRECLIP flag will be set in the flags WORD of the function number; otherwise, they will be passed as unclipped screen coordinates. Filling is inclusive at the left boundaries and exclusive at the right boundaries. The scan lines are ordered from bottom to top and from left to right.

The handling routine can assume that the two polyshortlines do not cross and both polyshortlines have the same height.

GrePolyScanline - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GrePolyShortLine

GrePolyShortLine - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GrePolyShortLine draws a series of shortlines. The current (X,Y) position is not changed. A polyshortline is a linked list of shortlines.

This function must be supported by all presentation drivers except those that hook GrePolyLine and all of the GreArc*xxx* functions that are simulated by handling routines in the graphics engine. All function calls to GrePolyShortLine come from either GrePolyLine or the GreArc*xxx* functions. GrePolyShortLine might be accessed from any of the curve-rendering functions. However, it is not guaranteed that curve rendering will call GrePolyShortLine.

```
#define INCL_GRE_LINES
#include <os2.h>

HDC           hdc;         /*  Device context handle. */
PSHORTLINE    psl;         /*  Pointer to SHORTLINE structure. */
PVOID         pInstance;   /*  Pointer to instance data. */
ULONG         lFunction;   /*  High-order WORD=flags; low-order WORD=NGrePolyShortLine. */
LONG          rc;          /*  Return Code. */

rc = GrePolyShortLine(hdc, psl, pInstance,
       lFunction);
```

GrePolyShortLine Parameter - hdc

hdc (HDC) - input
Device context handle.

GrePolyShortLine Parameter - psl

psl (PSHORTLINE) - input
Pointer to SHORTLINE structure.

The shortlines are defined in a list of linked SHORTLINE structures:

slh	SHORTLINEHEADER structure:	
	ulStyle	Line style.
	ulFormat	Line format.
	ptlStart	(X,Y) position of start (the start position is included in the line).
	ptlStop	(X,Y) position of end (the end position is not included in the line).
	lxLeft	Left edge of bounding rectangle.
	lxRight	Right edge of bounding rectangle.
	pslhNext	Pointer to next shortline.
	pslhPrev	Pointer to previous shortline.

The boundaries of the rectangle are inclusive at the start points of the lines, and exclusive at the stop points regardless of the direction.

This structure is a discrete representation of a curve that starts at point (x0,y0) and ends at point (x1,y1). For each of the (y1-y0+1) rows, there is exactly one X value contained in the X array. The final point in the series is not drawn.

ax — Steps. This is an array of X-coordinates in absolute values. There is one coordinate value for each shortline.

GrePolyShortLine Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GrePolyShortLine Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePolyShortLine.

GrePolyShortLine Return Value - rc

rc (LONG) - returns
Return Code.

On completion, the handling routine must return an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BITMAP_NOT_SELECTED

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePolyShortLine - Parameters

hdc (HDC) - input
Device context handle.

psl (PSHORTLINE) - input
Pointer to SHORTLINE structure.

The shortlines are defined in a list of linked SHORTLINE structures:

slh	SHORTLINEHEADER structure:	
	ulStyle	Line style.
	ulFormat	Line format.
	ptlStart	(X,Y) position of start (the start position is included in the line).
	ptlStop	(X,Y) position of end (the end position is not included in the line).
	lxLeft	Left edge of bounding rectangle.
	lxRight	Right edge of bounding rectangle.
	pslhNext	Pointer to next shortline.
	pslhPrev	Pointer to previous shortline.

The boundaries of the rectangle are inclusive at the start points of the lines, and exclusive at the stop points regardless of the direction.

This structure is a discrete representation of a curve that starts at point (x0,y0) and ends at point (x1,y1). For each of the (y1-y0+1) rows, there is exactly one X value contained in the X array. The final point in the series is not drawn.

ax	Steps. This is an array of X-coordinates in absolute values. There is one coordinate value for each shortline.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePolyShortLine.

rc (LONG) - returns
Return Code.

On completion, the handling routine must return an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA

PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePolyShortLine - Remarks

The function renders each SHORTLINE structure in the list until a NULL pslhNext is encountered. Coordinates are passed as screen coordinates and are already completely clipped.

GrePolyShortLine - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GrePolySpline

GrePolySpline - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GrePolySpline draws a sequence of one or more Bezier splines starting at the current (X,Y) position. As each spline is drawn, the specified end point for the spline becomes the start point for the next spline. Upon completion, the current (X,Y) position is the end point of the last spline.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_ARCS
#include <os2.h>

HDC         hdc;          /*  Device context handle. */
PPOINTL     paptlPoint;   /*  Pointer to coordinates array. */
LONG        cPoints;
PVOID       pInstance;    /*  Pointer to instance data. */
ULONG       lFunction;
LONG        rc;           /*  Return codes. */
```

```
rc = GrePolySpline(hdc, paptlPoint, cPoints,
       pInstance, lFunction);
```

GrePolySpline Parameter - hdc

hdc (HDC) - input
Device context handle.

GrePolySpline Parameter - paptlPoint

paptlPoint (PPOINTL) - input
Pointer to coordinates array.

An array of elements in which each group contains three pairs of (X,Y) coordinates for the control and end points for the splines. If COM_TRANSFORM is not set, the function expects the points to be in screen coordinates.

GrePolySpline Parameter - cPoints

cPoints (LONG) - input
Number of coordinate pairs in the array. When this is passed as 0, it returns Successful.

GrePolySpline Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GrePolySpline Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePolySpline.

GrePolySpline Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePolySpline - Parameters

hdc (HDC) - input
Device context handle.

paptlPoint (PPOINTL) - input
Pointer to coordinates array.

An array of elements in which each group contains three pairs of (X,Y) coordinates for the control and end points for the splines. If COM_TRANSFORM is not set, the function expects the points to be in screen coordinates.

cPoints (LONG) - input
Number of coordinate pairs in the array. When this is passed as 0, it returns Successful.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePolySpline.

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED

PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePolySpline - Remarks

The shape of each spline is controlled by a set of coordinates for three connected lines. The spline starts at the current position and ends at the end point of the third line. The end points of the first and second lines are used as control points. An individual spline always lies within the area bounded by the start, end, and control points.

Refer to the function "GpiPolySpline" in the *Presentation Manager Programming Reference* for more information.

GrePolySpline - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GrePtInRegion

GrePtInRegion - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GrePtInRegion checks whether a point lies within a region. GrePtInRegion raises an error when hrgn is the handle of the currently selected clip region. The point is always expected in device coordinates.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_REGIONS
```

```
#include <os2.h>

HDC         hdc;         /*  Device context handle. */
HRGN        hrgn;        /*  Region handle. */
PPOINTL     pptlPoint;   /*  Pointer to (x, y) point specified in device coordinates. */
PVOID       pInstance;   /*  Pointer to instance data. */
ULONG       lFunction;   /*  High-order WORD=flags; low-order WORD=NGrePtInRegion. */
LONG        rc;          /*  Return codes. */

rc = GrePtInRegion(hdc, hrgn, pptlPoint, pInstance,
       lFunction);
```

GrePtInRegion Parameter - hdc

hdc (HDC) - input
Device context handle.

GrePtInRegion Parameter - hrgn

hrgn (HRGN) - input
Region handle.

GrePtInRegion Parameter - pptlPoint

pptlPoint (PPOINTL) - input
Pointer to (x, y) point specified in device coordinates.

GrePtInRegion Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GrePtInRegion Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePtInRegion.

GrePtInRegion Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (lInside) indicating whether the point is inside the region:

RGN_ERROR	Error
PRGN_INSIDE	In region
PRGN_OUTSIDE	Not in region.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HRGN_BUSY
PMERR_INV_COORDINATE
PMERR_INV_HRGN
PMERR_INV_RECT
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePtInRegion - Parameters

hdc (HDC) - input
Device context handle.

hrgn (HRGN) - input
Region handle.

pptlPoint (PPOINTL) - input
Pointer to (x, y) point specified in device coordinates.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePtInRegion.

rc (LONG) - returns
Return codes.

This function returns an integer (lInside) indicating whether the point is inside the region:

RGN_ERROR	Error
PRGN_INSIDE	In region
PRGN_OUTSIDE	Not in region.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HRGN_BUSY
PMERR_INV_COORDINATE
PMERR_INV_HRGN
PMERR_INV_RECT
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePtInRegion - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GrePtVisible

GrePtVisible - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GrePtVisible checks whether a point is visible within the DC region of the specified device context. The "DC region" is defined as the intersection of the application clipping and the window clipping.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_CLIP
#include <os2.h>

HDC         hdc;         /*  Device context handle. */
PPOINTL     pptlPoint;   /*  Pointer to (X,Y) point in world or screen coordinates. */
PVOID       pInstance;   /*  Pointer to instance data. */
ULONG       lFunction;   /*  High-order WORD=flags; low-order WORD=NGrePtVisible. */
LONG        rc;          /*  Return codes. */

rc = GrePtVisible(hdc, pptlPoint, pInstance,
       lFunction);
```

GrePtVisible Parameter - hdc

hdc (HDC) - input
Device context handle.

GrePtVisible Parameter - pptlPoint

pptlPoint (PPOINTL) - input
Pointer to (X,Y) point in world or screen coordinates.

If COM_TRANSFORM is not set, the function expects the point to be in screen coordinates.

GrePtVisible Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GrePtVisible Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePtVisible.

GrePtVisible Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (lVisible) indicating the visibility of the point:

PVIS_ERROR	Error
PVIS_INVISIBLE	Not visible
PVIS_VISIBLE	Visible

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePtVisible - Parameters

hdc (HDC) - input
Device context handle.

pptlPoint (PPOINTL) - input
Pointer to (X,Y) point in world or screen coordinates.

If COM_TRANSFORM is not set, the function expects the point to be in screen coordinates.

pInstance (PVOID) - input

Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePtVisible.

rc (LONG) - returns
Return codes.

This function returns an integer (lVisible) indicating the visibility of the point:

PVIS_ERROR	Error
PVIS_INVISIBLE	Not visible
PVIS_VISIBLE	Visible

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GrePtVisible - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreQueryCharPositions

GreQueryCharPositions - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreQueryCharPositions stores, at the location addressed by paptXY, an array of world coordinates identifying the start points at which the device is to place each character of a given string.

This function must be supported by the presentation driver. GreQueryCharPositions is called by GpiQueryCharStringPos, and is used to get the position where the next string output would occur relative to the current presentation space position. It also returns the starting position of each character within that string.

```
#define INCL_GRE_STRINGS
#include <os2.h>
```

```
HDC         hdc;         /*  Device context handle. */
PPOINTL     pptlStart;   /*  Pointer to (X,Y) coordinates of optional starting position. */
ULONG       flOptions;   /*  Options flags. */
LONG        cChars;      /*  Number of bytes in string. */
PCH         pchString;   /*  Pointer to character string. */
PLONG       pAdx;        /*  Pointer to Increment array. */
PPOINTL     paptXY;      /*  Pointer to array where character positions are returned. */
PVOID       pInstance;   /*  Pointer to instance data. */
ULONG       lFunction;   /*  High-order WORD=flags; low-order WORD=NGreQueryCharPositions. */
BOOL        rc;          /*  Return Code. */

rc = GreQueryCharPositions(hdc, pptlStart,
       flOptions, cChars, pchString, pAdx,
       paptXY, pInstance, lFunction);
```

GreQueryCharPositions Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryCharPositions Parameter - pptlStart

pptlStart (PPOINTL) - input
Pointer to (X,Y) coordinates of optional starting position.

GreQueryCharPositions Parameter - flOptions

flOptions (ULONG) - input
Options flags.

The following flags can be set:

CHS_VECTOR	If set, increment vector is present.
CHS_START_XY	If set, starting position is present. Otherwise, pptlStart is ignored.

GreQueryCharPositions Parameter - cChars

cChars (LONG) - input
Number of bytes in string.

GreQueryCharPositions Parameter - pchString

pchString (PCH) - input
Pointer to character string.

GreQueryCharPositions Parameter - pAdx

pAdx (PLONG) - input
Pointer to Increment array.

Vector of increment values with one element for each character in the string. After writing a character, the increment specifies the absolute distance in world coordinates to get to the starting point of the next character.

GreQueryCharPositions Parameter - paptXY

paptXY (PPOINTL) - input
Pointer to array where character positions are returned.

Pointer to an array of (cChars+1) returned positions. The first value in the array is the initial current position; the last value is the current position on return.

GreQueryCharPositions Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryCharPositions Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryCharPositions.

GreQueryCharPositions Return Value - rc

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE Successful

FALSE Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CHAR_ANGLE_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CHAR_POS_OPTIONS
PMERR_INV_CODEPAGE
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MATRIX_ELEMENT
PMERR_INV_SETID
PMERR_INV_TRANSFORM_TYPE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryCharPositions - Parameters

hdc (HDC) - input
Device context handle.

pptlStart (PPOINTL) - input
Pointer to (X,Y) coordinates of optional starting position.

flOptions (ULONG) - input
Options flags.

The following flags can be set:

CHS_VECTOR	If set, increment vector is present.
CHS_START_XY	If set, starting position is present. Otherwise, pptlStart is ignored.

cChars (LONG) - input
Number of bytes in string.

pchString (PCH) - input
Pointer to character string.

pAdx (PLONG) - input
Pointer to Increment array.

Vector of increment values with one element for each character in the string. After writing a character, the increment specifies the absolute distance in world coordinates to get to the starting point of the next character.

paptXY (PPOINTL) - input
Pointer to array where character positions are returned.

Pointer to an array of (cChars+1) returned positions. The first value in the array is the initial current position; the last value is the current position on return.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryCharPositions.

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE Successful

FALSE Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CHAR_ANGLE_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CHAR_POS_OPTIONS
PMERR_INV_CODEPAGE
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MATRIX_ELEMENT
PMERR_INV_SETID
PMERR_INV_TRANSFORM_TYPE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryCharPositions - Remarks

GreQueryCharPositions is required for the management of device fonts in CM_MODE2 only. When the presentation driver has no device fonts, the handling routine must post PMERR_DEV_FUNC_NOT_INSTALLED.

GreQueryCharPositions - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreQueryClipRegion

GreQueryClipRegion - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreQueryClipRegion returns the handle of the currently selected clip region, or NULL (if none exists).

This function can be hooked by the presentation driver.

```
#define INCL_GRE_CLIP
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreQueryClipRegion. */
HRGN     rc;          /*  This function returns the handle of the clip region: */

rc = GreQueryClipRegion(hdc, pInstance, lFunction);
```

GreQueryClipRegion Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryClipRegion Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryClipRegion Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryClipRegion.

GreQueryClipRegion Return Value - rc

rc (HRGN) - returns
This function returns the handle of the clip region:

hrgn	Region handle
NULL	Null handle (no region selected)
HRGN_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryClipRegion - Parameters

hdc (HDC) - input
Device context handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryClipRegion.

rc (HRGN) - returns
This function returns the handle of the clip region:

hrgn	Region handle
NULL	Null handle (no region selected)
HRGN_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryClipRegion - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreQueryColorData

GreQueryColorData - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreQueryColorData stores an array of information about the currently available logical color table and device colors at the location addressed by pArray.

This function must be supported by the presentation driver. GreQueryColorData is called by GpiQueryColorData in response to an application's request for the currently selected color table data for the device context. This function can be handled by bit-map simulation.

```
#define INCL_GRE_COLORTABLE
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
LONG     cArray;      /*  Number of elements supplied in Array. */
PLONG    pArray;      /*  Pointer to Array. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreQueryColorData. */
BOOL     rc;          /*  Return Code. */

rc = GreQueryColorData(hdc, cArray, pArray,
       pInstance, lFunction);
```

GreQueryColorData Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryColorData Parameter - cArray

cArray (LONG) - input
Number of elements supplied in Array.

GreQueryColorData Parameter - pArray

pArray (PLONG) - input
Pointer to Array.

On return, this array contains:

lArray[QCD_LCT_FORMAT]	Format of loaded color table (if any):	
	LCOLF_DEFAULT	Default color table is in force.
	LCOLF_INDRGB	Color table loaded, which provides translation from index to RGB.
	LCOLF_RGB	Color index = RGB.
	LCOLF_PALETTE	DC has a palette selected.
lArray[QCD_LCT_LOINDEX]	Smallest color index loaded (always 0).	
lArray[QCD_LCT_HIINDEX]	Largest color index loaded (never less than 15).	

Information is only returned for the number of elements supplied. Any extra elements supplied must be zeroed by the handling routine.

GreQueryColorData Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryColorData Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryColorData.

GreQueryColorData Return Value - rc

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryColorData - Parameters

hdc (HDC) - input
Device context handle.

cArray (LONG) - input
Number of elements supplied in Array.

pArray (PLONG) - input
Pointer to Array.

On return, this array contains:

lArray[QCD_LCT_FORMAT]	Format of loaded color table (if any):	
	LCOLF_DEFAULT	Default color table is in force.

	LCOLF_INDRGB	Color table loaded, which provides translation from index to RGB.
	LCOLF_RGB	Color index = RGB.
	LCOLF_PALETTE	DC has a palette selected.
lArray[QCD_LCT_LOINDEX]	Smallest color index loaded (always 0).	
lArray[QCD_LCT_HIINDEX]	Largest color index loaded (never less than 15).	

Information is only returned for the number of elements supplied. Any extra elements supplied must be zeroed by the handling routine.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryColorData.

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryColorData - Remarks

When the current table is the default logical color table, presentation drivers that support less than 16 colors return the device colors to which the 16 colors from 0 (CLR_BACKGROUND) through 15 (CLR_PALEGRAY) have been mapped.

GreQueryColorData - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreQueryColorIndex

GreQueryColorIndex - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreQueryColorIndex returns the logical color index that is closest to the specified RGB color representation for the device. If the color index is in RGB mode, the supplied RGB value is returned.

This function must be supported by the presentation driver. GreQueryColorIndex is called by GpiQueryColorIndex when an application requests the index of the color most closely matching a specified color, relative to the current logical color table for the device context.

```
#define INCL_GRE_COLORTABLE
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    flOptions;   /*  Options flag. */
LONG     rgbColor;    /*  Color, as an RGB value. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreQueryColorIndex. */
LONG     rc;          /*  Return Code. */

rc = GreQueryColorIndex(hdc, flOptions, rgbColor,
       pInstance, lFunction);
```

GreQueryColorIndex Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryColorIndex Parameter - flOptions

flOptions (ULONG) - input
Options flag.

The only valid option is:

LCOLOPT_REALIZED	If set, the information is required when the logical color table (if any) is realized. Otherwise, the information is required when the logical color table is not realized.

Other flags are reserved and must be 0.

GreQueryColorIndex Parameter - rgbColor

rgbColor (LONG) - input
Color, as an RGB value.

GreQueryColorIndex Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryColorIndex Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryColorIndex.

GreQueryColorIndex Return Value - rc

rc (LONG) - returns
Return Code.

On completion, the handling routine must return the color index that provides the closest match to the specified color (lColorIndex), or GPI_ALTERROR if an error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_COLOR_OPTIONS
PMERR_INV_HDC
PMERR_INV_RGBCOLOR

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryColorIndex - Parameters

hdc (HDC) - input
Device context handle.

flOptions (ULONG) - input
Options flag.

The only valid option is:

LCOLOPT_REALIZED	If set, the information is required when the logical color table (if any) is realized. Otherwise, the information is required when the logical color table is not realized.

Other flags are reserved and must be 0.

rgbColor (LONG) - input
Color, as an RGB value.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryColorIndex.

rc (LONG) - returns
Return Code.

On completion, the handling routine must return the color index that provides the closest match to the specified color (lColorIndex), or GPI_ALTERROR if an error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_COLOR_OPTIONS
PMERR_INV_HDC
PMERR_INV_RGBCOLOR

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryColorIndex - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreQueryDeviceBitmaps

GreQueryDeviceBitmaps - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreQueryDeviceBitmaps stores a list of bit-map formats supported by the device in the array addressed by paOutData. The number of formats supported can be found by using GreQueryDeviceCaps.

This function must be supported by the presentation driver. GreQueryDeviceBitmaps is called by GpiQueryDeviceBitmapFormats in response to the request of an application for a list of the bit-map formats that the device context supports. This function can be handled by bit-map simulation.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
PLONG    paOutData;   /*  Pointer to array where the bit-map format data is returned. */
```

```
LONG     cOutData;    /*  Number of elements in the array pointed to by paOutData. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreQueryDeviceBitmaps. */
BOOL     rc;          /*  Return Code. */

rc = GreQueryDeviceBitmaps(hdc, paOutData,
       cOutData, pInstance, lFunction);
```

GreQueryDeviceBitmaps Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryDeviceBitmaps Parameter - paOutData

paOutData (PLONG) - input
Pointer to array where the bit-map format data is returned.

Excess elements are set to 0.

GreQueryDeviceBitmaps Parameter - cOutData

cOutData (LONG) - input
Number of elements in the array pointed to by paOutData.

This number must be even.

GreQueryDeviceBitmaps Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryDeviceBitmaps Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryDeviceBitmaps.

GreQueryDeviceBitmaps Return Value - rc

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_LENGTH_OR_COUNT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryDeviceBitmaps - Parameters

hdc (HDC) - input
Device context handle.

paOutData (PLONG) - input
Pointer to array where the bit-map format data is returned.

Excess elements are set to 0.

cOutData (LONG) - input
Number of elements in the array pointed to by paOutData.

This number must be even.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryDeviceBitmaps.

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_LENGTH_OR_COUNT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryDeviceBitmaps - Remarks

Each bit-map format is returned in a pair of array elements and is in the form (cPlanes, cBitsPerPel). The first pair in the array must be the

format that most closely matches the device.

GreQueryDeviceBitmaps - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreQueryDeviceCaps

GreQueryDeviceCaps - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreQueryDeviceCaps supports DevQueryCaps() at the API. See *Presentation Manager Programming Reference, Volume 1* for more information about DevQueryCaps(). The handling routine returns the required information in the device capabilities buffer addressed by *paOutData*.

This function must be supported by the presentation driver. GreQueryDeviceCaps is called by DevQueryCaps(), and is used to return information regarding the general capabilities of the device.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
LONG     lIndex;      /*  Identifies the first item required. */
PLONG    paOutData;   /*  Pointer to an array where information is returned. */
LONG     cOutData;    /*  Number of items of information to be returned at paOutData. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreQueryDeviceCaps. */
BOOL     rc;          /*  Return Code. */

rc = GreQueryDeviceCaps(hdc, lIndex, paOutData,
       cOutData, pInstance, lFunction);
```

GreQueryDeviceCaps Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryDeviceCaps Parameter - lIndex

lIndex (LONG) - input
Identifies the first item required.

Indicates the element within the device capabilities array at which the presentation driver must begin returning information. Device capabilities are held by the system in an array of ULONG fields:

CAPS_FAMILY	Device type.
CAPS_IO_CAPS	Device input/output capability. Has the following values: CAPS_IO_DUMMY Dummy device CAPS_SUPPORTS_OP Device supports output CAPS_SUPPORTS_IP Device supports input CAPS_SUPPORTS_IO Device supports output and input.
CAPS_TECHNOLOGY	Technology. Has the following values: CAPS_TECH_UNKNOWN Unknown CAPS_TECH_VECTOR_PLOTTER Vector plotter CAPS_TECH_RASTER_DISPLAY Raster display CAPS_TECH_RASTER_PRINTER Raster printer CAPS_TECH_RASTER_CAMERA Raster camera CAPS_TECH_POSTSCRIPT PostScript device.
CAPS_DRIVER_VERSION	Version identifier of the presentation driver. The high-order WORD of the version identifier is 0. The low-order WORD identifies the release. For example, 0x0120 is Release 1.2.
CAPS_WIDTH	Media width in pels of a full-screen, maximized window for displays.
CAPS_HEIGHT	Media depth in pels of a full-screen, maximized window for displays. For a plotter, a pel is defined as the smallest possible displacement of the pen and can be smaller than a pen width.
CAPS_WIDTH_IN_CHARS	Media width in default character columns of a full-screen, maximized window for displays.
CAPS_HEIGHT_IN_CHARS	Media depth in default character rows of a full-screen, maximized window for displays.
CAPS_HORIZONTAL_RESOLUTION	Horizontal resolution of device in pels per meter.
CAPS_VERTICAL_RESOLUTION	Vertical resolution of device in pels per meter.

CAPS_CHAR_WIDTH

Default character-box width for VIO, in pels

CAPS_CHAR_HEIGHT

Default character-box height for VIO, in pels.

CAPS_SMALL_CHAR_WIDTH

Default small-character box width for VIO, in pels. If 0, there is only one character-box size.

CAPS_SMALL_CHAR_HEIGHT

Default small-character box height for VIO, in pels. If 0, there is only one character-box size.

CAPS_COLORS

Number of distinct colors supported at the same time, including reset. (Note that gray scales count as distinct colors.) If loadable color tables are supported, this is the number of entries in the device color table. For plotters, the value returned is the "number of pens + 1" (for the background).

CAPS_COLOR_PLANES

Number of color planes.

CAPS_COLOR_BITCOUNT

Number of adjacent color bits for each pel within one plane.

CAPS_COLOR_TABLE_SUPPORT

Loadable color table support. Has the following values:

CAPS_COLTABL_RGB_8
: If 1, the RGB color table can be loaded with a minimum support of eight bits each for red, green, and blue.

CAPS_COLTABL_RGB_8_PLUS
: If 1, the color table can be loaded with more than eight bits for each primary color.

CAPS_COLTABL_TRUE_MIX
: If 1, true mixing occurs when the logical color table has been realized, providing that the size of the logical color table is not greater than the number of distinct colors supported (see CAPS_COLORS).

CAPS_COLTABL_REALIZE
: If 1, a loaded color table can be realized.

CAPS_MOUSE_BUTTONS

Number of buttons available. A returned value of 0 indicates that there are no buttons available.

CAPS_FOREGROUND_MIX_SUPPORT

Foreground mix support. Has the following values:

CAPS_FM_OR
: Logical OR.

CAPS_FM_OVERPAINT
: Overpaint.

CAPS_FM_XOR
: Logical XOR.

CAPS_FM_LEAVEALONE
: Leave alone.

CAPS_FM_AND
: Logical AND.

CAPS_FM_GENERAL_BOOLEAN

All other mix modes.

The value returned is the sum of the values appropriate to the mixes supported. A device capable of supporting OR must at least return CAPS_FM_OR + CAPS_FM_OVERPAINT + CAPS_FM_LEAVEALONE to signify support for the mandatory mixes OR, Overpaint, and Leave Alone.

These numbers correspond to the decimal representation of a bit string that is six bits long with each bit set to 1 if the appropriate mode is supported. Mixes that are returned as supported are guaranteed for all primitive types. For more information, refer to the function "GpiSetMix" in the *Presentation Manager Programming Reference* .

CAPS_BACKGROUND_MIX_SUPPORT

Background mix support. Has the following values:

CAPS_BM_OR
: Logical OR.

CAPS_BM_OVERPAINT
: Overpaint.

CAPS_BM_XOR
: Logical XOR.

CAPS_BM_LEAVEALONE
: Leave alone.

CAPS_BM_AND
: Logical AND.

CAPS_BM_SCRTRANSPARENT
: Provides for a transparent overlay function by not copying pels that match the presentation space background color from the source bit map to the output bit map.

CAPS_BM_DESTTRANSPARENT
: Provides for a transparent underlay function by only copying pels that match the presentation space background color from the source bit map to destination pels.

CAPS_BM_GENERAL_BOOLEAN
: All other mix modes.

If CAPS_BM_SCRTRANSPARENT and CAPS_BM_DESTTRANSPARENT are supported, these values can be selected by calling the function GpiSetBackMix and specifying them as the mix mode. The presentation driver then performs any BitBlt operations (for GreBitblt or GreDrawBits) by first checking the background mix for one of the transparent mix modes. Note that these mixes are performed only on BitBlt operations that do not include a pattern, that is, raster operations that include only the source or destination, or both. If the source bit map is not given, CAPS_BM_SCRTRANSPARENT does not operate and the entire bit map operation area is used.

	The returned value is the sum of the values appropriate to the mixes supported. A device must at least return CAPS_BM_OVERPAINT + CAPS_BM_LEAVEALONE to signify support for the mandatory background mixes Overpaint and Leave Alone. These numbers correspond to the decimal representation of a bit string that is four bits long, with each bit set to 1 if the appropriate mode is supported. Mixes that are returned as supported are guaranteed for all primitive types.
CAPS_VIO_LOADABLE_FONTS	Number of fonts that can be loaded for VIO.
CAPS_WINDOW_BYTE_ALIGNMENT	Indicates whether the client area of VIO windows should be byte-aligned. Has the following values: CAPS_BYTE_ALIGN_REQUIRED Must be byte-aligned. CAPS_BYTE_ALIGN_RECOMMENDED More efficient if byte-aligned, but not required. CAPS_BYTE_ALIGN_NOT_REQUIRED Does not matter whether byte-aligned.
CAPS_BITMAP_FORMATS	Number of bit-map formats supported by device.
CAPS_RASTER_CAPS	Capability for device raster operations. Has the following values: CAPS_RASTER_BITBLT If 1, GpiBitBlt and GpiWCBitBlt are supported. CAPS_RASTER_BANDING If 1, banding is supported. CAPS_RASTER_BITBLT_SCALING If 1, GpiBitBlt and GpiWCBitBlt (with scaling) are supported. CAPS_RASTER_SET_PEL If 1, GpiSetPel is supported. CAPS_RASTER_FONTS If 1, this device can draw raster fonts. CAPS_RASTER_FLOOD_FILL If 1, GpiFloodFill is supported.
CAPS_MARKER_HEIGHT	Default marker-box height in pels.
CAPS_MARKER_WIDTH	Default marker-box width in pels.
CAPS_DEVICE_FONTS	Number of device-specific fonts.
CAPS_GRAPHICS_SUBSET	Graphics drawing subset supported; for example, 3 indicates GOCA DR/3.
CAPS_GRAPHICS_VERSION	Graphics architecture version number supported; for example, 1 indicates Version 1.
CAPS_GRAPHICS_VECTOR_SUBSET	Graphics vector drawing subset supported; for example, 2 indicates GOCA VS/2.
CAPS_DEVICE_WINDOWING	Device windowing support. Has the following values: CAPS_DEV_WINDOWING_SUPPORT

	If 1, this device supports windowing. Other bits are reserved (0).
CAPS_ADDITIONAL_GRAPHICS	Additional graphics support. Has the following values: CAPS_GRAPHICS_KERNING_SUPPORT If 1, device supports kerning. CAPS_FONT_OUTLINE_DEFAULT If 1, device has a default outline font. CAPS_FONT_IMAGE_DEFAULT If 1, device has a default image font. CAPS_SCALED_DEFAULT_MARKERS If 1, default markers are to be scaled by the marker-box attribute. CAPS_COLOR_CURSOR_SUPPORT If 1, device supports colored cursors. CAPS_PALETTE_MANAGER If 1, device supports palette functions. CAPS_COSMETIC_WIDELINE_SUPPORT If 1, device supports cosmetic thick lines. CAPS_ENHANCED_TEXT If 1, device supports full-font file description and text alignment. Other bits are reserved (0). CAPS_CLIP_FILLS If 1, changes the behavior of the graphics engine to call GrePolyScanline with the COM_PRECLIP flag set. The lines are preclipped when filling areas. This flag should only be returned to the graphics engine version number 0x202 or later (see GreQueryEngineVersion).
CAPS_PHYS_COLORS	Maximum number of distinct colors available on the device.
CAPS_COLOR_INDEX	Maximum logical color-table index supported for this device. For EGA and VGA drivers, this value is 63.
CAPS_GRAPHICS_CHAR_WIDTH	Default graphics character-box width in pels.
CAPS_GRAPHICS_CHAR_HEIGHT	Default graphics character-box height in pels.
CAPS_HORIZONTAL_FONT_RES	Effective horizontal device resolution in pels per inch (for selecting fonts). For printers, this is the actual device resolution. For displays, it can differ from the actual resolution for reasons of legibility.
CAPS_VERTICAL_FONT_RES	Effective vertical device resolution in pels per inch (for selecting fonts).

CAPS_DEVICE_FONT_SIM	Identifies which simulations are valid on device fonts. Valid flags are: **CAPS_DEVICE_FONT_SIM_BOLD CAPS_DEVICE_FONT_SIM_ITALIC CAPS_DEVICE_FONT_SIM_UNDERSCOR E CAPS_DEVICE_FONT_SIM_STRIKEOUT**
CAPS_LINEWIDTH_THICK	Cosmetic thickness of lines and arcs on this device when fxLineWidth is LINEWIDTH_THICK (refer to the function "GpiSetLineWidth" in the *Presentation Manager Programming Reference*). The units are in pels. A value of 0 is interpreted as two pels.

Additional information is provided in the device capabilities array for communication between the presentation driver and the graphics engine. For example, the parameter CAPS_ADDITIONAL_GRAPHICS has two extra flags and a CAPS_DEVICE_FONT_SIM field is provided.

The additional flags in CAPS_ADDITIONAL_GRAPHICS are used (in conjunction with the CAPS_FONT_OUTLINE_DEFAULT and CAPS_FONT_IMAGE_DEFAULT flags) by the presentation driver to indicate when the graphics engine manages the transforms and mappings for the default fonts supplied by the driver. These flags are:

CAPS_FONT_OUTLINE_MANAGE	Set by the presentation driver to indicate that the graphics engine must manage the default outline font
CAPS_FONT_IMAGE_MANAGE	Set by the presentation driver to indicate that the graphics engine must manage the default image font

Note: If the presentation driver supplies the fonts but wants the graphics engine to manage them, it must pass the font address to the graphics engine using GreQueryDevResource.

The CAPS_DEVICE_FONT_SIM field contains flags that the presentation driver sets so that the graphics engine will handle simulations for the default fonts supplied by the driver:

CAPS_DEV_FONT_SIM_BOLD	Indicates that the graphics engine should simulate CDEF_BOLD for device fonts
CAPS_DEV_FONT_SIM_ITALIC	Indicates that the graphics engine should simulate CDEF_ITALIC for device fonts
CAPS_DEV_FONT_SIM_UNDERSCORE	Indicates that the graphics engine should simulate CDEF_UNDERSCORE for device fonts
CAPS_DEV_FONT_SIM_STRIKEOUT	Indicates that the graphics engine should simulate CDEF_STRIKEOUT for device fonts.

Note: The font attributes CDEF_*xxx* are identified by the *cdef.fFlags* field in the "Character Attributes" bundle (see Character Attributes). In the presentation driver, routines that write character strings should check the *cdef.fFlags* field to determine whether the call should be passed to the default handling routine in the graphics engine.

GreQueryDeviceCaps Parameter - paOutData

paOutData (PLONG) - input
Pointer to an array where information is returned.

GreQueryDeviceCaps Parameter - cOutData

cOutData (LONG) - input
Number of items of information to be returned at paOutData.

GreQueryDeviceCaps Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryDeviceCaps Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryDeviceCaps.

GreQueryDeviceCaps Return Value - rc

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_QUERY_ELEMENT_NO

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryDeviceCaps - Parameters

hdc (HDC) - input
Device context handle.

lIndex (LONG) - input
Identifies the first item required.

Indicates the element within the device capabilities array at which the presentation driver must begin returning information. Device capabilities are held by the system in an array of ULONG fields:

CAPS_FAMILY	Device type.

CAPS_IO_CAPS	Device input/output capability. Has the following values: CAPS_IO_DUMMY Dummy device CAPS_SUPPORTS_OP Device supports output CAPS_SUPPORTS_IP Device supports input CAPS_SUPPORTS_IO Device supports output and input.
CAPS_TECHNOLOGY	Technology. Has the following values: CAPS_TECH_UNKNOWN Unknown CAPS_TECH_VECTOR_PLOTTER Vector plotter CAPS_TECH_RASTER_DISPLAY Raster display CAPS_TECH_RASTER_PRINTER Raster printer CAPS_TECH_RASTER_CAMERA Raster camera CAPS_TECH_POSTSCRIPT PostScript device.
CAPS_DRIVER_VERSION	Version identifier of the presentation driver. The high-order WORD of the version identifier is 0. The low-order WORD identifies the release. For example, 0x0120 is Release 1.2.
CAPS_WIDTH	Media width in pels of a full-screen, maximized window for displays.
CAPS_HEIGHT	Media depth in pels of a full-screen, maximized window for displays. For a plotter, a pel is defined as the smallest possible displacement of the pen and can be smaller than a pen width.
CAPS_WIDTH_IN_CHARS	Media width in default character columns of a full-screen, maximized window for displays.
CAPS_HEIGHT_IN_CHARS	Media depth in default character rows of a full-screen, maximized window for displays.
CAPS_HORIZONTAL_RESOLUTION	Horizontal resolution of device in pels per meter.
CAPS_VERTICAL_RESOLUTION	Vertical resolution of device in pels per meter.
CAPS_CHAR_WIDTH	Default character-box width for VIO, in pels
CAPS_CHAR_HEIGHT	Default character-box height for VIO, in pels.
CAPS_SMALL_CHAR_WIDTH	Default small-character box width for VIO, in pels. If 0, there is only one character-box size.
CAPS_SMALL_CHAR_HEIGHT	Default small-character box height for VIO, in pels. If 0, there is only one character-box size.
CAPS_COLORS	Number of distinct colors supported at the same time, including reset. (Note that gray scales count as distinct colors.) If loadable

color tables are supported, this is the number of entries in the device color table. For plotters, the value returned is the "number of pens + 1" (for the background).

CAPS_COLOR_PLANES

Number of color planes.

CAPS_COLOR_BITCOUNT

Number of adjacent color bits for each pel within one plane.

CAPS_COLOR_TABLE_SUPPORT

Loadable color table support. Has the following values:

CAPS_COLTABL_RGB_8
: If 1, the RGB color table can be loaded with a minimum support of eight bits each for red, green, and blue.

CAPS_COLTABL_RGB_8_PLUS
: If 1, the color table can be loaded with more than eight bits for each primary color.

CAPS_COLTABL_TRUE_MIX
: If 1, true mixing occurs when the logical color table has been realized, providing that the size of the logical color table is not greater than the number of distinct colors supported (see CAPS_COLORS).

CAPS_COLTABL_REALIZE
: If 1, a loaded color table can be realized.

CAPS_MOUSE_BUTTONS

Number of buttons available. A returned value of 0 indicates that there are no buttons available.

CAPS_FOREGROUND_MIX_SUPPORT

Foreground mix support. Has the following values:

CAPS_FM_OR
: Logical OR.

CAPS_FM_OVERPAINT
: Overpaint.

CAPS_FM_XOR
: Logical XOR.

CAPS_FM_LEAVEALONE
: Leave alone.

CAPS_FM_AND
: Logical AND.

CAPS_FM_GENERAL_BOOLEAN
: All other mix modes.

The value returned is the sum of the values appropriate to the mixes supported. A device capable of supporting OR must at least return CAPS_FM_OR + CAPS_FM_OVERPAINT + CAPS_FM_LEAVEALONE to signify support for the mandatory mixes OR, Overpaint, and Leave Alone.

These numbers correspond to the decimal representation of a bit string that is six bits long with each bit set to 1 if the appropriate mode is supported. Mixes that are returned as supported are guaranteed for all primitive

types. For more information, refer to the function "GpiSetMix" in the *Presentation Manager Programming Reference* .

CAPS_BACKGROUND_MIX_SUPPORT

Background mix support. Has the following values:

CAPS_BM_OR
: Logical OR.

CAPS_BM_OVERPAINT
: Overpaint.

CAPS_BM_XOR
: Logical XOR.

CAPS_BM_LEAVEALONE
: Leave alone.

CAPS_BM_AND
: Logical AND.

CAPS_BM_SCRTRANSPARENT
: Provides for a transparent overlay function by not copying pels that match the presentation space background color from the source bit map to the output bit map.

CAPS_BM_DESTTRANSPARENT
: Provides for a transparent underlay function by only copying pels that match the presentation space background color from the source bit map to destination pels.

CAPS_BM_GENERAL_BOOLEAN
: All other mix modes.

If CAPS_BM_SCRTRANSPARENT and CAPS_BM_DESTTRANSPARENT are supported, these values can be selected by calling the function GpiSetBackMix and specifying them as the mix mode. The presentation driver then performs any BitBlt operations (for GreBitblt or GreDrawBits) by first checking the background mix for one of the transparent mix modes. Note that these mixes are performed only on BitBlt operations that do not include a pattern, that is, raster operations that include only the source or destination, or both. If the source bit map is not given, CAPS_BM_SCRTRANSPARENT does not operate and the entire bit map operation area is used.

The returned value is the sum of the values appropriate to the mixes supported. A device must at least return CAPS_BM_OVERPAINT + CAPS_BM_LEAVEALONE to signify support for the mandatory background mixes Overpaint and Leave Alone. These numbers correspond to the decimal representation of a bit string that is four bits long, with each bit set to 1 if the appropriate mode is supported. Mixes that are returned as supported are guaranteed for all primitive types.

CAPS_VIO_LOADABLE_FONTS

Number of fonts that can be loaded for VIO.

CAPS_WINDOW_BYTE_ALIGNMENT

Indicates whether the client area of VIO windows should be byte-aligned. Has the

following values:

CAPS_BYTE_ALIGN_REQUIRED
: Must be byte-aligned.

CAPS_BYTE_ALIGN_RECOMMENDED
: More efficient if byte-aligned, but not required.

CAPS_BYTE_ALIGN_NOT_REQUIRED
: Does not matter whether byte-aligned.

CAPS_BITMAP_FORMATS
: Number of bit-map formats supported by device.

CAPS_RASTER_CAPS
: Capability for device raster operations. Has the following values:

CAPS_RASTER_BITBLT
: If 1, GpiBitBlt and GpiWCBitBlt are supported.

CAPS_RASTER_BANDING
: If 1, banding is supported.

CAPS_RASTER_BITBLT_SCALING
: If 1, GpiBitBlt and GpiWCBitBlt (with scaling) are supported.

CAPS_RASTER_SET_PEL
: If 1, GpiSetPel is supported.

CAPS_RASTER_FONTS
: If 1, this device can draw raster fonts.

CAPS_RASTER_FLOOD_FILL
: If 1, GpiFloodFill is supported.

CAPS_MARKER_HEIGHT
: Default marker-box height in pels.

CAPS_MARKER_WIDTH
: Default marker-box width in pels.

CAPS_DEVICE_FONTS
: Number of device-specific fonts.

CAPS_GRAPHICS_SUBSET
: Graphics drawing subset supported; for example, 3 indicates GOCA DR/3.

CAPS_GRAPHICS_VERSION
: Graphics architecture version number supported; for example, 1 indicates Version 1.

CAPS_GRAPHICS_VECTOR_SUBSET
: Graphics vector drawing subset supported; for example, 2 indicates GOCA VS/2.

CAPS_DEVICE_WINDOWING
: Device windowing support. Has the following values:

CAPS_DEV_WINDOWING_SUPPORT
: If 1, this device supports windowing. Other bits are reserved (0).

CAPS_ADDITIONAL_GRAPHICS
: Additional graphics support. Has the following values:

CAPS_GRAPHICS_KERNING_SUPPORT
: If 1, device supports kerning.

CAPS_FONT_OUTLINE_DEFAULT
: If 1, device has a default outline font.

CAPS_FONT_IMAGE_DEFAULT
: If 1, device has a

	default image font. CAPS_SCALED_DEFAULT_MARKERS If 1, default markers are to be scaled by the marker-box attribute. CAPS_COLOR_CURSOR_SUPPORT If 1, device supports colored cursors. CAPS_PALETTE_MANAGER If 1, device supports palette functions. CAPS_COSMETIC_WIDELINE_SUPPORT If 1, device supports cosmetic thick lines. CAPS_ENHANCED_TEXT If 1, device supports full-font file description and text alignment. Other bits are reserved (0). CAPS_CLIP_FILLS If 1, changes the behavior of the graphics engine to call GrePolyScanline with the COM_PRECLIP flag set. The lines are preclipped when filling areas. This flag should only be returned to the graphics engine version number 0x202 or later (see GreQueryEngineVersion).
CAPS_PHYS_COLORS	Maximum number of distinct colors available on the device.
CAPS_COLOR_INDEX	Maximum logical color-table index supported for this device. For EGA and VGA drivers, this value is 63.
CAPS_GRAPHICS_CHAR_WIDTH	Default graphics character-box width in pels.
CAPS_GRAPHICS_CHAR_HEIGHT	Default graphics character-box height in pels.
CAPS_HORIZONTAL_FONT_RES	Effective horizontal device resolution in pels per inch (for selecting fonts). For printers, this is the actual device resolution. For displays, it can differ from the actual resolution for reasons of legibility.
CAPS_VERTICAL_FONT_RES	Effective vertical device resolution in pels per inch (for selecting fonts).
CAPS_DEVICE_FONT_SIM	Identifies which simulations are valid on device fonts. Valid flags are: **CAPS_DEVICE_FONT_SIM_BOLD** **CAPS_DEVICE_FONT_SIM_ITALIC** **CAPS_DEVICE_FONT_SIM_UNDERSCORE** **CAPS_DEVICE_FONT_SIM_STRIKEOUT**
CAPS_LINEWIDTH_THICK	Cosmetic thickness of lines and arcs on this device when fxLineWidth is LINEWIDTH_THICK (refer to the function "GpiSetLineWidth" in the *Presentation Manager Programming Reference*). The units are in pels. A value of 0 is interpreted as two pels.

Additional information is provided in the device capabilities array for communication between the presentation driver and the graphics engine. For example, the parameter CAPS_ADDITIONAL_GRAPHICS has two extra flags and a CAPS_DEVICE_FONT_SIM field is provided.

The additional flags in CAPS_ADDITIONAL_GRAPHICS are used (in conjunction with the CAPS_FONT_OUTLINE_DEFAULT and CAPS_FONT_IMAGE_DEFAULT flags) by the presentation driver to indicate when the graphics engine manages the transforms and mappings for the default fonts supplied by the driver. These flags are:

CAPS_FONT_OUTLINE_MANAGE	Set by the presentation driver to indicate that the graphics engine must manage the default outline font
CAPS_FONT_IMAGE_MANAGE	Set by the presentation driver to indicate that the graphics engine must manage the default image font

Note: If the presentation driver supplies the fonts but wants the graphics engine to manage them, it must pass the font address to the graphics engine using GreQueryDevResource.

The CAPS_DEVICE_FONT_SIM field contains flags that the presentation driver sets so that the graphics engine will handle simulations for the default fonts supplied by the driver:

CAPS_DEV_FONT_SIM_BOLD	Indicates that the graphics engine should simulate CDEF_BOLD for device fonts
CAPS_DEV_FONT_SIM_ITALIC	Indicates that the graphics engine should simulate CDEF_ITALIC for device fonts
CAPS_DEV_FONT_SIM_UNDERSCORE	Indicates that the graphics engine should simulate CDEF_UNDERSCORE for device fonts
CAPS_DEV_FONT_SIM_STRIKEOUT	Indicates that the graphics engine should simulate CDEF_STRIKEOUT for device fonts.

Note: The font attributes CDEF_*xxx* are identified by the *cdef.fFlags* field in the "Character Attributes" bundle (see Character Attributes). In the presentation driver, routines that write character strings should check the *cdef.fFlags* field to determine whether the call should be passed to the default handling routine in the graphics engine.

paOutData (PLONG) - input
Pointer to an array where information is returned.

cOutData (LONG) - input
Number of items of information to be returned at paOutData.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryDeviceCaps.

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_QUERY_ELEMENT_NO

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryDeviceCaps - Remarks

Calls to GreQueryDeviceCaps would not usually require the handling routine to return data in all fields in the buffer. The parameters lIndex and cOutData identify the offset to the first field and the count of consecutive field for returned data.

Note: In OS/2 2.0, if GreQueryDeviceCaps returns data in the CAPS_DRIVER_VERSION field, the return value must be hex 00000200.

GreQueryDeviceCaps - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreQueryDevResource

GreQueryDevResource - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreQueryDevResource indicates whether a specified resource is available. If the resource is loaded, its handle is returned so that it can be selected into the device context.

This function must be supported by the display drivers, which must support the full range of requests. GreQueryDevResource is used internally by the graphics engine. Hardcopy drivers are required to provide a minimal level of support. At a minimum, the hardcopy driver must return 0 to indicate that a requested resource is not available. If a hardcopy driver has a raster or outline font that it requests the graphics engine to use as the default, then the presentation driver must return the address of its raster or outline font when the parameter *id* is equal to RT_FONT.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    lType;       /*  Resource type. */
ULONG    id;          /*  Defined resource value. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreQueryDevResource. */
LONG     rc;          /*  Return Code. */

rc = GreQueryDevResource(hdc, lType, id, pInstance,
       lFunction);
```

GreQueryDevResource Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryDevResource Parameter - lType

lType (ULONG) - input
Resource type.

Resource types returned by the presentation driver:

RT_DISPLAYINFO — A structure containing some of the display constants required by the Window Manager. This information is required for all display devices that support windows. The format of the DISPLAYINFO structure is:

cb	Size of this structure (always set to 26).
cxIcon	Count of pels for x-width of icon.
cyIcon	Count of pels for y-height of icon. When WinLoadPointer is used to load the icon, it is stretched or compressed to the size indicated by cxIcon and cyIcon.
cxPointer	Count of pels for x-width of pointer.
cyPointer	Count of pels for y-height of pointer. When

	WinLoadPointer is used to load the pointer, it is stretched or compressed to the size indicated by cxPointer and cyPointer.
cxBorder	Count of pels for X-width of horizontal border.
cyBorder	Count of pels for Y-height vertical border.
cxHSlider	Count of pels for X-width of horizontal scroll bar slider.
cyVSlider	Count of pels for Y-height of vertical scroll bar slider.
cxSizeBorder	Count of pels for X-width of default border.
cySizeBorder	Count of pels for Y-height of default border.
cxDeviceAlign	Count of pels for horizont

		al device alignment.
	cyDeviceAlign	Count of pels for vertical device alignment. Some display devices operate faster when operation coordinates are aligned to a BYTE, WORD or DWORD boundary. The cxDeviceAlign and cyDeviceAlign parameters allow the presentation driver to align windows on these boundaries and so optimize window management operations.
RT_POINTER	Defined system pointers are: SPTR_ARROW	Left-pointing arrow, usually the system default.
	SPTR_TEXT	Text-in

	sertion pointer, typically used when the mouse pointer is on an edit control.
SPTR_WAIT	An hourglass used to tell the user to wait while the system is busy.
SPTR_MOVE	Four arrows together, pointing north, south, east and west, that tell the user that window can be dragged in any of these directions.
SPTR_SIZENWSE	An arrow pointing northwest and southeast, that tells the user that the

	window can be sized in these directions.
SPTR_SIZENESW	An arrow pointing north east and south west, that tells the user that the window can be sized in these directions.
SPTR_SIZEWE	An arrow pointing east and west, that tells the user that the window can be sized in these directions.
SPTR_SIZENS	An arrow pointing north and south, that tells the user that the window can be sized in these directions.

SPTR_APPICON	Usually a blank icon. This is used when a window that has been sized down to its minimum (and has no normal icon) is dragged across the screen.
SPTR_ICONINFORMATION	Pointer used as part of a message box when specified in a call to WinMessageBox.
SPTR_ICONQUESTION	Pointer used as part of a message box when specified in a call to WinMessageBox.
SPTR_ICONERROR	Pointer used

	as part of a message box when specified in a call to WinMessageBox.
SPTR_ICONWARNING	Pointer used as part of a message box when specified in a call to WinMessageBox.
SPTR_ILLEGAL	Pointer used by the filing system to notify the user of an illegal mouse-directed copy or move operation.
SPTR_MULTFILE	Pointer used by the file system to indicate a multiple file copy or move operation.

	SPTR_PROGRAM	Pointer used by the file system to indicate a copy or move operation on an executable program file.
	SPTR_FILE	Pointer used by the file system to indicate a copy or move operation on an ordinary file.
	SPTR_FOLDER	Pointer used by the file system to indicate a copy or move operation on an entire directory.
RT_BITMAP	The following defined system bit maps are required in display drivers:	
	SBMP_BTNCORNERS	Contains the rounded corners for push buttons. It is arranged as three

bit maps divided into 2x2 bit-map arrays describing the corners of each bit map. The three bit maps are defined as follows:

- Contains the corners of an unselected push button that is not a default

pushbutton.

- Holds the corners of a default pushbutton that is currently not selected.

- Contains the corners of a currently selected pushbutton, which can be either a default

or nondefault pushbutton.

SBMP_DRIVE	Used by the file system to display the logical disk drive.
SBMP_FILE	Used by the file system to indicate an unknown file type.
SBMP_FOLDER	Used by the file system to display a directory.
SBMP_MENUATTACHED	Drawn on the right edge of a menu item to indicate that a pull down menu is attached to that item.
SBMP_MENUCHECK	Displayed next to a menu item when the item is checked. Menu items are displayed in the system font and the menu checks are vertically aligned next to them. The

	height of this bit map must be no greater than the system font height to ensure that consecutive menu-check bit maps do not overlap. Its width is arbitrary, but is normally the same as the system font width.
SBMP_PROGRAM	Used by the file system to mark EXE and COM files.
SBMP_SIZEBOX	Used by some applications to display a "sizebox" in the bottom-right corner of a frame window.
SBMP_TREEMINUS	Used by the file system to indicate there are no more subdirectories to view.
SBMP_TREEPLUS	Used by the file system to indicate there are more subdirectories to view.

The following defined system bit maps are also required to ensure compatibility with OS/2 Version 1.x applications:

SBMP_OLD_CHILDSYSMENU
SBMP_OLD_CHECKBOXES
SBMP_OLD_MAXBUTTON
SBMP_OLD_MINBUTTON
SBMP_OLD_RESTOREBUTTON
SBMP_OLD_SBDNARROW
SBMP_OLD_SBLFARROW
SBMP_OLD_SBRGARROW

	SBMP_OLD_SBUPARROW SBMP_OLD_SYSMENU. The following defined system bit maps are provided by PMWIN.DLL and are optional in display drivers: SBMP_CHECKBOXES SBMP_CHILDSYSMENU SBMP_CHILDSYSMENUDEP SBMP_CLOSEBUTTON SBMP_CLOSEBUTTONDEP SBMP_COMBODOWN SBMP_MAXBUTTON SBMP_MAXBUTTONDEP SBMP_MINBUTTON SBMP_MINBUTTONDEP SBMP_RESTOREBUTTON SBMP_RESTOREBUTTONDEP SBMP_SBDNARROW SBMP_SBDNARROWDEP SBMP_SBDNARROWDIS SBMP_SBLFARROW SBMP_SBLFARROWDEP SBMP_SBLFARROWDIS SBMP_SBRGARROWDEP SBMP_SBRGARROWDIS SBMP_SBRGARRROW SBMP_SBUPARROW SBMP_SBUPARROWDEP SBMP_SBUPARROWDIS SBMP_SYSMENU SBMP_SYSMENUDEP Refer to the function "WinGetSysBitmap" in *Presentation Manager Programming Reference* for further explanation.	
RT_FONT	The default system fonts are:	
	SFONT_RASTER SFONT_OUTLINE	Default image font. Default outline font.

GreQueryDevResource Parameter - id

id (ULONG) - input
Defined resource value.

GreQueryDevResource Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryDevResource Parameter - lFunction

lFunction (ULONG) - input

High-order WORD=flags; low-order WORD=NGreQueryDevResource.

GreQueryDevResource Return Value - rc

rc (LONG) - returns
Return Code.

This function returns the address of the indicated resource-0 if no address is available, or GPI_ALTERROR if an error occurs.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_DEV_FUNC_NOT_INSTALLED

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreQueryDevResource - Parameters

hdc (HDC) - input
Device context handle.

lType (ULONG) - input
Resource type.

Resource types returned by the presentation driver:

RT_DISPLAYINFO	A structure containing some of the display constants required by the Window Manager. This information is required for all display devices that support windows. The format of the DISPLAYINFO structure is:	
	cb	Size of this structure (always set to 26).
	cxIcon	Count of pels for x-width of icon.
	cyIcon	Count of pels for y-height of icon. When WinLoadPointer is used to load the icon, it is stretched or compressed to

	the size indicated by cxIcon and cyIcon.
cxPointer	Count of pels for x-width of pointer.
cyPointer	Count of pels for y-height of pointer. When WinLoadPointer is used to load the pointer, it is stretched or compressed to the size indicated by cxPointer and cyPointer.
cxBorder	Count of pels for X-width of horizontal border.
cyBorder	Count of pels for Y-height vertical border.
cxHSlider	Count of pels for X-width of horizontal scroll bar slider.
cyVSlider	Count of pels for Y-height of vertical scroll bar slider.

cxSizeBorder	Count of pels for X-width of default border.
cySizeBorder	Count of pels for Y-height of default border.
cxDeviceAlign	Count of pels for horizontal device alignment.
cyDeviceAlign	Count of pels for vertical device alignment. Some display devices operate faster when operation coordinates are aligned to a BYTE, WORD or DWORD boundary. The cxDeviceAlign and cyDeviceAlign parameters allow the presentation driver to align windows on these boundaries and so optimize window

		management operations.
RT_POINTER	Defined system pointers are:	
	SPTR_ARROW	Left-pointing arrow, usually the system default.
	SPTR_TEXT	Text-insertion pointer, typically used when the mouse pointer is on an edit control.
	SPTR_WAIT	An hourglass used to tell the user to wait while the system is busy.
	SPTR_MOVE	Four arrows together, pointing north, south, east and west, that tell the user that window can be dragged in

	any of these directions.
SPTR_SIZENWSE	An arrow pointing north west and south east, that tells the user that the window can be sized in these directions.
SPTR_SIZENESW	An arrow pointing north east and south west, that tells the user that the window can be sized in these directions.
SPTR_SIZEWE	An arrow pointing east and west, that tells the user that the window can be sized in these directions.

SPTR_SIZENS	An arrow pointing north and south, that tells the user that the window can be sized in these directions.
SPTR_APPICON	Usually a blank icon. This is used when a window that has been sized down to its minimum (and has no normal icon) is dragged across the screen.
SPTR_ICONINFORMATION	Pointer used as part of a message box when specified in a call to WinMessageBox.
SPTR_ICONQUESTION	Pointe

	r used as part of a message box when specified in a call to WinMessageBox.
SPTR_ICONERROR	Pointer used as part of a message box when specified in a call to WinMessageBox.
SPTR_ICONWARNING	Pointer used as part of a message box when specified in a call to WinMessageBox.
SPTR_ILLEGAL	Pointer used by the filing system to notify the user of an illegal mouse-directed copy or move opera

	tion.
SPTR_MULTFILE	Pointer used by the file system to indicate a multiple file copy or move operation.
SPTR_PROGRAM	Pointer used by the file system to indicate a copy or move operation on an executable program file.
SPTR_FILE	Pointer used by the file system to indicate a copy or move operation on an ordinary file.
SPTR_FOLDER	Pointer used by the file system to indicate a

		copy or move operation on an entire directory.
RT_BITMAP	The following defined system bit maps are required in display drivers:	
	SBMP_BTNCORNERS	Contains the rounded corners for push buttons. It is arranged as three bit maps divided into 2x2 bit-map arrays describing the corners of each bit map. The three bit maps are defined as follows: • Contains the corners of an unselected pushbu

tton that is not a default push button.

- Holds the corners of a default push button that is

currentlynotselected.

- Contains the corners of a currentlyselected pushbutton, whi

chcanbeeitheradefaultornondefaultpushbutton.

SBMP_DRIVE	Used by the file system to display the logical disk drive.
SBMP_FILE	Used by the file system to indicate an unknown file type.
SBMP_FOLDER	Used by the file system to display a directory.
SBMP_MENUATTACHED	Drawn on the right edge of a menu item to indicate that a pull down menu is attached to that

	item.
SBMP_MENUCHECK	Displayed next to a menu item when the item is checked. Menu items are displayed in the system font and the menu checks are vertically aligned next to them. The height of this bit map must be no greater than the system font height to ensure that consecutive menu-check bit maps do not overlap. Its width is arbitrary, but is normally the same as the system font width.
SBMP_PROGRAM	Used by the file system to mark EXE and COM files.
SBMP_SIZEBOX	Used by some applications to display a "sizebox" in the bottom-right corner of a frame window.
SBMP_TREEMINUS	Used by the file system to indicate there are no more subdirectories to view.

	SBMP_TREEPLUS	Used by the file system to indicate there are more subdirectories to view.

The following defined system bit maps are also required to ensure compatibility with OS/2 Version 1.x applications:

SBMP_OLD_CHILDSYSMENU
SBMP_OLD_CHECKBOXES
SBMP_OLD_MAXBUTTON
SBMP_OLD_MINBUTTON
SBMP_OLD_RESTOREBUTTON
SBMP_OLD_SBDNARROW
SBMP_OLD_SBLFARROW
SBMP_OLD_SBRGARROW
SBMP_OLD_SBUPARROW
SBMP_OLD_SYSMENU.

The following defined system bit maps are provided by PMWIN.DLL and are optional in display drivers:

SBMP_CHECKBOXES
SBMP_CHILDSYSMENU
SBMP_CHILDSYSMENUDEP
SBMP_CLOSEBUTTON
SBMP_CLOSEBUTTONDEP
SBMP_COMBODOWN
SBMP_MAXBUTTON
SBMP_MAXBUTTONDEP
SBMP_MINBUTTON
SBMP_MINBUTTONDEP
SBMP_RESTOREBUTTON
SBMP_RESTOREBUTTONDEP
SBMP_SBDNARROW
SBMP_SBDNARROWDEP
SBMP_SBDNARROWDIS
SBMP_SBLFARROW
SBMP_SBLFARROWDEP
SBMP_SBLFARROWDIS
SBMP_SBRGARROWDEP
SBMP_SBRGARROWDIS
SBMP_SBRGARROW
SBMP_SBUPARROW
SBMP_SBUPARROWDEP
SBMP_SBUPARROWDIS
SBMP_SYSMENU
SBMP_SYSMENUDEP

Refer to the function "WinGetSysBitmap" in *Presentation Manager Programming Reference* for further explanation.

RT_FONT

The default system fonts are:

SFONT_RASTER	Default image font.
SFONT_OUTLINE	Default outline font.

id (ULONG) - input
Defined resource value.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryDevResource.

rc (LONG) - returns
Return Code.

This function returns the address of the indicated resource-0 if no address is available, or GPI_ALTERROR if an error occurs.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_DEV_FUNC_NOT_INSTALLED

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreQueryDevResource - Remarks

The resources (display information, pointers, bit maps, and fonts) are stored in DLL files. Some of these resources are linked by the presentation driver when it is first enabled, others are loaded by the application with WinLoadPointer.

The two system fonts are queried by the graphics engine when the presentation driver is loaded. When the presentation driver has a default font, it returns the handle of the font, as requested. When this function returns a NULL handle for the system font, the graphics engine default fonts are used instead (see Font Functions).

GreQueryDevResource - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreQueryHardcopyCaps

GreQueryHardcopyCaps - Syntax

Simulation support:

This function is mandatory for hardcopy drivers, but may simply return a DEVESC_NOTIMPLEMENTED code. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreQueryHardcopyCaps stores information about the hardcopy capabilities of the device in the buffer addressed by pInfo. The information is stored as a sequence of one or more HCINFO structures defining the hardcopy capabilities for one or more form codes.

This function must be supported by hardcopy drivers. It is not required for display drivers. GreQueryHardcopyCaps is called by the function DevQueryHardcopyCaps.

```
#define INCL_GRE_DEVICE
#include <os2.h>

HDC         hdc;         /*  Device context handle. */
LONG        lStart;      /*  Index. */
LONG        cCount;      /*  Number of forms. */
PHCINFO     pInfo;       /*  Pointer to buffer for returned form data. */
```

```
PVOID       pInstance;  /*  Pointer to instance data. */
ULONG       lFunction;  /*  High-order WORD=flags; low-order WORD=NGreQueryHardcopyCaps. */
LONG        rc;         /*  Return Code. */

rc = GreQueryHardcopyCaps(hdc, lStart, cCount,
       pInfo, pInstance, lFunction);
```

GreQueryHardcopyCaps Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryHardcopyCaps Parameter - lStart

lStart (LONG) - input
Index.

Index to the required starting HCINFO structure. A value of 0 identifies the HCINFO for the first form.

GreQueryHardcopyCaps Parameter - cCount

cCount (LONG) - input
Number of forms.

Number of structures to be returned in the buffer. A value of 0 requests the handling routine to set the return code to the number of forms that the driver supports.

GreQueryHardcopyCaps Parameter - pInfo

pInfo (PHCINFO) - input
Pointer to buffer for returned form data.

A pointer to the buffer for the returned data. The data is returned as a set of one or more HCINFO structures. When pInfo is not equal to NULL and lStart is greater than or equal to the number of form codes that the hardcopy drivers supports, the hardcopy driver should return 0 without modifying the memory block pointed to by pInfo.

szFormname[32]	Character string containing the name of the form
cx	Width (left-to-right) in millimeters
cy	Height (top-to-bottom) in millimeters
xLeftClip	Left clip limit in millimeters
yBottomClip	Bottom clip limit in millimeters
xRightClip	Right clip limit in millimeters
yTopClip	Top clip limit in millimeters
xPels	Number of pels between left and right clip limits
yPels	Number of pels between bottom and top clip limits
flAttributes	Attributes describing the availability of the form: HCAPS_SELECTABLE

The form is installed on the printer as given by the printer properties dialog (see OS2_PM_DRV_DEVMODE). It is available from an alternate form source without operator intervention. If the form does not have this bit set and is used (if the user selects it) in a print job, a "forms mismatch" error is generated by the printer object.

HCAPS_CURRENT
The form is the one currently selected by *pdriv* (the job properties), a field in OS2_DRV_ENABLE (FillPhysicalDeviceBlock) (see FillPhysicalDeviceBlock). The HCAPS_SELECTABLE bit is not necessarily set. This is also the form size returned by GreQueryDeviceCaps capabilities of CAPS_WIDTH and CAPS_HEIGHT.

GreQueryHardcopyCaps Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryHardcopyCaps Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryHardcopyCaps.

GreQueryHardcopyCaps Return Value - rc

rc (LONG) - returns
Return Code.

The value returned by the handling routine depends on the initial value of cCount. If cCount=0, return the total number of form codes that the presentation driver supports. For any other value, return the number of HCINFO structures that were transferred to the buffer.

Possible Errors Detected: When an error is detected, the handling routine must return DQHC_ERROR and call WinSetErrorInfo to post the condition. An error code for conditions that the handling routine is expected to check is:

PMERR_INV_LENGTH_OR_COUNT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryHardcopyCaps - Parameters

hdc (HDC) - input
Device context handle.

lStart (LONG) - input
Index.

Index to the required starting HCINFO structure. A value of 0 identifies the HCINFO for the first form.

cCount (LONG) - input
Number of forms.

Number of structures to be returned in the buffer. A value of 0 requests the handling routine to set the return code to the number of forms that the driver supports.

pInfo (PHCINFO) - input
Pointer to buffer for returned form data.

A pointer to the buffer for the returned data. The data is returned as a set of one or more HCINFO structures. When pInfo is not equal to NULL and lStart is greater than or equal to the number of form codes that the hardcopy drivers supports, the hardcopy driver should return 0 without modifying the memory block pointed to by pInfo.

szFormname[32]	Character string containing the name of the form
cx	Width (left-to-right) in millimeters
cy	Height (top-to-bottom) in millimeters
xLeftClip	Left clip limit in millimeters
yBottomClip	Bottom clip limit in millimeters
xRightClip	Right clip limit in millimeters
yTopClip	Top clip limit in millimeters
xPels	Number of pels between left and right clip limits
yPels	Number of pels between bottom and top clip limits
flAttributes	Attributes describing the availability of the form:
	HCAPS_SELECTABLE The form is installed on the printer as given by the printer properties dialog (see OS2_PM_DRV_DEVMODE). It is available from an alternate form source without operator intervention. If the form does not have this bit set and is used (if the user selects it) in a print job, a "forms mismatch" error is generated by the printer object.
	HCAPS_CURRENT The form is the one currently selected by *pdriv* (the job properties), a field in OS2_DRV_ENABLE (FillPhysicalDeviceBlock) (see FillPhysicalDeviceBlock). The HCAPS_SELECTABLE bit is not necessarily set. This is also the form size returned by GreQueryDeviceCaps capabilities of CAPS_WIDTH and CAPS_HEIGHT.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryHardcopyCaps.

rc (LONG) - returns
Return Code.

The value returned by the handling routine depends on the initial value of cCount. If cCount=0, return the total number of form codes that the presentation driver supports. For any other value, return the number of HCINFO structures that were transferred to the buffer.

Possible Errors Detected: When an error is detected, the handling routine must return DQHC_ERROR and call WinSetErrorInfo to post the condition. An error code for conditions that the handling routine is expected to check is:

PMERR_INV_LENGTH_OR_COUNT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryHardcopyCaps - Remarks

For presentation drivers that support more than one form code with the relevant data held in a structure of HCINFO structures, the parameters lStart and cCount identify the starting point in the main structure and the number of HCINFO structures to be stored.

It is usual for this function to be issued twice (initially with a value of 0 in cCount) to return the number of forms available. Storage can be allocated and the function called again with cCount set to the number of forms for which information is required.

GreQueryHardcopyCaps - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreQueryHWPaletteInfo

GreQueryHWPaletteInfo - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreQueryHWPaletteInfo fills a buffer with the RGB value information from the hardware palette. The order of the returned values is the same as the hardware. Both RGB values and the user defined xxxxx are returned. If cclr=0, only the size required for the buffer referenced by pclr is returned. The information returned can be used to create a palette. This allows a caller to create the same palette in another device context.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_PALETTE
#include <os2.h>

HDC       hdc;         /*  Device context handle. */
ULONG     ulStart;     /*  Starting index, first palette entry to query. */
ULONG     cclr;        /*  Count of palette entries. */
PULONG    pclr;        /*  Pointer to table of RGB2 palette entries. */
PVOID     pInstance;   /*  Pointer to instance data. */
ULONG     lFunction;   /*  High-order WORD=flags; low-order WORD=NGreQueryHWPaletteInfo. */
BOOL      fSuccess;    /*  Return codes. */

fSuccess = GreQueryHWPaletteInfo(hdc, ulStart,
             cclr, pclr, pInstance, lFunction);
```

GreQueryHWPaletteInfo Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryHWPaletteInfo Parameter - ulStart

ulStart (ULONG) - input
Starting index, first palette entry to query.

GreQueryHWPaletteInfo Parameter - cclr

cclr (ULONG) - input
Count of palette entries.

GreQueryHWPaletteInfo Parameter - pclr

pclr (PULONG) - input
Pointer to table of RGB2 palette entries.

GreQueryHWPaletteInfo Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryHWPaletteInfo Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryHWPaletteInfo.

GreQueryHWPaletteInfo Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_PALETTE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryHWPaletteInfo - Parameters

hdc (HDC) - input
Device context handle.

ulStart (ULONG) - input
Starting index, first palette entry to query.

cclr (ULONG) - input
Count of palette entries.

pclr (PULONG) - input
Pointer to table of RGB2 palette entries.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryHWPaletteInfo.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_PALETTE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryHWPaletteInfo - Remarks

The existing color table GreQuery*xxx* APIs are still supported.

GreQueryHWPaletteInfo - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreQueryLogColorTable

GreQueryLogColorTable - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreQueryLogColorTable stores an array of the current logical color values at the location addressed by pArray.

This function must be supported by the presentation driver. GreQueryLogColorTable is called by GpiQueryLogColorTable in response to an application's request for the currently configured logical color table. This function can be handled by bit-map simulation.

```
#define INCL_GRE_COLORTABLE
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    flOptions;   /*  Options flag. */
LONG     lStart;      /*  Starting index for which data is to be returned. */
LONG     cArray;      /*  Number of elements available in the array. */
PLONG    pArray;      /*  Pointer to the array in which the information is returned. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreQueryLogColorTable. */
LONG     rc;          /*  Return Code. */

rc = GreQueryLogColorTable(hdc, flOptions,
       lStart, cArray, pArray, pInstance,
       lFunction);
```

GreQueryLogColorTable Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryLogColorTable Parameter - flOptions

flOptions (ULONG) - input
Options flag.

The only valid option flag is:

LCOLOPT_INDEX	If set, the handling routine must return the index for each RGB value.

GreQueryLogColorTable Parameter - lStart

lStart (LONG) - input
Starting index for which data is to be returned.

GreQueryLogColorTable Parameter - cArray

cArray (LONG) - input
Number of elements available in the array.

GreQueryLogColorTable Parameter - pArray

pArray (PLONG) - input
Pointer to the array in which the information is returned.

When LCOLOPT_INDEX is set, pArray points to an array of alternating color indexes and values (index1, value1, index2, value2, and so forth). If the logical color table is not loaded with a contiguous set of indexes, any index values that are not loaded are skipped.

When LCOLOPT_INDEX is not set, pArray points to an array of RGB color values in which the information is to be returned. Each value is the same as those defined for GreCreateLogColorTable, starting with the specified index and ending when there are no further loaded entries in the table or when cCount has been exhausted. If the logical color table is not loaded with a contiguous set of indexes, QLCT_NOTLOADED is returned as the color value for any index values that are not loaded.

GreQueryLogColorTable Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryLogColorTable Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryLogColorTable.

GreQueryLogColorTable Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine must return a LONG value indicating the number of elements returned in pArray or:

QLCT_ERROR	Error.
QLCT_RGB	Color table is in RGB mode and no elements are returned.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes

for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_COLOR_OPTIONS
PMERR_INV_COLOR_START_INDEX
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryLogColorTable - Parameters

hdc (HDC) - input
Device context handle.

flOptions (ULONG) - input
Options flag.

The only valid option flag is:

LCOLOPT_INDEX	If set, the handling routine must return the index for each RGB value.

lStart (LONG) - input
Starting index for which data is to be returned.

cArray (LONG) - input
Number of elements available in the array.

pArray (PLONG) - input
Pointer to the array in which the information is returned.

When LCOLOPT_INDEX is set, pArray points to an array of alternating color indexes and values (index1, value1, index2, value2, and so forth). If the logical color table is not loaded with a contiguous set of indexes, any index values that are not loaded are skipped.

When LCOLOPT_INDEX is not set, pArray points to an array of RGB color values in which the information is to be returned. Each value is the same as those defined for GreCreateLogColorTable, starting with the specified index and ending when there are no further loaded entries in the table or when cCount has been exhausted. If the logical color table is not loaded with a contiguous set of indexes, QLCT_NOTLOADED is returned as the color value for any index values that are not loaded.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryLogColorTable.

rc (LONG) - returns
Return Code.

The handling routine must return a LONG value indicating the number of elements returned in pArray or:

QLCT_ERROR	Error.
QLCT_RGB	Color table is in RGB mode and no elements are returned.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_COLOR_OPTIONS
PMERR_INV_COLOR_START_INDEX
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryLogColorTable - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreQueryNearestColor

GreQueryNearestColor - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreQueryNearestColor returns the available color nearest to the specified color on the currently associated device even if it is not available in the logical color table. Both colors are specified as RGB values.

This function must be supported by the presentation driver. GreQueryNearestColor is called by GpiQueryNearestColor when the application wants the available colors nearest to the specified color.

```
#define INCL_GRE_COLORTABLE
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
ULONG    flOptions;    /*  Options flag. */
LONG     rgbColorIn;   /*  Color required. */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreQueryNearestColor. */
LONG     rc;           /*  Return Code. */

rc = GreQueryNearestColor(hdc, flOptions,
       rgbColorIn, pInstance, lFunction);
```

GreQueryNearestColor Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryNearestColor Parameter - flOptions

flOptions (ULONG) - input
Options flag.

The only significant flag is:

LCOLOPT_REALIZED	If set, the information is required when the logical color table (if any) is realized. When this flag is not set, the information is required when the logical color table is not realized.

Other flags are reserved.

GreQueryNearestColor Parameter - rgbColorIn

rgbColorIn (LONG) - input
Color required.

GreQueryNearestColor Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryNearestColor Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryNearestColor.

GreQueryNearestColor Return Value - rc

rc (LONG) - returns
Return Code.

The handling routine must return the nearest available RGB color to that requested (rgbColorOut), or GPI_ALTERROR if an error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_COLOR_OPTIONS
PMERR_INV_HDC
PMERR_INV_RGBCOLOR

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryNearestColor - Parameters

hdc (HDC) - input
Device context handle.

flOptions (ULONG) - input
Options flag.

The only significant flag is:

LCOLOPT_REALIZED	If set, the information is required when the logical color table (if any) is realized. When this flag is not set, the information is required when the logical color table is not realized.

Other flags are reserved.

rgbColorIn (LONG) - input
Color required.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryNearestColor.

rc (LONG) - returns
Return Code.

The handling routine must return the nearest available RGB color to that requested (rgbColorOut), or GPI_ALTERROR if an error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_COLOR_OPTIONS
PMERR_INV_HDC
PMERR_INV_RGBCOLOR

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryNearestColor - Remarks

The color used for drawing primitives, such as lines and text, is the color returned. GreQueryNearestColor does not consider the possibility of using dithered colors for filling areas. Where dithered colors are used for filling, the color used for text and lines is likely to be different even when the same color index is selected.

The nearest color is determined by finding its position in RGB space. "RGB space" can be defined as a cube with three axes (representing red, green and blue color intensities) radiating from one corner or origin. Moving up the Red axis results in increasing red intensity. Different intensities of cyan can be produced by moving along the Green and Blue axes. For EGA and VGA devices, dithering is performed by dividing the RGB space into 9x9x9 cubical cells representing the colors that can be created. The cell each RGB color falls into is determined and a lookup table is created to indicate which EGA planes to set ON or OFF to create each color.

When this function is called for a monochrome device, the color returned is either the reset color or the contrast color for the device. See Support for Monochrome Devices.

GreQueryNearestColor - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreQueryPaletteRealization

GreQueryPaletteRealization - Syntax

Simulation support:

This function is simulated by a handling routine in Revision 2.0 (or later) of the graphics engine.

Description

GreQueryPaletteRealization returns the mapping from the logical palette in the device context to the HW palette as an array of ULONGs. This is done by logically type-casting the index in the hardware palette to a ULONG. GreQueryPaletteRealization gives applications the ability to predict the outcome of color mixing operations.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_PALETTE
#include <os2.h>

HDC       hdc;         /*  Device context handle. */
ULONG     ulStart;     /*  Starting index, first palette entry to query. */
ULONG     cclr;        /*  Count of palette entries. */
PULONG    pclr;        /*  Pointer to an array of ULONGs. */
PVOID     pInstance;   /*  Pointer to instance data. */
ULONG     lFunction;   /*  HHigh-order WORD=flags; low-order WORD=NGreQueryPaletteRealization. */
ULONG     rc;          /*  Return codes. */

rc = GreQueryPaletteRealization(hdc, ulStart,
       cclr, pclr, pInstance, lFunction);
```

GreQueryPaletteRealization Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryPaletteRealization Parameter - ulStart

ulStart (ULONG) - input
Starting index, first palette entry to query.

GreQueryPaletteRealization Parameter - cclr

cclr (ULONG) - input
Count of palette entries.

GreQueryPaletteRealization Parameter - pclr

pclr (PULONG) - input
Pointer to an array of ULONGs.

GreQueryPaletteRealization Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryPaletteRealization Parameter - lFunction

lFunction (ULONG) - input
HHigh-order WORD=flags; low-order WORD=NGreQueryPaletteRealization.

GreQueryPaletteRealization Return Value - rc

rc (ULONG) - returns
Return codes.

This function returns the count of the palette entries.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_DC

GreQueryPaletteRealization - Parameters

hdc (HDC) - input
Device context handle.

ulStart (ULONG) - input
Starting index, first palette entry to query.

cclr (ULONG) - input
Count of palette entries.

pclr (PULONG) - input
Pointer to an array of ULONGs.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
HHigh-order WORD=flags; low-order WORD=NGreQueryPaletteRealization.

rc (ULONG) - returns
Return codes.

This function returns the count of the palette entries.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_DC

GreQueryPaletteRealization - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreQueryRealColors

GreQueryRealColors - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreQueryRealColors stores, in the array addressed by pArray, the RGB values of the distinct colors available on the currently associated device.

This function must be supported by the presentation driver. GreQueryRealColors is called by GpiQueryRealColors in response to an application requesting the currently available colors for the device context.

```
#define INCL_GRE_COLORTABLE
#include <os2.h>

HDC      hdc;        /*  Device context handle. */
ULONG    flOptions;  /*  Options flag. */
LONG     lStart;     /*  Ordinal number of the first color required. */
LONG     cArray;     /*  Number of elements available in the array. */
PLONG    pArray;     /*  Pointer to array in which data is returned. */
```

```
PVOID    pInstance;  /*  Pointer to instance data. */
ULONG    lFunction;  /*  High-order WORD=flags; low-order WORD=NGreQueryRealColors. */
LONG     rc;         /*  Return Code. */

rc = GreQueryRealColors(hdc, flOptions, lStart,
       cArray, pArray, pInstance, lFunction);
```

GreQueryRealColors Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryRealColors Parameter - flOptions

flOptions (ULONG) - input
Options flag.

Valid options are:

LCOLOPT_REALIZED	If set, the information is required when the logical color table (if any) is realized. When this flag is not set, the information is required when the logical color table is not realized.
LCOLOPT_INDEX	If set, the handling routine must return the index for each RGB value. Other flags are reserved and must be 0.

GreQueryRealColors Parameter - lStart

lStart (LONG) - input
Ordinal number of the first color required.

Typically, this is 0 to start the sequence. This value does not necessarily bear any relationship to the color index because the order in which the colors are returned is not defined.

GreQueryRealColors Parameter - cArray

cArray (LONG) - input
Number of elements available in the array.

GreQueryRealColors Parameter - pArray

pArray (PLONG) - input
Pointer to array in which data is returned.

When LCOLOPT_INDEX is set, this is an array of alternating color indexes and values (in the order, *index1* , *value1* , *index2* , *value2* and so on). If there is a color table, colors that are not in the table but are available on the device, have a special index of QLCT_NOTLOADED. In RGB mode, the RGB value is returned in the color indexes.

When LCOLOPT_INDEX is not set, this is an array of color values. Each value is the same as those defined for GreCreateLogColorTable.

GreQueryRealColors Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryRealColors Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryRealColors.

GreQueryRealColors Return Value - rc

rc (LONG) - returns
Return Code.

On completion, the handling routine must return the number of colors returned in the array (cColors), or GPI_ALTERROR if an error occurred

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_COLOR_OPTIONS
PMERR_INV_COLOR_START_INDEX
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryRealColors - Parameters

hdc (HDC) - input
Device context handle.

flOptions (ULONG) - input
Options flag.

Valid options are:

LCOLOPT_REALIZED	If set, the information is required when the logical color table (if any) is realized. When this flag is not set, the information is required when the logical color table is not realized.
LCOLOPT_INDEX	If set, the handling routine must return the index for each RGB value. Other flags are reserved and must be 0.

lStart (LONG) - input
Ordinal number of the first color required.

Typically, this is 0 to start the sequence. This value does not necessarily bear any relationship to the color index because the order in which the colors are returned is not defined.

cArray (LONG) - input
Number of elements available in the array.

pArray (PLONG) - input
Pointer to array in which data is returned.

When LCOLOPT_INDEX is set, this is an array of alternating color indexes and values (in the order, *index1* , *value1* , *index2* , *value2* and so on). If there is a color table, colors that are not in the table but are available on the device, have a special index of QLCT_NOTLOADED. In RGB mode, the RGB value is returned in the color indexes.

When LCOLOPT_INDEX is not set, this is an array of color values. Each value is the same as those defined for GreCreateLogColorTable.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryRealColors.

rc (LONG) - returns
Return Code.

On completion, the handling routine must return the number of colors returned in the array (cColors), or GPI_ALTERROR if an error occurred

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_COLOR_OPTIONS
PMERR_INV_COLOR_START_INDEX
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryRealColors - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreQueryRGBColor

GreQueryRGBColor - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreQueryRGBColor returns the actual RGB color that results from the specified color index for the specified device. If the color index is RGB mode, the nearest RGB color (the same as QueryNearestColor) is returned.

This function must be supported by the presentation driver. GreQueryRGBColor is called by GpiQueryRGBColor in response to the request of an application to convert a color index into the corresponding RGB value. If the logical table is currently in RGB mode, the nearest RGB color is returned. This function can be handled by bit-map simulation.

```
#define INCL_GRE_COLORTABLE
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    flOptions;   /*  Options flag. */
LONG     iColor;      /*  Color index. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreQueryRGBColor. */
LONG     rc;          /*  Return Code. */

rc = GreQueryRGBColor(hdc, flOptions, iColor,
       pInstance, lFunction);
```

GreQueryRGBColor Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryRGBColor Parameter - flOptions

flOptions (ULONG) - input
Options flag.

Valid options are:

LCOLOPT_REALIZED	If set, the information is required when the logical color table (if any) is realized. When this flag is not set, the information is required when the logical color table is not realized.
LCOLOPT_INDEX	When set, the handling routine must return the actual RGB color originally specified for this index. Otherwise, it must return the nearest RGB color for this index, that is, the one which would result from drawing on the specified device.

Other flags are reserved and must be 0.

GreQueryRGBColor Parameter - iColor

iColor (LONG) - input
Color index.

GreQueryRGBColor Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryRGBColor Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryRGBColor.

GreQueryRGBColor Return Value - rc

rc (LONG) - returns
Return Code.

On completion, the handling routine must return the nearest available RGB color to that requested (rgbColor), or GPI_ALTERROR if an error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COLOR_OPTIONS
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryRGBColor - Parameters

hdc (HDC) - input
Device context handle.

flOptions (ULONG) - input
Options flag.

Valid options are:

LCOLOPT_REALIZED — If set, the information is required when the logical color table (if any) is realized. When this flag is not set, the information is required when the

logical color table is not realized.

LCOLOPT_INDEX — When set, the handling routine must return the actual RGB color originally specified for this index. Otherwise, it must return the nearest RGB color for this index, that is, the one which would result from drawing on the specified device.

Other flags are reserved and must be 0.

iColor (LONG) - input
Color index.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryRGBColor.

rc (LONG) - returns
Return Code.

On completion, the handling routine must return the nearest available RGB color to that requested (rgbColor), or GPI_ALTERROR if an error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COLOR_OPTIONS
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryRGBColor - Remarks

All defined colors are valid except CLR_DEFAULT, which causes an error to be raised.

GreQueryRGBColor - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreQueryTextBox

GreQueryTextBox - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreQueryTextBox processes a character string as if it were being drawn.

This function must be supported by the presentation driver. GreQueryTextBox is called by the function GpiQueryTextBox, and is used to return a tight bounding box for the currently selected font of a given string relative to the current position.

```
#define INCL_GRE_STRINGS
#include <os2.h>

HDC         hdc;            /*  Device context handle. */
LONG        cChars;         /*  Number of bytes in string. */
PCH         pchString;      /*  Pointer to character string. */
LONG        cptPosition;    /*  Number of (X, Y) pairs that the Position array can contain. */
PPOINTL     paptPosition;   /*  Pointer to position array. */
PVOID       pInstance;      /*  Pointer to instance data. */
ULONG       lFunction;      /*  High-order WORD=flags; low-order WORD=NGreQueryTextBox. */
BOOL        rc;             /*  Return Code. */

rc = GreQueryTextBox(hdc, cChars, pchString,
       cptPosition, paptPosition, pInstance,
       lFunction);
```

GreQueryTextBox Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryTextBox Parameter - cChars

cChars (LONG) - input
Number of bytes in string.

GreQueryTextBox Parameter - pchString

pchString (PCH) - input
Pointer to character string.

GreQueryTextBox Parameter - cptPosition

cptPosition (LONG) - input
Number of (X, Y) pairs that the Position array can contain.

GreQueryTextBox Parameter - paptPosition

paptPosition (PPOINTL) - input
Pointer to position array.

Pointer to the array in which the function returns the requested values. Upon completion, the array contains cptPosition sets of (X, Y) coordinates in the following order:

TXTBOX_TOPLEFT	Top-left corner
TXTBOX_BOTTOMLEFT	Bottom-left corner
TXTBOX_TOPRIGHT	Top-right corner
TXTBOX_BOTTOMRIGHT	Bottom-right corner
TXTBOX_CONCAT	Start point of next character position

GreQueryTextBox Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryTextBox Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryTextBox.

GreQueryTextBox Return Value - rc

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CHAR_ANGLE_ATTR
PMERR_INV_CHAR_MODE_ATTR

PMERR_INV_CODEPAGE
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MATRIX_ELEMENT
PMERR_INV_SETID
PMERR_INV_TRANSFORM_TYPE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryTextBox - Parameters

hdc (HDC) - input
Device context handle.

cChars (LONG) - input
Number of bytes in string.

pchString (PCH) - input
Pointer to character string.

cptPosition (LONG) - input
Number of (X, Y) pairs that the Position array can contain.

paptPosition (PPOINTL) - input
Pointer to position array.

Pointer to the array in which the function returns the requested values. Upon completion, the array contains cptPosition sets of (X, Y) coordinates in the following order:

TXTBOX_TOPLEFT	Top-left corner
TXTBOX_BOTTOMLEFT	Bottom-left corner
TXTBOX_TOPRIGHT	Top-right corner
TXTBOX_BOTTOMRIGHT	Bottom-right corner
TXTBOX_CONCAT	Start point of next character position

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryTextBox.

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CHAR_ANGLE_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MATRIX_ELEMENT
PMERR_INV_SETID
PMERR_INV_TRANSFORM_TYPE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryTextBox - Remarks

The handling routine stores the coordinates of the current text box (relative to the current (X,Y) position) as an array at the location indicated by paptPosition. The first four coordinate pairs identify the bounding parallelogram for the given character string. The fifth coordinate pair is the (X,Y) position of the starting point for the next character position after the string, that is, the current position value that would be set by an equivalent call to GpiCharStringAt. The positions take account of current values for character spacing such as kerning and character space. The points on the borders of the character box are deemed to be inside the box.

When the character mode is CM_MODE2, this function is valid only if the character angle and shear attributes are set to their default values. See Character Attributes.

GreQueryTextBox - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreQueryWidthTable

GreQueryWidthTable - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreQueryWidthTable stores, at the location addressed by paWidthTable, an array of world coordinates representing the width-table information of the currently selected font.

This function must be supported by the presentation driver. GreQueryWidthTable is called by the function GpiQueryWidthTable.

```
#define INCL_GRE_STRINGS
#include <os2.h>

HDC      hdc;           /*  Device context handle. */
LONG     lFirstChar;    /*  Code point of the initial character for which width-table information is required. *
LONG     cWidthTable;   /*  Count of widths in the width-table data. */
PLONG    paWidthTable;  /*  Pointer to buffer of width-table data. */
PVOID    pInstance;     /*  Pointer to instance data. */
ULONG    lFunction;     /*  High-order WORD=flags; low-order WORD=NGreQueryWidthTable. */
BOOL     rc;            /*  Return Code. */
```

```
rc = GreQueryWidthTable(hdc, lFirstChar, cWidthTable,
       paWidthTable, pInstance, lFunction);
```

GreQueryWidthTable Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryWidthTable Parameter - lFirstChar

lFirstChar (LONG) - input
Code point of the initial character for which width-table information is required.

GreQueryWidthTable Parameter - cWidthTable

cWidthTable (LONG) - input
Count of widths in the width-table data.

GreQueryWidthTable Parameter - paWidthTable

paWidthTable (PLONG) - input
Pointer to buffer of width-table data.

GreQueryWidthTable Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryWidthTable Parameter - lFunction

lFunction (ULONG) - input

High-order WORD=flags; low-order WORD=NGreQueryWidthTable.

GreQueryWidthTable Return Value - rc

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COORD_SPACE
PMERR_INV_FIRST_CHAR
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MATRIX_ELEMENT
PMERR_INV_SETID
PMERR_INV_TRANSFORM_TYPE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryWidthTable - Parameters

hdc (HDC) - input
Device context handle.

lFirstChar (LONG) - input
Code point of the initial character for which width-table information is required.

cWidthTable (LONG) - input
Count of widths in the width-table data.

paWidthTable (PLONG) - input
Pointer to buffer of width-table data.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryWidthTable.

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COORD_SPACE
PMERR_INV_FIRST_CHAR
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MATRIX_ELEMENT
PMERR_INV_SETID
PMERR_INV_TRANSFORM_TYPE

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreQueryWidthTable - Remarks

The handling routine must use GreConvert (see GreConvert) to transform the width-table information from device coordinates to world coordinates.

GreQueryWidthTable - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreRealizeColorTable

GreRealizeColorTable - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description: *GreRealizeColorTable* causes the system to ensure that, for a realizable color table, the device physical color table is set to the closest possible match to the logical color table.

This function must be supported by the presentation driver. GreRealizeColorTable is called by GpiRealizeColorTable in response to the request of an application to realize the current logical color table to device output capabilities. This function can be handled by bit-map simulation.

```
#define INCL_GRE_COLORTABLE
#include <os2.h>
```

```
HDC      hdc;         /*  Device context handle. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreRealizeColorTable. */
BOOL     rc;          /*  Return Code. */

rc = GreRealizeColorTable(hdc, pInstance,
       lFunction);
```

GreRealizeColorTable Parameter - hdc

hdc (HDC) - input
Device context handle.

GreRealizeColorTable Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreRealizeColorTable Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRealizeColorTable.

GreRealizeColorTable Return Value - rc

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COL_TABLE_NOT_REALIZABLE
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_REALIZE_NOT_SUPPORTED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreRealizeColorTable - Parameters

hdc (HDC) - input
Device context handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRealizeColorTable.

rc (BOOL) - returns
Return Code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COL_TABLE_NOT_REALIZABLE
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_REALIZE_NOT_SUPPORTED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreRealizeColorTable - Remarks

A device context such as a hardcopy DC can implicitly realize a color table when it is created. In this case, the handling routine need only return Successful. An error must be posted if this function is called for a color table that cannot be realized.

Note: GreRealizeColorTable is provided for the support of older applications and is of less importance for new applications.

GreRealizeColorTable - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreRealizeFont

GreRealizeFont - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreRealizeFont requests the presentation driver to realize or delete a font.

This function must be supported by the presentation driver. GreRealizeFont is called during the processing of GpiCreateLogFont to realize the specified logical font. This function call can be handled by bit-map simulation.

```
#define INCL_GRE_DEVMISC2
#include <os2.h>

HDC         hdc;          /*  Device context handle. */
ULONG       cmdCommand;   /*  Command. */
PFATTRS     pLogFont;     /*  Pointer to a logical font. */
PULONG      pFont;        /*  Pointer or Handle to font. */
PVOID       pInstance;    /*  Pointer to instance data. */
ULONG       lFunction;    /*  High-order WORD=flags; low-order WORD=NGreRealizeFont. */
ULONG       rc;           /*  Return Code. */

rc = GreRealizeFont(hdc, cmdCommand, pLogFont,
       pFont, pInstance, lFunction);
```

GreRealizeFont Parameter - hdc

hdc (HDC) - input
Device context handle.

GreRealizeFont Parameter - cmdCommand

cmdCommand (ULONG) - input
Command.

Valid commands are:

RF_DEVICE_FONT

(1) The graphics engine asks the presentation driver if it is able to realize a device font for the logical font identified by pLogFont. The action taken by the handling routine depends on the match number of the logical font. The match number is the lMatch parameter in the FONTMETRICS structure of the font specified. See Font Matching for further information.

- If lMatch is negative, the request is for the device font with the corresponding match number. The handling routine must return the font handle, or 0 if no match was found.
- If lMatch is 0, the handling routine must

search the device fonts for an exact match to pLogFont and return its handle, or 0 if no match was found.

- If lMatch is a positive value, the handling routine must return 0, as if no font was found.

RF_LOAD_ENGINE_FONT	(2) The presentation driver is requested to convert an engine font into a device font. The presentation driver must return the font handle, or 0 if it is unable to convert the font.
RF_DELETE_FONT	(3) The presentation driver is requested to delete a device font. The 32-bit font handle is passed in the pFont parameter.
RF_DELETE_ENGINE_FONT	(4) Informs the presentation driver that a previously loaded engine font is being deleted. If font caching is not supported, the handling routine performs the action.

The font is passed to the presentation driver in the device character bundle during the GreDeviceSetAttribute call. (See Character Attributes.) A bit in the CHARDEFS structure indicates whether this is a pointer to an engine font or the handle to a device font.

GreRealizeFont Parameter - pLogFont

pLogFont (PFATTRS) - input
Pointer to a logical font.

The initial value of cmdCommand determines the nature of this parameter:

RF_DEVICE_FONT	Pointer to a logical font
RF_LOAD_ENGINE_FONT	Pointer to a logical font
RF_DELETE_FONT	NULL pointer
RF_DELETE_ENGINE_FONT	NULL pointer

GreRealizeFont Parameter - pFont

pFont (PULONG) - input
Pointer or Handle to font.

The initial value of cmdCommand determines the nature of this parameter:

RF_DEVICE_FONT	NULL pointer
RF_LOAD_ENGINE_FONT	Pointer to an engine font
RF_DELETE_FONT	32-bit device font handle
RF_DELETE_ENGINE_FONT	Pointer to an engine font.

GreRealizeFont Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreRealizeFont Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRealizeFont.

GreRealizeFont Return Value - rc

rc (ULONG) - returns
Return Code.

The value returned by this function depends on whether the presentation driver is requested to realize or to delete the font. When realizing or loading a font, the handling routine must return a 32-bit logical font handle (GPI_ALTERROR), or 0 if the match (or load) was unsuccessful.

When deleting a font, the handling routine must return:

GPI_OK	Successful
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT

Refer to the "Error Explanations" section on the *Presentation Manager Programming Reference* for further explanation.

GreRealizeFont - Parameters

hdc (HDC) - input
Device context handle.

cmdCommand (ULONG) - input
Command.

Valid commands are:

RF_DEVICE_FONT

(1) The graphics engine asks the presentation driver if it is able to realize a device font for the logical font identified by pLogFont. The action taken by the handling routine depends on the match number of the logical font. The match number is the lMatch parameter in the FONTMETRICS structure of the font specified. See Font Matching for further information.

- If lMatch is negative, the request is for the device font with the corresponding match number. The handling routine must return the font handle, or 0 if no match was found.

- If lMatch is 0, the handling routine must search the device fonts for an exact match to pLogFont and return its handle, or 0 if no match was found.
- If lMatch is a positive value, the handling routine must return 0, as if no font was found.

RF_LOAD_ENGINE_FONT	(2) The presentation driver is requested to convert an engine font into a device font. The presentation driver must return the font handle, or 0 if it is unable to convert the font.
RF_DELETE_FONT	(3) The presentation driver is requested to delete a device font. The 32-bit font handle is passed in the pFont parameter.
RF_DELETE_ENGINE_FONT	(4) Informs the presentation driver that a previously loaded engine font is being deleted. If font caching is not supported, the handling routine performs the action.

The font is passed to the presentation driver in the device character bundle during the GreDeviceSetAttribute call. (See Character Attributes.) A bit in the CHARDEFS structure indicates whether this is a pointer to an engine font or the handle to a device font.

pLogFont (PFATTRS) - input
Pointer to a logical font.

The initial value of cmdCommand determines the nature of this parameter:

RF_DEVICE_FONT	Pointer to a logical font
RF_LOAD_ENGINE_FONT	Pointer to a logical font
RF_DELETE_FONT	NULL pointer
RF_DELETE_ENGINE_FONT	NULL pointer

pFont (PULONG) - input
Pointer or Handle to font.

The initial value of cmdCommand determines the nature of this parameter:

RF_DEVICE_FONT	NULL pointer
RF_LOAD_ENGINE_FONT	Pointer to an engine font
RF_DELETE_FONT	32-bit device font handle
RF_DELETE_ENGINE_FONT	Pointer to an engine font.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRealizeFont.

rc (ULONG) - returns
Return Code.

The value returned by this function depends on whether the presentation driver is requested to realize or to delete the font. When realizing or loading a font, the handling routine must return a 32-bit logical font handle (GPI_ALTERROR), or 0 if the match (or load) was unsuccessful.

When deleting a font, the handling routine must return:

GPI_OK	Successful
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT

Refer to the "Error Explanations" section on the *Presentation Manager Programming Reference* for further explanation.

GreRealizeFont - Remarks

GreRealizeFont is called by the graphics engine to allow the presentation driver to satisfy logical font requests. The example below shows a typical font-matching algorithm for the presentation driver:

```
/* If the face name is empty, the application is requesting the default font.  Return NO_MATCH.*/

   if(Font->szFacename = NULL)
   {
      return(0);
   }

/* If the match number is positive, an engine font is required.  Return NO_MATCH.              */

   if(Font->lfMatch > 0)
   {
      return(0);
   }

/* If the match number is negative, a device font is required.  The presentation driver should  */
/* return the font if it exists; otherwise return NO_MATCH.                                     */

   if(Font->lMatch < 0)
   {
     if(font with required lMatch exists)
     {
     /* Check the fsSelection flags and szFacename.  If the font is unable to satisfy these     */
     /* flags, return NO_MATCH.  Otherwise, return the device font handle.                       */

       if((FATTR_FONTUSE_OUTLINE && font not outline)
         ||(FATTR_FONTUSE_TRANSFORMABLE && font not outline) ||(szFacename does not match font))
          return(0);
       else
          return(device_font_handle);
     }
     else
     {
     /* Attempt metrics match (i.e. all metrics including szFaceName, usCodePage, lAveCharWidth */
     /* and lMaxBaselineExt).  If match exists, return device font handle.  Otherwise, return   */
     /* NO_MATCH.                                                                                */

       if(metric match exists)
       {
          return(device_font_handle);
       }
       else
          return(0);
     }
   }

/* The match number is zero, the presentation driver should search for a font with the          */
/* specified metrics, and if an exact match exists, return the device font handle. Otherwise,   */
/* return NO_MATCH.                                                                             */

if(Font->lMatch = 0)
{
  if(metrics match exists - see above)
  {
  /* Check the fsSelection flags.  If the font is unable to satisfy these flags, return          */
  /* NO_MATCH.  Otherwise, return the device font handle.                                        */

    if((FATTR_FONTUSE_OUTLINE && font not outline)
    ||(FATTR_FONTUSE_TRANSFORMABLE && font not outline)
       return(0);
    else
       return(device_font_handle);
  }
else
   return(0);
}
```

Graphics Engine Simulation

1. The GRE calls GreRealizeFont with cmdCommand equal to RF_DEVICE_FONT if the driver returns 0.

2. The GRE calls again with cmdCommand == RF_LOAD_ENGINE_FONT and the pFont parameter points to a FOCAFONT structure.

3. If the fsDefn field for the font has FM_DEFN_GENERIC *and* the driver does not report that it is CAPS_RASTER for the GreQueryDeviceCaps() entry point, the FOCAFONT will be followed by the character glyphs as documented in the *Presentation Manager Programming Reference, Volume 3* .

4. Subsequent GreDeviceSetAttributes() calls will have the CDEF_GENERIC flag turned off for the font.

GreRealizeFont - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreRealizePalette

GreRealizePalette - Syntax

Simulation support:

This function is simulated by a handling routine in Revision 2.0 (or higher) of the graphics engine.

Description

GreRealizePalette is called by the Window Manager during processing of a call to WinRealizePalette. GreRealizePalette takes a logical palette and assigns it slots in the hardware palette.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_PALETTE
#include <os2.h>

HDC       hdc;              /*  Device context handle. */
PULONG    pflType;          /*  Specifies whether the DC is a foreground or background window. */
PULONG    pcSlotsChanged;   /*  Pointer to the returned count of hardware slots changed. */
PVOID     pInstance;        /*  Pointer to instance data. */
ULONG     lFunction;        /*  High-order WORD=flags; low-order WORD=NGreRealizePalette. */
ULONG     rc;               /*  Return codes. */

rc = GreRealizePalette(hdc, pflType, pcSlotsChanged,
       pInstance, lFunction);
```

GreRealizePalette Parameter - hdc

hdc (HDC) - input
Device context handle.

GreRealizePalette Parameter - pflType

pflType (PULONG) - input
Specifies whether the DC is a foreground or background window.

Used to indicate if the default colors have changed. The possible values are:

RP_BACKGROUND=0	When a palette is realized in the background (that is, the application making the WinRealizePalette call does not have the focus), then the driver should not change any entries in the hardware palette that are already mapped by other logical palettes. The presentation driver must either modify hardware palette entries that are currently unused, modify the default palette size to make additional slots available, or locate nearest color matches.
RP_FOREGROUND=1	When a logical palette is realized in the foreground (that is, the application making the WinRealizePalette call has the focus), then that palette receives priority over any other palettes that were previously realized. To satisfy the color requests of the foreground WinRealizePalette, the driver should overwrite entries in the hardware palette that are allocated to other applications, if necessary.
RP_DEFAULTSCHANGED=2	Returned by the presentation driver to indicate that all direct DCs need repainting, due to a change in the number of default system colors.

GreRealizePalette Parameter - pcSlotsChanged

pcSlotsChanged (PULONG) - input
Pointer to the returned count of hardware slots changed.

GreRealizePalette Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreRealizePalette Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRealizePalette.

GreRealizePalette Return Value - rc

rc (ULONG) - returns
Return codes.

On completion, this function returns the following value:

PAL-ERROR Error	Number of logical palette mappings changed.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_IN_AREA
PMERR_INV_IN_PATH

GreRealizePalette - Parameters

hdc (HDC) - input
Device context handle.

pflType (PULONG) - input
Specifies whether the DC is a foreground or background window.

Used to indicate if the default colors have changed. The possible values are:

RP_BACKGROUND=0	When a palette is realized in the background (that is, the application making the WinRealizePalette call does not have the focus), then the driver should not change any entries in the hardware palette that are already mapped by other logical palettes. The presentation driver must either modify hardware palette entries that are currently unused, modify the default palette size to make additional slots available, or locate nearest color matches.
RP_FOREGROUND=1	When a logical palette is realized in the foreground (that is, the application making the WinRealizePalette call has the focus), then that palette receives priority over any other palettes that were previously realized. To satisfy the color requests of the foreground WinRealizePalette, the driver should overwrite entries in the hardware palette that are allocated to other applications, if necessary.
RP_DEFAULTSCHANGED=2	Returned by the presentation driver to indicate that all direct DCs need repainting, due to a change in the number of default system colors.

pcSlotsChanged (PULONG) - input
Pointer to the returned count of hardware slots changed.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRealizePalette.

rc (ULONG) - returns
Return codes.

On completion, this function returns the following value:

PAL-ERROR Error	Number of logical palette mappings changed.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_IN_AREA
PMERR_INV_IN_PATH

GreRealizePalette - Remarks

If a palette has not been explicitly selected into the DC before a call to GreRealizePalette is made, the default palette is implicitly in effect and is realized.

This function requires no action for memory DCs. A call to GreRealizePalette is a real operation only when an actual device with a palette is referenced. The function does not return an error for memory DCs, therefore, an application need not be concerned with the DC type if it has a number of DCs with palettes.

When GreBitblt is performed between a memory DC and a device DC, the presentation driver has to perform the appropriate translation between the memory bit map's color table or logical palette and the physical palette.

GreRealizePalette - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreRectInRegion

GreRectInRegion - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreRectInRegion checks whether any part of a given rectangle lies within the specified region. GreRectInRegion raises an error if hrgn is the handle of the currently selected clip region.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_REGIONS
#include <os2.h>

HDC       hdc;        /*  Device context handle. */
HRGN      hrgn;       /*  Region handle. */
PRECTL    prclRect;   /*  Pointer to rectangle in device coordinates. */
```

```
PVOID     pInstance;  /*  Pointer to instance data. */
ULONG     lFunction;  /*  High-order WORD=flags; low-order WORD=NGreRectInRegion. */
LONG      rc;         /*  Return codes. */

rc = GreRectInRegion(hdc, hrgn, prclRect,
       pInstance, lFunction);
```

GreRectInRegion Parameter - hdc

hdc (HDC) - input
Device context handle.

GreRectInRegion Parameter - hrgn

hrgn (HRGN) - input
Region handle.

GreRectInRegion Parameter - prclRect

prclRect (PRECTL) - input
Pointer to rectangle in device coordinates.

RECTL structure:

xLeft	Minimum x-coordinate of rectangle
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of rectangle
yTop	Maximum y-coordinate

The rectangle passed is considered "exclusive" on the right and top sides. If the rectangle represents the bounding box of a graphic object to be drawn, the drawing might include the right and top sides. In this case, the caller should "inflate" the top and right sides by one unit in device-coordinate space.

GreRectInRegion Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreRectInRegion Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRectInRegion.

GreRectInRegion Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (lInside) indicating whether the rectangle is inside the region:

RRGN_ERROR	Error
RRGN_INSIDE	All in region
RRGN_PARTIAL	Partially in region
RRGN_OUTSIDE	Not in region

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HRGN_BUSY
PMERR_INV_COORDINATE
PMERR_INV_HRGN
PMERR_INV_RECT
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreRectInRegion - Parameters

hdc (HDC) - input
Device context handle.

hrgn (HRGN) - input
Region handle.

prclRect (PRECTL) - input
Pointer to rectangle in device coordinates.

RECTL structure:

xLeft	Minimum x-coordinate of rectangle
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of rectangle
yTop	Maximum y-coordinate

The rectangle passed is considered "exclusive" on the right and top sides. If the rectangle represents the bounding box of a graphic object to be drawn, the drawing might include the right and top sides. In this case, the caller should "inflate" the top and right sides by one unit in device-coordinate space.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRectInRegion.

rc (LONG) - returns
Return codes.

This function returns an integer (lInside) indicating whether the rectangle is inside the region:

RRGN_ERROR	Error
RRGN_INSIDE	All in region
RRGN_PARTIAL	Partially in region

RRGN_OUTSIDE	Not in region

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HRGN_BUSY
PMERR_INV_COORDINATE
PMERR_INV_HRGN
PMERR_INV_RECT
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreRectInRegion - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreRectVisible

GreRectVisible - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreRectVisible checks whether any part of the given rectangle is visible within the DC region. The DC region is the intersection of the application clipping and the window clipping.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_CLIP
#include <os2.h>

HDC       hdc;        /*  Device context handle. */
PRECTL    prclRect;   /*  Pointer to rectangle in world or screen coordinates. RECTL structure: */
PVOID     pInstance;  /*  Pointer to instance data. */
ULONG     lFunction;  /*  High-order WORD=flags; low-order WORD=NGreRectVisible. */
LONG      rc;         /*  Return codes. */

rc = GreRectVisible(hdc, prclRect, pInstance,
       lFunction);
```

GreRectVisible Parameter - hdc

hdc (HDC) - input
Device context handle.

GreRectVisible Parameter - prclRect

prclRect (PRECTL) - input
Pointer to rectangle in world or screen coordinates. RECTL structure:

xLeft	Minimum x-coordinate of rectangle
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of rectangle
yTop	Maximum y-coordinate

GreRectVisible Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreRectVisible Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRectVisible.

GreRectVisible Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (lVisible) indicating the visibility of the rectangle.

RVIS_ERROR	Error
RVIS_INVISIBLE	Not visible
RVIS_PARTIAL	Partially visible
RVIS_VISIBLE	All visible

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE

PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreRectVisible - Parameters

hdc (HDC) - input
Device context handle.

prclRect (PRECTL) - input
Pointer to rectangle in world or screen coordinates. RECTL structure:

xLeft	Minimum x-coordinate of rectangle
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of rectangle
yTop	Maximum y-coordinate

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRectVisible.

rc (LONG) - returns
Return codes.

This function returns an integer (lVisible) indicating the visibility of the rectangle.

RVIS_ERROR	Error
RVIS_INVISIBLE	Not visible
RVIS_PARTIAL	Partially visible
RVIS_VISIBLE	All visible

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreRectVisible - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreRegionSelectBitmap

GreRegionSelectBitmap - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreRegionSelectBitmap is called when a new bit-map handle is selected into a memory DC. It removes the old visible region and informs the presentation driver that the DC region must be recalculated.

Note: In a nondisplay DC, the visible region represents the device boundary.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_CLIP
#include <os2.h>

HDC         hdc;         /*  Device context handle. */
HBITMAP     hbm;         /*  Handle of bit map to be selected. */
PVOID       pInstance;   /*  Pointer to instance data. */
ULONG       lFunction;   /*  High-order WORD=flags; low-order WORD=NGreRegionSelectBitmap. */
BOOL        fSuccess;    /*  Return codes. */

fSuccess = GreRegionSelectBitmap(hdc, hbm,
             pInstance, lFunction);
```

GreRegionSelectBitmap Parameter - hdc

hdc (HDC) - input
Device context handle.

GreRegionSelectBitmap Parameter - hbm

hbm (HBITMAP) - input
Handle of bit map to be selected.

GreRegionSelectBitmap Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreRegionSelectBitmap Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRegionSelectBitmap.

GreRegionSelectBitmap Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreRegionSelectBitmap - Parameters

hdc (HDC) - input
Device context handle.

hbm (HBITMAP) - input
Handle of bit map to be selected.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRegionSelectBitmap.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED

PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreRegionSelectBitmap - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreResetBounds

GreResetBounds - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreResetBounds resets the bounds to their initial values, hex 07FFFFF for the minimum coordinates and hex F800000 for the maximum coordinates.

This function must be supported by the presentation driver. GreResetBounds is called by the function GpiResetBoundaryData, and is used to reset the boundary data for a presentation space or device context pair.

```
#define INCL_GRE_DEVMISC3
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    flOptions;   /*  Option flags. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreResetBounds. */
BOOL     rc;          /*  Return code. */

rc = GreResetBounds(hdc, flOptions, pInstance,
       lFunction);
```

GreResetBounds Parameter - hdc

hdc (HDC) - input
Device context handle.

GreResetBounds Parameter - flOptions

flOptions (ULONG) - input
Option flags.

Valid flags are:

RB_GPI	Reset the GPI bounds
RB_USER	Reset the user bounds

GreResetBounds Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreResetBounds Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreResetBounds.

GreResetBounds Return Value - rc

rc (BOOL) - returns
Return code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreResetBounds - Parameters

hdc (HDC) - input
Device context handle.

flOptions (ULONG) - input
Option flags.

Valid flags are:

RB_GPI	Reset the GPI bounds
RB_USER	Reset the user bounds

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreResetBounds.

rc (BOOL) - returns
Return code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreResetBounds - Remarks

Hardcopy drivers are required to support only GPI bounds. Display drivers must also support user bounds for the Window Manager.

GreResetBounds - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreRestorePath

GreRestorePath - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreRestorePath is called during RestoreDC and CloseDC to allow the path handling routines to restore their local data structures. When a DC is closed, GreRestorePath is called with a count of 0 to free its local data.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_PATHS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
LONG     cSave;       /*  DC save level. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreRestorePath. */
BOOL     fSuccess;    /*  Return codes. */

fSuccess = GreRestorePath(hdc, cSave, pInstance,
             lFunction);
```

GreRestorePath Parameter - hdc

hdc (HDC) - input
Device context handle.

GreRestorePath Parameter - cSave

cSave (LONG) - input
DC save level.

Indicates the saved DC state, which the handling routine uses to restore the path. When cSave is passed as 1, the handling routine resets the path to its initial state (-1 is passed to this routine from GreResetDC and is the only valid negative value). A value of 0 indicates that the path is restored to its initial state. Other positive values identify which saved level is restored. See RestoreDCState.

GreRestorePath Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreRestorePath Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRestorePath.

GreRestorePath Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreRestorePath - Parameters

hdc (HDC) - input
Device context handle.

cSave (LONG) - input
DC save level.

Indicates the saved DC state, which the handling routine uses to restore the path. When cSave is passed as 1, the handling routine resets the path to its initial state (-1 is passed to this routine from GreResetDC and is the only valid negative value). A value of 0 indicates that the path is restored to its initial state. Other positive values identify which saved level is restored. See RestoreDCState.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRestorePath.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreRestorePath - Remarks

This function is required to ensure that memory is freed.

GreRestorePath - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreRestoreRegion

GreRestoreRegion - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreRestoreRegion is called during RestoreDC and CloseDC to allow the region handling routines to restore their local data structures. When a DC is closed, GreRestoreRegion is called with a count of 0 to free its local data. The clip region currently selected into the DC is deleted by this function.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_CLIP
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
LONG     cSave;       /*  DC save level. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreRestoreRegion.

 */
BOOL     fSuccess;    /*  Return codes. */

fSuccess = GreRestoreRegion(hdc, cSave, pInstance,
            lFunction);
```

GreRestoreRegion Parameter - hdc

hdc (HDC) - input
Device context handle.

GreRestoreRegion Parameter - cSave

cSave (LONG) - input
DC save level.

Indicates the saved DC state, which the handling routine uses to restore the region. When this is passed as -1, the handling routine resets the region to its initial state (-1 is passed to this routine from GreResetDC and is the only valid negative value). A value of 0 indicates that the region is restored to its initial state. Other positive values identify which saved level is restored. See RestoreDCState.

GreRestoreRegion Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreRestoreRegion Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRestoreRegion.

GreRestoreRegion Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreRestoreRegion - Parameters

hdc (HDC) - input
Device context handle.

cSave (LONG) - input
DC save level.

Indicates the saved DC state, which the handling routine uses to restore the region. When this is passed as -1, the handling routine resets the region to its initial state (-1 is passed to this routine from GreResetDC and is the only valid negative value). A value of 0 indicates that the region is restored to its initial state. Other positive values identify which saved level is restored. See RestoreDCState.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRestoreRegion.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreRestoreRegion - Remarks

This function is required to ensure that memory is freed.

GreRestoreRegion - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreRestoreScreenBits

GreRestoreScreenBits - Syntax

Simulation support:

This function is mandatory for display drivers. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreRestoreScreenBits restores a rectangle of bits to a screen rectangle and can also free the handle of the saved bits.

```
#define INCL_GRE_BITMAPS
#include <os2.h>

HDC       hdc;         /*  The device context handle. */
ULONG     hsbBits;     /*  The handle to screen bits to be restored. */
PRECTL    prclRect;
ULONG     flOptions;
PVOID     pInstance;   /*  A pointer to instance data. */
ULONG     lFunction;
BOOL      fSuccess;    /*  Return codes. */

fSuccess = GreRestoreScreenBits(hdc, hsbBits,
             prclRect, flOptions, pInstance,
             lFunction);
```

GreRestoreScreenBits Parameter - hdc

hdc (HDC) - input
The device context handle.

GreRestoreScreenBits Parameter - hsbBits

hsbBits (ULONG) - input
The handle to screen bits to be restored.

GreRestoreScreenBits Parameter - prclRect

prclRect (PRECTL) - input
A pointer to a screen rectangle defined in screen coordinates.

GreRestoreScreenBits Parameter - flOptions

flOptions (ULONG) - input
Option flags, valid values are:

RSB_FREE	*1* (free the save bits handle)
RSB_RESTORE	*2* (restore the bits to the screen)

GreRestoreScreenBits Parameter - pInstance

pInstance (PVOID) - input
A pointer to instance data.

GreRestoreScreenBits Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRestoreScreenBits.

GreRestoreScreenBits Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_IS_SELECTED
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INCOMPATIBLE_BITMAP
PMERR_INCORRECT_DC_TYPE
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_BITBLT_MIX

PMERR_INV_BITBLT_STYLE
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_CHAR_DIRECTION_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CHAR_SET_ATTR
PMERR_INV_CHAR_SHEAR_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_FORMAT
PMERR_INV_COLOR_INDEX
PMERR_INV_COLOR_OPTIONS
PMERR_INV_COLOR_START_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_DC_DATA
PMERR_INV_DC_TYPE
PMERR_INV_DRIVER_NAME
PMERR_INV_GEOM_LINE_WIDTH_ATTR
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_ID
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_END_ATTR
PMERR_INV_LINE_JOIN_ATTR
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_LINE_WIDTH_ATTR
PMERR_INV_MARKER_SET_ATTR
PMERR_INV_MARKER_SYMBOL_ATTR
PMERR_INV_MIX_ATTR
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_PRIMITIVE_TYPE
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_SCAN_START
PMERR_INV_SETID
PMERR_INV_USAGE_PARM
PMERR_REALIZE_NOT_SUPPORTED
PMERR_UNSUPPORTED_ATTR
PMERR_UNSUPPORTED_ATTR_VALUE

GreRestoreScreenBits - Parameters

hdc (HDC) - input
The device context handle.

hsbBits (ULONG) - input
The handle to screen bits to be restored.

prclRect (PRECTL) - input
A pointer to a screen rectangle defined in screen coordinates.

flOptions (ULONG) - input
Option flags, valid values are:

RSB_FREE	*1* (free the save bits handle)
RSB_RESTORE	*2* (restore the bits to the screen)

pInstance (PVOID) - input
A pointer to instance data.

lFunction (ULONG) - input

High-order WORD=flags; low-order WORD=NGreRestoreScreenBits.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_IS_SELECTED
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INCOMPATIBLE_BITMAP
PMERR_INCORRECT_DC_TYPE
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_BITBLT_MIX
PMERR_INV_BITBLT_STYLE
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_CHAR_DIRECTION_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CHAR_SET_ATTR
PMERR_INV_CHAR_SHEAR_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_FORMAT
PMERR_INV_COLOR_INDEX
PMERR_INV_COLOR_OPTIONS
PMERR_INV_COLOR_START_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_DC_DATA
PMERR_INV_DC_TYPE
PMERR_INV_DRIVER_NAME
PMERR_INV_GEOM_LINE_WIDTH_ATTR
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_ID
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_END_ATTR
PMERR_INV_LINE_JOIN_ATTR
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_LINE_WIDTH_ATTR
PMERR_INV_MARKER_SET_ATTR
PMERR_INV_MARKER_SYMBOL_ATTR
PMERR_INV_MIX_ATTR
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_PRIMITIVE_TYPE
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_SCAN_START
PMERR_INV_SETID
PMERR_INV_USAGE_PARM
PMERR_REALIZE_NOT_SUPPORTED
PMERR_UNSUPPORTED_ATTR
PMERR_UNSUPPORTED_ATTR_VALUE

GreRestoreScreenBits - Remarks

Clipping is done on the restored bits, as necessary.

GreRestoreScreenBits - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreRestoreXform

GreRestoreXform - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreRestoreXform is called during RestoreDC and CloseDC to allow simulations to restore their local data structures. When a DC is closed, GreRestoreXform is called with a count of 0 to free its local data.

```
#define INCL_GRE_XFORMS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
LONG     cSave;       /*  DC save level. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreRestoreXform. */
BOOL     fSuccess;    /*  Return codes. */

fSuccess = GreRestoreXform(hdc, cSave, pInstance,
            lFunction);
```

GreRestoreXform Parameter - hdc

hdc (HDC) - input
Device context handle.

GreRestoreXform Parameter - cSave

cSave (LONG) - input
DC save level.

Indicates the saved DC state, which the handling routine uses to restore the transform. When cSave is passed as -1, the handling routine resets the transform to its initial state (-1 is passed to this routine from GreResetDC and is the only valid negative value). A value of 0 indicates that the transform is reset to its initial state. Other positive values identify which saved level is restored. See RestoreDCState.

GreRestoreXform Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreRestoreXform Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRestoreXform.

GreRestoreXform Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreRestoreXform - Parameters

hdc (HDC) - input
Device context handle.

cSave (LONG) - input
DC save level.

Indicates the saved DC state, which the handling routine uses to restore the transform. When cSave is passed as -1, the handling routine resets the transform to its initial state (-1 is passed to this routine from GreResetDC and is the only valid negative value). A value of 0 indicates that the transform is reset to its initial state. Other positive values identify which saved level is restored. See RestoreDCState.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRestoreXform.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreRestoreXform - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreRestoreXformData

GreRestoreXformData - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreRestoreXformData restores a previously saved transform state. The current transform state is overwritten.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_XFORMS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    ulSize;      /*  Size in bytes of pBuffer. */
PBYTE    pBuffer;     /*  Pointer to stored transform data. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreRestoreXformData. */
BOOL     fSuccess;    /*  Return codes. */

fSuccess = GreRestoreXformData(hdc, ulSize,
             pBuffer, pInstance, lFunction);
```

GreRestoreXformData Parameter - hdc

hdc (HDC) - input
Device context handle.

GreRestoreXformData Parameter - ulSize

ulSize (ULONG) - input
Size in bytes of pBuffer.

GreRestoreXformData Parameter - pBuffer

pBuffer (PBYTE) - input
Pointer to stored transform data.

GreRestoreXformData Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreRestoreXformData Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRestoreXformData.

GreRestoreXformData Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PICK_APERTURE_POSN
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreRestoreXformData - Parameters

hdc (HDC) - input
Device context handle.

ulSize (ULONG) - input
Size in bytes of pBuffer.

pBuffer (PBYTE) - input
Pointer to stored transform data.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRestoreXformData.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC

PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PICK_APERTURE_POSN
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreRestoreXformData - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreResurrection

GreResurrection - Syntax

Simulation support:

This function is mandatory for display drivers. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreResurrection reverses the condition set by GreDeath and restores the screen to the Presentation Manager interface.

```
#define INCL_GRE_DEVMISC3
#include <os2.h>

HDC       hdc;         /*  The device context handle. */
LONG      cbVmem;
PULONG    pReserved;   /*  A reserved pointer. Must be zero. */
PVOID     pInstance;   /*  A pointer to instance data. */
ULONG     lFunction;   /*  High-order WORD=flags; low-order WORD=NGreResurrection. */
LONG      lResult;     /*  Return codes. */

lResult = GreResurrection(hdc, cbVmem, pReserved,
            pInstance, lFunction);
```

GreResurrection Parameter - hdc

hdc (HDC) - input
The device context handle.

GreResurrection Parameter - cbVmem

cbVmem (LONG) - input
The number of bytes of video memory that have been damaged (determined by the VIO). The display driver can use this value to determine whether any of its video memory has been destroyed by the application. Some display drivers can ignore this parameter.

GreResurrection Parameter - pReserved

pReserved (PULONG) - input
A reserved pointer. Must be zero.

GreResurrection Parameter - pInstance

pInstance (PVOID) - input
A pointer to instance data.

GreResurrection Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreResurrection.

GreResurrection Return Value - lResult

lResult (LONG) - returns
Return codes.

On completion, the handling routine must return a LONG value (lResult):

0	Error.
1	The screen has been successfully redrawn.
2	The screen has not been completely redrawn; further action is required from the application.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

GreResurrection - Parameters

hdc (HDC) - input
The device context handle.

cbVmem (LONG) - input
The number of bytes of video memory that have been damaged (determined by the VIO). The display driver can use this value to determine whether any of its video memory has been destroyed by the application. Some display drivers can ignore this parameter.

pReserved (PULONG) - input
A reserved pointer. Must be zero.

pInstance (PVOID) - input
A pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreResurrection.

lResult (LONG) - returns
Return codes.

On completion, the handling routine must return a LONG value (lResult):

0	Error.
1	The screen has been successfully redrawn.
2	The screen has not been completely redrawn; further action is required from the application.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

GreResurrection - Remarks

Presentation Manager applications are set to the foreground. The presentation driver is enabled to update the screen for subsequent drawing calls.

This function goes directly to the Presentation Driver interface.

GreResurrection - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSavePath

GreSavePath - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreSavePath is called during SaveDC and OpenDC to allow the path handling routines to save their local data structures. When a new DC is created, GreSavePath is called with a count of 1 to initialize its local data. A Save with a count of one more than the current save level generates intervening levels.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_PATHS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
LONG     cSave;       /*  DC save level. This must not be a negative value. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreSavePath. */
BOOL     fSuccess;    /*  Return codes. */

fSuccess = GreSavePath(hdc, cSave, pInstance,
            lFunction);
```

GreSavePath Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSavePath Parameter - cSave

cSave (LONG) - input
DC save level. This must not be a negative value.

GreSavePath Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSavePath Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSavePath.

GreSavePath Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSavePath - Parameters

hdc (HDC) - input
Device context handle.

cSave (LONG) - input
DC save level. This must not be a negative value.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSavePath.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSavePath - Remarks

This function is required to ensure that the path data is saved.

GreSavePath - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSaveRegion

GreSaveRegion - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreSaveRegion is called during SaveDC and OpenDC to allow the region handling routines to save their local data structures. When a new DC is created, GreSaveRegion is called with a save level of 1 to initialize its local data. A Save with a count of one more than the current save level generates intervening levels. A negative save value is invalid. When initializing a new DC, a valid visible region handle must be created.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_CLIP
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
LONG     cSave;       /*  DC save level. This must not be a negative value. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreSaveRegion. */
BOOL     fSuccess;    /*  Return codes. */

fSuccess = GreSaveRegion(hdc, cSave, pInstance,
            lFunction);
```

GreSaveRegion Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSaveRegion Parameter - cSave

cSave (LONG) - input
DC save level. This must not be a negative value.

GreSaveRegion Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSaveRegion Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSaveRegion.

GreSaveRegion Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_RECT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSaveRegion - Parameters

hdc (HDC) - input
Device context handle.

cSave (LONG) - input
DC save level. This must not be a negative value.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSaveRegion.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_RECT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSaveRegion - Remarks

This function is required to ensure that the region data is saved.

GreSaveRegion - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSaveScreenBits

GreSaveScreenBits - Syntax

Simulation support:

This function is mandatory for display drivers. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreSaveScreenBits saves a rectangle of screen bits.

```
#define INCL_GRE_BITMAPS
#include <os2.h>

HDC       hdc;         /*  The device context handle. */
PRECTL    prclRect;    /*  A pointer to a screen rectangle defined in screen coordinates. */
PVOID     pInstance;   /*  A pointer to instance data. */
ULONG     lFunction;   /*  High-order WORD=flags; low-order WORD=NGreRestoreScreenBits. */
ULONG     rc;          /*  Return codes. */

rc = GreSaveScreenBits(hdc, prclRect, pInstance,
       lFunction);
```

GreSaveScreenBits Parameter - hdc

hdc (HDC) - input
The device context handle.

GreSaveScreenBits Parameter - prclRect

prclRect (PRECTL) - input
A pointer to a screen rectangle defined in screen coordinates.

GreSaveScreenBits Parameter - pInstance

pInstance (PVOID) - input
A pointer to instance data.

GreSaveScreenBits Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRestoreScreenBits.

GreSaveScreenBits Return Value - rc

rc (ULONG) - returns
Return codes.

On completion, the handling routine must return a handle to the saved bits (hsbBits) or 0 to indicate that the bits were not saved or that an error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

GreSaveScreenBits - Parameters

hdc (HDC) - input
The device context handle.

prclRect (PRECTL) - input
A pointer to a screen rectangle defined in screen coordinates.

pInstance (PVOID) - input
A pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRestoreScreenBits.

rc (ULONG) - returns
Return codes.

On completion, the handling routine must return a handle to the saved bits (hsbBits) or 0 to indicate that the bits were not saved or that an error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

GreSaveScreenBits - Remarks

It is permissible to implement this function by returning 0 to indicate that the bits were not saved and, therefore, must be saved by the calling routine.

This function lets the user-interface routines improve the performance of dialog boxes.

GreSaveScreenBits - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSaveXform

GreSaveXform - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreSaveXform is called during SaveDC and OpenDC to allow simulations to save their local data structures. A Save with a count of one more than the current save level generates intervening levels. When a new DC is created, GreSaveXform is called with a count of 1 to initialize its local data.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_XFORMS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
LONG     cSave;       /*  DC save level. This must not be a negative value. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreSaveXform. */
BOOL     fSuccess;    /*  Return codes. */

fSuccess = GreSaveXform(hdc, cSave, pInstance,
            lFunction);
```

GreSaveXform Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSaveXform Parameter - cSave

cSave (LONG) - input
DC save level. This must not be a negative value.

GreSaveXform Parameter - pInstance

pInstance (PVOID) - input

Pointer to instance data.

GreSaveXform Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSaveXform.

GreSaveXform Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_HDC.

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSaveXform - Parameters

hdc (HDC) - input
Device context handle.

cSave (LONG) - input
DC save level. This must not be a negative value.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSaveXform.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_HDC.

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSaveXform - Remarks

When initializing a new DC, the default transform must be set.

GreSaveXform - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSaveXformData

GreSaveXformData - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreSaveXformData stores the current transform state in pBuffer.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_XFORMS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    ulSize;      /*  Size in bytes required for pBuffer. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreSaveXformData. */
ULONG    rc;          /*  Return codes. */

rc = GreSaveXformData(hdc, ulSize, pInstance,
       lFunction);
```

GreSaveXformData Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSaveXformData Parameter - ulSize

ulSize (ULONG) - input
Size in bytes required for pBuffer.

Can be specified as 0. In this case, the function returns the size of the transform data:

```
ulSize=GreSaveXformData (hdc, 0, 0);     /* Find out how large the buffer must be, */
GreSaveXformData (hdc, ulSize, pBuffer); /* then save the transform state.         */
```

GreSaveXformData Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSaveXformData Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSaveXformData.

GreSaveXformData Return Value - rc

rc (ULONG) - returns
Return codes.

When called with ulSize specified as 0, it returns the size of the buffer required to save the transform state. Otherwise, it returns GPI_OK to indicate a successful completion. In either case, GPI_ERROR is returned to indicate failure.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSaveXformData - Parameters

hdc (HDC) - input
Device context handle.

ulSize (ULONG) - input
Size in bytes required for pBuffer.

Can be specified as 0. In this case, the function returns the size of the transform data:

```
ulSize=GreSaveXformData (hdc, 0, 0);     /* Find out how large the buffer must be, */
GreSaveXformData (hdc, ulSize, pBuffer); /* then save the transform state.         */
```

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSaveXformData.

rc (ULONG) - returns
Return codes.

When called with ulSize specified as 0, it returns the size of the buffer required to save the transform state. Otherwise, it returns GPI_OK to indicate a successful completion. In either case, GPI_ERROR is returned to indicate failure.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSaveXformData - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreScrollRect

GreScrollRect - Syntax

Simulation support:

This function is mandatory for display drivers. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreScrollRect scrolls the contents of the LVB through the DC.

```
#define INCL_AVIOP
#include <os2.h>

HDC                     hdc;            /*  The device context handle. */
VioPresentationSpace   *pVioPS;         /*  A pointer to the Vio presentation space. */
LPScrollRectRef         paScrollRect;
PVOID                   pInstance;      /*  A pointer to instance data. */
ULONG                   lFunction;      /*  High-order WORD=flags; low-order WORD=NGreScrollRect. */
LONG                    rc;             /*  Return codes. */

rc = GreScrollRect(hdc, pVioPS, paScrollRect,
       pInstance, lFunction);
```

GreScrollRect Parameter - hdc

hdc (HDC) - input
The device context handle.

GreScrollRect Parameter - pVioPS

pVioPS (VioPresentationSpace *) - input
A pointer to the Vio presentation space.

GreScrollRect Parameter - paScrollRect

paScrollRect (LPScrollRectRef) - input
Pointer to parameter block for this function. This block is defined as a SCROLLRECTREF structure:

StartRow	Starting row relative to the bottom left of the LVB
StartCol	Starting column in the logical video buffer of the character string to be output
RectWidth	Width of the scroll rectangle
RectDepth	Depth of the scroll rectangle
HorzMovement	Number of columns to be scrolled
VertMovement	Number of rows to be scrolled **Note:** Positive values mean movement downward or to the right, negative values mean upward or to the left.
lpFillCell	Pointer to a cell containing the character and attributes to be

used for filling the tail of the scroll region. This pointer is only used when GreBitblt is used to implement this function. When this lpFillCell is passed as NULL, the logical video buffer has been updated. The handling routine then must call GreCharRect to update the display.

GreScrollRect Parameter - pInstance

pInstance (PVOID) - input
A pointer to instance data.

GreScrollRect Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreScrollRect.

GreScrollRect Return Value - rc

rc (LONG) - returns
Return codes.

This function returns a LONG value as an error indicator.

NO_ERROR	Successful
CE_INVALID_PRESENTATION_SPACE	Error. For example, this message might be returned as a result of an invalid CellByteSize.

GreScrollRect - Parameters

hdc (HDC) - input
The device context handle.

pVioPS (VioPresentationSpace *) - input
A pointer to the Vio presentation space.

paScrollRect (LPScrollRectRef) - input
Pointer to parameter block for this function. This block is defined as a SCROLLRECTREF structure:

StartRow	Starting row relative to the bottom left of the LVB
StartCol	Starting column in the logical video buffer of the character string to be output
RectWidth	Width of the scroll rectangle

RectDepth	Depth of the scroll rectangle
HorzMovement	Number of columns to be scrolled
VertMovement	Number of rows to be scrolled **Note:** Positive values mean movement downward or to the right, negative values mean upward or to the left.
lpFillCell	Pointer to a cell containing the character and attributes to be used for filling the tail of the scroll region. This pointer is only used when GreBitblt is used to implement this function. When this lpFillCell is passed as NULL, the logical video buffer has been updated. The handling routine then must call GreCharRect to update the display.

pInstance (PVOID) - input
A pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreScrollRect.

rc (LONG) - returns
Return codes.

This function returns a LONG value as an error indicator.

NO_ERROR	Successful
CE_INVALID_PRESENTATION_SPACE	Error. For example, this message might be returned as a result of an invalid CellByteSize.

GreScrollRect - Remarks

The contents of the LVB are not affected by this function. Typically, the presentation driver responds to this call by calling GreCharRect. An alternative approach is to use the horizontal and vertical movement fields to define a new source rectangle in the DC and to use GreBitblt to transfer the bits. When new information is revealed from the LVB as a result of the scroll, the handling routine calls GreCharRect to update the display. This approach provides considerable performance advantages for devices that support GreBitblt

---

GreScrollRect - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSelectClipPath

GreSelectClipPath - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreSelectClipPath resets or modifies the currently selected clip path.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_PATHS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
LONG     idPath;      /*  Path ID. */
ULONG    flOptions;   /*  Option flags. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;
BOOL     fSuccess;    /*  Return codes. */

fSuccess = GreSelectClipPath(hdc, idPath,
             flOptions, pInstance, lFunction);
```

GreSelectClipPath Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSelectClipPath Parameter - idPath

idPath (LONG) - input
Path ID.

When this is passed as 0, it indicates "no clipping". Otherwise, the new clip path is intersected with the existing clip path.

GreSelectClipPath Parameter - flOptions

flOptions (ULONG) - input
Option flags.

The following flags determine the path mode:

SCP_ALTERNATE	(Default) Use alternate mode for intersection.
SCP_WINDING	Use winding mode for intersection.

The following flags determine whether the new path replaces or intersects the initial path:

SCP_AND	Intersect the two clip paths. If idPath is not 1, this flag is invalid and the handling routine must log an error.

SCP_RESET	(Default) Replace the initial clip path. If idPath is not 0, this flag is invalid and the handling routine must log an error.
SCP_EXCL	Path is exclusive of bottom and right border in device coordinate.

Other flags are reserved and must be 0.

GreSelectClipPath Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSelectClipPath Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSelectClipPath.

GreSelectClipPath Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CLIP_PATH_OPTIONS
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PATH_ID
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSelectClipPath - Parameters

hdc (HDC) - input
Device context handle.

idPath (LONG) - input
Path ID.

When this is passed as 0, it indicates "no clipping". Otherwise, the new clip path is intersected with the existing clip path.

flOptions (ULONG) - input
Option flags.

The following flags determine the path mode:

SCP_ALTERNATE	(Default) Use alternate mode for intersection.
SCP_WINDING	Use winding mode for intersection.

The following flags determine whether the new path replaces or intersects the initial path:

SCP_AND	Intersect the two clip paths. If idPath is not 1, this flag is invalid and the handling routine must log an error.
SCP_RESET	(Default) Replace the initial clip path. If idPath is not 0, this flag is invalid and the handling routine must log an error.
SCP_EXCL	Path is exclusive of bottom and right border in device coordinate.

Other flags are reserved and must be 0.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSelectClipPath.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CLIP_PATH_OPTIONS
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PATH_ID

PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSelectClipPath - Remarks

Before modifying the clip path, the handling routine must close all open figures in the path. The modified clip path consists of the areas common to both the old clip path and the figures in the specified path. When the constituent paths are defined, the modified clip path is bound in device coordinates and is used for all subsequent drawing, including filling. All of the boundaries of the area are considered to be part of the interior and are not clipped. Upon completion, the handling routine deletes the old clip path indicated by idPath, allowing a new path definition, which can coexist with the new clip path, to begin as required. Refer to function "GpiSetClipPath" in the *Presentation Manager Programming Reference* for further information.

GreSelectClipPath - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSelectClipRegion

GreSelectClipRegion - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreSelectClipRegion specifies the region to be used for clipping.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_CLIP
#include <os2.h>

HDC      hdc;        /*  Device context handle. */
HRGN     hrgn;       /*  Region handle. */
```

```
PHRGN     phrgnOld;
PVOID     pInstance;  /*  Pointer to instance data. */
ULONG     lFunction;
LONG      rc;         /*  Return codes. */

rc = GreSelectClipRegion(hdc, hrgn, phrgnOld,
       pInstance, lFunction);
```

GreSelectClipRegion Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSelectClipRegion Parameter - hrgn

hrgn (HRGN) - input
Region handle.

When NULL, the clip region is set to *clipping* (the initial state).

GreSelectClipRegion Parameter - phrgnOld

phrgnOld (PHRGN) - output
Pointer to previously selected region handle. If this is a NULL handle, there was no clip region.

GreSelectClipRegion Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSelectClipRegion Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSelectClipRegion.

GreSelectClipRegion Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (lComplexity) indicating the complexity of the DC region:

RGN_ERROR	Error
RGN_NULL	Null region
RGN_RECT	Rectangular region
RGN_COMPLEX	Complex region (more than 1 rectangle)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSelectClipRegion - Parameters

hdc (HDC) - input
Device context handle.

hrgn (HRGN) - input
Region handle.

When NULL, the clip region is set to *clipping* (the initial state).

phrgnOld (PHRGN) - output
Pointer to previously selected region handle. If this is a NULL handle, there was no clip region.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSelectClipRegion.

rc (LONG) - returns
Return codes.

This function returns an integer (lComplexity) indicating the complexity of the DC region:

RGN_ERROR	Error
RGN_NULL	Null region
RGN_RECT	Rectangular region
RGN_COMPLEX	Complex region (more than 1 rectangle)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORDINATE
PMERR_INV_HDC

PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSelectClipRegion - Remarks

A region can only be selected by one DC at a time. As soon as a region is selected, operations that reference its handle are invalid. The coordinates of the region are in device coordinates within the DC. Clipping is inclusive at the left and bottom boundaries, exclusive at the right and top boundaries. Functions that modify the clipping region also modify the region when its handle is returned by a subsequent GreSelectClipRegion.

GreSelectClipRegion passes the handle of the previously selected clip region back to the calling routine at the location addressed by phrgnOld. If this handle is not NULL, the calling routine is responsible for deleting the previous clip region. If the default clip region is in use when GreSelectClipRegion is called, a null region handle is returned.

Note: The clip region should be deleted whether it was created explicitly, or in a call to GreIntersectClipRectangle or GreExcludeClipRectangle.

GreSelectClipRegion - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSelectPathRegion

GreSelectPathRegion - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreSelectPathRegion merges a region representing a path into the DC region. A "path region" behaves exactly like any other region. It has no special properties relating to paths.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_CLIP
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
HRGN     hrgn;        /*  Region handle. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreSelectPathRegion. */
BOOL     fSuccess;    /*  Return codes. */

fSuccess = GreSelectPathRegion(hdc, hrgn,
             pInstance, lFunction);
```

GreSelectPathRegion Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSelectPathRegion Parameter - hrgn

hrgn (HRGN) - input
Region handle.

GreSelectPathRegion Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSelectPathRegion Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSelectPathRegion.

GreSelectPathRegion Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE Successful

FALSE Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSelectPathRegion - Parameters

hdc (HDC) - input
Device context handle.

hrgn (HRGN) - input
Region handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSelectPathRegion.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE Successful
FALSE Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSelectPathRegion - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreSetArcParameters

GreSetArcParameters - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreSetArcParameters sets the arc parameters.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_ARCS
#include <os2.h>

HDC           hdc;         /*  Device context handle. */
PARCPARAMS    pArcParms;   /*  Pointer to an array containing the arc parameters.

ARCPARAMS structure: */
PVOID         pInstance;   /*  Pointer to instance data. */
ULONG         lFunction;   /*  High-order WORD=flags; low-order WORD=NGreSetArcParameters. */
BOOL          fSuccess;    /*  Return codes. */

fSuccess = GreSetArcParameters(hdc, pArcParms,
             pInstance, lFunction);
```

GreSetArcParameters Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetArcParameters Parameter - pArcParms

pArcParms (PARCPARAMS) - input
Pointer to an array containing the arc parameters.

ARCPARAMS structure:

IP	P coefficient
IQ	Q coefficient
IR	R coefficient
IS	S coefficient

See the *Presentation Manager Programming Reference* for a full description of the arc parameters structure.

GreSetArcParameters Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetArcParameters Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetArcParameters.

GreSetArcParameters Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetArcParameters - Parameters

hdc (HDC) - input
Device context handle.

pArcParms (PARCPARAMS) - input
Pointer to an array containing the arc parameters.

ARCPARAMS structure:

lP	P coefficient
lQ	Q coefficient
lR	R coefficient
lS	S coefficient

See the *Presentation Manager Programming Reference* for a full description of the arc parameters structure.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetArcParameters.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetArcParameters - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreSetBitmapBits

GreSetBitmapBits - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreSetBitmapBits transfers bit-map data from application storage into the specified bit map or DC.

This function must be supported by the presentation driver. GreSetBitmapBits is called from the function GpiSetBitmapBits, and is used to transfer bit-map data from the device context to application storage. This function can be handled by bit-map simulation.

```
#define INCL_GRE_BITMAPS
#include <os2.h>

HDC             hdc;          /*  Device context handle. */
HBITMAP         hbm;          /*  Bit-map handle. */
LONG            lScanStart;   /*  Scan line number from where data transfer starts. */
LONG            cScanCount;   /*  Number of scan lines to be transferred. */
PBYTE           pBitmap;      /*  Pointer to bit-map data. */
```

```
PBITMAPINFO     pInfo;        /*  Pointer to BITMAPINFO or BITMAPINFO2 structure. */
PVOID           pInstance;    /*  Pointer to instance data. */
ULONG           lFunction;    /*  High-order WORD=flags; low-order WORD=NGreSetBitmapBits. */
LONG            rc;           /*  Return codes. */

rc = GreSetBitmapBits(hdc, hbm, lScanStart,
       cScanCount, pBitmap, pInfo, pInstance,
       lFunction);
```

GreSetBitmapBits Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetBitmapBits Parameter - hbm

hbm (HBITMAP) - input
Bit-map handle.

GreSetBitmapBits Parameter - lScanStart

lScanStart (LONG) - input
Scan line number from where data transfer starts.

0 is the first.

GreSetBitmapBits Parameter - cScanCount

cScanCount (LONG) - input
Number of scan lines to be transferred.

GreSetBitmapBits Parameter - pBitmap

pBitmap (PBYTE) - input
Pointer to bit-map data.

Pointer to the pel data of the bit map. This data is stored in the order that the coordinates appear on a display screen, that is, the pel in the lower-left corner is the first in the bit map. Pels are scanned to the right and upward from that position. The bits of the first pel are stored beginning with the most significant bits of the first byte. The data for pels in each scan line is packed together tightly. However, all scan lines are padded at the end so that each one begins on a ULONG boundary. That is, three bytes of pel data will hold one 24-bit pel,

three 8-bit pels, six 4-bit pels, or twenty-four 1-bit pels. If those three bytes are the only pel data for that scan line, one more byte of zeros would be required to pad the line to a ULONG boundary.

GreSetBitmapBits Parameter - pInfo

pInfo (PBITMAPINFO) - input
Pointer to BITMAPINFO or BITMAPINFO2 structure.

Pointer to either a BITMAPINFO structure:

cbFix	Length of structure
cx	Bit-map width
cy	Bit-map height
cPlanes	Number of color planes, 1 if standard format
cBitCount	Number of adjacent color bits per pel
argbColor[]	Color table array of RGB structures: bBlue bGreen bRed

or pointer to a BITMAPINFO2 structure:

cbFix	Length of structure
cx	Bit-map width
cy	Bit-map height
cPlanes	Number of planes, 1 if standard format
cBitCount	Number of adjacent color bits per pel
ulCompression	Compression scheme used to store the bit map: BCA_UNCOMP — Bit map is uncompressed (the only valid value).
cbImage	Length of bit-map storage data in bytes. If the bit map is uncompressed, 0 (default) can be specified for this.
cxResolution	Horizontal component of the resolution of the target device; that is, the resolution of the device the bit map is intended for in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cyResolution	Vertical component of the resolution of the target device; that is, the resolution of the device the bit map is intended for in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cclrUsed	The number of color indexes from the color table that are used by the bit map. If it is 0 (the default), all the indexes are used. If it is not zero, only the first cclrUsed entries in the table are accessed by the system; further entries can be omitted. For standard formats with a cBitCount of 1, 4, or 8 (and cPlanes=1), any indexes beyond cclrUsed are not valid. For example, a bit map with 64 colors can use the 8-bitcount format without having to supply the other 192 entries in the color table. For the 24-bitcount standard format, cclrUsed is the number of colors used by the bit map.
cclrImportant	Minimum number of color indexes for satisfactory appearance of the bit map. More colors can be used in the bit map but it is not necessary to assign them to the device palette. These additional colors can be mapped to the nearest colors available. Zero (default) means that all entries are important. For a 24-bitcount standard format, the cclrImportant colors are also listed in the color table relating to this bit map.

usUnits	Units of measure of the horizontal and vertical components of resolution:	
	BRU_METRIC	(Default) Pels per meter.
usReserved	Reserved field. If present, it must be 0.	
usRecording	Recording algorithm, the format in which bit-map data is recorded:	
	BRA_BOTTOMUP	(Default) Scan lines are recorded from bottom to top.
usRendering	Halftoning algorithm used to record bit-map data that has been digitally halftoned:	
	BRH_NOTHALFTONED	(Default) Bit-map data not halftoned.
	BRH_ERRORDIFFUSION	Error diffusion or damped error diffusion algorithm
	BRH_PANDA	Processing algorithm for noncoded document acquisition
	BRH_SUPERCIRCLE	Super circle algorithm
cSize1	Size value 1. If BRH_ERRORDIFFUSION is specified in usRendering, cSize1 is the error damping as a percentage in the range 0 - 100. A value of 100% indicates no damping. A value of 0% indicates that any errors are not diffused. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize1 is the X-dimension of the pattern used in pels.	
cSize2	Size value 2. If BRH_ERRORDIFFUSION is specified in usRendering, this parameter is ignored. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize2 is the Y-dimension in pels of the pattern used.	
ulColorEncoding	Color encoding:	
	BCE_RGB	(Default) Each element in the color array is an RGB2 data type.
ulIdentifier	Reserved for application use.	
argbColor[]	Color table array of RGB2 structures:	
	bBlue bGreen bRed fcOptions	Reserved. Must be 0.

GreSetBitmapBits Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetBitmapBits Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetBitmapBits.

GreSetBitmapBits Return Value - rc

rc (LONG) - returns
Return codes.

On completion, the handling routine must return a LONG value (lLines), indicating the number of lines transferred, or GPI_ALTERROR if an error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BITMAP_NOT_SELECTED
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INCORRECT_DC_TYPE
PMERR_INV_HBITMAP
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SCAN_START

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetBitmapBits - Parameters

hdc (HDC) - input
Device context handle.

hbm (HBITMAP) - input
Bit-map handle.

lScanStart (LONG) - input
Scan line number from where data transfer starts.

0 is the first.

cScanCount (LONG) - input
Number of scan lines to be transferred.

pBitmap (PBYTE) - input
Pointer to bit-map data.

Pointer to the pel data of the bit map. This data is stored in the order that the coordinates appear on a display screen, that is, the pel in the lower-left corner is the first in the bit map. Pels are scanned to the right and upward from that position. The bits of the first pel are stored beginning with the most significant bits of the first byte. The data for pels in each scan line is packed together tightly. However, all scan lines are padded at the end so that each one begins on a ULONG boundary. That is, three bytes of pel data will hold one 24-bit pel, three 8-bit pels, six 4-bit pels, or twenty-four 1-bit pels. If those three bytes are the only pel data for that scan line, one more byte of zeros would be required to pad the line to a ULONG boundary.

pInfo (PBITMAPINFO) - input
Pointer to BITMAPINFO or BITMAPINFO2 structure.

Pointer to either a BITMAPINFO structure:

cbFix	Length of structure
cx	Bit-map width

cy	Bit-map height
cPlanes	Number of color planes, 1 if standard format
cBitCount	Number of adjacent color bits per pel
argbColor[]	Color table array of RGB structures: bBlue bGreen bRed

or pointer to a BITMAPINFO2 structure:

cbFix	Length of structure
cx	Bit-map width
cy	Bit-map height
cPlanes	Number of planes, 1 if standard format
cBitCount	Number of adjacent color bits per pel
ulCompression	Compression scheme used to store the bit map: BCA_UNCOMP — Bit map is uncompressed (the only valid value).
cbImage	Length of bit-map storage data in bytes. If the bit map is uncompressed, 0 (default) can be specified for this.
cxResolution	Horizontal component of the resolution of the target device; that is, the resolution of the device the bit map is intended for in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cyResolution	Vertical component of the resolution of the target device; that is, the resolution of the device the bit map is intended for in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cclrUsed	The number of color indexes from the color table that are used by the bit map. If it is 0 (the default), all the indexes are used. If it is not zero, only the first cclrUsed entries in the table are accessed by the system; further entries can be omitted. For standard formats with a cBitCount of 1, 4, or 8 (and cPlanes=1), any indexes beyond cclrUsed are not valid. For example, a bit map with 64 colors can use the 8-bitcount format without having to supply the other 192 entries in the color table. For the 24-bitcount standard format, cclrUsed is the number of colors used by the bit map.
cclrImportant	Minimum number of color indexes for satisfactory appearance of the bit map. More colors can be used in the bit map but it is not necessary to assign them to the device palette. These additional colors can be mapped to the nearest colors available. Zero (default) means that all entries are important. For a 24-bitcount standard format, the cclrImportant colors are also listed in the color table relating to this bit map.
usUnits	Units of measure of the horizontal and vertical components of resolution: BRU_METRIC — (Default) Pels per meter.
usReserved	Reserved field. If present, it must be 0.
usRecording	Recording algorithm, the format in which bit-map data is recorded: BRA_BOTTOMUP — (Default) Scan lines are recorded from bottom to top.
usRendering	Halftoning algorithm used to record bit-map data that has been digitally halftoned: BRH_NOTHALFTONED — (Default) Bit-map data not halftoned.

		BRH_ERRORDIFFUSION	Error diffusion or damped error diffusion algorithm
		BRH_PANDA	Processing algorithm for noncoded document acquisition
		BRH_SUPERCIRCLE	Super circle algorithm
	cSize1	Size value 1. If BRH_ERRORDIFFUSION is specified in usRendering, cSize1 is the error damping as a percentage in the range 0 - 100. A value of 100% indicates no damping. A value of 0% indicates that any errors are not diffused. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize1 is the X-dimension of the pattern used in pels.	
	cSize2	Size value 2. If BRH_ERRORDIFFUSION is specified in usRendering, this parameter is ignored. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize2 is the Y-dimension in pels of the pattern used.	
	ulColorEncoding	Color encoding:	
		BCE_RGB	(Default) Each element in the color array is an RGB2 data type.
	ulIdentifier	Reserved for application use.	
	argbColor[]	Color table array of RGB2 structures:	
		bBlue bGreen bRed fcOptions	Reserved. Must be 0.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetBitmapBits.

rc (LONG) - returns
Return codes.

On completion, the handling routine must return a LONG value (lLines), indicating the number of lines transferred, or GPI_ALTERROR if an error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BITMAP_NOT_SELECTED
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INCORRECT_DC_TYPE
PMERR_INV_HBITMAP
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SCAN_START

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetBitmapBits - Remarks

When the bit-map handle is NULL, the DC must be a memory DC with a bit map currently selected. Otherwise, the DC must be valid for that

device. When the format of the supplied bit map does not match that of the device, the handling routine must use the supplied BITMAPINFO or BITMAPINFO2 structure to convert it. Only standard formats and device formats that are compatible with the target device are supported.

GreSetBitmapBits - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSetCodePage

GreSetCodePage - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreSetCodePage sets the current code page for characters written with the base (default) font.

This function must be supported by the presentation driver. GreSetCodePage is called by GpiSetCP in response to an application requesting to change the currently selected code page for the device context.

```
#define INCL_GRE_DEVMISC3
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    lCodePage;   /*  New code page. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreSetCodePage. */
BOOL     rc;          /*  Return codes. */

rc = GreSetCodePage(hdc, lCodePage, pInstance,
       lFunction);
```

GreSetCodePage Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetCodePage Parameter - lCodePage

lCodePage (ULONG) - input
New code page.

GreSetCodePage Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetCodePage Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetCodePage.

GreSetCodePage Return Value - rc

rc (BOOL) - returns
Return codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CODEPAGE
PMERR_INV_HDC
PMERR_INV_IN_AREA

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetCodePage - Parameters

hdc (HDC) - input
Device context handle.

lCodePage (ULONG) - input
New code page.

pInstance (PVOID) - input

Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetCodePage.

rc (BOOL) - returns
Return codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CODEPAGE
PMERR_INV_HDC
PMERR_INV_IN_AREA

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetCodePage - Remarks

The default is the font that the system uses when the cbnd_Set attribute is hex 0000. (See Character Attributes.) When the base font is not in use, lCodePage is saved until required. When a DC is initialized, the code page is set to the default code page obtained from WinQueryProcessCp.

GreSetCodePage - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSetColorCursor

GreSetColorCursor - Syntax

Simulation support:

This function is mandatory for display drivers. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreSetColorCursor sets the bit maps that define a color cursor or pointer.

```
#define INCL_WINPOINTERS
#include <os2.h>

HDC             hdc;           /*  The device context handle. */
PPOINTERINFO    pPointerInfo;  /*  A pointer to a pointer information structure. */
PVOID           pInstance;     /*  A pointer to instance data. */
ULONG           lFunction;     /*  High-order WORD=flags; low-order WORD=NGreDeviceSetCursor. */
BOOL            fSuccess;      /*  Return codes. */

fSuccess = GreSetColorCursor(hdc, pPointerInfo,
             pInstance, lFunction);
```

GreSetColorCursor Parameter - hdc

hdc (HDC) - input
The device context handle.

GreSetColorCursor Parameter - pPointerInfo

pPointerInfo (PPOINTERINFO) - input
A pointer to a pointer information structure.

GreSetColorCursor Parameter - pInstance

pInstance (PVOID) - input
A pointer to instance data.

GreSetColorCursor Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceSetCursor.

GreSetColorCursor Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_COORDINATE
PMERR_INV_CURSOR_BITMAP
PMERR_INV_HDC

GreSetColorCursor - Parameters

hdc (HDC) - input
The device context handle.

pPointerInfo (PPOINTERINFO) - input
A pointer to a pointer information structure.

pInstance (PVOID) - input
A pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeviceSetCursor.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_COORDINATE
PMERR_INV_CURSOR_BITMAP
PMERR_INV_HDC

GreSetColorCursor - Remarks

The handling routine in the presentation driver updates its copy of the pointer definition to that identified by pPointerInfo.

GreSetColorCursor - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSetCurrentPosition

GreSetCurrentPosition - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreSetCurrentPosition sets the current (X,Y) position and resets the line type sequence.

This function must be supported by all presentation drivers. GreSetCurrentPosition is called by the function GpiSetCurrentPosition. GreSetCurrentPosition might be called in response to any of the Presentation Manager drawing functions, which end their operations by updating the current presentation space position.

```
#define INCL_GRE_LINES
#include <os2.h>

HDC         hdc;            /*  Device context handle. */
PPOINTL     pptlPosition;   /*  Pointer to new current position. */
PVOID       pInstance;      /*  Pointer to instance data. */
ULONG       lFunction;      /*  High-order WORD=flags; low-order WORD=NGreSetCurrentPosition. */
BOOL        rc;             /*  Return code. */

rc = GreSetCurrentPosition(hdc, pptlPosition,
       pInstance, lFunction);
```

GreSetCurrentPosition Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetCurrentPosition Parameter - pptlPosition

pptlPosition (PPOINTL) - input
Pointer to new current position.

When COM_TRANSFORM is set, the current position is expressed in world coordinates. Otherwise, this value is passed in device coordinates. The handling routine must transform these values as appropriate. Typically, the presentation driver maintains the current position in both coordinate sets in the DC instance data structure.

GreSetCurrentPosition Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetCurrentPosition Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetCurrentPosition.

GreSetCurrentPosition Return Value - rc

rc (BOOL) - returns
Return code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_PATH_LIMIT_EXCEEDED

GreSetCurrentPosition - Parameters

hdc (HDC) - input
Device context handle.

pptlPosition (PPOINTL) - input
Pointer to new current position.

When COM_TRANSFORM is set, the current position is expressed in world coordinates. Otherwise, this value is passed in device coordinates. The handling routine must transform these values as appropriate. Typically, the presentation driver maintains the current position in both coordinate sets in the DC instance data structure.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetCurrentPosition.

rc (BOOL) - returns
Return code.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_PATH_LIMIT_EXCEEDED

GreSetCurrentPosition - Remarks

Typically, the handling routine also sets a flag in the DC instance data structure to indicate that the first pel of the next line must be drawn. When the COM_AREA or COM_PATH flag is set, this function is part of an area or path definition. In either case, the handling routine usually passes the call back to the graphics engine for processing by the default handling routine. The call would not be passed back to the graphics engine if the presentation driver had hooked all of the area and path functions.

When the current context is "in area," a figure closure line is generated (if necessary). This can cause a correlation hit to occur on an area boundary. The current position should only be correlated on, merged into the bounds, or both correlated and merged, if it is actually used in a drawing primitive.

The following is an example:

```
            ...
GreSetCurrentPosition (hdc, p1);
            ...
            ...
            ...
GreSetCurrentPosition (hdc, p2);
            ...
            ...
            ...
GrePolyLine (hdc, to p3, n);
            ...
```

Notice that the sequence does not merge p1 into the bounds or correlate on it.

GreSetCurrentPosition - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSetGlobalViewingXform

GreSetGlobalViewingXform - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Decscription

GreSetGlobalViewingXform sets the global viewing transform matrix elements.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_XFORMS
#include <os2.h>

HDC       hdc;          /*  Device context handle. */
PXFORM    paXformData;  /*  Pointer to an array. */
LONG      flOptions;    /*  Specifies how the supplied array is used to set the matrix. */
PVOID     pInstance;    /*  Pointer to instance data. */
ULONG     lFunction;    /*  High-order WORD=flags; low-order WORD=NGreSetGlobalViewingXform. */
BOOL      fSuccess;     /*  Return codes. */

fSuccess = GreSetGlobalViewingXform(hdc, paXformData,
             flOptions, pInstance, lFunction);
```

GreSetGlobalViewingXform Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetGlobalViewingXform Parameter - paXformData

paXformData (PXFORM) - input
Pointer to an array.

Pointer to an array of six matrix elements for two-dimensional formation. These are M11, M12, M21, M22, M41, and M42.

GreSetGlobalViewingXform Parameter - flOptions

flOptions (LONG) - input
Specifies how the supplied array is used to set the matrix.

Valid values are:

SX_UNITY	Set unity transform. Ignore array values.
SX_CAT_AFTER	Concatenate after.
SX_CAT_BEFORE	Concatenate before.
SX_OVERWRITE	Overwrite.

GreSetGlobalViewingXform Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetGlobalViewingXform Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetGlobalViewingXform.

GreSetGlobalViewingXform Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MATRIX_ELEMENT
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_TRANSFORM_TYPE
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetGlobalViewingXform - Parameters

hdc (HDC) - input
Device context handle.

paXformData (PXFORM) - input
Pointer to an array.

Pointer to an array of six matrix elements for two-dimensional formation. These are M11, M12, M21, M22, M41, and M42.

flOptions (LONG) - input
Specifies how the supplied array is used to set the matrix.

Valid values are:

SX_UNITY	Set unity transform. Ignore array values.
SX_CAT_AFTER	Concatenate after.
SX_CAT_BEFORE	Concatenate before.
SX_OVERWRITE	Overwrite.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetGlobalViewingXform.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MATRIX_ELEMENT
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_TRANSFORM_TYPE
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetGlobalViewingXform - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreSetGraphicsField

GreSetGraphicsField - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description:

GreSetGraphicsField sets the boundaries of the graphics field (clip) limits in page-coordinate space. The boundaries are inclusive so that points on them are not clipped. By default, no clipping is performed.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_XFORMS
#include <os2.h>

HDC       hdc;                /*  Device context handle. */
PRECTL    prclGraphicsField;  /*  Pointer to graphics field. */
PVOID     pInstance;          /*  Pointer to instance data. */
ULONG     lFunction;          /*  High-order WORD=flags; low-order WORD=NGreSetGraphicsField. */
BOOL      fSuccess;           /*  Return codes. */

fSuccess = GreSetGraphicsField(hdc, prclGraphicsField,
             pInstance, lFunction);
```

GreSetGraphicsField Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetGraphicsField Parameter - prclGraphicsField

prclGraphicsField (PRECTL) - input
Pointer to graphics field.

RECTL structure:

xLeft	Minimum x-coordinate of graphics field
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of graphics field
yTop	Maximum y-coordinate

An error is raised when xLeft is greater than xRight, or yBottom is greater than yTop.

GreSetGraphicsField Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetGraphicsField Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetGraphicsField.

GreSetGraphicsField Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_COORDINATE
PMERR_INV_GRAPHICS_FIELD
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetGraphicsField - Parameters

hdc (HDC) - input
Device context handle.

prclGraphicsField (PRECTL) - input
Pointer to graphics field.

RECTL structure:

xLeft	Minimum x-coordinate of graphics field
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of graphics field
yTop	Maximum y-coordinate

An error is raised when xLeft is greater than xRight, or yBottom is greater than yTop.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetGraphicsField.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_COORDINATE
PMERR_INV_GRAPHICS_FIELD
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetGraphicsField - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreSetLineOrigin

GreSetLineOrigin - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreSetLineOrigin sets the current line style and current position.

This function must be supported by the presentation driver. GreSetLineOrigin is used to enable the simultaneous update of line style and position. This function can be handled by bit-map simulation.

```
#define INCL_GRE_DEVMISC3
#include <os2.h>

HDC         hdc;         /*  Device context handle. */
PPOINTL     pptlXY;      /*  Pointer to an (X,Y) coordinate pair to which the current position is returned. */
ULONG       lStyle;      /*  Style number. */
```

```
PVOID       pInstance;  /*  Pointer to instance data. */
ULONG       lFunction;  /*  High-order WORD=flags; low-order WORD=NGreSetLineOrigin.

 */
BOOL        rc;         /*  Return codes. */

rc = GreSetLineOrigin(hdc, pptlXY, lStyle,
       pInstance, lFunction);
```

GreSetLineOrigin Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetLineOrigin Parameter - pptlXY

pptlXY (PPOINTL) - input
Pointer to an (X,Y) coordinate pair to which the current position is returned.

GreSetLineOrigin Parameter - lStyle

lStyle (ULONG) - input
Style number.

GreSetLineOrigin Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetLineOrigin Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetLineOrigin.

GreSetLineOrigin Return Value - rc

rc (BOOL) - returns
Return codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetLineOrigin - Parameters

hdc (HDC) - input
Device context handle.

pptlXY (PPOINTL) - input
Pointer to an (X,Y) coordinate pair to which the current position is returned.

lStyle (ULONG) - input
Style number.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetLineOrigin.

rc (BOOL) - returns
Return codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetLineOrigin - Remarks

If COM_TRANSFORM is set, the current position is expected in world coordinates. If COM_TRANSFORM is not set, the current position is expected in screen coordinates. The new line style is stored in the DC instance data structure. Some lines and curves can be drawn either by the presentation driver or by simulations, and therefore must be able to query and set the style as required.

The high-order WORD of the style number contains the first/last pel information. The low-order BYTE indicates the position in the style mask. The next byte is the state of the style error value. Refer to GreGetLineOrigin for further information.

The following C Language example outlines a strategy by which the handling routine could use the information from a call to GreSetLineOrigin.

```
/*****************************************************************/
/*                                                               */
/*   StyleMask is a single byte.  usState is a USHORT with the   */
/*   error byte as the low-order byte and the mask position as   */
/*   the high-order byte.  The three low-order bytes of the mask */
/*   position represent the number of bits by which StyleMask has */
/*   been rotated.                                               */
/*                                                               */
/*****************************************************************/


     while (necessary--)
           {

/*****************************************************************/
/*                                                               */
/*   Do we need to draw the first pel in the line?               */
/*                                                               */
/*****************************************************************/

           if (stylemask & 0x80)
               SetPel (x, y);

/*****************************************************************/
/*                                                               */
/*   Save the current style state.                               */
/*                                                               */
/*****************************************************************/

           usStateOld=usState;
           switch (LineMajor)
               {
               case yMajor:
                    usState=usState+yRatio;
                    break;
               default:
                    usState=usState+xRatio;
                    break;
               }

            if HIBYTE (usState) != HIBYTE (usStateOld)

/*****************************************************************/
/*                                                               */
/*   If the error byte has overflowed, rotate the style ratio.   */
/*   The style mask is reset every eighth rotation.              */
/*                                                               */
/*****************************************************************/

              RotateLeftOne (StyleRatio);
              UpDateNext (x, y);
           }
```

The values for pRatio are set and queried by GreSetStyleRatio and GreGetStyleRatio respectively. Alternatively, the programmer can specify device-specific defaults for xRatio and yRatio.

GreSetLineOrigin - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSetModelXform

GreSetModelXform - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreSetModelXform sets the model transform matrix elements as specified.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_XFORMS
#include <os2.h>

HDC       hdc;          /*  Device context handle. */
PXFORM    paXformData;  /*  Pointer to an array. */
LONG      flOptions;    /*  Specifies how the supplied array is used to set the matrix. Valid values are: */
PVOID     pInstance;    /*  Pointer to instance data. */
ULONG     lFunction;    /*  High-order WORD=flags; low-order WORD=NGreSetModelXform. */
BOOL      fSuccess;     /*  Return codes. */

fSuccess = GreSetModelXform(hdc, paXformData,
             flOptions, pInstance, lFunction);
```

GreSetModelXform Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetModelXform Parameter - paXformData

paXformData (PXFORM) - input
Pointer to an array.

Pointer to an array of six matrix elements for two-dimensional formation. These are M11, M12, M21, M22, M41, and M42.

GreSetModelXform Parameter - flOptions

flOptions (LONG) - input
Specifies how the supplied array is used to set the matrix. Valid values are:

SX_UNITY	Set unity transform. Ignore array values.
SX_CAT_AFTER	Concatenate after.
SX_CAT_BEFORE	Concatenate before.
SX_OVERWRITE	Overwrite.

GreSetModelXform Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetModelXform Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetModelXform.

GreSetModelXform Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT

PMERR_INV_MATRIX_ELEMENT
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_TRANSFORM_TYPE
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetModelXform - Parameters

hdc (HDC) - input
Device context handle.

paXformData (PXFORM) - input
Pointer to an array.

Pointer to an array of six matrix elements for two-dimensional formation. These are M11, M12, M21, M22, M41, and M42.

flOptions (LONG) - input
Specifies how the supplied array is used to set the matrix. Valid values are:

SX_UNITY	Set unity transform. Ignore array values.
SX_CAT_AFTER	Concatenate after.
SX_CAT_BEFORE	Concatenate before.
SX_OVERWRITE	Overwrite.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetModelXform.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MATRIX_ELEMENT
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_TRANSFORM_TYPE
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetModelXform - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreSetPageUnits

GreSetPageUnits - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreSetPageUnits sets the page units controlling the device transform.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_XFORMS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    ulUnits;     /*  Page units. */
LONG     lWidth;      /*  Page width (w). */
LONG     lHeight;     /*  Page height (h). See previous parameter, IWidth. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreSetPageUnits. */
BOOL     rc;          /*  Return codes. */

rc = GreSetPageUnits(hdc, ulUnits, lWidth,
       lHeight, pInstance, lFunction);
```

GreSetPageUnits Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetPageUnits Parameter - ulUnits

ulUnits (ULONG) - input
Page units.

Page units, as:

PU_ARBITRARY	Isotropic, arbitrary units defined by lHeight and lWidth. The page viewport is constructed to give equal X and Y spacing on the physical device, with at least one dimension of the page completely filling the corresponding default device dimension (that is, maximized window size and paper size). The origin is at the bottom left.
PU_PELS	Pel coordinates with the origin at the bottom left.
PU_LOMETRIC	Low resolution metric. These are units of 0.1 mm with the origin at the bottom left.
PU_HIMETRIC	High resolution metric. These are units of 0.01 mm with the origin at the bottom left.
PU_LOENGLISH	Units of 0.01 inch with the origin at the bottom left.
PU_HIENGLISH	Units of 0.001 inch with the origin at the bottom left.
PU_TWIPS	Twentieths of an imperial point. These are units of 1/1440 inch with the origin at the bottom left.

Other bits are reserved, and must be preserved and returned by GetPageUnits.

GreSetPageUnits Parameter - lWidth

lWidth (LONG) - input
Page width (w).

For PU_ARBITRARY. An lWidth of 0 is set to produce equal X and Y spacing on the physical device, with both dimensions completely filling the default device dimensions. When both are passed as 0, they are set to produce equal X and Y spacing on the physical device.

lWidth and lHeight completely fill the default device dimensions so that one is equal to the corresponding default device dimension, and the other is equal to or greater than its corresponding default device dimension. These dimensions are measured in pels. For other units, a value of 0 for "w" or "h" causes the page to be set to the corresponding default dimension (that is, maximized window size and paper size) in page units or pels for isotropic units.

GreSetPageUnits Parameter - lHeight

lHeight (LONG) - input
Page height (h). See previous parameter, lWidth.

GreSetPageUnits Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetPageUnits Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetPageUnits.

GreSetPageUnits Return Value - rc

rc (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MATRIX_ELEMENT
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_RECT
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetPageUnits - Parameters

hdc (HDC) - input
Device context handle.

ulUnits (ULONG) - input
Page units.

Page units, as:

PU_ARBITRARY	Isotropic, arbitrary units defined by lHeight and lWidth. The page viewport is constructed to give equal X and Y spacing on the physical device, with at least one dimension of the page completely filling the corresponding default device dimension (that is, maximized window size and paper size). The origin is at the bottom left.
PU_PELS	Pel coordinates with the origin at the bottom left.
PU_LOMETRIC	Low resolution metric. These are units of 0.1 mm with the origin at the bottom left.
PU_HIMETRIC	High resolution metric. These are units of 0.01 mm with the origin at the bottom left.
PU_LOENGLISH	Units of 0.01 inch with the origin at the bottom left.
PU_HIENGLISH	Units of 0.001 inch with the origin at the bottom left.
PU_TWIPS	Twentieths of an imperial point. These are units of 1/1440 inch with the origin at the bottom left.

Other bits are reserved, and must be preserved and returned by GetPageUnits.

lWidth (LONG) - input
Page width (w).

For PU_ARBITRARY. An lWidth of 0 is set to produce equal X and Y spacing on the physical device, with both dimensions completely filling the default device dimensions. When both are passed as 0, they are set to produce equal X and Y spacing on the physical device.

lWidth and lHeight completely fill the default device dimensions so that one is equal to the corresponding default device dimension, and the other is equal to or greater than its corresponding default device dimension. These dimensions are measured in pels. For other units, a value of 0 for "w" or "h" causes the page to be set to the corresponding default dimension (that is, maximized window size and paper size) in page units or pels for isotropic units.

lHeight (LONG) - input
Page height (h). See previous parameter, lWidth.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetPageUnits.

rc (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MATRIX_ELEMENT
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_RECT
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetPageUnits - Remarks

This function causes the Window/Viewport Transform, Page Viewport, and Device Transform matrixes to be updated as shown below. For PU_LOMETRIC, PU_HIMETRIC, PU_TWIPS, PU_LOENGLISH, PU_HIENGLISH, and PU_PELS:

Window/Viewport Transform	Unity
Page Viewport	(0,0) (sx*lWidth-1, sy*lHeight-1)
Device Transform	As defined by the mapping from Page Window to Page Viewport

Where "sx" is the horizontal scaling required by page units for the device (or 1 for PU_PELS), and "sy" is the vertical scaling required by page units for the device (or 1 for PU_PELS).

For PU_ARBITRARY:

Window/Viewport Transform	Unity
Page Viewport	(0,0) (X2,Y2)
Device Transform	As defined by the mapping from Page Window to Page Viewport

Where "Dh" is the default device (maximized window) height in pels, "Dw" is the default device (maximized window) width in pels, and "Par" is the pel (width/height) aspect ratio. X2 and Y2 are integers, determined as follows:

```
When (lWidth/lHeight) > Par*(Dw/Dh):
      X2 = Dw-1
      Y2 = Par*Dw * (lWidth/lHeight)-1

When (lWidth/lHeight) < Par*(Dw/Dh):
      X2 = (1/Par) * Dh * (lWidth/lHeight)-1
      Y2 = Dh-1

Otherwise, when (lWidth/lHeight) = Par*(Dw/Dh):
      X2 = Dw-1
      Y2 = Dh-1
```

GreSetPageUnits - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSetPageViewport

GreSetPageViewport - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreSetPageViewport name field sets the page viewport in device coordinates so that the graphics engine can update the device transform by using the page window and page viewport coordinates.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_XFORMS
#include <os2.h>

HDC       hdc;           /*  Device context handle. */
PRECTL    prclViewport;  /*  Pointer to page viewport boundaries in device coordinates. */
ULONG     flOptions;     /*  Reserved parameter.  The only valid value is 0. */
PVOID     pInstance;     /*  Pointer to instance data. */
ULONG     lFunction;     /*  High-order WORD=flags; low-order WORD=NGreSetPageViewport. */
BOOL      fSuccess;      /*  Return codes. */

fSuccess = GreSetPageViewport(hdc, prclViewport,
             flOptions, pInstance, lFunction);
```

GreSetPageViewport Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetPageViewport Parameter - prclViewport

prclViewport (PRECTL) - input
Pointer to page viewport boundaries in device coordinates.

RECTL structure:

xLeft	Minimum x-coordinate of viewport
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of viewport
yTop	Maximum y-coordinate

GreSetPageViewport Parameter - flOptions

flOptions (ULONG) - input
Reserved parameter. The only valid value is *0* .

GreSetPageViewport Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetPageViewport Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetPageViewport.

GreSetPageViewport Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MATRIX_ELEMENT
PMERR_INV_PAGE_VIEWPORT
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_SET_VIEWPORT_OPTION
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetPageViewport - Parameters

hdc (HDC) - input
Device context handle.

prclViewport (PRECTL) - input
Pointer to page viewport boundaries in device coordinates.

RECTL structure:

xLeft	Minimum x-coordinate of viewport
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of viewport
yTop	Maximum y-coordinate

flOptions (ULONG) - input
Reserved parameter. The only valid value is *0* .

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetPageViewport.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MATRIX_ELEMENT
PMERR_INV_PAGE_VIEWPORT
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_SET_VIEWPORT_OPTION
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetPageViewport - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreSetPel

GreSetPel - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreSetPel sets a pel to the current line attribute, color, and mix.

This function must be supported by the presentation driver. GreSetPel is called by the function GpiSetPel, and is used to set the value of a pel at a specified (X,Y) coordinate within a device context. This function can be handled by bit map-simulation.

```
#define INCL_GRE_BITMAPS
#include <os2.h>

HDC         hdc;         /*  Device context handle. */
PPOINTL     pptlPel;     /*  Pointer. */
PVOID       pInstance;   /*  Pointer to instance data. */
ULONG       lFunction;   /*  High-order WORD=flags; low-order WORD=NGreSetPel. */
LONG        rc;          /*  Return codes. */

rc = GreSetPel(hdc, pptlPel, pInstance, lFunction);
```

GreSetPel Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetPel Parameter - pptlPel

pptlPel (PPOINTL) - input
Pointer.

Pointer to pel position in world or screen coordinates.

GreSetPel Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetPel Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetPel.

GreSetPel Return Value - rc

rc (LONG) - returns
Return codes.

On completion, the handling routine must return a LONG integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by the display driver when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BITMAP_NOT_SELECTED

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetPel - Parameters

hdc (HDC) - input
Device context handle.

pptlPel (PPOINTL) - input
Pointer.

Pointer to pel position in world or screen coordinates.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetPel.

rc (LONG) - returns
Return codes.

On completion, the handling routine must return a LONG integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by the display driver when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetPel - Remarks

If COM_TRANSFORM is set, the pel position is expected in world coordinates. If COM_TRANSFORM is not set, the pel position is expected in screen coordinates. This function is subject to all usual clipping. No error is returned when the point is clipped.

GreSetPel - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSetPickWindow

GreSetPickWindow - Syntax

Simulation support:

This function is mandatory for display drivers. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreSetPickWindow sets the position and size of the pick window in page-coordinate space for subsequent correlation operations.

```
#define INCL_GRE_PICK
#include <os2.h>

HDC       hdc;        /*  The device context handle. */
PRECTL    pPick;      /*  A pointer to a pick window. */
PVOID     pInstance;  /*  A pointer to instance data. */
ULONG     lFunction;  /*  High-order WORD=flags; low-order WORD=NGreSetPickWindow. */
BOOL      fSuccess;   /*  Return codes. */

fSuccess = GreSetPickWindow(hdc, pPick, pInstance,
             lFunction);
```

GreSetPickWindow Parameter - hdc

hdc (HDC) - input
The device context handle.

GreSetPickWindow Parameter - pPick

pPick (PRECTL) - input
A pointer to a pick window.

The pick window is defined as a RECTL structure in page coordinate space:

xLeft	Minimum X-coordinate of window
yBottom	Minimum Y-coordinate
xRight	Maximum X-coordinate of window
yTop	Maximum Y-coordinate

GreSetPickWindow Parameter - pInstance

pInstance (PVOID) - input
A pointer to instance data.

GreSetPickWindow Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetPickWindow.

GreSetPickWindow Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN.

GreSetPickWindow - Parameters

hdc (HDC) - input
The device context handle.

pPick (PRECTL) - input
A pointer to a pick window.

The pick window is defined as a RECTL structure in page coordinate space:

xLeft	Minimum X-coordinate of window
yBottom	Minimum Y-coordinate
xRight	Maximum X-coordinate of window
yTop	Maximum Y-coordinate

pInstance (PVOID) - input
A pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetPickWindow.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_PICK_APERTURE_POSN.

GreSetPickWindow - Remarks

The boundary of the pick window is included in the correlated area.

GreSetPickWindow - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSetRectRegion

GreSetRectRegion - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreSetRectRegion sets the specified region to the region definition given by an array of rectangles, unless the region is in use as a clipping region (in which case, an error is raised). When no rectangles are specified, that is, cRect is 0, GreSetRectRegion produces an empty region.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_REGIONS
#include <os2.h>

HDC       hdc;         /*  Device context handle. */
HRGN      hrgn;        /*  Region handle. */
PRECTL    parcRgn;     /*  Pointer to region definition. */
LONG      cRect;       /*  Count of rectangles in the region definition. */
PVOID     pInstance;   /*  Pointer to instance data. */
ULONG     lFunction;   /*  High-order WORD=flags; low-order WORD=NGreSetRectRegion. */
BOOL      fSuccess;    /*  Return codes. */

fSuccess = GreSetRectRegion(hdc, hrgn, parcRgn,
             cRect, pInstance, lFunction);
```

GreSetRectRegion Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetRectRegion Parameter - hrgn

hrgn (HRGN) - input
Region handle.

GreSetRectRegion Parameter - parcRgn

parcRgn (PRECTL) - input
Pointer to region definition.

The region is defined as an array of rectangles. Each rectangle in the array is defined by a RECTL structure:

xLeft	Minimum X-coordinate of rectangle
yBottom	Minimum Y-coordinate
xRight	Maximum X-coordinate of rectangle

yTop	Maximum Y-coordinate

The region is defined by the OR of all the rectangles. For each rectangle, xRight must be equal to or greater than xLeft. yTop must be equal to or greater than yBottom. The bottom and left boundaries of each rectangle are part of the interior of the region, the top and right boundaries are not. If COM_TRANSFORM is not set, the function expects rectangles to be in device coordinates.

GreSetRectRegion Parameter - cRect

cRect (LONG) - input
Count of rectangles in the region definition.

GreSetRectRegion Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetRectRegion Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetRectRegion.

GreSetRectRegion Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORDINATE
PMERR_INV_HRGN
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetRectRegion - Parameters

hdc (HDC) - input
Device context handle.

hrgn (HRGN) - input
Region handle.

parcRgn (PRECTL) - input
Pointer to region definition.

The region is defined as an array of rectangles. Each rectangle in the array is defined by a RECTL structure:

xLeft	Minimum X-coordinate of rectangle
yBottom	Minimum Y-coordinate
xRight	Maximum X-coordinate of rectangle
yTop	Maximum Y-coordinate

The region is defined by the OR of all the rectangles. For each rectangle, xRight must be equal to or greater than xLeft. yTop must be equal to or greater than yBottom. The bottom and left boundaries of each rectangle are part of the interior of the region, the top and right boundaries are not. If COM_TRANSFORM is not set, the function expects rectangles to be in device coordinates.

cRect (LONG) - input
Count of rectangles in the region definition.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetRectRegion.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORDINATE
PMERR_INV_HRGN
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetRectRegion - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreSetScreenBits

GreSetScreenBits - Syntax

Simulation support:

This function is mandatory for display drivers that want to be DCAF-enabled.

Description *GreSetScreenBits* takes compressed data, generated by a previous call to GreGetScreenBits, from a buffer and decompresses it into the currently selected memory bit map. The call is only valid for a memory DC that has a bit map selected that is the same size as the screen of the machine on which the GreGetScreenBits call was performed. There is no clipping; if a rectangle exceeds the bit-map dimensions, the function will terminate immediately with an error logged. The bit map may be left in a partially drawn state as prior rectangles may have been copied into it.

This is a drawing primitive; therefore, correlation, boundary accumulation and drawing could take place. Function bits are ignored, however, and only drawing will be done.

The function may be passed a region handle, in which case the area defined by the set bits will be added to the region.

The DCAF-enabled driver may be passed 4bpp planar, 4bpp packed, 8bpp packed and 16bpp packed data.

```
#define INCL_GRE_SCREEN
#include <os2.h>
#include <pmddi.h>

HDC      hdc;      /*  Any valid, direct (screen) device context handle. */
PBYTE    pBuffer;  /*  Pointer to source (compressed) data. */
ULONG    cBytes;   /*  Length of the memory buffer pointed to by pBuffer. */
HRGN     hrgn;     /*  A valid region handle. */
PDC      pdcArg;   /*  Pointer to instance data.  */
ULONG    FunN;     /*  High-order word=Flags; Low-order word=NGreSetScreenBits. */
LONG     rc;

rc = GreSetScreenBits(hdc, pBuffer, cBytes,
       hrgn, pdcArg, FunN);
```

GreSetScreenBits Parameter - hdc

hdc (HDC) - input
Any valid, direct (screen) device context handle.

The valid handle must have selected into it a bit map the same size as the screen from which the source data was read.

GreSetScreenBits Parameter - pBuffer

pBuffer (PBYTE) - input
Pointer to source (compressed) data.

Pointer to the memory buffer where the source data is located.

GreSetScreenBits Parameter - cBytes

cBytes (ULONG) - input
Length of the memory buffer pointed to by *pBuffer* .

This must be the same value as that returned by the corresponding GreGetScreenBits call.

GreSetScreenBits Parameter - hrgn

hrgn (HRGN) - input
A valid region handle.

The area that is updated in the memory bit map by this call is added to the region identified by this handle.

GreSetScreenBits Parameter - pdcArg

pdcArg (PDC) - input
Pointer to instance data.

GreSetScreenBits Parameter - FunN

FunN (ULONG) - input
High-order word=Flags; Low-order word=NGreSetScreenBits.

GreSetScreenBits Return Value - rc

rc (LONG) - returns
This function returns one of the following:

1	The supplied data is successfully decompressed in the memory bit map.
0	An error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BITMAP_NOT_SELECTED
PMERR_INCOMPAT_COLOR_FORMAT
PMERR_INV_DC_TYPE
PMERR_INV_IMAGE_DIMENSION
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH

PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetScreenBits - Parameters

hdc (HDC) - input
Any valid, direct (screen) device context handle.

The valid handle must have selected into it a bit map the same size as the screen from which the source data was read.

pBuffer (PBYTE) - input
Pointer to source (compressed) data.

Pointer to the memory buffer where the source data is located.

cBytes (ULONG) - input
Length of the memory buffer pointed to by *pBuffer* .

This must be the same value as that returned by the corresponding GreGetScreenBits call.

hrgn (HRGN) - input
A valid region handle.

The area that is updated in the memory bit map by this call is added to the region identified by this handle.

pdcArg (PDC) - input
Pointer to instance data.

FunN (ULONG) - input
High-order word=Flags; Low-order word=NGreSetScreenBits.

rc (LONG) - returns
This function returns one of the following:

1	The supplied data is successfully decompressed in the memory bit map.
0	An error occurred.

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BITMAP_NOT_SELECTED
PMERR_INCOMPAT_COLOR_FORMAT
PMERR_INV_DC_TYPE
PMERR_INV_IMAGE_DIMENSION
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetScreenBits - Remarks

It is permissible to implement this function by returning 0 to indicate that the bits were not saved and, therefore, must be saved by the calling routine.

GreSetScreenBits calls EnterDriver and performs some initial error tests, checking the *FunN* field and the DC type, as follows:

- If the COM_PATH or the COM_AREA flags are set in *FunN,* the function logs PMERR_INV_IN_PATH or PMERR_INV_IN_AREA, respectively, and returns 0 (zero).
- If the DC is not a direct DC, the PMERR_INV_DC_TYPE error is logged and the call returns 0 (zero).
- If a bit map is not selected in the DC, the PMERR_BITMAP_NOT_SELECTED error is logged and the call returns 0 (zero).

Next, the data length is checked to see if it is at least the size of the header and that the passed data length matches the data length given in the packet header. If the data lengths do not match, the PMERR_INV_LENGTH_OR_COUNT error is logged and the call returns 0 (zero).

GreSetScreenBits also checks that the coupling local format and received data format are valid. This is done by creating a word containing the internal (screen) format and the external (data received) format. The AGSBValidDataFormats table is scanned for a match. This table is an array of valid source and destination format pairs, as defined in the VALID_DATA_FORMATS data structure. See VALID_DATA_FORMATS in Data Types for details on this data structure.

If a match is found, the format is valid. Otherwise, the format is invalid, the PMERR_INCOMPAT_COLOR_FORMAT error is logged and the call returns 0. If conversion is required for the pair, the pointer to the compression function for the required format combination is retrieved from the aGSBValidDataFormats structure, and the corresponding routine is called to create the conversion table. The creation calls for the conversion table of valid format pairs; all of them are defined in DCAFCNVT.C:

- CreateConvertTable_4ext_4int creates a 256-entry table of bytes to convert 4bpp external (default VGA) format to 4bpp internal (fudged).

  Pels are converted a pair at a time, which is why the conversion table is 256 entries, rather than the expected 16. To do this, after the memory allocation, the NearestRestrictedColourIndex (in CONVINT.C) is called, in order to map the entries in the VGA default table (see the appendix regarding default color palettes in the *IBM OS/2 Display Device Driver Reference*) to the nearest entry in the Reduced16DeviceDefaultPalette table (the 16-entry XGA default palette defined in EDDDATA.C).

  The remaining values in the internal table are obtained by combining pairs of conversion values, in the following way:

```
for (i = 0; i < 256; i++)
{
     pConvertTable[i] = (abLocalConvertTable[i >> 4] << 4) |
                        (abLocalConvertTable[i & 0x0F]) ; }
```

- CreateConvertTable_4ext_8int creates a 16-entry table of bytes to convert 4bpp to 8bpp.

  After the memory allocation, the NearestRestrictedColourIndex (in CONVINT.C) is called, in order to map the entries in the standard VGA table (see the appendix regarding default color palettes in the *IBM OS/2 Display Device Driver Reference*) to the nearest entry in the FullSizeDeviceDefaultPalette (the 256-entry XGA default palette defined in EDDDATA.C).

- CreateConvertTable_4ext_16int creates a 16-entry table of words to convert 4bpp to 16bpp.

  After the memory allocation, the RGB16FromPRGB2 (macro in CONVFUNS.H) is used to transform the VGA standard palette to 16 bpp values. Note that the VGA standard palette is in RGB2 format (see the appendix regarding default color palettes in the *IBM OS/2 Display Device Driver Reference* for details of these formats).

  The entries are returned in Intel format, as follows:

```
Bit7..Bit 0   Bit15..Bit8
  BYTE 0        BYTE 1
```

  and then must be converted to Motorola format, as follows:

```
Bit15...Bit8   Bit7..Bit0
   BYTE 0        BYTE 1
```

- CreateConvertTable_8ext_16int creates a 256-entry table of words to convert 8bpp external to 16bpp internal.

  After the memory allocation, the RGB16FromPRGB2 (macro in CONVFUNS.H) is used in order to transform the XGA device standard palette FullSizeDeviceDefaultPalette to 16 bpp values. The entries are returned in Intel format and must then be converted to Motorola format.

Next, WaitForRealHWFunction in HWACCESS.ASM is called to ensure that the hardware is ready before using the PHUNK buffers.

At this point, the main loop starts:

```
: for next byte in the input buffer < last buffer in the input buffer
: : check that the next rectangle in the buffer is valid; otherwise, log PMERR_INV_RECT and return 0.
: : calculate some data on the current rectangle (number of row, bytes per row, etc)
: : if source data is with 8-bit data fields
: : // expand the rectangle and return the pointer to the next rectangle in the input buffer
: : : call ExpandRect8
: : otherwise // source data is with 16-bit data fields
: : // expand the rectangle and return the pointer to the next rectangle in the input buffer
: : : call ExpandRect16
: : endif
: : subtract the just decompressed rectangle from the passed region using
    GreCombineRectRegion (CRGN_DIFF)
: endfor
return 1
```

At the end, ExitDriver is called and a return code of 1 is issued.

The ExpandRect8 and ExpandRect16 calls (in EXPAND.ASM) perform the decompression of the data and, if the internal display driver format is different from the data format, the conversion of the data takes place. The following routines (in DCAFCNV.ASM and EXPAND.ASM) are used:

```
convert_row_4pl_4pk           (scr 4bpp planar, dst  4bpp packed)
                                changes only the planes to linear

convert_row_4pl_4pkint        (src 4bpp planar, dst  4bpp packed)
                                calling: convert_row_4pl_4pk;
                                         convert_row_4pk_16pk

convert_row_4pl_8pk           (scr 4bpp planar, dst  8bpp packed)
                                calling: convert_row_4pl_4pk;
                                         convert_row_4pkt_8pk

convert_row_4pl_16pk          (scr 4bpp planar, dst 16bpp packed)
                                 calling: convert_row_4pl_4pk;
                                          convert_row_4pkext_4pkint

convert_row_4pkext_4pkint     (scr 4bpp packed, dst  4bpp packed)
                                 using ConvertTable_4ext_4int

convert_row_4pk_8pk           (scr 4bpp packed, dst  8bpp packed)
                                 using ConvertTable_4ext_8int

convert_row_4pk_16pk          (scr 4bpp packed, dst 16bpp packed)
                                 using ConvertTable_4ext_16int

convert_row_8pk_16pk          (scr 8bpp packed, dst 16bpp packed)
                                 using ConvertTable_8ext_16int
```

Conversion between different data formats (for example, 4bpp and 8bpp) is done independently from the data expansion. That is, if the source and destination formats differ, the source data is first expanded (in a intermediate buffer), and then converted to the destination format. If the source and destination formats match, the data is decompressed directly from the source buffer to the destination internal bitmap. As in GreGetScreenBits, the data conversion routines use tables wherever possible to improve performance (see Converting Data in Code Modifications Required in Base Driver for DCAF for more information).

GreSetScreenBits - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSetStyleRatio

GreSetStyleRatio - Syntax

Simulation support:

This function is mandatory for display drivers. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreSetStyleRatio sets the style ratio used by the presentation driver's line-drawing algorithm to determine which pels are turned ON to draw a sloping line.

```
#define INCL_GRE_DEVMISC2
#include <os2.h>

HDC      hdc;         /*  The device context handle. */
PBYTE    pRatio;
PVOID    pInstance;   /*  A pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreSetStyleRatio. */
BOOL     fSuccess;    /*  Return codes. */

fSuccess = GreSetStyleRatio(hdc, pRatio, pInstance,
             lFunction);
```

GreSetStyleRatio Parameter - hdc

hdc (HDC) - input
The device context handle.

GreSetStyleRatio Parameter - pRatio

pRatio (PBYTE) - input
Pointer to two unsigned bytes corresponding to the aspect of the pels on which a line is drawn. The style ratio is defined as a two-byte value. The low-order byte indicates a step in the X-direction, and the high-order byte a step in the Y-direction. Typical values for style ratios are:

- For EGA devices, X-direction equals 64 and Y-direction equals 85.
- For one-to-one devices, X-direction equals 64 and Y-direction equals 64. In this case, the style ratio steps are set to 64:64 rather than 1:1 to ensure that a single dot in a line-style pattern is a sensible length. The length of a single dot in the pattern is (256/step_value) pels.

GreSetStyleRatio Parameter - pInstance

pInstance (PVOID) - input
A pointer to instance data.

GreSetStyleRatio Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetStyleRatio.

GreSetStyleRatio Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

GreSetStyleRatio - Parameters

hdc (HDC) - input
The device context handle.

pRatio (PBYTE) - input
Pointer to two unsigned bytes corresponding to the aspect of the pels on which a line is drawn. The style ratio is defined as a two-byte value. The low-order byte indicates a step in the X-direction, and the high-order byte a step in the Y-direction. Typical values for style ratios are:

- For EGA devices, X-direction equals 64 and Y-direction equals 85.
- For one-to-one devices, X-direction equals 64 and Y-direction equals 64. In this case, the style ratio steps are set to 64:64 rather than 1:1 to ensure that a single dot in a line-style pattern is a sensible length. The length of a single dot in the pattern is (256/step_value) pels.

pInstance (PVOID) - input
A pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetStyleRatio.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC

GreSetStyleRatio - Remarks

Display drivers must support this function so that a hardcopy driver (whose device can have a different style ratio) can use the display driver to draw into a bit map that the hardcopy driver can use.

GreSetStyleRatio - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSetupDC

GreSetupDC - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreSetupDC initializes the device context to the region determined by flOptions.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_CLIP
#include <os2.h>

HDC       hdc;          /*  Device context handle. */
HRGN      hrgnVis;      /*  Visible region handle. */
LONG      xOrg;         /*  X-coordinate of DC origin, specified in screen coordinates. */
LONG      yOrg;         /*  Y-coordinate of DC origin, specified in screen coordinates. */
PRECTL    prclBounds;   /*  Bounding rectangle in device coordinates. */
ULONG     flOptions;    /*  Option flags. */
PVOID     pInstance;    /*  Pointer to instance data. */
ULONG     lFunction;    /*  High-order WORD=flags; low-order WORD=NGreSetupDC. */
BOOL      fSuccess;     /*  Return codes. */

fSuccess = GreSetupDC(hdc, hrgnVis, xOrg,
             yOrg, prclBounds, flOptions,
             pInstance, lFunction);
```

GreSetupDC Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetupDC Parameter - hrgnVis

hrgnVis (HRGN) - input
Visible region handle.

GreSetupDC Parameter - xOrg

xOrg (LONG) - input
X-coordinate of DC origin, specified in screen coordinates.

GreSetupDC Parameter - yOrg

yOrg (LONG) - input
Y-coordinate of DC origin, specified in screen coordinates.

GreSetupDC Parameter - prclBounds

prclBounds (PRECTL) - input
Bounding rectangle in device coordinates.

GreSetupDC Parameter - flOptions

flOptions (ULONG) - input
Option flags.

These flags determine the region returned in prclBounds:

SETUPDC_VISRGN	Replace the contents of the visible region of hdc with the contents of hrgnVis.
SETUPDC_ORIGIN	Set the DC origin to (xOrg, yOrg).
SETUPDC_ACCUMBOUNDSON	Turn on bounds accumulation. This only affects the COM_ALT_BOUND flag. If COM_ALT_BOUND (see page Calling Conventions) is not set, the bounds rectangle is reset to an empty rectangle. If COM_ALT_BOUND is already set, the bounds rectangle is not changed.
SETUPDC_ACCUMBOUNDSOFF	Turn off bounds accumulation.
SETUPDC_REALCLIP	Recalculate the true device clipping region. This bit is normally set, but can be 0 when immediate recalculation is not required.
SETUPDC_SETOWNER	When this bit is set, the DC must belong to the current process.
SETUPDC_CLEANDC	When this bit is set, the simulation marks the visible regions as valid and calls GreNotifyClipChange in the presentation driver.

GreSetupDC Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetupDC Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetupDC.

GreSetupDC Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE

PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetupDC - Parameters

hdc (HDC) - input
Device context handle.

hrgnVis (HRGN) - input
Visible region handle.

xOrg (LONG) - input
X-coordinate of DC origin, specified in screen coordinates.

yOrg (LONG) - input
Y-coordinate of DC origin, specified in screen coordinates.

prclBounds (PRECTL) - input
Bounding rectangle in device coordinates.

flOptions (ULONG) - input
Option flags.

These flags determine the region returned in prclBounds:

SETUPDC_VISRGN	Replace the contents of the visible region of hdc with the contents of hrgnVis.
SETUPDC_ORIGIN	Set the DC origin to (xOrg, yOrg).
SETUPDC_ACCUMBOUNDSON	Turn on bounds accumulation. This only affects the COM_ALT_BOUND flag. If COM_ALT_BOUND (see page Calling Conventions) is not set, the bounds rectangle is reset to an empty rectangle. If COM_ALT_BOUND is already set, the bounds rectangle is not changed.
SETUPDC_ACCUMBOUNDSOFF	Turn off bounds accumulation.
SETUPDC_REALCLIP	Recalculate the true device clipping region. This bit is normally set, but can be 0 when immediate recalculation is not required.
SETUPDC_SETOWNER	When this bit is set, the DC must belong to the current process.
SETUPDC_CLEANDC	When this bit is set, the simulation marks the visible regions as valid and calls GreNotifyClipChange in the presentation driver.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetupDC.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetupDC - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreSetViewingLimits

GreSetViewingLimits - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

*GreSetViewingLimit*s sets the boundaries of the viewing (clip) limits in model space to the specified values.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_XFORMS
#include <os2.h>

HDC       hdc;                /*  Device context handle. */
PRECTL    prclViewingLimits;  /*  Pointer to limits of viewing area. */
PVOID     pInstance;          /*  Pointer to instance data. */
ULONG     lFunction;          /*  High-order WORD=flags; low-order WORD=NGreSetViewingLimits.

 */
BOOL      fSuccess;           /*  Return codes. */

fSuccess = GreSetViewingLimits(hdc, prclViewingLimits,
             pInstance, lFunction);
```

GreSetViewingLimits Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetViewingLimits Parameter - prclViewingLimits

prclViewingLimits (PRECTL) - input
Pointer to limits of viewing area.

RECTL structure:

xLeft	Minimum X-coordinate of viewing limits
yBottom	Minimum Y-coordinate
xRight	Maximum X-coordinate of viewing limits
yTop	Maximum Y-coordinate

GreSetViewingLimits Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetViewingLimits Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetViewingLimits.

GreSetViewingLimits Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_COORDINATE
PMERR_INV_GRAPHICS_FIELD
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetViewingLimits - Parameters

hdc (HDC) - input
Device context handle.

prclViewingLimits (PRECTL) - input
Pointer to limits of viewing area.

RECTL structure:

xLeft	Minimum X-coordinate of viewing limits
yBottom	Minimum Y-coordinate
xRight	Maximum X-coordinate of viewing limits
yTop	Maximum Y-coordinate

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetViewingLimits.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_HDC_BUSY
PMERR_INV_COORDINATE
PMERR_INV_GRAPHICS_FIELD
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetViewingLimits - Remarks

This function sets the boundaries of the viewing (clip) limits in model space to the specified values. The boundaries are inclusive and are not clipped. The viewing-limit coordinates are transformed to make a clipping rectangle in page-coordinate or device-coordinate space. Any rotation or shear of this rectangle is ignored. When the left boundary is greater than the right, or the bottom boundary is greater than the top, a NULL rectangle is defined and all points are clipped.

GreSetViewingLimits - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSetWindowViewportXform

GreSetWindowViewportXform - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreSetWindowViewportXform sets the window or viewport transform matrix elements as specified.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_XFORMS
#include <os2.h>

HDC       hdc;          /*  Device context handle. */
PXFORM    paXformData;  /*  Pointer to an array. */
LONG      flOptions;    /*  Specifies how the supplied array should be used to set the matrix. */
PVOID     pInstance;    /*  Pointer to instance data. */
ULONG     lFunction;    /*  High-order WORD=flags; low-order WORD=NGreSetWindowViewportXform. */
BOOL      fSuccess;     /*  Return codes. */

fSuccess = GreSetWindowViewportXform(hdc,
             paXformData, flOptions, pInstance,
             lFunction);
```

GreSetWindowViewportXform Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetWindowViewportXform Parameter - paXformData

paXformData (PXFORM) - input
Pointer to an array.

Pointer to an array of six matrix elements for two-dimensional formation. These are M11, M12, M21, M22, M41, and M42.

GreSetWindowViewportXform Parameter - flOptions

flOptions (LONG) - input
Specifies how the supplied array should be used to set the matrix.

Valid values are:

SX_UNITY	Set unity transform. Ignore array values.
SX_CAT_AFTER	Concatenate after.
SX_CAT_BEFORE	Concatenate before.
SX_OVERWRITE	Overwrite.

GreSetWindowViewportXform Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetWindowViewportXform Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetWindowViewportXform.

GreSetWindowViewportXform Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY

PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MATRIX_ELEMENT
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_TRANSFORM_TYPE
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetWindowViewportXform - Parameters

hdc (HDC) - input
Device context handle.

paXformData (PXFORM) - input
Pointer to an array.

Pointer to an array of six matrix elements for two-dimensional formation. These are M11, M12, M21, M22, M41, and M42.

flOptions (LONG) - input
Specifies how the supplied array should be used to set the matrix.

Valid values are:

SX_UNITY	Set unity transform. Ignore array values.
SX_CAT_AFTER	Concatenate after.
SX_CAT_BEFORE	Concatenate before.
SX_OVERWRITE	Overwrite.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetWindowViewportXform.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MATRIX_ELEMENT
PMERR_INV_PATTERN_REF_PT_ATTR

PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_TRANSFORM_TYPE
PMERR_PATH_LIMIT_EXCEEDED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetWindowViewportXform - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreSetXformRect

GreSetXformRect - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreSetXformRect intersects a rectangle in device coordinates with the DC region. The rectangle is inclusive at the bottom and left boundaries, exclusive at the top and right boundaries.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_CLIP
#include <os2.h>

HDC       hdc;         /*  Device context handle. */
PRECTL    prclRect;    /*  Pointer to a rectangle in device coordinates. */
PVOID     pInstance;   /*  Pointer to instance data. */
ULONG     lFunction;   /*  High-order WORD=flags; low-order WORD=NGreSetXformRect. */
LONG      rc;          /*  Return codes. */

rc = GreSetXformRect(hdc, prclRect, pInstance,
       lFunction);
```

GreSetXformRect Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetXformRect Parameter - prclRect

prclRect (PRECTL) - input
Pointer to a rectangle in device coordinates.

RECTL structure:

xLeft	Minimum x-coordinate of rectangle
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of rectangle
yTop	Maximum y-coordinate

GreSetXformRect Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetXformRect Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetXformRect.

GreSetXformRect Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (lComplexity) indicating the complexity of the DC region:

RGN_ERROR	Error
RGN_NULL	Null region
RGN_RECT	Rectangular region
RGN_COMPLEX	Complex region (more than 1 rectangle)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetXformRect - Parameters

hdc (HDC) - input
Device context handle.

prclRect (PRECTL) - input
Pointer to a rectangle in device coordinates.

RECTL structure:

xLeft	Minimum x-coordinate of rectangle
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of rectangle
yTop	Maximum y-coordinate

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetXformRect.

rc (LONG) - returns
Return codes.

This function returns an integer (lComplexity) indicating the complexity of the DC region:

RGN_ERROR	Error
RGN_NULL	Null region
RGN_RECT	Rectangular region
RGN_COMPLEX	Complex region (more than 1 rectangle)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COORDINATE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreSetXformRect - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreStrokePath

GreStrokePath - Syntax

Simulation support:

This function is simulated by a handling routine in the graphics engine.

Description

GreStrokePath converts a path to the envelope of a wide line.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_PATHS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
LONG     idPath;      /*  Path ID. The only valid value is 1. */
ULONG    flOptions;   /*  Reserved. Must be 0. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreStrokePath. */
LONG     rc;          /*  Return codes. */

rc = GreStrokePath(hdc, idPath, flOptions,
       pInstance, lFunction);
```

GreStrokePath Parameter - hdc

hdc (HDC) - input
Device context handle.

GreStrokePath Parameter - idPath

idPath (LONG) - input
Path ID. The only valid value is 1.

GreStrokePath Parameter - flOptions

flOptions (ULONG) - input
Reserved. Must be 0.

GreStrokePath Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreStrokePath Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreStrokePath.

GreStrokePath Return Value - rc

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_FILL_PATH_OPTIONS
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MATRIX_ELEMENT
PMERR_INV_MODIFY_PATH_MODE
PMERR_INV_PATH_ID
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_TRANSFORM_TYPE
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreStrokePath - Parameters

hdc (HDC) - input
Device context handle.

idPath (LONG) - input
Path ID. The only valid value is 1.

flOptions (ULONG) - input
Reserved. Must be 0.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreStrokePath.

rc (LONG) - returns
Return codes.

This function returns an integer (cHits) indicating, where appropriate, whether correlation hits were detected:

GPI_OK	Successful
GPI_HITS	Successful with correlate hit (returned by display drivers when the correlate flag is ON, and a hit is detected)
GPI_ERROR	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_COLOR_DATA
PMERR_INV_COLOR_INDEX
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_FILL_PATH_OPTIONS
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_MATRIX_ELEMENT
PMERR_INV_MODIFY_PATH_MODE
PMERR_INV_PATH_ID
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PICK_APERTURE_POSN
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_TRANSFORM_TYPE
PMERR_PATH_LIMIT_EXCEEDED
PMERR_PATH_UNKNOWN
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreStrokePath - Remarks

This function converts a path to the envelope of a wide line by using the current geometric line attribute (see Line Attributes). The converted path is filled by using winding mode and the current area attributes (see Area (Pattern) Attributes), and is then drawn. When the path has been drawn, it is deleted. Note that GreStrokePath is equivalent to GreModifyPath followed by GreFillPath. It is provided to allow the presentation driver to optimize its storage.

Note: GreModifyPath and GreStrokePath are the only functions that can construct geometric wide lines.

GreStrokePath - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreUnlockDevice

GreUnlockDevice - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreUnlockDevice allows all pending screen input or output operations blocked by GreLockDevice to continue.

This function must be supported by all presentation drivers. For hardcopy devices, the hardcopy driver need do nothing except return TRUE (Successful). GreUnlockDevice is used to enable another process to access a resource (device context) that had been previously locked to prevent simultaneous update.

```
#define INCL_GRE_DEVMISC3
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreUnlockDevice. */
BOOL     rc;          /*  Return codes. */

rc = GreUnlockDevice(hdc, pInstance, lFunction);
```

GreUnlockDevice Parameter - hdc

hdc (HDC) - input
Device context handle.

GreUnlockDevice Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreUnlockDevice Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreUnlockDevice.

GreUnlockDevice Return Value - rc

rc (BOOL) - returns
Return codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. An error code for conditions that the handling routine is expected to check is:

PMERR_DEV_FUNC_NOT_INSTALLED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreUnlockDevice - Parameters

hdc (HDC) - input
Device context handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreUnlockDevice.

rc (BOOL) - returns
Return codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. An error code for conditions that the handling routine is expected to check is:

PMERR_DEV_FUNC_NOT_INSTALLED

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreUnlockDevice - Remarks

This function is used to synchronize the use and update of the visible region.

GreUnlockDevice - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreUnrealizeColorTable

GreUnrealizeColorTable - Syntax

Simulation support:

None. This function is mandatory for all drivers.

Description:

GreUnrealizeColorTable reverses GreRealizeColorTable by causing the default physical color table for the device to be reinstated.

This function must be supported by the presentation driver. GreUnrealizeColorTable is called by GpiUnRealizeColorTable in response to the request of an application to restore the application's logical color table prior to the last call to GpiRealizeColorTable. This function can be handled by bit-map simulation.

```
#define INCL_GRE_COLORTABLE
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreUnrealizeColorTable. */
BOOL     rc;          /*  Return codes. */

rc = GreUnrealizeColorTable(hdc, pInstance,
       lFunction);
```

GreUnrealizeColorTable Parameter - hdc

hdc (HDC) - input
Device context handle.

GreUnrealizeColorTable Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreUnrealizeColorTable Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreUnrealizeColorTable.

GreUnrealizeColorTable Return Value - rc

rc (BOOL) - returns
Return codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COL_TABLE_NOT_REALIZED
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreUnrealizeColorTable - Parameters

hdc (HDC) - input
Device context handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreUnrealizeColorTable.

rc (BOOL) - returns
Return codes.

On completion, the handling routine must return a BOOLEAN value to indicate success or an error.

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_COL_TABLE_NOT_REALIZED
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreUnrealizeColorTable - Remarks

The logical color table is unaffected by this function.

Note: GreUnrealizeColorTable is provided for the support of older applications and is of less importance for new applications.

GreUnrealizeColorTable - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreUpdateColors

GreUpdateColors - Syntax

Simulation support:

This function is simulated by a handling routine in Revision 2.0 (or later) of the graphics engine.

Description

GreUpdateColors causes a pel-by-pel remapping of the Device Context's visible area on the screen.

This function can be hooked by the presentation driver.

```
#define INCL_GRE_PALETTE
```

```
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreUpdateColor. */
BOOL     rc;          /*  Return codes. */

rc = GreUpdateColors(hdc, pInstance, lFunction);
```

GreUpdateColors Parameter - hdc

hdc (HDC) - input
Device context handle.

GreUpdateColors Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreUpdateColors Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreUpdateColor.

GreUpdateColors Return Value - rc

rc (BOOL) - returns
Return codes.

On completion, this function returns the following values:

Error	(0)
Successful	(1)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_IN_AREA
PMERR_INV_IN_PATH

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreUpdateColors - Parameters

hdc (HDC) - input
Device context handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreUpdateColor.

rc (BOOL) - returns
Return codes.

On completion, this function returns the following values:

Error	(0)
Successful	(1)

Possible Errors Detected: When an error is detected, the handling routine must call WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_INV_IN_AREA
PMERR_INV_IN_PATH

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreUpdateColors - Remarks

This function causes a pel-by-pel remapping of the hardware palette indexes for the DC's visible area on the screen. This function call indicates that the window colors might have changed to incorrect colors if the window is in the background. By calling GreUpdateColors, the pels are changed to the color in the current physical palette that comes closest to the desired color.

GreUpdateColors - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreUpdateCursor

GreUpdateCursor - Syntax

Simulation support:

This function is mandatory for display drivers. For other drivers, it is simulated by a handling routine in the graphics engine.

Description:

GreUpdateCursor updates the drawn alphanumeric cursor to match the cursor state information contained in the presentation space.

```
#define INCL_AVIOP
#include <os2.h>

HDC                     hdc;         /*  The device context handle. */
VioPresentationSpace   *pVioPS;      /*  A pointer to the Vio presentation space. */
PVOID                   pInstance;   /*  A pointer to instance data. */
ULONG                   lFunction;   /*  High-order WORD=flags; low-order WORD=NGreUpdateCursor. */
LONG                    rc;          /*  Return codes. */

rc = GreUpdateCursor(hdc, pVioPS, pInstance,
       lFunction);
```

GreUpdateCursor Parameter - hdc

hdc (HDC) - input
The device context handle.

GreUpdateCursor Parameter - pVioPS

pVioPS (VioPresentationSpace *) - input
A pointer to the Vio presentation space.

GreUpdateCursor Parameter - pInstance

pInstance (PVOID) - input
A pointer to instance data.

GreUpdateCursor Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreUpdateCursor.

GreUpdateCursor Return Value - rc

rc (LONG) - returns
Return codes.

This function returns a LONG value as an error indicator:

NO_ERROR	Successful
CE_INVALID_PRESENTATION_SPACE	Error. For example, this message might be returned as a result of an invalid CellByteSize.

GreUpdateCursor - Parameters

hdc (HDC) - input
The device context handle.

pVioPS (VioPresentationSpace *) - input
A pointer to the Vio presentation space.

pInstance (PVOID) - input
A pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreUpdateCursor.

rc (LONG) - returns
Return codes.

This function returns a LONG value as an error indicator:

NO_ERROR	Successful
CE_INVALID_PRESENTATION_SPACE	Error. For example, this message might be returned as a result of an invalid CellByteSize.

GreUpdateCursor - Remarks

This function usually involves removing the previous cursor from the window and drawing the new cursor, if visible, according to the presentation space information. The new cursor (if visible) is positioned and clipped according to this information and the window's cell-buffer origin and size.

The cursor is drawn as an Exclusive-OR bar. Its new position, size, and shape are saved by the handling routine in the VioPresentationSpace. Only one cursor can be visible on the screen at any time and it must be in the window with the input focus. This requirement is enforced by the operating system for VIO functions but not for AVIO. The AVIO application must alter the visibility of the cursor when changing input focus. When the text cursor collides with an AVIO and VIO drawing, the presentation driver must remove and redraw the cursor around the alphanumeric updates.

Note: GreBitblt copies everything including the cursor.

The presentation driver can assume that the values in the RowOrgLatch and CursorWidth fields of the VioPresentationSpace structure parallel the WindowOriginRow and TextCursorWidth, respectively.

GreUpdateCursor - Topics

Select an item:
Syntax

Parameters
Returns
Remarks
Glossary

SDBitBlt

SDBitBlt - Syntax

Simulation support:

This function is simulated by a handling routine in Revision 2.X of the graphics engine.

Description:

SDBitBlt is a preclipped BitBlt rendering routine. It processes BitBlt operations using all ROPs and common hardware mixes from Source/Pattern linear addresses to a Destination linear address. SDBitBlt is the default when OS2_PM_DRV_ENABLE: QueryDeviceSurface does not hook the pfnBitBlt function directly.

```
#include <os2.h>

PBITBLTINFO     pInput;    /*  Pointer to BITBLTINFO data structure. */
PVOID           pOutput;   /*  Reserved. */
ULONG           rc;        /*  Return codes. */

rc = SDBitBlt(pInput, pOutput);
```

SDBitBlt Parameter - pInput

pInput (PBITBLTINFO) - input
Pointer to BITBLTINFO data structure.

ulLength	Length of the BITBLTINFO data structure.	
ulBltFlags	Miscellaneous flags used by the graphics engine for rendering of rasterized data.	
	BF_DEFAULT_STATE	Blt direction is left to right and top to bottom.
	BF_DIR_X_NEGATIVE	Blt direction is towards X origin.
	BF_DIR_Y_NEGATIVE	Blt direction is towards Y origin.
	BF_ROP_INCL_SRC	ROP includes a source bit map.
	BF_ROP_INCL_PAT	

	ROP includes a pattern bit map.
BF_SRC_TRANSPARENT	Source transparent involved. SRC will not change when SRC = BG_COLOR.
BF_DST_TRANSPARENT	Destination transparent involved. DST will not change when DST = BG_COLOR.
BF_PAT_TRANSPARENT	Pattern transparent involved. Pattern not involved when PAT = BG_COLOR.
BF_PAT_SOLID	Pattern is solid; all Foreground color.
BF_PAT_HOLLOW	Pattern is hollow (empty); no Foreground color.
BF_APPLY_BACK_ROP	Treat ROP as Foreground mix and MonoBackROP as Background mix.
BF_SRC_MONOINVERT	Zero (0) bits are Foreground on monochrome SRC bit map.
BF_PAT_MONOINVERT	Zero (0) bits are Foreground on monochrome PAT bit map.
BF_SRC_BITS_EXTERNAL	Source bit map bits are in a nondevice-specific format.
BF_LAST_BLT	Defines last blit in a banded BitBlit.
BF_SRC_Y_FLIP	Source Y coordinates are inverted relative to device origin.
BMAP_VERTICAL_SCAN	Scan lines are in a vertical format instead of the default horizontal format.

cBlits	Count of Blts to be performed.
ulROP	Raster operation.
ulMonoBackROP	Background mix if BF_APPLY_BACK_ROP is set.
ulSrcFGColor	Monochrome source Foreground color.
ulSrcBGColor	Monochrome source Background color and transparent color.
ulPatFGColor	Monochrome pattern Foreground color.
ulPatBGColor	Monochrome pattern Background color.
abColors	Pointer to color translation table.

pSrcBmapInfo	Pointer to source bit map (BMAPINFO).
pDstBmapInfo	Pointer to destination bit map (BMAPINFO).
pPatBmapInfo	Pointer to pattern bit map (BMAPINFO).
aptlSrcOrg	Pointer to array of source origin POINTLs.
aptlPatOrg	Pointer to array of pattern origin POINTLs.
abrDst	Pointer to array of Blt rects.
prclSrcBounds	Pointer to source bounding rect of source Blts.
prclDstBounds	Pointer to destination bounding rect of destination Blts.

SDBitBlt Parameter - pOutput

pOutput (PVOID) - input
Reserved.

SDBitBlt Return Value - rc

rc (ULONG) - returns
Return codes.

Valid values are:

RC_SUCCESS
RC_ERROR

SDBitBlt - Parameters

pInput (PBITBLTINFO) - input
Pointer to BITBLTINFO data structure.

ulLength	Length of the BITBLTINFO data structure.
ulBltFlags	Miscellaneous flags used by the graphics engine for rendering of rasterized data.
BF_DEFAULT_STATE	Blt direction is left to right and top to bottom.
BF_DIR_X_NEGATIVE	Blt direction is towards X origin.
BF_DIR_Y_NEGATIVE	Blt direction is towards Y origin.
BF_ROP_INCL_SRC	

	ROP includes a source bit map.
BF_ROP_INCL_PAT	ROP includes a pattern bit map.
BF_SRC_TRANSPARENT	Source transparent involved. SRC will not change when SRC = BG_COLOR.
BF_DST_TRANSPARENT	Destination transparent involved. DST will not change when DST = BG_COLOR.
BF_PAT_TRANSPARENT	Pattern transparent involved. Pattern not involved when PAT = BG_COLOR.
BF_PAT_SOLID	Pattern is solid; all Foreground color.
BF_PAT_HOLLOW	Pattern is hollow (empty); no Foreground color.
BF_APPLY_BACK_ROP	Treat ROP as Foreground mix and MonoBackROP as Background mix.
BF_SRC_MONOINVERT	Zero (0) bits are Foreground on monochrome SRC bit map.
BF_PAT_MONOINVERT	Zero (0) bits are Foreground on monochrome PAT bit map.
BF_SRC_BITS_EXTERNAL	Source bit map bits are in a nondevice-specific format.
BF_LAST_BLT	Defines last blit in a banded BitBlit.
BF_SRC_Y_FLIP	Source Y coordinates are inverted relative to device origin.
BMAP_VERTICAL_SCAN	Scan lines are in a vertical format instead of the default horizontal format.

cBlits	Count of Blts to be performed.
ulROP	Raster operation.
ulMonoBackROP	Background mix if BF_APPLY_BACK_ROP is set.
ulSrcFGColor	Monochrome source Foreground color.
ulSrcBGColor	Monochrome source Background color and transparent color.
ulPatFGColor	Monochrome pattern Foreground color.

ulPatBGColor	Monochrome pattern Background color.
abColors	Pointer to color translation table.
pSrcBmapInfo	Pointer to source bit map (BMAPINFO).
pDstBmapInfo	Pointer to destination bit map (BMAPINFO).
pPatBmapInfo	Pointer to pattern bit map (BMAPINFO).
aptlSrcOrg	Pointer to array of source origin POINTLs.
aptlPatOrg	Pointer to array of pattern origin POINTLs.
abrDst	Pointer to array of Blt rects.
prclSrcBounds	Pointer to source bounding rect of source Blts.
prclDstBounds	Pointer to destination bounding rect of destination Blts.

pOutput (PVOID) - input
Reserved.

rc (ULONG) - returns
Return codes.

Valid values are:

RC_SUCCESS
RC_ERROR

SDBitBlt - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

SDLine

SDLine - Syntax

Simulation support:

This function is simulated by a handling routine in Revision 2.X of the graphics engine.

Description:

SDLine is a preclipped line rendering routine. It processes line operations onto a destination linear address. SDLine is the default when OS2_PM_DRV_ENABLE: QueryDeviceSurface does not hook the pfnLine function directly.

```
#include <os2.h>

PLINEINFO    pInput;    /*  Pointer to LINEINFO data structure. */
```

```
PVOID        pOutput;   /*  Reserved. */
ULONG        rc;        /*  Return codes. */

rc = SDLine(pInput, pOutput);
```

SDLine Parameter - pInput

pInput (PLINEINFO) - input
Pointer to LINEINFO data structure.

ulLength — Length of the LINEINFO data structure.

ulType — Defines line type.

- LINE_SOLID — Line will be solid in Foreground color.
- LINE_INVISIBLE — Line is not drawn.
- LINE_ALTERNATE — Line will be alternating Foreground and Background color; ignores style.

ulStyleMask — A 32-bit style mask.

cLines — Count of lines to be drawn.

ulFGColor — Line Foreground color.

ulBGColor — Line Background color.

usForeROP — Line Foreground mix.

usBackROP — Line Background mix.

pDstBmapInfo — Destination surface (BMAPINFO).

alpkLinePack — Array of line details information.

- ulStyleStep — The value to be added to ulStyleValue on each pel stepped along the line style's direction.
- ulStyleValue — The style value at the current pel. It is composed of an error value and a mask position, as follows:

```
high word        3 bits          5 bits
not used         not used        mask pos
```

- ulFlags — Miscellaneous flags used for the LINEPACK data structure.
  - LINE_DO_FIRST_PEL — Draws the first pel.
  - LINE_DIR_Y_POSITIVE — Indicates line direction is bottom-to-top.
  - LINE_HORIZONTAL

Indicates line is horizontal. No Bresenham algorithm.

LINE_X_MAJOR

Line is XMAJOR.

LINE_DIR_X_POSITIVE

Indicates line direction is right-to-left.

LINE_VERTICAL

Indicates line is vertical. No Bresenham algorithm.

LINE_STYLE_X_MAJOR

Line style is XMAJOR.

LINE_DO_LAST_PEL

Draws the last pel.

ulAbsDeltaX

Clipped Bresenham Delta X, absolute (ptlStart.x - ptlEnd.x).

ulAbsDeltaY

Clipped Bresenham Delta Y, absolute (ptlStart.y - ptlEnd.y).

ptlClipStart

Pointer to location where device performs the Bresenham algorithm. Sets only the pels from ptlClipStart to ptlClipEnd, inclusive.

ptlClipEnd

Ending location for Bresenham algorithm (see previous parameter, ptlClipStart).

ptlStart

Pointer to starting location for line. The device can perform the Bresenham algorithm from ptlStart or ptlClipStart.

ptlEnd

Ending location for line.

lClipStartError

The standard Bresenham error at the clipped start point. This error is calculated from the initial error at the start point and the error increments for major step and diagonal step. The initial error and the error increments are as follows:

MAX

Maximum (ulAbsDeltaX, ulAbsDeltaY)

MIN

Minimum (ulAbsDeltaX, ulAbsDeltaY)

Major Increment

Increment to the error for stepping along the major axis: 2 * MIN.

Diagonal Increment

Increment to the error for stepping along the major and minor axes: 2 * MIN - 2 * MAX.

Initial Error

Error at the start point: 2 * MIN - MAX, if LINE_DIR_X_POSITIVE is On

2 * MIN - MAX - 1, if LINE_DIR_Y_POSITIVE is Off.

Horizontal and vertical lines . The line is drawn from the clipped start to clipped end. The lClipStartError will not be given.

First pel consideration . Set the first pel at the ptlStart (not ptlClipStart) only if LINE_DO_FIRST_PEL is set and the first pel is not clipped.

Last pel consideration . Set the last pel at the ptlEnd (not ptlClipEnd) only if LINE_DO_LAST_PEL is set and the last pel is not clipped.

Styling . Lines are styled using the ulStyleMask, ulStyleStep and ulStyleValue.

Error Value
: Error value at the current pel.

Mask Position
: Bit position of the ulStyleMask.

  If this bit is on, set the current pel to the ulFGColor through usForeROP; otherwise, set the current pel to the ulBGColor through usBackROP.

prclBounds
: Bounding rect of the clipped line.

SDLine Parameter - pOutput

pOutput (PVOID) - input
: Reserved.

SDLine Return Value - rc

rc (ULONG) - returns
Return codes.

Valid values are:

RC_SUCCESS
RC_ERROR

SDLine - Parameters

pInput (PLINEINFO) - input
Pointer to LINEINFO data structure.

ulLength	Length of the LINEINFO data structure.
ulType	Defines line type.
	LINE_SOLID — Line will be solid in Foreground color.
	LINE_INVISIBLE — Line is not drawn.
	LINE_ALTERNATE — Line will be alternating Foreground and Background color; ignores style.
ulStyleMask	A 32-bit style mask.
cLines	Count of lines to be drawn.
ulFGColor	Line Foreground color.
ulBGColor	Line Background color.
usForeROP	Line Foreground mix.
usBackROP	Line Background mix.
pDstBmapInfo	Destination surface (BMAPINFO).
alpkLinePack	Array of line details information.
	ulStyleStep — The value to be added to ulStyleValue on each pel stepped along the line style's direction.
	ulStyleValue — The style value at the current pel. It is composed of an error value and a mask position, as follows:

```
high word       3 bits          5 bits          
not used        not used        mask pos        
```

	ulFlags — Miscellaneous flags used for the LINEPACK data structure.
	LINE_DO_FIRST_PEL — Draws the first pel.

LINE_DIR_Y_POSITIVE

Indicates line direction is bottom-to-top.

LINE_HORIZONTAL

Indicates line is horizontal. No Bresenham algorithm.

LINE_X_MAJOR

Line is XMAJOR.

LINE_DIR_X_POSITIVE

Indicates line direction is right-to-left.

LINE_VERTICAL

Indicates line is vertical. No Bresenham algorithm.

LINE_STYLE_X_MAJOR

Line style is XMAJOR.

LINE_DO_LAST_PEL

Draws the last pel.

ulAbsDeltaX

Clipped Bresenham Delta X, absolute (ptlStart.x - ptlEnd.x).

ulAbsDeltaY

Clipped Bresenham Delta Y, absolute (ptlStart.y - ptlEnd.y).

ptlClipStart

Pointer to location where device performs the Bresenham algorithm. Sets only the pels from ptlClipStart to ptlClipEnd, inclusive.

ptlClipEnd

Ending location for Bresenham algorithm (see previous parameter, ptlClipStart).

ptlStart

Pointer to starting location for line. The device can perform the Bresenham algorithm from ptlStart or ptlClipStart.

ptlEnd

Ending location for line.

lClipStartError

The standard Bresenham error at the clipped start point. This error is calculated from the initial error at the start point and the error increments for major step and diagonal step. The initial error and the error increments are as follows:

MAX

Maximum (ulAbsDeltaX, ulAbsDeltaY)

MIN

Minimum (ulAbsDeltaX, ulAbsDeltaY)

Major Increment

Increment to the error for stepping along the major axis: 2 * MIN.

Diagonal Increment

Increment to the error for stepping along the major and minor axes:

2 * MIN - 2 * MAX.

Initial Error

Error at the start point:
2 * MIN - MAX, if LINE_DIR_X_POSITIVE is On

2 * MIN - MAX - 1, if LINE_DIR_Y_POSITIVE is Off.

Horizontal and vertical lines . The line is drawn from the clipped start to clipped end. The lClipStartError will not be given.

First pel consideration . Set the first pel at the ptlStart (not ptlClipStart) only if LINE_DO_FIRST_PEL is set and the first pel is not clipped.

Last pel consideration . Set the last pel at the ptlEnd (not ptlClipEnd) only if LINE_DO_LAST_PEL is set and the last pel is not clipped.

Styling . Lines are styled using the ulStyleMask, ulStyleStep and ulStyleValue.

Error Value

Error value at the current pel.

Mask Position

Bit position of the ulStyleMask.

If this bit is on, set the current pel to the ulFGColor through usForeROP; otherwise, set the current pel to the ulBGColor through usBackROP.

prclBounds

Bounding rect of the clipped line.

pOutput (PVOID) - input
Reserved.

rc (ULONG) - returns
Return codes.

Valid values are:

RC_SUCCESS
RC_ERROR

SDLine - Remarks

The line starts from ptlStart and ends at ptlEnd (inclusive). The device can perform the Bresenham algorithm either from ptlStart or from ptlClipStart, using lClipStartError, and will set only the pels from ptlClipStart to ptlClipEnd (inclusive).

SDLine - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

Device Support Function Reference

All device support (helper) functions are listed alphabetically in the following table. The table includes port and queue drivers, which are discussed in Queue Drivers and Port Drivers.

A detailed description of each function follows the table.

Function	Type
Default_Primary	display driver
DspDefaultResolution	GRE exported API
DspQueryDriverInfo	GRE exported API
DspQueryDriverNames	GRE exported API
DspSetDriverInfo	GRE exported API
GetDriverInfo	GRE exported entry point
GreCloseDC	GRE internal (device context)
GreCopyDCLoadData	GRE internal (LCID)
GreCreateBitmap	GRE internal (device support)
GreCreateJournalFile	GRE internal (journaling)
GreCreateLogicalFont	GRE internal (font)
GreDeleteBitmap	GRE internal (device support)
GreDeleteJournalFile	GRE internal (journaling)
GreDeleteSetId	GRE internal (set id)
GreGetAttributes	GRE internal (device support)
GreGetBitmapDimension	GRE internal (device support)
GreGetBitmapParameters	GRE internal (device support)
GreGetDefaultArcParameters	GRE internal (device support)
GreGetDefaultAttributes	GRE internal (device support)
GreGetDefaultViewingLimits	GRE internal (device support)

GreGetHandle	GRE internal (device context)
GreGetProcessControl	GRE internal (device context)
GreInitializeAttributes	GRE internal (device support)
GreLoadFont	GRE internal (font)
GreOpenDC	GRE internal (device context)
GreOpenJournalFile	GRE internal (journaling)
GrePlayJournalFile	GRE internal (journaling)
GreQueryBitmapHandle	GRE internal (LCID)
GreQueryCodePageVector	GRE internal (font)
GreQueryEngineVersion	GRE internal (device context)
GreQueryFontAttributes	GRE internal (font)
GreQueryFontFileDescriptions	GRE internal (font)
GreQueryFonts	GRE internal (fonts)
GreQueryLogicalFont	GRE internal (fonts)
GreQueryNumberSetIds	GRE internal (set id)
GreQuerySetIds	GRE internal (set id)
GreRealizeString	GRE
GreResetDC	GRE internal (device context)
GreRestoreDC	GRE internal (device context)
GreSaveDC	GRE internal (device context)
GreSelectBitmap	GRE internal (device support)
GreSetAttributes	GRE internal (device support)
GreSetBitmapDimension	GRE internal (device support)
GreSetBitmapID	GRE internal (LCID)
GreSetDefaultArcParameters	GRE internal (device support)
GreSetDefaultAttributes	GRE internal (device support)
GreSetDefaultViewingLimits	GRE internal (device support)
GreSetDeviceSurface	GRE internal (device support)
GreSetGlobalAttribute	GRE internal (device support)
GreSetHandle	GRE internal (device context)
GreSetProcessControl	GRE internal (device context)
GreStartJournalFile	GRE internal (journaling)
GreStopJournalFile	GRE internal (journaling)
GreUnLoadFont	GRE internal (font)
PrtAbortDoc	GRE hardcopy
PrtClose	GRE hardcopy
PrtDevIOCtl	GRE hardcopy
PrtNewPage	GRE hardcopy
PrtOpen	GRE hardcopy
PrtQuery	GRE hardcopy
PrtResetAbort	GRE hardcopy
PrtSet	GRE hardcopy

PrtWrite	GRE hardcopy
SetDriverInfo	GRE exported entry point
SplCreatePort	GRE hardcopy
SplDeletePort	GRE hardcopy
SplDisable	GRE hardcopy
SplDisplayControlPanel	GRE hardcopy
SplEnable	GRE hardcopy
SplGetControlPanelList	GRE hardcopy
SplGetPortFromQ	GRE hardcopy
SplMessageBox	GRE hardcopy
SplPdAbortDoc	BIDI port driver
SplPdClose	BIDI port driver
SplPdEnumPort	port driver
SplPdGetPortIcon	port driver
SplPdInitPort	port driver
SplPdInstallPort	port driver
SplPdNewPage	BIDI port driver
SplPdOpen	BIDI port driver
SplPdQuery	BIDI port driver
SplPdQueryPort	port driver
SplPdRemoteSetPort	BIDI port driver
SplPdRemovePort	port driver
SplPdResetAbort	BIDI port driver
SplPdSendCmd	BIDI port driver
SplPdSet	BIDI port driver
SplPdSetPort	port driver
SplPdTermPort	port driver
SplPdWrite	BIDI port driver
SplProtSendCmd	BIDI protocol converter
SplProtWrite	BIDI protocol converter
SplProtXlateCmd	BIDI protocol converter
SplQmAbort	GRE hardcopy
SplQmAbortDoc	GRE hardcopy
SplQmClose	GRE hardcopy
SplQmEndDoc	GRE hardcopy
SplQmGetJobID	GRE hardcopy
SplQmNewPage	GRE hardcopy
SplQmOpen	GRE hardcopy
SplQmStartDoc	GRE hardcopy
SplQmWrite	GRE hardcopy
SplQpClose	queue driver

SplQpControl	queue driver
SplQpInstall	queue driver
SplQpOpen	queue driver
SplQpPrint	queue driver
SplQpQueryDt	queue driver
SplQpQueryFlags	queue driver
SplQueryControlPanel	GRE hardcopy
SplQueryPath	GRE hardcopy
SplQueryPort	GRE hardcopy
SplRegister	GRE hardcopy
SplRegisterControlPanel	GRE hardcopy
SplSetPort	GRE hardcopy
SplStdClose	GRE hardcopy
SplStdDelete	GRE hardcopy
SplStdGetBits	GRE hardcopy
SplStdOpen	GRE hardcopy
SplStdQueryLength	GRE hardcopy
SplStdStart	GRE hardcopy
SplStdStop	GRE hardcopy
SplUnRegister	GRE hardcopy
SSAllocMem	memory management
SSFreeMem	memory management
VisRegionNotify	system function
WinSetErrorInfo	system function

Default_Primary

Default_Primary - Syntax

Description:

This action routine is invoked when the user accepts the primary display selection of the Video Display Configuration PM panel. The other three DSPINSTL action routines have identical calling sequences and return codes.

```
#include <os2.h>

HWND              hWnd;
PDSPINSTL_DATA    dsp_stuc_ptr;
ULONG             rc;             /*  Return codes. */

rc = Default_Primary(hWnd, dsp_stuc_ptr);
```

Default_Primary Parameter - hWnd

hWnd (HWND) - input
The handle of the main DSPINSTL window.

Default_Primary Parameter - dsp_stuc_ptr

dsp_stuc_ptr (PDSPINSTL_DATA) - input

The global DSPINSTL data structure is defined as follows:

```
struct dspinstl_global_data
{
ULONG rev_num;                    /* Global DSPINSTL data structure       */
                                  /*   revision number                    */
PFN Link_Chain_Element_Ptr;       /* Address of the Link_Chain_Element    */
                                  /*   service function                   */
PFN Unlink_Chain_Element_Ptr;     /* Address of the Unlink_Chain_Element  */
                                  /*   service function                   */
PFN Next_Chain_Element_Ptr;       /* Address of the Next_Chain_Element    */
                                  /*   service function                   */
PFN Reserved_PTR1;                /* Reserved                             */
PFN Select_Display_Resolutions_Ptr;/* Address of Select_Display_Resolutions*/
                                  /*   service function                   */
PFN Reserved_PTR2;                /* Reserved                             */
PFN Reserved_PTR3;                /* Reserved                             */
PFN Reserved_PTR4;                /* Reserved                             */
PDSPINSTL_GLOBAL_DATA             /* Pointer to head of data element chain*/
PDSPINSTL_CHAIN_head              /* Pointer to head of data element chain*/
BOOL Reserved_Book                /* Reserved                             */
PSZ pszSourceDirectory            /* Source directory name (ASCIIZ string)*/
ULONG ulSizeSource                /* Size of source directory name buffer */
};
```

The source directory name string is the string that will be displayed in the Specify Source Directory PM panel that allows the user to override the default source directory for a given DSPINSTL step. The source directory name string, therefore, acts as the default source directory for that step. The string's inclusion in the global data structure provides the ability to modify the default source directory name to all action routines.

There are three service routines that are provided to manipulate the data element chain. The following data structure characterizes a DSPINSTL data element:

```
struct dspinstl_chain_element
{
 ULONG rev_num;         /* DSPINSTL data chain element revision number    */

 PDSPINSTL_CHAIN next;  /* Pointer to next DSPINSTL data chain element    */

 PSZ key;               /* Pointer to keyword string associated with this */
                        /*   chain element                                */

 PSZ general_prompt;    /* Pointer to the prompt string to be written in  */
                        /*   the Specify Source Directory panel when this */
                        /*   chain element is interpreted.  A NULL value  */
                        /*   indicates that the previous source directory */
                        /*   should be used again and that this panel     */
                        /*   should be bypassed for this chain element.   */

 PSZ diskette_prompt;   /* Pointer to the prompt string to be written in  */
                        /*   the Insert Diskette prompt panel             */
```

```
 PSZ volume_label;      /* Pointer to the (optional) diskette volume      */
                        /*   label string that is used to verify the      */
                        /*   correct identity of a DSP diskette           */

 PSZ resolution_string;/* Optional string that indicates a display driver*/
                        /*   resolution.  The string can be passed to     */
                        /*   Select_Display_Resolutions or left NULL      */
};
```

Note: The various strings that are found in DSPINSTL data elements are typically initialized from string values found in DSPINSTL configuration file entries.

Default_Primary Return Value - rc

rc (ULONG) - returns
Return codes.

Action routines return the following values:

0	Indicates that no problems were encountered and DSPINSTL should proceed forward.
0xFFFFFFFF	Indicates that the action routine is returning a Cancel indication, which might be triggered by the user pressing a **Cancel** push button within a PM panel that was invoked by the action routine. Note that the Cancel return code is supported so that DSPINSTL can back out of its current execution path.
0xFFFFFFFE	Is a reserved return code

All other non-zero values are interpreted as OS/2 system error codes.

Default_Primary - Parameters

hWnd (HWND) - input
The handle of the main DSPINSTL window.

dsp_stuc_ptr (PDSPINSTL_DATA) - input

The global DSPINSTL data structure is defined as follows:

```
struct dspinstl_global_data
{
ULONG rev_num;                  /* Global DSPINSTL data structure       */
                                /*   revision number                    */
PFN Link_Chain_Element_Ptr;     /* Address of the Link_Chain_Element    */
                                /*   service function                   */
PFN Unlink_Chain_Element_Ptr;   /* Address of the Unlink_Chain_Element  */
                                /*   service function                   */
PFN Next_Chain_Element_Ptr;     /* Address of the Next_Chain_Element    */
                                /*   service function                   */
PFN Reserved_PTR1;              /* Reserved                             */
PFN Select_Display_Resolutions_Ptr;/* Address of Select_Display_Resolutions*/
                                /*   service function                   */
PFN Reserved_PTR2;              /* Reserved                             */
PFN Reserved_PTR3;              /* Reserved                             */
PFN Reserved_PTR4;              /* Reserved                             */
PDSPINSTL_GLOBAL_DATA           /* Pointer to head of data element chain*/
PDSPINSTL_CHAIN_head            /* Pointer to head of data element chain*/
BOOL Reserved_Book              /* Reserved                             */
PSZ pszSourceDirectory          /* Source directory name (ASCIIZ string)*/
ULONG ulSizeSource              /* Size of source directory name buffer */
};
```

The source directory name string is the string that will be displayed in the Specify Source Directory PM panel that allows the user to override the default source directory for a given DSPINSTL step. The source directory name string, therefore, acts as the default source directory for that step. The string's inclusion in the global data structure provides the ability to modify the default source directory name to all action routines.

There are three service routines that are provided to manipulate the data element chain. The following data structure characterizes a DSPINSTL data element:

```
struct dspinstl_chain_element
{
 ULONG rev_num;         /* DSPINSTL data chain element revision number    */

 PDSPINSTL_CHAIN next; /* Pointer to next DSPINSTL data chain element    */

 PSZ key;               /* Pointer to keyword string associated with this */
                        /*   chain element                                */

 PSZ general_prompt;    /* Pointer to the prompt string to be written in  */
                        /*   the Specify Source Directory panel when this */
                        /*   chain element is interpreted.  A NULL value  */
                        /*   indicates that the previous source directory */
                        /*   should be used again and that this panel     */
                        /*   should be bypassed for this chain element.   */

 PSZ diskette_prompt;   /* Pointer to the prompt string to be written in  */
                        /*   the Insert Diskette prompt panel             */

 PSZ volume_label;      /* Pointer to the (optional) diskette volume      */
                        /*   label string that is used to verify the      */
                        /*   correct identity of a DSP diskette           */

 PSZ resolution_string;/* Optional string that indicates a display driver*/
                        /*   resolution.  The string can be passed to     */
                        /*   Select_Display_Resolutions or left NULL      */
};
```

Note: The various strings that are found in DSPINSTL data elements are typically initialized from string values found in DSPINSTL configuration file entries.

rc (ULONG) - returns
Return codes.

Action routines return the following values:

0	Indicates that no problems were encountered and DSPINSTL should proceed forward.
0xFFFFFFFF	Indicates that the action routine is returning a Cancel indication, which might be triggered by the user pressing a **Cancel** push button within a PM panel that was invoked by the action routine. Note that the Cancel return code is supported so that DSPINSTL can back out of its current execution path.
0xFFFFFFFE	Is a reserved return code

All other non-zero values are interpreted as OS/2 system error codes.

Default_Primary - Remarks

Action routines can perform whatever computation is necessary to modify the DSPINSTL data chain for eventual execution. Typically, action routines modify a key field within a data element, or they might add or subtract chain elements.

Generally, action routines should not change external system values such as OS2.INI entries because there is no guarantee that the user will not end the installation process after the action routine returns. Therefore, *persistent state* setting should be performed by the commands within DSP files rather than by action routines.

The action routine communicates with the DSPINSTL utility program by setting the data chain element key values. Therefore, if an action routine sets several alternative key values in different chain elements, an equivalent set of DSP files (at least one for each alternative key value) must be provided in the eventual source directories or diskettes. (See DSPINSTL Service Functions).

Default_Primary - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

DspDefaultResolution

DspDefaultResolution - Syntax

Description:

This function is used to set or query information about the system's current default resolution.

```
#include <os2.h>

PSCREENRESOLUTION    pScreenResolution;
PULONG               pcbScreenResolution;
ULONG                flOptions;
ULONG                fSuccess;             /*  Return codes. */

fSuccess = DspDefaultResolution(pScreenResolution,
             pcbScreenResolution, flOptions);
```

DspDefaultResolution Parameter - pScreenResolution

pScreenResolution (PSCREENRESOLUTION) - input
If flOptions==DSP_SET_DEFAULT_RESOLUTIONS, this parameter points to ScreenResolution structure to be saved to the INI file. If not, this parameter points to the buffer that receives the default values currently saved in the INI file.

DspDefaultResolution Parameter - pcbScreenResolution

pcbScreenResolution (PULONG) - input
Points to the count of bytes in the pScreenResolution buffer. If pScreenResolution is NULL on input, the function will store the count of bytes needed to store all of the resolution data in pcbScreenResolution. If not ZERO, upon return, that field contains the number of bytes actually returned.

DspDefaultResolution Parameter - flOptions

flOptions (ULONG) - input
Used to specify whether to SET or QUERY the default resolution. If flOptions==DSP_SET_DEFAULT_RESOLUTIONS, the values are SET (written) in the INI file. Otherwise, the INI file is QUERYED.

DSP_RESOLUTION_OBTAINABLE	Default resolution is obtained from the current hardware configuration.
DSP_RESOLUTION_DEFAULT	This is a default resolution.

Example of structure pointed to by pScreenResolution:

```
typedef struct _SCREENRESOLUTION {
   ULONG width;     /* width of the device in pels             */
   ULONG height;    /* height of the device in pels            */
   ULONG colors;    /* number of colors supported in this mode */
   ULONG planes;    /* number of display planes in this mode   */
   ULONG flOptions /* identifies optional information for each resolution*/
} SCREENRESOLUTION;
typedef SCREENRESOLUTION *PSCREENRESOLUTION;
```

DspDefaultResolution Return Value - fSuccess

fSuccess (ULONG) - returns
Return codes.

On completion, this installation routine must return BOOLEAN (fSuccess).

TRUE	Success
FALSE	Error

DspDefaultResolution - Parameters

pScreenResolution (PSCREENRESOLUTION) - input
If flOptions==DSP_SET_DEFAULT_RESOLUTIONS, this parameter points to ScreenResolution structure to be saved to the INI file. If not, this parameter points to the buffer that receives the default values currently saved in the INI file.

pcbScreenResolution (PULONG) - input
Points to the count of bytes in the pScreenResolution buffer. If pScreenResolution is NULL on input, the function will store the count of bytes needed to store all of the resolution data in pcbScreenResolution. If not ZERO, upon return, that field contains the number of bytes actually returned.

flOptions (ULONG) - input
Used to specify whether to SET or QUERY the default resolution. If flOptions==DSP_SET_DEFAULT_RESOLUTIONS, the values are SET (written) in the INI file. Otherwise, the INI file is QUERYED.

DSP_RESOLUTION_OBTAINABLE	Default resolution is obtained from the current hardware configuration.
DSP_RESOLUTION_DEFAULT	This is a default resolution.

Example of structure pointed to by pScreenResolution:

```
typedef struct _SCREENRESOLUTION {
```

```
   ULONG width;     /* width of the device in pels             */
   ULONG height;    /* height of the device in pels            */
   ULONG colors;    /* number of colors supported in this mode */
   ULONG planes;    /* number of display planes in this mode   */
   ULONG flOptions /* identifies optional information for each resolution*/
} SCREENRESOLUTION;
typedef SCREENRESOLUTION *PSCREENRESOLUTION;
```

fSuccess (ULONG) - returns
Return codes.

On completion, this installation routine must return BOOLEAN (fSuccess).

TRUE	Success
FALSE	Error

DspDefaultResolution - Remarks

It can be used to change the system's display resolution. Such a change takes effect the next time the system is restarted. PM display drivers that can support multiple resolutions use the function during their initialization to discover the current display resolution that has been selected by the user.

DspDefaultResolution - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

DspQueryDriverInfo

DspQueryDriverInfo - Syntax

Description:

DspQueryDriverInfo allows the caller to query driver information about any PM display driver that has been registered with the operating system (by using DspSetDriverInfo).

```
#include <os2.h>

PDSPINFO    pDSPInfo;   /*  Pointer to a data structure. */
ULONG       flOptions;
BOOL        fSuccess;   /*  Return codes. */

fSuccess = DspQueryDriverInfo(pDSPInfo, flOptions);
```

DspQueryDriverInfo Parameter - pDSPInfo

pDSPInfo (PDSPINFO) - input
Pointer to a data structure.

A pointer to a PDSPINFO data structure that contains the following values:

pszDriverName	A NULL-terminated string containing the name of the driver. All IBM drivers should start with "IBM", as in IBMVGA or IBMXGA.
cchDriverName	Length of the pszDriverName buffer.
pszDesc	Pointer to a string describing the driver. This description is used by the program that lists the available drivers and that gives information as to which driver is loaded.
cchDesc	Length of the pszDesc buffer.
pszFileName	The fully qualified path name (drive, path, and file name) of the display driver DLL.
cchFileName	Length of the pszFileName buffer.
pszOptionalParameters	Pointer to a string of optional parameters, which might be known only to the device driver.
cchOptionalParameters	Length of the pszOptionalParameters buffer.

DspQueryDriverInfo Parameter - flOptions

flOptions (ULONG) - input
The following flags are used to specify various options:

DSP_QUERY_OPT_PARMS	Get the optional parameters. The default does not query optional parameters. If this flag is not set, pszOptionalParameters is set to NULL.
DSP_QUERY_CURRENT_DRIVER	Get information about the current driver. In this case, no driver name is specified; instead, the name of the current driver is returned in pszDriverName.
DSP_QUERY_DEFAULT_DRIVER	Get information about the default driver. In this case, no driver name is specified; instead, the name of the default driver is returned in pszDriverName.

DspQueryDriverInfo Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, this installation routine must return BOOLEAN (fSuccess).

TRUE	Success
FALSE	Error.

Possible Errors Detected: When an error is detected, the handling routine must call WinGetLastError to post the condition. Error codes

for conditions that the handling routine is expected to check are as follows:

PMERR_INVALID_PARAMETER
PMERR_MEMORY_ALLOC
PMERR_MEMORY_ALLOCATION_ERR
PMERR_MEMORY_DEALLOCATION_ERR

DspQueryDriverInfo - Parameters

pDSPInfo (PDSPINFO) - input
Pointer to a data structure.

A pointer to a PDSPINFO data structure that contains the following values:

pszDriverName	A NULL-terminated string containing the name of the driver. All IBM drivers should start with "IBM", as in IBMVGA or IBMXGA.
cchDriverName	Length of the pszDriverName buffer.
pszDesc	Pointer to a string describing the driver. This description is used by the program that lists the available drivers and that gives information as to which driver is loaded.
cchDesc	Length of the pszDesc buffer.
pszFileName	The fully qualified path name (drive, path, and file name) of the display driver DLL.
cchFileName	Length of the pszFileName buffer.
pszOptionalParameters	Pointer to a string of optional parameters, which might be known only to the device driver.
cchOptionalParameters	Length of the pszOptionalParameters buffer.

flOptions (ULONG) - input
The following flags are used to specify various options:

DSP_QUERY_OPT_PARMS	Get the optional parameters. The default does not query optional parameters. If this flag is not set, pszOptionalParameters is set to NULL.
DSP_QUERY_CURRENT_DRIVER	Get information about the current driver. In this case, no driver name is specified; instead, the name of the current driver is returned in pszDriverName.
DSP_QUERY_DEFAULT_DRIVER	Get information about the default driver. In this case, no driver name is specified; instead, the name of the default driver is returned in pszDriverName.

fSuccess (BOOL) - returns
Return codes.

On completion, this installation routine must return BOOLEAN (fSuccess).

TRUE	Success
FALSE	Error.

Possible Errors Detected: When an error is detected, the handling routine must call WinGetLastError to post the condition. Error codes for conditions that the handling routine is expected to check are as follows:

PMERR_INVALID_PARAMETER
PMERR_MEMORY_ALLOC
PMERR_MEMORY_ALLOCATION_ERR
PMERR_MEMORY_DEALLOCATION_ERR

DspQueryDriverInfo - Remarks

Display drivers can be identified explicitly by name. If the DSP_QUERY_DEFAULT_DRIVER flag is set, the information for the default driver is returned. If the DSP_QUERY_CURRENT_DRIVER flag is set, the information for the current driver is returned. Otherwise, the information on the given driver is returned.

DspQueryDriverInfo - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

DspQueryDriverNames

DspQueryDriverNames - Syntax

Description:

This function returns a list of the names of all PM display drivers that have been registered with the operating system (by using DspSetDriverInfo).

```
#include <os2.h>

PVOID    pBuffer;  /*  Pointer to a buffer. */
ULONG    ulCount;
ULONG    rc;       /*  Return codes. */

rc = DspQueryDriverNames(pBuffer, ulCount);
```

DspQueryDriverNames Parameter - pBuffer

pBuffer (PVOID) - input
Pointer to a buffer.

Pointer to the buffer in which the driver names are returned. The names are NULL-terminated strings with one immediately following another. The final string is followed by two NULL characters.

DspQueryDriverNames Parameter - ulCount

ulCount (ULONG) - input
The maximum number of bytes that can be returned in pBuffer.

DspQueryDriverNames Return Value - rc

rc (ULONG) - returns
Return codes.

On completion, the return value is of type ULONG and is the number of bytes actually returned in pBuffer.

DspQueryDriverNames - Parameters

pBuffer (PVOID) - input
Pointer to a buffer.

Pointer to the buffer in which the driver names are returned. The names are NULL-terminated strings with one immediately following another. The final string is followed by two NULL characters.

ulCount (ULONG) - input
The maximum number of bytes that can be returned in pBuffer.

rc (ULONG) - returns
Return codes.

On completion, the return value is of type ULONG and is the number of bytes actually returned in pBuffer.

DspQueryDriverNames - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

DspSetDriverInfo

DspSetDriverInfo - Syntax

Description:

This function is used to set PM display driver configuration parameters.

```
#include <os2.h>

PDSPINFO    pDSPInfo;   /*  Pointer to a PDSPINFO data structure. */
ULONG       flOptions;
BOOL        fSuccess;   /*  Return codes. */

fSuccess = DspSetDriverInfo(pDSPInfo, flOptions);
```

DspSetDriverInfo Parameter - pDSPInfo

pDSPInfo (PDSPINFO) - input
Pointer to a PDSPINFO data structure.

Pointer to a PDSPINFO data structure that contains the following values:

pszDriverName	A NULL-terminated string containing the name of the driver. All IBM drivers should start with "IBM", as in IBMVGA or IBMXGA.
cchDriverName	Length of the pszDriverName buffer.
pszDesc	Pointer to a string describing the driver. This description is used by the program that lists the available drivers and that gives information as to which driver is loaded.
cchDesc	Length of the pszDesc buffer.
pszFileName	The fully qualified path name (drive, path, and file name) of the display driver DLL.
cchFileName	Length of the pszFileName buffer.
pszOptionalParameters	Pointer to a string of optional parameters, which might be known only to the device driver.
cchOptionalParameters	Length of the pszOptionalParameters buffer.

DspSetDriverInfo Parameter - flOptions

flOptions (ULONG) - input
The following flags are used to specify various options:

DSP_DELETE_DRIVER	Completely delete the driver information from the INI file.
DSP_OPT_PARMS_PRESENT	Optional parameters are present. The default does not use optional parameters. If this flag is not set, pszOptionalParameters is ignored and can be NULL.
DSP_SET_DEFAULT_DRIVER	Make this driver the default system display driver.
DSP_SET_INSTALL_ON_IPL	Install this driver as the current system display driver at next startup.

DspSetDriverInfo Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, this installation routine must return BOOLEAN (fSuccess).

TRUE	Success, no error
FALSE	NULL, error occurred

Possible Errors Detected: When an error is detected, the handling routine must call WinGetLastError to post the condition. Error codes for conditions that the handling routine is expected to check are:

PMERR_CURRENT_SYSTEM_DRIVER
PMERR_INVALID_PARAMETER
PMERR_MEMORY_ALLOC
PMERR_MEMORY_ALLOCATION_ERR
PMERR_MEMORY_DEALLOCATION_ERR

DspSetDriverInfo - Parameters

pDSPInfo (PDSPINFO) - input
Pointer to a PDSPINFO data structure.

Pointer to a PDSPINFO data structure that contains the following values:

pszDriverName	A NULL-terminated string containing the name of the driver. All IBM drivers should start with "IBM", as in IBMVGA or IBMXGA.
cchDriverName	Length of the pszDriverName buffer.
pszDesc	Pointer to a string describing the driver. This description is used by the program that lists the available drivers and that gives information as to which driver is loaded.
cchDesc	Length of the pszDesc buffer.
pszFileName	The fully qualified path name (drive, path, and file name) of the display driver DLL.
cchFileName	Length of the pszFileName buffer.
pszOptionalParameters	Pointer to a string of optional parameters, which might be known only to the device driver.
cchOptionalParameters	Length of the pszOptionalParameters buffer.

flOptions (ULONG) - input
The following flags are used to specify various options:

DSP_DELETE_DRIVER	Completely delete the driver information from the INI file.
DSP_OPT_PARMS_PRESENT	Optional parameters are present. The default does not use optional parameters. If this flag is not set, pszOptionalParameters is ignored and can be NULL.
DSP_SET_DEFAULT_DRIVER	Make this driver the default system display driver.
DSP_SET_INSTALL_ON_IPL	Install this driver as the current system display driver at next startup.

fSuccess (BOOL) - returns
Return codes.

On completion, this installation routine must return BOOLEAN (fSuccess).

TRUE	Success, no error

FALSE NULL, error occurred

Possible Errors Detected: When an error is detected, the handling routine must call WinGetLastError to post the condition. Error codes for conditions that the handling routine is expected to check are:

PMERR_CURRENT_SYSTEM_DRIVER
PMERR_INVALID_PARAMETER
PMERR_MEMORY_ALLOC
PMERR_MEMORY_ALLOCATION_ERR
PMERR_MEMORY_DEALLOCATION_ERR

DspSetDriverInfo - Remarks

The call is used to register display drivers with the operating system (informing the operating system of the existence and location of the drivers). It can be used to mark a display driver as the system's current display driver or the system's default display driver. If the DSP_DELETE_DRIVER flag is set, all of the information for a given driver is deleted and the pDSPInfo parameter is ignored.

DspSetDriverInfo - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GetDriverInfo

GetDriverInfo - Syntax

Description:

This function returns a driver object handle for the engine object identified by hEngObject.

```
#include <os2.h>

ULONG    hEngObject;  /*  Handle of a DC or bit map. */
ULONG    ulIndex;
HDC      hdc;         /*  Device context handle, which identifies the calling DC. */
LONG     rc;          /*  Return codes. */

rc = GetDriverInfo(hEngObject, ulIndex, hdc);
```

GetDriverInfo Parameter - hEngObject

hEngObject (ULONG) - input
Handle of a DC or bit map.

GetDriverInfo Parameter - ulIndex

ulIndex (ULONG) - input
Has the following settings:

Index=0	hEngObject is a DC handle
Index=1	hEngObject is a bit-map handle

GetDriverInfo Parameter - hdc

hdc (HDC) - input
Device context handle, which identifies the calling DC.

GetDriverInfo Return Value - rc

rc (LONG) - returns
Return codes.

If successful, the function returns the instance pointer associated with the object DC or the presentation driver's handle to the object bit map. If an error is detected, the function returns GPI_ALTERROR.

GetDriverInfo - Parameters

hEngObject (ULONG) - input
Handle of a DC or bit map.

ulIndex (ULONG) - input
Has the following settings:

Index=0	hEngObject is a DC handle
Index=1	hEngObject is a bit-map handle

hdc (HDC) - input
Device context handle, which identifies the calling DC.

rc (LONG) - returns
Return codes.

If successful, the function returns the instance pointer associated with the object DC or the presentation driver's handle to the object bit map. If an error is detected, the function returns GPI_ALTERROR.

GetDriverInfo - Remarks

The parameter hEngObject can be a DC handle or an engine bit-map handle (for example, the source handle, which would be passed to the presentation driver's GreBitblt routine). If hEngObject is a DC, the return code is a pointer (*pInstance*) to the instance data of that DC. If hEngObject is a bit map, the return code is the driver's handle for the bit map (that is the handle returned to the presentation driver when the bit map was created).

GetDriverInfo includes a check to ensure that the object is, or was created by, an instance of a DC for the same device as the DC identified by hdc. If the check is not successful, the function returns GPI_ALTERROR. GetDriverInfo is exported by the graphics engine at ordinal 30 and, if the presentation driver wants to call this function, it must be imported by the driver's module definition file.

GetDriverInfo - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreCloseDC

GreCloseDC - Syntax

Description

GreCloseDC closes a device context.

This function is supported by the graphics engine.

```
#define INCL_GRE_DCS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreCloseDC. */
BOOL     fSuccess;    /*  Return codes. */

fSuccess = GreCloseDC(hdc, pInstance, lFunction);
```

GreCloseDC Parameter - hdc

hdc (HDC) - input

Device context handle.

GreCloseDC Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreCloseDC Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCloseDC.

GreCloseDC Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_IS_SELECTED
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_CODEPAGE
PMERR_INV_COORDINATE
PMERR_INV_DC_TYPE
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_ID
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_SCAN_START

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreCloseDC - Parameters

hdc (HDC) - input
Device context handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCloseDC.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Error codes for conditions that the handling routine is expected to check include:

PMERR_BASE_ERROR
PMERR_BITMAP_IS_SELECTED
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_CODEPAGE
PMERR_INV_COORDINATE
PMERR_INV_DC_TYPE
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_ID
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_SCAN_START

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreCloseDC - Remarks

If the device context is a memory DC that has a bit map selected, the engine deselects the bit map before destroying the DC. This function deletes any visible regions and clip regions selected into the DC.

GreCloseDC - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreCopyDCLoadData

GreCopyDCLoadData - Syntax

Description

GreCopyDCLoadData copies the loaded fonts, bit maps, color table, and default attributes from one DC to another.

This function is supported by the graphics engine.

```
#define INCL_GRE_LCID
#include <os2.h>

HDC      hdc;         /*  Handle of target DC. */
ULONG    flCmd;       /*  Flags indicating which fonts to copy. */
HDC      hdcSrc;      /*  Handle of source DC. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreCopyDCLoadData. */
BOOL     fSuccess;    /*  Return codes. */

fSuccess = GreCopyDCLoadData(hdc, flCmd, hdcSrc,
             pInstance, lFunction);
```

GreCopyDCLoadData Parameter - hdc

hdc (HDC) - input
Handle of target DC.

GreCopyDCLoadData Parameter - flCmd

flCmd (ULONG) - input
Flags indicating which fonts to copy.

Valid flags are:

LCID_RANGE_GPI	GPI
LCID_RANGE_AVIO	AVIO
LCID_RANGE_BOTH	GPI and AVIO

GreCopyDCLoadData Parameter - hdcSrc

hdcSrc (HDC) - input

Handle of source DC.

GreCopyDCLoadData Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreCopyDCLoadData Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCopyDCLoadData.

GreCopyDCLoadData Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

GreCopyDCLoadData - Parameters

hdc (HDC) - input
Handle of target DC.

flCmd (ULONG) - input
Flags indicating which fonts to copy.

Valid flags are:

LCID_RANGE_GPI	GPI
LCID_RANGE_AVIO	AVIO
LCID_RANGE_BOTH	GPI and AVIO

hdcSrc (HDC) - input
Handle of source DC.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCopyDCLoadData.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

GreCopyDCLoadData - Remarks

Device-dependent attributes, such as pick aperture, character cell, and marker cell, are copied only when they have been set by the application. Otherwise, the target device defaults are used.

In response to this call, the graphics engine does the following:

- Transfers the contents of the source DC's lcid table to the target DC's lcid table.
- Translates any bit maps when the devices associated with the source and target DCs are different.
- Calls GreCreateLogicalFont for the target DC, as necessary.
- Transfers the color table and checks that the resulting color table in the target DC is the same as the one in the source DC.
- Resets the source DC using GreResetDC. When the save level of the source DC is not 1, it is restored to the default value before GreResetDC is called. The color table is also reset to the default value.

The values of the GPI handles in the source DC are preserved. This function fails if the target DC lcid table already has lcids set in the range specified. If the target DC does not support LCOL_REALIZABLE (see GreCreateLogColorTable), a warning (PMERR_REALIZE_NOT_SUPPORTED) is raised and the color table is treated as nonrealizable.

This function has the following effect on fonts:

- Any logical font in force on the source DC is reloaded onto the target with the original parameters specified by the application.
- A generic font selected by Match ID can be selected into the target DC provided that it is suitable. For example, an image font cannot be selected for a hardcopy DC.
- Any device font (selected by Match ID) can be selected for the target DC provided that the new device has a font with the same Match ID. If not, the defaults are used. This is managed by the presentation driver. When it is necessary to use a specific font on the new device, the presentation driver should call GreQueryFonts to determine the characteristics and Match ID of the font for the new device and then to reload it.
- A warning is raised when it is not possible to reload a font on association (not on reassociation). This warning is one of the following:

  PMERR_FONT_NOT_LOADED
  PMERR_KERNING_NOT_SUPPORTED

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreCopyDCLoadData - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreCreateBitmap

GreCreateBitmap - Syntax

Description

GreCreateBitmap creates a bit map and returns its handle.

This function is supported by the graphics engine.

```
#define INCL_GRE_DEVSUPPORT
#include <os2.h>

HDC                 hdc;        /*  Device context handle. */
PBITMAPINFOHEADER   pInfoHd;
ULONG               flUsage;
PBYTE               pBitmap;
PBITMAPINFO         paInfo;
PVOID               pInstance;  /*  Pointer to instance data. */
ULONG               lFunction;
HBITMAP             rc;         /*  Return codes. */

rc = GreCreateBitmap(hdc, pInfoHd, flUsage,
       pBitmap, paInfo, pInstance, lFunction);
```

GreCreateBitmap Parameter - hdc

hdc (HDC) - input
Device context handle.

GreCreateBitmap Parameter - pInfoHd

pInfoHd (PBITMAPINFOHEADER) - input
Pointer to BITMAPINFOHEADER or BITMAPINFOHEADER2 structure for new bit map.

Pointer to either a BITMAPINFOHEADER structure:

cbFix	Length in bytes of this structure
cx	Bit-map width
cy	Bit-map height
cPlanes	Number of color planes, 1 if standard format
cBitCount	Number of adjacent color bits per pel

Each plane has ((cx*cBitCount+31)/32*4*cy) bytes.

Or pointer to a BITMAPINFOHEADER2 structure:

cbFix	Length in bytes of this structure.
cx	Bit-map width.
cy	Bit-map height.
cPlanes	Number of color planes, 1 if standard format.
cBitCount	Number of adjacent color bits per pel.

ulCompression

Compression scheme used to store the bit map:

BCA_UNCOMP	Bit map is uncompressed (the only valid value).

cbImage

Length of bit-map storage data in bytes. If the bit map is uncompressed, 0 (default) can be specified for this.

cxResolution

Horizontal component of the resolution of the target device; that is, the resolution of the device for which the bit map is intended in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.

cyResolution

Vertical component of the resolution of the target device; that is, the resolution of the device for which the bit map is intended in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.

cclrUsed

The number of color indexes from the color table that are used by the bit map. If it is 0 (default), all the indexes are used. If it is nonzero, only the first cclrUsed entries in the table are accessed by the system; further entries can be omitted.

For standard formats with a cBitCount of 1, 4, or 8 (and cPlanes=1), any indexes beyond cclrUsed are not valid. For example, a bit map with 64 colors can use the 8-bitcount format without having to supply the other 192 entries in the color table. For the 24-bitcount standard format, cclrUsed is the number of colors used by the bit map.

cclrImportant

Minimum number of color indexes for satisfactory appearance of the bit map. More colors can be used in the bit map. However, it is not necessary to assign them to the device palette. These additional colors can be mapped to the nearest colors available. Zero (default) means that all entries are mapped. For a 24-bitcount standard format, the cclrImportant colors are also listed in the color table relating to this bit map.

usUnits

Units of measure of the horizontal and vertical components of resolution:

BRU_METRIC	(Default) Pels per meter.

usReserved

Reserved field. If present, it must be 0.

usRecording

Recording algorithm, the format in which bit-map data is recorded:

BRA_BOTTOMUP	(Default) Scan lines are recorded from bottom to top.

usRendering

Halftoning algorithm used to record bit-map data that has been digitally halftoned:

BRH_NOTHALFTONED	(Default) Bit-map data not halftoned.
BRH_ERRORDIFFUSION	Error diffusion or damped error diffusion algorithm
BRH_PANDA	Processing algorithm for noncoded document acquisition
BRH_SUPERCIRCLE	Super circle algorithm

cSize1	Size value 1. If BRH_ERRORDIFFUSION is specified in usRendering, cSize1 is the error damping as a percentage in the range 0 - 100. A value of 100% indicates no damping. A value of 0% indicates that any errors are not diffused. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize1 is the x-dimension of the pattern used in pels.	
cSize2	Size value 2. If BRH_ERRORDIFFUSION is specified in usRendering, this parameter is ignored. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize2 is the Y-dimension of the pattern used in pels.	
ulColorEncoding	Color encoding:	
	BCE_RGB	(Default) Each element in the color array is an RGB2 data type.
ulIdentifier	Reserved for applications.	

GreCreateBitmap Parameter - flUsage

flUsage (ULONG) - input
Additional information, used when creating a new bit map. The only defined flag is:

CBM_INIT	When set, the pBitmap and paInfo parameters are used to initialize the newly created bit map. If this flag is not set, the bit map initialization is device dependent. It is assumed that sufficient data is supplied to initialize the whole bit map. Other flags (16 - 31) can be used for special purposes known to be supported by a particular presentation driver.

GreCreateBitmap Parameter - pBitmap

pBitmap (PBYTE) - input
Pointer to bit-map initialization data, starting on a DWORD-aligned address. This data is stored in the order that the coordinates appear on a display screen, that is, the pel in the lower-left corner is the first in the bit map. Pels are scanned to the right and upward from that position. The bits of the first pel are stored beginning with the most significant bits of the first byte. The data for pels in each scan line is packed together tightly. However, all scan lines are padded at the end so that each one begins on a ULONG boundary. That is, 3 bytes of pel data will hold one 24-bit pel, three 8-bit pels, six 4-bit pels, or twenty-four 1-bit pels. If those 3 bytes are the only pel data for that scan line, 1 more byte of zeros would be required to pad the line to a ULONG boundary.

GreCreateBitmap Parameter - paInfo

paInfo (PBITMAPINFO) - input
Pointer to BITMAPINFO or BITMAPINFO2 structure for initialization data.

Pointer to either a BITMAPINFO structure:

cbFix	Length of structure
cx	Bit-map width
cy	Bit-map height
cPlanes	Number of color planes, 1 if standard format
cBitCount	Number of adjacent color bits per pel
argbColor[ ]	Color table array of RGB structures: bBlue

bGreen
bRed

Or to a BITMAPINFO2 structure:

cbFix	Length in bytes of this structure.
cx	Bit-map width.
cy	Bit-map height.
cPlanes	Number of color planes, 1 if standard format.
cBitCount	Number of adjacent color bits per pel.
ulCompression	Compression scheme used to store the bit map: BCA_UNCOMP — Bit map is uncompressed (the only valid value).
cbImage	Length of bit-map storage data in bytes. If the bit map is uncompressed, 0 (default) can be specified for this.
cxResolution	Horizontal component of the resolution of the target device; that is, the resolution of the device for which the bit map is intended in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cyResolution	Vertical component of the resolution of the target device; that is, the resolution of the device for which the bit map is intended in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cclrUsed	The number of color indexes from the color table that are used by the bit map. If it is 0 (default), all the indexes are used. If it is nonzero, only the first cclrUsed entries in the table are accessed by the system; further entries can be omitted. For standard formats with a cBitCount of 1, 4, or 8 (and cPlanes=1), any indexes beyond cclrUsed are not valid. For example, a bit map with 64 colors can use the 8-bitcount format without having to supply the other 192 entries in the color table. For the 24-bitcount standard format, cclrUsed is the number of colors used by the bit map.
cclrImportant	Minimum number of color indexes for satisfactory appearance of the bit map. More colors can be used in the bit map but it is not necessary to assign them to the device palette. These additional colors can be mapped to the nearest colors available. Zero (the default) means that all entries are important. For a 24-bitcount standard format, the cclrImportant colors are also listed in the color table relating to this bit map.
usUnits	Units of measure of the horizontal and vertical components of resolution: BRU_METRIC — (Default) Pels per meter.
usReserved	Reserved field. If present, it must be 0.
usRecording	Recording algorithm, the format in which bit-map data is recorded: BRA_BOTTOMUP — (Default.) Scan lines are recorded from bottom to top.
usRendering	Halftoning algorithm used to record bit-map data that has been digitally halftoned: BRH_NOTHALFTONED — (Default) Bit-map data not halftoned BRH_ERRORDIFFUSION — Error diffusion or damped error diffusion algorithm BRH_PANDA — Processing algorithm for noncoded document acquisition BRH_SUPERCIRCLE — Super circle

cSize1	Size value 1. If BRH_ERRORDIFFUSION is specified in usRendering, cSize1 is the error damping as a percentage in the range 0 - 100. A value of 100% indicates no damping. A value of 0% indicates that any errors are not diffused. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize1 is the X-dimension of the pattern used in pels.	
cSize2	Size value 2. If BRH_ERRORDIFFUSION is specified in usRendering, this parameter is ignored. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize2 is the Y-dimension of the pattern used in pels.	
ulColorEncoding	Color encoding:	
	BCE_RGB	(Default) Each element in the color array is an RGB2 data type.
ulIdentifier	Reserved for application use.	
argbColor[ ]	Color table array of RGB2 structures:	
	bBlue bGreen bRed fcOptions	Reserved. Must be 0.

Note: The same bit-map file format is used for bit maps, icons, and pointers. For details, refer to the *Presentation Manager Programming Reference* .

GreCreateBitmap Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreCreateBitmap Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCreateBitmap.

GreCreateBitmap Return Value - rc

rc (HBITMAP) - returns
Return codes.

On completion, the handling routine must return the handle of the bit map (hbm), or 0 if an error occurs.

Possible Errors Detected: Error codes posted by the engine for this function include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BITMAP_DIMENSION

PMERR_INV_HDC
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SCAN_START
PMERR_INV_USAGE_PARM

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreCreateBitmap - Parameters

hdc (HDC) - input
Device context handle.

pInfoHd (PBITMAPINFOHEADER) - input
Pointer to BITMAPINFOHEADER or BITMAPINFOHEADER2 structure for new bit map.

Pointer to either a BITMAPINFOHEADER structure:

cbFix	Length in bytes of this structure
cx	Bit-map width
cy	Bit-map height
cPlanes	Number of color planes, 1 if standard format
cBitCount	Number of adjacent color bits per pel

Each plane has ((cx*cBitCount+31)/32*4*cy) bytes.

Or pointer to a BITMAPINFOHEADER2 structure:

cbFix	Length in bytes of this structure.
cx	Bit-map width.
cy	Bit-map height.
cPlanes	Number of color planes, 1 if standard format.
cBitCount	Number of adjacent color bits per pel.
ulCompression	Compression scheme used to store the bit map:
	BCA_UNCOMP — Bit map is uncompressed (the only valid value).
cbImage	Length of bit-map storage data in bytes. If the bit map is uncompressed, 0 (default) can be specified for this.
cxResolution	Horizontal component of the resolution of the target device; that is, the resolution of the device for which the bit map is intended in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cyResolution	Vertical component of the resolution of the target device; that is, the resolution of the device for which the bit map is intended in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cclrUsed	The number of color indexes from the color table that are used by the bit map. If it is 0 (default), all the indexes are used. If it is nonzero, only the first cclrUsed entries in the table are accessed by the system; further entries can be omitted. For standard formats with a cBitCount of 1, 4, or 8 (and cPlanes=1), any indexes beyond cclrUsed are not valid. For example, a bit map with 64 colors can use the 8-bitcount format without having to supply the other 192 entries in the color table. For the 24-bitcount standard format, cclrUsed is the number of colors used by the bit map.

cclrImportant — Minimum number of color indexes for satisfactory appearance of the bit map. More colors can be used in the bit map. However, it is not necessary to assign them to the device palette. These additional colors can be mapped to the nearest colors available. Zero (default) means that all entries are mapped. For a 24-bitcount standard format, the cclrImportant colors are also listed in the color table relating to this bit map.

usUnits — Units of measure of the horizontal and vertical components of resolution:

BRU_METRIC	(Default) Pels per meter.

usReserved — Reserved field. If present, it must be 0.

usRecording — Recording algorithm, the format in which bit-map data is recorded:

BRA_BOTTOMUP	(Default) Scan lines are recorded from bottom to top.

usRendering — Halftoning algorithm used to record bit-map data that has been digitally halftoned:

BRH_NOTHALFTONED	(Default) Bit-map data not halftoned.
BRH_ERRORDIFFUSION	Error diffusion or damped error diffusion algorithm
BRH_PANDA	Processing algorithm for noncoded document acquisition
BRH_SUPERCIRCLE	Super circle algorithm

cSize1 — Size value 1. If BRH_ERRORDIFFUSION is specified in usRendering, cSize1 is the error damping as a percentage in the range 0 - 100. A value of 100% indicates no damping. A value of 0% indicates that any errors are not diffused.

If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize1 is the x-dimension of the pattern used in pels.

cSize2 — Size value 2. If BRH_ERRORDIFFUSION is specified in usRendering, this parameter is ignored. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize2 is the Y-dimension of the pattern used in pels.

ulColorEncoding — Color encoding:

BCE_RGB	(Default) Each element in the color array is an RGB2 data type.

ulIdentifier — Reserved for applications.

flUsage (ULONG) - input
Additional information, used when creating a new bit map. The only defined flag is:

CBM_INIT	When set, the pBitmap and paInfo parameters are used to initialize the newly created bit map. If this flag is not set, the bit map initialization is device dependent. It is assumed that sufficient data is supplied to initialize the whole bit map. Other flags (16 - 31) can be used for special purposes known to be supported by a particular presentation driver.

pBitmap (PBYTE) - input

Pointer to bit-map initialization data, starting on a DWORD-aligned address. This data is stored in the order that the coordinates appear on a display screen, that is, the pel in the lower-left corner is the first in the bit map. Pels are scanned to the right and upward from that position. The bits of the first pel are stored beginning with the most significant bits of the first byte. The data for pels in each scan line is packed together tightly. However, all scan lines are padded at the end so that each one begins on a ULONG boundary. That is, 3 bytes of pel data will hold one 24-bit pel, three 8-bit pels, six 4-bit pels, or twenty-four 1-bit pels. If those 3 bytes are the only pel data for that scan line, 1 more byte of zeros would be required to pad the line to a ULONG boundary.

paInfo (PBITMAPINFO) - input

Pointer to BITMAPINFO or BITMAPINFO2 structure for initialization data.

Pointer to either a BITMAPINFO structure:

cbFix	Length of structure
cx	Bit-map width
cy	Bit-map height
cPlanes	Number of color planes, 1 if standard format
cBitCount	Number of adjacent color bits per pel
argbColor[ ]	Color table array of RGB structures: bBlue bGreen bRed

Or to a BITMAPINFO2 structure:

cbFix	Length in bytes of this structure.	
cx	Bit-map width.	
cy	Bit-map height.	
cPlanes	Number of color planes, 1 if standard format.	
cBitCount	Number of adjacent color bits per pel.	
ulCompression	Compression scheme used to store the bit map:	
	BCA_UNCOMP	Bit map is uncompressed (the only valid value).
cbImage	Length of bit-map storage data in bytes. If the bit map is uncompressed, 0 (default) can be specified for this.	
cxResolution	Horizontal component of the resolution of the target device; that is, the resolution of the device for which the bit map is intended in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.	
cyResolution	Vertical component of the resolution of the target device; that is, the resolution of the device for which the bit map is intended in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.	
cclrUsed	The number of color indexes from the color table that are used by the bit map. If it is 0 (default), all the indexes are used. If it is nonzero, only the first cclrUsed entries in the table are accessed by the system; further entries can be omitted. For standard formats with a cBitCount of 1, 4, or 8 (and cPlanes=1), any indexes beyond cclrUsed are not valid. For example, a bit map with 64 colors can use the 8-bitcount format without having to supply the other 192 entries in the color table. For the 24-bitcount standard format, cclrUsed is the number of colors used by the bit map.	
cclrImportant	Minimum number of color indexes for satisfactory appearance of the bit map. More colors can be used in the bit map but it is not necessary to assign them to the device palette. These additional colors can be mapped to the nearest colors available. Zero (the default) means that all entries are important. For a 24-bitcount standard format, the cclrImportant colors are also listed in the color table relating to this bit map.	
usUnits	Units of measure of the horizontal and vertical components of resolution:	
	BRU_METRIC	(Default) Pels per meter.
usReserved	Reserved field. If present, it must be 0.	
usRecording	Recording algorithm, the format in which bit-map data is recorded:	

	BRA_BOTTOMUP	(Default.) Scan lines are recorded from bottom to top.
usRendering	Halftoning algorithm used to record bit-map data that has been digitally halftoned:	
	BRH_NOTHALFTONED	(Default) Bit-map data not halftoned
	BRH_ERRORDIFFUSION	Error diffusion or damped error diffusion algorithm
	BRH_PANDA	Processing algorithm for noncoded document acquisition
	BRH_SUPERCIRCLE	Super circle algorithm
cSize1	Size value 1. If BRH_ERRORDIFFUSION is specified in usRendering, cSize1 is the error damping as a percentage in the range 0 - 100. A value of 100% indicates no damping. A value of 0% indicates that any errors are not diffused. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize1 is the X-dimension of the pattern used in pels.	
cSize2	Size value 2. If BRH_ERRORDIFFUSION is specified in usRendering, this parameter is ignored. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize2 is the Y-dimension of the pattern used in pels.	
ulColorEncoding	Color encoding:	
	BCE_RGB	(Default) Each element in the color array is an RGB2 data type.
ulIdentifier	Reserved for application use.	
argbColor[ ]	Color table array of RGB2 structures:	
	bBlue bGreen bRed fcOptions	Reserved. Must be 0.

Note: The same bit-map file format is used for bit maps, icons, and pointers. For details, refer to the *Presentation Manager Programming Reference* .

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCreateBitmap.

rc (HBITMAP) - returns
Return codes.

On completion, the handling routine must return the handle of the bit map (hbm), or 0 if an error occurs.

Possible Errors Detected: Error codes posted by the engine for this function include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_HDC
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SCAN_START
PMERR_INV_USAGE_PARM

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreCreateBitmap - Remarks

Bit-map size is limited by available memory; the maximum width and height are 64KB. The following standard bit-maps formats are normally used:

Bitcount	Planes
1	1
4	1
8	1
24	1

Display drivers must support at least 4 bits per pel. For other devices, the presentation driver must be able to create and accept any of the standard formats even though they might not use the color information.

The DC handle supplied to this function must never be NULL. Bit maps always belong to some device. The bit map is created on the device specified and can be selected to a different device later as the engine can handle the transfer of bits from one device to another. When the value specified for cPlanes or cBitcount is incompatible with the physical device specified by the DC handle, an error is raised.

GreCreateBitmap - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreCreateJournalFile

GreCreateJournalFile - Syntax

Description

GreCreateJournalFile creates a journal file on disk. The presentation driver calls GreCreateJournal when responding to DevEscape DEVESC_STARTDOC.

This function is supported by the graphics engine.

```
#define INCL_GRE_JOURNALING
#include <os2.h>

PSZ      pszFileName;  /*  Pointer to a string containing the file name. */
ULONG    flOption;     /*  Options. */
```

```
ULONG    cSize;         /*  Size. */
PVOID    pInstance;     /*  Pointer to instance data. */
ULONG    lFunction;     /*  High-order WORD=flags; low-order WORD=NGreCreateJournalFile. */
ULONG    rc;            /*  Return codes. */

rc = GreCreateJournalFile(pszFileName, flOption,
       cSize, pInstance, lFunction);
```

GreCreateJournalFile Parameter - pszFileName

pszFileName (PSZ) - input
Pointer to a string containing the file name.

GreCreateJournalFile Parameter - flOption

flOption (ULONG) - input
Options.

Defined values are:

JNL_TEMP_FILE	An ordinary temporary journal file is created. The pszFileName parameter is ignored.
JNL_PERM_FILE	A permanent journal file is created. The pszFileName parameter points to a fully qualified path or file name.
JNL_ENGINERAM_FILE	A memory journal file is created in shared memory allocated by the engine. The pszFileName parameter is ignored.
JNL_USERRAM_FILE	A memory journal file is created in memory supplied by the caller. The location in memory is identified by the pointer passed in pszFileName.
JNL_DRAW_OPTIMIZATION	If set, the process control flag PCTL_DRAW is reset (optimization occurs). The Draw bit is not affected.
JNL_BOUNDS_OPTIMIZATION	If set, the process control flag PCTL_BOUND is reset (BOUNDS is turned OFF). Otherwise, current behavior continues.

GreCreateJournalFile Parameter - cSize

cSize (ULONG) - input
Size.

If greater than 0, cSize is an indication as to how large the file must be. If flOption is JNL_USERRAM_FILE, cSize must be greater than 0 and is the size of the buffer, which cannot be extended.

If cSize is 0, the calling routine does not know the size of the file.

GreCreateJournalFile Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreCreateJournalFile Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCreateJournalFile.

GreCreateJournalFile Return Value - rc

rc (ULONG) - returns
Return codes.

This function returns the journal file handle (ULONG), or if an error occurs, it returns NULL.

Possible Errors Detected: If this function fails, the graphics engine will set one of the following error codes:

PMERR_BASE_ERROR
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_JOURNAL_OPTION
PMERR_RAM_JNL_FILE_TOO_SMALL

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreCreateJournalFile - Parameters

pszFileName (PSZ) - input
Pointer to a string containing the file name.

flOption (ULONG) - input
Options.

Defined values are:

JNL_TEMP_FILE	An ordinary temporary journal file is created. The pszFileName parameter is ignored.
JNL_PERM_FILE	A permanent journal file is created. The pszFileName parameter points to a fully qualified path or file name.
JNL_ENGINERAM_FILE	A memory journal file is created in shared memory allocated by the engine. The pszFileName parameter is ignored.
JNL_USERRAM_FILE	A memory journal file is created in memory supplied by the caller. The location in memory is identified by the

	pointer passed in pszFileName.
JNL_DRAW_OPTIMIZATION	If set, the process control flag PCTL_DRAW is reset (optimization occurs). The Draw bit is not affected.
JNL_BOUNDS_OPTIMIZATION	If set, the process control flag PCTL_BOUND is reset (BOUNDS is turned OFF). Otherwise, current behavior continues.

cSize (ULONG) - input
Size.

If greater than 0, cSize is an indication as to how large the file must be. If flOption is JNL_USERRAM_FILE, cSize must be greater than 0 and is the size of the buffer, which cannot be extended.

If cSize is 0, the calling routine does not know the size of the file.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCreateJournalFile.

rc (ULONG) - returns
Return codes.

This function returns the journal file handle (ULONG), or if an error occurs, it returns NULL.

Possible Errors Detected: If this function fails, the graphics engine will set one of the following error codes:

PMERR_BASE_ERROR
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_JOURNAL_OPTION
PMERR_RAM_JNL_FILE_TOO_SMALL

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreCreateJournalFile - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreCreateLogicalFont

GreCreateLogicalFont - Syntax

Description

GreCreateLogicalFont sets the local identifier (lcid) for a logical font.

This function is supported by the graphics engine.

```
#define INCL_GRE_FONTS
#include <os2.h>

HDC         hdc;         /*  Device context handle. */
LONG        lcid;        /*  Local identifier that is to be assigned to the font. */
PSTR8       pchName;     /*  Pointer. */
PFATTRS     pAttrs;      /*  Pointer. */
PVOID       pInstance;   /*  Pointer to instance data. */
ULONG       lFunction;   /*  High-order WORD=flags; low-order WORD=NGreCreateLogicalFont. */
LONG        rc;          /*  Return Codes. */

rc = GreCreateLogicalFont(hdc, lcid, pchName,
       pAttrs, pInstance, lFunction);
```

GreCreateLogicalFont Parameter - hdc

hdc (HDC) - input
Device context handle.

GreCreateLogicalFont Parameter - lcid

lcid (LONG) - input
Local identifier that is to be assigned to the font.

GreCreateLogicalFont Parameter - pchName

pchName (PSTR8) - input
Pointer.

Pointer to an eight-character name used to describe the logical font.

GreCreateLogicalFont Parameter - pAttrs

pAttrs (PFATTRS) - input
Pointer.

Pointer to a FATTRS structure containing the font attributes:

usRecordLength	Length in bytes of this structure
fsSelection	Flags that can be set to define those features to be simulated by the system:
FATTR_SEL_ITALIC	Italic chara

cters are requir ed.

FATTR_SEL_UNDERSCORE

Under score d chara cters are requir ed.

FATTR_SEL_STRIKEOUT

Strikeo ut chara cters are requir ed.

FATTR_SEL_BOLD

Bold chara cters are requir ed.

lMatch

The match number for the font. GreQueryFonts returns a unique match number for each font that, together with szFaceName, can be used for font selection. If the match number is negative, a device font is selected. If it is positive, an engine font is selected.

szFaceName

The typeface name to which the font is designed. If the szFaceName is provided, an attempt is made to select the font with that face name. If the szFaceName is blank (0 length or NULL), then a default font is used.

idRegistry

The Registry number for the font. This should be set to the value returned by GreQueryFonts.

usCodePage

Defines the code page supported by the font.

lMaxBaselineExt

The required (average) height above the baseline for uppercase characters in world coordinates. For outline and transformable fonts (FATTR_FONTUSE_OUTLINE and FATTR_FONTUSE_TRANSFORM), lMaxBaselineExt should be set to 0.

lAveCharWidth

For image fonts, the required average intercharacter increment for the font in world coordinates. For outline and transformable fonts (FATTR_FONTUSE_OUTLINE and FATTR_FONTUSE_TRANSFORM), lAveCharWidth should be set to 0.

fsType

Type indicator. Setting fsType to FATTR_TYPE_KERNING indicates that kerning is to be used if the font provides kerning information.

fsFontUse

Flags containing information about how the font is to be related to the character attributes:

FATTR_FONTUSE_NOMIX

Specifies that permissive mixing is allowed when the font is used. The engine can ignore any interaction with graphics primitives and can use *overpaint* and *leave alone* as the foreground and background mixes rather than using the current mix attributes.

FATTR_FONTUSE_OUTLINE

Specifies that the font must be an outline font. When the font is not defined by lMatch and FATTR_FONTUSE_OUTLINE is specified, the system searches for an outline font. If the search fails, a default font is selected. When the font is not defined by lMatch and FATTR_FONTUSE_OUTLINE is not set, the system searches for an image font that matches lMaxBaselineExt and

lAveCharWidth. If this fails, it searches for an outline font with the required szFaceName and fsSelection flags.

FATTR_FONTUSE_TRANSFORMABLE
Specifies that the font must be transformable (that is, it can be scaled, rotated, and sheared). Transformable fonts are used only in CM_MODE3. Nontransformable fonts can be used in CM_MODE1 or CM_MODE2 but not in CM_MODE3.

GreCreateLogicalFont Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreCreateLogicalFont Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCreateLogicalFont.

GreCreateLogicalFont Return Value - rc

rc (LONG) - returns
Return Codes.

This function returns a LONG value indicating whether the font has been matched:

FONT_DEFAULT	The font was not matched. The default font is to be used.
FONT_MATCH	The font has been matched successfully.
GPI_ERROR	Error.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CODEPAGE
PMERR_INV_COORD_SPACE
PMERR_INV_EXTENDED_LCID
PMERR_INV_FONT_ATTRS
PMERR_INV_FONTDEF
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SETID
PMERR_SETID_IN_USE

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreCreateLogicalFont - Parameters

hdc (HDC) - input
Device context handle.

lcid (LONG) - input
Local identifier that is to be assigned to the font.

pchName (PSTR8) - input
Pointer.

Pointer to an eight-character name used to describe the logical font.

pAttrs (PFATTRS) - input
Pointer.

Pointer to a FATTRS structure containing the font attributes:

usRecordLength	Length in bytes of this structure
fsSelection	Flags that can be set to define those features to be simulated by the system:
	FATTR_SEL_ITALIC — Italic characters are required.
	FATTR_SEL_UNDERSCORE — Underscored characters are required.
	FATTR_SEL_STRIKEOUT — Strikeout characters are required.
	FATTR_SEL_BOLD — Bold characters are required.
lMatch	The match number for the font. GreQueryFonts returns a unique match number for each font that, together with szFaceName, can be used for font selection. If the match number is negative, a device font is selected. If it is positive, an engine font is selected.
szFaceName	The typeface name to which the font is designed. If the szFaceName is provided, an attempt is made to select the font with that face name. If the szFaceName is blank (0 length or NULL), then a default font is used.
idRegistry	The Registry number for the font. This should be set to the value returned by GreQueryFonts.
usCodePage	Defines the code page supported by the font.
lMaxBaselineExt	The required (average) height above the baseline for uppercase characters in world coordinates. For outline and transformable fonts

(FATTR_FONTUSE_OUTLINE and FATTR_FONTUSE_TRANSFORM), lMaxBaselineExt should be set to 0.

lAveCharWidth — For image fonts, the required average intercharacter increment for the font in world coordinates. For outline and transformable fonts (FATTR_FONTUSE_OUTLINE and FATTR_FONTUSE_TRANSFORM), lAveCharWidth should be set to 0.

fsType — Type indicator. Setting fsType to FATTR_TYPE_KERNING indicates that kerning is to be used if the font provides kerning information.

fsFontUse — Flags containing information about how the font is to be related to the character attributes:

FATTR_FONTUSE_NOMIX
: Specifies that permissive mixing is allowed when the font is used. The engine can ignore any interaction with graphics primitives and can use *overpaint* and *leave alone* as the foreground and background mixes rather than using the current mix attributes.

FATTR_FONTUSE_OUTLINE
: Specifies that the font must be an outline font. When the font is not defined by lMatch and FATTR_FONTUSE_OUTLINE is specified, the system searches for an outline font. If the search fails, a default font is selected. When the font is not defined by lMatch and FATTR_FONTUSE_OUTLINE is not set, the system searches for an image font that matches lMaxBaselineExt and lAveCharWidth. If this fails, it searches for an outline font with the required szFaceName and fsSelection flags.

FATTR_FONTUSE_TRANSFORMABLE
: Specifies that the font must be transformable (that is, it can be scaled, rotated, and sheared). Transformable fonts are used only in CM_MODE3. Nontransformable fonts can be used in CM_MODE1 or CM_MODE2 but not in CM_MODE3.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreCreateLogicalFont.

rc (LONG) - returns
Return Codes.

This function returns a LONG value indicating whether the font has been matched:

FONT_DEFAULT	The font was not matched. The default font is to be used.
FONT_MATCH	The font has been matched successfully.
GPI_ERROR	Error.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CODEPAGE
PMERR_INV_COORD_SPACE
PMERR_INV_EXTENDED_LCID
PMERR_INV_FONT_ATTRS
PMERR_INV_FONTDEF
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_LENGTH_OR_COUNT

PMERR_INV_SETID
PMERR_SETID_IN_USE

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreCreateLogicalFont - Remarks

The parameters passed to the function identify the name and attributes of the required font. The graphics engine selects a font from the list of available fonts that provides the best match for the font attributes addressed by pAttrs.

The selection is made in one of two ways:

- If the lMatch attribute is nonzero, the calling program has already determined (from a call to GreQueryFonts) which font it requires. In this case, the graphics engine selects the font identified by the lMatch and szFaceName attributes (or, if szFaceName is not specified, lMatch alone).
- If lMatch is 0 or a suitable match could not be found, the system examines the other fields in the attributes structure and selects the font that gives the best match.

If no match is found, the default font is used. When assigned, the system will not change the relationship of a specific lcid to a specific font.

The interaction between fonts and character attributes depends on the state of the FATTRS_FONTUSE_TRANSFORMABLE flag in the font attributes structure. When this flag is set:

- The size of the characters is determined by the values of the character attributes at the time that the characters are drawn.
- The characters are positioned, rotated, and sheared, as required.
- No checking is done.
- Any transformation is performed by mapping the box defined by the FONTMETRICS parameters, xMaxCharInc and yEmHeight, to the character box under the influence of character angle and shear.

When the FATTRS_FONTUSE_TRANSFORMABLE flag is not set, the lAveCharWidth and lBaselineExt parameters in the font attributes structure define the size of the font to be used. The character box attribute has no effect.

Transformable fonts cannot be used in Character Modes 1 and 2. Nontransformable fonts cannot be used in Character Mode 3. If the font is not compatible with the character mode, the engine raises an error when the presentation driver attempts to draw characters. The characteristics of the character modes are:

CM_MODE1	The position of characters after the first character is determined by the font metrics information. Character box, angle, shear, extra, break extra, and spacing are ignored.
CM_MODE2	The position of characters is determined by the font metrics information and the character attributes. Characters are not scaled, rotated, or sheared.
CM_MODE3	The position of characters is determined by the font metrics information and all character attributes. Characters can be scaled, rotated, and sheared.

Positioning is performed by using the character reference point defined within the font. When characters that are not hollow are drawn using an outline font, they are filled using the character foreground color and mix.

GreCreateLogicalFont - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreDeleteBitmap

GreDeleteBitmap - Syntax

Description

GreDeleteBitmap deletes the bit map identified by hbm. If the bit map is currently selected into a DC, GreDeleteBitmap returns an error.

This function is supported by the graphics engine.

```
#define INCL_GRE_DEVSUPPORT
#include <os2.h>

HBITMAP     hbm;         /*  Bit-map handle. */
PVOID       pInstance;   /*  Pointer to instance data. */
ULONG       lFunction;   /*  High-order WORD=flags; low-order WORD=NGreDeleteBitmap. */
BOOL        fSuccess;    /*  Return codes. */

fSuccess = GreDeleteBitmap(hbm, pInstance,
             lFunction);
```

GreDeleteBitmap Parameter - hbm

hbm (HBITMAP) - input
Bit-map handle.

GreDeleteBitmap Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreDeleteBitmap Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeleteBitmap.

GreDeleteBitmap Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BITMAP_IS_SELECTED
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HBITMAP_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SCAN_START

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreDeleteBitmap - Parameters

hbm (HBITMAP) - input
Bit-map handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeleteBitmap.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BITMAP_IS_SELECTED
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HBITMAP_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SCAN_START

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreDeleteBitmap - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreDeleteJournalFile

GreDeleteJournalFile - Syntax

Description

GreDeleteJournalFile deletes a journal file and frees any objects associated with the journal file handle (such as compatible DCs, private clone regions or bit maps, and global memory segments).

This function is supported by the graphics engine.

```
#define INCL_GRE_JOURNALING
#include <os2.h>

ULONG    hJournal;   /*  Journal file handle. */
PVOID    pInstance;  /*  Pointer to instance data. */
ULONG    lFunction;  /*  High-order WORD=flags; low-order WORD=NGreDeleteJournalFile. */
BOOL     fSuccess;   /*  Return codes. */

fSuccess = GreDeleteJournalFile(hJournal,
            pInstance, lFunction);
```

GreDeleteJournalFile Parameter - hJournal

hJournal (ULONG) - input
Journal file handle.

GreDeleteJournalFile Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreDeleteJournalFile Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeleteJournalFile.

GreDeleteJournalFile Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_BITMAP_IS_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_CHAR_DIRECTION_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CHAR_SET_ATTR
PMERR_INV_CHAR_SHEAR_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_DC_TYPE
PMERR_INV_GEOM_LINE_WIDTH_ATTR
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_HJOURNAL
PMERR_INV_HRGN
PMERR_INV_ID
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_END_ATTR
PMERR_INV_LINE_JOIN_ATTR
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_LINE_WIDTH_ATTR
PMERR_INV_MARKER_SET_ATTR
PMERR_INV_MARKER_SYMBOL_ATTR
PMERR_INV_METAFILE
PMERR_INV_MIX_ATTR
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_PRIMITIVE_TYPE
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_SCAN_START
PMERR_INV_SETID
PMERR_JFILE_BUSY
PMERR_REGION_IS_CLIP_REGION
PMERR_UNSUPPORTED_ATTR

PMERR_UNSUPPORTED_ATTR_VALUE

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreDeleteJournalFile - Parameters

hJournal (ULONG) - input
Journal file handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeleteJournalFile.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_BITMAP_IS_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_CHAR_DIRECTION_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CHAR_SET_ATTR
PMERR_INV_CHAR_SHEAR_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_DC_TYPE
PMERR_INV_GEOM_LINE_WIDTH_ATTR
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_HJOURNAL
PMERR_INV_HRGN
PMERR_INV_ID
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_END_ATTR
PMERR_INV_LINE_JOIN_ATTR
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_LINE_WIDTH_ATTR
PMERR_INV_MARKER_SET_ATTR
PMERR_INV_MARKER_SYMBOL_ATTR
PMERR_INV_METAFILE
PMERR_INV_MIX_ATTR
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT

PMERR_INV_PRIMITIVE_TYPE
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_SCAN_START
PMERR_INV_SETID
PMERR_JFILE_BUSY
PMERR_REGION_IS_CLIP_REGION
PMERR_UNSUPPORTED_ATTR
PMERR_UNSUPPORTED_ATTR_VALUE

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreDeleteJournalFile - Remarks

When the handle belongs to a temporary file, the file is also deleted. Finally, the file handle itself is freed.

GreDeleteJournalFile - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreDeleteSetId

GreDeleteSetId - Syntax

Description

GreDeleteSetId deletes the character set, marker set, or pattern set identified by lcid.

This function is supported by the graphics engine.

```
#define INCL_GRE_SETID
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
LONG     lcid;        /*  Local identifier. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreDeleteSetId. */
BOOL     fSuccess;    /*  Return codes. */

fSuccess = GreDeleteSetId(hdc, lcid, pInstance,
             lFunction);
```

GreDeleteSetId Parameter - hdc

hdc (HDC) - input
Device context handle.

GreDeleteSetId Parameter - lcid

lcid (LONG) - input
Local identifier.

When identifying a bit map, only the lcid is deleted. The bit map will have no LCID but it will still exist. When lcid=LCID_ALL, all loaded graphics local identifiers such as logical fonts and bit-map IDs are destroyed. In this case, AVIO fonts are unaffected and can only be explicitly deleted.

LCID_AVIO_1, LCID_AVIO_2, and LCID_AVIO_3 represent AVIO fonts 1, 2, and 3, respectively.

GreDeleteSetId Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreDeleteSetId Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeleteSetId.

GreDeleteSetId Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED

PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CODEPAGE
PMERR_INV_COORD_SPACE
PMERR_INV_EXTENDED_LCID
PMERR_INV_FONTDEF
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SETID
PMERR_SETID_IN_USE
PMERR_SETID_NOT_FOUND

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreDeleteSetId - Parameters

hdc (HDC) - input
Device context handle.

lcid (LONG) - input
Local identifier.

When identifying a bit map, only the lcid is deleted. The bit map will have no LCID but it will still exist. When lcid=LCID_ALL, all loaded graphics local identifiers such as logical fonts and bit-map IDs are destroyed. In this case, AVIO fonts are unaffected and can only be explicitly deleted.

LCID_AVIO_1, LCID_AVIO_2, and LCID_AVIO_3 represent AVIO fonts 1, 2, and 3, respectively.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreDeleteSetId.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CODEPAGE
PMERR_INV_COORD_SPACE
PMERR_INV_EXTENDED_LCID
PMERR_INV_FONTDEF
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SETID
PMERR_SETID_IN_USE
PMERR_SETID_NOT_FOUND

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreDeleteSetId - Remarks

Base sets cannot be deleted. An error is logged if GreDeleteSetId is called to delete the current character, marker, or pattern set.

GreDeleteSetId - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreGetAttributes

GreGetAttributes - Syntax

Description

This function returns the current value of the attributes indicated in flAttrsMask.

This function is supported by the graphics engine.

```
#define INCL_GRE_DEVSUPPORT
#include <os2.h>

HDC         hdc;           /*  Device context handle. */
LONG        lPrimType;     /*  Bundle primitive type. */
ULONG       flAttrsMask;   /*  Attribute mask. */
PBUNDLE     pAttrs;        /*  The returned attribute value (bundle). */
PVOID       pInstance;     /*  Pointer to instance data. */
ULONG       lFunction;     /*  High-order WORD=flags; low-order WORD=NGreGetAttributes. */
LONG        rc;

rc = GreGetAttributes(hdc, lPrimType, flAttrsMask,
       pAttrs, pInstance, lFunction);
```

GreGetAttributes Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetAttributes Parameter - lPrimType

lPrimType (LONG) - input
Bundle primitive type.

Indicates the bundle type. Valid primitive values are:

PRIM_LINE	Line attribute bundle
PRIM_CHAR	Character attribute bundle
PRIM_MARKER	Marker attribute bundle
PRIM_AREA	Pattern attribute bundle
PRIM_IMAGE	Image attribute bundle

GreGetAttributes Parameter - flAttrsMask

flAttrsMask (ULONG) - input
Attribute mask.

Specifies the attributes to be returned. This mask contains a bit corresponding to each attribute in the bundle record. For each set bit, the graphics engine returns the corresponding attribute values and default mask bits.

GreGetAttributes Parameter - pAttrs

pAttrs (PBUNDLE) - input
The returned attribute value (bundle).

Pointer to the fixed format bundle record containing the attributes returned. The only fields that are updated are those whose corresponding flags in flAttrsMask have been set.

GreGetAttributes Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetAttributes Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetAttributes.

GreGetAttributes Return Value - rc

rc (LONG) - returns
This function returns the default attribute bit mask. Only bits with corresponding set bits in flAttrsMask are updated. Otherwise, this function returns the error, GPI_ALTERROR.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC

GreGetAttributes - Parameters

hdc (HDC) - input
Device context handle.

lPrimType (LONG) - input
Bundle primitive type.

Indicates the bundle type. Valid primitive values are:

PRIM_LINE	Line attribute bundle
PRIM_CHAR	Character attribute bundle
PRIM_MARKER	Marker attribute bundle
PRIM_AREA	Pattern attribute bundle
PRIM_IMAGE	Image attribute bundle

flAttrsMask (ULONG) - input
Attribute mask.

Specifies the attributes to be returned. This mask contains a bit corresponding to each attribute in the bundle record. For each set bit, the graphics engine returns the corresponding attribute values and default mask bits.

pAttrs (PBUNDLE) - input
The returned attribute value (bundle).

Pointer to the fixed format bundle record containing the attributes returned. The only fields that are updated are those whose corresponding flags in flAttrsMask have been set.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetAttributes.

rc (LONG) - returns
This function returns the default attribute bit mask. Only bits with corresponding set bits in flAttrsMask are updated. Otherwise, this function returns the error, GPI_ALTERROR.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC

GreGetAttributes - Remarks

When a specified attribute is currently set to its default value, the corresponding flag in the returned defaults mask is set and the returned value for this attribute is undefined.

The graphics engine either:

- Returns the value of each specified attribute and resets the corresponding bit in the returned mask, or
- Sets the bit in the returned mask to indicate that the specified attribute is set to its default. Note that the corresponding value in the attribute buffer is *not* valid and is likely to have been overwritten by the engine.

GreGetAttributes - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreGetBitmapDimension

GreGetBitmapDimension - Syntax

Description

GreGetBitmapDimension renders height and width values for the bit map indicated by hbm.

This function is supported by the graphics engine.

```
#define INCL_GRE_DEVSUPPORT
#include <os2.h>

HBITMAP     hbm;          /*  Bit-map handle. */
PSIZEL      pDimension;   /*  Pointer to width and height values in 0.1mm units. */
PVOID       pInstance;    /*  Pointer to instance data. */
ULONG       lFunction;    /*  High-order WORD=flags; low-order WORD=NGreGetBitmapDimension. */
BOOL        fSuccess;     /*  Return codes. */

fSuccess = GreGetBitmapDimension(hbm, pDimension,
             pInstance, lFunction);
```

GreGetBitmapDimension Parameter - hbm

hbm (HBITMAP) - input
Bit-map handle.

GreGetBitmapDimension Parameter - pDimension

pDimension (PSIZEL) - input
Pointer to width and height values in 0.1mm units.

GreGetBitmapDimension Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetBitmapDimension Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetBitmapDimension.

GreGetBitmapDimension Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BITMAP_IS_SELECTED
PMERR_HBITMAP_BUSY
PMERR_INV_HBITMAP

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreGetBitmapDimension - Parameters

hbm (HBITMAP) - input
Bit-map handle.

pDimension (PSIZEL) - input
Pointer to width and height values in 0.1mm units.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input

High-order WORD=flags; low-order WORD=NGreGetBitmapDimension.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BITMAP_IS_SELECTED
PMERR_HBITMAP_BUSY
PMERR_INV_HBITMAP

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreGetBitmapDimension - Remarks

These are values that have been set by a previous call to GreSetBitmapDimension. They are not used by the system.

GreGetBitmapDimension - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreGetBitmapParameters

GreGetBitmapParameters - Syntax

Description

GreGetBitmapParameters returns, in the buffer addressed by pInfoHd, header information for the specified bit map.

This function is supported by the graphics engine.

```
#define INCL_GRE_DEVSUPPORT
#include <os2.h>

HBITMAP              hbm;         /*  Bit-map handle. */
PBITMAPINFOHEADER    pInfoHd;     /*  Pointer. */
PVOID                pInstance;   /*  Pointer to instance data. */
ULONG                lFunction;   /*  High-order WORD=flags; low-order WORD=NGreGetBitmapParameters. */
BOOL                 fSuccess;    /*  Return codes. */
```

```
fSuccess = GreGetBitmapParameters(hbm, pInfoHd,
             pInstance, lFunction);
```

GreGetBitmapParameters Parameter - hbm

hbm (HBITMAP) - input
Bit-map handle.

GreGetBitmapParameters Parameter - pInfoHd

pInfoHd (PBITMAPINFOHEADER) - input
Pointer.

Pointer to a BITMAPINFOHEADER or to a BITMAPINFOHEADER2 structure where the returned information is stored.

Pointer to a BITMAPINFOHEADER structure:

cbFix	Length in bytes of this structure
cx	Bit-map width
cy	Bit-map height
cPlanes	Number of color planes, 1 if standard format
cBitCount	Number of adjacent color bits per pel

Each plane has ((cx*cBitCount+31)/32*4*cy) bytes.

Pointer to a BITMAPINFOHEADER2 structure:

cbFix	Length in bytes of this structure.
cx	Bit-map width.
cy	Bit-map height.
cPlanes	Number of color planes, 1 if standard format.
cBitCount	Number of adjacent color bits per pel.
ulCompression	Compression scheme used to store the bit map: BCA_UNCOMP — Bit map is uncompressed (the only valid value).
cbImage	Length of bit map storage data in bytes. If the bit map is uncompressed, 0 (default) can be specified for this.
cxResolution	Horizontal component of the resolution of the target device; that is, the resolution of the device for which the bit map is intended in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cyResolution	Vertical component of the resolution of the target device; that is, the resolution of the device for which the bit map is intended in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cclrUsed	The number of color indexes from the color table that are used by the bit map. If it is 0 (default), all the indexes are used. If it is nonzero, only the first cclrUsed entries in the table are accessed by the system; further entries can be omitted.

For standard formats with a cBitCount of 1, 4, or 8 (and cPlanes=1), any indexes beyond cclrUsed are not valid. For example, a bit map with 64 colors can use the eight-bitcount format without having to supply the other 192 entries in the color table. For the 24-bitcount standard format, cclrUsed is the number of colors used by the bit map.

cclrImportant

Minimum number of color indexes for satisfactory appearance of the bit map. More colors can be used in the bit map. However, it is not necessary to assign them to the device palette. These additional colors can be mapped to the nearest colors available. Zero (default) means that all entries are mapped. For a 24-bitcount standard format, the cclrImportant colors are also listed in the color table relating to this bit map.

usUnits

Units of measure of the horizontal and vertical components of resolution:

BRU_METRIC	(Default) Pels per meter.

usReserved

Reserved field. If present, it must be 0.

usRecording

Recording algorithm, the format in which bit-map data is recorded:

BRA_BOTTOMUP	(Default) Scan lines are recorded from bottom to top.

usRendering

Halftoning algorithm used to record bit-map data that has been digitally halftoned:

BRH_NOTHALFTONED	(Default) Bit-map data not halftoned.
BRH_ERRORDIFFUSION	Error diffusion or damped error diffusion algorithm.
BRH_PANDA	Processing algorithm for noncoded document acquisition
BRH_SUPERCIRCLE	Super circle algorithm

cSize1

Size value 1. If BRH_ERRORDIFFUSION is specified in usRendering, cSize1 is the error damping as a percentage in the range 0 - 100. A value of 100% indicates no damping. A value of 0% indicates that any errors are not diffused.

If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize1 is the X-dimension of the pattern used, in pels.

cSize2

Size value 2. If BRH_ERRORDIFFUSION is specified in usRendering, this parameter is ignored. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize2 is the Y-dimension of the pattern used, in pels.

ulColorEncoding

Color encoding:

BCE_RGB	(Default) Each element in the color array is an RGB2 data type.

ulIdentifier

Reserved for application use.

GreGetBitmapParameters Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetBitmapParameters Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetBitmapParameters.

GreGetBitmapParameters Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BITMAP_IS_SELECTED
PMERR_HBITMAP_BUSY
PMERR_INV_HBITMAP

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreGetBitmapParameters - Parameters

hbm (HBITMAP) - input
Bit-map handle.

pInfoHd (PBITMAPINFOHEADER) - input
Pointer.

Pointer to a BITMAPINFOHEADER or to a BITMAPINFOHEADER2 structure where the returned information is stored.

Pointer to a BITMAPINFOHEADER structure:

cbFix	Length in bytes of this structure
cx	Bit-map width
cy	Bit-map height
cPlanes	Number of color planes, 1 if standard format
cBitCount	Number of adjacent color bits per pel

Each plane has ((cx*cBitCount+31)/32*4*cy) bytes.

Pointer to a BITMAPINFOHEADER2 structure:

cbFix	Length in bytes of this structure.

cx	Bit-map width.
cy	Bit-map height.
cPlanes	Number of color planes, 1 if standard format.
cBitCount	Number of adjacent color bits per pel.
ulCompression	Compression scheme used to store the bit map: BCA_UNCOMP — Bit map is uncompressed (the only valid value).
cbImage	Length of bit map storage data in bytes. If the bit map is uncompressed, 0 (default) can be specified for this.
cxResolution	Horizontal component of the resolution of the target device; that is, the resolution of the device for which the bit map is intended in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cyResolution	Vertical component of the resolution of the target device; that is, the resolution of the device for which the bit map is intended in the units specified by usUnits. This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.
cclrUsed	The number of color indexes from the color table that are used by the bit map. If it is 0 (default), all the indexes are used. If it is nonzero, only the first cclrUsed entries in the table are accessed by the system; further entries can be omitted. For standard formats with a cBitCount of 1, 4, or 8 (and cPlanes=1), any indexes beyond cclrUsed are not valid. For example, a bit map with 64 colors can use the eight-bitcount format without having to supply the other 192 entries in the color table. For the 24-bitcount standard format, cclrUsed is the number of colors used by the bit map.
cclrImportant	Minimum number of color indexes for satisfactory appearance of the bit map. More colors can be used in the bit map. However, it is not necessary to assign them to the device palette. These additional colors can be mapped to the nearest colors available. Zero (default) means that all entries are mapped. For a 24-bitcount standard format, the cclrImportant colors are also listed in the color table relating to this bit map.
usUnits	Units of measure of the horizontal and vertical components of resolution: BRU_METRIC — (Default) Pels per meter.
usReserved	Reserved field. If present, it must be 0.
usRecording	Recording algorithm, the format in which bit-map data is recorded: BRA_BOTTOMUP — (Default) Scan lines are recorded from bottom to top.
usRendering	Halftoning algorithm used to record bit-map data that has been digitally halftoned: BRH_NOTHALFTONED — (Default) Bit-map data not halftoned. BRH_ERRORDIFFUSION — Error diffusion or damped error diffusion algorithm. BRH_PANDA — Processing algorithm for

		noncoded document acquisition
	BRH_SUPERCIRCLE	Super circle algorithm
cSize1	Size value 1. If BRH_ERRORDIFFUSION is specified in usRendering, cSize1 is the error damping as a percentage in the range 0 - 100. A value of 100% indicates no damping. A value of 0% indicates that any errors are not diffused. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize1 is the X-dimension of the pattern used, in pels.	
cSize2	Size value 2. If BRH_ERRORDIFFUSION is specified in usRendering, this parameter is ignored. If the BRH_PANDA or BRH_SUPERCIRCLE is specified, cSize2 is the Y-dimension of the pattern used, in pels.	
ulColorEncoding	Color encoding:	
	BCE_RGB	(Default) Each element in the color array is an RGB2 data type.
ulIdentifier	Reserved for application use.	

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetBitmapParameters.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BITMAP_IS_SELECTED
PMERR_HBITMAP_BUSY
PMERR_INV_HBITMAP

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreGetBitmapParameters - Remarks

The header information is returned as a BITMAPINFOHEADER or BITMAPINFOHEADER2 structure and gives details such as the width, height, number of planes, and number of bits per pel.

GreGetBitmapParameters - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreGetDefaultArcParameters

GreGetDefaultArcParameters - Syntax

Description

GreGetDefaultArcParameters stores the default arc parameters in the buffer addressed by pArcParms.

This function is supported by the graphics engine.

```
#define INCL_GRE_DEVSUPPORT
#include <os2.h>

HDC           hdc;         /*  Device context handle. */
PARCPARAMS    pArcParms;   /*  Pointer to ARCPARAMS structure. */
PVOID         pInstance;   /*  Pointer to instance data. */
ULONG         lFunction;   /*  High-order WORD=flags; low-order WORD=NGreGetDefaultArcParameters. */
BOOL          fSuccess;    /*  Return codes. */

fSuccess = GreGetDefaultArcParameters(hdc,
             pArcParms, pInstance, lFunction);
```

GreGetDefaultArcParameters Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetDefaultArcParameters Parameter - pArcParms

pArcParms (PARCPARAMS) - input
Pointer to ARCPARAMS structure.

GreGetDefaultArcParameters Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetDefaultArcParameters Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetDefaultArcParameters.

GreGetDefaultArcParameters Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreGetDefaultArcParameters - Parameters

hdc (HDC) - input
Device context handle.

pArcParms (PARCPARAMS) - input
Pointer to ARCPARAMS structure.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetDefaultArcParameters.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreGetDefaultArcParameters - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreGetDefaultAttributes

GreGetDefaultAttributes - Syntax

Description

GreGetDefaultAttributes returns the default values of the attributes indicated in flAttrsMask.

This function is supported by the graphics engine.

```
#define INCL_GRE_DEVSUPPORT
#include <os2.h>

HDC         hdc;           /*  Device context handle. */
LONG        lPrimType;     /*  Bundle primitive type. */
ULONG       flAttrsMask;   /*  Attribute mask. */
PBUNDLE     pAttrs;
PVOID       pInstance;     /*  Pointer to instance data. */
ULONG       lFunction;
BOOL        rc;

rc = GreGetDefaultAttributes(hdc, lPrimType,
       flAttrsMask, pAttrs, pInstance, lFunction);
```

GreGetDefaultAttributes Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetDefaultAttributes Parameter - lPrimType

lPrimType (LONG) - input
Bundle primitive type.

Indicates the bundle type. Valid primitive values are:

PRIM_LINE	Line attribute bundle
PRIM_CHAR	Character attribute bundle
PRIM_MARKER	Marker attribute bundle

PRIM_AREA	Pattern attribute bundle
PRIM_IMAGE	Image attribute bundle

GreGetDefaultAttributes Parameter - flAttrsMask

flAttrsMask (ULONG) - input
Attribute mask.

Specifies the attributes to be returned corresponding to each attribute in the bundle record. For each set bit, the engine returns the corresponding attribute values and default mask bits.

GreGetDefaultAttributes Parameter - pAttrs

pAttrs (PBUNDLE) - input
Pointer to the fixed format bundle record containing the attributes returned.

GreGetDefaultAttributes Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetDefaultAttributes Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetDefaultAttributes.

GreGetDefaultAttributes Return Value - rc

rc (BOOL) - returns
This function returns the default attribute bit mask. Only bits with corresponding set bits in flAttrsMask are updated. Otherwise, this function returns the error, ATTRS_ERROR.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreGetDefaultAttributes - Parameters

hdc (HDC) - input
Device context handle.

lPrimType (LONG) - input
Bundle primitive type.

Indicates the bundle type. Valid primitive values are:

PRIM_LINE	Line attribute bundle
PRIM_CHAR	Character attribute bundle
PRIM_MARKER	Marker attribute bundle
PRIM_AREA	Pattern attribute bundle
PRIM_IMAGE	Image attribute bundle

flAttrsMask (ULONG) - input
Attribute mask.

Specifies the attributes to be returned corresponding to each attribute in the bundle record. For each set bit, the engine returns the corresponding attribute values and default mask bits.

pAttrs (PBUNDLE) - input
Pointer to the fixed format bundle record containing the attributes returned.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetDefaultAttributes.

rc (BOOL) - returns
This function returns the default attribute bit mask. Only bits with corresponding set bits in flAttrsMask are updated. Otherwise, this function returns the error, ATTRS_ERROR.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreGetDefaultAttributes - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreGetDefaultViewingLimits

GreGetDefaultViewingLimits - Syntax

Description

*GreGetDefaultViewingLimit*s loads the array indicated by prclViewingLimits with the default boundaries of the viewing window in graphic model space coordinates.

This function is supported by the graphics engine.

```
#define INCL_GRE_DEVSUPPORT
#include <os2.h>

HDC       hdc;                /*  Device context handle. */
PRECTL    prclViewingLimits;  /*  Pointer to limits of viewing area. */
PVOID     pInstance;          /*  Pointer to instance data. */
ULONG     lFunction;          /*  High-order WORD=flags; low-order WORD=NGreGetDefaultViewingLimits. */
BOOL      fSuccess;           /*  Return codes. */

fSuccess = GreGetDefaultViewingLimits(hdc,
             prclViewingLimits, pInstance,
             lFunction);
```

GreGetDefaultViewingLimits Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetDefaultViewingLimits Parameter - prclViewingLimits

prclViewingLimits (PRECTL) - input
Pointer to limits of viewing area.

RECTL structure:

xLeft	Minimum x-coordinate of viewing limits
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of viewing limits
yTop	Maximum y-coordinate

GreGetDefaultViewingLimits Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetDefaultViewingLimits Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetDefaultViewingLimits.

GreGetDefaultViewingLimits Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreGetDefaultViewingLimits - Parameters

hdc (HDC) - input
Device context handle.

prclViewingLimits (PRECTL) - input
Pointer to limits of viewing area.

RECTL structure:

xLeft	Minimum x-coordinate of viewing limits
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of viewing limits
yTop	Maximum y-coordinate

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetDefaultViewingLimits.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreGetDefaultViewingLimits - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreGetHandle

GreGetHandle - Syntax

Description

GreGetHandle returns the handle or variable (stored in the DC corresponding to iIndex) previously set by GreSetHandle.

This function is supported by the graphics engine.

```
#define INCL_GRE_DCS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    iIndex;      /*  Index value. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreGetHandle. */
LONG     rc;          /*  Return code. */

rc = GreGetHandle(hdc, iIndex, pInstance,
       lFunction);
```

GreGetHandle Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetHandle Parameter - iIndex

iIndex (ULONG) - input
Index value.

Index value of the returned handle in the range 0 - 3. A value of 1 can be used to get the associated AVIO presentation space handle.

GreGetHandle Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetHandle Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetHandle.

GreGetHandle Return Value - rc

rc (LONG) - returns
Return code.

On completion, the graphics engine returns the handle requested hHandle), or GPI_ALTERROR if an error occurs.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreGetHandle - Parameters

hdc (HDC) - input
Device context handle.

iIndex (ULONG) - input
Index value.

Index value of the returned handle in the range 0 - 3. A value of 1 can be used to get the associated AVIO presentation space handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetHandle.

rc (LONG) - returns
Return code.

On completion, the graphics engine returns the handle requested hHandle), or GPI_ALTERROR if an error occurs.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreGetHandle - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreGetProcessControl

GreGetProcessControl - Syntax

Description

GreGetProcessControl returns the process control flags.

This function is supported by the graphics engine.

```
#define INCL_GRE_DCS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreGetProcessControl. */
LONG     rc;          /*  Return codes. */

rc = GreGetProcessControl(hdc, pInstance,
       lFunction);
```

GreGetProcessControl Parameter - hdc

hdc (HDC) - input
Device context handle.

GreGetProcessControl Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreGetProcessControl Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetProcessControl.

GreGetProcessControl Return Value - rc

rc (LONG) - returns
Return codes.

This function returns the process control flags (flProcess), or DCTL_ERROR if an error occurs. The flags returned are:

PCTL_DRAW	Draw flag
PCTL_BOUND	GPI_BOUNDS flag
PCTL_CORRELATE	Correlate flag
PCTL_USERBOUNDS	USER_BOUNDS flag
PCTL_AREA	When set, an area definition is in progress
COM_PATH	When set, a path definition is in progress

Other flags are reserved and should be ignored.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreGetProcessControl - Parameters

hdc (HDC) - input
Device context handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreGetProcessControl.

rc (LONG) - returns
Return codes.

This function returns the process control flags (flProcess), or DCTL_ERROR if an error occurs. The flags returned are:

PCTL_DRAW	Draw flag
PCTL_BOUND	GPI_BOUNDS flag

PCTL_CORRELATE	Correlate flag
PCTL_USERBOUNDS	USER_BOUNDS flag
PCTL_AREA	When set, an area definition is in progress
COM_PATH	When set, a path definition is in progress

Other flags are reserved and should be ignored.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreGetProcessControl - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreInitializeAttributes

GreInitializeAttributes - Syntax

Description

This function sets the current default attributes to the initial standard default values.

This function is supported by the graphics engine.

```
#define INCL_GRE_DEVSUPPORT
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    flOptions;   /*  Option flags. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreInitializeAttributes. */
BOOL     fSuccess;    /*  Return codes. */

fSuccess = GreInitializeAttributes(hdc, flOptions,
             pInstance, lFunction);
```

GreInitializeAttributes Parameter - hdc

hdc (HDC) - input
Device context handle.

GreInitializeAttributes Parameter - flOptions

flOptions (ULONG) - input
Option flags.

Valid flags are:

INAT_DEFAULTATTRIBUTES	When set, all the current default attributes are set to their initial standard default values.
INAT_CURRENTATTRIBUTES	When set, all the current attributes are set to their current default values. Note that when both flags are set, INAT_DEFAULTATTRIBUTES is processed first.

GreInitializeAttributes Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreInitializeAttributes Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreInitializeAttributes.

GreInitializeAttributes Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

GreInitializeAttributes - Parameters

hdc (HDC) - input
Device context handle.

flOptions (ULONG) - input
Option flags.

Valid flags are:

INAT_DEFAULTATTRIBUTES	When set, all the current default attributes are set to their initial standard default values.
INAT_CURRENTATTRIBUTES	When set, all the current attributes are set to their current default values. Note that when both flags are set, INAT_DEFAULTATTRIBUTES is processed first.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreInitializeAttributes.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

GreInitializeAttributes - Remarks

This function can also be used to reset the current attributes to the current default values. GreInitializeAttributes affects all bundle attributes that can be set with GreSetAttributes, arc parameters, and viewing limits.

GreInitializeAttributes - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreLoadFont

GreLoadFont - Syntax

Description

This function loads fonts from a resource file. All fonts in the file are private and are only available to the process that called this function.

This function is supported by the graphics engine.

```
#define INCL_GRE_FONTS
#include <os2.h>

PSZ      pszFilename;  /*  Pointer. */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreLoadFont. */
BOOL     fSuccess;     /*  Return codes. */

fSuccess = GreLoadFont(pszFilename, pInstance,
            lFunction);
```

GreLoadFont Parameter - pszFilename

pszFilename (PSZ) - input
Pointer.

Pointer to a NULL-terminated string containing path and name of the font file, identified by the file extension FON.

GreLoadFont Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreLoadFont Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreLoadFont.

GreLoadFont Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_FONT_FILE_DATA

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreLoadFont - Parameters

pszFilename (PSZ) - input
Pointer.

Pointer to a NULL-terminated string containing path and name of the font file, identified by the file extension FON.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreLoadFont.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_FONT_FILE_DATA

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreLoadFont - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreOpenDC

GreOpenDC - Syntax

Description

GreOpenDC creates an output device context (DC). The new device context inherits the current code page of the process that created it.

This function is supported by the graphics engine.

```
#define INCL_GRE_DCS
#include <os2.h>

HDC             hdc;         /*  Device context handle. */
ULONG           ulType;      /*  Type of DC being opened. */
PSZ             pszToken;    /*  This parameter is ignored by the graphics engine. */
LONG            cData;       /*  Number of elements in the data structure. */
PDEVOPENDATA    pszData;     /*  Pointer to data structure. */
PVOID           pInstance;   /*  Pointer to instance data. */
ULONG           lFunction;   /*  High-order WORD=flags; low-order WORD=NGreOpenDC. */
HDC             rc;          /*  Return codes. */

rc = GreOpenDC(hdc, ulType, pszToken, cData,
       pszData, pInstance, lFunction);
```

GreOpenDC Parameter - hdc

hdc (HDC) - input
Device context handle.

When ulType is OD_ context compatible with bit maps that are used with this device context. If this is NULL, compatibility with the display is assumed.

GreOpenDC Parameter - ulType

ulType (ULONG) - input
Type of DC being opened.

Type of device context:

OD_QUEUED	The DC is for a hardcopy device for which output is to be queued by the spooler.
OD_DIRECT	The DC is for a device for which output is not to be queued by the spooler.
OD_INFO	This is similar to OD_DIRECT except that it is only used to retrieve information such as font metrics. Drawing can be done to a presentation space associated with such a DC but no output medium is updated.
OD_MEMORY	The new DC is a memory DC used to contain a bit map. The hdc parameter identifies the device with which the bit map is to be compatible.

GreOpenDC Parameter - pszToken

pszToken (PSZ) - input
This parameter is ignored by the graphics engine.

GreOpenDC Parameter - cData

cData (LONG) - input
Number of elements in the data structure.

GreOpenDC Parameter - pszData

pszData (PDEVOPENDATA) - input
Pointer to data structure.

Pointer to DEVOPENDATA structure:

pszLogAddress	Pointer to logical address, for example, *LPT1Q* .	
pszDriverName	Pointer to presentation driver name, for example, *LASERJET* .	
pdriv	Pointer to a DRIVDATA structure:	
	cb	Size in bytes of this structure
	lVersion	Version number of the data. Version numbers are defined by the presentation driver.
	szDeviceName[32]	String identifying the device. Valid values are supplied by the presentation driver.
	abGeneralData	General data defined by the presentation driver. This does not contain pointers as they might not be valid when passed to the driver.
pszDataType	Pointer to a data type of the queued file. Supported data types are:	
	PM_Q_STD PM_Q_RAW	User-defined data types can also be supported.
pszComment	Pointer to a description of the file that can be displayed by the spooler to the user.	
pszQueueProcName	Pointer to name of queue processor.	
pszQueueProcParams	Pointer to a string of queue processor parameters.	
pszSpoolerParams	Pointer to a string of spooler parameters separated by one or more blanks. Valid parameters are:	
	FORM=aaa	Identifies the form name for a print job. Multiple names are

		separated by commas (aaa,bbb,ccc). If this parameter is not present, the current form is used. Form names are defined by the presentation driver. Valid names are those that would be returned from a call to the GreQueryHardcopyCaps handling routine of the driver.
	PRTY=n	Identifies the priority for the print job. The priority can be any value from 1 - 99 (1 is lowest priority). If this parameter is not present, the priority value defaults to 50.
pszNetworkParams		Pointer to a string of networking parameters, which are used only in a network environment. Their nature is defined by the network application.

GreOpenDC Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreOpenDC Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreOpenDC.

GreOpenDC Return Value - rc

rc (HDC) - returns
Return codes.

On completion, the graphics engine returns the handle of the new device context (hdc), or DEV_ERROR if an error occurs.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_BITMAP_IS_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_CHAR_DIRECTION_ATTR

PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_DC_DATA
PMERR_INV_DC_TYPE
PMERR_INV_DRIVER_NAME
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_ID
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_MIX_ATTR
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_SCAN_START

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreOpenDC - Parameters

hdc (HDC) - input
Device context handle.

When ulType is OD_ context compatible with bit maps that are used with this device context. If this is NULL, compatibility with the display is assumed.

ulType (ULONG) - input
Type of DC being opened.

Type of device context:

OD_QUEUED	The DC is for a hardcopy device for which output is to be queued by the spooler.
OD_DIRECT	The DC is for a device for which output is not to be queued by the spooler.
OD_INFO	This is similar to OD_DIRECT except that it is only used to retrieve information such as font metrics. Drawing can be done to a presentation space associated with such a DC but no output medium is updated.
OD_MEMORY	The new DC is a memory DC used to contain a bit map. The hdc parameter identifies the device with which the bit map is to be compatible.

pszToken (PSZ) - input
This parameter is ignored by the graphics engine.

cData (LONG) - input
Number of elements in the data structure.

pszData (PDEVOPENDATA) - input
Pointer to data structure.

Pointer to DEVOPENDATA structure:

pszLogAddress	Pointer to logical address, for example, *LPT1Q* .
pszDriverName	Pointer to presentation driver name, for example, *LASERJET* .
pdriv	Pointer to a DRIVDATA structure:
	cb Size in bytes of

this structure

lVersion — Version number of the data. Version numbers are defined by the presentation driver.

szDeviceName[32] — String identifying the device. Valid values are supplied by the presentation driver.

abGeneralData — General data defined by the presentation driver. This does not contain pointers as they might not be valid when passed to the driver.

pszDataType — Pointer to a data type of the queued file. Supported data types are:

PM_Q_STD
PM_Q_RAW — User-defined data types can also be supported.

pszComment — Pointer to a description of the file that can be displayed by the spooler to the user.

pszQueueProcName — Pointer to name of queue processor.

pszQueueProcParams — Pointer to a string of queue processor parameters.

pszSpoolerParams — Pointer to a string of spooler parameters separated by one or more blanks. Valid parameters are:

FORM=aaa — Identifies the form name for a print job. Multiple names are separated by commas (aaa,bbb,ccc). If this parameter is not present, the current form is used.

Form names are defined by the presentation driver. Valid names are those that would be returned from a call to the GreQueryHardcopyCaps handling routine of the driver.

PRTY=n — Identifies the priority for the print job. The priority can be any value from 1 - 99 (1 is lowest priority). If this parameter is not present, the priority value defaults to 50.

pszNetworkParams — Pointer to a string of networking parameters, which are used only in a network environment. Their nature is defined by the network application.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreOpenDC.

rc (HDC) - returns
Return codes.

On completion, the graphics engine returns the handle of the new device context (hdc), or DEV_ERROR if an error occurs.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_BITMAP_IS_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_CHAR_DIRECTION_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_DC_DATA
PMERR_INV_DC_TYPE
PMERR_INV_DRIVER_NAME
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_ID
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_MIX_ATTR
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_SCAN_START

Refer to the "Error Explanations" section in the *Presentation Manager Programming Reference* for further explanation.

GreOpenDC - Remarks

Subsequent calls to DosSetCp do not alter the code page of an existing DC. Default VIO and KBD code pages are always in the last code page set by any application process.

When this function is called for the first time, the graphics engine performs the following sequence:

```
DosLoadModule("NEWDRIVER", &hDriver)
                  /*****************************************************/
                  /* Load the presentation driver DLL file.            */
                  /*****************************************************/

DosGetProcAddr(hDriver, "OS2_PM_DRV_ENABLE", &Enable)
                  /*****************************************************/
                  /* Find the presentation driver's Enable function.   */
                  /* See OS2_PM_DRV_ENABLE.          */
                  /*****************************************************/

*Enable(FillLogicalDeviceBlock, &DispatchTable)
                  /*****************************************************/
                  /* The presentation driver must:                     */
                  /*   Save the addresses of the engine simulations    */
                  /*   Overwrite the dispatch table, as necessary      */
                  /*   Hook the ExitList for the calling process       */
```

```
                    /*************************************************/

hPhysDev=*Enable(FillPhysicalDeviceBlock, pDevOpenStructure)
                    /*************************************************/
                    /* Create the physical device block.             */
                    /*************************************************/

pInstance=*Enable(EnableDeviceContext, hdc, Type, hPhysDev)
                    /*************************************************/
                    /* The presentation driver must create an instance  */
                    /* data structure for the DC.                    */
                    /*************************************************/

*Enable(CompleteOpenDC, hdc, pInstance)
                    /*************************************************/
                    /* The presentation driver must inform the system   */
                    /* that the DC is open and ready to receive output. */
                    /*************************************************/
```

For subsequent calls to GreOpenDC for the same DC, the graphics engine calls Enable (EnableDeviceContext) to create a new instance for the DC and then calls Enable (CompleteOpenDC). When GreOpenDC is called for the same DC by a different process, the graphics engine calls DosLoadModule to load the presentation driver. It then calls Enable (FillLogicalDeviceBlock), Enable (EnableDeviceContext), and Enable (CompleteOpenDC).

GreOpenDC - Topics

Select an item:

Syntax
Parameters
Returns
Remarks
Glossary

GreOpenJournalFile

GreOpenJournalFile - Syntax

Description

GreOpenJournalFile opens a journal file for play. The journal file must exist and must contain valid journaled calls.

This function is supported by the graphics engine.

```
#define INCL_GRE_JOURNALING
#include <os2.h>

PSZ      pszFileName;  /*  Pointer a string containing the file name. */
ULONG    flOption;     /*  An option flag that identifies the type of journal file. */
ULONG    cBufSize;     /*  Size of buffer required for the file. */
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreOpenJournalFile. */
ULONG    rc;           /*  Return codes. */

rc = GreOpenJournalFile(pszFileName, flOption,
       cBufSize, pInstance, lFunction);
```

GreOpenJournalFile Parameter - pszFileName

pszFileName (PSZ) - input
Pointer a string containing the file name.

GreOpenJournalFile Parameter - flOption

flOption (ULONG) - input
An option flag that identifies the type of journal file.

Valid values are:

JNL_PERM_FILE	A disk file with the name pszFileName, which was created by using GreCreateJournalFile (JNL_PERM_FILE).
Bit 3	A shared memory file pointed to by pszFileName and already filled with complete journal records. One way this journal file might have been created is by using GreCreateJournalFile (JNL_USERRAM_FILE) and being filled by the engine. .

GreOpenJournalFile Parameter - cBufSize

cBufSize (ULONG) - input
Size of buffer required for the file.

GreOpenJournalFile Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreOpenJournalFile Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreOpenJournalFile.

GreOpenJournalFile Return Value - rc

rc (ULONG) - returns
Return codes.

This function returns the journal file handle (ULONG), or if an error occurs, it returns NULL.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_JOURNAL_OPTION
PMERR_RAM_JNL_FILE_TOO_SMALL

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreOpenJournalFile - Parameters

pszFileName (PSZ) - input
Pointer a string containing the file name.

flOption (ULONG) - input
An option flag that identifies the type of journal file.

Valid values are:

JNL_PERM_FILE	A disk file with the name pszFileName, which was created by using GreCreateJournalFile (JNL_PERM_FILE).
Bit 3	A shared memory file pointed to by pszFileName and already filled with complete journal records. One way this journal file might have been created is by using GreCreateJournalFile (JNL_USERRAM_FILE) and being filled by the engine. .

cBufSize (ULONG) - input
Size of buffer required for the file.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreOpenJournalFile.

rc (ULONG) - returns
Return codes.

This function returns the journal file handle (ULONG), or if an error occurs, it returns NULL.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_JOURNAL_OPTION
PMERR_RAM_JNL_FILE_TOO_SMALL

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreOpenJournalFile - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GrePlayJournalFile

GrePlayJournalFile - Syntax

Description

GrePlayJournalFile plays a journal file to the specified DC.

This function is supported by the graphics engine.

```
#define INCL_GRE_JOURNALING
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    hJournal;    /*  Journal file handle. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGrePlayJournalFile. */
BOOL     fSuccess;    /*  Return codes. */

fSuccess = GrePlayJournalFile(hdc, hJournal,
             pInstance, lFunction);
```

GrePlayJournalFile Parameter - hdc

hdc (HDC) - input
Device context handle.

GrePlayJournalFile Parameter - hJournal

hJournal (ULONG) - input
Journal file handle.

GrePlayJournalFile Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GrePlayJournalFile Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePlayJournalFile.

GrePlayJournalFile Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_BITMAP_IS_SELECTED
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INCORRECT_DC_TYPE
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_CHAR_DIRECTION_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_DC_DATA
PMERR_INV_DC_TYPE
PMERR_INV_DRIVER_NAME
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_HJOURNAL
PMERR_INV_HRGN
PMERR_INV_ID
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_METAFILE
PMERR_INV_MIX_ATTR
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT

PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_REGION_MIX_MODE
PMERR_INV_SCAN_START
PMERR_INV_USAGE_PARM
PMERR_JFILE_BUSY
PMERR_RAM_JNL_FILE_TOO_SMALL
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GrePlayJournalFile - Parameters

hdc (HDC) - input
Device context handle.

hJournal (ULONG) - input
Journal file handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGrePlayJournalFile.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_BITMAP_IS_SELECTED
PMERR_BITMAP_NOT_SELECTED
PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_HRGN_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INCORRECT_DC_TYPE
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_CHAR_DIRECTION_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_DC_DATA
PMERR_INV_DC_TYPE
PMERR_INV_DRIVER_NAME
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_HJOURNAL
PMERR_INV_HRGN
PMERR_INV_ID
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT

PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_METAFILE
PMERR_INV_MIX_ATTR
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_REGION_MIX_MODE
PMERR_INV_SCAN_START
PMERR_INV_USAGE_PARM
PMERR_JFILE_BUSY
PMERR_RAM_JNL_FILE_TOO_SMALL
PMERR_REGION_IS_CLIP_REGION

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GrePlayJournalFile - Remarks

The journal file is read into memory and each journaled call is played. Each journaled record is processed before playing to fix-up data pointers, and create "clone objects," such as regions or bit maps, if necessary, from the journaled data. It is assumed that any single journaled function and associated data fits in a 32KB buffer. If the journaled record contains region rectangles or bit-map bits, they are not considered in this restriction.

GrePlayJournalFile - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreQueryBitmapHandle

GreQueryBitmapHandle - Syntax

Description

GreQueryBitmapHandle gets the bit-map handle for the specified local identifier (lcid). When lLcid does not reference a bit map, an error is raised by the graphics engine.

This function is supported by the graphics engine.

```
#define INCL_GRE_LCID
#include <os2.h>

HDC         hdc;         /*  Device context handle. */
LONG        lLcid;       /*  Local identifier for which the bit-map handle is required. */
PVOID       pInstance;   /*  Pointer to instance data. */
```

```
ULONG      lFunction;  /*  High-order WORD=flags; low-order WORD=NGreQueryBitmapHandle. */
HBITMAP    rc;         /*  Return codes. */

rc = GreQueryBitmapHandle(hdc, lLcid, pInstance,
       lFunction);
```

GreQueryBitmapHandle Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryBitmapHandle Parameter - lLcid

lLcid (LONG) - input
Local identifier for which the bit-map handle is required.

GreQueryBitmapHandle Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryBitmapHandle Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryBitmapHandle.

GreQueryBitmapHandle Return Value - rc

rc (HBITMAP) - returns
Return codes.

This function returns the bit-map handle (hbm), or it returns GPI_ERROR if an error occurs.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BITMAP_NOT_SELECTED
PMERR_HDC_BUSY
PMERR_INV_HDC

PMERR_INV_SETID

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreQueryBitmapHandle - Parameters

hdc (HDC) - input
Device context handle.

lLcid (LONG) - input
Local identifier for which the bit-map handle is required.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryBitmapHandle.

rc (HBITMAP) - returns
Return codes.

This function returns the bit-map handle (hbm), or it returns GPI_ERROR if an error occurs.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BITMAP_NOT_SELECTED
PMERR_HDC_BUSY
PMERR_INV_HDC
PMERR_INV_SETID

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreQueryBitmapHandle - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreQueryCodePageVector

GreQueryCodePageVector - Syntax

Description

GreQueryCodePageVector returns a pointer to a vector of 256 WORDs, which is the code point to the glyph mapping number.

This function is supported by the graphics engine.

```
#define INCL_GRE_FONTS
#include <os2.h>

ULONG      ulCodePage;   /*  Code page number. */
PVOID      pInstance;    /*  Pointer to instance data. */
ULONG      lFunction;    /*  High-order WORD=flags; low-order WORD=NGreQueryCodePageVector. */
PUSHORT    rc;           /*  Return codes. */

rc = GreQueryCodePageVector(ulCodePage, pInstance,
       lFunction);
```

GreQueryCodePageVector Parameter - ulCodePage

ulCodePage (ULONG) - input
Code page number.

GreQueryCodePageVector Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryCodePageVector Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryCodePageVector.

GreQueryCodePageVector Return Value - rc

rc (PUSHORT) - returns
Return codes.

This function returns a pointer to the code page vector, or it returns GPI_ERROR if an error occurs.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CODEPAGE

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreQueryCodePageVector - Parameters

ulCodePage (ULONG) - input
Code page number.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryCodePageVector.

rc (PUSHORT) - returns
Return codes.

This function returns a pointer to the code page vector, or it returns GPI_ERROR if an error occurs.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CODEPAGE

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreQueryCodePageVector - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreQueryEngineVersion

GreQueryEngineVersion - Syntax

Description

GreQueryEngineVersion returns the version number of the graphics engine.

This function is supported by the graphics engine.

```
#define INCL_GRE_DCS
#include <os2.h>

PVOID    pInstance;  /*  Pointer to instance data. */
ULONG    lFunction;  /*  High-order WORD=flags; low-order WORD=NGreQueryEngineVersion. */
LONG     rc;         /*  Return codes. */
```

```
rc = GreQueryEngineVersion(pInstance, lFunction);
```

GreQueryEngineVersion Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryEngineVersion Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryEngineVersion.

GreQueryEngineVersion Return Value - rc

rc (LONG) - returns
Return codes.

GreQueryEngineVersion returns the engine version number (lVersion), or it returns GPI_ALTERROR if an error occurs. For example, it might return `0X000000200L = 2.0`.

GreQueryEngineVersion - Parameters

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryEngineVersion.

rc (LONG) - returns
Return codes.

GreQueryEngineVersion returns the engine version number (lVersion), or it returns GPI_ALTERROR if an error occurs. For example, it might return `0X000000200L = 2.0`.

GreQueryEngineVersion - Topics

Select an item:
Syntax

Parameters
Returns
Glossary

GreQueryFontAttributes

GreQueryFontAttributes - Syntax

Description

GreQueryFontAttributes returns the metrics of the current font at the location addressed by pfmMetrics.

This function is supported by the graphics engine.

```
#define INCL_GRE_FONTS
#include <os2.h>

HDC             hdc;          /*  Device context handle. */
LONG            cbMetrics;    /*  Size in bytes of the font metrics buffer. */
PFONTMETRICS    pfmMetrics;   /*  Pointer. */
PVOID           pInstance;    /*  Pointer to instance data. */
ULONG           lFunction;    /*  High-order WORD=flags; low-order WORD=NGreQueryFontAttributes. */
BOOL            fSuccess;     /*  Return codes. */

fSuccess = GreQueryFontAttributes(hdc, cbMetrics,
            pfmMetrics, pInstance, lFunction);
```

GreQueryFontAttributes Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryFontAttributes Parameter - cbMetrics

cbMetrics (LONG) - input
Size in bytes of the font metrics buffer.

GreQueryFontAttributes Parameter - pfmMetrics

pfmMetrics (PFONTMETRICS) - input
Pointer.

Pointer to font metric block where the information is to be returned.

GreQueryFontAttributes Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryFontAttributes Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryFontAttributes.

GreQueryFontAttributes Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SETID

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreQueryFontAttributes - Parameters

hdc (HDC) - input
Device context handle.

cbMetrics (LONG) - input
Size in bytes of the font metrics buffer.

pfmMetrics (PFONTMETRICS) - input
Pointer.

Pointer to font metric block where the information is to be returned.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryFontAttributes.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SETID

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreQueryFontAttributes - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreQueryFontFileDescriptions

GreQueryFontFileDescriptions - Syntax

Description

GreQueryFontFileDescriptions determines whether a file is a font file and, if so, returns the family and face names for the fonts in the file.

This function is supported by the graphics engine.

```
#define INCL_GRE_FONTS
#include <os2.h>

HDC       hdc;          /*  Device context handle. */
```

```
PSZ       pszFileName;  /*  Pointer to the file path and name of the font file. */
PULONG    pcFonts;      /*  Pointer. */
PSZ       paszNames;    /*  Pointer. */
PVOID     pInstance;    /*  Pointer to instance data. */
ULONG     lFunction;    /*  High-order WORD=flags; low-order WORD=NGreQueryFontFileDescriptions.

 */
ULONG     rc;           /*  Return codes. */

rc = GreQueryFontFileDescriptions(hdc, pszFileName,
       pcFonts, paszNames, pInstance, lFunction);
```

GreQueryFontFileDescriptions Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryFontFileDescriptions Parameter - pszFileName

pszFileName (PSZ) - input
Pointer to the file path and name of the font file.

GreQueryFontFileDescriptions Parameter - pcFonts

pcFonts (PULONG) - input
Pointer.

Pointer to a count of the maximum number of family and face name pairs to be returned. On completion, this count is updated to the number of pairs actually returned.

GreQueryFontFileDescriptions Parameter - paszNames

paszNames (PSZ) - input
Pointer.

Pointer to an array of family and face name pairs whose size is 2 x pcFonts. The family name is the first in each pair.

GreQueryFontFileDescriptions Parameter - pInstance

pInstance (PVOID) - input

Pointer to instance data.

GreQueryFontFileDescriptions Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryFontFileDescriptions.

GreQueryFontFileDescriptions Return Value - rc

rc (ULONG) - returns
Return codes.

This function returns the number of fonts not returned, or it returns GPI_ALTERROR if an error occurs.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_INV_FONT_FILE_DATA

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreQueryFontFileDescriptions - Parameters

hdc (HDC) - input
Device context handle.

pszFileName (PSZ) - input
Pointer to the file path and name of the font file.

pcFonts (PULONG) - input
Pointer.

Pointer to a count of the maximum number of family and face name pairs to be returned. On completion, this count is updated to the number of pairs actually returned.

paszNames (PSZ) - input
Pointer.

Pointer to an array of family and face name pairs whose size is 2 x pcFonts. The family name is the first in each pair.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryFontFileDescriptions.

rc (ULONG) - returns
Return codes.

This function returns the number of fonts not returned, or it returns GPI_ALTERROR if an error occurs.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_INV_FONT_FILE_DATA

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreQueryFontFileDescriptions - Remarks

The names are returned at the location addressed by paszNames. Typically, the calling routine calls GreQueryFontFileDescriptions twice. The first call sets pcFonts to 0 and determines the number of fonts in the file. When sufficient storage is allocated for the names, GreQueryFontFileDescriptions is called again with pcFonts pointing to the font count.

GreQueryFontFileDescriptions - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreQueryFonts

GreQueryFonts - Syntax

Description

GreQueryFonts returns the details of fonts that match the face name addressed by pszFaceName.

This function is supported by the graphics engine.

```
#define INCL_GRE_FONTS
#include <os2.h>

HDC             hdc;          /*  Device context handle. */
ULONG           flOptions;    /*  Flags indicating whether private fonts, public fonts, or both are required. */
PSZ             pszFaceName;  /*  Pointer to a string specifying the face name. */
PFONTMETRICS    paMetrics;
ULONG           cMetricLen;   /*  Length in bytes of each metrics structure in the paMetrics array. */
PULONG          pcFontCount;
PVOID           pInstance;    /*  Pointer to instance data. */
ULONG           lFunction;    /*  High-order WORD=flags; low-order WORD=NGreQueryFonts. */
LONG            rc;           /*  Return codes. */

rc = GreQueryFonts(hdc, flOptions, pszFaceName,
       paMetrics, cMetricLen, pcFontCount,
```

```
        pInstance, lFunction);
```

GreQueryFonts Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryFonts Parameter - flOptions

flOptions (ULONG) - input
Flags indicating whether private fonts, public fonts, or both are required.

These flags can be used in combination:

QFF_PUBLIC	Enumerate public fonts
QFF_PRIVATE	Enumerate private fonts

GreQueryFonts Parameter - pszFaceName

pszFaceName (PSZ) - input
Pointer to a string specifying the face name.

When this is NULL, all faces are matched.

GreQueryFonts Parameter - paMetrics

paMetrics (PFONTMETRICS) - input
Pointer to an array of font element records to which the metrics of matching fonts are returned. This is a pointer to an array of up to cFontCount font metric records, each of which contains a maximum of cMetricLen bytes. Note that for multi-codepage fonts the *usCodePage* field has no meaning and is set to *0* .

GreQueryFonts Parameter - cMetricLen

cMetricLen (ULONG) - input
Length in bytes of each metrics structure in the paMetrics array.

GreQueryFonts Parameter - pcFontCount

pcFontCount (PULONG) - input
Pointer to cFontCount that specifies the number of fonts for which metrics are required. On return, this is updated to the number of fonts for which metrics were returned.

GreQueryFonts Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryFonts Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryFonts.

GreQueryFonts Return Value - rc

rc (LONG) - returns
Return codes.

This function returns the number of fonts not returned, or it returns GPI_ALTERROR if an error occurs.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_OR_INCOMPAT_OPTIONS

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreQueryFonts - Parameters

hdc (HDC) - input
Device context handle.

flOptions (ULONG) - input
Flags indicating whether private fonts, public fonts, or both are required.

These flags can be used in combination:

QFF_PUBLIC	Enumerate public fonts
QFF_PRIVATE	Enumerate private fonts

pszFaceName (PSZ) - input
Pointer to a string specifying the face name.

When this is NULL, all faces are matched.

paMetrics (PFONTMETRICS) - input
Pointer to an array of font element records to which the metrics of matching fonts are returned. This is a pointer to an array of up to cFontCount font metric records, each of which contains a maximum of cMetricLen bytes. Note that for multi-codepage fonts the *usCodePage* field has no meaning and is set to *0* .

cMetricLen (ULONG) - input
Length in bytes of each metrics structure in the paMetrics array.

pcFontCount (PULONG) - input
Pointer to cFontCount that specifies the number of fonts for which metrics are required. On return, this is updated to the number of fonts for which metrics were returned.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryFonts.

rc (LONG) - returns
Return codes.

This function returns the number of fonts not returned, or it returns GPI_ALTERROR if an error occurs.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_COORD_SPACE
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_OR_INCOMPAT_OPTIONS

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreQueryFonts - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreQueryLogicalFont

GreQueryLogicalFont - Syntax

Description

GreQueryLogicalFont returns the 8-character name and attributes for the logical font that is defined for the specified lcid.

This function is supported by the graphics engine.

```
#define INCL_GRE_FONTS
#include <os2.h>

HDC         hdc;         /*  Device context handle. */
LONG        lcid;        /*  Local identifier for the logical font. */
PSTR8       pchName;     /*  Pointer to an 8-character name used to describe the logical font. */
PFATTRS     pLogFont;    /*  Pointer to a font attribute structure. */
ULONG       cLogFont;    /*  Number of bytes of font attribute information requested. */
PVOID       pInstance;   /*  Pointer to instance data. */
ULONG       lFunction;   /*  High-order WORD=flags; low-order WORD=NGreQueryLogicalFont. */
BOOL        fSuccess;    /*  Return codes. */

fSuccess = GreQueryLogicalFont(hdc, lcid,
             pchName, pLogFont, cLogFont,
             pInstance, lFunction);
```

GreQueryLogicalFont Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryLogicalFont Parameter - lcid

lcid (LONG) - input
Local identifier for the logical font.

GreQueryLogicalFont Parameter - pchName

pchName (PSTR8) - input
Pointer to an 8-character name used to describe the logical font.

GreQueryLogicalFont Parameter - pLogFont

pLogFont (PFATTRS) - input
Pointer to a font attribute structure.

GreQueryLogicalFont Parameter - cLogFont

cLogFont (ULONG) - input
Number of bytes of font attribute information requested.

GreQueryLogicalFont Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryLogicalFont Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryLogicalFont.

GreQueryLogicalFont Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SETID

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreQueryLogicalFont - Parameters

hdc (HDC) - input
Device context handle.

lcid (LONG) - input

Local identifier for the logical font.

pchName (PSTR8) - input
Pointer to an 8-character name used to describe the logical font.

pLogFont (PFATTRS) - input
Pointer to a font attribute structure.

cLogFont (ULONG) - input
Number of bytes of font attribute information requested.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryLogicalFont.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SETID

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreQueryLogicalFont - Remarks

The data is returned in the locations addressed by pchName and pLogFont. When lcid identifies a bit-map ID, an error is raised by the graphics engine.

GreQueryLogicalFont - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreQueryNumberSetIds

GreQueryNumberSetIds - Syntax

Description

GreQueryNumberSetIds returns the total number of lcids, such as logical fonts and bit-map IDs, that have been created.

This function is supported by the graphics engine.

```
#define INCL_GRE_SETID
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    lRange;      /*  Indicates whether GPI, or AVIO lcids, or both are to be returned. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreQueryNumberSetIds. */
LONG     fSuccess;    /*  Return codes. */

fSuccess = GreQueryNumberSetIds(hdc, lRange,
             pInstance, lFunction);
```

GreQueryNumberSetIds Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQueryNumberSetIds Parameter - lRange

lRange (ULONG) - input
Indicates whether GPI, or AVIO lcids, or both are to be returned.

Valid ranges are:

LCID_RANGE_GPI	GPI
LCID_RANGE_AVIO	AVIO
LCID_RANGE_BOTH	GPI and AVIO

GreQueryNumberSetIds Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQueryNumberSetIds Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryNumberSetIds.

GreQueryNumberSetIds Return Value - fSuccess

fSuccess (LONG) - returns
Return codes.

This function returns the number of lcids, or it returns GPI_ALTERROR if an error occurs.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC
PMERR_INV_SETID
PMERR_INV_SETID_TYPE

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreQueryNumberSetIds - Parameters

hdc (HDC) - input
Device context handle.

lRange (ULONG) - input
Indicates whether GPI, or AVIO lcids, or both are to be returned.

Valid ranges are:

LCID_RANGE_GPI	GPI
LCID_RANGE_AVIO	AVIO
LCID_RANGE_BOTH	GPI and AVIO

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQueryNumberSetIds.

fSuccess (LONG) - returns
Return codes.

This function returns the number of lcids, or it returns GPI_ALTERROR if an error occurs.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC
PMERR_INV_SETID
PMERR_INV_SETID_TYPE

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreQueryNumberSetIds - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreQuerySetIds

GreQuerySetIds - Syntax

Description

GreQuerySetIds returns a list of created lcids with their names and types in the buffers addressed by paLcid, paszNames, and paTypes.

This function is supported by the graphics engine.

```
#define INCL_GRE_SETID
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
LONG     cSets;       /*  Number of sets to return. */
PLONG    paTypes;
PSTR8    paszNames;
PLONG    paLcid;      /*  Pointer to an array of cSets elements to which the lcids are returned. */
ULONG    lRange;      /*  Indicates whether GPI, or AVIO local identifiers, or both are to be returned. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreQuerySetIds. */
BOOL     fSuccess;    /*  Return codes. */

fSuccess = GreQuerySetIds(hdc, cSets, paTypes,
             paszNames, paLcid, lRange, pInstance,
             lFunction);
```

GreQuerySetIds Parameter - hdc

hdc (HDC) - input
Device context handle.

GreQuerySetIds Parameter - cSets

cSets (LONG) - input
Number of sets to return.

GreQuerySetIds Parameter - paTypes

paTypes (PLONG) - input
Pointer to an array of cSets elements indicating the type of the corresponding set. Each element in this array is a LONG value indicating the type of the corresponding local identifier:

LCIDT_FONT	Logical font
LCIDT_BITMAP	Bit-Map ID.

GreQuerySetIds Parameter - paszNames

paszNames (PSTR8) - input
Pointer to an array of eight-character names associated with the corresponding lcids. For a bit map, the name string is filled with *0s* .

GreQuerySetIds Parameter - paLcid

paLcid (PLONG) - input
Pointer to an array of cSets elements to which the lcids are returned.

GreQuerySetIds Parameter - lRange

lRange (ULONG) - input
Indicates whether GPI, or AVIO local identifiers, or both are to be returned.

Valid ranges are:

LCID_RANGE_GPI	GPI
LCID_RANGE_AVIO	AVIO
LCID_RANGE_BOTH	GPI and AVIO

LCID_AVIO_1, LCID_AVIO_2, and LCID_AVIO_3 represent AVIO sets of 1, 2, and 3.

GreQuerySetIds Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreQuerySetIds Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQuerySetIds.

GreQuerySetIds Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SETID
PMERR_INV_SETID_TYPE

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreQuerySetIds - Parameters

hdc (HDC) - input
Device context handle.

cSets (LONG) - input
Number of sets to return.

paTypes (PLONG) - input
Pointer to an array of cSets elements indicating the type of the corresponding set. Each element in this array is a LONG value indicating the type of the corresponding local identifier:

LCIDT_FONT	Logical font
LCIDT_BITMAP	Bit-Map ID.

paszNames (PSTR8) - input
Pointer to an array of eight-character names associated with the corresponding lcids. For a bit map, the name string is filled with *0s* .

paLcid (PLONG) - input
Pointer to an array of cSets elements to which the lcids are returned.

lRange (ULONG) - input
Indicates whether GPI, or AVIO local identifiers, or both are to be returned.

Valid ranges are:

LCID_RANGE_GPI	GPI
LCID_RANGE_AVIO	AVIO
LCID_RANGE_BOTH	GPI and AVIO

LCID_AVIO_1, LCID_AVIO_2, and LCID_AVIO_3 represent AVIO sets of 1, 2, and 3.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreQuerySetIds.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_SETID
PMERR_INV_SETID_TYPE

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreQuerySetIds - Remarks

When cSets is greater than the number of lcids, the unused elements of paTypes and paLcid are set to *0* .

GreQuerySetIds - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreResetDC

GreResetDC - Syntax

Description

GreResetDC restores a device context to its created state.

This function is supported by the graphics engine.

```
#define INCL_GRE_DCS
#include <os2.h>
```

```
HDC      hdc;        /*  Device context handle. */
ULONG    flOptions;  /*  Option flags. */
PVOID    pInstance;  /*  Pointer to instance data. */
ULONG    lFunction;  /*  High-order WORD=flags; low-order WORD=NGreResetDC. */
BOOL     fSuccess;   /*  Return codes. */

fSuccess = GreResetDC(hdc, flOptions, pInstance,
             lFunction);
```

GreResetDC Parameter - hdc

hdc (HDC) - input
Device context handle.

GreResetDC Parameter - flOptions

flOptions (ULONG) - input
Option flags.

Valid flags are:

RDC_SETOWNERTOSHELL	When set, the graphics engine sets the ownership of the reset DC to the process that first initialized the engine, (normally the Presentation Manager interface).
RDC_RGBMODE	Sets the DC's logical color table to RGB mode.

GreResetDC Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreResetDC Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreResetDC.

GreResetDC Return Value - fSuccess

fSuccess (BOOL) - returns

Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_BITMAP_IS_SELECTED
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_CODEPAGE
PMERR_INV_COORDINATE
PMERR_INV_DC_TYPE
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_ID
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_SCAN_START

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreResetDC - Parameters

hdc (HDC) - input
Device context handle.

flOptions (ULONG) - input
Option flags.

Valid flags are:

RDC_SETOWNERTOSHELL	When set, the graphics engine sets the ownership of the reset DC to the process that first initialized the engine, (normally the Presentation Manager interface).
RDC_RGBMODE	Sets the DC's logical color table to RGB mode.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreResetDC.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR

PMERR_BITMAP_IS_SELECTED
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_CODEPAGE
PMERR_INV_COORDINATE
PMERR_INV_DC_TYPE
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_ID
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_SCAN_START

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreResetDC - Remarks

All objects, such as fonts, patterns, and paths, are deleted. All attributes are set to their defaults. Any clip region selected into the DC is deleted.

This function does not alter Window Manager information stored in the DC instance data. Window Manager information includes:

- The visible region
- The DC origin
- User bounds
- Cached clipping rectangles
- The HDC_IS_DIRTY flag

GreResetDC - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreRestoreDC

GreRestoreDC - Syntax

Description

GreRestoreDC restores the contents of a previously saved device context.

This function is supported by the graphics engine.

```
#define INCL_GRE_DCS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
LONG     idDC;        /*  DC state identifier. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreRestoreDC. */
BOOL     fSuccess;    /*  Return codes. */

fSuccess = GreRestoreDC(hdc, idDC, pInstance,
             lFunction);
```

GreRestoreDC Parameter - hdc

hdc (HDC) - input
Device context handle.

GreRestoreDC Parameter - idDC

idDC (LONG) - input
DC state identifier.

Identifies the DC state to be restored. The values with which this parameter may be passed are as follows:

- A negative value indicates the number of device contexts that must be popped OFF the stack to access the required DC. A -1 indicates that the most recently saved DC must be restored.
- When this parameter is passed as a positive value (>0) and a corresponding DC does not exist, an error is returned. The current DC is not modified.
- When passed as a negative value and there are insufficient entries on the stack, an error is raised. The current DC is not modified.
- When the value passed corresponds to a saved DC, all entries on the stack up to the one indicated are discarded. Any clip regions selected into the discarded DCs are destroyed.
- A value of 1 resets the DC stack. All entries are removed from the stack.
- A value of 0 raises an error and the DC is not modified.

GreRestoreDC Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreRestoreDC Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRestoreDC.

GreRestoreDC Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CODEPAGE
PMERR_INV_COORDINATE
PMERR_INV_DC_TYPE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_ID
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreRestoreDC - Parameters

hdc (HDC) - input
Device context handle.

idDC (LONG) - input
DC state identifier.

Identifies the DC state to be restored. The values with which this parameter may be passed are as follows:

- A negative value indicates the number of device contexts that must be popped OFF the stack to access the required DC. A -1 indicates that the most recently saved DC must be restored.
- When this parameter is passed as a positive value (>0) and a corresponding DC does not exist, an error is returned. The current DC is not modified.
- When passed as a negative value and there are insufficient entries on the stack, an error is raised. The current DC is not modified.
- When the value passed corresponds to a saved DC, all entries on the stack up to the one indicated are discarded. Any clip

regions selected into the discarded DCs are destroyed.

- A value of 1 resets the DC stack. All entries are removed from the stack.
- A value of 0 raises an error and the DC is not modified.

plnstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreRestoreDC.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CODEPAGE
PMERR_INV_COORDINATE
PMERR_INV_DC_TYPE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_ID
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreRestoreDC - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreSaveDC

GreSaveDC - Syntax

Description

GreSaveDC saves the device context's state on a stack and returns an identifier to allow for its subsequent restoration.

This function is supported by the graphics engine.

```
#define INCL_GRE_DCS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreSaveDC. */
LONG     rc;          /*  Return codes. */

rc = GreSaveDC(hdc, pInstance, lFunction);
```

GreSaveDC Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSaveDC Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSaveDC Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSaveDC.

GreSaveDC Return Value - rc

rc (LONG) - returns
Return codes.

This function returns the identifier for the saved DC state (idDC), or it returns GPI_ERROR if an error occurs.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CODEPAGE
PMERR_INV_COORDINATE

PMERR_INV_DC_TYPE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSaveDC - Parameters

hdc (HDC) - input
Device context handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSaveDC.

rc (LONG) - returns
Return codes.

This function returns the identifier for the saved DC state (idDC), or it returns GPI_ERROR if an error occurs.

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_CODEPAGE
PMERR_INV_COORDINATE
PMERR_INV_DC_TYPE
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSaveDC - Remarks

The following information is saved:

- Current position
- Current attributes
- Current transforms, viewing limits, and clip path
- Any reference to a selected clip window
- Any loaded logical color table
- References to any loaded logical fonts
- References to the regions created on the associated DC

The following are not saved:

- Visible region
- Process controls

Any resources such as clip region and logical fonts, which are referenced in a saved DC, should not be deleted. The ID of a saved DC is only unique within the DC for which it is issued. Other DCs can have saved states with the same ID. The returned identifier can be used for a subsequent GreRestoreDC. This identifier represents the level of the saved DC on the save/restore stack. The first DC saved is identified by a value of 1, the second by 2, and so on.

GreSaveDC - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSelectBitmap

GreSelectBitmap - Syntax

Description

GreSelectBitmap selects a bit map into a memory DC or (if called with a NULL bit-map handle) deselects the existing bit map from the DC.

This function is supported by the graphics engine.

```
#define INCL_GRE_DEVSUPPORT
#include <os2.h>

HDC         hdc;         /*  Device context handle. */
HBITMAP     hbm;         /*  Bit-map handle. */
PVOID       pInstance;   /*  Pointer to instance data. */
ULONG       lFunction;   /*  High-order WORD=flags; low-order WORD=NGreSelectBitmap. */
HBITMAP     rc;          /*  On completion, this function returns an HBITMAP value: */

rc = GreSelectBitmap(hdc, hbm, pInstance,
       lFunction);
```

GreSelectBitmap Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSelectBitmap Parameter - hbm

hbm (HBITMAP) - input
Bit-map handle.

GreSelectBitmap Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSelectBitmap Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSelectBitmap.

GreSelectBitmap Return Value - rc

rc (HBITMAP) - returns
On completion, this function returns an HBITMAP value:

HBM_ERROR	Error
Null	Successful
Other	Handle of deselected bit map

An error is raised when the bit map:

- Is incompatible with the DC and cannot be converted
- Is already selected into a DC
- Has been assigned a Set ID for use as a pattern in an area fill operation

Possible Errors Detected: Error codes posted by the graphics engine for this function include:

PMERR_BITMAP_IS_SELECTED
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_COORDINATE
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_SCAN_START

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSelectBitmap - Parameters

hdc (HDC) - input
Device context handle.

hbm (HBITMAP) - input
Bit-map handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSelectBitmap.

rc (HBITMAP) - returns
On completion, this function returns an HBITMAP value:

HBM_ERROR	Error
Null	Successful
Other	Handle of deselected bit map

An error is raised when the bit map:

- Is incompatible with the DC and cannot be converted
- Is already selected into a DC
- Has been assigned a Set ID for use as a pattern in an area fill operation

Possible Errors Detected: Error codes posted by the graphics engine for this function include:

PMERR_BITMAP_IS_SELECTED
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_COORDINATE
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_HRGN
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_SCAN_START

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSelectBitmap - Remarks

If GreSelectBitmap is called to deselect a bit map, the return code is the handle of the deselected bit map.

When created, a bit map has to be selected into a DC before the presentation driver can write to it. A bit map can be selected into the DC that created it or into any DC that has a compatible bit-map format. Compatibility can be ensured by using one of the standard formats. Note that the presentation driver must select a bit map into a device context before attempting to draw it.

GreSelectBitmap - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSetAttributes

GreSetAttributes - Syntax

Description

*GreSetAttribute*s sets the attributes for the specified primitive type according to the flags set in flDefsMask and flAttrsMask.

This function is supported by the graphics engine.

```
#define INCL_GRE_DEVSUPPORT
#include <os2.h>

HDC         hdc;          /*  Device context handle. */
LONG        lPrimType;    /*  Bundle primitive type. */
ULONG       flDefsMask;   /*  Flags indicating the attributes to be set to default. */
ULONG       flAttrsMask;  /*  Flags indicating the attributes to be modified. */
PBUNDLE     pAttrs;       /*  Pointer. */
PVOID       pInstance;    /*  Pointer to instance data. */
ULONG       lFunction;    /*  High-order WORD=flags; low-order WORD=NGreSetAttributes. */
BOOL        rc;           /*  Return codes. */

rc = GreSetAttributes(hdc, lPrimType, flDefsMask,
       flAttrsMask, pAttrs, pInstance, lFunction);
```

GreSetAttributes Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetAttributes Parameter - lPrimType

lPrimType (LONG) - input
Bundle primitive type.

Indicates the bundle type. Valid primitive values are:

PRIM_LINE	Line attribute bundle
PRIM_CHAR	Character attribute bundle
PRIM_MARKER	Marker attribute bundle
PRIM_AREA	Pattern attribute bundle
PRIM_IMAGE	Image attribute bundle

GreSetAttributes Parameter - flDefsMask

flDefsMask (ULONG) - input
Flags indicating the attributes to be set to default.

GreSetAttributes Parameter - flAttrsMask

flAttrsMask (ULONG) - input
Flags indicating the attributes to be modified.

GreSetAttributes Parameter - pAttrs

pAttrs (PBUNDLE) - input
Pointer.

This is a pointer to the fixed format-bundle record containing the attribute values to be set as specified by flAttrsMask. Only the attribute fields corresponding to attribute flags set in flAttrsMask, and not set in flDefsMask, contain valid values.

This buffer need only be large enough to contain data for the highest offset attribute referenced.

GreSetAttributes Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetAttributes Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetAttributes.

GreSetAttributes Return Value - rc

rc (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_CHAR_DIRECTION_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CHAR_SET_ATTR
PMERR_INV_CHAR_SHEAR_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COORD_SPACE
PMERR_INV_GEOM_LINE_WIDTH_ATTR
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_END_ATTR
PMERR_INV_LINE_JOIN_ATTR
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_LINE_WIDTH_ATTR
PMERR_INV_MARKER_SET_ATTR
PMERR_INV_MARKER_SYMBOL_ATTR
PMERR_INV_MIX_ATTR
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_PRIMITIVE_TYPE
PMERR_INV_SETID
PMERR_UNSUPPORTED_ATTR
PMERR_UNSUPPORTED_ATTR_VALUE

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSetAttributes - Parameters

hdc (HDC) - input
Device context handle.

lPrimType (LONG) - input
Bundle primitive type.

Indicates the bundle type. Valid primitive values are:

PRIM_LINE	Line attribute bundle
PRIM_CHAR	Character attribute bundle
PRIM_MARKER	Marker attribute bundle
PRIM_AREA	Pattern attribute bundle
PRIM_IMAGE	Image attribute bundle

flDefsMask (ULONG) - input
Flags indicating the attributes to be set to default.

flAttrsMask (ULONG) - input
Flags indicating the attributes to be modified.

pAttrs (PBUNDLE) - input
Pointer.

This is a pointer to the fixed format-bundle record containing the attribute values to be set as specified by flAttrsMask. Only the attribute fields corresponding to attribute flags set in flAttrsMask, and not set in flDefsMask, contain valid values.

This buffer need only be large enough to contain data for the highest offset attribute referenced.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetAttributes.

rc (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_CHAR_DIRECTION_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CHAR_SET_ATTR
PMERR_INV_CHAR_SHEAR_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COORD_SPACE
PMERR_INV_GEOM_LINE_WIDTH_ATTR
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_END_ATTR
PMERR_INV_LINE_JOIN_ATTR
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_LINE_WIDTH_ATTR
PMERR_INV_MARKER_SET_ATTR
PMERR_INV_MARKER_SYMBOL_ATTR
PMERR_INV_MIX_ATTR
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_PRIMITIVE_TYPE
PMERR_INV_SETID
PMERR_UNSUPPORTED_ATTR
PMERR_UNSUPPORTED_ATTR_VALUE

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSetAttributes - Remarks

Note that:

- Only attributes whose flags are set in flAttrsMask are modified.
- Attributes whose flags are set in both flDefsMask and flAttrsMask are set to their default values.
- Attributes whose flags are set only in flDefsMask, or whose flags are not set either in flDefsMask or flAttrsMask, are unchanged.
- Attributes whose flags are set only in flAttrsMask are set to the value specified by pAttrs.

When GreSetAttributes occurs within a path bracket, it must not be used to set the geometric line width for line attributes, nor used to set the foreground and background colors and mixes for character and marker attributes.

GreSetAttributes - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSetBitmapDimension

GreSetBitmapDimension - Syntax

Description

GreSetBitmapDimension associates height and width values for the bit map indicated by hbm. These values can be read back later by calling GreGetBitmapDimension.

This function is supported by the graphics engine.

```
#define INCL_GRE_DEVSUPPOR
#include <os2.h>

HBITMAP     hbm;          /*  Bit-map handle. */
PSIZEL      pDimension;   /*  Pointer to width and height values in 0.1mm units. */
PVOID       pInstance;    /*  Pointer to instance data. */
ULONG       lFunction;    /*  High-order WORD=flags; low-order WORD=NGreSetBitmapDimension. */
BOOL        fSuccess;     /*  Return codes. */

fSuccess = GreSetBitmapDimension(hbm, pDimension,
             pInstance, lFunction);
```

GreSetBitmapDimension Parameter - hbm

hbm (HBITMAP) - input
Bit-map handle.

GreSetBitmapDimension Parameter - pDimension

pDimension (PSIZEL) - input
Pointer to width and height values in 0.1mm units.

Pointer to a pair of parameters:

ulWidth	Width of bit map in 0.1mm units
ulHeight	Height of bit map in 0.1mm units

GreSetBitmapDimension Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetBitmapDimension Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetBitmapDimension.

GreSetBitmapDimension Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BITMAP_IS_SELECTED
PMERR_HBITMAP_BUSY
PMERR_INV_HBITMAP

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSetBitmapDimension - Parameters

hbm (HBITMAP) - input
Bit-map handle.

pDimension (PSIZEL) - input
Pointer to width and height values in 0.1mm units.

Pointer to a pair of parameters:

ulWidth	Width of bit map in 0.1mm units
ulHeight	Height of bit map in 0.1mm units

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetBitmapDimension.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BITMAP_IS_SELECTED
PMERR_HBITMAP_BUSY
PMERR_INV_HBITMAP

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSetBitmapDimension - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreSetBitmapID

GreSetBitmapID - Syntax

Description

GreSetBitmapID sets the local identifier (lcid) for a bit map.

This function is supported by the graphics engine.

```
#define INCL_GRE_LCID
#include <os2.h>

HDC         hdc;         /*  Device context handle. */
HBITMAP     hbm;         /*  Bit-map handle. */
LONG        lLcid;       /*  Local identifier to be associated with the bit map. */
PVOID       pInstance;   /*  Pointer to instance data. */
ULONG       lFunction;   /*  High-order WORD=flags; low-order WORD=NGreSetBitmapID. */
BOOL        rc;          /*  Return codes. */

rc = GreSetBitmapID(hdc, hbm, lLcid, pInstance,
       lFunction);
```

GreSetBitmapID Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetBitmapID Parameter - hbm

hbm (HBITMAP) - input
Bit-map handle.

GreSetBitmapID Parameter - lLcid

lLcid (LONG) - input
Local identifier to be associated with the bit map.

GreSetBitmapID Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetBitmapID Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetBitmapID.

GreSetBitmapID Return Value - rc

rc (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BITMAP_IS_SELECTED
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_INV_HBITMAP
PMERR_INV_HDC

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSetBitmapID - Parameters

hdc (HDC) - input
Device context handle.

hbm (HBITMAP) - input
Bit-map handle.

lLcid (LONG) - input
Local identifier to be associated with the bit map.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetBitmapID.

rc (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BITMAP_IS_SELECTED
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_INV_HBITMAP
PMERR_INV_HDC

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSetBitmapID - Remarks

The presentation driver must assign an lcid before the bit map can be used for area shading or as the pattern in a BitBlt operation. The bit map can be of any format supported by the device. However, it can be simplified by the graphics engine before use.

Errors are raised by the graphics engine when:

- The local identifier is already in use.
- The bit map is already selected into a memory DC.

Note: When a bit map is destroyed, its lcid becomes undefined.

GreSetBitmapID - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSetDefaultArcParameters

GreSetDefaultArcParameters - Syntax

Description

*GreSetDefaultArcParameter*s sets the arc parameters to the default values.

This function is supported by the graphics engine.

```
#define INCL_GRE_DEVSUPPORT
#include <os2.h>

HDC           hdc;         /*  Device context handle. */
PARCPARAMS    pArcParms;   /*  Pointer to the default arc parameters. */
PVOID         pInstance;   /*  Pointer to instance data. */
ULONG         lFunction;   /*  High-order WORD=flags; low-order WORD=NGreSetDefaultArcParameters. */
BOOL          fSuccess;    /*  Return codes. */

fSuccess = GreSetDefaultArcParameters(hdc,
             pArcParms, pInstance, lFunction);
```

GreSetDefaultArcParameters Parameter - hdc

hdc (HDC) - input

Device context handle.

GreSetDefaultArcParameters Parameter - pArcParms

pArcParms (PARCPARAMS) - input
Pointer to the default arc parameters.

Pointer to an ARCPARAMS structure:

lP	P coefficient
lQ	Q coefficient
lR	R coefficient
lS	S coefficient

GreSetDefaultArcParameters Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetDefaultArcParameters Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetDefaultArcParameters.

GreSetDefaultArcParameters Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSetDefaultArcParameters - Parameters

hdc (HDC) - input
Device context handle.

pArcParms (PARCPARAMS) - input
Pointer to the default arc parameters.

Pointer to an ARCPARAMS structure:

lP	P coefficient
lQ	Q coefficient
lR	R coefficient
lS	S coefficient

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetDefaultArcParameters.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSetDefaultArcParameters - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreSetDefaultAttributes

GreSetDefaultAttributes - Syntax

Description

GreSetDefaultAttributes sets the default attributes for the specified primitive type according to the flags set in flAttrsMask.

This function is supported by the graphics engine.

```
#define INCL_GRE_DEVSUPPORT
#include <os2.h>

HDC         hdc;           /*  Device context handle. */
LONG        lPrimType;     /*  Bundle primitive type. */
ULONG       flAttrsMask;   /*  Flags indicating which attributes are to be modified. */
PBUNDLE     pAttrs;        /*  Pointer. */
PVOID       pInstance;     /*  Pointer to instance data. */
ULONG       lFunction;     /*  High-order WORD=flags; low-order WORD=NGreSetDefaultAttributes. */
BOOL        fSuccess;      /*  Return codes. */

fSuccess = GreSetDefaultAttributes(hdc, lPrimType,
             flAttrsMask, pAttrs, pInstance,
             lFunction);
```

GreSetDefaultAttributes Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetDefaultAttributes Parameter - lPrimType

lPrimType (LONG) - input
Bundle primitive type.

Indicates the bundle type. Valid primitive values are:

PRIM_LINE	Line attribute bundle
PRIM_CHAR	Character attribute bundle
PRIM_MARKER	Marker attribute bundle
PRIM_AREA	Pattern attribute bundle
PRIM_IMAGE	Image attribute bundle

GreSetDefaultAttributes Parameter - flAttrsMask

flAttrsMask (ULONG) - input
Flags indicating which attributes are to be modified.

GreSetDefaultAttributes Parameter - pAttrs

pAttrs (PBUNDLE) - input
Pointer.

This is a pointer to the fixed-format bundle record containing the attribute values to be set as specified by flAttrsMask. Only the attribute fields corresponding to attribute flags set in flAttrsMask contain valid values.

GreSetDefaultAttributes Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetDefaultAttributes Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetDefaultAttributes.

GreSetDefaultAttributes Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_CHAR_DIRECTION_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CHAR_SET_ATTR
PMERR_INV_CHAR_SHEAR_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COORD_SPACE
PMERR_INV_GEOM_LINE_WIDTH_ATTR
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_END_ATTR
PMERR_INV_LINE_JOIN_ATTR
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_LINE_WIDTH_ATTR
PMERR_INV_MARKER_SET_ATTR
PMERR_INV_MARKER_SYMBOL_ATTR
PMERR_INV_MIX_ATTR
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_PRIMITIVE_TYPE
PMERR_INV_SETID
PMERR_UNSUPPORTED_ATTR

PMERR_UNSUPPORTED_ATTR_VALUE

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSetDefaultAttributes - Parameters

hdc (HDC) - input
Device context handle.

lPrimType (LONG) - input
Bundle primitive type.

Indicates the bundle type. Valid primitive values are:

PRIM_LINE	Line attribute bundle
PRIM_CHAR	Character attribute bundle
PRIM_MARKER	Marker attribute bundle
PRIM_AREA	Pattern attribute bundle
PRIM_IMAGE	Image attribute bundle

flAttrsMask (ULONG) - input
Flags indicating which attributes are to be modified.

pAttrs (PBUNDLE) - input
Pointer.

This is a pointer to the fixed-format bundle record containing the attribute values to be set as specified by flAttrsMask. Only the attribute fields corresponding to attribute flags set in flAttrsMask contain valid values.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetDefaultAttributes.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_COORDINATE_OVERFLOW
PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HDC_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_CHAR_DIRECTION_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CHAR_SET_ATTR
PMERR_INV_CHAR_SHEAR_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COORD_SPACE
PMERR_INV_GEOM_LINE_WIDTH_ATTR
PMERR_INV_HDC
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_END_ATTR
PMERR_INV_LINE_JOIN_ATTR
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_LINE_WIDTH_ATTR
PMERR_INV_MARKER_SET_ATTR

PMERR_INV_MARKER_SYMBOL_ATTR
PMERR_INV_MIX_ATTR
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_PRIMITIVE_TYPE
PMERR_INV_SETID
PMERR_UNSUPPORTED_ATTR
PMERR_UNSUPPORTED_ATTR_VALUE

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSetDefaultAttributes - Remarks

Only attributes whose flags are set in flAttrsMask are modified. They are set to the value specified by pAttrs.

When this function occurs within a path bracket, it must not be used to set the geometric line width for line attributes. In addition, it must not be used to set the foreground and background colors and mixes for character and marker attributes.

GreSetDefaultAttributes - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSetDefaultViewingLimits

GreSetDefaultViewingLimits - Syntax

Description

GreSetDefaultViewingLimits sets the boundaries of the default viewing (clip) limits to the values specified by prclViewingLimits.

This function is supported by the graphics engine.

```
#define INCL_GRE_DEVSUPPORT
#include <os2.h>

HDC       hdc;                /*  Device context handle. */
PRECTL    prclViewingLimits;  /*  Pointer to limits of viewing area. */
PVOID     pInstance;          /*  Pointer to instance data. */
ULONG     lFunction;          /*  High-order WORD=flags; low-order WORD=NGreSetDefaultViewingLimits. */
BOOL      fSuccess;           /*  Return codes. */

fSuccess = GreSetDefaultViewingLimits(hdc,
             prclViewingLimits, pInstance,
             lFunction);
```

GreSetDefaultViewingLimits Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetDefaultViewingLimits Parameter - prclViewingLimits

prclViewingLimits (PRECTL) - input
Pointer to limits of viewing area.

RECTL structure:

xLeft	Minimum x-coordinate of viewing limits
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of viewing limits
yTop	Maximum y-coordinate

GreSetDefaultViewingLimits Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetDefaultViewingLimits Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetDefaultViewingLimits.

GreSetDefaultViewingLimits Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for

failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_COORDINATE
PMERR_INV_GRAPHICS_FIELD
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSetDefaultViewingLimits - Parameters

hdc (HDC) - input
Device context handle.

prclViewingLimits (PRECTL) - input
Pointer to limits of viewing area.

RECTL structure:

xLeft	Minimum x-coordinate of viewing limits
yBottom	Minimum y-coordinate
xRight	Maximum x-coordinate of viewing limits
yTop	Maximum y-coordinate

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetDefaultViewingLimits.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_COORDINATE
PMERR_INV_GRAPHICS_FIELD
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSetDefaultViewingLimits - Remarks

The current viewing limits are unaffected by a call to this function (see GreGetViewingLimits).

GreSetDefaultViewingLimits - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSetDeviceSurface

GreSetDeviceSurface - Syntax

Description:

GreSetDeviceSurface allows printer devices to dynamically optimize their frame buffer's height, width, bits per pel and DevCaps.

```
#include <os2.h>

HDC               hdc;   /*  The device context handle. */
PDEVICESURFACE    pDS;   /*  Pointer to information needed by the graphics engine. */
LONG              rc;    /*  Return codes. */

rc = GreSetDeviceSurface(hdc, pDS);
```

GreSetDeviceSurface Parameter - hdc

hdc (HDC) - input
The device context handle.

GreSetDeviceSurface Parameter - pDS

pDS (PDEVICESURFACE) - input
Pointer to information needed by the graphics engine.

ulLength	Length of the DEVICESURFACE data structure.
SurfaceBmapInfo	Information used by the graphics engine when rastering to a linear address. (See BMAPINFO in Data Types)
ulDSFlgs	Miscellaneous flags used by the graphics engine for rendering of rasterized data.
	Color format flag
	DS_MONO_INVERT Mono ROPs are

inverted.

Destination color format flags

DS_COLOR_RGB

Color is in RGB.

DS_COLOR_RGB2

Color is in RGB2 (default).

DS_COLOR_24BIT_BGR

First byte=blue (default).

DS_COLOR_24BIT_RGB

First byte=red, second byte=green, third byte=blue.

DS_COLOR_16BIT_565_BGR

Blue

5 bits (default)

Green

6 bits

Red

5 bits

|b|b|b|b|b|g|g|g|g|g|g|r

DS_COLOR_16BIT_565_RGB

Red

5 bits

Green

6 bits

Blue

5 bits

|r|r|r|r|r|g|g|g|g|g|g|b

DS_COLOR_16BIT_1555_BGR

Overlay

1

Blue

5 bits

Green

5 bits

Red

5 bits

|o|b|b|b|b|b|g|g|g|g|g|r

DS_COLOR_16BIT_1555_RGB

Overlay

1

Red

5 bits

Green

5 bits

Blue

5 bits

|o|r|r|r|r|r|g|g|g|g|g|b

Direction flags

DS_BOTTOMTOP	Bottom-to-top is Y position.
DS_TOPBOTTOM	Top-to-bottom is Y position.

Hardcopy flag

DS_QUEUED_RAW_DEFAULT	Set QUEUED to RAW as default.

Notification flag

DS_NOTIFY_LAST_SDBITBLT	Last banded blit per API bit map transfer.

Scanline alignment flags

DS_BYTE_ALIGNMENT	DS scan line alignment.
DS_2BYTE_ALIGNMENT	DS scan line alignment.
DS_4BYTE_ALIGNMENT	DS scan line alignment.
DS_KEEP_EXTFORMAT	Variable alignment; let application decide.

Surface description flag

DS_MEMORY	Memory surface.

ulStyleRatio — Default style ratio used by the current device.

abmapinfoDefPattern[ ] — Set of default patterns supplied by the graphics engine that can be changed by the presentation driver. [DEFAULT_PATTERNS_NUMBER].

pHWPalette — Default hardware palette for the current device.

DitherMatrix — Default is the generic dither support for hardcopy devices.

pfnDevLockDC — Pointer to allow the device driver to retrieve its device-specific information from the graphics engine. Pointer is stored by the device driver.

pfnDevUnLockDC — Pointer to the function to release the lock on the current DC. Pointer is stored by the device driver.

pfnBitBlt — Pointer defaults to SOFTDRAW's handling of the new SDBitBlt function. If a device does not have linear address support or would rather handle preclipped bit map transfers itself, then it should hook out this function with its own routine.

pfnLine — Pointer defaults to SOFTDRAW's handling of the new SDLine function. If a device does not have linear address support or would rather handle preclipped line operations itself, then it should hook out this function with its own routine.

pfnTextBlt — Used in conjunction with DS_DOWNLOAD_FONTS and DS_DEVICE_FONTS. If flags are not specified, text strings will be simulated through bitblts.

pfnLockPhysDev — Pointer prevents other processes from accessing the physical hardware between the lock and unlock.

pfnUnLockPhysDev — Pointer releases the physical hardware for access by other processes.

ulhddc — Handle to device's device context.

ulReserved[4] — Reserved by system.

ulCapsCnt — Size in ULONGs of the *DevCaps* field.

DevCaps[ ] — The device capabilities array in OS/2 2.0 and previous versions. Array must be filled out in order for the graphics engine to handle a

call to the DevQueryCaps function. [CAPS_MAX_CAPS + 1]

GreSetDeviceSurface Return Value - rc

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

Zero (0)	Success
Other	Failure

GreSetDeviceSurface - Parameters

hdc (HDC) - input
The device context handle.

pDS (PDEVICESURFACE) - input
Pointer to information needed by the graphics engine.

ulLength	Length of the DEVICESURFACE data structure.
SurfaceBmapInfo	Information used by the graphics engine when rastering to a linear address. (See BMAPINFO in Data Types)
ulDSFlgs	Miscellaneous flags used by the graphics engine for rendering of rasterized data.

Color format flag

DS_MONO_INVERT
Mono ROPs are inverted.

Destination color format flags

DS_COLOR_RGB
Color is in RGB.

DS_COLOR_RGB2
Color is in RGB2 (default).

DS_COLOR_24BIT_BGR
First byte=blue (default).

DS_COLOR_24BIT_RGB
First byte=red, second byte=green, third byte=blue.

DS_COLOR_16BIT_565_BGR

Blue

5
b
i
t

s
(
d
e
f
a
u
l
t
)

Green

6
b
i
t
s

Red

5
b
i
t
s

|b|b|b|b|b|g|g|g|g|g|g|r

DS_COLOR_16BIT_565_RGB

Red

5
b
i
t
s

Green

6
b
i
t
s

Blue

5
b
i
t
s

|r|r|r|r|r|g|g|g|g|g|g|b

DS_COLOR_16BIT_1555_BGR

Overlay

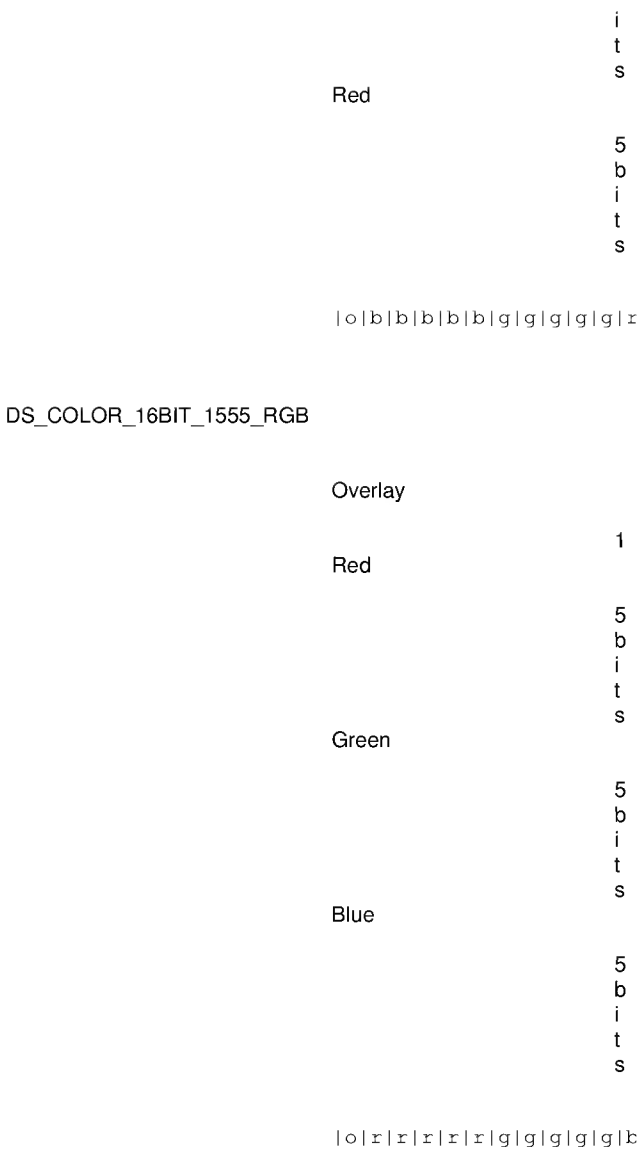
1

Blue

5
b
i
t
s

Green

5
b



Direction flags

DS_BOTTOMTOP	Bottom-to-top is Y position.
DS_TOPBOTTOM	Top-to-bottom is Y position.

Hardcopy flag

DS_QUEUED_RAW_DEFAULT	Set QUEUED to RAW as default.

Notification flag

DS_NOTIFY_LAST_SDBITBLT	Last banded blit per API bit map transfer.

Scanline alignment flags

DS_BYTE_ALIGNMENT	DS scan line alignment.
DS_2BYTE_ALIGNMENT	DS scan line alignment.
DS_4BYTE_ALIGNMENT	DS scan line alignment.
DS_KEEP_EXTFORMAT	Variable alignment; let application decide.

Surface description flag

DS_MEMORY	Memory surface.

ulStyleRatio	Default style ratio used by the current device.
abmapinfoDefPattern[ ]	Set of default patterns supplied by the graphics engine that can be changed by the presentation driver. [DEFAULT_PATTERNS_NUMBER].
pHWPalette	Default hardware palette for the current device.
DitherMatrix	Default is the generic dither support for hardcopy devices.
pfnDevLockDC	Pointer to allow the device driver to retrieve its device-specific information from the graphics engine. Pointer is stored by the device driver.
pfnDevUnLockDC	Pointer to the function to release the lock on the current DC. Pointer is stored by the device driver.
pfnBitBlt	Pointer defaults to SOFTDRAW's handling of the new SDBitBlt function. If a device does not have linear address support or would rather handle preclipped bit map transfers itself, then it should hook out this function with its own routine.
pfnLine	Pointer defaults to SOFTDRAW's handling of the new SDLine function. If a device does not have linear address support or would rather handle preclipped line operations itself, then it should hook out this function with its own routine.
pfnReserved	Reserved by system.
pfnLockPhysDev	Pointer prevents other processes from accessing the physical hardware between the lock and unlock.
pfnUnLockPhysDev	Pointer releases the physical hardware for access by other processes.
ulReserved[5]	Reserved by system.
ulCapsCnt	Size in ULONGs of the *DevCaps* field.
DevCaps[ ]	The device capabilities array in OS/2 2.0 and previous versions. Array must be filled out in order for the graphics engine to handle a call to the DevQueryCaps function. [CAPS_MAX_CAPS + 1]

rc (LONG) - returns
Return codes.

The handling routine should return a LONG integer. Valid values are:

Zero (0)	Success
Other	Failure

GreSetDeviceSurface - Remarks

Printer devices can change their surface definition between pages using the GreSetDeviceSurface call. (This call must be imported.)

GreSetDeviceSurface - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSetGlobalAttribute

GreSetGlobalAttribute - Syntax

Description

GreSetGlobalAttribute sets the specified attribute in the pen, pattern, character, image, and marker bundles.

This function is supported by the graphics engine.

```
#define INCL_GRE_DEVSUPPORT
#include <os2.h>

HDC      hdc;          /*  Device context handle. */
LONG     lAttrType;    /*  Specifies the attribute type. */
LONG     lAttribute;   /*  New attribute value. */
ULONG    flOptions;
PVOID    pInstance;    /*  Pointer to instance data. */
ULONG    lFunction;    /*  High-order WORD=flags; low-order WORD=NGreSetGlobalAttribute. */
BOOL     fSuccess;     /*  Return codes. */

fSuccess = GreSetGlobalAttribute(hdc, lAttrType,
             lAttribute, flOptions, pInstance,
             lFunction);
```

GreSetGlobalAttribute Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetGlobalAttribute Parameter - lAttrType

lAttrType (LONG) - input
Specifies the attribute type.

Attribute types:

ATYPE_COLOR	Foreground color
ATYPE_BACK_COLOR	Background color
ATYPE_MIX_MODE	Foreground mix
ATYPE_BACK_MIX_MODE	Background mix

Note: ATYPE_BACK_COLOR and ATYPE_BACK_MIX_MODE do not apply to the line bundle.

GreSetGlobalAttribute Parameter - lAttribute

lAttribute (LONG) - input
New attribute value.

GreSetGlobalAttribute Parameter - flOptions

flOptions (ULONG) - input
The only allowable option flag is GATTR_DEFAULT, which specifies that the attribute indicated by lAttrType should be set to its default value. If the GATTR_DEFAULT flag is not set, the function sets the attribute to the value specified by lAttribute.

GreSetGlobalAttribute Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetGlobalAttribute Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetGlobalAttribute.

GreSetGlobalAttribute Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_COLOR_ATTR
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_MIX_ATTR
PMERR_INV_RESET_OPTIONS

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSetGlobalAttribute - Parameters

hdc (HDC) - input
Device context handle.

lAttrType (LONG) - input
Specifies the attribute type.

Attribute types:

ATYPE_COLOR	Foreground color
ATYPE_BACK_COLOR	Background color
ATYPE_MIX_MODE	Foreground mix
ATYPE_BACK_MIX_MODE	Background mix

Note: ATYPE_BACK_COLOR and ATYPE_BACK_MIX_MODE do not apply to the line bundle.

lAttribute (LONG) - input
New attribute value.

flOptions (ULONG) - input
The only allowable option flag is GATTR_DEFAULT, which specifies that the attribute indicated by lAttrType should be set to its default value. If the GATTR_DEFAULT flag is not set, the function sets the attribute to the value specified by lAttribute.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetGlobalAttribute.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_HDC_BUSY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_COLOR_ATTR
PMERR_INV_HDC
PMERR_INV_IN_AREA
PMERR_INV_MIX_ATTR
PMERR_INV_RESET_OPTIONS

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSetGlobalAttribute - Remarks

The attribute can be set to its default value or to a specified value.

GreSetGlobalAttribute - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSetHandle

GreSetHandle - Syntax

Description

GreSetHandle stores a handle or variable in the device context.

This function is supported by the graphics engine.

```
#define INCL_GRE_DCS
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    hHandle;     /*  Handle to be associated with hdc. */
ULONG    iIndex;      /*  Index value of the handle in the range 0-3. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreSetHandle. */
BOOL     fSuccess;    /*  Return codes. */

fSuccess = GreSetHandle(hdc, hHandle, iIndex,
             pInstance, lFunction);
```

GreSetHandle Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetHandle Parameter - hHandle

hHandle (ULONG) - input
Handle to be associated with hdc.

GreSetHandle Parameter - iIndex

iIndex (ULONG) - input
Index value of the handle in the range *0-3* .

For a normal device context, this is a reserved parameter.

GreSetHandle Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetHandle Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetHandle.

GreSetHandle Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSetHandle - Parameters

hdc (HDC) - input
Device context handle.

hHandle (ULONG) - input
Handle to be associated with hdc.

iIndex (ULONG) - input
Index value of the handle in the range *0-3* .

For a normal device context, this is a reserved parameter.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetHandle.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSetHandle - Remarks

Up to four handles can be stored in a DC at one time. The presentation driver can only use this function for device contexts that it has created for its own use with GreOpenDC.

GreSetHandle - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreSetProcessControl

GreSetProcessControl - Syntax

Description

GreSetProcessControl provides a mechanism to control drawing, boundary computation, and correlation.

This function is supported by the graphics engine.

```
#define INCL_GRE_DCS
```

```
#include <os2.h>

HDC      hdc;        /*  Device context handle. */
ULONG    flMask;
ULONG    flProcess;  /*  Indicates which process-control flags to set. */
PVOID    pInstance;  /*  Pointer to instance data. */
ULONG    lFunction;  /*  High-order WORD=flags; low-order WORD=NGreSetProcessControl. */
BOOL     fSuccess;   /*  Return codes. */

fSuccess = GreSetProcessControl(hdc, flMask,
             flProcess, pInstance, lFunction);
```

GreSetProcessControl Parameter - hdc

hdc (HDC) - input
Device context handle.

GreSetProcessControl Parameter - flMask

flMask (ULONG) - input
Only those flags with the corresponding bit in this parameter set are modified.

GreSetProcessControl Parameter - flProcess

flProcess (ULONG) - input
Indicates which process-control flags to set.

Process-control flags:

PCTL_DRAW	Has no effect on GreErasePS. If set, drawing primitives should appear on screen. Otherwise, output operations such as GreBitblt, GrePaintRegion, GreSetPel and other drawing primitives are not displayed.
PCTL_BOUND	Set to indicate that GPI_BOUNDS must be accumulated.
PCTL_CORRELATE	Set to indicate that correlation is to be done.
PCTL_USERBOUNDS	When set, indicates that USER_BOUNDS are to be collected for the window manager.

GreSetProcessControl Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreSetProcessControl Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetProcessControl.

GreSetProcessControl Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSetProcessControl - Parameters

hdc (HDC) - input
Device context handle.

flMask (ULONG) - input
Only those flags with the corresponding bit in this parameter set are modified.

flProcess (ULONG) - input
Indicates which process-control flags to set.

Process-control flags:

PCTL_DRAW	Has no effect on GreErasePS. If set, drawing primitives should appear on screen. Otherwise, output operations such as GreBitblt, GrePaintRegion, GreSetPel and other drawing primitives are not displayed.
PCTL_BOUND	Set to indicate that GPI_BOUNDS must be accumulated.
PCTL_CORRELATE	Set to indicate that correlation is to be done.
PCTL_USERBOUNDS	When set, indicates that USER_BOUNDS are to be collected for the window manager.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreSetProcessControl.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_HDC_BUSY
PMERR_INV_HDC

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreSetProcessControl - Remarks

Bounds are returned in graphics model-space coordinates. If a composite transform is applied to the drawing primitives, the bounds values must be transformed back to their original values before merging with the previous bounds values.

Correlation is performed in page-coordinate space on the output of primitives that have been clipped only to the viewing limits and graphics field. Note that correlation is performed for all functions *except* alphanumeric functions and GreErasePS. Boundary computation is performed for all functions *except* GreErasePS.

GreSetProcessControl - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreStartJournalFile

GreStartJournalFile - Syntax

Description

GreStartJournalFile starts the journaling process.

This function is supported by the graphics engine.

```
#define INCL_GRE_JOURNALING
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    hJournal;    /*  Journal file handle. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreStartJournalFile. */
BOOL     fSuccess;    /*  Return codes. */
```

```
fSuccess = GreStartJournalFile(hdc, hJournal,
             pInstance, lFunction);
```

GreStartJournalFile Parameter - hdc

hdc (HDC) - input
Device context handle.

GreStartJournalFile Parameter - hJournal

hJournal (ULONG) - input
Journal file handle.

GreStartJournalFile Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreStartJournalFile Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreStartJournalFile.

GreStartJournalFile Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_BITMAP_IS_SELECTED
PMERR_COORDINATE_OVERFLOW

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_CHAR_DIRECTION_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_DC_DATA
PMERR_INV_DC_TYPE
PMERR_INV_DRIVER_NAME
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_HJOURNAL
PMERR_INV_HRGN
PMERR_INV_ID
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_METAFILE
PMERR_INV_MIX_ATTR
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_SCAN_START
PMERR_JFILE_BUSY
PMERR_RAM_JNL_FILE_TOO_SMALL

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreStartJournalFile - Parameters

hdc (HDC) - input
Device context handle.

hJournal (ULONG) - input
Journal file handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreStartJournalFile.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_BITMAP_IS_SELECTED
PMERR_COORDINATE_OVERFLOW

PMERR_DEV_FUNC_NOT_INSTALLED
PMERR_EXCEEDS_MAX_SEG_LENGTH
PMERR_HBITMAP_BUSY
PMERR_HDC_BUSY
PMERR_HUGE_FONTS_NOT_SUPPORTED
PMERR_INSUFFICIENT_MEMORY
PMERR_INV_BACKGROUND_COL_ATTR
PMERR_INV_BACKGROUND_MIX_ATTR
PMERR_INV_BITMAP_DIMENSION
PMERR_INV_CHAR_DIRECTION_ATTR
PMERR_INV_CHAR_MODE_ATTR
PMERR_INV_CODEPAGE
PMERR_INV_COLOR_ATTR
PMERR_INV_COORD_SPACE
PMERR_INV_COORDINATE
PMERR_INV_DC_DATA
PMERR_INV_DC_TYPE
PMERR_INV_DRIVER_NAME
PMERR_INV_HBITMAP
PMERR_INV_HDC
PMERR_INV_HJOURNAL
PMERR_INV_HRGN
PMERR_INV_ID
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_INFO_TABLE
PMERR_INV_LENGTH_OR_COUNT
PMERR_INV_LINE_TYPE_ATTR
PMERR_INV_METAFILE
PMERR_INV_MIX_ATTR
PMERR_INV_PATTERN_REF_PT_ATTR
PMERR_INV_PATTERN_SET_ATTR
PMERR_INV_PATTERN_SET_FONT
PMERR_INV_RECT
PMERR_INV_REGION_CONTROL
PMERR_INV_SCAN_START
PMERR_JFILE_BUSY
PMERR_RAM_JNL_FILE_TOO_SMALL

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreStartJournalFile - Remarks

This function opens the previously created journal file and turns ON the COM_RECORDING bit. Subsequent calls to this DC drop through GreAccumulateJournalFile until GreStopJournalFile is called.

Note: The COM_DRAW bit is turned OFF until GreStopJournalFile is called.

GreStartJournalFile - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

GreStopJournalFile

GreStopJournalFile - Syntax

Description

GreStopJournalFile writes the END_OF_JOURNALFILE marker into the journal file, closes the journal file, and turns OFF the COM_RECORDING bit.

This function is supported by the graphics engine.

```
#define INCL_GRE_JOURNALING
#include <os2.h>

HDC      hdc;         /*  Device context handle. */
ULONG    hJournal;    /*  Journal file handle. */
PVOID    pInstance;   /*  Pointer to instance data. */
ULONG    lFunction;   /*  High-order WORD=flags; low-order WORD=NGreStopJournalFile. */
BOOL     rc;          /*  Return codes. */

rc = GreStopJournalFile(hdc, hJournal, pInstance,
       lFunction);
```

GreStopJournalFile Parameter - hdc

hdc (HDC) - input
Device context handle.

GreStopJournalFile Parameter - hJournal

hJournal (ULONG) - input
Journal file handle.

GreStopJournalFile Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreStopJournalFile Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreStopJournalFile.

GreStopJournalFile Return Value - rc

rc (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN. (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_INV_DC_DATA
PMERR_INV_HDC
PMERR_INV_HJOURNAL
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_METAFILE
PMERR_JFILE_BUSY
PMERR_RAM_JNL_FILE_TOO_SMALL

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreStopJournalFile - Parameters

hdc (HDC) - input
Device context handle.

hJournal (ULONG) - input
Journal file handle.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreStopJournalFile.

rc (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN. (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_INV_DC_DATA
PMERR_INV_HDC
PMERR_INV_HJOURNAL
PMERR_INV_IN_AREA
PMERR_INV_IN_PATH
PMERR_INV_METAFILE

PMERR_JFILE_BUSY
PMERR_RAM_JNL_FILE_TOO_SMALL

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreStopJournalFile - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

GreUnLoadFont

GreUnLoadFont - Syntax

Description

GreUnLoadFont unloads the private font definitions previously loaded from the resource file indicated by pszFilename.

This function is supported by the graphics engine.

```
#define INCL_GRE_FONTS
#include <os2.h>

PSZ      pszFilename;
PVOID    pInstance;     /*  Pointer to instance data. */
ULONG    lFunction;     /*  High-order WORD=flags; low-order WORD=NGreUnLoadFont. */
BOOL     fSuccess;      /*  Return codes. */

fSuccess = GreUnLoadFont(pszFilename, pInstance,
            lFunction);
```

GreUnLoadFont Parameter - pszFilename

pszFilename (PSZ) - input
Pointer to a NULL-terminated string containing the file path and name of the font file.

GreUnLoadFont Parameter - pInstance

pInstance (PVOID) - input
Pointer to instance data.

GreUnLoadFont Parameter - lFunction

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreUnLoadFont.

GreUnLoadFont Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_FONT_FILE_NOT_LOADED

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreUnLoadFont - Parameters

pszFilename (PSZ) - input
Pointer to a NULL-terminated string containing the file path and name of the font file.

pInstance (PVOID) - input
Pointer to instance data.

lFunction (ULONG) - input
High-order WORD=flags; low-order WORD=NGreUnLoadFont.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

Possible Errors Detected: When an error is detected, the graphics engine calls WinSetErrorInfo to post the condition. Reasons for failure of this function include:

PMERR_BASE_ERROR
PMERR_FONT_FILE_NOT_LOADED

Refer to the "Error Explanations" section of the *Presentation Manager Programming Reference* for further explanation.

GreUnLoadFont - Remarks

Before unloading any fonts, the caller must ensure that:

- The fonts are not in use for the current character set.
- The relevant lcids have been deleted (see GreDeleteSetId).

GreUnLoadFont - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

PrtAbort

PrtAbort - Syntax

Description:

PrtAbort terminates operations to the output device identified by hDevice handle.

```
#include <os2.h>

HFILE    hDevice;  /*  The device handle returned by PrtOpen. */

PrtAbort(hDevice);
```

PrtAbort Parameter - hDevice

hDevice (HFILE) - input
The device handle returned by *PrtOpen* .

PrtAbort - Return Value

None.

PrtAbort - Parameters

hDevice (HFILE) - input
The device handle returned by *PrtOpen* .

None.

PrtAbort - Remarks

Any output data that is held in buffers for the physical device driver is emptied. PrtAbort does not close the device, therefore, the presentation driver must call PrtClose after ending operations. If PrtWrite is called to write to a device whose output has been ended, the call is not honored by the system.

Presentation drivers do not use PrtClose to end an output operation. The effect of PrtClose is to output any buffered data before closing the device.

PrtAbort - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

PrtAbortDoc

PrtAbortDoc - Syntax

Description:

PrtAbortDoc resets the printer after current print data is flushed. If FLG_COMPLETE_DATA is set, all prior data for this job is sent to the printer. If this flag is clear, PrtAbortDoc first flushes all current write requests for this job and then sends the buffer of data to reset the printer.

```
#define INCL_SPL
#define INCL_SPLDOSPRINT
```

```
#include <os2.h>

HFILE    hDevice;  /*  The device handle returned by PrtOpen. */
PVOID    pchData;  /*  Pointer to the data buffer to write after flushing all current write data. */
ULONG    cbData;   /*  Length of data, in bytes, in pchData buffer. */
ULONG    ulFlags;  /*  Abort processing flags. */
ULONG    rc;       /*  Return codes. */

rc = PrtAbortDoc(hDevice, pchData, cbData,
       ulFlags);
```

PrtAbortDoc Parameter - hDevice

hDevice (HFILE) - input
The device handle returned by PrtOpen.

PrtAbortDoc Parameter - pchData

pchData (PVOID) - input
Pointer to the data buffer to write after flushing all current write data.

PrtAbortDoc Parameter - cbData

cbData (ULONG) - input
Length of data, in bytes, in *pchData* buffer.

PrtAbortDoc Parameter - ulFlags

ulFlags (ULONG) - input
Abort processing flags.

Value is as follows:

FLG_COMPLETE_DATA - 0x00000001
If this flag is set, all prior data for this job must be sent to the printer. If this flag is clear, any buffers waiting to be sent to the printer can be flushed.

PrtAbortDoc Return Value - rc

rc (ULONG) - returns

Return codes.

0	Success
6(ERROR_INVALID_HANDLE)	Invalid handle given.
29(ERROR_WRITE_FAULT)	Failure attempting to write reset sequence to the device.
32(ERROR_SHARING_VIOLATION)	Another thread has access to this port using this Prt handle. This can occur if another thread is in the middle of a PrtClose using this Prt handle.

PrtAbortDoc - Parameters

hDevice (HFILE) - input
The device handle returned by PrtOpen.

pchData (PVOID) - input
Pointer to the data buffer to write after flushing all current write data.

cbData (ULONG) - input
Length of data, in bytes, in *pchData* buffer.

ulFlags (ULONG) - input
Abort processing flags.

Value is as follows:

FLG_COMPLETE_DATA - 0x00000001	If this flag is set, all prior data for this job must be sent to the printer. If this flag is clear, any buffers waiting to be sent to the printer can be flushed.

rc (ULONG) - returns
Return codes.

0	Success
6(ERROR_INVALID_HANDLE)	Invalid handle given.
29(ERROR_WRITE_FAULT)	Failure attempting to write reset sequence to the device.
32(ERROR_SHARING_VIOLATION)	Another thread has access to this port using this Prt handle. This can occur if another thread is in the middle of a PrtClose using this Prt handle.

PrtAbortDoc - Remarks

The return code must be checked to ensure that the reset sequence has been sent to the printer.

PrtClose must still be issued to close the device connection.

PrtAbortDoc - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

PrtClose

PrtClose - Syntax

Description:

PrtClose closes the output device identified by hDevice and releases the spooler port semaphore. It also calls the port driver for port termination.

```
#include <os2.h>

HFILE    hDevice;  /*  The device handle returned by PrtOpen. */
ULONG    rc;       /*  Return codes. */

rc = PrtClose(hDevice);
```

PrtClose Parameter - hDevice

hDevice (HFILE) - input
The device handle returned by *PrtOpen* .

PrtClose Return Value - rc

rc (ULONG) - returns
Return codes.

This function returns the same codes as DosClose:

ERROR_ACCESS_DENIED
ERROR_FILE_NOT_FOUND
ERROR_INVALID_HANDLE
NO_ERROR

PrtClose - Parameters

hDevice (HFILE) - input
The device handle returned by *PrtOpen* .

rc (ULONG) - returns
Return codes.

This function returns the same codes as DosClose:

ERROR_ACCESS_DENIED
ERROR_FILE_NOT_FOUND
ERROR_INVALID_HANDLE
NO_ERROR

PrtClose - Remarks

If this function returns an error, it is reissued to close the device.

PrtClose - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

PrtDevIOCtl

PrtDevIOCtl - Syntax

Description:

PrtDevIOCtl passes device-specific commands to the device. PrtDevIOCtl is an emulation of DosDevIOCtl.

Note: Under OS/2, PM port drivers are responsible for initializing printer ports; presentation drivers should not need to use this function.

```
#include <os2.h>

PVOID    pData;       /*  A pointer to a data packet. */
PVOID    pParms;      /*  The parameter list. */
ULONG    ulFunction;  /*  The function number. */
ULONG    ulCategory;  /*  The category code. */
HFILE    hDevice;     /*  The device handle returned by PrtOpen. */
ULONG    rc;          /*  Return codes. */
```

```
rc = PrtDevIOCtl(pData, pParms, ulFunction,
       ulCategory, hDevice);
```

PrtDevIOCtl Parameter - pData

pData (PVOID) - input
A pointer to a data packet.

PrtDevIOCtl Parameter - pParms

pParms (PVOID) - input
The parameter list.

PrtDevIOCtl Parameter - ulFunction

ulFunction (ULONG) - input
The function number.

PrtDevIOCtl Parameter - ulCategory

ulCategory (ULONG) - input
The category code.

PrtDevIOCtl Parameter - hDevice

hDevice (HFILE) - input
The device handle returned by *PrtOpen* .

PrtDevIOCtl Return Value - rc

rc (ULONG) - returns
Return codes.

This function returns the same codes as DosDevIOCtl:

ERROR_BAD_DRIVER_LEVEL
ERROR_GEN_FAILURE
ERROR_INVALID_CATEGORY
ERROR_INVALID_DRIVE
ERROR_INVALID_FUNCTION
ERROR_INVALID_HANDLE
ERROR_INVALID_PARAMETER
ERROR_MONITORS_NOT_SUPPORTED
ERROR_PROTECTION_VIOLATION
ERROR_UNCERTAIN_MEDIA
NO_ERROR

PrtDevIOCtl - Parameters

pData (PVOID) - input
A pointer to a data packet.

pParms (PVOID) - input
The parameter list.

ulFunction (ULONG) - input
The function number.

ulCategory (ULONG) - input
The category code.

hDevice (HFILE) - input
The device handle returned by *PrtOpen* .

rc (ULONG) - returns
Return codes.

This function returns the same codes as DosDevIOCtl:

ERROR_BAD_DRIVER_LEVEL
ERROR_GEN_FAILURE
ERROR_INVALID_CATEGORY
ERROR_INVALID_DRIVE
ERROR_INVALID_FUNCTION
ERROR_INVALID_HANDLE
ERROR_INVALID_PARAMETER
ERROR_MONITORS_NOT_SUPPORTED
ERROR_PROTECTION_VIOLATION
ERROR_UNCERTAIN_MEDIA
NO_ERROR

PrtDevIOCtl - Remarks

For a full description of the parameters, see the function "DosDevIOCtl" in the *Presentation Manager Programming Reference* . For further information about the IOCtl interface, see the chapter on generic IOCtl commands in the *OS/2 Physical Device Driver Reference* .

PrtDevIOCtl - Topics

Select an item:

Syntax
Parameters
Returns
Remarks
Glossary

PrtNewPage

PrtNewPage - Syntax

Description:

PrtNewPage signals the beginning of a new output page. This function call is issued for the first page in a job with page number 1, to signal to the spooler that the printer driver can determine print job page boundaries. This call also sets the *ulPagesSent* field in PRJINFO4 for the print job. See Data Types for a description of the PRJINFO4 data structure.

```
#define INCL_SPL
#define INCL_SPLDOSPRINT
#include <os2.h>

HFILE    hDevice;       /*  The device handle returned by PrtOpen. */
ULONG    ulPageNumber;  /*  The new page number. */
ULONG    rc;            /*  Return codes. */

rc = PrtNewPage(hDevice, ulPageNumber);
```

PrtNewPage Parameter - hDevice

hDevice (HFILE) - input
The device handle returned by PrtOpen.

PrtNewPage Parameter - ulPageNumber

ulPageNumber (ULONG) - input
The new page number.

PrtNewPage Return Value - rc

rc (ULONG) - returns

Return codes.

0	Success
6(ERROR_INVALID_HANDLE)	Invalid handle given.

PrtNewPage - Parameters

hDevice (HFILE) - input
The device handle returned by PrtOpen.

ulPageNumber (ULONG) - input
The new page number.

rc (ULONG) - returns
Return codes.

0	Success
6(ERROR_INVALID_HANDLE)	Invalid handle given.

PrtNewPage - Remarks

PrtNewPage might not be issued for raw jobs. Some printer drivers might not be able to determine page breaks for raw print jobs.

PrtNewPage - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

PrtOpen

PrtOpen - Syntax

Description:

PrtOpen opens a device (port), file, named pipe, or UNC name, and returns its spooler handle in the location addressed by phDevice.

```
#include <os2.h>

PSZ         pszDeviceName;
PHFILE      phDevice;
PUSHORT     psAction;
ULONG       cbFile;
ULONG       ulFat;          /*  The file attribute. */
ULONG       fnOpen;         /*  The Open function type. */
ULONG       flMode;         /*  The Open mode of the file. */
ULONG       lRes;           /*  Reserved.  Must be zero. */
ULONG       rc;             /*  Return codes. */

rc = PrtOpen(pszDeviceName, phDevice, psAction,
       cbFile, ulFat, fnOpen, flMode, lRes);
```

PrtOpen Parameter - pszDeviceName

pszDeviceName (PSZ) - input
A pointer to a string identifying the device.

PrtOpen Parameter - phDevice

phDevice (PHFILE) - input
A pointer to a location for the returned spooler file handle for the device.

PrtOpen Parameter - psAction

psAction (PUSHORT) - input
A pointer to location for the returned value that identifies the action taken by the system.

PrtOpen Parameter - cbFile

cbFile (ULONG) - input
The initial size in bytes, of the output file.

PrtOpen Parameter - ulFat

ulFat (ULONG) - input
The file attribute.

PrtOpen Parameter - fnOpen

fnOpen (ULONG) - input
The Open function type.

PrtOpen Parameter - flMode

flMode (ULONG) - input
The Open mode of the file.

PrtOpen Parameter - lRes

lRes (ULONG) - input
Reserved. Must be zero.

PrtOpen Return Value - rc

rc (ULONG) - returns
Return codes.

This function returns the same codes as DosOpen:

ERROR_ACCESS_DENIED
ERROR_CANNOT_MAKE
ERROR_DEVICE_IN_USE
ERROR_DISK_FULL
ERROR_DRIVE_LOCKED
ERROR_FILE_NOT_FOUND
ERROR_FILENAME_EXCED_RANGE
ERROR_INVALID_ACCESS
ERROR_INVALID_PARAMETER
ERROR_NOT_DOS_DISK
ERROR_OPEN_FAILED
ERROR_PATH_NOT_FOUND
ERROR_PIPE_BUSY
ERROR_SHARING_BUFFER_EXCEEDED
ERROR_SHARING_VIOLATION
ERROR_TOO_MANY_OPEN_FILES
NO_ERROR

PrtOpen - Parameters

pszDeviceName (PSZ) - input
A pointer to a string identifying the device.

phDevice (PHFILE) - input
A pointer to a location for the returned spooler file handle for the device.

psAction (PUSHORT) - input
A pointer to location for the returned value that identifies the action taken by the system.

cbFile (ULONG) - input
The initial size in bytes, of the output file.

ulFat (ULONG) - input
The file attribute.

fnOpen (ULONG) - input
The Open function type.

flMode (ULONG) - input
The Open mode of the file.

lRes (ULONG) - input
Reserved. Must be zero.

rc (ULONG) - returns
Return codes.

This function returns the same codes as DosOpen:

ERROR_ACCESS_DENIED
ERROR_CANNOT_MAKE
ERROR_DEVICE_IN_USE
ERROR_DISK_FULL
ERROR_DRIVE_LOCKED
ERROR_FILE_NOT_FOUND
ERROR_FILENAME_EXCED_RANGE
ERROR_INVALID_ACCESS
ERROR_INVALID_PARAMETER
ERROR_NOT_DOS_DISK
ERROR_OPEN_FAILED
ERROR_PATH_NOT_FOUND
ERROR_PIPE_BUSY
ERROR_SHARING_BUFFER_EXCEEDED
ERROR_SHARING_VIOLATION
ERROR_TOO_MANY_OPEN_FILES
NO_ERROR

PrtOpen - Remarks

If an attempt is made to open a device file that has already been opened by the spooler, this call will block on a semaphore. This call also calls the port driver to initialize the port. PrtOpen is an emulation of DosOpen. For a full description of the parameters, refer to the function "DosOpen" in the *Presentation Manager Programming Reference* .

PrtOpen - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

PrtQuery

PrtQuery - Syntax

Description:

PrtQuery is called to get information about the print device.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

PSZ       pszComputerName;  /*  Destination computer name or NULL for local query. */
PSZ       pszDeviceName;    /*  Port name on pszComputerName. */
ULONG     ulType;           /*  Indicates type of query and options. */
ULONG     ulCommand;        /*  Generic command code. */
PVOID     pInData;          /*  May contain information required by PrtQuery. */
ULONG     cbInData;         /*  Length of information in pInData, in bytes. */
PVOID     pOutData;         /*  Return buffer. */
PULONG    pcbOutData;       /*  Points to the length of the output buffer. */
ULONG     rc;               /*  Return codes. */

rc = PrtQuery(pszComputerName, pszDeviceName,
       ulType, ulCommand, pInData, cbInData,
       pOutData, pcbOutData);
```

PrtQuery Parameter - pszComputerName

pszComputerName (PSZ) - input
Destination computer name or NULL for local query.

Following is an example of a destination computer name:

LS:\\SERVER

PrtQuery Parameter - pszDeviceName

pszDeviceName (PSZ) - input
Port name on *pszComputerName*.

Following is an example of a port name:

LPT1

This may also be the name of a local printer device or local print queue to allow presentation drivers to query printer characteristics during DevOpenDC or DevPostDeviceModes. The spooler will change the destination to be the port that is connected to either the printer device or the print queue.

PrtQuery Parameter - ulType

ulType (ULONG) - input
Indicates type of query and options.

Values are as follows:

TYPE_CACHE_ONLY(0)

Returns information from the protocol converter or port driver's cache only. Does not send anything to the printer.

If the requested information is not available, ERROR_INFO_NOT_AVAIL is returned. The caller should reissue the command with a different *ulType* value.

TYPE_SHORT_WAIT(1)

Returns information via a communications channel with the printer that can be used while a job is being sent to the printer.

An example would be using NPAP or SNMP. If the requested information cannot be returned using this type of channel (for instance, if a PJL command is required in order to get this information), ERROR_INFO_NOT_AVAIL is returned.

TYPE_LONG_WAIT(2)

Returns information using any necessary communications channel with the printer, even if the request has to wait for the current job being sent to the printer to complete.

If the printer supports only one command channel, and this channel can be used while a job is being sent to the printer, TYPE_SHORT_WAIT(1) and TYPE_LONG_WAIT(2) will behave in the same way.

TYPE_CACHE_UPDATE(3)

Returns information from the protocol converter or port driver's cache and also updates the cache in the background.

This is an optional type of query to support and, if it is not supported, will return ERROR_INVALID_FUNCTION.

If the requested information is not available in the cache, ERROR_INFO_NOT_AVAIL is returned. The caller should reissue the command with a different *ulType* value.

TYPE_WITHOUT_WRAPPER(4)

Returns passthru information without any packet wrapper around it.

This is useful for BIDI_READ_PASSTHRU as it allows an application to send and receive printer-interpreter language commands without having to know the format of the BIDI protocol's data wrappers. An example would be removing the NPAP wrapper before returning PostScript response data to an application.

If BIDI_READ_PASSTHRU is used without this value, the calling application requests the data as the printer returns it, possibly with any data wrappers.

Note: If this value is used with any command other than BIDI_READ_PASSTHRU, ERROR_INVALID_PARAMETER is returned.

TYPE_CACHE_WAIT(5)

Returns information from the protocol converter or port driver's cache only. If the data is not available in the cache, use any mechanism available to get the information.

If TYPE_CACHE_WAIT(5) is not supported by the port driver or protocol converter, ERROR_INVALID_FUNCTION is returned.

The value of TYPE_CACHE_WAIT(5) may be used by the print object to issue BIDI_Q_STATUS, in order to get the current status of the printer without waiting to communicate with the printer.

PrtQuery Parameter - ulCommand

ulCommand (ULONG) - input
Generic command code.

See BIDI Command Structures and Command Flow for contents of *pInData* and *pOutData* for each of the following commands.

Current commands are as follows:

BIDI_READ_PASSTHRU(8001h)
Wait for any response from printer. This must be done from within a passthru session. See BIDI_START_PASSTHRU(1Ah) for more details. See BIDI_READ_PASSTHRU(8001h) for input and output parameters.

BIDI_Q_PORT(800Bh)
Query BIDI capabilities of the port. See BIDI_Q_PORT(800Bh) for input and output parameters.

BIDI_Q_SW(800Ch)
Query BIDI software capabilities. See BIDI_Q_SW(800Ch) for input and output parameters.

BIDI_Q_DEVICE(800Dh)
Query printer device characteristics. See BIDI_Q_DEVICE(800Dh) for input and output parameters.

BIDI_Q_INTERPRETER(800Eh)
Query interpreters available in the printer. See BIDI_Q_INTERPRETER(800Eh) for input and output parameters.

BIDI_Q_INPUTBINS(800Fh)
Query input bins. See BIDI_Q_INPUTBINS(800Fh) for input and output parameters.

BIDI_Q_OUTPUTBINS(8010h)
Query output bins. See BIDI_Q_OUTPUTBINS(8010h) for input and output parameters.

BIDI_Q_FONTS(8012h)
Query fonts for a particular interpreter. See BIDI_Q_FONTS(8012h) for input and output parameters.

BIDI_Q_JOBS_COMPLETE(8013h)
Query list of completed jobs. See BIDI_Q_JOBS_COMPLETE(8013h) for input and output parameters.

BIDI_Q_JOBS_QUEUED(8014h)
Query list of queued jobs. See BIDI_Q_JOBS_QUEUED(8014h) for input and output parameters.

BIDI_Q_STATUS(8015h)
Query current status of the print device. See BIDI_Q_STATUS(8015h) for input and output parameters.

BIDI_WAIT_ALERT(8016h)
Wait for unsolicited alerts. See BIDI_WAIT_ALERT(8016h) for input and output parameters.

BIDI_Q_JOBID(8017h)
Query the printer Job ID currently being sent to this port driver. See BIDI_Q_JOBID(8017h) for input and output parameters.

BIDI_Q_RESPONSE_FMT(8018h)
Return format of printer-to-host messages. See BIDI_Q_RESPONSE_FMT(8018h) for input and output parameters.

BIDI_Q_PORTDRV(8019h)
Query port driver's configuration for the port. See BIDI_Q_PORTDRV(8019h) for input and output parameters.

This command does not go to the printer or protocol converter.

BIDI_READ_ALERT(801Dh)
Read an extended alert. See BIDI_READ_ALERT(801Dh) for output parameter.

BIDI_Q_CONVERTER_INFO(8021h)
Return protocol converter information. See BIDI_Q_CONVERTER_INFO(8021h) for output parameter.

BIDI_Q_SPOOLER_VERSION(8022h)
Return spooler version information. See BIDI_Q_SPOOLER_VERSION(8022h) for output parameter.

BIDI_Q_STORAGE(8023h)
: Return printer storage media. See BIDI_Q_STORAGE(8023h) for input and output parameters.

0x00009000 - 0x00009FFF
: Protocol-converter-specific commands.

0x0000B000 - 0x0000BFFF
: Port-driver-specific commands.

PrtQuery Parameter - pInData

pInData (PVOID) - input
: May contain information required by PrtQuery.

See BIDI Command Structures and Command Flow for specific information definitions.

PrtQuery Parameter - cbInData

cbInData (ULONG) - input
: Length of information in *pInData* , in bytes.

PrtQuery Parameter - pOutData

pOutData (PVOID) - output
: Return buffer.

Format of buffer depends on *ulCommand*. For BIDI_READ_PASSTHRU, this will contain BIDI protocol-specific response data and will not be interpreted by the BIDI protocol converter. However, if *ulType* is TYPE_WITHOUT_WRAPPER, any protocol data wrapper is removed before returning the printer-specific information.

For other commands, it will return a structure containing queried information for the command as defined in BIDI Command Structures and Command Flow.

PrtQuery Parameter - pcbOutData

pcbOutData (PULONG) - in/out
: Points to the length of the output buffer.

On entry, this is set to the length of the buffer passed in. On exit, it is updated with the length of data available.

If ERROR_MORE_DATA(234) or NERR_BufTooSmall(2123) is set, this field has the buffer size required to get all available information on exit.

PrtQuery Return Value - rc

rc (ULONG) - returns
Return codes.

0 Success

1(ERROR_INVALID_FUNCTION)
The command code is not supported. This lack of support may occur if a new command is sent to an old spooler, port driver, or protocol converter, or if the printer rejects the command.

2(ERROR_FILE_NOT_FOUND)
The *pszDeviceName* given is not a valid port name with a port driver, a printer device name, or a printer queue name.

5(ERROR_ACCESS_DENIED)
Not authorized to access printer port.

8(ERROR_NOT_ENOUGH_MEMORY)
Not enough memory to satisfy request.

29(ERROR_WRITE_FAULT)
Failure attempting to write to the device. The printer might not have received the query request.

53(ERROR_BAD_NETPATH)
The computer name specified is not available or is invalid.

67(ERROR_BAD_NET_NAME)
The computer name cannot be found.

87(ERROR_INVALID_PARAMETER)
An invalid parameter was specified; most likely an invalid buffer pointer was given.

211(ERROR_INFO_NOT_AVAIL)
The requested information is not available in the protocol converter cache, or the request cannot be sent with the given *ulType* value. The request should be reissued with a different *ulType* value.

234(ERROR_MORE_DATA)
Part of the information requested was returned, but there was not enough room for all the data. **pcbOutData* contains the size of buffer needed to retrieve all the query response data.

640(ERROR_TIMEOUT)
The printer response was not received within the time allotted. The caller may reissue the query, at which time the information might be available.

683(ERROR_BIDI_NO_SUPPORT)
The printer is not in bidirectional mode, therefore query requests are not supported. The port also may be in the process of being shut down.

2123(NERR_BufTooSmall)
No information was returned because the output buffer was too small. **pcbOutData* contains the size of buffer needed to retrieve all the query response data.

0x4005(PMERR_SPL_INV_HSPL)
Not currently in a passthru session.

0x4009(PMERR_SPL_SPOOLER_NOT_INSTALLED)
The spooler is not enabled, therefore bidirectional query requests are not supported.

0x4011(PMERR_SPL_INV_JOB_ID)
No job is being actively sent to the printer port, therefore BIDI_Q_JOBID is not successful.

0x403D(PMERR_SPL_CNV_NOT_INIT)
The protocol converter or port driver has not been initialized for this port.

0x403E(PMERR_SPL_INIT_IN_PROGRESS)
Another thread is still processing BIDI_INIT_PORTDRV or the initial BIDI_Q_PORT for the given portname. The request should be tried again later.

0x403F(PMERR_SPL_TYPE_NOT_AVAIL)

The storage type requested for BIDI_Q_FONTS or BIDI_Q_STORAGE is not available in the printer.

0x4041(PMERR_SPL_SESSION_TERM)
The passthru session has been terminated.

Others
If *pszComputerName* is specified, other network errors may be returned.

Note: Currently you cannot receive more than 64KB of data from a spooler call over the network. Therefore, all returned structures should be limited to less than 64KB for now.

PrtQuery - Parameters

pszComputerName (PSZ) - input
Destination computer name or NULL for local query.

Following is an example of a destination computer name:

```
LS:\\SERVER
```

pszDeviceName (PSZ) - input
Port name on *pszComputerName*.

Following is an example of a port name:

```
LPT1
```

This may also be the name of a local printer device or local print queue to allow presentation drivers to query printer characteristics during DevOpenDC or DevPostDeviceModes. The spooler will change the destination to be the port that is connected to either the printer device or the print queue.

ulType (ULONG) - input
Indicates type of query and options.

Values are as follows:

TYPE_CACHE_ONLY(0)
Returns information from the protocol converter or port driver's cache only. Does not send anything to the printer.

If the requested information is not available, ERROR_INFO_NOT_AVAIL is returned. The caller should reissue the command with a different *ulType* value.

TYPE_SHORT_WAIT(1)
Returns information via a communications channel with the printer that can be used while a job is being sent to the printer.

An example would be using NPAP or SNMP. If the requested information cannot be returned using this type of channel (for instance, if a PJL command is required in order to get this information), ERROR_INFO_NOT_AVAIL is returned.

TYPE_LONG_WAIT(2)
Returns information using any necessary communications channel with the printer, even if the request has to wait for the current job being sent to the printer to complete.

If the printer supports only one command channel, and this channel can be used while a job is being sent to the printer, TYPE_SHORT_WAIT(1) and TYPE_LONG_WAIT(2) will behave in the same way.

TYPE_CACHE_UPDATE(3)
Returns information from the protocol converter or port driver's cache and also updates the cache in the background.

This is an optional type of query to support and, if it is not supported, will return ERROR_INVALID_FUNCTION.

If the requested information is not available in the cache, ERROR_INFO_NOT_AVAIL is returned. The caller should reissue the command with a different *ulType* value.

TYPE_WITHOUT_WRAPPER(4)

Returns passthru information without any packet wrapper around it.

This is useful for BIDI_READ_PASSTHRU as it allows an application to send and receive printer-interpreter language commands without having to know the format of the BIDI protocol's data wrappers. An example would be removing the NPAP wrapper before returning PostScript response data to an application.

If BIDI_READ_PASSTHRU is used without this value, the calling application requests the data as the printer returns it, possibly with any data wrappers.

Note: If this value is used with any command other than BIDI_READ_PASSTHRU, ERROR_INVALID_PARAMETER is returned.

TYPE_CACHE_WAIT(5)

Returns information from the protocol converter or port driver's cache only. If the data is not available in the cache, use any mechanism available to get the information.

If TYPE_CACHE_WAIT(5) is not supported by the port driver or protocol converter, ERROR_INVALID_FUNCTION is returned.

The value of TYPE_CACHE_WAIT(5) may be used by the print object to issue BIDI_Q_STATUS, in order to get the current status of the printer without waiting to communicate with the printer.

ulCommand (ULONG) - input
Generic command code.

See BIDI Command Structures and Command Flow for contents of *pInData* and *pOutData* for each of the following commands.

Current commands are as follows:

BIDI_READ_PASSTHRU(8001h)

Wait for any response from printer. This must be done from within a passthru session. See BIDI_START_PASSTHRU(1Ah) for more details. See BIDI_READ_PASSTHRU(8001h) for input and output parameters.

BIDI_Q_PORT(800Bh)

Query BIDI capabilities of the port. See BIDI_Q_PORT(800Bh) for input and output parameters.

BIDI_Q_SW(800Ch)

Query BIDI software capabilities. See BIDI_Q_SW(800Ch) for input and output parameters.

BIDI_Q_DEVICE(800Dh)

Query printer device characteristics. See BIDI_Q_DEVICE(800Dh) for input and output parameters.

BIDI_Q_INTERPRETER(800Eh)

Query interpreters available in the printer. See BIDI_Q_INTERPRETER(800Eh) for input and output parameters.

BIDI_Q_INPUTBINS(800Fh)

Query input bins. See BIDI_Q_INPUTBINS(800Fh) for input and output parameters.

BIDI_Q_OUTPUTBINS(8010h)

Query output bins. See BIDI_Q_OUTPUTBINS(8010h) for input and output parameters.

BIDI_Q_FONTS(8012h)

Query fonts for a particular interpreter. See BIDI_Q_FONTS(8012h) for input and output parameters.

BIDI_Q_JOBS_COMPLETE(8013h)

Query list of completed jobs. See BIDI_Q_JOBS_COMPLETE(8013h) for input and output parameters.

BIDI_Q_JOBS_QUEUED(8014h)

Query list of queued jobs. See BIDI_Q_JOBS_QUEUED(8014h) for input and output parameters.

BIDI_Q_STATUS(8015h)

Query current status of the print device. See BIDI_Q_STATUS(8015h) for input and output parameters.

BIDI_WAIT_ALERT(8016h)

Wait for unsolicited alerts. See BIDI_WAIT_ALERT(8016h) for input and output parameters.

BIDI_Q_JOBID(8017h)

Query the printer Job ID currently being sent to this port driver. See BIDI_Q_JOBID(8017h) for input and output parameters.

BIDI_Q_RESPONSE_FMT(8018h)

Return format of printer-to-host messages. See BIDI_Q_RESPONSE_FMT(8018h) for input and output parameters.

BIDI_Q_PORTDRV(8019h)
Query port driver's configuration for the port. See BIDI_Q_PORTDRV(8019h) for input and output parameters.

This command does not go to the printer or protocol converter.

BIDI_READ_ALERT(801Dh)
Read an extended alert. See BIDI_READ_ALERT(801Dh) for output parameter.

BIDI_Q_CONVERTER_INFO(8021h)
Return protocol converter information. See BIDI_Q_CONVERTER_INFO(8021h) for output parameter.

BIDI_Q_SPOOLER_VERSION(8022h)
Return spooler version information. See BIDI_Q_SPOOLER_VERSION(8022h) for output parameter.

BIDI_Q_STORAGE(8023h)
Return printer storage media. See BIDI_Q_STORAGE(8023h) for input and output parameters.

0x00009000 - 0x00009FFF
Protocol-converter-specific commands.

0x0000B000 - 0x0000BFFF
Port-driver-specific commands.

pInData (PVOID) - input
May contain information required by PrtQuery.

See BIDI Command Structures and Command Flow for specific information definitions.

cbInData (ULONG) - input
Length of information in *pInData* , in bytes.

pOutData (PVOID) - output
Return buffer.

Format of buffer depends on *ulCommand*. For BIDI_READ_PASSTHRU, this will contain BIDI protocol-specific response data and will not be interpreted by the BIDI protocol converter. However, if *ulType* is TYPE_WITHOUT_WRAPPER, any protocol data wrapper is removed before returning the printer-specific information.

For other commands, it will return a structure containing queried information for the command as defined in BIDI Command Structures and Command Flow.

pcbOutData (PULONG) - in/out
Points to the length of the output buffer.

On entry, this is set to the length of the buffer passed in. On exit, it is updated with the length of data available.

If ERROR_MORE_DATA(234) or NERR_BufTooSmall(2123) is set, this field has the buffer size required to get all available information on exit.

rc (ULONG) - returns
Return codes.

0 Success

1(ERROR_INVALID_FUNCTION)
The command code is not supported. This lack of support may occur if a new command is sent to an old spooler, port driver, or protocol converter, or if the printer rejects the command.

2(ERROR_FILE_NOT_FOUND)
The *pszDeviceName* given is not a valid port name with a port driver, a printer device name, or a printer queue name.

5(ERROR_ACCESS_DENIED)
Not authorized to access printer port.

8(ERROR_NOT_ENOUGH_MEMORY)
Not enough memory to satisfy request.

29(ERROR_WRITE_FAULT)
Failure attempting to write to the device. The printer might not have received the query request.

53(ERROR_BAD_NETPATH)
: The computer name specified is not available or is invalid.

67(ERROR_BAD_NET_NAME)
: The computer name cannot be found.

87(ERROR_INVALID_PARAMETER)
: An invalid parameter was specified; most likely an invalid buffer pointer was given.

211(ERROR_INFO_NOT_AVAIL)
: The requested information is not available in the protocol converter cache, or the request cannot be sent with the given *ulType* value. The request should be reissued with a different *ulType* value.

234(ERROR_MORE_DATA)
: Part of the information requested was returned, but there was not enough room for all the data. **pcbOutData* contains the size of buffer needed to retrieve all the query response data.

640(ERROR_TIMEOUT)
: The printer response was not received within the time allotted. The caller may reissue the query, at which time the information might be available.

683(ERROR_BIDI_NO_SUPPORT)
: The printer is not in bidirectional mode, therefore query requests are not supported. The port also may be in the process of being shut down.

2123(NERR_BufTooSmall)
: No information was returned because the output buffer was too small. **pcbOutData* contains the size of buffer needed to retrieve all the query response data.

0x4005(PMERR_SPL_INV_HSPL)
: Not currently in a passthru session.

0x4009(PMERR_SPL_SPOOLER_NOT_INSTALLED)
: The spooler is not enabled, therefore bidirectional query requests are not supported.

0x4011(PMERR_SPL_INV_JOB_ID)
: No job is being actively sent to the printer port, therefore BIDI_Q_JOBID is not successful.

0x403D(PMERR_SPL_CNV_NOT_INIT)
: The protocol converter or port driver has not been initialized for this port.

0x403E(PMERR_SPL_INIT_IN_PROGRESS)
: Another thread is still processing BIDI_INIT_PORTDRV or the initial BIDI_Q_PORT for the given portname. The request should be tried again later.

0x403F(PMERR_SPL_TYPE_NOT_AVAIL)
: The storage type requested for BIDI_Q_FONTS or BIDI_Q_STORAGE is not available in the printer.

0x4041(PMERR_SPL_SESSION_TERM)
: The passthru session has been terminated.

Others
: If *pszComputerName* is specified, other network errors may be returned.

Note: Currently you cannot receive more than 64KB of data from a spooler call over the network. Therefore, all returned structures should be limited to less than 64KB for now.

PrtQuery - Remarks

A port driver must exist or the PrtQuery call returns ERROR_FILE_NOT_FOUND(2).

PrtQuery - Topics

Select an item:

Syntax
Parameters
Returns
Remarks
Glossary

PrtResetAbort

PrtResetAbort - Syntax

Description:

PrtResetAbort resets an aborted print job to allow a print driver to send multiple reset sequences to the printer. After PrtResetAbort is called successfully, the print driver can issue PrtWrite for this device handle

```
#define INCL_SPL
#define INCL_SPLDOSPRINT
#include <os2.h>

HFILE    hDevice;  /*  The device handle returned by PrtOpen. */
ULONG    rc;       /*  Return codes. */

rc = PrtResetAbort(hDevice);
```

PrtResetAbort Parameter - hDevice

hDevice (HFILE) - input
The device handle returned by *PrtOpen* .

PrtResetAbort Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
6(ERROR_INVALID_HANDLE)	Invalid handle given.
127(ERROR_PROC_NOT_FOUND)	Port driver does not export the SplPdResetAbort API.

PrtResetAbort - Parameters

hDevice (HFILE) - input
The device handle returned by *PrtOpen* .

rc (ULONG) - returns
Return codes.

0	Success
6(ERROR_INVALID_HANDLE)	Invalid handle given.
127(ERROR_PROC_NOT_FOUND)	Port driver does not export the SplPdResetAbort API.

PrtResetAbort - Remarks

This function was added to OS/2 Warp, Version 3, to allow print drivers to abort jobs more quickly. The print drivers do this by checking for PrtResetAbort during the print driver's abort processing. If the API exists, the print driver calls PrtAbort to cancel all output being sent to the printer. After all output is aborted, the print driver calls PrtResetAbort, which allows the print driver to call PrtWrite to send the reset sequence to the printer.

PrtClose must still be issued to close the device connection.

PrtResetAbort - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

PrtSet

PrtSet - Syntax

Description:

PrtSet sets printer device information.

```
#define INCL_SPL
#define INCL_SPLBIDI
```

```
#include <os2.h>

PSZ      pszComputerName;  /*  Destination computer name or NULL for local printer. */
PSZ      pszDeviceName;    /*  Port name on pszComputerName. */
ULONG    ulType;           /*  Type of set and options. */
ULONG    ulCommand;        /*  Generic command code. */
PVOID    pInData;          /*  Contains either a command sequence to pass to, or information required by, the pr
ULONG    cbInData;         /*  Length of pInData, in bytes. */
ULONG    rc;               /*  Return codes. */

rc = PrtSet(pszComputerName, pszDeviceName,
       ulType, ulCommand, pInData, cbInData);
```

PrtSet Parameter - pszComputerName

pszComputerName (PSZ) - input
Destination computer name or NULL for local printer.

Following is an example of a destination computer name:

LS:\\SERVER

PrtSet Parameter - pszDeviceName

pszDeviceName (PSZ) - input
Port name on *pszComputerName*.

Following is an example of a port name:

LPT1

This may also be the name of a local printer device or local print queue to allow presentation drivers to set printer characteristics during DevOpenDC or DevPostDeviceModes. The spooler will change the destination to be the port that is connected to either the printer device or the print queue.

PrtSet Parameter - ulType

ulType (ULONG) - input
Type of set and options.

Values are as follows:

TYPE_SHORT_WAIT(1)
Sends this information via a communications channel with the printer that can be used while a job is being sent to the printer.

An example would be using NPAP or SNMP. If the requested information cannot be sent using this type of channel (for instance, if a PJL command is required in order to set this information), ERROR_INFO_NOT_AVAIL is returned.

TYPE_LONG_WAIT(2)
Sends information using any necessary communications channel with the printer, even if the request has to wait for the current job being sent to the printer to complete.

If the printer supports only one command channel, and this channel can be used while a job is being sent to the printer, TYPE_SHORT_WAIT(1) and TYPE_LONG_WAIT(2) will behave in the same way.

TYPE_WITHOUT_WRAPPER(4)

The information being sent does not have any packet wrapper around it.

This is useful for BIDI_SEND_PASSTHRU as it allows an application to send and receive printer-interpreter language commands without having to know the format of the BIDI protocol's data wrappers. An example would be sending a Postscript query to the printer and the NPAP protocol converter adds the necessary NPAP wrapper before transmitting the data to the printer.

If BIDI_SEND_PASSTHRU is used without this value, the calling application must have already wrapped the passthru data.

Note: If this value is used with any command other than BIDI_SEND_PASSTHRU, ERROR_INVALID_PARAMETER is returned.

PrtSet Parameter - ulCommand

ulCommand (ULONG) - input
Generic command code.

See BIDI Command Structures and Command Flow for contents of *pInData* and *pOutData* for each of the following commands.

Current commands are as follows:

BIDI_SEND_PASSTHRU(1h)

Printer-language-specific command will be issued to printer. See BIDI_SEND_PASSTHRU(1h) for input parameters.

BIDI_STARTJOB(2h)

Signals start of print job. See BIDI_STARTJOB(2h) for input parameters.

BIDI_ENDJOB(3h)

Signals end of print job. See BIDI_ENDJOB(3h) for input parameters.

BIDI_HOLDJOB(4h)

Holds printer job. See BIDI_HOLDJOB(4h) for input parameters.

BIDI_RELEASEJOB(5h)

Releases printer job. See BIDI_RELEASEJOB(5h) for input parameters.

BIDI_CANCELJOB(6h)

Cancels printer job. See BIDI_CANCELJOB(6h) for input parameters.

BIDI_INIT_PORTDRV(8h)

Initializes a port driver. See BIDI_INIT_PORTDRV(8h) for input parameters.

BIDI_RESET(9h)

Resets the printer. See BIDI_RESET(9h) for input parameters.

BIDI_SHUTDOWN(Ah)

Releases all threads waiting for this port. See BIDI_SHUTDOWN(Ah) for input parameters.

BIDI_INIT(Bh)

Sets printer in bidirectional mode. See BIDI_INIT(Bh) for input parameters.

BIDI_TERM(Ch)

Sets printer in unidirectional mode. See BIDI_TERM(Ch) for input parameters.

BIDI_RESPONSE_FMT(Dh)

Defines format of printer-to-host messages. See BIDI_RESPONSE_FMT(Dh) for input parameters.

BIDI_PACKET_SIZE(Eh)

Defines printer-to-host maximum packet size. See BIDI_PACKET_SIZE(Eh) for input parameters.

BIDI_INIT_PROTCNV(Fh)

Initializes the protocol converter for a port. See BIDI_INIT_PROTCNV(Fh) for input parameters.

BIDI_SET_SW(10h)
: Tells port driver the BIDI software capabilities of the printer. See BIDI_SET_SW(10h) for input parameters.

BIDI_SET_PORTDRV(19h)
: Stores port driver configuration. See BIDI_SET_PORTDRV(19h) for input parameters.

BIDI_START_PASSTHRU(1Ah)
: Initiates a passthru session with the printer. See BIDI_START_PASSTHRU(1Ah) for input parameters.

BIDI_END_PASSTHRU(1Bh)
: Terminates a passthru session with the printer. See BIDI_END_PASSTHRU(1Bh) for input parameters.

BIDI_NOTIFY_ENDJOBCONNECT(20h)
: Notifies port driver that jobs are complete. See BIDI_NOTIFY_ENDJOBCONNECT(20h) for input parameters.

BIDI_NOTIFY_PORT_SELECTED(21h)
: Notifies port driver that the printer is connected to a print queue. See BIDI_NOTIFY_PORT_SELECTED(21h) for input parameters.

BIDI_NOTIFY_PORT_RELEASED(22h)
: Notifies port driver that the printer is no longer connected to a print queue. See BIDI_NOTIFY_PORT_RELEASED(22h) for input parameters.

BIDI_SET_DEVICE_ID(23h)
: Tells spooler the printer's device ID. See BIDI_SET_DEVICE_ID(23h) for input parameters.

BIDI_ENABLE_ALERT(24h)
: Enables printer alerts. See BIDI_ENABLE_ALERT(24h) for input parameters.

BIDI_DISABLE_ALERT(25h)
: Disables printer alerts. See BIDI_DISABLE_ALERT(25h) for input parameters.

BIDI_ADD_VIRTUAL_PORT(26h)
: Adds a virtual port. See BIDI_ADD_VIRTUAL_PORT(26h) for input parameters.

BIDI_DEL_VIRTUAL_PORT(27h)
: Deletes a virtual port. See BIDI_DEL_VIRTUAL_PORT(27h) for input parameters.

BIDI_DEL_PORT(28h)
: Indicates a spooler port has been deleted. See BIDI_DEL_PORT(28h) for input parameters.

0x00009000 - 0x00009FFF
: Protocol-converter-specific commands.

0x0000B000 - 0x0000BFFF
: Port-driver-specific commands.

PrtSet Parameter - pInData

pInData (PVOID) - input
: Contains either a command sequence to pass to, or information required by, the printer.

If *ulCommand* is set to BIDI_SEND_PASSTHRU, *pInData* contains the command sequence to pass to the printer. The field will not be altered by the port driver or protocol converter unless *ulType* is set to TYPE_WITHOUT_WRAPPER.

If *ulCommand* is **not** set to BIDI_SEND_PASSTHRU, *pInData* may contain information required by the command (see BIDI Command Structures and Command Flow for more information).

PrtSet Parameter - cbInData

cbInData (ULONG) - input
Length of *pInData* , in bytes.

PrtSet Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
1(ERROR_INVALID_FUNCTION	The command code is not supported. This may occur if a new command is sent to an old spooler, port driver, or protocol converter, or if the printer rejects the command.
2(ERROR_FILE_NOT_FOUND)	*pszDeviceName* given is not a valid port name with a port driver, a printer device name, or a printer queue name.
5(ERROR_ACCESS_DENIED)	Not authorized to access a printer port.
8(ERROR_NOT_ENOUGH_MEMORY)	Not enough memory to satisfy request.
29(ERROR_WRITE_FAULT)	Failure attempting to write to the device. The printer might not have received the set request.
53(ERROR_BAD_NETPATH)	The computer name specified is not available or is invalid.
67(ERROR_BAD_NET_NAME)	The computer name cannot be found.
87(ERROR_INVALID_PARAMETER)	An invalid parameter was specified; most likely an invalid buffer pointer was given.
117(ERROR_INVALID_CATEGORY)	The protocol converter does not handle the category given with command BIDI_ENABLE_ALERT.
186(ERROR_INVALID_FLAG_NUMBER)	The protocol converter does not understand one of the core alert types set with command BIDI_ENABLE_ALERT.
211(ERROR_INFO_NOT_AVAIL)	The request cannot be sent with the given *ulType* value. The request should be reissued with a different *ulType* value.
640(ERROR_TIMEOUT)	The set command was not able to be sent to the printer within the time allotted.
683(ERROR_BIDI_NO_SUPPORT)	The printer is not in bidirectional mode, therefore query requests are not honored. The port may also be in the process of being shut down.
0x1403(PMERR_ALREADY_INITIALIZED)	The port driver has already been initialized for the port.
0x4005(PMERR_SPL_INV_HSPL)	Not currently in a passthru session.
0x4009(PMERR_SPL_SPOOLER_NOT_INSTALLED)	The spooler is not enabled, therefore bidirectional set requests are not supported.
0x4011(PMERR_SPL_INV_JOB_ID)	The job ID specified is not known.

0x403D(PMERR_SPL_CNV_NOT_INIT)
The protocol converter or port driver has not been initialized for this port.

0x403E(PMERR_SPL_INIT_IN_PROGRESS)
Another thread is still processing BIDI_INIT_PORTDRV or the initial BIDI_Q_PORT for the given portname. The request should be tried again later.

Others
If *pszComputerName* is specified, other network errors may be returned.

Note: Currently, you cannot send more than 64KB of data from a spooler call over the network. Therefore, all data used in this API should be limited to less than 64KB for now.

PrtSet - Parameters

pszComputerName (PSZ) - input
Destination computer name or NULL for local printer.

Following is an example of a destination computer name:

```
LS:\\SERVER
```

pszDeviceName (PSZ) - input
Port name on *pszComputerName*.

Following is an example of a port name:

```
LPT1
```

This may also be the name of a local printer device or local print queue to allow presentation drivers to set printer characteristics during DevOpenDC or DevPostDeviceModes. The spooler will change the destination to be the port that is connected to either the printer device or the print queue.

ulType (ULONG) - input
Type of set and options.

Values are as follows:

TYPE_SHORT_WAIT(1)
Sends this information via a communications channel with the printer that can be used while a job is being sent to the printer.

An example would be using NPAP or SNMP. If the requested information cannot be sent using this type of channel (for instance, if a PJL command is required in order to set this information), ERROR_INFO_NOT_AVAIL is returned.

TYPE_LONG_WAIT(2)
Sends information using any necessary communications channel with the printer, even if the request has to wait for the current job being sent to the printer to complete.

If the printer supports only one command channel, and this channel can be used while a job is being sent to the printer, TYPE_SHORT_WAIT(1) and TYPE_LONG_WAIT(2) will behave in the same way.

TYPE_WITHOUT_WRAPPER(4)
The information being sent does not have any packet wrapper around it.

This is useful for BIDI_SEND_PASSTHRU as it allows an application to send and receive printer-interpreter language commands without having to know the format of the BIDI protocol's data wrappers. An example would be sending a Postscript query to the printer and the NPAP protocol converter adds the necessary NPAP wrapper before transmitting the data to the printer.

If BIDI_SEND_PASSTHRU is used without this value, the calling application must have already wrapped the passthru data.

Note: If this value is used with any command other than BIDI_SEND_PASSTHRU, ERROR_INVALID_PARAMETER is returned.

ulCommand (ULONG) - input

Generic command code.

See BIDI Command Structures and Command Flow for contents of *pInData* and *pOutData* for each of the following commands.

Current commands are as follows:

BIDI_SEND_PASSTHRU(1h)
: Printer-language-specific command will be issued to printer. See BIDI_SEND_PASSTHRU(1h) for input parameters.

BIDI_STARTJOB(2h)
: Signals start of print job. See BIDI_STARTJOB(2h) for input parameters.

BIDI_ENDJOB(3h)
: Signals end of print job. See BIDI_ENDJOB(3h) for input parameters.

BIDI_HOLDJOB(4h)
: Holds printer job. See BIDI_HOLDJOB(4h) for input parameters.

BIDI_RELEASEJOB(5h)
: Releases printer job. See BIDI_RELEASEJOB(5h) for input parameters.

BIDI_CANCELJOB(6h)
: Cancels printer job. See BIDI_CANCELJOB(6h) for input parameters.

BIDI_INIT_PORTDRV(8h)
: Initializes a port driver. See BIDI_INIT_PORTDRV(8h) for input parameters.

BIDI_RESET(9h)
: Resets the printer. See BIDI_RESET(9h) for input parameters.

BIDI_SHUTDOWN(Ah)
: Releases all threads waiting for this port. See BIDI_SHUTDOWN(Ah) for input parameters.

BIDI_INIT(Bh)
: Sets printer in bidirectional mode. See BIDI_INIT(Bh) for input parameters.

BIDI_TERM(Ch)
: Sets printer in unidirectional mode. See BIDI_TERM(Ch) for input parameters.

BIDI_RESPONSE_FMT(Dh)
: Defines format of printer-to-host messages. See BIDI_RESPONSE_FMT(Dh) for input parameters.

BIDI_PACKET_SIZE(Eh)
: Defines printer-to-host maximum packet size. See BIDI_PACKET_SIZE(Eh) for input parameters.

BIDI_INIT_PROTCNV(Fh)
: Initializes the protocol converter for a port. See BIDI_INIT_PROTCNV(Fh) for input parameters.

BIDI_SET_SW(10h)
: Tells port driver the BIDI software capabilities of the printer. See BIDI_SET_SW(10h) for input parameters.

BIDI_SET_PORTDRV(19h)
: Stores port driver configuration. See BIDI_SET_PORTDRV(19h) for input parameters.

BIDI_START_PASSTHRU(1Ah)
: Initiates a passthru session with the printer. See BIDI_START_PASSTHRU(1Ah) for input parameters.

BIDI_END_PASSTHRU(1Bh)
: Terminates a passthru session with the printer. See BIDI_END_PASSTHRU(1Bh) for input parameters.

BIDI_NOTIFY_ENDJOBCONNECT(20h)
: Notifies port driver that jobs are complete. See BIDI_NOTIFY_ENDJOBCONNECT(20h) for input parameters.

BIDI_NOTIFY_PORT_SELECTED(21h)
: Notifies port driver that the printer is connected to a print queue. See BIDI_NOTIFY_PORT_SELECTED(21h) for input parameters.

BIDI_NOTIFY_PORT_RELEASED(22h)
: Notifies port driver that the printer is no longer connected to a print queue. See BIDI_NOTIFY_PORT_RELEASED(22h) for input parameters.

BIDI_SET_DEVICE_ID(23h)

Tells spooler the printer's device ID. See BIDI_SET_DEVICE_ID(23h) for input parameters.

BIDI_ENABLE_ALERT(24h)
Enables printer alerts. See BIDI_ENABLE_ALERT(24h) for input parameters.

BIDI_DISABLE_ALERT(25h)
Disables printer alerts. See BIDI_DISABLE_ALERT(25h) for input parameters.

BIDI_ADD_VIRTUAL_PORT(26h)
Adds a virtual port. See BIDI_ADD_VIRTUAL_PORT(26h) for input parameters.

BIDI_DEL_VIRTUAL_PORT(27h)
Deletes a virtual port. See BIDI_DEL_VIRTUAL_PORT(27h) for input parameters.

BIDI_DEL_PORT(28h)
Indicates a spooler port has been deleted. See BIDI_DEL_PORT(28h) for input parameters.

0x00009000 - 0x00009FFF
Protocol-converter-specific commands.

0x0000B000 - 0x0000BFFF
Port-driver-specific commands.

pInData (PVOID) - input
Contains either a command sequence to pass to, or information required by, the printer.

If *ulCommand* is set to BIDI_SEND_PASSTHRU, *pInData* contains the command sequence to pass to the printer. The field will not be altered by the port driver or protocol converter unless *ulType* is set to TYPE_WITHOUT_WRAPPER.

If *ulCommand* is **not** set to BIDI_SEND_PASSTHRU, *pInData* may contain information required by the command (see BIDI Command Structures and Command Flow for more information).

cbInData (ULONG) - input
Length of *pInData* , in bytes.

rc (ULONG) - returns
Return codes.

0 Success

1(ERROR_INVALID_FUNCTION
The command code is not supported. This may occur if a new command is sent to an old spooler, port driver, or protocol converter, or if the printer rejects the command.

2(ERROR_FILE_NOT_FOUND)
pszDeviceName given is not a valid port name with a port driver, a printer device name, or a printer queue name.

5(ERROR_ACCESS_DENIED)
Not authorized to access a printer port.

8(ERROR_NOT_ENOUGH_MEMORY)
Not enough memory to satisfy request.

29(ERROR_WRITE_FAULT)
Failure attempting to write to the device. The printer might not have received the set request.

53(ERROR_BAD_NETPATH)
The computer name specified is not available or is invalid.

67(ERROR_BAD_NET_NAME)
The computer name cannot be found.

87(ERROR_INVALID_PARAMETER)
An invalid parameter was specified; most likely an invalid buffer pointer was given.

117(ERROR_INVALID_CATEGORY)
The protocol converter does not handle the category given with command BIDI_ENABLE_ALERT.

186(ERROR_INVALID_FLAG_NUMBER)
The protocol converter does not understand one of the core alert types set with command BIDI_ENABLE_ALERT.

211(ERROR_INFO_NOT_AVAIL)
The request cannot be sent with the given *ulType* value. The request should be reissued with a

different *ulType* value.

640(ERROR_TIMEOUT)
: The set command was not able to be sent to the printer within the time allotted.

683(ERROR_BIDI_NO_SUPPORT)
: The printer is not in bidirectional mode, therefore query requests are not honored. The port may also be in the process of being shut down.

0x1403(PMERR_ALREADY_INITIALIZED)
: The port driver has already been initialized for the port.

0x4005(PMERR_SPL_INV_HSPL)
: Not currently in a passthru session.

0x4009(PMERR_SPL_SPOOLER_NOT_INSTALLED)
: The spooler is not enabled, therefore bidirectional set requests are not supported.

0x4011(PMERR_SPL_INV_JOB_ID)
: The job ID specified is not known.

0x403D(PMERR_SPL_CNV_NOT_INIT)
: The protocol converter or port driver has not been initialized for this port.

0x403E(PMERR_SPL_INIT_IN_PROGRESS)
: Another thread is still processing BIDI_INIT_PORTDRV or the initial BIDI_Q_PORT for the given portname. The request should be tried again later.

Others
: If *pszComputerName* is specified, other network errors may be returned.

Note: Currently, you cannot send more than 64KB of data from a spooler call over the network. Therefore, all data used in this API should be limited to less than 64KB for now.

PrtSet - Remarks

A port driver must exist or this function returns ERROR_FILE_NOT_FOUND(2).

PrtSet - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

PrtWrite

PrtWrite - Syntax

Description:

PrtWrite writes data to the device file identified by hDevice.

```
#include <os2.h>

HFILE     hDevice;     /*  The device handle returned by PrtOpen. */
PVOID     pvoidData;   /*  A pointer to the data buffer. */
ULONG     cData;       /*  The length of the data in bytes. */
PULONG    cWritten;    /*  A pointer to a count of bytes actually written to the device. */
ULONG     rc;          /*  Return codes. */

rc = PrtWrite(hDevice, pvoidData, cData, cWritten);
```

PrtWrite Parameter - hDevice

hDevice (HFILE) - input
The device handle returned by *PrtOpen* .

PrtWrite Parameter - pvoidData

pvoidData (PVOID) - input
A pointer to the data buffer.

PrtWrite Parameter - cData

cData (ULONG) - input
The length of the data in bytes.

PrtWrite Parameter - cWritten

cWritten (PULONG) - input
A pointer to a count of bytes actually written to the device.

PrtWrite Return Value - rc

rc (ULONG) - returns
Return codes.

This function returns the same codes as DosWrite:

ERROR_ACCESS_DENIED
ERROR_BAD_UNIT
ERROR_BROKEN_PIPE
ERROR_INVALID_HANDLE
ERROR_LOCK_VIOLATION
ERROR_NOT_DOS_DISK
ERROR_OUT_OF_PAPER
ERROR_WRITEFAULT
NO_ERROR

PrtWrite - Parameters

hDevice (HFILE) - input
The device handle returned by *PrtOpen* .

pvoidData (PVOID) - input
A pointer to the data buffer.

cData (ULONG) - input
The length of the data in bytes.

cWritten (PULONG) - input
A pointer to a count of bytes actually written to the device.

rc (ULONG) - returns
Return codes.

This function returns the same codes as DosWrite:

ERROR_ACCESS_DENIED
ERROR_BAD_UNIT
ERROR_BROKEN_PIPE
ERROR_INVALID_HANDLE
ERROR_LOCK_VIOLATION
ERROR_NOT_DOS_DISK
ERROR_OUT_OF_PAPER
ERROR_WRITEFAULT
NO_ERROR

PrtWrite - Remarks

Some *physical device drivers* return NO_ERROR even though cWritten is not equal to cData. Presentation drivers should compare cData to cWritten to determine if a request has completed successfully before checking for return codes. To complete the request when cData is not equal to cWritten, the call must be issued after calculating the new starting point (pvoidData = pvoidData+cWritten) and the remaining characters to transfer (cData = cData-cWritten).

PrtWrite - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SetDriverInfo

SetDriverInfo - Syntax

Description:

SetDriverInfo associates a driver object handle with the object identified by an engine object handle.

```
#include <os2.h>

ULONG    hDrvObject;  /*  Driver's handle for the object. */
ULONG    hEngObject;  /*  Engine's handle for the object. This value must not be -1. */
ULONG    ulIndex;     /*  Indicates the type of object. */
HDC      hdc;         /*  Device context handle that identifies the calling DC. */
LONG     rc;          /*  Return codes. */

rc = SetDriverInfo(hDrvObject, hEngObject,
       ulIndex, hdc);
```

SetDriverInfo Parameter - hDrvObject

hDrvObject (ULONG) - input
Driver's handle for the object.

This value must not be *-1* .

SetDriverInfo Parameter - hEngObject

hEngObject (ULONG) - input
Engine's handle for the object. This value must not be *-1* .

SetDriverInfo Parameter - ulIndex

ulIndex (ULONG) - input
Indicates the type of object.

Has the following settings:

lIndex = 0	Reserved
lIndex = 1	hObject is a bit-map handle

SetDriverInfo Parameter - hdc

hdc (HDC) - input
Device context handle that identifies the calling DC.

SetDriverInfo Return Value - rc

rc (LONG) - returns
Return codes.

If successful, the function returns the driver object handle that was associated with the object before any changes were made by SetDriverInfo. If an error is detected, the function returns GPI_ALTERROR.

SetDriverInfo - Parameters

hDrvObject (ULONG) - input
Driver's handle for the object.

This value must not be *-1* .

hEngObject (ULONG) - input
Engine's handle for the object. This value must not be *-1* .

ulIndex (ULONG) - input
Indicates the type of object.

Has the following settings:

lIndex = 0	Reserved
lIndex = 1	hObject is a bit-map handle

hdc (HDC) - input
Device context handle that identifies the calling DC.

rc (LONG) - returns
Return codes.

If successful, the function returns the driver object handle that was associated with the object before any changes were made by SetDriverInfo. If an error is detected, the function returns GPI_ALTERROR.

SetDriverInfo - Remarks

Object handles are stored in the bit maps when they are created. Calling SetDriverInfo provides a method the presentation driver can use to change the driver handle in the bit map. SetDriverInfo includes a check to ensure that the object is, or was created by, an instance of a DC for the same device as the DC identified by hdc. If the check is not successful, the function returns GPI_ALTERROR.

SetDriverInfo is exported by the graphics engine at ordinal 31. If a presentation driver calls this function, it must be imported by the driver's

module definition file.

SetDriverInfo - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplCreatePort

SplCreatePort - Syntax

Description:

This function is called to create a printer port. The printer port can be a virtual port.

Note: There is currently no support for using this API to remotely add a printer port.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

PSZ      pszComputerName;  /*  Name of computer where port is to be installed. */
PSZ      pszPortName;      /*  Name of printer port to install. */
PSZ      pszPortDriver;    /*  Name of the port driver assigned to this port. */
ULONG    ulVersion;        /*  Version of port to create. */
PVOID    pBuf;             /*  Port-driver-specific buffer. */
ULONG    cbBuf;            /*  Length of the data in pBuf, in bytes. */
ULONG    rc;               /*  Return codes. */

rc = SplCreatePort(pszComputerName, pszPortName,
       pszPortDriver, ulVersion, pBuf, cbBuf);
```

SplCreatePort Parameter - pszComputerName

pszComputerName (PSZ) - input
Name of computer where port is to be installed.

A NULL string is used for installing a port on the local machine. This parameter currently must be NULL.

SplCreatePort Parameter - pszPortName

pszPortName (PSZ) - input
Name of printer port to install.

This port, virtual or regular, must not already exist. Following is an example of a port name:
LPT5

SplCreatePort Parameter - pszPortDriver

pszPortDriver (PSZ) - input
Name of the port driver assigned to this port.

If not NULL, this field must specify an installed port driver; it is case-sensitive. Following is an example of a port driver:
PARALLEL

SplCreatePort Parameter - ulVersion

ulVersion (ULONG) - input
Version of port to create.

Values are as follows:

SPLPORT_VERSION_REGULAR - 0
Creates normal printer port that can be connected to a print queue. This port will remain after a reboot.

SPLPORT_VERSION_VIRTUAL - 1
Creates a virtual printer port that can only be used for querying and setting. This port exists only until the spooler is disabled or the machine is rebooted.

SplCreatePort Parameter - pBuf

pBuf (PVOID) - input
Port-driver-specific buffer.

If SPLPORT_VERSION_VIRTUAL is set, this is a port-driver-specific buffer that contains information necessary for the port driver to set up a virtual connection to the printer. There cannot be any imbedded pointers in this buffer. This buffer wil be passed to the port driver with the BIDI_ADD_VIRTUAL_PORT command.

SplCreatePort Parameter - cbBuf

cbBuf (ULONG) - input
Length of the data in *pBuf,* in bytes.

SplCreatePort Return Value - rc

rc (ULONG) - returns
Return codes.

0 Success

ERROR_DEVICE_IN_USE(99)
Port name is already defined.

ERROR_FILE_NOT_FOUND(2)
Port driver name is not valid.

ERROR_INVALID_LEVEL(124)
ulVersion is not 0 or 1.

ERROR_INVALID_PARAMETER(87)
An invalid buffer was given.

ERROR_NOT_ENOUGH_MEMORY(8)
Not enough memory to satisfy request.

ERROR_NOT_SUPPORTED(50)
pszComputerName is not supported, or the port driver does not support the virtual port.

PMERR_SPL_SPOOLER_NOT_INSTALLED(0x4009)
Spooler not enabled; virtual ports require spooler to be enabled.

SplCreatePort - Parameters

pszComputerName (PSZ) - input
Name of computer where port is to be installed.

A NULL string is used for installing a port on the local machine. This parameter currently must be NULL.

pszPortName (PSZ) - input
Name of printer port to install.

This port, virtual or regular, must not already exist. Following is an example of a port name:
LPT5

pszPortDriver (PSZ) - input
Name of the port driver assigned to this port.

If not NULL, this field must specify an installed port driver; it is case-sensitive. Following is an example of a port driver:
PARALLEL

ulVersion (ULONG) - input
Version of port to create.

Values are as follows:

SPLPORT_VERSION_REGULAR - 0
Creates normal printer port that can be connected to a print queue. This port will remain after a reboot.

SPLPORT_VERSION_VIRTUAL - 1
Creates a virtual printer port that can only be used for querying and setting. This port exists only until the spooler is disabled or the machine is rebooted.

pBuf (PVOID) - input

Port-driver-specific buffer.

If SPLPORT_VERSION_VIRTUAL is set, this is a port-driver-specific buffer that contains information necessary for the port driver to set up a virtual connection to the printer. There cannot be any imbedded pointers in this buffer. This buffer wil be passed to the port driver with the BIDI_ADD_VIRTUAL_PORT command.

cbBuf (ULONG) - input
Length of the data in *pBuf,* in bytes.

rc (ULONG) - returns
Return codes.

0	Success
ERROR_DEVICE_IN_USE(99)	Port name is already defined.
ERROR_FILE_NOT_FOUND(2)	Port driver name is not valid.
ERROR_INVALID_LEVEL(124)	*ulVersion* is not 0 or 1.
ERROR_INVALID_PARAMETER(87)	An invalid buffer was given.
ERROR_NOT_ENOUGH_MEMORY(8)	Not enough memory to satisfy request.
ERROR_NOT_SUPPORTED(50)	*pszComputerName* is not supported, or the port driver does not support the virtual port.
PMERR_SPL_SPOOLER_NOT_INSTALLED(0x4009)	Spooler not enabled; virtual ports require spooler to be enabled.

SplCreatePort - Remarks

None.

SplCreatePort - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplDeletePort

SplDeletePort - Syntax

Description:

This function is called to remove a printer port.

Note: There is currently no support for using this API to remotely remove a printer port.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

PSZ      pszComputerName;  /*  Name of computer where port is to be removed. */
PSZ      pszPortName;      /*  Name of printer port to remove. */
ULONG    rc;               /*  Return codes. */

rc = SplDeletePort(pszComputerName, pszPortName);
```

SplDeletePort Parameter - pszComputerName

pszComputerName (PSZ) - input
Name of computer where port is to be removed.

A NULL string identifies the local workstation. This currently must be NULL.

SplDeletePort Parameter - pszPortName

pszPortName (PSZ) - input
Name of printer port to remove.

This can be a virtual port name. Following is an example of a port name:
LPT5

Note: This port cannot be removed if it is referenced by a print device.

SplDeletePort Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
ERROR_DEVICE_IN_USE(99)	Port name is connected to a print queue.
ERROR_INVALID_PARAMETER(87)	An invalid buffer was given.
ERROR_NOT_SUPPORTED(50)	*pszComputerName* is not supported.

SplDeletePort - Parameters

pszComputerName (PSZ) - input
Name of computer where port is to be removed.

A NULL string identifies the local workstation. This currently must be NULL.

pszPortName (PSZ) - input
Name of printer port to remove.

This can be a virtual port name. Following is an example of a port name:
LPT5

Note: This port cannot be removed if it is referenced by a print device.

rc (ULONG) - returns
Return codes.

0	Success
ERROR_DEVICE_IN_USE(99)	Port name is connected to a print queue.
ERROR_INVALID_PARAMETER(87)	An invalid buffer was given.
ERROR_NOT_SUPPORTED(50)	*pszComputerName* is not supported.

SplDeletePort - Remarks

None.

SplDeletePort - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplDisable

SplDisable - Syntax

Description:

This function is called to disable the OS/2 spooler. The spooler remains disabled across reboots. It can be re-enabled using SplEnable.

Note: There is currently no support for using this API to remotely disable a print spooler.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

PSZ      pszComputerName;  /*  Name of the computer on which to disable the spooler. */
PVOID    pReserved;        /*  Must be NULL. Reserved for future use. */
ULONG    rc;               /*  Return codes. */

rc = SplDisable(pszComputerName, pReserved);
```

SplDisable Parameter - pszComputerName

pszComputerName (PSZ) - input
Name of the computer on which to disable the spooler.

This value currently must be NULL to disable the local spooler.

SplDisable Parameter - pReserved

pReserved (PVOID) - input
Must be NULL. Reserved for future use.

SplDisable Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
ERROR_INVALID_PARAMETER(87)	An invalid buffer given.
PMERR_SPL_SPOOLER_NOT_INSTALLED(0x4009)	The spooler is not enabled.

SplDisable - Parameters

pszComputerName (PSZ) - input
Name of the computer on which to disable the spooler.

This value currently must be NULL to disable the local spooler.

pReserved (PVOID) - input
Must be NULL. Reserved for future use.

rc (ULONG) - returns
Return codes.

0	Success
ERROR_INVALID_PARAMETER(87)	An invalid buffer given.
PMERR_SPL_SPOOLER_NOT_INSTALLED(0x4009)	The spooler is not enabled.

SplDisable - Remarks

None.

SplDisable - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplDisplayControlPanel

SplDisplayControlPanel - Syntax

Description:

This function displays a control panel for a printer that is communicating bidirectionally.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

PSZ      pszComputerName;  /*  Printer server name for displaying a remote printer panel from a client machine.
PSZ      pszPortName;      /*  Printer port name. */
PSZ      pszDeviceID;      /*  Device ID, if known, for the printer connected to pszPortName. */
PSZ      pszControlPanel;  /*  Name of control panel to call to display the front panel for this printer. */
HAB      hab;              /*  Anchor block handle. */
```

```
ULONG    flCapabilities;    /*  Capabilities requested. */
ULONG    rc;                /*  Return codes. */

rc = SplDisplayControlPanel(pszComputerName,
       pszPortName, pszDeviceID, pszControlPanel,
       hab, flCapabilities);
```

SplDisplayControlPanel Parameter - pszComputerName

pszComputerName (PSZ) - input
Printer server name for displaying a remote printer panel from a client machine.

This parameter is NULL for printers whose ports exist on the current machine.

SplDisplayControlPanel Parameter - pszPortName

pszPortName (PSZ) - input
Printer port name.

The name of the printer port for *pszComputerName,* used to display a control panel. Following is an example of a port name:
LPT1

SplDisplayControlPanel Parameter - pszDeviceID

pszDeviceID (PSZ) - input
Device ID, if known, for the printer connected to *pszPortName* .

SplDisplayControlPanel Parameter - pszControlPanel

pszControlPanel (PSZ) - input
Name of control panel to call to display the front panel for this printer.

This name can be one of the following:

- The name registered with SplRegisterControlPanel
- One of the control panel names returned by SplGetControlPanelList

---

SplDisplayControlPanel Parameter - hab

hab (HAB) - input
Anchor block handle.

SplDisplayControlPanel Parameter - flCapabilities

flCapabilities (ULONG) - input
Capabilities requested.

FL_ADMIN - 0x00000001
Set if an administrator wants the control panel.

SplDisplayControlPanel Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
ERROR_INVALID_NAME(123)	*pszName* is not the name of a control panel DLL.
ERROR_NOT_SUPPORTED(50)	A control panel for this printer is not supported by the called control panel DLL.

SplDisplayControlPanel - Parameters

pszComputerName (PSZ) - input
Printer server name for displaying a remote printer panel from a client machine.

This parameter is NULL for printers whose ports exist on the current machine.

pszPortName (PSZ) - input
Printer port name.

The name of the printer port for *pszComputerName,* used to display a control panel. Following is an example of a port name:
LPT1

pszDeviceID (PSZ) - input
Device ID, if known, for the printer connected to *pszPortName* .

pszControlPanel (PSZ) - input
Name of control panel to call to display the front panel for this printer.

This name can be one of the following:

- The name registered with SplRegisterControlPanel
- One of the control panel names returned by SplGetControlPanelList

hab (HAB) - input
Anchor block handle.

flCapabilities (ULONG) - input
Capabilities requested.

FL_ADMIN - 0x00000001
Set if an administrator wants the control panel.

rc (ULONG) - returns
Return codes.

0	Success
ERROR_INVALID_NAME(123)	*pszName* is not the name of a control panel DLL.
ERROR_NOT_SUPPORTED(50)	A control panel for this printer is not supported by the called control panel DLL.

SplDisplayControlPanel - Remarks

None.

SplDisplayControlPanel - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplEnable

SplEnable - Syntax

Description:

This function is called to enable the OS/2 spooler.

Note: There is currently no support for using this API to remotely enable a print spooler.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

PSZ      pszComputerName;  /*  Name of the computer on which to enable the spooler. */
```

```
PSZ      pszDirectory;      /*  Name of the directory containing the spool files. */
PVOID    pReserved;         /*  Must be NULL. Reserved for future use. */
ULONG    rc;                /*  Return codes. */

rc = SplEnable(pszComputerName, pszDirectory,
       pReserved);
```

SplEnable Parameter - pszComputerName

pszComputerName (PSZ) - input
Name of the computer on which to enable the spooler.

This value currently must be NULL to enable the local spooler.

SplEnable Parameter - pszDirectory

pszDirectory (PSZ) - input
Name of the directory containing the spool files.

This value must currently be set to NULL to use the spooler directory as defined in the SYSTEM INI file.

SplEnable Parameter - pReserved

pReserved (PVOID) - input
Must be NULL. Reserved for future use.

SplEnable Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
ERROR_INVALID_PARAMETER(87)	An invalid buffer given.
PMERR_SPL_ALREADY_INITIALIZED(0x4FFD)	The spooler is already enabled.
PMERR_SPL_NO_MEMORY(0x4007)	Not enough memory to enable spooler.

SplEnable - Parameters

pszComputerName (PSZ) - input
Name of the computer on which to enable the spooler.

This value currently must be NULL to enable the local spooler.

pszDirectory (PSZ) - input
Name of the directory containing the spool files.

This value must currently be set to NULL to use the spooler directory as defined in the SYSTEM INI file.

pReserved (PVOID) - input
Must be NULL. Reserved for future use.

rc (ULONG) - returns
Return codes.

0	Success
ERROR_INVALID_PARAMETER(87)	An invalid buffer given.
PMERR_SPL_ALREADY_INITIALIZED(0x4FFD)	The spooler is already enabled.
PMERR_SPL_NO_MEMORY(0x4007)	Not enough memory to enable spooler.

SplEnable - Remarks

None.

SplEnable - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplEnumQueue

SplEnumQueue - Syntax

This function lists print queues on the local workstation or on a remote server, optionally supplying additional information.

```
#define INCL_SPL /* Or use INCL_PM, */
#include <os2.h>

PSZ       pszComputerName;  /*  Name of computer where queues are to be listed. */
ULONG     ulLevel;          /*  Level of detail. */
PVOID     pBuf;             /*  Buffer. */
ULONG     cbBuf;            /*  Size, in bytes, of Buffer. */
PULONG    pcReturned;       /*  Number of entries returned. */
PULONG    pcTotal;          /*  Total number of entries available. */
PULONG    pcbNeeded;        /*  Size in bytes of available information. */
PVOID     pReserved;        /*  Reserved value, must be NULL. */
SPLERR    rc;               /*  Return code. */

rc = SplEnumQueue(pszComputerName, ulLevel,
       pBuf, cbBuf, pcReturned, pcTotal, pcbNeeded,
       pReserved);
```

SplEnumQueue Parameter - pszComputerName

pszComputerName (PSZ) - input
Name of computer where queues are to be listed.

A NULL string specifies the local workstation.

SplEnumQueue Parameter - ulLevel

ulLevel (ULONG) - input
Level of detail.

The level of detail required. This must be 3, 4, 5, 6 or 7.

SplEnumQueue Parameter - pBuf

pBuf (PVOID) - output
Buffer.

The returned contents in the buffer depend on the value specified in *ulLevel* as follows:

ulLevel	Buffer Contents
3	An array of PRQINFO3 structures. The *fsType* bit PRQ3_TYPE_APPDEFAULT is set for the application default queue only.
4	An array of **pcReturned* PRQINFO3 structures in which each PRQINFO3 structure is followed by an array of PRJINFO2 structures, one for each job in the queue. *cJobs* in the PRQINFO3 structure gives the number of jobs in the array.
5	An array of PSZ, each pointing to a queue name.
6	An array of PRQINFO6 structures
7	An array of *pcReturned* PRQINFO3 structures in which each PRQINFO3 structure is followed by an array of PRJINFO4 structures, one for each job in the queue. *cJobs* in the PRQINFO3 structure gives the number of jobs in the array.

SplEnumQueue Parameter - cbBuf

cbBuf (ULONG) - input
Size, in bytes, of Buffer.

It must be greater than 0. Some systems do not allow a size greater than 65535.

SplEnumQueue Parameter - pcReturned

pcReturned (PULONG) - output
Number of entries returned.

SplEnumQueue Parameter - pcTotal

pcTotal (PULONG) - output
Total number of entries available.

SplEnumQueue Parameter - pcbNeeded

pcbNeeded (PULONG) - output
Size in bytes of available information.

A value of 0 specifies that the size is not known. This can occur when enumerating queues on a print server. To determine the buffer size needed, the caller should allocate a buffer in 4096-byte increments until a successful return code is received, or until the buffer size exceeds 65535.

SplEnumQueue Parameter - pReserved

pReserved (PVOID) - output
Reserved value, must be NULL.

SplEnumQueue Return Value - rc

rc (SPLERR) - returns
Return code.

NO_ERROR (0)
No errors occurred.
ERROR_ACCESS_DENIED (5)
Access is denied.
ERROR_NOT_SUPPORTED (50)
This request is not supported by the network.
ERROR_BAD_NETPATH (53)
The network path cannot be located.
ERROR_INVALID_PARAMETER (87)
An invalid parameter is specified.
ERROR_INVALID_LEVEL (124)
The level parameter is invalid.
ERROR_MORE_DATA (234)
Additional data is available.
NERR_NetNotStarted (2102)
The network program is not started.
NERR_BufTooSmall (2123)
The API return buffer is too small.
NERR_InvalidComputer (2351)
The computer name is invalid.

SplEnumQueue - Parameters

pszComputerName (PSZ) - input
Name of computer where queues are to be listed.

A NULL string specifies the local workstation.

ulLevel (ULONG) - input
Level of detail.

The level of detail required. This must be 3, 4, 5, 6 or 7.

pBuf (PVOID) - output
Buffer.

The returned contents in the buffer depend on the value specified in *ulLevel* as follows:

ulLevel	Buffer Contents
3	An array of PRQINFO3 structures. The *fsType* bit PRQ3_TYPE_APPDEFAULT is set for the application default queue only.
4	An array of **pcReturned* PRQINFO3 structures in which each PRQINFO3 structure is followed by an array of PRJINFO2 structures, one for each job in the queue. *cJobs* in the PRQINFO3 structure gives the number of jobs in the array.
5	An array of PSZ, each pointing to a queue name.
6	An array of PRQINFO6 structures
7	An array of *pcReturned* PRQINFO3 structures in which each PRQINFO3 structure is followed by an array of PRJINFO4 structures, one for each job in the queue. *cJobs* in the PRQINFO3 structure gives the number of jobs in the array.

cbBuf (ULONG) - input
Size, in bytes, of Buffer.

It must be greater than 0. Some systems do not allow a size greater than 65535.

pcReturned (PULONG) - output
Number of entries returned.

pcTotal (PULONG) - output
Total number of entries available.

pcbNeeded (PULONG) - output
Size in bytes of available information.

A value of 0 specifies that the size is not known. This can occur when enumerating queues on a print server. To determine the buffer size needed, the caller should allocate a buffer in 4096-byte increments until a successful return code is received, or until the buffer size exceeds 65535.

pReserved (PVOID) - output
Reserved value, must be NULL.

rc (SPLERR) - returns
Return code.

NO_ERROR (0)
No errors occurred.
ERROR_ACCESS_DENIED (5)
Access is denied.
ERROR_NOT_SUPPORTED (50)
This request is not supported by the network.
ERROR_BAD_NETPATH (53)
The network path cannot be located.
ERROR_INVALID_PARAMETER (87)
An invalid parameter is specified.
ERROR_INVALID_LEVEL (124)
The level parameter is invalid.
ERROR_MORE_DATA (234)
Additional data is available.
NERR_NetNotStarted (2102)
The network program is not started.
NERR_BufTooSmall (2123)
The API return buffer is too small.
NERR_InvalidComputer (2351)
The computer name is invalid.

SplEnumQueue - Related Functions

- SplQueryJob
- SplQueryQueue
- SplSetJob
- SplSetQueue

SplEnumQueue - Remarks

This sample code enumerates all the queues and the jobs in them that are on the local workstation.

```
#define INCL_BASE
#define INCL_SPL
#define INCL_SPLDOSPRINT
#define INCL_SPLERRORS

#include <os2.h>
#include <stdio.h>

INT main ()
{
  SPLERR splerr;
  USHORT jobCount;
  ULONG  cbBuf;
  ULONG  cTotal;
  ULONG  cReturned;
  ULONG  cbNeeded;
  ULONG  ulLevel;
  ULONG  i,j;
```

```
PSZ    pszComputerName;
PBYTE  pBuf;
PPRQINFO3 prq;
PPRJINFO2 prj2;

ulLevel = 4L;
pszComputerName = (PSZ)NULL;
splerr = SplEnumQueue(pszComputerName,
                      ulLevel,
                      pBuf, 0L,       /* cbBuf */
                      &cReturned,
                      &cTotal,
                      &cbNeeded,
                      NULL)
if (splerr == ERROR_MORE_DATA ||
    splerr == NERR_BufTooSmall)
{
  if (!DosAllocMem(&pBuf,
                   cbNeeded,
                   PAG_READ  |
                   PAG_WRITE |
                   PAG_COMMIT))
  {
    cbBuf = cbNeeded;
    splerr = SplEnumQueue(pszComputerName,
                          ulLevel,
                          pBuf,
                          cbBuf,
                          &cReturned,
                          &cTotal,
                          &cbNeeded, NULL)
    if (splerr == NO_ERROR)
    {
      /* Set pointer to point to the beginning of the buffer */
      prq = (PPRQINFO3)pBuf;

      /* cReturned has the count of the number of PRQINFO3 structures */
      for (i=0;i < cReturned; i++)
      {
        printf("Queue info: name - %s\n", prq->pszName);
           if (prq->fsType & PRQ3_TYPE_APPDEFAULT)
              printf("  This is the application default print queue\n");
           printf("  priority - %d  starttime - %d  endtime - %d fsType - %X\n",
                     prq->uPriority,
                     prq->uStartTime,
                     prq->uUntilTime,
                     prq->fsType);
           printf("  pszSepFile   - %s\n", prq->pszSepFile);
           printf("  pszPrProc    - %s\n", prq->pszPrProc);
           printf("  pszParms     - %s\n", prq->pszParms);
           printf("  pszComment   - %s\n", prq->pszComment);
           printf("  pszPrinters  - %s\n", prq->pszPrinters);
           printf("  pszDriverName- %s\n", prq->pszDriverName);
           if (prq->pDriverData)
           {
             printf("  pDriverData->cb          - %ld\n",
                       (ULONG)prq->pDriverData->cb);
             printf("  pDriverData->lVersion    - %ld\n",
                       (ULONG)prq->pDriverData->lVersion);
             printf("  pDriverData->szDeviceName- %s\n",
                       prq->pDriverData->szDeviceName);
           }
           /* Save the count of jobs. There are this many PRJINFO2 */
           /* structures following the PRQINFO3 structure          */
           jobCount = prq->cJobs;
           printf("Job count in this queue is %d\n\n",jobCount);

           /* Increment the pointer past the PRQINFO3  structure */
           prq++;

           /* Set a pointer to point to the first PRJINFO2 structure */
           prj2=(PPRJINFO2)prq;
           for (j=0;j < jobCount ; j++)
           {
             printf("Job ID       = %d\n", prj2->uJobId);
             printf("Job Priority = %d\n", prj2->uPriority);
             printf("User Name    = %s\n", prj2->pszUserName);
             printf("Position     = %d\n", prj2->uPosition);
             printf("Status       = %d\n", prj2->fsStatus);
             printf("Submitted    = %ld\n",prj2->ulSubmitted);
             printf("Size         = %ld\n",prj2->ulSize);
             printf("Comment      = %s\n", prj2->pszComment);
             printf("Document     = %s\n\n",prj2->pszDocument);
```

```
              /* Increment the pointer to point to the next structure */
              prj2++;
            } /* endfor jobCount */

            /* After doing all the job structures, prj2 points to the next */
            /* queue structure. Set the pointer for a PRQINFO3 structure   */
            prq = (PPRQINFO3)prj2;
        }/*endfor cReturned */
      }
      DosFreeMem(pBuf);
    }
  } /* end if Q level given */
  else
  {
    /* If we are here we had a bad error code. */
    /* Print it and some other info.           */
    printf("SplEnumQueue Error=%ld,
            Total=%ld,
            Returned=%ld,
            Needed=%ld\n",
            splerr,
            cTotal,
            cReturned,
            cbNeeded);
  }
  DosExit(EXIT_PROCESS, 0);
  return(splerr);
}   /* end main */
```

SplEnumQueue - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Related Functions
Glossary

SplGetControlPanelList

SplGetControlPanelList - Syntax

Description:

This function queries installed control panel DLLs for support for the printer attached to the given port.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

PSZ       pszComputerName;  /*  Name of computer connected to the printer port. */
PSZ       pszPortName;      /*  Name of printer port on pszComputerName used to determine if a control panel can
PSZ       pszDeviceID;      /*  Device ID, if known, for the printer connected to pszPortName. */
```

```
ULONG     flCapabilities;   /*  Compatibilities requested. */
PSZ       pszListOfPanels;  /*  Receives comma-separated list of control panel names capable of displaying a con
ULONG     cbBuf;            /*  Length of buffer, in bytes, pointed to by pszListOfPanels. */
PULONG    pcbNeeded;        /*  Receives length, in bytes, of buffer needed to store entire list of control pane
ULONG     rc;               /*  Return codes. */

rc = SplGetControlPanelList(pszComputerName,
       pszPortName, pszDeviceID, flCapabilities,
       pszListOfPanels, cbBuf, pcbNeeded);
```

SplGetControlPanelList Parameter - pszComputerName

pszComputerName (PSZ) - input
Name of computer connected to the printer port.

This is NULL for printers whose port exists on the current machine.

SplGetControlPanelList Parameter - pszPortName

pszPortName (PSZ) - input
Name of printer port on *pszComputerName* used to determine if a control panel can be displayed.

SplGetControlPanelList Parameter - pszDeviceID

pszDeviceID (PSZ) - input
Device ID, if known, for the printer connected to *pszPortName* .

SplGetControlPanelList Parameter - flCapabilities

flCapabilities (ULONG) - input
Compatibilities requested.

FL_ADMIN - 0x00000001
Set if an administrator wants the control panel.

SplGetControlPanelList Parameter - pszListOfPanels

pszListOfPanels (PSZ) - output
Receives comma-separated list of control panel names capable of displaying a control panel for the printer.

SplGetControlPanelList Parameter - cbBuf

cbBuf (ULONG) - input
Length of buffer, in bytes, pointed to by *pszListOfPanels* .

SplGetControlPanelList Parameter - pcbNeeded

pcbNeeded (PULONG) - output
Receives length, in bytes, of buffer needed to store entire list of control panels for the printer.

SplGetControlPanelList Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
ERROR_NOT_SUPPORTED(50)	A control panel for this printer is not supported by any control panel DLL.
NERR_BufTooSmall(2123)	Need to call again with a bigger buffer.

SplGetControlPanelList - Parameters

pszComputerName (PSZ) - input
Name of computer connected to the printer port.

This is NULL for printers whose port exists on the current machine.

pszPortName (PSZ) - input
Name of printer port on *pszComputerName* used to determine if a control panel can be displayed.

pszDeviceID (PSZ) - input
Device ID, if known, for the printer connected to *pszPortName* .

flCapabilities (ULONG) - input
Compatibilities requested.

FL_ADMIN - 0x00000001	Set if an administrator wants the control panel.

pszListOfPanels (PSZ) - output
Receives comma-separated list of control panel names capable of displaying a control panel for the printer.

cbBuf (ULONG) - input
Length of buffer, in bytes, pointed to by *pszListOfPanels* .

pcbNeeded (PULONG) - output
Receives length, in bytes, of buffer needed to store entire list of control panels for the printer.

rc (ULONG) - returns
Return codes.

0	Success
ERROR_NOT_SUPPORTED(50)	A control panel for this printer is not supported by any control panel DLL.
NERR_BufTooSmall(2123)	Need to call again with a bigger buffer.

SplGetControlPanelList - Remarks

This function will return Success only for printers that are configured to communicate bidirectionally.

SplGetControlPanelList - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplGetPortFromQ

SplGetPortFromQ - Syntax

Description:

This function gets a list of ports connected to the print queue for the print object or applications. This function is used by the print object to determine if some menu items can be enabled based on the type of printer port connected to a print queue.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

PSZ      pszComputerName;  /*  Name of the computer on which pszQueueName exists. */
PSZ      pszQueueName;     /*  Name of print queue. */
ULONG    ulLevel;          /*  Level of information to return. */
PVOID    pBuf;             /*  Receives the GETPORTFROMQ data structure. */
ULONG    cbBuf;            /*  Length of buffer pointed to by pBuf, in bytes. */
```

```
ULONG    pcbNeeded;        /*  Receives length, in bytes, of buffer needed to return all port data. */
ULONG    rc;               /*  Return codes. */

rc = SplGetPortFromQ(pszComputerName, pszQueueName,
       ulLevel, pBuf, cbBuf, pcbNeeded);
```

SplGetPortFromQ Parameter - pszComputerName

pszComputerName (PSZ) - input
Name of the computer on which *pszQueueName* exists.

This value is NULL if the queue is local or if the queue references a network print queue.

SplGetPortFromQ Parameter - pszQueueName

pszQueueName (PSZ) - input
Name of print queue.

This is the print queue name on *pszComputerName* or a local print queue name that may reference a network print queue.

SplGetPortFromQ Parameter - ulLevel

ulLevel (ULONG) - input
Level of information to return.

Only level 1 is supported, which returns the GETPORTFROMQ data structure. See GETPORTFROMQ in Data Types for details on this data structure.

SplGetPortFromQ Parameter - pBuf

pBuf (PVOID) - output
Receives the GETPORTFROMQ data structure.

SplGetPortFromQ Parameter - cbBuf

cbBuf (ULONG) - input
Length of buffer pointed to by *pBuf,* in bytes.

SplGetPortFromQ Parameter - pcbNeeded

pcbNeeded (ULONG) - output
Receives length, in bytes, of buffer needed to return all port data.

SplGetPortFromQ Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
ERROR_BAD_NET_NAME(67)	The computer name cannot be found.
ERROR_BAD_NETPATH(53)	The computer name specified is not available or is invalid.
ERROR_INVALID_LEVEL(124)	*ulLevel* is not 1.
ERROR_INVALID_PARAMETER(87)	An invalid parameter was specified; most likely an invalid *pBuf* value.
ERROR_MORE_DATA(234)	Not all data could fit in *pBuf* .
ERROR_NOT_SUPPORTED(50)	Command not supported on *pszComputerName*.
NERR_BufTooSmall(2123)	No data could fit in *pBuf* .
NERR_QNotFound(2150)	The queue name does not exist.

SplGetPortFromQ - Parameters

pszComputerName (PSZ) - input
Name of the computer on which *pszQueueName* exists.

This value is NULL if the queue is local or if the queue references a network print queue.

pszQueueName (PSZ) - input
Name of print queue.

This is the print queue name on *pszComputerName* or a local print queue name that may reference a network print queue.

ulLevel (ULONG) - input
Level of information to return.

Only level 1 is supported, which returns the GETPORTFROMQ data structure. See GETPORTFROMQ in Data Types for details on this data structure.

pBuf (PVOID) - output
Receives the GETPORTFROMQ data structure.

cbBuf (ULONG) - input
Length of buffer pointed to by *pBuf,* in bytes.

pcbNeeded (ULONG) - output
Receives length, in bytes, of buffer needed to return all port data.

rc (ULONG) - returns
Return codes.

0	Success
ERROR_BAD_NET_NAME(67)	The computer name cannot be found.
ERROR_BAD_NETPATH(53)	The computer name specified is not available or is invalid.
ERROR_INVALID_LEVEL(124)	*ulLevel* is not 1.
ERROR_INVALID_PARAMETER(87)	An invalid parameter was specified; most likely an invalid *pBuf* value.
ERROR_MORE_DATA(234)	Not all data could fit in *pBuf* .
ERROR_NOT_SUPPORTED(50)	Command not supported on *pszComputerName*.
NERR_BufTooSmall(2123)	No data could fit in *pBuf* .
NERR_QNotFound(2150)	The queue name does not exist.

SplGetPortFromQ - Remarks

None.

SplGetPortFromQ - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplMessageBox

SplMessageBox - Syntax

Description:

SplMessageBox creates and displays a message box.

The message queue will be created (and destroyed) if necessary. In OS/2 2.1, this API may return Retry automatically if the PM message box is displayed for more than three minutes.

```
#include <os2.h>

PSZ      pszAddress;   /*  Pointer to a string containing the logical address of the device, such as LPT1. */
ULONG    flErrorInfo;  /*  Error information. */
ULONG    flErrorData;  /*  Length of the data in bytes. */
PSZ      pszText;      /*  Pointer to the text string for the message box. */
PSZ      pszCaption;   /*  Pointer to a string containing a meaningful title for the message box. */
ULONG    idWindow;     /*  Window ID of the message box window. */
ULONG    fsStyle;      /*  Bit array specifying the contents and function of the message box. */
ULONG    Response;     /*  Return codes. */

Response = SplMessageBox(pszAddress, flErrorInfo,
             flErrorData, pszText, pszCaption,
             idWindow, fsStyle);
```

SplMessageBox Parameter - pszAddress

pszAddress (PSZ) - input
Pointer to a string containing the logical address of the device, such as LPT1.

SplMessageBox Parameter - flErrorInfo

flErrorInfo (ULONG) - input
Error information.

One of the following flags must be set to identify where the error occurred:

SPLINFO_QPERROR	Spooler queue processor error
SPLINFO_DDERROR	Presentation driver error
SPLINFO_SPLERROR	Spooler error
SPLINFO_OTHERERROR	Any other error

One of the following flags is also set to indicate the severity of the error:

SPLINFO_INFORMATION	Information only, no error.
SPLINFO_WARNING	Warning.
SPLINFO_ERROR	Recoverable error.
SPLINFO_SEVERE	Severe, unrecoverable error.
SPLINFO_USERINTREQD	This flag is optional. It shows that recovery requires action from the user.

SplMessageBox Parameter - flErrorData

flErrorData (ULONG) - input
Length of the data in bytes.

Error data:

SPLDATA_PRINTERJAM	Printer is jammed, offline, or not turned on
SPLDATA_FORMCHGREQD	Form change required
SPLDATA_CARTCHGREQD	Font cartridge change required
SPLDATA_PENCHGREQD	Pen change required
SPLDATA_DATAERROR	Data error, such as missing file
SPLDATA_UNEXPECTERROR	Unexpected DOS error
SPLDATA_OTHER	Any other error

SplMessageBox Parameter - pszText

pszText (PSZ) - input
Pointer to the text string for the message box.

SplMessageBox Parameter - pszCaption

pszCaption (PSZ) - input
Pointer to a string containing a meaningful title for the message box.

The text is centered in the title bar. If more than 40 characters are supplied, excess characters at the beginning and end of the string are not displayed.

SplMessageBox Parameter - idWindow

idWindow (ULONG) - input
Window ID of the message box window.

SplMessageBox Parameter - fsStyle

fsStyle (ULONG) - input
Bit array specifying the contents and function of the message box.

SplMessageBox Return Value - Response

Response (ULONG) - returns
Return codes.

This function returns a ULONG value (sResponse) that indicates the user's response.

MBID_ENTER	ENTER push button was selected
MBID_OK	OK push button was selected
MBID_CANCEL	CANCEL push button was selected
MBID_ABORT	ABORT push button was selected
MBID_RETRY	RETRY push button was selected
MBID_IGNORE	IGNORE push button was selected
MBID_YES	YES push button was selected
MBID_NO	NO push button was selected
MBID_ERROR	Function not successful; an error occurred.

SplMessageBox - Parameters

pszAddress (PSZ) - input
Pointer to a string containing the logical address of the device, such as LPT1.

flErrorInfo (ULONG) - input
Error information.

One of the following flags must be set to identify where the error occurred:

SPLINFO_QPERROR	Spooler queue processor error
SPLINFO_DDERROR	Presentation driver error
SPLINFO_SPLERROR	Spooler error
SPLINFO_OTHERERROR	Any other error

One of the following flags is also set to indicate the severity of the error:

SPLINFO_INFORMATION	Information only, no error.
SPLINFO_WARNING	Warning.
SPLINFO_ERROR	Recoverable error.
SPLINFO_SEVERE	Severe, unrecoverable error.
SPLINFO_USERINTREQD	This flag is optional. It shows that recovery requires action from the user.

flErrorData (ULONG) - input
Length of the data in bytes.

Error data:

SPLDATA_PRINTERJAM	Printer is jammed, offline, or not turned on
SPLDATA_FORMCHGREQD	Form change required
SPLDATA_CARTCHGREQD	Font cartridge change required
SPLDATA_PENCHGREQD	Pen change required
SPLDATA_DATAERROR	Data error, such as missing file
SPLDATA_UNEXPECTERROR	Unexpected DOS error
SPLDATA_OTHER	Any other error

pszText (PSZ) - input
Pointer to the text string for the message box.

pszCaption (PSZ) - input
Pointer to a string containing a meaningful title for the message box.

The text is centered in the title bar. If more than 40 characters are supplied, excess characters at the beginning and end of the string are not displayed.

idWindow (ULONG) - input
Window ID of the message box window.

fsStyle (ULONG) - input
Bit array specifying the contents and function of the message box.

Response (ULONG) - returns
Return codes.

This function returns a ULONG value (sResponse) that indicates the user's response.

MBID_ENTER	ENTER push button was selected
MBID_OK	OK push button was selected
MBID_CANCEL	CANCEL push button was selected
MBID_ABORT	ABORT push button was selected
MBID_RETRY	RETRY push button was selected
MBID_IGNORE	IGNORE push button was selected
MBID_YES	YES push button was selected
MBID_NO	NO push button was selected
MBID_ERROR	Function not successful; an error occurred.

SplMessageBox - Remarks

SplMessageBox creates a message queue if the current thread does not have one.

SplMessageBox is similar to WinMessageBox.

SplMessageBox - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplPdAbortDoc

SplPdAbortDoc - Syntax

Description:

SplPdAbortDoc is an API exported by port drivers. It is called by PrtAbort and PrtAbortDoc for document-abort processing. If FLG_COMPLETE_DATA is set, all prior data for this job is sent to the printer. If FLG_COMPLETE_DATA is **not** set, all current write requests for this job are flushed and then the buffer of data is sent to reset the printer.

FLG_COMPLETE_DATA is always set if the printer driver calls PrtAbort, because some older printer drivers send the reset sequence to the printer using PrtWrite, then call PrtAbort to abort the job. In this case, the printer driver relies on only PrtWrite returning after the data was sent to the printer.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

HFILE    hDevice;  /*  Handle from SplPdOpen. */
PVOID    pchData;  /*  Pointer to data buffer to write after flushing all current write data. */
ULONG    cbData;   /*  Length of data, in bytes. */
ULONG    ulFlags;  /*  Abort processing flags. */
ULONG    rc;       /*  Return codes. */

rc = SplPdAbortDoc(hDevice, pchData, cbData,
       ulFlags);
```

SplPdAbortDoc Parameter - hDevice

hDevice (HFILE) - input
Handle from SplPdOpen.

SplPdAbortDoc Parameter - pchData

pchData (PVOID) - input
Pointer to data buffer to write after flushing all current write data.

SplPdAbortDoc Parameter - cbData

cbData (ULONG) - input
Length of data, in bytes.

May be 0 (zero) for no data to write.

SplPdAbortDoc Parameter - ulFlags

ulFlags (ULONG) - input
Abort processing flags.

Value is as follows:

FLG_COMPLETE_DATA - 0x00000001
: If set, then all prior data for this job must be sent to the printer.

SplPdAbortDoc Return Value - rc

rc (ULONG) - returns
Return codes.

0 : Success

6(ERROR_INVALID_HANDLE)
: Invalid handle given.

29(ERROR_WRITE_FAULT)
: Failure attempting to write reset sequence to the device.

SplPdAbortDoc - Parameters

hDevice (HFILE) - input
Handle from SplPdOpen.

pchData (PVOID) - input
Pointer to data buffer to write after flushing all current write data.

cbData (ULONG) - input
Length of data, in bytes.

May be 0 (zero) for no data to write.

ulFlags (ULONG) - input
Abort processing flags.

Value is as follows:

FLG_COMPLETE_DATA - 0x00000001
: If set, then all prior data for this job must be sent to the printer.

rc (ULONG) - returns
Return codes.

0 : Success

6(ERROR_INVALID_HANDLE)
: Invalid handle given.

29(ERROR_WRITE_FAULT)
: Failure attempting to write reset sequence to the device.

SplPdAbortDoc - Remarks

SplPdClose must still be issued to close the device connection.

SplPdAbortDoc - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplPdClose

SplPdClose - Syntax

Description:

*SplPdClo*se is an API exported by port drivers. It is called during PrtClose to close the device connection and frees the handle.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

HFILE    hDevice;  /*  Handle from SplPdOpen. */
ULONG    rc;       /*  Return codes. */

rc = SplPdClose(hDevice);
```

SplPdClose Parameter - hDevice

hDevice (HFILE) - input
Handle from SplPdOpen.

SplPdClose Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
6(ERROR_INVALID_HANDLE)	Invalid handle given. For all error codes except this one, the SplPdClose must be reissued.

29(ERROR_WRITE_FAULT)
Failure attempting to write buffered data to the device, or the BIDI_ENDJOB bracket failed to be transmitted to the printer.

Note: SplPdClose performs an implicit SplPdSet(BIDI_ENDJOB) if BIDI_STARTJOB has already been issued, unless the job has been aborted due to an offline printer. The port driver should call PrtSet(BIDI_ENDJOB) before closing the connection to the port.

SplPdClose - Parameters

hDevice (HFILE) - input
Handle from SplPdOpen.

rc (ULONG) - returns
Return codes.

0
Success

6(ERROR_INVALID_HANDLE)
Invalid handle given. For all error codes except this one, the SplPdClose must be reissued.

29(ERROR_WRITE_FAULT)
Failure attempting to write buffered data to the device, or the BIDI_ENDJOB bracket failed to be transmitted to the printer.

Note: SplPdClose performs an implicit SplPdSet(BIDI_ENDJOB) if BIDI_STARTJOB has already been issued, unless the job has been aborted due to an offline printer. The port driver should call PrtSet(BIDI_ENDJOB) before closing the connection to the port.

SplPdClose - Remarks

None.

SplPdClose - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplPdEnumPort

SplPdEnumPort - Syntax

Description:

SplPdEnumPort enumerates the port names and port descriptions that this port driver can manipulate.

```
#include <os2.h>

HAB       hab;          /*  The handle to the anchor block. */
PVOID     pBuf;         /*  The pointer to the buffer of data structures. */
ULONG     cbBuf;        /*  The size of the buffer in bytes. */
PULONG    pulReturned;  /*  The number of port entries returned. */
PULONG    pulTotal;     /*  The total number of port entries. */
PULONG    pcbNeeded;    /*  The size of the buffer required for all data. */
APIERR    rc;           /*  Return codes. */

rc = SplPdEnumPort(hab, pBuf, cbBuf, pulReturned,
       pulTotal, pcbNeeded);
```

SplPdEnumPort Parameter - hab

hab (HAB) - input
The handle to the anchor block.

SplPdEnumPort Parameter - pBuf

pBuf (PVOID) - input
The pointer to the buffer of data structures.

SplPdEnumPort Parameter - cbBuf

cbBuf (ULONG) - input
The size of the buffer in bytes.

SplPdEnumPort Parameter - pulReturned

pulReturned (PULONG) - input
The number of port entries returned.

SplPdEnumPort Parameter - pulTotal

pulTotal (PULONG) - input
The total number of port entries.

SplPdEnumPort Parameter - pcbNeeded

pcbNeeded (PULONG) - input
The size of the buffer required for all data.

SplPdEnumPort Return Value - rc

rc (APIERR) - returns
Return codes.

This handling routine returns the following errors:

ERROR_INSUFFICIENT_BUFFER
ERROR_MORE_DATA
NO_ERROR

The buffer, on return, consists of an array of the following data structure:

```
typedef struct _PORTNAMES {
        PSZ pszPortName;
        PSZ pszPortDesc;
} PORTNAMES;
```

SplPdEnumPort - Parameters

hab (HAB) - input
The handle to the anchor block.

pBuf (PVOID) - input
The pointer to the buffer of data structures.

cbBuf (ULONG) - input
The size of the buffer in bytes.

pulReturned (PULONG) - input
The number of port entries returned.

pulTotal (PULONG) - input
The total number of port entries.

pcbNeeded (PULONG) - input
The size of the buffer required for all data.

rc (APIERR) - returns
Return codes.

This handling routine returns the following errors:

ERROR_INSUFFICIENT_BUFFER
ERROR_MORE_DATA
NO_ERROR

The buffer, on return, consists of an array of the following data structure:

```
typedef struct _PORTNAMES {
        PSZ pszPortName;
        PSZ pszPortDesc;
} PORTNAMES;
```

SplPdEnumPort - Remarks

If the buffer is too small to hold even one data structure, the error code ERROR_INSUFFICIENT_BUFFER, is returned. If there is more data, ERROR_MORE_DATA is returned and cbRequired gives the size of buffer required to get all the data.

The Workplace Shell is the expected caller of this function and determines which ports to install in OS2SYS.INI. The port descriptions are used to label the icon for each port. The Workplace Shell does not allow the installation of ports that can be serviced by more than one port driver. The first port installed is used. The Workplace Shell does not display other ports for installation. If another port is required, the first port must be uninstalled.

SplPdEnumPort - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplPdGetPortIcon

SplPdGetPortIcon - Syntax

Description:

SplPdGetPortIcon queries the Resource ID of the icon that represents the port.

```
#include <os2.h>

HAB       hab;      /*  The handle to the anchor block. */
PULONG    idIcon;   /*  The resource ID of the icon bit map. */
BOOL      rc;       /*  Return codes. */

rc = SplPdGetPortIcon(hab, idIcon);
```

SplPdGetPortIcon Parameter - hab

hab (HAB) - input
The handle to the anchor block.

SplPdGetPortIcon Parameter - idIcon

idIcon (PULONG) - input
The resource ID of the icon bit map.

SplPdGetPortIcon Return Value - rc

rc (BOOL) - returns
Return codes.

This handling routine returns FALSE if no icon is available. The system then uses a default port icon.

SplPdGetPortIcon - Parameters

hab (HAB) - input
The handle to the anchor block.

idIcon (PULONG) - input
The resource ID of the icon bit map.

rc (BOOL) - returns
Return codes.

This handling routine returns FALSE if no icon is available. The system then uses a default port icon.

SplPdGetPortIcon - Remarks

Only one icon is used for all the ports supported by a port driver. This limitation is imposed to avoid confusing the user with too many different icons.

The Workplace Shell calls SplQpQueryIcon to load and draw port icons, as appropriate.

SplPdGetPortIcon - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplPdInitPort

SplPdInitPort - Syntax

Description:

SplPdInitPort initializes a port on behalf of the spooler.

```
#include <os2.h>

HFILE     hfile;         /*  The handle to an open file. */
PSZ       pszPortName;   /*  The name of the port to be initialized. */
APIERR    rc;            /*  Return codes. */

rc = SplPdInitPort(hfile, pszPortName);
```

SplPdInitPort Parameter - hfile

hfile (HFILE) - input
The handle to an open file.

SplPdInitPort Parameter - pszPortName

pszPortName (PSZ) - input
The name of the port to be initialized.

SplPdInitPort Return Value - rc

rc (APIERR) - returns
Return codes.

This handling routine returns the following errors:

ERROR_INVALID_PARAMETER
NO_ERROR

SplPdInitPort - Parameters

hfile (HFILE) - input
The handle to an open file.

pszPortName (PSZ) - input
The name of the port to be initialized.

rc (APIERR) - returns
Return codes.

This handling routine returns the following errors:

ERROR_INVALID_PARAMETER
NO_ERROR

SplPdInitPort - Remarks

The port driver reads the initialization data from the INI file, interprets the data, and issues the appropriate DosDevIOCtl calls to the open port given by the file handle.

This function is called from the spooler function, PrtOpen, to complete port opening. Note that PrtOpen issues the actual DosOpen first. PrtOpen holds a semaphore for each port in use, and a linked list of threads waiting on the semaphore because, although the spooler serializes output, it is still possible for applications to write directly to the port by using DevOpenDC (OD_DIRECT).

SplPdInitPort - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplPdInstallPort

SplPdInstallPort - Syntax

Description:

SplPdInstallPort tells the port driver the name of the port that needs to be installed.

```
#include <os2.h>

HAB       hab;          /*  The handle to anchor block. */
PSZ       pszPortName;  /*  The name of the port to be installed. */
APIERR    rc;           /*  Return codes. */

rc = SplPdInstallPort(hab, pszPortName);
```

SplPdInstallPort Parameter - hab

hab (HAB) - input
The handle to anchor block.

SplPdInstallPort Parameter - pszPortName

pszPortName (PSZ) - input
The name of the port to be installed.

SplPdInstallPort Return Value - rc

rc (APIERR) - returns
Return codes.

This handling routine returns the following errors:

ERROR_INVALID_PARAMETER
NO_ERROR

SplPdInstallPort - Parameters

hab (HAB) - input
The handle to anchor block.

pszPortName (PSZ) - input
The name of the port to be installed.

rc (APIERR) - returns
Return codes.

This handling routine returns the following errors:

ERROR_INVALID_PARAMETER
NO_ERROR

SplPdInstallPort - Remarks

This function is called from the Workplace Shell. The port driver writes the initialization data from the INI file. The Workplace Shell then typically calls SplPdSetPort.

SplPdInstallPort - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplPdNewPage

SplPdNewPage - Syntax

Description:

SplPdNewPage is an API exported by port drivers. It is called by PrtNewPage to signal the beginning of a new output page. This call is issued for the first page in a job with page number 1, to signal the spooler that the port driver can determine print job page boundaries.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

HFILE    hDevice;       /*  Handle from SplPdOpen. */
ULONG    ulPageNumber;  /*  The new page number. */
ULONG    rc;            /*  Return codes. */

rc = SplPdNewPage(hDevice, ulPageNumber);
```

SplPdNewPage Parameter - hDevice

hDevice (HFILE) - input
Handle from SplPdOpen.

SplPdNewPage Parameter - ulPageNumber

ulPageNumber (ULONG) - input
The new page number.

SplPdNewPage Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
6(ERROR_INVALID_HANDLE)	Invalid handle given.

SplPdNewPage - Parameters

hDevice (HFILE) - input
Handle from SplPdOpen.

ulPageNumber (ULONG) - input
The new page number.

rc (ULONG) - returns
Return codes.

0	Success
6(ERROR_INVALID_HANDLE)	Invalid handle given.

SplPdNewPage - Remarks

This call might not be issued for raw jobs. Some printer drivers might not be able to determine page breaks for raw print jobs.

SplPdNewPage - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplPdOpen

SplPdOpen - Syntax

Description:

SplPdOpen is an API exported by port drivers. It is called by PrtOpen to open a device for output. SPIPDOpen returns a handle into the port driver to be used on the following calls:

- SplPdAbortDoc
- SplPdClose
- SplPdNewPage
- SplPdResetAbort
- SplPdWrite

SplPdOpen performs an implicit SplPdSet(BIDI_STARTJOB). The port driver should call PrtSet(BIDI_STARTJOB) during this API call.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

PSZ       pszPortName;     /*  Local port name. */
PHFILE    phDevice;        /*  Returned port-driver handle for the device. */
PULONG    pDeviceFlags;    /*  Flag returns type of device being opened. */
PVOID     pPrtOpenStruct;  /*  Pointer to a parameter structure that describes the job being sent to the print dev
ULONG     rc;              /*  Return codes. */

rc = SplPdOpen(pszPortName, phDevice, pDeviceFlags,
       pPrtOpenStruct);
```

SplPdOpen Parameter - pszPortName

pszPortName (PSZ) - input
Local port name.

Following is an example of a port name:

LPT1

SplPdOpen Parameter - phDevice

phDevice (PHFILE) - output
Returned port-driver handle for the device.

SplPdOpen Parameter - pDeviceFlags

pDeviceFlags (PULONG) - output
Flag returns type of device being opened.

Values are as follows:

HANDTYPE_FILE	0x0000
HANDTYPE_DEVICE	0x0001
HANDTYPE_PIPE	0x0002
HANDTYPE_LPTDEVICE	0x0004
HANDTYPE_COMDEVICE	0x0008
HANDTYPE_NETWORK	0x8000

SplPdOpen Parameter - pPrtOpenStruct

pPrtOpenStruct (PVOID) - input
Pointer to a parameter structure that describes the job being sent to the print device.

Some fields may be derived from the *DevOpenDC()* parameter DEVOPENSTRUC. See Data Types for details on the PRTOPENSTRUCT0 data structure.

SplPdOpen Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
5(ERROR_ACCESS_DENIED)	Not authorized to open printer port.
8(ERROR_NOT_ENOUGH_MEMORY)	Not enough memory to satisfy request.
29(ERROR_WRITE_FAULT)	Failure attempting to write to the device. This could happen if a BIDI_START_JOB bracket is sent during open time, and the print device does not accept the start job command.
87(ERROR_INVALID_PARAMETER)	An invalid parameter was specified; most likely an invalid pointer was passed to PrtOpen.
99(ERROR_DEVICE_IN_USE)	Device is in use by another application.

240(ERROR_VC_DISCONNECTED)
Unable to connect to the remote printer.

SplPdOpen - Parameters

pszPortName (PSZ) - input
Local port name.

Following is an example of a port name:

LPT1

phDevice (PHFILE) - output
Returned port-driver handle for the device.

pDeviceFlags (PULONG) - output
Flag returns type of device being opened.

Values are as follows:

HANDTYPE_FILE	0x0000
HANDTYPE_DEVICE	0x0001
HANDTYPE_PIPE	0x0002
HANDTYPE_LPTDEVICE	0x0004
HANDTYPE_COMDEVICE	0x0008
HANDTYPE_NETWORK	0x8000

pPrtOpenStruct (PVOID) - input
Pointer to a parameter structure that describes the job being sent to the print device.

Some fields may be derived from the *DevOpenDC()* parameter DEVOPENSTRUC. See Data Types for details on the PRTOPENSTRUCT0 data structure.

rc (ULONG) - returns
Return codes.

0
Success

5(ERROR_ACCESS_DENIED)
Not authorized to open printer port.

8(ERROR_NOT_ENOUGH_MEMORY)
Not enough memory to satisfy request.

29(ERROR_WRITE_FAULT)
Failure attempting to write to the device. This could happen if a BIDI_START_JOB bracket is sent during open time, and the print device does not accept the start job command.

87(ERROR_INVALID_PARAMETER)
An invalid parameter was specified; most likely an invalid pointer was passed to PrtOpen.

99(ERROR_DEVICE_IN_USE)
Device is in use by another application.

240(ERROR_VC_DISCONNECTED)
Unable to connect to the remote printer.

SplPdOpen - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

SplPdQuery

SplPdQuery - Syntax

Description:

SplPdQuery is an API exported by poirt drivers. It is called by PrtQuery to acquire information about the print device.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

PSZ       pszPortName;  /*  Port name. */
ULONG     ulType;       /*  Type of query and options. */
ULONG     ulCommand;    /*  Generic command code. */
PVOID     pInData;      /*  May contain information required by SplPdQuery. */
ULONG     cbInData;     /*  Length of information in pInData, in bytes. */
PVOID     pOutData;     /*  Return buffer. */
PULONG    pcbOutData;   /*  Points to the length of the output buffer. */
ULONG     rc;           /*  Return codes. */

rc = SplPdQuery(pszPortName, ulType, ulCommand,
       pInData, cbInData, pOutData, pcbOutData);
```

SplPdQuery Parameter - pszPortName

pszPortName (PSZ) - input
Port name.

Following is an example of a port name:

LPT1

This will be a locally defined port name.

SplPdQuery Parameter - ulType

ulType (ULONG) - input
Type of query and options.

Values are as follows:

TYPE_CACHE_ONLY(0)
Returns information from the protocol converter or port driver's cache only. Does not send anything to the printer.

If the requested information is not available, ERROR_INFO_NOT_AVAIL is returned. The caller should reissue the command with a different *ulType* value.

TYPE_SHORT_WAIT(1)
Returns information via a communications channel with the printer that can be used while a job is being sent to the printer.

An example would be using NPAP or SNMP. If the requested information cannot be returned using this type of channel (for instance, if a PJL command is required to get this information), ERROR_INFO_NOT_AVAIL is returned.

TYPE_LONG_WAIT(2)
Returns information using any necessary communications channel with the printer, even if the request has to wait for the current job being sent to the printer to complete.

If the printer supports only one command channel, and this channel can be used while a job is being sent to the printer, TYPE_SHORT_WAIT(1) and TYPE_LONG_WAIT(2) will behave in the same way.

TYPE_CACHE_UPDATE(3)
Returns information from the protocol converter or port driver's cache and also update the cache in the background.

This is an optional type of query to support and, if it is not supported, will return ERROR_INVALID_FUNCTION.

If the requested information is not available in the cache, ERROR_INFO_NOT_AVAIL is returned. The caller should reissue the command with a different *ulType* value.

TYPE_WITHOUT_WRAPPER(4)
Returns passthru or alert information without any packet wrapper around it.

This is useful for BIDI_READ_PASSTHRU as it allows an application to send and receive printer-interpreter language commands without having to know the format of the BIDI protocol's data wrappers. An example would be removing the NPAP wrapper before returning Postscript response data to an application.

If BIDI_READ_PASSTHRU is used without this value, the calling application requests the data as the printer returns it, possibly with any data wrappers.

Note: If this value is used with any command other than BIDI_READ_PASSTHRU, ERROR_INVALID_PARAMETER is returned.

TYPE_CACHE_WAIT(5)
Returns information from the protocol converter or port driver's cache only. If the data is not available in the cache, use any mechanism available to get the information.

If TYPE_CACHE_WAIT(5) is not be supported by the port driver or protocol converter, ERROR_INVALID_PARAMETER is returned.

The value of TYPE_CACHE_WAIT(5) may be used by the print object to issue BIDI_Q_STATUS, to get the current status of the printer without waiting to communicate with the printer.

SplPdQuery Parameter - ulCommand

ulCommand (ULONG) - input
Generic command code.

For all commands except BIDI_WAIT_ALERT and BIDI_Q_PORT, the BIDI protocol converter may be called to generate protocol-specific command data to satisfy query requests. See BIDI Command Structures and Command Flow for contents of *pInData* and *pOutData* for each of the following commands.

Current command values are as follows:

BIDI_READ_PASSTHRU(8001h)
: Waits for any response from printer. This must be done from within a passthru session. See BIDI_START_PASSTHRU(1Ah) for more details. See BIDI_READ_PASSTHRU(8001h) for input and output parameters.

BIDI_Q_PORT(800Bh)
: Query BIDI capabilities of the port. See BIDI_Q_PORT(800Bh) for input and output parameters.

BIDI_Q_SW(800Ch)
: Query BIDI software capabilities. See BIDI_Q_SW(800Ch) for input and output parameters.

BIDI_Q_DEVICE(800Dh)
: Query printer device characteristics. See BIDI_Q_DEVICE(800Dh) for input and output parameters.

BIDI_Q_INTERPRETER(800Eh)
: Query interpreters available in the printer. See BIDI_Q_INTERPRETER(800Eh) for input and output parameters.

BIDI_Q_INPUTBINS(800Fh)
: Query input bins. See BIDI_Q_INPUTBINS(800Fh) for input and output parameters.

BIDI_Q_OUTPUTBINS(8010h)
: Query output bins. See BIDI_Q_OUTPUTBINS(8010h) for input and output parameters.

BIDI_Q_FONTS(8012h)
: Query fonts for a particular interpreter. See BIDI_Q_FONTS(8012h) for input and output parameters.

BIDI_Q_JOBS_COMPLETE(8013h)
: Query list of completed jobs. See BIDI_Q_JOBS_COMPLETE(8013h) for input and output parameters.

BIDI_Q_JOBS_QUEUED(8014h)
: Query list of queued jobs. See BIDI_Q_JOBS_QUEUED(8014h) for input and output parameters.

BIDI_Q_STATUS(8015h)
: Query current status of the print device. See BIDI_Q_STATUS(8015h) for input and output parameters.

BIDI_WAIT_ALERT(8016h)
: Wait for unsolicited alerts. See BIDI_WAIT_ALERT(8016h) for input and output parameters.

BIDI_Q_JOBID(8017h)
: Query the printer Job ID associated with the active PrtOpen/SplPdOpen handle. See BIDI_Q_JOBID(8017h) for input and output parameters.

BIDI_Q_RESPONSE_FMT(8018h)
: Return format of printer-to-host messages. See BIDI_Q_RESPONSE_FMT(8018h) for input and output parameters.

BIDI_Q_PORTDRV(8019h)
: Query port driver's configuration for the port. See BIDI_Q_PORTDRV(8019h) for input and output parameters.

  This command does not go to the printer or protocol converter.

BIDI_Q_STORAGE(8023h)
: Query storage media in printer. See BIDI_Q_STORAGE(8023h) for input and output parameters.

0x0000B000 - 0x0000BFFF
: Port-driver-specific commands.

SplPdQuery Parameter - pInData

pInData (PVOID) - input
May contain information required by SplPdQuery.

See BIDI Command Structures and Command Flow for specific information definitions.

SplPdQuery Parameter - cbInData

cbInData (ULONG) - input
Length of information in *pInData* , in bytes.

SplPdQuery Parameter - pOutData

pOutData (PVOID) - output
Return buffer.

Format of buffer depends on *ulCommand*. For BIDI_READ_PASSTHRU, this will contain BIDI protocol-specific response data and will not be interpreted by the BIDI protocol converter. However, if *ulType* is TYPE_WITHOUT_WRAPPER, any protocol data wrapper is removed before returning the printer-specific information.

For other commands, see BIDI Command Structures and Command Flow for the structure returned.

SplPdQuery Parameter - pcbOutData

pcbOutData (PULONG) - output
Points to the length of the output buffer.

On entry, this is set to the length of the buffer passed in. On exit, it is updated with the length of data available.

If ERROR_MORE_DATA(234) or NERR_BufTooSmall(2123) is set, this field has the buffer size required to get all available information.

SplPdQuery Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
1(ERROR_INVALID_FUNCTION)	The command code is not supported. This lack of support may occur if a new command is sent to an old spooler, port driver, or protocol converter.
2(ERROR_FILE_NOT_FOUND)	The *pszPortName* given is not a valid port name configured for this port driver.
5(ERROR_ACCESS_DENIED)	Not authorized to access printer port.
8(ERROR_NOT_ENOUGH_MEMORY)	Not enough memory to satisfy request.

29(ERROR_WRITE_FAULT)
Failure attempting to write to the device. The printer might not have received the query request.

87(ERROR_INVALID_PARAMETER)
An invalid parameter was specified; most likely an invalid buffer pointer was given.

211(ERROR_INFO_NOT_AVAIL)
The requested information is not available in the protocol converter cache, or the request cannot be sent with the given *ulType* value. The request should be reissued with a different *ulType* value.

234(ERROR_MORE_DATA)
Part of the information requested was returned, but there was not enough room for all the data. **pcbOutData* contains the size of buffer needed to retrieve all the query response data.

640(ERROR_TIMEOUT)
The printer response was not received within the time allotted. The caller may reissue the query, at which time the information might be available.

683(ERROR_BIDI_NO_SUPPORT)
The printer is not in bidirectional mode, therefore query requests are not honored. The port also may be in the process of being shut down.

2123(NERR_BufTooSmall)
No information was returned because the output buffer was too small. **pcbOutData* contains the size of buffer needed to retrieve all the query response data.

0x4005(PMERR_SPL_INV_HSPL)
Not currently in a passthru session.

0x4011(PMERR_SPL_INV_JOB_ID)
No job is being actively sent to the printer port, therefore BIDI_Q_JOBID is not successful.

0x4041(PMERR_SPL_SESSION_TERM)
The passthru session has been terminated.

0x403D(PMERR_SPL_CNV_NOT_INIT)
The protocol converter or port driver has not been initialized for this port.

0x403F(PMERR_SPL_TYPE_NOT_AVAIL)
The storage type requested for BIDI_Q_FONTS or BIDI_Q_STORAGE is not available in the printer.

SplPdQuery - Parameters

pszPortName (PSZ) - input
Port name.

Following is an example of a port name:

LPT1

This will be a locally defined port name.

ulType (ULONG) - input
Type of query and options.

Values are as follows:

TYPE_CACHE_ONLY(0)
Returns information from the protocol converter or port driver's cache only. Does not send anything to the printer.

If the requested information is not available, ERROR_INFO_NOT_AVAIL is returned. The caller should reissue the command with a different *ulType* value.

TYPE_SHORT_WAIT(1)
Returns information via a communications channel with the printer that can be used while a job is being sent to the printer.

An example would be using NPAP or SNMP. If the requested information cannot be returned using this type of channel (for instance, if a PJL command is required to get this information), ERROR_INFO_NOT_AVAIL is returned.

TYPE_LONG_WAIT(2)

Returns information using any necessary communications channel with the printer, even if the request has to wait for the current job being sent to the printer to complete.

If the printer supports only one command channel, and this channel can be used while a job is being sent to the printer, TYPE_SHORT_WAIT(1) and TYPE_LONG_WAIT(2) will behave in the same way.

TYPE_CACHE_UPDATE(3)

Returns information from the protocol converter or port driver's cache and also update the cache in the background.

This is an optional type of query to support and, if it is not supported, will return ERROR_INVALID_FUNCTION.

If the requested information is not available in the cache, ERROR_INFO_NOT_AVAIL is returned. The caller should reissue the command with a different *ulType* value.

TYPE_WITHOUT_WRAPPER(4)

Returns passthru or alert information without any packet wrapper around it.

This is useful for BIDI_READ_PASSTHRU as it allows an application to send and receive printer-interpreter language commands without having to know the format of the BIDI protocol's data wrappers. An example would be removing the NPAP wrapper before returning Postscript response data to an application.

If BIDI_READ_PASSTHRU is used without this value, the calling application requests the data as the printer returns it, possibly with any data wrappers.

Note: If this value is used with any command other than BIDI_READ_PASSTHRU, ERROR_INVALID_PARAMETER is returned.

TYPE_CACHE_WAIT(5)

Returns information from the protocol converter or port driver's cache only. If the data is not available in the cache, use any mechanism available to get the information.

If TYPE_CACHE_WAIT(5) is not be supported by the port driver or protocol converter, ERROR_INVALID_PARAMETER is returned.

The value of TYPE_CACHE_WAIT(5) may be used by the print object to issue BIDI_Q_STATUS, to get the current status of the printer without waiting to communicate with the printer.

ulCommand (ULONG) - input
Generic command code.

For all commands except BIDI_WAIT_ALERT and BIDI_Q_PORT, the BIDI protocol converter may be called to generate protocol-specific command data to satisfy query requests. See BIDI Command Structures and Command Flow for contents of *pInData* and *pOutData* for each of the following commands.

Current command values are as follows:

BIDI_READ_PASSTHRU(8001h)

Waits for any response from printer. This must be done from within a passthru session. See BIDI_START_PASSTHRU(1Ah) for more details. See BIDI_READ_PASSTHRU(8001h) for input and output parameters.

BIDI_Q_PORT(800Bh)

Query BIDI capabilities of the port. See BIDI_Q_PORT(800Bh) for input and output parameters.

BIDI_Q_SW(800Ch)

Query BIDI software capabilities. See BIDI_Q_SW(800Ch) for input and output parameters.

BIDI_Q_DEVICE(800Dh)

Query printer device characteristics. See BIDI_Q_DEVICE(800Dh) for input and output parameters.

BIDI_Q_INTERPRETER(800Eh)

Query interpreters available in the printer. See BIDI_Q_INTERPRETER(800Eh) for input and output parameters.

BIDI_Q_INPUTBINS(800Fh)

Query input bins. See BIDI_Q_INPUTBINS(800Fh) for input and output parameters.

BIDI_Q_OUTPUTBINS(8010h)
Query output bins. See BIDI_Q_OUTPUTBINS(8010h) for input and output parameters.

BIDI_Q_FONTS(8012h)
Query fonts for a particular interpreter. See BIDI_Q_FONTS(8012h) for input and output parameters.

BIDI_Q_JOBS_COMPLETE(8013h)
Query list of completed jobs. See BIDI_Q_JOBS_COMPLETE(8013h) for input and output parameters.

BIDI_Q_JOBS_QUEUED(8014h)
Query list of queued jobs. See BIDI_Q_JOBS_QUEUED(8014h) for input and output parameters.

BIDI_Q_STATUS(8015h)
Query current status of the print device. See BIDI_Q_STATUS(8015h) for input and output parameters.

BIDI_WAIT_ALERT(8016h)
Wait for unsolicited alerts. See BIDI_WAIT_ALERT(8016h) for input and output parameters.

BIDI_Q_JOBID(8017h)
Query the printer Job ID associated with the active PrtOpen/SplPdOpen handle. See BIDI_Q_JOBID(8017h) for input and output parameters.

BIDI_Q_RESPONSE_FMT(8018h)
Return format of printer-to-host messages. See BIDI_Q_RESPONSE_FMT(8018h) for input and output parameters.

BIDI_Q_PORTDRV(8019h)
Query port driver's configuration for the port. See BIDI_Q_PORTDRV(8019h) for input and output parameters.

This command does not go to the printer or protocol converter.

BIDI_Q_STORAGE(8023h)
Query storage media in printer. See BIDI_Q_STORAGE(8023h) for input and output parameters.

0x0000B000 - 0x0000BFFF
Port-driver-specific commands.

pInData (PVOID) - input
May contain information required by SplPdQuery.

See BIDI Command Structures and Command Flow for specific information definitions.

cbInData (ULONG) - input
Length of information in *pInData* , in bytes.

pOutData (PVOID) - output
Return buffer.

Format of buffer depends on *ulCommand*. For BIDI_READ_PASSTHRU, this will contain BIDI protocol-specific response data and will not be interpreted by the BIDI protocol converter. However, if *ulType* is TYPE_WITHOUT_WRAPPER, any protocol data wrapper is removed before returning the printer-specific information.

For other commands, see BIDI Command Structures and Command Flow for the structure returned.

pcbOutData (PULONG) - output
Points to the length of the output buffer.

On entry, this is set to the length of the buffer passed in. On exit, it is updated with the length of data available.

If ERROR_MORE_DATA(234) or NERR_BufTooSmall(2123) is set, this field has the buffer size required to get all available information.

rc (ULONG) - returns
Return codes.

0 Success

1(ERROR_INVALID_FUNCTION)
The command code is not supported. This lack of support may occur if a new command is sent to an old spooler, port driver, or protocol converter.

2(ERROR_FILE_NOT_FOUND)
The *pszPortName* given is not a valid port name configured for this port driver.

5(ERROR_ACCESS_DENIED)
Not authorized to access printer port.

8(ERROR_NOT_ENOUGH_MEMORY)
Not enough memory to satisfy request.

29(ERROR_WRITE_FAULT)
Failure attempting to write to the device. The printer might not have received the query request.

87(ERROR_INVALID_PARAMETER)
An invalid parameter was specified; most likely an invalid buffer pointer was given.

211(ERROR_INFO_NOT_AVAIL)
The requested information is not available in the protocol converter cache, or the request cannot be sent with the given *ulType* value. The request should be reissued with a different *ulType* value.

234(ERROR_MORE_DATA)
Part of the information requested was returned, but there was not enough room for all the data. **pcbOutData* contains the size of buffer needed to retrieve all the query response data.

640(ERROR_TIMEOUT)
The printer response was not received within the time allotted. The caller may reissue the query, at which time the information might be available.

683(ERROR_BIDI_NO_SUPPORT)
The printer is not in bidirectional mode, therefore query requests are not honored. The port also may be in the process of being shut down.

2123(NERR_BufTooSmall)
No information was returned because the output buffer was too small. **pcbOutData* contains the size of buffer needed to retrieve all the query response data.

0x4005(PMERR_SPL_INV_HSPL)
Not currently in a passthru session.

0x4011(PMERR_SPL_INV_JOB_ID)
No job is being actively sent to the printer port, therefore BIDI_Q_JOBID is not successful.

0x4041(PMERR_SPL_SESSION_TERM)
The passthru session has been terminated.

0x403D(PMERR_SPL_CNV_NOT_INIT)
The protocol converter or port driver has not been initialized for this port.

0x403F(PMERR_SPL_TYPE_NOT_AVAIL)
The storage type requested for BIDI_Q_FONTS or BIDI_Q_STORAGE is not available in the printer.

SplPdQuery - Remarks

None.

SplPdQuery - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplPdQueryPort

SplPdQueryPort - Syntax

Description:

SplPdQueryPort returns textual data that describes the port configuration in a way that can be printed by the Workplace Shell.

```
#include <os2.h>

HAB       hab;          /*  The handle to the anchor block. */
PSZ       pszPortName;  /*  The name of the port to be configured. */
PVOID     pBuf;         /*  The pointer to the buffer of data structures. */
ULONG     cbBuf;        /*  The size of the buffer in bytes. */
PULONG    cItems;       /*  The count of the number of strings of description returned. */
APIERR    rc;           /*  Return codes. */

rc = SplPdQueryPort(hab, pszPortName, pBuf,
       cbBuf, cItems);
```

SplPdQueryPort Parameter - hab

hab (HAB) - input
The handle to the anchor block.

SplPdQueryPort Parameter - pszPortName

pszPortName (PSZ) - input
The name of the port to be configured.

SplPdQueryPort Parameter - pBuf

pBuf (PVOID) - input
The pointer to the buffer of data structures.

SplPdQueryPort Parameter - cbBuf

cbBuf (ULONG) - input
The size of the buffer in bytes.

SplPdQueryPort Parameter - cItems

cItems (PULONG) - input
The count of the number of strings of description returned.

SplPdQueryPort Return Value - rc

rc (APIERR) - returns
Return codes.

This handling routine returns the following errors:

ERROR_INSUFFICIENT_BUFFER
NO_ERROR

SplPdQueryPort - Parameters

hab (HAB) - input
The handle to the anchor block.

pszPortName (PSZ) - input
The name of the port to be configured.

pBuf (PVOID) - input
The pointer to the buffer of data structures.

cbBuf (ULONG) - input
The size of the buffer in bytes.

cItems (PULONG) - input
The count of the number of strings of description returned.

rc (APIERR) - returns
Return codes.

This handling routine returns the following errors:

ERROR_INSUFFICIENT_BUFFER
NO_ERROR

SplPdQueryPort - Remarks

The buffer consists of an array of strings (PSZ), the number of which is given by cItems. Each string contains one line of text that can be 0 to 80 characters long. The port name and port description is not required; the Workplace Shell can retrieve these from the OS2SYS.INI file.

The maximum size of the data returned is 4KB. If the buffer is too small, the error code, ERROR_INSUFFICIENT_BUFFER, is returned.

SplPdQueryPort - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplPdRemoteSetPort

SplPdRemoteSetPort - Syntax

Description:

SplPdRemoteSetPort is an API exported by a port driver. It is called by the network print object to display a dialog to an administrator that allows configuration of a server's printer port from a client machine.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

HAB       hab;              /*  Anchor block handle. */
PSZ       pszComputerName;  /*  Name of print server being configured. */
PSZ       pszPortName;      /*  Name of printer port to configure on print server. */
PULONG    pflCapabilities;  /*  Pointer to a flag field. */
ULONG     rc;               /*  Return codes. */

rc = SplPdRemoteSetPort(hab, pszComputerName,
       pszPortName, pflCapabilities);
```

SplPdRemoteSetPort Parameter - hab

hab (HAB) - input
Anchor block handle.

SplPdRemoteSetPort Parameter - pszComputerName

pszComputerName (PSZ) - input

Name of print server being configured.

SplPdRemoteSetPort Parameter - pszPortName

pszPortName (PSZ) - input
Name of printer port to configure on print server.

Following is an example of a port name:

LPT1

SplPdRemoteSetPort Parameter - pflCapabilities

pflCapabilities (PULONG) - input
Pointer to a flag field.

Value is as follows:

FL_ADMIN - 0x00000001
On entry, if this flag is set the administrator wants the remote port settings displayed.

SplPdRemoteSetPort Return Value - rc

rc (ULONG) - returns
Return codes.

0 Success

87(ERROR_INVALID_PARAMETER)
An invalid parameter was specified; most likely an invalid *pszPortName* or *pflModified* was given.

SplPdRemoteSetPort - Parameters

hab (HAB) - input
Anchor block handle.

pszComputerName (PSZ) - input
Name of print server being configured.

pszPortName (PSZ) - input
Name of printer port to configure on print server.

Following is an example of a port name:

LPT1

pflCapabilities (PULONG) - input
Pointer to a flag field.

Value is as follows:

FL_ADMIN - 0x00000001
On entry, if this flag is set the administrator wants the remote port settings displayed.

rc (ULONG) - returns
Return codes.

0 Success

87(ERROR_INVALID_PARAMETER)
An invalid parameter was specified; most likely an invalid *pszPortName* or *pflModified* was given.

SplPdRemoteSetPort - Remarks

Note:

This optional API uses BIDI_Q_PORTDRV to get the current configuration parameters for the print server's port driver. These parameters are used to generate the port setup dialog box.

The port driver entry for this routine should not assume that the port driver is installed on this client machine. For instance, if the port driver is not installed on the client and the port driver module is available on the network, the print object can load and call the port driver at this entry point.

SplPdRemoteSetPort - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplPdRemovePort

SplPdRemovePort - Syntax

Description:

SplPdRemovePort tells the port driver the name of the port that needs to be removed.

```
#include <os2.h>

HAB       hab;          /*  The handle to anchor block. */
PSZ       pszPortName;  /*  The name of the port to be removed. */
APIERR    rc;           /*  Return codes. */

rc = SplPdRemovePort(hab, pszPortName);
```

SplPdRemovePort Parameter - hab

hab (HAB) - input
The handle to anchor block.

SplPdRemovePort Parameter - pszPortName

pszPortName (PSZ) - input
The name of the port to be removed.

SplPdRemovePort Return Value - rc

rc (APIERR) - returns
Return codes.

This handling routine returns the following errors:

ERROR_INVALID_PARAMETER
NO_ERROR

SplPdRemovePort - Parameters

hab (HAB) - input
The handle to anchor block.

pszPortName (PSZ) - input
The name of the port to be removed.

rc (APIERR) - returns
Return codes.

This handling routine returns the following errors:

ERROR_INVALID_PARAMETER
NO_ERROR

SplPdRemovePort - Remarks

This function is called from the Workplace Shell and allows the port driver to remove its data from the INI file.

The port driver must remove the printer port from the system INI file by removing ApplicationName "PM_SPOOLER_PORT" and KeyName "pszPortName". The port driver should also remove its own data for this port from the system INI file by removing ApplicationName "PM_<pszPortName>", where "pszPortName" is the designation of the port in question.

SplPdRemovePort - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplPdResetAbort

SplPdResetAbort - Syntax

Description:

SplPdResetAbort is an API exported by port drivers. It is called by PrtResetAbort to reset an aborted device handle to again allow writes. After the reset is successful, the printer driver is allowed to call PrtWrite for the device handle to send the reset sequence to the printer.

Note:

Resetting an aborted device handle does not initiate a new printer job. It only allows writes to be sent to the printer.

SplPdResetAbort is an optional API; it does not have to be exported by the port driver.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

HFILE    hDevice;  /*  Handle from SplPdOpen. */
ULONG    rc;       /*  Return codes. */

rc = SplPdResetAbort(hDevice);
```

SplPdResetAbort Parameter - hDevice

hDevice (HFILE) - input
Handle from SplPdOpen.

SplPdResetAbort Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
6(ERROR_INVALID_HANDLE)	Invalid handle given.

SplPdResetAbort - Parameters

hDevice (HFILE) - input
Handle from SplPdOpen.

rc (ULONG) - returns
Return codes.

0	Success
6(ERROR_INVALID_HANDLE)	Invalid handle given.

SplPdResetAbort - Remarks

SplPdClose must still be issued to close the device connection.

SplPdResetAbort - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplPdSendCmd

SplPdSendCmd - Syntax

Description:

SplPdSendCmd is an API exported by port drivers. It is called by a protocol converter to send protocol-specific commands to the printer. SplPdSendCmd does not return any response data **from** the printer. This is an optional API.

If the port driver exports SplPdSendCmd, the spooler passes the address of this function to the BIDI protocol converter SplProtSendCmd. The purpose is to achieve better performance of query requests.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

PSZ      pszPortName;  /*  Port name. */
ULONG    flOptions;    /*  Flags defining how to send the data to the printer. */
ULONG    ulCommand;    /*  Generic command code. */
PVOID    pInData;      /*  Contains the command sequence to pass to the printer. */
ULONG    cbInData;     /*  Length of command sequence in pInData, in bytes. */
ULONG    rc;           /*  Return codes. */

rc = SplPdSendCmd(pszPortName, flOptions,
       ulCommand, pInData, cbInData);
```

SplPdSendCmd Parameter - pszPortName

pszPortName (PSZ) - input
Port name.

Following is an example of a port name:

LPT1

This will be a locally defined port name.

SplPdSendCmd Parameter - flOptions

flOptions (ULONG) - input
Flags defining how to send the data to the printer.

All other bits must be clear. Values are as follows:

FLG_MUSTCOMPLETE - 0x00000001
If set, the caller does not want to return until the command sequence is sent to the printer.

If this flag is **not** set, the caller returns immediately if unable to send the command. For example, the command might not be sent if the printer is being sent a job and the printer cannot accept commands inside a job boundary.

FLG_SYNCH - 0x00000002
If set, waits for any current job to finish being sent to the printer before sending this command sequence.

FLG_MORECMDS - 0x00000004

If set, more SplPdSendCmd calls are necessary to complete the current query or set command.

FLG_DATACHANNEL - 0x00000008

If set, the data channel should be used for this command.

The data channel is the channel into the printer on which print job data is sent. Some printers also support a command channel, such as SNMP, which can be used to send commands into the printer.

However, some commands must be sent to the printer using the data channel instead of the command channel (for example, sending a PJL query requires sending the PJL command via the data channel.)

FLG_SNMPCHANNEL - 0x00000010

If set, the SNMP channel should be used for this command.

If this flag is set and there is no SNMP channel available, ERROR_INVALID_PATH(253) is returned.

FLG_RESERVEDCHANNEL - 0x00000020

This bit may be used to access a special channel in the port driver. If this flag is set and the port driver does not support a special channel, the default command channel should be used to send this data to the printer.

SplPdSendCmd Parameter - ulCommand

ulCommand (ULONG) - input

Generic command code.

This code is given to the port driver for information only. The *pInData* parameter contains the BIDI software protocol-specific packet to send to the printer.

Current commands are as follows:

BIDI_Q_SW(800Ch)

Query BIDI software capabilities.

BIDI_Q_DEVICE(800Dh)

Query printer device characteristics.

BIDI_Q_INTERPRETER(800Eh)

Query interpreters available in the printer.

BIDI_Q_INPUTBINS(800Fh)

Query input bins.

BIDI_Q_OUTPUTBINS(8010h)

Query output bins.

BIDI_Q_FONTS(8012h)

Query fonts for a particular interpreter.

BIDI_Q_JOBS_COMPLETE(8013h)

Query list of completed jobs.

BIDI_Q_JOBS_QUEUED(8014h)

Query list of queued jobs.

BIDI_Q_STATUS(8015h)

Query current status of the print device.

BIDI_Q_STORAGE(8023h)

Return printer storage media.

SplPdSendCmd Parameter - pInData

pInData (PVOID) - input
Contains the command sequence to pass to the printer.

This parameter will not be altered by the port driver.

SplPdSendCmd Parameter - cbInData

cbInData (ULONG) - input
Length of command sequence in *pInData* , in bytes.

SplPdSendCmd Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
2(ERROR_FILE_NOT_FOUND)	The *pszPortName* given is not a valid port name configured for this port driver.
5(ERROR_ACCESS_DENIED)	Not authorized to access printer port.
8(ERROR_NOT_ENOUGH_MEMORY)	Not enough memory to satisfy request.
29(ERROR_WRITE_FAULT)	Failure attempting to write to the device. The printer might not have received the command sequence.
87(ERROR_INVALID_PARAMETER)	An invalid buffer was given.
253(ERROR_INVALID_PATH)	Invalid channel requested.
683(ERROR_BIDI_NO_SUPPORT)	The printer is not in bidirectional mode, therefore query requests are not honored. The port also may be in the process of being shut down.
0x4013(PMERR_SPL_JOB_PRINTING)	A job is being sent to the printer, the printer cannot accept commands while receiving job data, and FLG_MUSTCOMPLETE is clear.
0x403D(PMERR_SPL_CNV_NOT_INIT)	The protocol converter or port driver has not been initialized for this port.

SplPdSendCmd - Parameters

pszPortName (PSZ) - input
Port name.

Following is an example of a port name:

LPT1

This will be a locally defined port name.

flOptions (ULONG) - input
Flags defining how to send the data to the printer.

All other bits must be clear. Values are as follows:

FLG_MUSTCOMPLETE - 0x00000001
If set, the caller does not want to return until the command sequence is sent to the printer.

If this flag is **not** set, the caller returns immediately if unable to send the command. For example, the command might not be sent if the printer is being sent a job and the printer cannot accept commands inside a job boundary.

FLG_SYNCH - 0x00000002
If set, waits for any current job to finish being sent to the printer before sending this command sequence.

FLG_MORECMDS - 0x00000004
If set, more SplPdSendCmd calls are necessary to complete the current query or set command.

FLG_DATACHANNEL - 0x00000008
If set, the data channel should be used for this command.

The data channel is the channel into the printer on which print job data is sent. Some printers also support a command channel, such as SNMP, which can be used to send commands into the printer.

However, some commands must be sent to the printer using the data channel instead of the command channel (for example, sending a PJL query requires sending the PJL command via the data channel.)

FLG_SNMPCHANNEL - 0x00000010
If set, the SNMP channel should be used for this command.

If this flag is set and there is no SNMP channel available, ERROR_INVALID_PATH(253) is returned.

FLG_RESERVEDCHANNEL - 0x00000020
This bit may be used to access a special channel in the port driver. If this flag is set and the port driver does not support a special channel, the default command channel should be used to send this data to the printer.

ulCommand (ULONG) - input
Generic command code.

This code is given to the port driver for information only. The *pInData* parameter contains the BIDI software protocol-specific packet to send to the printer.

Current commands are as follows:

BIDI_Q_SW(800Ch)
Query BIDI software capabilities.

BIDI_Q_DEVICE(800Dh)
Query printer device characteristics.

BIDI_Q_INTERPRETER(800Eh)
Query interpreters available in the printer.

BIDI_Q_INPUTBINS(800Fh)
Query input bins.

BIDI_Q_OUTPUTBINS(8010h)
Query output bins.

BIDI_Q_FONTS(8012h)
Query fonts for a particular interpreter.

BIDI_Q_JOBS_COMPLETE(8013h)
Query list of completed jobs.

BIDI_Q_JOBS_QUEUED(8014h)
Query list of queued jobs.

BIDI_Q_STATUS(8015h)
Query current status of the print device.

BIDI_Q_STORAGE(8023h)
Return printer storage media.

pInData (PVOID) - input
Contains the command sequence to pass to the printer.

This parameter will not be altered by the port driver.

cbInData (ULONG) - input
Length of command sequence in *pInData* , in bytes.

rc (ULONG) - returns
Return codes.

0 Success

2(ERROR_FILE_NOT_FOUND)
The *pszPortName* given is not a valid port name configured for this port driver.

5(ERROR_ACCESS_DENIED)
Not authorized to access printer port.

8(ERROR_NOT_ENOUGH_MEMORY)
Not enough memory to satisfy request.

29(ERROR_WRITE_FAULT)
Failure attempting to write to the device. The printer might not have received the command sequence.

87(ERROR_INVALID_PARAMETER)
An invalid buffer was given.

253(ERROR_INVALID_PATH)
Invalid channel requested.

683(ERROR_BIDI_NO_SUPPORT)
The printer is not in bidirectional mode, therefore query requests are not honored. The port also may be in the process of being shut down.

0x4013(PMERR_SPL_JOB_PRINTING)
A job is being sent to the printer, the printer cannot accept commands while receiving job data, and FLG_MUSTCOMPLETE is clear.

0x403D(PMERR_SPL_CNV_NOT_INIT)
The protocol converter or port driver has not been initialized for this port.

SplPdSendCmd - Remarks

Note:

The caller will only wait for a response inside the protocol converter, because the protocol converter is the only part of the system that can match a command with the printer's response.

If the last SplPdSendCmd has FLG_MORECMDS set and no new SplPdSendCmd is issued for the port driver's timeout, then the port driver should allow a job to be sent. This can happen only when there is a problem with the protocol converter.

SplPdSendCmd - Topics

Select an item:
Syntax
Parameters

Returns
Remarks
Glossary

SplPdSet

SplPdSet - Syntax

Description:

SplPdSet is an API exported by port drivers. It is called by PrtSet to set printer device information.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

PSZ      pszPortName;  /*  Port name. */
ULONG    ulType;       /*  Type of query and options. */
ULONG    ulCommand;    /*  Generic command code. */
PVOID    pInData;      /*  May contain information required by SplPdSet. */
ULONG    cbInData;     /*  Length of information in pInData, in bytes. */
ULONG    rc;           /*  Return codes. */

rc = SplPdSet(pszPortName, ulType, ulCommand,
       pInData, cbInData);
```

SplPdSet Parameter - pszPortName

pszPortName (PSZ) - input
Port name.

Following is an example of a port name:

LPT1

This will be a locally defined port name.

SplPdSet Parameter - ulType

ulType (ULONG) - input
Type of query and options.

Values are as follows:

TYPE_SHORT_WAIT(1)
Sends information via a communications channel with the printer that can be used while a job is being

sent to the printer.

An example would be using NPAP or SNMP. If the requested information cannot be sent using this type of channel (for instance, if a PJL command is required to get this information), ERROR_INFO_NOT_AVAIL is returned.

TYPE_LONG_WAIT(2)

Sends information using any necessary communications channel with the printer, even if the request has to wait for the current job being sent to the printer to complete.

If the printer supports only one command channel, and this channel can be used while a job is being sent to the printer, TYPE_SHORT_WAIT(1) and TYPE_LONG_WAIT(2) will behave in the same way.

TYPE_WITHOUT_WRAPPER(4)

Indicates the information being sent does not have any packet wrapper around it.

This is useful for BIDI_SEND_PASSTHRU as it allows an application to send and receive printer-interpreter language commands without having to know the format of the BIDI protocol's data wrappers. An example would be sending a PostScript query to the printer, and the NPAP protocol converter adds the necessary NPAP wrapper before transmitting the data to the printer.

If BIDI_SEND_PASSTHRU is used without this value, the calling application must have already wrapped the passthru data.

Note: If this value is used with any command other than BIDI_SEND_PASSTHRU, ERROR_INVALID_PARAMETER is returned.

SplPdSet Parameter - ulCommand

ulCommand (ULONG) - input
Generic command code.

The BIDI protocol converter may be called to generate protocol-specific command data to satisfy set requests. See BIDI Command Structures and Command Flow for contents of *pInData* and *pOutData* for each of the following commands.

Current command values are as follows:

BIDI_SEND_PASSTHRU(8001h)

Printer protocol-specific command will be issued to the printer. See BIDI_SEND_PASSTHRU(1h) for input parameters.

BIDI_STARTJOB(2h)

Signals start of print job. See BIDI_STARTJOB(2h) for input parameters.

BIDI_ENDJOB(3h)

Signals end of print job. See BIDI_ENDJOB(3h) for input parameters.

BIDI_HOLDJOB(4h)

Holds printer job. See BIDI_HOLDJOB(4h) for input parameters.

BIDI_RELEASEJOB(5h)

Releases printer job. See BIDI_RELEASEJOB(5h) for input parameters.

BIDI_CANCELJOB(6h)

Cancels printer job. See BIDI_CANCELJOB(6h) for input parameters.

BIDI_INIT_PORTDRV(8h)

Initializes a port driver. See BIDI_INIT_PORTDRV(8h) for input parameters.

BIDI_RESET(9h)

Resets the printer. See BIDI_RESET(9h) for input parameters.

BIDI_SHUTDOWN(Ah)

Releases all threads waiting for this port.

Used to shut down a printer port. Sent to the protocol converter before being sent to the port driver. See BIDI_SHUTDOWN(Ah) for input parameters.

BIDI_INIT(Bh)

Sets printer in BIDI mode.

Changes printer from unidirectional to bidirectional. Does not call the protocol converter. See BIDI_INIT(Bh) for input parameters.

BIDI_TERM(Ch)

Sets printer in unidirectional mode.

Changes printer from bidirectional to unidirectional. Does not call the protocol converter. See BIDI_TERM(Ch) for input parameters.

BIDI_RESPONSE_FMT(Dh)

Defines the format of printer-to-host messages. See BIDI_RESPONSE_FMT(Dh) for input parameters.

BIDI_PACKET_SIZE(Eh)

Defines printer-to-host maximum packet size. See BIDI_PACKET_SIZE(Eh) for input parameters.

BIDI_SET_PORTDRV(19h)

Sets port driver configuration. See BIDI_SET_PORTDRV(19h) for input parameters.

BIDI_START_PASSTHRU(1Ah)

Initiates a passthru session with the printer. See BIDI_START_PASSTHRU(1Ah) for input parameters.

BIDI_END_PASSTHRU(1Bh)

Terminates a passthru session with the printer. See BIDI_END_PASSTHRU(1Bh) for input parameters.

BIDI_NOTIFY_ENDJOBCONNECT(20h)

Notifies port driver that jobs are complete. See BIDI_NOTIFY_ENDJOBCONNECT(20h) for input parameters.

BIDI_NOTIFY_PORT_SELECTED(21h)

Notifies port driver that printer is connected to print queue. See BIDI_NOTIFY_PORT_SELECTED(21h) for input parameters.

BIDI_NOTIFY_PORT_RELEASED(22h)

Notifies port driver that printer is not connected to print queue. See BIDI_NOTIFY_PORT_RELEASED(22h) for input parameters.

BIDI_SET_DEVICE_ID(23h)

Tells port driver the printer's device ID. See BIDI_SET_DEVICE_ID(23h) for input parameters.

BIDI_ENABLE_ALERT(24h)

Enables printer alerts. See BIDI_ENABLE_ALERT(24h) for input parameters.

BIDI_DISABLE_ALERT(25h)

Disables printer alerts. See BIDI_DISABLE_ALERT(25h) for input parameters.

BIDI_ADD_VIRTUAL_PORT(26h)

Adds a virtual port. See BIDI_ADD_VIRTUAL_PORT(26h) for input parameters.

BIDI_DEL_VIRTUAL_PORT(27h)

Deletes a virtual port. See BIDI_DEL_VIRTUAL_PORT(27h) for input parameters.

BIDI_DEL_PORT(28h)

Indicates a spooler port has been deleted. See BIDI_DEL_PORT(28h) for input parameters.

0x0000B000 - 0x0000BFFF

Port-driver-specific commands.

SplPdSet Parameter - pInData

pInData (PVOID) - input

May contain information required by SplPdSet.

If BIDI_SEND_PASSTHRU is set, this parameter contains the command sequence to pass to the printer and will not be altered by the port driver.

If BIDI_SEND_PASSTHRU is **not** set, this parameter may contain information required by SplPdSet. See BIDI Command Structures and

Command Flow for specific information definitions.

SplPdSet Parameter - cbInData

cbInData (ULONG) - input
Length of information in *pInData* , in bytes.

SplPdSet Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
1(ERROR_INVALID_FUNCTION)	The command code is not supported. This lack of support may occur if a new command is sent to an old spooler, port driver, or protocol converter.
2(ERROR_FILE_NOT_FOUND)	The *pszPortName* given is not a valid port name configured for this port driver.
5(ERROR_ACCESS_DENIED)	Not authorized to access printer port.
8(ERROR_NOT_ENOUGH_MEMORY)	Not enough memory to satisfy request.
29(ERROR_WRITE_FAULT)	Failure attempting to write to the device. The printer might not have received the set request.
87(ERROR_INVALID_PARAMETER)	An invalid parameter was specified; most likely an invalid buffer pointer was given.
211(ERROR_INFO_NOT_AVAIL)	The requested information is not available in the protocol converter cache, or the request cannot be sent with the given *ulType* value. The request should be reissued with a different *ulType* value.
640(ERROR_TIMEOUT)	The set command was not able to be sent to the printer within the time allotted.
683(ERROR_BIDI_NO_SUPPORT)	The printer is not in bidirectional mode, therefore query requests are not honored. The port also may be in the process of being shut down.
0x1403(PMERR_ALREADY_INITIALIZED)	The port driver has already been initialized for the port.
0x4005(PMERR_SPL_INV_HSPL)	Not currently in a passthru session.
0x4011(PMERR_SPL_INV_JOB_ID)	The Job ID is not known.
0x403D(PMERR_SPL_CNV_NOT_INIT)	The protocol converter or port driver has not been initialized for this port.

SplPdSet - Parameters

pszPortName (PSZ) - input
Port name.

Following is an example of a port name:

LPT1

This will be a locally defined port name.

ulType (ULONG) - input
Type of query and options.

Values are as follows:

TYPE_SHORT_WAIT(1)
Sends information via a communications channel with the printer that can be used while a job is being sent to the printer.

An example would be using NPAP or SNMP. If the requested information cannot be sent using this type of channel (for instance, if a PJL command is required to get this information), ERROR_INFO_NOT_AVAIL is returned.

TYPE_LONG_WAIT(2)
Sends information using any necessary communications channel with the printer, even if the request has to wait for the current job being sent to the printer to complete.

If the printer supports only one command channel, and this channel can be used while a job is being sent to the printer, TYPE_SHORT_WAIT(1) and TYPE_LONG_WAIT(2) will behave in the same way.

TYPE_WITHOUT_WRAPPER(4)
Indicates the information being sent does not have any packet wrapper around it.

This is useful for BIDI_SEND_PASSTHRU as it allows an application to send and receive printer-interpreter language commands without having to know the format of the BIDI protocol's data wrappers. An example would be sending a PostScript query to the printer, and the NPAP protocol converter adds the necessary NPAP wrapper before transmitting the data to the printer.

If BIDI_SEND_PASSTHRU is used without this value, the calling application must have already wrapped the passthru data.

Note: If this value is used with any command other than BIDI_SEND_PASSTHRU, ERROR_INVALID_PARAMETER is returned.

ulCommand (ULONG) - input
Generic command code.

The BIDI protocol converter may be called to generate protocol-specific command data to satisfy set requests. See BIDI Command Structures and Command Flow for contents of *pInData* and *pOutData* for each of the following commands.

Current command values are as follows:

BIDI_SEND_PASSTHRU(8001h)
Printer protocol-specific command will be issued to the printer. See BIDI_SEND_PASSTHRU(1h) for input parameters.

BIDI_STARTJOB(2h)
Signals start of print job. See BIDI_STARTJOB(2h) for input parameters.

BIDI_ENDJOB(3h)
Signals end of print job. See BIDI_ENDJOB(3h) for input parameters.

BIDI_HOLDJOB(4h)
Holds printer job. See BIDI_HOLDJOB(4h) for input parameters.

BIDI_RELEASEJOB(5h)
Releases printer job. See BIDI_RELEASEJOB(5h) for input parameters.

BIDI_CANCELJOB(6h)
Cancels printer job. See BIDI_CANCELJOB(6h) for input parameters.

BIDI_INIT_PORTDRV(8h)
Initializes a port driver. See BIDI_INIT_PORTDRV(8h) for input parameters.

BIDI_RESET(9h)

Resets the printer. See BIDI_RESET(9h) for input parameters.

BIDI_SHUTDOWN(Ah)

Releases all threads waiting for this port.

Used to shut down a printer port. Sent to the protocol converter before being sent to the port driver. See BIDI_SHUTDOWN(Ah) for input parameters.

BIDI_INIT(Bh)

Sets printer in BIDI mode.

Changes printer from unidirectional to bidirectional. Does not call the protocol converter. See BIDI_INIT(Bh) for input parameters.

BIDI_TERM(Ch)

Sets printer in unidirectional mode.

Changes printer from bidirectional to unidirectional. Does not call the protocol converter. See BIDI_TERM(Ch) for input parameters.

BIDI_RESPONSE_FMT(Dh)

Defines the format of printer-to-host messages. See BIDI_RESPONSE_FMT(Dh) for input parameters.

BIDI_PACKET_SIZE(Eh)

Defines printer-to-host maximum packet size. See BIDI_PACKET_SIZE(Eh) for input parameters.

BIDI_SET_PORTDRV(19h)

Sets port driver configuration. See BIDI_SET_PORTDRV(19h) for input parameters.

BIDI_START_PASSTHRU(1Ah)

Initiates a passthru session with the printer. See BIDI_START_PASSTHRU(1Ah) for input parameters.

BIDI_END_PASSTHRU(1Bh)

Terminates a passthru session with the printer. See BIDI_END_PASSTHRU(1Bh) for input parameters.

BIDI_NOTIFY_ENDJOBCONNECT(20h)

Notifies port driver that jobs are complete. See BIDI_NOTIFY_ENDJOBCONNECT(20h) for input parameters.

BIDI_NOTIFY_PORT_SELECTED(21h)

Notifies port driver that printer is connected to print queue. See BIDI_NOTIFY_PORT_SELECTED(21h) for input parameters.

BIDI_NOTIFY_PORT_RELEASED(22h)

Notifies port driver that printer is not connected to print queue. See BIDI_NOTIFY_PORT_RELEASED(22h) for input parameters.

BIDI_SET_DEVICE_ID(23h)

Tells port driver the printer's device ID. See BIDI_SET_DEVICE_ID(23h) for input parameters.

BIDI_ENABLE_ALERT(24h)

Enables printer alerts. See BIDI_ENABLE_ALERT(24h) for input parameters.

BIDI_DISABLE_ALERT(25h)

Disables printer alerts. See BIDI_DISABLE_ALERT(25h) for input parameters.

BIDI_ADD_VIRTUAL_PORT(26h)

Adds a virtual port. See BIDI_ADD_VIRTUAL_PORT(26h) for input parameters.

BIDI_DEL_VIRTUAL_PORT(27h)

Deletes a virtual port. See BIDI_DEL_VIRTUAL_PORT(27h) for input parameters.

BIDI_DEL_PORT(28h)

Indicates a spooler port has been deleted. See BIDI_DEL_PORT(28h) for input parameters.

0x0000B000 - 0x0000BFFF

Port-driver-specific commands.

pInData (PVOID) - input

May contain information required by SplPdSet.

If BIDI_SEND_PASSTHRU is set, this parameter contains the command sequence to pass to the printer and will not be altered by the port driver.

If BIDI_SEND_PASSTHRU is **not** set, this parameter may contain information required by SplPdSet. See BIDI Command Structures and Command Flow for specific information definitions.

cbInData (ULONG) - input
Length of information in *pInData* , in bytes.

rc (ULONG) - returns
Return codes.

0	Success
1(ERROR_INVALID_FUNCTION)	The command code is not supported. This lack of support may occur if a new command is sent to an old spooler, port driver, or protocol converter.
2(ERROR_FILE_NOT_FOUND)	The *pszPortName* given is not a valid port name configured for this port driver.
5(ERROR_ACCESS_DENIED)	Not authorized to access printer port.
8(ERROR_NOT_ENOUGH_MEMORY)	Not enough memory to satisfy request.
29(ERROR_WRITE_FAULT)	Failure attempting to write to the device. The printer might not have received the set request.
87(ERROR_INVALID_PARAMETER)	An invalid parameter was specified; most likely an invalid buffer pointer was given.
211(ERROR_INFO_NOT_AVAIL)	The requested information is not available in the protocol converter cache, or the request cannot be sent with the given *ulType* value. The request should be reissued with a different *ulType* value.
640(ERROR_TIMEOUT)	The set command was not able to be sent to the printer within the time allotted.
683(ERROR_BIDI_NO_SUPPORT)	The printer is not in bidirectional mode, therefore query requests are not honored. The port also may be in the process of being shut down.
0x1403(PMERR_ALREADY_INITIALIZED)	The port driver has already been initialized for the port.
0x4005(PMERR_SPL_INV_HSPL)	Not currently in a passthru session.
0x4011(PMERR_SPL_INV_JOB_ID)	The Job ID is not known.
0x403D(PMERR_SPL_CNV_NOT_INIT)	The protocol converter or port driver has not been initialized for this port.

SplPdSet - Remarks

None.

SplPdSet - Topics

Select an item:
Syntax
Parameters
Returns
Remarks

SplPdSetPort

SplPdSetPort - Syntax

Description:

SplPdSetPort displays a dialog to allow the user to browse and modify port configurations.

```
#include <os2.h>

HAB       hab;          /*  The handle to the anchor block. */
PSZ       pszPortName;  /*  The name of the port to be removed. */
PULONG    flModified;   /*  A flag to indicate that the configuration has been modified. */
APIERR    rc;           /*  Return codes. */

rc = SplPdSetPort(hab, pszPortName, flModified);
```

SplPdSetPort Parameter - hab

hab (HAB) - input
The handle to the anchor block.

SplPdSetPort Parameter - pszPortName

pszPortName (PSZ) - input
The name of the port to be removed.

SplPdSetPort Parameter - flModified

flModified (PULONG) - input
A flag to indicate that the configuration has been modified.

SplPdSetPort Return Value - rc

rc (APIERR) - returns
Return codes.

This handling routine returns the following errors:

ERROR_INVALID_PARAMETER
NO_ERROR

SplPdSetPort - Parameters

hab (HAB) - input
The handle to the anchor block.

pszPortName (PSZ) - input
The name of the port to be removed.

flModified (PULONG) - input
A flag to indicate that the configuration has been modified.

rc (APIERR) - returns
Return codes.

This handling routine returns the following errors:

ERROR_INVALID_PARAMETER
NO_ERROR

SplPdSetPort - Remarks

This function is called from the Workplace Shell. The port driver retrieves the values previously stored in OS2SYS.INI and displays a dialog box. Default values are used the first time if no previous values were stored in OS2SYS.INI.

When the user selects **OK** from the dialog box, the port driver stores the changed values back into OS2SYS.INI. The flag flModified is not set if the user did not modify anything or if the user selected **CANCEL** from the dialog box.

Note: The dialog should contain everything required for port initialization and termination.

SplPdSetPort - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplPdTermPort

SplPdTermPort - Syntax

Description:

SplPdTermPort terminates a port on behalf of the spooler.

```
#include <os2.h>

HFILE     hfile;        /*  The handle to an open file. */
PSZ       pszPortName;  /*  The name of the port to be terminated (closed). */
APIERR    rc;           /*  Return codes. */

rc = SplPdTermPort(hfile, pszPortName);
```

SplPdTermPort Parameter - hfile

hfile (HFILE) - input
The handle to an open file.

SplPdTermPort Parameter - pszPortName

pszPortName (PSZ) - input
The name of the port to be terminated (closed).

SplPdTermPort Return Value - rc

rc (APIERR) - returns
Return codes.

This handling routine returns the following errors:

ERROR_INVALID_PARAMETER
NO_ERROR

SplPdTermPort - Parameters

hfile (HFILE) - input
The handle to an open file.

pszPortName (PSZ) - input
The name of the port to be terminated (closed).

rc (APIERR) - returns
Return codes.

This handling routine returns the following errors:

ERROR_INVALID_PARAMETER
NO_ERROR

SplPdTermPort - Remarks

The port driver reads the termination data from the INI file, interprets the data, and issues the appropriate DosDevIOCtls to the open port given by the file handle.

This function is called from the spooler function, PrtClose, to start port close down. Note that PrtClose then issues a DosClose.

SplPdTermPort - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplPdWrite

SplPdWrite - Syntax

Description:

SplPdWrite is an API exported by port drivers. It is called by PrtWrite to send print job data to the device. If the BIDI software protocol being used requires print job data to have a wrapper around it (see BIDI_Q_SW(800Ch) in BIDI Command Structures and Command Flow), then the spooler will call the protocol converter's API, SplProtWrite. SplProtWrite will then call the port driver's SplPdWrite routine.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

HFILE    hDevice;      /*  Handle from SplPdOpen. */
PVOID    pchData;      /*  Pointer to data buffer to write. */
ULONG    cbData;       /*  Length of data, in bytes. */
ULONG    pcbWritten;   /*  Pointer to count of bytes actually written to the device. */
ULONG    rc;           /*  Return codes. */
```

```
rc = SplPdWrite(hDevice, pchData, cbData,
       pcbWritten);
```

SplPdWrite Parameter - hDevice

hDevice (HFILE) - input
Handle from SplPdOpen.

SplPdWrite Parameter - pchData

pchData (PVOID) - input
Pointer to data buffer to write.

SplPdWrite Parameter - cbData

cbData (ULONG) - input
Length of data, in bytes.

SplPdWrite Parameter - pcbWritten

pcbWritten (ULONG) - input
Pointer to count of bytes actually written to the device.

This value can be updated even when an unsuccessful return code is given, if some of the data was sent to the printer.

SplPdWrite Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success **Note:** **pcbWritten* is updated with the number of bytes sent to the printer. If all bytes were not written, it should be considered an error.
6(ERROR_INVALID_HANDLE)	Invalid handle given.

28(ERROR_OUT_OF_PAPER)
The printer is out of paper.

29(ERROR_WRITE_FAULT)
Failure attempting to write to the device. Specific error alerts should be sent to the spooler to better define the problem.

87(ERROR_INVALID_PARAMETER)
Invalid buffer given.

SplPdWrite - Parameters

hDevice (HFILE) - input
Handle from SplPdOpen.

pchData (PVOID) - input
Pointer to data buffer to write.

cbData (ULONG) - input
Length of data, in bytes.

pcbWritten (ULONG) - input
Pointer to count of bytes actually written to the device.

This value can be updated even when an unsuccessful return code is given, if some of the data was sent to the printer.

rc (ULONG) - returns
Return codes.

0
Success

Note: **pcbWritten* is updated with the number of bytes sent to the printer. If all bytes were not written, it should be considered an error.

6(ERROR_INVALID_HANDLE)
Invalid handle given.

28(ERROR_OUT_OF_PAPER)
The printer is out of paper.

29(ERROR_WRITE_FAULT)
Failure attempting to write to the device. Specific error alerts should be sent to the spooler to better define the problem.

87(ERROR_INVALID_PARAMETER)
Invalid buffer given.

SplPdWrite - Remarks

None.

SplPdWrite - Topics

Select an item:
Syntax
Parameters

Returns
Remarks
Glossary

SplProtSendCmd

SplProtSendCmd - Syntax

Description:

SplProtSendCmd is an API exported by protocol converters. It is called by PrtQuery or PrtSet to convert generic commands into BIDI protocol-specific commands to send to the printer.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

PSZ       pszPortName;          /*  Port name. */
ULONG     ulType;               /*  Type of query and options. */
ULONG     ulCommand;            /*  Generic command code. */
PFN       pfnPdSendCmd;         /*  Port driver send-routine address. */
PFN       pfnBaseProtSendCmd;   /*  Base protocol converter address for ProtSendCmd routine. */
PVOID     pInData;              /*  May contain information required by PrtQuery and PrtSet. */
ULONG     cbInData;             /*  Length of information in pInData, in bytes. */
PVOID     pOutData;             /*  Return buffer. */
PULONG    pcbOutData;           /*  Points to the length of the output buffer. */
ULONG     rc;                   /*  Return codes. */

rc = SplProtSendCmd(pszPortName, ulType, ulCommand,
       pfnPdSendCmd, pfnBaseProtSendCmd, pInData,
       cbInData, pOutData, pcbOutData);
```

SplProtSendCmd Parameter - pszPortName

pszPortName (PSZ) - input
Port name.

Following is an example of a port name:

LPT1

This will be a locally defined port name.

SplProtSendCmd Parameter - ulType

ulType (ULONG) - input
Type of query and options.

Values are as follows:

TYPE_CACHE_ONLY(0)

Returns information from the protocol converter or port driver's cache only. Does not send anything to the printer.

If the requested information is not available, ERROR_INFO_NOT_AVAIL is returned. The caller should reissue the command with a different *ulType* value.

TYPE_SHORT_WAIT(1)

Returns information via a communications channel with the printer that can be used while a job is being sent to the printer.

An example would be using NPAP or SNMP. If the requested information cannot be returned using this type of channel (for instance, if a PJL command is required to get this information), ERROR_INFO_NOT_AVAIL is returned.

TYPE_LONG_WAIT(2)

Returns information using any necessary communications channel with the printer, even if the request has to wait for the current job being sent to the printer to complete.

If the printer supports only one command channel, and this channel can be used while a job is being sent to the printer, TYPE_SHORT_WAIT(1) and TYPE_LONG_WAIT(2) will behave in the same way.

TYPE_CACHE_UPDATE(3)

Returns information from the protocol converter or port driver's cache and also updates the cache in the background.

This is an optional type of query to support and, if it is not supported, will return ERROR_INVALID_FUNCTION.

If the requested information is not available in the cache, ERROR_INFO_NOT_AVAIL is returned. The caller should reissue the command with a different *ulType* value.

TYPE_WITHOUT_WRAPPER(4)

Returns or sends passthru or alert information without any packet wrapper around it.

This is useful for BIDI_READ_PASSTHRU and BIDI_SEND_PASSTHRU as it allows an application to send and receive printer interpreter language commands without having to know the format of the BIDI protocol's data wrappers. An example would be removing the NPAP wrapper before returning Postscript response data to an application.

If BIDI_READ_PASSTHRU is used without this value, the calling application requests the data as the printer returns it, possibly with any data wrappers.

If BIDI_SEND_PASSTHRU is used without this value, the calling application must have already wrapped the passthru data.

Note: If this value is used with any command other than BIDI_READ_PASSTHRU, ERROR_INVALID_PARAMETER is returned.

TYPE_CACHE_WAIT(5)

Returns information from the protocol converter or port driver's cache only. If the data is not available in the cache, use any mechanism available to get the information.

If TYPE_CACHE_WAIT(5) is not supported by the port driver or protocol converter, ERROR_INVALID_PARAMETER is returned.

The value of TYPE_CACHE_WAIT(5) may be used by the print object to issue BIDI_Q_STATUS, to get the current status of the printer without waiting to communicate with the printer.

SplProtSendCmd Parameter - ulCommand

ulCommand (ULONG) - input
Generic command code.

See BIDI Command Structures and Command Flow for contents of *pInData* and *pOutData* for each of the following commands.

Current commands are as follows:

BIDI_READ_PASSTHRU(8001h)
: Wait for any response from printer. This must be done from within a passthru session. See BIDI_START_PASSTHRU(1Ah) for more details. See BIDI_READ_PASSTHRU(8001h) for input and output parameters.

BIDI_Q_SW(800Ch)
: Query BIDI software capabilities. See BIDI_Q_SW(800Ch) for input and output parameters.

BIDI_Q_DEVICE(800Dh)
: Query printer device characteristics. See BIDI_Q_DEVICE(800Dh) for input and output parameters.

BIDI_Q_INTERPRETER(800Eh)
: Query interpreters available in the printer. See BIDI_Q_INTERPRETER(800Eh) for input and output parameters.

BIDI_Q_INPUTBINS(800Fh)
: Query input bins. See BIDI_Q_INPUTBINS(800Fh) for input and output parameters.

BIDI_Q_OUTPUTBINS(8010h)
: Query output bins. See BIDI_Q_OUTPUTBINS(8010h) for input and output parameters.

BIDI_Q_FONTS(8012h)
: Query fonts for a particular interpreter. See BIDI_Q_FONTS(8012h) for input and output parameters.

BIDI_Q_JOBS_COMPLETE(8013h)
: Query list of completed jobs. See BIDI_Q_JOBS_COMPLETE(8013h) for input and output parameters.

BIDI_Q_JOBS_QUEUED(8014h)
: Query list of queued jobs. See BIDI_Q_JOBS_QUEUED(8014h) for input and output parameters.

BIDI_Q_STATUS(8015h)
: Query current status of the print device. See BIDI_Q_STATUS(8015h) for input and output parameters.

BIDI_Q_JOBID(8017h)
: Query the printer Job ID currently being sent to this port driver. See BIDI_Q_JOBID(8017h) for input and output parameters.

BIDI_Q_RESPONSE_FMT(8018h)
: Return format of printer-to-host messages. See BIDI_Q_RESPONSE_FMT(8018h) for input and output parameters.

BIDI_READ_ALERT(801Dh)
: Read an extended alert. See BIDI_READ_ALERT(801Dh) for output parameter.

BIDI_Q_CONVERTER_INFO(8021h)
: Return protocol converter information. See BIDI_Q_CONVERTER_INFO(8021h) for output parameter.

BIDI_Q_STORAGE(8023h)
: Return printer storage media. See BIDI_Q_STORAGE(8023h) for input and output parameters.

0x00009000 - 0x00009FFF
: Protocol-converter-specific commands.

SplProtSendCmd Parameter - pfnPdSendCmd

pfnPdSendCmd (PFN) - input
Port driver send-routine address.

This is the address of a function in the port driver that will be called to send commands to the printer. Only the BIDI converters will call this function. SplProtSendCmd calls SplPdSendCmd, which sends protocol-specific commands to the printer.

SplProtSendCmd Parameter - pfnBaseProtSendCmd

pfnBaseProtSendCmd (PFN) - input
Base protocol converter address for ProtSendCmd routine.

This is NULL unless the protocol converter is an extension to the base protocol converter. If not NULL, the extension protocol converter can call the base converter at this address to let the base converter process the command.

SplProtSendCmd Parameter - pInData

pInData (PVOID) - input
May contain information required by PrtQuery and PrtSet.

SplProtSendCmd Parameter - cbInData

cbInData (ULONG) - input
Length of information in *pInData* , in bytes.

SplProtSendCmd Parameter - pOutData

pOutData (PVOID) - output
Return buffer.

If successful, this returns a structure as defined by PrtQuery and PrtSet. The format will be protocol-dependent.

SplProtSendCmd Parameter - pcbOutData

pcbOutData (PULONG) - output
Points to the length of the output buffer.

On entry, this is set to the length of the output buffer passed in. On exit, it is updated with the length of data available.

If ERROR_MORE_DATA(234) or NERR_BufTooSmall(2123) is set, this field has the buffer size required to get all available information.

SplProtSendCmd Return Value - rc

rc (ULONG) - returns
Return codes.

0 Success

1(ERROR_INVALID_FUNCTION)
The command code is not supported. This lack of support may occur if a new command is sent to an old spooler, port driver, or protocol converter, or if the printer rejects the command.

2(ERROR_FILE_NOT_FOUND)
The *pszPortNam*e given is not a valid port name.

5(ERROR_ACCESS_DENIED)
Not authorized to access printer port.

8(ERROR_NOT_ENOUGH_MEMORY)
Not enough memory to satisfy request.

29(ERROR_WRITE_FAULT)
Failure attempting to write to the device. The printer might not have received the command sequence.

87(ERROR_INVALID_PARAMETER)
An invalid buffer given.

117(ERROR_INVALID_CATEGORY)
The protocol converter does not handle the category given with command BIDI_ENABLE_ALERT.

186(ERROR_INVALID_FLAG_NUMBER)
The protocol converter does not understand one of the core alert types set with BIDI_ENABLE_ALERT.

211(ERROR_INFO_NOT_AVAIL)
The requested information is not available in the protocol converter cache, or the request cannot be sent with the given *ulType* value. The request should be reissued with a different *ulType* value.

234(ERROR_MORE_DATA)
Part of the information requested was returned, but there was not enough room for all the data. **pcbOutData* contains the size of buffer needed to retrieve all the query response data.

640(ERROR_TIMEOUT)
The printer response was not received within the time allotted. The caller may reissue the query, at which time the information might be available.

683(ERROR_BIDI_NO_SUPPORT)
The printer is not in bidirectional mode, therefore query requests are not honored. The port also may be in the process of being shut down.

2123(NERR_BufTooSmall)
No information was returned because the output buffer was too small. **pcbOutData* contains the size of buffer needed to retrieve all the query response data.

0x4005(PMERR_SPL_INV_HSPL)
Not currently in a passthru session.

0x4011(PMERR_SPL_INV_JOB_ID)
No job is being actively sent to the printer port, therefore BIDI_Q_JOBID is not successful.

0x4041(PMERR_SPL_SESSION_TERM)
The passthru session has been terminated.

0x403D(PMERR_SPL_CNV_NOT_INIT)
The protocol converter has not been initialized for this port.

0x403F(PMERR_SPL_TYPE_NOT_AVAIL)
The storage type requested for BIDI_Q_FONTS or BIDI_Q_STORAGE is not available in the printer.

SplProtSendCmd - Parameters

pszPortName (PSZ) - input
Port name.

Following is an example of a port name:

LPT1

This will be a locally defined port name.

ulType (ULONG) - input
Type of query and options.

Values are as follows:

TYPE_CACHE_ONLY(0)

Returns information from the protocol converter or port driver's cache only. Does not send anything to the printer.

If the requested information is not available, ERROR_INFO_NOT_AVAIL is returned. The caller should reissue the command with a different *ulType* value.

TYPE_SHORT_WAIT(1)

Returns information via a communications channel with the printer that can be used while a job is being sent to the printer.

An example would be using NPAP or SNMP. If the requested information cannot be returned using this type of channel (for instance, if a PJL command is required to get this information), ERROR_INFO_NOT_AVAIL is returned.

TYPE_LONG_WAIT(2)

Returns information using any necessary communications channel with the printer, even if the request has to wait for the current job being sent to the printer to complete.

If the printer supports only one command channel, and this channel can be used while a job is being sent to the printer, TYPE_SHORT_WAIT(1) and TYPE_LONG_WAIT(2) will behave in the same way.

TYPE_CACHE_UPDATE(3)

Returns information from the protocol converter or port driver's cache and also updates the cache in the background.

This is an optional type of query to support and, if it is not supported, will return ERROR_INVALID_FUNCTION.

If the requested information is not available in the cache, ERROR_INFO_NOT_AVAIL is returned. The caller should reissue the command with a different *ulType* value.

TYPE_WITHOUT_WRAPPER(4)

Returns or sends passthru or alert information without any packet wrapper around it.

This is useful for BIDI_READ_PASSTHRU and BIDI_SEND_PASSTHRU as it allows an application to send and receive printer interpreter language commands without having to know the format of the BIDI protocol's data wrappers. An example would be removing the NPAP wrapper before returning Postscript response data to an application.

If BIDI_READ_PASSTHRU is used without this value, the calling application requests the data as the printer returns it, possibly with any data wrappers.

If BIDI_SEND_PASSTHRU is used without this value, the calling application must have already wrapped the passthru data.

Note: If this value is used with any command other than BIDI_READ_PASSTHRU, ERROR_INVALID_PARAMETER is returned.

TYPE_CACHE_WAIT(5)

Returns information from the protocol converter or port driver's cache only. If the data is not available in the cache, use any mechanism available to get the information.

If TYPE_CACHE_WAIT(5) is not supported by the port driver or protocol converter, ERROR_INVALID_PARAMETER is returned.

The value of TYPE_CACHE_WAIT(5) may be used by the print object to issue BIDI_Q_STATUS, to get the current status of the printer without waiting to communicate with the printer.

ulCommand (ULONG) - input
Generic command code.

See BIDI Command Structures and Command Flow for contents of *pInData* and *pOutData* for each of the following commands.

Current commands are as follows:

BIDI_READ_PASSTHRU(8001h)
: Wait for any response from printer. This must be done from within a passthru session. See BIDI_START_PASSTHRU(1Ah) for more details. See BIDI_READ_PASSTHRU(8001h) for input and output parameters.

BIDI_Q_SW(800Ch)
: Query BIDI software capabilities. See BIDI_Q_SW(800Ch) for input and output parameters.

BIDI_Q_DEVICE(800Dh)
: Query printer device characteristics. See BIDI_Q_DEVICE(800Dh) for input and output parameters.

BIDI_Q_INTERPRETER(800Eh)
: Query interpreters available in the printer. See BIDI_Q_INTERPRETER(800Eh) for input and output parameters.

BIDI_Q_INPUTBINS(800Fh)
: Query input bins. See BIDI_Q_INPUTBINS(800Fh) for input and output parameters.

BIDI_Q_OUTPUTBINS(8010h)
: Query output bins. See BIDI_Q_OUTPUTBINS(8010h) for input and output parameters.

BIDI_Q_FONTS(8012h)
: Query fonts for a particular interpreter. See BIDI_Q_FONTS(8012h) for input and output parameters.

BIDI_Q_JOBS_COMPLETE(8013h)
: Query list of completed jobs. See BIDI_Q_JOBS_COMPLETE(8013h) for input and output parameters.

BIDI_Q_JOBS_QUEUED(8014h)
: Query list of queued jobs. See BIDI_Q_JOBS_QUEUED(8014h) for input and output parameters.

BIDI_Q_STATUS(8015h)
: Query current status of the print device. See BIDI_Q_STATUS(8015h) for input and output parameters.

BIDI_Q_JOBID(8017h)
: Query the printer Job ID currently being sent to this port driver. See BIDI_Q_JOBID(8017h) for input and output parameters.

BIDI_Q_RESPONSE_FMT(8018h)
: Return format of printer-to-host messages. See BIDI_Q_RESPONSE_FMT(8018h) for input and output parameters.

BIDI_READ_ALERT(801Dh)
: Read an extended alert. See BIDI_READ_ALERT(801Dh) for output parameter.

BIDI_Q_CONVERTER_INFO(8021h)
: Return protocol converter information. See BIDI_Q_CONVERTER_INFO(8021h) for output parameter.

BIDI_Q_STORAGE(8023h)
: Return printer storage media. See BIDI_Q_STORAGE(8023h) for input and output parameters.

0x00009000 - 0x00009FFF
: Protocol-converter-specific commands.

pfnPdSendCmd (PFN) - input
: Port driver send-routine address.

  This is the address of a function in the port driver that will be called to send commands to the printer. Only the BIDI converters will call this function. SplProtSendCmd calls SplPdSendCmd, which sends protocol-specific commands to the printer.

pfnBaseProtSendCmd (PFN) - input
: Base protocol converter address for ProtSendCmd routine.

  This is NULL unless the protocol converter is an extension to the base protocol converter. If not NULL, the extension protocol converter can call the base converter at this address to let the base converter process the command.

pInData (PVOID) - input
: May contain information required by PrtQuery and PrtSet.

cbInData (ULONG) - input
: Length of information in *pInData* , in bytes.

pOutData (PVOID) - output
: Return buffer.

If successful, this returns a structure as defined by PrtQuery and PrtSet. The format will be protocol-dependent.

pcbOutData (PULONG) - output
Points to the length of the output buffer.

On entry, this is set to the length of the output buffer passed in. On exit, it is updated with the length of data available.

If ERROR_MORE_DATA(234) or NERR_BufTooSmall(2123) is set, this field has the buffer size required to get all available information.

rc (ULONG) - returns
Return codes.

0	Success
1(ERROR_INVALID_FUNCTION)	The command code is not supported. This lack of support may occur if a new command is sent to an old spooler, port driver, or protocol converter, or if the printer rejects the command.
2(ERROR_FILE_NOT_FOUND)	The *pszPortName* given is not a valid port name.
5(ERROR_ACCESS_DENIED)	Not authorized to access printer port.
8(ERROR_NOT_ENOUGH_MEMORY)	Not enough memory to satisfy request.
29(ERROR_WRITE_FAULT)	Failure attempting to write to the device. The printer might not have received the command sequence.
87(ERROR_INVALID_PARAMETER)	An invalid buffer given.
117(ERROR_INVALID_CATEGORY)	The protocol converter does not handle the category given with command BIDI_ENABLE_ALERT.
186(ERROR_INVALID_FLAG_NUMBER)	The protocol converter does not understand one of the core alert types set with BIDI_ENABLE_ALERT.
211(ERROR_INFO_NOT_AVAIL)	The requested information is not available in the protocol converter cache, or the request cannot be sent with the given *ulType* value. The request should be reissued with a different *ulType* value.
234(ERROR_MORE_DATA)	Part of the information requested was returned, but there was not enough room for all the data. **pcbOutData* contains the size of buffer needed to retrieve all the query response data.
640(ERROR_TIMEOUT)	The printer response was not received within the time allotted. The caller may reissue the query, at which time the information might be available.
683(ERROR_BIDI_NO_SUPPORT)	The printer is not in bidirectional mode, therefore query requests are not honored. The port also may be in the process of being shut down.
2123(NERR_BufTooSmall)	No information was returned because the output buffer was too small. **pcbOutData* contains the size of buffer needed to retrieve all the query response data.
0x4005(PMERR_SPL_INV_HSPL)	Not currently in a passthru session.
0x4011(PMERR_SPL_INV_JOB_ID)	No job is being actively sent to the printer port, therefore BIDI_Q_JOBID is not successful.
0x4041(PMERR_SPL_SESSION_TERM)	The passthru session has been terminated.
0x403D(PMERR_SPL_CNV_NOT_INIT)	The protocol converter has not been initialized for this port.
0x403F(PMERR_SPL_TYPE_NOT_AVAIL)	The storage type requested for BIDI_Q_FONTS or BIDI_Q_STORAGE is not available in the printer.

SplProtSendCmd - Remarks

The protocol converter will call pfnPdSendCmd to send any commands to the printer. The protocol converter will hold the caller of SplProtSendCmd until the requested information is returned by the printer.

SplProtSendCmd - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplProtWrite

SplProtWrite - Syntax

Description:

SplProtWrite is an optional API and is exported by protocol converters. It is called by PrtWrite to put a BIDI protocol wrapper around print data and then send the data to the printer.

SplProtWrite is responsible for breaking write packets into sub-packets to accommodate packet size restrictions from the printer.

If the BIDI software protocol being used requires print job data to have a wrapper around it (see BIDI_Q_SW(800Ch) in BIDI Command Structures and Command Flow), the spooler calls SplProtWrite, the protocol converter's API. SplProtWrite then calls the port driver's SplPdWrite routine with an updated buffer.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

HFILE     hDevice;           /*  Handle from SplPdOpen to pass to SplPdWrite. */
PSZ       pszDeviceName;     /*  Device name. */
PFN       pfnPdWrite;        /*  Address of port driver function SplPdWrite. */
PFN       pfnBaseProtWrite;  /*  Base protocol converter address for ProtWrite routine. */
PVOID     pchData;           /*  Pointer to the data buffer to write. */
ULONG     cbInData;          /*  Length of information in pchData, in bytes. */
PULONG    pcbWritten;        /*  Pointer to count of bytes actually written to the device. */
ULONG     rc;                /*  Return codes. */

rc = SplProtWrite(hDevice, pszDeviceName,
       pfnPdWrite, pfnBaseProtWrite, pchData,
       cbInData, pcbWritten);
```

SplProtWrite Parameter - hDevice

hDevice (HFILE) - input
Handle from SplPdOpen to pass to SplPdWrite.

SplProtWrite Parameter - pszDeviceName

pszDeviceName (PSZ) - input
Device name.

Port name being written to.

SplProtWrite Parameter - pfnPdWrite

pfnPdWrite (PFN) - input
Address of port driver function SplPdWrite.

SplProtWrite Parameter - pfnBaseProtWrite

pfnBaseProtWrite (PFN) - input
Base protocol converter address for ProtWrite routine.

This is NULL unless the protocol converter is an extension to the base protocol converter. If not NULL, the extension protocol converter can call the base converter at this address to let the base converter process the command.

SplProtWrite Parameter - pchData

pchData (PVOID) - input
Pointer to the data buffer to write.

SplProtWrite Parameter - cbInData

cbInData (ULONG) - input
Length of information in *pchData* , in bytes.

SplProtWrite Parameter - pcbWritten

pcbWritten (PULONG) - output
Pointer to count of bytes actually written to the device.

This value can be updated even when an unsuccessful return code is given, if some of the data was sent to the printer.

SplProtWrite Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success **Note:** **pcbWritten* is updated with the number of bytes sent to the printer. If all bytes were not written, it should be considered an error.
6(ERROR_INVALID_HANDLE)	Invalid handle given.
28(ERROR_OUT_OF_PAPER)	The printer is out of paper.
29(ERROR_WRITE_FAULT)	Failure attempting to write to the device.
32(ERROR_SHARING_VIOLATION)	Another thread has access to this port using this handle. This situation can occur if another thread is in the process of a PrtWrite using this Prt handle.
87(ERROR_INVALID_PARAMETER)	An invalid buffer given.

SplProtWrite - Parameters

hDevice (HFILE) - input
Handle from SplPdOpen to pass to SplPdWrite.

pszDeviceName (PSZ) - input
Device name.

Port name being written to.

pfnPdWrite (PFN) - input
Address of port driver function SplPdWrite.

pfnBaseProtWrite (PFN) - input
Base protocol converter address for ProtWrite routine.

This is NULL unless the protocol converter is an extension to the base protocol converter. If not NULL, the extension protocol converter can call the base converter at this address to let the base converter process the command.

pchData (PVOID) - input
Pointer to the data buffer to write.

cbInData (ULONG) - input
Length of information in *pchData* , in bytes.

pcbWritten (PULONG) - output
Pointer to count of bytes actually written to the device.

This value can be updated even when an unsuccessful return code is given, if some of the data was sent to the printer.

rc (ULONG) - returns
Return codes.

0	Success **Note:** **pcbWritten* is updated with the number of bytes sent to the printer. If all bytes were not written, it should be considered an error.
6(ERROR_INVALID_HANDLE)	Invalid handle given.
28(ERROR_OUT_OF_PAPER)	The printer is out of paper.
29(ERROR_WRITE_FAULT)	Failure attempting to write to the device.
32(ERROR_SHARING_VIOLATION)	Another thread has access to this port using this handle. This situation can occur if another thread is in the process of a PrtWrite using this Prt handle.
87(ERROR_INVALID_PARAMETER)	An invalid buffer given.

SplProtWrite - Remarks

None.

SplProtWrite - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplProtXlateCmd

SplProtXlateCmd - Syntax

Description:

SplProtXlateCmd is an API exported by protocol converters. It is called only by port drivers when processing the BIDI_WAIT_ALERT(8016h) command in the port driver SplPdQuery API. The purpose is to convert a protocol-specific message received from the printer into a generic command structure that can be used by the port driver and spooler.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

PSZ       pszPortName;          /*  Port name. */
PFN       pfnBaseProtXlateCmd;  /*  Base protocol converter address for ProtXlateCmd routine. */
PVOID     pInData;              /*  Command sequence for printer. */
ULONG     cbInData;             /*  Length of information in pInData, in bytes. */
PVOID     pAlertInfo;           /*  Alert information buffer. */
PVOID     pOutData;             /*  Return buffer. */
PULONG    pcbOutData;           /*  Points to the length of the output buffer. */
ULONG     rc;                   /*  Return codes. */

rc = SplProtXlateCmd(pszPortName, pfnBaseProtXlateCmd,
       pInData, cbInData, pAlertInfo, pOutData,
       pcbOutData);
```

SplProtXlateCmd Parameter - pszPortName

pszPortName (PSZ) - input
Port name.

Following is an example of a port name:

LPT1

This will be a locally defined port name.

SplProtXlateCmd Parameter - pfnBaseProtXlateCmd

pfnBaseProtXlateCmd (PFN) - input
Base protocol converter address for ProtXlateCmd routine.

This is NULL unless the protocol converter is an extension to the base protocol converter. If not NULL, the extension protocol converter can call the base converter at this address to let the base converter process the command.

SplProtXlateCmd Parameter - pInData

pInData (PVOID) - input
Command sequence for printer.

On entry, this field contains the BIDI software protocol-specific command sequence received from the printer.

SplProtXlateCmd Parameter - cbInData

cbInData (ULONG) - input
Length of information in *pInData* , in bytes.

SplProtXlateCmd Parameter - pAlertInfo

pAlertInfo (PVOID) - in/out
Alert information buffer.

This field receives a PDALERTINFO data structure. See Data Types for details on this data structure.

SplProtXlateCmd Parameter - pOutData

pOutData (PVOID) - output
Return buffer.

If PD_ALERT is set, this returns a PRTALERT structure. See BIDI_WAIT_ALERT(8016h) in BIDI Command Structures and Command Flow for more information and Data Types for details on this data structure.

If PD_ALERT is not set, nothing is returned in this buffer.

SplProtXlateCmd Parameter - pcbOutData

pcbOutData (PULONG) - output
Points to the length of the output buffer.

On entry, this is set to the length of the output buffer passed in. On exit, it is updated with the length of the PRTALERT structure data copied into *pOutData* .

SplProtXlateCmd Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
8(ERROR_NOT_ENOUGH_MEMORY)	Not enough memory to satisfy request.
87(ERROR_INVALID_PARAMETER)	An invalid parameter was specified; most likely an invalid buffer pointer was given.
211(ERROR_INFO_NOT_AVAIL)	The caller set PD_NEXTALERT. However, the protocol converter does not have any more alerts to return for the previous data passed to SplProtXlateCmd for the port.
234(ERROR_MORE_DATA)	

Part of the information requested was returned, but there was not enough room for all the data. **pcbOutData* contains the size of buffer needed to retrieve all the query response data.

2123(NERR_BufTooSmall)
No information was returned because the output buffer was too small. **pcbOutData* contains the size of buffer needed to retrieve all the query response data.

SplProtXlateCmd - Parameters

pszPortName (PSZ) - input
Port name.

Following is an example of a port name:

LPT1

This will be a locally defined port name.

pfnBaseProtXlateCmd (PFN) - input
Base protocol converter address for ProtXlateCmd routine.

This is NULL unless the protocol converter is an extension to the base protocol converter. If not NULL, the extension protocol converter can call the base converter at this address to let the base converter process the command.

pInData (PVOID) - input
Command sequence for printer.

On entry, this field contains the BIDI software protocol-specific command sequence received from the printer.

cbInData (ULONG) - input
Length of information in *pInData* , in bytes.

pAlertInfo (PVOID) - in/out
Alert information buffer.

This field receives a PDALERTINFO data structure. See Data Types for details on this data structure.

pOutData (PVOID) - output
Return buffer.

If PD_ALERT is set, this returns a PRTALERT structure. See BIDI_WAIT_ALERT(8016h) in BIDI Command Structures and Command Flow for more information and Data Types for details on this data structure.

If PD_ALERT is not set, nothing is returned in this buffer.

pcbOutData (PULONG) - output
Points to the length of the output buffer.

On entry, this is set to the length of the output buffer passed in. On exit, it is updated with the length of the PRTALERT structure data copied into *pOutData* .

rc (ULONG) - returns
Return codes.

0 Success

8(ERROR_NOT_ENOUGH_MEMORY)
Not enough memory to satisfy request.

87(ERROR_INVALID_PARAMETER)
An invalid parameter was specified; most likely an invalid buffer pointer was given.

211(ERROR_INFO_NOT_AVAIL)
The caller set PD_NEXTALERT. However, the protocol converter does not have any more alerts to return for the previous data passed to SplProtXlateCmd for the port.

234(ERROR_MORE_DATA)
Part of the information requested was returned, but there was not enough room for all the data.

pcbOutData contains the size of buffer needed to retrieve all the query response data.

2123(NERR_BufTooSmall)

No information was returned because the output buffer was too small. *pcbOutData* contains the size of buffer needed to retrieve all the query response data.

SplProtXlateCmd - Remarks

None.

SplProtXlateCmd - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplQmAbort

SplQmAbort - Syntax

Description:

SplQmAbort stops the generation of the spool file. It automatically closes the spool file (see SplQmClose).

```
#define INCL_SPL /* Or use INCL_PM, */
#include <os2.h>

HSPL    hspl;  /*  Spooler handle. */
BOOL    rc;    /*  Success indicator. */

rc = SplQmAbort(hspl);
```

SplQmAbort Parameter - hspl

hspl (HSPL) - input
Spooler handle.

SplQmAbort Return Value - rc

rc (BOOL) - returns
Success indicator.

TRUE
Successful completion
FALSE
Error occurred.

SplQmAbort - Parameters

hspl (HSPL) - input
Spooler handle.

rc (BOOL) - returns
Success indicator.

TRUE
Successful completion
FALSE
Error occurred.

SplQmAbort - Error Condition List

Possible returns from WinGetLastError

PMERR_SPL_QUEUE_ERROR (0x4004)
No spooler queue supplied or found.

PMERR_SPL_INV_HSPL (0x4005)
The spooler handle is invalid.

SplQmAbort - Related Functions

Prerequisite Functions

- SplQmOpen

Related Functions

- DevEscape

SplQmAbort - Remarks

This function is used to stop the generation of spool files and automatically close the spool file.

```
#define INCL_SPL
#include <OS2.H>

HSPL hspl; /* spooler handle. */

SplQmAbort(hspl);
```

SplQmAbort - Topics

Select an item:
Syntax
Parameters
Returns
Error Condition List
Remarks
Related Functions
Glossary

SplQmAbortDoc

SplQmAbortDoc - Syntax

Description:

SplQmAbortDoc ends a print job.

```
#define INCL_SPL /* Or use INCL_PM, */
#include <os2.h>

HSPL    hspl;  /*  Spooler handle. */
BOOL    rc;    /*  Success indicator. */

rc = SplQmAbortDoc(hspl);
```

SplQmAbortDoc Parameter - hspl

hspl (HSPL) - input
Spooler handle.

SplQmAbortDoc Return Value - rc

rc (BOOL) - returns
Success indicator.

TRUE
Successful completion
FALSE
Error occurred.

SplQmAbortDoc - Parameters

hspl (HSPL) - input
Spooler handle.

rc (BOOL) - returns
Success indicator.

TRUE
Successful completion
FALSE
Error occurred.

SplQmAbortDoc - Error Condition List

Possible returns from WinGetLastError

PMERR_SPL_QUEUE_ERROR (0x4004)
No spooler queue supplied or found.

PMERR_STARTDOC_NOT_ISSUED (0x2104)
A request to write spooled output without first issuing a STARTDOC was attempted.

PMERR_SPL_INV_HSPL (0x4005)
The spooler handle is invalid.

SplQmAbortDoc - Related Functions

Prerequisite Functions

- SplQmOpen
- SplQmStartDoc

Related Functions

- DevEscape

SplQmAbortDoc - Remarks

This function is used to end a print job. Everything that has been written to the spool file for this job since the last SplQmStartDoc is erased, including the SplQmStartDoc.

```
#define INCL_SPL
#include <OS2.H>

HSPL hspl; /* spooler handle. */

SplQmAbortDoc(hspl);
```

SplQmAbortDoc - Topics

Select an item:
Syntax
Parameters
Returns
Error Condition List
Remarks
Related Functions
Glossary

SplQmClose

SplQmClose - Syntax

Description:

SplQmClose closes the spool file. It corresponds to the DevCloseDC function.

```
#define INCL_SPL /* Or use INCL_PM, */
#include <os2.h>

HSPL    hspl;  /*  Spooler handle. */
BOOL    rc;    /*  Success indicator. */

rc = SplQmClose(hspl);
```

SplQmClose Parameter - hspl

hspl (HSPL) - input

Spooler handle.

SplQmClose Return Value - rc

rc (BOOL) - returns
Success indicator.

TRUE
Successful completion
FALSE
Error occurred.

SplQmClose - Parameters

hspl (HSPL) - input
Spooler handle.

rc (BOOL) - returns
Success indicator.

TRUE
Successful completion
FALSE
Error occurred.

SplQmClose - Error Condition List

Possible returns from WinGetLastError

PMERR_SPL_QUEUE_ERROR (0x4004)
No spooler queue supplied or found.

PMERR_ENDDOC_NOT_ISSUED (0x210B)
A request to close the spooled output without first issuing a an ENDDOC was attempted.

PMERR_SPL_INV_HSPL (0x4005)
The spooler handle is invalid.

SplQmClose - Related Functions

Prerequisite Functions

- SplQmOpen

Related Functions

- DevCloseDC

SplQmClose - Remarks

This function is used to close a spool file that was opened with SplQmOpen.

```
#define INCL_SPL
#include <OS2.H>

HSPL hspl; /* spooler handle. */

SplQmClose(hspl);
```

SplQmClose - Topics

Select an item:
Syntax
Parameters
Returns
Error Condition List
Remarks
Related Functions
Glossary

SplQmEndDoc

SplQmEndDoc - Syntax

Description:

SplQmEndDoc ends a print job, and returns ulJob, a unique number to identify the job. This function corresponds to the DevEscape (DEVESC_ENDDOC) call.

```
#define INCL_SPL /* Or use INCL_PM, */
#include <os2.h>

HSPL     hspl;   /*  Spooler handle. */
ULONG    ulJob;  /*  Job identifier. */

ulJob = SplQmEndDoc(hspl);
```

SplQmEndDoc Parameter - hspl

hspl (HSPL) - input
Spooler handle.

SplQmEndDoc Return Value - ulJob

ulJob (ULONG) - returns
Job identifier.

Nonzero
Jobid (1 through 65,535)
SPL_ERROR
Error

SplQmEndDoc - Parameters

hspl (HSPL) - input
Spooler handle.

ulJob (ULONG) - returns
Job identifier.

Nonzero
Jobid (1 through 65,535)
SPL_ERROR
Error

SplQmEndDoc - Error Condition List

Possible returns from WinGetLastError

PMERR_SPL_QUEUE_ERROR (0x4004)
No spooler queue supplied or found.

PMERR_SPL_NO_DATA (0x400D)
No data supplied or found.

PMERR_SPL_INV_HSPL (0x4005)
The spooler handle is invalid.

SplQmEndDoc - Related Functions

Prerequisite Functions

- SplQmOpen
- SplQmStartDoc

Related Functions

- DevEscape

SplQmEndDoc - Remarks

This function is used to end a print job and return the job id. The print-job identifier is displayed to the user by the spooler while this job is on the queue, and while it is being printed.

```
#define INCL_SPL
#include <OS2.H>

HSPL hspl;  /* spooler handle. */
ULONG jobid;
CHAR szMsg[100];
HWND hwndClient;

jobid = SplQmEndDoc(hspl);

sprintf(szMsg, "ending job %d",jobid);
WinMessageBox(HWND_DESKTOP,
    hwndClient,               /* client-window handle  */
    szMsg,                    /* body of the message   */
    "Printing Information",   /* title of the message */
    0,                        /* message box id        */
    MB_NOICON | MB_OK);       /* icon and button flags */
```

SplQmEndDoc - Topics

Select an item:
Syntax
Parameters
Returns
Error Condition List
Remarks
Related Functions
Glossary

SplQmGetJobID

SplQmGetJobID - Syntax

Description:

This function returns the spooler print job ID for the spool file handle.

This function allows Presentation drivers (or applications) to get the spooler print job ID while still spooling the job. Currently, only SplQmEndDoc will return the spooler print job ID.

SplQmGetJobID will return an error if called after SplQmEndDoc and before the next SplQmStartDoc because the spool file handle does not

reference a spool job after SplQmEndDoc.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

HSPL      hspl;       /*  Spool file handle returned by SplQmOpen. */
ULONG     ulLevel;    /*  Level of job information; currently, level 0 is supported. */
PVOID     pBuf;       /*  Buffer to receive job information. */
ULONG     cbBuf;      /*  Length of pBuf, in bytes. */
PULONG    pcbNeeded;  /*  Receives length, in bytes, of buffer needed to store job information. */
ULONG     rc;         /*  Return codes. */

rc = SplQmGetJobID(hspl, ulLevel, pBuf, cbBuf,
       pcbNeeded);
```

SplQmGetJobID Parameter - hspl

hspl (HSPL) - input
Spool file handle returned by SplQmOpen.

SplQmGetJobID Parameter - ulLevel

ulLevel (ULONG) - input
Level of job information; currently, level 0 is supported.

SplQmGetJobID Parameter - pBuf

pBuf (PVOID) - output
Buffer to receive job information.

Level 0 will return the QMJOBINFO data structure. See QMJOBINFO in Data Types for details on this data structure.

SplQmGetJobID Parameter - cbBuf

cbBuf (ULONG) - input
Length of *pBuf,* in bytes.

SplQmGetJobID Parameter - pcbNeeded

pcbNeeded (PULONG) - output
Receives length, in bytes, of buffer needed to store job information.

SplQmGetJobID Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
ERROR_INVALID_HANDLE(6)	Invalid handle given.
ERROR_INVALID_PARAMETER(87)	*ulLevel* not 0 or invalid parameter given.
ERROR_MORE_DATA(234)	Part of the information requested was returned, but there was not enough room for all the data. The **pcbOutData* parameter contains the size of buffer needed to retrieve all the query response data.
ERROR_NOT_SUPPORTED(50)	This function is not supported by the print server processing the spool job.
NERR_BufTooSmall(2123)	No information was returned because the output buffer was too small. **pcbOutData* contains the size of buffer needed to retrieve all the query response data.
NERR_JobNotFound(2151)	No spooler job found.

SplQmGetJobID - Parameters

hspl (HSPL) - input
Spool file handle returned by SplQmOpen.

ulLevel (ULONG) - input
Level of job information; currently, level 0 is supported.

pBuf (PVOID) - output
Buffer to receive job information.

Level 0 will return the QMJOBINFO data structure. See QMJOBINFO in Data Types for details on this data structure.

cbBuf (ULONG) - input
Length of *pBuf,* in bytes.

pcbNeeded (PULONG) - output
Receives length, in bytes, of buffer needed to store job information.

rc (ULONG) - returns
Return codes.

0	Success
ERROR_INVALID_HANDLE(6)	Invalid handle given.
ERROR_INVALID_PARAMETER(87)	*ulLevel* not 0 or invalid parameter given.
ERROR_MORE_DATA(234)	Part of the information requested was returned, but there was not enough room for all the data. The **pcbOutData* parameter contains the size of buffer needed to retrieve all the query response data.
ERROR_NOT_SUPPORTED(50)	This function is not supported by the print server processing the spool job.
NERR_BufTooSmall(2123)	

NERR_JobNotFound(2151)	No information was returned because the output buffer was too small. **pcbOutData* contains the size of buffer needed to retrieve all the query response data. No spooler job found.

SplQmGetJobID - Remarks

None.

SplQmGetJobID - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplQmNewPage

SplQmNewPage - Syntax

Description:

This function notifies the spooler of a page boundary when spooling a print job. It is called by the Presentation driver just prior to the first SplQmWrite for a page, including before the first page of the print job being spooled.

SplQmNewPage allows prespool approximation of the length of print jobs and may be used to alter job scheduling. The function may be called once, when the entire metafile is spooled, with the total number of pages for the print job.

This call also sets *ulPagesSpooled* in PRJINFO4 for the print job. See PRJINFO4 in Data Types for details on this data structure.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

HSPL     hspl;          /*  Spool file handle returned by SplQmOpen. */
ULONG    ulPageNumber;  /*  Page number of next SplQmWrite. */
ULONG    rc;            /*  Return codes. */

rc = SplQmNewPage(hspl, ulPageNumber);
```

SplQmNewPage Parameter - hspl

hspl (HSPL) - input
Spool file handle returned by SplQmOpen.

SplQmNewPage Parameter - ulPageNumber

ulPageNumber (ULONG) - input
Page number of next SplQmWrite.

The first page number would be 1 (one).

SplQmNewPage Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
ERROR_INVALID_HANDLE(6)	Invalid handle given.

SplQmNewPage - Parameters

hspl (HSPL) - input
Spool file handle returned by SplQmOpen.

ulPageNumber (ULONG) - input
Page number of next SplQmWrite.

The first page number would be 1 (one).

rc (ULONG) - returns
Return codes.

0	Success
ERROR_INVALID_HANDLE(6)	Invalid handle given.

SplQmNewPage - Remarks

None.

SplQmNewPage - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplQmOpen

SplQmOpen - Syntax

Description:

SplQmOpen opens a spool file for generating a print job. It corresponds to the DevOpenDC call.

```
#define INCL_SPL /* Or use INCL_PM, */
#include <os2.h>

PSZ             pszToken;     /*  Reserved value. */
LONG            lCount;       /*  Number of items. */
PQMOPENDATA     pqmdopData;   /*  Pointer to DevOpenStruc. */
HSPL            hspl;         /*  Spooler handle. */

hspl = SplQmOpen(pszToken, lCount, pqmdopData);
```

SplQmOpen Parameter - pszToken

pszToken (PSZ) - input
Reserved value.

SplQmOpen Parameter - lCount

lCount (LONG) - input
Number of items.

This is the number of items present in the *pqmdopData* supplied. This can be shorter than the full list, if omitted items are irrelevant.

This parameter must be greater than 0.

SplQmOpen Parameter - pqmdopData

pqmdopData (PQMOPENDATA) - input
Pointer to DevOpenStruc.

SplQmOpen Return Value - hspl

hspl (HSPL) - returns
Spooler handle.

Nonzero
Spooler handle
SPL_ERROR
Error

SplQmOpen - Parameters

pszToken (PSZ) - input
Reserved value.

lCount (LONG) - input
Number of items.

This is the number of items present in the *pqmdopData* supplied. This can be shorter than the full list, if omitted items are irrelevant.

This parameter must be greater than 0.

pqmdopData (PQMOPENDATA) - input
Pointer to DevOpenStruc.

hspl (HSPL) - returns
Spooler handle.

Nonzero
Spooler handle
SPL_ERROR
Error

SplQmOpen - Error Condition List

Possible returns from WinGetLastError

PMERR_INVALID_PARM (0x1303)
A parameter to the function contained invalid data.

PMERR_SPL_INV_LENGTH_OR_COUNT (0x401A)
The length or count is invalid.

PMERR_SPL_QUEUE_NOT_FOUND (0x4024)
The spooler queue definition could not be found.

PMERR_BASE_ERROR (0x2006)

An OS/2 base error has occurred. The base error code can be accessed using the OffBinaryData field of the ERRINFO structure returned by WinGetErrorInfo.

SplQmOpen - Related Functions

- DevOpenDC

SplQmOpen - Remarks

This sample code will initialize a PDEVOPENSTRUC and use it to call the function.

```
#define INCL_SPL
#define INCL_SPLDOSPRINT
#define INCL_BASE
#define INCL_ERRORS

#include <os2.h>
#include <stdio.h>
#include <stdlib.h>

VOID main()
{
  HSPL hspl;
  PDEVOPENSTRUC pdata;         /* Pointer to a DEVOPENSTRUC structure */
  PSZ  pszToken = "*";         /* Spooler info identifier             */

  /* Allocate memory for pdata */
  if ( !DosAllocMem( &pdata,sizeof( DEVOPENSTRUC ),
     (PAG_READ|PAG_WRITE|PAG_COMMIT ) )
  {
    /* Initialize elements of pdata */
    pdata->pszLogAddress      = "LPT1Q1";
    pdata->pszDriverName      = "IBMNULL";
    pdata->pdriv              = NULL;
    pdata->pszDataType        = "PM_Q_STD";
    pdata->pszComment         = NULL;
    pdata->pszQueueProcName   = NULL;
    pdata->pszQueueProcParams = NULL;
    pdata->pszSpoolerParams   = NULL;
    pdata->pszNetworkParams   = NULL;

    hspl = SplQmOpen( pszToken,4L,( PQMOPENDATA )pdata );

    if ( hspl != SPL_ERROR )        /* Good spooler handle */
    {
       printf("SplQmOpen handle is %d\n",hspl);
    }
    else
    {
       printf("SplQmOpen failed.\n");
    }
  }
}
```

SplQmOpen - Topics

Select an item:

Syntax
Parameters
Returns
Error Condition List
Remarks
Related Functions
Glossary

SplQmStartDoc

SplQmStartDoc - Syntax

Description:

SplQmStartDoc starts a print job. It corresponds to the DevEscape (DEVESC_STARTDOC) call.

```
#define INCL_SPL /* Or use INCL_PM, */
#include <os2.h>

HSPL    hspl;         /*  Spooler handle. */
PSZ     pszDocName;   /*  Document name. */
BOOL    rc;           /*  Success indicator. */

rc = SplQmStartDoc(hspl, pszDocName);
```

SplQmStartDoc Parameter - hspl

hspl (HSPL) - input
Spooler handle.

SplQmStartDoc Parameter - pszDocName

pszDocName (PSZ) - input
Document name.

This is part of the job description that is displayed to the end user by the spooler.

SplQmStartDoc Return Value - rc

rc (BOOL) - returns
Success indicator.

TRUE
Successful completion
FALSE
Error occurred.

SplQmStartDoc - Parameters

hspl (HSPL) - input
Spooler handle.

pszDocName (PSZ) - input
Document name.

This is part of the job description that is displayed to the end user by the spooler.

rc (BOOL) - returns
Success indicator.

TRUE
Successful completion
FALSE
Error occurred.

SplQmStartDoc - Error Condition List

Possible returns from WinGetLastError

PMERR_INVALID_PARM (0x1303)
A parameter to the function contained invalid data.

PMERR_SPL_QUEUE_ERROR (0x4004)
No spooler queue supplied or found.

PMERR_ENDDOC_NOT_ISSUED (0x210B)
A request to close the spooled output without first issuing a an ENDDOC was attempted.

PMERR_SPL_INV_HSPL (0x4005)
The spooler handle is invalid.

SplQmStartDoc - Related Functions

Prerequisite Functions

- SplQmOpen

Related Functions

- DevEscape

SplQmStartDoc - Remarks

This function indicates the start of a print job. It allows the application to specify a document name to be associated with the print job.

Multiple print jobs can be generated, within a single queue manager open, by bracketing each job with SplQmStartDoc and SplQmEndDoc.

```
#define INCL_SPL
#include <OS2.H>

HSPL hspl;  /* spooler handle. */
CHAR szDocName[] = "Test Job";
CHAR szMsg[100];
HWND hwndClient;

sprintf(szMsg, "Starting job named: %s",szDocName);
WinMessageBox(HWND_DESKTOP,
    hwndClient,                /* client-window handle  */
    szMsg,                     /* body of the message   */
    "Printing Information",    /* title of the message */
    0,                         /* message box id        */
    MB_NOICON | MB_OK);  /* icon and button flags */

SplQmStartDoc(hspl,szDocName);
```

SplQmStartDoc - Topics

Select an item:
Syntax
Parameters
Returns
Error Condition List
Remarks
Related Functions
Glossary

SplQmWrite

SplQmWrite - Syntax

Description:

*SplQmWrit*e writes a buffer of data to the spool file for the print job.

```
#define INCL_SPL /* Or use INCL_PM, */
#include <os2.h>

HSPL     hspl;     /*  Spooler handle. */
LONG     lCount;   /*  Length in bytes. */
PVOID    pData;    /*  Buffer of data to be written to the spool file. */
BOOL     rc;       /*  Success indicator. */

rc = SplQmWrite(hspl, lCount, pData);
```

SplQmWrite Parameter - hspl

hspl (HSPL) - input
Spooler handle.

SplQmWrite Parameter - lCount

lCount (LONG) - input
Length in bytes.

This is the length of *pData* ; it must not be less than 1 or greater than 65 535. Data that is longer than this must be written by two or more calls.

SplQmWrite Parameter - pData

pData (PVOID) - input
Buffer of data to be written to the spool file.

SplQmWrite Return Value - rc

rc (BOOL) - returns
Success indicator.

TRUE	Successful completion
FALSE	Error occurred.

SplQmWrite - Parameters

hspl (HSPL) - input
Spooler handle.

lCount (LONG) - input
Length in bytes.

This is the length of *pData* ; it must not be less than 1 or greater than 65 535. Data that is longer than this must be written by two or more calls.

pData (PVOID) - input
Buffer of data to be written to the spool file.

rc (BOOL) - returns
Success indicator.

TRUE
Successful completion
FALSE
Error occurred.

SplQmWrite - Error Condition List

Possible returns from WinGetLastError

PMERR_INVALID_PARM (0x1303)
A parameter to the function contained invalid data.

PMERR_BASE_ERROR (0x2006)
An OS/2 base error has occurred. The base error code can be accessed using the OffBinaryData field of the ERRINFO structure returned by WinGetErrorInfo.

PMERR_SPL_INV_LENGTH_OR_COUNT (0x401A)
The length or count is invalid.

PMERR_SPL_QUEUE_ERROR (0x4004)
No spooler queue supplied or found.

PMERR_SPL_PRINT_ABORT (0x4008)
The job has already been aborted.

PMERR_STARTDOC_NOT_ISSUED (0x2104)
A request to write spooled output without first issuing a STARTDOC was attempted.

PMERR_SPL_CANNOT_OPEN_FILE (0x401C)
Unable to open the file.

PMERR_SPL_INV_HSPL (0x4005)
The spooler handle is invalid.

PMERR_SPL_NO_DISK_SPACE (0x4006)
There is not enough free disk space.

SplQmWrite - Related Functions

Prerequisite Functions

- SplQmOpen
- SplQmStartDoc

SplQmWrite - Remarks

This function writes a buffer of data to the spool file for the print job. The size of the data buffer must not be greater than 64KB. Print jobs that exceed the maximum buffer size must be written by repeatedly calling to this function.

```
#define INCL_SPL
```

```
#include <OS2.H>

HSPL hspl; /* spooler handle. */

SplQmWrite(hspl,
           strlen("DATA"),
           (PVOID)"DATA");
```

SplQmWrite - Topics

Select an item:
Syntax
Parameters
Returns
Error Condition List
Remarks
Related Functions
Glossary

SplQpClose

SplQpClose - Syntax

Description:

SplQpClose closes the queue drive (queue processor).

```
#include <os2.h>

HPROC    hproc;     /*  The queue processor handle. */
BOOL     fSuccess;  /*  Return codes. */

fSuccess = SplQpClose(hproc);
```

SplQpClose Parameter - hproc

hproc (HPROC) - input
The queue processor handle.

SplQpClose Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Success
FALSE	Error

SplQpClose - Parameters

hproc (HPROC) - input
The queue processor handle.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Success
FALSE	Error

SplQpClose - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

SplQpControl

SplQpControl - Syntax

Description:

SplQpControl controls the printing of a document and is usually called while another thread is inside SplQpPrint.

```
#include <os2.h>

HPROC     hproc;      /*  The queue processor handle. */
LONG      cmdCode;    /*  Printer control codes. */
BOOL      fSuccess;   /*  Return codes. */

fSuccess = SplQpControl(hproc, cmdCode);
```

SplQpControl Parameter - hproc

hproc (HPROC) - input
The queue processor handle.

SplQpControl Parameter - cmdCode

cmdCode (LONG) - input
Printer control codes.

Control codes are as follows:

SPLC_ABORT	Printing is terminated
SPLC_PAUSE	Printing is paused
SPLC_CONTINUE	Printing resumes on a paused job

SplQpControl Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Success
FALSE	Error

SplQpControl - Parameters

hproc (HPROC) - input
The queue processor handle.

cmdCode (LONG) - input
Printer control codes.

Control codes are as follows:

SPLC_ABORT	Printing is terminated
SPLC_PAUSE	Printing is paused
SPLC_CONTINUE	Printing resumes on a paused job

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE Success
FALSE Error

SplQpControl - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

SplQpInstall

SplQpInstall - Syntax

Description:

SplQpInstall allows the user to configure a queue driver. This function is optional.

```
#include <os2.h>

HWND    hwnd;       /*  The window handle. */
BOOL    fSuccess;   /*  Return codes. */

fSuccess = SplQpInstall(hwnd);
```

SplQpInstall Parameter - hwnd

hwnd (HWND) - input
The window handle.

SplQpInstall Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE Success

FALSE Error

SplQpInstall - Parameters

hwnd (HWND) - input
The window handle.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE Success
FALSE Error

SplQpInstall - Remarks

This function is called by the Workplace Shell. It is used to display a dialog to the user for queue driver (queue processor) configuration. The queue driver then stores the values in the OS2SYS.INI file.

SplQpInstall - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplQpOpen

SplQpOpen - Syntax

Description:

SplQpOpen opens the queue driver and returns its handle. This function is normally called by the spooler.

```
#include <os2.h>

LONG            cData;    /*  The number of elements in the SQPOPENDATA structure. */
PSQPOPENDATA    pbData;   /*  Pointer to the SQPOPENDATA structure. */
HPROC           rc;       /*  Return codes. */
```

```
rc = SplQpOpen(cData, pbData);
```

SplQpOpen Parameter - cData

cData (LONG) - input
The number of elements in the SQPOPENDATA structure.

SplQpOpen Parameter - pbData

pbData (PSQPOPENDATA) - input
Pointer to the SQPOPENDATA structure.

This structure is based on the DEVOPENSTRUC data structure and would typically contain data extracted from the presentation driver's physical device block.

pszLogAddress	Pointer to the output port name. Can also be a filename.	
pszDriverName	Pointer to presentation driver name.	
pdriv	Pointer to a DRIVDATA structure:	
	cb	Size, in bytes, of this structure.
	lVersion	Version number of the data. Version numbers are defined by the presentation driver.
	szDeviceName[32]	String identifying the device. Valid values are supplied by the presentation driver.
	abGeneralData	General data, the type of which is defined by the presentation driver.
pszDataType	Pointer to the data type of the queued file. All queue drivers must support PM_Q_STD and PM_Q_RAW. User-defined data types are optional.	
pszComment	Pointer to a description of the application that submitted the print job.	
pszProcParams	Pointer to a string of blank-separated queue driver parameters. **Note:** All queue drivers must support the "COP=*n* " parameter, where *n* is the number of copies of the job to print.	
pszSpoolParams	Pointer to a string of spooler parameters separated by one or more blanks. Valid parameters are:	
	FORM=aaa	Identifies the form name for the print job. Multiple names should be separated by commas (aaa,bbb,ccc). If this parameter is not present, the job is printed on the current form. Form names are defined by the presentation driver. Valid names are

		those that would be returned from a call to the driver's GreQueryHardcopyCaps handling routine.
	PRTY=n	Identifies the priority for the print job. The priority can be any value from 1 through 99 (1 is lowest priority). Notice that this parameter has no significance when passed to SplQpOpen.
pszNetworkParams		Pointer to string of networking parameters. These are only used in a network environment; their nature is defined by the network application. Typically, the queue driver ignores this parameter.
pszDocName		Pointer to a string containing the document name.
pszQueueName		Pointer to a string containing the name of the queue from which the job was sent by the spooler.
pszToken		Pointer to the device information token. This currently is not used by queue drivers, and is typically ' * ' (asterisk).
idJobId		This is a USHORT value that identifies the print job.

SplQpOpen Return Value - rc

rc (HPROC) - returns
Return codes.

On completion, the handling routine must return:

SPL_ERROR	Error
¬=0	Queue driver (queue processor) handle

SplQpOpen - Parameters

cData (LONG) - input
The number of elements in the SQPOPENDATA structure.

pbData (PSQPOPENDATA) - input
Pointer to the SQPOPENDATA structure.

This structure is based on the DEVOPENSTRUC data structure and would typically contain data extracted from the presentation driver's physical device block.

pszLogAddress		Pointer to the output port name. Can also be a filename.
pszDriverName		Pointer to presentation driver name.
pdriv		Pointer to a DRIVDATA structure:
	cb	Size, in bytes, of this structure.
	lVersion	Version number of the data. Version numbers are defined by the presentation driver.

szDeviceName[32] — String identifying the device. Valid values are supplied by the presentation driver.

abGeneralData — General data, the type of which is defined by the presentation driver.

pszDataType — Pointer to the data type of the queued file. All queue drivers must support PM_Q_STD and PM_Q_RAW. User-defined data types are optional.

pszComment — Pointer to a description of the application that submitted the print job.

pszProcParams — Pointer to a string of blank-separated queue driver parameters.

Note: All queue drivers must support the "COP=*n* " parameter, where *n* is the number of copies of the job to print.

pszSpoolParams — Pointer to a string of spooler parameters separated by one or more blanks. Valid parameters are:

FORM=aaa — Identifies the form name for the print job. Multiple names should be separated by commas (aaa,bbb,ccc). If this parameter is not present, the job is printed on the current form.

Form names are defined by the presentation driver. Valid names are those that would be returned from a call to the driver's GreQueryHardcopyCaps handling routine.

PRTY=n — Identifies the priority for the print job. The priority can be any value from 1 through 99 (1 is lowest priority). Notice that this parameter has no significance when passed to SplQpOpen.

pszNetworkParams — Pointer to string of networking parameters. These are only used in a network environment; their nature is defined by the network application. Typically, the queue driver ignores this parameter.

pszDocName — Pointer to a string containing the document name.

pszQueueName — Pointer to a string containing the name of the queue from which the job was sent by the spooler.

pszToken — Pointer to the device information token. This currently is not used by queue drivers, and is typically ' * ' (asterisk).

idJobId — This is a USHORT value that identifies the print job.

rc (HPROC) - returns
Return codes.

On completion, the handling routine must return:

SPL_ERROR — Error
¬=0 — Queue driver (queue processor) handle

SplQpOpen - Remarks

The information passed into this API will be used by SplQpPrint to format the spool file.

SplQpOpen - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplQpPrint

SplQpPrint - Syntax

Description:

SplQpPrint processes and prints the spool file.

```
#include <os2.h>

HPROC     hproc;         /*  Queue processor handle returned by SplQpOpen. */
PSZ       pszFileName;   /*  The pointer to the name of the file containing the data. */
BOOL      fSuccess;      /*  Return codes. */

fSuccess = SplQpPrint(hproc, pszFileName);
```

SplQpPrint Parameter - hproc

hproc (HPROC) - input
Queue processor handle returned by SplQpOpen.

SplQpPrint Parameter - pszFileName

pszFileName (PSZ) - input
The pointer to the name of the file containing the data.

SplQpPrint Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Success
FALSE	Error

SplQpPrint - Parameters

hproc (HPROC) - input
Queue processor handle returned by SplQpOpen.

pszFileName (PSZ) - input
The pointer to the name of the file containing the data.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Success
FALSE	Error

SplQpPrint - Remarks

The handling routine in the queue driver opens a DC for an OD_DIRECT device type using the parameters passed into SplQpOpen.

SplQpPrint - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplQpQueryDt

SplQpQueryDt - Syntax

Description:

SplQpQueryDt returns a list of supported data types.

```
#include <os2.h>

PLONG    pcDatatypes;
PSZ      papszDatatypes;
BOOL     fSuccess;        /*  Return codes. */

fSuccess = SplQpQueryDt(pcDatatypes, papszDatatypes);
```

SplQpQueryDt Parameter - pcDatatypes

pcDatatypes (PLONG) - input
A pointer to a value indicating the maximum number of data types. On return, the value is updated to show the number of data types returned.

SplQpQueryDt Parameter - papszDatatypes

papszDatatypes (PSZ) - input
A pointer to an array of pointers that address the locations for the returned data type names. Each location must be an array of 16 characters to accommodate a name of the maximum length with its terminating NULL.

SplQpQueryDt Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Success
FALSE	Error

SplQpQueryDt - Parameters

pcDatatypes (PLONG) - input
A pointer to a value indicating the maximum number of data types. On return, the value is updated to show the number of data types returned.

papszDatatypes (PSZ) - input
A pointer to an array of pointers that address the locations for the returned data type names. Each location must be an array of 16 characters to accommodate a name of the maximum length with its terminating NULL.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Success
FALSE	Error

SplQpQueryDt - Remarks

This function should be called once with pcDatatypes set to 0 to determine the number of data types. The array papszDatatypes is not updated in this instance. The application can then allocate storage for the array and call the function a second time to return a list of supported data types.

SplQpQueryDt - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplQpQueryFlags

SplQpQueryFlags - Syntax

Description:

SplQpQueryFlags queries print queue processor flags. This function is optional.

```
#include <os2.h>

PULONG    pulFlags;
BOOL      fSuccess;  /*  Return codes. */

fSuccess = SplQpQueryFlags(pulFlags);
```

SplQpQueryFlags Parameter - pulFlags

pulFlags (PULONG) - input
A pointer to a ULONG to receive flag values.

SplQpQueryFlags Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Success
FALSE	Error

SplQpQueryFlags - Parameters

pulFlags (PULONG) - input
A pointer to a ULONG to receive flag values.

fSuccess (BOOL) - returns
Return codes.

On completion, the handling routine must return BOOLEAN (fSuccess).

TRUE	Success
FALSE	Error

SplQpQueryFlags - Remarks

This function is called to determine if this print queue processor allows the spooler to bypass it for PM_Q_RAW jobs. By exporting SplQpQueryFlags and setting the contents of pulFlags to QP_RAWDATA_BYPASS, the spooler can bypass calling this print queue processor to print PM_Q_RAW jobs that are still spooling.

SplQpQueryFlags - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplQueryControlPanel

SplQueryControlPanel - Syntax

Description:

This function queries a specific control panel DLL for support for the printer attached to the given port.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

PSZ       pszComputerName;  /*  Name of computer connected to the printer port. */
PSZ       pszPortName;      /*  Printer port name. */
PSZ       pszDeviceID;      /*  Device ID, if known, for the printer connected to pszPortName. */
PSZ       pszControlPanel;  /*  Name of control panel to call to query support for this printer. */
ULONG     flCapabilities;   /*  Capabilities requested. */
PULONG    pulOptions;       /*  Option value returned by the control panel DLL. */
ULONG     rc;               /*  Return codes. */

rc = SplQueryControlPanel(pszComputerName,
       pszPortName, pszDeviceID, pszControlPanel,
       flCapabilities, pulOptions);
```

SplQueryControlPanel Parameter - pszComputerName

pszComputerName (PSZ) - input
Name of computer connected to the printer port.

This parameter is NULL for printers whose port exists on the current machine.

SplQueryControlPanel Parameter - pszPortName

pszPortName (PSZ) - input
Printer port name.

The name of the printer port in *pszComputerName* used to determine if a control panel can be displayed. Following is an example of a port name:
LPT1

This will be a locally defined port name.

SplQueryControlPanel Parameter - pszDeviceID

pszDeviceID (PSZ) - input
Device ID, if known, for the printer connected to *pszPortName* .

SplQueryControlPanel Parameter - pszControlPanel

pszControlPanel (PSZ) - input
Name of control panel to call to query support for this printer.

This name can be one of the following:

- The name registered with SplRegisterControlPanel
- One of the control panel names returned by SplGetControlPanelList

This parameter can be used to determine the type of support given by a control panel in the list.

SplQueryControlPanel Parameter - flCapabilities

flCapabilities (ULONG) - input
Capabilities requested.

FL_ADMIN - 0x00000001
Set if an administrator wants the control panel.

SplQueryControlPanel Parameter - pulOptions

pulOptions (PULONG) - output
Option value returned by the control panel DLL.

Values are as follows:

OPT_CUSTOM(2)
A control panel for the specific device is available.

OPT_GENERIC(1)
A generic control panel can be displayed for the printer.

SplQueryControlPanel Return Value - rc

rc (ULONG) - returns
Return codes.

0 Success

ERROR_INVALID_NAME(123)
pszName is not the name of a control panel DLL.

ERROR_NOT_SUPPORTED(50)
A control panel for this printer is not supported by the called control panel DLL.

SplQueryControlPanel - Parameters

pszComputerName (PSZ) - input
Name of computer connected to the printer port.

This parameter is NULL for printers whose port exists on the current machine.

pszPortName (PSZ) - input
Printer port name.

The name of the printer port in *pszComputerName* used to determine if a control panel can be displayed. Following is an example of a port name:
LPT1

This will be a locally defined port name.

pszDeviceID (PSZ) - input
Device ID, if known, for the printer connected to *pszPortName* .

pszControlPanel (PSZ) - input
Name of control panel to call to query support for this printer.

This name can be one of the following:

- The name registered with SplRegisterControlPanel
- One of the control panel names returned by SplGetControlPanelList

This parameter can be used to determine the type of support given by a control panel in the list.

flCapabilities (ULONG) - input
Capabilities requested.

FL_ADMIN - 0x00000001
Set if an administrator wants the control panel.

pulOptions (PULONG) - output
Option value returned by the control panel DLL.

Values are as follows:

OPT_CUSTOM(2)
A control panel for the specific device is available.

OPT_GENERIC(1)
A generic control panel can be displayed for the printer.

rc (ULONG) - returns
Return codes.

0 Success

ERROR_INVALID_NAME(123)
pszName is not the name of a control panel DLL.

ERROR_NOT_SUPPORTED(50)
A control panel for this printer is not supported by the called control panel DLL.

SplQueryControlPanel - Remarks

None.

SplQueryControlPanel - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplQueryPath

SplQueryPath - Syntax

Description:

This function returns the path to get printer drivers or port drivers.

When a new printer is being installed that requires a printer driver not yet installed, the print object calls this function to get the path to the print drivers.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

PSZ       pszComputerName;     /*  Name of the computer for which a network print object is being created. */
PSZ       pszRemoteQueueName;  /*  Name of the remote queue the network print object is referencing. */
ULONG     ulLevel;             /*  Level of information to return. */
PVOID     pBuf;                /*  Receives the path to printer drivers for level 0. */
ULONG     cbBuf;               /*  Length of buffer pointed to by pBuf, in bytes. */
PULONG    pcbNeeded;           /*  Receives length, in bytes, of buffer needed to return path. */
ULONG     rc;                  /*  Return codes. */

rc = SplQueryPath(pszComputerName, pszRemoteQueueName,
       ulLevel, pBuf, cbBuf, pcbNeeded);
```

SplQueryPath Parameter - pszComputerName

pszComputerName (PSZ) - input
Name of the computer for which a network print object is being created.

This value can be NULL.

SplQueryPath Parameter - pszRemoteQueueName

pszRemoteQueueName (PSZ) - input
Name of the remote queue the network print object is referencing.

This value can be NULL.

SplQueryPath Parameter - ulLevel

ulLevel (ULONG) - input
Level of information to return.

This must be 0 (zero).

SplQueryPath Parameter - pBuf

pBuf (PVOID) - output
Receives the path to printer drivers for level 0.

SplQueryPath Parameter - cbBuf

cbBuf (ULONG) - input
Length of buffer pointed to by *pBuf,* in bytes.

SplQueryPath Parameter - pcbNeeded

pcbNeeded (PULONG) - output
Receives length, in bytes, of buffer needed to return path.

SplQueryPath Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
ERROR_NOT_SUPPORTED(50)	A path to printer drivers cannot be found.
NERR_BufTooSmall(2123)	Must call again with a bigger buffer.

SplQueryPath - Parameters

pszComputerName (PSZ) - input
Name of the computer for which a network print object is being created.

This value can be NULL.

pszRemoteQueueName (PSZ) - input
Name of the remote queue the network print object is referencing.

This value can be NULL.

ulLevel (ULONG) - input
Level of information to return.

This must be 0 (zero).

pBuf (PVOID) - output
Receives the path to printer drivers for level 0.

cbBuf (ULONG) - input
Length of buffer pointed to by *pBuf,* in bytes.

pcbNeeded (PULONG) - output
Receives length, in bytes, of buffer needed to return path.

rc (ULONG) - returns
Return codes.

0	Success
ERROR_NOT_SUPPORTED(50)	A path to printer drivers cannot be found.
NERR_BufTooSmall(2123)	Must call again with a bigger buffer.

SplQueryPath - Remarks

The spooler will look in the following places for the path to printer drivers:

1. The spooler will first look in the system .INI file for applicationName PM_SPOOLER_DRVSHARE. If found, the spooler will return the keyValue for keyName *pszComputerName* if it exists, or for the first keyName that exists under the applicationName.
2. The spooler will check the *pszComputerName* field for a share called PRINTDRV.
3. The spooler will check the user's logon domain LS:*ALIAS for a share called PRINTDRV.

If a path is found, the spooler will return the path to the caller. The path can be one of the following:

- An extended UNC path (LS:\\SERVER1\PRINTDRV)
- A UNC path with additional subdirectories defined (LS:\\SERVER1\PRINTDRV\DIR1\NEXT)
- A fully qualified pathname (Z:\OS2\SHARED)

The print object will use the path as follows:

- Look in the root of the path for packed files (.DR_)
- Look in the root of the path for unpacked files (.DRV)

- Look in the subdirectory PMDD_n from path, where *n* is the OS2 printer driver diskette number.

SplQueryPath - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplQueryPort

SplQueryPort - Syntax

Description:

This function queries information about a printer port.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

PSZ       pszComputerName;  /*  Name of the computer connected to the port. */
PSZ       pszPortName;      /*  Name of the port to query. */
ULONG     ulLevel;          /*  Level of information to return. */
PVOID     pBuf;             /*  Receives the PRPORTINFO2 data structure. */
ULONG     cbBuf;            /*  Length of buffer pointed to by pBuf, in bytes. */
PULONG    pcbNeeded;        /*  Receives length, in bytes, of buffer needed to return all the port data. */
ULONG     rc;               /*  Return codes. */

rc = SplQueryPort(pszComputerName, pszPortName,
       ulLevel, pBuf, cbBuf, pcbNeeded);
```

SplQueryPort Parameter - pszComputerName

pszComputerName (PSZ) - input
Name of the computer connected to the port.

This value can be NULL for a local machine.

SplQueryPort Parameter - pszPortName

pszPortName (PSZ) - input
Name of the port to query.

SplQueryPort Parameter - ulLevel

ulLevel (ULONG) - input
Level of information to return.

This must be 2.

SplQueryPort Parameter - pBuf

pBuf (PVOID) - output
Receives the PRPORTINFO2 data structure.

See PRPORTINFO2 in Data Types page=no. for details on this data structure.

SplQueryPort Parameter - cbBuf

cbBuf (ULONG) - input
Length of buffer pointed to by *pBuf,* in bytes.

SplQueryPort Parameter - pcbNeeded

pcbNeeded (PULONG) - output
Receives length, in bytes, of buffer needed to return all the port data.

SplQueryPort Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
ERROR_INVALID_LEVEL(124)	*ulLevel* is not 2.

ERROR_INVALID_PARAMETER(87)
An invalid parameter was specified; most likely an invalid *pBuf* value.

ERROR_MORE_DATA(234)
Not all data could fit in *pBuf* .

ERROR_NOT_SUPPORTED(50)
Command not supported on *pszComputerName*.

NERR_BufTooSmall(2123)
No data could fit in *pBuf* .

NERR_DestNotFound(2152)
The port name does not exist.

SplQueryPort - Parameters

pszComputerName (PSZ) - input
Name of the computer connected to the port.

This value can be NULL for a local machine.

pszPortName (PSZ) - input
Name of the port to query.

ulLevel (ULONG) - input
Level of information to return.

This must be 2.

pBuf (PVOID) - output
Receives the PRPORTINFO2 data structure.

See PRPORTINFO2 in Data Types page=no. for details on this data structure.

cbBuf (ULONG) - input
Length of buffer pointed to by *pBuf,* in bytes.

pcbNeeded (PULONG) - output
Receives length, in bytes, of buffer needed to return all the port data.

rc (ULONG) - returns
Return codes.

0
Success

ERROR_INVALID_LEVEL(124)
ulLevel is not 2.

ERROR_INVALID_PARAMETER(87)
An invalid parameter was specified; most likely an invalid *pBuf* value.

ERROR_MORE_DATA(234)
Not all data could fit in *pBuf* .

ERROR_NOT_SUPPORTED(50)
Command not supported on *pszComputerName*.

NERR_BufTooSmall(2123)
No data could fit in *pBuf* .

NERR_DestNotFound(2152)
The port name does not exist.

SplQueryPort - Remarks

None.

SplQueryPort - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplQueryQueue

SplQueryQueue - Syntax

This function supplies information about a print queue, and, optionally, about the jobs in it.

```
#define INCL_SPL /* Or use INCL_PM, */
#include <os2.h>

PSZ       pszComputerName;  /*  Name of computer where queue is to be queried. */
PSZ       pszQueueName;     /*  Queue name. */
ULONG     ulLevel;          /*  Level of detail required. */
PVOID     pBuf;             /*  Buffer. */
ULONG     cbBuf;            /*  Size, in bytes, of Buffer. */
PULONG    pcbNeeded;        /*  Size in bytes of available information. */
SPLERR    rc;               /*  Return code. */

rc = SplQueryQueue(pszComputerName, pszQueueName,
       ulLevel, pBuf, cbBuf, pcbNeeded);
```

SplQueryQueue Parameter - pszComputerName

pszComputerName (PSZ) - input
Name of computer where queue is to be queried.

A NULL string specifies the local workstation.

SplQueryQueue Parameter - pszQueueName

pszQueueName (PSZ) - input
Queue name.

SplQueryQueue Parameter - ulLevel

ulLevel (ULONG) - input
Level of detail required.

This must be 3, 4, 5, 6 or 7.

SplQueryQueue Parameter - pBuf

pBuf (PVOID) - in/out
Buffer.

The returned contents of the buffer depend on the value specified in *ulLevel* as follows:

ulLevel	**Buffer Contents**
3	A PRQINFO3 structure, with associated variable information up to *cbBuf* . The *fsType* bit PRQ3_TYPE_APPDEFAULT is set for the application default queue only.
4	A PRQINFO3 structure, with associated variable information, and an array of PRJINFO2 structures, one for each job in the queue, up to *cbBuf* .
5	A queue name of type PSZ.
6	A PRQINFO6 structure, with associated variable information up to *cbBuf* .
7	A PRQINFO3 structure, with associated variable information, and an array of PRJINFO4 structures, one for each job in the queue, up to *cbBuf* .

SplQueryQueue Parameter - cbBuf

cbBuf (ULONG) - input
Size, in bytes, of Buffer.

It must be greater than 0.

SplQueryQueue Parameter - pcbNeeded

pcbNeeded (PULONG) - output
Size in bytes of available information.

SplQueryQueue Return Value - rc

rc (SPLERR) - returns
Return code.

NO_ERROR (0)
No errors occurred.
ERROR_ACCESS_DENIED (5)
Access is denied.
ERROR_NOT_SUPPORTED (50)
This request is not supported by the network.
ERROR_BAD_NETPATH (53)
The network path cannot be located.
ERROR_INVALID_PARAMETER (87)
An invalid parameter is specified.
ERROR_INVALID_LEVEL (124)
The level parameter is invalid.
ERROR_MORE_DATA (234)
Additional data is available.
NERR_NetNotStarted (2102)
The network program is not started.
NERR_BufTooSmall (2123)
The API return buffer is too small.
NERR_QNotFound (2150)
The printer queue does not exist.
NERR_InvalidComputer (2351)
The computer name is invalid.

SplQueryQueue - Parameters

pszComputerName (PSZ) - input
Name of computer where queue is to be queried.

A NULL string specifies the local workstation.

pszQueueName (PSZ) - input
Queue name.

ulLevel (ULONG) - input
Level of detail required.

This must be 3, 4, 5, 6 or 7.

pBuf (PVOID) - in/out
Buffer.

The returned contents of the buffer depend on the value specified in *ulLevel* as follows:

ulLevel	**Buffer Contents**
3	A PRQINFO3 structure, with associated variable information up to *cbBuf* . The *fsType* bit PRQ3_TYPE_APPDEFAULT is set for the application default queue only.
4	A PRQINFO3 structure, with associated variable information, and an array of PRJINFO2 structures, one for each job in the queue, up to *cbBuf* .
5	A queue name of type PSZ.
6	A PRQINFO6 structure, with associated variable information up to *cbBuf* .
7	A PRQINFO3 structure, with associated variable information, and an array of PRJINFO4 structures, one for each job in the queue, up to *cbBuf* .

cbBuf (ULONG) - input
Size, in bytes, of Buffer.

It must be greater than 0.

pcbNeeded (PULONG) - output
Size in bytes of available information.

rc (SPLERR) - returns
Return code.

NO_ERROR (0)
No errors occurred.
ERROR_ACCESS_DENIED (5)
Access is denied.
ERROR_NOT_SUPPORTED (50)
This request is not supported by the network.
ERROR_BAD_NETPATH (53)
The network path cannot be located.
ERROR_INVALID_PARAMETER (87)
An invalid parameter is specified.
ERROR_INVALID_LEVEL (124)
The level parameter is invalid.
ERROR_MORE_DATA (234)
Additional data is available.
NERR_NetNotStarted (2102)
The network program is not started.
NERR_BufTooSmall (2123)
The API return buffer is too small.
NERR_QNotFound (2150)
The printer queue does not exist.
NERR_InvalidComputer (2351)
The computer name is invalid.

SplQueryQueue - Remarks

If *ulLevel* is 3 or 4, and *pBuf* cannot hold the entire PRQINFO3 structure, SplQueryQueue returns NERR_BufTooSmall (2123).

If *ulLevel* is 6, and *pBuf* cannot hold the entire PRQINFO6 structure, SplQueryQueue returns NERR_BufTooSmall (2123).

If *ulLevel* is 4, and *pBuf* cannot hold all the available PRJINFO2 structures, SplQueryQueue returns ERROR_MORE_DATA (234).

To obtain the size of buffer required, call SplQueryQueue with the required value of *ulLevel* , *cbBuf* set to 0 (zero), and a NULL buffer. The number of bytes required is returned in *pcbNeeded* .

SplQueryQueue - Related Functions

- SplEnumQueue
- SplQueryJob
- SplSetJob
- SplSetQueue

SplQueryQueue - Remarks

This sample code queries the local workstation for a queue name that is entered at the command prompt. The query is done at level 4 which

returns returns in the buffer information in a PRQINFO3 structure and follows this with PRJINFO2 structures - one for each job in the queue.

```
#define INCL_SPL
#define INCL_SPLDOSPRINT
#define INCL_SPLERRORS

#include <os2.h>
#include <stdio.h>

INT main (argc, argv)
INT argc;
CHAR *argv[];

{
  ULONG  splerr;
  ULONG  cbBuf;
  ULONG  cbNeeded;
  ULONG  ulLevel;
  ULONG  i;
  USHORT uJobCount;
  PSZ    pszComputerName;
  PSZ    pszQueueName;
  PVOID  pBuf;
  PPRJINFO2 prj2;
  PPRQINFO3 prq3 ;

  if (argc != 2)
  {
    printf("Syntax:  setqryq  QueueName \n");
    DosExit(EXIT_PROCESS, 0);
  }

  pszComputerName = (PSZ)NULL;
  pszQueueName = argv[1];
  ulLevel = 4L;
  splerr = SplQueryQueue(pszComputerName,
                         pszQueueName,
                         ulLevel,
                         (PVOID)NULL,
                         0L,
                         &cbNeeded);

  if (splerr != NERR_BufTooSmall ||
      splerr != ERROR_MORE_DATA)
  {
    printf("SplQueryQueue Error=%ld, cbNeeded=%ld\n",splerr, cbNeeded);
    DosExit(EXIT_PROCESS, 0);
  }

  if (!DosAllocMem(&pBuf, cbNeeded,
                   PAG_READ  |
                   PAG_WRITE |
                   PAG_COMMIT))
  {
    cbBuf = cbNeeded;
    splerr = SplQueryQueue(pszComputerName,
                           pszQueueName,
                           ulLevel,
                           pBuf,
                           cbBuf,
                           &cbNeeded);
    prq3=(PPRQINFO3)pBuf;
    printf("Queue info: name- %s\n", prq3->pszName);
    if (prq3->fsType & PRQ3_TYPE_APPDEFAULT)
       printf(" This is the application default print queue\n");
       printf("  priority - %d  starttime - %d  endtime - %d fsType - %X\n",
                 prq3->uPriority,
                 prq3->uStartTime,
                 prq3->uUntilTime,
                 prq3->fsType);
       printf("  pszSepFile    - %s\n", prq3->pszSepFile);
       printf("  pszPrProc     - %s\n", prq3->pszPrProc);
       printf("  pszParms      - %s\n", prq3->pszParms);
       printf("  pszComment    - %s\n", prq3->pszComment);
       printf("  pszPrinters   - %s\n", prq3->pszPrinters);
       printf("  pszDriverName - %s\n", prq3->pszDriverName);

    if (prq3->pDriverData)
    {
      printf("  pDriverData->cb           - %ld\n",
                (ULONG)prq3->pDriverData->cb);
      printf("  pDriverData->lVersion     - %ld\n",
```

```
                (ULONG)prq3->pDriverData->lVersion);
      printf("  pDriverData->szDeviceName - %s\n",
                prq3->pDriverData->szDeviceName);
    }
    /* Store the job count for use later in the for loop */
    uJobCount = prq3->cJobs;
    printf("Job count in this queue is %d\n\n",uJobCount);

    /* Increment the pointer to the PRQINFO3 structure       */
    /* so that it points to the first structure after itself */
    prq3++;

    /* Cast the prq3 pointer to point to a PRJINFO2 structure, */
    /* and set a pointer to point to that place                */
    prj2=(PPRJINFO2)prq3;
    for (i=0 ; i<uJobCount ;i++)
    {
      printf("Job ID    = %d\n",  prj2->uJobId);
      printf("Priority  = %d\n",  prj2->uPriority);
      printf("User Name = %s\n",  prj2->pszUserName);
      printf("Position  = %d\n",  prj2->uPosition);
      printf("Status    = %d\n",  prj2->fsStatus);
      printf("Submitted = %ld\n", prj2->ulSubmitted);
      printf("Size      = %ld\n", prj2->ulSize);
      printf("Comment   = %s\n",  prj2->pszComment);
      printf("Document  = %s\n\n",prj2->pszDocument);

      /* Increment the pointer to point to the next structure */
      prj2++;
    } /* endfor */
    DosFreeMem(pBuf);
  }
  DosExit(EXIT_PROCESS, 0);
  return (splerr);
}
```

SplQueryQueue - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Remarks
Related Functions
Glossary

SplRegister

SplRegister - Syntax

Description:

This function allows applications to register for notification of events about printers. This function may be used for job accounting purposes or by printer drivers to keep aware of the operation of the print subsystem.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

PSZ       pszComputerName;  /*  Printer server name to register for alert notification. */
PSZ       pszName;          /*  Case-sensitive name of printer port to register for alerts. */
HWND      hwndNotify;       /*  Window handle to get notify messages. */
ULONG     ulCategory;       /*  Category of alert to register. */
ULONG     ulType;           /*  Type of alert to enable for the given category. */
PULONG    pulHandle;        /*  Pointer to spooler alert handle. */
ULONG     rc;               /*  Return codes. */

rc = SplRegister(pszComputerName, pszName,
       hwndNotify, ulCategory, ulType, pulHandle);
```

SplRegister Parameter - pszComputerName

pszComputerName (PSZ) - input
Printer server name to register for alert notification.

The field is used by an application running on a client machine to register for alerts on a print server. To register locally, this field must be NULL.

SplRegister Parameter - pszName

pszName (PSZ) - input
Case-sensitive name of printer port to register for alerts.

This value can be a virtual port name.

SplRegister Parameter - hwndNotify

hwndNotify (HWND) - input
Window handle to get notify messages.

The message posted is as follows:

```
Message ID is WM_SPOOLER_ALERT(0x00d8)
```

MP1 will define the type of alert as follows:

```
/* Alert type - SHORT1FROMMP(MP1) bits 0-7 */
#define SPLMSG_SPECIAL_ALERT          0x00FC
#define SPLMSG_EXTENDED_ALERT         0x00FD
#define SPLMSG_CORE_ALERT_MASK        0x00FF

/* Extended alert type - SHORT1FROMMP(MP1) bits 8-15 */
#define SPLMSG_EXTENDED_TYPE_MASK     0xFF00
```

If the object type is SPLMSG_EXTENDED_ALERT, then this is an extended alert. The extended alert category is SHORT2FROMMP(MP1). The type of alert in the given extended alert category is SHORT1FROMMP(MP1) bits 8-15. This limits the

notification given to applications registered for extended alerts to only the specific categories that are 0xFFFF or less and to only category types that are 0xFF or less.

If the object type is SPLMSG_SPECIAL_ALERT, then this is a special alert generated by the spooler and SHORT2FROMMP(MP1) will contain one of the following messages:

SPLMSG_CONNECTION_LOST - 0x0002
Connection to computer terminated; all alerts disabled for given computer.

SPLMSG_PORT_TERMINATED - 0x0003
Registration terminated for port.

SPLMSG_SPOOLER_DISABLED - 0x0001
Spooler disabled; all alerts for given computer are disabled.

If the object type is less than SPLMSG_SPECIAL_ALERT(0xFC), then this is a core alert. The specific core alert posted will be: SHORT1FROMMP(MP1) bits 0-7(SPLMSG_CORE_ALERT_MASK)

If the core alert is PRTALERT_TYPE_PAGE_PRINTED(9), then SHORT1FROMMP(MP1) bits 8-15(SPLMSG_PAGE_PRINTED_MASK) will be the page number printed. If the page number is above 255, the value hex 'FF' will be put in this byte field.

If the core alert is **not** PRTALERT_TYPE_PAGE_PRINTED(9), then SHORT1FROMMP(MP1) bits 8-15(SPLMSG_CORE_SEVERITY_MASK) will be the severity of the alert. SHORT2FROMMP(MP1) will be the spooler job ID if this is a core alert for a print job.

The core alert type will be one of the following:

Value	Description
1	PRTALERT_TYPE_INPUT Input alert (Example: out of paper).
2	PRTALERT_TYPE_OUTPUT Output alert (Example: output bin full).
3	PRTALERT_TYPE_JAM Paper jam alert.
4	PRTALERT_TYPE_OPERATOR Operator intervention required (Example: print ribbon jam).
5	PRTALERT_TYPE_CONFIG_CHANGE Configuration changed (Example: cartridge removed).
6	PRTALERT_TYPE_SUPPLIES Supplies alert (Example: out of toner).
7	PRTALERT_TYPE_JOB_START Job started printing.
8	PRTALERT_TYPE_JOB_STACKED Job completed and stacked.
9	PRTALERT_TYPE_PAGE_PRINTED Page printed.
10	PRTALERT_TYPE_JOB_CANCELLED Job cancelled inside printer.
11	PRTALERT_TYPE_JOB_HELD Job held inside printer.
12	PRTALERT_TYPE_COVER_OPEN Cover or panel open.
13	PRTALERT_TYPE_POWER_ON Printer powered on.
14	PRTALERT_TYPE_RESET Printer reset.
15	PRTALERT_TYPE_ONLINE Printer now online.
16	PRTALERT_TYPE_OFFLINE Printer now offline.

17	PRTALERT_TYPE_TIMED_ALERT Timed alert.
18	PRTALERT_TYPE_COMMUNICATION_PROBLEM Communication problem.
19	PRTALERT_TYPE_COMM_STATUS_CHANGED Communication status changed. BIDI_Q_PORT should be issued to get current state.
0xFF	PRTALERT_TYPE_OTHER Other alert type (0xFFFFFFFF).

The severity of the problem will be one of the following:

Value	Description
00	PRTALERT_SEV_INFORMATIONAL
01	PRTALERT_SEV_WARNING
02	PRTALERT_SEV_ERROR Printing stopped due to error.
03	PRTALERT_SEV_SERVICE Service is required.
04 - FD	Reserved
FE	PRTALERT_SEV_OTHER Other (0xFE)
FF	PRTALERT_SEV_UNKNOWN Unknown (0xFF)

MP2 contents are as follows:
SHORT1FROMMP(MP1) is the atomized print server name.
If the notification is from the local spooler, this value will be zero.

SHORT2FROMMP(MP2) is the atomized print queue name when the
message involves a print job (alert values 7 - 11);
otherwise, it will be the atomized printer port name, if applicable.

SplRegister Parameter - ulCategory

ulCategory (ULONG) - input
Category of alert to register.

This value can be the core alert category or an extended alert category as defined below:

PRTALERT_CATEGORY_CORE - 0x00000001
Core alert category

PRTALERT_CATEGORY_EXTENDED - 0x0000D000
Beginning of the extended alert categories. Protocol converters should limit the category range to between 0x0000D000 and 0x0000DFFFF.

PRTALERT_CATEGORY_EXT_MAX - 0x0000DFFF
End of the extended alert categories.

If the protocol converter does not support the given category, ERROR_INVALID_CATEGORY(117) is returned.

SplRegister Parameter - ulType

ulType (ULONG) - input

Type of alert to enable for the given category.

If the category is PRTALERT_CATEGORY_CORE, this is a bit-field that defines the additional core alerts to enable. The core alerts corresponding to the bits set in the *ulType* field will be enabled, in addition to any currently enabled core alerts. If the protocol converter does not understand one of the bits that are set, ERROR_INVALID_FLAG_NUMBER(186) should be returned.

If the printer is not capable of setting one of the core alerts whose bit is set, and if any core alerts can be enabled, a Success return code should be returned.

If the category is **not** PRTALERT_CATEGORY_CORE, then this is a value that defines the specific extended alert to enable.

Bit defintions for core alerts are as follows:

Bit	Description
0	PRTALERT_CORE_INPUT - 0x00000001 Input alert (Example: out of paper).
1	PRTALERT_CORE_OUTPUT - 0x00000002 Output alert (Example: output bin is full).
2	PRTALERT_CORE_JAM - 0x00000004 Paper jam alert.
3	PRTALERT_CORE_OPERATOR - 0x00000008 Operator intervention required (Example: print ribbon jam).
4	PRTALERT_CORE_CONFIG_CHANGE - 0x00000010 Configuration changed (Example: cartridge removed).
5	PRTALERT_CORE_SUPPLIES - 0x00000020 Supplies alert (Example: out of toner).
6	PRTALERT_CORE_JOB_START - 0x00000040 Job started printing.
7	PRTALERT_CORE_JOB_STACKED - 0x00000080 Job completed and stacked.
8	PRTALERT_CORE_PAGE_PRINTED - 0x00000100 Page printed.
9	PRTALERT_CORE_JOB_CANCELLED - 0x00000200 Job cancelled inside printer.
10	PRTALERT_CORE_JOB_HELD - 0x00000400 Job held inside printer.
11	PRTALERT_CORE_COVER_OPEN - 0x00000800 Cover or panel open.
12	PRTALERT_CORE_POWER_ON - 0x00001000 Printer powered on.
13	PRTALERT_CORE_RESET - 0x00002000 Printer reset.
14	PRTALERT_CORE_ONLINE - 0x00004000 Printer now online.
15	PRTALERT_CORE_OFFLINE - 0x00008000 Printer now offline.
16	PRTALERT_CORE_TIMED_ALERT - 0x00010000 Timed alert.
17	PRTALERT_CORE_COMMUNICATION_PROBLEM - 0x00020000 Communication problem.
18	PRTALERT_CORE_COMM_STATUS_CHANGED - 0x00040000 Communication status changed.

SplRegister Parameter - pulHandle

pulHandle (PULONG) - in/out
Pointer to spooler alert handle.

For initial registration, the value pointed to by this parameter should be 0 (zero). If an application wants to make better use of spooler resources, the application can use the same returned spooler handle for all of its alert registration.

SplRegister Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
ERROR_INVALID_CATEGORY(117)	The protocol converter does not support the extended alert category.
ERROR_INVALID_FLAG_NUMBER(186)	The protocol converter did not understand a core alert that was set.
ERROR_INVALID_HANDLE(6)	The value pointed to by *pulHandle* is not a valid spooler alert handle. The value at **pulHandle* should be set to 0 (zero) to allocate a new spooler alert handle.
ERROR_INVALID_PARAMETER(87)	An invalid buffer given.
ERROR_NOT_SUPPORTED(50)	The spooler on the given print server cannot support alert notification.
PMERR_SPL_SPOOLER_NOT_INSTALLED(0x4009)	The spooler is not enabled.

SplRegister - Parameters

pszComputerName (PSZ) - input
Printer server name to register for alert notification.

The field is used by an application running on a client machine to register for alerts on a print server. To register locally, this field must be NULL.

pszName (PSZ) - input
Case-sensitive name of printer port to register for alerts.

This value can be a virtual port name.

hwndNotify (HWND) - input
Window handle to get notify messages.

The message posted is as follows:

```
Message ID is WM_SPOOLER_ALERT(0x00d8)
```

MP1 will define the type of alert as follows:

```
/* Alert type - SHORT1FROMMP(MP1) bits 0-7 */
#define SPLMSG_SPECIAL_ALERT          0x00FC
#define SPLMSG_EXTENDED_ALERT         0x00FD
#define SPLMSG_CORE_ALERT_MASK        0x00FF

/* Extended alert type - SHORT1FROMMP(MP1) bits 8-15 */
#define SPLMSG_EXTENDED_TYPE_MASK     0xFF00
```

If the object type is SPLMSG_EXTENDED_ALERT, then this is an extended alert. The extended alert category is SHORT2FROMMP(MP1). The type of alert in the given extended alert category is SHORT1FROMMP(MP1) bits 8-15. This limits the notification given to applications registered for extended alerts to only the specific categories that are 0xFFFF or less and to only category types that are 0xFF or less.

If the object type is SPLMSG_SPECIAL_ALERT, then this is a special alert generated by the spooler and SHORT2FROMMP(MP1) will contain one of the following messages:

SPLMSG_CONNECTION_LOST - 0x0002
Connection to computer terminated; all alerts disabled for given computer.

SPLMSG_PORT_TERMINATED - 0x0003
Registration terminated for port.

SPLMSG_SPOOLER_DISABLED - 0x0001
Spooler disabled; all alerts for given computer are disabled.

If the object type is less than SPLMSG_SPECIAL_ALERT(0xFC), then this is a core alert. The specific core alert posted will be: SHORT1FROMMP(MP1) bits 0-7(SPLMSG_CORE_ALERT_MASK)

If the core alert is PRTALERT_TYPE_PAGE_PRINTED(9), then SHORT1FROMMP(MP1) bits 8-15(SPLMSG_PAGE_PRINTED_MASK) will be the page number printed. If the page number is above 255, the value hex 'FF' will be put in this byte field.

If the core alert is **not** PRTALERT_TYPE_PAGE_PRINTED(9), then SHORT1FROMMP(MP1) bits 8-15(SPLMSG_CORE_SEVERITY_MASK) will be the severity of the alert. SHORT2FROMMP(MP1) will be the spooler job ID if this is a core alert for a print job.

The core alert type will be one of the following:

Value	Description
1	PRTALERT_TYPE_INPUT Input alert (Example: out of paper).
2	PRTALERT_TYPE_OUTPUT Output alert (Example: output bin full).
3	PRTALERT_TYPE_JAM Paper jam alert.
4	PRTALERT_TYPE_OPERATOR Operator intervention required (Example: print ribbon jam).
5	PRTALERT_TYPE_CONFIG_CHANGE Configuration changed (Example: cartridge removed).
6	PRTALERT_TYPE_SUPPLIES Supplies alert (Example: out of toner).
7	PRTALERT_TYPE_JOB_START Job started printing.
8	PRTALERT_TYPE_JOB_STACKED Job completed and stacked.
9	PRTALERT_TYPE_PAGE_PRINTED Page printed.
10	PRTALERT_TYPE_JOB_CANCELLED Job cancelled inside printer.
11	PRTALERT_TYPE_JOB_HELD Job held inside printer.
12	PRTALERT_TYPE_COVER_OPEN

Cover or panel open.

13 PRTALERT_TYPE_POWER_ON
Printer powered on.

14 PRTALERT_TYPE_RESET
Printer reset.

15 PRTALERT_TYPE_ONLINE
Printer now online.

16 PRTALERT_TYPE_OFFLINE
Printer now offline.

17 PRTALERT_TYPE_TIMED_ALERT
Timed alert.

18 PRTALERT_TYPE_COMMUNICATION_PROBLEM
Communication problem.

19 PRTALERT_TYPE_COMM_STATUS_CHANGED
Communication status changed. BIDI_Q_PORT should be issued to get current state.

0xFF PRTALERT_TYPE_OTHER
Other alert type (0xFFFFFFFF).

The severity of the problem will be one of the following:

Value	Description
00	PRTALERT_SEV_INFORMATIONAL
01	PRTALERT_SEV_WARNING
02	PRTALERT_SEV_ERROR Printing stopped due to error.
03	PRTALERT_SEV_SERVICE Service is required.
04 - FD	Reserved
FE	PRTALERT_SEV_OTHER Other (0xFE)
FF	PRTALERT_SEV_UNKNOWN Unknown (0xFF)

MP2 contents are as follows:
SHORT1FROMMP(MP1) is the atomized print server name.
If the notification is from the local spooler, this value will be zero.

SHORT2FROMMP(MP2) is the atomized print queue name when the
message involves a print job (alert values 7 - 11);
otherwise, it will be the atomized printer port name, if applicable.

ulCategory (ULONG) - input
Category of alert to register.

This value can be the core alert category or an extended alert category as defined below:

PRTALERT_CATEGORY_CORE - 0x00000001
Core alert category

PRTALERT_CATEGORY_EXTENDED - 0x0000D000
Beginning of the extended alert categories. Protocol converters should limit the category range to between 0x0000D000 and 0x0000DFFFF.

PRTALERT_CATEGORY_EXT_MAX - 0x0000DFFF
End of the extended alert categories.

If the protocol converter does not support the given category, ERROR_INVALID_CATEGORY(117) is returned.

ulType (ULONG) - input
Type of alert to enable for the given category.

If the category is PRTALERT_CATEGORY_CORE, this is a bit-field that defines the additional core alerts to enable. The core alerts corresponding to the bits set in the *ulType* field will be enabled, in addition to any currently enabled core alerts. If the protocol converter does not understand one of the bits that are set, ERROR_INVALID_FLAG_NUMBER(186) should be returned.

If the printer is not capable of setting one of the core alerts whose bit is set, and if any core alerts can be enabled, a Success return code should be returned.

If the category is **not** PRTALERT_CATEGORY_CORE, then this is a value that defines the specific extended alert to enable.

Bit defintions for core alerts are as follows:

Bit	Description
0	PRTALERT_CORE_INPUT - 0x00000001 Input alert (Example: out of paper).
1	PRTALERT_CORE_OUTPUT - 0x00000002 Output alert (Example: output bin is full).
2	PRTALERT_CORE_JAM - 0x00000004 Paper jam alert.
3	PRTALERT_CORE_OPERATOR - 0x00000008 Operator intervention required (Example: print ribbon jam).
4	PRTALERT_CORE_CONFIG_CHANGE - 0x00000010 Configuration changed (Example: cartridge removed).
5	PRTALERT_CORE_SUPPLIES - 0x00000020 Supplies alert (Example: out of toner).
6	PRTALERT_CORE_JOB_START - 0x00000040 Job started printing.
7	PRTALERT_CORE_JOB_STACKED - 0x00000080 Job completed and stacked.
8	PRTALERT_CORE_PAGE_PRINTED - 0x00000100 Page printed.
9	PRTALERT_CORE_JOB_CANCELLED - 0x00000200 Job cancelled inside printer.
10	PRTALERT_CORE_JOB_HELD - 0x00000400 Job held inside printer.
11	PRTALERT_CORE_COVER_OPEN - 0x00000800 Cover or panel open.
12	PRTALERT_CORE_POWER_ON - 0x00001000 Printer powered on.
13	PRTALERT_CORE_RESET - 0x00002000 Printer reset.
14	PRTALERT_CORE_ONLINE - 0x00004000 Printer now online.
15	PRTALERT_CORE_OFFLINE - 0x00008000 Printer now offline.
16	PRTALERT_CORE_TIMED_ALERT - 0x00010000 Timed alert.
17	PRTALERT_CORE_COMMUNICATION_PROBLEM - 0x00020000 Communication problem.
18	PRTALERT_CORE_COMM_STATUS_CHANGED - 0x00040000 Communication status changed.

pulHandle (PULONG) - in/out
Pointer to spooler alert handle.

For initial registration, the value pointed to by this parameter should be 0 (zero). If an application wants to make better use of spooler resources, the application can use the same returned spooler handle for all of its alert registration.

rc (ULONG) - returns
Return codes.

0	Success
ERROR_INVALID_CATEGORY(117)	The protocol converter does not support the extended alert category.
ERROR_INVALID_FLAG_NUMBER(186)	The protocol converter did not understand a core alert that was set.
ERROR_INVALID_HANDLE(6)	The value pointed to by *pulHandle* is not a valid spooler alert handle. The value at **pulHandle* should be set to 0 (zero) to allocate a new spooler alert handle.
ERROR_INVALID_PARAMETER(87)	An invalid buffer given.
ERROR_NOT_SUPPORTED(50)	The spooler on the given print server cannot support alert notification.
PMERR_SPL_SPOOLER_NOT_INSTALLED(0x4009)	The spooler is not enabled.

SplRegister - Remarks

Note:

Registration is **not** maintained across reboots.

Any new registration is in addition to existing registration(s) for the spooler handle passed in (if any handle was passed).

To remove registration, use SplUnRegister.

For each spooler alert handle, only the **last** *hwndNotify* passed in with the alert handle will be posted with alert messages.

That is, if you register window handle X and receive alert handle 1, then register for more alerts using window handle Y and alert handle 1, window handle Y will get alerts for alert handle 1, but window handle X will not get any alert messages.

Registration for alerts from a client on a print server requires that the user logged onto the client have authority to open a named pipe on the server. This access is normally granted to those users who have access to the print queue on a server.

SplRegister - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplRegisterControlPanel

SplRegisterControlPanel - Syntax

Description:

This function registers a DLL with the spooler that can display a control panel for printers. By default, all protocol converters will be checked for control panel support, so converters must not register themselves using this API.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

PSZ      pszName;      /*  Name of control panel. */
PSZ      pszDllPath;   /*  Fully qualified path to the DLL that exports the control panel APIs. */
ULONG    ulFlags;      /*  Must be 0 (zero). */
PSZ      pszReserved;  /*  Must be NULL. */
ULONG    ulVersion;    /*  Must be 0 (zero). */
ULONG    rc;           /*  Return codes. */

rc = SplRegisterControlPanel(pszName, pszDllPath,
       ulFlags, pszReserved, ulVersion);
```

SplRegisterControlPanel Parameter - pszName

pszName (PSZ) - input
Name of control panel.

This name must not conflict with any installed protocol converter name or control panel name.

SplRegisterControlPanel Parameter - pszDllPath

pszDllPath (PSZ) - input
Fully qualified path to the DLL that exports the control panel APIs.

SplRegisterControlPanel Parameter - ulFlags

ulFlags (ULONG) - input
Must be 0 (zero).

SplRegisterControlPanel Parameter - pszReserved

pszReserved (PSZ) - input
Must be NULL.

SplRegisterControlPanel Parameter - ulVersion

ulVersion (ULONG) - input
Must be 0 (zero).

SplRegisterControlPanel Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
ERROR_INVALID_LEVEL(124)	*ulVersion* not 0.
ERROR_INVALID_NAME(123)	*pszName* is already used by a protocol converter or another control panel DLL.
ERROR_INVALID_PARAMETER(87)	*ulFlags* not 0 or invalid parameter given.
ERROR_MOD_NOT_FOUND(126)	Invalid path to DLL.
ERROR_PROC_NOT_FOUND(127)	*pszDllPath* does not export the two APIs for control panels (apiref refid=spqucp form=textonly. and SplDisplayControlPanel).

SplRegisterControlPanel - Parameters

pszName (PSZ) - input
Name of control panel.

This name must not conflict with any installed protocol converter name or control panel name.

pszDllPath (PSZ) - input
Fully qualified path to the DLL that exports the control panel APIs.

ulFlags (ULONG) - input
Must be 0 (zero).

pszReserved (PSZ) - input
Must be NULL.

ulVersion (ULONG) - input
Must be 0 (zero).

rc (ULONG) - returns
Return codes.

0	Success
ERROR_INVALID_LEVEL(124)	*ulVersion* not 0.
ERROR_INVALID_NAME(123)	*pszName* is already used by a protocol converter or another control panel DLL.
ERROR_INVALID_PARAMETER(87)	*ulFlags* not 0 or invalid parameter given.
ERROR_MOD_NOT_FOUND(126)	Invalid path to DLL.
ERROR_PROC_NOT_FOUND(127)	*pszDllPath* does not export the two APIs for control panels (apiref refid=spqucp form=textonly. and SplDisplayControlPanel).

SplRegisterControlPanel - Remarks

A control panel need register itself only once; registration is maintained across reboots.

SplRegisterControlPanel - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplSetPort

SplSetPort - Syntax

Description:

This function sets the BIDI capabilities of a printer port.

This function is used by the print object or other applications to enable or disable BIDI communications with a printer. If the port driver is capable of BIDI communications, SplSetPort can override the port driver and disable BIDI with the printer. This is useful in case there are problems with the BIDI components.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

PSZ      pszComputerName;  /*  Name of the computer connected to the port. */
```

```
PSZ      pszPortName;       /*  Name of the port to set. */
ULONG    ulLevel;           /*  Level of information to set. */
PVOID    pBuf;              /*  Buffer containing new port setting. */
ULONG    cbBuf;             /*  Length of buffer pointed to by pBuf, in bytes. */
ULONG    ulParmNum;         /*  Parameter number used to change port settings. */
ULONG    rc;                /*  Return codes. */

rc = SplSetPort(pszComputerName, pszPortName,
       ulLevel, pBuf, cbBuf, ulParmNum);
```

SplSetPort Parameter - pszComputerName

pszComputerName (PSZ) - input
Name of the computer connected to the port.

This value can be NULL for a local machine.

SplSetPort Parameter - pszPortName

pszPortName (PSZ) - input
Name of the port to set.

SplSetPort Parameter - ulLevel

ulLevel (ULONG) - input
Level of information to set.

This must be 2.

SplSetPort Parameter - pBuf

pBuf (PVOID) - input
Buffer containing new port setting.

SplSetPort Parameter - cbBuf

cbBuf (ULONG) - input
Length of buffer pointed to by *pBuf,* in bytes.

SplSetPort Parameter - ulParmNum

ulParmNum (ULONG) - input
Parameter number used to change port settings.

Parameter number must be one of the following:

Value	Description
1	PRPO_PORT_DRIVER Port driver. This will be the new port driver for the printer port.
2	PRPO_PROTOCOL_CNV Protocol converter. This will override the values returned by BIDI_Q_PORT. The spooler will use this protocol converter when using this port in BIDI mode.
3	PRPO_MODE BIDI mode of the port. The ULONG value pointed to by *pBuf* must be one of the following values:

Value	Description
1	PRPORT_AUTODETECT Auto-detect mode. The spooler will use the results of BIDI_Q_PORT to decide if BIDI should be enabled and which protocol converter to use. This is the default.
2	PRPORT_DISABLE_BIDI BIDI disabled. The spooler will not use the printer in BIDI mode. This overrides the port driver.
3	PRPORT_ENABLE_BIDI BIDI enabled. This is set only if an application knows a printer is capable of using a protocol converter, but the port driver does not know which converter to use. The application must also supply the protocol converter name when setting a port to this mode.

4	PRPO_PRIORITY Priority. Currently not supported.

SplSetPort Return Value - rc

rc (ULONG) - returns
Return codes.

0	Success
ERROR_INVALID_PARAMETER(87)	An invalid parameter was specified; most likely an invalid *pBuf* value or an invalid *ulParmNum* value.
ERROR_NOT_SUPPORTED(50)	Command not supported on *pszComputerName*.
NERR_DestNotFound(2152)	The port name does not exist.

SplSetPort - Parameters

pszComputerName (PSZ) - input
Name of the computer connected to the port.

This value can be NULL for a local machine.

pszPortName (PSZ) - input
Name of the port to set.

ulLevel (ULONG) - input
Level of information to set.

This must be 2.

pBuf (PVOID) - input
Buffer containing new port setting.

cbBuf (ULONG) - input
Length of buffer pointed to by *pBuf*, in bytes.

ulParmNum (ULONG) - input
Parameter number used to change port settings.

Parameter number must be one of the following:

Value	Description
1	PRPO_PORT_DRIVER Port driver. This will be the new port driver for the printer port.
2	PRPO_PROTOCOL_CNV Protocol converter. This will override the values returned by BIDI_Q_PORT. The spooler will use this protocol converter when using this port in BIDI mode.
3	PRPO_MODE BIDI mode of the port. The ULONG value pointed to by *pBuf* must be one of the following values:

Value	Description
1	PRPORT_AUTODETECT Auto-detect mode. The spooler will use the results of BIDI_Q_PORT to decide if BIDI should be enabled and which protocol converter to use. This is the default.
2	PRPORT_DISABLE_BIDI BIDI disabled. The spooler will not use the printer in BIDI mode. This overrides the port driver.
3	PRPORT_ENABLE_BIDI BIDI enabled. This is set only if an application knows a printer is capable of using a protocol converter, but the port driver does not know which converter to use. The application must also supply the protocol converter name when setting a port to this mode.

4	PRPO_PRIORITY Priority. Currently not supported.

rc (ULONG) - returns
Return codes.

0	Success
ERROR_INVALID_PARAMETER(87)	An invalid parameter was specified; most likely an invalid *pBuf* value or an invalid *ulParmNum* value.
ERROR_NOT_SUPPORTED(50)	Command not supported on *pszComputerName*.
NERR_DestNotFound(2152)	The port name does not exist.

SplSetPort - Remarks

None.

SplSetPort - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplStdClose

SplStdClose - Syntax

Description:

SplStdClose closes the PM_Q_STD buffer.

```
#include <os2.h>

HDC     hdc;        /*  The device context handle. */
BOOL    fSuccess;   /*  Return codes. */

fSuccess = SplStdClose(hdc);
```

SplStdClose Parameter - hdc

hdc (HDC) - input
The device context handle.

SplStdClose Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess): .

TRUE	Success
FALSE	Error

SplStdClose - Parameters

hdc (HDC) - input
The device context handle.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess): .

TRUE	Success
FALSE	Error

SplStdClose - Remarks

The call to SplStdClose is made from the presentation driver's BeginCloseDC routine when the device type is OD_QUEUED and the data type is PM_Q_STD. SplStdClose must not be called at any other time.

SplStdClose - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplStdDelete

SplStdDelete - Syntax

Description:

SplStdDelete deletes the PM_Q_STD buffer identified by hstd. Any data in the buffer is lost.

```
#include <os2.h>
```

```
HSTD    hstd;       /*  The handle to PM_Q_STD data. */
BOOL    fSuccess;   /*  Return codes. */

fSuccess = SplStdDelete(hstd);
```

SplStdDelete Parameter - hstd

hstd (HSTD) - input
The handle to PM_Q_STD data.

SplStdDelete Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess): .

TRUE	Success
FALSE	Error

SplStdDelete - Parameters

hstd (HSTD) - input
The handle to PM_Q_STD data.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess): .

TRUE	Success
FALSE	Error

SplStdDelete - Remarks

None.

SplStdDelete - Topics

Select an item:
Syntax
Parameters

Returns
Remarks
Glossary

SplStdGetBits

SplStdGetBits - Syntax

Description:

SplStdGetBits transfers data from the identified PM_Q_STD buffer into a buffer owned by the presentation driver.

```
#include <os2.h>

HSTD    hstd;       /*  The handle to PM_Q_STD data. */
LONG    lStart;
LONG    cBytes;     /*  The number of bytes to transfer. */
PCH     pAddress;
BOOL    fSuccess;   /*  Return codes. */

fSuccess = SplStdGetBits(hstd, lStart, cBytes,
             pAddress);
```

SplStdGetBits Parameter - hstd

hstd (HSTD) - input
The handle to PM_Q_STD data.

SplStdGetBits Parameter - lStart

lStart (LONG) - input
Offset to the byte at which transfer must start. Used when the data is obtained in a series of calls to SplStdGetBits.

SplStdGetBits Parameter - cBytes

cBytes (LONG) - input
The number of bytes to transfer.

SplStdGetBits Parameter - pAddress

pAddress (PCH) - input
A pointer to the presentation driver's data buffer.

SplStdGetBits Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Success
FALSE	Error

SplStdGetBits - Parameters

hstd (HSTD) - input
The handle to PM_Q_STD data.

lStart (LONG) - input
Offset to the byte at which transfer must start. Used when the data is obtained in a series of calls to SplStdGetBits.

cBytes (LONG) - input
The number of bytes to transfer.

pAddress (PCH) - input
A pointer to the presentation driver's data buffer.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Success
FALSE	Error

SplStdGetBits - Remarks

Before calling SplStdGetBits, the presentation driver calls SplStdQueryLength to determine the length of the PM_Q_STD data. Depending upon the length, the hardcopy driver allocates a buffer large enough to contain all of the data, or allocates a smaller buffer and receives the data in a series of calls to SplStdGetBits.

SplStdGetBits - Topics

Select an item:

Syntax
Parameters
Returns
Remarks
Glossary

SplStdOpen

SplStdOpen - Syntax

Description:

SplStdOpen opens a spool buffer for PM_Q_STD data.

```
#include <os2.h>

HDC     hdc;        /*  The device context handle. */
BOOL    fSuccess;   /*  Return codes. */

fSuccess = SplStdOpen(hdc);
```

SplStdOpen Parameter - hdc

hdc (HDC) - input
The device context handle.

SplStdOpen Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess): .

TRUE	Success
FALSE	Error

SplStdOpen - Parameters

hdc (HDC) - input

The device context handle.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess): .

TRUE	Success
FALSE	Error

SplStdOpen - Remarks

The call to SplStdOpen is made from the hardcopy driver's CompleteOpenDC routine when the device type is OD_QUEUED and the data type is PM_Q_STD. SplStdOpen must not be called at any other time.

SplStdOpen - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplStdQueryLength

SplStdQueryLength - Syntax

Description:

SpStdQueryLength returns the number of data bytes in the PM_Q_STD buffer identified by *hstd*.

```
#include <os2.h>

HSTD    hstd;  /*  The handle to the PM_Q_STD buffer. */
LONG    rc;    /*  Return codes. */

rc = SplStdQueryLength(hstd);
```

SplStdQueryLength Parameter - hstd

hstd (HSTD) - input

The handle to the PM_Q_STD buffer.

SplStdQueryLength Return Value - rc

rc (LONG) - returns
Return codes.

This function returns the size of *hstd* (cBytes) or SPL_ERROR, if an error occurred.

SplStdQueryLength - Parameters

hstd (HSTD) - input
The handle to the PM_Q_STD buffer.

rc (LONG) - returns
Return codes.

This function returns the size of *hstd* (cBytes) or SPL_ERROR, if an error occurred.

SplStdQueryLength - Remarks

None.

SplStdQueryLength - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplStdStart

SplStdStart - Syntax

Description:

SplStdStart starts the recording of GPI and DevEscape calls as a metafile in the PM_Q_STD buffer.

```
#include <os2.h>

HDC     hdc;        /*  The device context handle. */
BOOL    fSuccess;   /*  Return codes. */

fSuccess = SplStdStart(hdc);
```

SplStdStart Parameter - hdc

hdc (HDC) - input
The device context handle.

SplStdStart Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess):

TRUE	Success
FALSE	Error

SplStdStart - Parameters

hdc (HDC) - input
The device context handle.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess):

TRUE	Success
FALSE	Error

SplStdStart - Remarks

The calls are still processed by the system and passed on to the graphics engine and hardcopy driver as calls to the relevant Gre*xxx* functions.

The hardcopy driver's Escape routine usually calls SplStdStart when DEVESC_STARTDOC is received. Some previous versions of presentation drivers made this call from the CompleteOpenDC routine to accommodate applications that did not call DevEscape(DEVESC_STARTDOC) at the start of the document.

SplStdStart - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplStdStop

SplStdStop - Syntax

Description:

SplStdStop stops the recording of GPI and DevEscape calls in the PM_Q_STD buffer. The hardcopy driver's Escape routine calls SplStdStop when DEVESC_ENDDOC is received.

```
#include <os2.h>

HDC     hdc;  /*  The device context handle. */
HSTD    rc;   /*  Return codes. */

rc = SplStdStop(hdc);
```

SplStdStop Parameter - hdc

hdc (HDC) - input
The device context handle.

SplStdStop Return Value - rc

rc (HSTD) - returns
Return codes.

If successful, this function returns the handle to the buffer (hstd) that contains the recorded GPI and DevEscape data.

If an error occurs, the function returns SPL_ERROR.

SplStdStop - Parameters

hdc (HDC) - input
The device context handle.

rc (HSTD) - returns
Return codes.

If successful, this function returns the handle to the buffer (hstd) that contains the recorded GPI and DevEscape data.

If an error occurs, the function returns SPL_ERROR.

SplStdStop - Remarks

None.

SplStdStop - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SplUnRegister

SplUnRegister - Syntax

Description:

This function allows applications to unregister for notification of events about printers. This function should be used when an application no longer wants certain alerts, in order to limit the network traffic.

```
#define INCL_SPL
#define INCL_SPLBIDI
#include <os2.h>

PSZ      pszComputerName;  /*  Printer server name to unregister for alert notification. */
PSZ      pszName;          /*  Case-sensitive name of printer port to unregister for alerts. */
ULONG    ulHandle;         /*  Spooler alert handle previously allocated by SplRegister. */
ULONG    ulCategory;       /*  Category of alert to unregister. */
ULONG    ulType;           /*  Type of alert to unregister for the given category. */
ULONG    rc;               /*  Return codes. */
```

```
rc = SplUnRegister(pszComputerName, pszName,
       ulHandle, ulCategory, ulType);
```

SplUnRegister Parameter - pszComputerName

pszComputerName (PSZ) - input
Printer server name to unregister for alert notification.

This value is NULL for local printer ports.

SplUnRegister Parameter - pszName

pszName (PSZ) - input
Case-sensitive name of printer port to unregister for alerts.

This value can be a virtual port name.

SplUnRegister Parameter - ulHandle

ulHandle (ULONG) - input
Spooler alert handle previously allocated by SplRegister.

SplUnRegister Parameter - ulCategory

ulCategory (ULONG) - input
Category of alert to unregister.

This value can be the core alert category or an extended alert category as defined below:

PRTALERT_CATEGORY_ALL - 0x00000000
Unregister for all alerts for the given alert handle. This will remove registration for all printer ports (even remote ones) for the alert handle.

PRTALERT_CATEGORY_ALL_CORE - 0x00000002
Unregister for all core alerts for the port.

PRTALERT_CATEGORY_ALL_EXT - 0x00000003
Unregister for all extended alerts for the port.

PRTALERT_CATEGORY_CORE - 0x00000001
Core alert category. This will remove registration for only those core alerts whose bits are set in *ulType* for the port.

PRTALERT_CATEGORY_EXT_MAX - 0x0000DFFF
End of the extended alert categories.

PRTALERT_CATEGORY_EXTENDED - 0x0000D000
Beginning of the extended alert categories. Protocol converters should limit the category range to between 0x0000D000 and 0x0000DFFFF.

If the protocol converter does not support the given category, ERROR_INVALID_CATEGORY(117) is returned.

SplUnRegister Parameter - ulType

ulType (ULONG) - input
Type of alert to unregister for the given category.

If the category is PRTALERT_CATEGORY_CORE, then this is a bit-field that defines the core alerts to unregister.

If the category is **not** PRTALERT_CATEGORY_ALL, PRTALERT_CATEGORY_CORE, PRTALERT_CATEGORY_ALL_CORE, or PRTALERT_CATEGORY_ALL_EXT, this is a value that defines the specific extended alert to unregister.

SplUnRegister Return Value - rc

rc (ULONG) - returns
Return codes.

0
Success

ERROR_FILE_NOT_FOUND(2)
pszName is not a valid port name.

ERROR_INVALID_HANDLE(6)
The value *ulHandle* is not a valid spooler alert handle.

ERROR_INVALID_PARAMETER(87)
An invalid buffer given.

ERROR_NOT_SUPPORTED(50)
The spooler on the given print server cannot support alert notification.

PMERR_SPL_NOT_REGISTERED(0x4042)
The alert handle *ulHandle* is not currently registered for any alerts specified by *ulCategory* and *ulType* .

PMERR_SPL_SPOOLER_NOT_INSTALLED(0x4009)
The spooler is not enabled.

SplUnRegister - Parameters

pszComputerName (PSZ) - input
Printer server name to unregister for alert notification.

This value is NULL for local printer ports.

pszName (PSZ) - input
Case-sensitive name of printer port to unregister for alerts.

This value can be a virtual port name.

ulHandle (ULONG) - input
Spooler alert handle previously allocated by SplRegister.

ulCategory (ULONG) - input
Category of alert to unregister.

This value can be the core alert category or an extended alert category as defined below:

PRTALERT_CATEGORY_ALL - 0x00000000
Unregister for all alerts for the given alert handle. This will remove registration for all printer ports (even remote ones) for the alert handle.

PRTALERT_CATEGORY_ALL_CORE - 0x00000002
Unregister for all core alerts for the port.

PRTALERT_CATEGORY_ALL_EXT - 0x00000003
Unregister for all extended alerts for the port.

PRTALERT_CATEGORY_CORE - 0x00000001
Core alert category. This will remove registration for only those core alerts whose bits are set in *ulType* for the port.

PRTALERT_CATEGORY_EXT_MAX - 0x0000DFFF
End of the extended alert categories.

PRTALERT_CATEGORY_EXTENDED - 0x0000D000
Beginning of the extended alert categories. Protocol converters should limit the category range to between 0x0000D000 and 0x0000DFFFF.

If the protocol converter does not support the given category, ERROR_INVALID_CATEGORY(117) is returned.

ulType (ULONG) - input
Type of alert to unregister for the given category.

If the category is PRTALERT_CATEGORY_CORE, then this is a bit-field that defines the core alerts to unregister.

If the category is **not** PRTALERT_CATEGORY_ALL, PRTALERT_CATEGORY_CORE, PRTALERT_CATEGORY_ALL_CORE, or PRTALERT_CATEGORY_ALL_EXT, this is a value that defines the specific extended alert to unregister.

rc (ULONG) - returns
Return codes.

0 Success

ERROR_FILE_NOT_FOUND(2)
pszName is not a valid port name.

ERROR_INVALID_HANDLE(6)
The value *ulHandle* is not a valid spooler alert handle.

ERROR_INVALID_PARAMETER(87)
An invalid buffer given.

ERROR_NOT_SUPPORTED(50)
The spooler on the given print server cannot support alert notification.

PMERR_SPL_NOT_REGISTERED(0x4042)
The alert handle *ulHandle* is not currently registered for any alerts specified by *ulCategory* and *ulType* .

PMERR_SPL_SPOOLER_NOT_INSTALLED(0x4009)
The spooler is not enabled.

SplUnRegister - Remarks

Registration is **not** maintained across reboots.

SplUnRegister - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SSAllocMem

SSAllocMem - Syntax

Description:

SSAllocMem allocates a shared memory object that is managed by the selector server component of the graphics engine.

```
#include <os2.h>

PVOID    pBaseAddress;
ULONG    ObjectSize;
ULONG    Flags;          /*  Reserved. */
ULONG    rc;             /*  Return codes. */

rc = SSAllocMem(pBaseAddress, ObjectSize,
       Flags);
```

SSAllocMem Parameter - pBaseAddress

pBaseAddress (PVOID) - input
Pointer to a variable, which will receive the base address of the allocated memory.

SSAllocMem Parameter - ObjectSize

ObjectSize (ULONG) - input
Value that specifies the size in bytes of the shared memory to allocate. The size is rounded up to the next page boundary.

SSAllocMem Parameter - Flags

Flags (ULONG) - input
Reserved.

Reserved. Must be *0* .

SSAllocMem Return Value - rc

rc (ULONG) - returns
Return codes.

SSAllocMem returns a ULONG value:

NO_ERROR
ERROR_INVALID_PARAMETER
ERROR_NOT_ENOUGH_MEMORY.

Note: Other return codes listed under DosAllocSharedMem might also apply.

SSAllocMem - Parameters

pBaseAddress (PVOID) - input
Pointer to a variable, which will receive the base address of the allocated memory.

ObjectSize (ULONG) - input
Value that specifies the size in bytes of the shared memory to allocate. The size is rounded up to the next page boundary.

Flags (ULONG) - input
Reserved.

Reserved. Must be *0* .

rc (ULONG) - returns
Return codes.

SSAllocMem returns a ULONG value:

NO_ERROR
ERROR_INVALID_PARAMETER
ERROR_NOT_ENOUGH_MEMORY.

Note: Other return codes listed under DosAllocSharedMem might also apply.

SSAllocMem - Remarks

Presentation drivers should use SSAllocMem to allocate memory for objects, such as bit maps and regions, to ensure that the returned memory object is a *global* memory object for which the system can set a new owner, or which can be marked as having no owner.

In some presentation devices, the handling routine for FillLogicalDeviceBlock (OS2_PM_DRV_ENABLE, sub-function 2) allocates global heap space for use by the device contexts. The memory for this heap space is obtained by calling the SSAllocMem function.

SSAllocMem - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

SSFreeMem

SSFreeMem - Syntax

Description:

SSFreeMem frees shared memory that was allocated by a call to SSAllocMem.

```
#include <os2.h>

PVOID    BaseAddress;  /*  Base address */
ULONG    rc;           /*  Return codes. */

rc = SSFreeMem(BaseAddress);
```

SSFreeMem Parameter - BaseAddress

BaseAddress (PVOID) - input
Base address

Base address of the memory object to be freed.

SSFreeMem Return Value - rc

rc (ULONG) - returns
Return codes.

SSFreeMem returns a ULONG value:

NO_ERROR
ERROR_ACCESS_DENIED
ERROR_INVALID_PARAMETER.

SSFreeMem - Parameters

BaseAddress (PVOID) - input
Base address

Base address of the memory object to be freed.

rc (ULONG) - returns
Return codes.

SSFreeMem returns a ULONG value:

NO_ERROR
ERROR_ACCESS_DENIED
ERROR_INVALID_PARAMETER.

SSFreeMem - Topics

Select an item:
Syntax
Parameters
Returns
Glossary

VisRegionNotify

VisRegionNotify - Syntax

Description:

VisRegionNotify recalculates the visible region and validates the DC region.

```
#include <os2.h>

ULONG    hdc;       /*  Device context handle. */
BOOL     fSuccess;  /*  Return codes. */

fSuccess = VisRegionNotify(hdc);
```

VisRegionNotify Parameter - hdc

hdc (ULONG) - input
Device context handle.

VisRegionNotify Return Value - fSuccess

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

VisRegionNotify - Parameters

hdc (ULONG) - input
Device context handle.

fSuccess (BOOL) - returns
Return codes.

This function returns BOOLEAN (fSuccess).

TRUE	Successful
FALSE	Error

VisRegionNotify - Remarks

The handling routine in the graphics engine does the necessary calculations and calls the GreNotifyClipChange routine in the presentation driver so that the driver can receive the new clip regions.

VisRegionNotify is used only by display drivers. These drivers maintain an HDC_IS_DIRTY flag that is set ON by GreDeviceInvalidateClipRegion and cleared by GreNotifyClipChange. In the display driver, the handling routines for all drawing functions should check the HDC_IS_DIRTY flag, and if it is set, call VisRegionNotify before drawing.

VisRegionNotify is available across the system at Ring 3. Presentation drivers do not need to import this function through the module definition file.

VisRegionNotify - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

WinSetErrorInfo

WinSetErrorInfo - Syntax

Description:

*WinSetErrorInf*o posts an error message, which can be retrieved by an application that calls WinGetLastError.

```
#include <os2.h>

ERRORID     idError;       /*  An ERRORID structure */
ULONG       fsOptions;     /*  Option flags: */
ULONG       arg1...argN;   /*  Variable number of optional arguments. */

WinSetErrorInfo(idError, fsOptions, arg1...argN);
```

WinSetErrorInfo Parameter - idError

idError (ERRORID) - input
An ERRORID structure

An ERRORID structure with the following values:

lSeverity	Valid values are: SEVERITY_ERROR SEVERITY_NOERROR SEVERITY_SEVERE SEVERITY_WARNING SEVERITY_UNRECOVERABLE
lErrorCode	Error code. See Presentation Manager Error Codes for a list of defined values.

WinSetErrorInfo Parameter - fsOptions

fsOptions (ULONG) - input
Option flags:

4000H	Do not call DosBeep.
2000H	Do not prompt the user.
0008H	The next parameter is a base OS/2 error code.
0004H	fsOptions must be set to 0004H to use the variable arguments arg1...argN. arg1 must contain the number of arguments that follow, not including arg1 itself. Note that fsOptions cannot equal 000CH; that is, DOSERROR and argcount are mutually exclusive.

WinSetErrorInfo Parameter - arg1...argN

arg1...argN (ULONG) - input
Variable number of optional arguments.

These parameters have no significance except when the presentation driver is reporting a base OS/2 error. The two examples that follow show how WinSetErrorInfo is used to post a presentation driver error and a base OS/2 error:

Example #1. To post a presentation driver error:

```
WinSetErrorInfo (ERRORID (SEVERITY_ERROR, PMERR_INV_COORDINATE), NOBEEP);
```

Example #2. To post a base OS/2 error:

```
WinSetErrorInfo (ERRORID (SEVERITY_WARNING, ErrorCode),
(DOSERROR+NOBEEP+NOPROMPT), DosErrorCode);
```

WinSetErrorInfo - Return Value

There is no return value for this function.

WinSetErrorInfo - Parameters

idError (ERRORID) - input
An ERRORID structure

An ERRORID structure with the following values:

lSeverity	Valid values are: SEVERITY_ERROR SEVERITY_NOERROR SEVERITY_SEVERE SEVERITY_WARNING SEVERITY_UNRECOVERABLE
lErrorCode	Error code. See Presentation Manager Error Codes for a list of defined values.

fsOptions (ULONG) - input
Option flags:

4000H	Do not call DosBeep.
2000H	Do not prompt the user.
0008H	The next parameter is a base OS/2 error code.
0004H	fsOptions must be set to 0004H to use the variable arguments arg1...argN. arg1 must contain the number of arguments that follow, not including arg1 itself. Note that fsOptions cannot equal 000CH; that is, DOSERROR and argcount are mutually exclusive.

arg1...argN (ULONG) - input
Variable number of optional arguments.

These parameters have no significance except when the presentation driver is reporting a base OS/2 error. The two examples that follow show how WinSetErrorInfo is used to post a presentation driver error and a base OS/2 error:

Example #1. To post a presentation driver error:

```
WinSetErrorInfo (ERRORID (SEVERITY_ERROR, PMERR_INV_COORDINATE), NOBEEP);
```

Example #2. To post a base OS/2 error:

```
WinSetErrorInfo (ERRORID (SEVERITY_WARNING, ErrorCode),
(DOSERROR+NOBEEP+NOPROMPT), DosErrorCode);
```

There is no return value for this function.

WinSetErrorInfo - Remarks

The syntax of the call to WinSetErrorInfo allows a variable number of parameters. The minimum requirement is the appropriate ERRORID structure and a NULL value for fsOptions.

Note: Presentation drivers do not need to import this function through the module definition file.

WinSetErrorInfo - Topics

Select an item:
Syntax
Parameters
Returns
Remarks
Glossary

BIDI Command Structures and Command Flow

All structures used by PrtQuery and PrtSet use offsets into the return buffer for variable-length data, instead of returning pointers. These offsets are used to make transmission of the bidirectional structures easier between processes and machines.

For example, in a structure that returns a description string, the field that references the storage location of this string is a ULONG. If zero, that string is not returned. If nonzero, the field is the offset, from the beginning of the output buffer, to the description string.

All structures with variable-length data should pack this data immediately following all the fixed-length structures. As a result, **pcbOutData* contains the number of bytes to copy from the beginning of the *pOutData* buffer.

Note: All returned structure-offset fields are offsets from the beginning of the output buffer given, not offsets from the beginning of the structure.

The Query and Set command structures are listed in the following two tables. The tables show which part of the system is called for each command structure. The values indicate the following:

- A **PrCo** in the "Sent To" column indicates that the protocol converter is called first.
- A **PDrv** indicates that the port driver is called first.
- If both are called, they are listed in the order called.

The numbers in the "Init Sequence #/Description" column indicate, across both the Query and the Set tables, the order in which these command structures are called.

Query Command	Command Code	First Sent To: PrCo/PDrv	Init Sequence # and Description
BIDI_Q_CONVERTER_INFO	(8021h)	PrCo	
BIDI_Q_Device	(800Dh)	PrCo; PDrv	
BIDI_Q_FONTS	(8012h)	PrCo; PDrv	
BIDI_Q_INPUTBINS	(800Fh)	PrCo; PDrv	
BIDI_Q_INTERPRETER	(800Eh)	PrCo; PDrv	
BIDI_Q_JOBID	(8017h)	PrCo	During PrtOpen, after BIDI_STARTJOB
BIDI_Q_JOBS_COMPLETE	(8013h)	PrCo; PDrv	
BIDI_Q_JOBS_QUEUED	(8014h)	PrCo; PDrv	
BIDI_Q_OUTPUTBINS	(8010h)	PrCo; PDrv	
BIDI_Q_PORT	(800Bh)	PDrv	2 - Determines protocol converter to use
BIDI_Q_PORTDRV	(8019h)	PDrv	Free-format buffer
BIDI_Q_RESPONSE_FMT	(8018h)	PrCo	5 - Gets printer msg format
BIDI_Q_SPOOLER_VERSION	(8022h)		Handled by spooler
BIDI_Q_STATUS	(8015h)	PrCo; PDrv	
BIDI_Q_STORAGE	(8023h)	PrCo; PDrv	
BIDI_Q_SW	(800Ch)	PrCo	7
BIDI_READ_ALERT	(801Dh)	PrCo	
BIDI_READ_PASSTHRU	(8001h)	PrCo	Copies SplProtXlateCmd data
BIDI_WAIT_ALERT	(8016h)	PDrv	3 - One thread per port driver

Set Command	Command Code	First Sent To: PrCo/PDrv	Init Sequence # and Description
BIDI_ADD_VIRTUAL_PORT	(26h)	PDrv	
BIDI_CANCELJOB	(6h)	PrCo; PDrv	
BIDI_DEL_PORT	(28h)	PrCo; PDrv	
BIDI_DEL_VIRTUAL_PORT	(27h)	PDrv	
BIDI_DISABLE_ALERT	(25h)	PrCo; PDrv	
BIDI_ENABLE_ALERT	(24h)	PrCo; PDrv	9
BIDI_END_PASSTHRU	(1Bh)	PrCo	
BIDI_ENDJOB	(3h)	PrCo; PDrv	
BIDI_HOLDJOB	(4h)	PrCo; PDrv	

BIDI_INIT	(Bh)	PDrv	
BIDI_INIT_PORTDRV	(8h)	PDrv	1 - Once per PortDRV
BIDI_INIT_PROTCNV	(Fh)	PrCo	4 - Given Q_PORT for each port
BIDI_NOTIFY_ENDJOBCONNECT	(20h)	PDrv	
BIDI_NOTIFY_PORT_RELEASED	(22h)	PDrv	
BIDI_NOTIFY_PORT_SELECTED	(21h)	PDrv	10
BIDI_PACKET_SIZE	(Eh)	PrCo; PDrv	
BIDI_RELEASEJOB	(5h)	PrCo; PDrv	
BIDI_RESPONSE_FMT	(Dh)	PDrv	6
BIDI_RESET	(9h)	PrCo; PDrv	
BIDI_SEND_PASSTHRU	(1h)	PrCo; PDrv	
BIDI_SET_DEVICE_ID	(23h)	PDrv	
BIDI_SET_PORTDRV	(19h)	PDrv	Free-format buffer
BIDI_SET_SW	(10h)	PDrv	8
BIDI_SHUTDOWN	(Ah)	PrCo; PDrv	Sent to both protocol converter and port driver
BIDI_START_PASSTHRU	(1Ah)	PrCo	
BIDI_STARTJOB	(2h)	PrCo; PDrv	
BIDI_TERM	(Ch)	PrCo; PDrv	

Protocol Converters

A protocol converter is a DLL written to support printers that are capable of returning information to the host PC. A protocol converter is responsible for understanding the control language of the printer. Examples of control languages are as follows:

- Network Printer Alliance Protocol (NPAP)
- Printer Job Language (PJL)
- Simple Network Management Protocol (SNMP)

The file type of a protocol converter is ".CNV". The protocol converter does not get involved in the physical transmission of commands or data to the printer. The converter will generate appropriate printer commands to send to the printer, and the converter is able to translate the data sent from the printer to the host PC.

Developers can write a protocol converter if they have a port driver that can communicate with a group of printers that does not yet have a protocol converter. See the "Bidirectional Communications" chapter in the *Printer Device Driver Reference for OS/2* for more information.

The functions exported from a protocol converter are as follows:

- SplProtSendCmd
- SplProtXlateCmd

Note: SplProtWrite is an optional function that can be exported from a protocol converter.

A protocol converter is installed in a system by adding the following entry to the system .INI file:

AppName = PM_PROTOCOL_CONVERTER

KeyName = ConverterName
Example: "PJL", "NPAP"

KeyValue = Path to converter
Example: "C:\OS2\DLL\PROT.CNV"

Printer Control Panels

A printer control panel is a DLL written to display the current status of a printer. Optionally, this DLL also allows a user to change the front panel settings for a printer that can communicate bidirectionally with the OS/2 print subsystem.

This printer panel DLL is called when the user selects the "Printer Panel" menu item on a print object that is connected to a printer communicating bidirectionally with the OS/2 print subsystem.

The file type of a printer control panel is ".DLL". The printer control panels make use of the PrtQuery and PrtSet APIs to get current status of printers. See the "Bidirectional Communications" chapter in the *Printer Device Driver Reference for OS/2* for more information.

The functions exported from a printer control panel are as follows:

- SplQueryControlPanel
- SplRegisterControlPanel

A protocol converter is installed in a system by calling SplRegisterControlPanel. This adds the following entry to the system .INI file:

AppName = PM_SPOOLER_CONTROL_PANEL

KeyName = ControlPanelName
Example: "NewPrinter"

KeyValue = Path to DLL with ctrl panel
Example: "C:\OS2\DLL\CPANEL.DLL"

The print object typically calls SplGetControlPanelList, which causes the spooler to call the API SplQueryControlPanel for each installed printer control panel, in order to determine if the printer panel DLL can display a control panel for the given printer.

The print object gives the list of printer panel names to the user and, after one is selected, the print object will call SplDisplayControlPanel.

BIDI_ADD_VIRTUAL_PORT(26h)

BIDI_ADD_VIRTUAL_PORT(26h) adds a virtual printer port. This command is sent to the port driver to notify it that a port for querying purposes only has been added. The port driver does not know about the printer port passed into SplPdSet when BIDI_ADD_VIRTUAL_PORT(26h) is sent.

Parameters

- **pInData** (PVOID) - input

  Points to the buffer passed into SplCreatePort. This buffer is passed by an application attempting to add a virtual printer port. The contents of the buffer are defined by the port driver being used.

- **pOutData** (PVOID) - input

  Not used.

Remarks

None.

BIDI_CANCELJOB(6h)

BIDI_CANCELJOB(6h) cancels a job in the printer.

For all PRT_xx_JOB commands, if the caller does not know the printer job ID, the caller can set both *InterpreterID* and *ulPrinterJobID* to -1 and pass in the spooler job ID. The spooler will set the printer job ID based on information stored about the spooler job ID.

Parameters

- **pInData** (PVOID) - input

  Contains a PRTCONTROLJOB data structure.

- **pOutData** (PVOID) - input

  Not used.

Remarks

None.

BIDI_DEL_PORT(28h)

BIDI_DEL_PORT(28h) is sent to the protocol converter and the port driver **after** a spooler port has been removed, even if the port driver is the application removing the port. It is **not** sent when a virtual port is removed. The protocol converter and port driver can free any instance data stored for the printer port.

Note: Port drivers are normally called to remove a port by the print object, but any application can remove a port by updating the system .INI file.

Parameters

- **pInData** (PVOID) - input

  Not used.

- **pOutData** (PVOID) - input

  Not used.

Remarks

None.

BIDI_DEL_VIRTUAL_PORT(27h)

BIDI_DEL_VIRTUAL_PORT(27h) deletes a virtual printer port. This command is sent to the port driver when an application calls SplDeletePort to remove a virtual port.

Parameters

- **pInData** (PVOID) - input

  Not used.

- **pOutData** (PVOID) - input

  Not used.

Remarks

None.

BIDI_DISABLE_ALERT(25h)

BIDI_DISABLE_ALERT(25h) disables alerts for the printer. This command is sent as a result of an application calling SplUnRegister to unregister for alerts. Only the alert(s) specified are disabled.

Parameters

- **pInData** (PVOID) - input

  Contains a PRTSETALERT data structure.

- **pOutData** (PVOID) - input

  Not used.

Remarks

None.

BIDI_ENABLE_ALERT(24h)

BIDI_ENABLE_ALERT(24h) enables alerts for the printer. This command is sent as a result of an application calling SplRegister to register for alerts. The alert(s) specified are enabled, in addition to any currently enabled alerts for the printer.

Parameters

- **pInData** (PVOID) - input

  Contains a PRTSETALERT data structure.

- **pOutData** (PVOID) - input

  Not used.

Remarks

None.

BIDI_END_PASSTHRU(1Bh)

BIDI_END_PASSTHRU(1Bh) terminates a passthru reading session. This command releases the port's passthru session and allows another application to start a temporary passthru session.

Parameters

- **pInData** (PVOID) - input

  Not used.

- **pOutData** (PVOID) - input

  Not used.

Remarks

None.

BIDI_ENDJOB(3h)

BIDI_ENDJOB(3h) is called by the port driver during SplPdClose to signal the end of the print job being sent to the printer.

Note: For this and all other BIDI_xx_JOB command structures, the caller of PrtSet can set the following parameters in the PRTJOB data structure:

- *ulInterpreterID* - set to -1
- *ulPrinterJobID* - set to -1

This will let the spooler fill in the values based on the current job being sent to this printer port.

Parameters

- **pInData** (PVOID) - input

  Contains a PRTJOB data structure.

- **pOutData** (PVOID) - input

  Not used.

Remarks

None.

BIDI_HOLDJOB(4h)

BIDI_HOLDJOB(4h) holds a job in the printer.

Parameters

- **pInData** (PVOID) - input

  Contains a PRTCONTROLJOB data structure.

- **pOutData** (PVOID) - input

  Not used.

Remarks

None.

BIDI_INIT(Bh)

BIDI_INIT(Bh) sets the printer to BIDI mode and changes the printer from unidirectional to bidirectional. BIDI_INIT is called after a BIDI_TERM call to restart bidirectional printing.

Parameters

- **pInData** (PVOID) - input

  Not used.

- **pOutData** (PVOID) - input

  Not used.

Remarks

None.

BIDI_INIT_PORTDRV(8h)

BIDI_INIT_PORTDRV(8h) initializes a port driver. This command is sent during spooler initialization and when a port driver is first installed. BIDI_INIT_PORTDRV(8h) is called only once per port driver. No other commands are sent to the port driver until this command completes. As a result, a port driver that opens network adapters has the opportunity to do so while processing the BIDI_INIT_PORTDRV(8h) command.

A port driver can spawn threads at this time, if another thread is needed in order to keep the printer connection active.

Note: If the port driver returns an error, the spooler may unload the port driver. No jobs will be sent to this port driver until the BIDI_INIT_PORTDRV(8h) command succeeds.

Parameters

- **pInData** (PVOID) - input

  Not used.

- **pOutData** (PVOID) - input

  Not used.

Remarks

None.

BIDI_INIT_PROTCNV(Fh)

BIDI_INIT_PROTCNV(Fh) initializes the protocol converter for a port. The spooler passes the BIDI_Q_PORT command structure, returned by the port driver, to the protocol converter using BIDI_INIT_PROTCNV(Fh). The protocol converter should initialize its cache for the given port name.

Note: If a protocol converter or extension is unable to support the printer, it should return ERROR_NOT_SUPPORTED for BIDI_INIT_PROTCNV(Fh).

Parameters

- **pInData** (PVOID) - input

  Contains a PRTPORT data structure.

- **pOutData** (PVOID) - input

  Not used.

Remarks

None.

BIDI_NOTIFY_ENDJOBCONNECT(20h)

BIDI_NOTIFY_ENDJOBCONNECT(20h) notifies the port driver that a job connection is not needed. The spooler sends this command to notify the port driver that no jobs are waiting for confirmation from the printer.

BIDI_NOTIFY_ENDJOBCONNECT(20h) allows the port driver to know when the spooler is not currently using the printer for print jobs. It does **not** imply that the spooler will not use the printer again in the near future.

Parameters

- **pInData** (PVOID) - input

  Not used.

- **pOutData** (PVOID) - input

  Not used.

Remarks

None.

BIDI_NOTIFY_PORT_RELEASED(22h)

BIDI_NOTIFY_PORT_RELEASED(22h) notifies the port driver that the port is no longer selected in a print object. The spooler sends this command to notify the port driver that the output port is no longer connected to a print queue. BIDI_NOTIFY_PORT_SELECTED(21h) allows the port driver to know that the spooler will not be sending print jobs to this printer.

The port driver should not assume that no job can be sent to this port, because an application can still print OD_DIRECT to this port. BIDI_NOTIFY_PORT_RELEASED(22h) can assist the port driver in determining the type of connection necessary for the printer.

Parameters

- **pInData** (PVOID) - input

  Not used.

- **pOutData** (PVOID) - input

  Not used.

Remarks

None.

BIDI_NOTIFY_PORT_SELECTED(21h)

BIDI_NOTIFY_PORT_SELECTED(21h) notifies the port driver that the port is selected in a print object. The spooler sends this command to notify the port driver that the output port is selected by a print queue.

BIDI_NOTIFY_PORT_SELECTED(21h) allows the port driver to know that the spooler has a print queue connected to this printer and that jobs may be submitted to this printer. This command can assist the port driver in determining the type of connection necessary for the printer.

Parameters

- **pInData** (PVOID) - input

  Not used.

- **pOutData** (PVOID) - input

  Not used.

Remarks

None.

BIDI_PACKET_SIZE(Eh)

BIDI_PACKET_SIZE(Eh) defines the printer-to-host maximum packet size.

Parameters

- **ulPacketSize** (ULONG) - input

  Maximum size, in bytes, of a packet sent from the printer to the host.

- **pInData** (PVOID) - input

  Contains the value of *ulPacketSize*. For NPAP, the default maximum packet size is 64 bytes; it can be increased using BIDI_PACKET_SIZE(Eh).

- **pOutData** (PVOID) - input

  Not used.

Remarks

None.

BIDI_Q_CONVERTER_INFO(8021h)

BIDI_Q_CONVERTER_INFO(8021h) gets protocol converter information.

This function returns information about the protocol converter currently active for the printer. If an extension protocol converter is being used, the extension should call the base converter to get the information about the base converter. The extension converter should then copy the base converter version and strings into the caller's buffer, along with the extension's information.

Parameters

- **pInData** (PVOID) - input

  Not used.

- **pOutData** (PVOID) - output

  Contains a PRTCNVINFO data structure.

Remarks

None.

BIDI_Q_DEVICE(800Dh)

BIDI_Q_DEVICE(800Dh) queries the printer device characteristics. This function returns the capabilities and characteristics of the printer.

Parameters

- **pInData** (PVOID) - input

  Not used.

- **pOutData** (PVOID) - output

  Returns a PRTDEVCHAR data structure.

Remarks

None.

BIDI_Q_FONTS(8012h)

BIDI_Q_FONTS(8012h) queries fonts for a particular interpreter.

Parameters

- **pInData** (PVOID) - input

  Contains a PRTFONTS_Q data structure.

- **pOutData** (PVOID) - output

Returns a PRTFONTSL data structure.

Remarks

If the requested storage type is not available, PMERR_SPL_TYPE_NOT_AVAIL is returned.

BIDI_Q_INPUTBINS(800Fh)

BIDI_Q_INPUTBINS(800Fh) queries input bins.

Parameters

- **pInData** (PVOID) - input

  Contains a QBINS data structure.

- **pOutData** (PVOID) - output

  Returns a PRTINPUTSL data structure and a PRTINPUTS data structure.

Remarks

None.

BIDI_Q_INTERPRETER(800Eh)

BIDI_Q_INTERPRETER(800Eh) returns information on interpreters that are available in the printer.

Parameters

- **ulInterpreterID** (ULONG) - input

  ID of the interpreter. This value will be the logical unit number, or -1 for all interpreters.

- **pInData** (PVOID) - input

  Contains the interpreter ID.

- **pOutData** (PVOID) - output

Returns a PRTINTERPL data structure.

Remarks

None.

BIDI_Q_JOBID(8017h)

BIDI_Q_JOBID(8017h) returns the printer job ID for the job being sent to this port. This function call is issued by the spooler during PrtOpen, after a successful SplPdOpen to the port driver. It allows the spooler to synchronize the printer job ID with the spooler job ID.

Parameters

- **pInData** (PVOID) - input

  Not used.

- **pOutData** (PVOID) - output

  Returns a PRTJOB data structure.

Remarks

None.

BIDI_Q_JOBS_COMPLETE(8013h)

BIDI_Q_JOBS_COMPLETE(8013h) returns a list of completed jobs known to the printer.

Parameters

- **pInData** (PVOID) - input

  Contains a PRTJOBS_Q data structure.

- **pOutData** (PVOID) - output

  Returns a PRTJOBSL data structure and a PRTJOBS data structure.

Remarks

None.

BIDI_Q_JOBS_QUEUED(8014h)

BIDI_Q_JOBS_QUEUED(8014h) returns a list of queued jobs known to the printer.

Parameters

- **pInData** (PVOID) - input

  Contains a PRTJOBS_Q data structure.

- **pOutData** (PVOID) - output

  Returns a PRTJOBSL data structure and a PRTJOBS data structure.

Remarks

None.

BIDI_Q_OUTPUTBINS(8010h)

BIDI_Q_OUTPUTBINS(8010h) queries output bins.

Parameters

- **pInData** (PVOID) - input

  Contains a QBINS data structure.

- **pOutData** (PVOID) - output

  Returns a PRTOUTPUTSL data structure and a PRTOUTPUTS data structure.

Remarks

None.

BIDI_Q_PORT(800Bh)

BIDI_Q_PORT(800Bh) queries the port configuration from the port driver. This function returns BIDI capabilities of the port and the BIDI software protocol of the attached printer. It also performs an implicit BIDI_INIT(Bh) to the port driver to put the printer in BIDI mode.

BIDI_Q_PORT(800Bh) does not go to the BIDI protocol converter because the information returned is used to determine the BIDI protocol converter to be used for the port.

Parameters

- **pInData** (PVOID) - input

  Not used.

- **pOutData** (PVOID) - output

  Returns a PRTPORT data structure.

Remarks

None.

BIDI_Q_PORTDRV(8019h)

BIDI_Q_PORTDRV(8019h) queries the port driver's configuration for the port.

Parameters

- **pInData** (PVOID) - input

  Not used.

- **pOutData** (PVOID) - output

  Contains a free-format buffer.

  The format of the output buffer is known only to the port driver. BIDI_Q_PORTDRV(8019h) is called to get the port driver configuration data from a print server. This information enables a port driver to allow an administrator to change the port settings remotely via SplPdRemoteSetPort.

  This call also can be used during SplPdSetPort to allow the port driver dialog (which runs under the PMSHELL process) to get information from the part of the port driver that communicates with the printer (which runs under the spooler process).

Remarks

None.

BIDI_Q_RESPONSE_FMT(8018h)

BIDI_Q_RESPONSE_FMT(8018h) returns the format of the printer-to-host messages.

The spooler calls the protocol converter to get this information. The information is passed to the port driver as BIDI_RESPONSE_FMT(Dh), to let the port driver determine printer response message boundaries.

Parameters

- **pInData** (PVOID) - input

  Not used.

- **pOutData** (PVOID) - output

  Returns a PRTRESPONSE data structure.

Remarks

None.

BIDI_Q_SPOOLER_VERSION(8022h)

BIDI_Q_SPOOLER_VERSION(8022h) gets version information about the spooler.

This information can be used by an application to determine if the spooler on a remote server supports the PrtQuery and PrtSet APIs.

Parameters

- **pInData** (PVOID) - input

  Not used.

- **pOutData** (PVOID) - output

  Contains a PRTSPLRVER data structure.

Remarks

None.

BIDI_Q_STATUS(8015h)

BIDI_Q_STATUS(8015h) queries the protocol converter for the current status of the print device.

Parameters

- **pInData** (PVOID) - input

  Points to the value in *ulStatusFlags* to indicate the type of status to return.

- **ulStatusFlags** (ULONG) - input

  Type of status alerts to return. Values are as follows:

Bit	Description
0	PRTSTATUS_TYPE_INPUTS - 0x00000001 Inputs (0=No; 1=Yes)
1	PRTSTATUS_TYPE_OUTPUTS - 0x00000002 Outputs (0=No; 1=Yes)
2	PRTSTATUS_TYPE_JAMS - 0x00000004 Jams (0=No; 1=Yes)
3	PRTSTATUS_TYPE_OP_INTERV - 0x00000008 Operator intervention required (0=No; 1=Yes)
4	PRTSTATUS_TYPE_WARNINGS - 0x00000010 Warnings (0=No; 1=Yes)
5	PRTSTATUS_TYPE_SERVICE_REQ - 0x00000020 Service required (0=No; 1=Yes)
6	PRTSTATUS_TYPE_CONFIG_CHANGES - 0x00000040 Configuration changes (0=No; 1=Yes)
7	PRTSTATUS_TYPE_SUPPLIES - 0x00000080 Supplies (0=No; 1=Yes)
8	PRTSTATUS_TYPE_PRINTER_STATS - 0x00000100 Printer statistics (0=No; 1=Yes)
9 - 31	Reserved (set to 0)

- ***pcbOutData** (PVOID) - input/output

  Set to the length of data returned in *pOutData,* which returns a variable length structure.

- **pOutData** (PVOID) - output

  Returns a list of status fields and their values. The status fields are listed below:

```
BYTE        printerStatus;
BYTE        deviceAlerts1;
BYTE        deviceAlerts2;

BYTE        cInputStatus;

BYTE        InputId;
USHORT      InputStatus;

BYTE        cOutputStatus;

BYTE        OutputId;
USHORT      OutputStatus;

BYTE        cJamAlerts;

BYTE        JamLocationCode;
BYTE        JamLocationID;

BYTE        cOperatorAlerts;

BYTE        OperatorAlertCode;
BYTE        lOperatorAlertMsg;
CHAR        OperatorAlertMsg[];

BYTE        cWarningAlerts;

BYTE        WarningAlertCode;
BYTE        lWarningAlertMsg;
CHAR        WarningAlertMsg[];

BYTE        cServiceAlerts;

BYTE        ServiceAlertCode;
BYTE        lServiceAlertMsg;
CHAR        ServiceAlertMsg[];



BYTE        cConfigureAlerts;

BYTE        ConfigureAlertCode;
BYTE        lConfigureAlertMsg;
CHAR        ConfigureAlertMsg[];

BYTE        cSuppliesLevels;

BYTE        SuppliesLocation;
BYTE        SuppliesID;
USHORT      SuppliesLevel;

BYTE        cSuppliesAlerts;

BYTE        SuppliesAlertCode;
BYTE        lSuppliesAlertMsg;
CHAR        SuppliesAlertMsg[];

BYTE        cPrinterStatistics;

BYTE        PrinterStatisticsCode;
ULONG       PrinterStatisticsValue;
```

The specific flags and their values are listed below:

printerStatus	Overall printer status. Bit	Description
	0	PRTSTATUS_PROTCNV_HAS_STATUS - 0x01 Set if protocol converter can determine printer status. If clear, the remaining bits do not reflect the printer state.
	1	PRTSTATUS_POWER_ON - 0x02 Power on initialization (0=No; 1=Yes)
	2	PRTSTATUS_OFFLINE - 0x04

		Printer offline (0=No; 1=Yes)
	3	PRTSTATUS_PRINTER_BUSY - 0x08 Printer temporarily busy (0=No; 1=Yes)
	4	PRTSTATUS_NOT_ACCEPTING_DATA - 0x10 Unable to accept print job data (0=No; 1=Yes). If set, the spooler will not send a print job to this printer.
	5 - 7	Reserved
deviceAlerts1	Active device alerts 1. For each bit set, there is an active alert.	
	Bit	Description
	0	PRTSTATUS_DEV1_SUPPLY_ALERT - 0x01 Printing supply alert
	1	PRTSTATUS_DEV1_JAM_ALERT - 0x02 Paper jam alert
	2	PRTSTATUS_DEV1_OUTPUT_ALERT - 0x04 Output alert
	3	PRTSTATUS_DEV1_INPUT_ALERT - 0x08 Input alert
	4 - 7	Reserved
deviceAlerts2	Active device alerts 2. For each bit set, there is an active alert.	
	0	PRTSTATUS_DEV2_CONFIG_ALERT - 0x01 Configuration change alert
	1	PRTSTATUS_DEV2_WARNING_ALERT - 0x02 Warnings alert
	2	PRTSTATUS_DEV2_SERVICE_ALERT - 0x03 Device service-required alert
	3	PRTSTATUS_DEV2_OPERATOR_ALERT - 0x04 Operator intervention-required alert
	4 - 7	Reserved
cInputStatus	Count of input status alerts that follow. This value determines the number of times *InputID* and *InputStatus* are repeated. If 0 (zero), these fields are not returned.	
	InputID	Input identifier with this alert active.
	InputStatus	Current status of input, as follows:

Bit	Description
0 - 2	PRTSTATUS_LEVEL_MASK - 0x0007 Current

		input level. All zeros indicate the input bin is empty.
	3	PRTSTATUS_TRAY_MISSING - 0x0008 Tray missing (0=No; 1=Yes)
	4	PRTSTATUS_BROKEN - 0x0010 Unavailable/broken (0=No; 1=Yes)
	5	PRTSTATUS_INOUT_BUSY - 0x0020 Busy/temporarily unavailable (0=No; 1=Yes). Set when tray is allocated to some other process.
	6 - 14	Reserv

ed.

15	PRTSTATUS_ALERT_ACTIVE - 0x8000 Alert active (0=No; 1=Yes)

cOutputStatus

Count of output status alerts that follow. This value determines the number of times *OutputID* and *OutputStatus* are repeated. If 0 (zer0), these fields are not returned.

OutputID

Output identifier with this alert active.

OutputStatus

Current status of output, as follows:

Bit	Description
0 - 2	PRTSTATUS_LEVEL_MASK - 0x0007 Current output level. All zeros (0) indicate the output bin is empty. All ones (1) indicates the output bin is full.
3	PRTSTATUS_TRAY_MISSING - 0x0008 Tray missing (0=No;

				1=Yes)
			4	PRTSTATUS_BROKEN - 0x0010 Unavailable/broken (0=No; 1=Yes)
			5	PRTSTATUS_INOUT_BUSY - 0x0020 Busy/temporarily unavailable (0=No; 1=Yes). Set when tray is allocated to some other process.
			6 - 14	Reserved
			15	PRTSTATUS_ALERT_ACTIVE - 0x8000 Alert active (0=No; 1=Yes)
cJamAlerts	Count of printing-media-jam status alerts that follow. This value determines the number of times *JamLocationCode* and *JamLocationID* are repeated. If zero, these fields are not returned.			
	JamLocationCode			
		Indicates general location of jam, as follows:		
			Bit	

	Description
00	Reserved
01	PRTSTATUS_JAM_CODE_INPUT - 0x01
02	PRTSTATUS_JAM_CODE_OUTPUT - 0x02
03	PRTSTATUS_JAM_CODE_INTERNAL - 0x03
04	PRTSTATUS_JAM_CODE_DUPLEX - 0x04
05	PRTSTATUS_JAM_CODE_EXTERNAL - 0x05
06	PRTSTATUS_JAM_CODE_STACKER - 0x06
07	PRTSTATUS_JAM_CODE_FINISHER -

			0x07	
			08	PRTSTATUS_JAM_CODE_MARKER - 0x08
			09	PRTSTATUS_JAM_CODE_LOG_UNIT - 0x09
			0A - FD	Reserved
			FE	PRTSTATUS_JAM_CODE_OTHER - 0xFE
			FF	PRTSTATUS_JAM_CODE_UNKNOWN - 0xFF
		JamLocationID		Identification number of jam location. For example, if the jam occurred at an input, this value would be the input ID. A value of hex 'FF' indicates the location cannot be determined.
cOperatorAlerts		Count of operator alert messages that follow. This value determines the number of times *OperatorAlertCode, lOperatorAlertMsg,* and *OperatorAlertMsg* are repeated. If zero, these fields are not returned.		
		OperatorAlertCode		Defines the operator intervention reason, as follows:
			Bit	Description
			00	Reserved
			01	PRTSTATUS_OP_COVER_OPE

	N - 0x01 Cover or door open or ajar
02	PRTSTATUS_OP_WASTE_TONER_FULL - 0x02 Waste toner box full
03	PRTSTATUS_OP_RIBBON_JAM - 0x03 Print ribbon jam
04	PRTSTATUS_OP_WASTE_INK_FULL - 0x04 Waste ink reservoir full
05 - FD	Reserved
FE	PRTSTATUS_OP_OTHER - 0xFE Other
FF	PRTSTATUS_OP_UNKNOWN - 0xFF Unknown

lOperatorAlertMsg

Length of operator alert message that follows.

OperatorAlertMsg

ASCII operator alert message.

Note: This is **not** null terminated.

cWarningAlerts

Count of warning alert messages that follow. This value determines the number of times *WarningAlertCode, lWarningAlertMsg,* and *WarningAlertMsg* are repeated. If zero, these fields are not returned.

WarningAlertCode

Defines type of warning, as follows:

Bit	Description
00	Reserved
01	PRTSTATUS_WARNING_TONER_LOW - 0x01 Toner supply low
02	PRTSTATUS_WARNING_INK_LOW - 0x02 Supply low
03	PRTSTATUS_WARNING_INPUT_LOW - 0x03 Paper input low
04	PRTSTATUS_WARNING_OUTPUT_FULL - 0x04 Output bin nearly full

		05	PRTSTATUS_WARNING_COVER_OPEN - 0x05 Cover or panel open
		06	PRTSTATUS_WARNING_FUSING_LOW - 0x06 Fusing supplies low
		07 - FD	Reserved
		FE	PRTSTATUS_WARNING_OTHER - 0xFE Other
		FF	PRTSTATUS_WARNING_UNKNOWN - 0xFF Unknown.
	lWarningAlertMsg		Length of warning alert message that follows.
	WarningAlertMsg		ASCII warning alert message. **Note:** This is **not** null terminated.
cServiceAlerts	Count of device service-required alert messages that follow.		
	ServiceAlertCode		Defines type-of-service-required alert, as follows:
		Bit	Description

00	Reserved
01	PRTSTATUS_SERVICE_HARD_ERROR - 0x01 Hard error. A hard error indicates an error from which the system cannot recover.
02	PRTSTATUS_SERVICE_SOFT_ERROR - 0x02 Soft error. A soft error indicates an error that was detected and may or may not persist.
03 - FE	Reserved
FF	PRTSTATUS_SERVICE_UNKNOWN -

0xFF Unknown.

lServiceAlertMsg

Length of service-required message that follows.

ServiceAlertMsg

ASCII service-required alert message.

Note: This is **not** null terminated.

cConfigureAlerts

Count of configuration-change alert messages that follow. This value determines the number of times *ConfigureAlertCode, lConfigureAlertMsg* and *ConfigureAlertMsg* are repeated. If zero, these fields are not returned.

ConfigureAlertCode

Defines the type of configuration-change alert, as follows:

Bit	Description
00	Reserved
01	PRTSTATUS_CONFIG_INT_RES_DEL - 0x01 Interpreter-related resources deleted
02	PRTSTATUS_CONFIG_INT_RES_ADD - 0x02 Interpreter-related resources added
03	PRTSTATUS_CONFIG_TRAY_REMOVED - 0x03

04	PRTSTATUS_CONFIG_TRAY_INSERTED - 0x04
05	PRTSTATUS_CONFIG_CART_REMOVED - 0x05 Cartridge removed
06	PRTSTATUS_CONFIG_CART_INSERTED - 0x06 Cartridge inserted
07	PRTSTATUS_CONFIG_MEM_REDUCED - 0x07 Installation memory reduced
08	PRTSTATUS_CONFIG_MEM_ADDED - 0x08 Installation memory increased

		09 - FD	Reserved
		FE	PRTSTATUS_CONFIG_OTHER - 0xFE Configuration change. Printer is unable to classify change, but host can determine change by interpreting printer's query response.
		FF	PRTSTATUS_CONFIG_UNKNOWN - 0xFF Unknown
	lConfigureAlertMsg	Length of configuration-change message that follows.	
	ConfigureAlertMsg	ASCII configuration-change alert message. **Note:** This is **not** null terminated.	
cSuppliesLevels	Count of printing-supplies-level values that follow. This value determines the number of times *SuppliesLocation, lSuppliesID,* and *SuppliesLevel* are repeated. If zero, these fields are not returned.		
	SuppliesLevel	Defines the current status of printing supplies as follows:	
		Bit	Description
		0 - 2	PRTSTATUS

_LEV
EL_M
ASK -
0x00
07
Suppl
ies
level.
All
zeros
(0)
indica
te the
printi
ng
suppl
y is
empt
y. All
ones
(1)
indica
te the
suppl
y is
full.

3 - 14

Reserv
ed

15

PRTST
ATUS
_ALE
RT_A
CTIV
E -
0x80
00
Alert
active
(0=N
o;
1=Ye
s)

SuppliesLocation

Specifies the general location of the supply within the printer, as defined by *JamLocationCode*. A value of hex 'FF' is used if the location cannot be specified.

SuppliesID

Specifies the ID number of the location of the supply at *SuppliesLocation*. A value of hex 'FF' is used if an ID does not exist for the supply.

cSuppliesAlerts

Count of printing-supplies alert messages that follow. This value determines the number of times *SuppliesAlertCode, ISuppliesAlertMsg,* and *SuppliesAlertMsg* are repeated. If zero, these fields are not returned.

SuppliesAlertCode

Defines the type of printing-supplies alert.

Bit

Descri
ption

00

Reserv
ed

01

PRTST

			ATUS_SUPPLY_OUT_OF_TONER - 0x01
		02	PRTSTATUS_SUPPLY_OUT_OF_INK - 0x02
		03	PRTSTATUS_SUPPLY_RIBBON_OUT - 0x03
		04 - FD	Reserved
		FE	PRTSTATUS_SUPPLY_OTHER - 0xFE
		FF	PRTSTATUS_SUPPLY_UNKNOWN - 0xFF
	lSuppliesAlertMsg		Length of printing-supplies message that follows.
	SuppliesAlertMsg		ASCII printing-supplies alert message. **Note:** This is **not** null terminated.
cPrinterStatistics			Count of printer statistics that follow. This value determines the number of times *PrinterStatisticsCode* and *PrinterStatisticsValue* are repeated. If zero, these fields are not returned.
	PrinterStatisticsCode		Defines the type of statistic the printer is reporting, as follows:
		Bit	Description
		00	Reserved

01	PRTSTATUS_STATS_UNITS_TOTAL - 0x01 Total counter units printed since beginning of printer life
02	PRTSTATUS_STATS_UNITS_POWERON - 0x02 Total counter units printed since printer power on
03	PRTSTATUS_STATS_UNITS_SUPPLIES - 0x03 Total counter units on current printing supplies
04	PRTSTATUS_STATS_HOST_COUNTER - 0x04

		Current value of the counter, resetable by the host.
	05 - FE	Reserved
	FF	PRTSTATUS_STATS_UNKNOWN - 0xFF Unknown.
PrinterStatisticsValue		The value associated with the previous statistic code. The units associated with this value are defined by BIDI_Q_DEVICE, in the *counterUnit*s field of the PRTDEVCHAR data structure.

Remarks

None.

BIDI_Q_STORAGE(8023h)

BIDI_Q_STORAGE(8023h) returns printer storage media information.

Parameters

- **pInData** (PVOID) - input

  Not used.

- **pOutData** (PVOID) - output

  Returns a PRTSTORAGEL data structure and a PRTSTORAGE data structure.

Remarks

None.

BIDI_Q_SW(800Ch)

BIDI_Q_SW(800Ch) queries BIDI software capabilities.

Parameters

- **pInData** (PVOID) - input

  Not used.

- **pOutData** (PVOID) - output

  Returns a PRTSW data structure.

Remarks

None.

BIDI_READ_ALERT(801Dh)

BIDI_READ_ALERT(801Dh) requests extended alert information from the protocol converter.

Parameters

- **ulAlertHandle** (ULONG) - input

  Indicates the alert handle returned by SplRegister.

- **pInData** (PVOID) - input

  Contains the value from *ulAlertHandle*.

- **pOutData** (PVOID) - output

  Receives a PRTSETALERT data structure, followed by the extended alert data.

  The PRTSETALERT data structure is followed by the extended alert data. The format of this extended alert data is dependent on the protocol converter.

  Only applications that know the protocol converter's convention for returned extended alert information can use the BIDI_READ_ALERT(801Dh) command structure. An application must first register for an extended alert using SplRegister, and the application must have been posted a message indicating that an extended alert was received from the printer. Only then can the application issue a BIDI_READ_ALERT.

Remarks

None.

BIDI_READ_PASSTHRU(8001h)

BIDI_READ_PASSTHRU(8001h) waits for any response from the printer and must be done from within a passthru session.

See BIDI_START_PASSTHRU(1Ah) for more details. BIDI_SEND_PASSTHRU(1h) must be called to send any protocol-specific commands to the printer.

Parameters

- **pInData** (PVOID) - input

  Should be set to NULL; for future use.

- **pOutData** (PVOID) - output

  Contains the protocol-specific response.

  This parameter receives all responses and unknown data from the printer while the passthru session is active.

  Note: If TYPE_WITHOUT_WRAPPER is specified, any protocol converter wrapper will be removed before passing the data to the application.

Remarks

None.

BIDI_RELEASEJOB(5h)

BIDI_RELEASEJOB(5h) releases a job held in the printer.

Parameters

- **pInData** (PVOID) - input

  Contains a PRTCONTROLJOB data structure.

- **pOutData** (PVOID) - output

  Not used.

Remarks

None.

BIDI_RESET(9h)

BIDI_RESET(9h) resets the printer. This command resets the printer to the state defined in *ulResetValue*. If the printer is connected using a network card, the network card is not reset.

Parameters

- **ulResetValue** (PVOID) - input

  Indicates the reset state of the printer. Values are as follows:

Value	Description
1	PRTRESET_POWER_ON Reset printer to power-on condition. For NPAP, this returns the printer to the NPAP-initialized state.
2	PRTRESET_SW_ONLY Reset BIDI software protocol layer. This reset leaves the rest of the printer and interpreter configuration as is.
3	PRTRESET_POWER_ON_EXIT_SW Reset printer to power-on condition and exit BIDI software protocol. All temporary resources are lost, according to Power-On-Reset.
4	PRTRESET_EXIT_SW Reset printer and exit BIDI software protocol. Temporary resources are preserved.
5	PRTRESET_FACTORY_DEFAULTS Reset printer to factory defaults. All temporary resources are lost, according to Power-On-Reset.

- **pInData** (PVOID) - input

  Contains the value from *ulResetValue*.

- **pOutData** (PVOID) - output

  Not used.

Remarks

None.

BIDI_RESPONSE_FMT(Dh)

BIDI_RESPONSE_FMT(Dh) sets the format of the printer-to-host messages. The spooler queries the protocol converter with BIDI_Q_RESPONSE_FMT(8018h). The spooler then passes this information to the port driver, using BIDI_RESPONSE_FMT. This allows the port driver to determine printer response message boundaries.

Parameters

- **pInData** (PVOID) - input

  Contains a PRTRESPONSE data structure.

- **pOutData** (PVOID) - oytput

  Not used.

Remarks

None.

BIDI_SEND_PASSTHRU(1h)

BIDI_SEND_PASSTHRU(1h) is a protocol-specific command that can be issued to the printer. This command does not return any response data to the caller. BIDI_SEND_PASSTHRU(1h) can be sent at any time, not just during a passthru session. See BIDI_START_PASSTHRU(1Ah) for more details.

Parameters

- **pInData** (PVOID) - input

  Contains the protocol-specific set command sequence to send to the printer.

- **pOutData** (PVOID) - oytput

  Not used.

Remarks

None.

BIDI_SET_DEVICE_ID(23h)

BIDI_SET_DEVICE_ID(23h) sets the printer's device ID. The protocol converter calls PrtSet with this command structure to notify the spooler of the device ID of an attached printer.

BIDI_SET_DEVICE_ID(23h) is used when the port driver is unable to determine the device ID during BIDI_Q_PORT(800Bh), and allows the protocol converter to retrieve the device ID.

Parameters

- **pInData** (PVOID) - input

  Contains a null-terminating device ID string, as defined by the P1284 specification.

- **pOutData** (PVOID) - output

  Not used.

Remarks

None.

BIDI_SET_PORTDRV(19h)

BIDI_SET_PORTDRV(19h) stores port driver configuration data. A port driver on a client implements SplPdRemoteSetPort by calling PrtQuery(BIDI_Q_PORTDRV(8019h)). This call gets the port driver configuration from the server. Next, the port driver displays the port configuration and, if the user changes anything, updates the server's port driver settings using PrtSet(BIDI_SET_PORTDRV).

Parameters

- **pInData** (PVOID) - input

  Contains port-driver-specific data.

- **pOutData** (PVOID) - output

  Not used.

Remarks

None.

BIDI_SET_SW(10h)

BIDI_SET_SW(10h) tells the port driver the BIDI software capabilities of the printer. The spooler passes the BIDI_Q_SW(800Ch) command structure, returned by the protocol converter, to the port driver using BIDI_SET_SW(10h).

Parameters

- **pInData** (PVOID) - input

  Contains a PRTSW data structure.

- **pOutData** (PVOID) - output

  Not used.

Remarks

None.

BIDI_SHUTDOWN(Ah)

BIDI_SHUTDOWN(Ah) releases all threads waiting for this protocol converter and port driver. This command is used to shut down the protocol converter and port driver when disabling the spooler. The command is sent first to the protocol converter to shut down the port; then the port driver is called.

Parameters

- **pInData** (PVOID) - input

  Not used.

- **pOutData** (PVOID) - output

  Not used.

Remarks

None.

BIDI_START_PASSTHRU(1Ah)

BIDI_START_PASSTHRU(1Ah) starts a passthru reading session. An application or printer driver that needs to issue a passthru query to the printer, and get its response, must establish a temporary passthru session.

There can be only one passthru session active for a printer port. A passthru session is necessary in order to use the BIDI_READ_PASSTHRU(8001h) command structure. The passthru session receives all data returned by the printer via BIDI_READ_PASSTHRU(8001h). After the application has received its answer(s), it must use BIDI_END_PASSTHRU(1Bh) to allow other applications or printer drivers to get a passthru session.

Giving a remote control panel display to the user is an example of using a passthru session. The printer driver uses BIDI_START_PASSTHRU, then sends any printer-specific commands using BIDI_SEND_PASSTHRU(1h). Next, the printer driver waits for the printer responses using BIDI_READ_PASSTHRU(8001h) and, finally, after the query is satisfied, the printer driver must use BIDI_END_PASSTHRU(1Bh).

Passthru sessions must be responsive. The protocol converter will store only the last five (5) printer messages if a passthru session is active. If a passthru session is not active for more than three (3) minutes, the session can be terminated.

Parameters

- **ulPassthruWait** (ULONG) - input

  Contains the wait value, in seconds, to establish a passthru session. Values are as follows:

0	Return immediately if unable to establish passthru session. This could fail if there is already a passthru session for the port.
Nonzero	Wait this number of seconds to get the passthru session established. The caller must check for ERROR_TIMEOUT and retry if necessary.

- **pInData** (PVOID) - input

  Contains the value in *ulPassthruWait*.

- **pOutData** (PVOID) - output

  Not used.

Remarks

None.

BIDI_STARTJOB(2h)

BIDI_STARTJOB(2h) is called by the port driver during SplPdOpen to signal the protocol converter that the print job is starting. The port driver should use the *pPrtOpenStruct* parameter in SplPdOpen to derive the parameters for this command.

Note: The caller of PrtSet can set the following parameters in the PRTSTARTJOB data structure:

- *ulSpoolerJobID* - set to -1
- *ulInterpreterID* - set to -1

This will let the spooler fill in the values based on the current job being sent to this printer port.

Parameters

- **pInData** (PVOID) - input

  Contains a PRTSTARTJOB data structure.

- **pOutData** (PVOID) - output

  Not used.

Remarks

None.

BIDI_TERM(Ch)

BIDI_TERM(Ch) sets the printer to unidirectional mode. This tells the port driver and kernel device driver the port is set to unidirectional mode.

Parameters

- **pInData** (PVOID) - input

  Not used.

- **pOutData** (PVOID) - output

  Not used.

Remarks

None.

BIDI_WAIT_ALERT(8016h)

BIDI_WAIT_ALERT(8016h) waits for an alert. The port driver must be able to store the PRTPORT.ulMaxHeldResponses number of responses, to give the spooler time to ask for each printer-generated message. If an alert does not change the currently known state of the printer, the spooler may disregard the alert.

The spooler has a single thread dedicated to calling each port driver with BIDI_WAIT_ALERT. No other application can use the BIDI_WAIT_ALERT command.

If a protocol converter is used, the port driver will process the request by calling SplProtXlateCmd with all data returned by the printer. If the protocol converter indicates by its return that the data was an alert, the port driver returns this information to the caller of SplPdQuery (the spooler).

When an alert condition is cleared, an alert should be returned, with the current status of the printer (that is, PRTALERT_TYPE_ONLINE or PRTALERT_TYPE_OFFLINE), to the caller of BIDI_WAIT_ALERT.

Parameters

- **pInData** (PVOID) - input

  Not used.

- **pOutData** (PVOID) - output

  Returns a PRTALERT data structure.

Remarks

None.

OS/2 Version Compatibility Considerations

The following table lists new information that has been added to or changed in this documentation. Moreover, when writing a device driver, you will want to take into consideration these particular changes that can affect the downward compatibility of these items.

Item Added or Changed	Date Item Added or Changed	Compatible with Version
DEVESC_DBE_FONTMANAGEMENT	June 1996	OS/2 Warp, Version 3 and higher
Dynamic Job Property DevEscape interface in Mandatory and Simulated Graphics Engine Function Reference	June 1996	OS/2 Warp, FixPak XR_W017 and higher
BIDI APIs in Device Support Function Reference	November 1995	OS/2 Warp, Version 3 and higher
BIDI command structures in BIDI Command Structures and Command Flow	November 1995	OS/2 Warp, Version 3 and higher
DEVESC_ACQUIREFB	April 1995	OS/2 Warp, Version 3 and higher
DEVESC_DEACQUIREFB	April 1995	OS/2 Warp, Version 3 and higher
DEVESC_EXTGET	April 1995	OS/2 Warp, Version 3 and higher
DEVESC_EXTPUT	April 1995	OS/2 Warp, Version 3 and higher
DEVESC_EXTQUERY	April 1995	OS/2 Warp, Version 3 and higher
DEVESC_GETAPERTURE	April 1995	OS/2 Warp, Version 3 and higher
DEVESC_HWREQUEST	April 1995	OS/2 Warp, Version 3 and higher
DEVESC_QUERYFB	April 1995	OS/2 Warp, Version 3 and higher

DEVESC_REGISTER	April 1995	OS/2 Warp, Version 3 and higher
DEVESC_SWITCHBANK	April 1995	OS/2 Warp, Version 3 and higher
DEVESC_VRAMALLOC	April 1995	OS/2 Warp, Version 3 and higher
QueryDeviceSurface*	April 1995	OS/2 Warp, Version 3 and higher
SDBitBlit*	September 1994	OS/2 Warp, Version 3 and higher
SDLine*	September 1994	OS/2 Warp, Version 3 and higher

Note: *A fix pack to make these functions work on OS/2 2.11 available in late 1995.

Syntax Conventions

The programming statements in this book use the C language syntax. Support for code written in C is provided in header files identified by the filename extension ".h". Assembler support is provided in the include files identified by the filename extension ".INC".

Parameter Names

Parameter names are constructed to show the data type of the parameter and to indicate its use:

- A lowercase prefix of one or more characters that indicates the data type.
- An optional qualifier starting with an uppercase letter.

  Where possible, standard names have been used to describe parameters. Where multiple word qualifiers are used, the order of the words is not significant.

For example:

```
hdc          /* device context handle         */
pszFilename  /* pointer to a character string */
```

The following standard base tags and their associated type names are defined:

Tag	Data Type	Description
f	BOOL	Flag or Boolean variable. The qualifier describes the condition associated with the flag when it is TRUE. For example, fSuccess is TRUE if successful and FALSE if not; whereas fError is TRUE if an error occurred and FALSE if no error occurred. For objects of type BOOL, the value 0 implies FALSE and any nonzero value implies TRUE.
ch	CHAR	Signed eight-bit quantity; a character.
s	SHORT	Signed 16-bit quantity; a SHORT. This is

		often used in place of ***us*** when it does not matter whether the value is signed or unsigned.
l	LONG	Signed 32-bit quantity; a LONG. This is often used in place of ***ul*** when it does not matter whether the value is signed or unsigned.
uch	UCHAR	Unsigned eight-bit quantity; a byte. Same as ***b***.
us	USHORT	Unsigned 16-bit quantity.
ul	ULONG	Unsigned 32-bit quantity.
b	BYTE	Unsigned eight-bit quantity; a byte. Same as ***uch***.
sz	CHAR[ ]	NULL-terminated string of characters.
fb	UCHAR	Byte of flags, that is, an array of flags packed in a BYTE.
fs	USHORT	SHORT of flags, that is, an array of flags packed in a USHORT.
fl	ULONG	LONG of flags, that is, an array of flags packed in a ULONG. The three preceding types are used when more than one flag is combined into a byte, SHORT or LONG. Typically, the values are combined with the OR operator and are always unsigned.
r	REAL	Real number, single precision 32-bits.
rd	DOUBLE	Real number, double precision 64-bits.
pfn		Pointer to a function.
x		X-coordinate.
y		Y-coordinate.

The following standard prefixes are also defined:

Prefix	Description
p	32-bit pointer for an 80386 microprocessor. For example, ***pch*** is a pointer to a character.
a	Array. For example, ***ach*** is an array of characters.
i	Index to an array. For example, an ***ich*** is used to index an ***ach***.
c	Count. For example, ***cch*** is a count of characters.
d	Delta or difference between instances of a type. For example, ***dx*** is the difference between two values of ***x***.
h	Handle. A value that uniquely identifies an object but cannot directly be used to access it. For example, ***hps*** is a PS handle.
mp	Mapping array. This prefix is always followed by two base types rather than one and represents the most general case of an array. Mathematically, an array is a function mapping the index to the value stored in the array. The prefix ***mp*** is an abbreviation of map. In the construct ***mpab***, ***a*** is the type of the index and ***b*** is the type of the value stored in the array. In most cases, the only type that is important is the type of the value. The index is usually an integer with no other meaning (the ***a*** prefix is used in this instance).

off	Offset. Generally used as an offset within a data structure. The actual address of the element within the data structure is derived by adding an offset to a pointer, which points to the beginning of the data structure. Normally, OFF is a byte offset. For example: pfoo = (FOO *)((BYTE *)pfooBase + 0fff00)
id	Identifier. This is generally used for values that identify some object. Usually the association of the ID value and the object are established by the programmer. For example, all windows are identified by their Window ID, which can be set and queried by the programmer.
cmd	Command. Used for command values, typically as function parameters.

Some parameters are used in pairs; the qualifiers that are used reflect the relationship between the two variables. For example:

Parameter	Description

First/Last	First and last elements in a set. These are typically used with indexes or pointers (pchFirst, pchLast). Both values represent valid values (compare with Min/Max below). For all valid values of x: xFirst <= x <= xLast. The use of > with ***First*** or < with ***Last*** is almost always an "off-by-one" error.
	For example, to determine whether an ich is within ichFirst and ichLast:
	`if (ich >= ichFirst && ich <= ichLast)`
	`...`
	A typical loop:
	`for (ich = ichFirst; ich <= ichLast; ich++)`
	`...`

Min/Max	Similar to First/Last except that Max is not a valid value in the set (Min is a valid value). For all valid values of ***x*** in the set: xMin < = x < xMax. The use of > with Min or < = with Max is almost always an "off-by-one" error.
	For example, to determine whether an ich is within ichMin and ichMax:
	`if (ich >= ichMin && ich < ichMax)`
	`...`
	A typical loop:
	`for (ich = ichMin; ich < /* or != */ ichMax; ich++)`
	`...`
	The current value (Cur) qualifier can be used with Min and Max when Min or Max can change over time (for example, ***pbStackMaxCur***).

Old/New	Old and new. Typically used for values or states when it is necessary to compare the old and new states of the value.
Next/Prev	Next and previous. Typically used in situations in which items are being enumerated, such as with linked lists.
Src/Dst	Source and destination. Typically used in transfer operations.
T	A temporary value.
Save	A temporary saved value. Typically used when saving and restoring some state.
Cur	Current value.

The base types and their prefixes are defined as follows:

Data Type	Prefix
PSZ	psz
PCH	pch
HAB	hab
HPS	hps

```
HDC         hdc

HRGN        hrgn

HBITMAP     hbmp

PLONG       pl

POINTL      ptl

POINTL      pt

RECTL       rcl

RECTL       rc

HWND        hwnd

WPOINT      wpt

WRECT       wrc

FIXED       fx
```

Parameters for defined structures are the defined parameter names. For example:

```
AREADEFS struct
         {
               defSet
               fFlags
               CodePage
         }AREADEFS
```

System-defined constants and flags are represented as two or more uppercase WORDs or mnemonic abbreviations separated by underscores. For example, SYS_CONSTANT and SYS_FLAG.

Return Values

Function handling routines pass full 32-bit return codes back to the calling function. In MASM**, the return code is passed in the EAX Register.

Register Content Preservation

Registers EAX, ECX, and EDX can be destroyed. All other registers must be preserved.

Handles

All handles and pointers are 32-bit values.

Coordinates

All coordinates are passed as signed 32-bit values unless stated otherwise. World, model, and presentation-page space coordinates are restricted to the 28 low-order bits and lie within the range F8000000h through 07FFFFFFh. Device space coordinates are restricted to the 16 low-order bits and lie within the range FFFF0000h through 0000FFFFh.

Format of the Journal File

Note: The journal file format is subject to change and should only be accessed by the engine functions. It is presented here to aid debugging of hardcopy drivers that use journal files.

The OS/2 graphics engine journal functions create a file, either in memory or on disk. This file is used to record all OS/2 Presentation Manager graphics functions so that a balance can be achieved between memory usage, image detail, hardcopy device capabilities, and hardcopy device performance.

The format of the record is as follows:

Flags (Low Memory)	Length	Function Arguments	arg data (if any) (High Memory)
1 WORD	1 WORD	(arg cnt DWORDs)	Variable size

The **arg** data is the data pointed to by any arguments in the function argument list. The actual journaled argument is changed into an offset to the journal arg data and is fixed up at playback time.

If the bit-map bits or region rects are dumped, they will be written to disk immediately following the journal record to which they belong.

The format of this additional data for bit maps is:

Data Size	Bit Map Width	Height	Planes	Bit Count	BITS
1 DWORD	1 WORD	1 WORD	1 WORD	1 WORD	*n* bytes, where n = ((bitcount * width + 31) /32) * height * planes *4

The format of this additional data for regions rects is:

Data Size	DATA
1 DWORD	Data size/ 16 RECTs

The journal record for any function (including any *arg* data but not including dumped bit map or region data) is assumed to fit into the journal buffer. If regions or bit maps are written out, they are first dumped into the *region/bit-map* buffer. It is not necessary for the data to fit into this buffer all at once. However, it is assumed that regions do not have more than 64KB rectangles and bit maps do not have more than 64KB scans.

When private objects are created for the four special case functions, they are recorded. In this way, they can be destroyed by *DeleteJournalFile* and reused on subsequent playbacks to the same journal file handle. If metafiling this, avoid re-creating the regions from *rects* or the bit map from the bits for every play.

Bit-Map Simulation for 16-Bit Hardcopy Drivers

Presentation drivers for monochrome raster devices can use the system's display driver, DISPLAY.DLL, to draw the page image on a bit map. This technique reduces the amount of code in the hardcopy driver and ensures that all devices use the same drawing algorithms.

To use the display driver, the hardcopy driver has to open and manage a display DC. It must do this without invoking the graphics engine (the hardcopy driver has to act as if it were the engine). When an application opens a hardcopy DC, the Enable subfunctions in the hardcopy driver must issue appropriate calls to the Enable subfunctions in the display driver.

Note: This technique works well for IBM display drivers. However, when using OEM display drivers, the use of this technique might result in an incompatibility.

These Enable subfunctions perform the following actions:

- **FillLogicalDeviceBlock**
  1. Loads the display driver and gets the address of its enable entry point.
  2. Saves a copy of the default dispatch table to pass to the display driver's *FillLogicalDeviceBlock* routine.
  3. Calls the display driver's enable entry point with the parameters set for FillLogicalDeviceBlock and passes a pointer to the saved default dispatch table. The display driver initializes this dispatch table and it can then be used to pass Gre*xxx* routines to the display DC.
- **FillPhysicalDeviceBlock**
  1. Saves the address of the display driver's enable entry point in the hardcopy driver's physical device block.
  2. Calls the display driver's enable entry point with the parameters set for *FillPhysicalDeviceBlock* and OD_MEMORY device type.
  3. Saves the value returned from the display driver's *FillPhysicalDeviceBlock* routine. (The value, *ulStateInf*o , is passed back to the display driver's *EnableDeviceContext* routine.)

     Note: In some source code, the name *pDCI* is used for the *ulStateInf*o parameter.
- **EnableDeviceContext**
  1. Calls the display driver's enable entry point with the parameters set for *EnableDeviceContext* . The DENPARAMS structure passed to the *EnableDeviceContext* routine contains the same DC handle (*ulHDC*) that was received by the hardcopy driver.
  2. Saves the value returned from the display driver's *FillPhysicalDeviceBlock* routine. This value, *pInstance,* needs to be passed back to the display driver on every call through the dispatch table.

     Note: In the source code, *pInstance* might occasionally be referred to as the "magic cookie".
- **CompleteOpenDC**
  1. Calls the display driver's enable entry point with the parameters set for *CompleteOpenDC* .

Any Gre*xxx* functions called from the operating system to the DC will enter the hardcopy DC through its dispatch table. Function handling routines in the hardcopy driver monitor the incoming calls and redirect those calls that affect the image through the display DC's dispatch table. When a *GreEscape* routine for DEVESC_NEWFRAME or DEVESC_ENDDOC is detected, the hardcopy driver transfers the bit-map bits from the display DC to a local buffer and sends them as scan lines or bands to the physical device driver.

Data Types

A description of each data type follows.

ACQUIREFB

Structure describing action to take when acquiring the frame buffer.

```
typedef struct _ACQUIREFB {
  ULONG     fAFBFlags;     /*  Flag indicating whether bank switching is required. */
  ULONG     ulBankNumber;  /*  Bank number to switch to. */
  RECTL     rctlXRegion;   /*  Rectangle of cursor exclude region. */
} ACQUIREFB;

typedef ACQUIREFB *PACQUIREFB;
```

ACQUIREFB Field - fAFBFlags

fAFBFlags (ULONG)
Flag indicating whether bank switching is required.

This flag has the following value:

AFB_SWITCH	1

ACQUIREFB Field - ulBankNumber

ulBankNumber (ULONG)
Bank number to switch to.

ACQUIREFB Field - rctlXRegion

rctlXRegion (RECTL)
Rectangle of cursor exclude region.

ADDITIONALMETRICS

ADDITIONALMETRICS data structure.

If the font signature type is ADDITIONALMETRICS, additional information is included in the PANOSE datatype.

```
typedef struct _ADDITIONALMETRICS {
  ULONG      ulIdentity;  /*  Structure identity code. */
  ULONG      ulSize;      /*  Structure size in bytes. */
  PANOSE     panose;      /*  Quantitative descriptions of font faces. */
} ADDITIONALMETRICS;

typedef ADDITIONALMETRICS *PADDITIONALMETRICS;
```

ADDITIONALMETRICS Field - ulIdentity

ulIdentity (ULONG)
Structure identity code.

This must be one (1). This field is not part of the FONTMETRICS structure.

ADDITIONALMETRICS Field - ulSize

ulSize (ULONG)
Structure size in bytes.

This must be set to the size of the FOCAMETRICS structure. This field is not part of the FONTMETRICS structure.

ADDITIONALMETRICS Field - panose

panose (PANOSE)
Quantitative descriptions of font faces.

The PANOSE datatype field consists of 10 digits, each of which describes one of up to 16 variations, to allow for quantitative descriptions of the visual properties of the font faces.

APERTURE

Structure describing attributes of VRAM.

```
typedef struct _APERTURE {
  ULONG     ulPhysAddr;      /*  Physical address of VRAM. */
  ULONG     ulApertureSize;  /*  Size of VRAM. */
  ULONG     ulScanLineSize;  /*  Size of a scan line. */
  RECTL     rctlScreen;      /*  Rectangle of screen coordinates. */
} APERTURE;

typedef APERTURE *PAPERTURE;
```

APERTURE Field - ulPhysAddr

ulPhysAddr (ULONG)
Physical address of VRAM.

APERTURE Field - ulApertureSize

ulApertureSize (ULONG)
Size of VRAM.

APERTURE Field - ulScanLineSize

ulScanLineSize (ULONG)
Size of a scan line.

APERTURE Field - rctlScreen

rctlScreen (RECTL)
Rectangle of screen coordinates.

APIERR

The return for SPLPD API error return codes.

```
typedef unsigned long APIERR;
```

ARCPARAMS

Arc-parameters structure.

```
typedef struct _ARCPARAMS {
  LONG     lP;  /*  P coefficient. */
  LONG     lQ;  /*  Q coefficient. */
  LONG     lR;  /*  R coefficient. */
  LONG     lS;  /*  S coefficient. */
} ARCPARAMS;

typedef ARCPARAMS *PARCPARAMS;
```

ARCPARAMS Field - lP

lP (LONG)
P coefficient.

ARCPARAMS Field - lQ

lQ (LONG)
Q coefficient.

ARCPARAMS Field - IR

IR (LONG)
R coefficient.

ARCPARAMS Field - IS

IS (LONG)
S coefficient.

BANDRECT

Location to which the device coordinates of the next band are returned.

```
typedef struct _BANDRECT {
  LONG     xLeft;     /*  X-coordinate of the lower-left corner of the rectangular band. */
  LONG     yBottom;   /*  Y-coordinate of the lower-left corner of the rectangular band. */
  LONG     xRight;    /*  X-coordinate of the top-right corner of the rectangular band. */
  LONG     yTop;      /*  Y-coordinate of the to-right corner of the rectangular band. */
} BANDRECT;

typedef BANDRECT *PBANDRECT;
```

BANDRECT Field - xLeft

xLeft (LONG)
X-coordinate of the lower-left corner of the rectangular band.

BANDRECT Field - yBottom

yBottom (LONG)
Y-coordinate of the lower-left corner of the rectangular band.

BANDRECT Field - xRight

xRight (LONG)
X-coordinate of the top-right corner of the rectangular band.

BANDRECT Field - yTop

yTop (LONG)
Y-coordinate of the to-right corner of the rectangular band.

BITBLTINFO

BitBlt information structure. BitBlt information structure, used for the GHI_CMD_BITBLT and VMI_CMD_BITBLT functions.

```
typedef struct _BITBLTINFO {
  ULONG         ulLength;       /*  Length of the BITBLTINFO data structure, in bytes. */
  ULONG         ulBltFlags;     /*  Flags for rendering of rasterized data. */
  ULONG         cBlits;         /*  Count of Blts to be performed. */
  ULONG         ulROP;          /*  Raster operation. */
  ULONG         ulMonoBackROP;  /*  Background mix if B_APPLY_BACK_ROP is set. */
  ULONG         ulSrcFGColor;   /*  Monochrome source Foreground color. */
  ULONG         ulSrcBGColor;   /*  Monochrome source Background color and transparent color. */
  ULONG         ulPatFGColor;   /*  Monochrome pattern Foreground color. */
  ULONG         ulPatBGColor;   /*  Monochrome pattern Background color. */
  PBYTE         abColors;       /*  Pointer to color translation table. */
  PBMAPINFO     pSrcBmapInfo;   /*  Pointer to source bit map (BMAPINFO) */
  PBMAPINFO     pDstBmapInfo;   /*  Pointer to destination bit map (BMAPINFO). */
  PBMAPINFO     pPatBmapInfo;   /*  Pointer to pattern bit map (BMAPINFO). */
  PPOINTL       aptlSrcOrg;     /*  Pointer to array of source origin POINTLs. */
  PPOINTL       aptlPatOrg;     /*  Pointer to array of pattern origin POINTLs. */
  PBLTRECT      abrDst;         /*  Pointer to array of Blt rects. */
  PRECTL        prclSrcBounds;  /*  Pointer to source bounding rect of source Blts. */
  PRECTL        prclDstBounds;  /*  Pointer to destination bounding rect of destination Blts. */
} BITBLTINFO;

typedef BITBLTINFO *PBITBLTINFO;
```

BITBLTINFO Field - ulLength

ulLength (ULONG)
Length of the BITBLTINFO data structure, in bytes.

BITBLTINFO Field - ulBltFlags

ulBltFlags (ULONG)
Flags for rendering of rasterized data.

Miscellaneous flags used by the graphics engine for rendering of rasterized data:

BF_DEFAULT_STATE	Blt direction is left to right and top to bottom.

BF_DIR_X_NEGATIVE	Blt direction is towards X origin.
BF_DIR_Y_NEGATIVE	Blt direction is towards Y origin.
BF_ROP_INCL_SRC	ROP includes a source bit map.
BF_ROP_INCL_PAT	ROP includes a pattern bit map.
BF_SRC_TRANSPARENT	Source transparent involved. SRC will not change when SRC=BG_COLOR.
BF_DST_TRANSPARENT	Destination transparent involved. DST will not change when DST=BG_COLOR.
BF_PAT_TRANSPARENT	Pattern transparent involved. Pattern not involved when PAT=BG_COLOR.
BF_PAT_SOLID	Pattern is solid; all Foreground color.
BF_PAT_HOLLOW	Pattern is hollow (empty); no Foreground color.
BF_APPLY_BACK_ROP	Treat ROP as Foreground mix and MonoBackROP as Background mix.
BF_SRC_MONOINVERT	Zero (0) bits are Foreground on monochrome SRC bit map.
BF_PAT_MONOINVERT	Zero (0) bits are Foreground on monochrome PAT bit map.
BF_SR_BITS_EXTERNAL	Source bit map bits are in a nondevice-specific format.
BF_LAST_BLT	Defines last blit in a banded BitBlit.
BF_SRC_Y_FLIP	Source Y coordinates are inverted relative to device origin.

BITBLTINFO Field - cBlits

cBlits (ULONG)
Count of Blts to be performed.

BITBLTINFO Field - ulROP

ulROP (ULONG)
Raster operation.

BITBLTINFO Field - ulMonoBackROP

ulMonoBackROP (ULONG)
Background mix if B_APPLY_BACK_ROP is set.

BITBLTINFO Field - ulSrcFGColor

ulSrcFGColor (ULONG)
Monochrome source Foreground color.

BITBLTINFO Field - ulSrcBGColor

ulSrcBGColor (ULONG)
Monochrome source Background color and transparent color.

BITBLTINFO Field - ulPatFGColor

ulPatFGColor (ULONG)
Monochrome pattern Foreground color.

BITBLTINFO Field - ulPatBGColor

ulPatBGColor (ULONG)
Monochrome pattern Background color.

BITBLTINFO Field - abColors

abColors (PBYTE)
Pointer to color translation table.

BITBLTINFO Field - pSrcBmapInfo

pSrcBmapInfo (PBMAPINFO)
Pointer to source bit map (BMAPINFO)

BITBLTINFO Field - pDstBmapInfo

pDstBmapInfo (PBMAPINFO)
Pointer to destination bit map (BMAPINFO).

BITBLTINFO Field - pPatBmapInfo

pPatBmapInfo (PBMAPINFO)
Pointer to pattern bit map (BMAPINFO).

BITBLTINFO Field - aptlSrcOrg

aptlSrcOrg (PPOINTL)
Pointer to array of source origin POINTLs.

BITBLTINFO Field - aptlPatOrg

aptlPatOrg (PPOINTL)
Pointer to array of pattern origin POINTLs.

BITBLTINFO Field - abrDst

abrDst (PBLTRECT)
Pointer to array of Blt rects.

BITBLTINFO Field - prclSrcBounds

prclSrcBounds (PRECTL)
Pointer to source bounding rect of source Blts.

BITBLTINFO Field - prclDstBounds

prclDstBounds (PRECTL)
Pointer to destination bounding rect of destination Blts.

BITMAPINFO

Bit-map information structure.

Each bit plane logically contains (*cx * cy * cBitCount*) bits, although the actual length can be greater because of padding.

See also BITMAPINFO2, which is preferred.

```
typedef struct _BITMAPINFO {
  ULONG      cbFix;          /*  Length of fixed portion of structure. */
  USHORT     cx;             /*  Bit-map width in pels. */
  USHORT     cy;             /*  Bit-map height in pels. */
  USHORT     cPlanes;        /*  Number of bit planes. */
  USHORT     cBitCount;      /*  Number of bits per pel within a plane. */
  RGB        argbColor[1];   /*  Array of RGB values. */
} BITMAPINFO;

typedef BITMAPINFO *PBITMAPINFO;
```

BITMAPINFO Field - cbFix

cbFix (ULONG)
Length of fixed portion of structure.

This length can be determined using `sizeof(BITMAPINFOHEADER)`.

BITMAPINFO Field - cx

cx (USHORT)
Bit-map width in pels.

BITMAPINFO Field - cy

cy (USHORT)
Bit-map height in pels.

BITMAPINFO Field - cPlanes

cPlanes (USHORT)
Number of bit planes.

BITMAPINFO Field - cBitCount

cBitCount (USHORT)
Number of bits per pel within a plane.

BITMAPINFO Field - argbColor[1]

argbColor[1] (RGB)
Array of RGB values.

This is a packed array of 24-bit RGB values. If there are N bits per pel (N = c*Planes* * c*BitCount*), the array contains 2**N RGB values. However, if N = 24, the bit map does not need the c*olor* *color* array because the standard-format bit map, with 24 bits per pel, is assumed to contain RGB values.

BITMAPINFOHEADER

Bit-map information header structure.

Each bit plane logically contains (c*x* * c*y* * c*BitCount*) bits, although the actual length can be greater because of padding.

See also BITMAPINFOHEADER2, which is preferred.

```
typedef struct _BITMAPINFOHEADER {
  ULONG      cbFix;      /*  Length of structure. */
  USHORT     cx;         /*  Bit-map width in pels. */
  USHORT     cy;         /*  Bit-map height in pels. */
  USHORT     cPlanes;    /*  Number of bit planes. */
  USHORT     cBitCount;  /*  Number of bits per pel within a plane. */
} BITMAPINFOHEADER;

typedef BITMAPINFOHEADER *PBITMAPINFOHEADER;
```

BITMAPINFOHEADER Field - cbFix

cbFix (ULONG)
Length of structure.

BITMAPINFOHEADER Field - cx

cx (USHORT)
Bit-map width in pels.

BITMAPINFOHEADER Field - cy

cy (USHORT)
Bit-map height in pels.

BITMAPINFOHEADER Field - cPlanes

cPlanes (USHORT)
Number of bit planes.

BITMAPINFOHEADER Field - cBitCount

cBitCount (USHORT)
Number of bits per pel within a plane.

BITMAPINFOHEADER2

Bit-map information header structure.

Each bit plane logically contains (*cx * cy * cBitCount*) bits, although the actual length can be greater because of padding.

Note: Many functions can accept either this structure or the BITMAPINFOHEADER structure. Where possible, use BITMAPINFOHEADER2.

```
typedef struct _BITMAPINFOHEADER2 {
  ULONG      cbFix;             /*  Length of structure. */
  ULONG      cx;                /*  Bit-map width in pels. */
  ULONG      cy;                /*  Bit-map height in pels. */
  USHORT     cPlanes;           /*  Number of bit planes. */
  USHORT     cBitCount;         /*  Number of bits per pel within a plane. */
  ULONG      ulCompression;     /*  Compression scheme used to store the bit map. */
  ULONG      cbImage;           /*  Length of bit-map storage data, in bytes. */
  ULONG      cxResolution;      /*  Horizontal component of the resolution of target device. */
  ULONG      cyResolution;      /*  Vertical component of the resolution of target device. */
  ULONG      cclrUsed;          /*  Number of color indexes used. */
  ULONG      cclrImportant;     /*  Minimum number of color indexes for satisfactory appearance of the bit map. *
  USHORT     usUnits;           /*  Units of measure. */
  USHORT     usReserved;        /*  Reserved. */
  USHORT     usRecording;       /*  Recording algorithm. */
  USHORT     usRendering;       /*  Halftoning algorithm. */
  ULONG      cSize1;            /*  Size value 1. */
  ULONG      cSize2;            /*  Size value 2. */
  ULONG      ulColorEncoding;   /*  Color encoding. */
  ULONG      ulIdentifier;      /*  Reserved for application use. */
} BITMAPINFOHEADER2;
```

```
typedef BITMAPINFOHEADER2 *PBITMAPINFOHEADER2;
```

BITMAPINFOHEADER2 Field - cbFix

cbFix (ULONG)
Length of structure.

The structure can be truncated after *cBitCount* or any subsequent field.

BITMAPINFOHEADER2 Field - cx

cx (ULONG)
Bit-map width in pels.

BITMAPINFOHEADER2 Field - cy

cy (ULONG)
Bit-map height in pels.

BITMAPINFOHEADER2 Field - cPlanes

cPlanes (USHORT)
Number of bit planes.

BITMAPINFOHEADER2 Field - cBitCount

cBitCount (USHORT)
Number of bits per pel within a plane.

BITMAPINFOHEADER2 Field - ulCompression

ulCompression (ULONG)
Compression scheme used to store the bit map.

BCA_UNCOMP
Bit map is uncompressed.

BCA_HUFFMAN1D
The bit map is compressed by a modified Huffman encoding. This is valid for a bi-level (one bit per pel) bit map.

BCA_RLE4
The bit map is a 4-bit per pel run-length encoded bit map. See the following section, "Format of Compressed Data," for a description of the format of the compressed data.

BCA_RLE8
The bit map is an 8-bit per pel run-length encoded bit map. See the following section, "Format of Compressed Data," for a description of the format of the compressed data.

BCA_RLE24
The bit map is a 24-bit per pel run-length encoded bit map. See the following section, "Format of Compressed Data," for a description of the format of the compressed data.

Format of Compressed Data

Encoding a run length:

Run length encoded bit maps are encoded in the buffer in a controlled format. In all cases, if the first byte is non-zero, it is the length of a run of pels of a particular color or, in the case of a BCA_RLE4 bit map, a run of a length of pels of alternating colors.

```
1st-byte       pel repetition count >= 1
2nd-4th bytes  (BCA_RLE24 only) RGB value of pel.
2nd-byte       (BCA_RLE8) color index of pel to be repeated
               (BCA_RLE4) the second byte contains 2 4-bit
               color indexes.  The repetition count is
               completed by alternately choosing the high-order
               nibble followed by the low-order nibble for the
               succeeding pels until the count is exhausted.
```

Unencoded run:

An unencoded run is a string of pels to be placed in consecutive positions in the destination bit map.

```
1st-byte     0
2nd-byte     COUNT = a multiple of 3 for BCA_RLE24 bit maps, or
             COUNT >= 3 (for BCA_RLE4 and BCA_RLE8 bit maps).
```

followed by the bytes as follows:

BCA_RLE24
A string of bytes specifying the RGB color values of succeeding pels. If COUNT is odd, it must be padded by a zero byte for an even length overall.

BCA_RLE8
A string of bytes specifying color indexes for succeeding pels. If COUNT is odd, it must be padded by a zero byte for an even length overall.

BCA_RLE4
A string of bytes, each byte providing two color indexes, with the high-order nibble specifying the index of the pel preceding the low-order nibble. The COUNT specifies the number of indexes. The overall length of the string must be an even number of bytes, and thus may be padded with a zero byte, and the low order nibble of the last significant byte may also be zero and not used.

Delta record:

A delta record indicates a shift in position in the destination bit map before the next record is decoded.

```
1st-byte        0
2nd-byte        2
3rd-byte        Delta-x (unsigned)
4th-byte        Delta-y (unsigned)
```

This is a relative jump record. It implies that the next record is to be decoded into a position in the destination bit map at an offset from the current position, determined by changing the horizontal and vertical positions by Delta-x and Delta-y, respectively.

End-of-line record:

The end-of-line record signifies that the data for the current scan line is complete and that decoding of the next record should begin at the start of the next scan line.

```
1st-byte        0
2nd-byte        0
```

End-of-RLE record:

The end-of-RLE record signifies the end of the data in the RLE compressed bit map.

```
1st-byte        0
2nd-byte        1
```

BITMAPINFOHEADER2 Field - cbImage

cbImage (ULONG)
Length of bit-map storage data, in bytes.

If the bit map is uncompressed, zero (the default) can be specified for this.

BITMAPINFOHEADER2 Field - cxResolution

cxResolution (ULONG)
Horizontal component of the resolution of target device.

The resolution of the device the bit map is intended for, in the units specified by *usUnits* . This information enables applications to select from a resource group the bit map that best matches the characteristics of the current output device.

BITMAPINFOHEADER2 Field - cyResolution

cyResolution (ULONG)
Vertical component of the resolution of target device.

See the description of *cxResolution* .

BITMAPINFOHEADER2 Field - cclrUsed

cclrUsed (ULONG)
Number of color indexes used.

The number of color indexes from the color table that are used by the bit map. If this is zero (the default), all the indexes are used. If it is non-zero, only the first *cclrUsed* entries in the table are accessed by the system, and further entries can be omitted.

For the standard formats with a *cBitCount* of 1, 4, or 8 (and *cPlanes* equal to 1), any indexes beyond *cclrUsed* are invalid. For example, a bit map with 64 colors can use the 8-bitcount format without having to supply the other 192 entries in the color table. For the 24-bitcount standard format, *cclrUsed* is the number of colors used by the bit map.

BITMAPINFOHEADER2 Field - cclrImportant

cclrImportant (ULONG)
Minimum number of color indexes for satisfactory appearance of the bit map.

More colors may be used in the bit map, but it is not necessary to assign them to the device palette. These additional colors may be mapped to the nearest colors available.

Zero (the default) means that all entries are important.

For a 24-bitcount standard format bit map, the *cclrImportant* colors are also listed in the color table relating to this bit map.

BITMAPINFOHEADER2 Field - usUnits

usUnits (USHORT)
Units of measure.

Units of measure of the horizontal and vertical resolution, *cxResolution* and *cyResolution* .

BRU_METRIC	Pels per meter. This is the default value.

BITMAPINFOHEADER2 Field - usReserved

usReserved (USHORT)
Reserved.

This is a reserved field. If present, it must be zero.

BITMAPINFOHEADER2 Field - usRecording

usRecording (USHORT)
Recording algorithm.

The format in which the bit-map data is recorded.

BRA_BOTTOMUP	Scan lines are recorded bottom to top. This is the default value.

BITMAPINFOHEADER2 Field - usRendering

usRendering (USHORT)
Halftoning algorithm.

The algorithm used to record bit-map data that has been digitally halftoned.

BRH_NOTHALFTONED	Bit-map data is not halftoned. This is the default value.
BRH_ERRORDIFFUSION	Error Diffusion or Damped Error Diffusion algorithm.
BRH_PANDA	Processing Algorithm for Non-coded Document Acquisition.
BRH_SUPERCIRCLE	Super Circle algorithm.

BITMAPINFOHEADER2 Field - cSize1

cSize1 (ULONG)
Size value 1.

If BRH_ERRORDIFFUSION is specified in *usRendering* , *cSize1* is the error damping as a percentage in the range 0 through 100. A value of 100% indicates no damping, and a value of 0% indicates that any errors are not diffused.

If BRH_PANDA or BRH_SUPERCIRCLE is specified, *cSize1* is the x dimension of the pattern used, in pels.

BITMAPINFOHEADER2 Field - cSize2

cSize2 (ULONG)
Size value 2.

If BRH_ERRORDIFFUSION is specified in *usRendering* , this parameter is ignored.

If BRH_PANDA or BRH_SUPERCIRCLE is specified, *cSize2* is the y dimension of the pattern used, in pels.

BITMAPINFOHEADER2 Field - ulColorEncoding

ulColorEncoding (ULONG)
Color encoding.

BCE_RGB	Each element in the color array is an RGB2 datatype. This is the default value.

BITMAPINFOHEADER2 Field - ulIdentifier

ulIdentifier (ULONG)
Reserved for application use.

BITMAPINFO2

Bit-map information structure.

Each bit plane logically contains (c*x* * c*y* * c*BitCount*) bits, although the actual length can be greater because of padding.

Note: Many functions can accept either this structure or the BITMAPINFO structure. Where possible, BITMAPINFO2 should be used.

The c*bFix* field is used to find the color table, if any, that goes with the information in this structure. A color table is an array of color (RGB2) values. If there are N bits per pel (N = c*Plane*s * c*BitCount*), the array contains 2**N color values. However, if N = 24, the color table is not required (because the standard-format bit map, with 24 bits per pel, is assumed to contain RGB values), unless either cc*lrUsed* or cc*lrImportant* is non-zero.

```
typedef struct _BITMAPINFO2 {
  ULONG       cbFix;             /*  Length of fixed portion of structure. */
  ULONG       cx;                /*  Bit-map width in pels. */
  ULONG       cy;                /*  Bit-map height in pels. */
  USHORT      cPlanes;           /*  Number of bit planes. */
  USHORT      cBitCount;         /*  Number of bits per pel within a plane. */
  ULONG       ulCompression;     /*  Compression scheme used to store the bit map. */
  ULONG       cbImage;           /*  Length of bit-map storage data, in bytes. */
  ULONG       cxResolution;      /*  Horizontal component of the resolution of target device. */
  ULONG       cyResolution;      /*  Vertical component of the resolution of the target device. */
  ULONG       cclrUsed;          /*  Number of color indexes used. */
  ULONG       cclrImportant;     /*  Minimum number of color indexes for satisfactory appearance of the bit map. *
  USHORT      usUnits;           /*  Units of measure. */
  USHORT      usReserved;        /*  Reserved. */
  USHORT      usRecording;       /*  Recording algorithm. */
  USHORT      usRendering;       /*  Halftoning algorithm. */
  ULONG       cSize1;            /*  Size value 1. */
  ULONG       cSize2;            /*  Size value 2. */
  ULONG       ulColorEncoding;   /*  Color encoding. */
  ULONG       ulIdentifier;      /*  Reserved for application use. */
  RGB2        argbColor[1];      /*  Array of RGB values. */
} BITMAPINFO2;

typedef BITMAPINFO2 *PBITMAPINFO2;
```

BITMAPINFO2 Field - cbFix

cbFix (ULONG)
Length of fixed portion of structure.

The structure can be truncated after c*BitCount* or any subsequent field.

The length does not include the length of the color table. Where the color table is present, it is at an offset of c*bFix* from the start of the BITMAPINFO2 structure.

This length can range from 16 (BITMAPINFOHEADER through field c*BitCount*) up to `sizeof(BITMAPINFOHEADER2)` bytes.

BITMAPINFO2 Field - cx

cx (ULONG)
Bit-map width in pels.

BITMAPINFO2 Field - cy

cy (ULONG)
Bit-map height in pels.

BITMAPINFO2 Field - cPlanes

cPlanes (USHORT)
Number of bit planes.

BITMAPINFO2 Field - cBitCount

cBitCount (USHORT)
Number of bits per pel within a plane.

BITMAPINFO2 Field - ulCompression

ulCompression (ULONG)
Compression scheme used to store the bit map.

BCA_UNCOMP
Bit map is uncompressed.

BCA_HUFFMAN1D
The bit map is compressed by a modified Huffman encoding. This is valid for a bi-level (one bit per pel) bit map.

BCA_RLE4
The bit map is a 4-bit per pel run-length encoded bit map. See the following section, "Format of Compressed Data," for a description of the format of the compressed data.

BCA_RLE8
The bit map is an 8-bit per pel run-length encoded bit map. See the following section, "Format of Compressed Data," for a description of the format of the compressed data.

BCA_RLE24
The bit map is a 24-bit per pel run-length encoded bit map. See the following section, "Format of Compressed Data," for a description of the format of the compressed data.

Format of Compressed Data

Encoding a run length:

Run-length encoded bit maps are encoded in the buffer in a controlled format. In all cases, if the first byte is non-zero, it is the length of a run of pels of a particular color or, in the case of a BCA_RLE4 bit map, a run of a length of pels of alternating colors.

```
1st-byte       pel repetition count >= 1
2nd-4th bytes  (BCA_RLE24 only) RGB value of pel.
2nd-byte       (BCA_RLE8) color index of pel to be repeated
               (BCA_RLE4) the second byte contains 2 4-bit
               color indexes.  The repetition count is
               completed by alternately choosing the high-order
               nibble followed by the low-order nibble for the
               succeeding pels until the count is exhausted.
```

Unencoded run:

An unencoded run is a string of pels to be placed in consecutive positions in the destination bit map.

```
1st-byte     0
2nd-byte     COUNT = a multiple of 3 for BCA_RLE24 bit maps, or
             COUNT >= 3 (for BCA_RLE4 and BCA_RLE8 bit maps).
```

followed by the bytes as follows:

BCA_RLE24

A string of bytes specifying the RGB color values of succeeding pels. If COUNT is odd, it must be padded by a zero byte for an even length overall.

BCA_RLE8

A string of bytes specifying color indexes for succeeding pels. If COUNT is odd, it must be padded by a zero byte for an even length overall.

BCA_RLE4

A string of bytes, each byte providing two color indexes, with the high-order nibble specifying the index of the pel preceding the low-order nibble. The COUNT specifies the number of indexes. The overall length of the string must be an even number of bytes, and thus may be padded with a zero byte, and the low order nibble of the last significant byte may also be zero and not used.

Delta record:

A delta record indicates a shift in position in the destination bit map before the next record is decoded.

```
1st-byte        0
2nd-byte        2
3rd-byte        Delta-x (unsigned)
4th-byte        Delta-y (unsigned)
```

This is a relative jump record. It implies that the next record is to be decoded into a position in the destination bit map at an offset from the current position, determined by changing the horizontal and vertical positions by Delta-x and Delta-y, respectively.

End-of-line record:

The end-of-line record signifies that the data for the current scan line is complete and that decoding of the next record should begin at the start of the next scan line.

```
1st-byte        0
2nd-byte        0
```

End-of-RLE record:

The end-of-RLE record signifies the end of the data in the RLE compressed bit map.

```
1st-byte        0
2nd-byte        1
```

BITMAPINFO2 Field - cbImage

cbImage (ULONG)
Length of bit-map storage data, in bytes.

If the bit map is uncompressed, zero (default) can be specified for this.

BITMAPINFO2 Field - cxResolution

cxResolution (ULONG)
Horizontal component of the resolution of target device.

The resolution of the device the bit map is intended for, in the units specified by *usUnits* . This information enables an application to select from a resource group the bit map that best matches the characteristics of the current output device.

BITMAPINFO2 Field - cyResolution

cyResolution (ULONG)
Vertical component of the resolution of the target device.

See the description of *cxResolution* .

BITMAPINFO2 Field - cclrUsed

cclrUsed (ULONG)
Number of color indexes used.

The number of color indexes from the color table that are used by the bit map. If it is zero (the default), all the indexes are used. If it is non-zero, only the first *cclrUsed* entries in the table are accessed by the system, and further entries can be omitted.

For the standard formats with a *cBitCount* of 1, 4, or 8 (and *cPlanes* equal to 1), any indexes beyond *cclrUsed* are not valid. For example, a bit map with 64 colors can use the 8-bitcount format without having to supply the other 192 entries in the color table. For the 24-bitcount standard format, *cclrUsed* is the number of colors used by the bit map.

BITMAPINFO2 Field - cclrImportant

cclrImportant (ULONG)

Minimum number of color indexes for satisfactory appearance of the bit map.

More colors may be used in the bit map, but it is not necessary to assign them to the device palette. These additional colors may be mapped to the nearest colors available.

Zero (the default) means that all entries are important.

For a 24-bitcount standard format bit map, the *cclrImportant* colors are also listed in the color table following the BITMAPINFO2 structure.

BITMAPINFO2 Field - usUnits

usUnits (USHORT)
Units of measure.

Units of measure of the horizontal and vertical components of resolution, *cxResolution* and *cyResolution* .

BRU_METRIC	Pels per meter. This is the default value.

BITMAPINFO2 Field - usReserved

usReserved (USHORT)
Reserved.

This is a reserved field.

BITMAPINFO2 Field - usRecording

usRecording (USHORT)
Recording algorithm.

The format in which the bit map data is recorded.

BRA_BOTTOMUP	Scan lines are recorded bottom to top. This is the default value.

BITMAPINFO2 Field - usRendering

usRendering (USHORT)
Halftoning algorithm.

The algorithm used to record bit map data that has been digitally halftoned.

BRH_NOTHALFTONED	Bit-map data is not halftoned. This is the default value.
BRH_ERRORDIFFUSION	Error Diffusion or Damped Error Diffusion algorithm.
BRH_PANDA	Processing Algorithm for Non-coded Document Acquisition.

BRH_SUPERCIRCLE	Super Circle algorithm.

BITMAPINFO2 Field - cSize1

cSize1 (ULONG)
Size value 1.

If BRH_ERRORDIFFUSION is specified in *usRendering* , *cSize1* is the error damping as a percentage in the range 0 through 100. A value of 100% indicates no damping, and a value of 0% indicates that any errors are not diffused.

If BRH_PANDA or BRH_SUPERCIRCLE is specified, *cSize1* is the x dimension of the pattern used, in pels.

BITMAPINFO2 Field - cSize2

cSize2 (ULONG)
Size value 2.

If BRH_ERRORDIFFUSION is specified in *usRendering* , this parameter is ignored.

If BRH_PANDA or BRH_SUPERCIRCLE is specified, *cSize2* is the y dimension of the pattern used, in pels.

BITMAPINFO2 Field - ulColorEncoding

ulColorEncoding (ULONG)
Color encoding.

BCE_RGB	Each element in the color array is an RGB2 datatype. This is the default value.

BITMAPINFO2 Field - ulIdentifier

ulIdentifier (ULONG)
Reserved for application use.

BITMAPINFO2 Field - argbColor[1]

argbColor[1] (RGB2)
Array of RGB values.

This is a packed array of 24-bit RGB values. If there are N bits per pel (N = c*Planes* * c*BitCount*), the array contains 2**N RGB values. However, if N = 24, the bit map does not need the *color* array because the standard-format bit map, with 24 bits per pel, is assumed to contain RGB values.

BLTRECT

Destination rectangle for a bitblt operation.

```
typedef struct _BLTRECT {
  ULONG     ulXOrg;  /*  X origin of the destination Blt. */
  ULONG     ulYOrg;  /*  Y origin of the destination Blt. */
  ULONG     ulXExt;  /*  X extent of the BitBlt. */
  ULONG     ulYExt;  /*  Y extent of the BitBlt. */
} BLTRECT;

typedef BLTRECT *PBLTRECT;
```

BLTRECT Field - ulXOrg

ulXOrg (ULONG)
X origin of the destination Blt.

BLTRECT Field - ulYOrg

ulYOrg (ULONG)
Y origin of the destination Blt.

BLTRECT Field - ulXExt

ulXExt (ULONG)
X extent of the BitBlt.

BLTRECT Field - ulYExt

ulYExt (ULONG)
Y extent of the BitBlt.

BMAPINFO

Device-dependent bit map information structure.

```
typedef struct _BMAPINFO {
  ULONG     ulLength;        /*  Length of the BMAPINFO data structure, in bytes. */
  ULONG     ulType;          /*  Description of the Blt. */
  ULONG     ulWidth;         /*  Width in pels of the bit map. */
  ULONG     ulHeight;        /*  Height in pels of the bit map. */
  ULONG     ulBpp;           /*  Number of bits per pel/color depth. */
  ULONG     ulBytesPerLine;  /*  Number of aligned bytes per line. */
  PBYTE     pBits;           /*  Pointer to bit-map bits. */
} BMAPINFO;

typedef BMAPINFO *PBMAPINFO;
```

BMAPINFO Field - ulLength

ulLength (ULONG)
Length of the BMAPINFO data structure, in bytes.

BMAPINFO Field - ulType

ulType (ULONG)
Description of the Blt.

BMAP_VRAM	Bit map is in video memory.
BMAP_MEMORY	Bit map is in system memory.
BMAP_VERTICAL_SCAN	Scan lines are in a vertical format instead of the default horizontal format.
BMAP_FONT	BitBlit is a set of font character glyphs.

BMAPINFO Field - ulWidth

ulWidth (ULONG)
Width in pels of the bit map.

BMAPINFO Field - ulHeight

ulHeight (ULONG)
Height in pels of the bit map.

BMAPINFO Field - ulBpp

ulBpp (ULONG)
Number of bits per pel/color depth.

BMAPINFO Field - ulBytesPerLine

ulBytesPerLine (ULONG)
Number of aligned bytes per line.

BMAPINFO Field - pBits

pBits (PBYTE)
Pointer to bit-map bits.

The scanlines of the bit map can be either byte-aligned or word-aligned.

BOOL

Boolean.

Valid values are:

- FALSE, which is 0
- TRUE, which is 1

```
typedef unsigned long BOOL;
```

BOXDATA

Defines the dimensions of a box.

```
typedef struct _BOXDATA {
  POINTL     ptlOpposite;  /*  Defines opposite corner of the box. */
  LONG       IHRound;      /*  Horizontal length of the full axis of an ellipse. */
  LONG       IVRound;      /*  Vertical length of the full axis of an ellipse. */
} BOXDATA;

typedef BOXDATA *PBOXDATA;
```

BOXDATA Field - ptlOpposite

ptlOpposite (POINTL)
Defines opposite corner of the box.

If COM_TRANSFORM is not set, the function expects the point to be in screen coordinates.

x X-coordinate of opposite corner

y Y-coordinate of opposite corner

BOXDATA Field - lHRound

lHRound (LONG)
Horizontal length of the full axis of an ellipse.

This field is used for rounding each corner.

BOXDATA Field - lVRound

lVRound (LONG)
Vertical length of the full axis of an ellipse.

This field is used for rounding each corner.

BOXPARAMS

Defines the dimensions of a box.

```
typedef struct _BOXPARAMS {
  POINTL     ptl;             /*  Defines opposite corner of the box. */
  LONG       sizl.cx;  /*  Horizontal length of the full axis of an ellipse. */
  LONG       sizl.cy;  /*  Vertical length of the full axis of an ellipse. */
} BOXPARAMS;

typedef BOXPARAMS *PBOXPARAMS;
```

BOXPARAMS Field - ptl

ptl (POINTL)
Defines opposite corner of the box.

If COM_TRANSFORM is not set, the function expects the point to be in screen coordinates.

x	X-coordinate of opposite corner
y	Y-coordinate of opposite corner

BOXPARAMS Field - sizl.cx

sizl.cx (LONG)
Horizontal length of the full axis of an ellipse.

This field is used for rounding each corner.

BOXPARAMS Field - sizl.cy

sizl.cy (LONG)
Vertical length of the full axis of an ellipse.

This field is used for rounding each corner.

BYTE

A byte.

```
typedef unsigned char BYTE;
```

CHAR

Single-byte character.

```
#define CHAR char
```

CHARBUNDLE

Character-attributes bundle structure.

```
typedef struct _CHARBUNDLE {
  LONG       lColor;          /*  Character foreground color. */
  LONG       lBackColor;      /*  Character background color. */
  USHORT     usMixMode;       /*  Character foreground-mix mode. */
  USHORT     usBackMixMode;   /*  Character background-mix mode. */
  USHORT     usSet;           /*  Character set. */
  USHORT     usPrecision;     /*  Character precision. */
```

```
  SIZEF      sizfxCell;      /*  Character cell size. */
  POINTL     ptlAngle;       /*  Character angle. */
  POINTL     ptlShear;       /*  Character shear. */
  USHORT     usDirection;    /*  Character direction. */
  USHORT     usTextAlign;    /*  Text alignment. */
  FIXED      fxExtra;        /*  Character extra. */
  FIXED      fxBreakExtra;   /*  Character break extra. */
} CHARBUNDLE;

typedef CHARBUNDLE *PCHARBUNDLE;
```

CHARBUNDLE Field - lColor

lColor (LONG)
Character foreground color.

CHARBUNDLE Field - lBackColor

lBackColor (LONG)
Character background color.

CHARBUNDLE Field - usMixMode

usMixMode (USHORT)
Character foreground-mix mode.

CHARBUNDLE Field - usBackMixMode

usBackMixMode (USHORT)
Character background-mix mode.

CHARBUNDLE Field - usSet

usSet (USHORT)
Character set.

CHARBUNDLE Field - usPrecision

usPrecision (USHORT)
Character precision.

CHARBUNDLE Field - sizfxCell

sizfxCell (SIZEF)
Character cell size.

CHARBUNDLE Field - ptlAngle

ptlAngle (POINTL)
Character angle.

CHARBUNDLE Field - ptlShear

ptlShear (POINTL)
Character shear.

CHARBUNDLE Field - usDirection

usDirection (USHORT)
Character direction.

CHARBUNDLE Field - usTextAlign

usTextAlign (USHORT)
Text alignment.

CHARBUNDLE Field - fxExtra

fxExtra (FIXED)
Character extra.

CHARBUNDLE Field - fxBreakExtra

fxBreakExtra (FIXED)
Character break extra.

CODECINFO

Determines whether data is to be compressed or decompressed.

```
typedef struct _CODECINFO {
  ULONG      ulLength;      /*  Size of the CODECINFO data structure, in bytes. */
  FOURCC     fccCodecType;  /*  Describes compression type; 'RAW' if uncompressed. */
  ULONG      ulCodecCaps;   /*  Flag indicating CODEC capabilities. */
} CODECINFO;

typedef CODECINFO *PCODECINFO;
```

CODECINFO Field - ulLength

ulLength (ULONG)
Size of the CODECINFO data structure, in bytes.

CODECINFO Field - fccCodecType

fccCodecType (FOURCC)
Describes compression type; 'RAW' if uncompressed.

The FOURCC flags have the following values:

CODEC_ULTIMOTION	"ULTI"
CODEC_INDEO21	"RT21"
CODEC_INDEO31	"IV31"
CODEC_MPEG1	"MPG1"
CODEC_MPEG2	"MPG2"
CODEC_MJPEG	"MJPG"

CODECINFO Field - ulCodecCaps

ulCodecCaps (ULONG)
Flag indicating CODEC capabilities.

The following values are valid:

CODEC_ACCEL_STRETCHBLT	0x0002; hardware stretch
CODEC_ACCEL_BLTCOPROC	0x0002; hardware Blt
CODEC_ACCEL_DECOMP	0x0004; has hardware decompression
CODEC_ACCEL_COMP	0x0008; has hardware compression

COLORINFO

Color formats for source and destination data, organized in order of preference from the driver's perspective. This data structure is required in any driver supporting EnDIVE.

```
typedef struct _COLORINFO {
  ULONG      ulLength;          /*  Size of the COLORINFO data structure, in bytes. */
  FOURCC     fccColorEncoding;  /*  Field containing a name for the color space. */
} COLORINFO;

typedef COLORINFO *PCOLORINFO;
```

COLORINFO Field - ulLength

ulLength (ULONG)
Size of the COLORINFO data structure, in bytes.

COLORINFO Field - fccColorEncoding

fccColorEncoding (FOURCC)
Field containing a name for the color space.

The FOURCC represents a name (for example, RGB8, Y24, and so on). The bits per pixel and data layout of the color space is implied by the name.

This field can have the following values:

FOURCC(ch0,ch1,ch2,ch3)

```
((ULONG)(BYTE)(ch0) | ((ULONG)(BYTE)(ch1) <<8) | \
((ULONG)(BYTE)(CH2)<<16) | ((ULONG)(BYTE)(CH3)<<24))
```

FOURCC_LUT8 FOURCC('L','U','T','8')	8-bit palettized color space
FOURCC_R565 FOURCC('R','5','6','5')	RGB 565
FOURCC_R555 FOURCC('R','5','5','5')	RGB 555

FOURCC_R666 FOURCC('R','6','6','6')	RGB 666
FOURCC_R664 FOURCC('R','6','6','4')	RGB 664
FOURCC_RGB3 FOURCC('R','G','B','3')	RGB 24 in 3 bytes
FOURCC_BGR3 FOURCC('B','G','R','3')	BGR 24 in 3 bytes
FOURCC_RGB4 FOURCC('R','G','B','4')	RGB 24 in 4 bytes
FOURCC_BGR4 FOURCC('B','G','R','4')	BGR 24 in 4 bytes
FOURCC_Y888 FOURCC('Y','8','8','8')	YUV 24
FOURCC_Y411 FOURCC('Y','4','1','1')	YUV 411 interleaved 4 x 1 subsampled
FOURCC_Y422 FOURCC('Y','4','2','2')	YUV 422 (CCIR601)
FOURCC_YUV9 FOURCC('Y','U','V','9')	YUV9
FOURCC_Y2X2 FOURCC('Y','2','X','2')	YUV 2 by 2 subsampled multiplane
FOURCC_Y4X4 FOURCC('Y','4','X','4')	YUV 4 by 4 subsampled multiplane

---

CUSTPALINFO

Custom palette information.

```
typedef struct _CUSTPALINFO {
  ULONG     ulLength;      /*  Size of the CUSTPALINFO data structure, in bytes. */
  ULONG     fFlags;        /*  Palette flag. */
  ULONG     ulStartIndex;  /*  Starting palette index. */
  ULONG     ulNumEntries;  /*  Number of palette slots to query or set. */
  PRGB2     pRGBs;         /*  Pointer to the array of RGB values. */
} CUSTPALINFO;

typedef CUSTPALINFO *PCUSTPALINFO;
```

CUSTPALINFO Field - ulLength

ulLength (ULONG)
Size of the CUSTPALINFO data structure, in bytes.

CUSTPALINFO Field - fFlags

fFlags (ULONG)
Palette flag.

Not used at this time.

CUSTPALINFO Field - ulStartIndex

ulStartIndex (ULONG)
Starting palette index.

CUSTPALINFO Field - ulNumEntries

ulNumEntries (ULONG)
Number of palette slots to query or set.

CUSTPALINFO Field - pRGBs

pRGBs (PRGB2)
Pointer to the array of RGB values.

DEVICESURFACE

Pointer to information needed by the graphics engine that represents the device's physical surface.

```
typedef struct _DEVICESURFACE {
  ULONG            ulLength;                                                    /*  Length of the DEVICESURF
  BMAPINFO         SurfaceBmapInfo;                                             /*  Information for rasterin
  ULONG            ulDSFlgs;                                                    /*  Flags for rendering of r
  ULONG            ulStyleRatio;                                                /*  Default style ratio used
  BMAPINFO         abmapinfoDefPattern[DEFAULT_PATTERNS_NUMBER];  /*  Set of default patterns. */
  PVOID            pHWPalette;                                                  /*  Default hardware palette
  DITHERMATRIX     DitherMatrix;                                                /*  Default to generic dithe
  PFN              pfnDevLockDC;                                                /*  Default pointer to devic
  PFN              pfnDevUnLockDC;                                              /*  Default pointer to relea
  PFN              pfnBitBlt;                                                   /*  Pointer defaults to SOFTI
  PFN              pfnLine;                                                     /*  Pointer defaults to SOFTI
  PFN              pfnReserved;                                                 /*  Reserved by system. */
  PFN              pfnLockPhysDev;                                              /*  Prevents access to physi
  PFN              pfnUnLockPhysDev;                                            /*  Releases the physical de
  ULONG            ulReserved[5];                                  /*  Reserved by system. */
  ULONG            ulCapsCnt;                                                   /*  Size in ULONGs of the De
  ULONG            DevCaps[CAPS_MAX_CAPS + 1];              /*  The device capabilities array in OS/2 2.0 and p
ious versions. */
} DEVICESURFACE;

typedef DEVICESURFACE *PDEVICESURFACE;
```

DEVICESURFACE Field - ulLength

ulLength (ULONG)
Length of the DEVICESURFACE data structure.

DEVICESURFACE Field - SurfaceBmapInfo

SurfaceBmapInfo (BMAPINFO)
Information for rastering to a linear address.

Information used by the graphics engine when rastering to a linear address. (See BMAPINFO in this Appendix.) Device driver must include this field.

DEVICESURFACE Field - ulDSFlgs

ulDSFlgs (ULONG)
Flags for rendering of rasterized data.

Miscellaneous flags used by the graphics engine. Device driver must include this field.

Color format flag	DS_MONO_INVERT
Destination color format flags	DS_COLOR_RGB
	DS_COLOR_RGB2
	DS_COLOR_24BIT_BGR
	DS_COLOR_24BIT_RGB
	DS_COLOR_16BIT_565_BGR
	DS_COLOR_16BIT_565_RGB
	DS_COLOR_16BIT_1555_BGR
	DS_COLOR_16BIT_1555_RGB
Direction flags	DS_BOTTOMTOP
	DS_TOPBOTTOM
Hardcopy flag	DS_QUEUED_RAW_DEFAULT
Notification flag	DS_NOTIFY_LAST_SDBITBLT
Scanline alignment flags	DS_BYTE_ALIGNMENT
	DS_2BYTE_ALIGNMENT
	DS_4BYTE_ALIGNMENT
	DS_KEEP_EXTFORMAT
Surface description flag	DS_MEMORY

DEVICESURFACE Field - ulStyleRatio

ulStyleRatio (ULONG)
Default style ratio used by the current device.

Device driver must include this field.

DEVICESURFACE Field - abmapinfoDefPattern[DEFAULT_PATTE

abmapinfoDefPattern[DEFAULT_PATTERNS_NUMBER] (BMAPINFO)
Set of default patterns.

Default patterns supplied by the graphics engine that can be changed by the presentation driver. [DEFAULT_PATTERNS_NUMBER].

DEVICESURFACE Field - pHWPalette

pHWPalette (PVOID)
Default hardware palette for the current device.

DEVICESURFACE Field - DitherMatrix

DitherMatrix (DITHERMATRIX)
Default to generic dither support.

Defaults to the generic dither support for hardcopy devices.

DEVICESURFACE Field - pfnDevLockDC

pfnDevLockDC (PFN)
Default pointer to device-specific information.

Pointer that allows the device driver to retrieve its device-specific information from the graphics engine. Pointer is stored by the device driver.

DEVICESURFACE Field - pfnDevUnLockDC

pfnDevUnLockDC (PFN)
Default pointer to release DC lock.

Pointer to the function to release the lock on the current DC. Pointer is stored by the device driver.

DEVICESURFACE Field - pfnBitBlt

pfnBitBlt (PFN)
Pointer defaults to SOFTDRAW's handling of the SDBitBlt function.

If a device does not have linear address support or would rather handle preclipped bit map transfers itself, then it should hook out this function with its own routine.

DEVICESURFACE Field - pfnLine

pfnLine (PFN)
Pointer defaults to SOFTDRAW's handling of the SDLINE function.

If a device does not have linear address support or would rather handle preclipped line operations itself, then it should hook out this function with its own routine.

DEVICESURFACE Field - pfnReserved

pfnReserved (PFN)
Reserved by system.

DEVICESURFACE Field - pfnLockPhysDev

pfnLockPhysDev (PFN)
Prevents access to physical device.

Prevents other processes from accessing the physical hardware between the lock and unlock.

DEVICESURFACE Field - pfnUnLockPhysDev

pfnUnLockPhysDev (PFN)
Releases the physical device.

Releases the physical hardware for access by other processes.

DEVICESURFACE Field - ulReserved[5]

ulReserved[5] (ULONG)
Reserved by system.

DEVICESURFACE Field - ulCapsCnt

ulCapsCnt (ULONG)
Size in ULONGs of the *DevCaps* field.

Device driver must include this field.

DEVICESURFACE Field - DevCaps[CAPS_MAX_CAPS + 1]

DevCaps[CAPS_MAX_CAPS + 1] (ULONG)
The device capabilities array in OS/2 2.0 and previous versions.

Device driver must include this field. Array must be filled out in order for the graphics engine to handle a call to the DevQueryCaps() function. See *Presentation Manager Programming Reference, Volume 1* for more information on DevQueryCaps().

DEVOPENSTRUC

Open-device data structure.

```
typedef struct _DEVOPENSTRUC {
  PSZ           pszLogAddress;       /*  Logical address. */
  PSZ           pszDriverName;       /*  Driver name. */
  PDRIVDATA     pdriv;               /*  Driver data. */
  PSZ           pszDataType;         /*  Data type. */
  PSZ           pszComment;          /*  Comment. */
  PSZ           pszQueueProcName;    /*  Queue-processor name. */
  PSZ           pszQueueProcParams;  /*  Queue-processor parameters. */
  PSZ           pszSpoolerParams;    /*  Spooler parameters. */
  PSZ           pszNetworkParams;    /*  Network parameters. */
} DEVOPENSTRUC;

typedef DEVOPENSTRUC *PDEVOPENSTRUC;
```

DEVOPENSTRUC Field - pszLogAddress

pszLogAddress (PSZ)
Logical address.

This is required for an OD_DIRECT device being opened with DevOpenDC; it is the logical device address, such as "LPT1" on OS/2. Some drivers may accept a file name for this parameter or even a named pipe.

Where output is to be queued (for an OD_QUEUED device), this is the name of the queue for the output device. The queue name can be a UNC name.

Note: This parameter can be a port name for a printer device context.

DEVOPENSTRUC Field - pszDriverName

pszDriverName (PSZ)
Driver name.

Character string identifying the printer driver, for example, LASERJET. The Default Device Driver field of the PRQINFO3 structure, associated with the required print queue, gives the driver and device name, separated by a period, for example LASERJET.HP LaserJet IIID. It can contain only the name up to the period, for example LASERJET.

DEVOPENSTRUC Field - pdriv

pdriv (PDRIVDATA)
Driver data.

Data that is to be passed directly to the PM device driver. Whether any of this is required depends upon the device driver.

For printer device context, this is a pointer to the job properties data.

DEVOPENSTRUC Field - pszDataType

pszDataType (PSZ)
Data type.

For an OD_QUEUED or OD_DIRECT device, this parameter defines the type of data that is to be queued as follows:

PM_Q_STD	Standard format
PM_Q_RAW	Raw format

Note that a device driver can define other data types.

For OD_QUEUED or OD_DIRECT defice types, the default is supplied by the device driver if *pszDataType* is not specified. For any other device type, *pszDataType* is ignored.

DEVOPENSTRUC Field - pszComment

pszComment (PSZ)
Comment.

Optional character string that the printer object displays to the user in a job settings notebook. It is recommended that the application include its own name in this comment string.

Note: The job title text is derived from the document name passed to DevEscape (DEVESC_STARTDOC).

DEVOPENSTRUC Field - pszQueueProcName

pszQueueProcName (PSZ)
Queue-processor name.

This is the name of the queue processor, for queued output, and is usually the default.

DEVOPENSTRUC Field - pszQueueProcParams

pszQueueProcParams (PSZ)

Queue-processor parameters.

Queue processor parameters (optional). They can include information such as the number of copies you want to print and the size of the output area on the printed page.

The first parameter (*COP*) is used for all spool-file formats. The remaining parameters are valid for PM_Q_STD spool files only. Because PM_Q_STD data are used mainly for *graphic* data, these parameters are described in relation to the printing of picture files.

The PMPRINT/PMPLOT queue-processor parameters are separated by spaces and are:

COP=*n*

The *COP* parameter specifies the number of copies of the spool file that you want printed. The value of *n* must be an integer in the range of 1 through 999.

The default is *COP=1* .

ARE=C | *w,h,l,t*

The *ARE* parameter determines the size and position of the output area. This is the area of the physical page to which printing is restricted.

The default value of *ARE=C* means that the output area is the whole page. Note, however, that the printer cannot print outside its own device clip limits.

To size and position the output area at a specific point on the page, use *ARE= w,h,l,t* , where:

w, h	are the width and height of the desired output area.
l, t	are the offsets of the upper-left corner of the output area from the left (l) and from the top (t) of the maximum output area.

These four values must be given as percentages of the maximum output dimensions. The maximum output area is the area within the device clip limits.

FIT=S | *l,t*

The *FIT* parameter determines which part of the picture is to be printed. You can request the whole of the picture, scaled to fit the output area; or you can position the picture (actual size) anywhere within the output area. This could mean that the picture is clipped at the boundaries of the output area.

The default value of *FIT=S* causes the output to be scaled until the larger of the height or width just fits within the defined output area. The aspect ratio of the picture is maintained.

To print the picture in actual size, use *FIT= l,t* , where *l,t* are the coordinates of the point in the picture that you want positioned at the center of the output area: *l* is measured from the left edge of the picture; and *t* is measured from the top edge. The coordinates must be given as percentages of the actual dimensions of the picture.

XFM=0 | 1

The *XFM* parameter enables you to override the picture-positioning and clipping instructions that are provided by the *ARE* and *FIT* parameters, including their defaults.

The default value of *XFM=1* allows the appearance of the output to be determined by the settings of the *ARE* and *FIT* parameters.

A value of *XFM=0* yields output as specified in the picture file. For example, applications that use many different forms can define different positions on each form for their output.

COL=M | C

The *COL* parameter enables you to specify color output if you have a color printer.

A value of *COL=M* creates monochrome output (black foreground with no background color). This is supported by all devices.

A value of *COL=C* creates color output. If you request color output on a monochrome device, the printer presentation driver tries to satisfy your request, which can cause problems because the only color available is black. For example, if the picture file specifies a red line on a blue background, both are drawn in black.

The default is *COL=M* when you are addressing a monochrome printer and *COL=C* when you are addressing a color printer.

MAP=N | A

The *MAP* parameter enables you to decide how the *neutral* colors (those that are not specified in the picture file) are printed.

The default value of *MAP=N* yields a *normal* representation of the screen picture on a printed page, which means that the page background is white and the foreground is black.

A value of *MAP=A* provides the reverse of the normal representation: the background is black and the foreground is white on the printed page.

CDP=codepage

The *CDP* parameter overrides the code page to being used for PM_Q_RAW print jobs. The print queue driver uses DEVESC_SETMODE to set the code page, but not all printer drivers support this device escape.

XLT=0 | 1

The *XLT* parameter can eliminate the translation component when printing a metafile if *XLT=1* .

When the resolution of the device is higher than that of the world coordinate space, a small translation of world coordinate point (0,0) occurs on the device to preserve the accuracy of the mapping from world to device coordinate units. For example, (0,0) becomes (1,1) if there are 3 pels to every world coordinate.

Normally, this is not noticeable, but it can be a problem with some devices. For example, in order to draw a complete row of 80 characters using a device font, a device may require the text to start at device coordinate position zero. Starting at a position other than zero may cause one or more characters at the end of the row to be clipped. In such cases, elimination of the translation is important and can be accomplished by specifying *XLT=1* .

The default is *XLT=0* .

DEVOPENSTRUC Field - pszSpoolerParams

pszSpoolerParams (PSZ)
Spooler parameters.

Spooler parameters (optional) are separated by spaces. They are used for scheduling print jobs and are as follows:

- The form names that identify the paper to be used, for example, `FORM=A4,A5,ENV`. The form names are optional; but if they are provided, the spooler is able to hold off printing the jobs until the required form is installed in the printer. If the form name is not provided, the spooler attempts to print the job. The printer driver recognizes that there is a forms problem and displays a FORMS MISMATCH message box.
- Priority of the print job, for example, *PRTY=60* . The priority is specified as an integer in the range 1 through 99; 99 is the highest. The default priority value is 50. The application can use the spooler priority parameter to prioritize its own jobs; however, it is not good practice for an application always to use priority 99 in an attempt to get its jobs printed first.

DEVOPENSTRUC Field - pszNetworkParams

pszNetworkParams (PSZ)
Network parameters.

Optional parameter that can be used to specify network options; for example, *USER=JOESMITH* .

DBE_FDI_PARAMETERS

FONTMGRPARM data structure.

```
typedef struct _DBE_FDI_PARAMETERS {
  ULONG             fdi_1Match;
  ULONG             fdi_1FunctionID;
```

```
  ULONG             fdi_1CodePoint;
  VOID _far16      *fdi_pFontInfo;     /*  16:16 pointer. */
} DBE_FDI_PARAMETERS;

typedef DBE_FDI_PARAMETERS *PDBE_FDI_PARAMETERS;
```

DBE_FDI_PARAMETERS Field - fdi_1Match

fdi_1Match (ULONG)

DBE_FDI_PARAMETERS Field - fdi_1FunctionID

fdi_1FunctionID (ULONG)

DBE_FDI_PARAMETERS Field - fdi_1CodePoint

fdi_1CodePoint (ULONG)

DBE_FDI_PARAMETERS Field - fdi_pFontInfo

fdi_pFontInfo (VOID _far16 *)
16:16 pointer.

DJP_ITEM

This structure is used to set and query printer properties or job properties.

```
typedef struct _DJP_ITEM {
  ULONG     cb;             /*  Size of the DJP_ITEM structure. */
  ULONG     ulProperty;     /*  Identifies specific job property. */
  LONG      lType;          /*  Property type. */
  ULONG     ulNumReturned;  /*  Number of elements that have been returned. */
  ULONG     ulValue;        /*  Size is variable based on ulProperty. */
} DJP_ITEM;

typedef DJP_ITEM *PDJP_ITEM;
```

DJP_ITEM Field - cb

cb (ULONG)
Size of the DJP_ITEM structure.

The list of DJP_ITEM items are of variable size. Therefore, you must choose an appropriate method to determine the size of the item depending on the complexity. Simple items can use sizeof (DJP_ITEM). Complex items should use DJP_HEADER_SIZE and the size of the corresponding type. For example:

```
/* simple   */
pDJP->cb = sizeof (DJP_ITEM);

/* complex  */
pDJP->cb = DJP_HEADER_SIZE + sizeof (DJPT_FORM);
```

To move to the next structure, use the DJP_NEXT_STRUCTP() macro:

```
pDJP = DJP_NEXT_STRUCTP (pDJP);
```

DJP_ITEM Field - ulProperty

ulProperty (ULONG)
Identifies specific job property.

The 'S' and 'C' modifiers in the job property name indicate the complexity of *ulValue: simple* or *complex* . Simple types are ULONG in size and can be set directly using *ulValue* . Complex types are structures. For an example of how to set them, see the sample code under *ulValue* .

The 'J' and 'P' modifiers indicate the class of the property, where 'J' is a *job* and 'P' is a *printer* property.

Valid job properties are as follows:

DJP_NONE - 0L — Also used as end-of-list marker
DJP_SJ_ORIENTATION - 1L
DJP_CJ_RESOLUTION - 2L
DJP_SJ_BITSPERPEL - 3L
DJP_SJ_COLOR - 4L
DJP_SJ_PRINTQUALITY - 5L
DJP_SJ_PAPERSIZE - 6L
DJP_SJ_TRAYTYPE - 7L
DJP_SJ_MEDIA - 8L
DJP_SJ_MEDIA_COLOR - 9L
DJP_CJ_FORM - 10L
DJP_CJ_MIXEDFORMS - 11L
DJP_SJ_FONTDOWNLOADING - 12L
DJP_SJ_DUPLEX - 13L
DJP_SJ_COLLATE - 14L
DJP_SJ_FEED - 15L
DJP_SJ_COPIES - 16L
DJP_SJ_SCALING - 17L
DJP_SJ_FORMFEEDCONTROL - 18L
DJP_SJ_N_UP - 19L

DJP_ITEM Field - lType

lType (LONG)
Property type.

Valid input values are as follows:

DJP_ALL
The application requests all possible values. The list of values will be placed starting at the *ulValue* field.

DJP_CURRENT
The application requests the current value be taken from the input job properties (use the value from *ulProperty*).

Valid output values are as follows:

DJP_ALL
Unchanged; successful query

DJP_CURRENT
Unchanged; successful query

DJP_ERROR_NOT_SUPPORTED
Driver does not support the value in the *ulProperty* field

DJP_ERROR_OUT_OF_RANGE
The value in the *ulValue* field is not in the valid range.

DJP_ERROR_NOT_ENUM
This cannot be enumerated (for example, DJP_SJ_COPIES)

DJP_ERROR_INV_PARMS
The value in *ulValue* is not valid

DJP_ITEM Field - ulNumReturned

ulNumReturned (ULONG)
Number of elements that have been returned.

DJP_ITEM Field - ulValue

ulValue (ULONG)
Size is variable based on *ulProperty* .

There are two types of complexity: simple and complex. The _Sx_ and _Cx_ modifiers in the property definitions identify the complexity of *ulValue* . Simple types are ULONG in size and can be set directly using *ulValue*. Complex types are structures. To view them, first declare the corresponding property's type definition. Then choose either to use the DJP_SET_ELEMENT() macro to set the value, or to use the DJP_ELEMENTP() macro to return a pointer to the structure, as shown in the sample code below:

```
/* simple DJP_SJ_COLOR:
*/
PDJP_ITEM        pDJP;

if (DJP_CLR_MONOCHROME == pDJP->ulValue)
```

```
   printf ("Monochrome\n");
else if (DJP_CLR_COLOR == pDJP->ulValue)
   printf ("Color\n");

/* complex DJP_CJ_RESOLUTION:
*/
PDJP_ITEM         pDJP;
DJPT_RESOLUTION   pElm;

pElm = DJP_ELEMENTP  (*pDJP, DJPT_RESOLUTION);

printf ("The resolution is (%d, %d)\n",
        pElm->usXResolution,
        pElm->usYResolution);
```

If you choose to use the DJP_ELEMENT() macro, code similar to the following may be used:

```
/* simple DJP_SJ_COLOR:
*/
PDJP_ITEM         pDJP;

PDJP->ulValue = DJP_CLR_COLOR;

/* complex DJP_CJ_RESOLUTION:
*/
PDJP_ITEM         pDJP;
DJPT_RESOLUTION   djpResolution;

djpResolution.usXResolution = 720;
djpResolution.usYResolution = 720;

DJP_SET_ELEMENT (*pDJP, DJPT_RESOLUTION, djpResolution);
```

DRIVDATA

Driver-data structure.

```
typedef struct _DRIVDATA {
  LONG     cb;                /*  Length. */
  LONG     lVersion;          /*  Version. */
  CHAR     szDeviceName[32];  /*  Device name. */
  CHAR     abGeneralData[1];  /*  General data. */
} DRIVDATA;

typedef DRIVDATA *PDRIVDATA;
```

DRIVDATA Field - cb

cb (LONG)
Length.

The length of the structure.

DRIVDATA Field - lVersion

lVersion (LONG)
Version.

The version number of the data. Version numbers are defined by particular PM device drivers.

DRIVDATA Field - szDeviceName[32]

szDeviceName[32] (CHAR)
Device name.

A string in a 32-byte field identifying the particular device (model number, and so on). Again, valid values are defined by PM device drivers.

DRIVDATA Field - abGeneralData[1]

abGeneralData[1] (CHAR)
General data.

Data as defined by the Presentation Manager device driver.

The data type of this field is defined by the Presentation Manager device driver. It does not contain pointers, as these are not necessarily valid when passed to the device driver.

ERRINFO

Error-information structure.

```
typedef struct _ERRINFO {
  ULONG       cbFixedErrInfo;  /*  Length of fixed data to this structure. */
  ERRORID     idError;         /*  Error identity. */
  ULONG       cDetailLevel;    /*  Number of levels of detail. */
  ULONG       offaoffszMsg;    /*  Offset to the array of message offsets. */
  ULONG       offBinaryData;   /*  Offset to the binary data. */
} ERRINFO;

typedef ERRINFO *PERRINFO;
```

ERRINFO Field - cbFixedErrInfo

cbFixedErrInfo (ULONG)
Length of fixed data to this structure.

ERRINFO Field - idError

idError (ERRORID)
Error identity.

This is identical to the value returned by WinGetLastError.

ERRINFO Field - cDetailLevel

cDetailLevel (ULONG)
Number of levels of detail.

This is the number of entries in the array of words pointed to by the following field. One level of detail is provided.

ERRINFO Field - offaoffszMsg

offaoffszMsg (ULONG)
Offset to the array of message offsets.

This is an offset to an array of 16-bit offsets to null-terminated strings. Each string is a printable message that offers varying levels of information. The first level is the least amount of detail, and the remaining levels offer more and more detail.

The first level of detail is always an error message string, in the following format:

```
xxxnnnns

where    xxx    is the product identifier
         nnnn   is the message number
         s      is the message severity letter
                  W = warning
                  E = error
                  S = severe error
                  U = unrecoverable
```

ERRINFO Field - offBinaryData

offBinaryData (ULONG)
Offset to the binary data.

This can contain additional information relating to the error.

ERRORID

Error identity.

```
typedef ULONG ERRORID;
```

ESCSETMODE

Structure for setting printer mode. See DevEscape

```
typedef struct _ESCSETMODE {
  ULONG      mode;      /*  Mode to be set. */
  USHORT     codepage;  /*  Code page. */
} ESCSETMODE;

typedef ESCSETMODE *PESCSETMODE;
```

This data structure is a specific-case version of the ESCMODE data structure, used to set the code page of a printer.

ESCSETMODE Field - mode

mode (ULONG)
Mode to be set.

0	Set mode to specified code page. Any font can be used.

ESCSETMODE Field - codepage

codepage (USHORT)
Code page.

If zero is specified for the code page, the printer is set to the hardware default.

FATTRS

Font-attributes structure.

```
typedef struct _FATTRS {
  USHORT     usRecordLength;        /*  Length of record. */
  USHORT     fsSelection;           /*  Selection indicators. */
  LONG       lMatch;                /*  Matched-font identity. */
  CHAR       szFacename[FACESIZE];  /*  Typeface name. */
  USHORT     idRegistry;            /*  Registry identifier. */
  USHORT     usCodePage;            /*  Code page. */
  LONG       lMaxBaselineExt;       /*  Maximum baseline extension. */
  LONG       lAveCharWidth;         /*  Average character width. */
  USHORT     fsType;                /*  Type indicators. */
  USHORT     fsFontUse;             /*  Font-use indicators. */
} FATTRS;

typedef FATTRS *PFATTRS;
```

FATTRS Field - usRecordLength

usRecordLength (USHORT)
Length of record.

FATTRS Field - fsSelection

fsSelection (USHORT)
Selection indicators.

Flags causing the following features to be simulated by the system.

Note: If an italic flag is applied to a font that is itself defined as italic, the font is slanted further by italic simulation.

Underscore or strikeout lines are drawn using the appropriate attributes (for example, color) from the character bundle (see the CHARBUNDLE datatype), not the line bundle (see LINEBUNDLE). The width of the line, and the vertical position of the line in font space, are determined by the font. Horizontally, the line starts from a point in font space directly above or below the start point of each character, and extends to a point directly above or below the escapement point for that character.

For this purpose, the start and escapement points are those applicable to left-to-right or right-to-left character directions (see GpiSetCharDirection in *Graphics Programming Interface Programming Reference*), even if the string is currently being drawn in a top-to-bottom or bottom-to-top direction.

For left-to-right or right-to-left directions, any white space generated by the character extra and character break extra attributes (see GpiSetCharExtra and GpiSetCharBreakExtra in *Graphics Programming Interface Programming Reference*), as well as increments provided by the vector of increments on GpiCharStringPos and GpiCharStringPosAt, are also underlined/overstruck, so that in these cases the line is continuous for the string.

FATTR_SEL_ITALIC	Generate *italic* font.
FATTR_SEL_UNDERSCORE	Generate underscored font.
FATTR_SEL_BOLD	Generate **bold** font. (Note that the resulting characters are wider than those in the original font.)
FATTR_SEL_STRIKEOUT	Generate font with overstruck characters.
FATTR_SEL_OUTLINE	Use an outline font with hollow characters. If this flag is not set, outline font characters are filled. Setting this flag normally gives better performance, and for sufficiently small characters (depending on device resolution) there may be little visual difference.

FATTRS Field - lMatch

lMatch (LONG)
Matched-font identity.

FATTRS Field - szFacename[FACESIZE]

szFacename[FACESIZE] (CHAR)
Typeface name.

The typeface name of the font, for example, Tms Rmn.

FATTRS Field - idRegistry

idRegistry (USHORT)
Registry identifier.

Font registry identifier (zero if unknown).

FATTRS Field - usCodePage

usCodePage (USHORT)
Code page.

If zero, the current Gpi code page (see GpiSetCp in *Graphics Programming Interface Programming Reference*) is used. A subsequent GpiSetCp function changes the code page used for this logical font.

FATTRS Field - lMaxBaselineExt

lMaxBaselineExt (LONG)
Maximum baseline extension.

For raster fonts, this should be the height of the required font, in world coordinates.

For outline fonts, this should be zero.

FATTRS Field - lAveCharWidth

lAveCharWidth (LONG)
Average character width.

For raster fonts, this should be the width of the required font, in world coordinates.

For outline fonts, this should be zero.

FATTRS Field - fsType

fsType (USHORT)
Type indicators.

FATTR_TYPE_KERNING	Enable kerning (PostScript only).
FATTR_TYPE_MBCS	Font for mixed single- and double-byte code pages.
FATTR_TYPE_DBCS	Font for double-byte code pages.
FATTR_TYPE_ANTIALIASED	Antialiased font required. Only valid if supported by the device driver.

FATTRS Field - fsFontUse

fsFontUse (USHORT)
Font-use indicators.

These flags indicate how the font is to be used. They affect presentation speed and font quality.

FATTR_FONTUSE_NOMIX	Text is not mixed with graphics and can be written without regard to any interaction with graphics objects.
FATTR_FONTUSE_OUTLINE	Select an outline (vector) font. The font characters can be used as part of a path definition. If this flag is not set, an outline font might or might not be selected. If an outline font is selected, however, character widths are rounded to an integral number of pels.
FATTR_FONTUSE_TRANSFORMABLE	Characters can be transformed (for example, scaled, rotated, or sheared).

FBINFO

Information on frame buffer characteristics.

```
typedef struct _FBINFO {
  ULONG     ulLength;          /*  Length of FBINFO data structure, in bytes. */
  ULONG     ulFlags;           /*  Specifies the capabilities supported. */
  ULONG     ulBPP;             /*  Screen bits per pel. */
  ULONG     ulXres;            /*  Number of screen X pels. */
  ULONG     ulYres;            /*  Number of screen Y pels. */
  ULONG     ulScanLineBytes;   /*  Number of bytes per scanline. */
  ULONG     fccColorEncoding;  /*  Screen color encoding. */
  ULONG     ulENDIVEDrivers;   /*  Number of EnDIVE drivers installed under GRADD architecture.  */
} FBINFO;

typedef FBINFO *PFBINFO;
```

FBINFO Field - ulLength

ulLength (ULONG)
Length of FBINFO data structure, in bytes.

FBINFO Field - ulFlags

ulFlags (ULONG)
Specifies the capabilities supported.

This flag has the following value:

FB_SUPPORTSVRAMALLOC	0x0010; supports allocation of on-card memory

---

FBINFO Field - ulBPP

ulBPP (ULONG)
Screen bits per pel.

This value may be 32 for some 24-bit color adapters.

FBINFO Field - ulXres

ulXres (ULONG)
Number of screen X pels.

FBINFO Field - ulYres

ulYres (ULONG)
Number of screen Y pels.

FBINFO Field - ulScanLineBytes

ulScanLineBytes (ULONG)
Number of bytes per scanline.

FBINFO Field - fccColorEncoding

fccColorEncoding (ULONG)
Screen color encoding.

FBINFO Field - ulENDIVEDrivers

ulENDIVEDrivers (ULONG)
Number of EnDIVE drivers installed under GRADD architecture.

FIXED

Signed-integer fraction (16:16). This can be treated as a LONG where the value has been multiplied by 65 536.

```
typedef LONG FIXED;
```

FONTMETRICS

Font-metrics structure.

This structure is returned to applications on the GpiQueryFonts and GpiQueryFontMetrics calls and conveys information from the font creator to the application.

```
typedef struct _FONTMETRICS {
  CHAR        szFamilyname[FACESIZE];   /*  Family name. */
  CHAR        szFacename[FACESIZE];     /*  Face name. */
  USHORT      idRegistry;               /*  Registry identifier. */
  USHORT      usCodePage;               /*  Code page. */
  LONG        lEmHeight;                /*  Em height. */
  LONG        lXHeight;                 /*  X height. */
  LONG        lMaxAscender;             /*  Maximum ascender. */
  LONG        lMaxDescender;            /*  Maximum descender. */
  LONG        lLowerCaseAscent;         /*  Lowercase ascent. */
  LONG        lLowerCaseDescent;        /*  Lowercase descent. */
  LONG        lInternalLeading;         /*  Internal leading. */
  LONG        lExternalLeading;         /*  External leading. */
  LONG        lAveCharWidth;            /*  Average character width. */
  LONG        lMaxCharInc;              /*  Maximum character increment. */
  LONG        lEmInc;                   /*  Em increment. */
  LONG        lMaxBaselineExt;          /*  Maximum baseline extent. */
  SHORT       sCharSlope;               /*  Character slope. */
  SHORT       sInlineDir;               /*  Inline direction. */
  SHORT       sCharRot;                 /*  Character rotation. */
  USHORT      usWeightClass;            /*  Weight class. */
  USHORT      usWidthClass;             /*  Width class. */
  SHORT       sXDeviceRes;              /*  X-device resolution. */
  SHORT       sYDeviceRes;              /*  Y-device resolution. */
  SHORT       sFirstChar;               /*  First character. */
  SHORT       sLastChar;                /*  Last character. */
  SHORT       sDefaultChar;             /*  Default character. */
  SHORT       sBreakChar;               /*  Break character. */
  SHORT       sNominalPointSize;        /*  Nominal point size. */
  SHORT       sMinimumPointSize;        /*  Minimum point size. */
  SHORT       sMaximumPointSize;        /*  Maximum point size. */
  USHORT      fsType;                   /*  Type indicators. */
  USHORT      fsDefn;                   /*  Definition indicators. */
  USHORT      fsSelection;              /*  Selection indicators. */
```

```
  USHORT     fsCapabilities;          /*  Font capabilities. */
  LONG       lSubscriptXSize;         /*  Subscript x-size. */
  LONG       lSubscriptYSize;         /*  Subscript y-size. */
  LONG       lSubscriptXOffset;       /*  Subscript x-offset. */
  LONG       lSubscriptYOffset;       /*  Subscript y-offset. */
  LONG       lSuperscriptXSize;       /*  Superscript x-size. */
  LONG       lSuperscriptYSize;       /*  Superscript y-size. */
  LONG       lSuperscriptXOffset;     /*  Superscript x-offset. */
  LONG       lSuperscriptYOffset;     /*  Superscript y-offset. */
  LONG       lUnderscoreSize;         /*  Underscore size. */
  LONG       lUnderscorePosition;     /*  Underscore position. */
  LONG       lStrikeoutSize;          /*  Strikeout size. */
  LONG       lStrikeoutPosition;      /*  Strikeout position. */
  SHORT      sKerningPairs;           /*  Kerning pairs. */
  SHORT      sFamilyClass;            /*  Font family design classification. */
  LONG       lMatch;                  /*  Matched font identity. */
  PANOSE     panose;                  /*  Panose font descriptor. */
} FONTMETRICS;

typedef FONTMETRICS *PFONTMETRICS;
```

FONTMETRICS Field - szFamilyname[FACESIZE]

szFamilyname[FACESIZE] (CHAR)
Family name.

The family name of the font that describes the basic appearance of the font, for example, Times New Roman** This string is null terminated, and therefore is limited to 31 characters in length.

FONTMETRICS Field - szFacename[FACESIZE]

szFacename[FACESIZE] (CHAR)
Face name.

The typeface name that defines the particular font, for example, Times New Roman Bold Italic. This string is null terminated, and therefore is limited to 31 characters in length.

FONTMETRICS Field - idRegistry

idRegistry (USHORT)
Registry identifier.

The IBM registered number (or zero).

FONTMETRICS Field - usCodePage

usCodePage (USHORT)
Code page.

Defines the registered code page supported by the font. For example, the original IBM PC code page is 437. A value of 0 implies that the font may be used with any of the OS/2 supported code pages.

Where a font contains special symbols for which there is no registered code page, then code page 65400 is used.

FONTMETRICS Field - lEmHeight

lEmHeight (LONG)
Em height.

The height of the Em square in world coordinate units. This corresponds to the point size for the font.

FONTMETRICS Field - lXHeight

lXHeight (LONG)
X height.

The nominal height above the baseline for lowercase characters (ignoring ascenders) in world coordinate units.

FONTMETRICS Field - lMaxAscender

lMaxAscender (LONG)
Maximum ascender.

The maximum height above the baseline reached by any part of any symbol in the font in world coordinate units. This field may exceed *lEmHeight* .

FONTMETRICS Field - lMaxDescender

lMaxDescender (LONG)
Maximum descender.

The maximum depth below the baseline reached by any part of any symbol in the font in world coordinate units. This field may exceed *lEmHeight* .

FONTMETRICS Field - lLowerCaseAscent

lLowerCaseAscent (LONG)
Lowercase ascent.

The maximum height above the baseline reached by any part of any lowercase (Latin unaccented "a" through "z") symbol in the font in world coordinate units.

FONTMETRICS Field - lLowerCaseDescent

lLowerCaseDescent (LONG)
Lowercase descent.

The maximum depth below the baseline reached by any part of any lowercase (Latin unaccented "a" through "z") symbol in the font in world coordinate units.

FONTMETRICS Field - lInternalLeading

lInternalLeading (LONG)
Internal leading.

The amount of space which, when subtracted from *lMaxAscender* , gives a font-design dependent, but glyph-set independent, measure of the distance above the baseline that characters extend. This calculation approximates the visual top to a row of characters without actually looking at the characters in the row.

For optimum results, this field should be used by applications to position the first line of a block of text by subtracting it from *lMaxAscender* and positioning the baseline that distance below whatever is above the text.

Note: This does not guarantee that characters will not overwrite information above them, but does give a font designer's view of where to place the text. Collision should be tested for, and additional space allocated if necessary.

FONTMETRICS Field - lExternalLeading

lExternalLeading (LONG)
External leading.

The amount of guaranteed white space advised by the font designer to appear between adjacent rows of text. This value may be zero.

Note: The fonts built in to Presentation Manager have zero in this field.

FONTMETRICS Field - lAveCharWidth

lAveCharWidth (LONG)
Average character width.

This is determined by multiplying the width of each lowercase character by a constant, adding the products, and then dividing by 1000. The letters involved in this, plus their constants, are as follows:

Letter	Constant
a	64
b	14

c	27
d	35
e	100
f	20
g	14
h	42
i	63
j	3
k	6
l	35
m	20
n	56
o	56
p	17
q	4
r	49
s	56
t	71
u	31
v	10
w	18
x	3
y	18
z	2
space	166

Note: For fixed pitch fonts, this value will be the same as the (A width + B width + C width) escapement of each character.

FONTMETRICS Field - lMaxCharInc

lMaxCharInc (LONG)
Maximum character increment.

The maximum character increment for the font in world coordinate units.

FONTMETRICS Field - lEmInc

lEmInc (LONG)
Em increment.

The width of the Em square in world coordinate units. This corresponds to the point size of the font. When the horizontal device resolution equals the vertical device resolution this is equal to the em height.

FONTMETRICS Field - lMaxBaselineExt

lMaxBaselineExt (LONG)
Maximum baseline extent.

The maximum vertical space occupied by the font, in world coordinate units. This is the sum of *lMaxAscender* and *lMaxDescender* if both are positive. It is also the sum of *lInternalLeading* and *lEmHeight* .

One possible type of line spacing can be computed by adding *lMaxBaselineExt* to *lExternalLeading* . Such a line spacing, however, would be dependent on the glyph set included in the font. If a new version of the font should be made available, with new glyphs, then it is possible that this value will change because one of the new glyphs has gone above the previous *lMaxAscender* or below the previous

lMaxDescender . More sophisticated applications will base line spacing on the point size (*lEmHeight*) of the font, which is an invariant of the font, multiplied by some factor (for example, 120%) plus any external leading.

This field may exceed *lEmHeight* .

FONTMETRICS Field - sCharSlope

sCharSlope (SHORT)
Character slope.

Defines the nominal slope for the characters of a font. The slope is defined in degrees increasing clockwise from the vertical. An *italic* font is an example of a font with a nonzero slope.

Note: The units for this metric are degrees and minutes, encoded as shown in the following example:

```
180 degrees 59 minutes would be represented as :

    < byte 1  >     < byte 2   >

    < Minutes >     < Degrees >

 0 1 1 1 0 1 1   0 1 0 1 1 0 1 0 0

     59 min         180 degrees
```

FONTMETRICS Field - sInlineDir

sInlineDir (SHORT)
Inline direction.

The direction in which the characters in the font are designed for viewing. The direction is defined in degrees increasing clockwise from the horizontal (left-to-right). Characters are added to a line of text in the inline direction.

Note: The units for this metric are degrees and minutes, encoded as shown in *sCharSlope* .

FONTMETRICS Field - sCharRot

sCharRot (SHORT)
Character rotation.

The rotation of the character glyphs with respect to the baseline, the angle increasing counter clockwise. This is the angle assigned by the font designer.

Note: The units for this metric are degrees and minutes, encoded as shown in *sCharSlope* .

FONTMETRICS Field - usWeightClass

usWeightClass (USHORT)
Weight class.

Indicates the visual weight (thickness of strokes) of the characters in the font:

Value	Description
1	Ultra-light
2	Extra-light
3	Light
4	Semi-light
5	Medium (normal)
6	Semi-bold
7	Bold
8	Extra-bold
9	Ultra-bold

FONTMETRICS Field - usWidthClass

usWidthClass (USHORT)
Width class.

Indicates the relative aspect ratio of the characters of the font in relation to the normal aspect ratio for this type of font:

Value	Description	% of normal width
1	Ultra-condensed	50
2	Extra-condensed	62.5
3	Condensed	75
4	Semi-condensed	87.5
5	Medium (normal)	100
6	Semi-expanded	112.5
7	Expanded	125
8	Extra-expanded	150
9	Ultra-expanded	200

FONTMETRICS Field - sXDeviceRes

sXDeviceRes (SHORT)
X-device resolution.

For bit-map fonts this is the resolution in the X direction of the intended target device, measured in pels per inch.

For outline fonts this is the number of notional units in the X direction of the Em square, measured in notional units per Em. (Notional units are the units in which the outline is defined.)

FONTMETRICS Field - sYDeviceRes

sYDeviceRes (SHORT)
Y-device resolution.

For bit-map fonts this is the resolution in the Y direction of the intended target device, measured in pels per inch.

For outline fonts this is the number of notional units in the Y direction of the Em square, measured in notional units per Em. (Notional units are the units in which the outline is defined.)

FONTMETRICS Field - sFirstChar

sFirstChar (SHORT)
First character.

The code point of the first character in the font.

FONTMETRICS Field - sLastChar

sLastChar (SHORT)
Last character.

The code point of the last character in the font, expressed as an offset from *sFirstChar* .

All code points between the first and last character specified must be supported by the font.

FONTMETRICS Field - sDefaultChar

sDefaultChar (SHORT)
Default character.

The code point that is used if a code point outside the range supported by the font is used, expressed as an offset from *sFirstChar* .

FONTMETRICS Field - sBreakChar

sBreakChar (SHORT)
Break character.

The code point that represents the "space" or "break" character for this font, expressed as an offset from *sFirstChar* . For example, if the first character is the space in code page 850, *sFirstChar* = 32, and *sBreakChar* = 0.

FONTMETRICS Field - sNominalPointSize

sNominalPointSize (SHORT)
Nominal point size.

For a bit-map font, this field contains the height of the font.

For an outline font, this field contains the height intended by the font designer. For example, some fonts are designed for text use in which case a value of 120 (12 point) would probably be placed in this field, whereas other fonts are designed for "display" use ("display" is typographer's terminology for larger sizes). This is not the only size at which the font can be used.

Measured in decipoints (a decipoint is 1/720th of an inch).

FONTMETRICS Field - sMinimumPointSize

sMinimumPointSize (SHORT)
Minimum point size.

For a bit-map font, this field does not apply. For an outline font, this field contains the minimum height intended by the font designer. Note that this is not a restriction of the size at which the font can be used.

Measured in decipoints (a decipoint is 1/720th of an inch).

FONTMETRICS Field - sMaximumPointSize

sMaximumPointSize (SHORT)
Maximum point size.

For a bit-map font, this field does not apply.

For an outline font, this field contains the maximum height intended by the font designer. Note that this is not a restriction of the size at which the font can be used.

Measured in decipoints (a decipoint is 1/720th of an inch).

FONTMETRICS Field - fsType

fsType (USHORT)
Type indicators.

This field contains the following information:

FM_TYPE_FIXED	Characters in the font have the same fixed width.
FM_TYPE_LICENSED	Licensed (protected) font.
FM_TYPE_KERNING	Font contains kerning information.
FM_TYPE_64K	Font is larger than 64KB (KB equals 1024 bytes) in size. If the

following two bits are false, the font is for single-byte code pages. One of the bits may be set.

FM_TYPE_DBCS — Font is for double-byte code pages.

FM_TYPE_MBCS — Font is for mixed single- or double-byte code pages.

FONTMETRICS Field - fsDefn

fsDefn (USHORT)
Definition indicators.

Contains the following font definition data:

FM_DEFN_OUTLINE — Font is a vector (outline) font; otherwise, it is a bit-map font.

FM_DEFN_GENERIC — Font is in a format that can be used by the GPI; otherwise, it is a device font.

FONTMETRICS Field - fsSelection

fsSelection (USHORT)
Selection indicators.

Contains information about the font patterns in the physical font.

Note: The flags do not reflect simulations applied to the physical font.

Possible values are:

FM_SEL_ITALIC — True indicates that this font is designed as an italic font.

FM_SEL_UNDERSCORE — TRUE indicates that this font is designed with underscores included in each character.

FM_SEL_NEGATIVE — TRUE indicates that this font is designed with the background and foreground reversed.

FM_SEL_OUTLINE — TRUE indicates that this font is designed with outline (hollow) characters.

FM_SEL_STRIKEOUT — TRUE indicates that this font is designed with an overstrike through each character.

FM_SEL_BOLD — TRUE indicates that this font is designed with bold characters.

FM_SEL_ISO9241_TESTED — This flag indicates that the font has been tested for compliance to ISO 9241. The presence of this flag doesn't indicate whether the font passed or failed, only that it was tested.

Note: While the fonts were primarily tested for meeting the ISO standard, they have also been designed to meet the German standard DIN 66 234. Where the two standards differ, the fonts have been designed to meet the more stringent requirement.

FONTMETRICS Field - fsCapabilities

fsCapabilities (USHORT)
Font capabilities.

This attribute applies only to device fonts.

FM_CAP_NOMIX	Characters may not be mixed with graphics.
QUALITY	The most significant byte may contain the following numeric value:

0	Undefined
1	DP quality
2	DP draft
3	Near Letter Quality
4	Letter Quality

FONTMETRICS Field - lSubscriptXSize

lSubscriptXSize (LONG)
Subscript x-size.

The horizontal size recommended by the font designer for subscripts for this font in world coordinate units.

FONTMETRICS Field - lSubscriptYSize

lSubscriptYSize (LONG)
Subscript y-size.

The vertical size recommended by the font designer for subscripts for this font in world coordinate units.

FONTMETRICS Field - lSubscriptXOffset

lSubscriptXOffset (LONG)
Subscript x-offset.

The baseline x-offset recommended by the font designer for subscripts for this font in world coordinate units.

FONTMETRICS Field - lSubscriptYOffset

lSubscriptYOffset (LONG)
Subscript y-offset.

The baseline y-offset recommended by the font designer for subscripts for this font in world coordinate units.

Note: Positive numbers indicate an offset below the baseline.

FONTMETRICS Field - lSuperscriptXSize

lSuperscriptXSize (LONG)
Superscript x-size.

The horizontal size recommended by the font designer for superscripts for this font in world coordinate units.

FONTMETRICS Field - lSuperscriptYSize

lSuperscriptYSize (LONG)
Superscript y-size.

The vertical point size recommended by the font designer for superscripts for this font in world coordinate units.

FONTMETRICS Field - lSuperscriptXOffset

lSuperscriptXOffset (LONG)
Superscript x-offset.

The baseline x-offset recommended by the font designer for superscripts for this font in world coordinate units.

FONTMETRICS Field - lSuperscriptYOffset

lSuperscriptYOffset (LONG)
Superscript y-offset.

The baseline y-offset recommended by the font designer for superscripts for this font in world coordinate units.

FONTMETRICS Field - lUnderscoreSize

lUnderscoreSize (LONG)
Underscore size.

The width (thickness) of the underscore stroke in world coordinate units. This describes the actual underscore in the font if FM_SEL_UNDERSCORE is also set. Otherwise it describes what the engine will simulate if underscore is requested in GpiCreateLogFont.

FONTMETRICS Field - lUnderscorePosition

lUnderscorePosition (LONG)
Underscore position.

The position of the underscore stroke from the baseline in world coordinate units. This describes the actual underscore in the font if FM_SEL_UNDERSCORE is also set. Otherwise it describes what the engine will simulate if underscore is requested in GpiCreateLogFont.

Note: Positive values indicate an offset below the baseline.

FONTMETRICS Field - lStrikeoutSize

lStrikeoutSize (LONG)
Strikeout size.

The width of the strikeout stroke in world coordinate units. This describes the actual underscore in the font if FM_SEL_STRIKEOUT is also set. Otherwise it describes what the engine will simulate if overstrike is requested in GpiCreateLogFont.

FONTMETRICS Field - lStrikeoutPosition

lStrikeoutPosition (LONG)
Strikeout position.

The position of the strikeout stroke relative to the baseline in world coordinate units. This describes the actual underscore in the font if FM_SEL_STRIKEOUT is also set. Otherwise it describes what the engine will simulate if overstrike is requested in GpiCreateLogFont.

FONTMETRICS Field - sKerningPairs

sKerningPairs (SHORT)
Kerning pairs.

The number of kerning pairs in the kerning pair table.

FONTMETRICS Field - sFamilyClass

sFamilyClass (SHORT)
Font family design classification.

This value contains a font class and its subclass.

FONTMETRICS Field - lMatch

lMatch (LONG)
Matched font identity.

This uniquely identifies the font for a given device and device driver combination. A positive match number signifies that the font is a generic (engine) font while a negative number indicates a device font (a native or downloadable font). This value should not be used to identify a font across system boundaries.

FONTMETRICS Field - panose

panose (PANOSE)
Panose font descriptor.

This is the Panose descriptor identifying the visual characteristics of the font.

FONTMGRPARM

GreEscape DEVESC_DBE_FONTMANAGEMENT data structure.

```
union FONTMGRPARM {
  DBE_FDI_PARAMETERS     fdi;
  VIOFONTATTRIBUTE       vioa;
} FONTMGRPARM;
```

FONTMGRPARM Field - fdi

fdi (DBE_FDI_PARAMETERS)

FONTMGRPARM Field - vioa

vioa (VIOFONTATTRIBUTE)

FOCAFONT

FOCAFONT data structure.

```
typedef struct _FOCAFONT {
  FONTSIGNATURE                       fsSignature;                    /*  FOCAFONT data structure. */
  FOCAMETRICS                         fmMetrics;                      /*  Font metrics structure. */
  FONTDEFINITIONHEADER                fdDefinitions;                  /*  Defines format of character defini
  TYPE1CELL, TYPE2CELL or TYPE3CELL   font_cell [ ];  /*  Array of Type 1, 2, or 3 cells. */
} FOCAFONT;

typedef FOCAFONT *PFOCAFONT;
```

FOCAFONT Field - fsSignature

fsSignature (FONTSIGNATURE)
FOCAFONT data structure.

Indicates the font signature datatype, FONTSIGNATURE or ADDITIONALMETRICS. If the font signature type is ADDITIONALMETRICS, additional information is included in the PANOSE datatype.

FONT_SIGNATURE	Identifies header start
FONT_METRICS	Identifies metrics
FONT_DEFINITION	Identifies definition
FONT_KERNPAIRS	Identifies kern pairs
FONT_ADDITIONALMETRICS	Identifies additional metrics
FONT_ENDRECORD	Identifies record end

FOCAFONT Field - fmMetrics

fmMetrics (FOCAMETRICS)
Font metrics structure.

This structure is returned to applications on the GpiQueryFonts and GpiQueryFontMetrics calls and conveys information from the font creator to the application.

FOCAFONT Field - fdDefinitions

fdDefinitions (FONTDEFINITIONHEADER)
Defines format of character definition records.

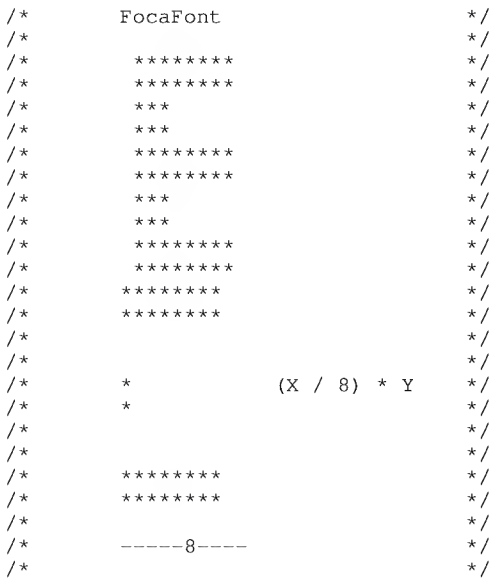
Defines the format of the character definition records in the FONTDEFINITIONHEADER data structure.

FOCAFONT Field - font_cell [ ]

font_cell [ ] (TYPE1CELL, TYPE2CELL or TYPE3CELL)
Array of Type 1, 2, or 3 cells.

Defines the cell used relative to the values in FONTDEFINITIONHEADER.

Here is an example of the FOCAFONT bit-map image:



FOCAMETRICS

FOCAMETRICS data structure.

This structure is returned to applications on the GPIQueryFonts and GPIQueryFontMetrics calls and conveys information from the font creator to the application.

```
typedef struct _FOCAMETRICS {
  ULONG      ulIdentity;                          /*  Structure identity code. */
  ULONG      ulSize;                              /*  Structure size in bytes. */
  CHAR       szFamilyname[32];  /*  Font family name. */
  CHAR       szFacename[32];    /*  Face name. The typeface name defines the particular font; for example, "Time
n Bold Italic."** This string is null terminated and is, therefore, limited to 32 characters in length. */
  SHORT      usRegistryId;                        /*  Registry identifier. */
  USHORT     usCodePage;                          /*  Code page supported by the font. */
  SHORT      yEmHeight;                           /*  Em height. */
  SHORT      yXHeight;                            /*  X height. */
  SHORT      yMaxAscender;                        /*  Maximum ascender. */
  SHORT      yMaxDescender;                       /*  Maximum descender. */
  SHORT      yLowerCaseAscent;                    /*  Lowercase ascent. */
  SHORT      yLowerCaseDescent;                   /*  Lowercase descent. */
  SHORT      yInternalLeading;                    /*  Internal leading. */
  SHORT      yExternalLeading;                    /*  External leading. */
  SHORT      xAveCharWidth;                       /*  Average character width. */
  SHORT      xMaxCharInc;                         /*  Maximum character increment. */
  SHORT      xEmInc;                              /*  Em increment. */
  SHORT      yMaxBaselineExt;                     /*  Maximum baseline extent. */
  SHORT      sCharSlope;                          /*  Character slope. */
  SHORT      sInlineDir;                          /*  Inline direction. */
  SHORT      sCharRot;                            /*  Character rotation. */
  USHORT     usWeightClass;                       /*  Weight class. */
  USHORT     usWidthClass;                        /*  Width class. */
  SHORT      xDeviceRes;                          /*  X-device resolution. */
  SHORT      yDeviceRes;                          /*  Y-device resolution. */
  SHORT      usFirstChar;                         /*  First character. */
  SHORT      usLastChar;                          /*  Last character. */
  SHORT      usDefaultChar;                       /*  Default character. */
```

```
  SHORT       usBreakChar;                  /*  Break character. */
  SHORT       usNominalPointSize;           /*  Nominal point size. */
  SHORT       usMinimumPointSize;           /*  Minimum point size. */
  SHORT       usMaximumPointSize;           /*  Maximum point size. */
  SHORT       usTypeFlags;                  /*  Type indicators. */
  SHORT       fsDefn;                       /*  Definition indicators. */
  SHORT       fsSelectionFlags;             /*  Selection indicators. */
  SHORT       fsCapabilities;               /*  Capabilities. */
  SHORT       ySubscriptXSize;              /*  Subscript X-size. */
  SHORT       ySubscriptYSize;              /*  Subscript Y-size. */
  SHORT       ySubscriptXOffset;            /*  Subscript X-offset. */
  SHORT       ySubscriptYOffset;            /*  Subscript Y-offset. */
  SHORT       ySuperscriptXSize;            /*  Superscript X-size. */
  SHORT       ySuperscriptYSize;            /*  Superscript Y-size. */
  SHORT       ySuperscriptXOffset;          /*  Superscript X-offset. */
  SHORT       ySuperscriptYOffset;          /*  Superscript Y-offset. */
  SHORT       yUnderscoreSize;              /*  Underscore size. */
  SHORT       yUnderscorePosition;          /*  Underscore position. */
  SHORT       yStrikeoutSize;               /*  Strikeout size. */
  SHORT       yStrikeoutPosition;           /*  Strikeout position. */
  SHORT       usKerningPairs;               /*  Kerning pairs. */
  SHORT       sFamilyClass;                 /*  Font family design classification. */
  PSZ         pszDeviceNameOffset;          /*  Address where device name is stored. */
} FOCAMETRICS;

typedef FOCAMETRICS *PFOCAMETRICS;
```

FOCAMETRICS Field - ulIdentity

ulIdentity (ULONG)
Structure identity code.

Value must be one (1). This field is not part of the FONTMETRICS structure.

FOCAMETRICS Field - ulSize

ulSize (ULONG)
Structure size in bytes.

This must be set to the size of the FOCAMETRICS structure. This field is not part of the FONTMETRICS structure.

FOCAMETRICS Field - szFamilyname[32]

szFamilyname[32] (CHAR)
Font family name.

The family name defines the basic appearance of the font; for example, "Times New Roman."** This string is null terminated and is, therefore, limited to 32 characters in length.

FOCAMETRICS Field - szFacename[32]

szFacename[32] (CHAR)
Face name. The typeface name defines the particular font; for example, "Times New Roman Bold Italic."** This string is null terminated and is, therefore, limited to 32 characters in length.

FOCAMETRICS Field - usRegistryId

usRegistryId (SHORT)
Registry identifier.

The IBM registered number (or zero).

FOCAMETRICS Field - usCodePage

usCodePage (USHORT)
Code page supported by the font.

Defines the registered code page supported by the font. A value of zero (0) implies that the font may be used with any of the code pages supported by OS/2.

When a font contains special symbols for which there is no registered code page, then code page 65400 is used.

FOCAMETRICS Field - yEmHeight

yEmHeight (SHORT)
Em height.

The height of the Em square in world coordinate units. This height corresponds to the point size for the font.

FOCAMETRICS Field - yXHeight

yXHeight (SHORT)
X height.

The nominal height above the baseline for lowercase characters (ignoring ascenders), in world coordinate units.

FOCAMETRICS Field - yMaxAscender

yMaxAscender (SHORT)
Maximum ascender.

The maximum height above the baseline reached by any part of any symbol in the font, in world coordinate units. This field may exceed *yEmHeight* .

FOCAMETRICS Field - yMaxDescender

yMaxDescender (SHORT)
Maximum descender.

The maximum depth below the baseline reached by any part of any symbol in the font, in world coordinate units. This field may exceed *yEmHeight* .

FOCAMETRICS Field - yLowerCaseAscent

yLowerCaseAscent (SHORT)
Lowercase ascent.

The maximum height above the baseline reached by any part of any lowercase (Latin unaccented ' a ' through ' z ') symbol in the font, in world coordinate units.

FOCAMETRICS Field - yLowerCaseDescent

yLowerCaseDescent (SHORT)
Lowercase descent.

The maximum depth below the baseline reached by any part of any lowercase (Latin unaccented ' a ' through ' z ') symbol in the font, in world coordinate units.

FOCAMETRICS Field - yInternalLeading

yInternalLeading (SHORT)
Internal leading.

The amount of space which, when subtracted from yMaxAscender, gives a measure of the distance above the baseline that characters extend. This measure is font-design dependent, but glyph-set independent. Therefore, this calculation approximates the visual top to a row of characters without actually looking at the characters in the row.

It is recommended that applications use this field to position the first line of a block of text by subtracting yInternalLeading from yMaxAscender. Then position the baseline that distance below the preceding information (text or graphic element).

Note: This use does not guarantee that characters will not overwrite those above them; it does give a font designer's view of where to place the text. Collision should be tested for and additional space allocated if necessary.

FOCAMETRICS Field - yExternalLeading

yExternalLeading (SHORT)
External leading.

The amount of guaranteed white space advised by the font designer to appear between adjacent rows of text. This value may be zero (0).

Note: The fonts built into Presentation Manager have zero (0) in this field.

FOCAMETRICS Field - xAveCharWidth

xAveCharWidth (SHORT)
Average character width.

This is determined by multiplying the width of each lowercase character by a constant, adding the products, and then dividing by 1000. The letters, plus their constants, are listed below:

Letter	Constant
a	64
b	14
c	27
d	35
e	100
f	20
g	14
h	42
i	63
j	3
k	6
l	35
m	20
n	56
o	56
p	17
q	4
r	49
s	56
t	71
u	31
v	10
w	18
x	3
y	18
z	2
space	166

Note: For fixed pitch fonts, this value will be the same as the (A width + B width + C width) escapement of each character.

FOCAMETRICS Field - xMaxCharInc

xMaxCharInc (SHORT)
Maximum character increment.

The maximum character increment for the font, in world coordinate units.

FOCAMETRICS Field - xEmInc

xEmInc (SHORT)
Em increment.

The width of the Em square in world coordinate units. This width corresponds to the point size of the font. When the horizontal device resolution equals the vertical device resolution, width is equal to the em height.

FOCAMETRICS Field - yMaxBaselineExt

yMaxBaselineExt (SHORT)
Maximum baseline extent.

The maximum vertical space occupied by the font, in world coordinate units. This space is the sum of *yMaxAscender* and *yMaxDescender* if both are positive. It is also the sum of *yInternalLeading* and *yEmHeight* .

One possible line spacing can be computed by adding *yMaxBaselineExt* to *yInternalLeading* . Such a line spacing, however, would be dependent on the glyph set included in the font. If a new version of the font is made available with new glyphs, then it is possible that this value will change because one of the new glyphs has gone above the previous *yMaxAscender* or below the previous *yMaxDescender* . More sophisticated applications will base their line spacing on the point size (*yEmHeight*) of the font, which is an invariant of the font, multiplied by some factor (for example, 120%), plus any external leading.

yMaxBaselineExt may exceed *yEmHeight* .

FOCAMETRICS Field - sCharSlope

sCharSlope (SHORT)
Character slope.

Defines the nominal slope of the characters of a font. The slope is defined in degrees, increasing clockwise from the vertical. An *italic* font is an example of a font with a nonzero slope.

Note: The units for this metric are degrees and minutes, encoded as shown in the following example.

```
180 degrees 59 minutes would be represented as:

    < byte 1  >     < byte 2   >

    < Minutes >     < Degrees >

 0 1 1 1 0 1 1   0 1 0 1 1 0 1 0 0

     59 min         180 degrees
```

FOCAMETRICS Field - sInlineDir

sInlineDir (SHORT)
Inline direction.

The direction in which the characters in the font are designed for viewing. Shown in degrees, increasing clockwise from the horizontal (left-to-right). Characters are added to a line of text in the inline direction.

Note: The units for this metric are degrees and minutes, encoded as shown in *sCharSlope* .

FOCAMETRICS Field - sCharRot

sCharRot (SHORT)
Character rotation.

The rotation of the character glyphs with respect to the baseline, the angle increasing counterclockwise. This is the angle assigned by the font designer.

Note: The units for this metric are degrees and minutes, encoded as shown in *sCharSlope* .

FOCAMETRICS Field - usWeightClass

usWeightClass (USHORT)
Weight class.

Indicates the visual weight (thickness of strokes) of the characters in the font:

Value	Description
1000	Ultra-light
2000	Extra-light
3000	Light
4000	Semi-light
5000	Medium (normal)
6000	Semi-bold
7000	Bold
8000	Extra-bold
9000	Ultra-bold

FOCAMETRICS Field - usWidthClass

usWidthClass (USHORT)
Width class.

Indicates the relative aspect ratio of the font characters in relation to the normal aspect ratio for this type of font:

Value	Description	% of Normal Width
1000	Ultra-condensed	50
2000	Extra-condensed	62.5
3000	Condensed	75
4000	Semi-condensed	87.5

5000	Medium (normal)	100
6000	Semi-expanded	112.5
7000	Expanded	125
8000	Extra-expanded	150
9000	Ultra-expanded	200

FOCAMETRICS Field - xDeviceRes

xDeviceRes (SHORT)
X-device resolution.

For bit-map fonts, this is the resolution in the X direction of the intended target device, measured in pels per inch.

For outline fonts, this is the number of notional units in the X direction of the Em square, measured in notional units per Em. (Notional units are the units in which the outline is defined.)

FOCAMETRICS Field - yDeviceRes

yDeviceRes (SHORT)
Y-device resolution.

For bit-map fonts, this is the resolution in the Y direction of the intended target device, measured in pels per inch.

For outline fonts, this is the number of notional units in the Y direction of the Em square, measured in notional units per Em. (Notional units are the units in which the outline is defined.)

FOCAMETRICS Field - usFirstChar

usFirstChar (SHORT)
First character.

The code point of the first character in the font.

FOCAMETRICS Field - usLastChar

usLastChar (SHORT)
Last character.

The code point of the last character in the font, expressed as an offset from *usFirstChar* .

All code points between the first and last character specified must be supported by the font.

FOCAMETRICS Field - usDefaultChar

usDefaultChar (SHORT)
Default character.

The code point that is used if a code point outside the range supported by the font is used, expressed as an offset from *usFirstChar* .

FOCAMETRICS Field - usBreakChar

usBreakChar (SHORT)
Break character.

The code point that represents the ”space” or ”break” character for this font, expressed as an offset from *usFirstChar* . For example, if the first character is the space in code page 850, *usFirstChar* = 32, and *usBreakChar* = 0.

FOCAMETRICS Field - usNominalPointSize

usNominalPointSize (SHORT)
Nominal point size.

For a bit-map font, this field contains the height of the font.

For an outline font, this field contains the height that the font designer intended for this font. For example, some fonts are designed for text use, in which case a value of 120 (12 point) would probably be placed in this field. Other fonts are designed for ”display” use (”display” is a typographer’s term for larger point sizes). This is not the only size at which the font can be used.

Measured in decipoints (a decipoint is 1/720th of an inch).

FOCAMETRICS Field - usMinimumPointSize

usMinimumPointSize (SHORT)
Minimum point size.

For a bit-map font, this field is not applicable.

For an outline font, this field contains the minimum height that the font designer intended for this font. Note that this is not a restriction on the size at which the font can be used.

Measured in decipoints (a decipoint is 1/720th of an inch).

FOCAMETRICS Field - usMaximumPointSize

usMaximumPointSize (SHORT)
Maximum point size.

For a bit-map font, this field is not applicable.

For an outline font, this field contains the maximum height that the font designer intended for this font. Note that this is not a restriction on the size at which the font can be used.

Measured in decipoints (a decipoint is 1/720th of an inch).

FOCAMETRICS Field - usTypeFlags

usTypeFlags (SHORT)
Type indicators.

These flags contain the following information:

FM_TYPE_FIXED	Characters in the font have the same fixed width.
FM_TYPE_LICENSED	Licensed (protected) font.
FM_TYPE_KERNING	Font contains kerning information.
FM_TYPE_64K	Font is larger than 64KB (KB equals 1024 bytes) in size. Only one of the following bits (FM_TYPE_DBCS or FM_TYPE_MCBS) may be set. If both of those bits are false (not set), the font is for single-byte code pages.
FM_TYPE_DBCS	Font is for double-byte code pages.
FM_TYPE_MBCS	Font is for mixed single- and double-byte code pages.
FM_TYPE_FACETRUNC	Font *szFacename[32]* has been truncated.
FM_TYPE_FAMTRUNC	Font *szFamilyname[32]* has been truncated.

FOCAMETRICS Field - fsDefn

fsDefn (SHORT)
Definition indicators.

These indicators contain the following font definition data:

FM_DEFN_OUTLINE	Font is a vector (outline) font; otherwise, it is a bit map font.
FM_DEFN_GENERIC	Font is in a format that can be used by the GPI; otherwise, it is a device font.

FOCAMETRICS Field - fsSelectionFlags

fsSelectionFlags (SHORT)
Selection indicators.

These indicators contain information about the font patterns in the physical font.

Note: The flags do not reflect simulations applied to the physical font.

FM_SEL_ITALIC	TRUE indicates that this font is designed as an italic font.
FM_SEL_UNDERSCORE	TRUE indicates that this font is designed with underscores included

in each character.

FM_SEL_NEGATIVE	TRUE indicates that this font is designed with the background and foreground reversed.
FM_SEL_OUTLINE	TRUE indicates that this font is designed with outline (hollow) characters.
FM_SEL_STRIKEOUT	TRUE indicates that this font is designed with an overstrike through each character.
FM_SEL_BOLD	TRUE indicates that this font is designed with bold characters.

FOCAMETRICS Field - fsCapabilities

fsCapabilities (SHORT)
Capabilities.

This attribute applies only to device fonts.

FM_CAP_NOMIX	Characters may not be mixed with graphics.	
QUALITY	The most significant byte may contain the following numeric value:	
	0	Undefined
	1	DP quality
	2	DP draft
	3	Near Letter Quality
	4	Letter Quality

FOCAMETRICS Field - ySubscriptXSize

ySubscriptXSize (SHORT)
Subscript X-size.

The recommended horizontal point size for subscripts for this font, in world coordinate units.

FOCAMETRICS Field - ySubscriptYSize

ySubscriptYSize (SHORT)
Subscript Y-size.

The recommended vertical point size for subscripts for this font, in world coordinate units.

FOCAMETRICS Field - ySubscriptXOffset

ySubscriptXOffset (SHORT)
Subscript X-offset.

The recommended baseline X-offset for subscripts for this font, in world coordinate units.

FOCAMETRICS Field - ySubscriptYOffset

ySubscriptYOffset (SHORT)
Subscript Y-offset.

The recommended baseline Y-offset for subscripts for this font, in world coordinate units.

FOCAMETRICS Field - ySuperscriptXSize

ySuperscriptXSize (SHORT)
Superscript X-size.

The recommended horizontal point size for superscripts for this font, in world coordinate units.

FOCAMETRICS Field - ySuperscriptYSize

ySuperscriptYSize (SHORT)
Superscript Y-size.

The recommended vertical point size for superscripts for this font, in world coordinate units.

FOCAMETRICS Field - ySuperscriptXOffset

ySuperscriptXOffset (SHORT)
Superscript X-offset.

The recommended baseline X-offset for superscripts for this font, in world coordinate units.

FOCAMETRICS Field - ySuperscriptYOffset

ySuperscriptYOffset (SHORT)
Superscript Y-offset.

The recommended baseline Y-offset for superscripts for this font, in world coordinate units.

FOCAMETRICS Field - yUnderscoreSize

yUnderscoreSize (SHORT)
Underscore size.

The width (thickness) of the underscore stroke, in world coordinate units. This describes the actual underscore in the font if FM_SEL_UNDERSCORE is also set. Otherwise, it describes what the graphics engine will simulate if underscore is requested in GpiCreateLogFont.

FOCAMETRICS Field - yUnderscorePosition

yUnderscorePosition (SHORT)
Underscore position.

The position of the underscore stroke from the baseline, in world coordinate units. This value describes the actual underscore in the font if FM_SEL_UNDERSCORE is also set. Otherwise, it describes what the graphics engine will simulate if underscore is requested in GpiCreateLogFont.

Note: Positive values mean below the baseline.

FOCAMETRICS Field - yStrikeoutSize

yStrikeoutSize (SHORT)
Strikeout size.

The width of the strikeout stroke, in world coordinate units. This value describes the actual strikeout in the font if FM_SEL_STRIKEOUT is also set. Otherwise, it describes what the graphics engine will simulate if overstrike is requested in GpiCreateLogFont.

FOCAMETRICS Field - yStrikeoutPosition

yStrikeoutPosition (SHORT)
Strikeout position.

The position of the strikeout stroke relative to the baseline, in world coordinate units. This value describes the actual strikeout in the font if FM_SEL_STRIKEOUT is also set. Otherwise, it describes what the graphics engine will simulate if overstrike is requested in GpiCreateLogFont.

FOCAMETRICS Field - usKerningPairs

usKerningPairs (SHORT)
Kerning pairs.

The number of kerning pairs in the kerning pair table.

FOCAMETRICS Field - sFamilyClass

sFamilyClass (SHORT)
Font family design classification.

This value contains a font class and its subclass.

FOCAMETRICS Field - pszDeviceNameOffset

pszDeviceNameOffset (PSZ)
Address where device name is stored.

FONTDEFINITIONHEADER

FONTDEFINITIONHEADER data structure.

This structure defines the format of the character definition records that follow it.

```
typedef struct _FONTDEFINITIONHEADER {
  ULONG     ulIdentity;       /*  Structure identity code. */
  ULONG     ulSize;           /*  Structure size in bytes. */
  SHORT     fsFontdef;        /*  Flags for font definition data. */
  SHORT     fsChardef;        /*  Fields present, per-character basis. */
  SHORT     usCellSize;       /*  Length of each character definition record. */
  SHORT     xCellWidth;       /*  Width of characters. */
  SHORT     yCellHeight;      /*  Height of characters. */
  SHORT     xCellIncrement;   /*  Stepping distance along character baseline. */
  SHORT     xCellA;           /*  Width of the space before a character. */
  SHORT     xCellB;           /*  Width of a character. */
  SHORT     xCellC;           /*  Width of the space following a character. */
  SHORT     pCellBaseOffset;  /*  Top position of a character definition. */
} FONTDEFINITIONHEADER;

typedef FONTDEFINITIONHEADER *PFONTDEFINITIONHEADER;
```

FONTDEFINITIONHEADER Field - ulIdentity

ulIdentity (ULONG)
Structure identity code.

Value must be one (1). This field is not part of the FONTMETRICS structure.

FONTDEFINITIONHEADER Field - ulSize

ulSize (ULONG)
Structure size in bytes.

Value must be set to the size of the FOCAMETRICS structure. This field is not part of the FONTMETRICS structure.

FONTDEFINITIONHEADER Field - fsFontdef

fsFontdef (SHORT)
Flags for font definition data.

Indicates which fields are present in the font definition data in the header, and whether the fields are Type 1, Type 2, or Type 3.

FONTDEFFONT1	Sets width, height, increment and base offset.
FONTDEFFONT2	Sets height and base offset.
FONTDEFFONT3	Sets height and base offset.
SPACE_UNDEF	Reserved by the system.

FONTDEFINITIONHEADER Field - fsChardef

fsChardef (SHORT)
Fields present, per-character basis.

Indicates which fields (TYPE 1, TYPE 2, or TYPE 3) are present on a per-character basis.

FONTDDEFCHAR1	Sets character offset and width.
FONTDEFCHAR2	Sets character offset and width.
FONTDEFCHAR3	Sets character offset, A, B, and C space.

FONTDEFINITIONHEADER Field - usCellSize

usCellSize (SHORT)
Length of each character definition record.

Indicates the length in bytes (the per-character data).

FONTDEFINITIONHEADER Field - xCellWidth

xCellWidth (SHORT)
Width of characters.

For image fonts, the width of the characters in pels. For outline fonts, the width in relative units.

FONTDEFINITIONHEADER Field - yCellHeight

yCellHeight (SHORT)
Height of characters.

For image fonts, the height of the characters in pels. For outline fonts, the height in relative units.

FONTDEFINITIONHEADER Field - xCellIncrement

xCellIncrement (SHORT)
Stepping distance along character baseline.

The distance along the character baseline required to step from one character to the next when forming a character string.

Type 1	Width of the characters
Type 2	Zero
Type 3	Zero

FONTDEFINITIONHEADER Field - xCellA

xCellA (SHORT)
Width of the space before a character.

The width of the space before a character in the inline direction (the a-space).

Type 1	Zero
Type 2	Zero
Type 3	a-space for all characters

FONTDEFINITIONHEADER Field - xCellB

xCellB (SHORT)
Width of a character.

The width of a character in the inline direction (the b-space).

Type 1	Zero
Type 2	Zero

Type 3	b-space for all characters

FONTDEFINITIONHEADER Field - xCellC

xCellC (SHORT)
Width of the space following a character.

The width of the space following a character in the inline direction (the c-space).

Type 1	Zero
Type 2	Zero
Type 3	c-space for all characters

FONTDEFINITIONHEADER Field - pCellBaseOffset

pCellBaseOffset (SHORT)
Top position of a character definition.

The position of the top of a character definition, relative to the baseline in the direction perpendicular to the baseline.

Type 1	Baseline offset for all characters
Type 2	Baseline offset for all characters
Type 3	Baseline offset for all characters

FONTSIGNATURE

FONTSIGNATURE data structure.

Indicates the font signature datatype, FONTSIGNATURE or ADDITIONALMETRICS. If the font signature type is ADDITIONALMETRICS, additional information is included in the PANOSE datatype.

```
typedef struct _FONTSIGNATURE {
  ULONG     ulIdentity;                         /*  Structure identity code. */
  ULONG     ulSize;                             /*  Structure size in bytes. */
  CHAR      achSignature[12];  /*  Signatures defined in PMFONT.H. */
} FONTSIGNATURE;

typedef FONTSIGNATURE *PFONTSIGNATURE;
```

FONTSIGNATURE Field - ulIdentity

ulIdentity (ULONG)
Structure identity code.

Value must be one (1). This field is not part of the FONTMETRICS structure.

FONTSIGNATURE Field - ulSize

ulSize (ULONG)
Structure size in bytes.

Value must be set to the size of the FOCAMETRICS structure. This field is not part of the FONTMETRICS structure.

FONTSIGNATURE Field - achSignature[12]

achSignature[12] (CHAR)
Signatures defined in PMFONT.H.

GETPORTFROMQ

Information about the ports connected to the specified queue.

```
typedef struct _GETPORTFROMQ {
  ULONG         ulPorts;                        /*  Number of ports connected to this print queue. */
  PORTFROMQ     PortFromQ[1];  /*  An array (size ulPorts) of ports connected to this queue. */
} GETPORTFROMQ;

typedef GETPORTFROMQ *PGETPORTFROMQ;
```

GETPORTFROMQ Field - ulPorts

ulPorts (ULONG)
Number of ports connected to this print queue.

GETPORTFROMQ Field - PortFromQ[1]

PortFromQ[1] (PORTFROMQ)
An array (size *ulPorts*) of ports connected to this queue.

This array is a list of PORTFROMQ structures.

GRADIENTL

Direction-vector structure.

```
typedef struct _GRADIENTL {
  LONG      x;   /*  X-component of direction. */
  LONG      y;   /*  Y-component of direction. */
} GRADIENTL;

typedef GRADIENTL *PGRADIENTL;
```

GRADIENTL Field - x

x (LONG)
X-component of direction.

GRADIENTL Field - y

y (LONG)
Y-component of direction.

HAB

Anchor-block handle.

```
typedef LHANDLE HAB;
```

HBITMAP

Bit-map handle.

```
typedef LHANDLE HBITMAP;
```

HCINFO

Hardcopy-capabilities structure.

```
typedef struct _HCINFO {
  CHAR      szFormname[32];  /*  Form name. */
  LONG      cx;              /*  Width (left-to-right) in millimeters. */
  LONG      cy;              /*  Height (top-to-bottom) in millimeters. */
  LONG      xLeftClip;       /*  Left clip limit in millimeters. */
  LONG      yBottomClip;     /*  Bottom clip limit in millimeters. */
  LONG      xRightClip;      /*  Right clip limit in millimeters. */
  LONG      yTopClip;        /*  Top clip limit in millimeters. */
  LONG      xPels;           /*  Number of pels between left and right clip limits. */
  LONG      yPels;           /*  Number of pels between bottom and top clip limits. */
  LONG      flAttributes;    /*  Attributes of the form identifier. */
} HCINFO;

typedef HCINFO *PHCINFO;
```

HCINFO Field - szFormname[32]

szFormname[32] (CHAR)
Form name.

HCINFO Field - cx

cx (LONG)
Width (left-to-right) in millimeters.

HCINFO Field - cy

cy (LONG)
Height (top-to-bottom) in millimeters.

HCINFO Field - xLeftClip

xLeftClip (LONG)
Left clip limit in millimeters.

HCINFO Field - yBottomClip

yBottomClip (LONG)
Bottom clip limit in millimeters.

HCINFO Field - xRightClip

xRightClip (LONG)
Right clip limit in millimeters.

HCINFO Field - yTopClip

yTopClip (LONG)
Top clip limit in millimeters.

HCINFO Field - xPels

xPels (LONG)
Number of pels between left and right clip limits.

HCINFO Field - yPels

yPels (LONG)
Number of pels between bottom and top clip limits.

HCINFO Field - flAttributes

flAttributes (LONG)
Attributes of the form identifier.

HCAPS_SELECTABLE	The form is installed on the printer as given by the printer properties dialog. It is available from an alternate form source without operator intervention. If the form does not have this bit set and is used (if the user selects it), a "forms mismatch" error is generated by the printer object.
HCAPS_CURRENT	The form is the one currently selected by the *pdriv* field (the job properties) in DEVOPENSTRUC

HDC

Device-context handle.

```
typedef LHANDLE HDC;
```

HFILE

File handle.

```
typedef LHANDLE HFILE;
```

HPROC

Processor handle.

```
typedef LHANDLE HPROC;
```

HPS

Presentation-space handle.

```
typedef LHANDLE HPS;
```

HRGN

Region handle.

```
typedef LHANDLE HRGN;
```

HSPL

Spooler handle.

```
typedef LHANDLE HSPL;
```

HSTD

The handle to the PM_Q_STD buffer.

```
typedef LHANDLE HSTD;
```

HWND

Window handle.

```
typedef LHANDLE HWND;
```

IMAGEBUF

Characteristics describing an image.

```
typedef struct _IMAGEBUF {
  ULONG            ulLength;        /*  Length of IMAGEBUF data structure, in bytes. */
  ULONG            ulFlags;         /*  Image buf flag. */
  ULONG            ulType;          /*  Flag indicating type of memory:  VRAM or system.  */
  PBYTE            pBits;           /*  Virtual address of the image. */
  ULONG            ulImgWidth;      /*  Image width, in pels. */
  ULONG            ulImgHeight;     /*  Image height, in pels. */
  ULONG            ulBytesPerScan;  /*  Bytes per scan line. */
  PCOLORINFO       pColorInfo;      /*  Color space of data in buffer. */
  PCODECINFO       pCodecInfo;      /*  Compression type of data in buffer. */
  PCUSTPALINFO     pCustPalInfo;    /*  Pointer to custom palette information. */
} IMAGEBUF;

typedef IMAGEBUF *PIMAGEBUF;
```

IMAGEBUF Field - ulLength

ulLength (ULONG)
Length of IMAGEBUF data structure, in bytes.

IMAGEBUF Field - ulFlags

ulFlags (ULONG)
Image buf flag.

This flag has the following values:

IBF_Y_ORG_BOTTOM

0x0001; origin is set at bottom

Note: This flag can be used as an optimization for 8-bpp images for the following two conditions:

1. The hardware has support for cached custom palettes, and the client application is playing frames using the same palette. The driver will return IMG_CAPS_PAL_CACHING if it has this support.

2. The screen is in 256-color mode and the source image bits have already been translated to the current hardware palette.

IBF_IGNORE_CUST_PAL

0x0002; ignore the custom palette

IMAGEBUF Field - ulType

ulType (ULONG)
Flag indicating type of memory: VRAM or system.

This flag has the following values:

IBT_SRC_VRAM

0x0000; indicates source is VRAM memory

IBT_SRC_MEM

0x0001; indicates source is system memory

IMAGEBUF Field - pBits

pBits (PBYTE)
Virtual address of the image.

Not required if source is in VRAM.

IMAGEBUF Field - ulImgWidth

ulImgWidth (ULONG)
Image width, in pels.

IMAGEBUF Field - ulImgHeight

ulImgHeight (ULONG)
Image height, in pels.

IMAGEBUF Field - ulBytesPerScan

ulBytesPerScan (ULONG)
Bytes per scan line.

IMAGEBUF Field - pColorInfo

pColorInfo (PCOLORINFO)
Color space of data in buffer.

IMAGEBUF Field - pCodecInfo

pCodecInfo (PCODECINFO)
Compression type of data in buffer.

IMAGEBUF Field - pCustPalInfo

pCustPalInfo (PCUSTPALINFO)
Pointer to custom palette information.

IMAGECAPS

Capabilities of the video accelerator driver.

```
typedef struct _IMAGECAPS {
  ULONG          ulLength;         /*  Length of IMAGECAPS data structure, in bytes. */
  ULONG          ulCaps;           /*  Flag that specifies image capabilities supported. */
  ULONG          ulMaxHorz;        /*  Maximum horizontal pels supported by scaler. */
  ULONG          ulMaxVert;        /*  Maximum vertical lines supported by scaler. */
  BOOL           fAccelMem;        /*  Flag indicating whether the hardware accelerator has additional VRAM for
  ULONG          ulPhysAddrVRAM;   /*  Physical address of the hardware accelerator VRAM. */
  ULONG          ulSize;           /*  Size of hardware accelerator VRAM, in bytes. */
  ULONG          ulScanLineBytes;  /*  Size of scan line, in bytes. */
  ULONG          ulNumCodecs;      /*  Number of FOURCCs that follow in this structure. */
  PCODECINFO     pCodecList;       /*  Pointer to array of CODEC FOURCCs. */
  ULONG          ulNumSrc;         /*  Number of source COLORINFO data structures pointed to by pSrcColor
Info. */
  PCOLORINFO     pSrcColorInfo;    /*  Pointer to array of source COLORINFO data structures. */
  ULONG          ulNumDst;         /*  Number of destination COLORINFO data structures pointed to by pDstColorIn
  PCOLORINFO     pDstColorInfo;    /*  Pointer to array of destination COLORINFO data structures. */
} IMAGECAPS;

typedef IMAGECAPS *PIMAGECAPS;
```

IMAGECAPS Field - ulLength

ulLength (ULONG)
Length of IMAGECAPS data structure, in bytes.

IMAGECAPS Field - ulCaps

ulCaps (ULONG)
Flag that specifies image capabilities supported.

This flag has the following values:

IMAGECAPS_STRETCHBLT	0x0001; has hardware stretch Blt assist
IMAGECAPS_CAPTURE	0x0002; has capture to VRAM hardware present
IMAGECAPS_1X_BLT	0x0004; has nonstretch Blt assist
IMAGECAPS_WINDOWCLIP	0x0008; supports clipping in hardware and/or software
IMAGECAPS_BUFALLOC	0x0010; supports allocation of on-card memory
IMAGECAPS_RECTALLOC	0x0020; supports allocation of rectangles in on-card memory
IMAGECAPS_COMP	0x0040; compression supported by driver
IMAGECAPS_DECOMP	0x0080; decompression supported by driver
IMAGECAPS_PAL_CACHING	0x0100; supports 8-bit palette caching
IMAGECAPS_INTERPOL	0x0200; supports color interpolation in hardware
IMAGECAPS_BYLOCSYSMEM	0x0400; able to read/write to system memory
IMIAGECAPS_SEM_BYPASS	0x0800; device serialization not required
IMAGECAPS_NO_SHARED_FB	0x1000; overlay device

IMAGECAPS Field - ulMaxHorz

ulMaxHorz (ULONG)
Maximum horizontal pels supported by scaler.

IMAGECAPS Field - ulMaxVert

ulMaxVert (ULONG)
Maximum vertical lines supported by scaler.

IMAGECAPS Field - fAccelMem

fAccelMem (BOOL)
Flag indicating whether the hardware accelerator has additional VRAM for use.

The following values are valid:

TRUE	VRAM available.
FALSE	VRAM not available.

IMAGECAPS Field - ulPhysAddrVRAM

ulPhysAddrVRAM (ULONG)
Physical address of the hardware accelerator VRAM.

This parameter is valid only if fAccelMem = TRUE.

IMAGECAPS Field - ulSize

ulSize (ULONG)
Size of hardware accelerator VRAM, in bytes.

This parameter is valid only if fAccelMem = TRUE.

IMAGECAPS Field - ulScanLineBytes

ulScanLineBytes (ULONG)
Size of scan line, in bytes.

Number of bytes includes padding. This parameter is valid only if fAccelMem = TRUE.

IMAGECAPS Field - ulNumCodecs

ulNumCodecs (ULONG)
Number of FOURCCs that follow in this structure.

IMAGECAPS Field - pCodecList

pCodecList (PCODECINFO)

Pointer to array of CODEC FOURCCs.

This field points to the array of FOURCCs supported by the hardware accelerator.

IMAGECAPS Field - ulNumSrc

ulNumSrc (ULONG)
Number of source COLORINFO data structures pointed to by *pSrcColorInfo* .

IMAGECAPS Field - pSrcColorInfo

pSrcColorInfo (PCOLORINFO)
Pointer to array of source COLORINFO data structures.

IMAGECAPS Field - ulNumDst

ulNumDst (ULONG)
Number of destination COLORINFO data structures pointed to by *pDstColorInfo* .

IMAGECAPS Field - pDstColorInfo

pDstColorInfo (PCOLORINFO)
Pointer to array of destination COLORINFO data structures.

IMAGEPACK

Information pertaining to PutImage and GetImage.

```
typedef struct _IMAGEPACK {
  ULONG          ulLength;     /*  Length of IMAGEPACK data structure, in bytes. */
  ULONG          ulFlags;      /*  Image pack flag. */
  ULONG          ulHandle;     /*  Unique ID returned by VRAMREGISTEROUT. */
  ULONG          ulID;         /*  Unique buffer ID. */
  ULONG          ulCmdMask;    /*  Specific changes since the last frame. */
  PIMAGEBUF      pPutBuf;      /*  Used as source image in PutImage command. */
  PIMAGEBUF      pGetBuf;      /*  Used as destination image in GetImage command. */
  POINTL         ptlDstOrg;    /*  Destination origin (origin at upper left). */
  ULONG          ulDstXext;    /*  Destination X extent. */
  ULONG          ulDstYext;    /*  Destination Y extent. */
  POINTL         ptlSrcOrg;    /*  Offset into the source image. */
  ULONG          ulSrcXext;    /*  Source X extent. */
  ULONG          ulSrcYext;    /*  Source Y extent. */
```

```
  ULONG          cVisRects;    /*  Number of output visible rectangles. */
  PRECTL         prctlVis;     /*  Pointer to array of visible regions. */
  POINTL         ptlWBOrg;     /*  Origin of off-screen VRAM work buffer. */
  ULONG          cWBBytes;     /*  Number of bytes in the work buffer. */
  PBYTE          pVirtVRAM;    /*  A 32-bit virtual pointer to the start of VRAM. */
  ULONG          ulGrafXPar;   /*  Transparent color for overlay type devices. */
} IMAGEPACK;

typedef IMAGEPACK *PIMAGEPACK;
```

IMAGEPACK Field - ulLength

ulLength (ULONG)
Length of IMAGEPACK data structure, in bytes.

IMAGEPACK Field - ulFlags

ulFlags (ULONG)
Image pack flag.

This flag has the following value:

IPF_DONT_INTERPOLATE
0x0001; don't use hardware color interpolation

IMAGEPACK Field - ulHandle

ulHandle (ULONG)
Unique ID returned by VRAMREGISTEROUT.

IMAGEPACK Field - ulID

ulID (ULONG)
Unique buffer ID.

IMAGEPACK Field - ulCmdMask

ulCmdMask (ULONG)
Specific changes since the last frame.

The following values are valid:

IPC_INTERMEDIATE_PUT	0x0001; not used at this time
IPC_RECTLS	0x0002; visible regions have changed
IPC_SRC_COLOR	0x0004; source color information has changed
IPC_DST_COLOR	0x0008; destination color information has changed
IPC_PALETTE	0x0010; palette color information has changed
IPC_SRC_SIZE	0x0020; source size information has changed
IPC_DST_SIZE	0x0040; destination size information has changed
IPC_DST_POS	0x0080; destination position information has changed
IPC_DROP_FRAME	0x0100; ignore this frame

IMAGEPACK Field - pPutBuf

pPutBuf (PIMAGEBUF)
Used as source image in PutImage command.

IMAGEPACK Field - pGetBuf

pGetBuf (PIMAGEBUF)
Used as destination image in GetImage command.

IMAGEPACK Field - ptlDstOrg

ptlDstOrg (POINTL)
Destination origin (origin at upper left).

IMAGEPACK Field - ulDstXext

ulDstXext (ULONG)
Destination X extent.

IMAGEPACK Field - ulDstYext

ulDstYext (ULONG)
Destination Y extent.

IMAGEPACK Field - ptlSrcOrg

ptlSrcOrg (POINTL)
Offset into the source image.

IMAGEPACK Field - ulSrcXext

ulSrcXext (ULONG)
Source X extent.

IMAGEPACK Field - ulSrcYext

ulSrcYext (ULONG)
Source Y extent.

IMAGEPACK Field - cVisRects

cVisRects (ULONG)
Number of output visible rectangles.

IMAGEPACK Field - prctlVis

prctlVis (PRECTL)
Pointer to array of visible regions.

IMAGEPACK Field - ptlWBOrg

ptlWBOrg (POINTL)
Origin of off-screen VRAM work buffer.

IMAGEPACK Field - cWBBytes

cWBBytes (ULONG)
Number of bytes in the work buffer.

IMAGEPACK Field - pVirtVRAM

pVirtVRAM (PBYTE)
A 32-bit virtual pointer to the start of VRAM.

IMAGEPACK Field - ulGrafXPar

ulGrafXPar (ULONG)
Transparent color for overlay type devices.

LINEBUNDLE

Line-attributes bundle structure.

```
typedef struct _LINEBUNDLE {
  LONG       lColor;          /*  Line foreground color. */
  LONG       lBackColor;      /*  Line background color. */
  USHORT     usMixMode;       /*  Line foreground-mix mode. */
  USHORT     usBackMixMode;   /*  Line background-mix mode. */
  FIXED      fxWidth;         /*  Line width. */
  LONG       lGeomWidth;      /*  Geometric line width. */
  USHORT     usType;          /*  Line type. */
  USHORT     usEnd;           /*  Line end. */
  USHORT     usJoin;          /*  Line join. */
  USHORT     usReserved;      /*  Reserved. */
} LINEBUNDLE;

typedef LINEBUNDLE *PLINEBUNDLE;
```

LINEBUNDLE Field - lColor

lColor (LONG)
Line foreground color.

LINEBUNDLE Field - lBackColor

lBackColor (LONG)
Line background color.

LINEBUNDLE Field - usMixMode

usMixMode (USHORT)
Line foreground-mix mode.

LINEBUNDLE Field - usBackMixMode

usBackMixMode (USHORT)
Line background-mix mode.

LINEBUNDLE Field - fxWidth

fxWidth (FIXED)
Line width.

LINEBUNDLE Field - lGeomWidth

lGeomWidth (LONG)
Geometric line width.

LINEBUNDLE Field - usType

usType (USHORT)
Line type.

LINEBUNDLE Field - usEnd

usEnd (USHORT)
Line end.

LINEBUNDLE Field - usJoin

usJoin (USHORT)
Line join.

LINEBUNDLE Field - usReserved

usReserved (USHORT)
Reserved.

LINEINFO

Line information data structure shared by cLines in the alpkLinePack array.

```
typedef struct _LINEINFO {
  ULONG         ulLength;       /*  Length of LINEINFO data structure. */
  ULONG         ulType;         /*  Defines line type. */
  ULONG         ulStyleMask;    /*  A 32-bit style mask. */
  ULONG         cLines;         /*  Count of lines to be drawn. */
  ULONG         ulFGColor;      /*  Line Foreground color. */
  ULONG         ulBGColor;      /*  Line Background color. */
  USHORT        usForeROP;      /*  Line Foreground mix. */
  USHORT        usBackROP;      /*  Line Background mix. */
  PBMAPINFO     pDstBmapInfo;   /*  Pointer to destination surface bit map. */
  PLINEPACK     alpkLinePack;   /*  Pointer to LINEPACK data structure. */
  PRECTL        prclBounds;     /*  Pointer to bounding rect of a clipped line. */
} LINEINFO;

typedef LINEINFO *PLINEINFO;
```

LINEINFO Field - ulLength

ulLength (ULONG)
Length of LINEINFO data structure.

LINEINFO Field - ulType

ulType (ULONG)
Defines line type.

LINE_SOLID	Line will be solid in Foreground color.
LINE_INVISIBLE	Line is not drawn.
LINE_ALTERNATE	Line will be alternating Foreground and Background color; ignores style.

LINEINFO Field - ulStyleMask

ulStyleMask (ULONG)
A 32-bit style mask.

LINEINFO Field - cLines

cLines (ULONG)
Count of lines to be drawn.

LINEINFO Field - ulFGColor

ulFGColor (ULONG)
Line Foreground color.

LINEINFO Field - ulBGColor

ulBGColor (ULONG)
Line Background color.

LINEINFO Field - usForeROP

usForeROP (USHORT)
Line Foreground mix.

LINEINFO Field - usBackROP

usBackROP (USHORT)
Line Background mix.

The following flags apply to ulForeROP and ulBackROP:

LR_ZERO
LR_INVERTMERGEPAT
LR_MASKINVERTPAT
LR_INVERTCOPYPAT
LR_MASKPATINVERT
LR_INVERT
LR_XORPAT
LR_INVERTMASKPAT
LR_MASKPAT
LR_INVERTXORPAT
LR_LEAVEALONE
LR_MERGEINVERTPAT
LR_PATCOPY
LR_MERGEPATINVERT
LR_MERGEPAT
LR_ONE

LINEINFO Field - pDstBmapInfo

pDstBmapInfo (PBMAPINFO)
Pointer to destination surface bit map.

LINEINFO Field - alpkLinePack

alpkLinePack (PLINEPACK)
Pointer to LINEPACK data structure.

LINEINFO Field - prclBounds

prclBounds (PRECTL)
Pointer to bounding rect of a clipped line.

LINEPACK

Line information data structure on a per-line basis.

```
typedef struct _LINEPACK {
  ULONG         ulStyleStep;       /*  Value to be added to ulStyleValue. */
  ULONG         ulStyleValue;      /*  Style value at the current pel. */
  ULONG         ulFlags;           /*  Flags used for the LINEPACK data structure. */
  LINEPACK      plpkNext;          /*  Pointer to next LINEPACK data structure. */
  ULONG         ulAbsDeltaX;       /*  Clipped Bresenham Delta X, absolute. */
  ULONG         ulAbsDeltaY;       /*  Clipped Bresenham Delta Y, absolute. */
  POINTL        ptlClipStart;      /*  Pointer to location for device to perform Bresenham algorithm. */
  POINTL        ptlClipEnd;        /*  Ending location for Bresenham algorithm (see ptlClipStart). */
  POINTL        ptlStart;          /*  Pointer to starting location for line. */
  POINTL        ptlEnd;            /*  Ending location for line. */
  LONG          lClipStartError;   /*  Standard Bresenham error at the clipped start point. */
} LINEPACK;

typedef LINEPACK *PLINEPACK;
```

LINEPACK Field - ulStyleStep

ulStyleStep (ULONG)
Value to be added to ulStyleValue.

The value to be added to ulStyleValue on each pel stepped along the style major direction.

LINEPACK Field - ulStyleValue

ulStyleValue (ULONG)
Style value at the current pel.

The style value is composed of an error value and a mask position, as follows:

high word	3 bits	5 bits	8 bits
not used	not used	mask pos	error value

LINEPACK Field - ulFlags

ulFlags (ULONG)
Flags used for the LINEPACK data structure.

LINE_DO_FIRST_PEL	Draws the first pel.
LINE_DIR_Y_POSITIVE	Indicates line direction is bottom-to-top.
LINE_HORIZONTAL	Indicates line is horizontal. No Bresenham algorithm.
LINE_X_MAJOR	Line is XMAJOR.
LINE_DIR_X_POSITIVE	Indicates line direction is right-to-left.

LINE_VERTICAL	Indicates line is vertical. No Bresenham algorithm.
LINE_STYLE_X_MAJOR	Line style is XMAJOR.
LINE_DO_LAST_PEL	Draws the last pel.

LINEPACK Field - plpkNext

plpkNext (LINEPACK)
Pointer to next LINEPACK data structure.

LINEPACK Field - ulAbsDeltaX

ulAbsDeltaX (ULONG)
Clipped Bresenham Delta X, absolute.

LINEPACK Field - ulAbsDeltaY

ulAbsDeltaY (ULONG)
Clipped Bresenham Delta Y, absolute.

LINEPACK Field - ptlClipStart

ptlClipStart (POINTL)
Pointer to location for device to perform Bresenham algorithm.

Pointer to location where device performs the Bresenham algorithm. Sets only the pels from ptlClipStart to ptlClipEnd, inclusive.

LINEPACK Field - ptlClipEnd

ptlClipEnd (POINTL)
Ending location for Bresenham algorithm (see ptlClipStart).

LINEPACK Field - ptlStart

ptlStart (POINTL)
Pointer to starting location for line.

The device can perform the Bresenham algorithm from ptlStart or ptlClipStart.

LINEPACK Field - ptlEnd

ptlEnd (POINTL)
Ending location for line.

LINEPACK Field - lClipStartError

lClipStartError (LONG)
Standard Bresenham error at the clipped start point.

Error is calculated from the initial error and the error increments for major step and diagonal step. The initial error and the error increments are as follows:

MAX
Maximum (ulAbsDeltaX, ulAbsDeltaY)

MIN
Minimum (ulAbsDeltaX, ulAbsDeltaY)

Major Increment
Increment to the error for stepping along the major axis:
2 * MIN.

Diagonal Increment
Increment to the error for stepping along the major and minor axes:
2 * MIN - 2 * MAX.

Initial Error
Error at the start point:
2 * MIN - MAX, if LINE_DIR_X_POSITIVE is On.

2 * MIN - MAX - 1, if LINE_DIR_X_POSITIVE is Off.

Horizontal and vertical lines
The line is drawn from the clipped start to clipped end.
The lClipStartError will not be given.

First pel consideration
Set the first pel at ptlStart (not ptlClipStart) only if LINE_DO_FIRST_PEL is set and the first pel is not clipped.

Last pel consideration
Set the last pel at ptlEnd (not ptlClipEnd) only if LINE_DO_LAST_PEL is set and the last pel is not clipped.

Styling
Lines are styled using the ulStyleMask, ulStyleStep, and ulStyleValue.
ulStyleMask
A 32-bit style mask.

Error Value
Error value at the current pel.

Mask Position
Bit position of the ulStyleMask.

If this bit is on, set the current pel to the ulFGColor through usForeROP; otherwise, set the current pel to the ulBGColor through usBackRop.

LONG

Signed integer in the range -2 147 483 648 through 2 147 483 647.

```
#define LONG long
```

Note: Where this data type represents a graphic coordinate in world or model space, its value is restricted to -134 217 728 through 134 217 727.

A graphic coordinate in device or screen coordinates is restricted to -32 768 through 32 767.

The value of a graphic coordinate may be further restricted by any transforms currently in force, including the positioning of the origin of the window on the screen. In particular, coordinates in world or model space must not generate coordinate values after transformation (that is, in device or screen space) outside the range -32 768 through 32 767.

LPGridRectRef

A pointer to a block of parameters in GridRectRef structure.

```
typedef GridRectRef *LPGridRectRef;
```

LPScrollRectRef

A pointer to a block of parameters in ScrollRectRef structure.

```
typedef ScrollRectRef *LPScrollRectRef;
```

PANOSE

The *Panose* field in the font metrics will allow for quantitative descriptions of the visual properties of font faces. The PANOSE definition contains ten digits, each of which currently describes up to sixteen variations.

```
typedef struct _PANOSE {
  BYTE     bFamilyType;        /*  Family kind. */
  BYTE     bSerifStyle;        /*  Serif style. */
  BYTE     bWeight;            /*  Weight. */
  BYTE     bProportion;        /*  Proportion. */
  BYTE     bContrast;          /*  Contrast. */
  BYTE     bStrokeVariation;   /*  Stroke Variation. */
  BYTE     bArmStyle;          /*  Arm Style. */
  BYTE     bLetterform;        /*  Letterform. */
  BYTE     bMidline;           /*  Midline. */
  BYTE     bXHeight;           /*  X-Height. */
  BYTE     fbPassedISO;        /*  Font passed ISO test. */
  BYTE     fbFailedISO;        /*  Font failed ISO test. */
} PANOSE;

typedef PANOSE *PPANOSE;
```

PANOSE Field - bFamilyType

bFamilyType (BYTE)
Family kind.

0	Any
1	No Fit
2	Text and Display
3	Script
4	Decorative
5	Pictorial

PANOSE Field - bSerifStyle

bSerifStyle (BYTE)
Serif style.

0	Any
1	No Fit
2	Cove
3	Obtuse Cove
4	Square Cove
5	Obtuse Square Cove
6	Square
7	Thin
8	Bone
9	Exaggerated
10	Triangle
11	Normal Sans
12	Obtuse Sans
13	Perp Sans
14	Flared
15	Rounded

PANOSE Field - bWeight

bWeight (BYTE)
Weight.

0	Any
1	No Fit
2	Very Light
3	Light
4	Thin
5	Book
6	Medium
7	Demi
8	Bold
9	Heavy
10	Black
11	Nord

PANOSE Field - bProportion

bProportion (BYTE)
Proportion.

0	Any
1	No Fit
2	Old Style
3	Modern
4	Even Width
5	Expanded
6	Condensed
7	Very Expanded
8	Very Condensed
9	Monospaced

PANOSE Field - bContrast

bContrast (BYTE)
Contrast.

0	Any
1	No Fit
2	None
3	Very Low
4	Low
5	Medium Low
6	Medium
7	Medium High
8	High
9	Very High

PANOSE Field - bStrokeVariation

bStrokeVariation (BYTE)
Stroke Variation.

0	Any
1	No Fit
2	Gradual/Diagonal
3	Gradual/Transitional
4	Gradual/Vertical
5	Gradual/Horizontal
6	Rapid/Vertical
7	Rapid/Horizontal
8	Instant/Vertical

PANOSE Field - bArmStyle

bArmStyle (BYTE)
Arm Style.

0	Any
1	No Fit
2	Straight Arms/Horizontal
3	Straight Arms/Wedge
4	Straight Arms/Vertical
5	Straight Arms/Single Serif
6	Straight Arms/Double Serif
7	Non-Straight Arms/Horizontal
8	Non-Straight Arms/Wedge
9	Non-Straight Arms/Vertical
10	Non-Straight Arms/Single Serif
11	Non-Straight Arms/Double Serif

PANOSE Field - bLetterform

bLetterform (BYTE)
Letterform.

0	Any
1	No Fit
2	Normal/Contact
3	ONormal/Weighted
4	ONormal/Boxed
5	ONormal/Flattened
6	ONormal/Rounded
7	ONormal/Off Center
8	ONormal/Square
9	Oblique/Contact
10	Oblique/Weighted
11	Oblique/Boxed
12	Oblique/Flattened
13	Oblique/Rounded
14	Oblique/Off Center
15	Oblique/Square

PANOSE Field - bMidline

bMidline (BYTE)
Midline.

0	Any
1	No Fit
2	Standard/Trimmed
3	Standard/Pointed
4	Standard/Serifed
5	High/Trimmed
6	High/Pointed
7	High/Serifed
8	Constant/Trimmed
9	Constant/Pointed
10	Constant/Serifed
11	Low/Trimmed
12	Low/Pointed
13	Low/Serifed

PANOSE Field - bXHeight

bXHeight (BYTE)
X-Height.

0	Any
1	No Fit
2	Constant/Small
3	Constant/Standard
4	Constant/Large
5	Ducking/Small
6	Ducking/Standard
7	Ducking/Large

PANOSE Field - fbPassedISO

fbPassedISO (BYTE)
Font passed ISO test.

The following flags indicate those displays and resolutions at which the font complied with ISO 9241:

FM_ISO_9518_640
FM_ISO_9515_640
FM_ISO_9515_1024
FM_ISO_9517_640
FM_ISO_9517_1024

PANOSE Field - fbFailedISO

fbFailedISO (BYTE)
Font failed ISO test.

The following flags indicate those displays and resolutions at which the font did not comply with ISO 9241:

FM_ISO_9518_640
FM_ISO_9515_640
FM_ISO_9515_1024
FM_ISO_9517_640
FM_ISO_9517_1024

PBITBLTATTRS

Pointer to BITBLTATTRS data structure.

```
typedef BITBLTATTRS *PBITBLTATTRS;
```

PCSP_INFO

Pointer to CSP_INFO data structure.

```
typedef CSP_INFO *PCSP_INFO;
```

PCH

Pointer to a character string.

```
typedef unsigned char *PCH;
```

PCSZ

Pointer to a constant null-terminated string.

```
typedef const char *PCSZ;
```

PCURVE

Pointer to CURVE data structure.

```
typedef CURVE *PCURVE;
```

PDALERTINFO

Input/output structure for the SplProtXlateCmd API.

```
typedef struct _PDALERTINFO {
  ULONG     ulVersion;  /*  Version of alert structure. */
  ULONG     ulFlags;    /*  Flags describing the returned alert structure. */
  ULONG     ulCommand;  /*  Command value describing pInData if PD_RESPONSE is set. */
} PDALERTINFO;

typedef PDALERTINFO *PPDALERTINFO;
```

PDALERTINFO Field - ulVersion

ulVersion (ULONG)
Version of alert structure.

This parameter must be set by the caller to the highest alert structure version it can handle. The initial version setting is 0 (zero). On exit, the protocol converter sets this field to the alert structure version returned.

PDALERTINFO Field - ulFlags

ulFlags (ULONG)
Flags describing the returned alert structure.

This parameter must be set by the caller to the appropriate bits as defined below. If none of these values apply, the caller should set this field to 0 (zero).

PD_SNMP - 0x00000008
Set when InData is received from the SNMP channel.

PD_ALTERNATE - 0x00000010
Set when InData is received from the alternate command channel.

PD_NEXTALERT - 0x00001000
Set when the port driver is given a PD_MOREALERTS flag, set on the previous call to SplProtXlateCmd. When PD_NEXTALERT is set, *plnData* is not used; the protocol converter returns the next PRTALERT structure that it was unable to return to the last caller. If the protocol converter does not have any more alerts to return, it should return ERROR_INFO_NOT_AVAIL.

On exit, the protocol converter sets some of the following bits. If all flags are clear, the protocol converter did not understand the data. The bits to be set are as follows:

PD_ALERT - 0x00000001
Set when the alert is returned.

PD_RESPONSE - 0x00000002
Set when a response to a query or set command is given.

PD_PARTIAL - 0x00000004
Set when not enough information is passed to create a full structure.

PD_IGNORE - 0x00000020
Set when the protocol converter handles the data and the data should not be passed on to the spooler.

PD_MOREALERTS - 0x00002000
Set when the data given to the protocol converter contains more than one alert. This flag should be set only when the PD_ALERT flag is also set.

PD_MOREALERTS signals the port driver that it should return the PRTALERT structure to the caller of BIDI_WAIT_ALERT. Then, when the port driver gets the next BIDI_WAIT_ALERT call, it should call SplProtXlateCmd with *plnData* and *cblnData* set to 0 (zero) and with the PD_NEXTALERT bit set.

When the protocol converter has returned all alerts for a given buffer, it must clear PD_MOREALERTS.

PDALERTINFO Field - ulCommand

ulCommand (ULONG)
Command value describing *plnData* if PD_RESPONSE is set.

If PD_RESPONSE is **not** set, *ulCommand* should be set to FFFFFFFF.

See BIDI Command Structures and Command Flow for a list of commands.

Note: The *ulCommand* field is only set for informational purposes. If the protocol converter does not know which command the data was from, it can set *ulCommand* to 0 (zero).

PDC_BLOCK

A pointer to an array of DC_BLOCK structures.

```
typedef DC_BLOCK *PDC_BLOCK;
```

PDELETERETURN

Pointer to DELETERETURN data structure.

```
typedef DELETERETURN *PDELETERETURN;
```

PDEVOPENDATA

Open device-data array.

This data type points to data whose format is described by the DEVOPENSTRUC data type.

```
typedef PSZ *PDEVOPENDATA;
```

PDSPINFO

Pointer to DSPINFO data structure.

```
typedef DSPINFO *PDSPINFO;
```

PDSPINSTL_CHAIN

Pointer to DSPINSTL_CHAIN data structure.

```
typedef DSPINSTL_CHAIN *PDSPINSTL_CHAIN;
```

PDSPINSTL_DATA

Pointer to DSPINSTL_DATA data structure.

```
typedef DSPINSTL_DATA *PDSPINSTL_DATA;
```

PKERNPAIRS

Pointer to KERNPAIRS data structure.

```
typedef KERNPAIRS *PKERNPAIRS;
```

PNOTIFYTRANSFORMDATA

Pointer to NOTIFYTRANSFORMDATA data structure.

```
typedef NOTIFYTRANSFORMDATA *PNOTIFYTRANSFORMDATA;
```

POINTL

Point structure (long integers).

```
typedef struct _POINTL {
  LONG     x;  /*  X-coordinate. */
  LONG     y;  /*  Y-coordinate. */
} POINTL;

typedef POINTL *PPOINTL;
```

POINTL Field - x

x (LONG)
X-coordinate.

POINTL Field - y

y (LONG)
Y-coordinate.

POLYGON

Polygon structure.

```
typedef struct _POLYGON {
  ULONG       ulPoints;  /*  Number of points in array. */
  PPOINTL     aPointl;   /*  Array of points. */
} POLYGON;

typedef POLYGON *PPOLYGON;
```

POLYGON Field - ulPoints

ulPoints (ULONG)
Number of points in array.

POLYGON Field - aPointl

aPointl (PPOINTL)
Array of points.

PPALETTEINFO

Pointer to PALETTEINFO data structure.

```
typedef PALETTEINFO *PPALETTEINFO;
```

PPOINTERINFO

Pointer information structure.

```
typedef struct _PPOINTERINFO {
  USHORT      fPointer;
  SHORT       xHotspot;
  SHORT       yHotspot;
  HBITMAP     hbmPointer;
  HBITMAP     hbmColor;
} PPOINTERINFO;
```

```
typedef PPOINTERINFO *PPOINTERINFO;
```

PPOINTERINFO Field - fPointer

fPointer (USHORT)

PPOINTERINFO Field - xHotspot

xHotspot (SHORT)

PPOINTERINFO Field - yHotspot

yHotspot (SHORT)

PPOINTERINFO Field - hbmPointer

hbmPointer (HBITMAP)

PPOINTERINFO Field - hbmColor

hbmColor (HBITMAP)

PORTFROMQ

Structure returned by SplGetPortFromQ.

```
typedef struct _PORTFROMQ {
  PSZ       pszComputer;         /*  Name of the computer on which the port exisits. */
```

```
  PSZ       pszPort;             /*  Name of the port connected to the queue. */
  PSZ       pszDeviceID;         /*  Device ID string returned by the printer. */
  ULONG     flBidiCapabilities;  /*  Printer capabilities. */
  ULONG     flJobs;              /*  Job control flags. */
} PORTFROMQ;

typedef PORTFROMQ *PPORTFROMQ;
```

PORTFROMQ Field - pszComputer

pszComputer (PSZ)
Name of the computer on which the port exisits.

This information is useful if the print queue references a network print queue.

PORTFROMQ Field - pszPort

pszPort (PSZ)
Name of the port connected to the queue.

PORTFROMQ Field - pszDeviceID

pszDeviceID (PSZ)
Device ID string returned by the printer.

PORTFROMQ Field - flBidiCapabilities

flBidiCapabilities (ULONG)
Printer capabilities.

See BIDI_Q_PORT(800Bh) in BIDI Command Structures and Command Flow for definitions of the PRTPORT_CAPS_xx defines.

PORTFROMQ Field - flJobs

flJobs (ULONG)
Job control flags.

See BIDI_Q_SW(800Ch) in BIDI Command Structures and Command Flow for definitions of the PRTSW_JOB_xx defines.

PQMOPENDATA

Open queue-manager data array.

This data type points to data whose format is described by the DEVOPENSTRUC data type.

```
typedef PSZ *PQMOPENDATA;
```

PRQINFO3

Print-queue information structure.

This structure is used at information levels 3 and 4.

```
typedef struct _PRQINFO3 {
  PSZ           pszName;        /*  Queue name. */
  USHORT        uPriority;      /*  Queue priority. */
  USHORT        uStartTime;     /*  Minutes after midnight when queue becomes active. */
  USHORT        uUntilTime;     /*  Minutes after midnight. when queue ceases to be active. */
  USHORT        fsType;         /*  Queue type. */
  PSZ           pszSepFile;     /*  Separator-page file. */
  PSZ           pszPrProc;      /*  Default queue-processor. */
  PSZ           pszParms;       /*  Queue parameters. */
  PSZ           pszComment;     /*  Queue description. */
  USHORT        fsStatus;       /*  Queue status. */
  USHORT        cJobs;          /*  Number of jobs in queue. */
  PSZ           pszPrinters;    /*  Print devices connected to queue. */
  PSZ           pszDriverName;  /*  Default device driver. */
  PDRIVDATA     pDriverData;    /*  Default queue job properties. */
} PRQINFO3;

typedef PRQINFO3 *PPRQINFO3;
```

PRQINFO3 Field - pszName

pszName (PSZ)
Queue name.

The maximum length of the name in the network case is 256 (including one byte for zero termination).

PRQINFO3 Field - uPriority

uPriority (USHORT)
Queue priority.

The range is 1 through 9, with 1 being the highest queue priority.

The default job priority (DefJobPrio) is determined from:
DefJobPrio=100-(10* *uPriority*).

If a job is added with *PRJ_NO_PRIORITY* specified, DefJobPrio is used. If a default priority higher than the default job priority is specified, the default job priority is used. If a default priority lower than the default is specified, the specified job priority is used.

PRQ_DEF_PRIORITY
: Default priority

PRQ_MAX_PRIORITY
: Highest priority

PRQ_MIN_PRIORITY
: Minimum priority

PRQ_NO_PRIORITY
: No priority.

PRQINFO3 Field - uStartTime

uStartTime (USHORT)
Minutes after midnight when queue becomes active.

For example, the value 75 represents 1:15 a.m.

If *uStartTime* and *uUntilTime* are both 0, the print queue is always available.

PRQINFO3 Field - uUntilTime

uUntilTime (USHORT)
Minutes after midnight. when queue ceases to be active.

For example, the value 1200 represents 8 p.m.

If *uUntilTime* and *uStartTime* are both 0, the print queue is always available.

PRQINFO3 Field - fsType

fsType (USHORT)
Queue type.

PRQ3_TYPE_RAW
: Data is always enqueued in the device specific format.

PRQ3_TYPE_BYPASS
: Allows the spooler to bypass the queue processor and send data directly to the Printer Driver. Setting this bit allows the spooler to print jobs of type PM_Q_RAW while they are still being spooled.

PRQ3_TYPE_APPDEFAULT
: This bit is set for the application default queue only.

PRQINFO3 Field - pszSepFile

pszSepFile (PSZ)
Separator-page file.

The path and file name of a separator-page file on the target computer.

This file contains formatting information for the page or pages to be used between print jobs. A relative path name is taken as relative to the current spool directory. A NULL string indicates no separator page.

PRQINFO3 Field - pszPrProc

pszPrProc (PSZ)
Default queue-processor.

PRQINFO3 Field - pszParms

pszParms (PSZ)
Queue parameters.

This can be any text string or a NULL string.

PRQINFO3 Field - pszComment

pszComment (PSZ)
Queue description.

A NULL string results in no comment. The maximum length is 48 characters (including one byte for the null terminator).

PRQINFO3 Field - fsStatus

fsStatus (USHORT)
Queue status.

PRQ3_PAUSED
Queue is paused (held).
PRQ3_PENDING
Queue is pending deletion.

PRQINFO3 Field - cJobs

cJobs (USHORT)
Number of jobs in queue.

PRQINFO3 Field - pszPrinters

pszPrinters (PSZ)
Print devices connected to queue.

This cannot be NULL.

PRQINFO3 Field - pszDriverName

pszDriverName (PSZ)
Default device driver.

PRQINFO3 Field - pDriverData

pDriverData (PDRIVDATA)
Default queue job properties.

Note: An application can use *pszDriverName* , *pDriverData* , *pszPrProc* , and *pszParms* to construct a valid DevOpenDC call based only on the queue name.

PRJINFO2

Print-job information structure.

This structure provides a subset of the information supplied by PRJINFO3. It minimizes the storage required for job-information retrieval, and is sufficient for most uses.

```
typedef struct _PRJINFO2 {
  USHORT     uJobId;       /*  Job identification number. */
  USHORT     uPriority;    /*  Job priority. */
  PSZ        pszUserName;  /*  User who submitted the job. */
  USHORT     uPosition;    /*  Job position in queue. */
  USHORT     fsStatus;     /*  Job status. */
  ULONG      ulSubmitted;  /*  Time job submitted. */
  ULONG      ulSize;       /*  Print-job size (bytes). */
  PSZ        pszComment;   /*  Comment string. */
  PSZ        pszDocument;  /*  Document name. */
} PRJINFO2;

typedef PRJINFO2 *PPRJINFO2;
```

PRJINFO2 Field - uJobId

uJobId (USHORT)
Job identification number.

PRJINFO2 Field - uPriority

uPriority (USHORT)
Job priority.

The job-priority range is 1 through 99, with 99 the highest job priority. (For queue priorities, 1 is the highest priority.)

The job priority determines the order of jobs in the queue. If multiple queues print to the same printer, the job at the front of each queue is examined. The job with the highest priority is printed first; if there is more than one job with the highest priority, the oldest job with this priority is printed first.

PRJ_MAX_PRIORITY
Highest priority
PRJ_MIN_PRIORITY
Lowest priority
PRJ_NO_PRIORITY
No priority.

PRJINFO2 Field - pszUserName

pszUserName (PSZ)
User who submitted the job.

This parameter applies only to jobs created by a user and enqueued on a remote server. A NULL string signifies a local job.

PRJINFO2 Field - uPosition

uPosition (USHORT)
Job position in queue.

If 1, the job is scheduled to be the next job printed from this queue.

PRJINFO2 Field - fsStatus

fsStatus (USHORT)
Job status.

To find the job status, use the PRJ_QSTATUS mask:

PRJ_QS_QUEUED	Queued
PRJ_QS_PAUSED	Paused by a SplHoldJob function
PRJ_QS_SPOOLING	Job being created

PRJ_QS_PRINTING	Printing (bits 2 through 11 are valid).

For further information, use the PRJ_DEVSTATUS mask:

PRJ_COMPLETE	Job complete
PRJ_INTERV	Intervention required
PRJ_ERROR	Error occurred.
PRJ_DESTOFFLINE	Print destination offline
PRJ_DESTPAUSED	Print destination paused
PRJ_NOTIFY	Alert should be raised
PRJ_DESTNOPAPER	Print destination out of paper
PRJ_DESTFORMCHG	Printer waiting for form change
PRJ_DESTCRTCHG	Printer waiting for cartridge change
PRJ_DESTPENCHG	Printer waiting for pen change.

This bit indicates that the job is deleted:

PRJ_DELETED	Job deleted.

PRJINFO2 Field - ulSubmitted

ulSubmitted (ULONG)
Time job submitted.

Time format is the same as that stored in the global information segment.

PRJINFO2 Field - ulSize

ulSize (ULONG)
Print-job size (bytes).

PRJINFO2 Field - pszComment

pszComment (PSZ)
Comment string.

Information about the print job. The maximum length of the string is 48 characters (including one byte for the null terminator).

PRJINFO2 Field - pszDocument

pszDocument (PSZ)
Document name.

The document name of the print job (set by the application that submitted the print job). The maximum length of the string is 260 characters.

PRQINFO6

Print-queue information structure.

This structure is used at information level 6.

```
typedef struct _PRQINFO6 {
  PSZ           pszName;               /*  Queue name. */
  USHORT        uPriority;             /*  Queue priority. */
  USHORT        uStartTime;            /*  Minutes after midnight when queue becomes active. */
  USHORT        uUntilTime;            /*  Minutes after midnight. when queue ceases to be active. */
  USHORT        fsType;                /*  Queue type. */
  PSZ           pszSepFile;            /*  Separator-page file. */
  PSZ           pszPrProc;             /*  Default queue-processor. */
  PSZ           pszParms;              /*  Queue parameters. */
  PSZ           pszComment;            /*  Queue description. */
  USHORT        fsStatus;              /*  Queue status. */
  USHORT        cJobs;                 /*  Number of jobs in queue. */
  PSZ           pszPrinters;           /*  Print devices connected to queue. */
  PSZ           pszDriverName;         /*  Default device driver. */
  PDRIVDATA     pDriverData;           /*  Default queue job properties. */
  PSZ           pszRemoteComputerName; /*  Remote computer name. */
  PSZ           pszRemoteQueueName;    /*  Remote queue name. */
} PRQINFO6;

typedef PRQINFO6 *PPRQINFO6;
```

PRQINFO6 Field - pszName

pszName (PSZ)
Queue name.

The maximum length of the name in the network case is 256 (including one byte for zero termination).

PRQINFO6 Field - uPriority

uPriority (USHORT)
Queue priority.

The range is 1 through 9, with 1 being the highest queue priority.

The default job priority (DefJobPrio) is determined from:
DefJobPrio=100-(10* *uPriority*).

If a job is added with *PRJ_NO_PRIORITY* specified, DefJobPrio is used. If a default priority higher than the default job priority is specified, the default job priority is used. If a default priority lower than the default is specified, the specified job priority is used.

PRQ_DEF_PRIORITY
: Default priority

PRQ_MAX_PRIORITY
: Highest priority

PRQ_MIN_PRIORITY
: Minimum priority

PRQ_NO_PRIORITY
: No priority.

PRQINFO6 Field - uStartTime

uStartTime (USHORT)
Minutes after midnight when queue becomes active.

For example, the value 75 represents 1:15 a.m.

If *uStartTime* and *uUntilTime* are both 0, the print queue is always available.

PRQINFO6 Field - uUntilTime

uUntilTime (USHORT)
Minutes after midnight. when queue ceases to be active.

For example, the value 1200 represents 8 p.m.

If *uUntilTime* and *uStartTime* are both 0, the print queue is always available.

PRQINFO6 Field - fsType

fsType (USHORT)
Queue type.

PRQ3_TYPE_RAW
Data is always enqueued in the device specific format.
PRQ3_TYPE_BYPASS
Allows the spooler to bypass the queue processor and send data directly to the Printer Driver. Setting this bit allows the spooler to print jobs of type PM_Q_RAW while they are still being spooled.
PRQ3_TYPE_APPDEFAULT
This bit is set for the application default queue only.

PRQINFO6 Field - pszSepFile

pszSepFile (PSZ)
Separator-page file.

The path and file name of a separator-page file on the target computer.

This file contains formatting information for the page or pages to be used between print jobs. A relative path name is taken as relative to the current spool directory. A NULL string indicates no separator page.

PRQINFO6 Field - pszPrProc

pszPrProc (PSZ)
Default queue-processor.

PRQINFO6 Field - pszParms

pszParms (PSZ)
Queue parameters.

This can be any text string or a NULL string.

PRQINFO6 Field - pszComment

pszComment (PSZ)
Queue description.

A NULL string results in no comment. The maximum length is 48 characters (including one byte for the null terminator).

PRQINFO6 Field - fsStatus

fsStatus (USHORT)
Queue status.

PRQ3_PAUSED
Queue is paused (held).
PRQ3_PENDING
Queue is pending deletion.

PRQINFO6 Field - cJobs

cJobs (USHORT)
Number of jobs in queue.

PRQINFO6 Field - pszPrinters

pszPrinters (PSZ)
Print devices connected to queue.

This cannot be NULL.

PRQINFO6 Field - pszDriverName

pszDriverName (PSZ)
Default device driver.

PRQINFO6 Field - pDriverData

pDriverData (PDRIVDATA)
Default queue job properties.

Note: An application can use *pszDriverName* , *pDriverData* , *pszPrProc* , and *pszParms* to construct a valid DevOpenDC call based only on the queue name.

PRQINFO6 Field - pszRemoteComputerName

pszRemoteComputerName (PSZ)
Remote computer name.

The computer name part of a remote queue for which this queue is a local alias.

PRQINFO6 Field - pszRemoteQueueName

pszRemoteQueueName (PSZ)
Remote queue name.

The queue name part of a remote queue for which this queue is a local alias.

PRJINFO4

Information about the PRJINFO4 data structure, which stores accounting information about print jobs.

```
typedef struct _PRJINFO4 {
  USHORT     uJobID;             /*  Spooler job identifier. */
  USHORT     uPriority;          /*  Priority of the print job. */
  PSZ        pszUserName;        /*  Name of job submitter. */
  USHORT     uPosition;          /*  Job position in the print queue. */
  USHORT     fsStatus;           /*  Job status bits. */
  ULONG      ulSubmitted;        /*  Date and time of job submission. */
  ULONG      ulSize;             /*  Size of the spooled job. */
  PSZ        pszComment;         /*  Job comment string. */
```

```
  PSZ         pszDocument;         /*  Job document name. */
  PSZ         pszSpoolFileName;    /*  Path to spool file if it is not in the standard spool directory. */
  PSZ         pszPortName;         /*  Port on which this job is printing. */
  PSZ         pszStatus;           /*  Job status string. */
  ULONG       ulPagesSpooled;      /*  Total pages spooled so far. */
  ULONG       ulPagesSent;         /*  Total pages sent to the printer for this job. */
  ULONG       ulPagesPrinted;      /*  Total pages stacked for this job. */
  ULONG       ulTimePrinted;       /*  Processing time used in printer. */
  ULONG       ulExtendJobStatus;   /*  Extended job status flags. */
  ULONG       ulStartPage;         /*  First page to begin printing. */
  ULONG       ulEndPage;           /*  Last page to print for this job. */
} PRJINFO4;

typedef PRJINFO4 *PPRJINFO4;
```

PRJINFO4 Field - uJobID

uJobID (USHORT)
Spooler job identifier.

PRJINFO4 Field - uPriority

uPriority (USHORT)
Priority of the print job.

PRJINFO4 Field - pszUserName

pszUserName (PSZ)
Name of job submitter.

PRJINFO4 Field - uPosition

uPosition (USHORT)
Job position in the print queue.

PRJINFO4 Field - fsStatus

fsStatus (USHORT)
Job status bits.

PRJINFO4 Field - ulSubmitted

ulSubmitted (ULONG)
Date and time of job submission.

PRJINFO4 Field - ulSize

ulSize (ULONG)
Size of the spooled job.

PRJINFO4 Field - pszComment

pszComment (PSZ)
Job comment string.

PRJINFO4 Field - pszDocument

pszDocument (PSZ)
Job document name.

PRJINFO4 Field - pszSpoolFileName

pszSpoolFileName (PSZ)
Path to spool file if it is not in the standard spool directory.

This field is null except when an application wants to submit a print job without spooling it (due to the size of the print file).
pszSpoolFileName can be set with SplSetJob using the PRJ_SPOOLFILENAME_PARMNUM(19) parameter.

PRJINFO4 Field - pszPortName

pszPortName (PSZ)
Port on which this job is printing.

PRJINFO4 Field - pszStatus

pszStatus (PSZ)
Job status string.

PRJINFO4 Field - ulPagesSpooled

ulPagesSpooled (ULONG)
Total pages spooled so far.

This value will be 0 (zero) for printer drivers that do not call SplQmNewPage. *ulPagesSpooled* can be set with SplSetJob using the PRJ_PAGESSPOOLED_PARMNUM(20) parameter.

PRJINFO4 Field - ulPagesSent

ulPagesSent (ULONG)
Total pages sent to the printer for this job.

This value reflects the number of pages sent to the printer so far, if PrtNewPage is used by the printer driver. A value of 0 (zero) indicates the printer driver is **not** calling PrtNewPage. *ulPagesSent* can be set with SplSetJob using the PRJ_PAGESSENT_PARMNUM(21) parameter.

PRJINFO4 Field - ulPagesPrinted

ulPagesPrinted (ULONG)
Total pages stacked for this job.

This value will be known only for BIDI printers. It indicates the number of pages the printer has confirmed were output. *ulPagesPrinted* can be set with SplSetJob using the PRJ_PAGESPRINTED_PARMNUM(22) parameter.

PRJINFO4 Field - ulTimePrinted

ulTimePrinted (ULONG)
Processing time used in printer.

If the printer is unable to tell host, 0 (zero) will be used. *ulTimePrinted* can be set with SplSetJob using the PRJ_TIMEPRINTED_PARMNUM(23) parameter.

PRJINFO4 Field - ulExtendJobStatus

ulExtendJobStatus (ULONG)
Extended job status flags.

Values are as follows:

Bit	Description
0	PRJ4_INPRINTER - 0x00000001 Job in printer (0=No; 1=Yes)
1	PRJ4_STACKED - 0x00000002 Job is stacked (0=No; 1=Yes)
2	PRJ4_HELDINPRINTER - 0x00000004 Job held in printer (0=No; 1=Yes)
3	PRJ4_JOBSTARTED - 0x00000008 Job has begun printing (0=No; 1=Yes)
4 - 31	Reserved

This field can be set with SplSetJob using the PRJ_EXTENDSTATUSus.PARMNUM(24) parameter.

PRJINFO4 Field - ulStartPage

ulStartPage (ULONG)
First page to begin printing.

This value indicates the first page to begin printing, if the printer supports this feature. A value of 0 (zero) indicates printing should begin with the first page. *ulStartPage* can be set with SplSetJob using the PRJ_STARTPAGE_PARMNUM(25) parameter.

PRJINFO4 Field - ulEndPage

ulEndPage (ULONG)
Last page to print for this job.

This value indicates the last page to print for this job, if the printer supports this feature. *ulEndPage* can be set with SplSetJob using the PRJ_ENDPAGE_PARMNUM(26) parameter.

PRLEHDR

Pointer to RLEHDR data structure.

```
typedef RLEHDR *PRLEHDR;
```

PRPORTINFO2

Information about the current port settings, used by SplQueryPort and SplSetPort.

```
typedef struct _PRPORTINFO2 {
  PSZ       pszPortName;           /*  Name of the port. */
  PSZ       pszPortDriver;         /*  Port driver used by the port. */
  PSZ       pszProtocolConverter;  /*  Protocol converter used by the port. */
  ULONG     ulReserved;            /*  Set to 0 (zero) for now. */
  ULONG     ulMode;                /*  BIDI mode of the port. */
  ULONG     ulPriority;            /*  Currently not used. */
} PRPORTINFO2;

typedef PRPORTINFO2 *PPRPORTINFO2;
```

PRPORTINFO2 Field - pszPortName

pszPortName (PSZ)
Name of the port.

PRPORTINFO2 Field - pszPortDriver

pszPortDriver (PSZ)
Port driver used by the port.

PRPORTINFO2 Field - pszProtocolConverter

pszProtocolConverter (PSZ)
Protocol converter used by the port.

PRPORTINFO2 Field - ulReserved

ulReserved (ULONG)
Set to 0 (zero) for now.

PRPORTINFO2 Field - ulMode

ulMode (ULONG)
BIDI mode of the port.

Values are as follows:

Value	Description
1	PRPORT_AUTODETECT Auto-detect mode. The port driver will tell the spooler if the printer is BIDI-capable. This is the default.
2	PRPORT_DISABLE_BIDI BIDI is disabled. The spooler will not use the printer in BIDI mode. This value overrides the port driver.
3	PRPORT_ENABLE_BIDI BIDI is enabled. This is set only if an application knows a printer is capable of using a protocol converter, but the port driver does not know which converter to use. The application must also supply the protocol converter name when setting a port to this mode.

PRPORTINFO2 Field - ulPriority

ulPriority (ULONG)
Currently not used.

PRTALERT

Information returned by the BIDI_WAIT_ALERT command.

```
typedef struct _PRTALERT {
  ULONG      ulpszPortName;    /*  Name of the port generating the alert. */
  ULONG      ulCategory;       /*  Category of alert being returned. */
  ULONG      ulType;           /*  Type of alert in a given category. */
  BYTE       interpreterID;    /*  Interpreter identifier that returns this alert. */
  BYTE       bReserved;        /*  Reserved; must be 0 (zero). */
  USHORT     usSeverity;       /*  Severity of problem. */
  ULONG      ulValue;          /*  Value associated with the alert. */
  ULONG      ulPrinterJobID;   /*  Printer job identifier if this is a print job core alert. */
  ULONG      ulpszAlertMsg;    /*  Optional alert text. */
} PRTALERT;

typedef PRTALERT *PPRTALERT;
```

PRTALERT Field - ulpszPortName

ulpszPortName (ULONG)
Name of the port generating the alert.

This value is returned to allow a single spooler thread to wait for alerts on any port supported by a port driver.

PRTALERT Field - ulCategory

ulCategory (ULONG)
Category of alert being returned.

Values are as follows:

Value	Description
0x00000001	PRTALERT_CATEGORY_CORE - 0x00000001 Core alert. The *ulType* field will indicate the specific alert being returned.
0x0000D000	PRTALERT_CATEGORY_EXTENDED - 0x0000D000 Extended alert. This is the **first** number available for extended alert categories.
0x0000DFFF	PRTALERT_CATEGORY_EXT_MAX - 0x0000DFFF Maximum extended alert. This is the **last** number available for extended alert categories.

PRTALERT Field - ulType

ulType (ULONG)
Type of alert in a given category.

If *ulCategory* is PRTALERT_CATEGORY_CORE, the type will be one of the following:

Value	Description
1	PRTALERT_TYPE_INPUT - 1 Input alert (for example, out of paper)
2	PRTALERT_TYPE_OUTPUT - 2 Output alert (for example, output bin is full)
3	PRTALERT_TYPE_JAM - 3 Paper jam alert
4	PRTALERT_TYPE_OPERATOR - 4 Operator intervention required (for example, print ribbon jam)
5	PRTALERT_TYPE_CONFIG_CHANGE - 5 Configuration changed (for example, cartridge removed)
6	PRTALERT_TYPE_SUPPLIES - 6 Supplies alert (for example, out of toner)
7	PRTALERT_TYPE_JOB_START - 7 Job started printing
8	PRTALERT_TYPE_JOB_STACKED - 8 Job completed and stacked
9	PRTALERT_TYPE_PAGE_PRINTED - 9 Page printed
10	PRTALERT_TYPE_JOB_CANCELLED - 10 Job cancelled inside printer
11	PRTALERT_TYPE_JOB_HELD - 11 Job held inside printer
12	PRTALERT_TYPE_COVER_OPEN - 12 Cover or panel open.
13	PRTALERT_TYPE_POWER_ON - 13 Printer powered on
14	PRTALERT_TYPE_RESET - 14 Printer reset

15	PRTALERT_TYPE_ONLINE - 15 Printer now online
16	PRTALERT_TYPE_OFFLINE - 16 Printer now offline
17	PRTALERT_TYPE_TIMED_ALERT - 17 Timed alert
18	PRTALERT_TYPE_COMMUNICATION_PROBLEM - 18 Communication problem
19	PRTALERT_TYPE_COMM_STATUS_CHANGED - 19 Communication status changed. BIDI_Q_PORT should be issued to get current state.
0xFFFFFFFF	PRTALERT_TYPE_OTHER - 0xFFFFFFFF Other alert type

PRTALERT Field - interpreterID

interpreterID (BYTE)
Interpreter identifier that returns this alert.

Values are as follows:

Value	Description
00	PRTALERT_INTERPRETER_CURRENT - 0x00 Current interpreter
FE	PRTALERT_INTERPRETER_NOTAPPLIC - 0xFE Not applicable
FF	PRTALERT_INTERPRETER_UNKNOWN - 0xFF Unknown

PRTALERT Field - bReserved

bReserved (BYTE)
Reserved; must be 0 (zero).

PRTALERT Field - usSeverity

usSeverity (USHORT)
Severity of problem.

Values are as follows:

Value	Description
00	PRTALERT_SEV_INFORMATIONAL - 0
01	PRTALERT_SEV_WARNING - 1
02	PRTALERT_SEV_ERROR - 2

	Error; printing stopped
03	PRTALERT_SEV_SERVICE - 3 Service required
04 - FD	Reserved
FE	PRTALERT_SEV_OTHER - 0xFE
FF	PRTALERT_SEV_UNKNOWN - 0xFF

PRTALERT Field - ulValue

ulValue (ULONG)
Value associated with the alert.

If this is a core alert and the *ulType* field is set to PRTALERT_TYPE_PAGE_PRINTED, this value is the page number printed.

PRTALERT Field - ulPrinterJobID

ulPrinterJobID (ULONG)
Printer job identifier if this is a print job core alert.

PRTALERT Field - ulpszAlertMsg

ulpszAlertMsg (ULONG)
Optional alert text.

If *alertSeverity* is "Error" or "Service required," the spooler will set the actively printing job's status string to this message.

PRTCNVINFO

Information returned by the BIDI_Q_CONVERTER_INFO command.

```
typedef struct _PRTCNVINFO {
  ULONG     ulBaseVersion;           /*  Version number of the base protocol converter. */
  ULONG     ulpszBaseConverterName;  /*  Name of the base protocol converter for this printer. */
  ULONG     ulpszBaseDescription;    /*  Descriptive name of the base converter. */
  ULONG     ulExtensionVersion;      /*  Version number of the extension protocol converter. */
  ULONG     ulpszExtConverterName;   /*  Name of the extension protocol converter. */
  ULONG     ulpszExtDescription;     /*  Descriptive name of the extension converter. */
} PRTCNVINFO;

typedef PRTCNVINFO *PPRTCNVINFO;
```

PRTCNVINFO Field - ulBaseVersion

ulBaseVersion (ULONG)
Version number of the base protocol converter.

PRTCNVINFO Field - ulpszBaseConverterName

ulpszBaseConverterName (ULONG)
Name of the base protocol converter for this printer.

The base protocol converters shipped with OS/2 are "NPAP" and "PJL."

PRTCNVINFO Field - ulpszBaseDescription

ulpszBaseDescription (ULONG)
Descriptive name of the base converter.

PRTCNVINFO Field - ulExtensionVersion

ulExtensionVersion (ULONG)
Version number of the extension protocol converter.

PRTCNVINFO Field - ulpszExtConverterName

ulpszExtConverterName (ULONG)
Name of the extension protocol converter.

If this port does not use an extension converter, this value is NULL.

PRTCNVINFO Field - ulpszExtDescription

ulpszExtDescription (ULONG)
Descriptive name of the extension converter.

PRTCONTROLJOB

Job structure used with the BIDI_Q_JOBID and BIDI_ENDJOB commands.

```
typedef struct _PRTCONTROLJOB {
  ULONG     interpreterID;    /*  Identifier of the interpreter that is processing this job. */
  ULONG     ulPrinterJobID;   /*  Printer job identifier. */
  ULONG     ulSpoolerJobID;   /*  Spooler job identifier. */
} PRTCONTROLJOB;

typedef PRTCONTROLJOB *PPRTCONTROLJOB;
```

PRTCONTROLJOB Field - interpreterID

interpreterID (ULONG)
Identifier of the interpreter that is processing this job.

PRTCONTROLJOB Field - ulPrinterJobID

ulPrinterJobID (ULONG)
Printer job identifier.

Values are as follows:

For a BIDI_HOLDJOB call:
This value is the printer job identifier returned by BIDI_Q_JOBID. A value of 0 (zero) indicates to hold all jobs in the printer.

For a BIDI_RELEASEJOB call:
This value is the printer job identifier returned by BIDI_Q_JOBID. A value of 0 (zero) indicates to release all jobs in the printer.

For a BIDI_CANCELJOB call:
This value is the printer job identifier returned by BIDI_Q_JOBID. A value of 0 (zero) indicates to cancel all jobs in the printer.

PRTCONTROLJOB Field - ulSpoolerJobID

ulSpoolerJobID (ULONG)
Spooler job identifier.

Values are as follows:

For a BIDI_HOLDJOB call:
This value is used if the caller of PrtSet had *ulPrinterJobID* set to -1.

For a BIDI_RELEASEJOB call:
This value is used if the caller of PrtSet had *ulPrinterJobID* set to -1.

For a BIDI_CANCELJOB call:
This value is used if the caller of PrtSet had *ulPrinterJobID* set to -1.

PRTDEVCHAR

Information about printer device characteristics, returned by the BIDI_Q_DEVICE command.

```
typedef struct _PRTDEVCHAR {
  USHORT     majorRev;              /*  Major revision level of software protocol. */
  USHORT     minorRev;              /*  Minor revision level of software protocol. */
  USHORT     cLogicalConnections;   /*  Number of logical connections with which the printer is capable of commu
  USHORT     cPhysicalConnections;  /*  Number of physical ports in the printer. */
  USHORT     cActiveConnections;    /*  Number of different connections active with this printer. */
  USHORT     usReserved;            /*  Must be 0 (zero). */
  USHORT     extensionRev;          /*  Revision extension-level of software protocol. */
  USHORT     maxJobsComplete;       /*  Maximum number of jobs about which the printer can retain information. *
  BYTE       markingTech;           /*  Marking technology used by the printer. */
  BYTE       colorCaps;             /*  Color capabilities of the printer. */
  USHORT     colorlevels;           /*  Number of color or monochrome levels supported. */
  BYTE       duplexCaps;            /*  Duplex capabilities of the printer. */
  BYTE       lengthUnits;           /*  Units used by the printer when reporting lengths. */
  USHORT     horizUnits;            /*  Logical units per lengthUnit in the horizontal direction. */
  USHORT     vertUnits;             /*  Logical units per lengthUnit in the vertical direction. */
  USHORT     speed;                 /*  Maximum speed of the printer. */
  BYTE       speedUnits;            /*  Units indicating speed capabilities of the printer. */
  BYTE       counterUnits;          /*  Units of measurement used in printer statistics. */
  BYTE       interpreters;          /*  Count of interpreters in the printer. */
  BYTE       inputs;                /*  Count of printing media inputs currently in the printer. */
  BYTE       outputs;               /*  Count of output destinations for printing media. */
  BYTE       options;               /*  Count of options installed in the printer. */
  BYTE       msgLanguage;           /*  Language used in text messages sent from printer. */
  BYTE       ubReserved;            /*  Reserved; must be 0 (zero). */
  ULONG      maxReceivePacketSize;  /*  Maximum size, in bytes, of a data packet that can be received by the pri
  USHORT     maxOutstandingCmds;    /*  Number of commands that the printer can receive and be working on at one
  ULONG      memory;                /*  Total RAM installed in printer in K bytes. */
  ULONG      ulpszPrinterName;      /*  Model and manufacturer of printer. */
  ULONG      ulpszRevision;         /*  Revision level of the printer. */
  ULONG      ulpszSerialNo;         /*  Serial number of the printer. */
} PRTDEVCHAR;

typedef PRTDEVCHAR *PPRTDEVCHAR;
```

PRTDEVCHAR Field - majorRev

majorRev (USHORT)
Major revision level of software protocol.

PRTDEVCHAR Field - minorRev

minorRev (USHORT)
Minor revision level of software protocol.

PRTDEVCHAR Field - cLogicalConnections

cLogicalConnections (USHORT)

Number of logical connections with which the printer is capable of communicating.

For a token-ring card, a printer might support more than one logical connection. A value of hex 'FFFF' indicates number of connections is unknown.

PRTDEVCHAR Field - cPhysicalConnections

cPhysicalConnections (USHORT)
Number of physical ports in the printer.

A value of hex 'FFFF' indicates number of ports is unknown.

PRTDEVCHAR Field - cActiveConnections

cActiveConnections (USHORT)
Number of different connections active with this printer.

A value of hex 'FFFF' indicates number of connections is unknown.

PRTDEVCHAR Field - usReserved

usReserved (USHORT)
Must be 0 (zero).

PRTDEVCHAR Field - extensionRev

extensionRev (USHORT)
Revision extension-level of software protocol.

PRTDEVCHAR Field - maxJobsComplete

maxJobsComplete (USHORT)
Maximum number of jobs about which the printer can retain information.

PRTDEVCHAR Field - markingTech

markingTech (BYTE)
Marking technology used by the printer.

Values are as follows:

Value	Description
00	Electrophotographic
01	Impact; moving head - dot matrix
02	Impact; moving head - formed
03	Impact; band
04	Aqueous ink jet
05	Solid ink jet
06	Pen
07	Thermal transfer
08	Thermal sensitive
09	Thermal diffusion (sublimation)
0A	Electrostatic
0B	Photographic
0C	Ion deposition
0D	E-beam
0E - FD	Reserved
FE	Other
FF	Unknown

PRTDEVCHAR Field - colorCaps

colorCaps (BYTE)
Color capabilities of the printer.

Values are as follows:

Value	Description
00	Monochrome
01	Spot color
02	3-primary (CMY) color
03	4-primary (CMYK) color
04 - FD	Reserved
FE	Other
FF	Unknown

PRTDEVCHAR Field - colorlevels

colorlevels (USHORT)

Number of color or monochrome levels supported.

For **monochrome** printers, this field contains the number of distinct gray-scale levels supported at the printer's maximum resolution, not including *white* or *unmarked*. For example, a black/white printer with no gray-scale capability should report 0001 in this field.

For **spot-color** printers, this field contains the number of spot colors supported, not including *white* or *unmarked*. For example, a black/white/red printer should report 0002 in this field.

For **3-primary** and **4-primary** color printers, this field contains the maximum number of distinct levels, not including *white* or *unmarked,* of any one primary color at the printer's maximum resolution. Note that the number of distinct colors achievable by combining primaries may be significantly less than the product of the number of levels in each primary.

PRTDEVCHAR Field - duplexCaps

duplexCaps (BYTE)

Duplex capabilities of the printer.

Values are as follows:

Bit	Description
0	Short-edge binding duplex-capable (0=No; 1=Yes)
1	Long-edge binding duplex-capable (0=No; 1=Yes)
2	Tumble duplex-capable (0=No; 1=Yes)
3 - 7	Reserved

PRTDEVCHAR Field - lengthUnits

lengthUnits (BYTE)

Units used by the printer when reporting lengths.

Values are as follows:

Value	Description
00	.001 inches
01	.01 inches
02	.1 inches
03	inches
04	.01 millimeters
05	.1 millimeter
06	millimeters
07	centimeters

08	1/1440 inch
09 - FD	Reserved
FE	Other
FF	Unknown

PRTDEVCHAR Field - horizUnits

horizUnits (USHORT)
Logical units per *lengthUnit* in the horizontal direction.

Horizontal direction is defined to be orthogonal (90 degrees) to the feed or motion of the printing media.

PRTDEVCHAR Field - vertUnits

vertUnits (USHORT)
Logical units per *lengthUnit* in the vertical direction.

Vertical resolution is defined to be the same direction as the feed or motion of the printing media.

PRTDEVCHAR Field - speed

speed (USHORT)
Maximum speed of the printer.

This parameter defines the maximum speed or how fast the printer can print. The value in this field is interpreted together with the value returned in the s*peedUnit*s field, which defines the units that apply to this number. As an example, the speed of a printer might be twenty-six impressions per minute.

PRTDEVCHAR Field - speedUnits

speedUnits (BYTE)
Units indicating speed capabilities of the printer.

This unsigned byte defines the units that a printer uses when reporting various printer speed capabilities. This field is used together with the value of s*peed,* which returns the printer speed.

Values for s*peedUnit*s are as follows:

Value	Description
00	Impressions or sheets per minute
01	Inches per second

02	feet per second
03	yards per second
04	millimeters per second
05	centimeters per second
06	decimeters per second
07	meters per second
08	characters per second
09	tens of characters per second
0A	hundreds of characters per second
0B	thousands of characters per second
0C	lines per minute
0D	tens of lines per minute
0E	hundreds of lines per minute
0F	thousands of lines per minute
10 - FD	Reserved
FE	Other
FF	Unknown

PRTDEVCHAR Field - counterUnits

counterUnits (BYTE)

Units of measurement used in printer statistics.

Values are as follows:

Value	Description
00	pages
01	inches
02	feet
03	yards
04	millimeters
05	centimeters
06	decimeters
07	meters
08	characters
09	tens of characters
0A	hundreds of characters
0B	thousands of characters
0C	lines

0D	tens of lines
0E	hundreds of lines
0F	thousands of lines
10 - FD	Reserved
FE	Other
FF	Unknown

PRTDEVCHAR Field - interpreters

interpreters (BYTE)
Count of interpreters in the printer.

PostScript or PCL are examples of interpreters.

PRTDEVCHAR Field - inputs

inputs (BYTE)
Count of printing media inputs currently in the printer.

PRTDEVCHAR Field - outputs

outputs (BYTE)
Count of output destinations for printing media.

PRTDEVCHAR Field - options

options (BYTE)
Count of options installed in the printer.

An option can be font or memory cards, mass storage devices, and so on.

PRTDEVCHAR Field - msgLanguage

msgLanguage (BYTE)

Language used in text messages sent from printer.

Values are as follows:

Value	Description
00	English
01	German
02	French
03	Canadian French
04	Spanish
05	Italian
06	Danish
07	Norwegian
08	Dutch
09	Japanese
0A	Korean
0B	Traditional Chinese
0C	Simplified Chinese
0D	Swedish
0E	Portugese
0F	Finnish
10 - FD	Reserved
FE	Other
FF	Unknown

PRTDEVCHAR Field - ubReserved

ubReserved (BYTE)
Reserved; must be 0 (zero).

PRTDEVCHAR Field - maxReceivePacketSize

maxReceivePacketSize (ULONG)
Maximum size, in bytes, of a data packet that can be received by the printer.

If 0, the printer may not have a maximum packet size, or data going to the printer may not be packetized.

PRTDEVCHAR Field - maxOutstandingCmds

maxOutstandingCmds (USHORT)
Number of commands that the printer can receive and be working on at one time.

A value of FFFF indicates there is no known limit.

PRTDEVCHAR Field - memory

memory (ULONG)
Total RAM installed in printer in K bytes.

PRTDEVCHAR Field - ulpszPrinterName

ulpszPrinterName (ULONG)
Model and manufacturer of printer.

The suggested format is: "Manufacturer's Name (as on the logo): Product Name: Model Number."

PRTDEVCHAR Field - ulpszRevision

ulpszRevision (ULONG)
Revision level of the printer.

PRTDEVCHAR Field - ulpszSerialNo

ulpszSerialNo (ULONG)
Serial number of the printer.

PRTFONTS

Information about fonts in the printer, returned by the BIDI_Q_FONTS structure.

```
typedef struct _PRTFONTS {
  BYTE       interpreterID;     /*  Interpreter identifier that returns this font's information. */
  BYTE       bReserved;         /*  Reserved; must be 0 (zero). */
  USHORT     usStorageTypeID;   /*  Font storage-type identifier. */
  ULONG      ulStorageType;     /*  Font storage type. */
  ULONG      ulFontID;          /*  Font identifier number. */
  ULONG      ulTechnology;      /*  Font technology. */
  ULONG      ulpszName;         /*  Offset to null-terminating font-name string. */
```

```
  ULONG      ulpszDescription;  /*  Offset to null-terminating font-description string. */
} PRTFONTS;

typedef PRTFONTS *PPRTFONTS;
```

PRTFONTS Field - interpreterID

interpreterID (BYTE)
Interpreter identifier that returns this font's information.

PRTFONTS Field - bReserved

bReserved (BYTE)
Reserved; must be 0 (zero).

PRTFONTS Field - usStorageTypeID

usStorageTypeID (USHORT)
Font storage-type identifier.

Values are as follows:

Value	Description
00	First font storage-type identifier.
FFFF	Unknown

PRTFONTS Field - ulStorageType

ulStorageType (ULONG)
Font storage type.

If the requested storage type is not available, PMERR_SPL_TYPE_NOT_AVAIL is returned.

Values are as follows:

Value	Description
00	PRTFONTS_STOR_TYPE_RESIDENT - 0x00000000
01	PRTFONTS_STOR_TYPE_REMOVABLE_CARD - 0x00000001
02	PRTFONTS_STOR_TYPE_DISK - 0x00000002
03	PRTFONTS_STOR_TYPE_RAM - 0x00000003

04	PRTFONTS_STOR_TYPE_FLASH - 0x00000004
05	PRTFONTS_STOR_TYPE_ROM - 0x00000005
06 - FFFFFFFD	Reserved
FFFFFFFE	PRTFONTS_STOR_TYPE_LIST_ABORTED - 0xFFFFFFFE Font details list aborted
FFFFFFFF	PRTFONTS_STOR_TYPE_ALL - 0xFFFFFFFF All locations

PRTFONTS Field - ulFontID

ulFontID (ULONG)
Font identifier number.

This number will depend on the hardware manufacturer. A value of hex 'FFFFFFFF' indicates that the font identifier is unknown.

PRTFONTS Field - ulTechnology

ulTechnology (ULONG)
Font technology.

Value	Description
00	PRTFONTS_TECH_OTHER - 0 Other (none of below)
01	PRTFONTS_TECH_ATM_TYPE1 - 1
02	PRTFONTS_TECH_TRUE_TYPE - 2
03	PRTFONTS_TECH_INTELLIFONT - 3
04	PRTFONTS_TECH_BITMAP - 4
05 - FFFFFFFF	Reserved

PRTFONTS Field - ulpszName

ulpszName (ULONG)
Offset to null-terminating font-name string.

PRTFONTS Field - ulpszDescription

ulpszDescription (ULONG)
Offset to null-terminating font-description string.

PRTFONTS_Q

Information about the PRTFONTS_Q data structure.

```
typedef struct _PRTFONTS_Q {
  BYTE       interpreterID;     /*  Interpreter identifier to get font information. */
  BYTE       bReserved;         /*  Reserved; must be 0 (zero). */
  USHORT     usStorageTypeID;   /*  Specific font-storage-type identifier. */
  ULONG      ulStorageType;     /*  Font storage type to return font information. */
  ULONG      ulFontID;          /*  Specific font-storage-type ID to query. */
} PRTFONTS_Q;

typedef PRTFONTS_Q *PPRTFONTS_Q;
```

PRTFONTS_Q Field - interpreterID

interpreterID (BYTE)
Interpreter identifier to get font information.

A value of hex 'FF' requests fonts from all interpreters.

PRTFONTS_Q Field - bReserved

bReserved (BYTE)
Reserved; must be 0 (zero).

PRTFONTS_Q Field - usStorageTypeID

usStorageTypeID (USHORT)
Specific font-storage-type identifier.

This field is used to get font information. A value of hex 'FFFF' requests fonts from all storage areas matching the search criteria.

PRTFONTS_Q Field - ulStorageType

ulStorageType (ULONG)
Font storage type to return font information.

If the requested storage type is not available, PMERR_SPL_TYPE_NOT_AVAIL is returned.

Values are as follows:

Value	Description
00	PRTFONTS_STOR_TYPE_RESIDENT - 0x00000000
01	PRTFONTS_STOR_TYPE_REMOVABLE_CARD - 0x00000001
02	PRTFONTS_STOR_TYPE_DISK - 0x00000002
03	PRTFONTS_STOR_TYPE_RAM - 0x00000003
04	PRTFONTS_STOR_TYPE_FLASH - 0x00000004
05	PRTFONTS_STOR_TYPE_ROM - 0x00000005
06 - FFFFFFFD	Reserved
FFFFFFFE	PRTFONTS_STOR_TYPE_LIST_ABORTED - 0xFFFFFFFE Font details list aborted
FFFFFFFF	PRTFONTS_STOR_TYPE_ALL - 0xFFFFFFFF All locations

PRTFONTS_Q Field - ulFontID

ulFontID (ULONG)
Specific font-storage-type ID to query.

A value of hex 'FFFFFFFF' requests all fonts matching the storage type.

PRTFONTSL

Structure returned by the BIDI_Q_FONTS command.

```
typedef struct _PRTFONTSL {
  ULONG        cTotal;     /*  Total fonts meeting this criteria. */
  ULONG        cReturned;  /*  Number of PRTFONTS data structures that follow. */
  PRTFONTS     prtFonts;   /*  Contains a PRTFONTS data structure. */
} PRTFONTSL;

typedef PRTFONTSL *PPRTFONTSL;
```

PRTFONTSL Field - cTotal

cTotal (ULONG)
Total fonts meeting this criteria.

PRTFONTSL Field - cReturned

cReturned (ULONG)
Number of PRTFONTS data structures that follow.

PRTFONTSL Field - prtFonts

prtFonts (PRTFONTS)
Contains a PRTFONTS data structure.

PRTINPUTS

Information about the installed printer input bins, returned by the BIDI_Q_INPUTBINS command.

```
typedef struct _PRTINPUTS {
  BYTE       logicalUnit;       /*  Logical unit number for this interpreter. */
  BYTE       bReserved;         /*  Reserved; must be 0 (zero). */
  USHORT     usReserved;        /*  Reserved; must be 0 (zero). */
  BYTE       inputID;           /*  Input source identifier. */
  BYTE       feedType;          /*  Input feed type. */
  BYTE       formChar;          /*  Form characteristic. */
  BYTE       formSize;          /*  Encoded form size. */
  ULONG      maxCapacity;       /*  Maximum input capacity. */
  USHORT     maxCrossFormSize;  /*  Maximum size of the form across feed direction. */
  USHORT     maxFeedFormSize;   /*  Maximum size of the form in the feed direction. */
  USHORT     curCrossFormSize;  /*  Current size of the form across feed direction. */
  USHORT     curFeedFormSize;   /*  Current size of the form in the feed direction. */
  USHORT     topMargin;         /*  Top unprintable area for this input. */
  USHORT     botMargin;         /*  Bottom unprintable area for this input. */
  USHORT     leftMargin;        /*  Left unprintable area for this input. */
  USHORT     rightMargin;       /*  Right unprintable area for this input. */
  ULONG      ulpszInput;        /*  Input description string. */
  ULONG      ulpszForm;         /*  Form description name. */
} PRTINPUTS;

typedef PRTINPUTS *PPRTINPUTS;
```

PRTINPUTS Field - logicalUnit

logicalUnit (BYTE)
Logical unit number for this interpreter.

A value of hex 'FF' indicates that this bin is returned from the physical printer, not an interpreter. To get information on a specific interpreter, pass the interpreter's ID in *ulInterpreterID*.

PRTINPUTS Field - bReserved

bReserved (BYTE)

Reserved; must be 0 (zero).

PRTINPUTS Field - usReserved

usReserved (USHORT)
Reserved; must be 0 (zero).

PRTINPUTS Field - inputID

inputID (BYTE)
Input source identifier.

The identification numbers for a particular input are assigned by the printer and printer manufacturer. The identification numbers for functionally equivalent input devices may be different from printer to printer, depending on the manufacturer of the printer. For example, a manual input may have an identification number of x'01' with one manufacturer and an identification number of x'03' with another manufacturer or printer model.

The printer will assign identification numbers sequentially each time the printer is initialized or turned on. As a result, if the number of input options is changed, a particular input may not have the same identification number as before the change in printer configuration.

PRTINPUTS Field - feedType

feedType (BYTE)
Input feed type.

Values are as follows:

Value	Description
00	Reserved
01	Sheet-feed (auto)
02	Sheet-feed (manual)
03	Continuous roll
04	Continuous fanfold
05 - FD	Reserved
FE	Other
FF	Unknown

PRTINPUTS Field - formChar

formChar (BYTE)
Form characteristic.

Values are as follows:

Value	Description
00	Reserved
01	Paper
02	Envelope
03	Transparency
04	Labels
05 - FD	Reserved
FE	Other
FF	Unknown

PRTINPUTS Field - formSize

formSize (BYTE)
Encoded form size.

Values are as follows:

Bit	Description
00	PRT_FORMSIZE_RESERVED - 0x00
01	PRT_FORMSIZE_A - 0x01 A; 8-1/2 x 11 (US Sizes inches)
02	PRT_FORMSIZE_B - 0x02 B; 11 x 17
03	PRT_FORMSIZE_C - 0x03 C; 17 x 22
04	PRT_FORMSIZE_D - 0x04 D; 22 x 34
05	PRT_FORMSIZE_E - 0x05 E; 34 x 44
06	PRT_FORMSIZE_F - 0x06 F; 44 x 68
07	PRT_FORMSIZE_LEGAL - 0x07 Legal; 8-1/2 x 14
08	PRT_FORMSIZE_1_2LETTER - 0x08 1/2 Letter; 8-1/2 x 5-1/2
09	PRT_FORMSIZE_EXECUTIVE - 0x09 Executive; 7-1/4 x 10-1/2
0A - 0F	Reserved
10	PRT_FORMSIZE_4A0 - 0x10 4A0; 1682 x 2378 (ISO A Series mm)
11	PRT_FORMSIZE_2A0 - 0x11 2A0; 1189 x 1682

12	PRT_FORMSIZE_A0 - 0x12 A0; 841 x 1189
13	PRT_FORMSIZE_A1 - 0x13 A1; 594 x 841
14	PRT_FORMSIZE_A2 - 0x14 A2; 420 x 594
15	PRT_FORMSIZE_A3 - 0x15 A3; 297 x 420
16	PRT_FORMSIZE_A4 - 0x16 A4; 210 x 297
17	PRT_FORMSIZE_A5 - 0x17 A5; 148 x 210
18	PRT_FORMSIZE_A6 - 0x18 A6; 105 x 148
19	PRT_FORMSIZE_A7 - 0x19 A7; 74 x 105
1A	PRT_FORMSIZE_A8 - 0x1A A8; 52 x 74
1B	PRT_FORMSIZE_A9 - 0x1B A9; 37 x 52
1C	PRT_FORMSIZE_A19 - 0x1C A10; 26 x 37
1D - 1F	Reserved
20	PRT_FORMSIZE_RA0 - 0x20 RA0; 860 x 1220
21	PRT_FORMSIZE_RA1 - 0x21 RA1; 610 x 860
22	PRT_FORMSIZE_RA2 - 0x22 RA2; 430 x 610
23	PRT_FORMSIZE_SRA0 - 0x23 SRA0; 900 x 1280
24	PRT_FORMSIZE_SRA1 - 0x24 SRA1; 640 x 900
25	PRT_FORMSIZE_SRA2 - 0x25 SRA2; 450 x 640
26 - 2F	Reserved
30	PRT_FORMSIZE_B0 - 0x30 B0; 1000 x 1414 (ISO B Series mm)
31	PRT_FORMSIZE_B1 - 0x31 B1; 707 x 1000
32	PRT_FORMSIZE_B2 - 0x32 B2; 500 x 707
33	PRT_FORMSIZE_B3 - 0x33 B3; 353 x 500
34	PRT_FORMSIZE_B4 - 0x34 B4; 250 x 353
35	PRT_FORMSIZE_B5 - 0x35 B5; 176 x 250
36	PRT_FORMSIZE_B6 - 0x36 B6; 125 x 176

37	PRT_FORMSIZE_B7 - 0x37 B7; 88 x 125
38	PRT_FORMSIZE_B8 - 0x38 B8; 62 x 88
39	PRT_FORMSIZE_B9 - 0x39 B9; 44 x 62
3A	PRT_FORMSIZE_B10 - 0x3A B10; 31 x 44
3B - 3F	Reserved
40	PRT_FORMSIZE_C0 - 0x40 C0; 917 x 1297 (ISO C Series mm)
41	PRT_FORMSIZE_C1 - 0x41 C1; 648 x 917
42	PRT_FORMSIZE_C2 - 0x42 C2; 458 x 648
43	PRT_FORMSIZE_C3 - 0x43 C3; 324 x 458
44	PRT_FORMSIZE_C4 - 0x44 C4; 229 x 324
45	PRT_FORMSIZE_C5 - 0x45 C5; 162 x 229
46	PRT_FORMSIZE_C6 - 0x46 C6; 114 x 162
47	PRT_FORMSIZE_C7 - 0x47 C7; 81 x 114
48	PRT_FORMSIZE_C8 - 0x48 C8; 57 x 81
49-4F	Reserved
50	PRT_FORMSIZE_ENVELOPEC3 - 0x50 Envelope C3; 324 x 458 (ISO Envelopes mm)
51	PRT_FORMSIZE_ENVELOPEB4 - 0x51 Envelope B4; 250 x 353
52	PRT_FORMSIZE_ENVELOPEC4 - 0x52 Envelope C4; 229 x 324
53	PRT_FORMSIZE_ENVELOPEB5 - 0x53 Envelope B5; 176 x 250
54	PRT_FORMSIZE_ENVELOPEC5 - 0x54 Envelope C5; 162 x 229
55	PRT_FORMSIZE_ENVELOPEB6_C4 - 0x55 Envelope B6/C4; 125 x 324
56	PRT_FORMSIZE_ENVELOPEB6 - 0x56 Envelope B6; 125 x 324
57	PRT_FORMSIZE_ENVELOPEC6 - 0x57 Envelope C6; 114 x 162
58	PRT_FORMSIZE_ENVELOPEDL - 0x58 Envelope DL; 110 x 220
59	PRT_FORMSIZE_ENVELOPEC7_6 - 0x59 Envelope C7/6; 81 x 162
5A	PRT_FORMSIZE_ENVELOPEC7 - 0x5A Envelope C7; 81 x 114

5B-5F	Reserved
60	PRT_FORMSIZE_POSTCARD - 0x60 Postcard; 100 x 148 (Japanese mm)
61	PRT_FORMSIZE_DOUBLEPOSTCARD - 0x61 Double Postcard; 200 x 148
62	PRT_FORMSIZE_ENVELOPE_LONG3 - 0x62 Envelope Long3; 120 x 235
63	PRT_FORMSIZE_ENVELOPE_LONG4 - 0x63 Envelope Long4; 90 x 205
64-6F	Reserved
70	PRT_FORMSIZE_B0_J - 0x70 B0; 1030 x 1456 (JIS B Series mm)
71	PRT_FORMSIZE_B1_J - 0x71 B1; 728 x 1030
72	PRT_FORMSIZE_B2_J - 0x72 B2; 515 x 728
73	PRT_FORMSIZE_B3_J - 0x73 B3; 364 x 515
74	PRT_FORMSIZE_B4_J - 0x74 B4; 257 x 364
75	PRT_FORMSIZE_B5_J - 0x75 B5; 182 x 257
76	PRT_FORMSIZE_B6_J - 0x76 B6; 128 x 182
77	PRT_FORMSIZE_B7_J - 0x77 B7; 91 x 128
78	PRT_FORMSIZE_B8_J - 0x78 B8; 64 x 91
79	PRT_FORMSIZE_B9_J - 0x79 B9; 45 x 64
7A	PRT_FORMSIZE_B10_J - 0x7A B10; 32 x 45
7B - 7F	Reserved
80	PRT_FORMSIZE_ENVELOPE7_3_4 - 0x80 Envelope 7-3/4; 3 7/8 x 7 1/2 (Envelopes inches)
81	PRT_FORMSIZE_ENVELOPE9 - 0x81 Envelope 9; 3 7/8 x 8 7/8
82	PRT_FORMSIZE_ENVELOPE10 - 0x82 Envelope 10; 4 1/4 x 9 1/2
83 - FD	Reserved
FE	PRT_FORMSIZE_OTHER - 0xFE Other
FF	PRT_FORMSIZE_UNKNOWN - 0xFF Unknown

PRTINPUTS Field - maxCapacity

maxCapacity (ULONG)
Maximum input capacity.

The units for this value are specified by *counterUnit*s in the PRTDEVCHAR data structure. This value does not indicate the current amount of input printing media present. The value of hex 'FFFFFFFF' indicates that the printer is unable to determine the value.

PRTINPUTS Field - maxCrossFormSize

maxCrossFormSize (USHORT)
Maximum size of the form across feed direction.

This value specifies the maximum size of the media, in the direction of ninety degrees to the feed or motion direction, that can be input from a particular input feature. The units for this value are specified by the *lengthUnit*s field in the PRTDEVCHAR data structure. If the size is unknown, the printer returns a value of hex 'FFFF.'

PRTINPUTS Field - maxFeedFormSize

maxFeedFormSize (USHORT)
Maximum size of the form in the feed direction.

This value specifies the maximum size of the media in the feed or motion direction that can be input from a particular input feature. The units for this value are specified in *lengthUnit*s in the PRTDEVCHAR data structure. If the size is unknown, the printer returns a value of hex 'FFFF.'

PRTINPUTS Field - curCrossFormSize

curCrossFormSize (USHORT)
Current size of the form across feed direction.

This value specifies the current size of the media, in the direction of ninety degrees to the feed or motion direction, that can be input from a particular input feature. The units for this measurement are defined by the *lengthUnit*s parameter in the PRTDEVCHAR data structure. If the size is unknown, the printer returns a value of hex 'FFFF.'

PRTINPUTS Field - curFeedFormSize

curFeedFormSize (USHORT)
Current size of the form in the feed direction.

This value specifies the current size of the media in the feed or motion direction that can be input from a particular input feature. The units for this value are specified by *lengthUnit*s in the PRTDEVCHAR data structure. If the size is unknown, the printer returns a value of hex 'FFFF.'

PRTINPUTS Field - topMargin

topMargin (USHORT)
Top unprintable area for this input.

The value in this field defines the top unprintable area for this input. The top margin is defined as that area above the first printable line written in the marking direction. The units for this value are specified by *lengthUnits* in the PRTDEVCHAR data structure. A value of hex 'FFFF' indicates that the printer does not have, nor can it determine, this value.

Note: This value specifies the limitation of the printing mechanism and also indicates that particular interpreters may place additional limitations on the margins.

PRTINPUTS Field - botMargin

botMargin (USHORT)
Bottom unprintable area for this input.

The value in this field defines the bottom unprintable area for this input. The bottom margin is defined as that area below the last printable line written in the marking direction. The units for this value are specified by *lengthUnits* in the PRTDEVCHAR data structure. A value of hex 'FFFF' indicates that the printer does not have, nor can it determine, this value.

Note: This value specifies the limitation of the printing mechanism and also indicates that particular page interpreters may place additional limitations on the margins.

PRTINPUTS Field - leftMargin

leftMargin (USHORT)
Left unprintable area for this input.

The value in this field defines the left unprintable area for this input. The left margin is defined as that area to the left of the first printable character in a line written in the marking direction. The units for this value are specified by *lengthUnits* in the PRTDEVCHAR data structure. A value of hex 'FFFF' indicates that the printer does not have, nor can it determine, this value.

Note: This value specifies the limitation of the printing mechanism and also indicates that particular interpreters may place additional limitations on the margins.

PRTINPUTS Field - rightMargin

rightMargin (USHORT)
Right unprintable area for this input.

The value in this field defines the right unprintable area for this input. The right margin is defined as that area to the right of the last printable character in a line written in the marking direction. The units for this value are specified by *lengthUnits* in the PRTDEVCHAR data structure. A value of hex 'FFFF' indicates that the printer does not have, nor can it determine, this value.

Note: This value specifies the limitation of the printing mechanism and also indicates that particular interpreters may place additional limitations on the margins.

PRTINPUTS Field - ulpszInput

ulpszInput (ULONG)
Input description string.

For example, "Automatic Envelope Feeder."

PRTINPUTS Field - ulpszForm

ulpszForm (ULONG)
Form description name.

For example, "Executive."

PRTINPUTSL

Structure returned by the BIDI_Q_INPUTBINS command.

```
typedef struct _PRTINPUTSL {
  ULONG         cTotal;                         /*  Total inputs in the printer. */
  ULONG         cReturned;                      /*  Number of PRTINPUTS data structures that follow. */
  PRTINPUTS     prtInputs[1];  /*  Contains a PRTINPUTS data structure. */
} PRTINPUTSL;

typedef PRTINPUTSL *PPRTINPUTSL;
```

PRTINPUTSL Field - cTotal

cTotal (ULONG)
Total inputs in the printer.

PRTINPUTSL Field - cReturned

cReturned (ULONG)
Number of PRTINPUTS data structures that follow.

PRTINPUTSL Field - prtInputs[1]

prtInputs[1] (PRTINPUTS)
Contains a PRTINPUTS data structure.

PRTINTERP

Information about the PRTINTERP data structure.

```
typedef struct _PRTINTERP {
  BYTE       logicalUnit;       /*  Logical unit number for this interpreter. */
  BYTE       bReserved;         /*  Reserved; must be 0 (zero). */
  USHORT     usReserved;        /*  Reserved; must be 0 (zero). */
  ULONG      features;          /*  Interpreter features summary. */
  ULONG      memory;            /*  Amount of memory in K bytes. */
  USHORT     fontcount;         /*  Number of fonts available to interpreter. */
  BYTE       inputs;            /*  Number of input trays supported by this interpreter. */
  BYTE       outputs;           /*  Number of output bins supported by this interpreter. */
  USHORT     horizRes;          /*  Maximum horizontal marking resolution supported by this interpreter. */
  USHORT     vertRes;           /*  Maximum vertical marking resolution supported by this interpreter. */
  ULONG      ulpszInterpreter;  /*  Interpreter name. */
} PRTINTERP;

typedef PRTINTERP *PPRTINTERP;
```

PRTINTERP Field - logicalUnit

logicalUnit (BYTE)
Logical unit number for this interpreter.

PRTINTERP Field - bReserved

bReserved (BYTE)
Reserved; must be 0 (zero).

PRTINTERP Field - usReserved

usReserved (USHORT)
Reserved; must be 0 (zero).

PRTINTERP Field - features

features (ULONG)
Interpreter features summary.

Values are as follows:

Bit	Description
0	If clear, the *feature*s parameter is not supported by the printer. The remaining bits are valid only if bit 0 is set.
1	Interpreter save state on switching (0=No; 1=Yes).
2	Interpreter can run simultaneously with others (0=No; 1=Yes).
3	Interpreter can report resident font information (0=No; 1=Yes).
4	Interpreter can report font card information (0=No; 1=Yes).
5	Interpreter can report download font information (0=No; 1=Yes).
6	Interpreter can report disk font information (0=No; 1=Yes).
7	Interpreter supports soft resource fault handling (0=No; 1=Yes).
8 - 31	Reserved; must be clear.

PRTINTERP Field - memory

memory (ULONG)
Amount of memory in K bytes.

This parameter contains the maximum amount of free memory available to this interpreter at this point in time. A value of hex 'FFFFFFFF' indicates that the printer is not able to supply this information.

PRTINTERP Field - fontcount

fontcount (USHORT)
Number of fonts available to interpreter.

A value of hex 'FFFF' indicates that the printer is not able to supply this information.

PRTINTERP Field - inputs

inputs (BYTE)
Number of input trays supported by this interpreter.

A value of hex 'FF' indicates that the printer is not able to supply this information.

PRTINTERP Field - outputs

outputs (BYTE)
Number of output bins supported by this interpreter.

A value of hex 'FF' indicates that the printer is not able to supply this information.

PRTINTERP Field - horizRes

horizRes (USHORT)
Maximum horizontal marking resolution supported by this interpreter.

The units for this measurement are defined by the *lengthUnits* parameter in the PRTDEVCHAR data structure.

PRTINTERP Field - vertRes

vertRes (USHORT)
Maximum vertical marking resolution supported by this interpreter.

The units for this measurement are defined by the *lengthUnits* parameter in the PRTDEVCHAR data structure.

PRTINTERP Field - ulpszInterpreter

ulpszInterpreter (ULONG)
Interpreter name.

"PostScript" is an example of an interpreter name.

PRTINTERPL

Structure returned by the BIDI_Q_INTERPRETER command to list interpreters in the printer.

```
typedef struct _PRTINTERPL {
  ULONG         cTotal;                          /*  Total interpreters in the printer. */
  ULONG         cReturned;                       /*  Number of PRTINTERP data structures that follow. */
  PRTINTERP     prtInterpreter[1];  /*  Contains a PRTINTERP data structure. */
} PRTINTERPL;

typedef PRTINTERPL *PPRTINTERPL;
```

PRTINTERPL Field - cTotal

cTotal (ULONG)
Total interpreters in the printer.

PRTINTERPL Field - cReturned

cReturned (ULONG)
Number of PRTINTERP data structures that follow.

PRTINTERPL Field - prtInterpreter[1]

prtInterpreter[1] (PRTINTERP)
Contains a PRTINTERP data structure.

PRTJOB

Job structure used with the BIDI_Q_JOBID and BIDI_ENDJOB commands.

```
typedef struct _PRTJOB {
  ULONG     interpreterID;   /*  Identifier of the interpreter that is processing this job. */
  ULONG     ulPrinterJobID;  /*  Printer job identifier. */
} PRTJOB;

typedef PRTJOB *PPRTJOB;
```

PRTJOB Field - interpreterID

interpreterID (ULONG)
Identifier of the interpreter that is processing this job.

PRTJOB Field - ulPrinterJobID

ulPrinterJobID (ULONG)
Printer job identifier.

PRTJOBS

Information about jobs in the printer returned by the BIDI_Q_JOBS_COMPLETE and BIDI_Q_JOBS_QUEUED commands.

```
typedef struct _PRTJOBS {
  BYTE       interpreterID;    /*  Interpreter identifier that returns this job's information. */
  BYTE       bReserved;        /*  Reserved; must be 0 (zero). */
  USHORT     fsStatus;         /*  Status of this job. */
  ULONG      ulPrinterJobID;   /*  Interpreter job identifier. */
  ULONG      ulProcessTime;    /*  Number of seconds it took to process the job. */
  ULONG      ulSheets;         /*  Total pieces of print media used. */
  ULONG      ulImpressions;    /*  Total sides printed. */
} PRTJOBS;

typedef PRTJOBS *PPRTJOBS;
```

PRTJOBS Field - interpreterID

interpreterID (BYTE)
Interpreter identifier that returns this job's information.

PRTJOBS Field - bReserved

bReserved (BYTE)
Reserved; must be 0 (zero).

PRTJOBS Field - fsStatus

fsStatus (USHORT)
Status of this job.

Values are as follows:

Bit	Description
0	PRTJOBS_STATUS_COMPLETE - 0x00001 Completed (0=No; 1=Yes)
1	PRTJOBS_STATUS_PRINTING - 0x000002 Printing (0=No; 1=Yes)
2	PRTJOBS_STATUS_PAUSED - 0x000004 Paused (0=No; 1=Yes)
3 - 15	Reserved

PRTJOBS Field - ulPrinterJobID

ulPrinterJobID (ULONG)
Interpreter job identifier.

PRTJOBS Field - ulProcessTime

ulProcessTime (ULONG)
Number of seconds it took to process the job.

This should not include any operator intervention time. A value of hex 'FFFF' indicates that the interpreter is unable to return this information.

PRTJOBS Field - ulSheets

ulSheets (ULONG)
Total pieces of print media used.

A value of hex 'FFFFFFFF' indicates that the interpreter is unable to return this information.

PRTJOBS Field - ulImpressions

ulImpressions (ULONG)
Total sides printed.

Duplex would have two impressions per sheet of paper. A value of hex 'FFFFFFFF' indicates that the interpreter is unable to return this information.

PRTJOBS_Q

Information passed to the BIDI_Q_JOBS_COMPLETE and BIDI_Q_JOBS_QUEUED commands to limit the information requested.

```
typedef struct _PRTJOBS_Q {
  ULONG     interpreterID;   /*  Interpreter identifier to get job information. */
  ULONG     ulPrinterJobID;  /*  Specific printer job to query. */
  ULONG     ulMaxJobs;       /*  Maximum jobs to return. */
} PRTJOBS_Q;

typedef PRTJOBS_Q *PPRTJOBS_Q;
```

PRTJOBS_Q Field - interpreterID

interpreterID (ULONG)

Interpreter identifier to get job information.

A value of hex 'FFFFFFFF' requests jobs from all interpreters.

PRTJOBS_Q Field - ulPrinterJobID

ulPrinterJobID (ULONG)
Specific printer job to query.

A value of hex 'FFFFFFFF' requests all jobs for the requested interpreter(s).

PRTJOBS_Q Field - ulMaxJobs

ulMaxJobs (ULONG)
Maximum jobs to return.

A value of hex 'FFFFFFFF' requests all known jobs.

PRTJOBSL

Information returned by the BIDI_Q_JOBS_COMPLETE and BIDI_Q_JOBS_QUEUED commands.

```
typedef struct _PRTJOBSL {
  ULONG       cTotal;     /*  Total jobs meeting criteria. */
  ULONG       cReturned;  /*  Number of PRTJOBS data structures that follow. */
  PRTJOBS     prtJobs;    /*  Contains a PRTJOBS data structure. */
} PRTJOBSL;

typedef PRTJOBSL *PPRTJOBSL;
```

PRTJOBSL Field - cTotal

cTotal (ULONG)
Total jobs meeting criteria.

PRTJOBSL Field - cReturned

cReturned (ULONG)
Number of PRTJOBS data structures that follow.

PRTJOBSL Field - prtJobs

prtJobs (PRTJOBS)
Contains a PRTJOBS data structure.

PRTOUTPUTS

Information about the installed printr output bins, returned by the BIDI_Q_OUTPUTBINS command.

```
typedef struct _PRTOUTPUTS {
  BYTE       logicalUnit;  /*  Logical unit number for this interpreter. */
  BYTE       bReserved;    /*  Reserved; must be 0 (zero). */
  USHORT     usReserved;   /*  Reserved; must be 0 (zero). */
  BYTE       outputID;     /*  Output identifier. */
  BYTE       positions;    /*  Number of output positions for this output device. */
  BYTE       features1;    /*  Output features support byte 1. */
  BYTE       features2;    /*  Output features support byte 2. */
  ULONG      maxCapacity;  /*  Maximum capacity per output position. */
  ULONG      ulpszOutput;  /*  Output description string. */
} PRTOUTPUTS;

typedef PRTOUTPUTS *PPRTOUTPUTS;
```

PRTOUTPUTS Field - logicalUnit

logicalUnit (BYTE)
Logical unit number for this interpreter.

A value of hex 'FF' indicates that this bin is returned from the physical printer, not an interpreter. To get information on a specific interpreter, pass the interpreter's ID in *ulInterpreterID*.

PRTOUTPUTS Field - bReserved

bReserved (BYTE)
Reserved; must be 0 (zero).

PRTOUTPUTS Field - usReserved

usReserved (USHORT)
Reserved; must be 0 (zero).

PRTOUTPUTS Field - outputID

outputID (BYTE)
Output identifier.

The identification numbers for a particular output are assigned by the printer and printer manufacturer. The identification numbers for functionally equivalent output devices may be different from printer to printer, depending on the manufacturer of the printer. For example, an output collator may have an identification number of x'01' with one manufacturer and an identification number of x'04' with another manufacturer or printer model.

The printer will assign identification numbers sequentially each time the printer is initialized or turned on. As a result, if the number of input options is changed, a particular output may not have the same identification number as before the change in printer configuration.

PRTOUTPUTS Field - positions

positions (BYTE)
Number of output positions for this output device.

As an example, a ten-bin collator is a multiple-position output device that would have a value of x'0A' in this field.

PRTOUTPUTS Field - features1

features1 (BYTE)
Output features support byte 1.

Values are as follows:

Bit	Description
0	Face up (0=No; 1=Yes)
1	Offset capability (0=No; 1=Yes)
2	Security available (0=No; 1=Yes)
3	Bursting available (0=No; 1=Yes)
4	Collation available (0=No; 1=Yes)
5	Face down (0=No; 1=Yes)
6	Reserved
7	Reserved

PRTOUTPUTS Field - features2

features2 (BYTE)

Output features support byte 2.

Values are as follows:

Bit	Description
0	Stapling available (0=No; 1=Yes)
1	Binding available (0=No; 1=Yes)
2	Punching available (0=No; 1=Yes)
3 - 7	Reserved

PRTOUTPUTS Field - maxCapacity

maxCapacity (ULONG)
Maximum capacity per output position.

The units for this value are specified by *counterUnits* in the PRTDEVCHAR data structure. This value does not indicate the current amount of printing media in the output. The value of hex 'FFFFFFFF' indicates that the printer is unable to determine the value.

PRTOUTPUTS Field - ulpszOutput

ulpszOutput (ULONG)
Output description string.

PRTOUTPUTSL

Information about the PRTOUTPUTSL data structure.

```
typedef struct _PRTOUTPUTSL {
  ULONG          cTotal;                          /*  Total output features in the printer. */
  ULONG          cReturned;                       /*  Number of PRTOUTPUTS data structures that follow. */
  PRTOUTPUTS     prtOutputs[1];  /*  Contains a PRTOUTPUTS data structure. */
} PRTOUTPUTSL;

typedef PRTOUTPUTSL *PPRTOUTPUTSL;
```

PRTOUTPUTSL Field - cTotal

cTotal (ULONG)
Total output features in the printer.

PRTOUTPUTSL Field - cReturned

cReturned (ULONG)
Number of PRTOUTPUTS data structures that follow.

PRTOUTPUTSL Field - prtOutputs[1]

prtOutputs[1] (PRTOUTPUTS)
Contains a PRTOUTPUTS data structure.

PRTPORT

Information about BIDI capabilities of the port, returned by the BIDI_Q_PORT command.

If the *ulpszProtocol* parameter in the PRTPORT data structure is returned, it will be used as the protocol converter for the given printer. If *ulpszProtocol* is null and *ulBidiType* bit 0 is clear, the spooler will attempt to fill in the protocol based on the *ulpszDeviceID* returned by the port driver.

```
typedef struct _PRTPORT {
  ULONG     flBidiCapabilities;  /*  Defines printer capabilities of the port. */
  ULONG     flBidiProtocol;      /*  BIDI software protocol types supported. */
  ULONG     ulPortType;          /*  Medium used to communicate with the printer. */
  ULONG     ulpszProtocol;       /*  Offset to the name of the default BIDI protocol to be used for this printer
  ULONG     ulpszDeviceID;       /*  Device identifier returned by the printer hardware, as defined by the P1284
  ULONG     ulBidiLevel;         /*  Version level of the protocol type being used. */
  ULONG     flPrinterStatus;     /*  Current status of the attached printer. */
  ULONG     ulAdapterLevel;      /*  Level of protocol used by the adapter. */
  ULONG     ulMaxSendSize;       /*  Port driver's maximum send packet size for messages or data to the printer.
  ULONG     ulMaxReceiveSize;    /*  Port driver's maximum receive packet size for messages from the printer. */
  ULONG     ulMaxHeldResponses;  /*  Maximum number of printer messages held, waiting for BIDI_WAIT_ALERT to rea
} PRTPORT;

typedef PRTPORT *PPRTPORT;
```

PRTPORT Field - flBidiCapabilities

flBidiCapabilities (ULONG)
Defines printer capabilities of the port.

Bit	Description
0	PRTPORT_CAPS_BIDI_CAPABLE - 0x00000001 Set if the port is capable of bidirectional communications. This flag is clear if the output port will never be able to communicate using a BIDI protocol. The printer currently attached does not have to be BIDI-capable for this bit to be set.
1	PRTPORT_CAPS_BIDI_ACTIVE - 0x00000002 Set if the printer is able to communicate using a known BIDI protocol. This flag is set only if the printer is currently connected and communicating bidirectionally.
2	PRTPORT_CAPS_DEVICEID_LATER - 0x00000004 Set if the device ID is not yet known, but may be known using a protocol converter. This flag assumes the port is configured to a particular protocol converter, and the model of printer is not available until

	the protocol converter is used. After the printer model is known, the protocol converter will call PrtSet with the BIDI_SET_DEVICE_ID command to tell the spooler the printer's device ID.
3	PRTPORT_CAPS_Q_ALL_PORTS - 0x00000008 Unused.
4	PRTPORT_CAPS_NOT_RESPONDING - 0x00000010 Set if unable to find printer. The printer may be powered off, or there may be a problem communicating with the printer over the LAN.
5	PRTPORT_CAPS_NOT_CONFIGURED - 0x00000020 Set if port not configured. The port is defined, but not connected to a LAN address. The port cannot be used for printing until it is configured.
6	PRTPORT_CAPS_NOT_CONNECTED - 0x00000040 Set if unable to make connection. The printer can be found, but is currently being used by another host, which prevents this host from getting a connection to the printer.
7	PRTPORT_CAPS_NO_ASYNCH_STATUS - 0x00000080 Set if asynchronous status is disabled when no jobs are being sent. The printer will return job alerts, but after all jobs sent by this host have completed, the printer will no longer return alerts (cover open, out of paper, and so on). This is an example of a job-based connection to a printer.
8 - 11	PRTPORT_CAPS_PDCNV_MASK - 0x00000F00 Reserved for special use by port driver/protocol converter interface.
12 - 31	Reserved; must be clear.

PRTPORT Field - flBidiProtocol

flBidiProtocol (ULONG)
BIDI software protocol types supported.

A printer may support more than one software protocol. The default protocol is *ulpszProtocol*.

Bit	Description
0	PRTPORT_TYPE_NO_CNV - 0x00000001 Port driver will handle protocol (0=No; 1=Yes)
1	PRTPORT_TYPE_NPAP - 0x00000002 NPAP supported (0=No; 1=Yes)
2	PRTPORT_TYPE_PJL - 0x00000004 PJL supported (0=No; 1=Yes)
3	PRTPORT_TYPE_SNMP - 0x00000008 SNMP supported (0=No; 1=Yes)
4	PRTPORT_TYPE_RESERVED - 0x00000010 Reserved for use by port driver/protocol converter interface
5 - 31	Reserved; must be clear

PRTPORT Field - ulPortType

ulPortType (ULONG)
Medium used to communicate with the printer.

Bit	Description

00	Reserved
01	PRTPORT_PORT_TYPE_LPT - 1
02	PRTPORT_PORT_TYPE_COM - 2
03	PRTPORT_PORT_TYPE_NETWORK - 3
04	PRTPORT_PORT_TYPE_SCSI - 4
05	PRTPORT_PORT_TYPE_INFRARED - 5
06 - FFFFFFFD	Reserved
FFFFFFFE	Other port type
FFFFFFFF	Unknown.

PRTPORT Field - ulpszProtocol

ulpszProtocol (ULONG)
Offset to the name of the default BIDI protocol to be used for this printer.

This field, like all offsets in these BIDI command structures, is relative to the beginning of the buffer; 0 (zero) indicates no string.

This field overrides any default protocol converter selection. For example, a printer may support PJL, but it can be best serviced by a protocol converter other than the PJL converter shipped with OS/2. The value pointed to by this field must match the name of an installed protocol converter in order for the port driver to be able to use a protocol converter. For example, for NPAP, the value would be 'NPAP;' for PJL, the value would be 'PJL.'

PRTPORT Field - ulpszDeviceID

ulpszDeviceID (ULONG)
Device identifier returned by the printer hardware, as defined by the P1284 specification.

This value may be used to determine the OS/2 printer driver to use for this printer.

PRTPORT Field - ulBidiLevel

ulBidiLevel (ULONG)
Version level of the protocol type being used.

PRTPORT Field - flPrinterStatus

flPrinterStatus (ULONG)
Current status of the attached printer.

Values are as follows:

Bit	Description
0	PRTPORT_STAT_OFFLINE - 0x00000001 Printer offline (0=No; 1=Yes)
1	PRTPORT_STAT_BUSY - 0x00000002 Printer busy (0=No; 1=Yes)
2 - 31	Reserved

PRTPORT Field - ulAdapterLevel

ulAdapterLevel (ULONG)
Level of protocol used by the adapter.

PRTPORT Field - ulMaxSendSize

ulMaxSendSize (ULONG)
Port driver's maximum send packet size for messages or data to the printer.

PRTPORT Field - ulMaxReceiveSize

ulMaxReceiveSize (ULONG)
Port driver's maximum receive packet size for messages from the printer.

PRTPORT Field - ulMaxHeldResponses

ulMaxHeldResponses (ULONG)
Maximum number of printer messages held, waiting for BIDI_WAIT_ALERT to read them.

PRTRESPONSE

Information about the printer response data format. This structure is returned by the BIDI_Q_RESPONSE_FMT command.

```
typedef struct _PRTRESPONSE {
  ULONG      flResponseType;     /*  Flag bits that define response values. */
  BYTE       bFirstChar;         /*  The beginning byte of printer-to-host messages, if bit 0 of flResponseType i
  BYTE       bLastChar;          /*  The terminator byte of printer-to-host messages, if bit 1 of flResponseType
  USHORT     usLenFieldOffset;   /*  Offset from the beginning of printer-to-host messages, if bit 2 of flRespons
```

```
  USHORT     usLenFieldWidth;    /*  Number of bytes in the length field of the printer-to-host messages, if bit
set. */
  USHORT     usReserved;         /*  Reserved; must be 0 (zero). */
} PRTRESPONSE;

typedef PRTRESPONSE *PPRTRESPONSE;
```

PRTRESPONSE Field - flResponseType

flResponseType (ULONG)
Flag bits that define response values.

Values are as follows:

Bit	Description
0	PRTRESPONSE_TYPE_FIRSTCHAR - 0x00000001 First byte character (0=No; 1=Yes). If set, *bFirstChar* defines the first character that is always the beginning of printer-to-host messages.
1	PRTRESPONSE_TYPE_LASTCHAR - 0x00000002 Last byte character (0=No; 1=Yes). If set, *bLastChar* defines the last character that is always the terminator of printer-to-host messages.
2	PRTRESPONSE_TYPE_LEN_USED - 0x00000004 Length field (0=No; 1=Yes). If set, a length field is always part of the printer-to-host message. The *ulLenFieldOffset* and *ulLenFieldWidth* fields define where the length field is stored in the message.
3	PRTRESPONSE_TYPE_LEN_PREFIX - 0x00000008 Length field that contains prefix byte (0=No; 1=Yes). If set, the length field in the printer-to-host message includes the first byte character, if any. This is applicable only if bit 2 is set. NPAP clears this bit.
4	PRTRESPONSE_TYPE_LEN_WIDTH - 0x00000010 Length field that contains length width (0=No; 1=Yes). If set, the length field in the printer-to-host message includes the width of the length field. This is applicable only if bit 2 is set. NPAP clears this bit to indicate the length field in the printer-to-host messages does not contain "Start of Packet(1 byte)" or the length field (2 bytes) itself.
5 - 31	Reserved; must be clear.

PRTRESPONSE Field - bFirstChar

bFirstChar (BYTE)
The beginning byte of printer-to-host messages, if bit 0 of *flResponseType* is set.

For NPAP, this would be 'A5.'

PRTRESPONSE Field - bLastChar

bLastChar (BYTE)
The terminator byte of printer-to-host messages, if bit 1 of *flResponseType* is set.

For PJL, this would be the FormFeed character '0C.'

PRTRESPONSE Field - usLenFieldOffset

usLenFieldOffset (USHORT)
Offset from the beginning of printer-to-host messages, if bit 2 of *flResponseType* is set.

This value is the length field for the message. For NPAP, this would be 1 because Printer Response packets have their packet length immediately after the Start of Packet Byte 'A5.'

PRTRESPONSE Field - usLenFieldWidth

usLenFieldWidth (USHORT)
Number of bytes in the length field of the printer-to-host messages, if bit 2 of *flResponseType* is set.

PRTRESPONSE Field - usReserved

usReserved (USHORT)
Reserved; must be 0 (zero).

PRTSETALERT

Structure used by the BIDI_DISABLE_ALERT command to disable printer alerts. This structure is also used by the BIDI_ENABLE_ALERT command to enable printer alerts.

```
typedef struct _PRTSETALERT {
  ULONG     ulCategory;  /*  Category of alert. */
  ULONG     ulType;      /*  Type of alert. */
} PRTSETALERT;

typedef PRTSETALERT *PPRTSETALERT;
```

PRTSETALERT Field - ulCategory

ulCategory (ULONG)
Category of alert.

This can be an extended alert category or the core alert category.

Values are as follows:

Value	Description
0x00000000	PRTALERT_CATEGORY_ALL - 0x00000000 **Disable** all alerts (core and extended) for this printer.
0x00000001	PRTALERT_CATEGORY_CORE - 0x00000001 Core alert category for alerts to **enable** or **disable.** The field *ulType* is the bitmask of the core alerts to **disable.**
0x00000002	PRTALERT_CATEGORY_ALL_CORE - 0x00000002 **Disable** all core alerts for this printer.
0x00000003	PRTALERT_CATEGORY_ALL_EXT - 0x00000003 **Disable** all extended alerts for this printer.
0x0000D000	PRTALERT_CATEGORY_EXTENDED - 0x0000D000 Beginning of the extended alert categories for **enabling** or **disabling.** If **disabling** an alert, this disables a specific extended alert category and type. Protocol converters should limit the category range to beween 0x0000D000 and 0x0000DFFF. When **enabling** an alert, if the protocol converter does not support the given category, ERROR_INVALID_CATEGORY(117) should be returned.
0x0000DFFF	PRTALERT_CATEGORY_EXT_MAX - 0x0000DFFF End of the extended alert categories.

PRTSETALERT Field - ulType

ulType (ULONG)
Type of alert.

If the category is PRTALERT_CATEGORY_CORE, this is a bit-field that defines the additional core alerts to **enable** via BIDI_ENABLE_ALERT(24h), or the specific core alerts to **disable** via BIDI_DISABLE_ALERT(25h).

If **enabling** core alerts, the core alerts corresponding to the bits set in the *ulType* field will be enabled (in addition to any currently enabled core alerts). If **disabling** core alerts, the core alerts corresponding to the bits set in the *ulType* field will be disabled. The state of any other enabled core alerts will not change.

If the protocol converter does not understand one of the bits that are set, ERROR_INVALID_FLAG_NUMBER(186) should be returned.

If the printer is not capable of setting one of the core alerts whose bit is set, a successful return code should be returned if any core alerts can be enabled (or disabled).

If the category is **not** PRTALERT_CATEGORY_CORE, this value defines the specific extended alert to **enable** via BIDI_ENABLE_ALERT(24h), or **disable** via BIDI_DISABLE_ALERT(25h).

Values are as follows:

Bit	Description
0	PRTALERT_CORE_INPUT - 0x00000001 Input alert (Example: out of paper)
1	PRTALERT_CORE_OUTPUT - 0x00000002 Output alert (Example: output bin full)
2	PRTALERT_CORE_JAM - 0x00000004 Paper jam alert
3	PRTALERT_CORE_OPERATOR - 0x00000008 Operator intervention required (Example: print ribbon jam)
4	PRTALERT_CORE_CONFIG_CHANGE - 0x00000010 Configuration change (Example: cartridge removed)

5	PRTALERT_CORE_SUPPLIES - 0x00000020 Supplies alert (Example: out of toner).
6	PRTALERT_CORE_JOB_START - 0x00000040 Job started printing.
7	PRTALERT_CORE_JOB_STACKED - 0x00000080 Job completed and stacked.
8	PRTALERT_CORE_PAGE_PRINTED - 0x00000100 Page printed.
9	PRTALERT_CORE_JOB_CANCELLED - 0x00000200 Job cancelled inside printer.
10	PRTALERT_CORE_JOB_HELD - 0x00000400 Job held inside printer.
11	PRTALERT_CORE_COVER_OPEN - 0x00000800 Cover or panel open.
12	PRTALERT_CORE_POWER_ON - 0x00001000 Printer powered on.
13	PRTALERT_CORE_RESET - 0x00002000 Printer reset.
14	PRTALERT_CORE_ONLINE - 0x00004000 Printer now online.
15	PRTALERT_CORE_OFFLINE - 0x00008000 Printer now offline.
16	PRTALERT_CORE_TIMED_ALERT - 0x00010000 Timed alert.
17	PRTALERT_CORE_COMMUNICATION_PROBLEM - 0x00020000 Communication problem.
18	PRTALERT_CORE_COMM_STATUS_CHANGED - 0x00040000 Communication status changed.

PRTSPLRVER

Information about the spooler version, returned by the BIDI_Q_SPOOLER_VERSION command.

```
typedef struct _PRTSPLRVER {
  ULONG     ulMajorVersion;  /*  Major version number of the spooler processing this request. */
  ULONG     ulMinorVersion;  /*  Minor version number of the spooler. */
} PRTSPLRVER;

typedef PRTSPLRVER *PPRTSPLRVER;
```

PRTSPLRVER Field - ulMajorVersion

ulMajorVersion (ULONG)

Major version number of the spooler processing this request.

The major version for this initial release is 1.

PRTSPLRVER Field - ulMinorVersion

ulMinorVersion (ULONG)
Minor version number of the spooler.

The minor version for this initial release is 0 (zero).

PRTSTARTJOB

Information regarding the job about to be sent to the printer. This information is passed to the BIDI_STARTJOB command.

```
typedef struct _PRTSTARTJOB {
  ULONG     ulSpoolerJobID;   /*  Spooler job identifier. */
  ULONG     ulInterpreterID;  /*  Interpreter identifier. */
  ULONG     ulStartPage;      /*  Page in job to begin printing. */
  ULONG     ulEndPage;        /*  Page in job to stop printing. */
} PRTSTARTJOB;

typedef PRTSTARTJOB *PPRTSTARTJOB;
```

PRTSTARTJOB Field - ulSpoolerJobID

ulSpoolerJobID (ULONG)
Spooler job identifier.

PRTSTARTJOB Field - ulInterpreterID

ulInterpreterID (ULONG)
Interpreter identifier.

PRTSTARTJOB Field - ulStartPage

ulStartPage (ULONG)
Page in job to begin printing.

Set to 0 (zero) to start printing with first page in job.

PRTSTARTJOB Field - ulEndPage

ulEndPage (ULONG)
Page in job to stop printing.

Set to 0 (zero) to print to the end of the job.

PRTSTORAGE

Information about the storage media available in the printer, returned by the BIDI_Q_STORAGE command.

```
typedef struct _PRTSTORAGE {
  ULONG     ulType;            /*  Type of storage media. */
  ULONG     ulID;              /*  Storage identifier unique to this storage type. */
  ULONG     ulSize;            /*  Total storage of media, in K bytes. */
  ULONG     ulAvail;           /*  Available storage of media, in K bytes. */
  ULONG     ulpszName;         /*  Offset to name of media. */
  ULONG     ulpszDescription;  /*  Offset to a descriptive string for the media. */
} PRTSTORAGE;

typedef PRTSTORAGE *PPRTSTORAGE;
```

PRTSTORAGE Field - ulType

ulType (ULONG)
Type of storage media.

Value	Description
1	PRTSTOR_TYPE_CARTRIDGE - 1 Font cartridge
2	PRTSTOR_TYPE_DISK - 2 Disk or hardfile
3	PRTSTOR_TYPE_RAM - 3 RAM memory
4	PRTSTOR_TYPE_FLASH - 4 Flash
0xFFFFFFFE	PRTSTOR_TYPE_OTHER - 0xFFFFFFFE Other type of media
0xFFFFFFFF	PRTSTOR_TYPE_UNKNOWN - 0xFFFFFFFF Media type unknown

---

PRTSTORAGE Field - ulID

ulID (ULONG)
Storage identifier unique to this storage type.

For example, if there are two disks, *ulID* could be 1 or 2.

PRTSTORAGE Field - ulSize

ulSize (ULONG)
Total storage of media, in K bytes.

If not applicable or available, set to 0 (zero).

PRTSTORAGE Field - ulAvail

ulAvail (ULONG)
Available storage of media, in K bytes.

If not applicable or available, set to 0 (zero).

PRTSTORAGE Field - ulpszName

ulpszName (ULONG)
Offset to name of media.

For example, the font cartridge name.

PRTSTORAGE Field - ulpszDescription

ulpszDescription (ULONG)
Offset to a descriptive string for the media.

PRTSTORAGEL

Information about the printer storage media, returned by the BIDI_Q_STORAGE command.

```
typedef struct _PRTSTORAGEL {
  ULONG          cTotal;                          /*  Total number of storage media in the printer. */
  ULONG          cReturned;                       /*  Number of PRTSTORAGE data structures that follow. */
  PRTSTORAGE     prtStorage[1];  /*  Contains a PRTSTORAGE data structure. */
} PRTSTORAGEL;

typedef PRTSTORAGEL *PPRTSTORAGEL;
```

PRTSTORAGEL Field - cTotal

cTotal (ULONG)
Total number of storage media in the printer.

PRTSTORAGEL Field - cReturned

cReturned (ULONG)
Number of PRTSTORAGE data structures that follow.

PRTSTORAGEL Field - prtStorage[1]

prtStorage[1] (PRTSTORAGE)
Contains a PRTSTORAGE data structure.

PRTSW

Information about BIDI capabilities of the printer.

```
typedef struct _PRTSW {
  ULONG     flJob;     /*  Job control flags. */
  ULONG     flDevice;  /*  Printer device flags. */
} PRTSW;

typedef PRTSW *PPRTSW;
```

PRTSW Field - flJob

flJob (ULONG)
Job control flags.

Values are as follows:

Bit	Description
0	PRTSW_JOB_BOUNDARY_SEND - 0x00000001 Set if the BIDI protocol used requires some query and set commands to be issued outside a job boundary. These commands use the same communications channel as the print data. Some printers do not have a separate communications channel for control functions, thus requiring the current job to be sent to the printer in its entirety before a query request can be issued.

1	PRTSW_JOB_BOUNDARY_PROCESS - 0x00000002 Set if the BIDI protocol used might not allow the printer to process query or set commands until the current job is processed. Note that some commands might be processed by the printer while the printer is getting print job data, if the command is sent via a different communications channel (for example, an SNMP channel). This flag may be used to determine if an immediate response will be received from the printer.
2	PRTSW_JOB_TRUE_END - 0x00000004 Set if the BIDI protocol used allows the printer to indicate true end-of-job. True end-of-job is a notification that the print device successfully output the job.
3	PRTSW_JOB_HOLD_RELEASE - 0x00000008 Set if the BIDI protocol used allows the printer to hold and release jobs inside the printer.
4	PRTSW_JOB_CANCEL - 0x00000010 Set if the BIDI protocol used allows the printer to cancel jobs inside the printer.
5	PRTSW_JOB_MUST_RESUBMIT - 0x00000020 Set if canceling a print job requires all later jobs to be resubmitted. Setting this will cause the spooler to resubmit all jobs not yet printed whenever a job in the printer is canceled.
6	PRTSW_JOB_WRAPPER_REQUIRED - 0x00000040 Set if data packets require a wrapper supplied by the protocol converter.
7	PRTSW_JOB_STARTPAGE - 0x00000080 Set if the printer supports the **startpage** feature. This feature allows the printer to not output any print pages until the **startpage** page number is encountered in the print job data. This lets users skip the first few pages of their output file.
8	PRTSW_JOB_ENDPAGE - .0x00000100 Set if the printer supports the **endpage** feature. This feature allows the printer to stop outputting any print pages after **endpage** page number is encountered in the print job data. This lets users skip the last few pages of their output file.
9 - 31	Reserved; must be clear.

PRTSW Field - flDevice

flDevice (ULONG)
Printer device flags.

Values are as follows:

Bit	Description
0	PRTSW_DEV_PAGEPRINT_ALERT - 0x00000001 Set if the printer can send alerts when a page has completed printing.
1	PRTSW_DEV_JOBBEGIN_ALERT - 0x00000002 Set if the printer can send alert when a job begins printing.
2	PRTSW_JOBSTACK_ALERT - 0x00000004 Set if the printer can send alert when a job is stacked (completes printing).
3	PRTSW_TIMED_ALERT - 0x00000008 Set if the printer can send alerts at a given time interval.
4	PRTSW_LU_REQUIRED - 0x00000010 Set if logical unit number is necessary to submit job to correct interpreter. This flag is clear if the datastream should contain the command to switch to correct interpreter.
5 - 31	Reserved; must be clear.

PSCANDATA

Pointer to SCANDATA data structure.

```
typedef SCANDATA *PSCANDATA;
```

PSHORTLINE

Pointer to SHORTLINE data structure.

```
typedef SHORTLINE *PSHORTLINE;
```

PSZ

Pointer to a null-terminated string.

If you are using C++ **, you may need to use PCSZ.

```
typedef unsigned char *PSZ;
```

PVOID

Pointer to a data type of undefined format.

```
typedef VOID *PVOID;
```

PXFORM

Pointer to XFORM data structure.

```
typedef XFORM *PXFORM;
```

PSQPOPENDATA

Pointer to SQPOPENDATA data structure.

```
typedef SQPOPENDATA *PSQPOPENDATA;
```

QBINS

Structure passed into the BIDI_Q_INPUTBINS and BIDI_Q_OUTPUTBINS commands. The BIDI_Q_ command used determines the information to be returned.

```
typedef struct _QBINS {
  ULONG     ulInterpreterID;  /*  Interpreter identifier to get input (or output) bin information. */
  ULONG     ulBinID;          /*  Input (or output) source ID, or -1 to query all input (or output) sources. */
} QBINS;

typedef QBINS *PQBINS;
```

QBINS Field - ulInterpreterID

ulInterpreterID (ULONG)
Interpreter identifier to get input (or output) bin information.

A value of hex 'FFFFFFFF' requests input (or output) bins from all interpreters.

QBINS Field - ulBinID

ulBinID (ULONG)
Input (or output) source ID, or -1 to query all input (or output) sources.

QMJOBINFO

Information about a job being spooled, returned by SplQmGetJobID.

```
typedef struct _QMJOBINFO {
  ULONG     ulJobID;          /*  Spooler print job identifier. */
  PSZ       pszComputerName;  /*  Name of server job being spooled. */
  PSZ       pszQueueName;     /*  Name of queue job being spooled. */
} QMJOBINFO;

typedef QMJOBINFO *PQMJOBINFO;
```

QMJOBINFO Field - ulJobID

ulJobID (ULONG)
Spooler print job identifier.

This ID may be used with SplSetJob.

QMJOBINFO Field - pszComputerName

pszComputerName (PSZ)
Name of server job being spooled.

This parameter is NULL if the job is spooled locally.

QMJOBINFO Field - pszQueueName

pszQueueName (PSZ)
Name of queue job being spooled.

QUERYSIZE

This data structure is used in the GreEscape DEVESC_QUERYSIZE call.

```
typedef struct _QUERYSIZE {
  ULONG          cb;                          /*  Size of the entire structure. */
  ULONG          ulSizeNeeded;                /*  Size returned. */
  QUERYTUPLE     aTuples[1];  /*  Start of the tuple list. */
} QUERYSIZE;

typedef QUERYSIZE *PQUERYSIZE;
```

QUERYSIZE Field - cb

cb (ULONG)
Size of the entire structure.

QUERYSIZE Field - ulSizeNeeded

ulSizeNeeded (ULONG)
Size returned.

QUERYSIZE Field - aTuples[1]

aTuples[1] (QUERYTUPLE)
Start of the tuple list.

aTuples[1] is a variable-length array of QUERYTUPLE data structures. These structures are used to determine the job properties that are being queried. Use aTuples[x].ulProperty = DJP_NONE to identify the end of the list.

QUERYTUPLE

This structure is used in the GreEscape DEVESC_QUERYSIZE call. It associates the type of property you are querying (*ulProperty*) and the kind of query you will do (DJP_ALL or DJP_CURRENT).

```
typedef struct _QUERYTUPLE {
  ULONG     ulProperty;  /*  Identifies specific job property. */
  LONG      lType;       /*  Property type. */
} QUERYTUPLE;

typedef QUERYTUPLE *PQUERYTUPLE;
```

QUERYTUPLE Field - ulProperty

ulProperty (ULONG)
Identifies specific job property.

The 'C' or 'S' in the job property name indicates the level of complexity: *complex* or s*imple* . The 'J' or 'P' indicates the class of the property, where J is job property and P is printer property.

Valid job properties are as follows:

DJP_NONE - 0L — Also used as end-of-list marker
DJP_SJ_ORIENTATION - 1L
DJP_CJ_RESOLUTION - 2L
DJP_SJ_BITSPERPEL - 3L
DJP_SJ_COLOR - 4L
DJP_SJ_PRINTQUALITY - 5L
DJP_SJ_PAPERSIZE - 6L
DJP_SJ_TRAYTYPE - 7L
DJP_SJ_MEDIA - 8L
DJP_SJ_MEDIA_COLOR - 9L
DJP_CJ_FORM - 10L
DJP_CJ_MIXEDFORMS - 11L
DJP_SJ_FONTDOWNLOADING - 12L
DJP_SJ_DUPLEX - 13L
DJP_SJ_COLLATE - 14L
DJP_SJ_FEED - 15L
DJP_SJ_COPIES - 16L
DJP_SJ_SCALING - 17L
DJP_SJ_FORMFEEDCONTROL - 18L
DJP_SJ_N_UP - 19L

QUERYTUPLE Field - lType

lType (LONG)

Property type.

Input values are as follows:

DJP_ALL

The application requests all possible values. The list of values will be placed starting at the *ulValue* field.

DJP_CURRENT

The application requests the current value be taken from the input job properties (use the value from *ulProperty*).

RECTL

Rectangle structure.

```
typedef struct _RECTL {
  LONG     xLeft;     /*  X-coordinate of left-hand edge of rectangle. */
  LONG     yBottom;   /*  Y-coordinate of bottom edge of rectangle. */
  LONG     xRight;    /*  X-coordinate of right-hand edge of rectangle. */
  LONG     yTop;      /*  Y-coordinate of top edge of rectangle. */
} RECTL;

typedef RECTL *PRECTL;
```

RECTL Field - xLeft

xLeft (LONG)
X-coordinate of left-hand edge of rectangle.

RECTL Field - yBottom

yBottom (LONG)
Y-coordinate of bottom edge of rectangle.

RECTL Field - xRight

xRight (LONG)
X-coordinate of right-hand edge of rectangle.

RECTL Field - yTop

yTop (LONG)
Y-coordinate of top edge of rectangle.

RGB

RGB color value.

```
typedef struct _RGB {
  BYTE     bBlue;    /*  Blue component of the color definition. */
  BYTE     bGreen;   /*  Green component of the color definition. */
  BYTE     bRed;     /*  Red component of the color definition. */
} RGB;

typedef RGB *PRGB;
```

RGB Field - bBlue

bBlue (BYTE)
Blue component of the color definition.

RGB Field - bGreen

bGreen (BYTE)
Green component of the color definition.

RGB Field - bRed

bRed (BYTE)
Red component of the color definition.

RGB2

RGB color value.

```
typedef struct _RGB2 {
  BYTE     bBlue;       /*  Blue component of the color definition. */
  BYTE     bGreen;      /*  Green component of the color definition. */
  BYTE     bRed;        /*  Red component of the color definition. */
  BYTE     fcOptions;   /*  Entry options. */
} RGB2;
```

```
typedef RGB2 *PRGB2;
```

RGB2 Field - bBlue

bBlue (BYTE)
Blue component of the color definition.

RGB2 Field - bGreen

bGreen (BYTE)
Green component of the color definition.

RGB2 Field - bRed

bRed (BYTE)
Red component of the color definition.

RGB2 Field - fcOptions

fcOptions (BYTE)
Entry options.

These can be ORed together if required:

PC_RESERVED	The color entry is reserved for animating color with the palette manager.
PC_EXPLICIT	The low-order word of the color table entry designates a physical palette slot. This allows an application to show the actual contents of the device palette as realized for other logical palettes. This does not prevent the color in the slot from being changed for any reason.

RGNRECT

Region-rectangle structure.

```
typedef struct _RGNRECT {
  ULONG     ircStart;     /*  Rectangle number from which to start enumerating. */
  ULONG     crc;          /*  Number of rectangles that can be returned. */
  ULONG     crcReturned;  /*  Number of rectangles returned. */
  ULONG     ulDirection;  /*  Direction in which the returned rectangles are to be ordered. */
} RGNRECT;
```

```
typedef RGNRECT *PRGNRECT;
```

RGNRECT Field - ircStart

ircStart (ULONG)
Rectangle number from which to start enumerating.

Numbering starts from 1.

RGNRECT Field - crc

crc (ULONG)
Number of rectangles that can be returned.

This must be 1 or greater.

RGNRECT Field - crcReturned

crcReturned (ULONG)
Number of rectangles returned.

A value of less than *crc* indicates that there are no more rectangles to enumerate.

RGNRECT Field - ulDirection

ulDirection (ULONG)
Direction in which the returned rectangles are to be ordered.

This ordering uses the leading edge of a rectangle:

RECTDIR_LFRT_TOPBOT	Left-to-right, top-to-bottom
RECTDIR_RTLF_TOPBOT	Right-to-left, top-to-bottom
RECTDIR_LFRT_BOTTOP	Left-to-right, bottom-to-top
RECTDIR_RTLF_BOTTOP	Right-to-left, bottom-to-top

SCA

Screen change area data structure.

```
typedef struct _SCA {
  SCA       pscaNext;                                       /*  Linked list pointer. */
  ULONG     cRects;                                         /*  Number of rectangles in area. */
  RECTL     arcl[MAX_SCA_RECTS+1];          /*  Rectangle dimensions. */
  ULONG     aulRectSize[MAX_SCA_RECTS+1];   /*  Cached rectangle sizes. */
} SCA;

typedef SCA *PSCA;
```

SCA Field - pscaNext

pscaNext (SCA)
Linked list pointer.

SCA Field - cRects

cRects (ULONG)
Number of rectangles in area.

SCA Field - arcl[MAX_SCA_RECTS+1]

arcl[MAX_SCA_RECTS+1] (RECTL)
Rectangle dimensions.

SCA Field - aulRectSize[MAX_SCA_RECTS+1]

aulRectSize[MAX_SCA_RECTS+1] (ULONG)
Cached rectangle sizes.

SCREENRESOLUTION

Output from OS2_PM_DRV_QUERYSCREENRESOLUTIONS is returned in an array of SCREENRESOLUTION structures, following the SCREENRESCOUNT structure.

```
typedef struct _SCREENRESOLUTION {
  ULONG     width;      /*  Width of the device, in pels. */
  ULONG     height;     /*  Height of the device, in pels. */
  ULONG     colors;     /*  Number of colors supported in this mode. */
```

```
  ULONG     planes;     /*  Number of display planes in this mode. */
  ULONG     floptions;  /*  Optional information for each resolution. */
} SCREENRESOLUTION;

typedef SCREENRESOLUTION *PSCREENRESOLUTION;
```

SCREENRESOLUTION Field - width

width (ULONG)
Width of the device, in pels.

SCREENRESOLUTION Field - height

height (ULONG)
Height of the device, in pels.

SCREENRESOLUTION Field - colors

colors (ULONG)
Number of colors supported in this mode.

SCREENRESOLUTION Field - planes

planes (ULONG)
Number of display planes in this mode.

SCREENRESOLUTION Field - floptions

floptions (ULONG)
Optional information for each resolution.

Valid values include:

DSP_RESOLUTION_OBTAINABLE	Obtainable with the hardware configuration
DSP_RESOLUTION_DEFAULT	Default resolution for the hardware configuration

---

SCREENRESCOUNT

Output from OS2_PM_DRV_QUERYSCREENRESOLUTIONS is returned in this structure.

```
typedef struct _SCREENRESCOUNT {
  ULONG     maxcount;           /*  Total number of screen resolutions supported. */
  ULONG     count;              /*  Number of SCREENRESOLUTION structures returned. */
  ULONG     res_struct_length;  /*  Length of a SCREENRESOLUTION data structure, in bytes. */
} SCREENRESCOUNT;

typedef SCREENRESCOUNT *PSCREENRESCOUNT;
```

SCREENRESCOUNT Field - maxcount

maxcount (ULONG)
Total number of screen resolutions supported.

SCREENRESCOUNT Field - count

count (ULONG)
Number of SCREENRESOLUTION structures returned.

This number will be less than *maxcount* if the defined size of the *pBuf* buffer is too small, as identified by *pcbBuf*.

SCREENRESCOUNT Field - res_struct_length

res_struct_length (ULONG)
Length of a SCREENRESOLUTION data structure, in bytes.

This value should be used to increment between structures so that any future increase in the size of the structure (to add information) will not cause a failure.

SETUPBLITTERNOTIFYIN

The display driver, if it supports this function, should indicate whether the hardware can handle the current image blitting parameters. If the display driver does not support this function, it is assumed the hardware can handle the blitter operation being set.

```
typedef struct _SETUPBLITTERNOTIFYIN {
  ULONG     ulLength;           /* length of this structure         */
  ULONG     ulHandle;           /* instance handle from register    */
  ULONG     ulInversionFlags;   /* current image inversion flags    */
                                /* 0x1 = vertical, 0x2 = horizontal */
  FOURCC    fccSrc;             /* source color format              */
```

```
  ULONG     ulSrcWidth;         /* source image width in pixels     */
  ULONG     ulSrcHeight;        /* source image height in pixels    */
  ULONG     ulSrcPosX;          /* src image X offset for blitting  */
  ULONG     ulSrcPosY;          /* src image Y offset for blitting  */
  FOURCC    fccDst;             /* destination color format         */
  ULONG     ulDstWidth;         /* destination image width in pels  */
  ULONG     ulDstHeight;        /* destination image height in pels */
  LONG      lScreenPosX;        /* destination screen X position    */
  LONG      lScreenPosY;        /* destination screen Y position    */
  ULONG     ulNumDstRects;      /* number of visible rectangles     */
  PRECTL    pVisDstRects;       /* ptr to visible rectangle array   */

} SETUPBLITTERNOTIFYIN;

typedef SETUPBLITTERNOTIFYIN *PSETUPBLITTERNOTIFYIN;
```

SETUPBLITTERNOTIFYIN Field - ulLength

ulLength (ULONG)
Length of this structure.

SETUPBLITTERNOTIFYIN Field - ulHandle

ulHandle (ULONG)
Instance handle from register.

SETUPBLITTERNOTIFYIN Field - ulInversionFlags

ulInversionFlags (ULONG)
Current image inversion flags.

SETUPBLITTERNOTIFYIN Field - fccSrc

fccSrc (FOURCC)
Source color format.

SETUPBLITTERNOTIFYIN Field - ulSrcWidth

ulSrcWidth (ULONG)
Source image width in pixels.

SETUPBLITTERNOTIFYIN Field - ulSrcHeight

ulSrcHeight (ULONG)
Source image height in pixels.

SETUPBLITTERNOTIFYIN Field - ulSrcPosX

ulSrcPosX (ULONG)
Source image X offset for blitting.

SETUPBLITTERNOTIFYIN Field - ulSrcPosY

ulSrcPosY (ULONG)
Source image Y offset for blitting.

SETUPBLITTERNOTIFYIN Field - fccDst

fccDst (FOURCC)
Destination color format.

SETUPBLITTERNOTIFYIN Field - ulDstWidth

ulDstWidth (ULONG)
Destination image width in pels.

SETUPBLITTERNOTIFYIN Field - ulDstHeight

ulDstHeight (ULONG)
Destination image height in pels.

SETUPBLITTERNOTIFYIN Field - lScreenPosX

lScreenPosX (LONG)
Destination screen X position.

SETUPBLITTERNOTIFYIN Field - lScreenPosY

lScreenPosY (LONG)
Destination screen Y position.

SETUPBLITTERNOTIFYIN Field - ulNumDstRects

ulNumDstRects (ULONG)
Number of visible rectangles.

SETUPBLITTERNOTIFYIN Field - pVisDstRects

pVisDstRects (PRECTL)
Pointer to a visible rectangle array (in screen coordinates).

SETUPBLITTERNOTIFYOUT

The DEVESC_SETUPBLITTERNOTIFY output data structure.

```
typedef struct _SETUPBLITTERNOTIFYOUT {
  ULONG     ulLength;           /* length of this structure             */
  ULONG     ulFlags;            /* 0 => HW can handle, SETUP_CANT_HANDLE */
} SETUPBLITTERNOTIFYOUT;

typedef SETUPBLITTERNOTIFYOUT *PSETUPBLITTERNOTIFYOUT;
```

SETUPBLITTERNOTIFYOUT Field - ulLength

ulLength (ULONG)
Length of this structure.

SETUPBLITTERNOTIFYOUT Field - ulFlags

ulFlags (ULONG)
Zero indicates that the hardware can perform the blitting operation, otherwise the device driver must perform the blitting operation.

SHORT

Signed integer in the range -32 768 through 32 767.

```
#define SHORT short
```

SIZEF

Size structure (FIXED values).

```
typedef struct _SIZEF {
  FIXED     cx;  /*  Width. */
  FIXED     cy;  /*  Height. */
} SIZEF;

typedef SIZEF *PSIZEF;
```

SIZEF Field - cx

cx (FIXED)
Width.

SIZEF Field - cy

cy (FIXED)
Height.

SIZEL

Size structure (LONG values).

```
typedef struct _SIZEL {
  LONG      cx;   /*  Width. */
  LONG      cy;   /*  Height. */
} SIZEL;

typedef SIZEL *PSIZEL;
```

SIZEL Field - cx

cx (LONG)
Width.

SIZEL Field - cy

cy (LONG)
Height.

SIZEROF

Size structure.

```
typedef follow _SIZEROF {
  FIXED     fxWidth;    /*  Width. */
  FIXED     fxHeight;   /*  Height. */
};
```

SIZEROF Field - fxWidth

fxWidth (FIXED)
Width.

SIZEROF Field - fxHeight

fxHeight (FIXED)
Height.

STR8

String of 8 characters.

```
typedef CHAR STR8[8];
```

STR32

String of characters, with an implicit length, in a 32-byte field.

```
typedef CHAR STR32[32];
```

STR64

String of characters, with an implicit length, in a 64-byte field.

```
typedef CHAR STR64[64];
```

TYPE1CELL

Indicates the width of a character in the inline direction (the b-space).

```
typedef struct _TYPE1CELL {
  LONG       offset;  /*  Pointer to glyph. */
  USHORT     width;   /*  Width of character, in world coordinates. */
} TYPE1CELL;

typedef TYPE1CELL *PTYPE1CELL;
```

TYPE1CELL Field - offset

offset (LONG)
Pointer to glyph.

TYPE1CELL Field - width

width (USHORT)
Width of character, in world coordinates.

TYPE2CELL

Indicates the width of a character in the inline direction (the b-space).

```
typedef struct _TYPE2CELL {
  LONG       offset;  /*  Pointer to glyph. */
  USHORT     width;   /*  Width of character, in world coordinates. */
} TYPE2CELL;

typedef TYPE2CELL *PTYPE2CELL;
```

TYPE2CELL Field - offset

offset (LONG)
Pointer to glyph.

TYPE2CELL Field - width

width (USHORT)
Width of character, in world coordinates.

TYPE3CELL

Indicates the width of the space before a character in the inline direction (the a-space) as the a-space for all characters. Indicates the width of a character in the inline direction (the b-space) as the b-space for all characters. Indicates the width of the space after a character in the inline direction (the c-space) as the c-space for all characters.

```
typedef struct _TYPE3CELL {
  LONG       offset;   /*  Pointer to glyph data. */
  SHORT      a_space;  /*  Width of the space before a character. */
  USHORT     b_space;  /*  Width of a character. */
  SHORT      c_space;  /*  Width of the space after a character. */
} TYPE3CELL;

typedef TYPE3CELL *PTYPE3CELL;
```

TYPE3CELL Field - offset

offset (LONG)
Pointer to glyph data.

TYPE3CELL Field - a_space

a_space (SHORT)
Width of the space before a character.

The width of the space before a character in the inline direction (the a-space).

TYPE3CELL Field - b_space

b_space (USHORT)
Width of a character.

The width of a character in the inline direction (the b-space).

TYPE3CELL Field - c_space

c_space (SHORT)
Width of the space after a character.

The width of the space after a character in the inline direction (the c-space).

UCHAR

Single-byte unsigned character or unsigned integer in the range 0 through 255.

```
typedef unsigned char UCHAR;
```

ULONG

32-bit unsigned integer in the range 0 through 4 294 967 295.

```
typedef unsigned long ULONG;
```

USHORT

Unsigned integer in the range 0 through 65 535.

```
typedef unsigned short USHORT;
```

VALID_DATA_FORMATS

Defines valid source and destination format pairs.

```
typedef struct _VALID_DATA_FORMATS {
  ULONG     ulSrcDstFormat;           /*  Combined source and destination codes. */
  PFN       pfnRowConversionRoutine;  /*  Address of the conversion routine for this pair. */
  PBYTE     *ppConvertTable;          /*  Address to the conversion table, if required. */
  PFN       pfnCreateConvertTable;    /*  Address to a routine that creates a conversion table. */
} VALID_DATA_FORMATS;

typedef VALID_DATA_FORMATS *PVALID_DATA_FORMATS;
```

VALID_DATA_FORMATS Field - ulSrcDstFormat

ulSrcDstFormat (ULONG)
Combined source and destination codes.

VALID_DATA_FORMATS Field - pfnRowConversionRoutine

pfnRowConversionRoutine (PFN)
Address of the conversion routine for this pair.

VALID_DATA_FORMATS Field - *ppConvertTable

***ppConvertTable** (PBYTE)
Address to the conversion table, if required.

VALID_DATA_FORMATS Field - pfnCreateConvertTable

pfnCreateConvertTable (PFN)
Address to a routine that creates a conversion table.

VIOSIZECOUNT

Count of VIO cell sizes. See DevEscape.

```
typedef struct _VIOSIZECOUNT {
  LONG     maxcount;  /*  Maximum number of VIO cell sizes supported. */
  LONG     count;     /*  Number of VIO cell sizes returned. */
} VIOSIZECOUNT;

typedef VIOSIZECOUNT *PVIOSIZECOUNT;
```

VIOSIZECOUNT Field - maxcount

maxcount (LONG)
Maximum number of VIO cell sizes supported.

VIOSIZECOUNT Field - count

count (LONG)
Number of VIO cell sizes returned.

VIOFONTATTRIBUTE

FONTMGRPARM data structure.

```
typedef struct _VIOFONTATTRIBUTE {
  ULONG     vioa_ulRecordLength;
  LONG      vioa_ulFunctionID;
  ULONG     vioa_cx;
  ULONG     vioa_cy;
} VIOFONTATTRIBUTE;

typedef VIOFONTATTRIBUTE *PVIOFONTATTRIBUTE;
```

VIOFONTATTRIBUTE Field - vioa_ulRecordLength

vioa_ulRecordLength (ULONG)

VIOFONTATTRIBUTE Field - vioa_ulFunctionID

vioa_ulFunctionID (LONG)

VIOFONTATTRIBUTE Field - vioa_cx

vioa_cx (ULONG)

VIOFONTATTRIBUTE Field - vioa_cy

vioa_cy (ULONG)

VIOFONTCELLSIZE

An array of VIOFONTCELLSIZE structures.

```
typedef struct _VIOFONTCELLSIZE {
  BYTE     xWidth;    /*  Width of the VIO character cell. */
  BYTE     yHeight;   /*  Height of the VIO character cell. */
} VIOFONTCELLSIZE;

typedef VIOFONTCELLSIZE *PVIOFONTCELLSIZE;
```

VIOFONTCELLSIZE Field - xWidth

xWidth (BYTE)
Width of the VIO character cell.

VIOFONTCELLSIZE Field - yHeight

yHeight (BYTE)
Height of the VIO character cell.

VioPresentationSpace *

A pointer to the Vio presentation space.

```
typedef VioPresentationSpace *VioPresentationSpace *;
```

VOID

A data area of undefined format.

```
#define VOID void
```

VRAMALLOCIN

Input structure to GreEscape DEVESC_VRAMALLOC. Used for VRAM allocation and deallocation requests.

```
typedef struct _VRAMALLOCIN {
  ULONG     ulLength;    /*  Length of VRAMALLOCIN data structure, in bytes. */
  ULONG     ulFlags;     /*  Allocation request flag. */
  ULONG     ulID;        /*  Buffer ID. */
  ULONG     ulFunction;  /*  Indicates allocation status. */
  ULONG     ulHandle;    /*  VRAMREGISTER instance handle. */
  ULONG     ulSize;      /*  Size to allocate, in bytes. */
  ULONG     ulWidth;     /*  Width of the image, in pels. */
  ULONG     ulHeight;    /*  Height of the image, in scanlines. */
} VRAMALLOCIN;

typedef VRAMALLOCIN *PVRAMALLOCIN;
```

VRAMALLOCIN Field - ulLength

ulLength (ULONG)
Length of VRAMALLOCIN data structure, in bytes.

VRAMALLOCIN Field - ulFlags

ulFlags (ULONG)
Allocation request flag.

This flag has the following values:

ENDIVE_ALLOC_SHARED
0x0001; indicates allocated memory is shared, not private

ENDIVE_ALLOC_RECTANGLE
0x0002; not implemented at this time

VRAMALLOCIN Field - ulID

ulID (ULONG)
Buffer ID.

Used when *ulFunction* is set for buffer deallocation.

VRAMALLOCIN Field - ulFunction

ulFunction (ULONG)
Indicates allocation status.

The valid values are as follows:

VRAM_ALLOCATE
0x0001; indicates allocation

VRAM_DEALLOCATE
0x0002; indicates deallocation

VRAM_QUERY
0x0003; returns maximum memory size available at time of call

---

VRAMALLOCIN Field - ulHandle

ulHandle (ULONG)
VRAMREGISTER instance handle.

VRAMALLOCIN Field - ulSize

ulSize (ULONG)
Size to allocate, in bytes.

VRAMALLOCIN Field - ulWidth

ulWidth (ULONG)
Width of the image, in pels.

VRAMALLOCIN Field - ulHeight

ulHeight (ULONG)
Height of the image, in scanlines.

VRAMALLOCOUT

Output structure from GreEscape DEVESC_VRAMALLOC. Contains information describing the results of VRAM allocation.

```
typedef struct _VRAMALLOCOUT {
  ULONG      ulLength;          /*  Length of VRAMALLOCOUT data structure, in bytes. */
  ULONG      ulFlags;           /*  Returned flag work buffer. */
  ULONG      ulID;              /*  Buffer ID. */
  POINTL     ptlStart;          /*  X and Y point offset into the VRAM. */
  ULONG      ulSize;            /*  Byte size allocated. */
  ULONG      ulScanLineBytes;   /*  Number of bytes per scanline. */
} VRAMALLOCOUT;

typedef VRAMALLOCOUT *PVRAMALLOCOUT;
```

VRAMALLOCOUT Field - ulLength

ulLength (ULONG)
Length of VRAMALLOCOUT data structure, in bytes.

VRAMALLOCOUT Field - ulFlags

ulFlags (ULONG)
Returned flag work buffer.

This flag has the following value:

ENDIVE_ALLOC_WORKBUFFER
0x0004; allocates memory in a temporary work buffer

VRAMALLOCOUT Field - ulID

ulID (ULONG)

Buffer ID.

Returned allocated buffer ID used when buffer is allocated.

VRAMALLOCOUT Field - ptlStart

ptlStart (POINTL)
X and Y point offset into the VRAM.

VRAMALLOCOUT Field - ulSize

ulSize (ULONG)
Byte size allocated.

VRAMALLOCOUT Field - ulScanLineBytes

ulScanLineBytes (ULONG)
Number of bytes per scanline.

VRAMALLOCOUT2

This data structure is identical to VRAMALLOCOUT structure except that it contains an additional field for the optional pointer to a system memory buffer and an expanded definition of the ulFlags field. The expanded definition indicates if a system memory buffer has been allocated instead of a VRAM buffer.

```
typedef struct _VRAMALLOCOUT2 {
  ULONG     ulLength;           /* length of this structure         */
  ULONG     ulFlags;            /* Returned flag work buffer        */
  ULONG     ulID;               /* Buffer ID                        */
  POINTL    ptlStart;           /* X and Y point offset into VRAM   */
  ULONG     ulSize;             /* Byte size allocated              */
  ULONG     ulScanLineBytes;    /* Number of bytes per scan line    */
  PVOID     pNewBuffer;         /* 32-bit linear ptr to new buffer  */
} VRAMALLOCOUT2;

typedef VRAMALLOCOUT2  *PVRAMALLOCOUT2;
```

VRAMALLOCOUT2 Field - ulLength

ulLength (ULONG)
Length of this structure.

VRAMALLOCOUT2 Field - ulFlags

ulFlags (ULONG)
Returned flag work buffer.

VRAMALLOCOUT2 Field - ulID

ulID (ULONG)
Buffer ID.

VRAMALLOCOUT2 Field - ptlStart

ptlStart (POINTL)
X and Y point offset into the VRAM.

VRAMALLOCOUT2 Field - ulSize

ulSize (ULONG)
Byte size allocated.

VRAMALLOCOUT2 Field - ulScanLineBytes

ulScanLineBytes (ULONG)
Number of bytes per scan line.

VRAMALLOCOUT2 Field - pNewBuffer

pNewBuffer (PVOID)
A 32-bit linear pointer to the new buffer.

VRAMREGISTEROUT

Output packet returned by the GRADD to the caller of the GHI_CMD_VRAM function via the VRAMIN data structure.

```
typedef struct _VRAMREGISTEROUT {
  ULONG     ulLength;  /*  Size of the VRAMREGISTEROUT data structure, in bytes.  */
  ULONG     ulFlags;   /*  Not used at this time. */
  ULONG     ulHandle;  /*  Handle returned on register. */
} VRAMREGISTEROUT;

typedef VRAMREGISTEROUT *PVRAMRGISTEROUT;
```

VRAMREGISTEROUT Field - ulLength

ulLength (ULONG)
Size of the VRAMREGISTEROUT data structure, in bytes.

VRAMREGISTEROUT Field - ulFlags

ulFlags (ULONG)
Not used at this time.

VRAMREGISTEROUT Field - ulHandle

ulHandle (ULONG)
Handle returned on register.

VRAMHWREQUESTIN

Input structure to GreEscape DEVESC_HWREQUEST. Used to acquire and deacquire hardware access.

```
typedef struct _VRAMHWREQUESTIN {
  ULONG          ulLength;     /*  Length of VRAMHWREQUESTIN data structure, in bytes. */
  ULONG          ulHandle;     /*  Instance handle from register. */
  ULONG          ulID;         /*  Buffer ID for off-screen VRAM. */
  PACQUIREFB     pAcquireFB;   /*  Old DIVE data structure. */
  ULONG          ulFlags;      /*  Flag indicating type of request. */
} VRAMHWREQUESTIN;

typedef VRAMHWREQUESTIN *PVRAMHWREQUESTIN;
```

VRAMHWREQUESTIN Field - ulLength

ulLength (ULONG)
Length of VRAMHWREQUESTIN data structure, in bytes.

VRAMHWREQUESTIN Field - ulHandle

ulHandle (ULONG)
Instance handle from register.

VRAMHWREQUESTIN Field - ulID

ulID (ULONG)
Buffer ID for off-screen VRAM.

VRAMHWREQUESTIN Field - pAcquireFB

pAcquireFB (PACQUIREFB)
Old DIVE data structure.

VRAMHWREQUESTIN Field - ulFlags

ulFlags (ULONG)
Flag indicating type of request.

This flag has the following value:

ENDIVE_REQUEST_HARDWARE
0x001; set this flag to request and clear it to release

VRAMHWREQUESTOUT

Output structure from GreEscape DEVESC_HWREQUEST. Contains information describing the state of VRAM at the time of the request.

```
typedef struct _VRAMHWREQUESTOUT {
  ULONG      ulLength;         /*  Length of VRAMHWREQUESTOUT data structure, in bytes. */
```

```
  ULONG      ulFlags;          /*  Flag indicating status of memory allocation. */
  POINTL     ptlStart;         /*  Location of buffer in VRAM. */
  PVOID      psysMem;          /*  Pointer to globally shared system memory. */
  ULONG      ulScanLineBytes;  /*  Number of bytes per scanline. */
} VRAMHWREQUESTOUT;

typedef VRAMHWREQUESTOUT *PVRAMHWREQUESTOUT;
```

VRAMHWREQUESTOUT Field - ulLength

ulLength (ULONG)
Length of VRAMHWREQUESTOUT data structure, in bytes.

VRAMHWREQUESTOUT Field - ulFlags

ulFlags (ULONG)
Flag indicating status of memory allocation.

This flag has the following values:

ENDIVE_REQUEST_UNAVAILABLE
0x0001; set if system memory could not be allocated.

ENDIVE_REQUEST_RELOCATED
0x0002; this process buffer was relocated since last request.

If the integrity of the off-screen VRAM memory cannot be guaranteed for any reason, the contents will be copied into globally shared system memory (*pSysMem*) and the ENDIVE_REQUEST_RELOCATED bit will be set.

If integrity can be guaranteed later on, contents will be relocated (copied back) to off-screen VRAM from system memory and this bit will be set again (*pSysMem* will be NULL).

VRAMHWREQUESTOUT Field - ptlStart

ptlStart (POINTL)
Location of buffer in VRAM.

VRAMHWREQUESTOUT Field - psysMem

psysMem (PVOID)
Pointer to globally shared system memory.

VRAMHWREQUESTOUT Field - ulScanLineBytes

ulScanLineBytes (ULONG)
Number of bytes per scanline.

VRAMREGISTERIN

Registers and unregisters EnDIVE usage with the driver.

```
typedef struct _VRAMREGISTERIN {
  ULONG     ulLength;  /*  Length of VRAMREGISTERIN data structure, in bytes.  */
  ULONG     ulHandle;  /*  Handle required when unregistering. */
  ULONG     ulFlags;   /*  Flag indicating type of register. */
} VRAMREGISTERIN;

typedef VRAMREGISTERIN *PVRAMREGISTERIN;
```

VRAMREGISTERIN Field - ulLength

ulLength (ULONG)
Length of VRAMREGISTERIN data structure, in bytes.

VRAMREGISTERIN Field - ulHandle

ulHandle (ULONG)
Handle required when unregistering.

VRAMREGISTERIN Field - ulFlags

ulFlags (ULONG)
Flag indicating type of register.

This flag has the following values:

ENDIVE_REGISTER_HANDLE
0x001; set this flag to register and clear it to unregister

VRAMREGISTEROUT

Handle returned upon registration.

```
typedef struct _VRAMREGISTEROUT {
  ULONG     ulLength;  /*  Length of VRAMREGISTEROUT data structure, in bytes.  */
  ULONG     ulFlags;   /*  Not used at this time. */
  ULONG     ulHandle;  /*  Unique ID. */
} VRAMREGISTEROUT;

typedef VRAMREGISTEROUT *PVRAMREGISTEROUT;
```

VRAMREGISTEROUT Field - ulLength

ulLength (ULONG)
Length of VRAMREGISTEROUT data structure, in bytes.

VRAMREGISTEROUT Field - ulFlags

ulFlags (ULONG)
Not used at this time.

VRAMREGISTEROUT Field - ulHandle

ulHandle (ULONG)
Unique ID.

This ID is returned when ENDIVE_REGISTER_HANDLE bit is set in VRAMREGISTERIN.

Notices

IBM Developer Connection Device Driver Kit for OS/2, Version 4.02 Edition (September 1997)

The following paragraph does not apply to the United Kingdom or any country where such provisions are inconsistent with local law: INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some states do not allow disclaimer of express or implied warranties in certain transactions, therefore, this statement may not apply to you.

This publication could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in new editions of the publication. IBM may make improvements and/or changes in the product(s) and/or the program(s) described in this publication at any time.

It is possible that this publication may contain reference to, or information about, IBM products (machines and programs), programming, or services that are not announced in your country. Such references or information must not be construed to mean that IBM intends to announce such IBM products, programming, or services in your country.

Requests for technical information about IBM products should be made to your IBM reseller or IBM marketing representative.

Copyright Notices

COPYRIGHT LICENSE: This publication contains printed sample application programs in source language, which illustrate OS/2 programming techniques. You may copy, modify, and distribute these sample programs in any form without payment to IBM, for the purposes of developing, using, marketing or distributing application programs conforming to the OS/2 application programming interface.

Each copy of any portion of these sample programs or any derivative work, which is distributed to others, must include a copyright notice as follows: "(C) (your company name) (year). All rights reserved."

(C) Copyright International Business Machines Corporation 1997. All rights reserved.
Note to U.S. Government Users - Documentation related to restricted rights - Use, duplication or disclosure is subject to restrictions set forth in GSA ADP Schedule Contract with IBM Corp.

Disclaimers

References in this publication to IBM products, programs, or services do not imply that IBM intends to make these available in all countries in which IBM operates. Any reference to an IBM product, program, or service is not intended to state or imply that only that IBM product, program, or service may be used. Subject to IBM's valid intellectual property or other legally protectable rights, any functionally equivalent product, program, or service may be used instead of the IBM product, program, or service. The evaluation and verification of operation in conjunction with other products, except those expressly designated by IBM, are the responsibility of the user.

IBM may have patents or pending patent applications covering subject matter in this document. The furnishing of this document does not give you any license to these patents. You can send license inquiries, in writing, to:

IBM Director of Licensing
IBM Corporation
500 Columbus Avenue
Thornwood, NY 10594
U.S.A.

Asia-Pacific users can inquire, in writing, to the IBM Director of Intellectual Property and Licensing, IBM World Trade Asia Corporation, 2-31 Roppongi 3-chome, Minato-ku, Tokyo 106, Japan.

Licensees of this program who wish to have information about it for the purpose of enabling: (i) the exchange of information between independently created programs and other programs (including this one) and (ii) the mutual use of the information which has been exchanged, should contact IBM Corporation, Department LZKS, 11400 Burnet Road, Austin, TX 78758 U.S.A. Such information may be available, subject to appropriate terms and conditions, including in some cases, payment of a fee.

Trademarks

The following terms are trademarks of the IBM Corporation in the United States or other countries or both:
Advanced Peer-to-Peer Networking
APPN
IBM
Multimedia Presentation Manager/2
OS/2
OS/2 Warp
Personal System/2
PS/2
Presentation Manager
WIN-OS/2
Workplace Shell
XGA

The following terms are trademarks of other companies:

Trademark	**Owner**
C++	AT&T, Inc.
LaserJet	Hewlett-Packard Company
MASM	Microsoft Corporation
Times New Roman	Monotype Corporation, Limited
Times New Roman Bold Italic	Monotype Corporation, Limited
PostScript	Adobe Systems Incorporated

Other company, product, and service names, which may be denoted by a double asterisk (**), may be trademarks or service marks of others.

Glossary

This glossary contains terms and definitions that are, for the most part, used for OS/2 products. This is not a complete dictionary of computer terms.

Introduction

This glossary defines many of the terms used in this book. It includes terms and definitions from the *IBM Dictionary of Computing* , as well as terms specific to the Presentation Manager, but it is not a complete glossary for OS/2.

Other primary sources for these definitions are:

- The *American National Standard Dictionary for Information Systems* , ANSI X3.172-1990, copyrighted 1990 by the American National Standards Institute, 11 West 42nd Street, New York, New York 10036. These definitions are identified by the symbol (A) after the definition.
- The *Information Technology Vocabulary* , developed by Subcommittee 1, Joint Technical Committee 1, of the International Organization for Standardization and the International Electrotechnical Commission (ISO/IEC JTC1/SC1). Definitions of published parts of this vocabulary are identified by the symbol (I) after the definition; definitions taken from draft international standards, committee drafts, and working papers being developed by ISO/IEC JTC1/SC1 are identified by the symbol (T) after the definition, indicating that final agreement has not yet been reached among the participating National Bodies of SC1.

Glossary Listing

Select a starting letter of glossary terms:

A
B
C
D
E
F
G
H
I
J
K
L
M
N
O
P
Q
R
S
T
U
V
W
X
Y
Z

Glossary - A

A

ABIOS - Advanced BIOS. See *BIOS* .

accumulator - (1) A register in which one operand of an operation can be stored and subsequently replaced by the result of that operation. (T) (2) In the IBM 3800 Printing Subsystem Models 3 and 8, a feature that supplies a separate storage that can hold data in raster form. It can be used either for composing a sheet of data that combines a large amount of variable and constant data, or for storing an electronic overlay in raster form that will be merged with variable data as the sheet is printed.

access permission - All access rights a user has regarding an object. (I)

adapter - A piece of hardware that modifies the system unit to allow it to operate in a particular way, often by connecting the system unit to an external device such as a video monitor.

adapter device driver - A device driver that provides hardware-dependent services for an *OEM* adapter.

address space - (1) The range of addresses available to a program. (A) (2) The area of virtual storage available for a particular job.

all points addressable (APA) - In computer graphics, pertaining to the ability to address and display or not display each picture element (pel) on a display surface.

anchor block - An area of the internal resources of OS/2 Presentation Manager which is allocated to a process or thread that calls WinInitialize.

anchor point - The position or choice from which selection or deselection is extended.

APA - All points addressable.

API - Application programming interface.

application programming interface (API) - A functional interface supplied by the operating system, or by a separately-orderable licensed program, that allows an application program written in a high-level language to use specific data or functions of the operating system or the licensed program.

archive flag - In the OS/2 operating system, a flag of files and directories that the operating system uses to determine which files are new or modified. Files with this flag are included when a backup copy is made or when all the files are restored on a hard disk. See flag.

area - In computer graphics, a filled shape such as a solid rectangle.

ASCIIZ - A string of ASCII characters that is terminated with a byte containing the value 0.

aspect ratio - (1) The ratio of the height of a rectangle to its width. A rectangle of width 10 inches and height 5 inches has an aspect ratio of 10/5 or 2. (2) On a display screen, the ratio of the maximum length of a display line to the maximum length of a display column.

asynchronous (ASYNC) - (1) Pertaining to two or more processes that do not depend upon the occurrence of specific events such as common timing signals. (T) (2) Without regular time relationship; unexpected or unpredictable with respect to the execution of program instructions.

atom - A constant that represents a string. Once a string has been defined as an atom, the atom can be used in place of the string to save space. Strings are associated with their respective atoms in an *atom table* . See *integer atom* .

atom table - A table used to associate *atom* s with the strings that they represent. This table contains the mechanism by which the presence of a string can be verified.

AVIO - Advanced Video Input/Output

Glossary - B

B

background color - The color assigned to a background image.

background mix - An attribute that determines how the background of a graphic primitive is combined with the existing color of the graphics presentation space.

base device driver - An OS/2 device driver that performs I/O during the OS/2 kernel boot sequence to provide IPL support. Base device drivers are loaded by way of the CONFIG.SYS BASEDEV keyword, rather than the DEVICE keyword. See *BASEDEV keyword* , *adapter device driver* , and *device manager* .

BASEDEV keyword - New CONFIG.SYS keyword; loads a base device driver into the operating system.

Basic Input/Output System (BIOS) - Code that controls basic hardware operations, such as interactions with diskette drives, hard disk drives, and the keyboard.

Bezier curve - A mathematical technique of specifying a smooth, continuous line or surface, requiring a starting point and an ending point, with several intermediate points that influence or control the path of the linking curve.

BIOS - Basic Input/Output System.

bit-block transfer (bitblt) - Transfer of a rectangular array of bit-map data.

bitblt - Bit-block transfer.

bit map - A representation of an image by an array of bits.

block - (1) In programming languages, a compound statement that coincides with the scope of at least one of the declarations contained within it. A block may also specify storage allocation or segment programs for other purposes. (I) (2) A string of data elements recorded or transmitted as a unit. The elements may be characters, words or physical records. (T) (3) A collection of contiguous records recorded as a unit. Blocks are separated by interblock gaps and each block may contain one or more records. (A)

Bit block transfer (bitblt) - The process of transferring one or more blocks of data.

border - A visual indicator of a window's boundaries.

BPB - BIOS Parameter Block.

breakpoint - (1) A point in a computer program where execution may be halted. A breakpoint is usually at the beginning of an instruction where halts, caused by external intervention, are convenient for resuming execution. (T) (2) An instruction in a program for halting execution. Breakpoints are usually established at positions in a program where halts, caused by external intervention, are convenient for restarting. (T) (3) A place in a program, specified by a command or a condition, where the system halts execution and gives control to the workstation user or to a specified program.

Bus Master adapter - An adapter capable of performing Reads and Writes to physical storage by communicating directly with the storage subsystem (memory) rather than depending on a host DMA channel or host CPU. Synonymous with *first-party DMA adapter* .

Glossary - C

C

cached micro presentation space - A presentation space from a Presentation Manager owned store of micro presentation spaces. It can be used for drawing to a window only, and must be returned to the store when the task is complete.

CDB - Command Descriptor Block.

cell - See character cell.

character box - (1) An imaginary parallelogram on a display surface that contains all parts of one graphic character. Synonymous with bounding box. (T) (2) The maximum area in which a symbol and all associated elements, such as a cursor, an underline, or space surrounding the symbol to separate it from other symbols, can be printed or displayed. Synonymous with character cell. (3) The imaginary parallelogram whose boundaries govern the size, orientation, and spacing of individual characters to be displayed on a graphics display device.

character cell - (1) An addressable location on a display surface or printing medium. (2) The physical width and height in pels of a font. See also bounding box. (3) The imaginary box whose boundaries govern the size, orientation, and spacing of individual characters to be displayed on a workstation.

character mode - A mode that, in conjunction with the font type, determines the extent to which graphics characters are affected by the character box, shear, and angle attributes.

clipping - In computer graphics, removing those parts of display elements that lie outside of given boundary.

clip limits - The area of the paper that can be reached by a printer or plotter.

clipping path - A clipping boundary in world-coordinate space.

code page - An assignment of graphic characters and control function meanings to all code points; for example, assignment of characters and meanings to 256 code points for an 8-bit code, assignment of characters and meanings to 128 code points for a 7-bit code.

code point - A 1-byte code representing one of 256 potential characters.

code segment - An executable section of programming code within a load module.

color conversion - Changing one color format to another. Required, for example, when the source color format is different from the destination color format. When going from the monochrome color format to the color format, 1 (one) bits are converted to the image foreground color, and 0 (zero) bits are converted to the image background color.

When going from color to monochrome, all pels that match the passed background color are converted to the image background color of the destination.

All other pels are converted to the image foreground color of the destination. The color conversion takes place prior to any mix mode.

color dithering - See dithering.

command code - In this specification, refers to a group of related commands that an adapter device driver can receive.

All command codes have a prefix of "IOCC_". For example, common I/O requests (such as Read, Write, etc.) are grouped under the command code IOCC_EXECUTE_IO.

command data block - A data structure defined by the Small Computer System Interface standard to send commands to devices that conform to SCSI standards.

command descriptor block (CDB) - The structure used to communicate commands from a source to a destination.

command modifier - In this specification, a specific operation that an adapter device driver is to perform.

All command modifiers have a prefix of "IOCM_". For example, an adapter device driver might receive an IOCC_EXECUTE_IO command with a command modifier of IOCM_READ.

compatibility kernel - The portion of the OS/2 kernel that exists to support DOS INT 20, 21, 25, 26, and 27 functions. It acts as an interface to common kernel functionality such as the file system.

CON - Character-device name reserved for the console keyboard and screen.

conditional compilation - Processing by the preprocessor of certain specified code in the file, depending on the evaluation of a specified condition.

context hook - Similar to a "force flag" in earlier versions of OS/2. These are events, signaled by a virtual device driver, that are processed at task time. Forcing an IRET, and simulating an NMI, can fall into this category.

control program - A computer program designed to schedule and to supervise the execution of programs of a computer system.

controller sector buffer - One or more buffers, managed by a hardware adapter, to improve I/O transfer rates by helping to match a device and software timing requirements.

Glossary - D

D

DASD - Direct-access storage device.

data bus - A bus used to communicate data internally and externally to and from a processing unit, storage, and peripheral devices. (A) See bus.

data structure - The syntactic structure of symbolic expressions and their storage allocation characteristics. (T)

DBCS - Double-byte character set.

DC - Device context.

DDB - Device-dependent bit map.

deinstantiation - See *instantiation* .

DevHlp - Device helper.

device context (DC) - A logical description of a data destination such as memory, metafile, display, printer, or plotter. See also *direct device context, information device context, memory device context, metafile device context, and screen device context* .

device driver - A file that contains the code needed to attach and use a device such as a display, printer, or plotter.

device driver initialization (init) time - See *initialization (init) time, device driver* .

device driver profile - A file with a "DDP" extension, containing a script that is interpreted by the OS/2 DDINSTAL utility. Among other things, it

defines which files to copy from installation diskettes to target directories and specifies how the CONFIG.SYS file will be updated.

device helper (DevHlp) - (1) A kernel service (memory, hardware interrupt, software interrupt, queuing, semaphore, and so forth) provided to physical device drivers. (2) A callable C-language or assembler-language routine that provides an operating system service for an OS/2 device driver.

device object - A device that provides a means of communication between a computer and the outside world. A printer is an example of a device object.

device table - A data structure containing a summary of the adapters an adapter device driver supports and a list of the I/O devices attached to each adapter. This data structure is built by the adapter device driver in response to an IOCC_CONFIGURATION IOCM_GET_DEVICE_TABLE request.

direct access storage device (DASD) - A device in which access time is effectively independent of the location of the data.

direct memory access (DMA) - (1) A technique for moving data directly between main storage and peripheral equipment without requiring processing of the data by the processing unit. (2) The transfer of data between memory and input/output units without processor intervention.

display frame - (1) In computer graphics, an area in storage in which a display image can be recorded. (2) In computer micrographics, an area on a microform in which a display image can be recorded.

dispatch table - (1) A block of memory, allocated by the graphics engine, for the containment of entry points for use by a display driver. (2) An array of pointers to function-handling routines.

dithering - A technique for interleaving dark and light pels so that the resulting image looks smoothly shaded from a distance.

DLL - Dynamic link library.

DMA - Direct memory access.

double-byte character set (DBCS) - A set of characters in which each character is represented by two bytes. Languages such as Japanese, Chinese, and Korean, which contain more characters than can be represented by 256 code points, require double-byte character sets. Because each character requires 2 bytes, the typing, display, and printing of DBCS characters requires hardware and programs that support DBCS. Contrast with single-byte character set.

driver - (1) A program (and possibly data files) that contain information needed to run a particular unit, such as a plotter, printer, port, or mouse. See also device driver and printer driver. (2) A system or device that enables a functional unit to operate.

dynamic link library (DLL) - A file containing executable code and data bound to a program at load time or run time, rather than during linking. The code and data in a dynamic link library can be shared by several applications simultaneously.

Glossary - E

E

entry point - (1) In a database, the record that is first accessed upon entry into a database, caused by a user's command. (T) (2) The address or label of the first instruction executed on entering a computer program, routine, or subroutine. A computer program, routine, or subroutine may have a number of different entry points, each perhaps corresponding to a different function or purpose. (I) (A) Synonymous with entrance, entry. (3) In a routine, any place to which control can be passed. (A) (4) In the C, FORTRAN, and Pascal languages, the address or label of the first instruction processed or entered in a program, routine, or subroutine. A program, routine, or subroutine can have a number of different entry points, each corresponding to a different function or purpose.

EOI - End Of Interrupt

Glossary - F

F

Far call - Code that calls from one segment into another segment.

fillet - An arc that is tangential to the end points of two adjacent lines. See also *polyfillet* .

filter adapter device driver - A special class of adapter device drivers that do not manage the hardware directly, but monitor the stream of commands between a device manager and an adapter device driver. See *Device Manager* and *adapter device driver* .

first-party DMA adapter - See *bus master adapter* .

flag - A characteristic of a file or directory that enables it to be used in certain ways. See also *archive flag* , *hidden flag* , and *read-only flag* .

flat address - See *linear address* .

frame styles - Standard window layouts provided by the Presentation Manager.

freeze and thaw services - Functions that prevent a DOS session from executing (VDHFreezeVDM) until the matching thaw function (VDHThawVDM) is called. The freeze occurs when the specified DOS session leaves *kernel mode* .

Glossary - G

G

GDT - Global descriptor table.

Global Descriptor Table (GDT) - A table that defines code and data segments available to all tasks in an application.

glyph - A graphic symbol whose appearance conveys information; for example, the vertical and horizontal arrows on cursor keys that indicate the directions in which they control cursor movement, the sunburst symbol on the screen illumination control of a display device.

GPI - Graphics programming interface

graphic primitive - In computer graphics, a basic element, such as an arc or a line, that is not made up of smaller parts and that is used to create diagrams and pictures.

graphics attributes - The attributes that apply to graphics primitives. Examples are color selection, line type, and shading pattern definition. Contrast with segment attributes.

Graphics programming interface (GPI) - The formally-defined programming language that lies between an IBM graphics program and the user of the program.

graphics segment - A sequence of related graphic primitives and graphics attributes. See also *graphic primitive* .

GRE - Graphics engine.

Glossary - H

H

handle - (1) An identifier that represents an object, such as a device or a window, to the Presentation Interface. (2) In the Advanced DOS and OS/2 operating systems, a binary value created by the system that identifies a drive, directory, and file so that the file can be found and opened.

handshaking - A method by which two pieces of hardware, such as a personal computer and a plotter, can communicate. Depending upon the devices communicating, handshaking occurs either as a hardware function or through software, such as a device driver.

hard error - An error condition on a network that requires that the network be reconfigured or that the source of the error be removed before the network can resume reliable operation.

hardware palette - The array of RGBs that the physical device is displaying.

heap - An area of free storage available for dynamic allocation by an application. Its size varies depending on the storage requirements of the application.

hex - See hexadecimal

hexadecimal - Pertaining to a system of numbers to the base 16; hexadecimal digits range from 0 through 9 and A through F, where A represents 10 and F represents 15.

hook - A point in a system-defined function where an application can supply additional code that the system processes as though it were part of the function.

hook chain - A sequence of hook procedures that are "chained" together so that each event is passed in turn to each procedure in the chain.

Glossary - I

I

IDC - Inter-device-driver communication.

in-memory buffer - A block of memory in the address space of the host machine, used for data transfer.

init time - See initialization time, device driver.

initialization time, device driver - After the OS/2 loads a device driver, it sends it an OS/2 request packet to initialize. During this initialization, certain DevHlp functions are not permitted. Also called init time.

Input/Output Control (IOCtl) - A system service that provides a way for an application to send device-specific control commands to a device driver.

Input/Output Privilege Level (IOPL) - Allows part of a Ring 3 application or device driver to execute at Ring 0.

input router - OS/2 internal process that removes messages from the system queue.

inter-device-driver communication (IDC) - A mechanism that enables a physical device driver to communicate with another physical device driver.

interprocess communication - In the OS/2 operating system, the exchange of information between processes or threads through semaphores, queues, and shared memory.

interrupt - An instruction that directs the microprocessor to suspend what it is doing and run a specified routine. When the routine is complete, the microprocessor resumes its original work. See also routine.

interrupt request (IR) - Broadly, an "interrupt request level", referring to pending or in-service interrupt requests, or to a specific level (for example, IR 4).

interrupt request flag - A bit in the 8259 PIC controller that indicates an interrupt is pending on particular level. The VPIC also maintains a virtual interrupt request flag for each interrupt level for each DOS session.

interrupt service flag - A bit in the 8259 PIC controller that indicates an interrupt request is being serviced. It is cleared when the PIC is sent EOI. The VPIC maintains a virtual interrupt service flag indicating that a simulated interrupt is in-progress in a DOS session.

interrupt time - When a device driver is run because of an interrupt rather than because of an application request. OS/2 device drivers receive interrupts either from the hardware they manage or from the system real-time clock.

During interrupt time, certain DevHlp functions are not permitted. Also, addresses received directly from OS/2 applications might not be valid unless they are converted system addresses.

IOCtl - Input/Output Control.

IOPL - Input/Output Privilege Level.

IORB - Input/Output Request Block.

Input/Output Request Block (IORB) - A data structure defined by this specification that is passed as a parameter on all calls to an adapter device driver. It contains a fixed section, followed by a command-dependent section.

IORBH - Input/Output Request Block Header

IRET - Interrupt return.

IRQ - Interrupt Request.

Glossary - J

J

journal - A special-purpose file or data set that can be used to provide an audit trail of operator and system actions, or as a means of recovering superseded data.

Glossary - K

K

kanji - A graphic character set consisting of symbols used in Japanese ideographic alphabets. Each character is represented by 2 bytes.

kernel - (1) The part of an operating system that performs basic functions such as allocating hardware resources. (2) A program that can run under different operating system environments.

kerning - The design of graphic characters so that their character boxes overlap. The toned picture elements (pels) of the character appear outside the character cell.

Note: Kerning allows character boxes to overlap and characters to run together, so that characters can be designed for cursive languages, ligatures, or any other kind of character that requires more than one character box. It also allows for design of proportional-spaced fonts. By overlapping character boxes, characters can be placed closer together, or they can be placed farther apart by using overlapped blank character boxes.

Glossary - L

L

LCT - logical color table.

LDT - Local descriptor table.

LIFO stack - A data structure from which data is retrieved in "Last-In, First-Out" order.

linked list - A list in which the data elements may be dispersed, but in which each data element contains information for locating the next. Synonym for chained list.

linear address - A unique value that identifies the memory object.

Local Descriptor Table (LDT) - A table that defines code and data segments specific to a single task.

logical palette - An array of RGB and mapping index pairs, created by the device driver when defining a palette (as a result of a GpqCreatePalette call).

LVB - Logical Video Buffer.

Glossary - M

M

memory device context - A logical description of a data destination that is a memory bit map. See also device context.

metafile - A file containing a series of attributes that set color, shape, and size, usually of a picture or a drawing. Using a program that can interpret these attributes, a user can view the assembled image.

metafile device context - A logical description of a data destination that is a metafile which is used for graphics interchange. See also device context.

mickey - A unit of measurement for physical mouse motion whose value depends on the mouse device driver that is currently loaded.

mixed character string - A string containing a mixture of one-byte and kanji or Hangeul (two-byte) characters.

mutex semaphore - (Mutual exclusion semaphore). A semaphore that enables threads to serialize their access to resources. Only the thread that currently owns the mutex semaphore can gain access to the resource, thus preventing one thread from interrupting operations being performed by another.

Glossary - N

N

named pipe - A named buffer that provides client-to-server, server-to-client or duplex communication between unrelated processes. Contrast with unnamed pipe.

notification callout - The feature that provides for a routine to be called on completion of an input/output request. See also notification routine.

notification routine - The routine indicated in an input/output request block to be called on completion of that request. See also notification callout.

null-terminated string - A string of (n+1) characters where the (n+1)th character is the "null" character (X'00') and is used to represent an n-character string with implicit length. Also called a "zero-terminated" string or an "ASCIIZ". string.

Glossary - O

O

Glossary - P

P

palette - A list of colors assigned to various areas on a panel. A user can change the color of these areas.

PDD - Physical Device Driver.

PDE - PageDirectoryEntry.

pel - Picture element.

permissible action - In a conceptual schema language, an action conforming to specified rules or constraints that changes a presumably consistent collection of sentences into a consistent one or makes known a consistent one present in the information base or conceptual schema.

phase alignment - Aligning source bits with destination bits. Often required in a Bitblt function move operation where byte blocks are moved on bit boundaries.

physical address - A 32-bit byte address giving the actual address in physical storage for a data item.

physical device driver (PDD) - A system interface that handles hardware interrupts and supports a set of input and output functions.

pipe - See named pipe, unnamed pipe.

picture element (pel, pixel) - (1) In computer graphics, the smallest element of a display surface that can be independently assigned color and intensity. (T) . (2) The area of the finest detail that can be reproduced effectively on the recording medium. (3) An element of a raster pattern about which a toned area on a photoconductor can appear.

PIO - Programmed I/O.

pixel - Picture element.

polyfillet - A curve based on a sequence of lines. The curve is tangential to the end points of the first and last lines, and tangential also to the midpoints of all other lines.

polyline - In computer graphics, a sequence of adjoining lines.

pop - To remove an item from the top of a pushdown list. Contrast with push.

prefetch - To locate and load a quantity of data in anticipation of a request.

presence-check function - A Ring 3 (non-privileged) .EXE program that determines whether a given hardware interface is present on a workstation.

PRESENCECHECK - A keyword, interpreted by the DDINSTAL utility, to determine whether to process the device driver profile file, based on the return code from PRESENCECHECK.

printer driver - A file that describes the physical characteristics of a printer, plotter, or other peripheral device, and is used to convert graphics and text into device-specific data at the time of printing or plotting.

Print Manager - In the Presentation Manager, the part of the spooler that manages the spooling process. It also allows the user to view print queues and to manipulate print jobs.

privilege level - A method of protection that allows only certain program instructions to be used by certain programs.

program group - Several programs that can be acted upon as a single entity.

protect mode - A method of program operation that limits or prevents access to certain instructions or areas of storage. Contrast with real mode.

push - To add an item to the top of a pushdown list. Contrast with pop.

Glossary - Q

Q

queued device context - A logical description of a data destination (for example, a printer or plotter) where the output is to go through the spooler. See also device context.

Glossary - R

R

read-only memory basic input/output system (ROM-BIOS) - Microcode in read-only memory that controls basic input/output operations such as interactions with cassettes, diskette drives, hard disk drives, and the keyboard. See also BIOS, NetBIOS.

Note: ROM BIOS allows the user to write programs and add or remove devices without concern for characteristics such as device addresses.

real mode - In the OS/2 operating system, a method of program operation that does not limit or prevent access to any instructions or areas of storage. The operating system loads the entire program into storage and gives the program access to all system resources.

reentrant - The attribute of a program or routine that allows the same copy of the program or routine to be used concurrently by two or more tasks.

removable-media indicator - A flag (bit) indicating that a device permits media removal.

resource - The means of providing extra information used in the definition of a window. A resource can contain definitions of fonts, templates, accelerators and mnemonics; the definitions are held in a resource file.

resurrection - The Presentation Manager event that occurs when switched back from a full-screen DOS or WIN-OS/2 session.

RETF - Return far.

reverse video - A form of highlighting a character, field, or cursor by reversing the color of the character, field, or cursor with its background; for example, changing a red character on a black background to a black character on a red background.

ROM BIOS - Read-Only Memory Basic Input/Output System.

ROP - Raster operation.

RTC - Real-Time Clock.

Glossary - S

S

SBCS - Single-byte character set

SCB - See subsystem control block architecture.

screen device context - A logical description of a data destination that is a particular window on the screen. See also device context.

SCSI - Small Computer System Interface.

seamless windows - An architecture contained within OS/2 which permits one or more applications to share windowed desktop graphical space and other resources, while executing concurrently. Application session windows managed by seamless windows can share border information, and pointing device transitions from session to session are handled smoothly and transparently.

second-party DMA adapter - See DMA slave.

semaphore - (1) A variable that is used to enforce mutual exclusion. (T) (2) An indicator used to control access to a file; for example, in a multiuser application, a flag that prevents simultaneous access to a file. (3) An entity used to control access to system resources. Processes can be locked to a resource with semaphores if the processes follow certain programming conventions.

sense data - Data which describes an I/O error as defined by the ANSI SCSI specifications.

single-byte character set (SBCS) - A character set in which each character is represented by a one-byte code. Contrast with double-byte character set.

Small Computer System Interface (SCSI) - An input and output bus that provides a standard interface between the OS/2 multimedia system and peripheral devices.

spline curve - In computer graphics, a shape created when a user specifies a series of points and the computer software draws a curve that smoothly approaches those points.

spooler - A program that intercepts data going to a device driver and writes it to a disk. The data is later printed or plotted when the required device is available. A spooler prevents output from different sources from being intermixed.

synchronous - Pertaining to two or more processes that depend upon the occurrence of specific events such as common timing signals.

Glossary - T

T

text window - See VIO window.

thread - The smallest unit of operation to be performed within a process.

thunk - term used to describe the process of address conversion, stack, and structure realignment that is necessary when passing control between 16-bit and 32-bit modules.

thunk layer - An interface that converts 32-bit parameters to 16-bit parameters, and maps linear addresses to segmented addresses.

time slice - (1) The period of processing time allocated for running a program. (2) An interval of time on the processing unit allocated for use in performing a task. After the interval has expired, processing unit time is allocated to another task, so a task cannot monopolize processing unit time beyond a fixed limit.

tuple - In a relational database, a part of a relation that uniquely describes an entity and its attribute.

Glossary - U

U

unnamed pipe - A circular buffer created in memory; used by related processes to communicate with one another. Contrast with named pipe.

Glossary - V

V

VBIOS - Virtual BIOS device driver

VCMOS - Virtual CMOS device driver

VDD - Virtual device driver

VDH - Virtual video Device Handler

VDM - Virtual DOS Machine; use DOS session.

VDMA - Virtual Direct Memory Access device driver

VDSK - Virtual hard DiSK device driver

video graphics adapter (VGA) - A computer adapter that provides high-resolution graphics and a total of 256 colors.

VIO - Virtual Input/Output

VIRR - Virtual Interrupt Request Register

Virtual Device Driver (VDD) - In the OS/2 operating system, a type of device driver used by DOS programs running in a DOS session to access devices, such as the screen or mouse, which must be shared with other processes in the system. The virtual device driver maps DOS device commands to the normal (physical) device driver under OS/2 2.0 and later versions of the operating system.

virtual DevHlp (VDH) - Kernel (linear memory, paging, hardware interrupt, event control, port control) services provided to virtual device drivers.

virtual I/O (VIO) - A facility that pages data into and out of external page storage.

virtual memory - Synonym for virtual storage.

Virtual Programmable Interrupt Controller - Virtualizes the 8259 Programmable Interrupt Controller (PIC). A special virtual device driver, in that it provides services to other virtual device drivers.

virtual storage - Addressable space that is apparent to the user as the processor storage space, from which the instructions and the data are mapped into the processor storage locations. Synonymous with virtual memory.

visible region - A window's presentation space clipped to the boundary of the window and the boundaries of any overlying window.

VPIC - Virtual Programmable Interrupt Controller device driver.

VRAM - Video Random-Access Memory.

VTIMER - Virtual TIMER device driver.

V86 mode - Virtual 8086 mode of the 80386 CPU.

Glossary - W

W

window coordinates - A set of coordinates by which a window position or size is defined; measured in device units, or *pels* .

Glossary - X

X

Glossary - Y

Y

There are no glossary terms for this initial letter.

Glossary - Z

Z